

Statistics for Biology and Health

Daniel Sorensen

Statistical Learning in Genetics

An Introduction Using R

Second Edition



Springer

Statistics for Biology and Health

Series Editors

Mitchell H. Gail, Bethesda, USA

Jonathan M. Samet, Department of Environmental & Occupational Health, University of Colorado Denver - Anschutz Medical Campus, Aurora, USA

Statistics for Biology and Health (SBH) includes monographs and textbooks on statistical topics relating to biostatistics, medicine, epidemiology, biology, and ecology.

Daniel Sorensen

Statistical Learning in Genetics

An Introduction Using R

Second Edition



Springer

Daniel Sorensen
Aarhus University
Aarhus, Denmark

ISSN 1431-8776 ISSN 2197-5671 (electronic)
Statistics for Biology and Health
ISBN 978-3-031-86273-1 ISBN 978-3-031-86274-8 (eBook)
<https://doi.org/10.1007/978-3-031-86274-8>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023, 2025

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Til min elskede Pia

Preface

This book evolved from a set of notes written for a graduate course on Likelihood and Bayesian Computations held at Aarhus University in 2016 and 2018. The audience was life science PhD students and post-docs with a background in either biology, agriculture, medicine or epidemiology, who wished to develop analytic skills to perform genomic research. This book is addressed to this audience of numerate biologists, who despite an interest in quantitative methods lack the formal mathematical background of the professional statistician. For this reason, I offer considerably more detail in explanations and derivations than may be needed for a more mathematically oriented audience. Nevertheless, some mathematical and statistical prerequisites are needed in order to extract maximum benefit from the book. These include introductory courses on calculus, linear algebra and mathematical statistics, as well as a grounding in linear and nonlinear regression and mixed models. Applied statistics and biostatistics students may also find the book useful, but may wish to browse hastily through the introductory chapters describing likelihood and Bayesian methods.

I have endeavoured to write in a style that appeals to the quantitative biologist, while remaining concise and using examples profusely. The intention is to cover ground at a good pace, facilitating learning by interconnecting theory with examples and providing exercises with their solutions. Many exercises involve programming with the open-source package R, a statistical software that can be downloaded and used with the free graphical user interface RStudio. Most of today's students are competent in R, and there are many tutorials online for the uninitiated. The R-code needed to solve the exercises is provided in all cases and is written, with few exceptions, with the objective of being transparent rather than efficient. The reader has the opportunity to run the codes and to modify input parameters in an experimental fashion. This hands-on computing contributes to a better understanding of the underlying theory.

The first objective of this introduction is to provide readers with an understanding of the techniques used for analysis of data, with emphasis on genetic data. The second objective is to teach them to implement these techniques. Meeting these objectives is an initial step towards acquiring the skills needed to perform data-

driven genetics/genomics research. Despite the focus on genetic applications, the mathematics of the statistical models and their implementation are relevant for many other branches of quantitative methods. An appendix in the opening chapter provides an overview of basic quantitative genomic concepts, making the book more accessible to an audience of “non-geneticists”.

I have attempted to give a balanced account of frequentist/likelihood and Bayesian methods. Both approaches are used in classical quantitative genetic and modern genomic analyses and constitute essential ingredients in the toolkit of the well-trained quantitative biologist.

The book is organised in three parts. Part I (Chaps. 2 to 5) presents an overview of likelihood and Bayesian inference. Chapter 2 introduces the basic elements of the likelihood paradigm, including the likelihood function, the score and the maximum likelihood estimator. Properties of the maximum likelihood estimator are summarised and several examples illustrate the construction of simple likelihood models, the derivation of the maximum likelihood estimators and their properties. Chapter 3 provides a review of three computational methods for fitting likelihood models: Newton-Raphson, the EM (expectation-maximisation) algorithm and gradient descent. After a brief description of the methods and the essentials of their derivation, several examples (13 in all) are developed to illustrate their implementation. Chapter 4 covers the basics of the Bayesian approach, mostly through examples. The first set of examples illustrate the type of inferences that are possible (joint, conditional and marginal inferences), when the posterior distributions have known closed forms. In this case, inferences can be exact using analytical methods, or can be approximated using Monte Carlo draws from the posterior distribution. A number of options are available when the posterior distribution is only known up to proportionality. After a very brief account of Bayesian asymptotics, the chapter focuses on Markov chain Monte Carlo (McMC) methods. These are recipes for generating approximate draws from posterior distributions. Using these draws one can obtain Monte Carlo estimates of the complete posterior distribution, or Monte Carlo estimates of summaries such as the mean, variance and posterior intervals. The chapter provides a description of the Gibbs sampling algorithm and of the joint and single-site updating of parameters based on the Metropolis-Hastings algorithm. An overview of the tools needed for analysis of the McMC output concludes the chapter. An appendix provides the mathematical details underlying the magic of McMC within the constraints imposed by the author’s limited mathematics. Chapter 5 illustrates applications of McMC. Several of the examples discussed in connection with Newton-Raphson and the EM algorithm are revisited and implemented from a Bayesian McMC perspective.

Part II of the book has the heading *Prediction*. The boundaries between Parts I and II should not be construed as rigid. However, the heading emphasises the main thread of Chaps. 6 to 11, with an important detour in Chap. 8 that discusses multiple testing. Chapter 6 introduces many important ingredients of prediction: best predictor, best linear predictor, overfitting, bias-variance trade-off, cross-validation. Among the topics discussed is the accuracy with which future observations can be predicted, how is this accuracy measured, the factors affecting it, and importantly,

how a measure of uncertainty can be attached to accuracy. The body of the chapter deals with prediction from a classical/frequentist perspective. Bayesian prediction is illustrated in several examples throughout the book and particularly in Chap. 10. In Chap. 6, many important ideas related to prediction are illustrated using a simple least-squares setting, where the number of records n is larger than the number of parameters p of the model; this is the $n > p$ set-up. However, in many modern genetic problems the number of parameters greatly exceeds the number of records; the $p \gg n$ set-up. This calls for some form of regularisation, a topic introduced in Chap. 7 under the heading, *Shrinkage Methods*. After an introduction to ridge regression, the chapter provides a description of the lasso (least absolute shrinkage and selection operator) and of a Bayesian spike and slab model. The spike and slab model can be used for both prediction and discovery of relevant covariates that have an effect on the records. In a genetic context, these covariates could be observed genetic markers and the challenge is how to find as many promising markers among the hundreds of thousands available, while incurring a low proportion of false positives. This leads to the topic reviewed in Chap. 8: False Discovery Rate. The subject is first presented from a frequentist perspective as introduced by Benjamini and Hochberg in their highly acclaimed work, and is also discussed using empirical Bayesian and fully Bayesian approaches. The latter is implemented within an McMC environment using the spike and slab model as driving engine. The complete marginal posterior distribution of the false discovery rate can be obtained as a by-product of the McMC algorithm. Chapter 9 describes some of the technical details associated with prediction for binary data. The topics discussed include logistic regression for the analysis of case-control studies, where the data are collected in a non-random fashion, penalised logistic regression, lasso and spike and slab models implemented for the analysis of binary records, area under the curve (AUC) and prediction of a genetic disease of an individual, given information on the disease status of its parents. The chapter concludes with an appendix providing technical details for an approximate analysis of binary traits. The approximation can be useful as a first step, before launching the full McMC machinery of a more formal approach. Chapter 10 deals with Bayesian prediction, where many of the ideas scattered in various parts of the book are brought into focus. The chapter discusses the sources of uncertainty of predictors from a Bayesian and frequentist perspective and how they affect accuracy of prediction as measured by the Bayesian and frequentist expectations of the sample mean squared error of prediction. The final part of the chapter introduces, via an example, how specific aspects of a Bayesian model can be tested using posterior predictive simulations, a topic that combines frequentist and Bayesian ideas. Chapter 11 completes Part II and provides an overview of selected nonparametric methods. After an introduction of traditional nonparametric models, such as the binned estimator and kernel smoothing methods, the chapter concentrates on four more recent approaches: kernel methods using basis expansions, neural networks, classification and regression trees, and bagging and random forests.

Part III of the book consists of exercises and their solutions. The exercises (Chap. 12) are designed to provide the reader with deeper insight of the subject

discussed in the body of the book. A complete set of solutions, many involving programming, is available in Chap. 13.

The majority of the datasets used in the book are simulated and intend to illustrate important features of real-life data. The size of the simulated data is kept within the limits necessary to obtain solutions in reasonable CPU time, using straightforward R-code, although the reader may modify size by changing input parameters. Advanced computational techniques required for the analysis of very large datasets are not addressed. This subject requires a specialised treatment beyond the scope of this book.

The book has not had the benefit of having been used as material in repeated courses by a critical mass of students, who invariably stimulate new ideas, help with a deeper understanding of old ones, and not least, spot errors in the manuscript and in the problem sections. Despite these shortcomings, the book is completed and out of my hands. I hope the critical reader will make me aware of the errors. These will be corrected and listed on the web at <https://github.com/SorensenD/SLGDS>. The GitHub site also contains most of the R-codes used in the book, which can be downloaded, as well as notes that include comments, clarifications or additions of themes discussed in the book.

Aarhus, Denmark
May 2023

Daniel Sorensen

Preface to Second Edition

In this second edition the first chapter has been expanded, and a section on inferences using selected data has been added in Chap. 2. In Chap. 4 a new example using the normal multiple regression model has been added to illustrate how exact Bayesian inferences proceed when distributions have known forms, the section on the normal linear model has been extended, a section on sensitivity analysis based on importance sampling and on sampling importance resampling is now included and part of the appendix has been rewritten. Chapters 6 and 8 have undergone a number of changes and an appendix has been added to Chap. 8 with a proof of Benjamini and Hochberg's false discovery rate result. Part of Chap. 9 has been rewritten and extended, the example discussing estimation of the frequency of a disease in a population using an imperfect test has been expanded and an example illustrating some of the challenges involved in the discovery of genes related to rare diseases and in prediction of a rare disease based on genetic marker information has been added. Part of Chap. 10 has been rewritten and a section on Bayesian leave-one-out cross-validation has been added, together with an example. All the corrections and modifications listed in the file `errata.pdf` located on the website <https://github.com/SorensenD/SLGDS> have been incorporated.

Aarhus, Denmark
May 2025

Daniel Sorensen

Competing Interests The author has no competing interests to declare that are relevant to the content of this manuscript.

Acknowledgements

Many friends and colleagues have assisted in a variety of ways. Bernt Guldbrandtsen (University of Copenhagen) has been a stable helping hand and helping mind. Bernt has generously shared his deep biological and statistical knowledge with me on many, many occasions, and also provided endless advice with LaTeX and Markdown issues, with programming details, always with good spirits and patience. I owe much to him. Ole Fredslund Christensen (Aarhus University) read several chapters and wrote a meticulous list of corrections and suggestions. I am very grateful to him for this effort. Gustavo de los Campos (Michigan State University) has shared software codes and tricks and contributed with insight in many parts of the book, particularly in Prediction and Kernel Methods. I have learned much during the years of our collaboration. Parts of the book were read by Andres Legarra (INRA), Miguel Pérez Enciso (University of Barcelona), Bruce Walsh (University of Arizona), Rasmus Waagepetersen (Aalborg University), Peter Sørensen (Aarhus University), Kenneth Enevoldsen (Aarhus University), Agustín Blasco (Universidad Politécnica de Valencia), Jens Ledet Jensen (Aarhus University), Fabio Morgante (Clemson University), Doug Speed (Aarhus University), Bruce Weir (University of Washington), Rohan Fernando (retired from Iowa State University) and Daniel Gianola (retired from University of Wisconsin-Madison). I received many helpful comments, suggestions and corrections from them. However, I am only responsible for the errors that escaped scrutiny. I would be thankful if I could be made aware of these errors.

I acknowledge Eva Hiripi, Senior Editor, Statistics Books, Springer, for consistent support during this project.

I am the grateful recipient of many gifts from my wife Pia. One has been essential for concentrating on my task: happiness.

Contents

1	Overview	1
1.1	Introduction	1
1.2	The Sampling Distribution of a Random Variable	3
1.3	The Likelihood and the Maximum Likelihood Estimator	5
1.4	Incorporating Prior Information	8
1.5	Frequentist or Bayesian?	17
1.6	Prediction	20
1.7	Appendix: A Short Overview of Quantitative Genomics	36
 Part I Fitting Likelihood and Bayesian Models		
2	Likelihood	55
2.1	A Little Intuition	55
2.2	Summary of Likelihood Results	58
2.3	Example: The Likelihood Function of Transformed Data	63
2.4	Example: Linear Regression	64
2.5	Example: Bivariate Normal Model with Missing Records	67
2.6	Remarks on Inferences Under Selection	70
2.7	Example: Likelihood Inferences Using Selected Records	72
2.8	Example: The Likelihood Function with Truncated Data	76
2.9	Example: The Likelihood Function of a Genomic Model	78
3	Computing the Likelihood	83
3.1	Newton-Raphson and the Method of Scoring	83
3.2	Gradient Descent and Stochastic Gradient Descent	105
3.3	The EM Algorithm	109
4	Bayesian Methods	147
4.1	Example: Estimating the Mean and Variance of a Normal Distribution	149
4.2	Example: The Normal Multiple Regression Model	159

4.3	Example: Monte Carlo Inferences of the Joint Posterior Distribution of Mean and Variance	164
4.4	Approximating a Marginal Distribution	166
4.5	Example: The Normal Linear Mixed Model	167
4.6	Example: Inferring a Variance Component from a Marginal Posterior Distribution	174
4.7	Example: Bayesian Learning: Inheritance of Haemophilia	176
4.8	Example: Bayesian Learning: Updating Additive Genetic Values	179
4.9	A Brief Account of Bayesian Asymptotics	183
4.10	An Overview of Markov Chain Monte Carlo	185
4.11	The Metropolis-Hastings Algorithm	186
4.12	The Gibbs Sampling Algorithm	193
4.13	Output Analysis	197
4.14	Sensitivity Analysis	208
4.15	Appendix: A Closer Look at the MCMC Machinery	213
5	MCMC in Practice	229
5.1	Example: Estimation of Gene Frequencies from ABO Blood Group Phenotypes	229
5.2	Example: A Regression Model for Binary Data	235
5.3	Example: A Regression Model for Correlated Binary Data	242
5.4	Example: A Genomic Model	249
5.5	Example: A Mixture Model of Two Gaussian Components	257
5.6	Example: An Application of the EM Algorithm in a Bayesian Context. Estimation of SNP Effects	261
5.7	Example: Bayesian Analysis of the Truncated Normal Model	266
5.8	A Digression on Model Comparison	270
 Part II Prediction		
6	Fundamentals of Prediction	281
6.1	Best Predictor and Best Linear Predictor	281
6.2	Estimating the Regression Function in Practice: Least Squares ..	287
6.3	Overview of Things to Come	302
6.4	The Bias-variance Trade-off	304
6.5	Estimation of Validation MSE of Prediction in Practice	308
6.6	On Average Training MSE Underestimates Validation MSE	311
6.7	Least Squares Prediction	317
7	Shrinkage Methods	327
7.1	Ridge Regression	328
7.2	The Lasso	338
7.3	An Extension of the Lasso: The Elastic Net	346
7.4	Example: Prediction Using Ridge Regression and Lasso	347
7.5	A Bayesian Spike and Slab Model	349

8	Digression on Multiple Testing: False Discovery Rates	361
8.1	Introduction	363
8.2	Preliminaries	364
8.3	The Benjamini-Hochberg False Discovery Rate	366
8.4	A Bayesian Approach for a Simple Two-groups Mixture Model	371
8.5	Empirical Bayes Estimation	373
8.6	Local False Discovery Rates	376
8.7	Storey's q -values	378
8.8	Fully Bayesian MCMC False Discovery Rate	380
8.9	Example: A Two-component Gaussian Mixture	383
8.10	Example: The Spike and Slab Model with Genetic Markers	392
8.11	Appendix: Proof of Benjamini and Hochberg's Theorem	398
9	Binary Data	401
9.1	Prediction for Binary Observations	402
9.2	Mean Squared Error for Binary Data?	403
9.3	Logistic Regression with Non-random Sampling	408
9.4	Penalised Logistic Regression	410
9.5	The Lasso with Binary Records	413
9.6	A Bayesian Spike and Slab Model for Binary Records	414
9.7	Area Under the Curve	435
9.8	Prediction of Disease Status of Individual Given Disease Status of Relatives	452
9.9	Appendix: Approximate Analysis of Binary Traits	461
10	Bayesian Prediction and Model Checking	471
10.1	Levels of Uncertainty	472
10.2	Prior and Posterior Predictive Distributions	473
10.3	Bayesian Expectations of MSE	482
10.4	Example: Bayesian and Frequentist Measures of Uncertainty	485
10.5	Bayesian Leave-one-out Cross-validation	491
10.6	Model Checking Using Posterior Predictive Distributions	499
11	Nonparametric Methods: A Selected Overview	509
11.1	Local Kernel Smoothing	510
11.2	Kernel Methods Using Basis Expansions	524
11.3	Neural Networks	553
11.4	Classification and Regression Trees	576
11.5	Bagging and Random Forests	585
11.6	Appendix	597
Part III Exercises and Solutions		
12	Exercises	607
12.1	Likelihood Exercises I	607
12.2	Likelihood Exercises II	613

12.3	Bayes Exercises I.....	619
12.4	Bayes Exercises II.....	621
12.5	Prediction Exercises.....	626
13	Solution to Exercises	641
13.1	Likelihood Exercises I.....	641
13.2	Likelihood Exercises II.....	660
13.3	Bayes Exercises I.....	681
13.4	Bayes Exercises II.....	697
13.5	Prediction Exercises.....	719
	References.....	743
	Author Index.....	751
	Subject Index	755

Chapter 1

Overview



1.1 Introduction

Suppose there is a set of data consisting of observations in humans on forced expiratory volume (FEV, a measure of lung function; lung function is a predictor of health and a low lung function is a risk factor for mortality), or on presence or absence of heart disease and that there are questions that could be answered using these data. For example, a statistical geneticist may wish to know:

1. Is there a genetic component contributing to the total variance of these traits?
A positive answer suggests that genetic factors are at play. The next step would be to investigate the following:
2. Is the genetic component of the traits driven by a few genes located on a particular chromosome, or are there many genes scattered across many chromosomes? How many genes are involved and is this a scientifically sensible question?
3. Are the genes detected, protein-coding genes, or are there also noncoding genes involved in gene regulation?
4. How is the strength of the signals captured in a statistical analysis related to the two type of genes? What fraction of the total genetic variation is allocated to both type of genes?
5. What are the frequencies of the genes in the sample? Are the frequencies associated with the magnitude of their effects on the traits?
6. What is the mode of action of the genes?
7. What proportion of the genetic variance estimated in 1 can be explained by the discovered genes?
8. Given information on the set of genes carried by an individual, will a genetic score constructed before observing the trait help with early diagnosis and prevention?
9. How should the predictive ability of the score be measured?

10. Are there other non-genetic factors that affect the traits, such as smoking behaviour, alcohol consumption, blood pressure measurements, body mass index, level of physical exercise?
11. Could the predictive ability of the genetic score be improved by incorporation of these non-genetic sources of information, either additively, or considering interactions? What is the relative contribution from the different sources of information?

The first question has been the focus of quantitative genetics during many years long before the so-called “genomic revolution”. That is, before breakthroughs in molecular and computational biology made technically and economically possible the sequencing of whole genomes resulting in hundreds of thousands or millions of genetic markers (for example, SNP, single nucleotide polymorphisms) for each individual in the data set. Until the end of the twentieth century before dense genetic marker data were available, genetic variation of a given trait was inferred using resemblance between relatives. This requires equating the expected proportion of the genome shared identical by descent, given a pedigree, with the observed phenotypic correlation between relatives. The fitted models also retrieve “estimates of random effects”, the predicted genetic values that act as genetic scores and are used in selection programs of farm animals and plants.

Answers to questions 2–7 would provide insight into genetic architecture, and thereby, into the roots of many complex traits and diseases. This has important practical implications for drug therapies targeted to particular metabolic pathways, for personalised medicine and for improved prediction. These questions could not be sensibly addressed before dense marker data became available (perhaps with the exception provided by complex segregation analysis that allowed searching for single genes).

Shortly after a timid start where use of low density genetic marker information made its appearance, the first decade of the twenty-first century saw the construction of large biomedical databases that could be accessed for research purposes where health information was collected. One such was the *British 1958-cohort study* including medical records from approximately 3000 individuals genotyped for 1 million SNPs. These data provided for the first time the opportunity to begin addressing questions 2–7. However a problem had to be faced: how to fit and validate a model with one million unknowns to a few thousand records and moreover, how to find a few promising genetic markers from the million available avoiding a large proportion of false positives? This resulted in a burst of activity in the fields of computer science and statistics leading to development of methodology designed to meet the challenge posed by *Big Data*.

In recent years the amount of information in modern data sets has grown and become formidable and the challenges have not diminished. One example is the *UK Biobank* that provides a wealth of health information from half a million UK participants. The database is regularly updated and a team of scientists recently reported that the complete exome sequence was completed (about 2% of the genome involved in coding for proteins and considered to be important for

identifying disease-causing or rare genetic variants). The study involved more than 150,000 individuals genotyped for more than 500 million SNP (Halldorsson et al. 2022). These data are paired with detailed medical information and constitute an unparalleled resource for linking human genetic variation to human biology and disease.

An important task for the statistical geneticist is to adapt, develop and implement models that can extract information from these large-scale data and to contribute to finding answers to the 11 questions posed above. This is an exercise on inference (such as estimation of genetic variation), on gene detection (among the millions of genetic markers that may be included in a probability model, how to screen the “relevant” ones for further study?), on prediction (how does the quality of prediction of future records, for example outcome of a disease, improve with this new knowledge about the trait?) and on how to fit the probability models. There are several areas of expertise that must be developed in order to fulfil this data-driven research task. An initial step is to understand the methodology that underlies the probability models and to learn the modern computer-intensive methods required for fitting these models. The objective of this book is to guide the reader to take this first step.

This opening chapter gives an overview of the book’s content omitting many technicalities that are revealed in later chapters and is intended to give a flavour of the way ahead. The first part is about methodology and introduces, by means of an example, the concepts of probability distribution, likelihood and the maximum likelihood estimator. This is followed by a brief description of Bayesian methods indicating how prior knowledge can be incorporated in a probability model and how it can affect inferences. The second part of the chapter presents models for prediction and for detection of genes using parametric and nonparametric approaches. There is an appendix that offers a brief tour of the quantitative genetics/genomics model. The goal is to introduce the jargon and the basic quantitative genetics/genomics concepts used in the book.

1.2 The Sampling Distribution of a Random Variable

A useful starting point is to establish the distinction between a probability distribution and a likelihood function. For example, assume a random variable X that has a Bernoulli probability distribution. This random variable can take 1 or 0 as possible values (more generally, it can have two modalities) with probabilities θ and $1 - \theta$, respectively. The mean of the distribution is

$$E(X|\theta) = 0 \times \Pr(X = 0|\theta) + 1 \times \Pr(X = 1|\theta) = \theta$$

and the variance is

$$\begin{aligned}\text{Var}(X|\theta) &= E(X^2|\theta) - [E(X|\theta)]^2 \\ &= \theta - \theta^2 = \theta(1 - \theta).\end{aligned}$$

A binomial distribution arises from the sum of n mutually independent Bernoulli random variables all having the same probability θ . Therefore, the expected value and the variance of a binomially distributed random variable are $n\theta$ and $n\theta(1 - \theta)$, respectively.

With this background, imagine that a sample of size n of unrelated haploid individuals is obtained from some population with the objective of estimating allele frequency at a biallelic locus. The sample contains x copies of allele A and $n - x$ copies of allele a . The n data points are draws assumed to be identically and independently distributed and in each draw, the probability of observing an A allele is θ . Since the random variable can take two modalities (A or a), the number of copies drawn, X , is binomially distributed with parameters n and θ and probability mass function equal to

$$\Pr(X = x|n, \theta) = \binom{n}{x} \theta^x (1 - \theta)^{n-x}, \quad x = 0, 1, \dots, n, \quad 0 < \theta < 1. \quad (1.1)$$

For fixed values of n and θ , one can plot (1.1) as a function of $x = 0, 1, \dots, n$ and this defines the *probability distribution* of X . Figure 1.1 shows two different binomial distributions. Importantly in (1.1), the parameters n and θ are fixed and the random variable is X , the number of A alleles drawn (here I distinguish between

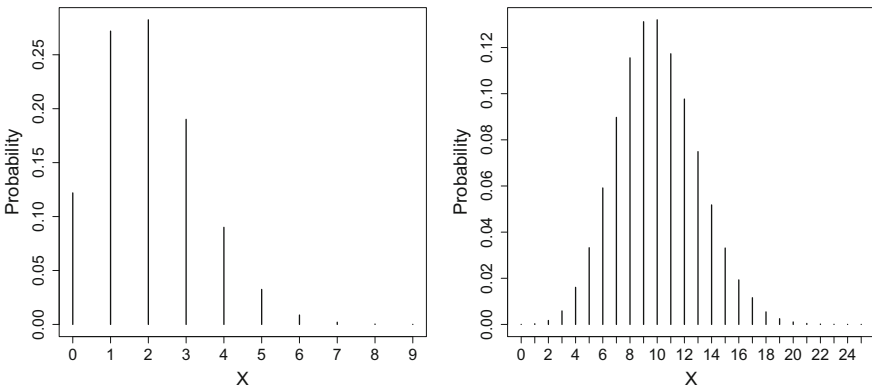


Fig. 1.1 Left: binomial probability distribution with parameters $n = 20, \theta = 0.1$; Right: binomial probability distribution with parameters $n = 100, \theta = 0.1$

the random variable X and its realised value, x . This distinction is not necessarily followed throughout the book).

The probability distribution in the right panel of Fig. 1.1 is more symmetrical than in the left panel. This is due to the different sample sizes n . As sample size increases further, X will approach its limiting distribution which is the normal distribution by virtue of the central limit theorem.

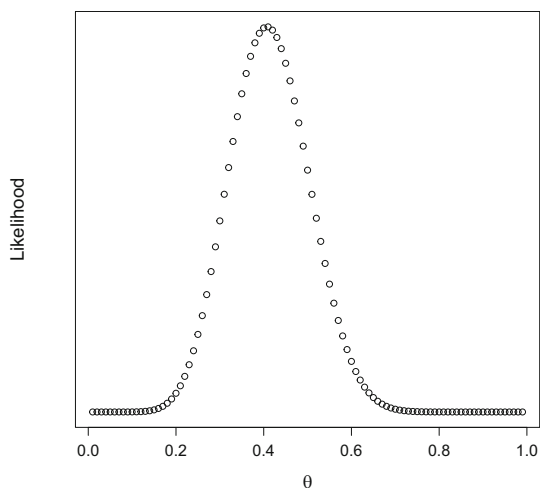
1.3 The Likelihood and the Maximum Likelihood Estimator

Consider now viewing (1.1) in a different manner, whereby x and n are fixed and θ varies. To be specific, assume that the sample size is $n = 27$ and the number of copies of allele A in the sample is $x = 11$. One can plot the probability of obtaining $x = 11$ copies of A in a sample of size $n = 27$, for all permissible values of θ as in Fig. 1.2. For example, for $\theta = 0.1$, $\Pr(X = 11|n = 27, \theta = 0.1) = 0.242 \times 10^{-4}$ and for $\theta = 0.6$, $\Pr(X = 11|n = 27, \theta = 0.6) = 0.203 \times 10^{-1}$. This plot is the *likelihood function* for θ , $L(\theta|x, n)$ and the value of θ that maximises this function is known as the maximum likelihood estimate of θ (I will use MLE short for maximum likelihood estimator or maximum likelihood estimate and ML for maximum likelihood).

One way of finding the *maximum likelihood estimate* of θ is to differentiate (1.1) and find the maximizer. It is equivalent—but often easier—to maximize the logarithm of the likelihood function, the loglikelihood, denoted $\ell(\theta|x, n)$,

$$\frac{\partial \ell(\theta|x, n)}{\partial \theta} = \frac{\partial}{\partial \theta} \left[\log \binom{n}{x} + x \log(\theta) + (n - x) \log(1 - \theta) \right].$$

Fig. 1.2 Binomial model: Likelihood function for θ , given data $n = 27$, $x = 11$



Carrying out the differentiation and setting the result equal to zero shows that the MLE of θ must satisfy

$$\frac{x}{\theta} - \frac{n-x}{1-\theta} = 0.$$

Solving for θ yields the MLE

$$\hat{\theta} = \frac{x}{n}, \quad (1.2)$$

that in the case of the example, with $x = 11$ and $n = 27$, gives $\hat{\theta} = 0.41$.

The Sampling Variance of the Maximum Likelihood Estimator

Usually one needs to quantify the degree of uncertainty associated with an estimate. In classical likelihood the uncertainty is described by the *sampling distribution* of the MLE. In the case of the example, the sampling distribution of $\hat{\theta}$ is the probability distribution of this estimator obtained by drawing repeated binomial samples of fixed size n , with the probability parameter fixed at its MLE, $\theta = \hat{\theta}$. The MLE is computed in each sample and the sampling distribution of the MLE is characterised by these estimates.

In this binomial example the sampling distribution of $\hat{\theta}$ is known exactly; it is proportional to a binomial distribution (since X is binomial and n is fixed). The small sample variance of the maximum likelihood estimator is

$$\text{Var}(\hat{\theta}) = \text{Var}\left(\frac{X}{n}\right) = \frac{\theta(1-\theta)}{n}. \quad (1.3)$$

The parameter θ is typically not known and is replaced by the MLE $\hat{\theta}$. Then

$$\widehat{\text{Var}}(\hat{\theta}) = \frac{\hat{\theta}(1-\hat{\theta})}{n}.$$

In many cases the MLE does not have a closed form and the small sample variance is not known. One can then appeal to large sample properties of MLE; one of these is that, asymptotically, the MLE is normally distributed, with mean equal to the parameter and variance given by minus the inverse of the second derivative of

the loglikelihood evaluated at $\theta = \hat{\theta}$. The second derivative of the loglikelihood is

$$\begin{aligned}\frac{\partial^2 \ell(\theta|x, n)}{(\partial \theta)^2} &= \frac{\partial^2}{(\partial \theta)^2} \left[\log \binom{n}{x} + x \log(\theta) + (n-x) \log(1-\theta) \right] \\ &= -\frac{x}{\theta^2} - \frac{n-x}{(1-\theta)^2}.\end{aligned}$$

In this expression, substituting θ with the MLE $\hat{\theta}$ and taking a reciprocal yields

$$-\left(\frac{\partial^2 \ell(\theta|x, n)}{(\partial \theta)^2} \right)^{-1} \Big|_{\theta=\hat{\theta}} = \widehat{\text{Var}}(\hat{\theta}) = \frac{\hat{\theta}(1-\hat{\theta})}{n} \approx 0.009. \quad (1.4)$$

In this simple example the asymptotic variance agrees with the small sample variance. An approximate 95% confidence interval for θ based on asymptotic theory is

$$0.41 \pm 1.96 \times 0.095 = (0.22, 0.60). \quad (1.5)$$

This means that there is a 95% probability that this interval contains the true parameter θ . The “probability” is interpreted with respect to a set of hypothetical repetitions of the entire data collection and analysis procedure. These repetitions consist of many random samples of data drawn under the same conditions and where a confidence interval is computed for each sample. The random variable is the confidence interval that is computed for each sample; in 95 intervals out of 100 (in a 95% confidence interval), the interval will contain the unobserved θ .

Figure 1.3 (left) shows the result of simulating 100,000 times from a binomial distribution with $n = 27$ and $\theta = 0.41$, computing the MLE in each replicate and plotting the distribution as a histogram. This represents the (small sample) Monte Carlo sampling distribution of the MLE. Overlaid is the asymptotic distribution of the MLE that is normal with mean 0.41 and variance given by (1.4) equal to 0.009. The right panel of Fig. 1.3 displays the result of a similar exercise with $n = 100$ and $\theta = 0.41$. The fit of the asymptotic approximation is better with the larger sample size.

A glance at a standard calculus book reveals that the curvature of a function f at a point θ is given by

$$c(\theta) = \frac{f''(\theta)}{[1 + f'(\theta)^2]^{3/2}}.$$

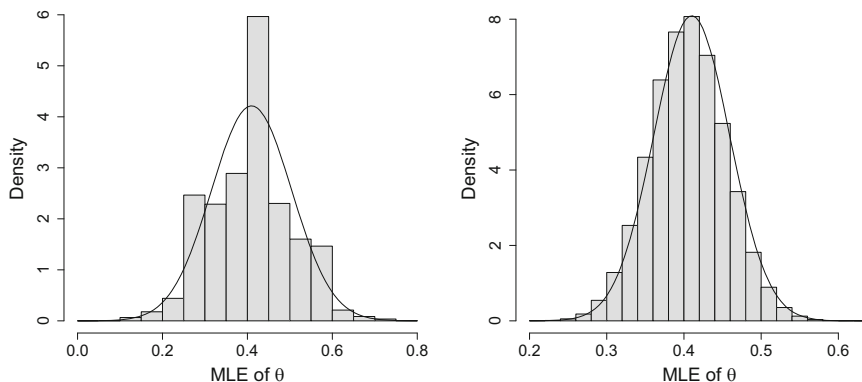


Fig. 1.3 Left: histogram of the Monte Carlo distribution of the MLE for the binomial model, with $n = 27, \theta = 0.41$. Right: histogram of the Monte Carlo distribution of the MLE for the binomial model, with $n = 100, \theta = 0.41$. The overlaid normal curves represent the asymptotic approximation of the distribution of the MLE

In the present case, the function f is the loglikelihood ℓ whose first derivative evaluated at $\theta = \hat{\theta}$ is equal to zero. The curvature of the loglikelihood at $\theta = \hat{\theta}$ is

$$c(\hat{\theta}) = \ell''(\hat{\theta}).$$

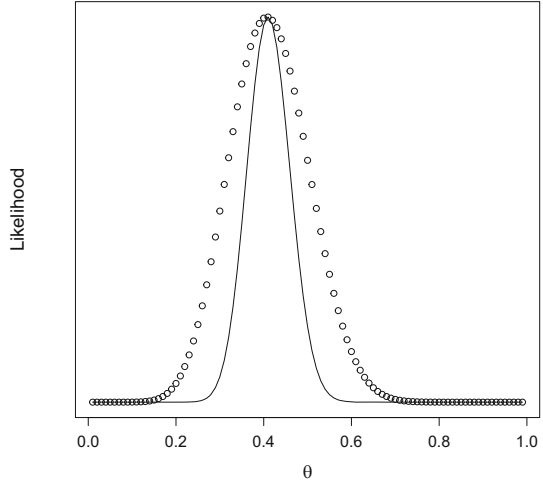
(I use the standard notation $\ell''(\hat{\theta})$ for $\left(\frac{\partial^2 \ell(\theta|x, n)}{(\partial \theta)^2}\right)\bigg|_{\theta=\hat{\theta}}$).

Note As the loglikelihood increases or decreases so does the likelihood; therefore the value of the parameter that maximises one also maximises the other. Working with the loglikelihood is to be preferred to working with the likelihood function because it is easier to differentiate a sum than a product. The curvature of the loglikelihood at the MLE is related to the sample variance of the MLE. This last point is illustrated in Fig. 1.4. As n increases from 27 to 100, the likelihood function becomes sharper, more concentrated about the MLE.

1.4 Incorporating Prior Information

Imagine that there is prior information about the frequency θ of allele A from comparable populations. Bayesian methods provide a natural way of incorporating such prior information into the model. This requires eliciting a *prior distribution* for θ that captures what is known about θ before obtaining the data sample. This prior distribution is combined with the likelihood (which, given the model, contains all the information arising from the data) to form the *posterior distribution* that is the

Fig. 1.4 Circled line: likelihood function for θ , given $n = 27, x = 11$. Full line: likelihood function for θ , given $n = 100, x = 41$



basis for the Bayesian inference. Specifically using Bayes theorem,

$$\text{Posterior} \propto \text{Prior} \times \text{Likelihood}. \quad (1.6)$$

If the prior density of θ is labelled $g(\theta)$, (1.6) becomes

$$p(\theta|x, n) \propto g(\theta) L(\theta|x, n), \quad (1.7)$$

indicating that the posterior density is proportional to the prior density times the likelihood. Probability statements about θ require scaling (1.7). This involves dividing the right hand side of (1.7) by

$$\sum_i g(\theta_i) L(\theta_i|x, n), \quad (1.8)$$

if θ is discrete (in which case g is a probability mass function), or by

$$\int g(\theta) L(\theta|x, n) d\theta, \quad (1.9)$$

if it is continuous (in which case g is a probability density function).

Using a Discrete Prior

This example is adapted from Albert (2009). Continuing with the binomial model, a simple approach to incorporate prior information on θ is to write down possible

values and to assign weights to these values. A list of possible values of θ could be

$$0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95. \quad (1.10)$$

Based on previous knowledge, one is prepared to assign the weights

$$1.0, 5.2, 8.0, 7.2, 4.6, 2.1, 0.7, 0.1, 0.0, 0.0$$

that are converted to probabilities dividing each weight by the sum; this gives the prior probability distribution of θ

$$0.04, 0.18, 0.28, 0.25, 0.16, 0.07, 0.02, 0.00, 0.00, 0.00. \quad (1.11)$$

The likelihood for θ is proportional to (1.1). The combinatorial term does not contain information about θ , so one can write

$$L(\theta|x = 11, n = 27) \propto \theta^{11} (1 - \theta)^{27-11}.$$

Evaluating this expression for all the possible values of θ in (1.10) yields a list of ten numbers (too small to be written down here). Label these ten numbers

$$L(0.05), L(0.15), \dots, L(0.95). \quad (1.12)$$

To obtain the posterior (1.7), the terms in (1.11) are multiplied by the corresponding term in (1.12). For example,

$$p(\theta = 0.05|x = 11, n = 27) \propto 0.04 \times L(0.05),$$

$$p(\theta = 0.15|x = 11, n = 27) \propto 0.18 \times L(0.15),$$

and so on with the remaining 8 terms. After scaling with the sum

$$0.04 \times L(0.05) + 0.18 \times L(0.15) + \dots + 0.00 \times L(0.95),$$

posterior probabilities can be assigned to the ten possible values of θ . These posterior probabilities are (rounded to two decimal places)

$$0.00, 0.00, 0.13, 0.48, 0.33, 0.06, 0.00, 0.00, 0.00, 0.00.$$

Based on these posterior probabilities, the posterior mean of θ is 0.38 and the probability that θ falls in the set $\{0.25, 0.35, 0.45\}$ is 0.94.

Using a Beta Prior: The Beta-Binomial Model

Another possible prior is to assign a beta distribution to θ with the appropriate parameters to reflect prior information. This is a continuous distribution with support between 0 and 1 and has two parameters denoted as a and b that determine the shape. When $a = b$, the distribution is symmetric.

One way of using a beta distribution that matches the prior probabilities (1.11) is as follows. Notice that the sum of the first 3 probabilities in (1.11) represents the probability that θ is smaller than or equal to 0.25. This probability is equal to 0.50. Similarly, the sum of the first 5 probabilities is the probability that θ is smaller than or equal to 0.45. This probability is equal to 0.91. The values of θ equal to 0.25 and 0.45 are two quantiles. Let $F(0.25; a, b)$ and $F(0.45; a, b)$ represent the cumulative distribution functions (cdf) of the beta distribution for $\theta = 0.25$ and for $\theta = 0.45$, respectively (the cdf is $F(x; a, b) = \Pr(X \leq x; a, b)$). Then the parameters a and b of the beta distribution that match the prior probabilities (1.11) can be found by minimising the function

$$(F(0.25; a, b) - 0.5)^2 + (F(0.45; a, b) - 0.91)^2$$

with respect to a and b . This can be achieved using the function `OPTIM` in R as indicated in the following code

```
mod <- function(par){
  a <- par[1]
  b <- par[2]
  fct <- (pbeta(0.25,a,b)-0.5)^2 + (pbeta(0.45,a,b)-0.91)^2
  return(fct)
}
res <- optim(par=c(3,3),mod)
res$par
```

```
## [1] 2.89705 8.04717
```

The function returns $a = 2.90$, $b = 8.05$. As a check one can compute the cumulative distribution functions

```
pbeta(0.45,2.9,8.05)
```

```
## [1] 0.9099298
```

```
pbeta(0.25,2.9,8.05)
```

```
## [1] 0.4995856
```

Figure 1.5 displays the discrete prior defined by (1.10) and (1.11) and the prior based on $Be(2.90, 8.05)$.

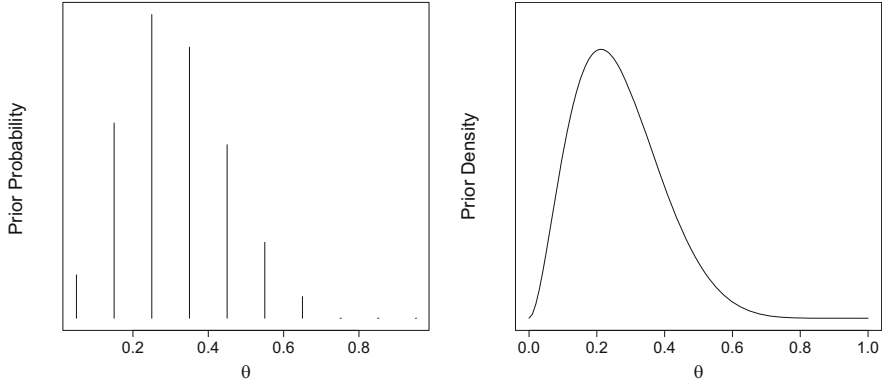


Fig. 1.5 Left: Discrete prior distribution defined by (1.10) and (1.11). Right: Beta prior $Be(2.90, 8.05)$

As mentioned above the likelihood is proportional to (1.1); that is

$$L(\theta|x, n) \propto \theta^x (1 - \theta)^{n-x}. \quad (1.13)$$

Seen as a function of θ , this is the kernel of a beta distribution with $a = x + 1$ and $b = n - x + 1$. The pdf (probability density function, sometimes referred to as density function) of the beta distribution is

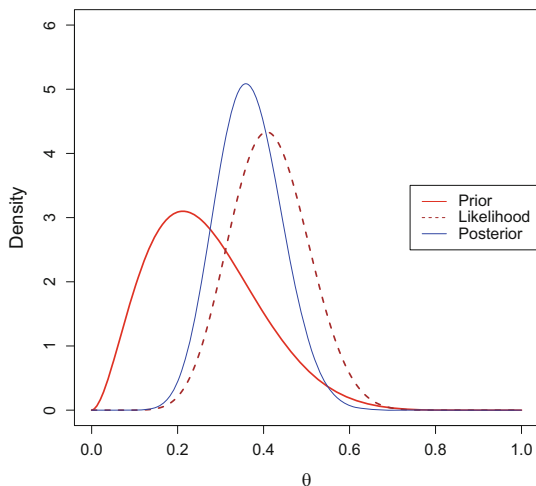
$$p(\theta) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{a-1} (1-\theta)^{b-1}, \quad \theta \in [0, 1], \quad a, b > 0,$$

where Γ is the gamma function. The posterior distribution is obtained by combining this likelihood with the prior $Be(2.90, 8.05)$. This results in a posterior distribution that has the beta density with parameters are $x + 1 + 2.90 - 1$ and $n - x + 1 + 8.05 - 1$. The posterior distribution has the form

$$p(\theta|x = 11, n = 27) = Be(13.90, 24.05). \quad (1.14)$$

Figure 1.6 displays plots of the prior, the likelihood and the posterior for the example. The posterior distribution is sharper than the prior distribution and its probability mass is concentrated between that of the prior distribution and the likelihood. In Bayesian inference, the posterior distribution provides the necessary information for drawing conclusions about θ . For example, the mean, mode and median of (1.14) are 0.37, 0.36, 0.36. The 95% posterior interval for the posterior mean is (0.24, 0.50). The inference using the Bayesian approach is sharper than that based on the likelihood (1.5). Further, the frequentist confidence interval and the Bayesian posterior intervals have different interpretations. In the latter the

Fig. 1.6 The prior density $Be(2.90, 8.05)$, the likelihood $Be(12, 17)$ and the posterior density $Be(13.90, 24.05)$ of the probability θ



confidence interval is fixed and the associated probability is the probability that the true parameter falls in the interval.

Note If a prior for θ is $Be(a, b)$, then $p(\theta) \propto \theta^{a-1} (1-\theta)^{b-1}$. The likelihood of the binomial model is proportional to $\theta^x (1-\theta)^{n-x}$ which is the kernel of $Be(x+1, n-x+1)$. The posterior is then proportional to $\theta^{a+x-1} (1-\theta)^{b+n-x-1}$ that is the kernel of $Be(a+x, b+n-x)$.

Prior Influence on Inferences

A Bayesian analysis is seldom complete without investigating how prior information affects the conclusions. In this example one can compare the inference about θ using either the discrete or the beta prior. With the former the mean is 0.38 and the probability that θ falls in the set $\{0.25, 0.35, 0.45\}$ is 0.94. The numerical values arrived at with the beta prior are quite similar.

In a situation where no prior information is available about θ , one could either resort to maximum likelihood, or to a Bayesian approach with a non-informative prior. The development of non-informative (or reference) priors can become quite technical, especially in complex model scenarios; a pragmatic approach is often chosen along the following lines. In the absence of information about θ , the investigator may consider three possible beta distributions as displayed in Fig. 1.7 (taken from Carlin and Louis 1996). The so-called Jeffrey's prior is transformation invariant; the $Be(1, 1)$ is a special case that retrieves a uniform distribution (assigning equal probabilities to all values of θ); the $Be(2, 2)$ is mildly informative assigning larger probabilities to intermediate values of θ . The combination of these

Fig. 1.7 The densities $Be(0.5, 0.5)$, $Be(1, 1)$ and $Be(2, 2)$ to model prior information about θ

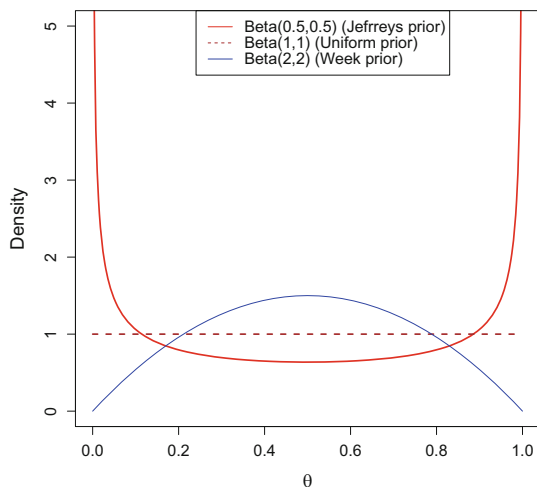
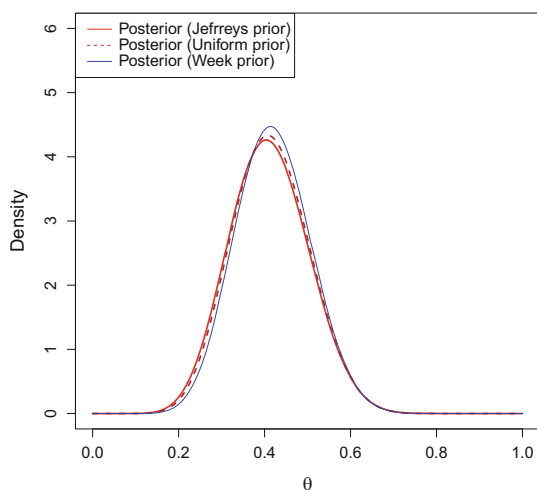


Fig. 1.8 The three posterior distributions corresponding to the three priors of Fig. 1.7 when $n = 27$, $x = 11$



priors with the likelihood $\theta^{11} (1 - \theta)^{16}$ gives rise to the three posterior distributions shown in Fig. 1.8.

In this particular example three very different prior distributions give rise to very similar posterior distributions. This is often the case when the likelihood is very informative relative to the prior distribution. Using the uniform prior $Be(1, 1)$, the posterior is $Be(12, 17)$ with mean value (the mode and median are almost the same) and 95% posterior interval equal to 0.41 and (0.24, 0.59), respectively. The posterior interval is a little wider than that based on the sharper prior $Be(2.90, 8.05)$ and almost identical (numerically) to the one based on the normal approximation to

the maximum likelihood estimator. The posterior probability that θ is less than or equal to 0.2 is

$$\int_{\theta=0}^{\theta=1} I(\theta \leq 0.2) p(\theta|y, n) d\theta = 0.00496. \quad (1.15)$$

Simulating from the Posterior Distribution

Inferences drawn in the examples above were exact. This is possible when the analytical form of the posterior distribution is known and features from it (such as probability intervals and moments) can be calculated. In principle, features from any posterior distribution can also be obtained using samples drawn from it, making use of standard theorems from the time series literature. For example, any function of the random variable X , $h(X)$ with finite expectation $E(h(X))$ can be approximated by

$$E(h(X)) \approx \frac{\sum_{i=1}^N h(x_i)}{N}, \quad (1.16)$$

where N is sample size. Using R, a sample $(x_i)_{i=1}^{100,000}$ of size 100,000 from the posterior $Be(12, 17)$ results in a sample mean

$$E(X) \approx \sum_{i=1}^{100,000} \frac{x_i}{100,000} = 0.41.$$

A 95% posterior interval for θ can be estimated as the 2.5th to 97.5th percentiles of the empirical distribution of the draws x_i using the R function `quantile`. If the original dataset is denoted `dat`, the R code is:

```
set.seed(7117)
dat <- rbeta(100000, 12, 17)
quantile(dat, c(0.025, 0.975))
```

```
##      2.5%      97.5%
## 0.2454898 0.5942647
```

Finally, using the simulated values, a Monte Carlo estimate of the posterior probability that θ is less than or equal to 0.2 is obtained using

$$\hat{\Pr}(\theta \leq 0.2) = \frac{1}{100,000} \sum_{i=1}^{100,000} I(x_i \leq 0.2) = 0.00499,$$

which is a Monte Carlo estimator of (1.15). These figures are in good agreement with the exact results.

In this example it was straightforward to sample directly from the posterior distribution because the normalising constant (1.9) is known, and therefore the form of the posterior is fully specified. Often the normalising constant cannot be obtained in closed form, particularly when θ contains many elements. Chapter 4 discusses how Monte Carlo draws from the approximate posterior distribution can still be obtained using Markov chain Monte Carlo (MCMC) methods.

An important issue with inferences based on Monte Carlo samples from posterior distributions is the accuracy of posterior summaries. The latter are subject to sampling uncertainty that depends on the size of the Monte Carlo sample and on the degree of autocorrelation of the samples. Methods to quantify this uncertainty are reviewed in Chap. 4.

Estimating Moments Using (Correlated) Samples from Posterior Distributions

Result (1.16) is extremely useful and is applied routinely in an MCMC environment to estimate features from posterior distributions. Typically, the elements that constitute the sample are correlated. However, despite this correlation structure, consistent estimators of features of posterior distributions can be obtained. For example, the sample mean

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^N x_i \quad (1.17)$$

the lag- k sample autocovariance

$$\hat{\gamma}(k) = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})(x_{i+k} - \hat{\mu}) \quad (1.18)$$

and the lag- k sample autocorrelation

$$\hat{\rho}(k) = \frac{\hat{\gamma}(k)}{\hat{\gamma}(0)} \quad (1.19)$$

are consistent estimators of the respective population parameters. In (1.19), $\hat{\gamma}(0)$ is the sample variance.

As an example, consider generating draws from the lag-1 autoregressive model

$$x_t = \rho x_{t-1} + e_t, \quad |\rho| < 1, \quad e_t \sim N(0, \sigma^2 = 1), \quad t = 1, \dots, N, \quad (1.20)$$

where the e'_s are iid (independently and identically distributed). Using R, I simulated $N = 1000, 10,000$ and $100,000$ observations from (1.20) using $\rho = 0.8$ and $\sigma^2 = 1$, with the initial condition $x_1 \sim N(0, 1)$. This generates a strongly autocorrelated structure among the draws. The marginal mean and variance of this process are

$$\begin{aligned} E(x_t) &= 0, \\ \text{Var}(x_t) &= \frac{\sigma^2}{1 - \rho^2}. \end{aligned}$$

The estimates of the mean (0.0), variance (2.778) and correlation (0.8) with samples of size $N = 1000, N = 10,000$ and $N = 100,000$ are $(-0.060, 2.731, 0.793)$, $(-0.046, 2.749, 0.797)$ and $(0.018, 2.774, 0.800)$, respectively. Despite the rather strong degree of autocorrelation, the estimates are quite acceptable and get closer to the true values as sample size increases.

1.5 Frequentist or Bayesian?

There has been much heated debate between frequentists and Bayesians about the advantages and shortcomings of both methods of inference. One can easily construct examples where one of the methods gives silly answers and the other performs fairly well. One such is the following. Imagine that in the situation discussed above, rather than obtaining $x = 11$ copies of allele A in a sample of size $n = 27$, one obtained $x = 0$. This outcome is not unlikely when θ is small; for example, the probability of obtaining $x = 0$ when $n = 27$ and $\theta = 0.04$ is $\Pr(x = 0 | n = 27, \theta = 0.04) = 0.33$. In this situation the maximum likelihood (ML) estimate (1.2) is 0 and its variance is also 0; clearly a silly result (classical maximum likelihood is problematic when the estimate lies on the boundary of the parameter space).

How does the Bayesian approach behave in such a situation? The three posterior distributions corresponding to the three priors of Fig. 1.7 are shown in Fig. 1.9. Jeffrey's prior and the uniform prior, when combined with the likelihood, proportional to $(1 - \theta)^{27}$, lead to posterior distributions $Be(0.5, 27.5)$ and $Be(1, 28)$, respectively; these have modal values of 0. The weakly informative prior $Be(2, 2)$ yields a posterior of the form $Be(2, 29)$, which has a mode at $\theta \approx 0.034$. The posterior means of these three distributions are 0.018, 0.034 and 0.065, respectively. The 95% posterior intervals are

$$\begin{aligned} &(0.18 \times 10^{-4}, 0.88 \times 10^{-1}), \\ &(0.90 \times 10^{-3}, 0.12), \\ &(0.81 \times 10^{-2}, 0.17), \end{aligned}$$

Fig. 1.9 The three posterior distributions corresponding to the three priors of Fig. 1.7 when $n = 27$, $x = 0$

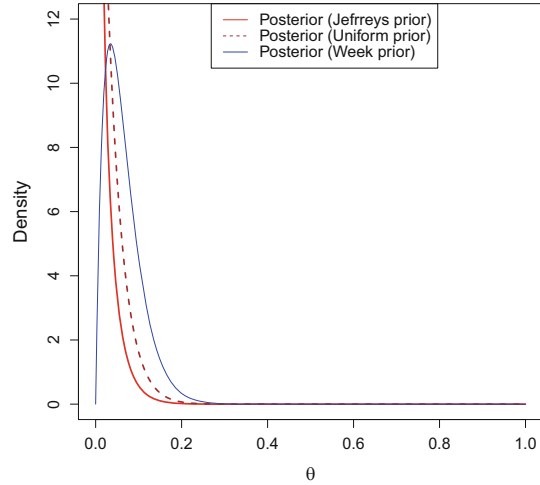
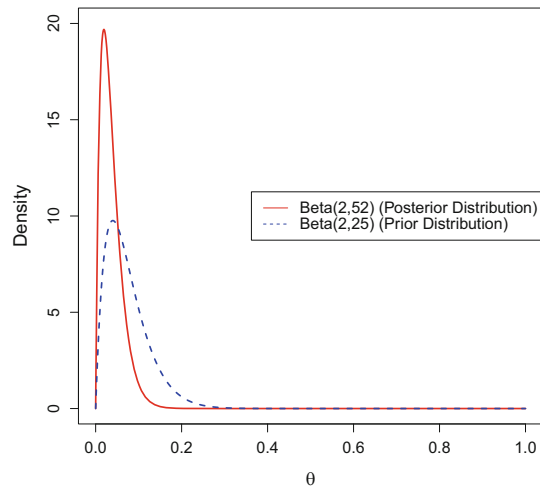


Fig. 1.10 A $Be(2, 25)$ prior distribution and a $Be(2, 52)$ posterior distribution when $n = 27$, $x = 0$



respectively. The posterior probability that θ is less then or equal to 0.05 for $Be(0.5, 27.5)$, $Be(1, 28)$ and $Be(2, 29)$ are 0.91, 0.76 and 0.45, respectively. In this extreme situation, prior information plays an important role (certainly, compared to the case $n = 27$, $x = 11$, displayed in Fig. 1.8).

One may consider other priors from the beta family that put strong probability mass in the neighbourhood of zero. One possibility is to use a $Be(2, 25)$ that has a mode at $\theta = 0.04$. With $n = 27$ and $x = 0$ this results in a posterior $Be(2, 52)$. The prior and posterior distributions are plotted in Fig. 1.10. The mean and the mode of the posterior distribution are 0.037 and 0.019, respectively. The 95% posterior interval is now $(0.005, 0.101)$ and $\Pr(\theta \leq 0.05 | n = 27, x = 0) = 0.75$.

The beta prior does not assign probability mass to $\theta = 0$ and this rules out the possibility that $\theta = 0$ in its posterior distribution. One way of including 0 as a

possible value for θ is to use a two-component mixture prior, where one component is a point mass at zero and the other is a beta prior. Mixture distributions are discussed in Chaps. 3, 7, and 9.

There is flexibility associated with the Bayesian approach and a carefully chosen prior distribution will lead to stable inferences about θ . The cost is a result which is partly affected by prior input. With small amount of data and when parameters lie on the border of the parameter space, there is little else to choose from. In such a situation the most important role of prior distributions may well be to obtain inferences about θ that are stable and that provide a fair picture of posterior uncertainty, *conditional on the model*.

Frequentist and Bayesian

There are situations where instead of choosing between *Frequentist* or *Bayesian* one could use *Frequentist and Bayesian* tools in a meaningful way. This is the case with model checking where one is interested in studying the ability of a model to account for particular features of the data, or to give reasonable predictions of future observations. Key literature is Rubin (1984), Gelman et al. (1995), and Gelman et al. (1996). Suppose that data vector y (length n) is a realisation from the sampling model $p(y|\theta_M, M)$, where θ_M is a vector of parameters and M represents the assumed model. If this assumption is adequate, then one would expect that a new realisation from $p(\cdot|\theta_M, M)$ should result in a vector y_{rep} say, that resembles y . Instead of working in n dimensions, one can construct a scalar function T of the data and θ_M , $T(y, \theta_M)$, designed to study a particular feature of the data that is scientifically relevant. One can then compare the observed value of $T(y, \theta_M)$ with its sampling distribution under $p(\cdot|\theta_M, M)$. An observed value that falls in the extreme tails of the sampling distribution indicates a potential failing of the model to account for T . Equivalently one can study whether zero is an atypical value in the distribution of the difference $T(y, \theta_M) - T(y_{rep}, \theta_M)$.

Parameter θ_M is typically unknown. The frequentist proposition is to replace θ_M by some point estimator $\hat{\theta}_M$ and then proceed as above treating θ_M as known and equal to $\hat{\theta}_M$.

A Bayesian, rather than generating data y_{rep} from $p(y_{rep}|\hat{\theta}_M, M)$ does so from $p(y_{rep}|y, M)$, the density of the posterior predictive distribution, given by

$$p(y_{rep}|y, M) = \int p(y_{rep}|\theta_M, y, M) p(\theta_M|y, M) d\theta_M \quad (1.21)$$

where often, $p(y_{rep}|\theta_M, y, M) = p(y_{rep}|\theta_M, M)$. One then observes whether zero is an extreme value in the posterior predictive distribution $T(y, \theta_M) - T(y_{rep}, \theta_M)$, where θ_M is generated from the posterior $[\theta_M|y, M]$ and given θ_M , y_{rep} is generated from $p(y_{rep}|\theta_M, M)$. This “Bayesian frequentist”, like the frequentist, accounts for the uncertainty in y_{rep} due to the sampling process from $p(y_{rep}|\theta_M, y, M)$. However, unlike the frequentist, account is also taken of the uncertainty about θ_M described by its posterior distribution $[\theta_M|y, M]$. Model checking applied in this manner, although embedded in the Bayesian paradigm, is frequentist in spirit

because it decides whether the observed data look reasonable under the posterior predictive distribution based on repetitions of data that could have been generated by the model. All this is typically carried out using MCMC methods that provide great flexibility to question models.

Model checking using posterior predictive distributions is discussed in Chap. 10.

1.6 Prediction

The second part of the book provides an introductory overview of prediction. A stylised setup is as follows. Data matrix z has the structure $z_i = (x_i, y_i)$, $i = 1, 2, \dots, n$, where x_i is a p -dimensional vector of covariates (or predictor variables), y_i , a scalar, is a response variable and the n vectors are independent and identically distributed realisations from some distribution. Examples of both parametric (frequentist and Bayesian) and nonparametric models are given here. In the case of parametric models where the response y is quantitative, a general form for the association between x and y is

$$y_i = f(x_i) + e_i, \quad (1.22)$$

where f , the conditional mean, is a fixed unknown function of x_i and of parameters; e_i is a random error term, assumed independent of x_i , with mean zero. Using the data z , an estimate of f labelled $\hat{f}$ is obtained by some method that leads to $\hat{E}(y_0|x_0) = \hat{y}_0$, a point prediction of the average value of y_0 , evaluated at a new value of the covariate $x = x_0$ (in a frequentist setting, conditional on estimates of parameters that index f)

$$\hat{y}_0 = \hat{f}(z, x_0). \quad (1.23)$$

The notation emphasises that the estimation procedure inputs data z and yields a prediction $\hat{y}_0$ for $x = x_0$. For example, in standard least squares regression $f(x_i) = E(y_i|x_i) = x_i'b$, $\hat{f}(z, x_0) = x_0'\hat{b}$, where $\hat{b} = (x'x)^{-1}x'y$ and x_i' is the i th row of matrix x .

With binary responses one may fit a logistic regression. Here the modelling is at the level of the probability. Specifically, letting $\Pr(y_i = 1|x_i) = \pi(x_i)$, the logistic model can be written as

$$\ln \left[\frac{\pi(x_i)}{1 - \pi(x_i)} \right] = x_i'b, \quad i = 1, 2, \dots, n.$$

A maximum likelihood estimate $\hat{b}$ together with a new input x_0 results in a predicted probability $\hat{\pi}(x_0)$, that can be transformed into a predicted value according to the rule

$$\hat{y}_0 = \begin{cases} 1 & \text{if } \hat{\pi}(x_0) \geq 0.5 \\ 0 & \text{if } \hat{\pi}(x_0) < 0.5. \end{cases} \quad (1.24)$$

Measuring Prediction Performance

The performance of the predictions can be evaluated measuring how well they match observed data. One measure of predictive performance is the sample mean squared error (MSE)

$$\text{MSE}_t = \frac{1}{n_t} \sum_{i=1}^{n_t} (y_i - \hat{y}_i)^2 \quad (1.25)$$

where n_t is the number of records and $\hat{y}_i = \hat{f}(z, x_i)$. When MSE_t (1.25) is computed using the data that was used to fit the model, the *training data*, it is known as the sample *training mean squared error*. If the objective is to study how well the model predicts a yet-to-be-observed record, MSE_t can be misleading as it overestimates predictive performance. In fact it can be made arbitrarily small by including a large number of covariates.

A more reliable measure of the prediction ability of the model is to test how well predictions match observations from a new sample of data z_0 (or hold-out data), the *validating data*, drawn from the same distribution as the training data. The validating data is $z_{0i} = (y_{0i}, x_{0i})$, $i = 1, 2, \dots, n_v$ and the sample *validating mean squared error* is

$$\text{MSE}_v = \frac{1}{n_v} \sum_{i=1}^{n_v} (y_{0i} - \hat{y}_{0i})^2. \quad (1.26)$$

In (1.26) $\hat{y}_{0i}$ is the i th prediction computed using the training data z evaluated at the value of the i th covariate x_{0i} . That is, $\hat{y}_{0i} = \hat{f}(z, x_{0i})$. With binary observations (1.25) and (1.26) represent the proportion of cases where $\hat{y}_{0i} \neq y_{0i}$, or misclassification error. Other approaches to evaluate predictors of binary data are discussed in Chap. 9.

As an illustration of some of these concepts I use data from a microarray study of prostate cancer from Singh et al. (2002). The study includes 52 men with tumour, 50 healthy men and a total of $n = 102$ men. The genetic expression of a panel of $p = 6033$ genes was measured for each man. The level of gene expression is associated with the level of activity of the gene. A larger number implies a more active gene. The $n \times p$ matrix of covariates is then $x = \{x_{ij}\}$, $i = 1, 2, \dots, n = 102$; $j = 1, 2, \dots, p = 6033$, with $p \gg n$.

Table 1.1 Training and validating mean squared errors for the prostate cancer data, as the number of covariates included in the linear predictor increases from 5 to 30. A standard logistic model is implemented and the mean squared errors represent the proportion of misclassifications in the training and validating data

No. covariates	5	10	15	20	25	30
MSE_t	0.29	0.31	0.16	0.12	0.06	0.00
MSE_v	0.27	0.37	0.25	0.33	0.35	0.39

Fitting Traditional Logistic Regression

To illuminate some of the consequences of overfitting, a first analysis is undertaken with traditional logistic regression models involving $p < n$. Data are divided into training and validating sets with equal numbers in each. The logistic models are fitted to the training data using maximum likelihood and the estimates of the parameters are used to predict the outcome (healthy/not healthy) in the validating data. The models differed in the number of covariates included. The change in MSE_t and MSE_v as the number of covariates (columns in x) increases in the different models from $p = 5$ to $p = 30$ in steps of 5, is shown in Table 1.1 for one training/validating split. The covariates were arbitrarily chosen as the first 5 columns of x , the first 10 and so on. The figures in the table show an increase in MSE_v as the number of covariates increases beyond 15 and a parallel increase of the overstatement of the model's predictive ability, as judged by the steady fall in MSE_t when the number of covariates is larger than 10.

The code below reads the data (singh2002), splits it into a training and a validating/testing set (y.test and y.train) and in the bottom part,

- fits a logistic regression to the training data using the R-function GLM
- using the ML estimates computes the predicted liabilities in the training and validating data
- based on these liabilities computes $\Pr(Y = 1|\hat{b})$, where $\hat{b}$ is the ML estimate
- transforms the probabilities into the 0/1 scale
- computes MSE (misclassification error) in the training and validating data

The figures in Table 1.1 were generated using this code. The code illustrates the case with 15 covariates (first 15 columns of matrix x). The output agrees with the figures in the third column of the table.

```
# CODE0101
# READING SINGH ET AL 2002 DATA
rm(list=ls()) # CLEAR WORKSPACE
# Lasso solutions using package glmnet
#install.packages("glmnet", .libPaths()[1])
#install.packages("sda")
library("sda")
```

```
library(glmnet)
```

```
data(singh2002)
X<-singh2002$x
y<-ifelse(singh2002$y=="cancer",1,0)
n<-nrow(X)
Xlasso<-X
set.seed(3037)
train=sample(1:nrow(X),nrow(X)/2)
test=(-train)
y.test=y[test]
y.train<-y[train]
#
# RESAMPLES TRAIN/TEST DATA AND COMPUTES MSE
# FOR EACH RESAMPLE/REPLICATE
t1 <- seq(1,15,1) # CHOOSE THE FIRST 15 COLUMNS OF x
X1 <-X[,t1]
n <- length(t1)
datarf <- data.frame(cbind(y,X))
nc <- 1 # EXAMPLE WITH 1 REPLICATE ONLY
res <- matrix(data=NA, nrow=nc,ncol=3)
for(i in 1:nc){
  if(i > 1){train <- sample(1:nrow(datarf),nrow(datarf)/2)}
  glm.fit <- glm(y[train] ~ X1[train,] ,
                 family=binomial(link="logit"))
  # CALCULATE PREDICTED LIABILITY FOR THE TRAINING (liabT) AND THE
  # VALIDATING DATA (liabV)
  liabV <- X1[-train,1:n]%%glm.fit$coefficients[2:(n+1)]+
    glm.fit$coefficients[1]
  liabT <- X1[train,1:n]%%glm.fit$coefficients[2:(n+1)]+
    glm.fit$coefficients[1]
  # COMPUTE Pr(Y=1) BASED ON THESE LIABILITIES
  probT <- exp(liabT)/(1+exp(liabT))
  probV <- exp(liabV)/(1+exp(liabV))
  # COMPUTE PREDICTED VALUES IN TRAINING AND VALIDATING DATA
  # ON THE 0/1 SCALE
  predT <- ifelse(probT > 0.5, "1", "0")
  predV <- ifelse(probV > 0.5, "1", "0")
  # COMPUTE MISCLASSIFICATION ERROR
  predclassT <- mean((as.numeric(predT) - y.train)^2)
  predclassV <- mean((as.numeric(predV) - y.test)^2)

  # IF CURIOUS COMPUTE LOG-LIKELIHOOD, DEVIANCE,
  # AIC USING TRAINING DATA
  # ll <- sum(y.train*liabT) - sum(log(1+exp(liabT)))
  # dev <- -2*ll
  # AIC <- dev + 2*(n+1)
  # *****

  res[i,] <- c(n,predclassT,predclassV)
}
res
```

```
##      [,1]      [,2]      [,3]
## [1,]    15 0.1568627 0.254902
```

The Setup Where the Number of Covariates Is Larger than the Number of Individuals

In the vast majority of genomic studies the number of covariates p (for example, genetic markers, gene expressions) measured in each individual is considerably larger than the number of individuals n ; this is the classical $p \gg n$ scenario. In the context of prediction, these covariates are incorporated in linear regression or logistic regression models, but only a subset are likely to contribute meaningfully. Inclusion of redundant covariates may improve model fit (reflected in small values of training mean squared error, MSE_T), but will definitely result in poor predictions. From the point of view of model implementation, when $p \gg n$, some form of regularisation or shrinkage is needed. This constitutes an important topic of the book. Three examples are provided in this overview. The first is a parametric model; the other two are nonparametric approaches specifically developed, but not restricted to, to deal with the $p \gg n$ situation. Like many of the tools used for the analysis of the type of data sets commonly found in genomic studies, these models can be useful as guidance in the choice of predictors for further study.

Fitting the Lasso

The parametric example is based on the *lasso* (Tibshirani 1996, “least absolute shrinkage and selection operator”) that is a regularisation method with a tuning parameter governing the amount of shrinkage of the regression parameters towards zero. Since the lasso solutions typically include many coefficients equal to zero when the tuning parameter is sufficiently large, it does model selection and shrinkage simultaneously.

The lasso logistic regression model is fitted using the public package **glmnet** (Friedman et al. 2009) implemented in R. Documentation about **glmnet** can be found in Hastie and Qian (2016) and in Friedman et al. (2010).

To obtain predictions the code below executes first the function **cv.glmnet** on the training data in order to find the value of the tuning parameter (λ) that optimises prediction ability measured by MSE. In a second step **glmnet** is executed again on the training data using this best λ to obtain the final values of the regression parameters. The code then constructs the predictions from the output of this second run. The model is finally tested on the training and on the validating data.

A more direct implementation of **glmnet** without the need to generate estimates of regression parameters is indicated at the bottom of the code.

The lasso logistic regression was run on the prostate data including all 6033 covariates representing the gene expression profiles. Lasso chooses 36 covariates and sets the remaining equal to zero. The model with these 36 covariates was used to classify the observations in the validating data and resulted in a MSE_v equal to 0.25. In other words, $51 \times 0.25 \approx 13$ out of the 51 observations in the validating data are incorrectly classified. At face value, the result for MSE_v matches that obtained in Table 1.1 when the first 15 columns of x were included in the linear predictor.

The latter can be interpreted as a logistic regression model where 15 out of 6033 covariates are randomly chosen. The result based on the lasso is not encouraging.

```
# CODE0102
# READING SINGH ET AL 2002 DATA
rm(list=ls()) # CLEAR WORKSPACE
# Lasso solutions using package glmnet
#install.packages("glmnet", .libPaths()[1])
#install.packages("sda")
library("sda")
library(glmnet)
data(singh2002)
X<-singh2002$x
y<-ifelse(singh2002$y=="cancer",1,0)
n<-nrow(X)
Xlasso<-X
set.seed(3037)
train=sample(1:nrow(X),nrow(X)/2)
test=(-train)
y.test=y[test]
y.train<-y[train]
#
# ***** FOR PREDICTION USING LASSO *****
repl <- 1 # NUMBER OF REPLICATES
# (RESAMPLES TRAINING/VALIDATING)
result <- matrix(data=NA, nrow=repl,ncol=4)
set.seed(3037)
for (i in 1:repl){
  if(i > 1){train <- sample(1:nrow(Xlasso),nrow(Xlasso)/2)}
  y.train <- y[train]
  y.test <- y[-train]
# STEP 1: cross-validation; find best value of lambda
# alpha=1: LASSO; alpha=0: RIDGE REGRESSION
  cv.out=cv.glmnet(Xlasso[train,],y[train],alpha=1,
    family="binomial",type = "class")
  #plot(cv.out)
  bestlam=cv.out$lambda.min
  #bestlam

# Using best lambda, fit model on training data
# to obtain final parameter estimates

# STEP 2
  fm=glmnet(y=y[train],x=Xlasso[train,],alpha=1,lambda=bestlam,
    family="binomial",type.measure= "class")
  nzcfc<-coef(fm)
  cf<-which(fm$beta[,1]!=0)
  if (length(cf) == 0){
    out <-c(i,length(cf))
    print(out)
    break
  }
}
#length(cf) # NO. REGRESSION PARAMETERS IN FINAL MODEL
```

```
# CONSTRUCT PREDICTIONS FROM OUTPUT OF fm
# 1. VALIDATING DATA
predglmnet<-fm$a0+Xlasso[-train,cf]%*%fm$beta[cf]
probs <- exp(predglmnet)/(1+exp(predglmnet))
predclass_test <- as.numeric(ifelse(probs > 0.5, "1", "0"))
# 2. TRAINING DATA
predglmnet<-fm$a0+Xlasso[train,cf]%*%fm$beta[cf]
probs <- exp(predglmnet)/(1+exp(predglmnet))
predclass_train <- as.numeric(ifelse(probs > 0.5, "1", "0"))
result[i,] <- c(mean((predclass_train-y.train)^2),
               mean((predclass_test-y.test)^2),bestlam,length(cf))
}
result
```

```
##      [,1]      [,2]      [,3] [,4]
## [1,]    0 0.254902 0.01948886   36
```

```
#NOTE: for prediction, GLMNET can be implemented more directly,
# using in STEP2:
#####
# fm.predclass=predict(cv.out,s=bestlam,newx=Xlasso[test,],
#   family="binomial",type="class")
# mean((as.numeric(fm.predclass)-y.test)^2) # VALIDATION ERROR
# RATE (BASED ON CLASS LABELS)
#####
```

The somewhat disappointing performance of the lasso was investigated further by random splitting the training/testing dataset 50 times. This provides a picture of the sampling variation of the MSE over the joint distribution of training/testing data. The mean validating mean squared error, over the 50 replicates was 0.30, with a minimum of 0.16 and a maximum of 0.53. The number of covariates not set equal to zero ranged from 2 to 41 with a median of 24. There is not a model that is consistently singled out as a good predictor over replications. This is a reflection of the ubiquitous multicollinearity in a multidimensional setting where covariates become highly correlated (a covariate can be expressed as a linear combination of others). Therefore, a different set of covariates is chosen in each replication.

Fitting a Classification Tree

The first of the two nonparametric models that are fitted to the data is a **classification tree** (Breiman et al. 1984), that is described briefly via the example generated by the R-code below. The code executes the R function `tree`; this requires installation of the package `tree`. The data set `singh2002` includes 6033 covariates and the response variable is `y`: a binary classification variable with modalities “healthy” and “cancer”.

```
# CODE0103
rm(list=ls()) # CLEAR WORKSPACE
set.seed(30331)
#install.packages("tree")
library(sda)
library(tree)
# library(glmnet)
data(singh2002)
d <- data.frame(singh2002$x)
d$y <- singh2002$y
nrep <- 1 # NUMBER OF REPLICATES
res <- matrix(data=NA,nrow=nrep,ncol=3)
ptm<-proc.time()
for ( i in 1:nrep ) {
  cat(i,"\n",sep="")
  train <- c(sample(1:50,25),sample(51:102,26))
# FIT THE TREE TO THE TRAINING DATA
  trees <- tree(y ~ . , data=d[train,])
# FIT FUNCTION PREDICT TO THE TRAINING AND VALIDATING DATA
  predtreev <- predict(trees,d[-train,],type="class")
  predtreet <- predict(trees,d[train,],type="class")
# CLASSIFICATION ERROR IN TRAINING AND VALIDATING DATA
  predv <- sum(predtreev==d$y[-train])/length(d$y[-train])
  predt <- sum(predtreet==d$y[train])/length(d$y[train])
# RECORD TRAINING/VALIDATING CLASSIFICATION ERROR AND
# NUMBER OF COVARIATES IN TREE
  res[i,]<-c((1-predt),(1-predv),length(summary(trees)$used))
}
```

```
## 1
```

```
proc.time()-ptm
```

```
##      user  system elapsed
##    2.95    0.17    3.12
```

```
res
```

```
##      [,1]      [,2] [,3]
## [1,]    0 0.1764706    2
```

```
tab <- table(predtreev,d$y[-train])
tab
```

```
##
## predtreev cancer healthy
## cancer      17      0
## healthy      9     25
```

```
# CHECK CLASSIFICATION ERROR
(tab[1,2]+tab[2,1])/(length(d$y[-train]))
```

```
## [1] 0.1764706
```

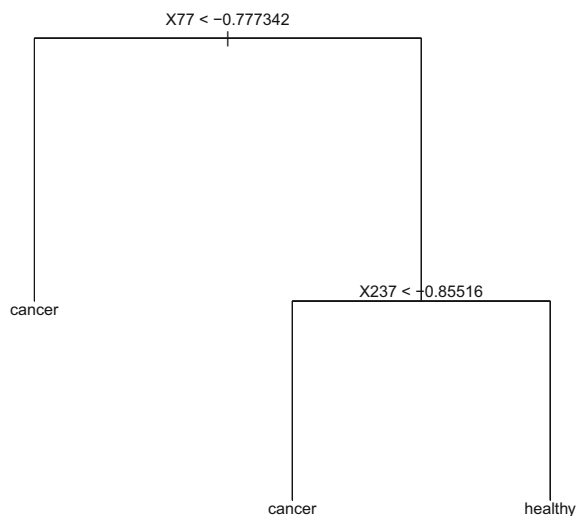
```
#summary(res)
#plot(trees)
#text(trees,pretty=0)
```

Figure 1.11 indicates that in the particular replicate, the algorithm isolated 2 of the 6033 covariates, X_{77} and X_{237} . Starting at the top of the tree, the 51 cases in the training data have been split into two groups: one, to the left, that shows expression profile for X_{77} less than a threshold $t_1 = -0.777$ and those to the right with $t_1 > -0.777$. The group on the left is not split further and constitutes a *terminal node*. On the right side, a second split based on the profile of X_{237} and a threshold $t_2 = -0.855$ gives rise to two terminal nodes. The result can be interpreted as an interaction between the two markers.

A predicted value is attached to each terminal node. For a new individual, a predicted value is obtained by starting at the top of the tree and following the splits downwards until the terminal node with its predicted value is reached. The new individual is assigned the prediction given by that terminal node. In the example, if the new individual were to show a value for $X_{77} = -0.3$ and for $X_{237} = -0.1$, following the tree from top to bottom would lead to a prediction taking the modality “healthy”.

Typing the tree object (in this case, *trees*) gives further details associated with the figure. Terminal nodes are indicated with asterisks. R prints output from each branch of the tree in the form of the *node*, *split criterion* t , the number of observations in the

Fig. 1.11 Output from a classification tree fitted to data `singh2002` from Singh et al. (2002) using the R function `tree` (R-code CODE0103). Results from one replicate



branch n , the *deviance*, the classification for the branch (“cancer”/“healthy” in the present case) and the proportion of observations in the branch that take the values “cancer”/“healthy”.

```
library(tree)
trees
```

```
## node), split, n, deviance, yval, (yprob)
##      * denotes terminal node
##
## 1) root 51 70.68 cancer ( 0.5098 0.4902 )
## 2) X77 < -0.777342 20  0.00 cancer ( 1.0000 0.0000 ) *
## 3) X77 > -0.777342 31 30.46 healthy ( 0.1935 0.8065 )
## 6) X237 < -0.85516 6  0.00 cancer ( 1.0000 0.0000 ) *
## 7) X237 > -0.85516 25 0.00 healthy ( 0.0000 1.0000 ) *
```

The output above indicates that at the top of the tree at X_{77} (which is the root since the tree is upside down), there are 51 records (the training data), the proportion of “cancer” is 0.5098. After the first split, to the left, the split is $t_1 < -0.777$, 20 observations are classified as “cancer” and 0 as “healthy” leading to proportions of (1.00, 0.00). To the right, the split is $t_1 > -0.777$ that gives rise to 31 records, with a proportion of “healthy” equal to 0.8065 (25 out of the 31 records are “healthy”, those whose $t_2 > -0.855$, associated with covariate X_{237}).

Various algorithms are available to decide which variable to split and the splitting value t to use for the construction of the tree. Some of these topics are deferred to the chapter on nonparametric methods. Here I concentrate on the predictive ability of the method. For the particular replicate the classification error in the training and validating data is 0 and 0.18, respectively. Replicating the experiment 50 times gives a mean classification error in training and validating data equal to 0.016 and 0.197, respectively, with (minimum, maximum) values of (0.000, 0.078) and (0.098, 0.333), respectively. With the parameters for the utility function `tree.control` set at the default values the number of covariates over the 50 replicates included in each tree fluctuates between 2 and 3 and these covariates vary over replicates. For these data the classification tree performs considerably better than the lasso.

Interestingly in all the cases the classification trees capture what can be interpreted as an interaction involving 2 or 3 covariates. These are not the same covariates for the 50 trees. Perhaps this must not come as a surprise: 6033 covariates give rise to more than 18 million different two-way interactions. Therefore, predictors based on interacting covariates are prone to be highly correlated. More generally and as noted with the lasso, in the high-dimensional setting the multicollinearity of the covariate matrix is often extreme and any of the p covariates in the $n \times p$ matrix can be written as a linear combination of the others. This means that there are likely many sets of pairs of covariates (other than X_{77} and X_{237}) that could predict just as well. It does not follow that the model cannot be trusted as a prediction tool, but rather that one must not overstate the importance of X_{77} and X_{237} as the only genes associated

with the response variable. As with the lasso, the analysis with the classification tree provides inconclusive evidence of specific genes affecting prostate cancer.

Fitting a Random Forest

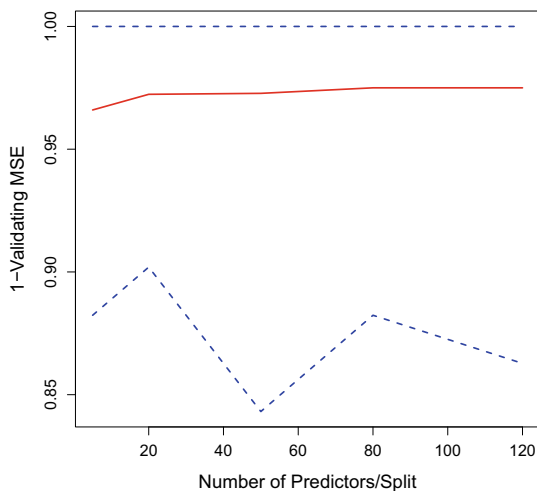
A problem often mentioned with trees is that they exhibit high variability. Small changes in the data can result in the construction of very different trees and their predictions can be impaired. However, they are an integral part of another method known as *random forest* (Breiman 2001) whose prediction performance benefits by the process of averaging. The random forest consists of many classification trees and each is created as follows:

- Create a sample of size n_v by drawing with replacement from the n_v data points in the training data. Repeat this B times to generate B samples. (With random forests there is an alternative way of estimating validating mean squared error using the entire data, without cross-validation. Details are discussed in Chap. 11).
- For each sample generate a classification tree. Each time a split in the tree is considered, a random sample of m unique predictors is chosen as split candidates from the p predictors. For classification it is customary to use $m \approx \sqrt{p}$. This step has the effect of *decorrelating* the ensemble of B trees (in classification trees the construction of a split involves all the predictors).

For classification, once the trees are available the final prediction is obtained by a majority vote. Thus, for $B = 10$, say, if for a particular observation in the validating data 6 or more trees classify it as “1”, the predicted value for this observation is “1”. The prediction obtained in this manner usually outperforms the prediction of classification trees. This improvement in performance arises from the fact that a prediction based on B predictors with very low correlation has smaller variance than a single prediction. The low correlation is ensured in the second step. The first step involving the bootstrapping of the training data is known as *bagging*, short for *bootstrap aggregating*, whereby the results of several bootstrap samples are averaged. (The *mean squared error* is a measure of the performance of a predictor, whose expectation includes the variance of the predictor, a squared bias term and a pure noise term associated with the variance of the predictand. Therefore, the performance of a predictor improves as its variance is reduced. This reduction is achieved by constructing a prediction averaged over several bootstrap samples whose variance is smaller than the variance of an estimate based on a single sample).

To study prediction ability, the random forest was implemented on the *singh2002* data set using the R function `RandomForest`. I executed 200 replicates (200 splits of training/testing data) and in each replicate the number of covariates included in the split of a particular tree ranged from 5 to 120 as indicated in the code below in the variable `mtry <- c(5, 20, 50, 80, 120)`. The average proportion of correct classifications in the validating data (“cancer”, “healthy”), as well as the minimum and maximum over the 200 replicates as a function of the number of covariates (*mtry* in the x -axis), is shown in Fig. 1.12.

Fig. 1.12 Average proportion of correct classifications in the validating data (in red) of a *random forest* over 200 replicates against the number of covariates included in the ensemble of trees. Maximum and minimum over replicates in blue



The results are quite impressive. The average proportion of correct classifications in the validating data is of the order of 97% with a minimum–maximum in the range 84–100%.

The code used to implement the random forest is shown below.

```
# CODE0104
#install.packages("randomForest")
rm(list=ls()) # CLEAR WORKSPACE
library(sda)
library(randomForest)
data(singh2002)
d <- data.frame(X=singh2002$x)
d$y <- singh2002$y
n0 <- sum(d$y=="healthy")
n1 <- sum(d$y=="cancer")
set.seed(3037)
p <- .5
nrep <- 1
mtry <- c(5,20,50,80,120)
sumd <- data.frame()
res <- rep(NA,nrep)
ptm<-proc.time()
for ( m in mtry) {
  cat("mtry ",m,"\\n",sep="")
  for ( rep in 1:nrep ) {
    cat("Replicate ",rep,"\\n",sep="")
    train <- c(sample( 1:n0,floor(p*n0) ),
               sample( (n0+1):(n0+n1),floor(p*n1) ))
    rf.singh =randomForest(y ~.,
                          data=d,
                          subset =train,
                          mtry=m,
```

```

                                importance =TRUE)
predict <- predict(rf.singh,d[-train,])
observed <- d$y[-train]
t <- table(observed,predict)
print(t)
res[rep] <- (t[1,1]+t[2,2])/sum(t)
}
sumd <- rbind(sumd,c(m,min(res),mean(res),median(res),
                    max(res),var(res)))
}
proc.time()-ptm
names(sumd) <- c("mtry","min","mean","median","max","var")

with(sumd,plot(mtry,mean,type="l",col="red",ylim=c(min(min),1),
              ylab="1 - Mean Squared Error",
              xlab="Number of Predictors Considered at each Split"))
with(sumd,lines(mtry,min,lty=2,col="blue"))
with(sumd,lines(mtry,max,lty=2,col="blue"))

```

While in this particular set of data the random forest was the clear winner among the prediction machines tested, it is important to mention that there is no uniformly best prediction machine. A different set of data may produce different results. Very marked differences among prediction methods ought to raise suspicion and warrants careful investigation of the data (Efron 2020). This is particularly important in this era of increasingly larger data sets where the consequence of bias due to non-random sampling is magnified. The point is elaborated in Meng (2018). Spurious results may be obtained by complex interactions between a prediction method and a particular structure in the data at hand that may not be reproduced when the model is deployed using a new sample of data.

Examples of Statistical Approaches for Discovery of Promising Covariates

One of the objectives of the experiment above is to find genes that may have an effect on prostate cancer. This section outlines the results of applying two approaches for gene discovery. The first approach is a classical genomewide association study (GWAS) whereby covariates are fitted one at a time, regressing the phenotype on each covariate in turn using a logistic regression. The statistical significance of each covariate is gauged by a simple t -test using a Bonferroni adjustment to correct for multiple testing. The classical GWAS with a Bonferroni implementation has the attraction of simplicity and of minimising the proportion of false positive results, but it suffers from lack of power relative to other more refined alternatives. It is therefore complemented with the calculation of false discovery rate (Benjamini and Hochberg 1995) to create a discovery set of potentially interesting genes for further study. The subject is pursued in Chap. 8.

The second approach is based on a Bayesian implementation of a spike and slab model. In the parametrisations used in this book, the average substitution effects of the marker loci are a priori assigned a two-component mixture distribution. With unknown probability π , one component (the slab) is Gaussian with mean zero and

unknown variance; with probability $(1 - \pi)$ the other component (the spike) is a degenerate distribution with a point mass at zero (see page 350 for details). One of the outputs of the implementation of the spike and slab model is the posterior probability that a covariate is a draw from the slab (has an effect on the response), for all the covariates. This information is particularly useful for gene discovery and as shown in Chap. 8, it is an important ingredient of the Bayesian estimator of false discovery rate (the proportion of false discoveries among the panel of covariates included in the discovery set). In addition, measures of uncertainty are easily accessed using the MCMC implementation of the spike and slab model that produces a richer output than the GWAS but is more demanding computationally.

An important detail of the implementation with the spike and slab model is that the data-collection mechanism is not accounted for (see page 408) and in both implementations, covariates are assumed to act additively on the output. The possibility of interactions, explored in the case of classification trees and random forests, is not investigated.

Figure 1.13 displays the results of the analysis of the Singh et al. (2002) data using both methods. The left panel corresponds to the GWAS implementation. Four genes are in the proximity of reaching the Bonferroni threshold (5.08) and one (gene 610) mildly exceeds it (5.67). The Bonferroni approach is very conservative and prone to missing potentially useful covariates. To mitigate this shortcoming, applying methods discussed in Chap. 8 that use as input the p -values from the GWAS study (Benjamini and Hochberg 1995), a discovery set of 5 genes is identified as candidates for further study. As applied here, the method asserts that on average, in conceptual replications of the experiment, the proportion of false positive genes in the discovery set is less than or equal to 10%.

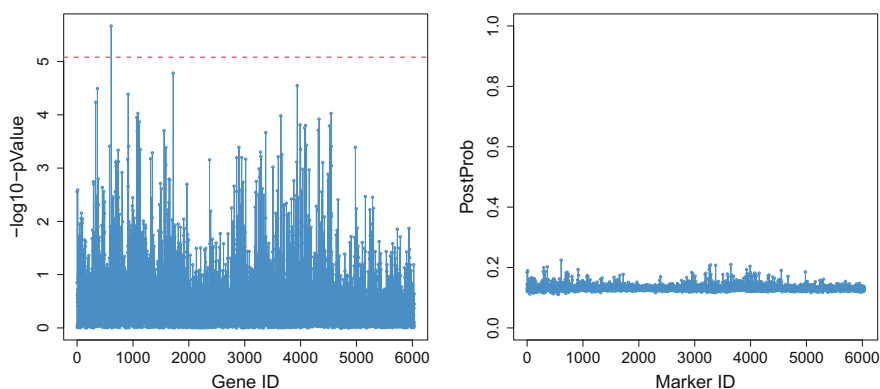


Fig. 1.13 Left: $-\log_{10} p$ -values for testing the null hypothesis that the gene has no effect on prostate cancer (measured by gene expression) using GWAS with a Bonferroni adjustment. The horizontal red line is the Bonferroni threshold corresponding to $-\log_{10}(0.05/6033)$. Right: posterior probability that the gene has an effect on prostate cancer obtained using a Bayesian, Markov chain Monte Carlo implementation of a spike and slab model, for each of the 6033 gene expressions. The minimum posterior probability is 0.11 and the maximum is 0.22

The result of the Bayesian analysis based on the spike and slab model is shown in the right panel of Fig. 1.13. Only a handful of genes reach posterior probabilities in the vicinity of 20%; the largest probability (0.22) is associated with gene 610 and the genes that overlap with those in the discovery set constructed using the first approach have posterior probabilities around 0.18 or larger. What is not revealed by Fig. 1.13 nor by the manner that the discovery set is arrived at, is the uncertainty associated with these inferences. The spike and slab model outputs the Monte Carlo estimate of the 95% posterior interval of the posterior probability that gene 610 has an effect on prostate cancer. The figure is almost (0; 1) spanning the whole support of the posterior distribution. This large posterior variance, that propagates on to the remaining inferences, has its roots in the uncertain estimates of the marker substitution effects. For example, the Monte Carlo estimates of the mean and of the 95% posterior interval (in brackets) of the substitution effect of marker 610 is 0.041(−0.021; 0.404). The only possible message to draw from these preliminary analyses is that the methods implemented here point towards a number of specific genes that may be involved in prostate cancer. However firmer conclusions merit further analyses and particularly, collection of more data.

I would like to end the section with less ambiguous results based simulated data. You have access to a binary trait (such presence or absence of a particular condition) measured in a sample of 2500 individuals. In the sample, 30% of the individuals show presence of the condition. There are 15,000 covariates available for each individual representing genotypic markers (a modern data set scales the size of these data up by a factor of one hundred or one thousand). GWAS and a Bayesian spike and slab model implemented using Markov chain Monte Carlo are the tools available for analysis. The objective is to discover covariates that have an additive effect on the response variable (presence or absence of the condition).

The true (simulated) model assumes that the probability that an individual shows the condition is a function of the additive effects of a genotypic value and of an environmental deviation. The genotypic value is the result of the sum of the effects of 25 genes. Additivity of effects at the level of the genotypic value holds for the fitted models and for the data-generating model. The difference between the true model and the fitted model is that the former includes 25 covariates only (the 25 genes) whereas the latter includes all the 15,000 covariates. The 25 genes are part of the panel of the 15,000 covariates included in the fitted model; the challenge for the analyst is to discover these genes, without having a priori knowledge of their number or position.

The results of the analyses are displayed in Fig. 1.14. The left panel represents the output of the GWAS analysis based on 2500 records. Out of the 25 genes, 5 are detected with zero false positive results in the discovery set. The spike and slab model detects 15 out of the 25 genes, with zero false positive results, as indicated in the right panel. The discovery set is based on declaring all the genes whose posterior probability of belonging to the slab is equal to or larger than 80%. The choice of threshold is here based on a visual inspection of the figure. Another option is to choose a discovery set based on estimates of the posterior distribution of false discovery rates for a range of thresholds. Chap. 8 elaborates on this subject. Details

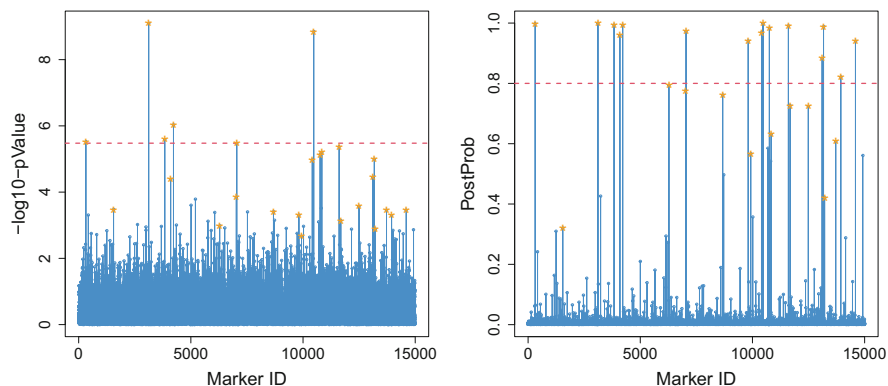


Fig. 1.14 Detection of promising marker genotypes; 2500 observations. Left: GWAS with a Bonferroni correction; 5 out of 25 markers detected; 0 false discoveries. The horizontal red line is the Bonferroni threshold corresponding to $-\log_{10}(0.05/15,000)$. Right: spike and slab model; 15 out of 25 markers detected; 0 false discoveries. The horizontal red line is the arbitrary threshold corresponding to a posterior probability equal to 0.8. Orange stars represent the 25 true (causal) genes

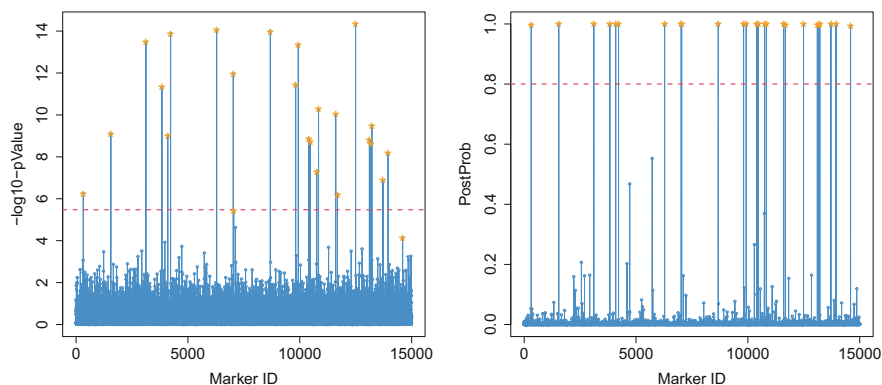


Fig. 1.15 Detection of promising marker genotypes; 5000 observations. Left: GWAS with a Bonferroni correction; 23 out of 25 markers detected; 0 false discoveries. Right: spike and slab model; 25 out of 25 markers detected; 0 false discoveries. Orange stars represent the 25 true (causal) genes

of the spike and slab model are in Chap. 7 (for conditionally Gaussian traits) and in Chap. 9 (for binary traits).

When the number of records increases from 2500 to 5000 the results are more encouraging (Fig. 1.15). In this larger data set, the GWAS detects 23 out of 25 genes with 0 false positive results. The spike and slab model detects all the 25 genes with 0 false positive results.

These examples illustrate how the information content in the data (here, in the form of the amount of data) increases the probability of detection of genes that affect the trait, when these are part of the data generating mechanism. The fitted model (or *operational model*) plays also an important role in detection, particularly when it is well aligned with important features of the true model. This true model is unknown but there are tools that can be employed to study the ability of alternative fitted models to account for specific features of the observed data. One such tool, posterior probability model checking, is described in Chap. 10.

1.7 Appendix: A Short Overview of Quantitative Genomics

I provide a brief and compact description of the quantitative genetics/genomics model and introduce terms used repeatedly in the book, such as *allele*, *locus*, *diploid*, *haploid*, *genotype*, *Hardy-Weinberg law*, *single nucleotide polymorphisms (SNP's)*, *genomewide association study (GWAS)*, *allele content*, *quantitative trait loci (QTL)*, *linkage*, *linkage disequilibrium*, *phenotype*, *genotype*, *genetic value*, *genetic variance*, *additive genetic value (breeding value)*, *additive genetic effect (additive effect of a gene substitution)*, *additive genetic variance*, *heritability*, *expected additive genetic relationship*, *additive genetic relationship matrix*, *genomic relationship matrix*, *genomic model*, *genomic value*, *genomic variance*.

The Classical Quantitative Genetics Model

The starting point of the mathematical genetics model is the metaphor that describes chromosomes as strings of beads, with each bead representing a gene. Genes are the unit of inheritance. In mammals and many other groups, each cell carries two copies of each chromosome; they are said to be diploid. Most fungi, algae and human gametes have only one chromosome set and are haploid.

The complete set of chromosomes of an organism includes sex chromosomes and autosomes. For example, in humans there are 23 pairs of chromosomes and of these, 22 pairs are non-sex chromosomes known as autosomes. The majority of genes are located on the autosomes and in this book I consider autosomal loci only.

In diploid organisms, at a specific location on the chromosome called the locus, each of the two copies of the chromosome carries a gene. The pair of genes constitute the genotype at the particular locus. Genes exist in different forms known as alleles. Here I consider biallelic loci, so for a given locus in diploid individuals, if the two alleles are A and a , the three genotypes could be denoted AA , Aa , aa , say (no distinction made between Aa or aA). For example, an individual with genotype Aa , received one allele (A , say) from the mother and the other from the father.

The standard quantitative genetic model assumes that the expression of a continuously distributed trait value y (the phenotype, here centred with zero mean)

in diploid individuals is determined by the additive contributions of a genetic value G and a continuously distributed environmental value e with variance σ^2 ,

$$y = G + e,$$

where $e \sim (0, \sigma^2)$ is often assumed independent of G . The genetic value is defined as the conditional mean of the phenotype, given genotype, $E(y|G)$ and is the result of the joint action of a typically unknown number q of quantitative trait loci (QTL).

The Single Locus Model

Consider first a trait affected by a single biallelic locus with the three genotypes labelled AA , Aa , aa . Let p denote the frequency of allele A in the population (assumed to be the same in both sexes). In a large population, assuming random mating among parents and in the absence of random genetic drift, selection and mutation, in the offspring generation, the frequency of genotype AA is p^2 , of genotype Aa is $2p(1 - p)$ and of genotype aa is $(1 - p)^2$. In this overview gene frequencies p are treated as known constants that remain unchanged over repeated cycles of random mating.

Define the random variable z^* as

$$z^* = \begin{cases} 2, & \text{with probability } p^2, \\ 1, & \text{with probability } 2p(1 - p), \\ 0, & \text{with probability } (1 - p)^2, \end{cases}$$

The random variable z^* is known as the allele content (here, allele A is arbitrarily taken as reference and $z^* = 2$ if the genotype has 2 copies of allele A). For this locus, $E(z^*) = 2p$, $\text{Var}(z^*) = 2p(1 - p)$ and for individuals k and j , $\text{Cov}(z_j^*, z_k^*) = a_{jk}2p(1 - p)$ where a_{jk} is the expected additive genetic relationship (given a pedigree) between k and j (for example, $a_{jk} = 0.5$ if j and k are non-inbred full sibs or parent and an offspring), also interpreted as the expected proportion of alleles shared identical by descent between j and k (genes that are identical by descent (IBD) are copies of a specific gene carried by some ancestral individual). The note note0101.pdf at <https://github.com/SorensenD/SLGDS> has a derivation of these results.

In a large (idealised) random mating population, in the absence of selection, mutation or migration, the relationship between gene frequency p and genotype frequency p^2 remains constant from generation to generation. The property is derived from a theorem known as the Hardy-Weinberg law that provides one explanation for the maintenance of genetic variation in such idealised random mating population. In a population in Hardy-Weinberg equilibrium, genotype frequencies at a particular locus in the offspring generation depend on gene frequencies in the parent generation.

From now on the codes for the three genotypes are centred as $z = z^* - E(z^*)$ so that $E(z) = 0$, $\text{Var}(z) = 2p(1-p)$ and phenotypic values are also centred, so that $E(y) = 0$.

The genetic value $G(z)$ at the locus can take three modalities corresponding to the three genotypes at the biallelic locus and can be decomposed into two terms

$$G(z) = \alpha z + \delta, \quad (1.27)$$

where αz is the best linear predictor of genetic value and δ is orthogonal to z . The best linear predictor αz is also known as the additive genetic value or breeding value: the best linear approximation describing the relationship between genetic value and allele content z (best linear prediction is discussed on page 283; see also the example on page 285 for more details on the concepts of additive genetic values and effects, where it is shown that α , the additive genetic effect of the locus or average substitution effect at the locus is also the regression of y on z).

The genetic variance contributed by the locus in the population (based on the law of total variance)

$$\text{Var}(G(z)) = \text{Var}_z(E[G(z)|z]) + E_z(\text{Var}[G(z)|z]) \quad (1.28)$$

is orthogonally decomposed into an additive genetic component of variance σ_a^2 , the first term in the right hand side and a residual or dominant component of genetic variance, $\text{Var}(\delta)$, the second term. The additive genetic variance (variance of the additive genetic values) in this single locus model, assuming Hardy-Weinberg equilibrium is

$$\sigma_a^2 = \text{Var}_z(E[G(z)|z]) = \text{Var}(\alpha z|\alpha) = 2\alpha^2 p(1-p).$$

If the linear fit is perfect, the genetic variance is equal to the additive genetic variance. Importantly, additive genetic variation at the locus arises due to variation in allele content z among individuals at the locus. The substitution effect α is treated as a fixed albeit unknown parameter (this is stressed by conditioning on α).

The (narrow sense) heritability of the trait is defined as the ratio of the additive genetic variance to the phenotypic variance: $h^2 = \sigma_a^2 / \sigma_y^2$, where $\sigma_y^2 = \text{Var}(y)$, the marginal variance of the phenotype.

Models with Many Loci

The extension to q biallelic loci involves a random vector $z = (z_1, \dots, z_q)'$ of allele contents of the q genotypes. Under random mating, $\text{Var}(z_k) = 2p_k(1-p_k)$, $k = 1, \dots, q$ and $\text{Cov}(z_k, z_l) = 2D_{kl}$, where the linkage disequilibrium (LD) parameter D_{kl} between loci k and l is defined as follows. Choose the paternal (or maternal) gamete and let the random variable U take the value 1 if allele A_k is

present in the paternal gamete at locus k and zero otherwise; let the random variable V take the value 1 if allele A_l is present in the paternal gamete at locus l and zero otherwise. Then D_{kl} is defined as the covariance between U and V

$$D_{kl} = \text{Cov}(U, V),$$

and $\text{Cov}(z_k, z_l) = 2D_{kl}$ since in the diploid model the genotype results from the random union of two gametes. Covariances involving alleles of different loci between gametes are zero. Linkage disequilibrium is created by evolutionary forces such as selection, mutation and drift and is broken down by random mating, as a function of time (measured in generations) and of the distance that separates the intervening loci. Generally, loci that are physically close together show stronger LD.

With random mating in a large population the variance structure of vector z containing the q allele contents, a population parameter, is

$$\Sigma_z = \begin{bmatrix} 2p_1(1-p_1) & 2D_{12} & \cdots & 2D_{1q} \\ 2D_{21} & 2p_2(1-p_2) & \cdots & 2D_{2q} \\ \vdots & \vdots & \ddots & \vdots \\ 2D_{q1} & 2D_{q2} & \cdots & 2p_q(1-p_q) \end{bmatrix}. \quad (1.29)$$

The combined action of the q loci defines the genetic value $G(z)$. A useful starting point is to assume that the effects of the loci combine additively. A genetic value has the same structure as (1.27), $G(z) = \alpha'z + \delta$, where now α is the $q \times 1$ column vector of average substitution effects (or additive genetic effects) at the q loci. The additive genetic variance contributed by the q loci is

$$\sigma_a^2 = \text{Var}(\alpha'z|\alpha) = \alpha'\Sigma_z\alpha. \quad (1.30)$$

Expanding (1.30) reveals that the additive genetic variance takes the form

$$\sigma_a^2 = 2 \sum_{i=1}^q \alpha_i^2 p_i (1-p_i) + 4 \sum_{i=1}^{q-1} \sum_{j=i+1}^q \alpha_i \alpha_j D_{ij}. \quad (1.31)$$

The first term in the right hand side is the *equilibrium additive genetic variance* and the second term is the contribution due to disequilibrium (Bulmer 1971). If the loci are in linkage equilibrium ($D_{ij} = 0$ for all i, j), genotypes in the population are uncorrelated, (1.29) is a diagonal matrix and the additive genetic variance reduces to the first term. Among the loci of neutral traits, some of the disequilibrium terms are positive and others negative. In contrast, among loci of selected traits, depending on the type of selection, in large populations the disequilibrium terms are either positive (disruptive selection) or negative (optimising and directional selection) (Bulmer 1971).

The phenotypic value y of an individual in the multilocus model can be written

$$\begin{aligned} y &= G(z) + e \\ &= \alpha'z + (\delta + e) \\ &= \alpha'z + \epsilon, \end{aligned} \quad (1.32)$$

where the residual term ϵ includes environmental effects e and non-additive genetic effects δ , and is orthogonal to z . The genetic value is the conditional expected value of the phenotype given the genotype

$$G(z) = E[y|z].$$

Covariance Between Relatives

Consider two related individuals i and j whose causal genotypes at q loci are described by the random vectors of allele contents z_i and z_j , where $z_i = (z_{i1}, \dots, z_{iq})'$. While (1.29) describes associations within individuals, here we seek associations between individuals. The covariance between i and j at the same locus k is $\text{Cov}(z_{ik}, z_{jk}) = 2a_{ij}p_k(1 - p_k)$, where a_{ij} is the coefficient of expected additive genetic relationship between i and j . The covariance involving two different loci k in i and l in j is $\text{Cov}(z_{ik}, z_{jl}) = 2a_{ij}D_{kl}$. The contribution to the covariance between the additive genetic values of i and j from loci k and l is then

$$\begin{aligned} \text{Cov}(\alpha'z_i, z_j'\alpha|\alpha) &= \alpha' \text{Cov}(z_i, z_j)\alpha \\ &= 2a_{ij} \sum_{m=k,l} \alpha_m^2 p_m (1 - p_m) + 4a_{ij}\alpha_k\alpha_l D_{kl}, \end{aligned} \quad (1.33)$$

that has the same form as (1.31) except for the factor a_{ij} .

Expression (1.33) generalises to n individuals displaying a family structure given by a pedigree. Based on the pedigree, the $n \times n$ matrix A can be constructed. This matrix, known as the additive genetic relationship matrix, has elements a_{ij} that denote the expected additive genetic relationship between individuals i and j .

Let $Z\alpha$ represent the vector of n additive genetic values, where Z is the $n \times q$ matrix of genotypic codes (allele contents) and α is the $q \times 1$ vector of additive genetic effects for the q loci. Then the variance-covariance matrix of the n additive

genetic values, treating Z as random and α as a fixed parameter is

$$\begin{aligned}
 \text{Var}(Z\alpha|\alpha) &= \begin{bmatrix} a_{11}\alpha'\Sigma_z\alpha & a_{12}\alpha'\Sigma_z\alpha & \dots & a_{1n}\alpha'\Sigma_z\alpha \\ a_{21}\alpha'\Sigma_z\alpha & a_{22}\alpha'\Sigma_z\alpha & \dots & a_{2n}\alpha'\Sigma_z\alpha \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1}\alpha'\Sigma_z\alpha & \dots & \dots & a_{nn}\alpha'\Sigma_z\alpha \end{bmatrix} \\
 &= A \left[\sum_{k=1}^q \alpha_k^2 2p_k(1-p_k) + \sum_{k=1}^{q-1} \sum_{l=k+1}^q 4\alpha_k\alpha_l D_{kl} \right] \\
 &= A\sigma_a^2,
 \end{aligned} \tag{1.34}$$

where σ_a^2 is given by (1.31) and is equal to the term in square brackets.

Expressions (1.33) and (1.34) assume that the (large) population is maintained in a steady state of gene frequency and linkage disequilibrium from generation to generation resulting in constant σ_a^2 and that the computation of the expected additive genetic relationship between individuals i and j involving loci k and l is tracing i and j to the most recent common ancestor. The constancy of σ_a^2 is an approximation that can also be justified by considering cohorts spanning 1 or 2 generations as often found in human pedigrees.

The covariance between relatives in multilocus systems is part of a subject of difficult entry. An exact general treatment involving only pairs of loci constitutes a formidable challenge leading to unwieldy expressions, as shown by Weir and Cockerham (1977). The curious reader may wish to glance with awe at formula (6) for the genetic variance in their article that is almost two pages long! Results assuming lack of inbreeding, epistasis and assortative mating, but accounting for dominance, linkage and for the dynamics of the linkage disequilibrium parameter over generations lead to simpler expressions and are given by Weir et al. (1980). Further details and an informal derivation of the covariance between allele contents of two individuals at different loci are in the note note0101.pdf at <https://github.com/SorensenD/SLGDS>.

Expected Value of the Genomic Relationship Matrix

In the population with n related individuals and q causal loci, the centred matrix Z contains the coded genotypes of the causal loci, z_{ij} , $i = 1, \dots, n$; $j = 1, \dots, q$. These are realisations from a random Mendelian process. The expectation $E(ZZ')$ is proportional to the expected additive genetic relationship matrix, as sketched below.

The ij th element of ZZ' involving individuals i and j , is computed using the inner product $z'_i z_j$, where z'_i is the i th row of Z , containing the causal genotypes of individual i . The expectation of the inner product $z'_i z_j$ involves only the diagonal elements of the matrix Σ_z in (1.29). Then conditional on gene frequencies,

$$E(z'_i z_j) = a_{ij} \sum_{k=1}^q 2p_k (1 - p_k)$$

and

$$E(ZZ') = A \sum_{k=1}^q 2p_k (1 - p_k), \quad (1.35)$$

proportional to A , as in Habier et al. (2007). This expectation, here derived for causal (QTL) genotypes, holds for any set of randomly drawn autosomal genotypes, including genetic marker loci.

The Genomic Model

The description so far is in essence the standard model of quantitative genetics. It is viewed as the model that generates the data, the *true model*. “True” must be interpreted in a statistical sense and not as a mechanistic description of the complex biological process that results in an observed phenotype. It is perhaps best understood relative to an *operational model* that by its very nature cannot be viewed as having generated the data. A *genomic model* is such an operational model. A genomic model incorporates genetic markers in a regression equation, often assuming a linear association between the phenotype and the genetic markers. This association arises as a result of the correlation (linkage disequilibrium) between the observed genetic markers and the unobserved causal loci (QTL). The genetic markers are DNA sequences with a known location on a chromosome. One type are single base-pairs (single nucleotide polymorphisms, SNPs), that give rise to different alleles containing alternative bases at a given position. SNPs are most often phenotypically neutral, but sometimes they can affect the phenotype, for example, by causing a change in the amino acid sequence of a gene product, or by affecting gene regulation in noncoding sequences. In this case the SNP becomes a causal locus and has a direct effect on phenotype.

The use of genetic markers as covariates instead of causal loci induces a new set of parameters that must be defined: the *genomic value*, the *genomic variance* and the *genomic heritability*.

At the population level, using p markers with centred marker genotype codes x_i in vector $x = (x_1, \dots, x_p)'$, the genomic value (the part of the additive genetic value explained by the linear regression on markers) is defined as

$$\begin{aligned} E[\alpha'z|x] &= \alpha' E(z|x) \\ &= \alpha' \Sigma_{zx} \Sigma_x^{-1} x = \alpha' \hat{z} \\ &= \beta' x. \end{aligned} \tag{1.36}$$

In this expression the marker effects $\beta = \Sigma_x^{-1} \Sigma_{zx} \alpha$ are population parameters, defined as the vector of coefficients in a multiple linear regression of additive genetic values on marker genotypes. The term $\hat{z} = \Sigma_{zx} \Sigma_x^{-1} x$ is the best linear predictor of allele contents at the QTL, z , given allele content at markers x . The parameter Σ_{zx} is the covariance matrix involving marker and QTL genotypes and the parameter Σ_x is the variance covariance matrix of marker genotypes that has a structure similar to (1.29).

The genomic variance or variance of genomic values $\beta'x$, is the part of the additive genetic variance explained by the linear regression on marker genotypes

$$\text{Var}_x(E[\alpha'z|x]) = \beta' \Sigma_x \beta = \alpha' \Sigma_{zx} \Sigma_x^{-1} \Sigma_{xz} \alpha. \tag{1.37}$$

The *genomic heritability* is the ratio of genomic variance to phenotypic variance, also a population parameter. These are the parameters to be inferred when the statistical models are fitted to data.

There are a couple of details to note in relation to (1.37). The first and obvious one is that the proportion of the additive genetic variance captured by the marker genotypes depends on the strength of the association between the markers and the QTL genotypes (defined by the degree of linkage disequilibrium between marker loci and causal loci). This is dictated by Σ_{zx} . When LD between marker genotypes and causal genotypes is perfect, $\Sigma_{zx} = \Sigma_z$, $\Sigma_x = \Sigma_z$ and the genomic variance is equal to the additive genetic variance. In this situation, the genomic heritability achieves its upper bound, equal to the heritability of the trait. The second is that when the QTL genotypes are part of the marker panel (for example, with full genome sequence) the (population parameter) genomic variance (1.37) is equal to the (population parameter) additive genetic variance (1.30). In other words, $\text{Var}(\beta'x) = \alpha' \Sigma_z \alpha$. The proof of this result involves a little manipulation with the inverse of partitioned matrices and can be found in de los Campos et al. (2015). It does not follow that implementing a statistical model that incorporates full genome sequence will automatically uncover the additive genetic variance of the trait. This depends on the input data and on the properties of the estimator of the genomic variance of the statistical model.

In several examples used in the book, I generate a matrix of genotypic markers drawing independent samples from a binomial distribution and a proportion is assigned as QTL. Since the QTL are part of the marker panel, the variance of the

independent marker genotypes fully accounts for the equilibrium additive genetic variance.

Further details on these subjects can be found in Gianola et al. (2009) and de los Campos et al. (2015) who lay the background for most of what is written here.

Note

Typically marker genotypes are fitted to phenotypic records using a multiple linear regression model of the form $x\beta$, where x is the observed matrix of marker genotypes and β is the unobserved vector of multiple regression coefficients. Given the marker genotypes, the variance explained by this regression model, $\text{Var}_x(E(y|x))$ is equal to the genomic variance provided that, as a minimum, a couple of conditions are fulfilled as shown in this note. Referring to (1.32), the conditional expectation is

$$\begin{aligned} E(y|x) &= E(\alpha'z + \epsilon|x) \\ &= E_{z|x} [E(\alpha'z|x, z)] + E(\epsilon|x) \\ &= \alpha' E(z|x) + E(\epsilon|x) \\ &= \beta'x + E(\epsilon|x). \end{aligned}$$

If ϵ , that includes environmental effects and non-additive genetic effects, is (i) linearly related to x and (ii) the covariance between ϵ and x is zero (a condition unlikely to be fulfilled), then $E(\epsilon|x) = E(\epsilon) = 0$. In this case, the variance explained by the model is

$$\text{Var}_x [E(y|x)] = \beta' \Sigma_x \beta$$

equal to the genomic variance.

Fitting Models Incorporating Marker Genotypes

When marker genotypes are fitted one at a time, the analysis is known as *genome-wide association study* (GWAS). GWAS exploit the association, at the population level, between observed genetic markers scattered across the genome and unobserved causal loci. The significance of markers is assessed by regressing the phenotype on each marker in turn. This is based on a simple t -test using often a Bonferroni adjustment to correct for multiple testing (see Chap. 8). GWAS have identified thousands of genetic variants associated with human complex traits and diseases. Despite this success the significant SNPs explain a small proportion of the genetic variation estimated from pedigrees. This has been known as the “missing heritability” problem and indicates that most of the genetic variation uncovered by GWAS remains unexplained. Part of the explanation lies in the multiple testing procedure, in the method’s attempt to avoid false discoveries, on the size of the

effects and frequency of the causal genes affecting the complex trait and on the incomplete degree of association between marker and causal genotypes. Many loci contributing to genetic variation do not reach the stringent significance level imposed and are missed. Despite the limitations the success of GWAS as a gene discovery tool cannot be denied, particularly when used together with a number of refinements designed to partially overcome some of its shortcomings (Yang et al. 2012). In 2008 a GWAS study involving several thousand individuals detected approximately 40 genetic variants associated with height that account for about 5% of the heritability (Visscher 2008). In 2022, with larger size of testing populations (resulting in higher power) and larger number of genetic markers (leading to larger amounts of linkage disequilibrium), the number of discovered SNPs increased to approximately 12,000, explaining 40% of the heritability of height (Yengo et al. 2022).

An alternative to fitting one marker at a time is to use a genomic model that incorporates all the markers simultaneously in a multiple linear regression disregarding statistical significance and in contrast with traditional GWAS, accounting for the structure among marker genotypes. A specification could be

$$y|X \sim N(1\mu + Xb, I\sigma^2), \quad (1.38)$$

where 1 is an $n \times 1$ vector of 1's, μ is a scalar, y is $n \times 1$ vector of observed records, X is a centred matrix of observed marker genotypes of order $n \times p$ and b is an unobserved $p \times 1$ vector of marker effects at the level of the fitted model. When typically $p > n$, some additional structure must be incorporated to fit the p marker effects b to the n data points. One common approach is to treat b as the random variable

$$b|\sigma_b^2 \sim N(0, I\sigma_b^2), \quad (1.39)$$

where σ_b^2 can be interpreted as quantifying prior uncertainty about the genetic marker effects b . The specification of marker effects as random variables is in contrast with their definition in (1.36) and has a direct consequence on the relationship between the genomic variance defined in (1.37) and the genomic variance of the fitted model. To see this, define the vector of genomic values (at the level of this operational model) as $g = Xb$. The genomic variance of the fitted model is defined as the unconditional variance

$$\begin{aligned} \sigma_g^2 &= \text{Var}(b'X) = E_b[\text{Var}(b'X|b)] + \text{Var}_b[E(b'X|b)] \\ &= E_b(b'\Sigma_X b) + 0 \\ &= \sigma_b^2 \text{tr}(\Sigma_X) = \sigma_b^2 \sum_{j=1}^p 2p_j(1 - p_j). \end{aligned} \quad (1.40)$$

In this expression, $\text{Var}_b [E(b'X|b)] = 0$ because with centred X , $E(b'X|b) = b'E(X) = 0$. A glance at the genomic variance of the fitted model (1.40) and at the parameter defined in (1.37), reveals a lack of correspondence between the two. This is elaborated a little further in the first *Comment* on page 47 and a deeper discussion is in de los Campos et al. (2015).

Let $k = \sum_{j=1}^p 2p_j (1 - p_j)$. An alternative parametrisation in terms of g that is useful when $p > n$ is

$$y|X \sim N(1\mu + g, I\sigma^2), \quad (1.41a)$$

$$g|X \sim N(0, G_g\sigma_g^2), \quad (1.41b)$$

where the genomic relationship matrix $G_g = \frac{1}{k}XX'$ has rank at most $n - 1$ if X is column-centred and if $p > n$. This is equivalent to constructing the genomic relationship matrix from $G_g = XX'$, when the elements of X are centred and scaled by $\sqrt{k}$. In this parametrisation, $\sigma_g^2 = \sigma_b^2$, independent of the number of markers. When X is column-centred and $p > n$, the vector of genomic values g has a singular normal distribution.

The model of marker effects defined in (1.39) shrinks all marker effects homogeneously towards zero and is therefore not ideal for discovery. It is best deployed for inferences of genomic variance σ_g^2 and for prediction of genomic values g . An early application of a version of this genomic model in humans to infer genomic heritability (Yang et al. 2010) incorporated 294,831 SNPs genotyped on 3925 unrelated individuals (meaning distantly related) and reported that 45% of the variance of height could be accounted for by considering all the SNPs simultaneously. The increase in variance explained relative to the GWAS analysis is attributed to the many SNPs with small effects that do not pass the significant threshold in GWAS. This was still considerably less than the 70–80% explained by traditional methods using pedigrees. The authors conjectured that most of the unaccounted variance could be due to causal variants at low frequency, leading to very low correlations between markers and causal variants. Genetic markers are typically at intermediate frequencies and the disparity of gene frequencies between markers and causal variants results in low correlations. Support for this conjecture was obtained years later using whole genome sequence data and larger population sizes, leading to an estimate of the proportion of variance of height attributed to SNPs (the genomic heritability) of 68% (Wainschtein et al. 2022), quite close to the estimate based on pedigrees.

The picture that is emerging is interpreted in a paradigm in which complex traits are driven by a very large number of loci spread along the genome. However, complex traits are not only affected by protein-coding genes, but especially by noncoding variants, perhaps related to gene regulation that play a direct role in the expression of the trait. Despite their relevance, the noncoding variants show weak signals in GWAS studies and are difficult to detect. Most of the genetic variation

of complex trait seems to be driven by a very large number of peripheral genes of small effect (Boyle et al. 2017).

The new insights of the genetics of complex traits stimulated further developments and refinements of the genomic model for estimation of genomic parameters and for prediction. For example, prior knowledge can be incorporated to partition genomic variance across groups defined by allele frequency of genetic markers, linkage disequilibrium, genotype certainty and accounting for SNP function and metabolic pathways (Speed et al. 2017, 2021; Patxot et al. 2021).

The genomic model (1.41) is revisited several times in the book. A likelihood model using Newton-Raphson and using the EM algorithm is described in Chap. 3 and in the Problems section on pages 617 and 676. A Bayesian model is presented in Chap. 5 and in the Problems section on pages 624 and 713.

The genomic model (1.41) can be extended for use in simultaneous inference, prediction and gene discovery. An early review is provided by de los Campos et al. (2013b). One approach is based on modifying the prior distribution of SNP effects (1.39) and adopting instead a mixture prior of two densities: one with small variance (the spike) and one with large variance (the slab). These are known as spike and slab models. For example if a two component mixture is chosen, these could be two Gaussian densities (George and McCulloch 1993) or one Gaussian and a point mass at zero (Mitchell and Beauchamp 1988; Meuwissen et al. 2001; Habier et al. 2011). If two Gaussian distributions are used, they are often parametrised such that one has a mean of zero and a very small user-tuned variance, generating very small SNP effects and the other a mean of zero and a larger variance, allowing for larger effects. For gene discovery, the idea is to obtain an estimate of the posterior probability that a genetic marker is drawn from either mixture component, for all genetic markers. An example of a spike and slab model with a point mass at zero applied to gene discovery can be found in Chap. 8. Chapter 7 provides a detailed Bayesian-McMC implementation of this model for the analysis of continuous traits and Chap. 9 extends the algorithm for the analysis of binary data.

Comments on the Genomic Variance and the Genomic Relationship Matrix

- The genomic variance σ_g^2 (1.40) of the fitted model has a tenuous correspondence with the genomic variance of the genomic model defined in (1.37). An approach that results in an estimate of genomic variance that is better aligned with parameter (1.37) can be derived drawing from Sorensen et al. (2001) as follows. Consider a group or cohort consisting of n individuals; the genomic value of individual i is g_i . This is a random variable that can take n possible values

each with probability n^{-1} . By definition, the variance of g_i conditional on the statistical model adopted and on the particular group of the n individuals is

$$\sigma_{gen}^2 = E(g_i^2) - [E(g_i)]^2 = \frac{1}{n} \sum_{i=1}^n g_i^2 - \bar{g}^2, \quad (1.42)$$

where $\bar{g} = n^{-1} \sum_i g_i$. Let $g_i = \beta' x_i$, where x_i is the column vector of the p centred marker genotypes of individual i and $\beta = \Sigma_x^{-1} \Sigma_{xz} \alpha$ is the column vector of p marker effects (the parameters of a multiple regression of additive genetic values $\alpha' z$ on marker genotypes x). Then if β is known and the elements of x are centred, $\bar{g} = 0$ and the genomic variance in the cohort is

$$\begin{aligned} \sigma_{gen}^2 &= n^{-1} \sum_{i=1}^n \beta' x_i x_i' \beta \\ &= \beta' \left[n^{-1} X' X \right] \beta. \end{aligned} \quad (1.43)$$

This expression has the same form as (1.37) with Σ_x replaced by its method of moments estimator $n^{-1} X' X$. The marker effects β are not observed and adopting a Bayesian approach can be inferred from their marginal posterior distribution $[\beta|y]$. The posterior mean of the distribution of the resulting genomic variance is then

$$\begin{aligned} E(\sigma_{gen}^2 | y) &= E(\beta' Q \beta | y) \\ &= \text{tr}(Q \text{Var}(\beta|y)) + E(\beta|y)' Q E(\beta|y) \\ &= \text{tr}[Q (E(\beta\beta'|y) - E(\beta|y) E(\beta|y)')] + \text{tr}[E(\beta|y)' Q E(\beta|y)] \\ &= \text{tr}[Q E(\beta\beta'|y)], \end{aligned} \quad (1.44)$$

where $Q = n^{-1} X' X$ (the equality in the final line stems from applying the cyclic property of the trace operator in the second trace of the third line). All this is typically implemented using MCMC. A Monte Carlo estimate of the posterior distribution of the genomic variance in the cohort σ_{gen}^2 is easily obtained by calculating the variance among the draws from $[g|y]$, without the need to infer the marker effects. A draw from this posterior distribution at round t of an MCMC implementation is

$$\sigma_{gen}^{2[t]} = \frac{1}{n} \sum_{i=1}^n (g_i^{2[t]} - \bar{g}^{2[t]})^2,$$

where $g_i^{[t]}$ is sampled from $[g_i|y]$ belonging to the i th individual in the cohort. The posterior mean of the genomic variance under the parametrisation $g = X\beta$

takes the form

$$E\left(\sigma_{gen}^2|y\right) = \frac{1}{n} \text{tr}\left[E\left(gg'|y\right)\right]. \quad (1.45)$$

The genomic variance σ_{gen}^2 describes variation among individuals in the cohort. Therefore, by appropriate construction of the cohorts, it can be used, for example, to monitor changes over time.

The subject is revisited on page 255 where an MCMC implementation is shown to involve computations using scalar quantities.

- The genomic relationship matrix constructed using genetic markers describes realised patterns of inheritance in a particular sample of individuals in the region of the genome defined by the marker loci. There are a number of ways of constructing the genomic relationship matrix and most use moment-based estimators (VanRaden 2008; de los Campos et al. 2013b). Here it was defined in connection with (1.41b)

$$G_g = \frac{1}{k} X X', \quad k = \sum_{i=1}^p 2p_i (1 - p_i), \quad (1.46)$$

where the elements of X (observed marker genotypes) are centred. Provided the model includes an intercept as in (1.41a), centring does not affect predictions or inferences of variances. However, standardising the elements of X may affect predictions, depending on the degree of similarity between the distributions of observed marker genotypes and of unobserved causal loci genotypes. Inferences, such as estimation of genomic heritability, are also sensitive to the way the genomic relationship matrix is constructed (Speed et al. 2012). In a prediction context, the choice of a method can be evaluated by cross-validation. A couple of other ways of constructing the genomic relationship matrix are mentioned on page 535.

- The genomic model builds on the incorporation of a large number of genetic marker loci spread along the genome of individuals from which G_g is constructed. These genetic marker loci are not necessarily causal, but are correlated with the causal loci. The latter are typically unobserved, the genetic markers are observed and can provide information about the genetics of a trait. The ability of the genomic model to separate true signal (the genetic value at the causal loci) from noise, depends on how well the genomic relationship matrix G_g describes relationships realised at causal loci. Realised relationships in different regions of the genome are the result of a random process, with expected value dictated by the pedigree and variation due to Mendelian sampling. This creates variability in the patterns of genetic similarity across the genome; this pattern can be very different for marker genotypes and for causal genotypes. The variability is accentuated for data composed of nominally unrelated individuals (Hill and Weir 2011), often seen in human data and plays a smaller role when pedigrees display

strong family relationships as in livestock. With distantly related individuals, fitting a large number of markers, many of which are potentially uncorrelated with genotypes at causal loci, can lead to an incorrect specification of the covariance structure and this can affect inferences of variance parameters of the model (de los Campos et al. 2015). This problem can be partly addressed fitting mixture models (spike and slab models) where inferences of variance parameters and model selection are performed simultaneously. These models are introduced in Chap. 7.

- Genomic models that incorporate dense marker genotypes, particularly in humans, typically include phenotypes on nominally unrelated individuals (to be understood as distantly related). These models are often used to estimate genomic heritability. Inference about genomic heritability is driven by quantifying the proportion of the phenotypic resemblance between distant relatives explained by the short genome segments they share (which is tagged by the observed marker loci via the genomic relationship matrix). In such a situation, there is a need to avoid bias due to non-genetic effects, such as common environment among close relatives, that inflates the phenotypic resemblance between relatives. Also, by excluding close relatives such as twins, full-sibs and parent and offspring that would explain most of the phenotypic resemblance, the estimate obtained reflects the proportion of the short genome sequences shared among the distantly related individuals. These short shared sequences transmitted from remote common ancestors generate linkage disequilibrium; this LD is exploited via dense marker genotyping by capturing contributions from unobserved causal loci. In so doing, a proportion of the additive genetic variation is unmasked: the genomic variation.
- A decomposition that has attractive computational properties introduced in Chap. 3 is the eigenvalue or spectral decomposition of XX'

$$\begin{aligned} XX' &= U\Delta U' \\ &= \sum_{i=1}^n \lambda_i U_i U_i', \end{aligned}$$

where X , here centred and scaled, has dimension $n \times m$, (m being the number of marker genotypes), $U = [U_1, U_2, \dots, U_n]$, of dimension $n \times n$ (n being the number of individuals) is the matrix of eigenvectors of XX' , U_j is the j th column (dimension $n \times 1$) and Δ is a diagonal matrix with elements equal to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ associated to the n eigenvectors. Since XX' must be non-negative definite the eigenvalues are $\lambda_i \geq 0$, $i = 1, 2, \dots, n$. The eigenvectors satisfy $U'U = UU' = I$. Then an alternative representation of the genomic relationship matrix (when X is centred and scaled) is

$$\begin{aligned} G_g &= \frac{1}{m} XX' \\ &= \frac{1}{m} U\Delta U'. \end{aligned}$$

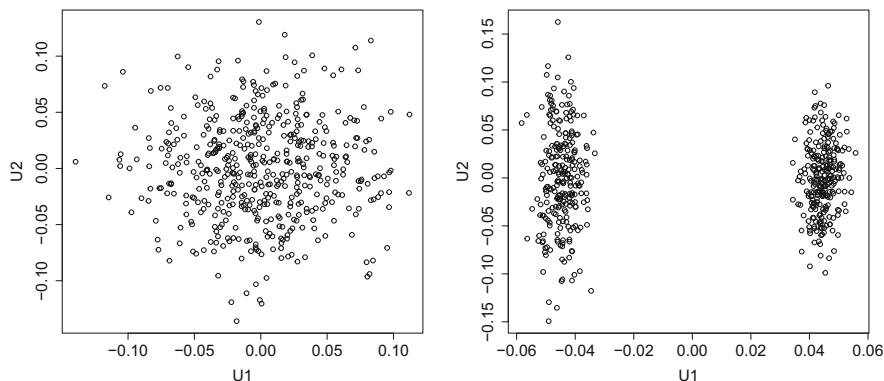


Fig. 1.16 Left: plot of the first two eigenvectors for homogeneous matrix composed of draws from $Bi(x; n = 2, q = 0.5)$; Right: plot of the first two eigenvectors for heterogeneous matrix composed of draws from $Bi(x; n = 2, q = 0.5)$ and draws from $Bi(x; n = 2, q = 0.3)$

This decomposition can be used to investigate the presence of unobserved substructure in X , that can cause spurious associations between marker genotypes and phenotype and also artifact genomic variation. As an illustration, the R-code below constructs two genomic relationship matrices drawing independent samples from binomial distributions. In the first genomic relationship matrix, each x representing a marker genotype is a draw from $Bi(x; n = 2, q = 0.5)$, where x can take values 0, 1, 2 and q is the probability of drawing allele A . The draws are elements of X that in the example is of order 500×1300 . The second relationship matrix is generated by appending two submatrices one on top of the other, where the first (order 250×1300) has elements drawn from $Bi(x; n = 2, q = 0.5)$ and the second (order 250×1300) has elements drawn from $Bi(x; n = 2, q = 0.3)$. This matrix displays fairly strong substructure caused by the difference of allele frequencies. The eigenvalue decomposition is performed on both matrices and Fig. 1.16 shows the plot of the first two eigenvectors. The left subfigure corresponding to the homogeneous matrix does not reveal any form of structure, but the right subfigure does. Estimation of marker effects without accounting for population structure can be a problem when they are used to predict records of a sample that has a different structure.

```
# CODE0105
# EIGEN DECOMPOSITION
rm(list=ls()) # CLEAR WORKSPACE
set.seed(30371171)
# SIMULATE Z FROM AN UNSTRUCTURED POPULATION
Z <- matrix(nrow= 500, ncol= 1300, rbinom(500*1300, size=2, p=.5))
Gz <- tcrossprod(scale(Z)) # CENTRING AND SCALING
EVD <- eigen(Gz)
U <- EVD$vector
tU<-t(U)
```

```
val <- EVD$values
#qr(Gz)$rank
#plot(U[,1],U[,2],xlab='U1',ylab='U2')

# SIMULATE Z FROM TWO POPULATIONS WITH
# DIFFERENT GENE FREQUENCIES
Z1 <- matrix(nrow= 250,ncol= 1300,rbinom(250*1300,size=2,p=.5))
Z2 <- matrix(nrow= 250,ncol= 1300,rbinom(250*1300,size=2,p=.3))
Z <- rbind(Z1,Z2)
Gz <- tcrossprod(scale(Z))
EVD <- eigen(Gz)
U <- EVD$vector
tU<-t(U)
val <- EVD$values
qr(Gz)$rank
#plot(U[,1],U[,2],xlab='U1',ylab='U2')
```


Part I

Fitting Likelihood and Bayesian Models

Chapter 2

Likelihood



A central problem in statistics is the estimation of parameters that index a probability model proposed to describe aspects of the state of nature. In the classical approach to inference these parameters have a “true” but unknown value and given the model, can be estimated from a set of observations. A firmly entrenched inferential approach in statistics is the method of maximum likelihood proposed and termed by Fisher (1922), although, as is often the case in science, the subject had been in the air long before Fisher. An excellent account is in Hald (1998).

This chapter starts by providing an intuition for the concept of likelihood emphasising the conceptual difference between a likelihood function and a probability function. This is followed by a summary of the basic results needed for classical likelihood inference. Proofs of most of the results presented here are not difficult and can be found, for example, in Sorensen and Gianola (2002). The final part of the chapter consists of examples that illustrate the construction of simple likelihood models, the derivation of the maximum likelihood estimators and some of their properties.

Readers that require a quick brush up of concepts of basic probability distributions and random variables may consider browsing through the first two chapters of Sorensen and Gianola (2002) that cover what is necessary of the subject to follow comfortably the material in this book.

2.1 A Little Intuition

Imagine that you are given the following *iid* (independently and identically distributed) data y sampled from a normal distribution $N(\mu, \sigma^2 = 2)$ with unknown mean μ and known variance $\sigma^2 = 2$:

$$y = (y_1 = 11.8, \quad y_2 = 6.1, \quad y_3 = 7.1, \quad y_4 = 11.1, \quad y_5 = 5.8).$$

You will agree that 10 is a better candidate for μ than 20, given the sample y (in fact, the sample was drawn from $N(\mu = 10, \sigma^2 = 2)$ using the R function `rnorm(5, 10, sqrt(2))`). In a general setting, suppose that a sample y from the distribution of Y is drawn, given the true (typically unknown) value of the parameter μ and the true known $\sigma^2 = 2$. This sample drawn with a certain probability is to provide information about μ . For fixed $Y = y$, compute $p(Y = y | \mu = \mu_1, \sigma^2 = 2)$ and $p(Y = y | \mu = \mu_2, \sigma^2 = 2)$, in order to choose between μ_1 and μ_2 as two possible values of μ . Viewed as a function of μ , $p(Y = y | \mu, \sigma^2 = 2)$ is the *likelihood function*. If $p(Y = y | \mu_1, \sigma^2 = 2) > p(Y = y | \mu_2, \sigma^2 = 2)$ we would choose $\mu = \mu_1$ on the grounds that, given the sample y , this is a “more likely” value than $\mu = \mu_2$. In fact, it is possible that if the true value of μ is μ_2 , we could still observe for the particular sample y , that $p(Y = y | \mu_1, \sigma^2 = 2) > p(Y = y | \mu_2, \sigma^2 = 2)$, but this occurs with small probability and this probability approaches zero as sample size tends to infinity (Lehmann and Casella 1998). This is what is meant by “more likely”. The value of the likelihood function $p(Y = y | \mu, \sigma^2 = 2)$ evaluated at $\mu = \mu_1$ agrees with the *probability of observing* $Y = y$ only when the *true value* of the parameter μ is in fact μ_1 (this is so if Y is discrete; otherwise for continuous Y the likelihood evaluated at $\mu = \mu_1$ is proportional to the probability that Y takes values in a small set containing y). However, the likelihood function is not a probability distribution. To quote Fisher (Fisher 1922): “The likelihood that any parameter (or set of parameters) should have any assigned value (or set of values) is proportional to the probability that if this were so, the totality of observations should be that observed”.

Irrespective of the true value of the parameter, the value of μ that maximises the likelihood function is the maximum likelihood estimator of μ , $\hat{\mu}$. The great appeal of the method is mostly due to the desirable statistical properties of the maximum likelihood estimator, many of which hold asymptotically. To be specific, given that the true value of μ is 10, the probability of observing $Y = y_1 = 11.8$ is proportional to $N(11.8 | \mu = 10, \sigma^2 = 2)$:

$$\Pr(Y = 11.8) \propto \text{dnorm}(11.8, 10, \text{sqrt}(2)) = 0.1255.$$

The probability of observing $Y = y$ is proportional to

$$\prod_{i=1}^5 N(y_i | \mu = 10, \sigma^2 = 2) = \text{prod}(\text{dnorm}(y, 10, \text{sqrt}(2))) = 1.9457 \times 10^{-8}.$$

On the other hand, the contribution to the likelihood function of $Y = y_1$ is

$$N(11.8 | \mu, \sigma^2 = 2) = (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{1}{2 \times 2} (11.8 - \mu)^2\right)$$

and in view of the *iid* nature of the data, the contribution from the whole data y to the likelihood function is

$$\prod_{i=1}^5 N(y_i | \mu, \sigma^2 = 2) = (2\pi 2)^{-\frac{5}{2}} \exp\left(-\frac{1}{2 \times 2} \sum_{i=1}^5 (y_i - \mu)^2\right). \quad (2.1)$$

The value of μ that maximises (2.1) is $\hat{\mu} = 8.38$. This is the maximum likelihood estimate.

Notation

- The shorthand used for probability density function is pdf, for probability mass function, pmf and for cumulative distribution function (or distribution function) is cdf.
- The shorthand used for independent and identically distributed is iid.
- MLE is shorthand for maximum likelihood estimator or maximum likelihood estimate and ML for maximum likelihood
- Often I use $E_X [T(X)]$, where $T(X)$ is some function of the random variable X with probability density function $p(x|\theta)$. By this I mean “integration over the distribution of X ”

$$E_X [T(X)] = \int T(x) p(x|\theta) dx$$

where θ is a parameter that indexes the probability density of the random variable X .

- The notation $X \sim N(\mu, \sigma^2)$ stands for X is distributed as $N(\mu, \sigma^2)$, a normal distribution with mean μ and variance σ^2 .
- The notation $N(x|\mu, \sigma^2)$ is used to specify the probability density function of X at the value $X = x$, when $X \sim N(\mu, \sigma^2)$.
- If λ is a $k \times 1$ vector and θ is a $p \times 1$ vector, then the j th row of the $k \times p$ matrix $\partial\lambda/\partial\theta'$ is

$$\left[\partial\lambda_j/\partial\theta_1 \quad \partial\lambda_j/\partial\theta_2 \quad \cdots \quad \partial\lambda_j/\partial\theta_p \right]$$

and the j th row of the $p \times k$ matrix $\partial\lambda'/\partial\theta = (\partial\lambda/\partial\theta')'$ is

$$\left[\partial\lambda_1/\partial\theta_j \quad \partial\lambda_2/\partial\theta_j \quad \cdots \quad \partial\lambda_k/\partial\theta_j \right].$$

- When f is a function of several variables $f(x) = f(x_1, x_2, \dots, x_n)$, the *gradient* of f is the column vector of n partial derivatives $\left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)'$ and is denoted ∇f .

- I use $[\theta|y]$ (with or without square brackets) to represent the distribution of the random variable θ given y . This may be a posterior distribution.
- Random variables Y , say, are not consistently distinguished in notation from their realised values y , unless the need to avoid confusion demands it. For example, I may write *the distribution of y given x* (symbolised as $[y|x]$), rather than: *the distribution of Y given $X = x$* (symbolised as $[Y|X = x]$), or for the probability mass function, $\Pr(y = 1|x)$ rather than $\Pr(Y = 1|X = x)$.
- I do not use boldface to distinguish matrices or vectors from scalars. Depending on context, matrices may be denoted with capital letters X or with lower case letters x . In both cases I make clear the dimension of the quantity at hand.
- The transpose of a matrix X (or of matrix x) is denoted by X' (or by x').

2.2 Summary of Likelihood Results

Let $p(y|\theta)$ represent the sampling density or mass function of the data vector y indexed by a parameter vector θ with p elements. The likelihood of the parameter θ given the data y , $L(\theta|y)$ is equal to $p(y|\theta)$,

$$L(\theta|y) = p(y|\theta). \quad (2.2)$$

If the elements of $y = (y_1, \dots, y_n)'$ are *iid*,

$$L(\theta|y) = \prod_{i=1}^n p(y_i|\theta). \quad (2.3)$$

The loglikelihood (also known as *support*) is

$$\ln L(\theta|y) = \ell(\theta|y) = \sum_{i=1}^n \ln p(y_i|\theta). \quad (2.4)$$

The *maximum likelihood estimator* $\hat{\theta}$ is the value of θ that maximises (2.4) (or (2.3)) over the parameter space of θ . Formally this involves a constrained maximisation. Often an unconstrained maximisation is performed; therefore one must check that the resulting maximum likelihood estimate is within the allowed parameter space.

One may always multiply a likelihood function by a constant c independent of θ . Therefore likelihood functions $L(\theta|y)$ and $cL(\theta|y)$ are equivalent in the sense that both lead to the same maximum likelihood estimate. Similarly, two loglikelihood functions that differ only by an additive constant lead to the same maximum likelihood estimate.

The first derivative of ℓ with respect to the vector θ is known as the *score function*, $S(\theta)$. Letting p be number of components in θ and using ℓ for $\ell(\theta|y)$,

$$S(\theta) = \frac{\partial}{\partial \theta} \ell(\theta|y) = \begin{pmatrix} \partial \ell / \partial \theta_1 \\ \vdots \\ \partial \ell / \partial \theta_p \end{pmatrix}, \quad (2.5)$$

so the score is an $p \times 1$ vector. It can be shown (Sorensen and Gianola 2002, pp. 132–133) that the score has zero mean and variance given by Fisher's expected information matrix (see (2.9) below); that is

$$S(\theta) \sim [0, E_y [S(\theta) S(\theta)']] . \quad (2.6)$$

Evaluated at the maximum $\hat{\theta}$

$$S(\hat{\theta}) = 0$$

when $\hat{\theta}$ is in the interior of the parameter space.

The matrix of second derivatives of the loglikelihood is known as the *Hessian*, $H(\theta|y)$. For example, if $p = 2$, $H(\theta|y)$ takes the form

$$H(\theta|y) = \frac{\partial^2}{\partial \theta \partial \theta'} \ell(\theta|y) = \begin{bmatrix} \partial^2 \ell / (\partial \theta_1)^2 & \partial^2 \ell / (\partial \theta_1 \partial \theta_2) \\ \partial^2 \ell / (\partial \theta_2 \partial \theta_1) & \partial^2 \ell / (\partial \theta_2)^2 \end{bmatrix}.$$

The *observed information matrix* (of dimension $p \times p$) is given by

$$I(\theta|y) = -\frac{\partial^2}{\partial \theta \partial \theta'} \ell(\theta|y) = -H(\theta|y) \quad (2.7)$$

which involves second derivatives and is a function of both θ and y . An approximation that uses first derivatives only is given by the outer product of the vector of first derivatives of the score

$$(S(\theta) S(\theta)') . \quad (2.8)$$

For example, if $p = 2$,

$$\begin{aligned} S(\theta) S(\theta)' &= \begin{bmatrix} \partial \ell / \partial \theta_1 \\ \partial \ell / \partial \theta_2 \end{bmatrix} \begin{bmatrix} \partial \ell / \partial \theta_1 & \partial \ell / \partial \theta_2 \end{bmatrix} \\ &= \begin{bmatrix} (\partial \ell / \partial \theta_1)^2 & (\partial \ell / \partial \theta_1) (\partial \ell / \partial \theta_2) \\ (\partial \ell / \partial \theta_2) (\partial \ell / \partial \theta_1) & (\partial \ell / \partial \theta_2)^2 \end{bmatrix}. \end{aligned}$$

Fisher's *expected information matrix* is defined to be the average of (2.7) or (2.8) over conceptual replications of y :

$$i(\theta) = E_y \left[\left(\frac{\partial}{\partial \theta} \ell(\theta|y) \right) \left(\frac{\partial}{\partial \theta} \ell(\theta|y) \right)' \right] = -E_y \left(\frac{\partial^2}{\partial \theta \partial \theta'} \ell(\theta|y) \right) \quad (2.9)$$

and is a function of θ only. Typically these information matrices are evaluated at $\theta = \hat{\theta}$.

Let θ_0 denote the true value of θ . A standard result says that the sampling distribution of the MLE has a limiting normal distribution

$$\hat{\theta} \rightarrow N(\theta_0, i(\theta_0)^{-1}) \quad (2.10)$$

as sample size goes to infinity. This sampling distribution may be approximated using

$$N(\hat{\theta}, i(\hat{\theta})^{-1}) \text{ or } N(\hat{\theta}, I(\hat{\theta})^{-1}). \quad (2.11)$$

Estimates of standard errors are obtained from

$$\sqrt{i(\hat{\theta})_{jj}^{-1}} \text{ or } \sqrt{I(\hat{\theta})_{jj}^{-1}}. \quad (2.12)$$

Cramér-Rao Theorem given certain regularity conditions, the variance of any unbiased estimator of a parameter θ (scalar or vector) must be at least as large as

$$\text{Var}(\hat{\theta}) \geq i(\theta)^{-1}. \quad (2.13)$$

When θ is a vector, the above inequality implies that

$$\text{Var}(\hat{\theta}) - i(\theta)^{-1}$$

is positive semidefinite. Any unbiased estimator that achieves this lower bound is *efficient* and no better unbiased estimator can be found. It follows from (2.10) that the ML estimator satisfies the Cramér-Rao lower bound, asymptotically; therefore the asymptotic variance is the smallest possible.

An important property of ML is *functional invariance*. If $\hat{\theta}$ is the MLE of θ and if $\lambda = g(\theta)$ is a one-to-one transformation such that $\theta = g^{-1}(\lambda)$ exists, then the MLE of $g(\theta)$ is $g(\hat{\theta})$.

If θ is a scalar, under the new parametrisation the score can be written

$$\frac{d\ell(g^{-1}(\lambda)|y)}{d\lambda} = \frac{d\ell(\theta|y)}{d\theta} \frac{d\theta}{d\lambda} \quad (2.14)$$

and if θ and λ are vectors

$$\frac{\partial \ell(g^{-1}(\lambda)|y)}{\partial \lambda} = \frac{\partial \theta'}{\partial \lambda} \frac{\partial \ell(\theta|y)}{\partial \theta}. \quad (2.15)$$

If θ is a scalar, the expected information about the new parameter λ contained in the sample is

$$i(\lambda) = i(\theta) \left(\frac{d\theta}{d\lambda} \right)^2 = i(g^{-1}(\lambda)) \left[\frac{dg^{-1}(\lambda)}{d\lambda} \right]^2 \quad (2.16)$$

and the asymptotic variance is

$$\text{Var}(\hat{\lambda}) = [i(\lambda)]^{-1} = i(\theta)^{-1} \left(\frac{d\lambda}{d\theta} \right)^2 = \text{Var}(\hat{\theta}) \left(\frac{d\lambda}{d\theta} \right)^2. \quad (2.17)$$

When θ and λ are $p \times 1$ vectors the expected information matrix is

$$i(\lambda) = \frac{\partial \theta'}{\partial \lambda} i(\theta) \frac{\partial \theta}{\partial \lambda'} = \frac{\partial [g^{-1}(\lambda)]'}{\partial \lambda} i(g^{-1}(\lambda)) \frac{\partial [g^{-1}(\lambda)]}{\partial \lambda'}, \quad (2.18)$$

where $\frac{\partial \theta'}{\partial \lambda}$ and $i(\theta)$ are $p \times p$ matrices. The asymptotic variance evaluated at $\lambda = \hat{\lambda}$ is

$$\begin{aligned} \text{Var}(g(\hat{\theta})) &= [i(\hat{\lambda})]^{-1} \\ &= \frac{\partial \lambda}{\partial \theta'} \Big|_{\theta=\hat{\theta}} \text{Var}(\hat{\theta}) \frac{\partial \lambda'}{\partial \theta} \Big|_{\theta=\hat{\theta}} \end{aligned} \quad (2.19)$$

where $\frac{\partial \lambda}{\partial \theta'}$ and $\text{Var}(\hat{\theta})$ are $p \times p$ matrices. Expression (2.19) is the multiparameter extension of (2.17) that can also be derived using the “delta method” (Sorensen and Gianola 2002, p. 95). The derivation based on the delta method does not require that vectors θ and λ have the same number of elements. An application of (2.19) is in the Exercises on page 679.

Also asymptotically, for scalar θ and $g(\theta)$

$$g(\hat{\theta}) \sim N \left(g(\theta), i(\theta)^{-1} \left(\frac{dg(\theta)}{d\theta} \right)^2 \right). \quad (2.20)$$

If $g(\hat{\theta})$ is a $p \times 1$ vector a similar results holds, except that the asymptotic variance in (2.20) is replaced by the $p \times p$ matrix given by (2.19). In applications θ is replaced by $\hat{\theta}$.

The MLE is also invariant to transformations of the data that do not depend on the parameter θ . If data X that have density function $p_X(x|\theta)$ are transformed to $Y = f(X)$ so that the inverse transformation is $f^{-1}(Y) = X$ exists, then the density of the transformed data Y is

$$\begin{aligned} p_Y(y|\theta) &= p_X(f^{-1}(y)|\theta) \left| \frac{df^{-1}(y)}{dy} \right| \\ &= p_X(x|\theta) \left| \frac{dx}{dy} \right| \end{aligned} \quad (2.21)$$

and therefore the likelihood function based on X and Y differ only by a factor (the Jacobian of the transformation) that does not depend on the parameter θ .

Note

The equality in the second line of (2.19) is based on the following two results. First, for full rank square matrices A, B, C

$$[ABC]^{-1} = C^{-1}B^{-1}A^{-1}.$$

Secondly,

$$\left[\frac{\partial \theta}{\partial \lambda'} \right]^{-1} = \left[\frac{\partial \lambda}{\partial \theta'} \right].$$

The derivation holds when the transformation $\lambda = g(\theta)$ is one-to-one.

When the dimension of λ is $k \times 1$ and of θ is $p \times 1$, direct application of the delta method yields

$$\text{Var}(g(\hat{\theta})) = \left. \frac{\partial \lambda}{\partial \theta'} \right|_{\theta=\hat{\theta}} \text{Var}(\hat{\theta}) \left. \frac{\partial \lambda'}{\partial \theta} \right|_{\theta=\hat{\theta}}$$

as in (2.19), but now $\frac{\partial \lambda}{\partial \theta'}$ is a $k \times p$ matrix. The stronger assumption of normality of the asymptotic distribution of $g(\hat{\theta})$ arrived at using the delta method requires that $k \leq p$ and that the rank of $\frac{\partial \lambda}{\partial \theta'}$ is equal to k .

Summary of Properties of Maximum Likelihood Estimators

Little is known about small sample properties of MLE. In many situations it is known that MLE are not unbiased in small samples.

In large samples, MLE

- (i) are consistent
- (ii) are asymptotically normal (i.e., (2.10))
- (iii) in a well specified model the MLE achieves the Cramér-Rao lower bound

Things to be aware of:

- (i) The MLE may not be a turning point (the likelihood increases continuously)
- (ii) The MLE may not be unique (the maximum is found over a flat region spanning a wide interval comprising the parameter—“flat” likelihood functions)
- (iii) If iterative procedures are used to find a maximum, rate of convergence may be extremely slow, particularly in complex multiparameter models
- (iv) An iterative procedure may converge to a local maximum, not necessarily to a global maximum
- (v) Asymptotic normality of the MLE is compromised if the parameter lies on the border of the parameter space
- (vi) The zeroes of the first derivatives only locate extreme points in the interior of the domain of a function. If extrema only occur on the boundaries or corners, first derivatives may not be zero at those points

2.3 Example: The Likelihood Function of Transformed Data

An example of (2.21) is the following. Let X have density

$$p(x) = \frac{1}{\sqrt{2\pi}} \exp \left[-\frac{1}{2} (x - \mu)^2 \right] \quad (2.22)$$

which is the density of the normal process $N(\mu, 1)$. Assume that X is transformed to $Y = f(X) = \exp(X)$. The inverse transformation is $X = f^{-1}(Y) = \ln(Y)$. The density function of Y is then, using (2.21),

$$\begin{aligned} p(y) &= p(f^{-1}(y)) \left| \frac{d}{dy} f^{-1}(y) \right| \\ &= \frac{1}{y\sqrt{2\pi}} \exp \left[-\frac{1}{2} (\ln y - \mu)^2 \right], \end{aligned} \quad (2.23)$$

where $1/y$ enters through the Jacobian of the transformation. Then $Y = \exp(X)$ is said to have a lognormal distribution with density as given above. The likelihood based on Y is proportional to this density. Inspection of (2.22) and (2.23) reveals that the two likelihoods are maximised when the exponential term is equal to 1. Therefore the MLE of μ based on X is $\bar{x}$ and based on Y is $\ln \bar{y}$. In other words, the MLE of the lognormal likelihood is the same as that of the normal likelihood fitted to the logarithm of the data.

2.4 Example: Linear Regression

Linear regression is one of the most widely adopted statistical methods and its use in genetics probably dates back to Galton (1885). For example, a simple approach for estimating heritability of quantitative traits is based on regressing offspring records on the records of their fathers and mothers, or regressing offspring records on mid-parental records.

This example, taken from Sorensen and Gianola (2002), is based on such a linear regression model where observations (for example, the offspring records) are postulated to be linked to an intercept β_0 and to a slope parameter β_1 via the relationship

$$y_i = \beta_0 + \beta_1 x_i + e_i.$$

In this expression, the scalars x_i ($i = 1, 2, \dots, n$) are known values of an explanatory variable (for example, the mid-parental values) and $e_i \sim N(0, \sigma^2)$ is an unobserved random residual term. The n residuals are assumed to be iid. The parameter vector is $[\beta_0, \beta_1, \sigma^2]$. In a genetics context, β_1 (the regression of offspring on mid-parents) is interpreted as the heritability of the quantitative trait (Falconer and Mackay 1996).

The likelihood function can be written as

$$L(\beta_0, \beta_1, \sigma^2 | y) \propto (\sigma^2)^{-\frac{n}{2}} \exp \left[-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \right]$$

and the corresponding loglikelihood, apart from an additive constant, is

$$l(\beta_0, \beta_1, \sigma^2 | y) = -\frac{n}{2} \ln(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

The score vector is given by

$$\begin{bmatrix} \frac{\partial l}{\partial \beta_0} \\ \frac{\partial l}{\partial \beta_1} \\ \frac{\partial l}{\partial \sigma^2} \end{bmatrix} = \begin{bmatrix} \frac{1}{\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \\ \frac{1}{\sigma^2} \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) \\ -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \end{bmatrix}.$$

Setting the score vector equal to zero and solving simultaneously for the unknown parameters gives explicit solutions to the ML equations. The MLEs are

$$\hat{\sigma}^2 = \frac{\sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2}{n},$$

where $\hat{\beta}_0$ and $\hat{\beta}_1$ are the solutions to the matrix equation

$$\begin{bmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{bmatrix} \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i y_i \end{bmatrix}.$$

Explicitly,

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

and

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n x_i y_i - \frac{1}{n} \sum_{i=1}^n x_i \sum_{i=1}^n y_i}{\sum_{i=1}^n x_i^2 - \frac{1}{n} (\sum_{i=1}^n x_i)^2}.$$

The first two columns of the 3×3 matrix of the negative of second derivatives of the loglikelihood with respect to the parameters, or observed information matrix, are

$$\begin{bmatrix} \sigma^{-2} n & \sigma^{-2} \sum_{i=1}^n x_i \\ \sigma^{-2} \sum_{i=1}^n x_i & \sigma^{-2} \sum_{i=1}^n x_i^2 \\ \sigma^{-4} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) & \sigma^{-4} \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) \end{bmatrix}$$

and the last column is

$$\begin{bmatrix} \sigma^{-4} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \\ \sigma^{-4} \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) \\ -(2\sigma^4)^{-1} n + \sigma^{-6} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \end{bmatrix}.$$

It is easy to verify that the expected value of each of the elements of the score vector is null. For example,

$$E\left(\frac{\partial l}{\partial \beta_1}\right) = \frac{1}{\sigma^2} \sum_{i=1}^n x_i E(y_i - \beta_0 - \beta_1 x_i) = 0$$

and

$$\begin{aligned} E\left(\frac{\partial l}{\partial \sigma^2}\right) &= -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^n E(y_i - \beta_0 - \beta_1 x_i)^2 \\ &= -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} n\sigma^2 = 0. \end{aligned}$$

Further, the expected information matrix is given by

$$i(\theta) = \sigma^{-2} \begin{bmatrix} n & \sum_{i=1}^n x_i & 0 \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 & 0 \\ 0 & 0 & (2\sigma^2)^{-1} n \end{bmatrix}.$$

Note that the elements (1, 2) and (2, 1) of this matrix are not null. This illustrates that in a multiparameter model it is often more sensible to speak about joint information on a set of parameters rather than about information on individual parameters themselves.

Invariance

The original parametrisation was in terms of $\theta = [\beta_0, \beta_1, \sigma^2]'$. Imagine that there is interest in a new parametrisation consisting of the vector

$$\eta = \begin{bmatrix} \eta_1 \\ \eta_2 \\ \eta_3 \end{bmatrix} = \begin{bmatrix} \beta_0/\beta_1 \\ \beta_1 \\ \sigma^2 \end{bmatrix}$$

with inverse

$$\theta = f^{-1}(\eta) = \begin{bmatrix} \eta_1 \eta_2 \\ \eta_2 \\ \eta_3 \end{bmatrix}.$$

The 3×3 matrix defined after (2.15) would be

$$\frac{\partial \theta'}{\partial \eta} = \begin{bmatrix} \partial \theta_1 / \partial \eta_1 & \partial \theta_2 / \partial \eta_1 & \partial \theta_3 / \partial \eta_1 \\ \partial \theta_1 / \partial \eta_2 & \partial \theta_2 / \partial \eta_2 & \partial \theta_3 / \partial \eta_2 \\ \partial \theta_1 / \partial \eta_3 & \partial \theta_2 / \partial \eta_3 & \partial \theta_3 / \partial \eta_3 \end{bmatrix} = \begin{bmatrix} \eta_2 & 0 & 0 \\ \eta_1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (2.24)$$

Differentiating the loglikelihood twice under the new parametrisation, multiplying by -1 and taking expectations, yields the information matrix

$$I(\eta) = \frac{1}{\eta_3} \begin{bmatrix} n\eta_2^2 & \eta_2 (n\eta_1 + \sum_{i=1}^n x_i) & 0 \\ \eta_2 (n\eta_1 + \sum_{i=1}^n x_i) & \sum_{i=1}^n (\eta_1 + x_i)^2 & 0 \\ 0 & 0 & \frac{n}{2\eta_3} \end{bmatrix}.$$

It can be verified that the same result is obtained from the matrix product given in (2.18), employing the matrix (2.24).

2.5 Example: Bivariate Normal Model with Missing Records

Consider a data set consisting of two rows; the first has phenotypic records on fathers (x), the second on sons (y), one son per father. Some of the records of the sons are missing. The ordered data can be represented as

$$\begin{aligned} x_1, x_2, \dots, x_m, x_{m+1}, \dots, x_n, \\ y_1, y_2, \dots, y_m. \end{aligned} \quad (2.25)$$

There are m complete independent bivariate observations (x_i, y_i) , $i = 1, 2, \dots, m$, and $(n - m)$ univariate independent records from fathers without sons. Each father that left offspring is assumed to have mated with a randomly chosen unknown female and produced one son. At this stage it is assumed that the observed pattern of missing records may depend on the observed x 's but not on the missing values themselves. In other words, the missing data $(y_{m+1}, \dots, y_n)$ are *missing at random* (Rubin 1976, 2002) and the missing data mechanism is *ignorable*, in the sense that this mechanism does not have to be incorporated as part of the likelihood model for correct inferences.

The data could represent LDL cholesterol levels in humans collected with the objective of estimating the parameters of a probability model entertained by the data analyst. The parameters could be mean levels of LDL cholesterol and variance components and functions thereof, such as heritability. Information on the latter indicates whether genetic factors may affect LDL cholesterol.

Assume that a record of a father and his son is a draw from the bivariate normal model

$$[(y_i, x_i) | \mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx}] \sim N \left(\begin{bmatrix} \mu_y \\ \mu_x \end{bmatrix}, \begin{bmatrix} \sigma_{yy} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{xx} \end{bmatrix} \right), \quad i = 1, \dots, n, \quad (2.26)$$

and assume that the pairs (y_i, x_i) are *iid*. The covariance term is usually interpreted as $\sigma_{xy} = \sigma_{xx}h^2/2$, where h^2 is the heritability (twice the regression of sons on fathers) in the population from which fathers were sampled (with assumptions, such as absence of common environmental effects between parents and offspring). The correlation coefficient is $\rho = \sigma_{xy}/\sqrt{\sigma_{yy}\sigma_{xx}}$ and is equal to $h^2/2$ when $\sigma_{xx} = \sigma_{yy}$.

In the absence of missing records (if the $(n - m)$ y 's were not missing), the joint density of the n pairs (y_i, x_i) is

$$\begin{aligned} p(y, x | \mu_y, \mu_x, \rho, \sigma_{yy}, \sigma_{xx}) = \\ \left(2\pi\sigma_{yy}\sigma_{xx} (1 - \rho^2) \right)^{-\frac{n}{2}} \exp \left[-\frac{1}{2(1 - \rho^2)} \left(\frac{\sum_{i=1}^n (x_i - \mu_x)^2}{\sigma_{xx}} \right. \right. \end{aligned}$$

$$\left. + \frac{\sum_{i=1}^n (y_i - \mu_y)^2}{\sigma_{yy}} - 2\rho \frac{\sum_{i=1}^n (x_i - \mu_x)(y_i - \mu_y)}{(\sigma_{xx}\sigma_{yy})^{\frac{1}{2}}} \right) \Bigg], \quad (2.27)$$

and the likelihood is proportional to (2.27). In this situation (no missing records), the ML estimators are in closed form and are given by

$$\begin{aligned} \hat{\mu}_x &= \frac{1}{n} \sum_{i=1}^n x_i, & \hat{\mu}_y &= \frac{1}{n} \sum_{i=1}^n y_i, \\ \hat{\sigma}_{xx} &= \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu}_x)^2, & \hat{\sigma}_{yy} &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\mu}_y)^2, \\ \hat{\sigma}_{xy} &= \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu}_x)(y_i - \hat{\mu}_y), \\ \hat{\rho} &= \frac{\hat{\sigma}_{xy}}{(\hat{\sigma}_{xx}\hat{\sigma}_{yy})^{\frac{1}{2}}}. \end{aligned} \quad (2.28)$$

However, with the present pattern of missing records the likelihood is proportional to

$$\begin{aligned} p(y, x | \mu_y, \mu_x, \rho, \sigma_{yy}, \sigma_{xx}) = & \\ & \left(2\pi\sigma_{yy}\sigma_{xx} (1 - \rho^2) \right)^{-\frac{m}{2}} \exp \left[-\frac{1}{2(1 - \rho^2)} \left(\frac{\sum_{i=1}^m (x_i - \mu_x)^2}{\sigma_{xx}} \right. \right. \\ & \left. \left. + \frac{\sum_{i=1}^m (y_i - \mu_y)^2}{\sigma_{yy}} - 2\rho \frac{\sum_{i=1}^m (x_i - \mu_x)(y_i - \mu_y)}{(\sigma_{xx}\sigma_{yy})^{\frac{1}{2}}} \right) \right] (2\pi\sigma_{xx})^{-\frac{(n-m)}{2}} \\ & \exp \left[-\frac{\sum_{i=m+1}^n (x_i - \mu_x)^2}{2\sigma_{xx}} \right]. \end{aligned} \quad (2.29)$$

The likelihood equations derived from this likelihood function are difficult to solve analytically.

Rather than expressing the likelihood as in (2.29), an alternative strategy can be followed. The normal density of the joint distribution (2.26) can be factorised as the product of two normal densities

$$p(y_i, x_i | \mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx}) = p(x_i | \mu_x, \sigma_{xx}) p(y_i | x_i, \mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx}). \quad (2.30)$$

The parameters of the two terms in the right hand side of (2.30) are not distinct because (μ_x, σ_{xx}) are common to both factors. Therefore, a global maximisation of the likelihood requires that the two terms of the right hand side of (2.30) are maximized jointly. The resulting algebra is quite messy.

Considering all the records, the first term in the right hand side of (2.30) is

$$(2\pi\sigma_{xx})^{-\frac{n}{2}} \exp \left[-\frac{\sum_{i=1}^n (x_i - \mu_x)^2}{2\sigma_{xx}} \right] \quad (2.31)$$

and the second term is (2.30) and

$$(2\pi\sigma_{y.x})^{-\frac{m}{2}} \exp \left[-\frac{\sum_{i=1}^m \left(y_i - \mu_y - \frac{\sigma_{xy}}{\sigma_{xx}} (x_i - \mu_x) \right)^2}{2\sigma_{y.x}} \right], \quad (2.32)$$

where

$$\sigma_{y.x} = \sigma_{yy} - \frac{(\sigma_{xy})^2}{\sigma_{xx}},$$

is the variance of the conditional distribution of y given x . The mean and variance of the distribution $[x_i | \mu_x, \sigma_{xx}]$ are then μ_x and σ_{xx} , respectively and the corresponding parameters of $[y_i | x_i, \mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx}]$ are

$$\begin{aligned} E(y_i | x_i, \mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx}) &= \mu_y + \frac{\sigma_{xy}}{\sigma_{xx}} (x_i - \mu_x), \\ \text{Var}(y_i | x_i, \mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx}) &= \sigma_{yy} - \frac{(\sigma_{xy})^2}{\sigma_{xx}}. \end{aligned}$$

Consider using the factorised likelihood (2.30) but rather than parametrising with $\theta = (\mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx})$, use instead $\phi = (\mu_x, \sigma_{xx}, \beta_0, \beta_1, \sigma_{y.x})$, where (μ_x, σ_{xx}) are common to both parametrisations and the other components of ϕ are given by the following functions of the components of θ

$$\begin{aligned} \beta_0 &= \mu_y - \beta_1 \mu_x, \\ \beta_1 &= \frac{\sigma_{xy}}{\sigma_{xx}}, \\ \sigma_{y.x} &= \sigma_{yy} - \frac{(\sigma_{xy})^2}{\sigma_{xx}}. \end{aligned}$$

Under this parametrisation the density of the pair (y_i, x_i) becomes

$$p(y_i, x_i | \mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx}) = p(x_i | \mu_x, \sigma_{xx}) p(y_i | x_i, \beta_0, \beta_1, \sigma_{y.x}). \quad (2.33)$$

The form of the first term in the right hand side of (2.33) is the same as in (2.31) and the form of the second term is

$$p(y_i | x_i, \beta_0, \beta_1, \sigma_{y.x}) = (2\pi\sigma_{y.x})^{-\frac{m}{2}} \exp \left[-\frac{\sum_{i=1}^m (y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma_{y.x}} \right]. \quad (2.34)$$

The mean and variance of the distribution corresponding to (2.34) are

$$\begin{aligned} E(y_i | x_i, \beta_0, \beta_1) &= \beta_0 + \beta_1 x_i \quad (= \mu_y + \beta_1 (x_i - \mu_x)), \\ \text{Var}(y_i | x_i, \sigma_{y.x}) &= \sigma_{y.x}. \end{aligned}$$

The parameters in the two terms of the right hand side of (2.33) are distinct; the global maximisation of the likelihood can proceed by maximising the likelihood of each term separately. Each of the two terms corresponds to the likelihood of a straightforward problem: $p(x_i | \mu_x, \sigma_{xx})$ is the likelihood of $N(\mu_x, \sigma_{xx})$, the marginal distribution of x_i and $p(y_i | x_i, \beta_0, \beta_1, \sigma_{y.x})$ is the likelihood of $N(\beta_0 + \beta_1 x_i, \sigma_{y.x})$, the likelihood of the regression of father on son. This development was apparently first given by Anderson (1957).

2.6 Remarks on Inferences Under Selection

The two examples that follow discuss how inferences may be affected when data are non-randomly sampled. This section provides a little background.

Consider two random vectors $x \in \Omega$ and $y \in \Phi$ where Ω and Φ are the respective sample spaces. The joint density of x and y is

$$p(x, y) = p(y|x) p(x), \quad x \in \Omega, y \in \Phi, \quad (2.35)$$

where the parameters of these distributions are omitted from the notation. The likelihood is proportional to (2.35).

Imagine that x is non-randomly sampled and that $S \subset \Omega$, a subset of Ω , includes the non-randomly sampled vector x . What are the consequences of selection operating on x , when inferences are based on the conditional likelihood $p(y|x)$? The conditional likelihood of y given that $x \in S$ is

$$\begin{aligned} p(y|x, x \in S) &= \frac{p(y, x|x \in S)}{p(x|x \in S)} \\ &= \frac{p(y, x)}{\Pr(x \in S)} \frac{\Pr(x \in S)}{p(x)}, \\ &= p(y|x), \quad x \in S, y \in \Phi, \end{aligned} \quad (2.36)$$

that takes the same mathematical form as if selection on x had not taken place. Selection on x can be ignored.

Let $z \in \Psi$ represent a third random vector with sample space Ψ . Assume that z is selected and $S \subset \Psi$, a subset of Ψ , includes the non-randomly sampled vector z . What are the consequences of non-random sampling of z if inferences are to be based on the conditional likelihood $p(y|x)$? The joint likelihood is $p(x, y, z)$ and the conditional likelihood is

$$\begin{aligned} p(y|x, z \in S) &= \frac{\int_S p(x, y, z) dz}{\int_S \int_\Phi p(x, y, z) dy dz} \\ &= \frac{\int_S p(x, y, z) dz}{\int_S p(x, z) dz}, \quad x \in \Omega, y \in \Phi, \end{aligned} \quad (2.37)$$

which is different from (2.36). For correct inferences, selection on z cannot be ignored and must be incorporated as part of the likelihood.

If z is independent of (x, y) so that $p(x, y|z) = p(x, y)$ and $p(x|z) = p(x)$, factorising numerator and denominator in (2.37):

$$\begin{aligned} p(y|x, z \in S) &= \frac{\int_S p(x, y|z) p(z) dz}{\int_S p(x|z) p(z) dz} \\ &= \frac{p(x, y)}{p(x)} \frac{\int_S p(z) dz}{\int_S p(z) dz} \\ &= p(y|x), \quad x \in \Omega, y \in \Phi, \end{aligned} \quad (2.38)$$

and the likelihood based on (2.38) is equal to the conditional likelihood of y given x as if selection on z had not occurred. These are special cases of a more general theory of inferences under missing data (Rubin 1976; Little and Rubin 1987).

2.7 Example: Likelihood Inferences Using Selected Records

Consider the following problem discussed in Lush and Shrode (1950) and in Henderson et al. (1959). One is to obtain an estimate of the difference in milk production between years. The data on which inferences are to be based are selected: second year records are obtained only from those cows that had the highest first year records. The lowest producers of first year records are not allowed to produce in year 2 and are culled from the herd. The pattern of missing records in this example is identical to the pattern displayed in (2.25): the missing data mechanism is only a function of the observed data (the first lactation records in this example) and therefore the data are *missing at random* (Rubin 2002) and the selection process does not have to be incorporated as part of the likelihood model; the selection mechanism is ignorable. A heuristic conditional argument provides an alternative explanation. Given a data structure of the form in (2.25), the likelihood can be factorised as

$$\begin{aligned} & p(x_1, x_2, \dots, x_m, x_{m+1}, \dots, x_n, y_1, y_2, \dots, y_m; \theta) \\ &= p(y_1, y_2, \dots, y_m | x_1, x_2, \dots, x_m; \theta_1) p(x_1, x_2, \dots, x_m, x_{m+1}, \dots, x_n; \theta_2), \end{aligned}$$

where the θ 's are parameters of the respective densities. The conditional likelihood, given x (first term on the right hand side) is unaffected by selection on x and the second term includes all the parental data. Therefore selection is ignorable.

Let y_{i1} and y_{i2} be age-adjusted records of the i th cow in years 1 and 2, respectively, with

$$y_{i1} = \mu + c_i + e_{i1}, \quad i = 1, 2, \dots, N \quad (2.39)$$

and

$$y_{i2} = \mu + c_i + \delta + e_{i2}, \quad i = 1, 2, \dots, n, \quad (2.40)$$

where c_i is an effect of cow assumed $N(0, \sigma_c^2)$, e_{i1} and $e_{i2} \sim N(0, \sigma_e^2)$ and δ is the difference between second and first year records that one wishes to estimate. There are N cows that have a first lactation record and n have a second lactation record. In the absence of culling $n = N$. With culling based on first lactation information $n < N$.

From the models,

$$\begin{aligned} E(y_{i1}) &= \mu \\ E(y_{i2}) &= \mu + \delta \\ E(y_{i2}) - E(y_{i1}) &= \delta \\ \text{Var}(y_{i1}) &= \text{Var}(y_{i2}) = \sigma^2 = \sigma_c^2 + \sigma_e^2 \end{aligned}$$

$$\begin{aligned}\text{Cov}(y_{i1}, y_{i2}) &= \sigma_c^2 = \rho\sigma^2 \\ \text{Corr}(y_{i1}, y_{i2}) &= \rho = \frac{\text{Cov}(y_{i1}, y_{i2})}{\sqrt{\text{Var}(y_{i1})}\sqrt{\text{Var}(y_{i2})}} = \frac{\sigma_c^2}{\sigma^2}.\end{aligned}$$

Above, ρ is the repeatability or correlation between y_{i1} and y_{i2} . Assume that, in the absence of selection, y_{i1} and y_{i2} are bivariate normally distributed. Then,

$$\begin{bmatrix} y_{i1} \\ y_{i2} \end{bmatrix} \sim N \left[\begin{bmatrix} \mu \\ \mu + \delta \end{bmatrix}, \begin{bmatrix} \sigma^2 & \rho\sigma^2 \\ \rho\sigma^2 & \sigma^2 \end{bmatrix} \right]. \quad (2.41)$$

Therefore,

$$y_{i2}|y_{i1} \sim N \left[\mu + \delta + \rho(y_{i1} - \mu), \sigma^2(1 - \rho^2) \right] \quad (2.42)$$

and

$$y_{i1} \sim N(\mu, \sigma^2). \quad (2.43)$$

If the conditional expectation of a second year record, given the first year record

$$E(y_{i2}|y_{i1}) = \mu + \delta + \rho(y_{i1} - \mu), \quad (2.44)$$

is averaged over all cows that have a year 1 record

$$\begin{aligned}E_{y_{i1}}[E(y_{i2}|y_{i1})] &= E(y_{i2}) = \mu + \delta + \rho E_{y_{i1}}[(y_{i1} - \mu)] \\ &= \mu + \delta,\end{aligned} \quad (2.45)$$

as expected from the model. However, only the best cows in year 1 are allowed to produce in year 2. This means that, with selection, the expected value of year 1 records is not μ but

$$E(y_{i1}|y_{i1} \in S) = \mu_s \neq \mu, \quad (2.46)$$

where S is the set of cows with records in year 1 that are allowed to produce a second record in year 2 and μ_s is the mean of the selected records of year 1. Then we have

$$\begin{aligned}E_{y_{i1}}[E(y_{i2}|y_{i1} \in S)] &= \mu + \delta + \rho E_{y_{i1}}[(y_{i1} - \mu) | y_{i1} \in S] \\ &= \mu + \delta + \rho(\mu_s - \mu).\end{aligned} \quad (2.47)$$

Estimation by Least Squares

Properties of the least squares estimator of δ , the difference between milk production in year 1 and 2 are presented. A least squares estimator of this difference, including only cows that have a record on both years, is

$$\hat{d}_1 = \bar{y}_{.2} - \bar{y}_{.1}, \quad y_{i1} \in S.$$

The expected value of $\hat{d}_1$ is

$$\begin{aligned} E(\hat{d}_1) &= E_{y_{i1}} [E(\bar{y}_{.2} - \bar{y}_{.1} | y_{i1} \in S)] = (\mu + \delta + \rho(\mu_s - \mu)) - \mu_s \\ &= \delta - (1 - \rho)(\mu_s - \mu). \end{aligned} \quad (2.48)$$

Since $\mu_s > \mu$, the estimator of the difference is biased downwards, unless $\rho = +1$. With selected data, least squares estimates of year effects based on (2.48) give the impression of negative environmental trend with time.

Another possibility would be to estimate δ by taking the average of all year 2 records minus all year 1 records. The expected value of this second estimator is

$$\begin{aligned} E(\hat{d}_2) &= E_{y_{i1}} [E(\bar{y}_{.2} | y_{i1} \in S)] - E(\bar{y}_{.1}) = (\mu + \delta + \rho(\mu_s - \mu)) - \mu \\ &= \delta + \rho(\mu_s - \mu), \end{aligned} \quad (2.49)$$

which is biased unless $\rho = 0$, in which case selection would not be effective in changing milk production.

The conclusion is that with this type of selection (culling type), the least squares estimate of the difference in milk production is biased. On the other hand, BLUP (best linear unbiased predictor) yields unbiased estimates of genetic and environmental trends under certain types of selection. The paper where this is discussed at length is Henderson (1975). The following subsection describes in a simplified manner estimation of year effects using maximum likelihood. An important paper on the topic is Curnow (1961).

Estimation by Maximum Likelihood

The parameters of the model defined by (2.42) and (2.43) are

$$\theta' = (\mu, \delta, \sigma^2, \rho).$$

However, in this simplified problem, it will be assumed that ρ and σ^2 are known. In view of (2.42) and (2.43), the likelihood is

$$L(\theta|\mathbf{y}) \propto (2\pi\sigma^2)^{-\frac{N}{2}} \exp\left[-\frac{\sum_{i=1}^N (y_{i1} - \mu)^2}{2\sigma^2}\right] \\ \times (2\pi(1-\rho^2)\sigma^2)^{-\frac{n}{2}} \exp\left[-\frac{\sum_{i=1}^n (y_{i2} - \mu - \delta - \rho(y_{i1} - \mu))^2}{2(1-\rho^2)\sigma^2}\right]. \quad (2.50)$$

With ρ and σ^2 assumed known, the loglikelihood, apart from an additive constant is

$$l(\mu, \delta|\mathbf{y}) = -\frac{(1-\rho^2)\sum_{i=1}^N (y_{i1} - \mu)^2 + \sum_{i=1}^n (y_{i2} - \mu - \delta - \rho(y_{i1} - \mu))^2}{2(1-\rho^2)\sigma^2}. \quad (2.51)$$

This loglikelihood can be written as

$$l(\mu, \delta|\mathbf{y}) = k - \frac{N(\bar{y} - \mu)^2}{2\sigma^2} \\ - \frac{n(\bar{y}_2 - \mu - \delta - \rho(\bar{y}_1 - \mu))^2}{2(1-\rho^2)\sigma^2}, \quad (2.52)$$

where k is a constant that does not depend on (μ, δ) and

$$\bar{y} = \frac{\sum_{i=1}^N y_{i1}}{N} \\ \bar{y}_1 = \frac{\sum_{i=1}^n y_{i1}}{n} \\ \bar{y}_2 = \frac{\sum_{i=1}^n y_{i2}}{n}.$$

With ρ and σ^2 assumed known, the loglikelihood (2.52) is a function of μ and δ only. To obtain the maximum likelihood estimates of μ and δ , (2.52) must be maximized with respect to μ and δ . One way of doing this is to take partial derivatives of (2.52) with respect to μ and δ , to set the resulting two equations equal to zero and to solve for μ and δ . The resulting maximum likelihood estimators of μ and δ are

$$\hat{\mu} = \bar{y}, \quad (2.53)$$

and

$$\hat{\delta} = \bar{y}_2 - \bar{y} - \rho (\bar{y}_1 - \bar{y}). \quad (2.54)$$

If all first lactation cows produce a second lactation (i.e., no selection), then $\bar{y}_1 = \bar{y}$ and the maximum likelihood estimator of δ reduces to $\bar{y}_2 - \bar{y}$, the difference between mean production in years two and one.

To study whether (2.53) and (2.54) are biased, I compute their expectations. From (2.43),

$$E(\hat{\mu}) = E(\bar{y}) = \mu. \quad (2.55)$$

Using (2.46), (2.55) and (2.47), the expected value of (2.54) is

$$\begin{aligned} E(\hat{\delta}) &= E_{y_{i1}} [E(\bar{y}_2 | y_{i1} \in S)] - E(\bar{y}) - \rho [E(\bar{y}_1 | y_{i1} \in S) - E(\bar{y})] \\ &= \mu + \delta + \rho(\mu_s - \mu) - \mu - \rho(\mu_s - \mu) \\ &= \delta. \end{aligned} \quad (2.56)$$

Therefore, the maximum likelihood estimators of μ and δ are unbiased by selection of the records, despite the fact that the selection mechanism is not accounted for in the computation of the likelihood (2.51). This holds in the present example because the data are *missing at random* in the sense defined by Rubin (1976) and therefore the selection mechanism is *ignorable*. If this is not the case, inferences based on the likelihood require incorporating the selection process as part of the model. This is illustrated in the next example.

2.8 Example: The Likelihood Function with Truncated Data

Consider data collected on n individuals with the objective of inferring the mean and variance of some trait. For some reason beyond the control of the analyst, only those individuals that exceed a known threshold become available for analysis. Can the mean and variance of the (unselected) trait be correctly inferred using these data? The example is presented in a stylised fashion, but the data structure is not uncommon for field data in animal breeding, where records of culled individuals are typically excluded.

As a reminder, let X have probability density function $p(x)$ and cumulative distribution function $F(x) = \Pr(X \leq x)$. Let a and b be constants lying within the support of the distribution of X (the support of the distribution is the set of values of x where the pdf (or pmf) is positive). Then,

$$p(x|a < X \leq b) = \frac{p(x)}{F(b) - F(a)}, \quad (2.57a)$$

$$p(x|X > a) = \frac{p(x)}{1 - F(a)}, \quad (2.57b)$$

$$p(x|X \leq b) = \frac{p(x)}{F(b)}. \quad (2.57c)$$

In each case, the unconditional density is adjusted by a scaling constant so that the conditional density still integrates to unity. In other words, the selection mechanism is taken into account.

With this detail in place, consider data vector $Y = (Y_o, Y_m)$ of length n from a normal distribution with mean μ and variance σ^2 that consists of r observed records Y_o and $n - r$ missing records Y_m . The $n - r$ missing records were discarded because they were smaller than an observed threshold T . In this particular case the missing data mechanism involves the complete data (the observed and the missing data). Therefore, the missing data mechanism must be incorporated in the likelihood for correct inferences.

The contribution to the likelihood function from each element of Y_o is

$$L_{Y_o}(\mu, \sigma^2 | y_i > T) \propto \frac{p(y_i | \mu, \sigma^2)}{\int_T^\infty p(y_i | \mu, \sigma^2) dy_i} = \frac{p(y_i | \mu, \sigma^2)}{1 - \Phi(z)}, \quad i = 1, \dots, r;$$

$$z = \frac{T - \mu}{\sigma},$$

where $p(y_i | \mu, \sigma^2)$, in this example, is the density of $N(\mu, \sigma^2)$ and $\Phi(\cdot)$ is the distribution function (or cumulative distribution function) of the standard normal.

Concerning the $n - r$ missing records, the only information available is that they are smaller than T . Therefore the contribution to the likelihood function from each missing record is

$$L_{Y_m}(\mu, \sigma^2 | y_i < T) = \Pr(y_i < T | \mu, \sigma^2) = \Phi(z), \quad i = r + 1, \dots, n.$$

Assuming the records are *iid*, after ordering, the likelihood becomes

$$\prod_{i=1}^r L_{Y_o}(\mu, \sigma^2 | y_i \geq T) \prod_{i=r+1}^n L_{Y_m}(\mu, \sigma^2 | y_i < T) \propto \frac{\prod_{i=1}^r p(y_i | \mu, \sigma^2)}{(1 - \Phi(z))^r} (\Phi(z))^{n-r}, \quad (2.58)$$

where z is a function of μ and σ . Maximisation over μ and σ^2 provides inferences of the base (untruncated) population parameters. This would not be the case if the selection mechanism had been ignored and if inferences of μ and σ^2 had instead been based on the likelihood constructed assuming $Y_o \sim N(\mu, \sigma^2)$. The resulting estimators would be biased.

Likelihood (2.58) is a non linear function of μ and σ^2 and no analytic form for the estimators exists. However solutions using the method of moments, the EM (*expectation maximisation*) algorithm or McMC are easily available as shown in the chapters ahead.

A slightly different situation arises if information is restricted to the r observed records only, known to have been selected because each was larger than T . There is no information about how many records were discarded. In this case, the likelihood is equal to (2.58) with the second term $(\Phi(z))^{n-r}$ omitted.

The Exercises section, pages 611 and 658, introduces a censored model where data are drawn from an exponential distribution.

2.9 Example: The Likelihood Function of a Genomic Model

So far no distinction has been made between the model assumed to generate the data, the *true model*, and the model used for analysis, the *operational model*. When the likelihood based on an operational model differs meaningfully from that based on the true model, the likelihood is *misspecified* and **inferences** can be affected. I underline **inferences** because in the context of prediction this is not necessarily a problem. The true model may constitute a poorer prediction machinery than an operational alternative.

I revisit (see page 42) a model that is used repeatedly in this book: the *genomic model*, that has its origins in the influential work of Meuwissen et al. (2001). Advances in molecular genetic techniques have allowed genotyping vast numbers of markers spread across the whole genome of individuals. These genetic marker loci are not necessarily causal but are correlated (they are in *linkage disequilibrium*) with the causal loci, the so-called quantitative trait loci (QTL). The latter are typically unobserved, the genetic markers are observed and can provide information about the genetics of a trait.

Genetic marker information on a large number of loci allows the assessment of kinship (additive genetic relationships) among nominally unrelated individuals (meaning distantly related). However, due to Mendelian sampling, the patterns of allele sharing at markers and at causal loci may be very different (Hill and Weir 2011). As a consequence, the variance structure specified by the operational model can differ from that of the true model, leading to misspecification. This disparity between the true model and the operational model may affect inferences but is not necessarily a problem in the context of prediction. The topic is elaborated in de los Campos et al. (2015).

With this warning in mind, I introduce the (operational) likelihood function of a genomic model. Consider, first in general terms, the following mixed model:

$$y|\alpha, g, \sigma_e^2 \sim N\left(Z\alpha + g, I\sigma_e^2\right), \quad (2.59a)$$

$$g|G, \sigma_g^2 \sim N\left(0, G\sigma_g^2\right). \quad (2.59b)$$

The first line specifies that the conditional distribution of the data y (vector of order $n \times 1$), given all the parameters, is normal with conditional mean $Z\alpha + g$ and conditional variance $I\sigma_e^2$. The scalar σ_e^2 is the residual variance, Z is an observed matrix and α is an unobserved vector of systematic effects. The second line describes the model for the random variable g given parameters. This is again normal with mean 0 and variance $G\sigma_g^2$. The system described by (2.59) is an example of a hierarchical model. Vector g could represent genomic or additive genetic values, G could represent a genomic or additive genetic relationship matrix and σ_g^2 , a scalar, can be a genomic or an additive genetic variance component. When g is a vector of additive genetic values, taken to be equal to the sum of the contributions from a very large number of unobserved loci, the model is known as the *infinitesimal model* (Bulmer 1980). This has been the standard model for analysis of many quantitative traits in the pre-genomics era. Matrix G specifies the conditional expected value of allele sharing between individuals, given an observed pedigree, also known as the additive genetic relationship matrix.

In the genomic model, where g is a vector of genomic values, it takes the form $g = Wb$, where b is a vector of p unobserved genetic marker effects, often assigned the normal distribution

$$b|\sigma_b^2 \sim N\left(0, I\sigma_b^2\right).$$

The scalar parameter σ_b^2 can be interpreted as the prior variance associated with each element of the $p \times 1$ vector b . The observed matrix W , of order $n \times p$, consists of marker labels X_{ij} (often column-centred and scaled, in which case the rank of W is at most $n - 1$ if $p > n$)

$$W_{ij} = \frac{X_{ij} - E(X_{ij})}{\sqrt{\text{Var}(X_{ij})}}.$$

(In practice, the expectation and the variance are replaced by the respective column mean and variance.) When the rank of W is smaller than n , g in (2.59b) has a singular normal distribution. The random variable X_{ij} can take values 0, 1, 2 according to the number of the arbitrarily chosen allele of marker locus j of individual i . Therefore $E(W_{ij}) = 0$, $\text{Var}(W_{ij}) = 1$. Matrix W is a random variable that takes a particular realised value in the data at hand. Inferences based on genomic

models are typical conditional on W . Parametrising in terms of g rather than in terms of b is useful when $p > n$.

The vector g of genomic values is a proxy for the true additive genetic values determined by the causal QTL. The conditional variance of g given W is

$$\begin{aligned}\text{Var}(g|W) &= WW'\sigma_b^2 \\ &= \frac{1}{m}WW'\sigma_g^2,\end{aligned}$$

where the genomic relationship matrix $G_g = (1/m)WW'$ is the average (over marker loci) realised observed matrix of genetic relationships among the n individuals, and $\sigma_g^2 = m\sigma_b^2$ is the unconditional (with respect to W) variance of an element of g , interpreted as the amount of additive genetic variance captured by marker loci, or *genomic variance* at the level of the operational model. The equality $\sigma_g^2 = m\sigma_b^2$ stems from the fact that

$$\begin{aligned}\text{Var}(g_i) &= \text{E}[\text{Var}(g_i|W_i)] + \text{Var}[\text{E}(g_i|W_i)] \\ &= \text{E}[\text{Var}(g_i|W_i)]\end{aligned}$$

because $\text{E}(g_i|W_i) = 0$. Labelling W'_i as the i th row of matrix W , the i th diagonal term WW' is $W'_iW_i = \sum_{j=1}^p W_{ij}^2$. Then

$$\text{Var}(g_i) = \text{E}\left[\sum_{j=1}^p W_{ij}^2\right] = m\sigma_b^2 = \sigma_g^2$$

because $\text{E}(W_{ij}^2) = 1$. A *genomic heritability* or proportion of variance accounted for by the genetic markers is defined as

$$h_g^2 = \frac{\sigma_g^2}{\sigma_g^2 + \sigma_e^2}. \quad (2.60)$$

The construction of the likelihood involves integration over g :

$$L(\alpha, \sigma_g^2, \sigma_e^2 | y, G_g) = \int_g p(y | \alpha, g, \sigma_e^2) p(g | G_g, \sigma_g^2) dg, \quad (2.61)$$

which in view of the normality assumptions is the likelihood of the normal distribution

$$N(Z\alpha, G_g\sigma_g^2 + I\sigma_e^2).$$

In a classical likelihood setup, the g 's are not “parameters” but latent variables that do not feature in the likelihood. The only parameters are α , σ_g^2 and σ_e^2 .

In this example the likelihood is explicit but the maximum likelihood estimators do not have a closed form. Numerical methods are needed to obtain solutions. This is the subject of the next chapter. In most non-Gaussian setups the integration in (2.61) cannot be written in closed form and therefore neither the likelihood function nor the maximum likelihood estimators can be obtained explicitly.

Chapter 3

Computing the Likelihood



Estimation using the likelihood function proceeds by solving for θ the equation $S(\theta) = 0$ where $S(\theta)$ is the score. In many cases there may be no explicit solution, either because the system of equations is not linear, or because the likelihood cannot be written explicitly. Therefore, numerical methods must be employed to obtain the maximum likelihood estimates. There is a large number of optimisation algorithms for nonlinear problems (see for example, Dahlquist and Björck 1974). The chapter provides an outline of two classical approaches that are used for fitting models using maximum likelihood and a third one that is often used in high dimensional settings such as neural networks. The first two are Newton-Raphson and the EM algorithm and the third is gradient descent. Examples illustrate implementation of the methods.

3.1 Newton-Raphson and the Method of Scoring

Newton-Raphson is a general procedure to solve $g(x) = 0$. Newton-Raphson finds the root of a function and therefore it is applied to the derivative of the loglikelihood. To illustrate assume that x is a scalar. The starting point is a Taylor expansion around an initial estimate of x labelled x_0

$$g(x) \approx g(x_0) + g'(x_0)(x - x_0) = 0, \quad (3.1)$$

where

$$g'(x_0) = \left. \frac{dg(x)}{dx} \right|_{x=x_0}.$$

From (3.1),

$$x = x_0 - \frac{g(x_0)}{g'(x_0)}$$

and the iterative process is

$$x_{t+1} = x_t - \frac{g(x_t)}{g'(x_t)}. \quad (3.2)$$

ML estimation involves finding the value of θ that satisfies $S(\theta) = 0$, where θ is a vector of p parameters and $S(\theta)$ is the score function, the first derivative of the loglikelihood with respect to the parameters. The vectorial expression equivalent to (3.1) is

$$S(\theta) = S(\theta_0) + S'(\theta_0)(\theta - \theta_0) \quad (3.3)$$

where

$$S'(\theta_0) = \left. \frac{\partial^2 \ell(\theta|y)}{\partial \theta \partial \theta'} \right|_{\theta=\theta_0}.$$

The component ij of the $p \times p$ matrix of second derivatives $S'(\theta)$ (the *Hessian*) is

$$\frac{\partial^2 \ell(\theta|y)}{\partial \theta_i \partial \theta_j}$$

and $-S'(\theta_0) = I(\theta_0|y)$ is the *observed information*. After setting (3.3) equal to zero and solving for θ , the iterative process is

$$\theta_{t+1} = \theta_t - [S'(\theta_t)]^{-1} S(\theta_t). \quad (3.4)$$

The *method of scoring* replaces the inverse of the observed information in (3.4) by the *expected information* matrix and the iterative process is now

$$\theta_{t+1} = \theta_t - [E_y(S'(\theta_t))]^{-1} S(\theta_t). \quad (3.5)$$

The expected information is

$$E_y(S'(\theta_t)) = - \left[E_y \left(\frac{\partial^2}{\partial \theta \partial \theta'} \ell(\theta|y) \right) \right] \Big|_{\theta=\theta_t} = - \int \frac{\partial^2 \ell(\theta|y)}{\partial \theta \partial \theta'} p(y|\theta) dy.$$

Typically, the method of scoring requires fewer calculations in each iteration because many expressions vanish or simplify in the process of taking expectations.

However, it may converge at a slower rate. The two methods may not converge at all and, even if they do, there is no assurance that a global maximum would be reached. This is not surprising, as the methods search for stationarity without reference to the possible existence of multiple maxima. This is a potential problem in models having many parameters and it is expected to occur more frequently when sample sizes are small, as the likelihood may have several “peaks and valleys”. Both methods involve calculating the matrix of second derivatives of the loglikelihood.

A third variant (and there are others) of this iterative procedure avoids the use of second derivatives and is implemented with first derivatives only as

$$\theta_{t+1} = \theta_t - \left[(S(\theta) S(\theta)') \right]^{-1} S(\theta_t). \quad (3.6)$$

This is known as the BHHH algorithm and was proposed by Berndt, Hall, Hall, and Hausman (Berndt et al. 1974). The performance of this method can be more erratic than the previous two.

All these methods are based on a linear approximation to the score function that is equivalent to a quadratic approximation to the likelihood.

Example: Estimation of Gene Frequencies from ABO Blood Group Phenotypes

The ABO blood group antigens in man are encoded by one genetic locus on chromosome 9, the ABO locus that has three allelic forms: *A*, *B* and *O*. A child receives one of the three alleles from each parent, giving rise to six possible genotypes but only four phenotypic classes (blood types) can be observed.

Consider the blood group data in Table 3.1. The expected frequency of each genotype in the last column is derived assuming Hardy–Weinberg equilibrium. The problem at hand is to infer p_A , p_B and p_O , the frequency of alleles *A*, *B* and *O*, respectively, subject to the constraint $p_A + p_B + p_O = 1$. The observed data are (n_A, n_{AB}, n_B, n_O) .

Before writing the likelihood corresponding to this problem, imagine that all 6 phenotypes were observed (one for each genotype). This would lead to the classical multinomial likelihood. To illustrate, assume that the complete data were instead

Table 3.1 Frequency of genotypes and phenotypes of ABO blood group data

Genotype	Phenotype	Observed counts	Frequency
<i>AA</i>	<i>A</i>	n_A	p_A^2
<i>AO</i>	<i>A</i>		$2p_A p_O$
<i>AB</i>	<i>AB</i>	n_{AB}	$2p_A p_B$
<i>BB</i>	<i>B</i>	n_B	p_B^2
<i>BO</i>	<i>B</i>		$2p_B p_O$
<i>OO</i>	<i>O</i>	n_O	p_O^2

$(n_{AA}, n_{AO}, n_{AB}, n_{BB}, n_{BO}, n_{OO})$ associated with the 6 phenotypes and the total number of observations is

$$N = n_{AA} + n_{AO} + n_{AB} + n_{BB} + n_{BO} + n_{OO}.$$

The 4 phenotypic classes (blood types) are $n_A = n_{AA} + n_{AO}$, $n_{AB} = n_{AB}$, $n_B = n_{BB} + n_{BO}$ and $n_O = n_{OO}$. Under multinomial sampling the joint probability function of the complete data is

$$\begin{aligned} & \Pr(n_{AA}, n_{AO}, n_{AB}, n_{BB}, n_{BO}, n_{OO} | p_A, p_B) \\ &= N! \frac{(p_A^2)^{n_{AA}}}{n_{AA}!} \frac{(2p_A p_O)^{n_{AO}}}{n_{AO}!} \frac{(2p_A p_B)^{n_{AB}}}{n_{AB}!} \frac{(p_B^2)^{n_{BB}}}{n_{BB}!} \frac{(2p_B p_O)^{n_{BO}}}{n_{BO}!} \frac{(p_O^2)^{n_{OO}}}{n_{OO}!}. \end{aligned}$$

The likelihood function is

$$\begin{aligned} & L(p_A, p_B | n_{AA}, n_{AO}, n_{AB}, n_{BB}, n_{BO}, n_{OO}) \\ & \propto (p_A^2)^{n_{AA}} (2p_A p_O)^{n_{AO}} (2p_A p_B)^{n_{AB}} (p_B^2)^{n_{BB}} (2p_B p_O)^{n_{BO}} (p_O^2)^{n_{OO}}, \quad (3.7) \end{aligned}$$

and the loglikelihood, up to an additive constant is

$$\begin{aligned} & l(p_A, p_B | n_{AA}, n_{AO}, n_{AB}, n_{BB}, n_{BO}, n_{OO}) \\ &= 2n_{AA} \ln(p_A) + n_{AO} \ln(2p_A p_O) + n_{AB} \ln(2p_A p_B) + 2n_{BB} \ln(p_B) \\ & \quad + n_{BO} \ln(2p_B p_O) + 2n_{OO} \ln(p_O). \quad (3.8) \end{aligned}$$

Substituting $p_O = 1 - p_A - p_B$, taking partial derivatives with respect to p_A and p_B and setting these equal to zero leads to the equations

$$\begin{aligned} & \frac{2n_{AA} + n_{AB} + n_{AO}}{p_A} - \frac{2n_{OO} + n_{AO} + n_{BO}}{1 - p_A - p_B} = 0, \\ & \frac{2n_{BB} + n_{AB} + n_{BO}}{p_B} - \frac{2n_{OO} + n_{AO} + n_{BO}}{1 - p_A - p_B} = 0. \end{aligned}$$

The unique solutions are the closed form ML estimators

$$\hat{p}_A = \frac{2n_{AA} + n_{AB} + n_{AO}}{2N}, \quad (3.9a)$$

$$\hat{p}_B = \frac{2n_{BB} + n_{AB} + n_{BO}}{2N}. \quad (3.9b)$$

In the case of the data in Table 3.1, n_{AO} and n_{BO} are not observed and the likelihood becomes nonlinear. Given the data in Table 3.1, the loglikelihood is:

$$l(p_A, p_B | n_A, n_{AB}, n_B, n_O) \propto n_A \ln[p_A(2 - p_A - 2p_B)] + n_{AB} \ln[2p_A p_B] \\ + n_B \ln[p_B(2 - p_B - 2p_A)] + 2n_O \ln[(1 - p_A - p_B)]. \quad (3.10)$$

Differentiating with respect to p_A and p_B yields the nonlinear system of equations

$$\frac{\partial l(p_A, p_B | n_A, n_{AB}, n_B, n_O)}{\partial p_A} = \frac{n_{AB}}{p_A} + \frac{n_A(2 - 2p_A - 2p_B)}{p_A(2 - p_A - 2p_B)} - \frac{2n_B}{2 - 2p_A - p_B} \\ - \frac{2n_O}{1 - p_A - p_B}, \quad (3.11)$$

$$\frac{\partial l(p_A, p_B | n_A, n_{AB}, n_B, n_O)}{\partial p_B} = \frac{n_{AB}}{p_B} + \frac{n_B(2 - 2p_A - 2p_B)}{p_B(2 - 2p_A - p_B)} - \frac{2n_A}{2 - p_A - 2p_B} \\ - \frac{2n_O}{1 - p_A - p_B}. \quad (3.12)$$

A solution can be obtained using Newton-Raphson. This requires the following second derivatives

$$\frac{\partial^2 l(p_A, p_B | n_A, n_{AB}, n_B, n_O)}{(\partial p_A)^2} = \frac{n_A(2 - 2p_A - 2p_B)}{p_A(2 - p_A - 2p_B)^2} - \frac{2n_A}{p_A(2 - p_A - 2p_B)} \\ - \frac{n_A(2 - 2p_A - 2p_B)}{p_A^2(2 - p_A - 2p_B)} - \frac{n_{AB}}{p_A^2} - \frac{2n_O}{(1 - p_A - p_B)^2} \\ - \frac{4n_B}{(2 - 2p_A - p_B)^2},$$

$$\frac{\partial^2 l(p_A, p_B | n_A, n_{AB}, n_B, n_O)}{(\partial p_B)^2} = \frac{n_B(2 - 2p_A - 2p_B)}{p_B(2 - 2p_A - p_B)^2} - \frac{2n_B}{p_B(2 - 2p_A - p_B)} \\ - \frac{n_B(2 - 2p_A - 2p_B)}{p_B^2(2 - 2p_A - p_B)} - \frac{n_{AB}}{p_B^2} - \frac{2n_O}{(1 - p_A - p_B)^2} \\ - \frac{4n_A}{(2 - p_A - 2p_B)^2},$$

$$\frac{\partial^2 l(p_A, p_B | n_A, n_{AB}, n_B, n_O)}{\partial p_A \partial p_B} = -\frac{2n_A}{(2 - p_A - 2p_B)^2} - \frac{2n_B}{(2 - 2p_A - p_B)^2} \\ - \frac{2n_O}{(1 - p_A - p_B)^2}.$$

Suppose the data are $n_A = 725$, $n_{AB} = 72$, $n_B = 258$, $n_O = 1073$. This example is discussed in Weir (1996) where references to the original source of the data can be found. Using these expressions in (3.4) yields, at convergence, the ML estimates: $\hat{p}_A = 0.2091$ and $\hat{p}_B = 0.0808$. The observed information matrix evaluated at the ML estimates is

$$I(\hat{p}_A, \hat{p}_B | n_A, n_{AB}, n_B, n_O) = \begin{bmatrix} 23, 210.6 & 5, 031.56 \\ 5, 031.56 & 56, 008.53 \end{bmatrix},$$

resulting in an estimate of the asymptotic covariance matrix equal to:

$$\begin{aligned} \text{Var}(\hat{p}_A, \hat{p}_B | n_A, n_{AB}, n_B, n_O) &= [I(\hat{p}_A, \hat{p}_B | n_A, n_{AB}, n_B, n_O)]^{-1} \\ &= 10^{-6} \begin{bmatrix} 43.939 & -3.947 \\ -3.947 & 18.209 \end{bmatrix}. \end{aligned}$$

The R-code for the Newton-Raphson computations, spelled out line by line is as follows:

```
# CODE0301
rm(list=ls())
set.seed(30371)
fd<-matrix(data=NA,nrow=2,ncol=1)
sd<-matrix(data=NA,nrow=2,ncol=2)
freq<-matrix(data=NA,nrow=2,ncol=1)
niter<-20
# DATA
n_A<-725
n_AB<-72
n_B<-258
n_O<-1073
# INITIALISE GENE FREQ
freq[1,1]<-0.3
freq[2,1]<-0.2
p_A<-freq[1,1]
p_B<-freq[2,1]
# ITERATION LOOP
for (i in 1:niter){
  fd[1,1] <- n_AB/p_A+n_A*(2-2*p_A-2*p_B)/(p_A*(2-p_A-2*p_B))-
    2*n_B/(2-2*p_A-p_B)-2*n_O/(1-p_A-p_B)
  fd[2,1] <- n_AB/p_B+n_B*(2-2*p_A-2*p_B)/(p_B*(2-2*p_A-p_B))-
    2*n_A/(2-p_A-2*p_B) - 2*n_O/(1-p_A-p_B)
  s11a <- -n_AB/((p_A)^2)
  s11b <- -(n_A*(2-2*p_A-2*p_B))/((p_A*(2-p_A-2*p_B)^2))
  s11c <- - (2*n_A)/((p_A*(2-p_A-2*p_B)))
  s11d <- - (n_A*(2-2*p_A-2*p_B))/((p_A^2*(2-p_A-2*p_B)))
  s11e <- - (4*n_B)/((2-2*p_A-p_B)^2)
  s11f <- - (2*n_O)/((1-p_A-p_B)^2)
  sd[1,1] <- s11a + s11b + s11c + s11d + s11e + s11f
  s22a <- - n_AB/((p_B)^2)
  s22b <- - n_B*(2-2*p_A-2*p_B)/(((2-2*p_A-p_B)^2) * p_B)
```

```

s22c <- - (2*n_B)/(p_B*(2-2*p_A-p_B))
s22d <- - (n_B*(2-2*p_A-2*p_B))/((p_B^2*(2-2*p_A-p_B)))
s22e <- -(4*n_A)/((2-p_A-2*p_B)^2)
s22f <- -(2*n_0)/((1-p_A-p_B)^2)
sd[2,2] <- s22a + s22b + s22c + s22d + s22e + s22f
sd[1,2] <- -2*n_A/((2-p_A-2*p_B)^2) - 2*n_0/((1-p_A-p_B)^2) -
  2*n_B/((2-2*p_A-p_B)^2)
sd[2,1] <- sd[1,2]
freq<-freq-solve(sd)%*%fd
p_A<-freq[1,1]
p_B<-freq[2,1]
}
# ML ESTIMATES ARE
freq

```

```

##           [,1]
## [1,] 0.20913065
## [2,] 0.08080101

```

```

# OBSERVED INFORMATION IS
-sd

```

```

##           [,1]      [,2]
## [1,] 23210.614 5031.559
## [2,] 5031.559 56008.529

```

```

# ASYMPTOTIC VAR-COVAR MATRIX IS
-solve(sd)

```

```

##           [,1]      [,2]
## [1,] 4.393943e-05 -3.947325e-06
## [2,] -3.947325e-06 1.820903e-05

```

A computationally simpler alternative is to use the R function `OPTIM` to solve minimisation problems (to maximise a function such as the loglikelihood (3.10) the function multiplied by -1 must be supplied). The code is as follows:

```

# CODE0302
rm(list=ls()) # CLEAR WORKSPACE
fr<-function(par){
  p_A <- par[1]
  p_B <- par[2]
  -(725*log(p_A*(2 - p_A - 2* p_B)) + 72*log(2*p_A*p_B) +
    258*log(p_B*(2 - p_B - 2*p_A)) + 2*1073*log(1 - p_A - p_B))}
result <- optim(par=c(0.3,0.2),fr,hessian=TRUE)
result$par

```

```

## [1] 0.20913767 0.08081225

```

```
solve(result$hessian)
```

```
##           [,1]           [,2]
## [1,]  4.393974e-05 -3.947440e-06
## [2,] -3.947440e-06  1.820851e-05
```

```
-result$value
```

```
## [1] -2303.55
```

The value of the loglikelihood at convergence can be extracted from `OPTIM` and is equal to -2303.55 .

Example: A Regression Model for Binary Data

Categorical data are ubiquitous in genetics and arise when the outcome is an assignment into one of several mutually exclusive classes. A distinction is made based on whether the classes are unordered or ordered. The ABO blood groups illustrate the former. Another example is hair colour. In these cases, there is no clear ordering from lowest to highest. In contrast, the degree of severity of a disease can be meaningfully classified into several categories, ranging from very low severity to very high severity.

Only two categories are assumed here, so the response is said to be binary. If y_i denotes the response, without loss of generality, the two possible values may be coded as 1 and 0. Therefore,

$$E(y_i) = 1 \Pr(y_i = 1) + 0 \Pr(y_i = 0) = \Pr(y_i = 1). \quad (3.13)$$

This section describes methods to study how covariates or explanatory variables influence $\Pr(y_i = 1)$. Later sections will consider joint inferences of explanatory variables and of components of variance of random effects as factors related to $\Pr(y_i = 1)$. The variance of the random effects in such a mixed model could, for example, inform on the existence of genetic variation underlying disease data.

Assume that binary records are available on $i = 1, 2, \dots, N$ individuals $(y_1, x_1), (y_2, x_2), \dots, (y_N, x_N)$, where $y_i = 0, 1$, and x_i is a vector with p elements representing covariates. The objective is to construct a model to study the influence of the covariates on the observations y . A possible choice is the regression model

$$y_i = x_i' \beta + e_i \quad (3.14)$$

and to estimate the $p \times 1$ vector of regression coefficients β using least squares. The problem with this approach is as follows. Taking expected values under the linear model (3.14)

$$E(y_i | x_i) = x_i' \beta.$$

From (3.13) this expectation is equal to $\Pr(y_i = 1 | x_i)$. This probability must satisfy

$$0 \leq \Pr(y_i = 1 | x_i) \leq 1 \quad (3.15)$$

but the prediction using an estimate of $x_i' \beta$ may yield outcomes outside the constraint (3.15).

A way of building a model in which the constraint (3.15) is automatically satisfied is to use a transformation of the output y , generating a nonlinear relationship between it and the covariate. This is achieved by defining the probability that $y_i = 1$ as a nonlinear function of x_i of the form

$$\Pr(y_i = 1 | x_i) = F(x_i' \beta), \quad (3.16)$$

where F is any distribution function (cumulative distribution function). Two common choices for F are the normal distribution, leading to the probit model, and the logistic distribution, leading to the logistic model.

For the probit model, $F = \Phi$ and

$$\Pr(y_i = 1 | x_i) = \Phi(x_i' \beta), \quad (3.17)$$

where Φ is the standard normal integral

$$\Phi(t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^t \exp\left(-\frac{1}{2}u^2\right) du.$$

The relationship (3.17) is linearised by the inverse normal transformation

$$\Phi^{-1}(\Pr(y_i = 1 | x_i)) = x_i' \beta. \quad (3.18)$$

For the logistic model

$$\begin{aligned} F(t) &= \int_{-\infty}^t p(u) du = \int_{-\infty}^t \frac{\exp(u)}{[1 + \exp(u)]^2} du \\ &= \frac{\exp(t)}{1 + \exp(t)}, \end{aligned} \quad (3.19)$$

where $p(u)$ is the pdf of the standard logistic distribution, with $E(u) = 0$ and $\text{Var}(u) = \pi^2/3$. Therefore, for the logistic model

$$\Pr(y_i = 1|x_i) = \frac{\exp(x'_i\beta)}{1 + \exp(x'_i\beta)}, \quad (3.20a)$$

$$\Pr(y_i = 0|x_i) = \frac{1}{1 + \exp(x'_i\beta)}. \quad (3.20b)$$

The relationship is linearised by the logit transformation

$$\ln \left[\frac{\Pr(y_i = 1|x_i)}{\Pr(y_i = 0|x_i)} \right] = x'_i\beta. \quad (3.21)$$

The Liability Model

Ordered categorical data and binary data are often analysed using a threshold model, first used by Wright (1934) to study the number of digits in guinea pigs and introduced in human genetics in a more modern version by Falconer (1965). This model assumes that there is an underlying or latent unobservable variable, u , in genetics of disease often called the liability after Falconer (1965). The categories of response result from the value of u relative to a fixed unobserved threshold t . The liability u is often assumed to follow a normal distribution or a logistic distribution. The objective is to show that in the first case, the liability model is equivalent to the probit model (3.17) and in the second to the logistic model (3.20).

Let the dichotomy be “survival” versus “death”, say. If $u > t$ then the individual survives and the binary variable (the observed datum) takes the value $y = 1$. If $u \leq t$ the individual dies and the observed datum is $y = 0$. Denote the liability associated with datum i as u_i and suppose that it is related to an unknown parameter vector β of order $p \times 1$ via the linear structure

$$u_i = x'_i\beta + e_i, \quad i = 1, 2, \dots, N, \quad (3.22)$$

where x'_i is the i th row of a known $N \times p$ matrix x of explanatory variables (covariates) and e_i is a random residual with pdf $p(e_i)$. Assume that the residuals are independent and identically distributed. The probability of survival of individual i (which is the pmf of the random variable y_i) is

$$\begin{aligned} \Pr(y_i = 1|\beta, x) &= \Pr(u_i > t|\beta, x) = \Pr(u_i - x'_i\beta > t - x'_i\beta|\beta, x) \\ &= \Pr(e_i > t - x'_i\beta|\beta, x) = \int_{t-x'_i\beta}^{\infty} p(e_i) de_i \\ &= \int_{-\infty}^{-(t-x'_i\beta)} p(e_i) de_i = \int_{-\infty}^{x'_i\beta-t} p(e_i) de_i. \end{aligned} \quad (3.23)$$

The equality in the third line requires e_i to be symmetrically distributed around 0. The liabilities cannot be observed and a convenient origin is to set the value of the threshold t equal to 0. Hence the scale is one of deviations from the threshold. This constraint makes the likelihood model identifiable and the Hessian becomes negative definite. Then,

$$\Pr(y_i = 1|\beta, x) = \int_{-\infty}^{x'_i\beta} p(e_i) de_i = F(x'_i\beta), \quad (3.24)$$

as in (3.16), where F is the cdf of the random variable e_i , indicating the equivalence between the liability model and the original formulation of the model (3.16).

The use of an underlying liability is a computational trick. However, the liability can have an intrinsic mechanistic significance. For example in the case of twinning in cattle or humans, the liability can be thought of as levels of hormones, which play a central role in factors determining monozygotic twinning. The liability formulation has also advantages in Gibbs sampling computations and provides a simple framework for extensions of the model to include hierarchical structures and for analysis of ordered categorical traits.

A Digression on Parameter Interpretation

The liability u_i changes with x_i at a constant rate, but this is not so at the level of the probabilities. This is verified by noting that

$$\frac{\partial u_i}{\partial x_i} = \beta,$$

whereas from expression (3.24)

$$\begin{aligned} \frac{\partial \Pr(y = 1|\beta, x)}{\partial x_i} &= \frac{\partial}{\partial x_i} \left[\int_{-\infty}^{x'_i\beta} p(e_i) de_i \right] \\ &= \frac{\partial}{\partial x'_i\beta} \left[\int_{-\infty}^{x'_i\beta} p(e_i) de_i \right] \frac{\partial x'_i\beta}{\partial x_i} \\ &= F(x'_i\beta) \beta. \end{aligned} \quad (3.25)$$

The change is not constant and depends on the value of the explanatory vector x_i .

In the Bernoulli distribution, $E(y|\beta, x) = \Pr(y = 1|\beta, x)$. The model at the level of the expectation of y is nonlinear, in contrast with the standard linear regression model, where $E(y|\beta, x) = \mu + \beta x$. According to (3.25) a unit change in the covariate x leads to a nonlinear change in the probability $E(y|\beta, x) = \Pr(y = 1|\beta, x)$.

To get a little insight into the meaning of β , consider the *odds ratio* of the logistic model

$$\frac{\Pr(y = 1|\beta, x)}{\Pr(y = 0|\beta, x)} = \exp(x'\beta) \quad (3.26)$$

or the *logit* or *logodds*

$$\ln \left[\frac{\Pr(y = 1|\beta, x)}{\Pr(y = 0|\beta, x)} \right] = x'\beta, \quad (3.27)$$

showing that β describes the linear change per unit change of x at the level of the logit. To be specific, imagine that y denotes disease or absence of disease and (x_1, x_2) denotes level of exposure to two conditions. Let $x_i'\beta = \mu + \beta_1 x_{1i} + \beta_2 x_{2i}$. Then in terms of (3.26),

$$\frac{\Pr(y_i = 1|\beta, x)}{\Pr(y_i = 0|\beta, x)} = \exp[(\mu + \beta_1 x_{1i} + \beta_2 x_{2i})]. \quad (3.28)$$

The parameter $\exp(\mu)$ is the odds of disease for an unexposed individual ($x = 0$).

How does the *odds ratio* change when one of the conditions, x_2 say, changes by one unit?

$$\begin{aligned} \frac{\Pr(y_i = 1|\beta, x_i)}{\Pr(y_i = 0|\beta, x_i)} &= \exp[(\mu + \beta_1 x_{1i} + \beta_2 (x_{2i} + 1))] \\ &= \exp[(\mu + \beta_1 x_{1i} + \beta_2 x_{2i})] \exp(\beta_2). \end{aligned}$$

The increase in x_2 by one unit to $x_2 + 1$ while keeping x_1 fixed, multiplies the odds ratio by $\exp(\beta_2)$. In terms of the logit we have

$$\ln \left[\frac{\Pr(y_i = 1|\beta, x_i)}{\Pr(y_i = 0|\beta, x_i)} \right] = (\mu + \beta_1 x_{1i} + \beta_2 x_{2i}) + \beta_2,$$

indicating that increasing x_2 by one unit to $x_2 + 1$ while keeping x_1 constant, increases the logit by β_2 .

Likelihood Function

Assuming the logistic model, the pmf of a datum using (3.20) is

$$\Pr(Y_i = y_i | x_i) = \left[\frac{\exp(x_i'\beta)}{1 + \exp(x_i'\beta)} \right]^{y_i} \left[\frac{1}{1 + \exp(x_i'\beta)} \right]^{1-y_i}, \quad y_i = 1, 0.$$

For N independent binary observations collected in the vector y and the covariates in matrix x , the pmf is

$$\Pr(Y = y|x) = \prod_{i=1}^N \left[\frac{\exp(x'_i \beta)}{1 + \exp(x'_i \beta)} \right]^{y_i} \left[\frac{1}{1 + \exp(x'_i \beta)} \right]^{1-y_i}. \quad (3.29)$$

This is the likelihood function when viewed as a function of β . The loglikelihood is obtained taking natural logarithms

$$l(\beta|y, x) = \sum_{i=1}^N \{y_i x'_i \beta - \ln[1 + \exp(x'_i \beta)]\}. \quad (3.30)$$

The Iterative System

To obtain ML estimators using Newton-Raphson first and second derivatives of the loglikelihood are needed. The score vector is:

$$\begin{aligned} l'(\beta|y) &= \sum_{i=1}^N \frac{\partial}{\partial \beta} \{y_i x'_i \beta - \ln[1 + \exp(x'_i \beta)]\} \\ &= \sum_{i=1}^N \left\{ y_i x_i - \frac{\exp(x'_i \beta)}{1 + \exp(x'_i \beta)} x_i \right\} \\ &= \sum_{i=1}^N [y_i - \pi(x'_i \beta)] x_i, \end{aligned} \quad (3.31)$$

where

$$\Pr(y_i = 1|x_i) = \frac{\exp(x'_i \beta)}{1 + \exp(x'_i \beta)} = \pi(x'_i \beta).$$

Let the $N \times 1$ vector of probabilities of survival for the N individuals be

$$\pi(X\beta) = [\pi(x'_1 \beta), \dots, \pi(x'_N \beta)]'$$

and observe that the score vector can be written

$$\begin{aligned} \sum_{i=1}^N [y_i - \pi(x'_i \beta)] x_i &= \{x_1 [y_1 - \pi(x'_1 \beta)] + \dots + x_N [y_N - \pi(x'_N \beta)]\} \\ &= X' [y - \pi(X\beta)]. \end{aligned}$$

The vector $y - \pi(X\beta)$ consists of deviations of the observations from their expectations. Using this representation in (3.31), it can be seen that the first-order condition for a maximum is satisfied if

$$X' \pi(X\hat{\beta}) = X'y, \quad (3.32)$$

where $\pi(X\hat{\beta})$ is the vector of probabilities of survival for the N individuals evaluated at the ML estimator $\hat{\beta}$, if this exists. The estimating Eqs.(3.32) are not explicit in $\hat{\beta}$ and must be solved iteratively. To obtain second derivatives, an additional differentiation of the loglikelihood with respect to the parameters gives

$$\begin{aligned} l''(\beta|y) &= \frac{\partial^2 l(\beta|y)}{\partial \beta \partial \beta'} = \frac{\partial}{\partial \beta'} \left\{ \sum_{i=1}^N [y_i - \pi(x'_i \beta)] x_i \right\} \\ &= - \sum_{i=1}^N x_i \frac{\partial}{\partial \beta'} \pi(x'_i \beta). \end{aligned} \quad (3.33)$$

Now,

$$\begin{aligned} \frac{\partial}{\partial \beta'} \pi(x'_i \beta) &= [1 + \exp(x'_i \beta)]^{-2} \exp(x'_i \beta) x'_i \\ &= \pi(x'_i \beta) [1 - \pi(x'_i \beta)] x'_i. \end{aligned}$$

Using this in (3.33)

$$l''(\beta|y) = - \sum_{i=1}^N x_i \pi(x'_i \beta) [1 - \pi(x'_i \beta)] x'_i = -X'D(\beta)X, \quad (3.34)$$

where $D(\beta)$ is an $N \times N$ diagonal matrix with the i th diagonal element $\pi(x'_i \beta) [1 - \pi(x'_i \beta)]$. Because the second derivatives do not depend on the observations, the expected information is equal to the observed information in this case. Hence, the Newton-Raphson and the scoring algorithms are identical. From (3.4), multiplying by $[X'D(\beta^{[l]})X]$, the iteration can be represented as

$$[X'D(\beta^{[l]})X] \beta^{[l+1]} = [X'D(\beta^{[l]})X] \beta^{[l]} + X'v(\beta^{[l]}), \quad (3.35)$$

where the vector $v(\beta^{[l]}) = y - \pi(X\beta^{[l]})$. Now let

$$y^*(\beta^{[l]}) = X\beta^{[l]} + D^{-1}(\beta^{[l]})v(\beta^{[l]})$$

be a pseudo-data vector evaluated at iteration $[t]$. Then the system (3.35) can be written as

$$\left[X' D \left(\beta^{[t]} \right) X \right] \beta^{[t+1]} = X' D \left(\beta^{[t]} \right) y^* \left(\beta^{[t]} \right). \quad (3.36)$$

This is an iterative reweighted least squares system with the matrix of weights equal to $D \left(\beta^{[t]} \right)$, with i th diagonal element

$$\pi \left(x_i' \beta^{[t]} \right) \left[1 - \pi \left(x_i' \beta^{[t]} \right) \right].$$

The Newton–Raphson algorithm is iterated until the change in successive rounds is negligible. If convergence is to a global maximum, $\hat{\beta}$ is the ML estimate. The asymptotic variance covariance matrix is estimated as

$$\widehat{\text{Var}} \left(\hat{\beta} \right) = \left[X' D \left(\hat{\beta} \right) X \right]^{-1}. \quad (3.37)$$

The variance of the maximum likelihood estimator is larger at extreme probabilities of survival.

Example: A Genomic Model

The genomic model was introduced on page 42 and discussed briefly on page 78. In the present likelihood setting, one may be interested in estimating the proportion of trait variance explained by marker information. This would give a first indication that genetic factors are influencing the trait. The justification for this conjecture is that marker genotypes are correlated (in LD) with unobserved causal genotypes and/or that causal genotypes are part of the marker panel.

In this example the genomic model is implemented with Newton-Raphson using a decomposition of the genomic relationship matrix that simplifies computations. This decomposition is used repeatedly in the book.

For simplicity, here it is assumed that the data are centred and have zero mean. The genomic model is

$$y|g, \sigma_e^2 \sim N \left(g, I \sigma_e^2 \right), \quad (3.38a)$$

$$g|W, \sigma_g^2 \sim SN \left(0, G \sigma_g^2 \right), \quad (3.38b)$$

$$G = \frac{1}{m} W W', \quad (3.38c)$$

$$W = \{ W_{ij} \}. \quad (3.38d)$$

In these expressions, g is the vector of genomic values, σ_g^2 is the genomic variance at the level of this operational model, SN is a shorthand for singular normal, m is the number of genetic markers and W_{ij} is a label for the j th marker in individual i , ($i = 1, \dots, n$; $j = 1, \dots, m$; $m > n$)

$$W_{ij} = \frac{X_{ij} - E(X_{ij})}{SD(X_{ij})}, \quad X_{ij} = 0, 1, 2,$$

where SD stands for standard deviation. In practice, $E(X_{ij})$ and $SD(X_{ij})$ are replaced by the j th column mean and the j th column standard deviation of W . Due to the column centring, when $m > n$, matrix W has rank $(n - 1)$, matrix G is singular and $(g|W, \sigma_g^2)$ is singular normally distributed.

Background

The following results are useful for deriving the likelihood for the genomic model. The eigenvalue decomposition of WW' is

$$\begin{aligned} WW' &= U \Delta U' \\ &= \sum_{i=1}^n \lambda_i U_i U_i', \end{aligned}$$

where $U = [U_1, U_2, \dots, U_n]$, of order $n \times n$ is the matrix of eigenvectors of WW' , U_j is the j th column (dimension $n \times 1$) and Δ is a diagonal matrix with elements equal to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ associated to the n eigenvectors. Since WW' is non-negative definite the eigenvalues are $\lambda_i \geq 0$, $i = 1, 2, \dots, n$. The eigenvectors satisfy $U'U = UU' = I$.

In general I work with the genomic relationship matrix G , defined as

$$\begin{aligned} G &= \frac{1}{m} WW' \\ &= \frac{1}{m} U \Delta U' \\ &= U D U' \end{aligned}$$

where

$$D = \frac{1}{m} \Delta.$$

The following result on determinants will be used.

$$\begin{aligned}
 |UDU'k + I| &= |U(Dk + I)U'| \\
 &= |U||Dk + I||U'| \\
 &= |Dk + I| \\
 &= \prod_{i=1}^n (\lambda_i k + 1),
 \end{aligned}$$

where $k = \frac{\sigma_g^2}{\sigma_e^2}$ and $|U| = \pm 1$, a property of orthogonal matrices ($UU' = I$ implies $|U|^2 = 1$).

Due to column centring of W , when $m > n$, the rank of G is typically $(n - 1)$ and the singular normal density is

$$p(g|W, \sigma_g^2) = \frac{1}{(2\pi)^{\frac{n-1}{2}} (\lambda_1 \sigma_g^2 \dots \lambda_{n-1} \sigma_g^2)^{\frac{1}{2}}} \exp\left(-\frac{g'G^-g}{2\sigma_g^2}\right) \quad (3.39)$$

where the λ' s are the $(n - 1)$ non-zero eigenvalues of G and G^- is any generalised inverse of G (Mardia et al. 1979). One choice of generalised inverse of G is

$$G^- = UD^-U', \quad (3.40)$$

where

$$D^- = \begin{bmatrix} \frac{1}{\lambda_1} & 0 & \dots & 0 \\ 0 & \ddots & \dots & 0 \\ \vdots & \vdots & \frac{1}{\lambda_{n-1}} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} D_1^{-1} & 0_c \\ 0'_c & 0 \end{bmatrix}. \quad (3.41)$$

Above, $D_1 = \text{diag}(\lambda_i)_{i=1}^{n-1}$, a diagonal matrix of dimension $(n - 1 \times n - 1)$ that contains non-zero eigenvalues, after removing the last row and column of D , 0_c is a column vector of zeroes of size $(n - 1 \times 1)$ and the element in the last row and column of D is the scalar 0.

A Probabilistically Equivalent Reparameterisation

Define a random variable α with distribution

$$\alpha|U, \sigma_g^2 \sim SN(0, D\sigma_g^2). \quad (3.42)$$

Since the last diagonal element of D is 0, the last element of vector α is equal to zero (with probability 1) and therefore α has the singular normal distribution with density

$$\begin{aligned} & \left(\lambda_1 \sigma_g^2 \dots \lambda_{n-1} \sigma_g^2 \right)^{-\frac{1}{2}} \exp \left[-\frac{1}{2\sigma_g^2} \alpha' D^- \alpha \right] \\ & \propto \left(\sigma_g^2 \right)^{-\left(\frac{n-1}{2}\right)} \exp \left[-\frac{1}{2\sigma_g^2} [(\alpha_1, \alpha_0)' D^- (\alpha_1, \alpha_0)] \right], \end{aligned} \quad (3.43)$$

In this expression $\alpha' = (\alpha_1', \alpha_0')$, where α_1 is a column vector with $n - 1$ elements and the scalar $\alpha_0 = 0$ with probability 1. It is simpler to work with vector α_1 that has probability density function

$$p(\alpha_1 | U, \sigma_g^2) \propto \left(\sigma_g^2 \right)^{-\left(\frac{n-1}{2}\right)} \exp \left[-\frac{1}{2\sigma_g^2} \alpha_1' D_1^{-1} \alpha_1 \right]. \quad (3.44)$$

This is the density of the $n - 1$ dimensional multivariate normal distribution $N(0, D_1 \sigma_g^2)$. Notice that $\alpha' D^- \alpha = \alpha_1' D_1^{-1} \alpha_1$.

Writing the Likelihood

The vector of genomic values $g = U\alpha$ and the model defined in (3.38) can be written as

$$y | \alpha, \sigma_e^2 \sim N(U\alpha, I\sigma_e^2), \quad (3.45a)$$

$$\alpha | U, \sigma_g^2 \sim SN(0, D\sigma_g^2), \quad (3.45b)$$

$$G = UDU', \quad (3.45c)$$

$$y | \sigma_g^2, \sigma_e^2 \sim N(0, UDU'\sigma_g^2 + I\sigma_e^2). \quad (3.45d)$$

Then the likelihood takes the form

$$p(\sigma_g^2, \sigma_e^2 | y, W) \propto |UDU'\sigma_g^2 + I\sigma_e^2|^{-\frac{1}{2}} \exp \left(-\frac{1}{2} y' (UDU'\sigma_g^2 + I\sigma_e^2)^{-1} y \right). \quad (3.46)$$

Write

$$U D U' \sigma_g^2 + I \sigma_e^2 = \sigma_e^2 (U D U' k + I), \quad k = \frac{\sigma_g^2}{\sigma_e^2}.$$

Then

$$\begin{aligned} |U D U' \sigma_g^2 + I \sigma_e^2| &= |\sigma_e^2 (U D U' k + I)| \\ &= |\sigma_e^2 U (Dk + I) U'| \\ &= (\sigma_e^2)^n |U (Dk + I) U'| \\ &= (\sigma_e^2)^n |Dk + I| \\ &= (\sigma_e^2)^n \prod_{i=1}^n (\lambda_i k + 1). \end{aligned}$$

Also,

$$(U D U' \sigma_g^2 + I \sigma_e^2)^{-1} = \frac{1}{\sigma_e^2} U (Dk + I)^{-1} U',$$

using $U^{-1} = U'$. The likelihood is

$$\begin{aligned} p(\sigma_g^2, \sigma_e^2 | y, W) &\propto (\sigma_e^2)^{-\frac{n}{2}} \prod_{i=1}^n (\lambda_i k + 1) \exp \left[-\frac{1}{2\sigma_e^2} y' U (Dk + I)^{-1} U' y \right] \\ &= (\sigma_e^2)^{-\frac{n}{2}} \prod_{i=1}^n (\lambda_i k + 1)^{-\frac{1}{2}} \exp \left[-\frac{1}{2\sigma_e^2} \tilde{y}' (Dk + I)^{-1} \tilde{y} \right] \\ &= (\sigma_e^2)^{-\frac{n}{2}} \prod_{i=1}^n (\lambda_i k + 1)^{-\frac{1}{2}} \exp \left[-\frac{1}{2\sigma_e^2} \sum_{i=1}^n \frac{\tilde{y}_i^2}{\lambda_i k + 1} \right], \quad (3.47) \end{aligned}$$

where $\tilde{y} = U'y$, a column vector of length n whose i th element is $\tilde{y}_i$.

The loglikelihood, up to an additive constant, is

$$\ln p(k, \sigma_e^2 | y, W) = -\frac{1}{2} \left\{ n \ln \sigma_e^2 + \sum_{i=1}^n \ln (\lambda_i k + 1) + \frac{1}{\sigma_e^2} \sum_{i=1}^n \frac{\tilde{y}_i^2}{\lambda_i k + 1} \right\}. \quad (3.48)$$

It can be informative to plot contours of (3.48) as a function of the two variance components.

Note: it may be numerically useful to parametrise in terms of the one-to-one transformation

$$\begin{aligned} v_e &= \ln \sigma_e^2, \\ v_g &= \ln \sigma_g^2, \end{aligned}$$

with inverse function

$$\begin{aligned} \sigma_e^2 &= \exp(v_e), \\ \sigma_g^2 &= \exp(v_g), \\ k &= \frac{\sigma_g^2}{\sigma_e^2} = \exp(v_g - v_e). \end{aligned}$$

Then the loglikelihood (3.48) takes the form

$$\begin{aligned} \ln p(v_g, v_e | y, W) = \\ -\frac{1}{2} \left\{ n v_e + \sum_{i=1}^n \ln(\lambda_i \exp(v_g - v_e) + 1) + \frac{1}{\exp(v_e)} \sum_{i=1}^n \frac{\tilde{y}_i^2}{\lambda_i \exp(v_g - v_e) + 1} \right\}. \end{aligned} \quad (3.49)$$

This avoids negative values of the variance components under unconstrained maximisation.

Implementation Using Newton-Raphson

To fit the model using Newton-Raphson first and second derivatives are needed. It is easier to work with (3.48).

First Derivatives

$$\frac{\partial}{\partial \sigma_e^2} \ln p(k, \sigma_e^2 | y, W) = -\frac{1}{2} \left(\frac{n}{\sigma_e^2} - \frac{1}{(\sigma_e^2)^2} \sum_i \frac{\tilde{y}_i^2}{1 + k \lambda_i} \right). \quad (3.50)$$

$$\frac{\partial}{\partial k} \ln p(k, \sigma_e^2 | y, W) = -\frac{1}{2} \left(\sum_i \frac{\lambda_i}{1 + k \lambda_i} - \frac{1}{\sigma_e^2} \sum_i \frac{\lambda_i \tilde{y}_i^2}{(1 + k \lambda_i)^2} \right). \quad (3.51)$$

Second Derivatives

$$\frac{\partial^2}{(\partial \sigma_e^2)^2} \ln p(k, \sigma_e^2 | y, W) = -\frac{1}{2} \left(\frac{2}{(\sigma_e^2)^3} \sum_i \frac{\tilde{y}_i^2}{1 + k\lambda_i} - \frac{n}{(\sigma_e^2)^2} \right). \quad (3.52)$$

$$\frac{\partial^2}{(\partial k)^2} \ln p(k, \sigma_e^2 | y, W) = -\frac{1}{2} \left(\frac{1}{\sigma_e^2} \sum_i \frac{2\lambda_i^2 \tilde{y}_i^2}{(1 + k\lambda_i)^3} - \sum_i \frac{\lambda_i^2}{(1 + k\lambda_i)^2} \right). \quad (3.53)$$

$$\frac{\partial^2}{\partial k \partial \sigma_e^2} \ln p(k, \sigma_e^2 | y, W) = -\frac{1}{2(\sigma_e^2)^2} \sum_i \frac{\tilde{y}_i^2 \lambda_i}{(1 + k\lambda_i)^2}. \quad (3.54)$$

The iterative system is

$$\begin{bmatrix} \sigma_e^2 \\ k \end{bmatrix}_{t+1} = \begin{bmatrix} \sigma_e^2 \\ k \end{bmatrix}_t - \begin{bmatrix} \frac{\partial^2}{(\partial \sigma_e^2)^2} \ln p(k, \sigma_e^2 | y, W) & \frac{\partial^2}{\partial k \partial \sigma_e^2} \ln p(k, \sigma_e^2 | y, W) \\ \frac{\partial^2}{\partial k \partial \sigma_e^2} \ln p(k, \sigma_e^2 | y, W) & \frac{\partial^2}{(\partial k)^2} \ln p(k, \sigma_e^2 | y, W) \end{bmatrix}^{-1} \begin{bmatrix} \frac{\partial}{\partial \sigma_e^2} \ln p(k, \sigma_e^2 | y, W) \\ \frac{\partial}{\partial k} \ln p(k, \sigma_e^2 | y, W) \end{bmatrix}. \quad (3.55)$$

The partial derivatives in (3.55) are evaluated at $\sigma_e^2 = \sigma_e^{2[t]}$, $k = k^{[t]}$.

An R-Code That Performs the Eigenvalue Decomposition

As an example, an R-code that performs the eigenvalue decomposition of the genomic relationship matrix is shown below. First, matrix X of dimension equal to the number of individuals times the number of markers ($nindiv \times nmark$) is generated by allocating marker codes drawn from a binomial distribution $Bi(2, p = 0.5)$. The matrix of standardised marker genotypes W is obtained by centring and scaling X . Using W the code generates the genomic relationship matrix G ($nindiv \times nindiv$), and the eigen decomposition is performed on G . This decomposition gives rise to matrix U ($nindiv \times nindiv$) and to the vector of eigenvalues val , from which the diagonal matrix D ($nindiv \times nindiv$) is produced.

The last line of the code checks whether G and UDU' are equal: they are within the limits of numerical precision.

```
# CODE0303
# DATA BASED ON GENOMIC MODEL; OBTAIN THE SVD OF WW'(1/m)
rm(list=ls()) # CLEAR WORKSPACE
set.seed(1953)
nindiv<-10
nmark<-20
```

```

X<-matrix(nrow= nindiv,ncol= nmark,
          rbinom(n=nindiv*nmark,size=2,p=.5))
W <- matrix(data=NA,nrow= nindiv,ncol=nmark)
U <- matrix(data=NA,nrow= nindiv,ncol= nindiv)
G<-matrix(data=NA,nrow= nindiv,ncol= nindiv)
cm <- colMeans(X)
# CREATE MATRIX OF STANDARDISED MARKER GENOTYPE CODES
for (i in 1:nmark)
{
W[,i] <-( X[,i]-cm[i]) / sd(X[,i])
}
# THIS IS MORE EFFICIENT THAN THE LOOP:
# W <- scale(X, center=TRUE, scale=TRUE)
qr(X)$rank

```

```
## [1] 10
```

```
qr(W)$rank
```

```
## [1] 9
```

```

# GENOMIC RELATIONSHIP MATRIX G
G <- (1/nmark)*W%*%t(W)
# THIS IS MORE EFFICIENT THAN THE LINE ABOVE:
# G <- (1/nmark)*tcrossprod(W)
# SVD OF G
EVD <- eigen(G)
names(EVD)

```

```
## [1] "values" "vectors"
```

```
head(EVD$values[1:5])
```

```
## [1] 2.3596077 1.8733148 1.3984700 1.0492670 0.7181604
```

```

U <- EVD$vector
val <- EVD$values
val[nindiv] <-0
D <- diag(val,nrow=nindiv)
# CHECK THAT G = UDU':
identical(G, U%*%D%*%t(U))

```

```
## [1] FALSE
```

```
max(abs(G - U%*%D%*%t(U)))
```

```
## [1] 2.88658e-15
```

3.2 Gradient Descent and Stochastic Gradient Descent

Gradient descent is a first-order iterative optimisation algorithm for finding a local minimum of a differentiable function. This is achieved by taking repeated steps in the opposite sign of the gradient of the function at the current point, because this is the direction of greatest rate of decrease of the function. The method does not rely on matrix inversions and is often used in high dimensional settings as encountered in machine learning.

Let θ_t be the value of a parameter $\theta \in \mathbb{R}^d$ at step t and let the column vector $\Delta\theta \in \mathbb{R}^d$ be the move taken at step t . Consider the first order Taylor expansion of the function f to be minimised around θ_t

$$f(\theta_t + \Delta\theta) \approx f(\theta_t) + \Delta\theta' \nabla f|_{\theta_t}$$

where $\nabla f|_{\theta_t}$ is the gradient of f evaluated at $\theta = \theta_t$, i.e., a column vector in $\mathbb{R}^d$ whose i th coordinate is the partial derivative of f with respect to element i of θ evaluated at $\theta = \theta_t$. Since the objective is to find a minimum of the function f , the move must result in

$$f(\theta_t + \Delta\theta) \leq f(\theta_t)$$

and therefore $\Delta\theta' \nabla f|_{\theta_t} \leq 0$. This suggests setting

$$\Delta\theta = -\alpha \nabla f|_{\theta_t}, \quad \alpha > 0 \tag{3.56}$$

where α is a small positive number known as the *learning rate*. If

$$\theta_{t+1} = \theta_t - \alpha \nabla f|_{\theta_t} \tag{3.57}$$

then

$$f(\theta_{t+1}) = f(\theta_t) - \alpha \left((\nabla f|_{\theta_t})' \nabla f|_{\theta_t} \right) < f(\theta_t) \tag{3.58}$$

since $(\nabla f|_{\theta_t})' \nabla f|_{\theta_t} > 0$.

In the case of Newton's method, $\alpha = [S'(\theta_t)]^{-1}$, that involves the inverse of second derivatives. The attraction of gradient descent, is that it only uses first derivatives and does not require matrix inversions. The downside is that it can be slow to converge and the appropriate choice of α can be quite challenging, especially in high dimensional settings where the gradients differ markedly among the θ 's. In such cases one may consider using a *diagonally-scaled gradient descent* where α is replaced by a diagonal matrix with elements tuned to each gradient. Alternatively, one can scale the features x_i so that they fall approximately in the range ± 1 . Centring and scaling the features by their standard deviation is yet another option.

It can also help to change α as a function of t , with larger values at the start of the iterative process.

A popular modification, especially with very large data sets, is *stochastic gradient descent*. At each iteration, the actual gradient is replaced by an estimate obtained from a randomly selected subset of the data. An extreme version uses a single data point at a time (see example below). These versions of gradient descent are also known as *mini batch gradient descent* as opposed to *batch gradient descent* when the complete set of data is used. In common with many optimisation algorithms, when data sets are very large, stochastic gradient descent converges faster in terms of total computation by rapidly calculating approximate values rather than exact values of the gradient. Stochastic gradient descent does not converge exactly to a local minimum of the cost function (negative of the loglikelihood) as classical gradient descent in theory does. It rather may oscillate around the local minimum. Recent theoretical and empirical results indicate that this does not seem to be an issue in large networks (LeCun et al. 2015). The noise associated with stochastic gradient descent can be beneficial to escape local minima, particularly in complex networks with non-convex loss functions.

There are several refinements to gradient descent as well as rules guiding the choice of α . A useful reference is Bottou (2012) and an overview of numerical computations for machine learning algorithms can be found in Goodfellow et al. (2016).

If the cost function is concave such as many likelihood functions, the sign in (3.57) becomes positive in order to move towards a maximum and this turns into a *gradient ascent* algorithm.

Note

The maximum likelihood estimator of a parameter θ involves the maximisation of the likelihood function with respect to θ , or the minimisation of the negative of the likelihood function with respect to θ . Both are tasks on optimisation. When minimisation is involved the function can be referred to as a *cost function*. The cost function may include a penalty term, as for example in penalised logistic regression discussed on page 410.

A Toy Example

As an illustration, the R-code below applies classical gradient descent to find the minimum of a cost function (negative loglikelihood) of a simple linear model $y = Xb$ where b includes an intercept. The result is compared to the exact least squares solution $\hat{b} = (X'X)^{-1} X'y$ at the bottom of the code. The code immediately below implements stochastic gradient descent.

```

# CODE0304
# GRADIENT DESCENT WITH A LINEAR MODEL
rm(list=ls()) # Clear the workspace
set.seed(195021)
N<-100
x<-seq(from=0,to=5,length=N)
signal<-10 + 0.2*x
error<-rnorm(N)
y<-signal+error
one <- rep(1,N)
X <- cbind(one,x)
LHS <- crossprod(X) # LEFT HAND SIDE
RHS <- crossprod(X,y) # RIGHT HAND SIDE
bhat<- solve(LHS,RHS) # SOLUTION TO THE LEAST SQUARES EQUATIONS

#####
nit <- 200
alfa <- 0.002
miu <- matrix(data=NA, nrow=nit+1,ncol=1)
b <- matrix(data=NA, nrow=nit+1,ncol=1)
c <- matrix(data=NA, nrow=nit+1,ncol=1)
cost <- function(miu,b){sum(y-miu-b*x)^2}

miu[1] <- 5
b[1] <- 1
c[1] <- cost(miu[1],b[1])
for(i in 1:nit) {
  fdmiu <- -sum(y - miu[i] - b[i] * x)
  fdbeta <- -sum((y - miu[i] - b[i] * x) * x)
  fd <- matrix(c(fdmiu, fdbeta), nrow = 2, ncol = 1)
  sol0 <- matrix(c(miu[i], b[i]), nrow = 2, ncol = 1)
  alfa <- 0.002
  sol1 <- sol0 - alfa * fd
  miu[i + 1] <- sol1[1, 1]
  b[i + 1] <- sol1[2, 1]
}
## CHECK #####
beta <- c(miu[i],b[i])
beta # GRADIENT DESCENT SOLUTION

```

```
## [1] 10.1511233 0.1727193
```

```
bhat # LEAST SQUARES SOLUTION
```

```
##           [,1]
## one 10.1515028
## x    0.1726028
```

```
cost(miu[i],b[i]) # COST FUNCTION AFTER nit ITERATIONS
```

```
## [1] 7.781174e-05
```

The following R-code implements stochastic gradient descent with a decaying learning rate α . The implementation is based on two loops. The internal loop computes the gradient and updates parameters one record at a time, for all records. The external loop repeats the procedure `nit` times. Each internal loop that defines a pass through the data is known as an *epoch* in the machine learning literature.

As implemented below, the algorithm does not reshuffle the data at the start. Some versions of the mini batch algorithm do this to achieve better behaviour and to break down structures in the data that may affect the results.

```
# STOCHASTIC GRADIENT DESCENT WITH A LINEAR MODEL
# CODE0305
rm(list=ls()) # Clear the workspace
set.seed(195021)
N <- 1000
x<-seq(from=0,to=5,length=N)
signal<-10 + 0.2*x
error<-rnorm(N)
y<-signal+error
one <- rep(1,N)
X <- cbind(one,x)
LHS <- crossprod(X) # LEFT HAND SIDE
RHS <- crossprod(X,y) # RIGHT HAND SIDE
bh<- solve(LHS,RHS) # SOLUTION TO THE LEAST SQUARES EQUATIONS

#####
nit <- 10
alfa_0 <- 0.0038
c <- matrix(data=NA, nrow=nit+1,ncol=1)
cost <- function(miu,b){sum(y-miu-b*x)^2}

miu <- 5
b <- 1
c[1] <- cost(miu,b)
for(j in 1:nit){
  alfa <- (1-(j/nit))*alfa_0 + ((j/nit)*alfa_0*0.01)
  for(i in 1:length(y)) {
    # cat(i, "\n",sep="")
    fdmiu <- -2*(y[i] - miu - b * x[i])
    fdbeta <- -2*((y[i] - miu - b * x[i]) * x[i])
    fd <- matrix(c(fdmiu, fdbeta), nrow = 2, ncol = 1)
    sol0 <- matrix(c(miu, b), nrow = 2, ncol = 1)
    sol1 <- sol0 - alfa * fd
    miu <- sol1[1, 1]
    b <- sol1[2, 1]
  }
}
## CHECK #####
beta <- c(miu,b)
beta # STOCHASTIC GRADIENT DESCENT SOLUTION
```

```
## [1] 10.0067857 0.1833919
```

```
bh # LEAST SQUARES SOLUTION
```

```
##           [,1]
## one 10.0305748
## x    0.1859502
```

Gradient descent is revisited in the chapter on nonparametric methods where it is applied to solve more demanding problems.

3.3 The EM Algorithm

An overview is provided of one of the most versatile iterative algorithms for computing maximum likelihood and posterior modes: the expectation–maximisation, or EM algorithm. More details can be found in Sorensen and Gianola (2002), considerably more in McLachlan and Krishnan (1997) and of course a tour de force is the celebrated paper by Dempster et al. (1977). The algorithm is conceptually simple, at least in its basic form, and brings considerable insight into the statistical structure of a maximum likelihood or posterior mode problem, contrary to Newton–Raphson or scoring that are based primarily on numerical considerations. The basic idea behind the method is to transform an incomplete- into a complete data problem for which the required maximisation is computationally more tractable. Also, the algorithm is numerically stable: each iteration increases the likelihood or posterior density and convergence is nearly always to a local maximum.

The concept of missing data is fairly broad. It includes, for example, missing data in an unbalanced layout, but it extends to observations from truncated distributions, censored data and latent variables. In these cases, one can view the complete data x as consisting of the vectors (y, z) , where y is the observed data or incomplete data and z is the missing data. More generally, many statistical problems that at first glance do not appear to involve missing data can be reformulated into missing-data problems by judicious augmentation of the data set with unobserved values. As such, one can view the observations at hand and the parameters of the posed model as data: part of these data is observed (the records) and another part is missing (the parameters). Mixed effects models, hierarchical models and models with latent variables, such as the threshold model, are typically amenable to an EM formulation. An example is an additive genetic model where inference may focus on $\theta = (\beta', \sigma_a^2, \sigma_e^2)'$, where β is a vector of location parameters and (σ_a^2, σ_e^2) are variance components. Here, one may augment the observed data y , with the missing data a , the unobserved vector of additive genetic values. As shown later, this simplifies the computations involved in finding the ML estimates of θ , or the maximum of $p(\beta, \sigma_a^2, \sigma_e^2 | y)$, or mode of the posterior distribution $[\beta, \sigma_a^2, \sigma_e^2 | y]$. On the other hand, if one wishes to find the mode of the distribution with density $p(\sigma_a^2, \sigma_e^2 | y)$, an EM strategy is to consider (β, a) as the missing data.

Derivation

The EM algorithm is derived here from a likelihood (as opposed to a posterior distribution) perspective. Conceptually there is no difference. Consider the identity

$$p(y|\theta) = \frac{p(y, z|\theta)}{p(z|y, \theta)},$$

where y is the observed data and z is the missing data. Taking logarithms on both sides leads to

$$\ln p(y|\theta) = \ln p(y, z|\theta) - \ln p(z|y, \theta), \quad (3.59)$$

where the first term on the right-hand side is known as the *complete data loglikelihood* (there is an equivalent complete data logposterior). The next step is to take expectations on both sides with respect to $[z|\theta^{[l]}, y]$, where $\theta^{[l]}$ is the current guess of θ . The left-hand side of (3.59) does not depend on z , so averaging over z , providing the integrals exist, gives

$$\ln p(y|\theta) = \int \ln p(y, z|\theta) p(z|\theta^{[l]}, y) dz - \int \ln p(z|y, \theta) p(z|\theta^{[l]}, y) dz. \quad (3.60)$$

The first term on the right-hand side of (3.60) is a function of θ for fixed y and fixed $\theta^{[l]}$ and is denoted as $Q(\theta|\theta^{[l]})$ in the EM literature. The second term is denoted $H(\theta|\theta^{[l]})$. Thus,

$$\ln p(y|\theta) = Q(\theta|\theta^{[l]}) - H(\theta|\theta^{[l]}). \quad (3.61)$$

The EM algorithm involves working with the first term only, $Q(\theta|\theta^{[l]})$, disregarding $H(\theta|\theta^{[l]})$. The two steps are:

1. E-step: calculation of $Q(\theta|\theta^{[l]})$, the expectation of the complete data loglikelihood (logposterior) with respect to the conditional distribution of the missing data, given the observed data and the current guess for θ .
2. M-step: maximisation of $Q(\theta|\theta^{[l]})$ with respect to θ , solving for θ and setting the result equal to $\theta^{[l+1]}$, the new value of the parameter. If $\theta^{[l+1]}$ maximises $Q(\theta|\theta^{[l]})$, the M-step is such that

$$Q(\theta^{[l+1]}|\theta^{[l]}) \geq Q(\theta|\theta^{[l]}), \quad \text{for all } \theta \in \Omega, \quad (3.62)$$

which implies that $\theta^{[t+1]}$ is a solution to the equation

$$\frac{\partial Q(\theta|\theta^{[t]})}{\partial \theta} = 0. \quad (3.63)$$

The two steps are repeated iteratively until convergence is reached.

An important property of EM (not of Newton-Raphson) is that the iterative sequence leads to a monotonic increase of the loglikelihood $\ln p(y|\theta)$. Therefore,

$$\ln p(y|\theta^{[t+1]}) \geq \ln p(y|\theta^{[t]}). \quad (3.64)$$

Since the loglikelihood increases in each step, the EM algorithm, with few exceptions, converges to a local mode.

In some models the calculation of $Q(\theta|\theta^{[t]})$ in the E-step may be difficult. Wei and Tanner (1990) propose a Monte Carlo approach for overcoming this difficulty. This consists of simulating $z_1, z_2, \dots, z_m$ from $p(z|\theta^{[t]}, y)$ and then forming the simulation consistent estimator

$$\hat{Q}(\theta|\theta^{[t]}) \approx \frac{1}{m} \sum_{i=1}^m \ln p(y, z_i|\theta).$$

In its original formulation, the EM algorithm does not yield estimates of asymptotic variances. However several approaches have been suggested to remedy this and some are described and illustrated in Sorensen and Gianola (2002).

A Digression on a Multivariate Transformation for Discrete Random Variables

A result from the theory of transformations is presented that is relevant for the next example. A multinomial distribution with three distinctive classes is considered.

Let n_i be the number of observations falling into the i th class and let p_i be the probability that an observation falls in the i th class, for $i = 1, 2, 3$, with $p_1 + p_2 + p_3 = 1$. Then $n = n_1 + n_2 + n_3$ and the joint probability mass function of (n_1, n_2) is (the dependence on parameters p_1, p_2 and n is omitted)

$$p(n_1, n_2) = \frac{n!}{n_1! n_2! (n - n_1 - n_2)!} p_1^{n_1} p_2^{n_2} (1 - p_1 - p_2)^{n - n_1 - n_2}. \quad (3.65)$$

Let $(n_1, n_2) = (X, Y)$ and $(n_1, n_1 + n_2) = (U, V)$. Suppose that one needs to find the conditional probability distribution of n_1 , given $n_1 + n_2$,

$$p(n_1 | n_1 + n_2) = \frac{p(n_1, n_1 + n_2)}{p(n_1 + n_2)} = \frac{p(U, V)}{p(V)}. \quad (3.66)$$

To derive the numerator in (3.66), $p(n_1, n_1 + n_2) = p(U, V)$, note that the transformation $(X, Y) \rightarrow (U, V)$ can be written as

$$\begin{bmatrix} U \\ V \end{bmatrix} = f(X, Y) = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} X \\ X + Y \end{bmatrix} \quad (3.67)$$

with inverse transformation

$$\begin{bmatrix} X \\ Y \end{bmatrix} = f^{-1}(U, V) = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} U \\ V \end{bmatrix} = \begin{bmatrix} U \\ V - U \end{bmatrix}.$$

Therefore

$$p_{U,V}(U, V) = \frac{n!}{U!(V-U)!(n-V)!} p_1^U p_2^{V-U} (1-p_1-p_2)^{n-V}. \quad (3.68)$$

To obtain $p(n_1 | n_1 + n_2)$, (3.68) must be divided by $p(V)$. The random variable $V = n_1 + n_2$ is binomially distributed

$$V \sim Bi(p_1 + p_2, n).$$

This is so because the three classes can be regrouped into two “wider” categories, one where the counts $(n_1 + n_2)$ are observed to fall and the other involving the third original category with counts n_3 . In view of the independence of the draws, it follows that $(n_1 + n_2)$ is binomially distributed. Dividing $p_{U,V}(U, V)$ in (3.68) by the marginal probability mass function $p_V(V)$ yields

$$\begin{aligned} p(n_1 | n_1 + n_2) &= p(U | V) \\ &= \frac{p(U, V)}{p(V)} \\ &= \frac{V!}{U!(V-U)!} \frac{p_1^U p_2^{V-U}}{(p_1 + p_2)^V} \\ &= \frac{(n_1 + n_2)!}{n_1! n_2!} \frac{p_1^{n_1} p_2^{n_2}}{(p_1 + p_2)^{n_1 + n_2}} \\ &= \frac{(n_1 + n_2)!}{n_1! n_2!} \left(\frac{p_1}{p_1 + p_2} \right)^{n_1} \left(\frac{p_2}{p_1 + p_2} \right)^{n_2}. \end{aligned}$$

This implies that

$$[n_1|n_1 + n_2] \sim Bi\left(\frac{p_1}{p_1 + p_2}, n_1 + n_2\right). \quad (3.69)$$

Hence, the conditional distribution $[n_1|n_1 + n_2]$ has mean,

$$E(n_1|n_1 + n_2) = (n_1 + n_2) \frac{p_1}{p_1 + p_2}$$

and variance

$$Var(n_1|n_1 + n_2) = (n_1 + n_2) \frac{p_1}{p_1 + p_2} \frac{p_2}{p_1 + p_2}.$$

Example: Estimation of Gene Frequencies from ABO Blood Group Phenotypes

The ABO blood groups problem introduced on page 85 is studied using the EM algorithm. Let $n = (n_A, n_{AB}, n_B, n_O)'$ be the observed data, with $n_A = 725$, $n_{AB} = 72$, $n_B = 258$, $n_O = 1073$. It is sensible to treat the unobserved counts n_{AO} , n_{AA} , n_{BB} and n_{BO} as missing data. The resulting complete data vector is

$$n_c = (n_{AA}, n_{AO}, n_{AB}, n_{BB}, n_{BO}, n_O)'$$

The complete data loglikelihood, excluding an additive constant is

$$\begin{aligned} \ln f(p_A, p_B|n_c) &= 2n_{AA} \ln(p_A) + n_{AO} \ln(2p_A p_O) + n_{AB} \ln(2p_A p_B) \\ &\quad + 2n_{BB} \ln(p_B) + n_{BO} \ln(2p_B p_O) + 2n_O \ln(p_O), \end{aligned}$$

where $p_O = (1 - p_A - p_B)$. The E-step consists of computing the expected value of the complete data loglikelihood, conditional on the observed counts n and on the value of the parameters at iteration t , $(p_A^{[t]}, p_B^{[t]})$. Explicitly, this is

$$\begin{aligned} Q(p_A, p_B|p_A^{[t]}, p_B^{[t]}) &= E[\{2n_{AA} \ln(p_A) + n_{AO} \ln(2p_A p_O) + n_{AB} \ln(2p_A p_B) \\ &\quad + 2n_{BB} \ln(p_B) + n_{BO} \ln(2p_B p_O) + 2n_O \ln(p_O)\} | p_A^{[t]}, p_B^{[t]}, n] \\ &= 2\tilde{n}_{AA} \ln(p_A) + \tilde{n}_{AO} \ln(2p_A p_O) + n_{AB} \ln(2p_A p_B) + 2\tilde{n}_{BB} \ln(p_B) \\ &\quad + \tilde{n}_{BO} \ln(2p_B p_O) + 2n_O \ln(p_O), \end{aligned} \quad (3.70)$$

where

$$\tilde{n}_{AA} = E \left(n_{AA} | p_A^{[t]}, p_B^{[t]}, n \right),$$

$$\tilde{n}_{AO} = E \left(n_{AO} | p_A^{[t]}, p_B^{[t]}, n \right),$$

$$\tilde{n}_{BB} = E \left(n_{BB} | p_A^{[t]}, p_B^{[t]}, n \right),$$

$$\tilde{n}_{BO} = E \left(n_{BO} | p_A^{[t]}, p_B^{[t]}, n \right).$$

The M-step consists of maximizing (3.70) with respect to p_A and p_B . This yields the following closed-form solution for p_A and p_B at a round $(t + 1)$:

$$p_A^{[t+1]} = \frac{2\tilde{n}_{AA} + n_{AB} + \tilde{n}_{AO}}{2(n_A + n_{AB} + n_B + n_O)}, \quad (3.71)$$

$$p_B^{[t+1]} = \frac{2\tilde{n}_{BB} + n_{AB} + \tilde{n}_{BO}}{2(n_A + n_{AB} + n_B + n_O)}. \quad (3.72)$$

The unobserved counts at iteration t are imputed via their expected values, given n and $(p_A^{[t]}, p_B^{[t]})$. Using (3.69), the unobserved counts are distributed binomially as follows:

$$n_{AA} \sim Bi \left(\frac{p_A^2}{p_A^2 + 2p_A(1 - p_A - p_B)}, n_A \right),$$

$$n_{AO} \sim Bi \left(\frac{2p_A(1 - p_A - p_B)}{p_A^2 + 2p_A(1 - p_A - p_B)}, n_A \right),$$

$$n_{BB} \sim Bi \left(\frac{p_B^2}{p_B^2 + 2p_B(1 - p_A - p_B)}, n_B \right),$$

and

$$n_{BO} \sim Bi \left(\frac{2p_B(1 - p_A - p_B)}{p_B^2 + 2p_B(1 - p_A - p_B)}, n_B \right).$$

Hence, expectations can be computed immediately. For example,

$$\tilde{n}_{AA} = n_A \frac{p_A^{2[t]}}{p_A^{2[t]} + 2p_A^{[t]}(1 - p_A^{[t]} - p_B^{[t]})},$$

and similarly for the other components of the missing data. Using starting values for the gene frequencies, the missing counts $\tilde{n}_{ij}$ are imputed and the next round of gene frequency values are computed from (3.71) and (3.72). In the case of the present example, starting with $p_A^{[0]} = p_B^{[0]} = 0.2$, 9 EM iterations result in $\hat{p}_A = 0.2091$ and $\hat{p}_B = 0.0808$ (with $\hat{p}_0 = 1 - \hat{p}_A - \hat{p}_B$).

The R-code below fits the EM algorithm to the *ABO* data. The third column of the output below displays the increasing values of the loglikelihood function with each EM iteration. This compares well with the output from OPTIM on page 90.

```
# CODE0306
#### EM algorithm #####
rm(list=ls())
set.seed(30371)
niter<-9
result <- matrix(data=NA,nrow=niter,ncol=3)

# DATA
n_A<-725
n_AB<-72
n_B<-258
n_0<-1073
# START VALUES FOR P_A and p_B
p_A <- 0.2
p_B <- 0.2
for (i in 1:niter){
# E-step
n_AA <- n_A * p_A^2/(p_A^2 + 2*p_A*(1-p_A-p_B))
n_A0 <- n_A * (2*p_A*(1-p_A-p_B))/(p_A^2 + 2*p_A*(1-p_A-p_B))
n_BB <- n_B * p_B^2/(p_B^2 + 2*p_B*(1-p_A-p_B))
n_B0 <- n_B * (2*p_B*(1-p_A-p_B))/(p_B^2 + 2*p_B*(1-p_A-p_B))
# M-step
p_A <- (2*n_AA + n_AB + n_A0)/(2*(n_A + n_AB + n_B + n_0))
p_B <- (2*n_BB + n_AB + n_B0)/(2*(n_A + n_AB + n_B + n_0))
loglik <- (725*log(p_A*(2 - p_A - 2* p_B)) + 72*log(2*p_A*p_B) +
258*log(p_B*(2 - p_B - 2*p_A)) + 2*1073*log(1 - p_A - p_B))
result[i,] <- c(p_A,p_B,loglik)
}
result
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.2116004 0.08619764 -2304.473
## [2,] 0.2095705 0.08104311 -2303.555
## [3,] 0.2091891 0.08081323 -2303.551
## [4,] 0.2091379 0.08080178 -2303.550
## [5,] 0.2091315 0.08080107 -2303.550
## [6,] 0.2091308 0.08080101 -2303.550
## [7,] 0.2091307 0.08080101 -2303.550
## [8,] 0.2091307 0.08080101 -2303.550
## [9,] 0.2091307 0.08080101 -2303.550
```

Example: A Regression Model for Binary Data

I return to the likelihood of the logit model fitted using Newton-Raphson. Here a probit model is implemented with the EM algorithm.

The binary data y is interpreted as arising from the following process involving the unobserved liability (or latent variable) u :

$$y_i = \begin{cases} 1 & \text{if } u_i < 0, \\ 0 & \text{if } u_i > 0. \end{cases} \quad (3.73)$$

The liability is modelled as

$$u_i = x_i' \beta + e_i, \quad i=1,2,\dots,N, \quad (3.74)$$

where in the probit model the error terms are *iid* $\mathcal{N}(0, 1)$ and therefore $[u_i | x_i, \beta] \sim \mathcal{N}(x_i' \beta, 1)$. The row vector x_i' contains observed covariates with p elements and β is an $p \times 1$ vector of unknown regression coefficients. In the probit model specified by (3.73)

$$\begin{aligned} \Pr(y_i = 1 | \beta, x_i) &= \Pr(u_i < 0 | \beta, x_i) \\ &= \Pr(x_i' \beta + e_i < 0 | \beta, x_i) \\ &= \Pr(e_i < -x_i' \beta | \beta, x_i) \\ &= 1 - \Phi(x_i' \beta) \end{aligned}$$

and

$$\begin{aligned} \Pr(y_i = 0 | \beta, x_i) &= \Pr(u_i > 0 | \beta, x_i) \\ &= \Phi(x_i' \beta). \end{aligned}$$

The probit likelihood is

$$L(\beta | x, y) \propto \prod_{i=1}^N \left[(1 - \Phi(x_i' \beta))^{y_i} (\Phi(x_i' \beta))^{1-y_i} \right]. \quad (3.75)$$

Augmenting the observed data y with the missing data u , the complete data for the i th observation is (y_i, u_i) . The joint density of the complete data can be written as

$$\Pr(Y_i = y_i | u_i) p(u_i | \beta, x_i) \propto \begin{cases} \mathcal{N}(u_i | x_i' \beta, 1) I(u_i < 0), & \text{for } y_i = 1, \\ \mathcal{N}(u_i | x_i' \beta, 1) I(u_i > 0), & \text{for } y_i = 0, \end{cases} \quad (3.76)$$

where the results follow from (3.73) and (3.74). The term $\Pr(Y_i = y_i | u_i)$ is a degenerate probability mass function: given the model specified by (3.73), once u_i is observed Y_i is not stochastic; it is known with certainty. This translates into the indicator functions in the right hand side of (3.76). However, these indicator functions will be omitted because they become additive constants of the complete data loglikelihood; they are not a function of β and vanish in the computation of the M-step (the form of the complete data likelihood including indicator functions is given in Eq. (5.26) on page 239 that is an alternative way of expressing the right hand side of (3.76)). Therefore the complete data likelihood for the i th observation is proportional to $p(u_i | \beta, x_i)$.

The *complete data likelihood* is proportional to the joint distribution of the liabilities. Due to independence, the density of this joint distribution is

$$p(u | \beta, x) = \prod_{i=1}^N p(u_i | \beta, x_i) \quad (3.77)$$

where each term in (3.77) is

$$p(u_i | \beta, x_i) = \mathcal{N}(u_i | x_i' \beta, 1). \quad (3.78)$$

The complete data loglikelihood (excluding additive terms that do not include β) is

$$\ell(\beta | u, x) = \sum_{i=1}^N \ln(p(u_i | \beta, x_i)). \quad (3.79)$$

The E-step consists of averaging (3.79) over the conditional distribution $[u | x, \beta^{[t]}, y]$; that is

$$\begin{aligned} Q(\beta, \beta^{[t]}) &= \int \ell(\beta | u, x) p(u | x, \beta^{[t]}, y) du \\ &= \int \sum_{i=1}^N \ln(p(u_i | \beta, x_i)) p(u_i | x_i, \beta^{[t]}, y_i) du_i \\ &= \sum_{i=1}^N \int \ln(p(u_i | \beta, x_i)) p(u_i | x_i, \beta^{[t]}, y_i) du_i. \end{aligned} \quad (3.80)$$

E-Step

The calculation of (3.80) needs the following results. For $y_i = 0$:

$$\begin{aligned} E(u_i | \beta, x_i, y_i = 0) &= x_i' \beta + E(e_i | y_i = 0) \\ &= x_i' \beta + E(e_i | x_i' \beta + e_i > 0) \end{aligned}$$

$$\begin{aligned}
&= x'_i \beta + E(e_i | e_i > -x'_i \beta) \\
&= x'_i \beta + \frac{1}{\Phi(x'_i \beta)} \int_{-x'_i \beta}^{\infty} e_i (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{e_i^2}{2}\right) de_i \\
&= x'_i \beta + \frac{\phi(x'_i \beta)}{\Phi(x'_i \beta)}. \tag{3.81}
\end{aligned}$$

For $y_i = 1$:

$$\begin{aligned}
E(u_i | \beta, x_i, y_i = 1) &= x'_i \beta + E(e_i | y_i = 1) \\
&= x'_i \beta + E(e_i | x'_i \beta + e_i < 0) \\
&= x'_i \beta + E(e_i | e_i < -x'_i \beta) \\
&= x'_i \beta + \frac{1}{\Phi(-x'_i \beta)} \int_{-\infty}^{-x'_i \beta} e_i (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{e_i^2}{2}\right) de_i \\
&= x'_i \beta - \frac{\phi(x'_i \beta)}{1 - \Phi(x'_i \beta)}. \tag{3.82}
\end{aligned}$$

In these expressions, $\phi(z)$ is the density of $\mathcal{N}(0, 1)$ at $Z = z$ and $\Phi(t) = \Pr(U \leq t)$, the cumulative distribution function of the standard normal distribution. The ratios $\phi(x'_i \beta) / \Phi(x'_i \beta)$ and $-\phi(x'_i \beta) / (1 - \Phi(x'_i \beta))$ are known as the *intensity of selection* in the quantitative genetics literature (for normally distributed characters). To arrive at the last line in (3.81) and (3.82) use

$$\int_{-x'_i \beta}^{\infty} e_i (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{e_i^2}{2}\right) de_i = - \int_{-\infty}^{-x'_i \beta} e_i (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{e_i^2}{2}\right) de_i = \phi(x'_i \beta).$$

With these results one proceeds with the evaluation of (3.80). Each term $\ln(p(u_i | \beta, x_i))$ is equal to (excluding an additive constant)

$$\ln(p(u_i | \beta, x_i)) = -\frac{(u_i - x'_i \beta)^2}{2} = -\frac{u_i^2 - 2u_i x'_i \beta + \beta' x_i x'_i \beta}{2}.$$

For the i th term of (3.80), the expectation over the distribution $[u_i | x'_i \beta^{[t]}, y_i]$ is

$$\begin{aligned}
Q_i(\beta, \beta^{[t]}) &= E[\ln(p(u_i | \beta, x_i))] \\
&= -\frac{1}{2} \left[E(u_i^2 | x_i, \beta^{[t]}, y_i) - 2x'_i \beta E(u_i | x_i, \beta^{[t]}, y_i) + \beta' x_i x'_i \beta \right]. \tag{3.83}
\end{aligned}$$

M-Step

The M-step involves a differentiation of (3.80) with respect to β , setting the result equal to zero and solving for β . The solution leads to $\beta^{[t+1]}$.

Since the first term in (3.83) $E(u_i^2 | x_i' \beta^{[t]}, y_i)$, does not involve β , the partial derivative is

$$\begin{aligned} & \frac{\partial}{\partial \beta} \left\{ -\frac{1}{2} \left[E(u_i^2 | x_i, \beta^{[t]}, y_i) - 2x_i' \beta E(u_i | x_i, \beta^{[t]}, y_i) + \beta' x_i x_i' \beta \right] \right\} \\ &= -\frac{1}{2} \left[-2x_i E(u_i | x_i, \beta^{[t]}, y_i) + 2x_i x_i' \beta \right], \end{aligned}$$

where $E(u_i | x_i, \beta^{[t]}, y_i)$ is given by (3.81) or by (3.82). Setting this expression equal to zero yields

$$x_i \left[E(u_i | x_i, \beta^{[t]}, y_i) - x_i' \beta \right] = 0.$$

For the N records we obtain

$$\sum_{i=1}^N x_i x_i' \beta = \sum_{i=1}^N x_i E(u_i | x_i, \beta^{[t]}, y_i).$$

The resulting iterative system is

$$\beta^{[t+1]} = \left[\sum_{i=1}^N (x_i x_i') \right]^{-1} \sum_{i=1}^N x_i E(u_i | x_i, \beta^{[t]}, y_i). \quad (3.84)$$

This solution is used back again in (3.80) to start a new iteration round that requires calculation of $E(u_i | x_i, \beta^{[t+1]}, y_i)$ using (3.81) or (3.82) depending on whether $y_i = 1$ or $y_i = 0$. Therefore the implementation of EM for the probit model is as follows:

1. Start with a guess value $\beta^{[0]}$
2. Compute $E(u_i | x_i, \beta^{[0]}, y_i)$ using (3.81) or (3.82)
3. Solve for β using (3.84) and obtain $\beta^{[1]}$
4. Go back to 2, update using $\beta^{[1]}$ and continue iterating until convergence

Notes

If u denotes the $N \times 1$ vector of liabilities, then the model for the missing data can be written

$$u = X\beta + e, \quad (3.85)$$

where X is an $N \times p$ observed matrix of covariates whose i th row is x_i' . The i th row of this linear system is given by (3.74). With this formulation, in (3.84),

$$\sum_{i=1}^N (x_i x_i') = X'X.$$

Let $\tilde{E}^{[t]}$ denote the $N \times 1$ vector whose i th element is $E(u_i | x_i, \beta^{[t]}, y_i)$. Then (3.84) can be written as

$$\beta^{[t+1]} = (X'X)^{-1} X' \tilde{E}^{[t]}. \quad (3.86)$$

Example: A Binomial Regression Model

An extension of the binary probit regression model is the binomial probit regression model. The unobserved original observations y_{ij} are independent binary (0, 1) as before, but now the observed records n_i are counts of one class and $(N_i - n_i)$ counts of the other class associated with covariate x_i , where N_i is the total number of counts associated with covariate x_i . There are $i = 1, 2, \dots, C$ levels of the covariate and the total number of counts is $\sum_{i=1}^C N_i = N$. In other words, the observed counts n_i are the result of adding the unobserved original records y_{ij}

$$n_i = \sum_{j=1}^{N_i} y_{ij}.$$

The binomial likelihood is

$$L(\beta | x, y) \propto \prod_{i=1}^C \left[(1 - \Phi(x_i' \beta))^{n_i} (\Phi(x_i' \beta))^{N_i - n_i} \right] \quad (3.87)$$

and the loglikelihood, up to an additive constant, is

$$\ell(\beta | x, y) = \sum_{i=1}^C [n_i \ln(1 - \phi(x_i' \beta)) + (N_i - n_i) \ln(\Phi(x_i' \beta))]. \quad (3.88)$$

The extension of the EM algorithm to accommodate the binomial model consists of associating the liability defined in (3.74) to each of the $\sum_{i=1}^C N_i = N$ binary outcomes. The complete data loglikelihood takes the form

$$\ell(\beta | u, x) = \sum_{i=1}^C \sum_{j=1}^{N_i} \ln(p(u_{ij} | \beta, x_i)), \quad (3.89)$$

where

$$p(u_{ij}|\beta, x_i) = N(u_{ij}|x'_i\beta, 1), \quad j = 1, \dots, N_i. \quad (3.90)$$

For example, x_i can represent the level i of a drug administered to N_i mice, n_i is the number of mice that are observed dead and $N_i - n_i$ are the number alive. The liability for all the N_i mice has the same form given by (3.90). The E-step of the algorithm is now

$$\begin{aligned} Q(\beta, \beta^{[t]}) &= \int \ell(\beta|u, x) p(u|x, \beta^{[t]}, y) du \\ &= \int \sum_{i=1}^C \sum_{j=1}^{N_i} \ln(p(u_{ij}|\beta, x_i)) \left[I(y_{ij} = 1) p(u_{ij}|\beta^{[t]}, x_i, y_{ij}) \right. \\ &\quad \left. + I(y_{ij} = 0) p(u_{ij}|\beta^{[t]}, x_i, y_{ij}) \right] du_{ij} \\ &= \sum_{i=1}^C \{n_i E_{[y=1]} \ln(p(u_{ij}|\beta, x_i)) + (N_i - n_i) E_{[y=0]} \ln(p(u_{ij}|\beta, x_i))\} \end{aligned} \quad (3.91)$$

where $E_{[y=1]}$ indicates expectation over $[u_{ij}|\beta^{[t]}, x_i, y_{ij} = 1]$ and $E_{[y=0]}$ indicates expectation over $[u_{ij}|\beta^{[t]}, x_i, y_{ij} = 0]$. These two expectations are given by (3.81) and (3.82); that is

$$E(u_{ij}|\beta, x_i, y_{ij} = 1) = x'_i\beta - \frac{\phi(x'_i\beta)}{1 - \Phi(x'_i\beta)}, \quad (3.92a)$$

$$E(u_{ij}|\beta, x_i, y_{ij} = 0) = x'_i\beta + \frac{\phi(x'_i\beta)}{\Phi(x'_i\beta)}. \quad (3.92b)$$

The term $\ln(p(u_{ij}|\beta, x_i))$, up to an additive constant is equal to

$$-\frac{(u_{ij} - x'_i\beta)^2}{2} = -\frac{u_{ij}^2 - 2u_{ij}x'_i\beta + \beta'x_ix'_i\beta}{2}$$

and

$$\begin{aligned} E_{[y=1]} \ln(p(u_{ij}|\beta, x_i)) &= -\frac{1}{2} \left[E_{[y=1]} (u_{ij}^2|x_i, \beta^{[t]}) \right. \\ &\quad \left. - 2x'_i\beta E_{[y=1]} (u_{ij}|x_i, \beta^{[t]}) + \beta'x_ix'_i\beta \right] \end{aligned}$$

with a similar structure for $E_{[y=0]}$.

The M-step involves differentiation of (3.91) with respect to β . For the i th term in (3.91) this gives, after a little simplification,

$$\begin{aligned} & n_i x_i E_{[y=1]}(u_{ij}|x_i, \beta^{[t]}) - n_i x_i x'_i \beta + (N_i - n_i) x_i \\ & E_{[y=0]}(u_{ij}|x_i, \beta^{[t]}) - (N_i - n_i) x_i x'_i \beta \\ &= n_i x_i E_{[y=1]}(u_{ij}|x_i, \beta^{[t]}) + (N_i - n_i) x_i \\ & E_{[y=0]}(u_{ij}|x_i, \beta^{[t]}) - N_i x_i x'_i \beta. \end{aligned}$$

Therefore differentiation of (3.91) with respect to β is equal to

$$\begin{aligned} \frac{\partial Q(\beta, \beta^{[t]})}{\partial \beta} &= \sum_{i=1}^C n_i x_i E_{[y=1]}(u_{ij}|x_i, \beta^{[t]}) \\ &+ \sum_{i=1}^C (N_i - n_i) x_i E_{[y=0]}(u_{ij}|x_i, \beta^{[t]}) - \sum_{i=1}^C N_i x_i x'_i \beta \end{aligned}$$

Setting this equation equal to zero yields

$$\begin{aligned} \sum_{i=1}^C N_i x_i x'_i \beta &= \sum_{i=1}^C n_i x_i E_{[y=1]}(u_{ij}|x_i, \beta^{[t]}) \\ &+ \sum_{i=1}^C (N_i - n_i) x_i E_{[y=0]}(u_{ij}|x_i, \beta^{[t]}) \end{aligned}$$

and the iterative system becomes

$$\begin{aligned} \beta^{[t+1]} &= \left[\sum_{i=1}^C N_i x_i x'_i \right]^{-1} \left\{ \sum_{i=1}^C n_i x_i E_{[y=1]}(u_{ij}|x_i, \beta^{[t]}) \right. \\ &\quad \left. + \sum_{i=1}^C (N_i - n_i) x_i E_{[y=0]}(u_{ij}|x_i, \beta^{[t]}) \right\}. \end{aligned} \quad (3.93)$$

Let D_N , D_n and D_δ denote $C \times C$ diagonal matrices with diagonal elements N_i , n_i and $(N_i - n_i)$ respectively. Let $\tilde{E}_{[y=1]}^{[t]}$ be the $C \times 1$ vector whose i th element ($i = 1, \dots, C$) is $E(u_{ij}|x_i, \beta^{[t]}, y_{ij} = 1)$ and let $\tilde{E}_{[y=0]}^{[t]}$ be the $C \times 1$ vector whose

i th element is $E(u_{ij}|x_i, \beta^{[t]}, y_{ij} = 0)$. Then (3.93) can be written more compactly as

$$\beta^{[t+1]} = (X' D_N X)^{-1} \left[X' D_n \tilde{E}_{[y=1]}^{[t]} + X' D_\delta \tilde{E}_{[y=0]}^{[t]} \right]. \quad (3.94)$$

With binary records, $N_1 = N_2 = \dots = N_C = 1$ and the iterative system (3.94) reduces to (3.86).

A Digression on Some Matrix Algebra Results

The following results are useful for the examples that follow.

Assume that the joint distribution of the vector of data y and the vector of additive genetic values a is multivariate normal

$$\begin{matrix} y \\ a \end{matrix} \Big| \theta \sim N \left(\begin{bmatrix} X\beta \\ 0 \end{bmatrix}, \begin{bmatrix} V & ZA\sigma_a^2 \\ AZ'\sigma_a^2 & A\sigma_a^2 \end{bmatrix} \right),$$

with $V = ZAZ'\sigma_a^2 + I\sigma_e^2$, $\theta = (\beta, \sigma_a^2, \sigma_e^2)$, β is a location vector of “fixed” effects, A is the additive genetic relationship matrix, X and Z are observed incidence matrices and σ_a^2, σ_e^2 are components of variance.

- *Conditional mean.* From properties of the multivariate normal distribution

$$a|\theta, y \sim N(E(a|\theta, y), \text{Var}(a|\theta, y)),$$

where

$$E(a|\theta, y) = AZ'V^{-1}(y - X\beta)\sigma_a^2.$$

Let $k = \sigma_e^2/\sigma_a^2$. Substituting V^{-1} with

$$V^{-1} = \frac{1}{\sigma_e^2}I - \frac{1}{\sigma_e^2}Z\left(Z'Z + A^{-1}k\right)^{-1}Z',$$

leads to the following expression for the conditional mean:

$$\begin{aligned} & E(a|\theta, y) \\ &= AZ' \left[\frac{1}{\sigma_e^2}I - \frac{1}{\sigma_e^2}Z\left(Z'Z + A^{-1}k\right)^{-1}Z' \right] (y - X\beta)\sigma_a^2 \end{aligned}$$

$$\begin{aligned}
&= \left[A \frac{1}{k} - \frac{1}{k} A Z' Z \left(Z' Z + A^{-1} k \right)^{-1} \right] Z' (y - X\beta) \\
&= \left(Z' Z + A^{-1} k \right)^{-1} Z' (y - X\beta). \tag{3.95}
\end{aligned}$$

The last line follows because

$$A \frac{1}{k} - \frac{1}{k} A Z' Z \left(Z' Z + A^{-1} k \right)^{-1} = \left(Z' Z + A^{-1} k \right)^{-1}.$$

This can be verified by post-multiplying the left-hand side by the inverse of the right-hand side, which recovers I .

- *Conditional variance.* The conditional variance is

$$\text{Var}(a|\theta, y) = \left(Z' Z + A^{-1} k \right)^{-1} \sigma_e^2. \tag{3.96}$$

This is obtained using properties of the multivariate normal distribution:

$$\begin{aligned}
\text{Var}(a|\theta, y) &= A \sigma_a^2 - A Z' \sigma_a^2 V^{-1} Z A \sigma_a^2 \\
&= A \sigma_a^2 - \left[A \frac{1}{k} - \frac{1}{k} A Z' Z \left(Z' Z + A^{-1} k \right)^{-1} \right] Z' Z A \sigma_a^2 \\
&= A \sigma_a^2 - \left(Z' Z + A^{-1} k \right)^{-1} Z' Z A \sigma_a^2 \\
&= \left(Z' Z + A^{-1} k \right)^{-1} \sigma_e^2.
\end{aligned}$$

The last line can be verified by premultiplying the third line by

$$\left(Z' Z + A^{-1} k \right),$$

that recovers $I \sigma_e^2$.

An alternative derivation from a Bayesian perspective (that implies assigning a uniform, improper prior distribution to the location vector β) is to start from

$$\beta \Big| \sigma_a^2, \sigma_e^2, y \sim N \left(\begin{bmatrix} \hat{\beta} \\ \hat{a} \end{bmatrix}, \begin{bmatrix} C^{11} & C^{12} \\ C^{21} & C^{22} \end{bmatrix} \sigma_e^2 \right) \tag{3.97}$$

where

$$\begin{bmatrix} C^{11} & C^{12} \\ C^{21} & C^{22} \end{bmatrix}$$

is the inverse of the coefficient matrix of the mixed model equations and $(\hat{\beta}, \hat{a})$ is the posterior mean vector

$$\begin{bmatrix} \hat{\beta} \\ \hat{a} \end{bmatrix} = \begin{bmatrix} C^{11} & C^{12} \\ C^{21} & C^{22} \end{bmatrix} \begin{bmatrix} X'y \\ Z'y \end{bmatrix}. \quad (3.98)$$

Using results outlined on page 173 or in Example 1.18 of Chap. 1 in Sorensen and Gianola (2002), one can show that the mean vector of $[a|\beta, \sigma_a^2, \sigma_e^2, y]$ is equal to

$$\left(Z'Z + A^{-1}k \right)^{-1} Z' (y - X\beta),$$

and its covariance matrix is equal to (3.96).

Example: ML Estimation in the Mixed Linear Model

This example illustrates the derivation of the iterative EM equations for ML estimation of fixed effects and of variance components in the univariate Gaussian mixed linear model with 2 variance components. A classical quantitative genetics setup is the mixed linear model, that includes fixed and random effects entering linearly into the conditional (given the random effects) expectation of the observations. Fixed effects could, for example, represent systematic sex differences or breed differences and the random effects could represent additive genetic values. In this case, the two variance components involve the residual variance and the component of the variance-covariance structure of the vector of additive genetic values. The implicit assumption of the model is that variance components are the same in the two sexes and breeds.

Specifically, assume that the data y (vector of dimension $n \times 1$) is a realisation from

$$y|\beta, a, \sigma_e^2 \sim N(X\beta + Za, I\sigma_e^2),$$

and the unobserved $q \times 1$ vector of the additive genetic values is multivariate normally distributed

$$a|A\sigma_a^2 \sim N(0, A\sigma_a^2).$$

The vector of fixed effects β has order $p \times 1$; X and Z are known incidence matrices and the unknown variance components are the scalars σ_a^2 and σ_e^2 . The matrix A is known; it describes expected additive genetic relationships among individuals, given

a known pedigree. The focus of inference is $\theta = (\beta', \sigma_a^2, \sigma_e^2)'$. The observed data likelihood is

$$L(\theta|y) = \int p(y|\beta, a, \sigma_e^2) p(a|A\sigma_a^2) da \\ \propto |V|^{-\frac{1}{2}} \exp\left[-\frac{1}{2} (y - X\beta)' V^{-1} (y - X\beta)\right], \quad (3.99)$$

where $V = ZAZ'\sigma_a^2 + I\sigma_e^2$ is the unconditional variance–covariance matrix of the observed data y . Rather than working with (3.99), the ML estimate of θ is obtained using the EM algorithm.

Treating the random effects a as the missing data, the complete data likelihood is

$$L(\theta, a|y) = |I\sigma_e^2|^{-\frac{1}{2}} \exp\left[-\frac{1}{2\sigma_e^2} (y - X\beta - Za)' (y - X\beta - Za)\right] \\ \times |A\sigma_a^2|^{-\frac{1}{2}} \exp\left[-\frac{1}{2\sigma_a^2} a' A^{-1} a\right], \quad (3.100)$$

and the corresponding loglikelihood is

$$\ln p(\theta, a|y) = \text{constant} - \frac{n}{2} \ln \sigma_e^2 - \frac{q}{2} \ln \sigma_a^2 - \frac{1}{2\sigma_a^2} a' A^{-1} a \\ - \frac{1}{2\sigma_e^2} (y - X\beta - Za)' (y - X\beta - Za). \quad (3.101)$$

The E-step is

$$Q(\theta|\theta^{[t]}) = \int \ln p(\theta, a|y) p(a|\theta^{[t]}, y) da \\ = -\frac{n}{2} \ln \sigma_e^2 - \frac{q}{2} \ln \sigma_a^2 - \frac{1}{2\sigma_a^2} E_{a|\theta^{[t]}, y} [a' A^{-1} a] \\ - \frac{1}{2\sigma_e^2} E_{a|\theta^{[t]}, y} (y - X\beta - Za)' (y - X\beta - Za).$$

Let

$$E_{a|\theta^{[t]}, y} [a|\theta^{[t]}, y] = \tilde{a}^{[t]}, \quad (3.102)$$

and

$$\text{Var}_{a|\theta^{[t]}, y} [a|\theta^{[t]}, y] = \tilde{v}_a^{[t]}. \quad (3.103)$$

Using results for expectation of quadratic forms (Searle 1971), (see also Note on page 141)

$$\begin{aligned} Q(\theta|\theta^{[t]}) = & -\frac{n}{2} \ln \sigma_e^2 - \frac{q}{2} \ln \sigma_a^2 - \frac{1}{2\sigma_a^2} \left[\tilde{a}^{[t]'} A^{-1} \tilde{a}^{[t]} + \text{tr} \left(A^{-1} \tilde{V}_a^{[t]} \right) \right] \\ & - \frac{1}{2\sigma_e^2} \left[\left(y - X\beta - Z\tilde{a}^{[t]} \right)' \left(y - X\beta - Z\tilde{a}^{[t]} \right) + \text{tr} \left(Z' Z \tilde{V}_a^{[t]} \right) \right]. \end{aligned}$$

The M-step consists of setting the following equations equal to zero:

$$\frac{\partial Q(\theta|\theta^{[t]})}{\partial \beta} = \frac{1}{\sigma_e^2} X' \left(y - X\beta - Z\tilde{a}^{[t]} \right),$$

$$\frac{\partial Q(\theta|\theta^{[t]})}{\partial \sigma_a^2} = -\frac{q}{2\sigma_a^2} + \frac{1}{2(\sigma_a^2)^2} \left[\tilde{a}^{[t]'} A^{-1} \tilde{a}^{[t]} + \text{tr} \left(A^{-1} \tilde{V}_a^{[t]} \right) \right],$$

and

$$\begin{aligned} \frac{\partial Q(\theta|\theta^{[t]})}{\partial \sigma_e^2} = & -\frac{n}{2\sigma_e^2} \\ & + \frac{1}{2(\sigma_e^2)^2} \left[\left(y - X\beta - Z\tilde{a}^{[t]} \right)' \left(y - X\beta - Z\tilde{a}^{[t]} \right) + \text{tr} \left(Z' Z \tilde{V}_a^{[t]} \right) \right]. \end{aligned}$$

Solving for θ one obtains the iterative system

$$\beta^{[t+1]} = (X'X)^{-1} X' \left(y - Z\tilde{a}^{[t]} \right), \quad (3.104)$$

$$\sigma_a^{2[t+1]} = \frac{1}{q} \left[\tilde{a}^{[t]'} A^{-1} \tilde{a}^{[t]} + \text{tr} \left(A^{-1} \tilde{V}_a^{[t]} \right) \right], \quad (3.105)$$

and

$$\begin{aligned} \sigma_e^{2[t+1]} = & \frac{1}{n} \left[\left(y - X\beta^{[t+1]} - Z\tilde{a}^{[t]} \right)' \left(y - X\beta^{[t+1]} - Z\tilde{a}^{[t]} \right) \right. \\ & \left. + \text{tr} \left(Z' Z \tilde{V}_a^{[t]} \right) \right]. \end{aligned} \quad (3.106)$$

Explicit expressions for $\tilde{a}^{[t]}$ and $\tilde{V}_a^{[t]}$ were given in (3.95) and (3.96).

Example: REML (Restricted Maximum Likelihood) Estimation in the Mixed Linear Model

The model is as in the preceding example, but now the focus of inference is $\theta = (\sigma_a^2, \sigma_e^2)$, with β and a viewed as nuisance parameters. The restricted maximum likelihood (REML) equations are derived using a Bayesian perspective, and the mode of the posterior distribution with density $p(\sigma_a^2, \sigma_e^2 | y)$ is chosen as the “REML” point estimator (Patterson and Thompson 1971). You may want to return to this example after reading Chap. 4.

Assigning improper uniform prior distributions to each of (σ_a^2, σ_e^2) and to β , then

$$p(\sigma_a^2, \sigma_e^2 | y) \propto \int p(y | \beta, a, \sigma_e^2) p(a | A, \sigma_a^2) da d\beta.$$

In this setting the mode of the posterior distribution of the variance components is identical to the REML estimator (Harville 1977). Joint maximisation of this expression is difficult. However, it is relatively easy to structure an EM algorithm, where the missing data are now $z = (\beta', a')'$. The complete data posterior distribution $p(\sigma_a^2, \sigma_e^2, z | y)$ is identical to (3.101) and the E-step is now

$$\begin{aligned} Q(\theta | \theta^{[t]}) &= \int \ln p(\theta, \beta, a | y) p(\beta, a | \theta^{[t]}, y) da d\beta \\ &= -\frac{n}{2} \ln \sigma_e^2 - \frac{q}{2} \ln \sigma_a^2 - \frac{1}{2\sigma_a^2} E_{\beta, a | \theta^{[t]}, y} [a' A^{-1} a] \\ &\quad - \frac{1}{2\sigma_e^2} E_{\beta, a | \theta^{[t]}, y} (y - X\beta - Za)' (y - X\beta - Za). \end{aligned} \quad (3.107)$$

Taking expectations over the quadratic forms leads to

$$\begin{aligned} Q(\theta | \theta^{[t]}) &= -\frac{n}{2} \ln \sigma_e^2 - \frac{q}{2} \ln \sigma_a^2 \\ &\quad - \frac{1}{2\sigma_a^2} [\hat{a}^{[t]'} A^{-1} \hat{a}^{[t]} + \text{tr}(A^{-1} C^{22[t]}) \sigma_e^{2[t]}] \\ &\quad - \frac{1}{2\sigma_e^2} [\hat{e}^{[t]'} \hat{e}^{[t]} + \text{tr}([X, Z] C^{-1[t]} [X, Z]') \sigma_e^{2[t]}], \end{aligned}$$

where $\hat{e}^{[t]} = (y - X\hat{\beta}^{[t]} - Z\hat{a}^{[t]})$, $\hat{a}$ is defined in (3.98) and

$$C^{-1} = \begin{bmatrix} C^{11} & C^{12} \\ C^{21} & C^{22} \end{bmatrix}.$$

Since the missing data are now $z = (\beta', a')'$, expectations in (3.107) are taken with respect to $[\beta, a | \theta^{[t]}, y]$, displayed in (3.97). The M-step is

$$\frac{\partial Q(\theta | \theta^{[t]})}{\partial \sigma_a^2} = -\frac{q}{2\sigma_a^2} + \frac{1}{2(\sigma_a^2)^2} \left[\widehat{a}^{[t]'} A^{-1} \widehat{a}^{[t]} + \text{tr} \left(A^{-1} C^{22[t]} \right) \sigma_e^{2[t]} \right],$$

and

$$\begin{aligned} \frac{\partial Q(\theta | \theta^{[t]})}{\partial \sigma_e^2} &= -\frac{n}{2\sigma_e^2} \\ &+ \frac{1}{2(\sigma_e^2)^2} \left[\widehat{e}^{[t]'} \widehat{e}^{[t]} + \text{tr} \left[[X, Z] C^{-1[t]} [X, Z]' \right] \sigma_e^{2[t]} \right]. \end{aligned}$$

Setting these equations equal to zero yields the iterative system

$$\sigma_a^{2[t+1]} = \frac{\widehat{a}^{[t]'} A^{-1} \widehat{a}^{[t]} + \text{tr} \left(A^{-1} C^{22[t]} \right) \sigma_e^{2[t]}}{q}, \quad (3.108)$$

$$\sigma_e^{2[t+1]} = \frac{\widehat{e}^{[t]'} \widehat{e}^{[t]} + \text{tr} \left[[X, Z] C^{-1[t]} [X, Z]' \right] \sigma_e^{2[t]}}{n}. \quad (3.109)$$

Contrary to ML estimation, REML estimation or inference via the posterior mode of $[\sigma_a^2, \sigma_e^2 | y]$ requires inverting the entire coefficient matrix C (in each iteration).

Example: Bivariate Normal Model with Missing Records

The example on page 67 is revisited and maximum likelihood inferences are drawn using the EM algorithm. The data consist of records of fathers and of their sons, with a number of the latter missing. As was discussed in connection with the example, the construction of the likelihood does not require incorporation of the missing data mechanism for correct inferences. The ordered data are represented as

$$x_1, x_2, \dots, x_m, x_{m+1}, \dots, x_n,$$

$$y_1, y_2, \dots, y_m.$$

To derive the EM equations, an obvious choice for the missing data is $y_* = y_{m+1}, \dots, y_n$. Then the complete data loglikelihood, ignoring an additive constant is

$$l(\mu_y, \mu_x, \rho, \sigma_{yy}, \sigma_{xx} | x, y, y_*) = -\frac{n}{2} \left[\ln \sigma_{yy} + \ln \sigma_{xx} + \ln (1 - \rho^2) \right]$$

$$\begin{aligned}
& -\frac{1}{2(1-\rho^2)} \left[\frac{\sum_{i=1}^n (x_i - \mu_x)^2}{\sigma_{xx}} + \frac{\sum_{i=1}^n (y_i - \mu_y)^2}{\sigma_{yy}} \right. \\
& \quad \left. - 2\rho \frac{\sum_{i=1}^n (x_i - \mu_x)(y_i - \mu_y)}{(\sigma_{xx})^{\frac{1}{2}} (\sigma_{yy})^{\frac{1}{2}}} \right] \quad (3.110)
\end{aligned}$$

As mentioned before, if $y_{m+1}, \dots, y_n$ were observed, the ML estimators have a simple closed form.

Expanding the quadratic forms in (3.110),

$$\begin{aligned}
l(\mu_y, \mu_x, \rho, \sigma_{yy}, \sigma_{xx} | x, y, y_*) &= -\frac{n}{2} \left[\ln \sigma_{yy} + \ln \sigma_{xx} + \ln(1 - \rho^2) \right] \\
&\quad - \frac{1}{2(1 - \rho^2)} \\
&\quad \left(\frac{\sum_{i=1}^n x_i^2 + n\mu_x^2 - 2\mu_x \sum_{i=1}^n x_i}{\sigma_{xx}} + \frac{\sum_{i=1}^n y_i^2 + n\mu_y^2 - 2\mu_y \sum_{i=1}^n y_i}{\sigma_{yy}} \right. \\
&\quad \left. - 2\rho \frac{\sum_{i=1}^n x_i y_i - \mu_x \sum_{i=1}^n y_i - \mu_y \sum_{i=1}^n x_i + n\mu_x \mu_y}{(\sigma_{xx})^{\frac{1}{2}} (\sigma_{yy})^{\frac{1}{2}}} \right). \quad (3.111)
\end{aligned}$$

In this expression the following equalities hold:

$$\begin{aligned}
\sum_{i=1}^n y_i^2 &= \sum_{i=1}^m y_i^2 + \sum_{i=m+1}^n y_{*,i}^2, \\
\sum_{i=1}^n x_i y_i &= \sum_{i=1}^m x_i y_i + \sum_{i=m+1}^n x_i y_{*,i}, \\
\sum_{i=1}^n y_i &= \sum_{i=1}^m y_i + \sum_{i=m+1}^n y_{*,i},
\end{aligned} \quad (3.112)$$

where $y_{*,i}$ is the i th missing observation. The computation of $Q(\theta | \theta^{[t]})$ requires the expectations of (3.111) with respect to the distribution of the missing data, given the parameters evaluated at round $[t]$ and given the observed data (x, y) . Thus,

$$Q(\theta | \theta^{[t]}) = \int l(\mu_y, \mu_x, \rho, \sigma_{yy}, \sigma_{xx} | x, y, y_*) p(y_* | \theta^{[t]}, x, y) dy_*.$$

This calculation involves expectations of (3.112):

$$\begin{aligned} \mathbb{E} \left[\sum_{i=1}^n y_i^2 | \theta^{[t]}, x, y \right] &= \sum_{i=1}^m y_i^2 + \mathbb{E} \left[\sum_{i=m+1}^n y_{*,i}^2 | \theta^{[t]}, x, y \right], \\ \mathbb{E} \left[\sum_{i=1}^n x_i y_i | \theta^{[t]}, x, y \right] &= \sum_{i=1}^m x_i y_i + \mathbb{E} \left[\sum_{i=m+1}^n x_i y_{*,i} | \theta^{[t]}, x, y \right], \\ \mathbb{E} \left[\sum_{i=1}^n y_i | \theta^{[t]}, x, y \right] &= \sum_{i=1}^m y_i + \mathbb{E} \left[\sum_{i=m+1}^n y_{*,i} | \theta^{[t]}, x, y \right]. \end{aligned}$$

Using properties of the bivariate normal distribution, the following results are easily derived:

$$\begin{aligned} \mathbb{E} (y_{*,i} | \theta^{[t]}, x, y) &= \mathbb{E} (y_{*,i} | \theta^{[t]}, x_i) \\ &= \mu_y^{[t]} + \frac{\sigma_{xy}^{[t]}}{\sigma_{xx}^{[t]}} (x_i - \mu_x^{[t]}) \\ &= \mu_y^{[t]} + \rho^{[t]} \frac{[\sigma_{yy}^{[t]}]^{1/2}}{[\sigma_{xx}^{[t]}]^{1/2}} (x_i - \mu_x^{[t]}), \\ \mathbb{E} (x_i y_{*,i} | \theta^{[t]}, x, y) &= x_i \mathbb{E} (y_{*,i} | \theta^{[t]}, x, y), \end{aligned}$$

and

$$\begin{aligned} \mathbb{E} (y_{*,i}^2 | \theta^{[t]}, x, y) &= \sigma_{yy}^{[t]} - \frac{(\sigma_{xy}^{[t]})^2}{\sigma_{xx}^{[t]}} + \left[\mathbb{E} (y_{*,i} | \theta^{[t]}, x, y) \right]^2 \\ &= \sigma_{yy}^{[t]} \left(1 - (\rho^{[t]})^2 \right) + \left[\mathbb{E} (y_{*,i} | \theta^{[t]}, x, y) \right]^2. \end{aligned}$$

Then the E step is

$$\begin{aligned} Q(\theta | \theta^{[t]}) &= -\frac{n}{2} \left[\ln \sigma_{yy} + \ln \sigma_{xx} + \ln (1 - \rho^2) \right] \\ &\quad - \frac{1}{2(1 - \rho^2)} \left(\frac{\sum_{i=1}^n x_i^2 + n\mu_x^2 - 2\mu_x \sum_{i=1}^n x_i}{\sigma_{xx}} \right. \\ &\quad \left. + \frac{\mathbb{E} [\sum_{i=1}^n y_i^2 | \theta^{[t]}, x, y] + n\mu_y^2 - 2\mu_y \mathbb{E} [\sum_{i=1}^n y_i | \theta^{[t]}, x, y]}{\sigma_{yy}} \right) \end{aligned}$$

$$-2\rho \frac{\mathbb{E}[\sum_{i=1}^n x_i y_i | \theta^{[t]}, x, y] - \mu_x \mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y] - \mu_y \sum_{i=1}^n x_i + n\mu_x \mu_y}{(\sigma_{xx})^{\frac{1}{2}} (\sigma_{yy})^{\frac{1}{2}}}, \quad (3.113)$$

and the M step consists of finding the value of $\theta = (\mu_y, \mu_x, \rho, \sigma_{yy}, \sigma_{xx})$ that maximises (3.113); this maximiser becomes $\theta^{[t+1]}$. To find this maximiser first derivatives are needed:

$$\frac{\partial Q(\theta | \theta^{[t]})}{\partial \mu_x} = \frac{n}{(\sigma_{xx})^{\frac{1}{2}} (1 - \rho^2)} \left[\frac{\frac{\sum_{i=1}^n x_i}{n} - \mu_x}{(\sigma_{xx})^{\frac{1}{2}}} - \rho \frac{\frac{\mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y]}{n} - \mu_y}{(\sigma_{yy})^{\frac{1}{2}}} \right], \quad (3.114)$$

$$\frac{\partial Q(\theta | \theta^{[t]})}{\partial \mu_y} = \frac{n}{(\sigma_{yy})^{\frac{1}{2}} (1 - \rho^2)} \left[\frac{\frac{\mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y]}{n} - \mu_y}{(\sigma_{yy})^{\frac{1}{2}}} - \rho \frac{\frac{\sum_{i=1}^n x_i}{n} - \mu_x}{(\sigma_{xx})^{\frac{1}{2}}} \right], \quad (3.115)$$

$$\begin{aligned} \frac{\partial Q(\theta | \theta^{[t]})}{\partial \sigma_{xx}} &= -\frac{1}{2\sigma_{xx} (1 - \rho^2)} \left[n(1 - \rho^2) - \frac{\sum_{i=1}^n (x_i - \mu_x)^2}{\sigma_{xx}} \right. \\ &\quad \left. + \rho \frac{\mathbb{E}[\sum_{i=1}^n x_i y_i | \theta^{[t]}, x, y] - \mu_x \mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y] - \mu_y \sum_{i=1}^n x_i + n\mu_x \mu_y}{(\sigma_{xx})^{\frac{1}{2}} (\sigma_{yy})^{\frac{1}{2}}} \right], \end{aligned} \quad (3.116)$$

$$\begin{aligned} \frac{\partial Q(\theta | \theta^{[t]})}{\partial \sigma_{yy}} &= -\frac{1}{2\sigma_{yy} (1 - \rho^2)} \left[n(1 - \rho^2) \right. \\ &\quad \left. - \frac{\mathbb{E}[\sum_{i=1}^n y_i^2 | \theta^{[t]}, x, y] + n\mu_y^2 - 2\mu_y \mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y]}{\sigma_{yy}} \right. \\ &\quad \left. + \rho \frac{\mathbb{E}[\sum_{i=1}^n x_i y_i | \theta^{[t]}, x, y] - \mu_x \mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y] - \mu_y \sum_{i=1}^n x_i + n\mu_x \mu_y}{(\sigma_{xx})^{\frac{1}{2}} (\sigma_{yy})^{\frac{1}{2}}} \right] \end{aligned} \quad (3.117)$$

and

$$\begin{aligned} \frac{\partial Q(\theta | \theta^{[t]})}{\partial \rho} &= \frac{1}{(1 - \rho^2)} \left\{ n\rho - \frac{1}{(1 - \rho^2)} \left[\rho \left(\frac{\sum_{i=1}^n (x_i - \mu_x)^2}{\sigma_{xx}} \right. \right. \right. \\ &\quad \left. \left. + \frac{\mathbb{E}[\sum_{i=1}^n y_i^2 | \theta^{[t]}, x, y] + n\mu_y^2 - 2\mu_y \mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y]}{\sigma_{yy}} \right) \right] - (1 + \rho)^2 \right\} \end{aligned}$$

$$\left. \frac{\mathbb{E} \left[\sum_{i=1}^n x_i y_i | \theta^{[t]}, x, y \right] - \mu_x \mathbb{E} \left[\sum_{i=1}^n y_i | \theta^{[t]}, x, y \right] - \mu_y \sum_{i=1}^n x_i + n \mu_x \mu_y}{(\sigma_{xx})^{\frac{1}{2}} (\sigma_{yy})^{\frac{1}{2}}} \right\}. \quad (3.118)$$

These equations must be solved simultaneously. After setting derivatives equal to zero, Eqs. (3.114) and (3.115) reduce to

$$\frac{\frac{\sum_{i=1}^n x_i}{n} - \mu_x}{(\sigma_{xx})^{\frac{1}{2}}} = \rho \frac{\frac{\mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y]}{n} - \mu_y}{(\sigma_{yy})^{\frac{1}{2}}}$$

and

$$\frac{\frac{\mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y]}{n} - \mu_y}{(\sigma_{yy})^{\frac{1}{2}}} = \rho \frac{\frac{\sum_{i=1}^n x_i}{n} - \mu_x}{(\sigma_{xx})^{\frac{1}{2}}}.$$

The only solution is

$$\hat{\mu}_x^{[t+1]} = \hat{\mu}_x = \frac{\sum_{i=1}^n x_i}{n}, \quad (3.119)$$

$$\hat{\mu}_y^{[t+1]} = \frac{\mathbb{E}[\sum_{i=1}^n y_i | \theta^{[t]}, x, y]}{n}. \quad (3.120)$$

Since x is completely observed, the solution for $\hat{\mu}_x$ is explicit.

Expressions for σ_{xx} , σ_{yy} and ρ involve (3.116), (3.117) and (3.118). A little algebra results in

$$\hat{\sigma}_{xx} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu}_x)^2, \quad (3.121)$$

$$\begin{aligned} \hat{\sigma}_{yy}^{[t+1]} &= \frac{1}{n} \left\{ \mathbb{E} \left[\sum_{i=1}^n y_i^2 | \theta^{[t]}, x, y \right] + n \left(\hat{\mu}_y^{[t+1]} \right)^2 - 2 \hat{\mu}_y^{[t+1]} \mathbb{E} \left[\sum_{i=1}^n y_i | \theta^{[t]}, x, y \right] \right\} \\ &= \frac{1}{n} \mathbb{E} \left[\sum_{i=1}^n y_i^2 | \theta^{[t]}, x, y \right] - \left(\hat{\mu}_y^{[t+1]} \right)^2. \end{aligned} \quad (3.122)$$

and

$$\hat{\rho}^{[t+1]} = \frac{\frac{1}{n} \mathbb{E} \left[\sum_{i=1}^n x_i y_i | \theta^{[t]}, x, y \right] - \hat{\mu}_x \hat{\mu}_y^{[t+1]}}{(\hat{\sigma}_{xx})^{\frac{1}{2}} (\hat{\sigma}_{yy}^{[t+1]})^{\frac{1}{2}}}. \quad (3.123)$$

The EM iterations involve computation of $\hat{\mu}_x$ using (3.119) and of $\hat{\sigma}_{xx}$ using (3.121) and then looping over (3.120), (3.122) and (3.123) to update $\hat{\mu}_y^{[t+1]}$, $\hat{\sigma}_{yy}^{[t+1]}$ and $\hat{\rho}^{[t+1]}$.

Example: A Two-Component Mixture Model

Finite mixtures constitute a very flexible modelling tool that arise in practice when measurements are assumed to be drawn from several subpopulations or mixture components, where the component to which an observation belongs is not identified. Mixture models have a long history; perhaps the first major analysis involving the use of mixtures was undertaken in 1894 by Karl Pearson (Pearson 1894) that fitted a mixture of two normal probability densities with different means and variances to data on crabs.

This example also assumes that the data are realisations from two subpopulations. A classical scenario in genomics is to decide which of the thousands or millions of genetic markers scattered along the genome has or has not an effect on a particular trait. The trait could have a binary expression, such as presence or absence of a disease, or it could be continuously distributed, such as height in humans. The model poses that each genetic marker effect (an unobserved quantity) is drawn from either a component that generates very small values or from the component that allows for larger values. The objective is to allocate each marker to one of the two components. In the example discussed here the mixture operates at the level of the observed data.

Let z_i be the (unobserved) binary variable that assigns an observation y_i to a specific mixture component. Assume the marginal probability that Z_i is a draw from the mixture component j is

$$\Pr(Z_i = j | \pi_j) = \pi_j.$$

Conditional on $z_i = j$, $j = 1, 0$, assume y_i has density $p_j(y_i | \theta_j)$. That is

$$\begin{aligned} p(y_i, z_i | \theta, \pi) &= p_j(y_i | z_i = j, \theta_j) \Pr(Z_i = j | \pi_j) \\ &= p_j(y_i | z_i = j, \theta_j) \pi_j. \end{aligned}$$

Treating z as missing data, let $x = (z, y)$ denote the complete data. The complete data loglikelihood is

$$\ell(\theta|x) = \sum_i \ell(\theta|x_i),$$

where $\ell(\theta|x_i)$ is the loglikelihood of the i th complete datum that takes the form

$$\ell(\theta|x_i) = \sum_{j=0}^1 [I(z_i = j) \ln p_j(y_i|\theta_j) + I(z_i = j) \ln \pi_j]. \quad (3.124)$$

E-Step

The E-step for the i th observation consists of averaging (3.124) over the conditional distribution $[z_i|\pi^{[t]}, \theta^{[t]}, y]$. The contribution from the i th complete datum is

$$Q_i(\pi, \theta, \pi^{[t]}, \theta^{[t]}) = E \left\{ \sum_{j=0}^1 [I(z_i = j) \ln p_j(y_i|\theta_j) + I(z_i = j) \ln \pi_j] \right\} \quad (3.125)$$

and the contribution from the entire data,

$$\begin{aligned} Q(\pi, \theta, \pi^{[t]}, \theta^{[t]}) &= E \left\{ \sum_{i=1}^n \sum_{j=0}^1 \{I(z_i = j) [\ln p_j(y_i|\theta_j) + \ln \pi_j]\} \right\} \\ &= \sum_{i=1}^n \sum_{j=0}^1 \left\{ E \left[I(z_i = j) | \pi^{[t]}, \theta^{[t]}, y \right] \right. \\ &\quad \left. [\ln p_j(y_i|\theta_j) + \ln \pi_j] \right\} \\ &= \sum_{i=1}^n \sum_{j=0}^1 \{ \hat{p}_{ij} [\ln p_j(y_i|\theta_j) + \ln \pi_j] \}, \end{aligned} \quad (3.126)$$

where the term in squared brackets $[\ln p_j(y_i|\theta_j) + \ln \pi_j]$ is a constant with respect to $[z_i|\pi^{[t]}, \theta^{[t]}, y]$ and

$$\begin{aligned} \hat{p}_{ij} &= E \left[I(Z_i = j) | \pi^{[t]}, \theta^{[t]}, y \right] \\ &= \Pr(Z_i = j | \pi^{[t]}, \theta^{[t]}, y) \end{aligned} \quad (3.127)$$

to be derived shortly. At this point note that $\sum_j \pi_j = 1$, $\sum_j \hat{p}_{ij} = 1$ and therefore $\sum_{i=1}^n \sum_j \hat{p}_{ij} = \sum_{i=1}^n 1 = n$.

M-Step

The Q function must be maximised with respect to the two sets of parameters, the θ_j 's and π_j . The derivation of π_j requires a constrained maximisation using Lagrange multipliers. Maximising (3.126) with respect to π_j subject to $\sum_j \pi_j = 1$ gives

$$\frac{\partial}{\partial \pi_j} \left[\sum_{i=1}^n \sum_{j=0}^1 \hat{p}_{ij} [\ln p_j(y_i|\theta_j) + \ln \pi_j] + \lambda \left(\sum_j \pi_j - 1 \right) \right] = 0$$

or

$$\frac{1}{\pi_j} \sum_{i=1}^n \hat{p}_{ij} + \lambda = 0.$$

Therefore,

$$-\frac{1}{\lambda} \sum_{i=1}^n \hat{p}_{ij} = \pi_j. \quad (3.128)$$

Summing over j on both sides

$$-\frac{1}{\lambda} \sum_{i=1}^n \sum_{j=0}^1 \hat{p}_{ij} = \sum_{j=0}^1 \pi_j$$

leading to

$$-\frac{1}{\lambda} \sum_{i=1}^n 1 = 1$$

which gives $\lambda = -n$. Substituting in (3.128) leads to the iterate

$$\pi_j^{[t+1]} = \frac{1}{n} \sum_{i=1}^n \hat{p}_{ij}. \quad (3.129)$$

The derivation of θ_j requires maximisation of (3.126) with respect to θ_j :

$$\frac{\partial}{\partial \theta_j} \left[\sum_{i=1}^n \sum_{j=0}^1 \hat{p}_{ij} [\ln p_j(y_i|\theta_j) + \ln \pi_j] \right] = 0$$

or

$$\sum_{i=1}^n \hat{p}_{ij} \frac{\partial}{\partial \theta_j} \ln p_j(y_i|\theta_j) = 0. \quad (3.130)$$

To be specific, assume that $p_j(y_i|\theta_j) = N(y_i|\theta_j, \sigma^2)$ and that observations are identically and independently distributed. Then ignoring terms that do not contain θ_j ,

$$\begin{aligned} \sum_{i=1}^n \hat{p}_{ij} \frac{\partial}{\partial \theta_j} \ln p_j(y_i|\theta_j) &= \sum_{i=1}^n \hat{p}_{ij} \frac{\partial}{\partial \theta_j} \left[-\frac{(y_i - \theta_j)^2}{2\sigma^2} \right] \\ &= \sum_{i=1}^n \hat{p}_{ij} \frac{(y_i - \theta_j)}{\sigma^2} = 0. \end{aligned}$$

Multiplying out by σ^2 yields

$$\theta_j^{[t+1]} = \frac{\sum_{i=1}^n \hat{p}_{ij} y_i}{\sum_{i=1}^n \hat{p}_{ij}}. \quad (3.131)$$

Similarly, for σ^2

$$\begin{aligned} \frac{\partial Q}{\partial \sigma^2} &= \sum_{i=1}^n \sum_{j=0}^1 \hat{p}_{ij} \frac{\partial}{\partial \sigma^2} \left[-\frac{1}{2} \ln \sigma^2 - \frac{(y_i - \theta_j)^2}{2\sigma^2} \right] \\ &= \sum_{i=1}^n \sum_{j=0}^1 \hat{p}_{ij} \left[-\frac{1}{2\sigma^2} + \frac{(y_i - \theta_j)^2}{2(\sigma^2)^2} \right] = 0. \end{aligned}$$

Multiplying out by $2\sigma^2$,

$$\sum_{i=1}^n \sum_{j=0}^1 \left[-\hat{p}_{ij} + \frac{\hat{p}_{ij} (y_i - \theta_j)^2}{\sigma^2} \right] = 0.$$

Then

$$\sum_{i=1}^n \sum_{j=0}^1 \frac{\hat{p}_{ij} (y_i - \theta_j)^2}{\sigma^2} = \sum_{i=1}^n \sum_{j=0}^1 \hat{p}_{ij} = \sum_{i=1}^n 1 = n$$

and finally

$$\sigma^{2[t+1]} = \frac{1}{n} \sum_{i=1}^n \sum_{j=0}^1 \hat{p}_{ij} (y_i - \theta_j)^2. \quad (3.132)$$

An expression for $\hat{p}_{ij}$ defined in (3.127) is obtained as follows:

$$\begin{aligned}
 \hat{p}_{ij}^{[t+1]} &= \Pr \left(Z_i = j | \pi^{[t]}, \theta_j^{[t]}, y_i \right) \\
 &= \frac{p_j \left(y_i, Z_i = j | \theta_j^{[t]}, \pi^{[t]} \right)}{\sum_j p_j \left(y_i, Z_i = j | \theta_j^{[t]}, \pi^{[t]} \right)} \\
 &= \frac{p_j \left(y_i | \theta_j^{[t]} \right) \Pr \left(Z_i = j | \pi^{[t]} \right)}{\sum_j p_j \left(y_i | \theta_j^{[t]} \right) \Pr \left(Z_i = j | \pi^{[t]} \right)} \\
 &= \frac{p_j \left(y_i | \theta_j^{[t]} \right) \pi_j^{[t]}}{\sum_j p_j \left(y_i | \theta_j^{[t]} \right) \pi_j^{[t]}}. \tag{3.133}
 \end{aligned}$$

A Bayesian implementation of this mixture model is on page 258 and examples using likelihood and Bayesian models are illustrated on pages 383 and 386.

Example: Genomic Model

This model was implemented using Newton-Raphson and here the EM algorithm is used instead. The focus of inference are the variance components with a view to learning how much of the total variance of the trait is explained by the regression on marker information. Full details of the hierarchical model and of the notation used below are in the example on page 97.

If

$$\begin{aligned}
 y|U, \alpha, \sigma_e^2 &\sim N \left(U\alpha, I\sigma_e^2 \right), \\
 \alpha|U, \sigma_g^2 &\sim SN \left(0, D\sigma_g^2 \right),
 \end{aligned}$$

then the distribution $[\alpha|y, \sigma_e^2, \sigma_g^2]$ is singular normal, with conditional mean

$$\begin{aligned}
 E \left(\alpha | y, \sigma_e^2, \sigma_g^2 \right) &= \tilde{\alpha} = \sigma_g^2 D U' \left(U D U' \sigma_g^2 + I \sigma_e^2 \right)^{-1} y \\
 &= k D (Dk + I)^{-1} \tilde{y} \tag{3.134}
 \end{aligned}$$

where $\tilde{y} = U'y$ and $k = \sigma_g^2/\sigma_e^2$. The conditional variance is

$$\begin{aligned}\text{Var}\left(\alpha|y, \sigma_e^2, \sigma_g^2\right) &= \tilde{V}_\alpha = D\sigma_g^2 - \sigma_g^2 D U' \left(U D U' \sigma_g^2 + I \sigma_e^2 \right)^{-1} U D \sigma_g^2 \\ &= D\sigma_g^2 - \sigma_g^2 k D (Dk + I)^{-1} D.\end{aligned}\quad (3.135)$$

Both (3.134) and (3.135) have a diagonal structure. Simple operations on these expressions reveal that for the i th element,

$$\text{E}\left(\alpha_i|y, \sigma_e^2, \sigma_g^2\right) = \tilde{\alpha}_i = \frac{\lambda_i k}{\lambda_i k + 1} \tilde{y}_i, \quad (3.136a)$$

$$\text{Var}\left(\alpha_i|y, \sigma_e^2, \sigma_g^2\right) = \tilde{V}_{\alpha_i} = \sigma_g^2 \frac{\lambda_i}{\lambda_i k + 1}, \quad i = 1, \dots, n, \quad (3.136b)$$

and when $i = n$ both terms are equal to zero.

The joint density of y and α , given the variance components is

$$\begin{aligned}p\left(y|\alpha, \sigma_e^2\right) p\left(\alpha|\sigma_g^2\right) &= \left|I\sigma_e^2\right|^{-\frac{1}{2}} \exp\left[-\frac{1}{2\sigma_e^2} (y - U\alpha)' (y - U\alpha)\right] \left(\sigma_g^2\right)^{-\left(\frac{n-1}{2}\right)} \\ &\quad \exp\left[-\frac{1}{2\sigma_g^2} \alpha' D^- \alpha\right].\end{aligned}\quad (3.137)$$

The full data likelihood (where α acts as the missing data) is proportional to (3.137) and the full data loglikelihood, up to an additive constant is given by

$$\begin{aligned}\ell\left(\sigma_e^2, \sigma_g^2, \alpha|y\right) &= -\frac{n}{2} \ln \sigma_e^2 - \left(\frac{n-1}{2}\right) \sigma_g^2 - \frac{1}{2\sigma_e^2} (y - U\alpha)' (y - U\alpha) - \\ &\quad - \frac{1}{2\sigma_g^2} \alpha' D^- \alpha.\end{aligned}\quad (3.138)$$

The Q function that defines the E-step of the algorithm is

$$\begin{aligned}Q\left(\sigma_e^2, \sigma_g^2|\sigma_e^{2[l]}, \sigma_g^{2[l]}\right) &= \\ -\frac{n}{2} \ln \sigma_e^2 - \left(\frac{n-1}{2}\right) \sigma_g^2 - \frac{1}{2\sigma_e^2} \text{E}\left[(y - U\alpha)' (y - U\alpha)\right] - \frac{1}{2\sigma_g^2} \text{E}\left[\alpha' D^- \alpha\right]\end{aligned}\quad (3.139)$$

Using results from expectations of quadratic forms (see Note at the end of the example) and the notation defined in (3.134) and (3.135) yields

$$\begin{aligned} Q\left(\sigma_e^2, \sigma_g^2 | \sigma_e^{2[t]}, \sigma_g^{2[t]}\right) &= -\frac{n}{2} \ln \sigma_e^2 - \left(\frac{n-1}{2}\right) \sigma_g^2 - \\ &\quad - \frac{1}{2\sigma_e^2} \left[(y - U\tilde{\alpha}^{[t]})' (y - U\tilde{\alpha}^{[t]}) + \text{tr}(\tilde{V}_\alpha^{[t]}) \right] \\ &\quad - \frac{1}{2\sigma_g^2} \left[\tilde{\alpha}^{[t]'} D^- \tilde{\alpha}^{[t]} + \text{tr}(D^- \tilde{V}_\alpha^{[t]}) \right]. \end{aligned} \quad (3.140)$$

Differentiation with respect to σ_e^2 and σ_g^2 constitute the M-step. This gives

$$\begin{aligned} \frac{\partial}{\partial \sigma_e^2} Q\left(\sigma_e^2, \sigma_g^2 | \sigma_e^{2[t]}, \sigma_g^{2[t]}\right) &= -\frac{n}{2\sigma_e^2} + \frac{1}{2(\sigma_e^2)^2} \left[(y - U\tilde{\alpha}^{[t]})' (y - U\tilde{\alpha}^{[t]}) + \text{tr}(\tilde{V}_\alpha^{[t]}) \right], \\ \frac{\partial}{\partial \sigma_g^2} Q\left(\sigma_e^2, \sigma_g^2 | \sigma_e^{2[t]}, \sigma_g^{2[t]}\right) &= -\frac{n-1}{2\sigma_g^2} + \frac{1}{2(\sigma_g^2)^2} \left[\tilde{\alpha}^{[t]'} D^- \tilde{\alpha}^{[t]} + \text{tr}(D^- \tilde{V}_\alpha^{[t]}) \right]. \end{aligned}$$

Setting these two equations equal to zero leads to the EM iterative system

$$\sigma_g^{2[t+1]} = \frac{\tilde{\alpha}^{[t]'} D^- \tilde{\alpha}^{[t]} + \text{tr}(D^- \tilde{V}_\alpha^{[t]})}{n-1}, \quad (3.141a)$$

$$\sigma_e^{2[t+1]} = \frac{(y - U\tilde{\alpha}^{[t]})' (y - U\tilde{\alpha}^{[t]}) + \text{tr}(\tilde{V}_\alpha^{[t]})}{n}. \quad (3.141b)$$

This system is computationally easy to implement because all operations involve diagonal matrices. Specifically,

$$\begin{aligned} \tilde{\alpha}^{[t]'} D^- \tilde{\alpha}^{[t]} &= k^{2[t]} \sum_{i=1}^n \tilde{y}_i^2 \frac{\lambda_i}{(\lambda_i k^{[t]} + 1)^2}, \\ \text{tr}(D^- \tilde{V}_\alpha^{[t]}) &= \sigma_g^{2[t]} \sum_{i=1}^n \frac{1}{\lambda_i k^{[t]} + 1} I(\lambda_i \neq 0), \end{aligned}$$

$$\begin{aligned} \left(y - U\tilde{\alpha}^{[t]} \right)' \left(y - U\tilde{\alpha}^{[t]} \right) &= \sum_{i=1}^n y_i^2 - 2 \sum_{i=1}^n \tilde{\alpha}_i^{[t]} \tilde{y}_i + \sum_{i=1}^n \tilde{\alpha}_i^{2[t]}, \\ \text{tr} \left(\tilde{V}_\alpha^{[t]} \right) &= \sigma_g^{2[t]} \sum_{i=1}^n \frac{\lambda_i}{\lambda_i k^{[t]} + 1}, \end{aligned}$$

where $\tilde{\alpha}_i^{[t]}$ is computed using (3.136a). The iterative system consists of cycling through the loop

1. Compute expectations (3.136)
2. Compute (3.141) and return to 1.

Note

The step from (3.139) to (3.140) requires knowledge of expectations of quadratic forms. The standard result is that if the vector x has mean μ and variance V , then

$$\mathbb{E} \left(x' A x \right) = \mu' A \mu + \text{tr} (A V). \quad (3.142)$$

We need

$$\mathbb{E} \left[(y - U\alpha)' (y - U\alpha) \mid y, \sigma_e^{2[t]}, \sigma_g^{2[t]} \right].$$

Now,

$$\mathbb{E} \left((y - U\alpha) \mid y, \sigma_e^{2[t]}, \sigma_g^{2[t]} \right) = y - U\tilde{\alpha}$$

and

$$\text{Var} \left((y - U\alpha) \mid y, \sigma_e^{2[t]}, \sigma_g^{2[t]} \right) = U \tilde{V}_\alpha^{[t]} U'.$$

Then using (3.142) results in

$$\mathbb{E} \left[(y - U\alpha)' (y - U\alpha) \mid y, \sigma_e^{2[t]}, \sigma_g^{2[t]} \right] = \left(y - U\tilde{\alpha}^{[t]} \right)' \left(y - U\tilde{\alpha}^{[t]} \right) + \text{tr} \left(\tilde{V}_\alpha^{[t]} \right),$$

where, for the second term in the right hand side, I used

$$\text{tr} \left(U \tilde{V}_\alpha^{[t]} U' \right) = \text{tr} \left(U' U \tilde{V}_\alpha^{[t]} \right) = \text{tr} \left(\tilde{V}_\alpha^{[t]} \right).$$

Finally, direct application of (3.142) gives

$$\mathbb{E} \left[(\alpha' D^{-1} \alpha) | y, \sigma_e^{2[l]}, \sigma_g^{2[l]} \right] = \tilde{\alpha}^{[l]'} D^{-1} \tilde{\alpha}^{[l]} + \text{tr} \left(D^{-1} \tilde{V}_\alpha^{[l]} \right).$$

Example: Likelihood Inferences with Truncated Data and Using the Method of Moments

The likelihood function for truncated data was introduced in the previous chapter on page 76. Here I indicate how to derive ML estimates of parameters using the EM algorithm. The example concludes with a short outline of estimation using the method of moments.

The focus of inference are the parameters of the original, untruncated distribution, assumed in this example to be the Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$. The information available is as follows: from the original sample of size N (drawn randomly from $\mathcal{N}(\mu, \sigma^2)$), m observations were discarded because they were smaller than an observed threshold C . The observed data consist of n records larger than the threshold C . The complete data z consist of $N = m + n$ records, $z = (y^*, y)$ where y^* are the m unobserved missing records and y are the n observed records.

Let $\theta = (\mu, \sigma^2)$. The complete data loglikelihood excluding an additive constant is

$$\ln p(y^*, y | \theta) = -\frac{1}{2} (m + n) \ln(\sigma^2) - \frac{\sum_{i=1}^m (y_i^* - \mu)^2}{2\sigma^2} - \frac{\sum_{i=1}^n (y_i - \mu)^2}{2\sigma^2}. \quad (3.143)$$

The E-step consists of computing expectations of this complete data loglikelihood, conditional on the current value of θ , $\theta^{[l]}$ (remembering that the random variable is the unobserved missing data):

$$\begin{aligned} Q(\theta | \theta^{[l]}) &= -\frac{1}{2} (m + n) \ln(\sigma^2) \\ &\quad - \frac{\sum_{i=1}^m \mathbb{E}[(y_i^* - \mu)^2 | \theta^{[l]}]}{2\sigma^2} - \frac{\sum_{i=1}^n (y_i - \mu)^2}{2\sigma^2}. \end{aligned} \quad (3.144)$$

The M-step needs $\partial Q(\theta|\theta^{[l]})/\partial\mu$ and $\partial Q(\theta|\theta^{[l]})/\partial\sigma^2$. A little algebra shows that

$$\frac{\partial Q(\theta|\theta^{[l]})}{\partial\mu} = \frac{1}{\sigma^2} \left[m E(y_i^*|\theta^{[l]}) - m\mu + \sum_{i=1}^n (y_i - \mu) \right],$$

$$\frac{\partial Q(\theta|\theta^{[l]})}{\partial\sigma^2} = \frac{1}{2} (\sigma^2)^{-2} \left[\sum_{i=1}^m E[(y_i^* - \mu)^2|\theta^{[l]}] + \sum_{i=1}^n (y_i - \mu)^2 - (m+n) \right].$$

This leads to the iterative system

$$\mu^{[t+1]} = \frac{m E(y_i^*|\theta^{[t]}) + \sum_{i=1}^n y_i}{m+n}, \quad (3.145a)$$

$$\sigma^{2[t+1]} = \frac{\sum_{i=1}^m E[(y_i^* - \mu^{[t+1]})^2|\theta^{[t]}] + \sum_{i=1}^n (y_i - \mu^{[t+1]})^2}{m+n}. \quad (3.145b)$$

Using results from the conditional expectations of truncated normal variables spelled out on page 457, one finds

$$E(y_i^*|\theta^{[t]}) = \mu^{[t]} - \sigma^{[t]} \frac{\phi(c^{[t]})}{\Phi(c^{[t]})},$$

$$E[(y_i^* - \mu)^2|\theta^{[t]}] = m\sigma^{2[t]} \left(1 - c^{[t]} \frac{\phi(c^{[t]})}{\Phi(c^{[t]})} \right),$$

where $\phi(\cdot)$ is the pdf of $\mathcal{N}(0, 1)$ and $c = (C - \mu)/\sigma$. The two expressions above replace the expectations in (3.145).

The R-code below illustrates details of the EM computations. The first part generates $N = 50,000$ records from $\mathcal{N}(\mu = 10, \sigma^2 = 3)$ (the complete data) and those larger than $C = \mu + 1.5\sigma$ are kept, the remaining are discarded. Inferences about the parameters of the original distribution, $\mu = 10$ and $\sigma^2 = 3$, are drawn using the truncated records available.

```
# CODE0307
# EM FOR TRUNCATED DATA; ESTIMATE MEAN OF UNTRUNCATED
# GENERATE Y ~ N(MEAN,VAR)
# TRUNCATE AT T SO THAT Z = Y > T ARE OBSERVED
# Y < T ARE MISSING (KNOWN INFORMATION)
rm(list=ls()) # CLEAR WORKSPACE
set.seed(12371)
nindiv<-50000
mean <- 10
var <- 3
T <- mean + 1.5*sqrt(var)
```

```
# CREATE DATA
y <- rnorm(nindiv,mean,sqrt(var))
z <- y[y>T]
length(z)
```

```
## [1] 3462
```

```
m <- length(y)-length(z)
mean(y)
```

```
## [1] 10.00188
```

```
mean(z)
```

```
## [1] 13.37054
```

```
var(z)
```

```
## [1] 0.4728943
```

```
# START VALUES FOR MEAN (mu) AND VARIANCE (sigmasq)
mu <- 0
sigmasq <- 2
sigma <- sqrt(sigmasq)
iter <- 1000
res <- matrix(data=NA, nrow=iter,ncol=2)
### EM LOOP #####
for (i in 1:iter){
  T_star <- (T-mu)/sigma
  expymiss <- mu - (sigma * dnorm(T_star))/(pnorm((T_star)))
  mu <- (sum(z)+m*expymiss)/length(y)
  e <- z-mu
  sigmasq <- (m*sigmasq*(1-T_star*dnorm(T_star)/pnorm(T_star))+
    sum(e*e))/length(y)
  sigma <- sqrt(sigmasq)
  res[i,] <- c(mu,sigmasq)
}
tail(res)
```

```
##           [,1]      [,2]
## [995,]  9.998747  3.077332
## [996,]  9.998747  3.077332
## [997,]  9.998747  3.077332
## [998,]  9.998747  3.077332
## [999,]  9.998747  3.077332
## [1000,] 9.998747  3.077332
```

```
emmu <- res[iter,1]
emmu
```

```
## [1] 9.998747
```

```
emsigmasq <- res[iter,2]
emsigmasq
```

```
## [1] 3.077332
```

```
emsel <- mu + sigma*dnorm(T_star)/(1-pnorm(T_star))
emsel
```

```
## [1] 13.37243
```

```
i <- dnorm(T_star)/(1-pnorm(T_star))
varsel <- sigmasq*(1-i*(i-T_star))
varsel
```

```
## [1] 0.4649044
```

The ML estimates are $\hat{\mu} = 9.999$ and $\hat{\sigma}^2 = 3.077$, in good agreement with the parameters of the unselected population (10 and 3).

Ignoring the selection process and basing inferences on the likelihood of $Y_i \sim N(\mu, \sigma^2)$, $i = 1, \dots, n$, one obtains

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n y_i = 13.4, \quad (3.146a)$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\mu})^2 = 0.47, \quad (3.146b)$$

quite off the mark from the unselected population parameters. Notwithstanding, in this simple setting (no systematic effects other than a common mean, absence of clustering and variance heterogeneity) (3.146) are consistent estimators of the parameters of the selected population, of which the present data are a random draw. As indicated on page 457,

$$\begin{aligned} E(y|y > C) &= \mu + \sigma \frac{\phi(c)}{1 - \Phi(c)}, \\ \text{Var}(y|y > C) &= \sigma^2 \left[1 - \frac{\phi(c)}{1 - \Phi(c)} \left(\frac{\phi(c)}{1 - \Phi(c)} - c \right) \right]. \end{aligned}$$

Likelihood properties sketched in the previous chapter indicate that MLE of these parameters are

$$\begin{aligned}\widehat{E}(y|y > C) &= \widehat{\mu} + \widehat{\sigma} \frac{\phi(\widehat{c})}{1 - \Phi(\widehat{c})}, \\ \widehat{\text{Var}}(y|y > C) &= \widehat{\sigma}^2 \left[1 - \frac{\phi(\widehat{c})}{1 - \Phi(\widehat{c})} \left(\frac{\phi(\widehat{c})}{1 - \Phi(\widehat{c})} - \widehat{c} \right) \right].\end{aligned}$$

Substituting in these expressions the converged values $\widehat{\mu} = 10.000$ and $\widehat{\sigma}^2 = 3.077$ yields the ML estimates

$$\begin{aligned}\widehat{E}(y|y > C) &= 13.37, \\ \widehat{\text{Var}}(y|y > C) &= 0.46,\end{aligned}$$

in good agreement with (3.146).

Estimation Using the Method of Moments

There is a simple alternative to the EM algorithm using the *method of moments* and properties of the normal distribution. A known truncation point $C = 12.59808$ leads to a “proportion selected” $P = \text{length}(y[y > C])/50,000 = 0.06924$ (see R-code CODE0307 that uses T for the known truncation point). Then the “z-value” corresponding to P is $\text{qnorm}(1-P) = 1.481475$; the pdf of the standard normal at this z-value is $\phi(z) = \text{dnorm}(z) = 0.1331442$ and the “intensity of selection” is (the difference between the mean of the truncated group and the mean of the original population, measured in units of standard deviation) $i = \text{dnorm}(z)/P = 1.922937$.

The method of moments consists of equating sample moments with population moments. The moment estimators of μ and σ^2 are obtained by solving the two linear equations (the right hand sides represent the mean and variance of truncated normal variables derived on page 457)

$$\begin{aligned}\frac{1}{n} \sum_{i=1}^n y_i &= \hat{\mu}_t, \\ \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2 &= \hat{\sigma}_t^2,\end{aligned}$$

where $\mu_t = \mu + i\sigma$ and $\sigma_t^2 = \sigma^2(1 - i(i - z))$. This yields $\hat{\mu} \approx 9.97$ and $\hat{\sigma}^2 \approx 3.13$.

Chapter 4

Bayesian Methods



In classical likelihood an important goal is to learn about a parameter θ regarded as a fixed unknown quantity. This is accomplished collecting data y assumed to be a realisation from a probability model $p(y|\theta)$ indexed by θ . This probability model gives rise to the likelihood, a function of θ conditional on the realised y , from which the maximum likelihood estimate $\hat{\theta}$ is obtained. The ML estimator is a random variable (a function of y), whose distribution is characterised by conceptual replications of y . This distribution describes the (sampling) uncertainty of $\hat{\theta}$ and is typically unknown. It can be approximated using resampling techniques or one can appeal to asymptotic results so, informally,

$$\hat{\theta} \sim N\left(\theta, I(\theta)^{-1}\right), \quad (4.1)$$

where $I(\theta)$ is the observed information, as explained before. Probability statements involving $\hat{\theta}$ (such as confidence intervals for θ) can be retrieved from this sampling distribution.

The Bayesian approach is radically different. First of all, the concept of probability is linked to a person's degree of belief. The goal is again to learn about θ but Bayesians regard θ as a random variable, in the sense that there is uncertainty about the range of values it may take. This uncertainty is characterised by the probability distribution of θ . Before data are available and based on previous knowledge, mechanistic considerations or mathematical convenience, a *prior* probability distribution is elicited. The prior probability distribution has density $p(\theta)$. During the learning process data are collected; these data are regarded as a realisation from a probability model $p(y|\theta)$, with θ random and unknown and y fixed and observed. This is often labelled the likelihood. The final stage of the learning process consists of constructing the posterior probability distribution of θ , given data y , with density $p(\theta|y)$, using $p(\theta)$ and $p(y|\theta)$. This posterior probability distribution of θ is a description of the new (posterior) uncertainty about θ in the light of y . It mirrors how prior beliefs have been modified after observing the

evidence: data y . During this process, the data have remained fixed and the inference is conditional on the particular realisation of these data. Specifically, Bayes theorem takes the form

$$p(\theta|y) = \frac{p(y|\theta) p(\theta)}{p(y)} \propto p(y|\theta) p(\theta) \quad (4.2)$$

The contentious part of the Bayesian approach is the elicitation of the prior probability $p(\theta)$ and how it may affect the conclusions of the analysis.

This chapter provides an introduction to Bayesian analysis mostly through examples. The objective of the first examples is to show the type of inferences that can be drawn (joint inferences, conditional inferences, marginal inferences) when the posterior distribution is known. In this case, inferences can be exact using analytical methods, or can be estimated using Monte Carlo draws from the appropriate distribution. The simple Gaussian probability model $N(\mu, \sigma^2)$ and the Gaussian multiple linear regression model $N(Xb, I\sigma^2)$ are chosen as classical examples where all the necessary distributions can be obtained in closed form.

The normal linear model is heavily used for the analysis of continuous data. The chapter includes an example providing first, a description of classical mixed model results leading to the best linear unbiased estimators (BLUE) of “fixed” effects and best linear unbiased predictors (BLUP) of random effects. These can be obtained solving the so-called mixed model equations (e.g. Henderson et al. 1959). This is followed by a computation of the posterior distribution of location parameters (the “fixed” and “random” effects of the mixed model), conditional on dispersion parameters. The connection between both approaches is highlighted. The fully conditional posterior distributions of location parameters are also derived; these are an essential ingredient for a Gibbs sampling implementation. A further example based on a classical regression model shows how the variance component can be inferred from its marginal posterior distribution. The modal value coincides with the restricted maximum likelihood estimator and the marginal posterior distribution has the form of a restricted likelihood.

The concept of *Bayesian learning* is the subject of two examples taken from Sorensen and Gianola (2002). This is a process by which prior information (with uncertainty described by the prior distribution) is modified by new data (generated with uncertainty under a sampling model), to become a posterior distribution, reflecting posterior uncertainty. If data accrue sequentially, at a particular stage i of the learning process, this posterior distribution acts as prior for stage $i + 1$. A single Bayesian analysis carried out at the end leads to the same inferences as one carried out sequentially. The two examples illustrate this probability driven inductive process for discrete and for continuous data.

Most often the exact form of the posterior distribution is not known or only known up to proportionality. A large number of options are available to arrive at approximate results. After a very brief account of Bayesian asymptotics, the chapter focuses on Markov chain Monte Carlo (MCMC) methods. These are recipes that make use of Markov chain theory to generate approximate draws from

posterior distributions. Using these draws one can obtain Monte Carlo estimates of expectations to construct summaries of the posterior distribution such as the posterior mean, variance, posterior intervals or one can compute a posterior mode. Joint and single site updating of parameters using Metropolis-Hastings are explained and illustrated in the simple setting of the $N(\mu, \sigma^2)$ model. The same model is used to introduce the Gibbs sampler. The chapter concludes with an overview of tools needed for the analysis of the output from a Monte Carlo Markov chain and with a discussion of sensitivity assessment.

An appendix provides the mathematical details of the MCMC machinery.

4.1 Example: Estimating the Mean and Variance of a Normal Distribution

Consider the scalar random variable y that is normally distributed with probability density function

$$p(y_i | \mu, \sigma^2) = (2\pi\sigma^2)^{-\frac{1}{2}} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right),$$

$$\sigma^2 > 0, -\infty < \mu < \infty, -\infty < y_i < \infty.$$

The central limit theorem provides a justification for the wide use of the normal distribution. It states that under very general conditions, the sum or the mean of a set of random variables is approximately normally distributed. Data that can be approximated using a continuous function and that result from the sum of the additive effects of a large number of factors fall into this category. The distribution has two parameters, the mean μ and the variance σ^2 and is denoted $N(\mu, \sigma^2)$.

This and the following example illustrate the type of posterior inferences that can be drawn when the forms of the posterior distributions are known.

Posterior Distribution of μ and σ^2

Consider now data vector $y = \{y_i\}_{i=1}^n$. Direct application of Bayes theorem leads to the posterior density of μ and σ^2

$$p(\mu, \sigma^2 | y) \propto p(\mu, \sigma^2) p(y | \mu, \sigma^2). \quad (4.3)$$

The second term in (4.3), the contribution from the n data points or likelihood is

$$p(y | \mu, \sigma^2) = (2\pi\sigma^2)^{-\frac{n}{2}} \exp\left(-\frac{\sum_{i=1}^n (y_i - \mu)^2}{2\sigma^2}\right)$$

$$\begin{aligned}
&= (2\pi\sigma^2)^{-\frac{n}{2}} \exp\left(-\frac{1}{2\sigma^2} \left[\sum_{i=1}^n (y_i - \bar{y})^2 + n(\bar{y} - \mu)^2 \right]\right) \\
&= (2\pi\sigma^2)^{-\frac{n}{2}} \exp\left(-\frac{1}{2\sigma^2} \left[(n-1)S^2 + n(\bar{y} - \mu)^2 \right]\right), \quad (4.4)
\end{aligned}$$

where

$$\begin{aligned}
S^2 &= \frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n-1}, \\
\bar{y} &= \frac{\sum_{i=1}^n y_i}{n},
\end{aligned}$$

are the sample variance and mean, respectively.

The term $p(\mu, \sigma^2) = p(\mu|\sigma^2) p(\sigma^2)$ is the joint prior density. The choice for the form of this density is dictated by prior knowledge, the nature of the scientific problem and/or by mathematical/computational convenience. The latter is less of an issue if the model is fit with Markov chain Monte Carlo (McMC) methods. For the time being I settle for mathematical convenience and choose a set of conjugate priors. As in Gelman et al. (1995), one possibility is to assume prior distributions

$$[\mu|\sigma^2] \sim N(\mu_0, \sigma^2/n_0), \quad (4.5a)$$

$$[\sigma^2] \sim \chi^{-2}(v_0, S_0^2). \quad (4.5b)$$

The form of (4.5a) indicates an a priori dependence between μ and σ^2 . The parameter n_0 can be regarded as prior sample size. The density is

$$p(\mu|\sigma^2) = \left(2\pi \frac{\sigma^2}{n_0}\right)^{-\frac{1}{2}} \exp\left(-\frac{1}{2} \frac{n_0(\mu - \mu_0)^2}{\sigma^2}\right). \quad (4.6)$$

The prior (4.5b) is a scaled inverted chi-square distribution with scale S_0^2 and degrees of freedom v_0 . The density is

$$p(\sigma^2) = \left(\frac{v_0 S_0^2}{2}\right)^{\frac{v_0}{2}} \left(\Gamma\left(\frac{v_0}{2}\right)\right)^{-1} (\sigma^2)^{-\left(\frac{v_0}{2}+1\right)} \exp\left(-\frac{v_0 S_0^2}{2\sigma^2}\right), \quad v_0, S_0 > 0. \quad (4.7)$$

The term S_0^2 can be regarded as a prior variance component, so that $v_0 S_0^2$ is a prior sum of squares. The density of the joint prior distribution is

$$p(\mu, \sigma^2) \propto (\sigma^2)^{-\frac{1}{2}} (\sigma^2)^{-\left(\frac{v_0}{2}+1\right)} \exp\left(-\frac{1}{2\sigma^2} [v_0 S_0^2 + n_0 (\mu - \mu_0)^2]\right), \quad (4.8)$$

proportional to a normal-inverse-chi-square density. As shown below, the assignment of a normal-inverse-chi-square prior distribution combined with a normal likelihood results in a normal-inverse-chi-square posterior. The property that the posterior distribution follows the same parametric form as the prior distribution is called *conjugacy*.

The joint posterior distribution is obtained replacing (4.4) and (4.8) in (4.3). This gives

$$\begin{aligned} p(\mu, \sigma^2 | y) &\propto (\sigma^2)^{-\frac{n}{2}} \exp\left(-\frac{1}{2\sigma^2} [(n-1) S^2 + n (\bar{y} - \mu)^2]\right) \times \\ &\quad \times (\sigma^2)^{-\frac{1}{2}} (\sigma^2)^{-\left(\frac{v_0}{2}+1\right)} \exp\left(-\frac{1}{2\sigma^2} [v_0 S_0^2 + n_0 (\mu - \mu_0)^2]\right) \\ &= (\sigma^2)^{-\left(\frac{n+v_0+3}{2}\right)} \exp\left(-\frac{1}{2\sigma^2} [v_0 S_0^2 + n_0 (\mu - \mu_0)^2 + (n-1) S^2 + n (\bar{y} - \mu)^2]\right). \end{aligned} \quad (4.9)$$

This expression is proportional to a normal-inverse-chi-square density confirming the conjugacy of the prior distribution.

Conditional Posterior Distribution of μ Given σ^2

The joint posterior distribution $[\mu, \sigma^2 | y]$ can be factorised as follows:

$$p(\mu, \sigma^2 | y) = p(\mu | \sigma^2, y) p(\sigma^2 | y).$$

The densities of these two components are derived in turn. The distribution $[\mu | y, \sigma^2]$ can be obtained either by extracting from (4.9) those terms that are function of μ or by direct use of (4.2)

$$p(\mu | \sigma^2, y) \propto p(\mu | \sigma^2) p(y | \mu, \sigma^2), \quad (4.10)$$

where $p(\mu | \sigma^2)$ is the prior density of the conditional distribution $[\mu | \sigma^2]$. In both cases this leads to (including terms in μ only)

$$p(\mu | \sigma^2, y) \propto \exp\left(-\frac{1}{2\sigma^2} [n_0 (\mu - \mu_0)^2 + n (\bar{y} - \mu)^2]\right).$$

There is an identity for combining quadratic forms (Box and Tiao 1973, pp. 74, 418) stating that

$$A(z-a)^2 + B(z-b)^2 = (A+B)(z-c)^2 + \frac{AB}{A+B}(a-b)^2,$$

$$c = \frac{1}{A+B}(Aa+Bb). \quad (4.11)$$

With

$$\begin{aligned} A &= n_0, \\ B &= n, \\ z &= \mu, \\ a &= \mu_0, \\ b &= \bar{y}, \\ c &= \left(\frac{n_0\mu_0 + n\bar{y}}{n_0 + n} \right), \end{aligned}$$

then

$$\begin{aligned} &\exp\left(-\frac{1}{2\sigma^2}\left[n_0(\mu-\mu_0)^2 + n(\bar{y}-\mu)^2\right]\right) = \\ &= \exp\left(-\frac{1}{2\sigma^2}\left[(n_0+n)\left(\mu - \frac{n_0\mu_0 + n\bar{y}}{n_0+n}\right)^2 + \frac{n_0n}{n_0+n}(\bar{y}-\mu_0)^2\right]\right) \\ &\propto \exp\left(-\frac{n_0+n}{2\sigma^2}\left[\left(\mu - \frac{n_0\mu_0 + n\bar{y}}{n_0+n}\right)^2\right]\right), \end{aligned} \quad (4.12)$$

where the third line includes the only terms that are function of μ . This is the kernel of the normal distribution

$$[\mu|\sigma^2, y] \sim N\left(\mu_1, \sigma_1^2\right). \quad (4.13)$$

where

$$E\left(\mu|\sigma^2, y\right) = \mu_1 = \frac{n_0\mu_0 + n\bar{y}}{n_0 + n} \quad (4.14)$$

is the posterior mean and

$$\text{Var}(\mu | \sigma^2, y) = \sigma_1^2 = \frac{\sigma^2}{n_0 + n} \quad (4.15)$$

is the posterior variance. Dividing numerator and denominator of (4.14) by σ^2 , defining $\pi_0 = n_0/\sigma^2$ as the prior precision, $\pi_n = n/\sigma^2$ as the precision arising from the data, then (4.14) can also be written in the form of the shrunk estimator

$$\mu_1 = \bar{y} - (\bar{y} - \mu_0) \frac{\pi_0}{\pi_0 + \pi_n}. \quad (4.16)$$

Expressions (4.14) and (4.16) express the posterior mean as a compromise between the prior mean μ_0 and the mean of the data $\bar{y}$.

Note

A matrix extension of (4.11), taken from Box and Tiao (1973), p. 418, is as follows:

$$\begin{aligned} & (z - m)' M (z - m) + (z - b)' B (z - b) \\ &= (z - c)' (M + B) (z - c) + (m - b)' M (M + B)^{-1} B (m - b), \end{aligned}$$

with

$$c = (M + B)^{-1} (Mm + Bb).$$

In these expressions, z , m and b are $k \times 1$ vectors and M and B are $k \times k$ symmetric matrices such that $(M + B)^{-1}$ exists.

Marginal Posterior Distribution of σ^2

There are two ways to obtain $p(\sigma^2 | y)$. The **first** is to compute the integral

$$p(\sigma^2 | y) = \int p(\mu, \sigma^2 | y) d\mu$$

where $p(\mu, \sigma^2 | y)$ is given up to proportionality in (4.9). Then

$$\begin{aligned} p(\sigma^2 | y) &\propto \int (\sigma^2)^{-\left(\frac{n+v_0+3}{2}\right)} \exp\left(-\frac{1}{2\sigma^2} \left[v_0 S_0^2 + (n-1) S^2 + n_0 (\mu - \mu_0)^2 + n (\bar{y} - \mu)^2 \right] \right) d\mu = \\ &= (\sigma^2)^{-\left(\frac{n+v_0+3}{2}\right)} \exp\left(-\frac{1}{2\sigma^2} \left[v_0 S_0^2 + (n-1) S^2 \right] \right) \\ &\int \exp\left(-\frac{1}{2\sigma^2} \left[n_0 (\mu - \mu_0)^2 + n (\bar{y} - \mu)^2 \right] \right) d\mu = \end{aligned}$$

$$\begin{aligned}
& \left(\sigma^2\right)^{-\left(\frac{n+v_0+3}{2}\right)} \exp \left(-\frac{1}{2\sigma^2} \left[v_0 S_0^2 + (n-1) S^2 + \frac{n_0 n}{n_0 + n} (\bar{y} - \mu_0)^2 \right] \right) \\
& \int \exp \left(-\frac{n_0 + n}{2\sigma^2} \left[\left(\mu - \frac{n_0 \mu_0 + n \bar{y}}{n_0 + n} \right)^2 \right] \right) d\mu.
\end{aligned} \tag{4.17}$$

The result in the last two lines are derived using equality (4.12) and the expression in the last line has a simple solution since it involves integration over a normal distribution. In general,

$$\int \exp \left(-\frac{(\mu - \mu_1)^2}{2\sigma_1^2} \right) d\mu = (2\pi\sigma_1^2)^{\frac{1}{2}} \propto (\sigma^2)^{\frac{1}{2}}.$$

Substituting in (4.17),

$$\begin{aligned}
p(\sigma^2|y) & \propto (\sigma^2)^{-\left(\frac{n+v_0}{2}+1\right)} \exp \left(-\frac{\left[v_0 S_0^2 + (n-1) S^2 + \frac{n_0 n}{n_0 + n} (\bar{y} - \mu_0)^2 \right]}{2\sigma^2} \right) \\
& = (\sigma^2)^{-\left(\frac{\tilde{v}}{2}+1\right)} \exp \left(-\frac{\tilde{v} \tilde{S}^2}{2\sigma^2} \right),
\end{aligned} \tag{4.18}$$

where

$$\begin{aligned}
\tilde{S}^2 & = \frac{v_0 S_0^2 + (n-1) S^2 + \frac{n_0 n}{n_0 + n} (\bar{y} - \mu_0)^2}{\tilde{v}}, \\
\tilde{v} & = n + v_0.
\end{aligned}$$

This is proportional to the pdf of a scaled inverted chi-square distribution, with $\tilde{v}$ degrees of freedom and scale parameter $\tilde{S}^2$. This is symbolised as

$$[\sigma^2|y] \sim \chi^{-2}(\tilde{v}, \tilde{S}^2). \tag{4.19}$$

For example, to draw samples from (4.19), first draw from a chi-square distribution with $\tilde{v}$ degrees of freedom, invert this number and multiply by

$$\tilde{v} \tilde{S}^2 = v_0 S_0^2 + (n-1) S^2 + \frac{n_0 n}{n_0 + n} (\bar{y} - \mu_0)^2. \tag{4.20}$$

A **second** way of arriving at $p(\sigma^2|y)$ is to use the identity

$$p(\sigma^2|y) = \frac{p(\mu, \sigma^2|y)}{p(\mu|\sigma^2, y)}. \tag{4.21}$$

Using (4.9)

$$\begin{aligned}
 p(\mu, \sigma^2 | y) &\propto (\sigma^2)^{-\left(\frac{n+v_0+3}{2}\right)} \exp\left(-\frac{1}{2\sigma^2} \right. \\
 &\quad \left. \left[v_0 S_0^2 + (n-1) S^2 + n_0 (\mu - \mu_0)^2 + n (\bar{y} - \mu)^2 \right] \right) = \\
 &= (\sigma^2)^{-\left(\frac{n+v_0+3}{2}\right)} \exp\left(-\frac{1}{2\sigma^2} \right. \\
 &\quad \left. \left[v_0 S_0^2 + (n-1) S^2 + (n_0 + n) (\mu - \mu_1)^2 + \frac{n_0 n}{n_0 + n} (\bar{y} - \mu_0)^2 \right] \right) \quad (4.22)
 \end{aligned}$$

and

$$p(\mu | \sigma^2, y) \propto (\sigma^2)^{-\frac{1}{2}} \exp\left(-\frac{n_0 + n}{2\sigma^2} [(\mu - \mu_1)^2]\right).$$

Substituting in (4.21) yields (4.19).

Marginal Posterior Distribution of μ

The distribution $[\mu | y]$ is easily obtained regarding the t -distribution as a mixture of normal distributions with a common mean and variances distributed as scaled inverse chi-square random variables. For example, if $[y_i | V_i] \sim N(\mu, V_i)$ and $V_i \sim \chi^{-2}(v, \sigma^2)$ then $y_i \sim t(v, \mu, \sigma^2)$, a t -distribution with v degrees of freedom, mean μ and scale σ^2 .

In the present case

$$[\mu | \sigma^2, y] \sim N\left(\mu_1, \frac{\sigma^2}{n_0 + n}\right) \quad (4.23)$$

and

$$[\sigma^2 | y] \sim \chi^{-2}(\tilde{v}, \tilde{S}^2). \quad (4.24)$$

Therefore,

$$p(\mu | y) = \int p(\mu | \sigma^2, y) p(\sigma^2 | y) d\sigma^2 \sim t\left(\tilde{v}, \mu_1, \frac{\tilde{S}^2}{n_0 + n}\right), \quad (4.25)$$

a t -distribution with $\tilde{v}$ degrees of freedom, mean μ_1 and scale $\tilde{S}^2/(n_0 + n)$. The density is

$$p(\mu|y) = \frac{\Gamma\left(\frac{\tilde{v}+1}{2}\right)}{\Gamma\left(\frac{\tilde{v}}{2}\right)\sqrt{\pi\tilde{v}}}\left(\frac{\tilde{S}^2}{n_0+n}\right)^{-\frac{1}{2}}\left(1 + \frac{n_0+n}{\tilde{v}\tilde{S}^2}(\mu - \mu_1)^2\right)^{-\left(\frac{\tilde{v}+1}{2}\right)}. \quad (4.26)$$

The mean and mode are given by

$$E(\mu|y) = \mu_1,$$

and the variance is

$$\text{Var}(\mu|y) = \left(\frac{\tilde{S}^2}{n_0+n}\right)\left(\frac{\tilde{v}}{\tilde{v}-2}\right), \quad \tilde{v} > 2.$$

The proof of this result is as follows. Using (4.22) and (4.20) the joint posterior of μ and σ^2 is

$$\begin{aligned} p(\mu, \sigma^2|y) &= \\ &= K(\sigma^2)^{-\left(\frac{n+v_0+3}{2}\right)} \exp\left(-\frac{1}{2\sigma^2}\right. \\ &\quad \left.\left[v_0 S_0^2 + (n-1)S^2 + (n_0+n)(\mu - \mu_1)^2 + \frac{n_0 n}{n_0+n}(\bar{y} - \mu_0)^2\right]\right) \\ &= K(\sigma^2)^{-\left(\frac{\tilde{v}+1}{2}+1\right)} \exp\left(-\frac{1}{2\sigma^2}\left[\tilde{v}\tilde{S}^2 + (n_0+n)(\mu - \mu_1)^2\right]\right) \\ &= K(\sigma^2)^{-\alpha-1} \exp\left(-\frac{1}{2\sigma^2}\left[\tilde{v}\tilde{S}^2 + (n_0+n)(\mu - \mu_1)^2\right]\right) \end{aligned}$$

where K is the constant of integration and $\alpha = \frac{\tilde{v}+1}{2}$. Then

$$p(\mu|y) = \int_0^\infty K(\sigma^2)^{-\alpha-1} \exp\left(-\frac{1}{2\sigma^2}\left[\tilde{v}\tilde{S}^2 + (n_0+n)(\mu - \mu_1)^2\right]\right) d\sigma^2. \quad (4.27)$$

Let

$$\begin{aligned} A &= \tilde{v}\tilde{S}^2 + (n_0 + n)(\mu - \mu_1)^2, \\ x &= \frac{A}{2\sigma^2}, \\ d\sigma^2 &= -\frac{A}{2}x^{-2}dx \end{aligned}$$

Therefore (4.27) can be written as

$$\begin{aligned} p(\mu|y) &= -\left(\frac{A}{2}\right) \int_0^\infty K\left(\frac{A}{2x}\right)^{-\alpha-1} \exp(-x) x^{-2} dx \\ &= -\left(\frac{A}{2}\right) \left(\frac{A}{2}\right)^{-\alpha-1} \int_0^\infty K x^{\alpha-1} \exp(-x) dx \\ &\propto A^{-\alpha} \int_0^\infty x^{\alpha-1} \exp(-x) dx \\ &\propto A^{-\alpha}, \end{aligned}$$

where the last line results from the fact that the integral, known as the Gamma function is

$$\int_0^\infty x^{\alpha-1} \exp(-x) dx = \Gamma(\alpha), \quad \alpha > 0,$$

a constant with respect to μ . Then

$$\begin{aligned} p(\mu|y) &\propto \left(\tilde{v}\tilde{S}^2 + (n_0 + n)(\mu - \mu_1)^2\right)^{-\left(\frac{\tilde{v}+1}{2}\right)} \\ &\propto \left(1 + \frac{(n_0 + n)(\mu - \mu_1)^2}{\tilde{v}\tilde{S}^2}\right)^{-\left(\frac{\tilde{v}+1}{2}\right)}, \end{aligned}$$

which is the kernel of (4.26).

Posterior Predictive Distribution for a New Observation

Anticipating a topic that will be discussed later in the book, I outline the derivation of the posterior predictive distribution for a new to-be-observed observation $\tilde{y}_i$. A natural application of this distribution is in Bayesian prediction problems, discussed in Chap. 10. In the context of the above Gaussian setup, $\tilde{y}_i$, given μ and σ^2 ,

is assumed to be a draw from $N(\mu, \sigma^2)$. The density of the posterior predictive distribution of $\tilde{y}_i$ is

$$\begin{aligned} p(\tilde{y}|y) &= \iint p(\tilde{y}|\mu, \sigma^2, y) p(\mu, \sigma^2|y) d\mu d\sigma^2 \\ &= \iint p(\tilde{y}|\mu, \sigma^2) p(\mu|\sigma^2, y) p(\sigma^2|y) d\mu d\sigma^2. \end{aligned} \quad (4.28)$$

The first term in the integrand of the second line follows from the fact that, given (μ, σ^2) , $\tilde{y}$ is independent of y .

A first integration with respect to μ results in

$$p(\tilde{y}|\sigma^2, y) = \int p(\tilde{y}|\mu, \sigma^2) p(\mu|\sigma^2, y) d\mu. \quad (4.29)$$

Before deriving the density of $[\tilde{y}|\sigma^2, y]$, one can easily derive the conditional mean and variance. First note that a future observation and a data point have the same sampling distribution. That is

$$\tilde{y}|\mu, \sigma^2 \sim N(\mu, \sigma^2).$$

Then the desired mean and variance are

$$\begin{aligned} E[\tilde{y}|\sigma^2, y] &= E_{\mu|\sigma^2, y}[E(\tilde{y}|\mu, \sigma^2)] \\ &= E_{\mu|\sigma^2, y}(\mu) \\ &= \frac{n_0\mu_0 + n\bar{y}}{n_0 + n} = \mu_1 \end{aligned} \quad (4.30)$$

using (4.14) and

$$\begin{aligned} \text{Var}(\tilde{y}|\sigma^2, y) &= \text{Var}_{\mu|\sigma^2, y}[E(\tilde{y}|\mu, \sigma^2)] + E_{\mu|\sigma^2, y}[\text{Var}(\tilde{y}|\mu, \sigma^2)] \\ &= \text{Var}_{\mu|\sigma^2, y}(\mu) + \sigma^2 \\ &= \frac{\sigma^2}{n_0 + n} + \sigma^2 = \sigma_2^2(\sigma^2) \end{aligned} \quad (4.31)$$

using (4.15). The notation makes explicit that σ_2^2 is a function of σ^2 . There are two sources of uncertainty in the posterior predictive distribution. The term σ^2 is the variance of the sampling model. The second represents uncertainty about μ as a result of the finiteness of the sample y . This term vanishes with large n but the first source of predictive uncertainty remains.

The form of the distribution $[\tilde{y}|\sigma^2, y]$ is readily obtained noting that the integrand in (4.29) involves two Gaussian distributions. That is

$$p(\tilde{y}|\sigma^2, y) = \int N(\tilde{y}|\mu, \sigma^2) N(\mu|\mu_1, \sigma_1^2) d\mu$$

Using the identity (4.11) it follows that

$$\tilde{y}|\sigma^2, y \sim N(\mu_1, \sigma_2^2(\sigma^2)). \quad (4.32)$$

To arrive at the desired result we need a second integration

$$p(\tilde{y}|y) = \int p(\tilde{y}|\sigma^2, y) p(\sigma^2|y) d\sigma^2.$$

Using (4.24) and (4.32) this involves

$$p(\tilde{y}|y) = \int N(\tilde{y}|\mu_1, \sigma_2^2(\sigma^2)) \chi^{-2}(\sigma^2|\tilde{v}, \tilde{S}^2) d\sigma^2,$$

the integration of a normal distribution and a scaled inverse chi-square distribution. It follows from the result spelled out in (4.23), that $[\tilde{y}|y]$ has the density of a t -distribution with $\tilde{v}$ degrees of freedom, mean μ_1 and scale $\tilde{S}^2 \left(1 + \frac{1}{n_0+n}\right)$. That is

$$\tilde{y}|y \sim t\left(\tilde{v}, \mu_1, \tilde{S}^2 \left(1 + \frac{1}{n_0+n}\right)\right). \quad (4.33)$$

4.2 Example: The Normal Multiple Regression Model

This example considers a Bayesian analysis of the following Gaussian linear regression model, distinguishing between two sets of coefficients b_1 , $(p_1 \times 1)$ and b_2 , $(p_2 \times 1)$. The $(n \times 1)$ vector of observations y is linked to b_1 and b_2 via the relationship

$$\begin{aligned} y &= X_1 b_1 + X_2 b_2 + e \\ &= [X_1 \quad X_2] \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} + e = Xb + e, \end{aligned} \quad (4.34)$$

where X_1 and X_2 are $(n \times p_1)$ and $(n \times p_2)$ full-rank observed matrices, respectively, matrix X has dimension $(n \times p)$, $p = p_1 + p_2$, the residual term $e \sim$

$N(0, I\sigma^2)$, I is the n -dimensional identity matrix and the scalar σ^2 is an unknown dispersion parameter. The likelihood function is then

$$\begin{aligned} L(b_1, b_2, \sigma^2 | y) &\propto (\sigma^2)^{-\frac{n}{2}} \exp \left[-\frac{(y - Xb)'(y - Xb)}{2\sigma^2} \right] \\ &\propto (\sigma^2)^{-\frac{n}{2}} \exp \left[-\frac{vS^2 + (b - \hat{b})' X' X (b - \hat{b})}{2\sigma^2} \right], \end{aligned} \quad (4.35)$$

where

$$\begin{aligned} v &= n - p_1 - p_2 = n - p, \\ S^2 &= \frac{(y - X\hat{b})'(y - X\hat{b})}{v}, \\ \hat{b} &= (X'X)^{-1} X'y. \end{aligned}$$

Prior Distributions

A specification based on improper uniform priors will be developed in this example. Here, b_1 and b_2 are allowed to vary in the p_1 - and p_2 -dimensional spaces $\mathbb{R}^{p_1}$ and $\mathbb{R}^{p_2}$, respectively, while σ^2 falls between 0 and ∞ . Hence, the joint posterior distribution is proportional to the likelihood function. The joint posterior density can be written

$$p(b_1, b_2, \sigma^2 | y) = \frac{(\sigma^2)^{-\frac{n}{2}} \exp \left[-\frac{vS^2 + (b - \hat{b})' X' X (b - \hat{b})}{2\sigma^2} \right]}{\int_{\mathbb{R}^{p_1}} \int_{\mathbb{R}^{p_2}} \int_0^\infty (\sigma^2)^{-\frac{n}{2}} \exp \left[-\frac{vS^2 + (b - \hat{b})' X' X (b - \hat{b})}{2\sigma^2} \right] db_1 db_2 d\sigma^2}. \quad (4.36)$$

The limits of integration are omitted in the notation hereinafter.

Conditional Posterior Distribution of the Regression Coefficients

This distribution can be found from (4.36) retaining the terms that vary with b . One obtains

$$p(b | \sigma^2, y) \propto \exp \left[-\frac{1}{2\sigma^2} (b - \hat{b})' X' X (b - \hat{b}) \right]$$

that is the kernel of the Gaussian distribution

$$[b | \sigma^2, y] \sim N(\hat{b}, (X'X)^{-1}\sigma^2). \quad (4.37)$$

If, in addition to σ^2 , one treats either b_1 or b_2 as known, the resulting conditional posterior distributions are also normal given by

$$\begin{aligned} [b_1|b_2, \sigma^2, y] &\sim N\left(\tilde{b}_1, (X_1'X_1)^{-1}\sigma^2\right), \\ [b_2|b_1, \sigma^2, y] &\sim N\left(\tilde{b}_2, (X_2'X_2)^{-1}\sigma^2\right), \\ \tilde{b}_i &= (X_i'X_i)^{-1}X_i'(y - X_jb_j), \quad i = 1, 2; i \neq j. \end{aligned}$$

Marginal Distribution of the Dispersion Parameter

Integration of the joint posterior density gives

$$p(\sigma^2|y) = \int p(b, \sigma^2|y) db.$$

From (4.35),

$$\begin{aligned} \int p(b, \sigma^2|y) db &\propto (\sigma^2)^{-\frac{n}{2}} \exp\left[-\frac{vS^2}{2\sigma^2}\right] \\ &\quad \int \exp\left[-\frac{(b - \hat{b})'X'X(b - \hat{b})}{2\sigma^2}\right] db. \end{aligned} \quad (4.38)$$

The integrand is the kernel of the multivariate normal distribution; that is

$$\int \exp\left[-\frac{(b - \hat{b})'X'X(b - \hat{b})}{2\sigma^2}\right] db = (2\pi)^{\frac{p}{2}} |(X'X)^{-1}\sigma^2|^{\frac{1}{2}} \propto (\sigma^2)^{\frac{p}{2}}.$$

Substituting in (4.38):

$$\begin{aligned} \int p(b, \sigma^2|y) db &\propto (\sigma^2)^{-\frac{n}{2}} \exp\left[-\frac{vS^2}{2\sigma^2}\right] (\sigma^2)^{\frac{p}{2}} \\ &\propto (\sigma^2)^{-\left(\frac{n-p-2}{2}+1\right)} \exp\left[-\frac{vS^2}{2\sigma^2}\right] \\ &= (\sigma^2)^{-\left(\frac{\tilde{v}}{2}+1\right)} \exp\left[-\frac{\tilde{v}\tilde{S}^2}{2\sigma^2}\right] \end{aligned} \quad (4.39)$$

where

$$\tilde{v} = n - p - 2,$$

$$\tilde{S}^2 = \frac{(y - X\hat{b})'(y - X\hat{b})}{\tilde{v}}.$$

Expression (4.39) is the kernel of a scale inverted chi-square distribution with $\tilde{v}$ degrees of freedom and scale $\tilde{S}^2$; that is

$$[\sigma^2|y] \sim \chi^{-2}(\tilde{v}, \tilde{S}^2). \quad (4.40)$$

The mean of the distribution is

$$E(\sigma^2|y) = \frac{\tilde{v}\tilde{S}^2}{\tilde{v} - 2} = \frac{vS^2}{v - 4}, \quad v > 4.$$

Notice that $\tilde{v}\tilde{S}^2 = vS^2 = (y - X\hat{b})'(y - X\hat{b})$.

Marginal Distribution of the Regression Coefficients

Integration of the joint posterior density over σ^2 results in

$$p(b|y) = \int p(b|\sigma^2, y) p(\sigma^2|y) dy.$$

The first term in the integrand is the density of the Gaussian distribution $N(\hat{b}, (X'X)^{-1}\sigma^2)$ and the second is the density of the scaled inverted chi-square distribution (4.40). Since the t -distribution can be interpreted as a mixture of normal distributions with common mean and a variance that varies according to a scale inverted chi-square distribution,

$$[b|y] \sim t(\hat{b}, \tilde{v}, \tilde{S}^2(X'X)^{-1}), \quad (4.41)$$

a multivariate t -distribution with mean $\hat{b}$, degrees of freedom $\tilde{v}$ and scale $\tilde{S}^2(X'X)^{-1}$. Any marginal and conditional distribution of the regression coefficients (after integrating σ^2 out) is either univariate or multivariate t . The variance of $[b|y]$ is

$$\begin{aligned} \text{Var}(b|y) &= \frac{\tilde{v}\tilde{S}^2(X'X)^{-1}}{\tilde{v} - 2} \\ &= \frac{vS^2(X'X)^{-1}}{v - 4}, \quad v > 4. \end{aligned} \quad (4.42)$$

Posterior Predictive Distribution of a Future Observation

Assume that an n_f vector of future observations y_f is a draw from the sampling model

$$y_f = X_f b + e_f, \quad e_f \sim N(0, I\sigma^2)$$

with the same set of regression coefficients b as in (4.34). The posterior predictive density of y_f , conditional on the “old” records y , is

$$p(y_f|y) = \int \int p(y_f|b, \sigma^2) p(b|\sigma^2, y) p(\sigma^2|y) db d\sigma^2, \quad (4.43)$$

assuming $p(y_f|b, \sigma^2, y) = p(y_f|b, \sigma^2)$. Before deriving the form of this density one can obtain the mean and variance. These are

$$E(y_f|y) = E_{b, \sigma^2|y} [E(y_f|b, \sigma^2, y)] = X_f \hat{b} \quad (4.44)$$

and

$$\begin{aligned} \text{Var}(y_f|y) &= E_{b, \sigma^2|y} [\text{Var}(y_f|b, \sigma^2, y)] + \text{Var}_{b, \sigma^2|y} [E(y_f|b, \sigma^2, y)] \\ &= E_{\sigma^2|y} (I\sigma^2) + \text{Var}_{b, \sigma^2|y} (X_f b) \\ &= \frac{vS^2}{v-4} + X_f \frac{vS^2 (X'X)^{-1}}{v-4} X_f' \\ &= \frac{vS^2}{v-4} [I + X_f (X'X)^{-1} X_f'], \quad v > 4. \end{aligned} \quad (4.45)$$

A straightforward approach to disclose the form of the posterior predictive distribution $[y_f|y]$, avoiding lengthy algebra, is as follows. Consider the first integration in (4.43):

$$\int p(y_f|b, \sigma^2) p(b|\sigma^2, y) db.$$

The integrand involves two Gaussian densities; therefore the integration results in a Gaussian density. The second integration over $[\sigma^2|y]$ involves a scale inverted chi-square distribution. Invoking again the mixture property alluded to before, it follows that $[y_f|y]$ is in the form of a multivariate t -distribution. The parameters of this t -distribution can be derived from knowledge of its mean and variance, together with the parameters of the intervening scale inverted chi-square. A little reflection indicates that

$$[y_f|y] \sim t(\tilde{v}, X_f \hat{b}, [X_f' (X'X)^{-1} X_f + I] \tilde{S}^2), \quad (4.46)$$

a t -distribution with $\tilde{v} = v - 2$ degrees of freedom, mean $X_f' \hat{b}$ and scale $X_f' (X'X)^{-1} X_f \tilde{S}^2$. The variance in terms of v and S^2 is

$$\text{Var}(y_f|y) = \frac{vS^2}{v-4} [X_f' (X'X)^{-1} X_f + I], \quad v > 4. \quad (4.47)$$

as in (4.45).

4.3 Example: Monte Carlo Inferences of the Joint Posterior Distribution of Mean and Variance

In the example on page 149, the forms of the joint posterior distribution $[\mu, \sigma^2|y]$ and the marginal posterior distributions of $[\mu|y]$ and $[\sigma^2|y]$ are known. Therefore, inferences can be obtained directly using these distributions. These “exact” inferences are compared with those based on Monte Carlo draws from the posterior distribution $[\mu, \sigma^2|y]$.

A simple way of drawing samples from $[\mu, \sigma^2|y]$ is based on the method of composition (Tanner 1996):

1. First draw σ_i^{2*} from $[\sigma^2|y]$
2. Then draw μ_i^* from $[\mu|\sigma_i^{2*}, y]$

Repeating this loop n times generates $(\mu_1^*, \sigma_1^{2*}), \dots, (\mu_n^*, \sigma_n^{2*})$ which are n realisations from $[\mu, \sigma^2|y]$. Further, $(\mu_1^*), \dots, (\mu_n^*)$ and $(\sigma_1^{2*}), \dots, (\sigma_n^{2*})$ are n realisations from the marginal distributions $[\mu|y]$ and $[\sigma^2|y]$, respectively. There are no issues of convergence with the method of composition. The first extraction (μ_1^*, σ_1^{2*}) is a draw from $[\mu, \sigma^2|y]$.

The following data y mimic $n = 9$ bristle number measurements of male *Drosophila* flies which are assumed to be a realisation from $N(\mu, \sigma^2)$.

$$y = (35.14, 37.01, 30.97, 34.42, 38.22, 34.04, 30.79, 42.31, 33.78)'$$

The sample mean and variance are $\bar{y} = 35.19$ and $S^2 = 13.03$. As a first approximation one may wish to compute a 95% confidence interval for μ based on $[\mu|\sigma^2, y]$. From previous knowledge about the trait you are willing to assign a value to $\mu_0 = 34$ and you choose $n_0 = 1$. Using $\sigma^2 = S^2 = 13.03$, from (4.14) and (4.15)

$$\begin{aligned} \mu_1 &= \frac{n_0\mu_0 + n\bar{y}}{n_0 + n} = \frac{1 \times 34 + 9 \times 35.19}{1 + 9} = 35.07, \\ \sigma_1^2 &= \frac{\sigma^2}{n_0 + n} = \frac{13.03}{10} = 1.303. \end{aligned}$$

Then

$$\left[\mu | \sigma^2 = 13.03, y \right] \sim N(35.07, 1.303).$$

A 95% quantile-based confidence interval for μ based on this conditional posterior distribution (given σ^2) gives (32.88; 37.31). This is obtained using the R-code

```
ci <- qnorm(c(0.0275, 0.975), 35.07, sqrt(1.303))
```

that yields (32.88, 37.31). This interval ignores the uncertainty about σ^2 since it assumes $\sigma^2 = S^2$, exactly.

To account properly for uncertainty, the method of composition is implemented drawing samples from $[\mu, \sigma^2 | y]$. This requires to specify (4.23) and (4.24). Based on previous knowledge of the trait, one may assign $S_0^2 = 11.22$ and the parameters of (4.23) and (4.24) are

$$\begin{aligned} \mu_1 &= 35.07, \\ \frac{\sigma^2}{n_0 + n} &= \frac{\sigma^2}{1 + 9}, \\ \tilde{v} &= v_0 + n = 1 + 9 = 10, \\ \tilde{S}^2 &= \frac{v_0 S_0^2 + (n - 1) S^2 + \frac{n_0 n}{n_0 + n} (\bar{y} - \mu_0)^2}{\tilde{v}} = \\ &= \frac{1 \times 11.22 + (9 - 1) 13.03 + \frac{1 \times 9}{1 + 9} (35.19 - 34)^2}{10} = 11.67 \end{aligned}$$

With these values one can repeatedly draw

1. σ_i^{2*} from $\chi^{-2}(10, 11.67)$
2. μ_i^* from $N\left(35.07, \frac{\sigma_i^{2*}}{1+9}\right)$

The R-code to perform these calculations is

```
set.seed(377)
s <- (11.67*10)/rchisq(10000, 10)
m <- rnorm(10000, 35.07, sqrt(s/10))
quantile(m, c(0.025, 0.975))
```

```
##      2.5%      97.5%
## 32.64573 37.45952
```

```
quantile(s, c(0.05, 0.95))
```

```
##      5%      95%
## 6.403362 29.220816
```

The 95% quantile-based confidence interval for μ is now a little broader than before because the joint inference accounts for uncertainty about σ^2 .

4.4 Approximating a Marginal Distribution

The conditional posterior distribution $[\mu|\sigma^2 = 13.03, y]$ can be regarded as an approximation to

$$p(\mu|y) = \int p(\mu|\sigma^2, y) p(\sigma^2|y) d\sigma^2.$$

The explanation lies in the following result. Assume a model that has two parameters, θ_1 and θ_2 . Then

$$p(\theta_1|y) = \int p(\theta_1|\theta_2, y) p(\theta_2|y) d\theta_2. \quad (4.48)$$

Define $\hat{\theta}_2$ as the mode of $[\theta_2|y]$ and expand $p(\theta_1|\theta_2, y)$ in a Taylor series about $\hat{\theta}_2$:

$$p(\theta_1|\theta_2, y) \approx p(\theta_1|\hat{\theta}_2, y) + (\theta_2 - \hat{\theta}_2) \left. \frac{\partial p(\theta_1|\theta_2, y)}{\partial \theta_2} \right|_{\theta_2=\hat{\theta}_2}.$$

Let

$$f(\hat{\theta}_2) = \left. \frac{\partial p(\theta_1|\theta_2, y)}{\partial \theta_2} \right|_{\theta_2=\hat{\theta}_2}$$

and substitute in (4.48)

$$\begin{aligned} p(\theta_1|y) &\approx \int [p(\theta_1|\hat{\theta}_2, y) + (\theta_2 - \hat{\theta}_2) f(\hat{\theta}_2)] p(\theta_2|y) d\theta_2 \\ &= p(\theta_1|\hat{\theta}_2, y) \int p(\theta_2|y) d\theta_2 + f(\hat{\theta}_2) \int (\theta_2 - \hat{\theta}_2) p(\theta_2|y) d\theta_2 \\ &\quad - f(\hat{\theta}_2) \hat{\theta}_2 \int p(\theta_2|y) d\theta_2 \\ &= p(\theta_1|\hat{\theta}_2, y) + f(\hat{\theta}_2) [E(\theta_2|y) - \hat{\theta}_2]. \end{aligned} \quad (4.49)$$

When the posterior mode $\hat{\theta}_2$ is equal to the posterior mean $E(\theta_2|y)$ the second term drops out and

$$p(\theta_1|y) \approx p(\theta_1|\hat{\theta}_2, y) \quad (4.50)$$

In the example above the mean and the mode are markedly different and both different from the value conditioned upon $\sigma^2 = 13.03$. The mean is

$$\begin{aligned} E(\sigma^2|y) &= \frac{\tilde{v}}{\tilde{v} - 2} \tilde{S}^2 \\ &= \frac{10}{8} 11.67 = 14.59 \end{aligned}$$

and the mode

$$\begin{aligned} \arg \max p(\sigma^2|y) &= \frac{\tilde{v}}{\tilde{v} + 2} \tilde{S}^2 \\ &= \frac{10}{12} 11.67 = 9.73, \end{aligned}$$

so in this case the approximation is rather coarse.

4.5 Example: The Normal Linear Mixed Model

The mixed linear model or (co)variance components model (Henderson 1953), includes fixed and random effects entering linearly into the conditional expectation of the observations (given the random effects). These effects are the location parameters of the model. Typically, random effects and model residuals are assigned Gaussian distributions which depend on components of variance or covariance. A classical example in animal genetics is modelling continuous data, where explanatory variables are “fixed” effects such as sex and breed and random effects are additive genetic values. In this setup variance components could include the additive genetic variance and the residual variance. This example provides first, an overview of classical mixed model results (Henderson et al. 1959), followed by a description of the computation of the posterior distribution of the location parameters, conditional on the variance components, using a Bayesian perspective. The connection between both approaches is highlighted. The third part of the example provides the derivation of the so-called, fully conditional posterior distribution of location parameters; these are essential ingredients of Gibbs sampling algorithms.

Assume that the variance components are known and let b , of dimension $p \times 1$ and a , of dimension $q \times 1$, represent vectors of ‘fixed’ and random effects, respectively. The conditional distribution (or sampling model) of data vector y , of dimension $n \times 1$ is

$$y|b, a, R \sim N(Xb + Za, R), \quad (4.51)$$

where X , of dimension $n \times p$ and Z , of dimension $n \times q$, are observed incidence matrices and R is the residual $n \times n$ (co)variance matrix. Invoking an infinitesimal

model, the vector a is assigned the normal distribution

$$a | \sigma_a^2 \sim N(0, G), \quad (4.52)$$

where $G = A\sigma_a^2$, A is the $q \times q$ additive genetic relationship matrix and σ_a^2 is the additive genetic variance.

- This first section offers a brief overview of some classical mixed model results. In the classical approach, b is treated as a fixed unknown parameter. Given the definition of the hierarchical model (4.51) and (4.52), the joint density of the data y and of the random additive genetic values a , conditional on b and the variance components, can be written

$$p(y|b, a, R) p(a|G) \propto \exp \left[-\frac{1}{2} (y - Xb - Za)' R^{-1} (y - Xb - Za) \right] \exp \left[-\frac{1}{2} a' G^{-1} a \right]. \quad (4.53)$$

Henderson et al. (1959) developed a set of equations that simultaneously result in the best linear unbiased estimator (BLUE) of b , and the best linear unbiased predictor (BLUP) of a , by maximising the joint density (4.53) with respect to elements of b and of a to yield

$$\begin{bmatrix} X'R^{-1}X & X'R^{-1}Z \\ Z'R^{-1}X & Z'R^{-1}Z + G^{-1} \end{bmatrix} \begin{bmatrix} \hat{b} \\ \hat{a} \end{bmatrix} = \begin{bmatrix} X'R^{-1}y \\ Z'R^{-1}y \end{bmatrix}, \quad (4.54)$$

known as the mixed model equations (see the book Variance Components (Searle et al. 1992) for various other derivations of the mixed model equations, some of which do not invoke the assumption of normality).

We examine the nature of the solutions $\hat{b}$ and $\hat{a}$. Using the second equation solve for $\hat{a}$

$$\hat{a} = \left(Z'R^{-1}Z + G^{-1} \right)^{-1} \left(Z'R^{-1}y - Z'R^{-1}X\hat{b} \right)$$

and substitute in the first equation:

$$\begin{aligned} & X' \left[R^{-1} - R^{-1}Z \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} \right] X\hat{b} \\ &= X' \left[R^{-1} - R^{-1}Z \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} \right] y. \end{aligned}$$

On establishing that the marginal variance of y is $V = ZGZ' + R$ and using the *Sherman-Morrison-Woodbury formula* (Seber and Lee 2003, p. 467) or Henderson et al. (1959)

$$V^{-1} = R^{-1} - R^{-1}Z \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1},$$

leads to the solution for $\hat{b}$ that satisfies the linear system

$$\left(X'V^{-1}X \right) \hat{b} = X'V^{-1}y. \quad (4.55)$$

In (4.55), $\hat{b}$ is the generalised least squares estimator of b or the maximum likelihood estimator of b in this normal linear setup with known variance components.

To examine the solution for a , expand the second equation,

$$Z'R^{-1}X\hat{b} + \left(Z'R^{-1}Z + G^{-1} \right) \hat{a} = Z'R^{-1}y$$

and replacing $\hat{b}$ by its solution (4.55), BLUP of $\hat{a}$, eliminating b by absorption, can also be written

$$\hat{a} = \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} (y - X\hat{b}). \quad (4.56)$$

There is an interesting connection between (4.56) and the conditional mean under multivariate normality. Assume

$$(y, a) \sim N \left[\begin{pmatrix} Xb \\ 0 \end{pmatrix}, \begin{pmatrix} V & C \\ C' & G \end{pmatrix} \right]. \quad (4.57)$$

Then standard normal theory asserts that

$$E(a|y) = C'V'^{-1}(y - Xb). \quad (4.58)$$

Henderson (1963) proves that

$$\left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} = C'V^{-1} \quad (4.59)$$

and therefore (4.56) can be written

$$\hat{a} = C'V'^{-1}(y - X\hat{b}) \quad (4.60)$$

that has the same form as the conditional mean under normality, with the unknown b replaced by its GLS estimator. Henderson's proof is as follows. Using (4.57),

$$\begin{aligned}
 C'V'^{-1} &= GZ'V'^{-1} \\
 &= GZ' \left(R^{-1} - R^{-1}Z \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} \right) \\
 &= G \left[Z'R^{-1} - Z'R^{-1}Z \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} \right] \\
 &= G \left[Z'R^{-1} - \left(Z'R^{-1}Z + G^{-1} \right) \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} \right. \\
 &\quad \left. + G^{-1} \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} \right] \\
 &= G \left[Z'R^{-1} - Z'R^{-1} + G^{-1} \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1} \right] \\
 &= \left(Z'R^{-1}Z + G^{-1} \right)^{-1} Z'R^{-1}.
 \end{aligned}$$

In the classical frequentist setting with b “fixed” and a “random”, the inverse of the coefficient matrix of the mixed model equations (4.54) corresponds to

$$\text{Var} \begin{bmatrix} \hat{b} \\ \hat{a} - a \end{bmatrix}$$

where $\text{Var}(\hat{b})$ is the variance (in repeated sampling of y) of BLUE(b) (the best linear unbiased estimator of b) and $\text{Var}(\hat{a} - a) = E_y(\text{Var}(a|y))$ is the prediction error variance of the predicted additive genetic values (Henderson 1975).

- This section adopts a Bayesian perspective and as before, the dispersion parameters G and R are assumed known. The focus is on the location parameters of the Bayesian linear model, b and a . A benchmark paper is Lindley and Smith (1972).

The hierarchical specification of the model is as follows:

$$\begin{aligned}
 y|b, a, R &\sim N(Xb + Za, R), \\
 a|a_0, G &\sim N(a_0, G), \\
 b|b_0, B &\sim N(b_0, B),
 \end{aligned}$$

and (a, b, e) are assumed independent, where $\text{Var}(e) = \text{Var}(y|b, a, R) = R$ and often $B = \bar{B}\sigma_b^2$ for some positive definite matrix $\bar{B}$ and scalar σ_b^2 . It follows that

$$\text{Cov}(y, b') = XB,$$

$$\text{Cov}(y, a') = ZG.$$

The objective is to show that conditional on B, G and R , assigning a vague prior distribution to b represented by $B = \infty$, the mean of the posterior distribution $[a, b|y, b_0, a_0, B, G, R]$ is mathematically identical to BLUE of b and BLUP of a developed under the classical mixed model approach. An early connection between both approaches was established by Dempfle (1977).

Since all distributions are normal, the joint distribution of y, a and b is

$$\begin{bmatrix} y \\ b \\ a \end{bmatrix} \sim N \left[\begin{pmatrix} Xb_0 + Za_0 \\ b_0 \\ a_0 \end{pmatrix}, \begin{pmatrix} XB X' + ZG Z' + R & XB & ZG \\ BX' & B & 0 \\ GZ' & 0 & G \end{pmatrix} \right]. \quad (4.61)$$

The posterior distribution $[a, b|y, b_0, a_0, B, G, R]$ can be read off directly from (4.61). However, it is easier to show the similarity between the Bayesian model and the classical model, proceeding as follows. From (4.61), the logposterior distribution $[a, b|y]$ is

$$\begin{aligned} \ln p(a, b|y, b_0, a_0, B, G, R) &\propto \ln p(y|a, b, R) + \ln p(b|B) + \ln p(a|G) \\ &\propto -\frac{1}{2}(y - Xb - Za)' R^{-1}(y - Xb - Za) - \frac{1}{2}(b - b_0)' B^{-1}(b - b_0) \\ &\quad - \frac{1}{2}(a - a_0)' G^{-1}(a - a_0) \end{aligned} \quad (4.62)$$

The mode of the distribution (and the mean, given the normal assumption) is obtained by maximising the logposterior with respect to (a, b) . Taking partial derivatives yields

$$\begin{aligned} \frac{\partial \ln p(a, b|y, b_0, a_0, B, G, R)}{\partial b} &= -B^{-1}(b - b_0) + XR^{-1}(y - Xb - Za), \\ \frac{\partial \ln p(a, b|y, b_0, a_0, B, G, R)}{\partial a} &= -G^{-1}(a - a_0) + Z'R^{-1}(y - Xb - Za). \end{aligned}$$

Setting the two equations equal to zero and rearranging results in the linear system

$$\begin{bmatrix} X'R^{-1}X + B^{-1} & X'R^{-1}Z \\ Z'R^{-1}X & Z'R^{-1}Z + G^{-1} \end{bmatrix} \begin{bmatrix} \hat{b} \\ \hat{a} \end{bmatrix} = \begin{bmatrix} X'R^{-1}y + B^{-1}b_0 \\ Z'R^{-1}y + G^{-1}a_0 \end{bmatrix}. \quad (4.63)$$

When B^{-1} implying $B = \infty$ (vague prior information about b), the system of equations is identical to the system generated under the classical approach, assuming $a|G, a_0 \sim N(a_0, G)$. If $a_0 = 0$, (4.63) reduces to (4.54).

- A full characterisation of the posterior distribution $[a, b|y, b_0, a_0, B, G, R]$ can be obtained as follows. The starting point is the logposterior (4.62). Define

$$\begin{aligned} W &= (X \ Z), \\ \theta' &= (b', a'), \\ \theta'_0 &= (b'_0, a'_0), \\ \Sigma &= \begin{bmatrix} B^{-1} & 0 \\ 0 & G^{-1} \end{bmatrix}. \end{aligned}$$

Write the mixed model equations (4.63) as

$$\left(W' R^{-1} W + \Sigma \right) \hat{\theta} = W' R^{-1} y + \Sigma \theta_0. \quad (4.64)$$

Then (4.62) times (-2) can be written

$$\begin{aligned} & (y - W\theta)' R^{-1} (y - W\theta) + (\theta - \theta_0)' \Sigma (\theta - \theta_0) \\ &= y' R^{-1} y - 2\theta' W' R^{-1} y + \theta' W' R^{-1} W \theta + (\theta - \theta_0)' \Sigma (\theta - \theta_0) \\ &= y' R^{-1} y - 2\theta' \left(W' R^{-1} W + \Sigma \right) \hat{\theta} + 2\theta' \Sigma \theta_0 + \theta' W' R^{-1} W \theta \\ &\quad + (\theta - \theta_0)' \Sigma (\theta - \theta_0) \\ &= y' R^{-1} y - 2\theta' \left(W' R^{-1} W + \Sigma \right) \hat{\theta} + \theta' \left(W' R^{-1} W + \Sigma \right) \theta \\ &\quad + \theta'_0 \Sigma \theta_0. \end{aligned}$$

Finally, add and subtract $\hat{\theta}' (W' R^{-1} W + \Sigma) \hat{\theta}$ and rearrange to yield

$$\begin{aligned} & y' R^{-1} y + (\theta - \hat{\theta})' \left(W' R^{-1} W + \Sigma \right) (\theta - \hat{\theta}) - \hat{\theta}' \left(W' R^{-1} W + \Sigma \right) \hat{\theta} \\ &+ \theta'_0 \Sigma \theta_0 \end{aligned}$$

This trick that I use here for combining the two quadratic forms is used repeatedly in the book, as well as formulas given in Box and Tiao (1973), p. 418 (reproduced on pages 152 and 153).

Since the first and the last two terms do not depend on θ ,

$$\ln p(\theta|y, \theta_0, B, G, R) \propto -\frac{1}{2} (\theta - \hat{\theta})' \left(W' R^{-1} W + \Sigma \right) (\theta - \hat{\theta})'$$

that is the kernel of the normal distribution

$$\theta|y, \theta_0, B, G, R \sim N\left(\widehat{\theta}, \left(W'R^{-1}W + \Sigma\right)^{-1}\right). \quad (4.65)$$

- Let $(W'R^{-1}W + \Sigma) = C$, $W'R^{-1}y + \Sigma\theta_0 = r$ and write equations (4.64) as

$$C\widehat{\theta} = r.$$

Partition $\theta' = (\theta_1, \theta_2)$ arbitrarily, such that θ_1 can be scalar or vector. *What is the conditional posterior distribution of $(\theta_1|\theta_2, y, \theta_0, B, G, R)$?* From multivariate normal theory,

$$\theta_1, \theta_2|y, \theta_0, B, G, R \sim N\left[\begin{pmatrix} \widehat{\theta}_1 \\ \widehat{\theta}_2 \end{pmatrix}, \begin{pmatrix} C^{11} & C^{12} \\ C^{21} & C^{22} \end{pmatrix}\right]$$

where

$$C^{-1} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}^{-1} = \begin{pmatrix} C^{11} & C^{12} \\ C^{21} & C^{22} \end{pmatrix}.$$

Since $(\theta_1, \theta_2|y, \theta_0, B, G, R)$ is multivariate normal, so is the conditional distribution $(\theta_1|\theta_2, y, \theta_0, B, G, R)$.

Define $r' = (r'_1, r'_2)$. The conditional expected value of $[\theta_1|\theta_2, y, \theta_0, B, G, R]$ is

$$E(\theta_1|\theta_2, y, \theta_0, B, G, R) = \widehat{\theta}_1 + C^{12} \left(C^{22}\right)^{-1} (\theta_2 - \widehat{\theta}_2).$$

Using the mixed model equations, write

$$\begin{aligned} \widehat{\theta}_1 &= C^{11}r_1 + C^{12}r_2, \\ \widehat{\theta}_2 &= C^{21}r_1 + C^{22}r_2. \end{aligned}$$

Substituting above

$$\begin{aligned} E(\theta_1|\theta_2, y, \sigma^2, \sigma_a^2) &= C^{11}r_1 + C^{12}r_2 + C^{12} \left(C^{22}\right)^{-1} (\theta_2 - C^{21}r_1 - C^{22}r_2) \\ &= \left(C^{11} - C^{12} \left(C^{22}\right)^{-1} C^{21}\right) r_1 + C^{12} \left(C^{22}\right)^{-1} \theta_2 \\ &= C_{11}^{-1} \left(r_1 + C_{11}C^{12} \left(C^{22}\right)^{-1} \theta_2\right) \\ &= C_{11}^{-1} (r_1 - C_{12}\theta_2). \end{aligned} \quad (4.66)$$

The derivation above uses the partition matrix results

$$\begin{aligned} C^{11} - C^{12} (C^{22})^{-1} C^{21} &= C_{11}^{-1}, \\ C_{11} C^{12} (C^{22})^{-1} &= -C_{12}. \end{aligned}$$

The conditional variance is

$$\begin{aligned} \text{Var}(\theta_1 | \theta_2, y, \theta_0, B, G, R) &= \left(C^{11} - C^{12} (C^{22})^{-1} C^{21} \right) \\ &= C_{11}^{-1}. \end{aligned}$$

Therefore the conditional posterior distribution is

$$[\theta_1 | \theta_2, y, \theta_0, B, G, R] \sim N \left[C_{11}^{-1} (r_1 - C_{12} \theta_2), C_{11}^{-1} \right]. \quad (4.67)$$

These results play a key role in the implementation of MCMC methods that rely on conditional posterior distributions such as the Gibbs sampler. Notice that even if vector θ contains many parameters and C is a very large matrix, the conditional posterior distribution $(\theta_1 | \theta_2, y, \theta_0, B, G, R)$ involves inverse of matrices of the order of the number of elements in θ_1 . For example, if θ_1 is a scalar, only the inverse of the appropriate scalar element is needed.

4.6 Example: Inferring a Variance Component from a Marginal Posterior Distribution

Let the sampling distribution of the $n \times 1$ data vector y be

$$y | b, \sigma^2 \sim N(Xb, I\sigma^2), \quad (4.68)$$

where X is an observed $n \times p$ matrix of full rank, b is a $p \times 1$ vector of unobserved regressions, I is the $n \times n$ identity matrix and the unobserved scalar σ^2 is the residual variance. On assigning independent improper prior distributions to the parameters of the model (b, σ^2) , the posterior distribution is proportional to the likelihood (4.68):

$$p(b, \sigma^2 | y) \propto p(y | b, \sigma^2). \quad (4.69)$$

The model in this example is identical to that on page 159. Here, the vector b is regarded as a nuisance parameter that is integrated out of the joint posterior

distribution $p(b, \sigma^2|y)$. This leads to the marginal posterior distribution $[\sigma^2|y]$, that as shown in (4.40), has the form of a scale inverted chi-square process; that is

$$p(\sigma^2|y) \propto (\sigma^2)^{-(\frac{v}{2}+1)} \exp\left(-\frac{vS^2}{2\sigma^2}\right), \quad (4.70)$$

where here, $v = n - p - 2$ and $S^2 = \frac{1}{v} (y - X\hat{b})' (y - X\hat{b})$. Inferences about σ^2 or of functions thereof can be performed analytically from this marginal posterior distribution. It is also possible to draw samples from the posterior distribution of σ^2 and to use the samples to construct a Monte Carlo estimate of the posterior distribution of any function. To obtain a sample from this distribution, draw a chi-square variable with v degrees of freedom, invert this number and multiply by $(y - X\hat{b})' (y - X\hat{b})$.

The modal value of this marginal posterior distribution is obtained by maximising (4.70) over σ^2 that leads to

$$\arg \max_{\sigma^2} p(\sigma^2|y) = \frac{v}{v+2} S^2 = \frac{(y - X\hat{b})' (y - X\hat{b})}{n - p}. \quad (4.71)$$

In this particular example where (b, σ^2) have a priori an improper uniform distribution, (4.71) has the same exact form as the REML estimator of σ^2 and (4.70) has the form of a restricted likelihood, here, arrived at by marginalising over the “fixed” effects b . The REML estimator for unbalanced data was originally derived from a frequentist perspective by maximising a likelihood that is invariant to the “fixed effects” in the highly cited work of Patterson and Thompson (1971).

It is revealing to compare the marginal posterior distribution $[\sigma^2|y]$ with the conditional posterior distribution $[\sigma^2|b, y]$. The latter takes the form

$$p(\sigma^2|b, y) \propto (\sigma^2)^{-\frac{n}{2}} \exp\left[-\frac{1}{2\sigma^2} (y - Xb)' (y - Xb)\right], \quad (4.72)$$

proportional to a scaled inverse chi square distribution with $v = n - 2$ degrees of freedom. Maximising over σ^2 leads to

$$\arg \max_{\sigma^2} p(\sigma^2|b, y) = \frac{(y - Xb)' (y - Xb)}{n}. \quad (4.73)$$

A comparison with (4.71) shows how marginalisation “accounts for the loss of degrees of freedom” associated with estimation of the p unknowns in b .

4.7 Example: Bayesian Learning: Inheritance of Haemophilia

This and the example below (both taken from Sorensen and Gianola 2002) illustrate the sequential nature of Bayesian inference. In this first example data are discrete; continuous data involving additive genetic values (assumed to be multivariate normally distributed) are discussed in the next example.

The following is adapted from Gelman et al. (1995). Haemophilia is a genetic disease in humans. The locus responsible for its expression is located on the sex chromosomes (these are denoted as XX in women and XY in men). The condition is observed in women only in double recessive individuals (aa) and in men that are carriers of the a allele on the X chromosome. Suppose there is a nonhaemophiliac woman whose father and mother are not affected by the disease, but her brother is known to be haemophiliac. This implies that her nonhaemophiliac mother must be heterozygote, a carrier of a . What is the probability that the propositus woman is also a carrier of the gene? Let θ be a random variable taking one of two mutually exclusive and exhaustive values. Either $\theta = 1$ if the woman is a carrier, or $\theta = 0$ otherwise. Since it is known that the mother of the woman must be a carrier (this constitutes part of the system within which probabilities are assigned), the prior distribution of θ is

$$\Pr(\theta = 1) = \Pr(\theta = 0) = \frac{1}{2}.$$

In the absence of additional information, it is not possible to make a very sharp probability assignment. Suppose now that the woman has two sons, none of which is affected. Let Y_i be a binary random variable taking the value 0 if son i is not affected, or 1 if he has the disease. The values taken by Y_1 and Y_2 constitute the evidence. Given that $\theta = 1$, the probability of the observed data is

$$\begin{aligned} & \Pr(Y_1 = 0, Y_2 = 0 | \theta = 1) \\ &= \Pr(Y_1 = 0 | \theta = 1) \Pr(Y_2 = 0 | \theta = 1) = \left(\frac{1}{2}\right)^2 = \frac{1}{4}. \end{aligned}$$

This follows because:

- (a) the observations are assumed to be independent, conditionally on θ and
- (b) if the woman is a carrier ($\theta = 1$), there is a 50% probability that she will not transmit the allele.

On the other hand, if she is not a carrier ($\theta = 0$):

$$\begin{aligned} & \Pr(Y_1 = 0, Y_2 = 0 | \theta = 0) \\ &= \Pr(Y_1 = 0 | \theta = 0) \Pr(Y_2 = 0 | \theta = 0) = 1 \times 1 = 1, \end{aligned}$$

this being so because it is impossible for a son to have the disease unless the mother is a carrier (ignoring mutation). Hence, the data confer four times more likelihood to the hypothesis that the mother is not a carrier. Using the information that none of the sons is diseased, the posterior distribution of θ is then

$$\begin{aligned}\Pr(\theta = 1|Y_1 = 0, Y_2 = 0) &= \frac{\Pr(\theta = 1) \Pr(Y_1 = 0, Y_2 = 0|\theta = 1)}{\Pr(Y_1 = 0, Y_2 = 0)} \\ &= \frac{\Pr(\theta = 1) \Pr(Y_1 = 0, Y_2 = 0|\theta = 1)}{\sum_{i=0}^1 \Pr(\theta = i) \Pr(Y_1 = 0, Y_2 = 0|\theta = i)} \\ &= \frac{\frac{1}{2} \frac{1}{4}}{\frac{1}{2} \frac{1}{4} + \frac{1}{2} \frac{1}{4}} = \frac{1}{5}\end{aligned}$$

and

$$\Pr(\theta = 0|Y_1 = 0, Y_2 = 0) = 1 - \frac{1}{5} = \frac{4}{5}.$$

A sharper probability assignment can be made now and the combination of prior information with the evidence suggests that the mother is probably not a carrier. The latter possibility cannot be ruled out, however, as there is a 20% probability that the mother is heterozygote. The posterior odds in favour of the hypothesis that the mother is not a carrier is given by the ratio

$$\begin{aligned}\frac{\Pr(\theta = 0|Y_1 = 0, Y_2 = 0)}{\Pr(\theta = 1|Y_1 = 0, Y_2 = 0)} &= \frac{\Pr(Y_1 = 0, Y_2 = 0|\theta = 0) \Pr(\theta = 0)}{\Pr(Y_1 = 0, Y_2 = 0|\theta = 1) \Pr(\theta = 1)} \\ &= \frac{1 \cdot \frac{1}{2}}{\frac{1}{4} \cdot \frac{1}{2}} = 4,\end{aligned}$$

where the ratio

$$\frac{\Pr(\theta = 0)}{\Pr(\theta = 1)} = 1$$

is called the prior odds in favour of the hypothesis. Further,

$$B_{01} = \frac{\Pr(Y_1 = 0, Y_2 = 0|\theta = 0)}{\Pr(Y_1 = 0, Y_2 = 0|\theta = 1)} = 4$$

is called the *Bayes factor*, the factor by which the prior odds about the hypotheses are modified by the evidence and converted into posterior odds. In this example, the odds in favour of the hypothesis that $\theta = 0$ relative to $\theta = 1$ increase by a factor of 4 after observing two sons that are not affected by the disease. Suppose that the

woman suspected of being a carrier has n children. The posterior distribution of θ can be represented as

$$\Pr(\theta = i|y) = \frac{\Pr(\theta = i) \Pr(y|\theta = i)}{\Pr(\theta = i) \Pr(y|\theta = i) + \Pr(\theta \neq i) \Pr(y|\theta \neq i)}, \quad i = 0, 1,$$

where $y = [Y_1, Y_2, \dots, Y_n]'$. Partition the data as $y = [y_A' y_B']'$, with y_A being the records on presence or absence of the disease for the first m progeny and with y_B containing data on the last $n - m$ children. The posterior distribution is

$$\Pr(\theta = i|y) = \frac{\Pr(\theta = i) p(y_A|\theta = i) p(y_B|y_A, \theta = i)}{\sum_{i=0}^1 \Pr(\theta = i) p(y_A|\theta = i) p(y_B|y_A, \theta = i)}.$$

Dividing the numerator and denominator by the marginal probability of observing y_A , $p(y_A)$, gives

$$\Pr(\theta = i|y) = \frac{\frac{\Pr(\theta = i) p(y_A|\theta = i)}{p(y_A)} p(y_B|y_A, \theta = i)}{\sum_{i=0}^1 \frac{\Pr(\theta = i) p(y_A|\theta = i)}{p(y_A)} p(y_B|y_A, \theta = i)}.$$

Note, however, that

$$\frac{\Pr(\theta = i) p(y_A|\theta = i)}{p(y_A)} = \Pr(\theta = i|y_A)$$

is the posterior probability after observing y_A , which acts as a prior before observing y_B . Then, it follows that

$$\Pr(\theta = i|y) = \frac{\Pr(\theta = i|y_A) p(y_B|y_A, \theta = i)}{\sum_{i=0}^1 \Pr(\theta = i|y_A) p(y_B|y_A, \theta = i)}$$

illustrating the “memory” property of Bayes theorem. If the observations are conditionally independent, as assumed in this example, then

$$p(y_B|y_A, \theta = i) = p(y_B|\theta = i).$$

Suppose now that the woman has a third, unaffected son. The prior distribution now assigns probabilities of $\frac{4}{5}$ and $\frac{1}{5}$ to the events “not being a carrier” and “carrying the allele”, respectively. The posterior probability of the woman being a carrier,

after observing a third, unaffected child is

$$\begin{aligned}
 & \Pr(\theta = 1 | Y_1 = 0, Y_2 = 0, Y_3 = 0) \\
 &= \frac{\frac{1}{5} \Pr(Y_3 = 0 | \theta = 1)}{\frac{1}{5} \Pr(Y_3 = 0 | \theta = 1) + \frac{4}{5} \Pr(Y_3 = 0 | \theta = 0)} \\
 &= \frac{\frac{1}{5} \frac{1}{2}}{\frac{1}{5} \frac{1}{2} + \frac{4}{5} 1} = \frac{1}{9}.
 \end{aligned}$$

The same result is obtained starting from the prior before observing any children

$$\begin{aligned}
 \Pr(\theta = 1 | Y_1 = 0, Y_2 = 0, Y_3 = 0) &= \frac{\frac{1}{2} \cdot \left(\frac{1}{2}\right)^3}{\frac{1}{2} \cdot \left(\frac{1}{2}\right)^3 + \frac{1}{2} \cdot (1)^3} \\
 &= \frac{1}{9}.
 \end{aligned}$$

4.8 Example: Bayesian Learning: Updating Additive Genetic Values

The setting is in the same spirit as in the preceding example. Here the vector of data $y = (y'_1, y'_2)'$ is collected first at stage 1, y_1 , and then at stage 2, y_2 . Suppose that at stage 1 (2), measurements y_1 (y_2) are taken on n_1 (n_2) different individuals (so that an individual measured at any stage is not recorded at the other stage) and that the objective is to infer their additive genetic effects a_1 (a_2). The protocol for the evaluation in stages is as follows:

- Using data at stage 1, y_1 , infer a_1 and a_2 and derive the marginal posterior distributions $[a_i | y_1]$, $i = 1, 2$. From $[a_2 | y_1]$ obtain the mean and variance

$$\tilde{a}_2 = E(a_2 | y_1), \quad \tilde{C}_2 = \text{Var}(a_2 | y_1).$$

(dropping here the conditioning on the location parameters and variances to avoid cluttering the notation).

- at stage 2, using as prior for a_2 the posterior from stage 1, obtain

$$p(a_2 | y_2, y_1) \propto p(y_2 | a_2) p(a_2 | \tilde{a}_2, \tilde{C}_2)$$

and finally, with knowledge of the posterior density, obtain $E(a_2 | y_2, y_1)$ and $\text{Var}(a_2 | y_2, y_1)$.

Consider the following linear model

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1_1 \mu_1 \\ 1_2 \mu_2 \end{bmatrix} + \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} + \begin{bmatrix} e_1 \\ e_2 \end{bmatrix}, \quad (4.74)$$

where μ_i , $i = 1, 2$, is a known location parameter common to records collected in stages i , 1_i , $i = 1, 2$, is a vector of 1's for stage i records and

$$\begin{bmatrix} e_1 \\ e_2 \end{bmatrix} \Big| \sigma_e^2 \sim N \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} I_1 & 0 \\ 0 & I_2 \end{bmatrix} \sigma_e^2 \right)$$

is a vector of independently distributed residual effects, where σ_e^2 is the (known) residual variance; I_i is an identity matrix of order $n_i \times n_i$, ($i = 1, 2$).

In the classical infinitesimal model of inheritance, the additive genetic effects are assumed to follow the multivariate normal distribution (acting as a prior in the Bayesian sense):

$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} \Big| \sigma_a^2 \sim N \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \sigma_a^2 \right) \quad (4.75)$$

where σ_a^2 is the additive genetic variance in the population (also assumed known) and

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

is the matrix of additive genetic relationships between individuals, or twice the matrix of coefficients of coancestry. This matrix is assumed to have full rank; clones or identical twins are not encountered.

The standard treatment using all the data y is presented first, where data collection at stage 2 has been completed and additive genetic values are inferred for all individuals. The posterior density of the additive genetic values can be obtained using results for the linear model derived in the example of page 167 and is given by

$$\begin{aligned} & p(a_1, a_2 | y_1, y_2, \mu_1, \mu_2, \sigma_e^2, \sigma_a^2) \\ & \propto \exp \left\{ -\frac{1}{2\sigma_e^2} \left[(a - \hat{a})' \left(I + A^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right) (a - \hat{a}) \right] \right\}, \end{aligned} \quad (4.76)$$

which is the kernel of a normal process with mean

$$\hat{a} = \left(I + A^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right)^{-1} \begin{bmatrix} y_1 - 1_1 \mu_1 \\ y_2 - 1_2 \mu_2 \end{bmatrix} = \left(I + A^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right)^{-1} w, \quad (4.77)$$

and variance–covariance matrix

$$\text{Var} \left(a_1, a_2 | y_1, y_2, \mu_1, \mu_2, \sigma_e^2, \sigma_a^2 \right) = \left(I + A^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right)^{-1} \sigma_e^2. \quad (4.78)$$

Using (4.77) and (4.78), the posterior density of a_1 of the “first-stage” distribution

$$\left[a_1 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right]$$

is immediately found to be

$$\begin{aligned} & p \left(a_1 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right) \\ & \propto \exp \left\{ -\frac{1}{2\sigma_e^2} \left[(a_1 - \tilde{a}_1)' \left(I_1 + A_{11}^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right) (a_1 - \tilde{a}_1) \right] \right\}. \end{aligned} \quad (4.79)$$

The posterior mean at stage 1 is then

$$\tilde{a}_1 = \left(I_1 + A_{11}^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right)^{-1} (y_1 - 1_1 \mu_1) = \left(I_1 + A_{11}^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right)^{-1} w_1 \quad (4.80)$$

and the posterior covariance is

$$\text{Var} \left(a_1 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right) = \left(I_1 + A_{11}^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right)^{-1} \sigma_e^2 = \tilde{C}_1. \quad (4.81)$$

The analysis in stages proceeds as follows. What can be said about all additive genetic effects at stage 1? The joint posterior at stage 1 is

$$p \left(a_1, a_2 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right) \propto \left[p \left(y_1 | \mu_1, a_1, \sigma_e^2 \right) p \left(a_1 | \sigma_a^2 \right) \right] p \left(a_2 | a_1, \sigma_a^2 \right).$$

Noting that the expression in square brackets is the posterior of a_1 after stage 1, $p \left(a_1 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right)$, one can write

$$p \left(a_1, a_2 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right) \propto p \left(a_2 | a_1, \sigma_a^2 \right) p \left(a_1 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right)$$

and this is the density of a normal process because the two intervening densities are in normal forms. Hence, the marginal distribution of a_2 at stage 1 is normal as well, with marginal density

$$p \left(a_2 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right) = \int p \left(a_2 | a_1, \sigma_a^2 \right) p \left(a_1 | y_1, \mu_1, \sigma_e^2, \sigma_a^2 \right) da_1.$$

Therefore the mean of the posterior distribution of a_2 at stage 1 is

$$\begin{aligned}
 E(a_2|y_1, \mu_1, \sigma_e^2, \sigma_a^2) &= \int \left[\int a_2 p(a_2|a_1, \sigma_a^2) da_2 \right] p(a_1|y_1, \mu_1, \sigma_e^2, \sigma_a^2) da_1 \\
 &= E_{a_1|y_1} [E(a_2|a_1)] \\
 &= A_{21} A_{11}^{-1} \tilde{a}_1 \\
 &= A_{21} A_{11}^{-1} \left(I_1 + A_{11}^{-1} \frac{\sigma_e^2}{\sigma_a^2} \right)^{-1} w_1.
 \end{aligned} \tag{4.82}$$

This has the form of

$$\begin{aligned}
 E(a_2|a_1) &= E(a_2) + A_{21} A_{11}^{-1} [a_1 - E(a_1)] \\
 &= A_{21} A_{11}^{-1} a_1
 \end{aligned}$$

but with a_1 replaced by its posterior expectation $\tilde{a}_1$. Likewise,

$$\begin{aligned}
 \tilde{C}_2 &= \text{Var}(a_2|y_1, \mu_1, \sigma_e^2, \sigma_a^2) \\
 &= E_{a_1|y_1} [\text{Var}(a_2|a_1)] + \text{Var}_{a_1|y_1} [E(a_2|a_1)] \\
 &= E_{a_1|y_1} \left[A_{22} \sigma_a^2 - A_{21} A_{11}^{-1} A_{12} \sigma_a^2 \right] + \text{Var}_{a_1|y_1} (A_{21} A_{11}^{-1} a_1) \\
 &= A_{22} \sigma_a^2 - A_{21} A_{11}^{-1} A_{12} \sigma_a^2 + A_{21} A_{11}^{-1} \tilde{C}_1 A_{11}^{-1} A_{12}.
 \end{aligned} \tag{4.83}$$

The first term represents the variance of a_2 before observing anything; the second term is the reduction in variance that would be obtained if a_1 were known and the third term is a penalty that results from having to infer a_1 from y_1 . Thus, the prior distribution of a_2 at stage 2 (which is the posterior distribution of a_2 at stage 1) is a normal process with mean vector (4.82) and covariance matrix (4.83). Label this distribution $[a_2|\tilde{a}_2, \tilde{C}_2]$. Finally, at stage 2, the posterior density of a_2 is

$$\begin{aligned}
 p(a_2|y_1, y_2, \mu_1, \mu_2, \sigma_e^2, \sigma_a^2) &\propto p(y_2|\mu_2, a_2, \sigma_e^2) p(a_2|\tilde{a}_2, \tilde{C}_2) \\
 &\propto \exp \left[-\frac{(y_2 - 1_2 \mu_2 - a_2)' (y_2 - 1_2 \mu_2 - a_2)}{2\sigma_e^2} \right. \\
 &\quad \left. -\frac{(a_2 - \tilde{a}_2)' \tilde{C}_2^{-1} \sigma_e^2 (a_2 - \tilde{a}_2)}{2\sigma_e^2} \right].
 \end{aligned} \tag{4.84}$$

We know that the density is in a normal form, so the quadratics on a_2 can be combined in the usual manner, to arrive at the mean vector and covariance matrix of the distribution. Alternatively, noting that a normal distribution is unimodal (so

the mean is identical to the mode), the posterior mean at stage 2 can be found by maximizing the logarithm of (4.84). Let

$$F(a_2) = - \left[\frac{(y_2 - 1_2\mu_2 - a_2)'(y_2 - 1_2\mu_2 - a_2)}{2\sigma_e^2} + \frac{(a_2 - \tilde{a}_2)' \tilde{C}_2^{-1} \sigma_e^2 (a_2 - \tilde{a}_2)}{2\sigma_e^2} \right]$$

so

$$\frac{\partial F(a_2)}{\partial a_2} = (-1) \frac{-2(y_2 - 1_2\mu_2 - a_2) + 2\tilde{C}_2^{-1} \sigma_e^2 (a_2 - \tilde{a}_2)}{2\sigma_e^2}.$$

Setting to 0 and solving for a_2 yields

$$\hat{a}_2 = \left(I_2 + \tilde{C}_2^{-1} \sigma_e^2 \right)^{-1} \left(y_2 - 1_2\mu_2 + \tilde{C}_2^{-1} \sigma_e^2 \tilde{a}_2 \right) \quad (4.85)$$

as mean of the posterior distribution of a_2 , after stages 1 and 2. This is a matrix weighted average of $\tilde{a}_2$ and of $y_2 - 1_2\mu_2$. The variance–covariance matrix of the distribution is given by

$$\text{Var} \left(a_2 | y_1, y_2, \mu_1, \mu_2, \sigma_e^2, \sigma_a^2 \right) = \left(I_2 + \tilde{C}_2^{-1} \sigma_e^2 \right)^{-1} \sigma_e^2. \quad (4.86)$$

It can be verified that this is equal to the inverse of minus the matrix of second derivatives of $F(a_2)$ with respect to a_2 . Using tedious algebra one can also verify that (4.85) is identical to the a_2 -component of the solution to (4.77).

4.9 A Brief Account of Bayesian Asymptotics

When the form of the posterior distribution is unknown or when limiting results are of interest, a simple alternative is to obtain approximate Bayesian inferences appealing to asymptotic arguments. A result from Bayesian asymptotics (first order approximations, and assuming a number of regularity conditions) says that as sample size n increases, the posterior distribution of a parameter θ converges to a normal distribution with mean equal to the mode of $[\theta|y]$ and asymptotic variance equal to the inverse of minus the second derivative of the logposterior (inverse of the observed information) evaluated at the mode θ ,

$$[\theta|y] \sim N \left(\hat{\theta}, [I(\hat{\theta})]^{-1} \right), \quad (4.87)$$

where the observed information evaluated at the mode of scalar θ is

$$I(\hat{\theta}) = - \frac{\partial^2 \log(p(\theta|y))}{(\partial \theta)^2} \Big|_{\theta=\hat{\theta}}.$$

For large n the amount of prior information will tend to be small compared with the information provided by the data and is ignored in this development. The posterior mode $\hat{\theta}$ is now the ML estimator and the variance can be replaced by the inverse of either $I(\theta|y)$ or $i(\theta)$ defined in (2.9) and (2.7).

A heuristic proof of (4.87) is as follows. Write $p(\theta|y) = \exp[\log(p(\theta|y))]$ and expand in a Taylor series about the posterior mode of $[\theta|y]$, $\hat{\theta}$. Consider first the case where θ is a scalar. This gives

$$\begin{aligned} p(\theta|y) = \exp & \left[\log(p(\hat{\theta}|y)) + (\theta - \hat{\theta}) \left[\frac{\partial \log(p(\theta|y))}{\partial \theta} \right]_{\theta=\hat{\theta}} \right. \\ & \left. + \frac{1}{2} (\theta - \hat{\theta})^2 \left[\frac{\partial^2 \log(p(\theta|y))}{(\partial \theta)^2} \right]_{\theta=\hat{\theta}} + R \right], \end{aligned}$$

where R is of smaller order than $(\theta - \hat{\theta})^2$. At the mode the first derivative is zero, the term $\log(p(\hat{\theta}|y))$ does not depend of θ and is absorbed into the normalising constant. Then

$$p(\theta|y) \propto \exp \left[-\frac{1}{2} (\theta - \hat{\theta})^2 \left[\frac{-\partial^2 \log(p(\theta|y))}{(\partial \theta)^2} \right]_{\theta=\hat{\theta}} + R \right]$$

In the neighbourhood of $\hat{\theta}$ as $n \rightarrow \infty$, R becomes negligible and $p(\theta|y)$ is proportional to the kernel of the normal distribution (4.87). Alternatively, Fisher's expected information $i(\theta)$ defined in (2.9) could be used in place of the observed information $I(\theta|y)$ in (4.87).

The quality of the approximation can be improved by applying a transformation to θ so that the transformed parameter looks more normal (see Likelihood exercises I, problem 2). The approximation can be accurate when the number of parameters is small relative to the number of data points, when there are no boundary problems associated with θ , and of course it requires the posterior distribution to be proper.

When θ is a vector the same result (4.87) holds. Now $\hat{\theta}$ is the vector of posterior modes and the variance is the inverse of the matrix

$$- \frac{\partial^2}{\partial \theta \partial \theta'} p(\theta|y).$$

The reader is referred to Bernardo and Smith (1994) and references therein for a discussion of this delicate subject.

4.10 An Overview of Markov Chain Monte Carlo

The examples discussed so far do not pose computational challenges because the posterior distributions have standard forms. As a consequence, one can draw inferences analytically or using Monte Carlo methods by simulating directly from these posterior distributions. The posterior distributions of more complex models have seldom standard forms and so alternative methods are required. A very general method to obtain approximate samples from posterior distributions that does not require knowledge of their integration constant is Markov chain Monte Carlo (MCMC). MCMC has been successfully applied in many branches of science since its introduction to the statistical community in the early 1990s.

MCMC algorithms are recipes for constructing a Markov chain that has the posterior distribution of interest as its *stationary distribution*. The goal is to obtain draws from this posterior distribution. Since doing this directly may be very complicated or impossible, a Markov chain X_i , $i = 1, 2, \dots$, is constructed so that its stationary distribution is the posterior distribution. To achieve this, the variables X_i are drawn from a *proposal distribution*, but because this is not the same distribution as the posterior distribution, the drawn value is accepted in a stochastic manner by means of an *acceptance probability*. The derivation of this acceptance probability guarantees that the X_i 's are an approximate Monte Carlo sample from the stationary distribution. To meet this, the Markov chain must satisfy a number of conditions which hold in most cases (but not always!). The samples can be used for Monte Carlo estimation of various expectations with respect to the stationary distribution. Thus, Monte Carlo estimates of the mean, variance or posterior intervals can be obtained using the draws from the Markov chain.

In what follows, two algorithms are described that are special cases of a third very general MCMC algorithm. The focus is on examples that illustrate how the algorithms are applied. Technical details, including the derivation of the acceptance probability of the standard and general MCMC algorithms can be found in the appendix at the end of the chapter.

The algorithms are motivated with a simple example based on the normal linear model $y_i \stackrel{iid}{\sim} N(\mu, \sigma^2)$, where the vector of observations $y = \{y_i\}$ has n elements and the scalars μ and σ^2 are the mean and variance. The likelihood is proportional to

$$y|\mu, \sigma^2 \sim N(1\mu, I\sigma^2), \quad (4.88)$$

where 1 is a vector of ones with n elements and I is an $n \times n$ identity matrix. It is assumed that the prior distribution of the parameters factorises into two independent prior distributions of the form

$$\mu \propto \text{constant}, \quad (4.89a)$$

$$\sigma^2 \propto \frac{1}{\sigma^2}. \quad (4.89b)$$

The prior for the variance is equivalent to assuming an improper uniform prior for $\ln \sigma^2$. The resulting joint posterior is

$$\begin{aligned} p(\mu, \sigma^2 | y) &\propto p(y | \mu, \sigma^2) p(\sigma^2) p(\mu) \\ &\propto \exp \left[-\frac{\sum_{i=1}^n (y_i - \mu)^2}{2\sigma^2} \right] (\sigma^2)^{-(\frac{n}{2}+1)}. \end{aligned} \quad (4.90)$$

For this particular model, there is no need to use MCMC because explicit solutions for $p(\mu | y)$ and for $p(\sigma^2 | y)$ are available. However, an MCMC approach will be followed here.

4.11 The Metropolis-Hastings Algorithm

Two strategies are described. In the first, all the parameters of the model are updated simultaneously, whilst in the other, parameters are updated one at a time.

Joint Updating

Denote the target distribution by π (this is typically the posterior distribution) and let x be the current realisation of the chain. In the Metropolis-Hastings algorithm, a proposal Y is extracted from an arbitrary density $q(\cdot | x)$ that may depend on x . Then the proposal Y is accepted with probability $\alpha(x, Y)$ given by (4.91). More specifically, given that at cycle t , $X^{(t)} = x = (x_1^{(t)}, x_2^{(t)}, \dots, x_m^{(t)})$, one cycle of the algorithm is:

1. Generate Y from the proposal density $q(\cdot | x)$
2. Generate a uniform number U on $[0, 1]$
3. If

$$U < \alpha(x, Y) = \min \left\{ 1, \frac{\pi(Y) q(x | Y)}{\pi(x) q(Y | x)} \right\} \quad (4.91)$$

then $X^{[t+1]} = Y$. Otherwise $X^{[t+1]} = x$.

This is repeated until enough samples from the posterior distribution have been collected. I return shortly to evaluate what is “enough”.

In this formulation of the Metropolis-Hastings algorithm, all the components of $X^{(t)}$ are updated if the proposal is accepted.

Joint Updating: Example Using the Normal Model (4.88), (4.89)

Let Y_μ and Y_{σ^2} denote the proposed values for μ and for σ^2 , so that

$$Y = (Y_\mu, Y_{\sigma^2}),$$

and the state of the chain at stage t is

$$X^{(t)} = (\mu^{(t)}, \sigma^{2(t)}) = x.$$

Then,

$$\begin{aligned} \pi(Y) &= \exp \left[-\frac{\sum_{i=1}^n (y_i - Y_\mu)^2}{2Y_{\sigma^2}} \right] (Y_{\sigma^2})^{-\left(\frac{n}{2}+1\right)}, \\ \pi(x) &= \exp \left[-\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}} \right] (\sigma^{2(t)})^{-\left(\frac{n}{2}+1\right)}. \end{aligned}$$

Assume that a $N(m, D)$ random walk proposal is used for generating

$$(\ln Y_{\sigma^2}, Y_\mu),$$

with

$$\begin{aligned} m &= (\ln \sigma^2, \mu), \\ D &= \begin{bmatrix} k_{\sigma^2} & 0 \\ 0 & k_\mu \end{bmatrix}, \end{aligned}$$

and where k_{σ^2} and k_μ are user-tuned variance parameters for σ^2 and for μ . Then (see **Note** below: if σ^2 (a variance) is a variable that is restricted to be positive and $\ln \sigma^2$ is normally distributed, then σ^2 is lognormally distributed),

$$q(x|Y) = q(\mu^{(t)}, \sigma^{2(t)} | Y_\mu, Y_{\sigma^2}) = q(\mu^{(t)} | Y_\mu) q(\sigma^{2(t)} | Y_{\sigma^2}),$$

where

$$q\left(\sigma^{2(t)}|Y_{\sigma^2}\right) = (2\pi k_{\sigma^2})^{-\frac{1}{2}} \exp\left[-\frac{(\ln \sigma^{2(t)} - \ln Y_{\sigma^2})^2}{2k_{\sigma^2}}\right] \frac{1}{\sigma^{2(t)}},$$

$$q\left(\mu^{(t)}|Y_{\mu}\right) = (2\pi k_{\mu})^{-\frac{1}{2}} \exp\left[-\frac{(\mu^{(t)} - Y_{\mu})^2}{2k_{\mu}}\right].$$

Similarly,

$$q(Y|x) = q\left(Y_{\mu}, Y_{\sigma^2}|\mu^{(t)}, \sigma^{2(t)}\right) = q\left(Y_{\mu}|\mu^{(t)}\right) q\left(Y_{\sigma^2}|\sigma^{2(t)}\right),$$

where

$$q\left(Y_{\sigma^2}|\sigma^{2(t)}\right) = (2\pi k_{\sigma^2})^{-\frac{1}{2}} \exp\left[-\frac{(\ln Y_{\sigma^2} - \ln \sigma^{2(t)})^2}{2k_{\sigma^2}}\right] \frac{1}{Y_{\sigma^2}}, \quad (4.92)$$

$$q\left(Y_{\mu}|\mu^{(t)}\right) = (2\pi k_{\mu})^{-\frac{1}{2}} \exp\left[-\frac{(Y_{\mu} - \mu^{(t)})^2}{2k_{\mu}}\right]. \quad (4.93)$$

Manipulating these expressions shows that the proposal $Y = (Y_{\mu}, Y_{\sigma^2})$ is accepted with probability given by (4.91), equal to

$$\begin{aligned} & \frac{\pi(Y) q(x|Y)}{\pi(x) q(Y|x)} = \\ & = \exp\left[\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}} - \frac{\sum_{i=1}^n (y_i - Y_{\mu})^2}{2Y_{\sigma^2}}\right] \left(\frac{Y_{\sigma^2}}{\sigma^{2(t)}}\right)^{-\frac{n}{2}}. \end{aligned} \quad (4.94)$$

In a programming environment in order to avoid under or overflows, one computes

$$\begin{aligned} & \ln \frac{\pi(Y) q(Y; x)}{\pi(x) q(x; Y)} = \\ & = \left[\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}} - \frac{\sum_{i=1}^n (y_i - Y_{\mu})^2}{2Y_{\sigma^2}}\right] + \frac{n}{2} (\ln \sigma^{2(t)} - \ln Y_{\sigma^2}) \end{aligned}$$

and in the final stage

$$\frac{\pi(Y) q(x|Y)}{\pi(x) q(Y|x)}$$

is calculated.

The user must choose the tuning parameters judiciously because this has a strong influence on the behaviour of the chain.

Note on the Theory of Transformations of Random Variables

Let $X = \ln \sigma^2$. Assume

$$X \sim N(\mu, v). \quad (4.95)$$

Then $Y = \sigma^2 = \exp X$ follows a lognormal distribution

$$Y = \sigma^2 = \exp X \sim \ln N(\mu, v), \quad 0 < Y < \infty, \quad -\infty < \mu < \infty, \quad v > 0, \quad (4.96)$$

with density

$$p(y) = (2\pi v)^{-\frac{1}{2}} \exp\left(-\frac{1}{2v} (\ln y - \mu)^2\right) \frac{1}{y}. \quad (4.97)$$

To show this I use the theory of transformation of random variables. If X has probability density function $p_X(x)$, that is

$$X \sim p_X(x),$$

and $Y = f(X)$ such that the inverse function f^{-1} exists, from which X can be retrieved

$$X = f^{-1}(Y),$$

then the probability density function of Y , $p_Y(y)$ is

$$p_Y(y) = p_X(f^{-1}(Y)) \left| \frac{df^{-1}(Y)}{dY} \right|, \quad (4.98)$$

where $|df^{-1}(Y)/dY|$ is the absolute value of the Jacobian of the transformation. The notation $p_X(f^{-1}(Y))$ implies that one uses the form of the density of X (in the example, a $N(\mu, v)$) and replaces in this density, x by $f^{-1}(y)$. In the present case the objective is to derive the distribution of the random variable $Y = \exp X = \sigma^2$. Then,

$$Y = f(X) = \exp(X) \implies X = f^{-1}(Y) = \ln Y, \quad \left| \frac{df^{-1}(Y)}{dY} \right| = \frac{1}{Y}.$$

Using (4.98)

$$p_Y(y) = (2\pi v)^{-\frac{1}{2}} \exp \left[-\frac{(\ln y - \mu)^2}{2v} \right] \frac{1}{y}, \quad 0 < Y < \infty, \quad -\infty < \mu < \infty, \\ v > 0, \quad (4.99)$$

which is the density of the lognormal distribution with parameters μ and v .

Single Site Updating

For a single-site updating Metropolis-Hastings algorithm, the components $x_1^{(t)}, x_2^{(t)}, \dots, x_m^{(t)}$ are updated individually in a random or systematic order. Typically, a systematic order is chosen starting with $x_1^{(t)}$, then $x_2^{(t)}$ and so on, until the last component $x_m^{(t)}$ is updated.

Assume that the current state is $X^{(t)} = (x_1^{(t)}, x_2^{(t)}, \dots, x_m^{(t)}) = x$ and that the j th component is to be updated. Then the next stage of the chain $X^{(t+1)}$ only differs from $X^{(t)}$ on the j th component that is generated as follows:

1. Generate a proposal Y_j from the proposal density $q_j(\cdot|x)$. Let

$$Y = (x_1^{(t)}, x_2^{(t)}, \dots, x_{j-1}^{(t)}, Y_j, x_{j+1}^{(t)}, \dots, x_m^{(t)})$$

2. Generate a uniform number U on $[0, 1]$
3. If

$$U < \alpha_j(x, Y) = \min \left\{ 1, \frac{\pi(Y) q_j(x_j|Y)}{\pi(x) q_j(Y_j|x)} \right\}, \quad (4.100)$$

then $X^{(t+1)} = Y$. Otherwise $X^{(t+1)} = x$.

Single-Site Updating: Example Using the Normal Model (4.88), (4.89)

1. Updating σ^2 .

As before, the state of the chain at stage t is

$$X^{(t)} = (\mu^{(t)}, \sigma^{2(t)}) = x,$$

but now

$$Y = \left(\mu^{(t)}, Y_{\sigma^2} \right).$$

Then,

$$\pi(Y) \propto \exp \left(-\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2Y_{\sigma^2}} \right) (Y_{\sigma^2})^{-(\frac{n}{2}+1)},$$

and

$$\pi(x) \propto \exp \left(-\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}} \right) (\sigma^{2(t)})^{-(\frac{n}{2}+1)}.$$

The proposal density of Y_{σ^2} is

$$\begin{aligned} q_j(Y_j|x) &= q_{\sigma^2}(Y_{\sigma^2}|\sigma^{2(t)}) \\ &= (2\pi k_{\sigma^2})^{-\frac{1}{2}} \exp \left[-\frac{(\ln Y_{\sigma^2} - \ln \sigma^{2(t)})^2}{2k_{\sigma^2}} \right] \frac{1}{Y_{\sigma^2}}. \end{aligned}$$

Likewise,

$$\begin{aligned} q_j(x_j|Y) &= q_{\sigma^2}(\sigma^{2(t)}|Y_{\sigma^2}) \\ &= (2\pi k_{\sigma^2})^{-\frac{1}{2}} \exp \left[-\frac{(\ln \sigma^{2(t)} - \ln Y_{\sigma^2})^2}{2k_{\sigma^2}} \right] \frac{1}{\sigma^{2(t)}}. \end{aligned}$$

Manipulating these expressions, it is easy to show that the proposal is accepted with probability (4.100), where

$$\begin{aligned} &\frac{\pi(Y) q_j(x_j|Y)}{\pi(x) q_j(Y_j|x)} \\ &= \exp \left[\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}} - \frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2Y_{\sigma^2}} \right] \left(\frac{Y_{\sigma^2}}{\sigma^{2(t)}} \right)^{-\frac{n}{2}}. \end{aligned} \quad (4.101)$$

In practice and in order to avoid under or overflows, one computes

$$\ln \frac{\pi(Y) q_j(x_j|Y)}{\pi(x) q_j(Y_j|x)} = \left[\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}} - \frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2Y_{\sigma^2}} \right] + \frac{n}{2} (\ln \sigma^{2(t)} - \ln Y_{\sigma^2})$$

and at the final stage

$$\frac{\pi(Y) q_j(x_j|Y)}{\pi(x) q_j(Y_j|x)}$$

is calculated.

2. Updating μ .

The state of the chain at stage t is again

$$X^{(t)} = (\mu^{(t)}, \sigma^{2(t)}) = x,$$

but now,

$$\begin{aligned} Y &= (Y_\mu, \sigma^{2(t)}), \\ \pi(Y) &\propto \exp \left[-\frac{\sum_{i=1}^n (y_i - Y_\mu)^2}{2\sigma^{2(t)}} \right] (\sigma^{2(t)})^{-(\frac{n}{2}+1)}, \\ \pi(x) &\propto \exp \left[-\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}} \right] (\sigma^{2(t)})^{-(\frac{n}{2}+1)}. \end{aligned}$$

The proposal densities are:

$$\begin{aligned} &q_j(Y_j|x) \\ &= q_\mu(Y_\mu|\mu^{(t)}) = (2\pi k_\mu)^{-\frac{1}{2}} \exp \left[-\frac{(Y_\mu - \mu^{(t)})^2}{2k_\mu} \right], \end{aligned}$$

and

$$\begin{aligned} &q_j(x_j|Y) \\ &= q_\mu(\mu^{(t)}|Y_\mu) = (2\pi k_\mu)^{-\frac{1}{2}} \exp \left[-\frac{(\mu^{(t)} - Y_\mu)^2}{2k_\mu} \right]. \end{aligned}$$

These densities are symmetric and therefore cancel each other in the ratio. The proposal is accepted with probability (4.100), where

$$\frac{\pi(Y) q_j(x_j|Y)}{\pi(x) q_j(Y_j|x)} = \exp \left[\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}} - \frac{\sum_{i=1}^n (y_i - Y_\mu)^2}{2\sigma^{2(t)}} \right].$$

Note: A slightly different implementation of the algorithm is to parametrise the model in terms of $\ln \sigma^2$. Then the posterior is now proportional to

$$\exp \left[-\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^2} \right] (\sigma^2)^{-\frac{n}{2}},$$

$q_{\sigma^2}(Y_{\sigma^2} | \ln \sigma^{2(t)})$ is given by

$$(2\pi k_{\sigma^2})^{-\frac{1}{2}} \exp \left[-\frac{(\ln Y_{\sigma^2} - \ln \sigma^{2(t)})^2}{2k_{\sigma^2}} \right],$$

and one can verify that the acceptance probability for Y_{σ^2} is the same as in (4.101).

4.12 The Gibbs Sampling Algorithm

The Gibbs sampler is a very popular MCMC algorithm because of its computational simplicity. The mechanics is as follows. Consider the vector of parameters of a model $(\theta_1, \theta_2, \dots, \theta_p)$, with posterior density $p(\theta_1, \theta_2, \dots, \theta_p | y)$ known up to proportionality. Assume that the user supplies “legal” starting values

$$(\theta_1^{(0)}, \theta_2^{(0)}, \dots, \theta_p^{(0)}),$$

in the sense that $p(\theta_1^{(0)}, \theta_2^{(0)}, \dots, \theta_p^{(0)} | y) > 0$. The implementation of the Gibbs sampler consists of iterating through the loop:

draw $\theta_1^{(1)}$ from $p(\theta_1 | \theta_2^{(0)}, \dots, \theta_p^{(0)}, y)$,

draw $\theta_2^{(1)}$ from $p(\theta_2 | \theta_1^{(1)}, \theta_3^{(0)}, \dots, \theta_p^{(0)}, y)$,

draw $\theta_3^{(1)}$ from $p(\theta_3|\theta_1^{(1)}, \theta_2^{(1)}, \theta_4^{(0)}, \dots, \theta_p^{(0)}, y)$,
 $\vdots$
 draw $\theta_p^{(1)}$ from $p(\theta_p|\theta_1^{(1)}, \dots, \theta_{p-1}^{(1)}, y)$,
 draw $\theta_1^{(2)}$ from $p(\theta_1|\theta_2^{(1)}, \dots, \theta_p^{(1)}, y)$,
 $\vdots$
 and so on.

After an initial period during which samples are dependent on the starting value (burn-in period), the draws $\theta_1^{(i)}, \theta_2^{(i)}, \dots, \theta_p^{(i)}$, for sufficiently large i , when the sampler converges, are regarded as samples from the normalised posterior distribution with density

$$p(\theta_1, \theta_2, \dots, \theta_p|y) / \int p(\theta_1, \theta_2, \dots, \theta_p|y) d\theta_1 \dots d\theta_p.$$

The coordinate $\theta_j^{(i)}$ is regarded as a draw from its marginal posterior distribution with density

$$p(\theta_j|y) / \int p(\theta_j|y) d\theta_j.$$

In more general terms, let

$$\theta_{-i} = (\theta_1, \dots, \theta_{i-1}, \theta_{i+1}, \dots, \theta_p)$$

be the vector of dimension $(p - r)$, $p > r$, $r \geq 1$, which is equal to θ with its i th component, θ_i , deleted and where r is the number of elements in θ_i . The density of the fully conditional posterior distribution (fcpd) of θ_i is

$$\begin{aligned}
 p(\theta_i|\theta_{-i}, y) &= \frac{p(\theta_1, \dots, \theta_{i-1}, \theta_i, \theta_{i+1}, \dots, \theta_p|y)}{\int p(\theta_1, \dots, \theta_{i-1}, \theta_i, \theta_{i+1}, \dots, \theta_p|y) d\theta_i} \\
 &\propto p(\theta_1, \dots, \theta_{i-1}, \theta_i, \theta_{i+1}, \dots, \theta_p|y). \tag{4.102}
 \end{aligned}$$

In many applications $r = 1$ and parameters are updated one at a time. In general, single-site updating leads to moves along each coordinate, whereas updating several components in a block allows for more general moves. Joint updating, which incorporates information on the correlation structure among the components in the joint conditional posterior distribution, can result in faster convergence when correlations are strong (Liu et al. 1994). A computational strategy that makes it

feasible to draw the entire vector θ for some Gaussian linear models is described in Sorensen and Gianola (2002), p. 584, originally proposed by García-Cortés and Sorensen (1996).

The Gibbs sampler is a special case of the more general Metropolis-Hastings algorithm. In the Gibbs sampler the proposals (drawn from the fully conditional posterior distributions) are always accepted (see the **Note** below). It is simpler to implement than the Metropolis-Hastings algorithm because it does not require tuning parameters. However, it requires knowledge of the form of the fully conditional posterior distributions.

Example Using the Normal Model

Consider the normal model defined by Eqs. (4.88) and (4.89). The fully conditional posterior distributions with density $p(\mu|\sigma^2, y)$ and $p(\sigma^2|\mu, y)$ are derived using the posterior distribution (4.90) as starting point. To obtain $[\mu|\sigma^2, y]$, one extracts from the posterior distribution (4.90) those terms that include μ . This results in

$$\begin{aligned} p(\mu|\sigma^2, y) &\propto \exp\left(-\frac{\sum_{i=1}^n (y_i - \mu)^2}{2\sigma^2}\right) \\ &= \exp\left(-\frac{1}{2\sigma^2} \left[n(\bar{y} - \mu)^2\right]\right) \end{aligned}$$

which is the kernel of a normal distribution with mean $\bar{y}$ and variance σ^2/n . Therefore

$$\mu|\sigma^2, y \sim N\left(\bar{y}, \frac{\sigma^2}{n}\right). \quad (4.103)$$

To obtain $[\sigma^2|\mu, y]$, one extracts from the posterior distribution (4.90) those terms that include σ^2 . This results in

$$p(\sigma^2|\mu, y) \propto \exp\left(-\frac{\sum_{i=1}^n (y_i - \mu)^2}{2\sigma^2}\right) (\sigma^2)^{-(\frac{n}{2}+1)}$$

which seen as a function of σ^2 is the kernel of a scale inverted chi-square distribution with n degrees of freedom and scale

$$s^2 = \frac{\sum_{i=1}^n (y_i - \mu)^2}{n}.$$

Therefore

$$\sigma^2 | \mu, y \sim n s^2 \chi_n^{-2}. \quad (4.104)$$

Note: The Acceptance Probability of the Gibbs Sampler Is Equal to 1

Let the state of the chain at stage t be

$$X_t = (\mu^{(t)}, \sigma^{2(t)}) = x$$

and let

$$Y = (\mu^{(t)}, Y_{\sigma^2}).$$

Then

$$\pi(Y) = \pi(\mu^{(t)}, Y_{\sigma^2} | y) \propto \exp\left(-\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2Y_{\sigma^2}}\right) (Y_{\sigma^2})^{-(\frac{n}{2}+1)},$$

and

$$\pi(x) = \pi(\mu^{(t)}, \sigma^{2(t)} | y) \propto \exp\left(-\frac{\sum_{i=1}^n (y_i - \mu^{(t)})^2}{2\sigma^{2(t)}}\right) (\sigma^{2(t)})^{-(\frac{n}{2}+1)}.$$

The proposal density of Y_{σ^2} is

$$q(Y_{\sigma^2} | x) = \pi(Y_{\sigma^2} | \mu^{(t)}, y)$$

and

$$q(x | Y_{\sigma^2}) = \pi(\sigma^{2(t)} | \mu^{(t)}, y).$$

Then the acceptance probability for updating σ^2 is

$$\begin{aligned} \frac{\pi(Y) q(x | Y_{\sigma^2})}{\pi(x) q(Y_{\sigma^2} | x)} &= \frac{\pi(Y)}{\pi(x)} \frac{\pi(\sigma^{(t)}, \mu^{(t)} | y)}{\pi(\mu^{(t)} | y)} \frac{\pi(\mu^{(t)} | y)}{\pi(Y_{\sigma^2}, \mu^{(t)} | y)} \\ &= \frac{\pi(Y)}{\pi(x)} \frac{\pi(x)}{\pi(\mu^{(t)} | y)} \frac{\pi(\mu^{(t)} | y)}{\pi(Y)} = 1. \end{aligned}$$

A similar result is obtained for the acceptance probability for updating μ .

4.13 Output Analysis

McMC is used to obtain approximate samples from posterior distributions. The information about unknowns contained in these samples depends on their size (length of the chain) and on the degree of autocorrelation among sampled values. When a particular feature from a posterior distribution is estimated from the samples, the uncertainty associated with the McMC estimate is the Monte Carlo error of estimation. This is a classical frequentist sampling uncertainty that depends on the samples. In contrast, the posterior uncertainty, given the model, depends on the data. It is important to distinguish between these two sources of uncertainty.

There is a large literature on McMC convergence diagnosis and output analysis. A useful practical reference is Kass et al. (1998). Here the focus is on a few important issues of output analysis.

The first is to determine the length of the so-called *burn-in period*. This is related to the question of convergence of the sampled values to the target distribution. The chain is initialised with values of parameters that are not drawn from the posterior distribution. Since convergence, if reached, to the posterior distribution is gradual, one must decide how many of the initial values must be discarded in order to include only representative ones to draw Monte Carlo inferences. Figure 4.1 provides an illustration. The figure displays traceplots from the simulated values from $[\mu|y]$ (left panel) and from $[\sigma^2|y]$ (right panel) obtained with the single-site Metropolis-Hastings algorithm applied to model (4.88), (4.89a), (4.89b). Such a traceplot discloses information about the burn-in period and at the same time provides an informal check for convergence. The values of μ and σ^2 start far away from the support of the respective marginal posterior distributions but seem to reach convergence after a few iterations (burn-in). There are a number of methods

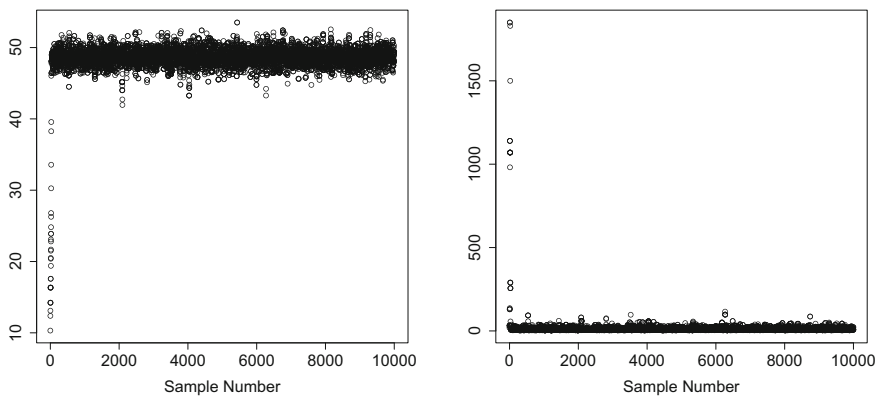


Fig. 4.1 Draws from the posterior distribution $[\mu|y]$ (left panel) and from $[\sigma^2|y]$ (right panel) obtained from the single-site Metropolis-Hastings algorithm

that attempt to provide more formal checks of convergence than the “eye-balling” approach displayed by traceplots. None can guarantee convergence but can flag lack of convergence. A useful reference is Cowles and Carlin (1996).

A second concern is to determine the *degree of autocorrelation* of the sampled values and the so-called *effective chain length or effective sample size*. These determine the *McMC-sample variance or MC variance* of estimates of features of posterior distributions. Because the number of sampled values from the estimated posterior distribution is finite, there is always sampling uncertainty associated with an estimator of features of posterior distributions. In principle, the sampling uncertainty can be made as small as desired by taking a sufficiently large number of samples.

The MC variance can be estimated by running several independent chains and then calculating the empirical, between-chain variance of the estimates obtained for each chain. Since this is often computationally expensive, the algorithm is often implemented running a single long chain and one resorts to theoretical estimators of the MC variance. These estimators account for the autocorrelation among the samples taken from the target distribution. Useful references are Ripley (1987), Geyer (1992), and Chen et al. (2000).

When the Bayesian model is implemented with the Metropolis-Hastings algorithm, the success of the method depends on striking some “right” rate of acceptance. Parameterisation and choice of the proposal density play a fundamental role here. Acceptance ratios in the neighbourhood of 1 imply very similar values between previous and proposed states and the chain will move very slowly (unless, of course, the proposal distribution is the equilibrium distribution, leading to an acceptance ratio of 1!). On the other hand, if the proposed displacement is too large and falls where the posterior has no support, this will lead to a high rejection rate. Here, the chain will remain in the same state for many iterations. No general optimization rules are available. However, Roberts et al. (1997) obtained optimal acceptance rates of 23.4% for high-dimensional problems under quite general conditions. Guidance on the proper choice of proposal densities can be found, for example, in Chib and Greenberg (1995).

Consider the output of a Markov chain consisting of m samples $X^{(1)}, X^{(2)}, \dots, X^{(m)}$, where the $X^{(i)}$ are approximate draws from the posterior distribution $[X|y]$, where X is a vector of parameters, y is a vector of data and m is the length of the chain. The goal is to obtain and estimate of the mean of some function of X , $h(X)$, over the posterior distribution of X , with density $p(x|y)$

$$E[h(X)|y] = \int h(x) p(x|y) dx$$

using

$$\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m h(X^{(i)}). \quad (4.105)$$

Here $h(X)$ is any function of X with finite expectation. If h is the identity function (4.105) retrieves an estimator of the mean.

A central limit theorem asserts

$$\sqrt{m} (\hat{\mu}_m - E[h(X) | y]) \xrightarrow{\mathcal{D}} N(0, V_{\text{asympt}}), \quad (4.106)$$

where V_{asympt} is the asymptotic variance of X . A little more informally this can be written

$$\hat{\mu}_m \sim N(E[h(X) | y], V_{\text{asympt}}/m).$$

From (4.106) an approximate 95% confidence interval for the unknown true value $E[h(X) | y]$ is, $\hat{\mu}_m \pm 1.96\sqrt{V_{\text{asympt}}/m}$. With independent sampling, measures of sampling uncertainty are obtained by replacing the unknown V_{asympt} by the sample variance of X .

The sampling variance of the estimator (4.105) is V_{asympt}/m where

$$\begin{aligned} V_{\text{asympt}} &= \lim_{m \rightarrow \infty} \text{Var}(\sqrt{m}\hat{\mu}_m) \\ &= \text{Var}[h(X) | y] \left(1 + 2 \sum_{j=1}^{\infty} \rho_j \right). \end{aligned} \quad (4.107)$$

(see the **Note 2** below) and $\text{Var}[h(X) | y]$ is the posterior variance of $h(X)$ under the limiting distribution. Above,

$$\begin{aligned} \rho_j &= \frac{\text{Cov}[(h(X^{(i)}), h(X^{(i+j)})) | y]}{\text{Var}[h(X^{(i)}) | y]} \\ &= \frac{\gamma(j)}{\gamma(0)}, \quad j = 1, 2, \dots \end{aligned}$$

for all i , where i refers to the i th draw and ρ_j is the lag- j autocorrelation of $X^{(1)}, X^{(2)}, \dots, X^{(m)}$. The above assumes that, under stationarity, the lag-covariance is constant for constant lag j (for example, for $j = 2$, $\gamma(2) = \text{Cov}[(h(X^{(1)}), h(X^{(3)})) | y] = \text{Cov}[(h(X^{(2)}), h(X^{(4)})) | y]$) and that $\text{Cov}[(h(X^{(i)}), h(X^{(i)})) | y] = \text{Var}[h(X^{(i)}) | y] = \gamma(0)$.

Note 1

The important property defined by expression (4.105) is used repeatedly in the book. For example (4.105) is an estimator of

- the posterior mean if $h(X) = X$;
- the posterior variance if $h(X) = [X - E(X|y)]^2$. The estimator of the posterior variance is

$$\frac{1}{m} \sum_{i=1}^m \left[X^{(i)2} - [\widehat{E}(X|y)]^2 \right],$$

where $\widehat{E}(X|y) = \frac{1}{m} \sum_i X^{(i)}$;

- the posterior probability that $X \in A$, if $h(X) = I(X \in A)$, such that

$$\Pr(X \in A|) = \int I(X \in A) p(X|y) dX.$$

Here, the estimator is $\sum_{i=1}^m I(X^{(i)} \in A) / m$. As a special case, the cumulative distribution function is estimated

$$\widehat{F}(t) = \frac{1}{m} \sum_{i=1}^n I(X^{(i)} < t).$$

Therefore, the estimator of the posterior probability that $t_1 < X < t_2$ is

$$\widehat{\Pr}(t_1 < X < t_2|y) = \frac{1}{m} \left[\sum_{i=1}^m I(t_1 < X^{(i)} < t_2) \right];$$

- the posterior predictive density:

$$p(z|y) = \int p(z|X, y) p(X|y) dX,$$

where, usually, the form of the problem is such that $p(z|X, y) = p(z|X)$. In this setting, $h(X) = p(z|X)$, and the estimator of the predictive density is $\sum_{i=1}^m p(z|X^{(i)}) / m$.

This important property of MCMC is used to obtain Monte Carlo estimates of the marginal posterior distribution of mean squared errors and of false discovery rate.

Note 2

To arrive at (4.107) one writes

$$\begin{aligned}
 m \operatorname{Var}(\hat{\mu}_m) &= m^{-1} \sum_{i,j} \operatorname{Cov} \left[\left(h(X^{(i)}) \right), h(X^{(j)}) \right] |y \\
 &= \operatorname{Var}[h(X) | y] \left[1 + 2 \sum_{j=1}^{m-1} \left(1 - \frac{j}{m} \right) \rho_j \right]. \quad (4.108)
 \end{aligned}$$

When $\rho_j = 0$ for all j ,

$$\operatorname{Var}(\hat{\mu}_m) = \frac{\operatorname{Var}[h(X) | y]}{m}$$

the familiar equation for the variance of the sample mean (assuming independent samples). The term in square brackets in (4.108) converges to $\left[1 + 2 \sum_{j=1}^{\infty} \rho_j \right]$ as $m \rightarrow \infty$, in which case

$$\begin{aligned}
 m \operatorname{Var}(\hat{\mu}_m) &= V_{\text{asympt}} \\
 \lim_{m \rightarrow \infty} m \operatorname{Var}(\hat{\mu}_m) &= \operatorname{Var}[h(X) | y] \left[1 + 2 \sum_{j=1}^{\infty} \rho_j \right] \\
 &= \operatorname{Var}[h(X) | y] \tau,
 \end{aligned}$$

where $\operatorname{Var}[h(X^{(i)}) | y] = \operatorname{Var}[h(X) | y]$ for all i , the posterior variance that, given the model, depends on data y and $\tau = 1 + 2 \sum_{j=1}^{\infty} \rho_j$ is the *integrated autocorrelation* that is modulated by the MCMC method used. The *effective sample size* is

$$m_{\text{eff}} = \frac{m}{\tau},$$

which is equal to m if the draws from $[X|y]$ are uncorrelated. It is important to distinguish between $\operatorname{Var}[h(X) | y]$, the variance of $h(X)$ under the limiting distribution $[X|y]$ and $\operatorname{Var}(\sqrt{m}\hat{\mu}_m)$ the limiting variance of $\sqrt{m}\hat{\mu}_m$, equal to $\operatorname{Var}[h(X) | y] \tau$. The former is associated with the posterior uncertainty of $h(X)$ while the latter is associated with the sampling scheme from the posterior distribution. Under independent sampling $\tau = 1$ and therefore both are equal to $\operatorname{Var}[h(X) | y]$.

There are a number of methods that can be used to estimate the Monte Carlo variance of features from posterior distributions, two of which are sketched below.

Geyer's Estimator of the Monte Carlo Variance

Geyer (1992) suggests the following estimator of $\text{Var}(\hat{\mu}_m)$ based on time-series theory (a classical reference is Priestley 1981). The lag- t autocovariance is

$$\gamma(t) = \text{Cov} \left[\left(h(X^{(i)}) \right), h(X^{(i+t)}) \mid y \right],$$

which under stationarity is the same for all i . The estimator is

$$\hat{\gamma}(t) = \frac{1}{m} \sum_{i=1}^{m-t} \left\{ \left[h(X^{(i)}) - \hat{\mu}_m \right] \left[h(X^{(i+t)}) - \hat{\mu}_m \right] \right\}.$$

A Monte Carlo estimator of the asymptotic variance is

$$\hat{V}_{\text{asympt}} = \hat{\gamma}(0) + 2 \sum_{i=1}^{i=2\delta+1} \hat{\gamma}(i) \quad (4.109)$$

and a Monte Carlo estimator of the variance of (4.105) is

$$\widehat{\text{Var}}(\hat{\mu}_m) = \frac{1}{m} \left[\hat{\gamma}(0) + 2 \sum_{i=1}^{i=2\delta+1} \hat{\gamma}(i) \right] \quad (4.110)$$

where δ is chosen such that it is the largest integer satisfying

$$\hat{\gamma}(2\delta') + \hat{\gamma}(2\delta' + 1) > 0, \quad \delta' = 0, 1, \dots, \delta.$$

Above,

$$\begin{aligned} \hat{\gamma}(0) &= \widehat{\text{Var}} \left[h(X^{(i)} \mid y) \right] \\ &= \widehat{\text{Var}} [h(X) \mid y], \quad \text{for all } i. \end{aligned}$$

An estimator of the integrated autocorrelation is obtained from (4.110) as follows:

$$\begin{aligned} \widehat{\text{Var}}(\hat{\mu}_m) &= \frac{\hat{\gamma}(0)}{m} \left[1 + 2 \sum_{i=1}^{i=2\delta+1} \frac{\hat{\gamma}(i)}{\hat{\gamma}(0)} \right] \\ &= \frac{\hat{\gamma}(0)}{m} \left[1 + 2 \sum_{i=1}^{i=2\delta+1} \hat{\rho}(i) \right]. \end{aligned} \quad (4.111)$$

Therefore,

$$\begin{aligned}\hat{\tau} &= 1 + 2 \sum_{i=1}^{i=2\delta+1} \hat{\rho}(i) \\ &= \frac{m \widehat{\text{Var}}(\hat{\mu}_m)}{\hat{\gamma}(0)}.\end{aligned}\tag{4.112}$$

The estimator of effective sample size is

$$\begin{aligned}\hat{m}_{\text{eff}} &= \frac{m}{\hat{\tau}} \\ &= \frac{\hat{\gamma}(0)}{\widehat{\text{Var}}(\hat{\mu}_m)}.\end{aligned}\tag{4.113}$$

The Method of Batching

A popular method of estimating Monte Carlo variances that is easy to implement is known as “batching” (Hastings 1970). It is based on the idea that if individual draws $X^{(j)}$ are correlated, grouping successive draws into b batches or groups of size s each and computing the raw averages will lead to b batch means that are less strongly inter-correlated than the original draws. This can be so, provided that s is chosen appropriately. The larger the autocorrelation among samples the larger s must be. Suppose that a chain of total length m is divided into b batches each of size s . Let the average of the i th batch be

$$\bar{x}_i = \frac{1}{s} \sum_{j=1}^s X^{(j)}, \quad i = 1, 2, \dots, b.$$

Here, $h(X^{(j)})$ is some feature of the posterior distribution evaluated at the sampled value $X^{(j)}$. The batch estimator of the variance of (4.105), assuming that s is large enough so that the $\bar{x}_i$'s are uncorrelated, is equal to

$$\widehat{\text{Var}}_b(\hat{\mu}_m) = \frac{\sum_{i=1}^b (\bar{x}_i - \hat{\mu}_m)^2}{b(b-1)}.\tag{4.114}$$

An estimate of the batch-effective chain length can be obtained as

$$\hat{m}_{\text{eff}_b} = \frac{\sum_{i=1}^m [X^{(i)} - \hat{\mu}_m]^2}{(m-1) \widehat{\text{Var}}_b(\hat{\mu}_m)}. \quad (4.115)$$

When the samples are independent, $s = 1$, $b = m$, $\bar{x}_i = X^{(i)}$ for all i and $\hat{m}_{\text{eff}_b} = m$.

If the autocorrelation among the samples of the chain is very high (>0.95), estimator (4.110) seems to be preferred over (4.114).

Example: A Simulated Autoregressive Process

These concepts are illustrated with an autoregressive model that generates draws (vector y) of length 10,000 mimicking the correlated structure of a Monte Carlo Markov chain. A plot of the autocorrelations among the elements of y versus lag gives a visual impression of the output. The R function `acf(y)` (see the bottom of the second R-code below) is used to generate Fig. 4.2 (left panel). The figure indicates that at a lag of approximately 20 the autocorrelation among the y 's vanishes.

The R-code below has 2 sections. The first generates the 10,000 correlated samples y using an autocorrelation equal to 0.8. The second section computes the Monte Carlo variance of the mean of y , based on (4.110), the effective chain length based on (4.113) and the integrated autocorrelation based on (4.112).

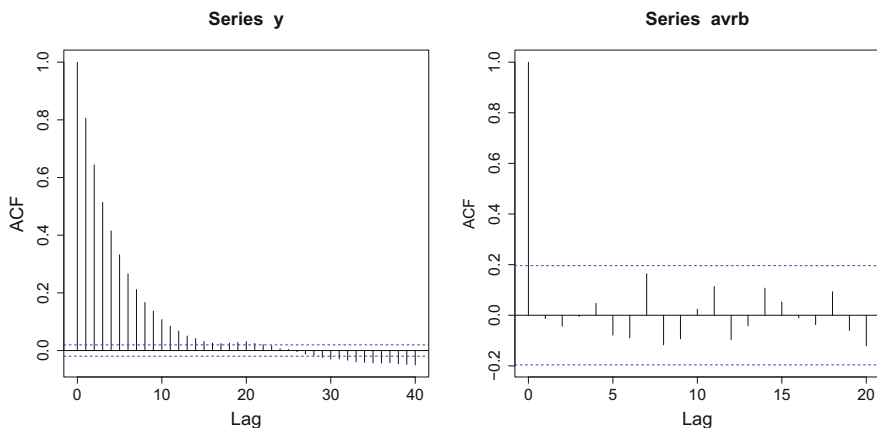


Fig. 4.2 Left: Autocorrelations between samples of y_i versus lag; Right: Autocorrelations between batch means

The R-code is as follows:

```
# CODE0401
# CODE FOR THE MC VARIANCE TESTED ON
#A FIRST-ORDER AUTOREGRESSIVE PROCESS WITH CORRELATION RHO
rm(list=ls()) # Clear the workspace
set.seed(1237)
#GENERATE DATA: AUTOREGRESSIVE PROCESS WITH CORRELATION RHO
ns <- 10000
y <- rep(NA,ns)
rho<-0.8
sum <- 0
y[1] <- rnorm(1,0,1)
for(i in 2:ns)
{
  y[i] <- rho*y[i-1] + rnorm(1,0,1)
  sum <- sum + y[i]*y[i-1]
}
cov <- sum/ns
rhat <- cov/var(y)
muhat <- mean(y)
gama0 <- var(y)
rhat

## [1] 0.7971166

# CODE FOR THE MC VARIANCE BASED ON GEYER
svar<-var(y)*(ns-1)/ns
tau<-1
tausum<-0
ptm <- proc.time()
for(i in 0:ns)
{
  gamaj<-0.0
  gamak<-0.0
  j<-2*i
  k<-(2*i)+1
# FASTER CODE: JUMP THE LOOP AND USE FUNCTION ACF
# for (ii in 1:(ns-j))
# {
#   cov<-(y[ii]*y[ii+j]-mean(y)*(y[ii]+y[ii+j])+mean(y)*mean(y))
#   gamaj<-gamaj+cov
# }
# for(ii in 1:(ns-k))
# {
#   gamak<-gamak+(y[ii]*y[ii+k]-mean(y)*(y[ii]+y[ii+k])+
#     mean(y)*mean(y))
# }
# gamaj<-gamaj/ns
# gamak<-gamak/ns

lag1<-j
lag2<-k
```

```

#USE THE R-FUNCTION ACF TO COMPUTE AUTOCORRELATIONS
cov1<-acf(y,type="covariance",lag.max=lag1,plot=FALSE)
cov2<-acf(y,type="covariance",lag.max=lag2,plot=FALSE)
gama<-cov1$acf[lag1+1]
gama2<-cov2$acf[lag2+1]
tau<-gama+gama2
if(tau<0)
{
  break
}
tausum<-tausum+tau
}
proc.time()-ptm

```

```

##      user  system elapsed
##    0.06    0.02    0.08

```

```
muhat
```

```
## [1] -0.003605249
```

```
gama0
```

```
## [1] 2.723108
```

```
varch<- svar+2*tausum
varch
```

```
## [1] 26.93673
```

```
mcvar<-varch/ns
mcvar
```

```
## [1] 0.002693673
```

```
efchsize<-svar/mcvar
efchsize
```

```
## [1] 1010.826
```

```
integrautoc<-varch/svar
integrautoc
```

```
## [1] 9.892896
```


The estimated feature is the mean of the distribution

$$\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m X^{(i)} = -0.0036$$

where $m = 10,000$ is the length of the Markov chain. Using the method of Geyer,

$$\begin{aligned}\widehat{V}_{\text{asympt}} &= \widehat{\text{Var}}\left(X^{(i)}|y\right) \hat{\tau} = 2.72 \times 9.89 = 26.94, \\ \widehat{\text{Var}}\left(\hat{\mu}_m\right) &= 0.0027, \\ \hat{m}_{\text{eff}} &= 1011, \\ \hat{\tau} &= 9.89.\end{aligned}$$

where $\widehat{\text{Var}}\left(X^{(i)}|y\right)$ is the Monte Carlo estimate of the variance of the posterior distribution $[X|y]$.

The R-code below uses the method of batching to compute the Monte Carlo variance of the mean of y , based on (4.114) and the effective chain length based on (4.115). The batch size was set equal to 100. Figure 4.2, right panel, is generated using `acf(avrb)` and shows that batch means are effectively uncorrelated. The output is printed at the bottom of the code and compares well with the previous results.

```
#CODE FOR THE MC VARIANCE BASED ON BATCHING
#choose size of batch s
s<-100
x<-matrix(y,ncol=s, byrow=FALSE)
avrb<-apply(x,2,mean)
mcvarb<-var(avrb)/length(avrb)
efchsizebatch<-var(y)/mcvarb
mcvarb
```

```
## [1] 0.002656311
```

```
efchsizebatch
```

```
## [1] 1025.147
```

```
#PLOT THE AUTOCORRELATION VS LAG USING R-FUNCTION acf
#require(graphics)
#acf(y)
#acf(avrb)
```

The top code was also executed simulating again 10,000 samples with an autocorrelation now of $\rho = 0.5$. The Geyer statistics for this run are

$$\begin{aligned}\widehat{V}_{\text{asympt}} &= 3.97, \\ \widehat{\tau} &= 3.03, \\ \widehat{\text{Var}}(\hat{\mu}_m) &= 0.0004, \\ \widehat{m}_{\text{eff}} &= 3294.\end{aligned}$$

4.14 Sensitivity Analysis

The output of a MCMC study consists typically of samples $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(T)}$ from an approximate posterior distribution with density $p(\theta|y)$, where θ is the vector of parameters of the Bayesian model and y is the observed data vector. Expectations of functions $h(\theta)$ with respect to $p(\theta|y)$ can be estimated using the sample average:

$$\widehat{E}(h(\theta)) = \frac{1}{T} \sum_{i=1}^T h(\theta^{(i)}). \quad (4.116)$$

As part of the statistical analysis one may wish to study the robustness of the conclusions drawn from (4.116) to either different prior specifications or perturbations of the likelihood (such as a new functional form or part of the data could be omitted). Alternatively, interest may focus on modifying samples from one distribution to form samples from another distribution from which inferences can be drawn. This section describes two computational techniques that are useful to achieve these goals.

Importance Sampling

Imagine that (4.116) is an estimator of the expectation of $h(\theta)$ with respect to a posterior density

$$p_1(\theta|y) = c_1 p_1(\theta) p(y|\theta), \quad (4.117)$$

where c_1 is the typically unknown normalising constant, $p_1(\theta)$ is the prior density assigned to θ and $p(y|\theta)$ is the likelihood. Suppose that one wishes to study the consequences of changing the prior specification, such that the new posterior becomes

$$p_2(\theta|y) = c_2 p_2(\theta) p(y|\theta). \quad (4.118)$$

Here, $p_2(\theta)$ is the density of the “new” prior distribution, and c_2 is the corresponding integration constant. Using the draws $\theta^{(i)}$ generated under the distribution

with density $p_1(\theta|y)$, inferences about $h(\theta)$ under the new posterior with density $p_2(\theta|y)$ can be obtained without having to run the MCMC procedure again, employing a computational technique known as importance sampling (Hammersley and Handscomb 1964). This is done by using $p_1(\theta|y)$ as importance sampling density. Thus, expectations under $p_2(\theta|y)$ can be obtained as follows

$$\begin{aligned}
 E_2[h(\theta)] &= \frac{\int h(\theta) \frac{p_2(\theta|y)}{p_1(\theta|y)} p_1(\theta|y) d\theta}{\int \frac{p_2(\theta|y)}{p_1(\theta|y)} p_1(\theta|y) d\theta} \\
 &= \frac{\int h(\theta) \frac{c_2 p_2(\theta) p(y|\theta)}{c_1 p_1(\theta) p(y|\theta)} p_1(\theta|y) d\theta}{\int \frac{c_2 p_2(\theta) p(y|\theta)}{c_1 p_1(\theta) p(y|\theta)} p_1(\theta|y) d\theta} \\
 &= \frac{\int h(\theta) w(\theta) p_1(\theta|y) d\theta}{\int w(\theta) p_1(\theta|y) d\theta}, \tag{4.119}
 \end{aligned}$$

where

$$w(\theta) = \frac{p_2(\theta)}{p_1(\theta)}.$$

Note that in the second line of (4.119) the ratio of constants of integration and the likelihood cancel out in the numerator and denominator. A consistent estimator of (4.119) based on (4.116) is

$$\hat{E}_2[h(\theta)] = \frac{\sum_{i=1}^T h(\theta^{(i)}) w(\theta^{(i)})}{\sum_{i=1}^T w(\theta^{(i)})}, \tag{4.120}$$

where $\theta^{(i)}$, $(i = 1, 2, \dots, T)$ are the draws from the distribution with density $p_1(\theta|y)$. The weight function w_i is equal to

$$w(\theta^{(i)}) = \frac{p_2[\theta^{(i)}]}{p_1[\theta^{(i)}]}.$$

If $w_i = 1$ for all i , $p_2(\theta|y) = p_1(\theta|y)$ and (4.119) is equal to (4.116).

The Monte Carlo standard error of (4.120) is

$$SE(\hat{E}_2[h(\theta)]) = \frac{\sqrt{\sum_{i=1}^T [(h(\theta^{(i)}) - \hat{E}_2[h(\theta)]) w(\theta^{(i)})]^2}}{\sum_{i=1}^T w(\theta^{(i)})} \tag{4.121}$$

(Geweke 1989). The presence of large relative weights leads to poor estimates.

Moments and quantiles under the new posterior distribution can be obtained along the same lines, using the draws from the original posterior distribution. For instance

$$\begin{aligned}\widehat{Var}_2[h(\theta)] &= \widehat{E}_2[h^2(\theta)] - [\widehat{E}_2[h(\theta)]]^2 \\ &= \frac{\sum_{i=1}^T h^2(\theta^{(i)}) w(\theta^{(i)})}{\sum_{i=1}^T w(\theta^{(i)})} - [\widehat{E}_2[h(\theta)]]^2\end{aligned}\quad (4.122)$$

and

$$\widehat{Pr}_2[h(\theta) < c] = \frac{\sum_{i=1}^T I[h(\theta^{(i)}) < c] w(\theta^{(i)})}{\sum_{i=1}^T w(\theta^{(i)})}, \quad (4.123)$$

where subscript 2 indicates that inferences are being drawn from the posterior distribution with density $p_2[\theta|y]$ and $\theta^{(i)}$ is the i th sample drawn from the posterior distribution with density $p_1[\theta|y]$.

Often, it can be computationally advantageous to fit a particular model elicited under a certain prior or likelihood specification. However, the analyst may have in mind an alternative model which is less tractable computationally. The approach described above provides a powerful tool for doing this in a rather straightforward manner. This is illustrated in the following example, taken from Sorensen and Gianola (2002).

Example: Inferences from Two Beta Distributions

Suppose n independent draws are made from a Bernoulli distribution with unknown probability of success θ . Let x denote the number of successes and y the number of failures. The likelihood is

$$p(x|\theta, n) \propto \theta^x (1 - \theta)^y. \quad (4.124)$$

The experimenter wishes to perform the Bayesian analysis under two different sets of prior assumptions. The first model assumes a uniform prior distribution for θ , $Un(0, 1)$:

$$p_1(\theta) = 1, \quad 0 \leq \theta \leq 1. \quad (4.125)$$

Under this prior, the posterior density is proportional to (4.124)

$$p_1(\theta|x, n) \propto \theta^x (1 - \theta)^y, \quad (4.126)$$

which is recognized as the density of a beta distributed random variable with parameters $x + 1, y + 1$, that is $Be(\theta|x + 1, y + 1)$. The second model assumes the same likelihood, but the prior distribution for θ is beta, with parameters a and b . The posterior density is now

$$p_2(\theta|x, n) \propto \theta^{a+x-1} (1 - \theta)^{b+y-1}, \quad (4.127)$$

which is the density $Be(\theta|a + x, b + y)$. In this example, the form of the posterior distribution is known under either prior, so it is straightforward to draw inferences from (4.126) or from (4.127). To illustrate, independent samples will be drawn from (4.126), and then importance sampling will be used to obtain inferences based on (4.127), using the draws from (4.126). Further, the Monte Carlo-based estimates will then be compared with exact results. The results for $\theta = 0.8$, obtained with $n = x + y = 5$ or 15, are shown in Table 4.1, for three importance sampling sample sizes. The focus of inference is on the posterior mean and variance, and on the probability that the value of θ lies between 0.75 and 0.85. In the model that provides the basis of inference, $p_2(\theta|x, n)$, the parameters of the Beta prior are $a = b = 3$. The results in the table illustrate that the estimator is consistent: as the number of samples increases from 4000 to 1 million, the estimates based on (4.120), (4.122), and (4.123) converge to the true values.

When the probability to be estimated is small, a larger number of importance samples must be drawn to achieve the same level of precision. For example, the true probability that θ lies between 0.3 and 0.4, based on $p_2(\theta|x, n)$, is 15.68×10^{-4} . Estimates obtained with sample sizes of four thousand, one hundred thousand and one million were 12.10×10^{-4} , 14.66×10^{-4} , and 15.75×10^{-4} , respectively.

Table 4.1 Comparison between exact results and estimates based on importance sampling (IS). S: number of samples in thousands; x : number of successes; y : number of failures; Probability: posterior probability that the binomial parameter takes a value between 0.75 and 0.85. From Sorensen and Gianola (2002)

S	x	y	Mean $\times 10$		Variance $\times 10^2$		Probability	
			Exact	IS	Exact	IS	Exact	IS
4	4	1	6.364	6.356	1.9284	1.9757	0.1742	0.1710
100	4	1	6.364	6.361	1.9284	1.9322	0.1742	0.1728
1000	4	1	6.364	6.365	1.9284	1.9286	0.1742	0.1743
4	12	3	7.143	7.134	0.9276	0.9515	0.3155	0.3004
100	12	3	7.143	7.141	0.9276	0.9283	0.3155	0.3124
1000	12	3	7.143	7.143	0.9276	0.9277	0.3155	0.3157

While in this example the importance sampling approach performs satisfactorily, in higher-dimensional problems the relative weights

$$\frac{w(\theta^{(i)})}{\sum_{i=1}^n w(\theta^{(i)})}$$

may be concentrated on a small number of samples. As a consequence, the Monte Carlo sampling error associated with estimates of posterior features is likely to be large. A larger effective sample size is required in order to mitigate this drawback.

Sampling Importance Resampling

Rubin (1987), Rubin (1988), Smith and Gelfand (1992), and Albert (2009) describe how another computational technique known as importance resampling (or sampling importance resampling, SIR) can be used to modify samples from one distribution to form samples from another distribution. Suppose that samples $\theta_1 = (\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(T)})$ are available from a distribution with density p_1 and there is interest in transforming θ_1 into samples from a distribution with density p_2 . This is achieved as follows:

1. compute weights $w^{(j)} = \frac{p_2(\theta^{(j)})}{p_1(\theta^{(j)})}$, $j = 1, 2, \dots, T$,
2. convert the weights to probabilities $w_j = \frac{w^{(j)}}{\sum_{j=1}^T w^{(j)}}$,
3. sample with replacement from the vector θ_1 with probabilities w_j . This results in a new vector θ_2 that is an approximate sample from the distribution with density p_2 .

This procedure, named sampling importance resampling (SIR), is a weighted bootstrap where the elements of θ_2 are sampled from the sample θ_1 with unequal probabilities.

Chapter 10 shows an application where samples from a posterior distribution $[\theta|y]$ are transformed to samples from a posterior distribution $[\theta|y_{-i}]$, where θ is the vector of parameters of the Bayesian model and data vector $y = (y_i, y_{-i})$, $i = 1, 2, \dots, N$, with y_{-i} equal to y with the i th observation excluded. Following the three steps above, using $p_2(\theta) = p(\theta|y_{-i})$, $p_1(\theta) = p(\theta|y)$, results in weights converted to probabilities of the form (see Eq. (10.26) on page 493)

$$w_{ij} = \frac{p^{-1}(y_i|\theta^{(j)})}{\sum_{j=1}^T p^{-1}(y_i|\theta^{(j)})}, \quad i = 1, 2, \dots, N; \quad j = 1, 2, \dots, T. \quad (4.128)$$

Sampling with replacement with probabilities $(w_{i1}, w_{i2}, \dots, w_{iT})$ from the vector $\theta_1 = (\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(T)})$ drawn from $[\theta|y]$, generates an approximate sample from the posterior distribution $[\theta|y_{-i}]$. Indeed, for example when θ is a scalar on

the real line,

$$\begin{aligned}
 \Pr(\theta \leq a | y_{-i}) &= \frac{\int_{-\infty}^a p(\theta | y_{-i}) d\theta}{\int_{-\infty}^{\infty} p(\theta | y_{-i}) d\theta} = \frac{\int_{-\infty}^a \frac{p(\theta | y_{-i})}{p(\theta | y)} p(\theta | y) d\theta}{\int_{-\infty}^{\infty} \frac{p(\theta | y_{-i})}{p(\theta | y)} p(\theta | y) d\theta} \\
 &= \frac{\frac{p(y)}{p(y_{-i})} \int_{-\infty}^a \frac{1}{p(y_i | \theta)} p(\theta | y) d\theta}{\frac{p(y)}{p(y_{-i})} \int_{-\infty}^{\infty} \frac{1}{p(y_i | \theta)} p(\theta | y) d\theta} \\
 &= \frac{\int_{-\infty}^a \frac{1}{p(y_i | \theta)} p(\theta | y) d\theta}{\int_{-\infty}^{\infty} \frac{1}{p(y_i | \theta)} p(\theta | y) d\theta}, \tag{4.129}
 \end{aligned}$$

so the distribution $[\theta | y_{-i}]$ is generated by appropriate weighting of the distribution $[\theta | y]$, as indicated in the bottom right hand side of (4.129). A Monte Carlo consistent estimator of (4.129) is

$$\begin{aligned}
 \hat{\Pr}(\theta \leq a | y_{-i}) &= \frac{\sum_{j=1}^T \frac{1}{p(y_i | \theta^{(j)})} I(\theta^{(j)} \leq a)}{\sum_{j=1}^T \frac{1}{p(y_i | \theta^{(j)})}} \\
 &= \sum_{j=1}^T w_{ij} I(\theta^{(j)} \leq a), \quad i = 1, 2, \dots, N, \tag{4.130}
 \end{aligned}$$

where w_{ij} is given by (4.128), $\theta^{(j)}$ is the j th draw from $[\theta | y]$ once the McMC has converged to its stationary distribution and $I(\cdot)$ is the indicator function that takes the value 1 if the argument is satisfied and 0 otherwise.

In Chap. 10 these ideas are developed to obtain Bayesian estimators of leave-one-out cross-validation running through the data only once and an example is provided. Attention is drawn to the fact that the estimates using the weighted bootstrap are sensitive to the presence of large values of the weights and some form of smoothing can produce less noisy estimates (Vehtari et al. 2017).

4.15 Appendix: A Closer Look at the McMC Machinery

The appendix provides a brief description of the Metropolis-Hastings algorithm in its standard and general form. The intention is to provide a little background for the rationale behind the algorithm. The material may seem a little daunting at first sight, particularly due to the nature of the ideas and the notation than the mathematics. In fact, the appendix uses nuts and bolts mathematics and basic theory of transformation of random variables. Much of what is written below is taken from Waagepetersen and Sorensen (2001), where more details and an example can be found. A benchmark paper for the general Metropolis-Hastings algorithm is Green

(1995); the paper is technical and its detailed understanding requires command of measure theory. An accessible overview of discrete and continuous Markov chains can be found in Chaps. 10 and 11 of Sorensen and Gianola (2002).

The Standard Metropolis-Hastings Ratio

Consider first a finite state discrete Markov chain with state space S . The Markov chain is defined by a sequence of discrete random variables $X_i, i = 1, 2, \dots$ each of which can take one of N values in the finite set $S = \{0, 1, \dots, N-1\}$. The subscript i in X_i can be interpreted as stages or time periods, and the initial stage is $i = 0$. These random variables X_i satisfy the Markov property

$$\Pr(X_i | X_{i-1}, X_{i-2}, \dots, X_0) = \Pr(X_i | X_{i-1}). \quad (4.131)$$

Stationarity

A probability density π is *stationary* or *invariant*, if $X_i \sim \pi$ implies $X_{i+1} \sim \pi$, $i \geq 0$. Notationally x and x' often will be used to represent values of the random variable X at two time periods or stages (not to be confused with the same symbol “ ρ ” used elsewhere to denote the transpose of a matrix or vector). A definition of stationarity is

$$\sum_x \pi(x) \Pr(X_{i+1} = x' | X_i = x) = \pi(x'), \quad x, x' \in S. \quad (4.132)$$

The idea behind stationarity is as follows. Consider the relationship

$$\Pr(X_{i+1} = x' | X_0 = x_0) = \sum_x \Pr(X_{i+1} = x' | X_i = x) \Pr(X_i = x | X_0 = x_0). \quad (4.133)$$

If the chain converges to a stationary distribution π that is independent of the starting point x_0 , then

$$\begin{aligned} \Pr(X_{i+1} = x' | X_0 = x_0) &= \pi(x'), \\ \Pr(X_i = x | X_0 = x_0) &= \pi(x) \end{aligned}$$

and (4.133) reduces to (4.132).

The Metropolis-Hastings (MH) algorithm is a recipe for constructing a Markov chain that has π as its *stationary or invariant distribution*. The goal is to obtain draws from π which could be a posterior distribution. Since doing this directly may be very complicated or impossible, a draw from a *proposal distribution* (denoted here as $q(\cdot|x)$ that may depend on x) is taken instead and because this is not

the same distribution as π , the drawn value is accepted in a stochastic manner by means of an *acceptance probability*. This acceptance probability is derived in a way ensuring that the sequence $X_0, X_1, \dots$, is an approximate Monte Carlo sample from the stationary distribution π . In order for this to hold, the Markov chain must be aperiodic and irreducible and hence, ergodic (an aperiodic chain does not return to the same state at regular time intervals; in an irreducible chain, every state is reachable from every other state in a finite number of transitions). An ergodic chain is one that converges to the stationary distribution, regardless of the starting value. This Markov chain can be used for Monte Carlo estimation of various expectations with respect to the stationary distribution. Before showing how the algorithm works, I define the concept of detailed balance on which the MH algorithm builds.

Detailed Balance

Let $T(x'|x) = \Pr(X_{i+1} = x' | X_i = x)$ be the conditional probability that $X_{i+1} = x'$, given that $X_i = x$. This conditional pmf is known as a *transition probability*, that has the standard property

$$\sum_{x'} T(x'|x) = 1. \quad (4.134)$$

In the case of finite state discrete Markov chains, $T(x'|x)$ is an element in an $N \times N$ stochastic matrix (transition probability matrix) and (4.134) indicates that the sum of the elements of the rows of the stochastic matrix adds to one. Then π satisfies detailed balance with respect to T , if

$$\pi(x) T(x'|x) = \pi(x') T(x|x'). \quad (4.135)$$

The left hand side is equal to the right hand side with x and x' reversed. Note that when $x = x'$, (4.135) holds trivially because

$$\pi(x') T(x'|x') = \pi(x') T(x'|x'). \quad (4.136)$$

The detailed balance equation (4.135) can also be written as

$$\Pr(X_i = x) \Pr(X_{i+1} = x' | X_i = x) = \Pr(X_i = x') \Pr(X_{i+1} = x | X_i = x')$$

or

$$\Pr(X_i = x, X_{i+1} = x') = \Pr(X_i = x', X_{i+1} = x),$$

provided $X \sim \pi$.

A Markov chain that has stationary distribution π that satisfies detailed balance with respect to T is said to be a reversible Markov chain.

An intuition for expression (4.135) is that the total probability mass in the move from x to x' is equal to that of the reverse move, from x' to x . In the left hand side, the probability mass at x is $\pi(x)$ and only a proportion $T(x'|x)$ moves to the right hand side. The total probability mass in the move from x to x' is the product of these two quantities. Likewise, the probability mass on the right hand side at $X_i = x'$ is $\pi(x')$ that may be different from $\pi(x)$. A proportion of $\pi(x')$ equal to $T(x|x')$ moves from x' to x and the total probability mass is again the product of these two quantities. The total probability mass for this pair of states (x, x') on both sides is the same. This holds for all possible pairs x and x' that belong in S .

An important consequence of imposing the strong condition of detailed balance (4.135), is that an ergodic Markov chain that is reversible has π as its stationary distribution when it converges. To show this, start with (4.135) and sum over x on both sides (if (4.135) holds, the equality still holds if we sum both sides over x):

$$\begin{aligned}\sum_x \pi(x) T(x'|x) &= \sum_x \pi(x') T(x|x') \\ &= \pi(x') \sum_x T(x|x') \\ &= \pi(x'),\end{aligned}\tag{4.137}$$

that is the definition of a stationary distribution. In the case of a Markov chain in continuous space with transition kernel p , the equivalent to (4.137) is obtained by integrating both sides with respect to x :

$$\begin{aligned}\int \pi(x) p(x'|x) dx &= \int \pi(x') p(x|x') dx \\ &= \pi(x') \int p(x|x') dx \\ &= \pi(x')\end{aligned}$$

that again is the definition of a stationary distribution. This means that if (X_i, X_{i+1}) has stationary distribution π , then the time-reversed subchain (X_{i+1}, X_i) has the same stationary distribution, whenever X_i has density π .

The acceptance probability of the MH algorithm is derived assuming that the Markov chain satisfies detailed balance. If the Markov chain is ergodic, when it converges, detailed balance guarantees that π is the stationary distribution.

The Acceptance Probability of the Metropolis-Hastings Algorithm

The acceptance probability is constructed subject to detailed balance as specified in Eq. (4.135). Consider a move from x to x' . In the MH algorithm a proposed value y for x' is drawn from the proposal distribution $q(y|x)$. If the proposed value is accepted, $x' = y$ and if its is rejected, $x' = x$, the value at the previous stage of the

chain. There are two ways in which the state in the next stage is equal to x' . One is to draw the proposal y and to accept it with probability $a(y|x)$. The other way in which the state at the next stage can take the value x' , is to reject the proposal, to set $x' = x$ but X was already equal to x' . Let

$$s(x) = \Pr(y \text{ rejected} | X_i = x) \quad (4.138)$$

Then

$$\begin{aligned} T(y|x) &= \Pr(X_{i+1} = y | X_i = x) \\ &= q(y|x) a(y|x) + s(x) I(x = y). \end{aligned}$$

The left hand side of (4.135) can now be written as

$$\pi(x) T(y|x) = \pi(x) q(y|x) a(y|x) + \pi(x) s(x) I(x = y), \quad (4.139)$$

and by symmetry the right hand side is equal to

$$\pi(y) T(x|y) = \pi(y) q(x|y) a(x|y) + \pi(y) s(y) I(y = x). \quad (4.140)$$

The second terms in the right hand side of (4.139) and (4.140) are equal, both in the case when $x \neq y$ in which case they are zero because the indicator function is zero, or trivially when $x = y$. Therefore detailed balance is satisfied if

$$\pi(x) q(y|x) a(y|x) = \pi(y) q(x|y) a(x|y). \quad (4.141)$$

Then,

$$\frac{a(y|x)}{a(x|y)} = \frac{\pi(y) q(x|y)}{\pi(x) q(y|x)}. \quad (4.142)$$

Relationship (4.142) is satisfied if $a(y|x) = \pi(y) q(x|y) / b(y|x)$, for some $b(y|x) \geq \pi(y) q(x|y)$, to ensure that the acceptance probability $a(y|x) \leq 1$ (a similar argument holds for $a(x|y)$ due to symmetry). A valid $b(y|x)$ is

$$b(y|x) = \max(\pi(y) q(x|y), \pi(x) q(y|x)) \quad (4.143)$$

but other choices are possible and these have an impact on the properties of the Markov chain. The subject is rather technical and is discussed by Hastings (1970) and Peskun (1973), where it is shown that (4.143) leads to a Markov chain with the largest possible acceptance probabilities resulting in minimum asymptotic

variances of moment estimates. This choice translates into the most commonly cited expression for the Metropolis-Hastings acceptance probability, given by

$$a(y|x) = \min \left(1, \frac{\pi(y) q(x|y)}{\pi(x) q(y|x)} \right), \quad (4.144)$$

that gives rise to the following algorithm:

- Initialise setting $X = x$, so that $\pi(x) > 0$
- Choose the proposal distribution q
Given current state $X_i = x$, go through the loop
- Draw y from the proposal $q(\cdot|x)$
- Draw u from $Un(0, 1)$
- If $u < a(y|x)$, accept y and set $X_{i+1} = y = x'$, otherwise $X_{i+1} = x$

The original idea of using Markov chain simulation of probability distributions is often attributed to Metropolis et al. (1953).

The General Metropolis-Hastings Ratio

In the standard Metropolis-Hastings algorithm, the next state x' is obtained by drawing the candidate y from q . A more general mechanism to generate the move to x' , is to construct a proposal y' by applying a deterministic mapping to x and to a random component u , that has density $q_U(u|x)$, which may depend on x (Green 1995). The proposal y' is

$$y' = g_1(x, u).$$

If the proposal is accepted, $x' = y'$. To ensure dimension matching in the move from (x, u) to x' , if x is of size n_x and u of size n_u , it may be necessary to include a random variable u' of size $n_{u'}$ with density $q_{U'}(u'|x')$ so that

$$n_x + n_u = n_{x'} + n_{u'}. \quad (4.145)$$

Then the vectors of Markov chain states and proposal random variables (x, u) and (x', u') are of equal dimension and the densities $\pi(x) q_U(u|x)$ in the move from (x, u) to (x', u') , and $\pi(x') q_{U'}(u'|x')$ in the move from (x', u') to (x, u) are joint densities on spaces of equal dimension. The mapping is then

$$(x', u') = g(x, u) = (g_1(x, u), g_2(x, u)) \quad (4.146)$$

and the move in the opposite direction is

$$(x, u) = g^{-1}(x', u') = \left(g_1^{-1}(x', u'), g_2^{-1}(x', u') \right). \quad (4.147)$$

A necessary condition for the existence of the one-to-one mapping is that (4.145) is satisfied. When moves occur within models, as discussed in this appendix, the dimension matching condition (4.145) is automatically fulfilled.

An **important detail** that becomes obvious in the derivation (more below), is that there is a constraint in the form of the deterministic function. The constraint is that

$$g = g^{-1}. \quad (4.148)$$

Therefore, we need

$$\begin{aligned} (x', u') &= g(x, u) = (g_1(x, u), g_2(x, u)), \\ (x, u) &= g^{-1}(x', u') = g(x', u') = (g_1(x', u'), g_2(x', u')). \end{aligned} \quad (4.149)$$

The general Metropolis-Hastings acceptance ratio takes the form

$$a(x'|x) = \min \left\{ 1, \frac{\pi(x') q_{U'}(u'|x')}{\pi(x) q_U(u|x)} |J| \right\} \quad (4.150)$$

where $|J| = \left| \det \frac{\partial g(x, u)}{\partial (x, u)} \right|$ is the absolute value of the Jacobian of the transformation g . A detailed derivation is given below.

Note

This appendix is restricted to moves within a given model. The general case of jumps across models involving model comparison and simulation from posterior distributions on spaces of varying dimensions is a little more demanding technically, especially requiring more complex notation. The reader is referred to Waagepetersen and Sorensen (2001) and to Sorensen and Gianola (2002), pp. 498–500 and 517–532; the benchmark paper is Green (1995).

Stationarity

The Markov chain in continuous space is defined in terms of the distribution for the initial state X_0 and the transition kernel $P(\cdot, \cdot)$ which specifies the conditional distribution of X_{t+1} given the previous state X_t . If the current state is $X_t = x$, then the probability that X_{t+1} is in a subset $B \subseteq \mathbb{R}^d$ is given by

$$P(x, B) = \Pr(X_{t+1} \in B | X_t = x). \quad (4.151)$$

Assume that π is a complex target distribution for a stochastic vector Z of dimension $d \geq 1$. Since expectations with respect to π cannot be evaluated analytically or by using techniques for numerical integration, a Markov chain X_i , $i = 1, 2, \dots$, is constructed whose stationary distribution is π . If the chain is ergodic then it can be used for Monte Carlo estimation of expectations $E(h(Z))$ for any function h , with respect to the invariant density π . That is

$$E(h(Z)) = \int h(z) \pi(z) dz \approx \frac{1}{n} \sum_{i=1}^n h(X_i) \quad (4.152)$$

as n tends to infinity. Thus $E(h(Z))$ can be approximated by the sample average for large n , the length of the chain. The autocorrelation among the draws of the chain implies that the size of the Markov chain must be larger than when the draws are independent, in order to achieve a given level of accuracy.

For a continuous space Markov chain, the definition of stationarity is

$$\int P(x, B) \pi(x) dx = \int_B \pi(x) dx = \Pr(X \in B) \quad (4.153)$$

The distribution π is invariant (or stationary) for the Markov chain, if the transition kernel $P(\cdot, \cdot)$ of the Markov chain preserves π , so that $X_t \sim \pi$ implies $X_{t+1} \sim \pi$. In order to verify that π is the invariant density using (4.153) is an infeasible task, since this involves integration with respect to π . The difficulty of doing this was the reason for using MCMC in the first place. However, choosing a kernel that imposes the stronger condition of reversibility with respect to π is sufficient to guarantee that π is the invariant density of the ergodic Markov chain.

Reversibility

For a continuous state space Markov chain, the reversibility condition

$$P_{t,t+1}(X_t \in A, X_{t+1} \in B) = P_{t,t+1}(X_t \in B, X_{t+1} \in A) \quad (4.154)$$

requires that the equilibrium probability that the state of the chain belongs to A and moves to B must be the same with A and B reversed, for subsets $A, B \subseteq \mathbb{R}^d$. In other words, expression (4.154) states that the joint probability that $X_t \in A$ and $X_{t+1} \in B$ (left hand side) is the same as the joint probability that $X_t \in B$ and $X_{t+1} \in A$ (right hand side). The left hand side will be referred to as the move from x to x' and the left hand side as the opposite move.

The reversibility condition (4.154) can also be written in terms of the transition kernel

$$\int_A P(x, B) \pi(x) dx = \int_B P(x, A) \pi(x) dx. \quad (4.155)$$

Reversibility (4.155) implies (4.153) by taking $A = \mathbb{R}^d$. Then $P(x, A) = 1$ and (4.155) reduces to

$$\int P(x, B) \pi(x) dx = \int_B \pi(x) dx$$

which is equal to (4.153). Therefore, an ergodic Markov chain that satisfies (4.155) has stationary distribution π .

The Acceptance Probability for a General Metropolis-Hastings Algorithm

The acceptance probability for the general MH algorithm is derived ensuring that detailed balance (or reversibility) as defined in Eq. (4.154) is satisfied. **In a general setting**, instead of generating the proposal directly from $q(\cdot|x)$ as is practised in the standard Metropolis-Hastings algorithm, the proposal y' can be defined in terms of a stochastic component $u \sim q(u|x)$ and a deterministic mapping g . If the proposal is accepted, $x' = y'$. As explained in connection with (4.146) and (4.147), in the move from x to x' the mapping is

$$(x', u') = g(x, u) = (g_1(x, u), g_2(x, u)) \quad (4.156)$$

and in the reverse move

$$(x, u) = g^{-1}(x', u') = (g_1(x', u'), g_2(x', u')). \quad (4.157)$$

The transition kernel in the move from $X_t \in A$ to $X_{t+1} \in B$ (move from x to x'), $\Pr(X_{t+1} \in B | X_t = x)$, is now constructed in three steps. The Metropolis-Hastings protocol requires first drawing u from $q(\cdot|x)$, secondly, constructing the proposal $y' = g_1(x, u)$ and thirdly accepting it with probability $a(g_1(x, u) | x)$. Define

$$S(x) = \Pr(y' \text{ is rejected} | X_t = x).$$

Then

$$\begin{aligned} \Pr(X_{t+1} \in B | X_t = x) &= \int I(g_1(x, u) \in B) q(u|x) a(g_1(x, u) | x) du \\ &\quad + S(x) I(x \in B), \end{aligned} \quad (4.158)$$

where the second term in the right hand side specifies the probability of rejecting y' but the current state x already belongs to the subset B . The left hand side of (4.154) takes the form

$$\begin{aligned} & \int_A \Pr(X_{t+1} \in B | X_t = x) \pi(x) dx \\ &= \int \int I(x \in A, g_1(x, u) \in B) \pi(x) q(u|x) a(g_1(x, u) | x) dx du \\ &+ \int S(x) I(x \in A \cap B) \pi(x) dx. \end{aligned} \quad (4.159)$$

(The second term in the right hand side is equal to $\int_A S(x) I(x \in B) \pi(x) dx$).

A transition from $X_t \in B$ to $X_{t+1} \in A$ (move from x' to x) is accomplished by first drawing u' from $q(\cdot|x')$, secondly, constructing the proposal $y = g_1(x', u')$ and thirdly accepting it with probability $a(g_1(x', u') | x')$. The right hand side of (4.154) takes the form

$$\begin{aligned} & \int \int I(x' \in B, g_1(x', u') \in A) \pi(x') q(u'|x') a(g_1(x', u') | x') dx' du' \\ &+ \int S(x') I(x' \in A \cap B) \pi(x') dx'. \end{aligned} \quad (4.160)$$

The second terms in (4.159) and (4.160) are equal (x and x' are dummy variables of integration) and therefore a sufficient condition for (4.154) to hold is

$$\begin{aligned} & \int \int I(x \in A, g_1(x, u) \in B) \pi(x) q(u|x) a(g_1(x, u) | x) dx du \\ &= \int \int I(x' \in B, g_1(x', u') \in A) \pi(x') q(u'|x') a(g_1(x', u') | x') dx' du'. \end{aligned} \quad (4.161)$$

The final step is to find a way to equalise the indicator functions of both sides of equation (4.161). This is achieved performing the change of variables defined by the mappings (4.156) and (4.157) and making use of

$$g^{-1}(x', u') = g(x', u'). \quad (4.162)$$

Substituting in the argument of the indicator function of the right hand side of (4.161), both indicator functions are equalised. It is at this point of the derivation that the constraint (4.148) or (4.162) becomes relevant (see also (4.149)). Using this

transformation and setting $dx' du' = |J| dx du$, the right hand side of (4.161) takes the form

$$\begin{aligned} & \iint I(g_1(x, u) \in B, x \in A) \pi(g_1(x, u)) q(g_2(x, u) | g_1(x, u)) \\ & \times a(x | g_1(x, u)) |J| dx du, \end{aligned} \quad (4.163)$$

where

$$J = \det \begin{bmatrix} \frac{\partial g_1(x, u)}{\partial x} & \frac{\partial g_2(x, u)}{\partial x} \\ \frac{\partial g_1(x, u)}{\partial u} & \frac{\partial g_2(x, u)}{\partial u} \end{bmatrix}.$$

Examination of the left hand side of (4.161) and of (4.163) shows that the reversibility condition (4.154) is satisfied if

$$\pi(x) q(u|x) a(g_1(x, u) | x) = \pi(g_1(x, u)) q(g_2(x, u) | g_1(x, u)) a(x | g_1(x, u)) |J|.$$

The same argument leading to (4.144) indicates that a valid choice for $a(x' | x)$ is

$$a(x' | x) = \min \left[1, \frac{\pi(x') q(u' | x')}{\pi(x) q(u | x)} |J| \right]. \quad (4.164)$$

Note

Using the framework of the general setting, in the move from x to x' consider drawing the proposal y' directly from $q(y' | x)$. If y' is accepted, $x' = y'$. In the opposite move from x' to x , y is drawn from $q(y | x')$, and if y is accepted, $x = y$. The mapping in this strategy, with abuse of notation is

$$\begin{aligned} x &= g_1(y) = y, \\ x' &= g_2(y') = y', \end{aligned}$$

where g_1 and g_2 are the identity mapping. The left hand side of (4.154) takes the form (move from x to x')

$$\begin{aligned} & \int_A \Pr(X_{t+1} \in B | X_t = x) \pi(x) dx \\ &= \int \int I(x \in A, y' \in B) \pi(x) q(y' | x) a(y' | x) dx dy' \\ &+ \int S(x) I(x \in A \cap B) \pi(x) dx. \end{aligned} \quad (4.165)$$

The right hand side of (4.154) (move from x' to x) is given by

$$\begin{aligned} & \int \int I(x' \in B, y \in A) \pi(x') q(y|x') a(y|x') dx' dy \\ & + \int S(x') I(x' \in A \cap B) \pi(x') dx'. \end{aligned} \quad (4.166)$$

The second terms in (4.165) and (4.166) are equal and therefore a sufficient condition for (4.154) to hold is

$$\begin{aligned} & \int \int I(x \in A, y' \in B) \pi(x) q(y'|x) a(y'|x) dx dy' \\ & = \int \int I(x' \in B, y \in A) \pi(x') q(y|x') a(y|x') dx' dy. \end{aligned} \quad (4.167)$$

Performing the change of variables $x = y$ and $x' = y'$ and since the Jacobian of the transformation is equal to 1, the right hand side of (4.167) is

$$\int \int I(y' \in B, x \in A) \pi(y') q(x|y') a(x|y') dy' dx.$$

The arguments of the indicator functions are equalised and therefore the reversibility condition (4.154) is satisfied if

$$\pi(x) q(y'|x) a(y'|x) = \pi(y') q(x|y') a(x|y').$$

The same rationale leading to (4.144) indicates that a valid choice for $a(y'|x)$ is

$$a(y'|x) = \min \left(1, \frac{\pi(y') q(x|y')}{\pi(x) q(y'|x)} \right), \quad (4.168)$$

the same acceptance probability (4.144) derived for the standard MH algorithm (here the notation y' is used for the proposal in the move from x to x' , rather than y used in the derivation of (4.144)).

A Toy Example

The model is $[y|\mu, \lambda] \sim N(\mu, \lambda)$. Assume μ is known and a Metropolis-Hastings algorithm is constructed to update the variance λ .

Strategy 1

This is accomplished generating $u \sim Un(a, b)$ and letting $\lambda' = \lambda u$. The current state of the Markov chain is $(z, u) = (\lambda, u)$ and the move is to

$$\begin{aligned} (z', u') &= g(\lambda, u) \\ &= ((\lambda u), 1/u) \end{aligned}$$

where $u = 1/u'$. The inverse function that makes the move in the opposite direction possible is

$$\begin{aligned} (z, u) &= g^{-1}(z', u') \\ &= g(\lambda', u') \\ &= ((\lambda' u'), 1/u') \end{aligned}$$

where $u' = 1/u$ is also generated from $Un(a, b)$. The Jacobian of the transformation $g(\lambda, u) = (\lambda u, 1/u)$ is

$$\begin{aligned} J &= \left| \det \left[\frac{\partial g(\lambda, u)}{\partial (\lambda, u)} \right] \right| \\ &= \left| \det \begin{bmatrix} u & \lambda \\ 0 & -u^{-2} \end{bmatrix} \right| = u^{-1}. \end{aligned}$$

Since u and u' are drawn from $Un(a, b)$, $1/(b - a)$ cancels in the ratio $q_{U'}(u'|z')/q_U(u|z)$ in (4.150) and the Metropolis-Hastings acceptance probability is

$$\min \left\{ 1, \frac{p(\mu, \lambda'|y)}{p(\mu, \lambda|y)} u^{-1} \right\}, \quad u \in (a, b). \quad (4.169)$$

Note

The above strategy satisfies the constraint (4.148) which in the example takes the form

$$\begin{aligned} (z', u') &= \left((zu), \frac{1}{u} \right) \\ (z, u) &= \left((z'u'), \frac{1}{u'} \right) \end{aligned}$$

with $u = 1/u'$. This is so because (recall, $g_1(a, b) = a \times b$, is a function that multiplies its arguments)

$$\begin{aligned} z' &= g_1(z, u) \\ &= g_1(g_1(z', u'), u) \\ &= g_1(z', u') \times u \\ &= (z' \times u') \times u \end{aligned}$$

and therefore $u = 1/u'$.

Strategy 2

An alternative way of arriving at (4.169) is to use as the Metropolis-Hastings ratio

$$\frac{p(\mu, \lambda u | y) p(\lambda | \lambda')}{p(\mu, \lambda | y) p(\lambda' | \lambda)}. \quad (4.170)$$

This is the standard form of the Metropolis-Hastings ratio, using the proposal for the parameter rather than the auxiliary variables (u, u') . Then with $\lambda' = u\lambda$,

$$p(\lambda' | \lambda) = q_U(u) \left| \frac{du}{d\lambda'} \right| = \frac{1}{b-a} \frac{1}{\lambda}, \quad \lambda' \in (\lambda a, \lambda b).$$

By symmetry,

$$p(\lambda | \lambda') = \frac{1}{b-a} \frac{1}{\lambda'}, \quad \lambda \in (\lambda' a, \lambda' b).$$

The Metropolis-Hastings ratio is now

$$\begin{aligned} \frac{p(\mu, \lambda u | y) p(\lambda | \lambda')}{p(\mu, \lambda | y) p(\lambda' | \lambda)} &= \frac{p(\mu, \lambda u | y) \lambda}{p(\mu, \lambda | y) \lambda'} \\ &= \frac{p(\mu, \lambda u | y) 1}{p(\mu, \lambda | y) u}, \quad u \in (a, b). \end{aligned} \quad (4.171)$$

and the resulting acceptance probability is

$$\min \left\{ 1, \frac{p(\mu, \lambda' | y) 1}{p(\mu, \lambda | y) u} \right\}, \quad u \in (a, b). \quad (4.172)$$

Strategy 3

The third strategy consists of updating the variance using a random walk proposal density on the logvariance. That is

$$\ln \lambda' \sim N(\ln \lambda, k),$$

a normal distribution with mean equal to the natural logarithm of the previous realisation of λ and variance given by k , a user-tuned parameter.

As a reminder, if X has density $p(x) = N(m, k)$ and $Y = f(X) = \exp(X)$, such that the inverse function f^{-1} exists and results in $X = f^{-1}(Y) = \ln Y$, then the Jacobian of the transformation from X to Y is $1/y$ and $p(y) = p(f^{-1}(y)) \frac{1}{y}$. In this particular case, in the move from λ to λ' we have $X = \ln \lambda'$; $Y = \exp(\ln \lambda') = \lambda'$; $f^{-1}(Y) = \ln \lambda'$. Therefore, if

$$q(\ln \lambda' | \ln \lambda, k) = N(\ln \lambda, k),$$

then

$$\begin{aligned} q(\lambda' | \ln \lambda, k) &= q(\ln \lambda' | \ln \lambda, k) \frac{1}{\lambda'} \\ &= N(\ln \lambda, k) \frac{1}{\lambda'}, \end{aligned} \quad (4.173)$$

which is the density of the lognormal distribution with parameters $(\ln \lambda, k)$. In these expressions, the variance of the normal distribution k is a user-tuned parameter. Then (4.173) is the proposal density evaluated at λ' and by symmetry,

$$q(\lambda | \ln \lambda', k) = q(\ln \lambda | \ln \lambda', k) \frac{1}{\lambda}.$$

Since $q(\ln \lambda' | \ln \lambda, k) = q(\ln \lambda | \ln \lambda', k)$ the Metropolis-Hastings ratio is

$$\frac{p(\mu, \lambda' | y)}{p(\mu, \lambda | y)} \frac{q(\lambda | \ln \lambda, k)}{q(\lambda' | \ln \lambda, k)} = \frac{p(\mu, \lambda' | y)}{p(\mu, \lambda | y)} \frac{\lambda'}{\lambda},$$

different from (4.171).

Chapter 5

McMC in Practice



This chapter illustrates applications of McMC in a Bayesian context. The treatment is mostly schematic; the objective is to present the mechanics of McMC in different modelling scenarios. Many of the examples discussed in connection with implementation of maximum likelihood (using Newton-Raphson and EM), are revisited from a Bayesian McMC perspective. These include the analysis of ABO blood group data, binary regression, the genomic model, the two-component mixture model and the Bayesian analysis of truncated data. Further examples are discussed in the second part of the book on Prediction and in the Exercise sections, including their solutions, at the end of the book.

5.1 Example: Estimation of Gene Frequencies from ABO Blood Group Phenotypes

The ABO blood group data discussed in Chap. 3 is reproduced in Table 5.1. The problem is to infer p_A , p_B and p_0 , the frequency of the three alleles A , B and 0 , respectively, subject to $p_A + p_B + p_0 = 1$. Three alleles give rise to six genotypes but only four phenotypic classes are observed. The observed data are $n = (n_A, n_{AB}, n_B, n_0)$ and the multinomial likelihood is proportional to the pmf $f(n|p_A, p_B, p_0)$,

$$\begin{aligned} L(p_A, p_B|n) &\propto f(n|p_A, p_B, p_0) \\ &= \left(p_A^2 + 2p_A p_B\right)^{n_A} (2p_A p_B)^{n_{AB}} \left(p_B^2 + 2p_B p_0\right)^{n_B} \left(p_0^2\right)^{n_0}. \end{aligned} \quad (5.1)$$

A Dirichlet prior will be chosen as distribution for the gene frequencies p_A , p_B , p_0 . The Dirichlet is a multivariate generalisation of the univariate Beta distribution and

Table 5.1 Frequency of genotypes and phenotypes of ABO blood group data

Genotype	Phenotype	Observed counts	Frequency
AA	A	n_A	p_A^2
AO	A		$2p_A p_O$
AB	AB	n_{AB}	$2p_A p_B$
BB	B	n_B	p_B^2
BO	B		$2p_B p_O$
OO	O	n_O	p_O^2

is symbolised $Di(\alpha)$ where α is a vector of positive real numbers. The pdf is

$$f(x_1, \dots, x_k | \alpha_1, \dots, \alpha_k) = \frac{1}{B(\alpha)} \prod_{i=1}^k x_i^{\alpha_i - 1},$$

$$x_1, \dots, x_{k-1} > 0,$$

$$x_1 + \dots + x_{k-1} < 1,$$

$$x_k = 1 - x_1 - \dots - x_{k-1}$$

and the normalising constant

$$B(\alpha) = \frac{\prod_{i=1}^k \Gamma(\alpha_i)}{\Gamma\left(\sum_{i=1}^k \alpha_i\right)}$$

is the multinomial beta function. A common special case is the symmetric Dirichlet distribution where all the elements of vector α have the same value. The pdf simplifies to

$$f(x_1, \dots, x_k | \alpha) = \frac{\Gamma(\alpha k)}{\Gamma(\alpha)^k} \prod_{i=1}^k x_i^{\alpha - 1}. \quad (5.2)$$

When $\alpha = 1$ this becomes a uniform distribution in $k - 1$ dimensions. In this example, the Dirichlet prior is

$$f(p_A, p_B, p_O) \propto (p_A)^{\alpha_A - 1} (p_B)^{\alpha_B - 1} (p_O)^{\alpha_O - 1}. \quad (5.3)$$

The likelihood (5.1) and the Dirichlet prior give rise to the posterior density

$$f(p_A, p_B, p_O | n) \propto \left(p_A^2 + 2p_A p_B\right)^{n_A} (2p_A p_B)^{n_{AB}} \left(p_B^2 + 2p_B p_O\right)^{n_B} \left(p_O^2\right)^{n_O} \\ (p_A)^{\alpha_A - 1} (p_B)^{\alpha_B - 1} (p_O)^{\alpha_O - 1}. \quad (5.4)$$

The fully conditional posterior distributions do not have a closed form. An McMC approach requires a Metropolis-Hastings algorithm or using *data augmentation* that

may lead to standard conditional posterior distributions. In the present example data augmentation facilitates a computationally simple Gibbs sampling implementation and I explore this alternative. First, data augmentation is described in general terms.

Data Augmentation

Imagine that there is interest in obtaining the posterior distribution of a parameter θ . Due to analytical intractability one chooses to approximate $p(\theta|y)$ using MCMC. Often, the fully conditional posterior distribution $p(\theta|y)$ does not have a standard form and the MCMC algorithm can be difficult to implement. The idea of data augmentation is to augment with the so-called latent data or missing data φ , in order to exploit the simplicity of the resulting conditional posterior distribution $p(\theta|\varphi, y)$. This is in the same spirit as in the EM algorithm: by increasing the dimensionality of the problem, possibly at the expense of extra computing time, although this is not always the case (Swendsen and Wang 1987), the problem is simplified algorithmically. The focus of inference is

$$p(\theta|y) = \int p(\theta|\varphi, y)p(\varphi|y)d\varphi,$$

and this marginalisation is carried out via MCMC. A key paper is Tanner and Wong (1987).

The mechanics works as follows. Start with an initial legal value of $\varphi^{[0]}$. Given $\varphi^{[t]}$ at iteration t ,

1. Draw $\theta^{[t+1]}$ from $[\theta|\varphi^{[t]}, y]$
2. Draw $\varphi^{[t+1]}$ from $[\varphi|\theta^{[t+1]}, y]$

At convergence the iterative procedure results in draws from the posterior distribution $[\theta, \varphi|y]$ and the margins correspond to draws from $[\theta|y]$ and $[\varphi|y]$.

To apply data augmentation to the ABO blood group data one can define $n_A = n_{AA} + n_{A0}$ and $n_B = n_{BB} + n_{B0}$. Then the *missing data* is

$$n_m = (n_{A0}, n_{AA}, n_{BB}, n_{B0}).$$

If the missing data were observed,

$$\begin{aligned} & f(n, n_m | p_A, p_B, p_0) \\ &= \left(p_A^2\right)^{n_{AA}} (2p_A p_0)^{n_{A0}} (2p_A p_B)^{n_{AB}} \left(p_B^2\right)^{n_{BB}} (2p_B p_0)^{n_{B0}} \left(p_0^2\right)^{n_0}. \end{aligned}$$

This is the pmf of the standard multinomial distribution. The augmented posterior can be written

$$\begin{aligned}
 f(n_m, p_A, p_B, p_0|n) &\propto f(n_m, p_A, p_B, p_0) f(n|n_m, p_A, p_B, p_0) \\
 &= f(n, n_m, p_A, p_B, p_0) \\
 &= f(n, n_m|p_A, p_B, p_0) f(p_A, p_B, p_0) \\
 &= \left(p_A^2\right)^{n_{AA}} (2p_A p_0)^{n_{A0}} (2p_A p_B)^{n_{AB}} \left(p_B^2\right)^{n_{BB}} \\
 &\quad (2p_B p_0)^{n_{B0}} \left(p_0^2\right)^{n_0} (p_A)^{\alpha_A-1} (p_B)^{\alpha_B-1} (p_0)^{\alpha_0-1}.
 \end{aligned} \tag{5.5}$$

This is proportional to

$$\begin{aligned}
 f(n_m, p_A, p_B, p_0|n) &\propto (p_A)^{2n_{AA}} (p_A)^{n_{A0}} (p_A)^{n_{AB}} (p_A)^{\alpha_A-1} \\
 &\quad (p_B)^{2n_{BB}} (p_B)^{n_{B0}} (p_B)^{n_{AB}} (p_B)^{\alpha_B-1} \\
 &\quad (p_0)^{2n_0} (p_0)^{n_{B0}} (p_0)^{n_{A0}} (p_0)^{\alpha_0-1} \\
 &= (p_A)^{2n_{AA}+n_{A0}+n_{AB}+\alpha_A-1} (p_B)^{2n_{BB}+n_{B0}+n_{AB}+\alpha_B-1} \\
 &\quad (p_0)^{2n_0+n_{A0}+n_{B0}+\alpha_0-1}.
 \end{aligned} \tag{5.6}$$

The Gibbs sampling algorithm requires extracting the fully conditional posterior distributions

$$f(p_A, p_B, p_0|n, n_m), \tag{5.7a}$$

$$f(n_{AA}|p_A, p_B, p_0, n_A, n), \tag{5.7b}$$

$$f(n_{BB}|p_A, p_B, p_0, n_B, n) \tag{5.7c}$$

from (5.5) or (5.6). The derivation of (5.7a) is straightforward: direct inspection of (5.6) indicates that the required fully conditional is

$$[p_A, p_B, p_0|n, n_m] \sim Di(a, b, c), \tag{5.8}$$

a Dirichlet distribution with parameters

$$a = 2n_{AA} + n_{A0} + n_{AB} + \alpha_A,$$

$$b = 2n_{BB} + n_{B0} + n_{AB} + \alpha_B,$$

$$c = 2n_0 + n_{A0} + n_{B0} + \alpha_0.$$

To derive (5.7b) first replace in (5.5) $n_{A0} = n_A - n_{AA}$. Extracting terms in n_{AA} only yields

$$f(n_{AA}|p_A, p_B, p_0, n_A, n) \propto (p_A^2)^{n_{AA}} (2p_A p_0)^{n_A - n_{AA}},$$

which is the kernel of the binomial distribution

$$[n_{AA}|p_A, p_B, p_0, n_A, n] \sim Bi\left(\frac{p_A^2}{p_A^2 + 2p_A p_0}, n_A\right). \quad (5.9)$$

Similarly, after replacing $n_{B0} = n_B - n_{BB}$,

$$f(n_{BB}|p_A, p_B, p_0, n_B, n) \propto (p_B^2)^{n_{BB}} (2p_B p_0)^{n_B - n_{BB}}$$

which is the kernel of the binomial distribution

$$[n_{BB}|p_A, p_B, p_0, n_B, n] \sim Bi\left(\frac{p_B^2}{p_B^2 + 2p_B p_0}, n_B\right). \quad (5.10)$$

The Gibbs Sampling Implementation

The data in Weir (1996) used in Chap. 3 is $n_A = 725$, $n_{AB} = 72$, $n_B = 258$, $n_0 = 1073$. The R-code below implements the Bayesian model with a Gibbs sampler, assuming $\alpha_A = \alpha_B = \alpha_0 = \alpha = 2$, start values $p_A = 0.33$, $p_B = 0.33$.

```
# CODE0501
# ABO BLOOD GROUP DATA - GIBBS SAMPLING
rm(list=ls()) # Clear the workspace
set.seed(1237)
#install.packages("MCMCpack", .libPaths()[1])
# to access function rdirichlet
library(MCMCpack)
#CHOOSE LENGTH OF GIBBS CHAIN rep
rep<-3000
result<-matrix(data=NA,nrow=rep,ncol=4)
# INITIALISE PARAMETERS
p_A<-0.33
p_B<-0.33
p_0<-1-p_A-p_B
alfa<-2
# DATA
n_A<-725
n_AB<-72
n_B<-258
n_0<-1073
```

```

#START WITH THE GIBBS LOOP
for (i in 1:rep)
{
  # SAMPLE n_AA AND n_BB
  n_AA<-rbinom(1,n_A,p_A^2/(p_A^2+2*p_A*p_0))
  n_BB<-rbinom(1,n_B,p_B^2/(p_B^2+2*p_B*p_0))
  n_A0<-n_A-n_AA
  n_B0<-n_B-n_BB
  # SAMPLE p_A,p_B,p_0
  a<-2*n_AA+n_A0+n_AB+alfa
  b<-2*n_BB+n_B0+n_AB+alfa
  c<-2*n_0+n_A0+n_B0+alfa
  draws<- rdirichlet(1,c(a,b,c))
  p_A<-draws[1,1]
  p_B<-draws[1,2]
  p_0<-draws[1,3]
  result[i, ]<-c(i,p_A,p_B,p_0)
}
# END OF GIBBS LOOP
meanp_A<-mean(result[,2])
meanp_A

```

```
## [1] 0.2092435
```

```

varp_A<-var(result[,2])
cip_A<- quantile(result[,2],c(0.025,0.975))
cip_A

```

```
##          2.5%          97.5%
## 0.1967966 0.2220510
```

```

meanp_B<-mean(result[,3])
meanp_B

```

```
## [1] 0.0811496
```

```

varp_B<-var(result[,3])
cip_B<- quantile(result[,3],c(0.025,0.975))
cip_B

```

```
##          2.5%          97.5%
## 0.07306027 0.08997164
```

```
covp_Ap_B<-cov(result[,2],result[,3])
```

These results are in good agreement with those from previous analyses.

5.2 Example: A Regression Model for Binary Data

The binary regression discussed in Chap. 3 is here parametrised as

$$\Pr(y_i = 1|x_i'\beta) = \Pr(u_i < 0|x_i, \beta),$$

$$\Pr(y_i = 0|x_i'\beta) = \Pr(u_i > 0|x_i, \beta),$$

where u_i is the unobserved liability. The logistic loglikelihood is

$$l(\beta|y, x) = \sum_{i=1}^N \{(1 - y_i) x_i' \beta - \ln[1 + \exp(x_i' \beta)]\}. \quad (5.11)$$

At the level of the liability u the linear model for the i th datum is

$$u_i = x_i' \beta + e_i, \quad i = 1, 2, \dots, N, \quad (5.12)$$

where x_i' is the i th row of the known non-stochastic $N \times p$ matrix of explanatory variables (covariates), β is an unknown parameter vector (of order $p \times 1$) and the residuals e_i are *iid* variables drawn from a standard logistic distribution with mean 0 and variance $\pi^2/3$.

A Bayesian implementation requires assigning a prior specification to β . One possibility is to assume a uniform prior. In this case the posterior $p(\beta|y, x)$ is proportional to the likelihood and therefore the logposterior takes the same form as the loglikelihood (5.11).

Note the notation for the posterior density $p(\beta|y, x)$ indicates that conditioning is on the observations (1's and 0's) and the covariates $(x_1, \dots, x_N)$ where N is the number of individuals. Hyperparameters are not included and often covariates are also omitted from the notation. When this is the case the posterior density is written as $p(\beta|y)$.

Metropolis-Hastings Algorithm

An implementation with the Metropolis-Hastings algorithm requires a proposal density for β . A possibility is a normal distribution centred at the current value $\beta^{[t]}$ and covariance matrix λC . Let the proposed value be $Y = Y_\beta$ and let the current state of the chain be $X^{[t]} = x = \beta^{[t]}$. Then the algorithm is as follows:

1. Set $t = 0$ and choose a starting value for β equal to $\beta^{[0]}$, choose C and the tuning parameter λ .
2. Let $t = t + 1$

3. Draw $Y_\beta \sim N(\beta^{[t]}, \lambda C)$. The density of this normal distribution evaluated at the proposed value Y_β is $q(Y_\beta | \beta^{[t]}, \lambda C)$
4. Draw $u \sim Un(0, 1)$
5. If $u < \alpha$, $\beta^{[t+1]} = Y_\beta$. Otherwise $\beta^{[t+1]} = \beta^{[t]}$
6. Goto 2

This is repeated until a chain of sufficient length is obtained. The decision is based on computation of effective chain length and Monte Carlo standard errors of estimates of features from the posterior distribution of β . The tuning parameter λ is used to obtain acceptance ratios in the range (0.25; 0.50).

The acceptance probability in step 5 is

$$\alpha = \frac{p(Y_\beta | y, x)}{p(\beta^{[t]} | y, x)} \frac{q(\beta^{[t]} | Y_\beta, \lambda C)}{q(Y_\beta | \beta^{[t]}, \lambda C)}. \quad (5.13)$$

In contrast with the general notation used in (4.91), here a notation specific to the model is used. Comparing with (4.91):

$$\begin{aligned} \pi(Y) &= p(Y_\beta | y, x), \\ \pi(x) &= p(\beta^{[t]} | y, x), \\ q(x|Y) &= q(\beta^{[t]} | Y_\beta, \lambda C), \\ q(Y|x) &= q(Y_\beta | \beta^{[t]}, \lambda C). \end{aligned}$$

The notation $q(\beta^{[t]} | Y_\beta, \lambda C)$ agrees with that used for $q(\cdot | Y)$ in (4.91). The proposal density $q(\cdot | Y)$ is evaluated at $\beta^{[t]}$ and $(Y_\beta, \lambda C)$ are respectively, the mean and variance of the distribution. Due to the symmetry properties of the normal distribution, the second ratio in the right hand side of (5.13) is equal to 1 and the acceptance probability reduces to the ratio of the posterior distributions $p(\cdot | y, x)$ evaluated at $\beta = Y_\beta$ and at $\beta = \beta^{[t]}$, respectively. The acceptance probability is

$$\alpha = \frac{p(Y_\beta | y, x)}{p(\beta^{[t]} | y, x)}. \quad (5.14)$$

An alternative approach is to use a *probit model*. The probit likelihood was given in (3.75):

$$L(\beta | x, y) \propto \prod_{i=1}^N \left[(1 - \Phi(x'_i \beta))^{y_i} (\Phi(x'_i \beta))^{1-y_i} \right]. \quad (5.15)$$

Assuming a uniform prior for β results in a posterior distribution proportional to the likelihood. The logposterior is obtained by taking logarithms of (5.15). This gives, up to an additive constant

$$\ln p(\beta|x, y) = \sum_{i=1}^N [y_i \ln(1 - \Phi(x_i' \beta)) + (1 - y_i) \ln(\Phi(x_i' \beta))]. \quad (5.16)$$

The Metropolis-Hastings algorithm based on the probit model follows along the same lines as with the logistic model. In the problems section Bayes Exercises II, you are asked to fit a logistic and a probit Bayesian model to binary data.

Gibbs Sampling Algorithm

The Gibbs sampling algorithm requires knowledge of the fully conditional posterior distributions (fcpd). The logposteriors (5.11) and (5.16) do not lead to fcpd that have a standard form. An alternative strategy is to augment the posterior distribution with the unobserved liability u . This is illustrated using the probit model. At the level of the liability the linear model for the i th record is

$$u_i = x_i' \beta + e_i, e_i \stackrel{iid}{\sim} N(0, 1), i = 1, \dots, N. \quad (5.17)$$

As before assume

$$\Pr(y_i = 1|\beta, x_i) = \Pr(u_i < 0|\beta, x_i) \quad (5.18)$$

and

$$\Pr(y_i = 0|\beta, x_i) = \Pr(u_i > 0|\beta, x_i). \quad (5.19)$$

The density of the augmented posterior is

$$\begin{aligned} p(u, \beta|y) &\propto p(u, \beta) p(y|u, \beta) \quad \text{Bayes theorem} \\ &= p(u|\beta) p(\beta) p(y|u) \quad \text{given } u, y \text{ is independent of } \beta \\ &\propto p(u|\beta) p(y|u) \quad \text{assuming a uniform prior for } \beta. \end{aligned} \quad (5.20)$$

In this expression $p(y|u)$ is a degenerate density: if u is observed, y is not stochastic because its value is known unambiguously as indicated in (5.18) and (5.19). That is

$$\begin{aligned} \Pr(y_i = 1|u_i > 0) &= \Pr(y_i = 0|u_i < 0) = 0, \\ \Pr(y_i = 1|u_i < 0) &= \Pr(y_i = 0|u_i > 0) = 1. \end{aligned}$$

Therefore, the joint density of vector y , given u , is

$$p(y|u) = \prod_{i=1}^N \left[I(u_i < 0)^{y_i} + I(u_i > 0)^{1-y_i} \right]. \quad (5.21)$$

It is important to realise that the focus of inference here is

$$p(\beta|y) = \int p(u, \beta|y) du$$

and not necessarily the augmented posterior $[u, \beta|y]$. When the system converges to the stationary distribution, the Gibbs sampler and in general, MCMC, generates Monte Carlo draws from the joint posterior $[u, \beta|y]$; the margins are draws from $[\beta|y]$ and from $[u|y]$. In this sense, MCMC is an automatic algorithm that performs the desired marginalisations. The draws from $[u, \beta|y]$ are obtained by sampling repeatedly from $[\beta|u, y]$ and $[u|\beta, y]$.

The fully conditional posterior distributions $[\beta|u, y]$ and $[u|\beta, y]$ are derived from (5.20) as follows.

- The fcpd $[\beta|u, y]$ is obtained by extracting from (5.20) the terms containing β . This yields

$$\begin{aligned} p(\beta|u, y) &\propto p(u|\beta) \\ &\propto \exp\left(-\frac{1}{2}(u - X\beta)'(u - X\beta)\right). \end{aligned} \quad (5.22)$$

As a function of β this density does not have a recognisable form. A little more work is needed. The quadratic form can be written as

$$(u - X\beta)'(u - X\beta) = u'u - 2u'X\beta + \beta'X'X\beta.$$

Define

$$X'X\hat{\beta} = X'u. \quad (5.23)$$

Then,

$$\begin{aligned} u'X &= \hat{\beta}'X'X, \\ u'X\beta &= \hat{\beta}'X'X\beta. \end{aligned}$$

Now replace $-2u'X\beta$ by $-2\hat{\beta}'X'X\beta$ in the quadratic form

$$(u - X\beta)'(u - X\beta) = u'u - 2\hat{\beta}'X'X\beta + \beta'X'X\beta.$$

Adding and subtracting $\widehat{\beta}' X' X \widehat{\beta}$ and keeping terms containing β yields

$$(u - X\beta)' (u - X\beta) = (\beta - \widehat{\beta})' X' X (\beta - \widehat{\beta}) + k,$$

where k is a constant that does not depend on β . Substituting in (5.22) gives

$$p(\beta|u, y) \propto \exp\left(-\frac{1}{2} (\beta - \widehat{\beta})' X' X (\beta - \widehat{\beta})\right)$$

which is the kernel of a normal distribution with mean $\widehat{\beta}$ and variance $(X'X)^{-1}$. Therefore

$$[\beta|u, y] \sim N\left(\widehat{\beta}, (X'X)^{-1}\right). \quad (5.24)$$

- The fcpd $[u|\beta, y]$ is obtained by extracting from (5.20) those terms that include u . This results in

$$p(u|\beta, y) \propto p(u|\beta) p(y|u).$$

The first term, seen as a function of u is obtained directly from (5.17). This gives

$$[u|\beta] \sim N(X\beta, I). \quad (5.25)$$

The term $p(y|u)$ is given in (5.21). Therefore the density of the fcpd $[u_i|\beta, y]$ can be written

$$p(u_i|\beta, y) \propto N(u_i|x'_i\beta, 1) \left[I(u_i < 0)^{y_i} + I(u_i > 0)^{1-y_i} \right]. \quad (5.26)$$

This means that if $y_i = 1$, $u_i < 0$ and the fully conditional posterior distribution of u_i is a truncated normal, with mean $x'_i\beta$, variance 1 and support $(-\infty, 0)$. If $y_i = 0$, then $u_i > 0$ and the fully conditional posterior distribution of u_i is a truncated normal, with mean $x'_i\beta$, variance 1 and support $(0, \infty)$.

In Bayes Exercises II, problem 3, you are asked to analyse binary data with a Bayesian probit model implemented with a Gibbs sampler using the data augmentation algorithm.

Drawing Samples from Truncated Distributions

An efficient algorithm to sample from univariate truncated distributions is as follows. Let Y be a random variable from a normal distribution truncated between

a (lower bound) and b (upper bound). To obtain a draw from the truncated normal $TN_{(a,b)}(\mu, \sigma^2)$, where μ and σ^2 are the mean and variance before truncation:

- Simulate U from a uniform distribution $Un(p_1, p_2)$, where $p_1 = \Phi[(a - \mu)/\sigma]$ and $p_2 = \Phi[(b - \mu)/\sigma]$.
- The truncated normal is

$$y = \mu + \sigma \Phi^{-1}(U),$$

where $\Phi(\cdot)$ is the cdf and $\Phi^{-1}(\cdot)$ is the inverse cdf of the standard normal distribution.

A more general method to sample from any univariate distribution truncated in the interval $[a, b]$ is as follows. If the cdf of the untruncated variate is F , then a draw from the truncated distribution is (Devroye 1986, p. 38, Example 10)

$$y = F^{-1}\{F(a) + U[F(b) - F(a)]\}, \quad (5.27)$$

where U is a draw from a uniform distribution in the interval $[0, 1]$. If Φ is the distribution function of the standard normal, then to draw from $TN_{(a,b)}(\mu, \sigma^2)$, application of (5.27) yields

$$y = \mu + \left\{ \sigma \Phi^{-1}[\Phi(\ell) + U(\Phi(u) - \Phi(\ell))] \right\} \quad (5.28)$$

where the lower bound is $\ell = (a - \mu)/\sigma$, the upper bound $u = (b - \mu)/\sigma$ and U is a draw from a uniform distribution in the interval $[0, 1]$. With binary data and $u \sim N(\mu, 1)$, if $y = 1$, $a = -\infty$, $b = 0$ and

$$\left. \begin{array}{l} \Phi(\ell) = 0, \\ \Phi(u) = \Phi(-\mu) \end{array} \right\} \text{ if } y = 1.$$

If $y = 0$, $a = 0$, $b = \infty$ and

$$\left. \begin{array}{l} \Phi(\ell) = \Phi(-\mu), \\ \Phi(u) = 1 \end{array} \right\} \text{ if } y = 0.$$

An R-code below generates N samples from (5.26) using algorithm (5.28).

```
# CODE0502
rm(list=ls()) # CLEAR WORKSPACE
##### DRAWING ALL U's IN ONE GO
### REQUIRES TO GENERATE BINARY RECORDS y
## USING PARAMETERISATION A OR B BELOW
# A
##### y = 0 -> TN(mean,1)(0,Infinity)
##### y = 1 -> TN(mean,1)(-Infinity,0)
##### OR #####
```

```

# B
##### y = 1 -> TN(mean,1)(0,Infinity)
##### y = 0 -> TN(mean,1)(-Infinity,0)
#####
set.seed(237777)
nrow <- 10
ncol <- 5
mu <- 0
# GENERATE X MATRIX
X<-matrix(nrow= nrow,ncol= ncol,rbinom(nrow*ncol,size=2,p=.5))
# GENERATE VECTOR b
b <- rnorm(ncol,0.5,1)
xb<-X%*%b
# LOGIT MODEL
#p1<-exp(mu+xb)/(1+exp(mu+xb)) # IF B
#p1 <- 1 - exp(mu+xb)/(1+exp(mu+xb)) # IF A
# PROBIT MODEL- GENERATE VECTOR OF DATA y
#p1 <- pnorm(mu+xb) # IF B
p1 <- 1 - pnorm(mu+xb) # IF A
y <- rbinom(nrow,1,p1)
mean <- mu+xb
sd <- 1

interm<-(1-y)*pnorm(0,mean=mean,sd=sd)+runif(length(y))*
  (pnorm(0,mean=mean,sd=sd)*(y)+
  (1-pnorm(0,mean=mean,sd=sd))*(1-y))

u <- qnorm(interm,mean=mean,sd=sd)
p1[1:5]

## [1] 0.70255823 0.82055114 0.05872622 0.52202695 0.37241559

p1[6:nrow]

## [1] 0.04796642 0.56227502 0.37241559 0.05222701 0.89662931

y[1:5]

## [1] 1 1 0 0 0

y[6:nrow]

## [1] 0 1 1 0 1

u[1:5]

## [1] -0.4976322 -0.6461563 3.8849389 0.4510179 1.1993790

```

```
u[6:nrow]
```

```
## [1] 1.962061 -1.652975 -1.563791 1.138155 -2.003726
```

As a more transparent case consider drawing a sample y of size 1 from a normal distribution, with mean 10 and variance 4, that is truncated in the interval $[11.0, 11.5]$. Then application of (5.28) leads to the following algorithm.

```
rm(list=ls()) # CLEAR WORKSPACE
set.seed(1237)
l <- (11-10)/2
u <- (11.5-10)/2
y <- 10 + (2*qnorm(pnorm(1) + runif(1)*(pnorm(u) - pnorm(1))))
y
```

```
## [1] 11.15587
```

5.3 Example: A Regression Model for Correlated Binary Data

Very often data cluster in groups where members of the same group are correlated and groups can be uncorrelated. An example consists of independent full-sib families. In this setup the model must be extended to account for the correlated structure of the observations (see Likelihood exercises II, problem 3 iii). This section illustrates a probit Bayesian model that can handle this situation.

The first step is to include a term in (5.17) that accounts for the correlated structure. The standard approach is to use a random variable f that represents a family effect:

$$\begin{aligned}
 u_{ij} &= x'_{ij}\beta + f_i + e_{ij}, \\
 f_i | \sigma_f^2 &\stackrel{iid}{\sim} N(0, \sigma_f^2), \quad \sigma_f^2 \in [0, 1], \\
 e_{ij} &\stackrel{iid}{\sim} N(0, 1), \\
 i &= 1, \dots, n_f, \quad j = 1, \dots, n.
 \end{aligned} \tag{5.29}$$

Assume that

$$\begin{aligned}
 \Pr(y_i = 1 | \beta, x_i, f_i) &= \Pr(u_i < 0 | \beta, x_i, f_i) \\
 &= \Pr(x'_{ij}\beta + f_i + e_{ij} < 0 | \beta, x_i, f_i) \\
 &= \Pr(e_{ij} < -x'_{ij}\beta - f_i | \beta, x_i, f_i) \\
 &= 1 - \Phi(x'_{ij}\beta + f_i)
 \end{aligned}$$

and

$$\begin{aligned}\Pr(y_i = 0 | \beta, x_i, f_i) &= \Pr(u_i > 0 | \beta, x_i) \\ &= \Phi(x'_{ij}\beta + f_i).\end{aligned}$$

In this specification, x'_{ij} is a row vector with p columns (number of elements in β) corresponding to individual j of family i , f_i is the effect of the i th full-sib family, σ_f^2 is the variance between families (or covariance between full-sibs), n_f is the number of full-sib families, n is the number of full-sibs per family (here assumed to be the same for all families) and the total number of records is $N = n_f \times n$. Since according to quantitative genetic theory, in the absence of common environmental effects and assuming an additive genetic model, the heritability at the level of the liability is equal to

$$h^2 = \frac{2\sigma_f^2}{\sigma_f^2 + 1}, \quad 0 < h^2 < 1,$$

then it follows that $\sigma_f^2 \in [0, 1]$. The heritability on the liability scale does not depend on the prevalence of the disease in the population.

Gibbs Sampling Implementation

Using data augmentation the parameters of the model are $[u, \beta, f, \sigma_f^2]$. The density of the prior distribution admits the following factorisation

$$\begin{aligned}p(u, \beta, f, \sigma_f^2) &\propto p(u | \beta, f) p(f | \sigma_f^2) p(\sigma_f^2) \\ &\propto p(u | \beta, f) p(f | \sigma_f^2).\end{aligned}$$

It is assumed that $p(\beta)$ and $p(\sigma_f^2)$ are proportional to constants. The augmented posterior is

$$\begin{aligned}p(u, \beta, f, \sigma_f^2 | y) &\propto p(u, \beta, f, \sigma_f^2) p(y | u, \beta, f, \sigma_f^2) \\ &\propto p(f | \sigma_f^2) \prod_{ij} p(u_{ij} | \beta, f_i) p(y_{ij} | u_{ij}), \quad \sigma_f^2 \in [0, 1].\end{aligned}\tag{5.30}$$

The Bayesian model can be implemented with a variety of MCMC algorithms. One possibility that is explored here is to use data augmentation and update the

parameters one at a time. The required fcpd are

$$[\beta|D]; [u|D]; [f|D]; [\sigma_f^2|D],$$

where D is a vector containing the observations y and all the parameters of the model except the one to be updated.

Deriving $[\beta|D]$

Extracting from (5.30) the terms that depend on β yields

$$\begin{aligned} p(\beta|D) &\propto \prod_{ij} p(u_{ij}|\beta, f_i) \\ &\propto \exp\left(-\frac{1}{2}(u - X\beta - Zf)'(u - X\beta - Zf)\right) \\ &= \exp\left(-\frac{1}{2}(\tilde{u} - X\beta)'(\tilde{u} - X\beta)\right), \end{aligned} \quad (5.31)$$

where $\tilde{u} = u - Zf$. This has the same form as (5.22). Then it is easy to see that

$$[\beta|D] \sim N\left(\hat{\beta}, (X'X)^{-1}\right), \quad (5.32)$$

where here,

$$\hat{\beta} = (X'X)^{-1} X'(u - Zf).$$

Deriving $[u|D]$

Inspection of (5.30) and (5.20) reveals that $[u|D] = [u|y]$. Therefore, since the u_{ij} are conditionally independent, given f_i ,

$$[u_{ij}|\beta, f, y] \sim N\left(x'_{ij}\beta + f_i, 1\right) \left[I(u_{ij} < 0)^{y_{ij}} + I(u_{ij} > 0)^{1-y_{ij}}\right], \quad (5.33)$$

as in the previous example.

Deriving $[f|D]$

The fcpd $[f|D]$ is obtained by extracting from (5.30) those terms that depend on f . This gives

$$\begin{aligned}
 p(f|D) &\propto p(f|\sigma_f^2) \prod_{ij} p(u_{ij}|\beta, f_i) \\
 &\propto \exp\left(-\frac{1}{2\sigma_f^2} f' f\right) \exp\left(-\frac{1}{2} (u - X\beta - Zf)' (u - X\beta - Zf)\right) \\
 &= \exp\left(-\frac{1}{2} [(\tilde{u} - Zf)' (\tilde{u} - Zf) + kf' f]\right) \tag{5.34}
 \end{aligned}$$

where $\tilde{u} = u - X\beta$ and $k = \sigma_f^{-2}$. The quadratic form can be written as

$$\begin{aligned}
 (\tilde{u} - Zf)' (\tilde{u} - Zf) + kf' f &= \tilde{u}' \tilde{u} - 2f' Z' \tilde{u} + f' Z' Z f + kf' f \\
 &= \text{constant} - 2f' Z' \tilde{u} + f' (Z' Z + Ik) f.
 \end{aligned}$$

Define

$$(Z' Z + Ik) \hat{f} = Z' \tilde{u} \tag{5.35}$$

and substitute $Z' \tilde{u}$ by $(Z' Z + Ik) \hat{f}$. Then

$$(\tilde{u} - Zf)' (\tilde{u} - Zf) + kf' f = \text{constant} - 2f' (Z' Z + Ik) \hat{f} + f' (Z' Z + Ik) f.$$

Adding and subtracting $\hat{f}' (Z' Z + Ik) \hat{f}$ and keeping terms in f only yields

$$(\tilde{u} - Zf)' (\tilde{u} - Zf) + kf' f = \text{constant} + (f - \hat{f})' (Z' Z + Ik) (f - \hat{f}).$$

Substituting the above in the exponential of (5.34), reveals that the fully conditional is

$$[f|\sigma_f^2, D] \sim N(\hat{f}, (Z' Z + Ik)^{-1}) \tag{5.36}$$

where $\hat{f}$ is given in (5.35). In this simple model with unrelated families, the term $Z' Z$ is diagonal and computations are straightforward.

Deriving $[\sigma_f^2|D]$

Finally, the term $[\sigma_f^2|D]$ is obtained from

$$\begin{aligned} p(\sigma_f^2|D) &\propto p(f|\sigma_f^2) \\ &\propto (\sigma_f^2)^{-\frac{n_f}{2}} \exp\left(-\frac{1}{2\sigma_f^2} f'f\right) \\ &= (\sigma_f^2)^{-\left(\frac{v_f}{2}+1\right)} \exp\left(-\frac{v_f S_f}{2\sigma_f^2}\right), \quad \sigma_f^2 \in [0, 1], \end{aligned} \quad (5.37)$$

which is the kernel of the density of a truncated scaled inverted chi-square distribution with scale $S_f = f'f/v_f$ and $v_f = n_f - 2$ degrees of freedom. A draw σ_f^{2*} from this distribution is

$$\sigma_f^{2*} \sim \frac{f'f}{\chi^2(v_f)}, \quad \sigma_f^{2*} \in [0, 1].$$

where $\chi^2(v_f)$ is a draw from a chi-square distribution with v_f degrees of freedom.

One must confirm that the draw is within the constraint $\sigma_f^{2*} \in [0, 1]$.

The code CODE1312 on page 709 implements the Gibbs sampling algorithm. The code can be downloaded from <https://github.com/SorensenD/SLGDS> in the folder Codes.

A Metropolis Within Gibbs Implementation

In the Gibbs sampling implementation the bound on h^2 imposes the constraint $\sigma_f^2 \in [0, 1]$ but this was not formally incorporated in the prior distribution of σ_f^2 . An unbounded uniform prior for σ_f^2 was chosen instead, even though the posterior is defined within the constraint $\sigma_f^2 \in [0, 1]$. The code of the MCMC implementation must account for this constraint. In certain cases this may require the need to draw from truncated distributions.

One may wish to incorporate the constraint on σ_f^2 assuming a Beta prior. However, depending on how the algorithm is tailored, there may still be a need to confirm that the MCMC samples are within the bounds imposed by the model.

A detail to observe is that the beta density does not assign probability mass to $\sigma_f^2 = 0$. This implies that a value of exactly 0 is excluded in the posterior distribution

of σ_f^2 . A mixture prior with a component that has a probability mass of 1 for $\sigma_f^2 = 0$ would remedy this shortcoming.

With this reservation in mind, one may explore the use of a beta prior distribution that may lead to more stable inferences when the likelihood is not very informative. Assuming a Beta prior for σ_f^2 causes the density of the posterior distribution to change from (5.30) to

$$\begin{aligned} p(u, \beta, f, \sigma_f^2 | y) &\propto p(u, \beta, f, \sigma_f^2) p(y | u, \beta, f, \sigma_f^2) \\ &\propto p(f | \sigma_f^2) p(\sigma_f^2) \prod_{ij} p(u_{ij} | \beta, f_i) p(y_{ij} | u_{ij}), \end{aligned} \quad (5.38)$$

$$\sigma_f^2 \in [0, 1]$$

where

$$p(\sigma_f^2) \propto (\sigma_f^2)^{a-1} (1 - \sigma_f^2)^{b-1}, \quad \sigma_f^2 \in [0, 1], a > 0, b > 0. \quad (5.39)$$

The choice of hyperparameters a and b can be governed by the problem at hand. For example, if the heritability on the underlying scale is likely to lie between 13% and 54%, setting $a = 5$ and $b = 20$ would result in a modal value of σ_f^2 equal to 0.17 and a 95% confidence interval of approximately (0.07; 0.37).

As in the Gibbs sampling implementation the fcpd are

$$[\beta | D]; [u | D]; [f | D]; [\sigma_f^2 | D]$$

which, with the exception of $[\sigma_f^2 | D]$ are unchanged. The fcpd of σ_f^2 is obtained by extracting from (5.38) the terms that are a function of σ_f^2 . This gives

$$\begin{aligned} p(\sigma_f^2 | D) &\propto p(f | \sigma_f^2) p(\sigma_f^2) \\ &\propto (\sigma_f^2)^{-\frac{n_f}{2}} \exp\left(-\frac{1}{2\sigma_f^2} f' f\right) (\sigma_f^2)^{a-1} (1 - \sigma_f^2)^{b-1} \\ &= (\sigma_f^2)^{-\left(\frac{v_f}{2} + 1\right)} \exp\left(-\frac{v_f S_f}{2\sigma_f^2}\right) (\sigma_f^2)^{a-1} (1 - \sigma_f^2)^{b-1}. \end{aligned} \quad (5.40)$$

Above, $v_f = n_f - 2$ and $S_f = f' f / v_f$ where f is the vector with the sampled values of family effects. This density does not have a distinguishable closed form. Therefore a Metropolis-Hastings step can be used to update σ_f^2 .

Let $Y_{\sigma_f^2}$ denote the proposed value of σ_f^2 drawn from the proposal distribution with density $q(\cdot | \sigma_f^2)$ that may depend on σ_f^2 (it does not have to). Then the Metropolis-Hastings step at iteration t is as follows:

1. At iterate $t = 0$ choose a start value for σ_f^2 ; label it $\sigma_f^{2[0]}$
2. Set $t = t + 1$
3. Draw $Y_{\sigma_f^2}$ from the proposal distribution. If $Y_{\sigma_f^2} > 0.999$ then set $Y_{\sigma_f^2} = \sigma_f^{2[t]}$ and go to 7
4. Draw $u \sim Un(0, 1)$
5. Compute the acceptance probability α
6. If $u < \alpha$, $\sigma_f^{2[t+1]} = Y_{\sigma_f^2}$. Otherwise $\sigma_f^{2[t+1]} = \sigma_f^{2[t]}$
7. Continue with the Gibbs sampling step and when completed go to 2.

The acceptance ratio is

$$\begin{aligned} \alpha &= \frac{p(Y_{\sigma_f^2}|y, x) q(\sigma_f^{2[t]}|Y_{\sigma_f^2})}{p(\sigma_f^{2[t]}|y, x) q(Y_{\sigma_f^2}|\sigma_f^{2[t]})} \\ &= \frac{(Y_{\sigma_f^2})^{-\left(\frac{v_f}{2}+1\right)} \exp\left(-\frac{v_f S_f}{2Y_{\sigma_f^2}}\right) (Y_{\sigma_f^2})^{a-1} (1 - Y_{\sigma_f^2})^{b-1} q(\sigma_f^{2[t]}|Y_{\sigma_f^2})}{(\sigma_f^{2[t]})^{-\left(\frac{v_f}{2}+1\right)} \exp\left(-\frac{v_f S_f}{2\sigma_f^{2[t]}}\right) (\sigma_f^{2[t]})^{a-1} (1 - \sigma_f^{2[t]})^{b-1} q(Y_{\sigma_f^2}|\sigma_f^{2[t]})} \end{aligned} \quad (5.41)$$

The behaviour of the algorithm is influenced by the choice of the proposal distribution q . One possibility is to use a beta distribution that guarantees that the proposed value is within the permissible range. The first hyperparameter can be set equal to $a = 5$ as in the prior distribution, but at each round choose the other hyperparameter b^* so that the modal value of the proposal distribution is equal to the current realisation $\sigma_f^{2[t]}$. Since the mode of the beta distribution is given by

$$\text{mode} = \sigma_f^{2[t]} = \frac{a - 1}{a + b^* - 2},$$

choose

$$b^* = \frac{(a - 1) - (a - 2)\sigma_f^{2[t]}}{\sigma_f^{2[t]}}.$$

The ratio of proposal terms is

$$\frac{q(\sigma_f^{2[t]}|Y_{\sigma_f^2})}{q(Y_{\sigma_f^2}|\sigma_f^{2[t]})} = \frac{(\sigma_f^{2[t]})^{a-1} (1 - \sigma_f^{2[t]})^{b^*-1}}{(Y_{\sigma_f^2})^{a-1} (1 - Y_{\sigma_f^2})^{b^*-1}}$$

and the Metropolis-Hastings acceptance ratio (5.41) after a little simplification reduces to

$$\begin{aligned} \alpha &= \frac{p(Y_{\sigma_f^2}|y, x) q(\sigma_f^{2[l]})}{p(\sigma_f^{2[l]}|y, x) q(Y_{\sigma_f^2})} \\ &= \frac{(Y_{\sigma_f^2})^{-\left(\frac{v_f}{2}+1\right)} \exp\left(-\frac{v_f S_f}{2Y_{\sigma_f^2}}\right) (1 - Y_{\sigma_f^2})^{b-1} (1 - \sigma_f^{2[l]})^{b^*-1}}{(\sigma_f^{2[l]})^{-\left(\frac{v_f}{2}+1\right)} \exp\left(-\frac{v_f S_f}{2\sigma_f^{2[l]}}\right) (1 - \sigma_f^{2[l]})^{b-1} (1 - Y_{\sigma_f^2})^{b^*-1}}. \end{aligned} \quad (5.42)$$

The R-code CODE1312MH executes the algorithm with the Metropolis-Hastings step to update σ_f^2 . The code can be downloaded from <https://github.com/SorensenD/SLGDS> in the folder Codes.

5.4 Example: A Genomic Model

The genomic model was implemented in a likelihood setting with Newton-Raphson and with the EM algorithm. Here the model is fitted using a Bayesian MCMC approach.

To recapitulate, in its original parametrisation, the initial hierarchy of the genomic model is defined by the expressions

$$\begin{aligned} y|\mu, b, \sigma_e^2 &\sim N(1\mu + Wb, I\sigma_e^2), \\ b|\sigma_b^2 &\sim N(0, I\sigma_b^2), \end{aligned}$$

where the matrix of marker genotype codes W of dimension $n \times m$, $m > n$, is centred and scaled and the vector of marker effects b has m elements. The genomic values are defined as $g = Wb$.

The reparametrised model uses the decomposition

$$\begin{aligned} G &= \frac{1}{m} W W' \\ &= \frac{1}{m} U \Delta U' \\ &= U \Lambda U', \quad \Lambda = \frac{1}{m} \Delta. \end{aligned}$$

The diagonal matrix Λ contains the (scaled) eigenvalues λ_i of U , $i = 1, \dots, n$. The last eigenvalue is $\lambda_n = 0$ when the rank of W is equal to $(n - 1)$.

On defining $\alpha \sim N(0, \Lambda\sigma_g^2)$, where $\sigma_g^2 = m\sigma_b^2$, the alternative parametrisation as defined on page 99 is

$$\begin{aligned} y|\mu, \alpha, \sigma_e^2 &\sim N(1\mu + U\alpha, I\sigma_e^2), \\ \alpha|U, \sigma_g^2 &\sim SN(0, \Lambda\sigma_g^2), \\ G &= U\Lambda U', \\ y|\mu, \sigma_g^2, \sigma_e^2 &\sim N(1\mu, U\Lambda U'\sigma_g^2 + I\sigma_e^2). \end{aligned}$$

In contrast with (3.38), here a scalar mean μ is included. In the reparametrised model the column vector of genomic values is $g = U\alpha$, where α is the regression of genomic values on eigenvectors.

The vector

$$\begin{aligned} \alpha' &= (\alpha_1, \dots, \alpha_{n-1}, \alpha_n)' \\ &= (\alpha'_{-n} \alpha_n) \end{aligned}$$

of dimension n has its first $n-1$ elements different from zero (the vector α_{-n} above) and the last element $\alpha_n = 0$. As mentioned before, the marginal density of α_n is a point mass at zero, because $\lambda_n = 0$ and therefore $[\alpha|U, \sigma_g^2]$ has the form of a singular normal distribution.

The parameters of the Bayesian model are $(\mu, \alpha, \sigma_g^2, \sigma_e^2)$ and the density of the posterior distribution is

$$\begin{aligned} p(\mu, \alpha, \sigma_g^2, \sigma_e^2|y) &\propto p(\mu, \alpha, \sigma_g^2, \sigma_e^2) p(y|\mu, \alpha, \sigma_g^2, \sigma_e^2) \\ &\propto p(\alpha|\sigma_g^2) p(y|\mu, \alpha, \sigma_e^2) \end{aligned} \quad (5.43)$$

where it is assumed that, a priori

$$p(\mu, \sigma_g^2, \sigma_e^2) \propto \text{constant}.$$

A Gibbs sampling algorithm requires to update the parameters from the fcpd's

$$[\mu|D]; [\alpha|D]; [\sigma_g^2|D]; [\sigma_e^2|D].$$

Deriving $[\mu|D]$

Extracting from (5.43) those terms that include μ gives

$$\begin{aligned} p(\mu|D) &\propto p(y|\mu, \alpha, \sigma_e^2) \\ &\propto \exp \left[-\frac{1}{2\sigma_e^2} (y - 1\mu - U\alpha)' (y - 1\mu - U\alpha) \right] \\ &= \exp \left[-\frac{1}{2\sigma_e^2} (y^* - 1\mu)' (y^* - 1\mu) \right], \end{aligned}$$

where $y^* = y - U\alpha$. This has the same form as (5.31) and therefore

$$[\mu|D] \sim N \left(\hat{\mu}, (1'1)^{-1} \sigma_e^2 \right), \quad (5.44a)$$

where

$$\hat{\mu} = (1'1)^{-1} 1' (y - U\alpha).$$

Deriving $[\alpha|D]$

Extracting from (5.43) those terms that include α gives

$$\begin{aligned} p(\alpha|D) &\propto p(\alpha|\sigma_g^2) p(y|\mu, \alpha, \sigma_e^2) \\ &\propto \exp \left[-\frac{1}{2\sigma_g^2} \alpha' \Lambda^- \alpha \right] \exp \left[-\frac{1}{2\sigma_e^2} (y - 1\mu - U\alpha)' (y - 1\mu - U\alpha) \right]. \end{aligned}$$

In this expression

$$\Lambda^- = \begin{bmatrix} (\Lambda_{-n})_{(n-1) \times (n-1)}^{-1} & 0_{(n-1) \times 1} \\ 0_{1 \times (n-1)} & 0_{1 \times 1} \end{bmatrix}_{n \times n}$$

and $(\Lambda_{-n})^{-1}$ is the $(n-1) \times (n-1)$ diagonal matrix whose diagonal elements are the inverse of the non-zero eigenvalues $\lambda_1, \dots, \lambda_{n-1}$. The two terms in the exponentials can be brought together to obtain

$$p(\alpha|D) \propto \exp \left[-\frac{1}{2\sigma_e^2} \{ (y - 1\mu - U\alpha)' (y - 1\mu - U\alpha) + k\alpha' \Lambda^- \alpha \} \right], \quad (5.45)$$

where $k = \sigma_e^2 / \sigma_g^2$. The term in curly brackets that only contains terms in α is

$$\begin{aligned} & \{(y - 1\mu - U\alpha)'(y - 1\mu - U\alpha) + k\alpha' \Lambda^- \alpha\} = \\ & = -2\alpha' U' y + 2\alpha' U' 1\mu + \alpha' U' U \alpha + k\alpha' \Lambda^- \alpha \\ & = -2\alpha' U' (y - 1\mu) + \alpha' \alpha + k\alpha' \Lambda^- \alpha \\ & = -2\alpha' U' (y - 1\mu) + \alpha' (I + \Lambda^- k) \alpha, \end{aligned}$$

excluding an additive constant. Defining

$$(I + \Lambda^- k) \hat{\alpha} = U' (y - 1\mu) \quad (5.46)$$

and replacing $-2U' (y - 1\mu)$ by the left hand side of this expression gives

$$\begin{aligned} & \{(y - 1\mu - U\alpha)'(y - 1\mu - U\alpha) + k\alpha' \Lambda^- \alpha\} \propto \\ & \propto -2\alpha' (I + \Lambda^- k) \hat{\alpha} + \alpha' (I + \Lambda^- k) \alpha. \end{aligned}$$

excluding an additive constant. Adding and subtracting $\hat{\alpha}' (I + \Lambda^- k) \hat{\alpha}$ and keeping only terms that contain α leads to (excluding an additive constant)

$$\begin{aligned} & \{(y - 1\mu - U\alpha)'(y - 1\mu - U\alpha) + k\alpha' \Lambda^- \alpha\} = \\ & = (\alpha - \hat{\alpha})' (I + \Lambda^- k) (\alpha - \hat{\alpha}). \end{aligned}$$

Substituting in (5.45),

$$p(\alpha|D) \propto \exp \left[-\frac{1}{2\sigma_e^2} (\alpha - \hat{\alpha})' (I + \Lambda^- k) (\alpha - \hat{\alpha}) \right]$$

which is the kernel of the normal distribution

$$[\alpha|D] \sim N \left(\hat{\alpha}, (I + \Lambda^- k)^{-1} \sigma_e^2 \right). \quad (5.47)$$

In practice one implements the Gibbs sampler updating the non-zero $n - 1$ elements α_{-n} from the $(n - 1)$ dimensional normal distribution

$$\alpha_{-n}|D \sim N \left(\hat{\alpha}_{-n}, \left(I + \Lambda_{-n}^{-1} k \right)^{-1} \sigma_e^2 \right) \quad (5.48)$$

where $\hat{\alpha}_{-n}$ contains the first $(n - 1)$ non-zero elements of $\hat{\alpha}$ defined in (5.46). The n th element of $\hat{\alpha}$ is equal to zero.

The update is computationally straightforward since the α 's are conditionally independent. A little manipulation with (5.47) or (5.48) shows that

$$E(\alpha_i|D) = \hat{\alpha}_i = \frac{\lambda_i}{\lambda_i + k} U'_i(y - 1\mu),$$

$$Var(\alpha_i|D) = \frac{\lambda_i \sigma_e^2}{\lambda_i + k}, \quad i = 1, \dots, n-1$$

and therefore each α_i is updated from $N(E(\alpha_i|D), Var(\alpha_i|D))$. These expressions make it clear that when $\lambda_i = 0$, the mean and variance are zero and the density collapses to a point mass at zero.

Deriving $[\sigma_g^2|D]$

The fcpd of the genomic variance component defined at the level of this operational model is obtained by extracting from (5.43) those terms that include σ_g^2 . This results in

$$p(\sigma_g^2|D) \propto p(\alpha|\sigma_g^2) \propto (\sigma_g^2)^{-\left(\frac{n-1}{2}\right)} \exp\left(-\frac{\alpha' \Lambda^{-1} \alpha}{2\sigma_g^2}\right), \quad (5.49)$$

that is proportional to the kernel of a scaled inverse chi-square distribution with $v_\alpha = n - 3$ degrees of freedom and scale $S_\alpha = (\alpha' \Lambda^{-1} \alpha / v_\alpha)$. The density up to proportionality is equal to

$$p(\sigma_g^2|D) \propto (\sigma_g^2)^{-\left(\frac{v_\alpha}{2} + 1\right)} \exp\left(-\frac{v_\alpha S_\alpha}{2\sigma_g^2}\right). \quad (5.50)$$

To extract a sample from (5.50) draw from a chi-square distribution with v_α degrees of freedom

$$\chi^2(v_\alpha)$$

and then compute

$$\frac{\alpha'_{-n} (\Lambda_{-n})^{-1} \alpha_{-n}}{\chi^2(v_\alpha)}.$$

Deriving $[\sigma_e^2|D]$

Extracting from (5.43) those terms that include σ_e^2 gives

$$\begin{aligned} p(\sigma_e^2|D) &\propto p(y|\mu, \alpha, \sigma_e^2) \\ &\propto (\sigma_e^2)^{-\frac{n}{2}} \exp\left[-\frac{1}{2\sigma_e^2} (y - 1\mu - U\alpha)' (y - 1\mu - U\alpha)\right]. \end{aligned}$$

Regarded as a function of σ_e^2 this is proportional to a scaled inverted chi-square density, with scale

$$S_e = \frac{(y - 1\mu - U\alpha)' (y - 1\mu - U\alpha)}{v_e}$$

and with $v_e = n - 2$ degrees of freedom. That is

$$p(\sigma_e^2|D) \propto (\sigma_e^2)^{-\left(\frac{v_e}{2}+1\right)} \exp\left(-\frac{v_e S_e}{2\sigma_e^2}\right). \quad (5.51)$$

To extract a sample from (5.51) draw from a chi-square distribution with v_e degrees of freedom

$$\chi^2(v_e)$$

and compute

$$\frac{(y - 1\mu - U\alpha)' (y - 1\mu - U\alpha)}{\chi^2(v_e)}.$$

An Alternative Definition of Genomic Variance

The Appendix of Chap. 1 discusses the distinction between the parameters defined at the level of the quantitative genetic model, regarded as the data generating mechanism and the parameters defined at the level of the operational model used to analyse the data. The genomic model is such an operational model. It was noted that the genomic variance of the genomic operational model (1.40), that is a similar parameter to σ_g^2 in the present example (here, the matrix of marker genotypes W is centred and scaled; in (1.40), it is centred), has an unclear connection with the parameter (1.37), that is argued, is the real focus of inference.

An alternative estimator of genomic variance (1.42) was defined on page 48, that is better aligned with (1.37). This subsection shows that estimator (1.42) can be obtained with one line of extra code, when the Bayesian model is implemented in an MCMC computing environment.

The $n \times 1$ vector of genomic values is $g = Wb = U\alpha$, where W is the $n \times m$ centred and scaled matrix of marker genotypes, b is the $m \times 1$ vector of genetic marker effects and α is the regression of genomic value on eigenvectors. The genomic variance or variance of genomic values was defined in (1.42) and in terms of the reparametrised model can be written as

$$\begin{aligned}\sigma_G^2 &= \frac{1}{n} g'g - \left(\frac{1}{n} 1'g \right)^2 \\ &= \frac{1}{n} \alpha' U' U \alpha \\ &= \frac{1}{n} \sum_{i=1}^n \alpha_i^2.\end{aligned}\tag{5.52}$$

The equality in the second line follows because when matrix W is column centred, $1'g = 1'Wb = 0$. Vector α is unknown and is therefore inferred from its marginal posterior distribution. When the Markov chain converges to its stationary distribution, extractions from this distribution are given by (5.48). For example, if $\alpha^{[t]}$ is the vector of draws from (5.48) at round t of the MCMC sampler, an extraction of σ_G^2 from its marginal posterior distribution is $\sigma_G^{2[t]} = \frac{1}{n} \alpha^{[t]'} \alpha^{[t]}$. The R-code is `mean(alpha*alpha)`, where `alpha` is the vector $\alpha^{[t]}$.

Scaled Inverted Chi-Square Prior Distributions for the Variance Components

A slightly different parametrisation that has the advantage of mitigating potential problems due to impropriety of posterior distributions that may arise with the use of improper priors of variance components (Hobert and Casella 1996), is to assume the prior specification

$$\begin{aligned}[\mu] &\propto \text{constant}, \\ [\sigma_g^2 | S_g, v_g] &\sim \chi^{-2}(S_g, v_g), \\ [\sigma_e^2 | S_e, v_e] &\sim \chi^{-2}(S_e, v_e),\end{aligned}$$

where $\chi^{-2}(S_i, v_i)$, $i = g, e$, is the scaled inverted chi-square distribution with v_i degrees of freedom and scale parameter S_i whose density function is

$$\begin{aligned} p(\sigma_i^2 | S_i, v_i) &= \frac{(v_i/2)^{(v_i/2)}}{\Gamma(v_i/2)} S_i^{(v_i/2)} (\sigma_i^2)^{-\left(\frac{v_i}{2}+1\right)} \exp\left(-\frac{v_i S_i}{2\sigma_i^2}\right) \\ &\propto (\sigma_i^2)^{-\left(\frac{v_i}{2}+1\right)} \exp\left(-\frac{v_i S_i}{2\sigma_i^2}\right), \quad v_i, S_i > 0. \end{aligned}$$

This specification leads to the posterior density

$$p(\mu, \alpha, \sigma_g^2, \sigma_e^2 | y) \propto p(\alpha | \sigma_g^2) p(y | \mu, \alpha, \sigma_e^2) p(\sigma_g^2 | S_g, v_g) p(\sigma_e^2 | S_e, v_e).$$

The fully conditional posterior distribution of σ_g^2 is now

$$\begin{aligned} p(\sigma_g^2 | D) &\propto p(\alpha | \sigma_g^2) p(\sigma_g^2 | S_g, v_g) \\ &\propto (\sigma_g^2)^{-\left(\frac{n-1}{2}\right)} \exp\left(-\frac{\alpha' \Lambda^{-1} \alpha}{2\sigma_g^2}\right) (\sigma_g^2)^{-\left(\frac{v_g}{2}+1\right)} \exp\left(-\frac{v_g S_g}{2\sigma_g^2}\right) \\ &= (\sigma_g^2)^{-\left(\frac{v_g^*}{2}+1\right)} \exp\left(-\frac{v_g^* S_g^*}{2\sigma_g^2}\right), \end{aligned}$$

where $v_g^* = (n-1) + v_g$ and $S_g^* = (\alpha' \Lambda^{-1} \alpha + v_g S_g) / v_g^*$. A Monte Carlo extraction from this distribution is obtained by drawing from a chi-square distribution with v_g^* degrees of freedom and computing

$$\frac{\alpha'_{-n} \Lambda^{-1} \alpha_{-n} + v_g S_g}{\chi^2(v_g^*)}.$$

The fully conditional posterior distribution of σ_e^2 is

$$\begin{aligned} p(\sigma_e^2 | D) &\propto p(y | \mu, \alpha, \sigma_e^2) p(\sigma_e^2 | S_e, v_e) \\ &\propto (\sigma_e^2)^{-\frac{\eta}{2}} \exp\left[-\frac{1}{2\sigma_e^2} \varepsilon' \varepsilon\right] (\sigma_e^2)^{-\left(\frac{v_e}{2}+1\right)} \exp\left(-\frac{v_e S_e}{2\sigma_e^2}\right) \\ &= (\sigma_e^2)^{-\left(\frac{v_e^*}{2}+1\right)} \exp\left(-\frac{v_e^* S_e^*}{2\sigma_e^2}\right), \end{aligned}$$

where $\varepsilon = (y - 1\mu - U\alpha)$, $v_e^* = v_e + n$ and $S_e^* = (\varepsilon'\varepsilon + v_e S_e) / v_e^*$. A Monte Carlo extraction from this distribution is obtained by drawing from a chi-square distribution with v_e^* degrees of freedom and computing

$$\frac{\varepsilon'\varepsilon + v_e S_e}{\chi^2(v_e^*)}.$$

The R-code CODE1313 on page 713 has an option to choose scaled inverted chi-square prior distributions for the variance components of the genomic model. The code can also be found and downloaded from <https://github.com/SorensenD/SLGDS> in the folder Codes.

5.5 Example: A Mixture Model of Two Gaussian Components

A description of a maximum likelihood analysis of a mixture model of two Gaussian components implemented with the EM algorithm was presented on page 134. Here, the mixture model is implemented with McMC in a Bayesian setup.

Let z_i , $i = 1, \dots, n$, be the (unobserved) binary random variable that indicates which of the two mixture components observation i comes from. Assume that the marginal probability of Z_i is Bernoulli $Br(\pi_1)$

$$\Pr(Z_i = z_i | \pi_1) = \pi_1^{z_i} (1 - \pi_1)^{1-z_i}, \quad z_i = 0, 1,$$

with

$$\Pr(Z_i = 1 | \pi_1) = \pi_1,$$

$$\Pr(Z_i = 0 | \pi_0) = \pi_0 = 1 - \pi_1.$$

An McMC data augmentation strategy is to use the unobserved Z to generate an augmented posterior distribution. Conditional on $z_i = j$, $j = 1, 0$, assume y_i has density $p_j(y_i | \theta_j, z_j)$. The pairs (Y_i, Z_i) are assumed to be *iid* with joint density

$$\begin{aligned} p(y_i, z_i | \theta, \pi) &= p_j(y_i | z_i = j, \theta_j) \Pr(Z_i = j | \pi_j) \\ &= p_j(y_i | z_i = j, \theta_j) \pi_j. \end{aligned}$$

To be specific, let $p_j(y_i | \theta_j, \sigma^2)$ be the density of the normal distribution $N(\theta_j, \sigma^2)$. The parameters of the model are $\theta_0, \theta_1, \sigma^2, \pi_0, z_i, i = 1, \dots, n$.

Assume that the priors for the θ 's are improper uniforms and that the density of the prior distribution of the remaining parameters factorises as follows

$$p(\theta_j, \sigma^2, \pi_1, z) \propto p(\sigma^2) p(z | \pi_1) p(\pi_1),$$

where $p(z|\pi_1) = \prod_{i=1}^n \Pr(Z_i = z_i|\pi_1)$. The probability π_1 is assigned a beta prior distribution with user tuned hyperparameters α and β ,

$$\pi_1|\alpha, \beta \sim Be(\alpha, \beta) \quad (5.53)$$

and the conditional variance of the data is a priori a scaled inverted chi-square distribution

$$\sigma^2|S, v \sim \chi^{-2}(S, v), \quad (5.54)$$

where S and the v are user-tuned hyperparameters. The density of the augmented posterior distribution takes the form

$$p(\theta_j, \sigma^2, \pi_1, z|y) \propto p(y|z, \theta_j, \sigma^2) p(\sigma^2) p(z|\pi_1) p(\pi_1). \quad (5.55)$$

The MCMC Gibbs sampling algorithm for this model is straightforward and can be implemented extracting the fully conditional posterior distributions (fcpd) from (5.55).

Deriving $[\theta_1|D]$

From (5.55) the fcpd of θ_j is

$$\begin{aligned} p(\theta_j|D) &\propto p(y|z, \theta_j, \sigma^2) \\ &\propto \exp\left[-\frac{1}{2\sigma^2} \sum_{i=1}^n \left\{I(z_i = j)(y_{ij} - \theta_j)^2\right\}\right]. \end{aligned}$$

A little algebra leads to

$$[\theta_j|D] \sim N\left(\hat{\theta}_j, \frac{\sigma^2}{n_j}\right). \quad (5.56)$$

In this expression,

$$\begin{aligned} \hat{\theta}_j &= \frac{1}{n_j} \sum_{i=1}^n y_i I(z_i = j), \\ n_j &= \sum_{i=1}^n I(z_i = j). \end{aligned}$$

Deriving $[z_i|D]$

From (5.55) the fcpd of z_i is

$$\Pr(Z_i = j|D) \propto p(y_i|z_i, \theta_j, \sigma^2) \Pr(z_i = j|\pi_j).$$

When $j = 0$,

$$p(y_i | z_i = 0, \theta_j, \sigma^2) \propto \exp \left[-\frac{1}{2\sigma^2} (y_i - \theta_0)^2 \right]$$

and

$$\Pr(Z_i = 0 | D) \propto \exp \left[-\frac{1}{2\sigma^2} (y_i - \theta_0)^2 \right] (1 - \pi_1).$$

When $j = 1$,

$$p(y_i | z_i = 1, \theta_j, \sigma^2) \propto \exp \left[-\frac{1}{2\sigma^2} (y_i - \theta_1)^2 \right]$$

and

$$\Pr(Z_i = 1 | D) \propto \exp \left[-\frac{1}{2\sigma^2} (y_i - \theta_1)^2 \right] \pi_1.$$

The fcpd is then the Bernoulli process

$$\Pr(Z_i = 1 | D) = \frac{\exp \left[-\frac{1}{2\sigma^2} (y_i - \theta_1)^2 \right] \pi_1}{\exp \left[-\frac{1}{2\sigma^2} (y_i - \theta_1)^2 \right] \pi_1 + \exp \left[-\frac{1}{2\sigma^2} (y_i - \theta_0)^2 \right] (1 - \pi_1)} \quad (5.57)$$

and the logodds is given by

$$\begin{aligned} \ln \left[\frac{\Pr(Z_i = 1 | D)}{\Pr(Z_i = 0 | D)} \right] &= \frac{1}{2\sigma^2} \left[(y_i - \theta_0)^2 - (y_i - \theta_1)^2 \right] \\ &\quad - [\ln(1 - \pi_1) - \ln \pi_1] = K_i. \end{aligned} \quad (5.58)$$

In a computing environment an extraction from (5.57) can be obtained efficiently as follows. Let un be a realisation from

$$un \sim Un(0, 1).$$

Then

$$z_i = \begin{cases} 1, & \text{if } \ln \left(\frac{un}{1-un} \right) \leq \ln \left[\frac{\Pr(Z_i=1|D)}{\Pr(Z_i=0|D)} \right], \\ 0, & \text{otherwise.} \end{cases} \quad (5.59)$$

There are two ways of characterising $\Pr(Z_i = 1|y)$. One is using the MCMC draws $z_i^{[j]}$, the j th sample from $\Pr(Z_i = 1|D)$. When the system converges, these are MCMC draws from marginal posterior distribution $[Z_i = 1|y]$ and their average is a Monte Carlo point estimate of the marginal posterior distribution

$$\varphi_i = \Pr(Z_i = 1|y).$$

The estimator is

$$\widehat{\varphi}_i = \widehat{\Pr}(Z_i = 1|y) = \frac{1}{l} \sum_{j=1}^l z_i^{[j]}, \quad (5.60)$$

where l is the length of the Gibbs chain.

The second way of characterising $\Pr(Z_i = 1|y)$ is by constructing φ_i at each round of the MCMC chain. Using the logodds (5.58),

$$\varphi_i = \frac{\exp(K_i)}{1 + \exp(K_i)}. \quad (5.61)$$

When the MCMC algorithm converges the value constructed from (5.61) at iteration j , $\varphi_i^{[j]}$, is an extraction from the marginal posterior distribution $[\varphi_i|y]$. The draws $\varphi_i^{[j]}$, $j = 1, \dots, l$, provide a Monte Carlo description of the complete marginal posterior distribution $[\varphi_i|y]$.

Deriving $[\sigma^2|D]$

From (5.55) the fcpd σ^2 is

$$\begin{aligned} p(\sigma^2|D) &\propto p(\sigma^2) p(y|z, \theta_j, \sigma^2) \\ &\propto (\sigma^2)^{-(1+\frac{v}{2})} \exp\left[-\frac{vS}{2\sigma^2}\right] (\sigma^2)^{-\frac{n}{2}} \\ &\quad \exp\left[-\frac{\sum_{i=1}^n \sum_{j=0}^1 I(z_i = j) \{(y_i - \theta_j)^2\}}{2\sigma^2}\right] \\ &= (\sigma^2)^{-(1+\frac{\tilde{v}}{2})} \exp\left[-\frac{\tilde{v}\tilde{S}}{2\sigma^2}\right], \end{aligned}$$

where $\tilde{v} = v + n$ and $\tilde{S} = \left[\left\{\sum_{i=1}^n \sum_{j=0}^1 I(z_i = j) (y_i - \theta_j)^2\right\} + vS\right]/\tilde{v}$. This is the kernel of a scaled inverted chi-square distribution with parameters $\tilde{v}$ and $\tilde{S}$:

$$[\sigma^2|D] \sim \chi^{-2}(\tilde{v}, \tilde{S}). \quad (5.62)$$

To obtain a sample from (5.62) draw a chi-square distribution with $\tilde{v}$ degrees of freedom and the reciprocal of this number is multiplied by $\tilde{v}\tilde{S}$.

Deriving $[\pi_1|D]$

From (5.55) the fcpd of π_1 is

$$\begin{aligned} p(\pi_1|D) &\propto p(z|\pi_1) p(\pi_1) \\ &= \pi_1^{\alpha-1} (1-\pi_1)^{\beta-1} \prod_{i=1}^n \Pr(Z_i = 1|\pi_1)^{z_i} (1 - \Pr(Z_i = 1|\pi_1))^{1-z_i} \\ &= \pi_1^{\sum_{i=1}^n z_i + \alpha - 1} (1-\pi_1)^{n - \sum_{i=1}^n z_i + \beta - 1} \end{aligned} \quad (5.63)$$

which is the kernel of a beta distribution with parameters $\sum_{i=1}^n z_i + \alpha$ and $n - \sum_{i=1}^n z_i + \beta$. Thus,

$$[\pi_1|D] \sim Be\left(\sum_{i=1}^n z_i + \alpha, n - \sum_{i=1}^n z_i + \beta\right). \quad (5.64)$$

An implementation of this Bayesian model is on page 386.

5.6 Example: An Application of the EM Algorithm in a Bayesian Context. Estimation of SNP Effects

A classical parametrisation of the genomic model is based on the multiple linear regression

$$y = Z\alpha + X\beta + e, \quad (5.65)$$

where Z is an observed incidence matrix of dimension $n \times p$ that associates fixed effects α to the data y (a column vector of length n) and matrix X , of dimension $n \times m$, is an observed matrix of SNP genotypes where each element takes the value 0, 1 or 2. Row i of X contains the marker genotypes of the m SNPs of individual i . Column vector β (of length m) represents unobserved SNP effects and e is a vector of residuals with *iid* elements $N(0, \sigma_e^2)$.

Sun et al. (2012) present an interesting application of the EM algorithm for estimation of SNP effects in a Bayesian framework. In contrast with model (1.39) on page 45 that poses a single variance parameter for all SNP effects, Sun et al. (2012) assume that the vector of SNP effects can be represented with the normal structure

$$\beta|\sigma^2 \sim N(0, D), \quad \sigma^2 = (\sigma_1^2, \sigma_2^2, \dots, \sigma_m^2), \quad (5.66)$$

where

$$D = \text{diag} \left\{ \sigma_j^2 \right\}, \quad j = 1, \dots, m, \quad (5.67)$$

and

$$\sigma_j^2 \stackrel{iid}{\sim} \chi^{-2} (v_\beta, S_\beta), \text{ for all } j. \quad (5.68)$$

The SNP model assumes that a priori, SNP effects β are conditionally (given D) independent and normally distributed and that each SNP has its own variance σ_j^2 . The variances of the m SNPs are *iid* realisations from a common scaled inverse chi-square distribution $\chi^{-2} (v_\beta, S_\beta)$ with hyperparameters v_β (the degrees of freedom) and scale S_β . It follows that the density of the marginal distribution of a SNP effect is

$$p(\beta_j | v_\beta, S_\beta) = \int p(\beta_j | \sigma_j^2) p(\sigma_j^2 | v_\beta, S_\beta) d\sigma_j^2,$$

the density of a t -distribution with v_β degrees of freedom and scale S_β (see page 155 for a characterisation of a t -distributed random variable as a mixture).

The EM algorithm proposed by Sun et al. (2012) treats the variance of each SNP effect σ_j^2 as missing data and maximises the resulting Q function with respect to α , β and σ_e^2 . The objective is to find the joint modal value of α , β and σ_e^2 .

Preliminaries

Before deriving the EM equations a little background is needed. The Q function involves expectation of $\ln p(y, \beta, \sigma, \sigma_e) = \ln [p(y | \alpha, \beta, \sigma_e^2) p(\beta | \sigma^2) p(\sigma^2)]$ with respect to the distribution $[\sigma^2 | \beta^{[l]}, y]$. First, the form of the conditional posterior distribution $[\sigma_j^2 | \beta^{[l]}, y]$ is needed. The posterior density is proportional to $p(y | \alpha, \beta, \sigma_e) p(\beta | \sigma^2) p(\sigma^2)$ and extracting terms including σ_j^2 yields

$$\begin{aligned} p(\sigma_j^2 | \beta, y) &\propto p(\beta_j | \sigma_j^2) p(\sigma_j^2) \\ &= (\sigma_j^2)^{-\frac{1}{2}} \exp\left(-\frac{\beta_j^2}{2\sigma_j^2}\right) (\sigma_j^2)^{-\left(\frac{v_\beta}{2}+1\right)} \exp\left(-\frac{v_\beta S_\beta}{2\sigma_j^2}\right) \\ &= (\sigma_j^2)^{-\left(\frac{v_\beta}{2}+\frac{3}{2}\right)} \exp\left(-\frac{\beta_j^2 + v_\beta S_\beta}{2\sigma_j^2}\right) \\ &= (\sigma_j^2)^{-\left(\frac{v_\beta^*}{2}+1\right)} \exp\left(-\frac{v_\beta^* S_\beta^*}{2\sigma_j^2}\right) \end{aligned} \quad (5.69)$$

that is the kernel of a scaled inverse chi-square density with v_β^* degrees of freedom and scale parameter S_β^* where

$$v_\beta^* = v + 1,$$

$$S_\beta^* = \frac{\beta_j^2 + v_\beta S_\beta}{v_\beta^*}.$$

As shown below, evaluation of the Q function involves the expected value of $(\sigma_j^2)^{-1}$. Using the theory of transformation of random variables, the pdf of $w = \frac{1}{\sigma_j^2}$, from (5.69), is

$$p(w|\beta, y) \propto (w^{-1})^{-\left(\frac{v_\beta^*}{2} + 1\right)} \exp\left(-\frac{v_\beta^* S_\beta^* w}{2}\right) w^{-2}$$

$$= w^{\frac{v_\beta^*}{2} - 1} \exp\left(-\frac{v_\beta^* S_\beta^* w}{2}\right). \quad (5.70)$$

Defining $a = v_\beta^*/2$ and $b = v_\beta^* S_\beta^*/2$, (5.70) can be written as

$$w^{a-1} \exp(-bw), \quad (5.71)$$

which is the kernel of a Gamma distribution whose mean is

$$E(w|\beta, y) = \frac{a}{b} = \frac{1}{S_\beta^*} = \left(\frac{\beta_j^2 + v_\beta S_\beta}{v_\beta^*}\right)^{-1}. \quad (5.72)$$

E-step

The Q function involved in the E step is

$$Q = E\left(\ln\left[p(y|\alpha, \beta, \sigma_e) p(\beta|\sigma^2) p(\sigma^2)\right]\right)$$

where integration is with respect to $[\sigma^2|\beta^{[t]}, y]$. Taking logarithms and noting that the first term is a constant with respect to $[\sigma^2|\beta^{[t]}, y]$ yields

$$Q = \ln p(y|\alpha, \beta, \sigma_e) + E\left[\ln p(\beta|\sigma^2)\right] + E\left[\ln p(\sigma^2)\right]. \quad (5.73)$$

The last term in the right hand side does not involve $\alpha, \beta, \sigma_e^2$ and therefore does not need to be evaluated (it drops out in the M -step). Working with the middle term of

the right hand side, due to independence, the expectation can be computed element-wise yielding

$$\begin{aligned}
 \mathbb{E} \left[\ln p(\beta | \sigma^2) \right] &= \sum_{i=1}^m \mathbb{E} \left(\ln \left[\left(2\pi \sigma_i^2 \right)^{-\frac{1}{2}} \exp \left(-\frac{\beta_i^2}{2\sigma_i^2} \right) \right] \right) \\
 &= -\frac{1}{2} \sum_{i=1}^m \mathbb{E} \left(\ln 2\pi + \ln \sigma_i^2 + \beta_i^2 \frac{1}{\sigma_i^2} \right) \\
 &= -\frac{1}{2} \sum_{i=1}^m \mathbb{E} \left(\beta_i^2 \frac{1}{\sigma_i^2} \right) + R \\
 &= -\frac{1}{2} \sum_{i=1}^m \beta_i^2 \left(\frac{\beta_i^{2[t]} + v_\beta S_\beta}{v_\beta^*} \right)^{-1} + R, \tag{5.74}
 \end{aligned}$$

where R includes terms that do not involve $\alpha, \beta, \sigma_e^2$. The expectation in the third line is obtained from (5.72) where β_i^2 is factored out as a constant and this operation leads to the fourth line. Then the Q function ignoring an additive constant reduces to

$$\begin{aligned}
 Q &= \ln p(y | \alpha, \beta, \sigma_e^2) - \frac{1}{2} \sum_{i=1}^m \beta_i^2 \left(\frac{\beta_i^{2[t]} + v_\beta S_\beta}{v_\beta^*} \right)^{-1} + R \\
 &= \ln \left\{ \left(\sigma_e^2 \right)^{-\frac{n}{2}} \exp \left[-\frac{1}{2\sigma_e^2} (y - Z\alpha - X\beta)' (y - Z\alpha - X\beta) \right] \right\} \\
 &\quad - \frac{1}{2} \beta' D^{[t]} \beta + R \\
 &= -\frac{n}{2} \ln \sigma_e^2 - \frac{1}{2\sigma_e^2} (y - Z\alpha - X\beta)' (y - Z\alpha - X\beta) - \frac{1}{2} \beta' D^{[t]} \beta + R \tag{5.75}
 \end{aligned}$$

where

$$D^{[t]} = \text{diag} \left[\left(\frac{\beta_i^{2[t]} + v_\beta S_\beta}{v_\beta^*} \right)^{-1} \right], \quad i = 1, \dots, m.$$

M-step

The M-step involves maximisation of (5.75) with respect to σ_e^2, α and β . Maximising with respect to σ_e^2 , setting the derivative equal to zero and solving for σ_e^2 yields

$$\sigma_e^{2[t+1]} = \frac{1}{n} \left(y - Z\alpha^{[t+1]} - X\beta^{[t+1]} \right)' \left(y - Z\alpha^{[t+1]} - X\beta^{[t+1]} \right). \tag{5.76}$$

Similarly for α

$$\begin{aligned}\frac{\partial Q}{\partial \alpha} &= \frac{1}{\sigma_e^2} Z' (y - Z\alpha - X\beta) = 0, \\ Z'Z\alpha + Z'X\beta &= Z'y,\end{aligned}\tag{5.77}$$

and for β

$$\begin{aligned}\frac{\partial Q}{\partial \beta} &= \frac{1}{\sigma_e^2} X' (y - Z\alpha - X\beta) - D^{[t]}\beta = 0, \\ X'Z\alpha + [X'X + \sigma_e^2 D^{[t]}]\beta &= X'y.\end{aligned}\tag{5.78}$$

Equations (5.77) and (5.78) result in the system

$$\begin{bmatrix} Z'Z & Z'X \\ X'Z & X'X + \sigma_e^2 D^{[t]} \end{bmatrix} \begin{bmatrix} \alpha^{[t+1]} \\ \beta^{[t+1]} \end{bmatrix} = \begin{bmatrix} Z'y \\ X'y \end{bmatrix}.\tag{5.79}$$

These equations have the same structure as those that originate from a mixed model where the random effect β has variance

$$\left(D^{[t]}\right)^{-1} = \text{diag} \left[\left(\frac{\beta_i^{2[t]} + v_\beta S_\beta}{v_\beta^*} \right) \right], \quad i = 1, \dots, m.\tag{5.80}$$

An Alternative Parametrisation

The system of equations (5.79) has dimension $(p + m) \times (p + m)$. When the number of individuals (n) is smaller than the number of markers (m) the following strategy is computationally more attractive. Define genomic values as $g = X\beta$ and assume that

$$g|X \sim N \left(0, X \left(D^{[t]} \right)^{-1} X' \right),$$

where $\left(D^{[t]}\right)^{-1}$ is defined in (5.80). Secondly, find the maximiser of $p(y|\alpha, g, \sigma_e^2) p(g|D^{[t]})$. This requires

$$\begin{aligned}\frac{\partial p(y|\alpha, g, \sigma_e^2) p(g|D^{[t]})}{\partial \alpha} &= 0, \\ \frac{\partial p(y|\alpha, g, \sigma_e^2) p(g|D^{[t]})}{\partial g} &= 0.\end{aligned}$$

The maximisation involves the following operations:

$$\frac{\partial \left[-\frac{1}{2\sigma_e^2} (y - X\alpha - g)' (y - X\alpha - g) - \frac{1}{2} g' G^{-1} g \right]}{\partial \alpha} = 0,$$

$$\frac{\partial \left[-\frac{1}{2\sigma_e^2} (y - X\alpha - g)' (y - X\alpha - g) - \frac{1}{2} g' G^{-1} g \right]}{\partial g} = 0,$$

where $G = X (D^{[t]})^{-1} X'$. This yields

$$\begin{bmatrix} Z'Z & Z' \\ I & I + \sigma_e^2 G^{-1} \end{bmatrix} \begin{bmatrix} \alpha^{[t+1]} \\ g^{[t+1]} \end{bmatrix} = \begin{bmatrix} Z'y \\ y \end{bmatrix}. \quad (5.81)$$

The system of equations is of dimension $(p + n) \times (p + n)$. To retrieve marker effects from (5.81) use (5.80) and write

$$\begin{bmatrix} \beta^{[t+1]} \\ g^{[t+1]} \end{bmatrix} \sim SN \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} (D^{[t]})^{-1} & (D^{[t]})^{-1} X' \\ X (D^{[t]})^{-1} & X (D^{[t]})^{-1} X' \end{bmatrix} \right).$$

Then

$$E \left(\beta^{[t+1]} | g^{[t+1]} \right) = (D^{[t]})^{-1} X' \left(X (D^{[t]})^{-1} X' \right)^{-1} g^{[t+1]}. \quad (5.82)$$

5.7 Example: Bayesian Analysis of the Truncated Normal Model

The setup here is the same as on page 142 where a likelihood analysis was implemented with the EM algorithm. To recapitulate, the data available originates from an initial draw of N records from the normal distribution $\mathcal{N}(\mu, \sigma^2)$. In a second stage only records larger than an observed threshold C are kept and those smaller than C are discarded. After this procedure one can confirm that there are n observable records and that m are missing. The complete data z consist of $N = m + n$ records, $z' = ((y^*)', y')$ where y^* are the m (unobserved) missing records and y are the n observed records.

A Bayesian MCMC analysis can be easily implemented augmenting the parameter space with the missing data. An observed datum is conceptually generated from

$$y_i | \mu, \sigma^2 \sim \mathcal{N}(\mu, \sigma^2) I(z_i > C)$$

and a missing datum from

$$y_i^* | \mu, \sigma^2 \sim \mathcal{N}(\mu, \sigma^2) I(z_i \leq C)$$

where $I(x \in A)$ is the indicator function that takes the value 1 if $x \in A$ and zero otherwise. Assuming independence the conditional density of the complete data is

$$p(z | \mu, \sigma^2) \propto \prod_{i=1}^N \mathcal{N}(z_i | \mu, \sigma^2) [I(z_i > C) + I(z_i \leq C)]. \quad (5.83)$$

The augmented posterior density is

$$\begin{aligned} p(\mu, \sigma^2, y^* | y) &\propto p(\mu, \sigma^2, y^*) p(y | \mu, \sigma^2, y^*) \\ &= p(\mu, \sigma^2) p(y, y^* | \mu, \sigma^2). \end{aligned} \quad (5.84)$$

Assuming independent uniform prior distributions for μ and σ^2 , this augmented posterior is proportional to (5.83). The parameters of the augmented Bayesian model are (μ, σ^2, y^*) and a Gibbs sampling implementation requires drawing from $[y^* | D]$, $[\mu | D]$ and $[\sigma^2 | D]$, where as before, D is a vector containing the observations y and all the parameters of the model except the one to be updated.

The update of y^* requires choosing the terms that include y^* from the augmented posterior (5.83). The resulting fully conditional posterior distribution is

$$[y^* | D] \propto \prod_{i=1}^m p(y_i^* | \mu, \sigma^2) I(y_i^* \leq C)$$

that is in the form of a truncated normal distribution with mean μ and variance σ^2 with probability density function

$$p(y_i^* | \mu, \sigma^2, y_i^* \leq C) = \frac{p(y_i^* | \mu, \sigma^2)}{\Phi(c)}, \quad c = \frac{C - \mu}{\sigma},$$

where $\Phi(\cdot)$ is the cdf of the $\mathcal{N}(0, 1)$. An efficient algorithm to draw from truncated distributions was given in (5.27).

The update for σ^2 is again based on the construction of its fully conditional distribution $[\sigma^2 | D]$. From (5.83) this takes the form

$$[\sigma^2 | D] \propto \prod_{i=1}^N p(z_i | \mu, \sigma^2).$$

The density is proportional to

$$(\sigma^2)^{-\frac{N}{2}} \exp \left[-\frac{\sum_{i=1}^N (z_i - \mu)^2}{2\sigma^2} \right].$$

This is in the form of a scaled inverted chi-square distribution with scale parameter $S = \sum_{i=1}^N (z_i - \mu)^2$ and $N - 2$ degrees of freedom. Therefore,

$$[\sigma^2|D] \sim \chi^{-2}(N - 2, S).$$

To obtain a draw first sample from a chi-square distribution with $N - 2$ degrees of freedom, invert this number and multiply by S .

The final update requires drawing from $[\mu|D]$. This distribution is proportional to (5.83), now seen as a function of μ . Adding a subtracting $\bar{z} = \sum_{i=1}^N z_i / N$ in the squared term and dropping the terms that do not contain μ results in

$$\frac{\sum_{i=1}^N [(z_i - \bar{z}) + (\bar{z} - \mu)]^2}{2\sigma^2} = \frac{N(\bar{z} - \mu)^2}{2\sigma^2} + k,$$

where k is an additive constant. Regarded as a function of μ this expression is recognised as the logarithm of the kernel of $N(\bar{z}, \sigma^2 / N)$. Therefore

$$[\mu|D] \sim N\left(\bar{z}, \sigma^2 / N\right).$$

The R-code below executes a Gibbs sampler using the same data as used on page 142.

```
# CODE0503
# EM FOR TRUNCATED DATA; ESTIMATE MEAN OF UNTRUNCATED
# GENERATE Y ~ N(MEAN,VAR)
# TRUNCATE AT T SO THAT Z = Y > T ARE OBSERVED
# Y < T ARE MISSING (KNOWN INFORMATION)
rm(list=ls()) # CLEAR WORKSPACE
set.seed(12371)
nindiv<-50000
mean <- 10
var <- 3
T <- mean + 1.5*sqrt(var) # ASSUMED KNOWN
# CREATE COMPLETE DATA
y <- rnorm(nindiv,mean,sqrt(var))
# TRUNCATE: OBSERVED DATA
z <- y[y>T]
#length(z)
m <- length(y)-length(z)
#mean(y)
#mean(z)
#var(z)
##### MCMC #####
nrep <- 1000
resmc <- matrix(data=NA,nrow=nrep,ncol=2)
w <- rep(0,m)
# START VALUES FOR MEAN (mu) AND VARIANCE (sigmasq)
mu <- 0
```

```

sigmasq <- 2
sigma <- sqrt(sigmasq)
ptm <- proc.time()
for (j in 1:nrep){
  # print(j)
  T_star <- (T-mu)/sigma
  std <- sqrt(var)
  # sample m missing records in one go (left from threshold T)
  w <- mu + std*qnorm(runif(m)*pnorm(T_star))
  # sample the variance
  scale <- sum((w-mu)^2) + sum((z-mu)^2)
  sigmasq <- scale/rchisq(1,length(y)-2)
  sigma <- sqrt(sigmasq)
  # sample the mean
  xbar <- (sum(w)+sum(z))/(length(w)+length(z))
  disp <- sigmasq/(length(w)+length(z))
  mu <- rnorm(1,xbar,sqrt(disp))
  resmc[j,] <- c(mu,sigmasq)
}
proc.time()-ptm

```

```

##      user  system elapsed
##      2.81    0.14    2.96

```

```

postmean <- mean(resmc[100:nrep,1])
postvar <- mean(resmc[100:nrep,2])
postmean

```

```
## [1] 10.03055
```

```
postvar
```

```
## [1] 3.005774
```

```

# 95% POSTERIOR INTERVAL FOR THE MEAN
pimean <- quantile(resmc[100:nrep,1],c(0.025,0.975))
# 95% POSTERIOR INTERVAL FOR THE VARIANCE
pivar <- quantile(resmc[100:nrep,2],c(0.025,0.975))
pimean

```

```

##      2.5%    97.5%
## 10.00086 10.06202

```

```
pivar
```

```

##      2.5%    97.5%
## 2.953886 3.059747

```

The MC estimates of the posterior mean and variance are 10.031 and 3.006, in good agreement with the parameters of the untruncated distribution (and with the

ML estimates introduced on page 142). The sampler generates the complete joint posterior distribution of (μ, σ^2, y^*) and the margins correspond to the marginal posterior distribution of each variable in turn. MC estimates of posterior intervals are obtained from the `nrep` draws (here I exclude the first 100 draws as burn-in). The MC estimate of the 95% posterior interval for the posterior mean is (10.001, 10.062) and for the posterior variance (2.954, 3.060).

5.8 A Digression on Model Comparison

The general problem of model criticism and model choice occupies a vast literature including classical and Bayesian approaches. Here the topic is visited briefly and it is shown that, within an McMC environment, a few extra calculations lead to a useful and versatile measure of model comparison: the *pseudo log-marginal probability of the data*. The basic ideas are summarised and an example is provided.

The *pseudo log-marginal probability of the data* is a standard measure of model comparison (Gelfand 1996) and is defined and computed as follows. Consider data vector $y' = (y_i, y'_{-i})$, $i = 1, \dots, n$, where y_i is the i th datum and y_{-i} is the vector of data with the i th datum deleted. The *conditional predictive distribution* of y_i has density

$$p(y_i | y_{-i}) = \int p(y_i | \theta, y_{-i}) f(\theta | y_{-i}) d\theta, \quad (5.85)$$

where θ is the vector of parameters of the model. This density can be interpreted as the probability of each data point given the remainder of the data; a low value indicates that the datum is poorly fit by the model. The actual value of $p(y_i | y_{-i})$ is known as the *conditional predictive ordinate* (CPO) for the i th observation. A plot of the CPO's versus the index for the observations can serve as a useful tool for outlier detection. The *pseudo log-marginal probability of the data* or *pseudo marginal likelihood* is given by

$$\sum_i \ln p(y_i | y_{-i}). \quad (5.86)$$

The *pseudo Bayes factor* for comparing two models M_1 and M_2 (Gelfand et al. 1992; Gelfand 1996) is

$$PBF_{12} = \prod_{i=1}^n \frac{\Pr(Y_i = y_i | y_{-i}, M_1)}{\Pr(Y_i = y_i | y_{-i}, M_2)}. \quad (5.87)$$

A Monte Carlo approximation of the CPO (5.85) for observation i is given by (Gelfand 1996)

$$\hat{p}(y_i|y_{-i}, M_k) = N \left[\sum_{j=1}^N \frac{1}{p(y_i|y_{-i}, \theta^{(j)}, M_k)} \right]^{-1}, \quad (5.88)$$

where N is the number of McMC draws, M_k is a label for model k and $\theta^{(j)}$ is the j th draw from the posterior of θ under model M_k . Often the y_i 's are conditionally independent given θ ; then the term $p(y_i|y_{-i}, \theta^{(j)}, M_k)$ in (5.88) simplifies to $p(y_i|\theta^{(j)}, M_k)$.

The so-called *LogCPO*'s are based on

$$\sum_i \ln \hat{p}(y_i|y_{-i}, M_k). \quad (5.89)$$

Larger values indicate a relative better fit.

An appealing side of (5.88) is that only one analysis is required, rather than n (n is sample size), where one out of n observations is left out in each of the n analyses.

A very useful property is that $p(y_i|y_{-i})$ is always a proper density, provided the posterior density is proper. The models under comparison do not need to be nested and since asymptotics are not involved, there is no problem with testing values of parameters that lie on the border of the parameter space.

The derivation of (5.88) is instructive and straightforward. The details are as follows (Gelfand 1996):

$$\begin{aligned} p(y_i|y_{-i}, M_r) &= \frac{p(y|M_r)}{p(y_{-i}|M_r)} \\ &= \frac{1}{\frac{p(y_{-i}|M_r)}{p(y|M_r)}} \\ &= \frac{1}{\int p(y_{-i}, \theta_r|M_r) \frac{1}{p(y|M_r)} d\theta_r} \\ &= \frac{1}{\int p(y_{-i}, \theta_r|M_r) \frac{p(\theta_r|y, M_r)}{p(\theta_r, y|M_r)} d\theta_r} \\ &= \frac{1}{\int \frac{p(\theta_r, y|M_r)}{p(y_i|y_{-i}, \theta_r, M_r)} \frac{p(\theta_r|y, M_r)}{p(\theta_r, y|M_r)} d\theta_r} \\ &= \frac{1}{\int \frac{1}{p(y_i|y_{-i}, \theta_r, M_r)} p(\theta_r|y, M_r) d\theta_r}, \end{aligned} \quad (5.90)$$

where the fifth line is a consequence of the equality

$$p(y_i | y_{-i}, \theta_r, M_r) = \frac{p(\theta_r, y | M_r)}{p(y_{-i}, \theta_r | M_r)}.$$

A Monte Carlo estimator of (5.90) is (5.88).

Example

Data are available from the following “true” model

$$\begin{aligned} y_{ij} | \mu, f_i &\stackrel{iid}{\sim} N(\mu + f_i, \sigma_e^2), \\ f_i | \sigma_f^2 &\stackrel{iid}{\sim} N(0, \sigma_f^2), \\ i &= 1, \dots, n_f, \quad j = 1, \dots, n. \end{aligned}$$

There are n_f full-sib families, with n full-sibs per family and a total of $N = n_f \times n$ records. In matrix notation the model is

$$y = 1\mu + Zf + e,$$

where 1 is a vector of ones of length N , Z is of dimension $N \times n_f$ and f and e contain family and residual effects, respectively.

The data set is analysed using the “true” model and using a “wrong” model that assumes $\sigma_f^2 = 0$. This model is

$$\begin{aligned} y_i | \mu &\stackrel{iid}{\sim} N(\mu, \sigma_e^2), \\ i &= 1, \dots, N. \end{aligned}$$

The two Bayesian models will be implemented with the Gibbs sampler and the *pseudo marginal likelihood* (5.89) will be calculated for each.

The values of n and n_f are set equal to 3 and 400, respectively, leading to $N = 1200$ records. Variance components are assumed to be $\sigma_f^2 = 10$ and $\sigma_e^2 = 50$.

A Gibbs Sampler for the “True” Model

The Bayesian model assumes improper, uniform prior distributions for $(\mu, \sigma_f^2, \sigma_e^2)$. The posterior density is

$$\begin{aligned}
 p(\mu, \sigma_f^2, \sigma_e^2 | y) &\propto p(f | \sigma_f^2) p(y | \mu, \sigma_f^2, \sigma_e^2) \\
 &\propto (\sigma_e^2)^{-\frac{N}{2}} (\sigma_f^2)^{-\frac{n_f}{2}} \exp\left(-\frac{1}{2\sigma_f^2} f' f\right) \\
 &\quad \exp\left(-\frac{1}{2\sigma_e^2} (y - 1\mu - Zf)' (y - 1\mu - Zf)\right) \\
 &= (\sigma_e^2)^{-\frac{N}{2}} (\sigma_f^2)^{-\frac{n_f}{2}} \\
 &\quad \exp\left[-\frac{1}{2\sigma_e^2} \{(y - 1\mu - Zf)' (y - 1\mu - Zf) + kf' f\}\right],
 \end{aligned}$$

where $k = \sigma_e^2 / \sigma_f^2$. The Gibbs sampler is implemented drawing from the following fully conditional posterior distributions:

$$\begin{aligned}
 [\mu | D] &\sim N(\hat{\mu}, (1'1)^{-1} \sigma_e^2), \\
 \hat{\mu} &= (1'1)^{-1} 1' (y - Zf), \\
 [f | D] &\sim N(\hat{f}, (Ik + Z'Z)^{-1} \sigma_e^2), \\
 \hat{f} &= (Ik + Z'Z)^{-1} Z' (y - 1\mu), \\
 [\sigma_f^2 | D] &\sim \frac{f' f}{\chi^2 (n_f - 2)}, \\
 [\sigma_e^2 | D] &\sim \frac{(y - 1\mu - Zf)' (y - 1\mu - Zf)}{\chi^2 (N - 2)}.
 \end{aligned}$$

The computation of (5.89) requires a few intermediate calculations. In each Gibbs round one first calculates

$$p(y_i | \mu^{[j]}, f^{[j]}, \sigma_e^{2[j]}, M_{correct}) \quad i = 1, \dots, N.$$

This can be accomplished for all the data (vector y) in one go using the R function

```
dnorm(y, mean=mu+Z%*%f, sd=sqrt(ve))
```

Specifically, for each round compute

```
pyinv< - 1/(dnorm(y,mean=mu+Z%*%f,sd=sqrt(ve)))
sumpyinv< - sumpyinv+pyinv
```

Once all the iterations have been executed one computes (5.88) and finally (5.89). The code is

```
phatyt< - rep*(sumpyinv)^(-1)}
logcpot< - sum(log(phatyt))
```

A Gibbs Sampler for the “Wrong” Model

Using improper uniform priors for $(\mu, \sigma_\varepsilon^2)$, the posterior density under the incorrect model is

$$p\left(\mu, \sigma_\varepsilon^2 | y\right) \propto p\left(y | \mu, \sigma_\varepsilon^2\right) \\ \propto \left(\sigma_\varepsilon^2\right)^{-\frac{N}{2}} \exp\left(-\frac{1}{2\sigma_\varepsilon^2} (y - 1\mu)' (y - 1\mu)\right).$$

It is straightforward to derive the fully conditional posterior distributions. These are

$$[\mu | D] \sim N\left(\hat{\mu}, (1'1)^{-1} \sigma_\varepsilon^2\right), \\ \hat{\mu} = (1'1)^{-1} 1'y, \\ \left[\sigma_\varepsilon^2 | D\right] \sim \frac{(y - 1\mu)' (y - 1\mu)}{\chi^2(N - 2)}.$$

The R-code below generates data under the true model and then runs a Gibbs sampler with the two models. The *pseudo marginal likelihoods* (5.89) are calculated for each model. The code is spelled out line by line avoiding the use of more efficient and more compact programming that is to be preferred with more demanding computations.

```
# CODE0504
#CPO EXAMPLE
rm(list=ls()) # Clear the workspace
set.seed(123771)
ptm<-proc.time()
require(graphics)
# GENERATE CORRELATED (FULL-SIBS DATA)
#install.packages("MCMCpack", .libPaths()[1])
#install.packages("mvtnorm", .libPaths()[1])
library(MCMCpack)
```

```

# INITIALISE PARAMETERS
mus<-10 # MEAN
vfs<-10 #VARIANCE BETWEEN FULL-SIBS
# RESIDUAL VARIANCE
ves<-50
nf<-400 # NUMBER OF FULL-SIB FAMILIES
n<-3 #FULL-SIB FAMILY SIZE
N<-nf*n
y<-matrix(data=0,nrow=nf*n,ncol=1)
# z IS COLUMN MATRIX WITH FAMILY ID (ID=1,..,nfs)
z<-matrix(data=0,nrow=nf*n,ncol=1)
# GENERATE nf FULL-SIB EFFECTS f
fs<-rnorm(nf,mean=0,sd=sqrt(vfs))
# GENERATE nf*n RESIDUAL EFFECTS f
es<-rnorm(nf*n,mean=0,sd=sqrt(ves))
# GENERATE FULL SIBS (CAN CHOOSE MORE TRANSPARENT LOOP ABOVE)
z <- rep(1:nf,each=n)
y <- mus+fs[z]+es
d<-data.frame(y,z)
# GENERATE INCIDENCE MATRIX Z
family<-z
family <- as.factor(family)
Z<-model.matrix(~0+family)
# WITH INDEPENDENT FAMILIES Z'Z IS DIAGONAL
ztz<-rep(n,nf)
#END OF GENERATION OF DATA Y
#CHOOSE LENGTH OF CHAIN
rep<-1000
resulttt<-matrix(data=NA,nrow=rep,ncol=4)
resulttw<-matrix(data=NA,nrow=rep,ncol=3)
#INITIALISE THE VECTOR OF FAMILY EFFECTS f
f<-rep(0,nf)
# INITIALISE BETWEEN FAMILY VARIANCE COMPONENT vf
vf<-5
# INITIALISE RESIDUAL VARIANCE
ve<-5
# INITIALISE k
k<-ve/vf
# INITIALISE THE MEAN
mu<-0
sumpyinv<-0
#START GIBBS LOOP TRUE MODEL
for (i in 1:rep)
{
  # SAMPLE mu
  meanmu<-sum(y-Z**f)/(nf*n)
  mu<-rnorm(1,mean=meanmu,sd=sqrt(ve/(nf*n)))
  # SAMPLE FAMILY EFFECTS f
  varf<-(k+n)^(-1)
  fmean<- varf*(t(Z)**(y-mu))
  f<-rnorm(nf,mean=fmean, sd=sqrt(varf*ve))
  #SAMPLE vf
  #COMPUTE SCALE

```

```

ftf<-sum(f*f)
vfx<-ftf/rchisq(1,nf-2)
vf<-as.numeric(vfx)
# SAMPLE ve
# COMPUTE SCALE
e<-(y-mu-Z**%f)
ete<-t(e)**e
vex<-ete/rchisq(1,N-2)
ve<-as.numeric(vex)
k<-ve/vf
resultt[i,]<-c(i,mu,vf,ve)
# print(resultt[i,])
# COMPUTE CPOs FOR TRUE MODEL
pyinv<-1/(dnorm(y,mean=mu+Z**%f,sd=sqrt(ve)))
sumpyinv<-sumpyinv+pyinv
}
phatyt<-rep*(sumpyinv)^(-1)
logcpot<-sum(log(phatyt))
proc.time()-ptm

```

```

##      user  system elapsed
##  14.14      0.41      3.67

```

```

#START GIBBS LOOP WRONG MODEL
vew<-20
sumpyinvw<-0
for (i in 1:rep)
{
  # SAMPLE muw
  meanmuw<-sum(y)/N
  muw<- rnorm(1,mean=meanmuw,sd=sqrt(vew/(N)))
  # SAMPLE vew
  vew<-sum((y-muw)*(y-muw))/rchisq(1,N-2)
  resultw[i,]<-c(i,muw,vew)
  # COMPUTE CPOs FOR WRONG MODEL
  pyinvw<-1/(dnorm(y,mean=mu,sd=sqrt(vew)))
  sumpyinvw<-sumpyinvw+pyinvw
}
phatyw<- rep*(sumpyinvw)^(-1)
logcpow<-sum(log(phatyw))
# PRINT OUTPUT
# LOG CPO TRUE MODEL
logcpot

```

```

## [1] -4176.398

```

```

# 95% POSTERIOR INTERVALS FOR THE MEAN AND THE TWO VARIANCES
quantile(resultt[,2],c(0.025,0.975))

```

```

##      2.5%      97.5%
##  9.344941 10.410468

```

```
quantile(resulttt[,3],c(0.025,0.975))
```

```
##      2.5%      97.5%
##  5.504737 13.107484
```

```
quantile(resulttt[,4],c(0.025,0.975))
```

```
##      2.5%      97.5%
## 49.94121 60.39323
```

```
# LOG CPO FOR WRONG MODEL
logcpow
```

```
## [1] -4194.195
```

```
# 95% POSTERIOR INTERVALS FOR THE MEAN AND VARIANCE
quantile(resultw[,2],c(0.025,0.975))
```

```
##      2.5%      97.5%
##  9.422171 10.308375
```

```
quantile(resultw[,3],c(0.025,0.975))
```

```
##      2.5%      97.5%
## 58.87403 69.06082
```

McMC estimates of 95% posterior intervals for the parameters of both models are printed at the bottom of the code. The $\ln CPO$ based on (5.89) for the true model is -4176 and for the incorrect model is -4194 . The difference in favour of the true model is very large. This may not be the case if σ_f^2 is small. For example, running the program with $\sigma_f^2 = 5$ results in a $\ln CPO$ for the true model equal to -4045 and for the wrong model, $\text{LogCPO} = -4048$. The difference is markedly smaller.

Part II

Prediction

Chapter 6

Fundamentals of Prediction



This chapter provides an overview of prediction with examples taken from quantitative genetics. The first part summarises *best prediction* and *best linear prediction* and offers a brief tour of the standard linear least squares regression. Many important ideas related to prediction can be introduced using the simple least squares setting. The more specific topics on prediction are introduced in Sect. 6.4, where the central question that dominates the remaining of the chapter is posed: suppose that observed data in the form of $(y_1, x_1), (y_2, x_2), \dots, (y_n, x_n)$ are available, where scalars y_i are outcomes or responses and vectors $x_i \in \mathbb{R}^p$ are signals, covariates or features that constitute the feature space. The goal is to construct a predictor that specifies the form of the relationship between the response and the covariate and generates a prediction for a future outcome given an observed x . The construction of the predictor involves estimation of parameters of the model that describes the relationship between y and x . Among the topics discussed is the accuracy with which future outcomes can be predicted, how this accuracy is measured and what are the factors affecting it. A distinction is made between the ability to predict (or to fit) those same observations that were used for estimation of parameters, or to predict new, yet-to-be observed outcomes.

The body of the chapter deals with prediction from a classical/frequentist perspective. Bayesian prediction is illustrated in several examples in the coming chapters and particularly in Chap. 10.

6.1 Best Predictor and Best Linear Predictor

Consider a scalar random variable y that can represent data such as height measurements in humans and a scalar or vector random variable x that can represent explanatory variables or covariates such as genetic marker genotypes. One may wish to establish the form of the association between y and x either to make inferences

of the parameters that describe this association, or to make predictions of a yet-to-be observed y_0 using a predictor $f(x)$, a function of x . One way of deriving the predictor f is by minimising the expected value of the *mean squared error of prediction* over the joint distribution $[y_0, x]$:

$$\begin{aligned} E_{y_0, x} [\text{MSE}(f(x))] &= E_{y_0, x} [(y_0 - f(x))^2] \\ &= E_x \left[E_{y_0|x} [(y_0 - f(x))^2 | x] \right] \\ &= E_x \left[\text{Var}(y_0|x) + (E(y_0|x) - f(x))^2 \right], \end{aligned} \quad (6.1)$$

where the notation $E_{y_0, x}$ means expectation over the joint distribution $[y_0, x]$. Adding and subtracting $E(y_0|x)$ in $(y_0 - f(x))$ in the second line, expanding the square and taking expectations leads to the third line. The first term in the right hand side does not depend on f and therefore (6.1) is minimised when

$$f(x) = E(y_0|x). \quad (6.2)$$

This is the *best predictor*; it minimises $E_{y_0, x} [\text{MSE}(f(x))]$ with respect to both, the joint distribution of (x, y_0) and the conditional distribution of y_0 given x . In Gaussian linear models the conditional distribution of y_0 given x depends on x only through the conditional mean. In the case of other distributions, such as the Bernoulli where independent binary data are drawn with probability $p(x)$, both the conditional mean $E(y_0|x) = p(x)$ and the conditional variance $\text{Var}(y_0|x) = p(x)[1 - p(x)]$ depend on x .

Once y_0 is observed, the error of prediction is

$$e = y_0 - E(y_0|x)$$

that motivates the model for y_0

$$y_0 = E(y_0|x) + e.$$

Therefore, by construction,

$$E(e|x) = E[(y_0 - E(y_0|x) | x)] = 0. \quad (6.3)$$

It does not follow that the distribution of e is independent of x unless it is imposed by assumption.

Using the law of iterated expectations reveals that the unconditional mean is also zero:

$$E(e) = E_x [E(e|x)] = 0. \quad (6.4)$$

The conditional expectation is typically not available and neither is the form of the joint distribution of (x, y_0) . To make progress requires choice of a model. If the choice falls on a linear relationship, whereby the scalar y_0 is predicted using $\alpha + x'\beta$ where α is an intercept, β is the $(p-1) \times 1$ column vector of population coefficients and x is the $(p-1) \times 1$ column vectors of covariates, then, minimising

$$E_{y_0, x} \left[(y_0 - \alpha - \beta'x)^2 \right] \quad (6.5)$$

with respect to α and β yields the population parameters

$$\beta = \text{Var}(x)^{-1} \text{Cov}(y_0, x), \quad \alpha = E(y_0) - \beta' E(x). \quad (6.6)$$

The dimensions are as follows: $\text{Var}(x)$ is $(p-1) \times (p-1)$, $\text{Cov}(y_0, x)$ is $(p-1) \times 1$, β is $(p-1) \times 1$ and $E(x)$ is $(p-1) \times 1$.

The resulting *best linear predictor* is

$$\hat{y}_0 = E(y_0) + \text{Cov}(y_0, x') [\text{Var}(x)]^{-1} (x - E(x)) \quad (6.7)$$

that is the best linear approximation of y_0 given x . Writing

$$y_0 = \hat{y}_0 + \epsilon, \quad (6.8)$$

then the following properties can be derived:

$$E(\epsilon) = E(y_0) - E(\hat{y}_0) = 0, \quad (6.9a)$$

$$\text{Cov}(\epsilon, \hat{y}_0) = \text{Cov}(y_0, \hat{y}_0) - \text{Var}(\hat{y}_0) = 0, \quad (6.9b)$$

$$\text{Var}(\epsilon) = \text{Var}(y_0) - \text{Var}(\hat{y}_0). \quad (6.9c)$$

In (6.9) expectations are taken over $[y_0, x]$. The first equality (6.9a) indicates that the marginal expectation of ϵ over $[y_0, x]$ is zero, but using (6.8) and writing $E(\epsilon|x) = E(y_0|x) - E(\hat{y}_0|x)$, reveals that in contrast to (6.3), the expectation of ϵ given x is not zero in general, unless $E(y_0|x)$ is linear.

The best linear predictor shares properties (6.9) with the best predictor as indicated below in (6.10).

A classical animal breeding setup where some of these concepts are illustrated is in Part III, on pages 627 and 719.

Properties of the Best Predictor

Let $E(y_0|x) = \tilde{y}_0$. Properties of the best predictor are

$$E_x[\tilde{y}_0] = E_x[E(y_0|x)] = E(y_0). \quad (6.10a)$$

$$\text{Cov}_{y_0x} [y_0, \tilde{y}_0] = \text{Var}_x [\tilde{y}_0]. \quad (6.10b)$$

$$\begin{aligned} \text{PEV}(\tilde{y}_0) &= \text{Var}(\tilde{y}_0 - y_0) \\ &= E_x \{ \text{Var}[(\tilde{y}_0 - y_0) | x] \} + \text{Var}_x \{ E[(\tilde{y}_0 - y_0) | x] \} \\ &= E_x [\text{Var}(y_0|x)] \end{aligned} \quad (6.10c)$$

where PEV is the prediction error variance, the variance of the deviation between the predictand and the conditional mean. The corresponding term in (6.9c) is the variance of the deviation between the *predictand* and its best linear approximation.

The top line (6.10a) indicates that the expectation of the predictor is equal to the expectation of the predictand, the classical frequentist definition of unbiasedness for “prediction of random variables”.

The proof of (6.10b) is based on the equality

$$\begin{aligned} \text{Cov}_{y_0x} [y_0, E(y_0|x)] &= E_x [\text{Cov}(y_0, E(y_0|x) | x)] + \text{Cov}_x [E(y_0|x), E(y_0|x)] \\ &= \text{Var}_x [\tilde{y}_0]. \end{aligned}$$

The squared correlation between predictand and predictor is

$$\begin{aligned} r_{y_0\tilde{y}_0}^2 &= \frac{[\text{Cov}_{y_0x}(y_0, \tilde{y}_0)]^2}{\text{Var}(y_0) \text{Var}(\tilde{y}_0)} = \frac{\text{Var}_x(\tilde{y}_0)}{\text{Var}(y_0)} \\ &= 1 - \frac{\text{PEV}(\tilde{y}_0)}{\text{Var}(y_0)}. \end{aligned} \quad (6.11)$$

For simplicity take x to be a scalar. Then

$$\begin{aligned} E_{y_0x}(y_0x) &= E_x [E_{y_0|x}(y_0x|x)] \\ &= E_x [x E_{y_0|x}(y_0|x)] \\ &= E_x(\tilde{y}_0x). \end{aligned}$$

When the equality $E_{y_0x}(y_0x) = E_x(\tilde{y}_0x)$ is written as

$$E_{y_0x} [x(y_0 - \tilde{y}_0)] = 0$$

it takes the form of an orthogonality condition indicating that the prediction errors $(y_0 - \tilde{y}_0)$ are uncorrelated with the covariate x .

Example: Additive Genetic Values as Best Linear Predictors

Consider a continuously distributed trait such as height in humans, $y_i, i = 1, \dots, n$, measured on n individuals. Assume that the trait is genetically determined by q bi-allelic loci coded as $z_{ij}, j = 1, \dots, q$, and that the genotypic codes are centred so that $E(z_{ij}) = 0$ for all individuals and loci. The genetic value of individual i is the deviation between the expected phenotypic value y_i , given z_i and the population mean, where $E(y_i|z_i) = \mu + G(z_i)$, μ is the population mean, $G(z_i)$ is the genetic value and $z'_i = (z_{i1}, \dots, z_{iq})$ is the row vector of genotype codes for individual i . The centred genotype codes are discrete random variables denoted allele contents. The genetic values of each of the q bi-allelic loci can take three modalities corresponding to the three possible genotypes.

The phenotype is assumed to be the result of the additive contribution of a mean, the genetic value and an environmental effect

$$y_i = \mu + G(z_i) + e_i \quad (6.12)$$

where the continuously distributed environmental effect e_i has mean zero and variance σ^2 and marginally, $E(y_i) = \mu$.

The conditional expectation $\mu + G(z_i)$ may not be linear on z_i due to the genetic mechanism operating within and across loci. However, one can always define a linear relationship of the form

$$m + \alpha' z_i. \quad (6.13)$$

Then m and α are obtained minimising

$$E_{y_i z_i} \left[(y_i - m - \alpha' z_i)^2 \right]$$

with respect to m and α . The expected squared error is a minimum with

$$\begin{aligned} \alpha &= [\text{Var}(z_i)]^{-1} \text{Cov}(y_i, z_i) \\ &= [\text{Var}(z_i)]^{-1} \text{Cov}(z_i, G(z_i)) \end{aligned} \quad (6.14)$$

and

$$m = E(y_i) - \alpha' E(z_i). \quad (6.15)$$

The equality in the second line of (6.14) is the result of the following:

$$\begin{aligned} \text{Cov}(y_i, z_i) &= E(y_i z_i) \\ &= E_{z_i} [E(y_i z_i | z_i)] \end{aligned}$$

$$\begin{aligned}
&= E_{z_i} [z_i E(y_i | z_i)] \\
&= E_{z_i} [z_i (\mu + G(z_i))] \\
&= \text{Cov}(z_i, G(z_i)).
\end{aligned}$$

Substituting (6.14) and (6.15) in (6.13) yields the best linear predictor of the phenotypic value

$$\begin{aligned}
\hat{y}_i &= E(y_i) + \text{Cov}(z'_i, G(z_i)) [\text{Var}(z_i)]^{-1} (z_i - E(z_i)) \\
&= \mu + \alpha' z_i
\end{aligned}$$

because $E(z_i) = 0$. The column vector α has q elements representing the multiple regression coefficients of phenotype on allele content. They are the additive genetic effects of the causal loci or effects of gene substitutions for each of the q loci affecting the trait (Falconer and Mackay 1996) and $\alpha' z_i$ is the additive genetic value of individual i , also known as the breeding value, the best linear approximation describing the relationship between genetic value and allele content.

Using a Biased Predictor

Imagine that a biased predictor is used instead of the best predictor $E(y_0|x) = \tilde{y}_0$. It is not unusual that a best predictor cannot be derived and compromises must be sought. Assume that the biased predictor is equal to $b\tilde{y}_0$, for some small positive constant b . How does this affect prediction error variance and the squared correlation between predictand and predictor? The prediction error variance is

$$\text{Var}(y_0 - b\tilde{y}_0) = \text{Var}(y_0) + b^2 \text{Var}(\tilde{y}_0) - 2b \text{Cov}(y_0, \tilde{y}_0).$$

Using $\text{Cov}(y_0, \tilde{y}_0) = \text{Var}(\tilde{y}_0)$ shows that the prediction error variance using the biased predictor increases to

$$\text{Var}(y_0 - b\tilde{y}_0) = \text{PEV}(\tilde{y}_0) + \text{Var}(\tilde{y}_0) (1 - b)^2.$$

The squared correlation between predictand and predictor is not affected.

6.2 Estimating the Regression Function in Practice: Least Squares

The population parameters of the best linear predictor (6.7) are typically estimated using a sample. Anticipating a terminology to be used later in the book, this sample is labelled *training data*. The training data may consist of independent draws $\{y_i, X_i\}_{i=1}^n$ from some distribution, where $X_i \in \mathbb{R}^{p-1}$ is a vector of $p-1$ covariates (it will be convenient here to use capital X_i to denote the vector of covariates). Collecting the responses onto the vector $y = \{y_i\}_{i=1}^n \in \mathbb{R}^n$ and the row vector of covariates X'_i onto the rows of a full rank $n \times (p-1)$ matrix X ,

$$X = \begin{bmatrix} X'_1 \\ X'_2 \\ \vdots \\ X'_n \end{bmatrix} \in \mathbb{R}^{n \times (p-1)},$$

the linear regression model can be written as

$$y = 1\alpha + X\beta + e, \quad e \sim (0, \sigma^2 I), \quad (6.16)$$

where scalar α is an intercept, e is independent of X (here an assumption), 1 is an $n \times 1$ column vector of ones and the scalar σ^2 is the residual variance. Define the column vector of parameters with p elements $b = (\alpha, \beta)$ and the $n \times p$ matrix $Z = (1 \ X)$. With this definitions the linear regression model (6.16) takes the form

$$y = Zb + e. \quad (6.17)$$

The sum of squared errors is

$$\sum_{i=1}^n (y_i - Z'_i b)^2 \quad (6.18)$$

where Z'_i is the i th row of matrix Z . The sample regression coefficient (or least squares estimator) obtained by minimising (6.18) is:

$$\begin{aligned} \hat{b} &= (Z'Z)^{-1} Z'y \\ &= (Z'Z)^{-1} Z' (Zb + e) \\ &= b + (Z'Z)^{-1} Z'e \end{aligned} \quad (6.19)$$

provided the relationship (6.17) holds. From (6.3), e has conditional mean zero and the conditional expectation of $\hat{b}$ is

$$\begin{aligned} E(\hat{b}|Z) &= b + (Z'Z)^{-1} Z' E(e|Z) \\ &= b. \end{aligned}$$

Therefore, unconditionally, $E(\hat{b}) = b$.

A fitted value is defined as

$$\hat{y}_i = Z_i' \hat{b}$$

and the estimated residual is

$$\hat{e}_i = y_i - \hat{y}_i = y_i - Z_i' \hat{b}.$$

Each datum can also be expressed as

$$y_i = Z_i' \hat{b} + \hat{e}_i.$$

The variance of $\hat{b}$ given Z is readily obtained from (6.19) and is given by

$$\begin{aligned} \text{Var}(\hat{b}|Z, \sigma^2) &= (Z'Z)^{-1} Z'Z (Z'Z)^{-1} \sigma^2 \\ &= (Z'Z)^{-1} \sigma^2. \end{aligned} \tag{6.20}$$

To derive the unconditional (over covariates X) variance of the least squares estimator of the regression parameter β of parametrisation (6.16) we look into the structure of the least squares equations

$$(Z'Z) \hat{b} = Z'y. \tag{6.21}$$

Expanding these equations yields

$$\begin{aligned} 1'1\hat{\alpha} + 1'X\hat{\beta} &= 1'y, \\ X'1\hat{\alpha} + X'X\hat{\beta} &= X'y. \end{aligned}$$

Solving for $\hat{\alpha}$ in the first equation and replacing the solution in the second equation results in the system

$$X'(I - P)X\hat{\beta} = X'(I - P)y, \tag{6.22}$$

where the operator P is given by

$$P = 1(1'1)^{-1}1' = \frac{1}{n}11' \quad (6.23)$$

and $I - P$ is symmetric and idempotent. It is easy to verify that the effect of P on the system (6.22) is such that

$$\begin{aligned} X'(I - P)X &= x'x, \\ X'(I - P)y &= x'y, \end{aligned}$$

where

$$x = X - \bar{X}. \quad (6.24)$$

The $n \times (p - 1)$ matrix $\bar{X}$ has the i th generic row equal to

$$(\bar{X}_1, \bar{X}_2, \dots, \bar{X}_{p-1}), \quad \bar{X}_j = \sum_{i=1}^n X_{ij},$$

where $\bar{X}_j$ is the mean of the j th column of X . Therefore the least squares estimator of β is

$$\hat{\beta} = (x'x)^{-1}x'y \quad (6.25)$$

with sampling variance

$$\text{Var}(\hat{\beta}|X, \sigma^2) = (x'x)^{-1}\sigma^2. \quad (6.26)$$

With $\hat{\beta}$ available, the estimator of α is

$$\hat{\alpha} = \bar{y} - \sum_{i=1}^p \bar{X}_i \hat{\beta}_i. \quad (6.27)$$

Expressions (6.25) and (6.27) yield, of course, the same estimates as those derived from (6.21).

The marginal (with respect to X) asymptotic variance of $\hat{\beta}$ is obtained as follows. Write (6.26) as

$$\text{Var}(\hat{\beta}|X, \sigma^2) = \frac{\sigma^2}{n} \left(\frac{1}{n}x'x \right)^{-1}. \quad (6.28)$$

The i th diagonal element of $\frac{1}{n}x'x$ is

$$\frac{1}{n} \left[(X_{1i} - \bar{X}_i)^2 + (X_{2i} - \bar{X}_i)^2 + \cdots + (X_{ni} - \bar{X}_i)^2 \right].$$

The element in row i and column j of $\frac{1}{n}x'x$ is

$$\frac{1}{n} \left[(X_{1i} - \bar{X}_i)(X_{1j} - \bar{X}_j) + (X_{2i} - \bar{X}_i)(X_{2j} - \bar{X}_j) + \cdots + (X_{ni} - \bar{X}_i)(X_{nj} - \bar{X}_j) \right].$$

These are sampling variances of the i th covariate and sampling covariances between covariates i and j , respectively. As n increases towards infinity, these sample moments converge to the true variances and covariances and

$$\frac{1}{n}x'x \rightarrow V$$

the true variance-covariance matrix of X . Therefore the marginal unconditional variance of $\hat{\beta}$ is

$$\begin{aligned} \text{Var}(\hat{\beta}|\sigma^2) &= E_X \left[\text{Var}(\hat{\beta}|X, \sigma^2) \right] + \text{Var}_X \left[E(\hat{\beta}|X, \sigma^2) \right] \\ &= E_X \left[\text{Var}(\hat{\beta}|X, \sigma^2) \right] \\ &= \frac{\sigma^2}{n} E \left\{ \left[\frac{1}{n}x'x \right]^{-1} \right\} \\ &\rightarrow \frac{\sigma^2}{n} V^{-1}. \end{aligned} \tag{6.29}$$

The variance of the regression estimator decreases as sample size n increases, increases when the linear model $E(y|x)$ accounts for a relatively smaller proportion of the marginal variance of y (σ^2 increases), decreases with increasing variance of X and when the covariates are less correlated. The latter point can be easily illustrated for the case of two covariates. Let the asymptotic variance of X be equal to

$$V = \begin{bmatrix} V_{11} & V_{12} \\ V_{12} & V_{22} \end{bmatrix}.$$

If the two covariables are independent, $V_{12} = 0$ and the variance of the estimator of the first regression is proportional to $(V_{11})^{-1}$. When the covariates are not independent, $V_{12} \neq 0$ and the variance of the estimator of the first regression is proportional to V_{22}/Δ where

$$V^{-1} = \frac{1}{\Delta} \begin{bmatrix} V_{22} & -V_{12} \\ -V_{12} & V_{11} \end{bmatrix}.$$

The determinant $\Delta = V_{11}V_{22} - (V_{12})^2 > 0$ because V is positive definite. This implies that $V_{11}V_{22} > \Delta$. Dividing both sides by Δ and multiplying by $(V_{11})^{-1}$ reveals that

$$\frac{V_{22}}{\Delta} > (V_{11})^{-1},$$

indicating that when the covariables are not independent the uncertainty increases. When $V_{12} = 0$, $V_{22}/\Delta = (V_{11})^{-1}$.

The conditional variance of y_i in the linear model specified in (6.17), given Z , is

$$\begin{aligned}\text{Var}(y_i|Z_i) &= \mathbb{E} \left[(y_i - \mathbb{E}(y_i|Z_i))^2 \right] \\ &= \mathbb{E} \left(e_i^2 | Z_i \right)\end{aligned}$$

that by assumption, is independent of Z and therefore $\text{Var}(y_i|Z_i) = \mathbb{E}(e_i^2|Z_i) = \sigma^2$, for all i . This defines an homoscedastic error. In a heteroscedastic model the conditional variance depends on the covariate X .

A more general approach to study factors affecting the conditional variance is to specify a model of the form

$$\begin{aligned}y_i|X_i &\sim N \left(X_i' \beta, \sigma^2(X_i) \right), \\ \ln \left(\sigma^2(X_i) \right) &= X_i' \beta^*\end{aligned}$$

where the logerror variance is modelled explicitly as in San Cristobal-Gaudy et al. (1998). When $\ln(\sigma^2(X_i)) = \beta^*$, a parameter in $\mathbb{R}$, the model reduces to its homoscedastic form (6.16). This is a fruitful area of research in quantitative and evolutionary genetics that is briefly revisited in Chap. 10 on page 499.

The unconditional error variance is by construction, independent of X . Indeed, the unconditional variance is

$$\mathbb{E}(e_i^2) = \mathbb{E}_{X_i} \left[\mathbb{E}(e_i^2|X_i) \right],$$

the average conditional variance.

Least Squares Linear Regression and the ‘Hat’ Matrix

The fitted value of y (a column vector with n elements) is

$$\begin{aligned}\hat{y} &= x\hat{\beta} \\ &= x(x'x)^{-1}x'y \\ &= Hy.\end{aligned}\tag{6.30}$$

In this subsection the linear model is specified using the notation $y = x\beta + e$, $e \sim (0, I\sigma^2)$. In (6.30) the $n \times n$ idempotent, symmetric, positive semidefinite matrix $H = x(x'x)^{-1}x'$ is known as the “hat” matrix (an idempotent matrix is a matrix which, when multiplied by itself, yields itself) and matrix x , whose first column is a vector of 1s to accommodate the constant term, is of dimension $n \times p$, where $p - 1$ is the number of covariates. Since H is positive semidefinite, its i th diagonal element $h_{ii} \geq 0$. Other properties of H are

1. $Hx = x$
2. Partition x as $x = (x_1 \ x_2)$. Then $Hx_1 = x_1$
3. If matrix x contains a constant term, $H1 = 1$, where 1 is a column vector of 1's.
It follows that $\sum_{j=1}^n h_{ij} = \sum_{i=1}^n h_{ij} = 1$
4. $h_{ii} = \sum_{j=1}^n h_{ij}^2 = h_{ii}^2 + \sum_{j \neq i} h_{ij}^2$
5. If matrix x contains a constant term, $n^{-1} \leq h_{ii} \leq 1$ and in general, $0 \leq h_{ii} \leq 1$

Writing

$$y = x\hat{\beta} + \hat{e} = \hat{y} + \hat{e}, \quad (6.31)$$

then

$$x'\hat{e} = x'(y - \hat{y}) = x'(y - Hy) = x'y - x'x(x'x)^{-1}x'y = 0$$

and therefore, the estimated residuals $\hat{e}$ are orthogonal to the covariable x .

The estimated residuals are also orthogonal to the predicted values. Indeed,

$$\hat{e}'\hat{y} = ((I - H)y)'Hy = y'Hy - y'HHy = 0,$$

because H is idempotent and therefore $HH = H$. The estimated residuals $\hat{e} = y - \hat{y}$ can be generated using

$$\begin{aligned} \hat{e} &= (I - H)y \\ &= (I - H)(x\beta + e) \\ &= (I - H)e, \end{aligned} \quad (6.32)$$

because $Hx = x$. For the homoscedastic model the variance of the vector of estimated residuals is

$$\text{Var}(\hat{e}|x) = \text{Var}((I - H)e|x) = \sigma^2(I - H)$$

because $I - H$ is idempotent, indicating that estimated residuals are heteroscedastic and correlated despite homoscedasticity of the uncorrelated errors e . In particular for record i ,

$$\text{Var}(\hat{e}_i|x) = \sigma^2 (1 - h_{ii}) \quad (6.33)$$

where h_{ii} is the i th diagonal element of H .

An estimator of the conditional variance given x is

$$\begin{aligned} \hat{\sigma}^2 &= \frac{1}{n} \hat{e}' \hat{e} \\ &= \frac{1}{n} y' (I - H) (I - H) y \\ &= \frac{1}{n} y' (I - H) y, \end{aligned}$$

because $(I - H)$ is idempotent. This is a method of moments estimator equal to the maximum likelihood estimator in the normal regression model. Using $Hx = x$ and the formula for the expectation of a quadratic form (see Note on page 141), one can show that the expected value of the estimator is

$$E(\hat{\sigma}^2) = \frac{n - p}{n} \sigma^2,$$

indicating that there is a downward bias. The usual unbiased estimator is $\frac{1}{n-p} \hat{e}' \hat{e}$.

Using the hat matrix, it is easy to show that

$$E(\hat{y}|x) = E(y|x)$$

and that

$$\text{Var}(\hat{y}|x) = \text{Var}(Hy|x) = H \text{Var}(y|x) H = \sigma^2 H. \quad (6.34)$$

Also

$$\text{Cov}(y, \hat{y}|x) = \text{Cov}(y, y'H|x) = \sigma^2 H. \quad (6.35)$$

For the i th record

$$\begin{aligned} \text{Cov}(y_i, \hat{y}_i|x) &= \text{Cov}\left(y_i, \sum_{j=1}^n h_{ij} y_j|x\right) \\ &= \text{Cov}(y_i, h_{ii} y_i|x) = h_{ii} \sigma^2, \end{aligned} \quad (6.36)$$

where h_{ij} is the element in row i and column j of the hat matrix H , known as the leverage. The equality in the second line follows because $\text{Cov}(y_i, h_{ij}y_j|x) = 0$ for $i \neq j$. This covariance can be interpreted as the influence that a datum has on its own prediction. It plays an important role in understanding the relationship between overfitting and the model's prediction ability and is related to the concept of effective number of parameters of a model (Hastie et al. 2009).

Also,

$$\text{Var}(\hat{y}_i|x) = h_{ii}\sigma^2. \quad (6.37)$$

Then it follows that, conditional on x ,

$$r_{y_i, \hat{y}_i}^2 = \frac{[\text{Cov}(y_i, \hat{y}_i|x)]^2}{\text{Var}(y_i|x) \text{Var}(\hat{y}_i|x)} = \frac{\text{Var}(\hat{y}_i|x)}{\text{Var}(y_i|x)} = h_{ii}, \quad (6.38)$$

indicating the two related interpretations of $r_{y_i, \hat{y}_i}^2$: (1) as the proportion of the variance in y explained by the linear predictor; (2) the squared correlation between predictor $\hat{y}_i$ and predictand y_i . The ratio of variances in the second equality emphasises that $r_{y_i, \hat{y}_i}^2$ is a measure of how much the full model (6.16) improves on the reduced model that only includes an intercept, and not on how good the full model is in an absolute sense. It also indicates that the full and reduced models are nested.

The correlation can be regarded as a parameter of the joint (bivariate) distribution of fitted values $\hat{y}_i$ and the data y_i used to obtain the fitted values (the training data). Indeed, assuming normality

$$\begin{pmatrix} \hat{y}_i \\ y_i \end{pmatrix} | x \sim N \left[\begin{pmatrix} x_i' \beta \\ x_i' \beta \end{pmatrix}, \begin{pmatrix} h_{ii}\sigma^2 & h_{ii}\sigma^2 \\ h_{ii}\sigma^2 & \sigma^2 \end{pmatrix} \right] \quad (6.39)$$

with correlation between $\hat{y}_i$ and y_i equal to $\sqrt{h_{ii}}$ and with regression of $\hat{y}_i$ on y_i equal to h_{ii} .

With n records the average squared correlation in a model with intercept is

$$\frac{\sum_{i=1}^n h_{ii}}{n} = \frac{\text{tr}(H)}{n} = \frac{p}{n},$$

indicating that the regression and the squared correlation increase towards 1 as p increases towards n reflecting overfitting. The proportion of variance explained of the training records or squared correlation between $\hat{y}_i$ and y_i depends only on the structure of the covariates for a particular data set and is a poor reflection of the model's ability to predict records from an independent set of data.

The sum of squared deviations between the observations and their fitted values (the residual sum of squares) is

$$\begin{aligned}
 \sum_{i=1}^n (y_i - \hat{y}_i)^2 &= (y - \hat{y})' (y - \hat{y}) \\
 &= y' (I - H) y \\
 &= \sum_{i=1}^n y_i^2 (1 - h_{ii}) - \sum_{i \neq j} y_i y_j h_{ij}.
 \end{aligned}$$

As p increases towards n , h_{ii} increases towards 1, and h_{ij} , ($i \neq j$) and the residual sum of squares decreases towards 0.

The estimator (6.19) was obtained by minimising the residual sum of squares (6.18), here with $f(x_i) = x_i' \beta$, a linear function of the parameters. In principle any arbitrary function passing through the data points (x_i, y_i) may represent a solution to the minimisation problem. However, we shall see that such solutions yielding perfect or nearly perfect fits obtained using a given sample of data, may be poor predictors of data points from a new sample of data.

Prediction of Out-of-sample Values

Given a value of the covariate vector x_0 one may wish to obtain a prediction of a yet-to-be-observed scalar y_0 (the *validating data*). The model for y_0 is

$$y_0 = x_0' \beta + e_0,$$

where e_0 is homoscedastic with known variance σ^2 and assumed to be independent of x_0 . This new datum is assumed to be independent of the data used to estimate β , the vector of *training data* and drawn from the same distribution. The model for the training data is $y = x\beta + e$. The classical approach is to use an estimate of the conditional mean as predictor

$$\hat{y}_0 = E(y_0 | x_0, \beta = \hat{\beta}) = x_0' \hat{\beta} \quad (6.40)$$

where $\hat{\beta}$ is estimated using the training data. This is a point prediction of the average value of yet-to-be-observed data evaluated at x_0 . It does not account for any form for uncertainty, but can be applied in an initial stage to compare predictors derived from different models. Conditional on training and validating data the predictor is

a point mass at $x'_0 \hat{\beta}$. The variance of the predictor over conceptual replications of training data is (assuming x is of full column rank)

$$\begin{aligned}\text{Var}(\hat{y}_0|x_0) &= \text{Var}_{\hat{\beta}} \left[\mathbb{E}(\hat{y}_0|x_0, \hat{\beta}) \right] \\ &= x'_0 (x'x)^{-1} x_0 \sigma^2.\end{aligned}$$

Averaging over conceptual replications of validating data as well, the variance of the error of prediction is

$$\begin{aligned}\text{Var}[(y_0 - \hat{y}_0) | x_0] &= \text{Var}(y_0|x_0) + \text{Var}(\hat{y}_0|x_0) \\ &= \sigma^2 \left[1 + x'_0 (x'x)^{-1} x_0 \right].\end{aligned}\quad (6.41)$$

The covariance term cancels due to the independence y_0 and $\hat{y}_0$. This expression has an extra term compared to (6.10c) accounting for the use of an estimate of the conditional expectation.

A Bayesian interpretation of the predictor $\hat{y}_0$ can be justified along the following lines. Consider the Bayesian model where the training (y) and validating (y_0) data are drawn from the sampling model

$$\begin{bmatrix} y \\ y_0 \end{bmatrix} \Big| x, x_0, \beta, \sigma^2 \sim N \left[\begin{pmatrix} x\beta \\ x'_0\beta \end{pmatrix}, \begin{pmatrix} I & 0 \\ 0 & 1 \end{pmatrix} \sigma^2 \right].$$

Assign an improper uniform prior distribution to β and let σ^2 be known. Assume that β is inferred from the training data y . Then the posterior distribution of β can be shown to take the Gaussian form (assuming x is of full column rank)

$$\beta|y, x, \sigma^2 \sim N \left(\hat{\beta}, (x'x)^{-1} \sigma^2 \right), \quad \hat{\beta} = (x'x)^{-1} x'y. \quad (6.42)$$

A predictor y_0^* of the validating datum y_0 is obtained drawing β^* from (6.42), and given β^* , drawing y_0^* from $[y_0|x_0, \beta^*, \sigma^2]$. Then the marginal (with respect to β) posterior predictive distribution of y_0^* can be shown to be normal with mean

$$\begin{aligned}\mathbb{E}(y_0^*|y, x, x_0, \sigma^2) &= \mathbb{E}_{\beta^*|y, x, x_0, \sigma^2} \left[\mathbb{E}(y_0^*|y, x_0, \sigma^2, \beta^*) \right] \\ &= \mathbb{E}_{\beta^*|y, x, x_0, \sigma^2} (x_0 \beta^*) = x'_0 \hat{\beta},\end{aligned}\quad (6.43)$$

and variance

$$\begin{aligned}\text{Var}(y_0^*|y, x, x_0, \sigma^2) &= \mathbb{E}_{\beta^*|y, x, x_0, \sigma^2} \left[\text{Var}(y_0^*|y, x_0, \sigma^2, \beta^*) \right] \\ &\quad + \text{Var}_{\beta^*|y, x, x_0, \sigma^2} \left[\mathbb{E}(y_0^*|y, x_0, \sigma^2, \beta^*) \right] \\ &= \sigma^2 + x'_0 (x'x)^{-1} x_0 \sigma^2.\end{aligned}\quad (6.44)$$

The mean (6.43) and variance (6.44) are mathematically identical to, respectively, (6.40) and (6.41) derived using the classical approach. The first term in (6.44) is the sampling variance (here, assumed known) of the draws y_0^* and the second term reflects uncertainty of β , inferred from its posterior distribution, that propagates on to the distribution of the predictor y_0^* .

A Justification for the Choice of a Linear Relationship

The conditional expectation was approximated using the linear function $x\beta$. While this may appear arbitrary, some justification can be found in the following argument. If f is a smooth function, a first order Taylor expansion about a chosen point x^* results in

$$f(x) = f(x^*) + \sum_{i=1}^p \left(\frac{\partial f}{\partial x_i} \Big|_{x=x^*} \right) (x_i - x_i^*) + \text{higher order terms in } (x_i - x_i^*),$$

where the partial derivatives of the function become the regression coefficients. If x^* is close to x , higher order terms are small and the linear approximation is (locally) acceptable.

Some Caveats of the Linear Regression Model

Several issues arise when the linear regression model $y = x\beta + e$ is a poor description of the model that generated the data.

- The first issue is concerned with the dependence of the population regression coefficients on the distribution of the input variables x . Specifically

$$\begin{aligned} & \text{Var}(x)^{-1} \text{Cov}(x, y) \\ &= \text{Var}(x)^{-1} \text{Cov}(x, x'\beta + e) \\ &= \text{Var}(x)^{-1} (\text{Var}(x) \beta + \text{Cov}(x, e)) \\ &= \beta, \end{aligned}$$

if the true model is linear and the error terms are uncorrelated with x . If this is not the case the coefficient β is sensitive to changes in the input variable x .

- Another potential problem arises when the regression model omits variables that are part of the true model. If one postulates

$$y = x\beta + e \tag{6.45}$$

where e has zero mean and is independent of x , when the true model is

$$y = x\beta + z\gamma + \varepsilon \quad (6.46)$$

where ε has zero mean and is independent of x and z , then the residual of the postulated model is $e = z\gamma + \varepsilon$. This residual may depend on x if z and x are associated. A consequence is that the estimator is biased with respect to β . Indeed,

$$\begin{aligned} \hat{\beta} &= (x'x)^{-1} x'y \\ &= (x'x)^{-1} x'(x\beta + z\gamma + \varepsilon) \end{aligned}$$

and

$$\begin{aligned} E(\hat{\beta}|x, z) &= \beta + (x'x)^{-1} x'z\gamma + E(\varepsilon|x, z) \\ &= \beta + (x'x)^{-1} x'z\gamma \end{aligned}$$

which is biased unless $x'z = 0$.

- On the other hand, if γ in (6.46) is a random variable with zero mean so that $E(y|x, z) = x\beta$ and the fitted model is (6.45) (that does not include the random variable γ), then $E(\hat{\beta}|x, z) = (x'x)^{-1} x'E(y|x, z) = \beta$, an unbiased estimator. The variance of this estimator is larger than the variance of the correct, generalised least squares estimator (the BLUE, best linear unbiased estimator).
- Adding superfluous variables to a linear regression model does not introduce bias but increases the variance of estimation. Specifically, consider now that the true model is defined in (6.45), but we fit

$$y = x\beta_1 + z\beta_2 + \varepsilon. \quad (6.47)$$

Above, z is redundant when $\beta_2 = 0$. I sketch the proof that including z in the model does not bias the estimator of β_1 , which has the same form as the estimator of β based on the true model (6.45). The least squares equations based on (6.47) are

$$\begin{bmatrix} x'x & x'z \\ z'x & z'z \end{bmatrix} \begin{bmatrix} \hat{\beta}_1 \\ \hat{\beta}_2 \end{bmatrix} = \begin{bmatrix} x'y \\ z'y \end{bmatrix}. \quad (6.48)$$

Carrying out the matrix multiplication yields after simple algebra

$$\hat{\beta}_1 = (x'x)^{-1} x'(y - z\hat{\beta}_2). \quad (6.49)$$

A little more effort (involving substituting $\hat{\beta}_1$ in terms of $\hat{\beta}_2$) leads to

$$\hat{\beta}_2 = [z'(I - H)z]^{-1} z'(I - H)y \quad (6.50)$$

where the hat matrix defined in (6.30) is

$$H = x(x'x)^{-1}x'.$$

Taking expectation under the true model in (6.45) yields as the $E(\hat{\beta}_2|x)$,

$$\begin{aligned} E\left\{[z'(I - H)z]^{-1} z'(I - H)y|x\right\} &= [z'(I - H)z]^{-1} z'(I - H)x\beta \\ &= [z'(I - H)z]^{-1} z'(x - x(x'x)^{-1}x')\beta = 0. \end{aligned}$$

Therefore, the expectation of $\hat{\beta}_1$ in (6.49) is

$$E\left[(x'x)^{-1}x'(y - z\hat{\beta}_2)|x\right] = (x'x)^{-1}x'x\beta = \beta, \quad (6.51)$$

showing that $\hat{\beta}_1$ is an unbiased estimator.

The sampling variance of $\hat{\beta}_1$ under (6.47) is proportional to the upper left hand block of the inverse of the coefficient matrix constructed on the basis of (6.47). Using results from partitioned matrices it is easy to show that this variance is larger than the variance under the true model (6.45) (which is proportional to $(x'x)^{-1}$), unless $x'z = 0$.

- Finally, consider fitting the operational model to data y (a vector with n elements),

$$y = z\gamma + \epsilon \quad (6.52)$$

where the regression parameter $\gamma = 0$ (a scalar), when the true model, for scalar β is

$$y = x\beta + e, \quad E(y|x) = x\beta.$$

Assume that ϵ has mean 0 and is independent of z and assume e is independent of z . The least squares estimator of γ is

$$\hat{\gamma} = (z'z)^{-1} z'y = \left(\frac{1}{n}z'z\right)^{-1} \frac{1}{n}z'y.$$

With centred covariates and data, as n increases, $n^{-1}z'z \rightarrow \text{Var}(z) = \sigma_z^2$, $n^{-1}z'y \rightarrow \beta\sigma_{xz}$, $\sigma_{xz} = \text{Cov}(x, z)$ and the least squares estimator converges to

$$\hat{\gamma} \rightarrow \frac{\sigma_{xz}}{\sigma_z^2} \beta. \quad (6.53)$$

This situation mimics a GWAS study, where the effects of multiple marker loci are studied by fitting one marker at a time. In this case, the causal locus is x and z is a marker genotype that is correlated with x ($\sigma_{xz} \neq 0$; the marker is in linkage disequilibrium with x and does not have a direct effect on the dependent variable y ; it is not a causal locus). The consequence of the single marker regression is that neutral markers that are correlated with causal loci, may be assigned phantom significant p -values, a problem that is aggravated with sample size (large sample increases the power of detection). This issue is briefly revisited on page 395 in connection with the computation of false discovery rates.

Least Squares Prediction as an Approximation to Best Linear Prediction

Consider the problem of predicting the scalar random variable y_0 from random scalars $X_1, X_2, \dots, X_p$ and assume that y_0 and the X 's have finite mean and finite non-zero variance. A linear function $\alpha + \beta'X$ predicts y_0 with mean squared error

$$\text{E} \left[(y_0 - \alpha - \beta'X)^2 \right].$$

This is minimised with

$$\begin{aligned} \alpha &= \text{E}(y_0) - \beta' \text{E}(X), \\ \beta &= [\text{Var}(X)]^{-1} \text{Cov}(X, y_0). \end{aligned}$$

The best linear predictor is

$$\hat{y}_0 = \text{E}(y_0) + \beta'(X - \text{E}(X)). \quad (6.54)$$

Let $(y_1, X_1), (y_2, X_2), \dots, (y_n, X_n)$ be an *iid* sequence of random vectors, where y_i are scalars and $X_i \in \mathbb{R}^p$, $i = 1, 2, \dots, n$. If one postulates the linear model

$$y_i = \alpha + X_i' \beta + e_i$$

where α is a scalar intercept, X'_i is the i th row of the $n \times p$ full rank matrix X , then the least squares estimators of α and β are the solution to

$$\begin{bmatrix} n & 1'X \\ X'1 & X'X \end{bmatrix} \begin{bmatrix} \hat{\alpha} \\ \hat{\beta} \end{bmatrix} = \begin{bmatrix} 1'y \\ X'y \end{bmatrix}.$$

This yields

$$\hat{\alpha} = \bar{y} - \sum_{i=1}^p \bar{X}_i \hat{\beta}_i \quad (6.55a)$$

$$\begin{aligned} \hat{\beta} &= (x'x)^{-1} x'y \\ &= \left(\frac{1}{n} x'x \right)^{-1} \frac{1}{n} x'y. \end{aligned} \quad (6.55b)$$

In (6.55), $\bar{X}_i = \frac{1}{n} \sum_{j=1}^n X_{ij}$, $x = X - \bar{X}$, and the $n \times p$ matrix $\bar{X}$ has the i th generic row equal to $(\bar{X}_1, \bar{X}_2, \dots, \bar{X}_p)$. As $n \rightarrow \infty$, $\bar{y}$ approaches $E(y)$, $\bar{X}_i$ approaches $E(X_i)$, $(n^{-1}x'x)$ approaches $\text{Var}(X)$, $(n^{-1}x'y)$ approaches $\text{Cov}(y, X)$ and the least squares predictor $\hat{y}_0 = \hat{\alpha} + X'_0 \hat{\beta}$ approaches the best linear predictor (6.54) irrespective of the true relationship between y and X .

Note

The j th row of $x'y$ is

$$\sum_{i=1}^n (X_{ij} - \bar{X}_j) y_i = \sum_{i=1}^n (X_{ij} - \bar{X}_j) (y_i - \bar{y}).$$

Appealing to asymptotics, as $n \rightarrow \infty$

$$\frac{1}{n} \sum_{i=1}^n (X_{ij} - \bar{X}_j) (y_i - \bar{y}) \rightarrow \text{Cov}(y, X_j).$$

The Linear Regression as a Linear Smoother

Consider the case where x_i is a scalar and assume the regression through the origin model

$$y_i = \beta x_i + e_i.$$

Then

$$\hat{\beta} = \frac{\sum_{i=1}^N x_i y_i}{\sum_{i=1}^N x_i^2}$$

and

$$\begin{aligned} \hat{f}(x_i) &= x_i \hat{\beta} \\ &= x_i \frac{\sum_{j=1}^N x_j y_j}{\sum_{j=1}^N x_j^2} \\ &= \sum_{j=1}^N h_{ij} y_j, \end{aligned} \tag{6.56}$$

where h_{ij} is the element in the i th row and j th column of the hat matrix H . Expression (6.56) indicates that the prediction based on least squares, $\hat{f}(x_i)$, is a smoothing of the data whereby the prediction is a weighted average of the output variables y_j with weights proportional to x_j .

The influence that output y_j has on the smoother's fitted value $\hat{f}(x_i)$ is h_{ij} . In general for a linear smoother, the influence that each data point y_i has on its own predicted value $\hat{y}_i$ is equal to $h_{ii}\sigma^2$. For the case of this regression through the origin model $h_{ii} = x_i^2 / \sum_i x_i^2$, $i = 1, 2, \dots, N$.

To summarise, the general form for a linear smoother or a linear fitting method is given by

$$\hat{f}(x) = Hy, \tag{6.57}$$

where the $n \times n$ hat matrix H depends on x but not on y .

6.3 Overview of Things to Come

In this chapter models are regarded as prediction machines as opposed to descriptions of the real world. Having collected data and obtained a predictor, one may wish to know how accurate it is for predicting future observations. The estimation of parameters used to construct the predictor is accomplished using *training data* consisting of N *iid* observations $\{y_i, x_i\}_{i=1}^N$ drawn from some joint distribution. The conditional mean and variance of the distribution of a training datum $[y_i|x_i]$ are $f(x_i)$ and σ^2 , respectively. Clearly, the predictor is a function of the training data (y, x) ; its quality can be studied in a second stage using the *validating* or *testing data* $\{y_{v,i}, x_i\}_{i=1}^N$. The validating data are an independent sample from the same distribution that generated the training data (here training and validating data are of the same length N but need not be and the subscript v is omitted in the covariate

x_i). Let $\hat{f}(x_i)$ be the predictor of the i th data point evaluated at x_i , $i = 1, 2, \dots, N$, obtained using the training data y . How good is $\hat{f}(x_i)$ to predict the validating datum $y_{v,i}$? A common measure of prediction ability is the *expected validating squared error* of prediction, which evaluated at x_i is given by

$$E[\text{MSE}_v(x_i)] = E_{y_{v,y}} \left((y_{v,i} - \hat{f}(x_i)) | x_i \right)^2. \quad (6.58)$$

(Expectations are conditional on covariables x but this is not mandatory). It will be shown that when the expectation is taken over both training and validating data, this measure of prediction ability of a future record has three terms:

$$E[\text{MSE}_v(x_i)] = \text{Var}(y_{v,i} | x_i) + \text{Var}(\hat{f}(x_i) | x_i) + \text{bias}^2, \quad (6.59)$$

where the squared bias is defined as $\left(E_{y_v}(y_{v,i} | x_i) - E_y(\hat{f}(x_i)) \right)^2$. The first term, $\text{Var}(y_{v,i} | x_i)$, is the conditional variance of a future record, also labelled irreducible variance since it reflects sampling variation beyond the control of the experimenter. The second term $\text{Var}(\hat{f}(x_i) | x_i)$ is the variance of the predictor. It describes sampling uncertainty associated with estimation of $\hat{f}$ due to finite size of the training data. The bias term reflects average systematic discrepancy between the predictor and the predictand. Increasing the number of parameters of a model tends to reduce bias and to increase the variance of the predictor. The choice of a prediction model often involves a trade-off involving these two terms.

Expression (6.59) cannot be obtained directly because parameters are not known. In practice the sample *mean squared error* is computed. For a particular realisation of validating data, the sample validating *mean squared error* is

$$\text{MSE}_v = \frac{1}{N} \sum_{i=1}^N (y_{v,i} - \hat{f}(x_i))^2, \quad (6.60)$$

where $\hat{f}$ is the predictor calculated using the training data. The *expected mean squared error* in the validating data is

$$E_{y_{v,y}}(\text{MSE}_v), \quad (6.61)$$

with a corresponding expression for the *expected mean squared error* in the training data

$$E_y(\text{MSE}_t) = E_y \left[\frac{1}{N} \sum_{i=1}^N (y_i - \hat{f}(x_i) | x_i)^2 \right]. \quad (6.62)$$

The expectation in (6.61) is taken over conceptual replications of training and validating data and in (6.62) over the training data. Monte Carlo estimates of these expectations can be obtained using the bootstrap or simulating the data, given a parametric model. Monte Carlo methods generate estimates of the complete sampling distribution of MSE. An example is discussed on page 485.

An important theme of this chapter is the distinction between the predictive ability of $\hat{f}(x_i)$ quantified using either the training data used to estimate $\hat{f}(x_i)$, or the validating data. It will be shown that the former overestimates predictive performance in the validating data, on average, as reflected in the expectations (6.61) and (6.62). These are given by

$$E_{y_v y}(\text{MSE}_v) = \frac{1}{N} \sum_i \text{Var}(y_{v,i} | x_i) + \frac{1}{N} \sum_i \text{bias}(i)^2 + \frac{1}{N} \sum_i \text{Var}(\hat{f}(x_i) | x_i)$$

and when covariates in training and validating data take the same values,

$$E_{y_v y}(\text{MSE}_t) = E_{y_v y}(\text{MSE}_v) - \frac{2}{N} \sum_i \text{Cov}(y_i, \hat{f}(x_i))$$

The covariance term in the second equation was introduced in (6.36) and reflects the influence that observation i has on its own prediction. It is the amount by which, on average, the training mean squared error underestimates the validating mean squared error and is labelled *expected optimism* by Efron (1986). The following section elaborates on these ideas.

6.4 The Bias-variance Trade-off

Consider two random variables (Y, X) that have a joint distribution and suppose that one wishes to predict Y from X using a function evaluated at $X = x$, $f(x)$. This unknown function specifies the true association between Y and X and is estimated using N training observations giving rise to the predictor $\hat{f}$. The N training observations are realisations from the joint distribution $[Y, X]$. The model for datum i in the training data is

$$y_i = f(x_i) + e_i, \quad e_i \stackrel{iid}{\sim} (0, \sigma^2), \quad i = 1, 2, \dots, N,$$

where residuals e have mean zero and variance σ^2 independent of x . The assumption of normality at this stage is not required. According to the model, $E(y_i | x_i) = f(x_i)$.

Consider another sample (y_0, x_0) drawn independently from the same joint distribution $[Y, X]$. This constitutes the validating data. How well does the function $\hat{f}$ constructed using the training data $(y_1, x_1), \dots, (y_N, x_N)$ predict the new test

point y_0 at the input point x_0 ? The predicted value is $\hat{y}_0 = \hat{f}(x_0)$. For a particular datum and its prediction, the *validating mean squared error* is

$$\text{MSE}_v = (y_0 - \hat{y}_0)^2 = y_0^2 + \hat{y}_0^2 - 2y_0\hat{y}_0. \quad (6.63)$$

MSE_v is a point mass and is a common measure of prediction ability. Alternatively, one may be interested in incorporating different sources of uncertainty into the mean squared error of prediction. The expected validating mean squared error is studied below in three scenarios that account for the sampling uncertainty of the predictor and/or for the sampling uncertainty of new records. The measures of prediction ability are evaluated at $X = x_0$ and the expected validating mean squared error is denoted $E[\text{MSE}_v(x_0)]$.

Training Data Treated as Fixed and Validating Data as Random

The first scenario accounts for variation of single records; expectations are taken over the distribution of validating data, conditional on the training data y . Therefore, the predictor $\hat{y}_0$ is regarded as a fixed constant. In this case

$$\begin{aligned} E_{y_0}[\text{MSE}_v(x_0)] &= E(y_0^2) + \hat{y}_0^2 - 2\hat{y}_0 E(y_0) \\ &= \text{Var}(y_0) + (E(y_0))^2 + \hat{y}_0^2 - 2\hat{y}_0 E(y_0) \\ &= \text{Var}(y_0) + (E(y_0) - \hat{y}_0)^2. \end{aligned} \quad (6.64)$$

The first term accounts for the conditional sampling variance (given covariates) of new validating data and the second reflects the squared discrepancy between the predictor $\hat{y}_0$ and the mean of the distribution of validating data. The squared discrepancy term depends on the particular realisation of the training data used to obtain $\hat{y}_0$. This second term would be equal to $\text{Var}(\hat{y}_0)$ if training data were treated as random, when $E(y_0) = E(\hat{y}_0)$.

The focus is on the quality of a predictor, measured by $E(\text{MSE}_v)$, computed from a single sample of training data, measured over conceptual replications of validating data. Single observations are predicted accounting for their sampling uncertainty, but ignoring sampling uncertainty of the parameters that index the predictor.

Training Data Treated as Random and Validating Data as Fixed

The second scenario accounts for variation of the predictor and the mean validating squared error is averaged over conceptual replications of training data, keeping

validating data fixed. Using similar algebra leads to

$$E_y [\text{MSE}_v (x_0)] = \text{Var} (\hat{y}_0) + (E (\hat{y}_0) - y_0)^2 \quad (6.65)$$

indicating that there is a component from the sampling variation of the predictor over conceptual replications of training data and a squared discrepancy between the validating data y_0 and the mean of the predictor (averaged over training data). The second term would be equal to $\text{Var}(y_0)$ if validating data were treated as random, when $E(\hat{y}_0) = E(y_0)$.

The focus here is on the quality of a predictor, computed repeatedly from conceptual replications of training data, measured in a single sample of validating data. Average values of new observations, all evaluated at $X = x_0$ are predicted, accounting for sampling variation of the parameters that index the predictor.

Training and Validating Data Are Treated as Random

The third scenario accounts for variation of single records and of the predictor and the average performance of the MSE_v is studied over conceptual replications of training and validating data, given $X = x_0$, as in (6.59):

$$\begin{aligned} E_{yy_0} [\text{MSE}_v (x_0)] &= E_{y_0} (y_0^2) + E_y (\hat{y}_0^2) - 2 E_{yy_0} (y_0 \hat{y}_0) \\ &= \text{Var} (y_0) + (E (y_0))^2 + \text{Var} (\hat{y}_0) + (E (\hat{y}_0))^2 - 2 E (y_0) E (\hat{y}_0) \\ &= \text{Var} (y_0) + \text{Var} (\hat{y}_0) + (E (y_0) - E (\hat{y}_0))^2 \end{aligned} \quad (6.66)$$

where the second line is the result of using $E_{yy_0} (y_0 \hat{y}_0) = E (y_0) E (\hat{y}_0)$ due to the independence of training and validating data in this simple least squares setting.

The focus here is on the quality of a predictor, computed repeatedly from conceptual replications of training data and measured over conceptual replications of validating data. Single observations are predicted, accounting for their sampling uncertainty and also accounting for sampling uncertainty of the parameters that index the predictor.

Expression (6.66) indicates how the expected validating mean squared error $E_{yy_0} [\text{MSE}_v (x_0)]$ evaluated at x_0 is affected by the inherent variation of y_0 , by the variance of the predictor $\text{Var} (\hat{y}_0)$ that describes how the values of $\hat{y}_0$ for the different training data sets vary around their average and the squared bias of the predictor. As the complexity of a model increases, the variance of the predictor tends to increase and the bias term tends to decrease. In practice most operational models are distortions of the true state of nature and hence are biased. In a prediction setup one often chooses the degree of complexity of a predictive model to trade bias off with variance in such a way as to minimise $E [\text{MSE}_v (x_0)]$. Therefore, choosing

a predictive model amounts to striking a balance between bias and variance. This is particularly relevant with the advent of new technologies that allows vast amounts of covariates to be collected. Even when the covariates are relevant, the variance incurred in estimating their effects may outweigh the reduction in bias. The “true model” may not lead to the best predictive tool.

For a given number of observations, positively correlated data due to underlying random effects, such as family effects, lead to a similar decomposition as (6.66) except that the terms $\text{Var}(y_0)$ (conditional on covariates but averaged over the distribution of random effects) and $\text{Var}(\hat{y}_0)$, are larger and often validating and training data are correlated resulting in $E_{yy_0}(y_0\hat{y}_0) = \text{Cov}(y_0, \hat{y}_0) + E(y_0)E(\hat{y}_0)$.

The three scenarios described above, developed from a frequentist perspective, have a Bayesian counterpart where all expectations are conditional on training and validating data. The topic is discussed in Chap. 10, in the Exercises on Prediction on pages 626 and 719 and also on page 485. Bayesian expectations of means squared error are derived on page 482.

Briefly, with y_0 representing validating data, y representing training data and θ the parameter of the model used to construct the predictor $\hat{y}_0$ of the validating data, the Bayesian scenario 1 corresponds to drawing the predictor $\hat{y}_0$ from $[y_0|\hat{\theta}, y, x_0]$, where $\hat{\theta}$ is some point estimate of θ . In this situation account is taken of sampling variation of the predictor of single records $\hat{y}_0$ and this variation is one of the components of the Bayesian expected validating mean squared error. By replicating data from $[y_0|\hat{\theta}, y, x_0]$ we are studying the frequency properties of the Bayesian procedure.

The second scenario corresponds to constructing the predictor of an average of records using a function f , $\hat{y}_0 = f(\theta^*, x_0)$, where θ^* is a draw from the posterior distribution $[\theta|y]$. In this case account is taken of posterior uncertainty of θ that propagates on to the expected validating mean squared error.

The third scenario takes care of uncertainty of θ and of sampling variation of a new record $\hat{y}_0$. In this case, θ^* is drawn from $[\theta|y]$ and given θ^* , $\hat{y}_0$ is drawn from $[y_0|\theta^*, y, x_0]$. Both sources of uncertainty are incorporated in the Bayesian expected validating mean squared error.

An alternative description of the three scenarios is as follows. Each scenario gives rise to a predictor that is characterised by its distribution. The mean squared errors are transformations with their own distributions that here are summarised by their expected value. This general narrative holds for the frequentist and Bayesian approaches with the obvious caveat that what is considered random and uncertain and what is considered non-random is peculiar to each approach to inference.

An attraction of the Bayesian approach implemented in an MCMC environment, is the ease with which the marginal posterior distribution of MSE can be estimated. If two models are evaluated in terms of their predictive ability, the computation of the marginal posterior distribution of the difference in MSE between the models only requires a few extra lines of code. This can be particularly relevant when the merits of different prediction machines are to be judged on the basis of relatively

small differences. However, a usual way of evaluating models is to estimate parameters and to use these to construct the point predictor ignoring uncertainty.

6.5 Estimation of Validation MSE of Prediction in Practice

The measure of prediction ability often used in practice is the sample mean squared error. A possibility is to use the same training data $\{y_i, x_i\}_{i=1}^N$, from which the predictions $\hat{f}(x_i)$ were estimated, in order to compute the sample *training mean squared error*

$$\text{MSE}_t = \frac{1}{N} \sum_{i=1}^N \left(y_i - \hat{f}(x_i) \right)^2. \quad (6.67)$$

This is the average squared discrepancy between observations and their predictions. The training mean squared error is not a good measure of prediction ability because it has the property of decreasing consistently with model complexity. A model with close to zero training squared error is overfit to the training data, as it captures not only true signal but also the noise peculiar to the training data and will predict new, independent observations poorly.

A better proposition for model validation or model selection would be to study the predictive ability of $\hat{f}$, computed using the training data, in an independent set of data: the testing or validating data $\{y_{v,i}, x_{v,i}\}_{i=1}^N$. This validating data is conceptually a new realisation from the same distribution that generated the training data. Using the validating data one can compute

$$\text{MSE}_v = \frac{1}{N} \sum_{i=1}^N \left(y_{v,i} - \hat{f}(x_{v,i}) \right)^2, \quad (6.68)$$

the sample *testing or validating mean squared error*. However with limited data or without access to real validating data, one must resort to alternative procedures. These consist of artificially splitting the data into training and validating sets or to use approximations derived from theory without splitting the available data.

One possibility is to use *leave-one-out cross-validation*. Each of the N observations (y_i, x_i) is held out in turn and the function $\hat{f}^{(-i)}$ is estimated with the remaining $N - 1$ records from the training data, with the i th pair (y_i, x_i) omitted. Then the squared discrepancy between y_i and $\hat{f}^{(-i)}(x_i)$ is computed and this is repeated N times. The resulting estimate of the leave-one-out prediction mean squared error is

$$\text{MSE}_1 = \frac{1}{N} \sum_{i=1}^N \left(y_i - \hat{f}^{(-i)}(x_i) \right)^2. \quad (6.69)$$

A more common practice is to use *K-fold cross-validation* whereby the complete set of N records is divided into K groups or folds of approximately equal size. For the k th fold the model is fitted to the other $K - 1$ folds of the data (this yields $\hat{f}^{(-k)}$ the estimate of the predictive function based on the $K - 1$ folds of the data with the k th fold excluded) and the prediction mean squared error of the fitted model is estimated using data from the k th fold. This is repeated for the K groups and the K estimates of prediction mean squared error are combined to obtain the cross-validation estimate of prediction mean squared error. Specifically, *K-fold cross-validation* proceeds as follows:

1. Divide the N records into K folds of approximately equal size n_k ; $N = \sum_{k=1}^K n_k$
2. Fit the model to the remaining $K - 1$ parts of the data (with fold k excluded) and obtain the estimate $\hat{f}^{(-k)}$
3. Using the records from the k th fold only, estimate the mean squared error

$$\text{MSE}_k \left(\hat{f}^{(-k)} \right) = \frac{1}{n_k} \sum_{i \in k\text{th fold}} \left(y_i - \hat{f}^{(-k)}(x_i) \right)^2$$

4. Repeat using all the K folds to obtain $\text{MSE}_1 \left(\hat{f}^{(-1)} \right) \dots \text{MSE}_K \left(\hat{f}^{(-K)} \right)$
5. Obtain the K -fold cross-validation combined (weighted according to fold size n_k) estimate of mean squared error

$$\text{MSE}_K = \frac{1}{N} \sum_{k=1}^K n_k \text{MSE}_k \left(\hat{f}^{(-k)} \right). \quad (6.70)$$

If all groups are of equal size n , $N = Kn$ and this reduces to

$$\text{MSE}_K = \frac{1}{K} \sum_{k=1}^K \text{MSE}_k \left(\hat{f}^{(-k)} \right). \quad (6.71)$$

Common choices for K are $K = 5$ or $K = 10$. If $n = 1$ the procedure recovers *leave-one-out cross-validation*. The K -fold cross-validation may be repeated several times by reconstructing the folds at random, to obtain estimates of the distribution of MSE_K .

Compared to K -fold cross-validation, leave-one-out cross-validation can suffer from highly correlated out-of-sample predictions. Indeed, with N observations there are N forecasts and any two training sets share $N - 2$ points. Therefore despite averaging N forecasts these tend to be highly correlated and therefore the leave-one-out estimator tends to have high variance (taken over conceptual replications of training data). With K -fold cross-validation the folds share relatively fewer observations and are less correlated. This tends to reduce the variance of the estimator. However, a relatively small number of the observations in the training data are included to estimate parameters and the K -fold estimator may be biased

upwards with respect to the squared error obtained using the entire data set. This is not the case for leave-one-out cross-validation, since the method repeatedly trains the model on $N - 1$ observations, almost as many as in the entire training data.

Implementation Shortcut for Leave-one-out Cross-validation

For linear smoothers or linear fitting methods as defined in connection with (6.30), a computational advantage of leave-one-out cross-validation is that it requires running through the training data only once, rather than N times (Seber and Lee 2003). This relies on a little trick that works as follows. From (6.30), the i th predicted value at x_i is

$$\hat{y}_i = h_{i1}y_1 + \cdots + h_{ii}y_i + \cdots + h_{iN}y_N$$

with $\sum_{j=1}^N h_{ij} = 1$ when the model includes an intercept. Eliminating the i th datum and rescaling so that the $N - 1$ weights sum to one yields

$$\hat{y}_{-i} = \frac{\hat{y}_i - h_{ii}y_i}{1 - h_{ii}}.$$

A more formal proof of this identity can be found in Seber and Lee (2003), pages 268 and 403.

The leave-one-out residual or prediction error is

$$\begin{aligned} r_i &= y_i - \hat{y}_{-i} \\ &= y_i - \frac{\hat{y}_i - h_{ii}y_i}{1 - h_{ii}} \\ &= \frac{y_i(1 - h_{ii}) - (\hat{y}_i - h_{ii}y_i)}{1 - h_{ii}} \\ &= \frac{y_i - \hat{y}_i}{1 - h_{ii}} \end{aligned} \tag{6.72}$$

and the leave-one-out cross-validation prediction mean squared error (6.69) reduces to

$$\text{MSE}_1 = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_{-i})^2 = \frac{1}{N} \sum_{i=1}^N \left(\frac{y_i - \hat{y}_i}{1 - h_{ii}} \right)^2, \tag{6.73}$$

that also holds when the model does not have an intercept (Seber and Lee 2003). The calculation of (6.73) requires one pass through the training data rather than the N implied in the left hand side.

A computationally faster approximation known as the *generalised cross-validation*, is

$$\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_{-i})^2 = \frac{1}{N (1 - N^{-1} \text{tr}(H))^2} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (6.74)$$

whereby rather than weighting each term by 1 minus the i th diagonal element of the hat matrix H , all terms are given the same weight (1 minus the average of the diagonal elements of H). The generalised cross-validation can be a numerically more robust alternative to (6.73) in cases where a few records may have a strong influence due to the terms $1 - h_{ii}$ in the denominator.

Using the approximation

$$\frac{1}{(1-x)^2} \approx 1 + 2x$$

allows expressing (6.74) as

$$\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_{-i})^2 \approx \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{N} \left(1 + 2 \frac{\text{tr}(H)}{N} \right). \quad (6.75)$$

For the least squares linear regression with p parameters $\text{tr}(H) = p$ and (6.75) reduces to

$$\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_{-i})^2 \approx \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{N} \left(1 + 2 \frac{p}{N} \right) \quad (6.76)$$

which establishes a connection between leave-one-out cross-validation and Mallows' C_p (Mallows 1973, see (6.90) below). The latter is an instance where approximate prediction ability is derived from theory using training data only, without resorting to cross-validation. There are several other theory-based measures of prediction ability such as Akaike's information criterion, Bayesian information criterion and many others. An entry to the subject is Gelman et al. (2014a).

6.6 On Average Training MSE Underestimates Validation MSE

The within sample prediction ability based on $\hat{f}$ will be overstated, in expectation, relative to the out-of-sample prediction, leading to $E(\text{MSE}_t) < E(\text{MSE}_v)$. The algebra underlying this result is simple and illuminating and is described in this section.

Independently Distributed Data

Let $\{y_i, x_i\}_{i=1}^N$ and $\{y_{v,i}, x_{v,i}\}_{i=1}^N$ denote the N observations in the training and validating data, respectively. Assume that under the true model the observations have means $f(x_i)$ and variance σ^2 ; that is

$$y_i = f(x_i) + e_i, e_i \sim (0, \sigma^2),$$

$$y_{v,i} = f(x_{v,i}) + e_{v,i}, e_{v,i} \sim (0, \sigma^2).$$

In the derivation that follows, (y_i, x_i) and $(y_{v,i}, x_{v,i})$ are different realisations from the same probability model. The random quantities are $(y_i, y_{v,i})$, whereas the covariates x are treated as fixed and known. Importantly, records are assumed to be independent. The modification needed for correlated data is discussed below.

The true model is unknown and a number of operational or instrumental models may be available to choose as prediction machines. The prediction ability will be studied specifically as follows:

1. Fit the operational model to the training data $\{y_i, x_i\}$ and obtain predictive values for the validating data $\hat{y}_{v,i} = \hat{f}(x_{v,i})$, $i = 1, 2, \dots, N$.
2. Obtain predictions for the training records $\hat{y}_i = \hat{f}(x_i)$, $i = 1, 2, \dots, N$ and compute the training mean squared error

$$\text{MSE}_t = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2 \quad (6.77)$$

3. Use the validating data $\{y_{v,i}, x_{v,i}\}$ to compute the testing or validating mean squared error

$$\text{MSE}_v = \frac{1}{N} \sum_i (y_{v,i} - \hat{y}_{v,i})^2 \quad (6.78)$$

The expected value of MSE_v is derived first. Taking expectations of (6.78) with respect to the distribution of (y, y_v)

$$\mathbb{E}_{y, y_v} \left[\frac{1}{N} \sum_i (y_{v,i} - \hat{y}_{v,i})^2 \right] = \frac{1}{N} \sum_i \mathbb{E}_{y_i y_{v,i}} (y_{v,i} - \hat{y}_{v,i})^2 \quad (6.79)$$

The squared term is expanded as in (6.63) and going through the same algebra used in the derivation of (6.66) leads to the expected validating mean squared error:

$$\begin{aligned}
 E_{y,y_v}(\text{MSE}_v) &= E_{y,y_v} \left[\frac{1}{N} \sum_i (y_{v,i} - \hat{y}_{v,i})^2 \right] \\
 &= \frac{1}{N} \sum_i \sigma^2 + \frac{1}{N} \sum_i \text{bias}^2(i) + \frac{1}{N} \sum_i \text{Var}(\hat{y}_{v,i}) \\
 &= \sigma^2 + \text{bias}^2 + \frac{1}{N} \sum_i \text{Var}(\hat{y}_{v,i}), \tag{6.80}
 \end{aligned}$$

where $\text{bias}^2(i) = (E(y_{v,i}) - E(\hat{y}_{v,i}))^2$.

The expected value of the training mean squared error is derived in a similar manner. Taking expectations of (6.77) with y random and covariates x fixed yields

$$E_y(\text{MSE}_t) = E_y \left[\frac{1}{N} \sum_i (y_i - \hat{y}_i)^2 \right] = \frac{1}{N} \sum_i E_{y_i} (y_i - \hat{y}_i)^2.$$

Expanding the square results in

$$\begin{aligned}
 E_{y_i} (y_i - \hat{y}_i)^2 &= E_{y_i} (y_i^2 + \hat{y}_i^2 - 2y_i \hat{y}_i)^2 \\
 &= \text{Var}(y_i) + (E(y_i))^2 + \text{Var}(\hat{y}_i) + (E(\hat{y}_i))^2 - 2 \text{Cov}(y_i, \hat{y}_i) - 2 E(y_i) E(\hat{y}_i) \\
 &= \text{Var}(y_i) + \text{Var}(\hat{y}_i) + (E(y_i) - E(\hat{y}_i))^2 - 2 \text{Cov}(y_i, \hat{y}_i).
 \end{aligned}$$

Therefore,

$$E_y(\text{MSE}_t) = \sigma^2 + \frac{1}{N} \sum_i \text{Var}(\hat{y}_i) + \text{bias}^2 - \frac{2}{N} \sum_i \text{Cov}(y_i, \hat{y}_i) \tag{6.81}$$

and from (6.80), when covariates in training and validating data take the same values (see note0603.pdf at <https://github.com/SorensenD/SLGDS> for elaborations on the subject)

$$E_y(\text{MSE}_t) = E_{yy_v}(\text{MSE}_v) - \frac{2}{N} \sum_i \text{Cov}(y_i, \hat{y}_i), \tag{6.82}$$

(Efron 1986) indicating that on average, the training MSE_t underestimates validating MSE_v especially if y_i and $\hat{y}_i$ are highly correlated. This happens when the operational model has a large number of parameters that result in a very close fit, with little discrepancy between observations y_i and fitted values $\hat{y}_i$. The term

$\text{Cov}(y_i, \hat{y}_i)$ is not observable and unless an analytic form is available (as for linear smoothers), must be estimated using, for example, the bootstrap or Monte Carlo. This subject is revisited in the Examples section, page 635.

The form of (6.82) encompasses two terms, reflecting model fit, $E_y(\text{MSE}_t)$ and model complexity, $\sum_i \text{Cov}(y_i, \hat{y}_i)$. The second term on the right hand side of (6.82) is labelled *expected optimism* by Efron (1986),

$$E_{y y_v}(\text{MSE}_v) - E_y(\text{MSE}_t) = \frac{2}{N} \sum_i \text{Cov}(y_i, \hat{y}_i) = \frac{2}{N} \text{tr}(\text{Cov}(y, \hat{y}')), \quad (6.83)$$

the difference between the expected validating mean squared error and the expected training mean squared error.

Invoking the law of large numbers, a natural estimate of the expected validating mean squared error obtained using the training data is (Efron and Hastie 2016)

$$\hat{E}(\text{MSE}_v) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \frac{2}{N} \sum_{i=1}^N \widehat{\text{Cov}}(y_i, \hat{y}_i). \quad (6.84)$$

The first term on the right hand side is the sample training mean squared error MSE_t and $\widehat{\text{Cov}}(y_i, \hat{y}_i)$ is some estimate obtained using Monte Carlo simulation or the bootstrap. Expression (6.84) obviates estimation of σ^2 and is simple to compute if an analytic form is available for $\text{Cov}(y_i, \hat{y}_i)$. Otherwise one incurs the cost of a bootstrap or a Monte Carlo analysis. An important issue with estimator (6.84) is that the rate of convergence to its expectation depends on the ratio p/N .

An estimate of expected optimism provides an approximation to the evaluation of prediction ability using training data without incurring the cost of cross-validation. See Exercise 12.5 on page 637 for a Bayesian take on the subject.

Correlated Data

Often data are correlated such as in genetic studies where family structure is a classical source of correlation. The correlation structure can be patterned and simple as in independent full-sib families, where correlation arises between members of the same family, or can be highly complex with pedigrees spanning several overlapping generations.

With correlated data the expected training mean squared error (6.81) is partitioned into the same 4 components, while the expectation of the validating mean squared error (6.80) has an extra term that describes the covariance between an observation in the validating set and its predictor. The degree of this association depends on the correlation structure in the data. With highly correlated data as found in animal breeding, a predicted value is constructed using information in the training data from many related individuals and the validating datum correlates with these

training data. The expected value of the validating mean squared error takes the form

$$E_{y,y_v}(\text{MSE}_v) = \sigma^2 + \text{bias}^2 + \frac{1}{N} \sum_i \text{Var}(\hat{y}_{v,i}) - \frac{2}{N} \sum_{i=1}^N \text{Cov}(y_{v,i}, \hat{y}_{v,i}) \quad (6.85)$$

and the expected optimism is

$$E_{y,y_v}(\text{MSE}_v) - E_y(\text{MSE}_t) = \frac{2}{N} \left[\sum_{i=1}^N \text{Cov}(y_i, \hat{y}_i) - \sum_{i=1}^N \text{Cov}(y_{v,i}, \hat{y}_{v,i}) \right]. \quad (6.86)$$

Here it is assumed that training and validating data are of size N .

In general, the whole setup of cross-validation builds on the assumption that validating data represent an independent draw from the same distribution as the training data, conditional on the model parameters. When data with a complicated hierarchical structure (such as a pedigree structure) are divided into training and validating sets, the assumption of conditional independence is often violated. The conclusions will then be restricted to the data at hand and cannot be justifiably extrapolated to a new random sample of future data, particularly when it does not reproduce the same structure as the training data. The problem can be partly mitigated by dividing the data judiciously accounting for the structure in the data and the prediction problem at hand. This may not always be possible and results must therefore be interpreted cautiously, and, as a general rule, supplemented with other approaches of model criticism and comparison.

Estimating Optimism of the Training Sample

Often the covariance term $\text{Cov}(y_i, \hat{y}_i)$ cannot be obtained analytically due to the nature of the fitting procedure but can be estimated either using the nonparametric or (the model dependent) parametric bootstrap (Efron and Hastie 2016). The parametric approach is easier to use when data are correlated. In the parametric bootstrap repeated samples of the vector of data are drawn from the parametric model used to analyse the original observations, conditional on parameter estimates. The latter are obtained using these original observations.

If a nonparametric bootstrap is chosen, at least one of two strategies can be followed. Assume that the training data consist of independent samples $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$. Then a **first** bootstrap strategy is to cycle through a loop from $b = 1$ to B ($B = 1000$, say)

1. draw N bootstrap samples (i.e., sampling with replacement from the data)

$$(x_i^b, y_i^b), \quad i = 1, \dots, N.$$

calculate the regression function $\hat{f}^b$ using (x_i^b, y_i^b) , $i = 1, \dots, N$ and obtain the fitted values $\hat{y}_i^b = \hat{f}^b(x_i^b)$, $i = 1, \dots, N$

2. store y_i^b , $i = 1, \dots, N$ and $\hat{y}_i^b$, $i = 1, \dots, N$
3. after B cycles calculate the empirical covariance between y_i and $\hat{y}_i$

$$\widehat{\text{Cov}}(y_i, \hat{y}_i) = \frac{1}{B} \sum_{b=1}^B (y_i^b - \bar{y}_i^b) (\hat{y}_i^b - \bar{\hat{y}}_i^b),$$

where

$$\bar{y}_i^b = \frac{1}{B} \sum_{b=1}^B y_i^b, \quad \bar{\hat{y}}_i^b = \frac{1}{B} \sum_{b=1}^B \hat{y}_i^b.$$

4. finally sum these over $i = 1, \dots, N$ to obtain the bootstrap estimate

$$\sum_{i=1}^N \widehat{\text{Cov}}(y_i, \hat{y}_i).$$

For continuous data, a **second** bootstrap strategy is to use the *residual bootstrap* that proceeds as follows.

1. using the training data $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ calculate $\hat{y}_i = \hat{f}(x_i)$, $i = 1, \dots, N$ and compute the empirical residuals

$$\hat{e}_i = y_i - \hat{y}_i, \quad i = 1, \dots, N.$$

Then cycle through the loop from $b = 1$ to B :

2. draw N samples with replacement of the empirical residuals

$$\hat{e}_i^b, \quad i = 1, \dots, N. \tag{6.87}$$

3. using the N bootstrap residuals (6.87) calculate

$$y_i^b = \hat{y}_i + \hat{e}_i^b, \quad i = 1, \dots, N$$

4. using (x_i, y_i^b) , $i = 1, \dots, N$ calculate the regression function $\hat{f}^b$ and obtain the fitted values $\hat{y}_i^b = \hat{f}^b(x_i^b)$, $i = 1, \dots, N$. Notice that the x_i are not sampled; only the residuals are sampled to compute $y_i^b = \hat{y}_i + \hat{e}_i^b$

5. store y_i^b , $i = 1, \dots, N$ and $\hat{y}_i^b$, $i = 1, \dots, N$
6. after B cycles calculate the empirical covariance between y_i^b and $\hat{y}_i^b$

$$\widehat{\text{Cov}}(y_i^b, \hat{y}_i^b) = \frac{1}{B} \sum_{b=1}^B (y_i^b - \bar{y}_i^b) (\hat{y}_i^b - \bar{\hat{y}}_i^b)$$

where

$$\bar{y}_i^b = \frac{1}{B} \sum_{b=1}^B y_i^b, \quad \bar{\hat{y}}_i^b = \frac{1}{B} \sum_{b=1}^B \hat{y}_i^b$$

7. finally sum these over $i = 1, \dots, N$ to obtain the bootstrap estimate

$$\sum_{i=1}^N \widehat{\text{Cov}}(y_i, \hat{y}_i). \quad (6.88)$$

An alternative approach to use when data are correlated or have a complicated structure is the parametric bootstrap. In the Prediction Exercises, page 635, you are asked to derive an expression for the expected optimism for a hierarchical model, to obtain numerical results using simulated data and to write a code to obtain a Monte Carlo estimate of expected optimism.

The two nonparametric bootstrap strategies described in this section are drawn from lecture notes of the course Advanced Methods for Data Analysis (R. Tibshirani) and can be found at <https://www.stat.cmu.edu/~ryantibs/advmethods/>.

6.7 Least Squares Prediction

The form of covariance penalty $\text{Cov}(y_i, \hat{y}_i)$ depends on the operational model used for prediction. Suppose that the operational model takes the standard regression form $y = Xb + e$, where the vector of residual terms $e \sim (0, I\sigma^2)$, the full-rank matrix X is of dimension $N \times p$ and the column vector of parameters b (of dimension p) is estimated by least squares. In this case as indicated in (6.35)

$$\text{Cov}(y_i, \hat{y}_i | X, \sigma^2) = h_{ii} \sigma^2$$

where h_{ii} is the i th diagonal element of the hat matrix H . Then the penalty term is

$$\begin{aligned}
 \frac{2}{N} \sum_i \text{Cov} \left(y_i, \hat{y}_i | X, \sigma^2 \right) &= \frac{2}{N} \text{tr} (H) \sigma^2 \\
 &= \frac{2}{N} \text{tr} \left(X (X' X)^{-1} X' \right) \sigma^2 \\
 &= \frac{2}{N} \text{tr} \left((X' X)^{-1} X' X \right) \sigma^2 \\
 &= 2 \frac{p}{N} \sigma^2
 \end{aligned}$$

and (6.82) takes the form

$$E_{yy_v} (\text{MSE}_t) = E_{yy_v} (\text{MSE}_v) - 2 \frac{p}{N} \sigma^2. \quad (6.89)$$

In view of this result and (6.84), a natural unbiased estimator of the validating mean squared error for the standard regression model, using the training mean squared error, could be

$$\begin{aligned}
 \hat{E}_{yy_v} (\text{MSE}_v) &= \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \frac{2}{N} \text{tr} (H) \sigma^2 \\
 &= \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + 2 \frac{p}{N} \sigma^2
 \end{aligned} \quad (6.90)$$

which is a version of Mallows' C_p (Mallows 1973). When σ^2 is not known it must be replaced by an estimate. The unbiased property holds if an unbiased estimator of σ^2 is available.

Limiting Distribution of the Validating Mean Squared Error

The distribution of MSE_v as N (the length of the vector of validating records) tends to infinity is studied as follows. Consider again the standard linear regression model $y = Xb + e$, where now the vector of residual terms $e \sim N(0, I\sigma^2)$. The validating mean squared error is

$$\text{MSE}_v = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

In this expression y_i is the i th validating record, $\hat{y}_i = x_i' \hat{b}$ its predictor, where x_i' is the i th row of matrix X and $\hat{b}$ is the standard least squares estimator of b . Add and subtract Xb inside the square brackets and expand the square. This yields

$$\begin{aligned} \frac{1}{N} \sum_{i=1}^N (y_i - x_i' \hat{b})^2 &= \frac{1}{N} [(y - Xb) - (X\hat{b} - Xb)]' [(y - Xb) - (X\hat{b} - Xb)] \\ &= \frac{1}{N} (y - Xb)' (y - Xb) - \frac{2}{N} (y - Xb)' X (\hat{b} - b) + \frac{1}{N} (\hat{b} - b)' X' X (\hat{b} - b). \end{aligned}$$

As $N \rightarrow \infty$ the first term tends to σ^2 and the second tends to zero in probability. Therefore,

$$\text{MSE}_v \sim \sigma^2 + \frac{1}{N} (\hat{b} - b)' X' X (\hat{b} - b). \quad (6.91)$$

Define $z'z = (\hat{b} - b)' A (\hat{b} - b)$, with $A = X'X \frac{1}{\sigma^2}$. Since $A \text{Var}(\hat{b} - b) = I_p$, an idempotent matrix, then, assuming normality of $\hat{b}$ that requires normality of the vector of residuals, $z'z \sim \chi^2(p)$, a central chi-squared distribution with p degrees of freedom. It follows that

$$\text{MSE}_v \sim \sigma^2 + \frac{\sigma^2}{N} \chi^2(p), \quad (6.92)$$

a central scaled chi-squared distribution with p degrees of freedom plus a constant, because in this least squares setting the non-centrality parameter is

$$\lambda = (E(\hat{b}) - b)' X' X (E(\hat{b}) - b) \frac{1}{\sigma^2} = 0. \quad (6.93)$$

The expected value of MSE_v (6.92) is

$$E(\text{MSE}_v) = \sigma^2 + \frac{p}{N} \sigma^2.$$

A connection with (6.80) can be established by noting that the bias term is zero and the third term is

$$\begin{aligned} \frac{1}{N} \sum_i \text{Var}(\hat{y}_i | X, \sigma^2) &= \frac{1}{N} \text{tr} [\text{Var}(X\hat{b} | X, \sigma^2)] \\ &= \frac{p}{N} \sigma^2. \end{aligned}$$

Example: Prediction Ability of the Least Squares Estimator

Some of the ideas discussed so far are illustrated using simulated data. These consist of $N = 55$ observations from a training set and $N = 55$ from a testing or validating set. The structure of the simulated data (not its size!), is representative of a genomic data set. A matrix X of genetic marker genotypes of dimension $N \times p_{\max}$ (centred and scaled) was generated representing genotype codes whose elements consist of *iid* (identically and independently distributed) draws from a binomial distribution with parameters $n = 2$ and probability 0.5. The number of columns of X in the example is set to $p_{\max} = 50$ and the first $p_{\ell} = 10$ columns are assigned as quantitative trait loci (QTL) affecting the trait.

Since this is the first example where marker genotypes are simulated, a comment is in order. In a typical situation, matrix X consists of correlated observed marker genotypes that are not themselves causal. The idea is that the correlation structure induces a correlation between the marker loci and the unobserved causal loci. In this way, the marker genotypes may inform on the underlying genetics of a trait. Here, the simulated marker genotypes are uncorrelated, on average, but importantly, the “unobserved” causal loci are part of the marker panel. As mentioned in the appendix of Chap. 1, when the causal loci are included in the marker panel, the genetic markers capture the genetic variation of the trait. Including information from non-causal markers inflates the validating mean squared error and the prediction ability of the model is compromised.

The true model for the training data is

$$y_t = X_{\ell} b_{\ell} + e, \quad (6.94)$$

where y_t is an $N \times 1$ vector of training observations, X_{ℓ} is the $N \times p_{\ell}$ matrix of QTL genotype codes and b_{ℓ} is an $p_{\ell} \times 1$ vector of allele substitution effects of causal loci with elements set equal to 0.5. Due to centring and scaling the elements of X are random variables with mean 0 and variance 1. The resulting equilibrium additive genetic variance is equal to $\text{Var}(x'_{i,\ell} b_{\ell} | b_{\ell}) = 10 \times (1 \times 0.5^2) = 2.5$ squared units, where $x'_{i,\ell}$ is the i th row of matrix X_{ℓ} . The vector e consists of random residuals $e \sim N(0, I 5)$ and is of dimension $N \times 1$. The proportion of variance of y explained by the true model (the R^2 or equilibrium heritability) is expected to be 33% ($(10 \times 1 \times 0.5^2 / 7.5) \times 100$). Training and validating data are generated using the same model and different samples of X and of e .

Training data y_t are analysed using operational models that regress y_t on markers genotypes X as covariates. Ten operational linear models are used to analyse y_t of the form

$$y_t = X_j b_j + \varepsilon_j, \quad j = 1, 2, \dots, 10 \quad (6.95)$$

that differ in the number of markers included. This is reflected in the number of columns of X_j and the number of elements of b_j , which range from 5 to 50 in

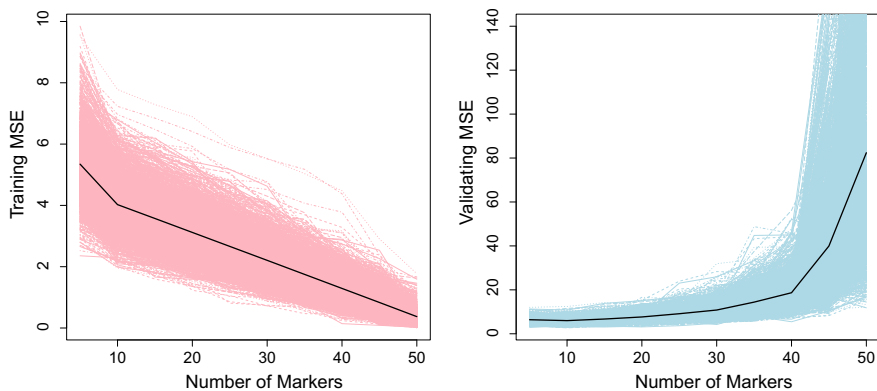


Fig. 6.1 Left panel, red: Training mean squared error (6.67) and Right panel, blue: validating mean squared error (6.68) of the least squares predictor using simulated data generated with the linear model. Results from 2000 training/validating replicates; black lines: average over replicates

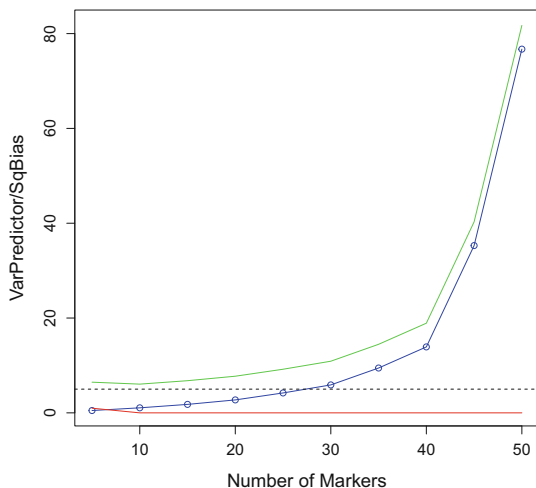
increments of 5 and the subscript j indexes the model. When $j = 2$ the operational model is the same as the true model including 10 covariates (the 10 QTL).

Figure 6.1 displays the training (left panel, red) and validating (right panel, blue) sample mean squared errors (6.67) and (6.68) using data analysed with the ten linear models (6.95) with increasing number of covariates. The figures show the behaviour of individual replicates (2000 replications of training/validating records, with genetic marker covariates X fixed at the initial simulated value) as well as the average MSE over replicate lines that is a Monte Carlo estimate of the expected MSE. The training mean squared errors (left panel) show a marked decline as the number of covariates increases from 5 to 10. At this point, the true model and the operational model are identical. As more covariates are included the training squared error falls, albeit at a lower rate. The validating squared error decreases slightly from its initial value, reaches a minimum when the number of covariates is 10, and increases subsequently as additional covariates are included. The rate of increase is very pronounced when 40 or more covariates are included in the operational models.

The figures emphasise an important feature of the training-testing split as a tool to monitor prediction ability: the large variation among replications. A correct specification of this variation may be especially relevant in a situation where the goal is to learn how well a given statistical model will perform on independent data. On the other hand, one may be interested in comparing models and therefore locating the minimum of the MSE curve may be an adequate goal, rather than the actual value of the MSE. The curves of the different replicates in Fig. 6.1, right panel, seem to identify the correct model with 10 covariates despite the fact that each replicate may deviate quite markedly from the average MSE.

The expected validating mean squared error and its three components (6.80) are shown in Fig. 6.2. The expected validating mean squared error agrees well with

Fig. 6.2 The expected validating mean squared error (green) and its three components (6.66): expected squared bias (red), variance of predictor (blue) and residual variance (dashed lines). Data simulated with the linear model



the simulated value displayed in the right panel of Fig. 6.1. The variance of the predictor increases with the number of covariates, particularly when this exceeds 20 and the increase becomes very pronounced with 40 or more covariates. The expected squared bias is positive when the number of covariates is equal to 5. When this number is equal to 10, the true model and the operational model are the same and the bias falls to zero. As the number of covariates increases beyond 10, the expected squared bias remains zero because in the case of the present example, the least squares estimator using models with 10 or more covariates is unbiased (see (6.51)). The dashed line represents the true residual variance of the observations.

Quantifying Optimism

The simulation example compares the Monte Carlo estimate of the expected optimism (6.83) with its expected value based on the second term of (6.90), for different values of the ratio p/N . When the number of records in training and validating data N is equal to 55, the average difference between the validating (6.78) and training (6.77) mean squared errors (over 500 Monte Carlo replicates) is as follows. Using an operational model with p equal to 10, 15, 25, or 35 covariates, the observed average difference between training and validating mean squared errors (O) and the predicted (P) based on $2(\hat{\sigma}^2/N)p$, where $\hat{\sigma}^2 = 4.99$ and $\sigma^2 = 5$, is (O; P): (3.1; 1.8), (3.5; 2.7), (6.9; 4.5), (12.7; 6.6). When instead of 55 records, 550 are used, with $\hat{\sigma}^2 = 5.00$, these figures are (O; P): (0.18; 0.18), (0.27; 0.28), (0.46; 0.45), (0.63; 0.64).

Another way of visualising optimism is illustrated in Fig. 6.3 that shows the regression of predictions versus observations in the training and validating data (55 observations) for a randomly chosen simulation replicate, using 50 covariates. In

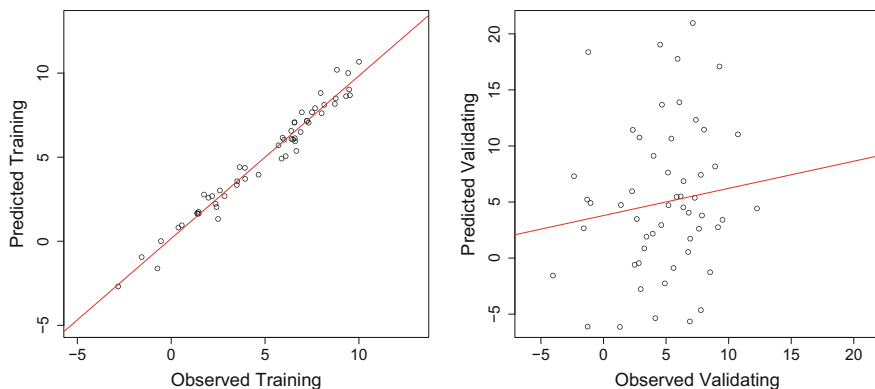


Fig. 6.3 Regression of predicted versus observed values in training (Left) and validating (Right) data

the validating data the regression is not significantly different from 0 and the linear model explains 1.5% of the variance of the predicted values. In contrast, in the training data the regression is equal to 0.97 (not significantly different from 1) and the linear model explains 97% of the variance of the predicted values.

Example: What Measure of Prediction Error Does Leave-one-out Estimate?

This example illustrates the ability of leave-one-out cross validation to estimate, either the conditional validating mean squared error (6.64) (averaging MSE_v over validating data, with training data fixed), or the unconditional validating squared error (6.66) (averaging MSE_v over training and validating data). The setup is the same as in the previous example, with 55 observations in training and testing data, 10 covariates assigned as loci affecting the trait and 10 operational models that differ in the number of covariates used for analysis.

The conditional validating squared error (6.64) has two terms: the irreducible error (equal to 5 in the simulation) and the squared term representing systematic difference between predictor and the expectation of the predictand. In least squares linear regression, this last quantity is zero on average over replications of training data, but it is not zero in any one realisation. For a given realisation of training data, as the number of covariates in the model increases, the variance of the predictor becomes larger and so does the size of the second term in (6.64). This is illustrated in the left panel of Fig. 6.4. The setup is one where there is a single realisation of training data and 1000 simulation replicates of validating data. The average (over 1000 simulation replicates of validating data) conditional validating squared error, shown in red, is equal to the conditional variance of the record (5 squared units), plus

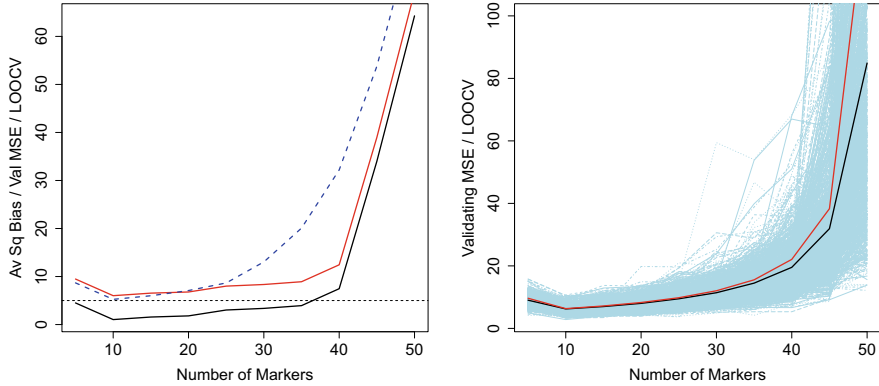


Fig. 6.4 Left: Components of Monte Carlo estimates of $E(\text{MSE})$ obtained by averaging over replications of validating data, given a single sample of training data. Dashed horizontal black line: conditional variance of a datum based on the true model (5 squared units). Solid black line: Average (over 1000 replicated validating data sets; a single training data is simulated) squared discrepancies between predictor and the expectation of the predictand y_0 , obtained using operational models with increasing number of covariates. Solid red line: average (over 1000 replicated validating data sets) conditional validating mean squared error, given training data. Dashed blue line: leave-one-out cross-validation estimate based on the single training data. Right: Blue: Validating mean squared errors for each of the 1000 replicated training and testing data sets; Solid black line: average validating mean squared error over the 1000 replicated training and testing data sets; Solid line red: average leave-one-out estimate of mean squared error over 1000 replicated training data sets

the average (over the 55 observations and 1000 simulation replicates of validating data) of the squared discrepancies between predictor and average predictand (shown in black). The figure also shows that the leave-one-out estimate of squared error (dashed line, blue) based on the single realisation of training data, overestimates the conditional validating mean squared error (6.64).

The right panel of Fig. 6.4 displays the validating squared error for each of the 1000 replications of training and testing data (blue), the average validating squared error over the 1000 replicates (thick black line), which is an estimate of (6.66) and the average leave-one-out estimate of squared error over the 1000 replications of training data (red line). The average leave-one-out estimator follows closely the average unconditional validating squared error (6.66), with a little overestimation particularly as the number of markers approaches the size of the training data. The agreement between the average leave-one-out estimator and the average unconditional validating squared error (6.66) could be anticipated by glancing at (6.76) and (6.90).

Example: Variation of Leave-one-out Cross-validation

This example illustrates the variation of the leave-one out estimator (6.73) in conceptual replications of training data, in the context of least squares. This implies that the number of markers is smaller than the number of records.

Three scenarios are illustrated. In the first scenario the amount of data is quite large relative to the number of covariates of the operational model ($N = 200$, $p = 50$). In the second, it is intermediate ($N = 100$) and the third is an extreme example where the number of covariates in the most complex operational model is large relative to the number of records ($N = 55$, $p = 50$).

The setup is as before, with 10 covariates assigned as loci affecting the observations (the true model) and 10 operational models that differ in the number of covariates used for analysis, ranging from 5 to 50 in steps of 5. The leave-one-out cross-validation estimate of MSE (6.73) for the 1000 simulation replicates of training data is shown in Fig. 6.5. With $N = 55$ records the degree of uncertainty is very large and it explodes as p approaches N . The variance of the leave-one-out cross-validation estimate of MSE decreases as the ratio number of makers (p) to size of training data (N) decreases (centre and right panels of Fig. 6.5).

Table 6.1 displays the average validating MSE over 2000 simulated training and validating data sets and the average leave-one-out estimates of MSE based on expression (6.73) over 2000 simulated training data, for various combinations of number of records N and covariates p . The leave-one-out estimates use the N training records, whereas computation of the validating mean squared error requires the N training and N validating records.

In general, the leave-one-out estimates have a tendency to overpredict the average validating mean squared error when the ratio p/N approaches 1. This is not a restriction when leave-one-out cross-validation is used with linear smoothers that use shrinkage, the topic of the next chapter.

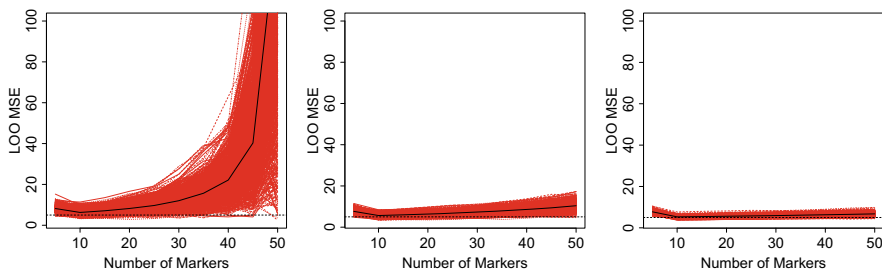


Fig. 6.5 Leave-one-out cross-validation estimates of mean squared error (6.73) for three sizes of training data (left panel: $N = 55$; centre panel: $N = 100$; right panel: $N = 200$) and operational models with increasing number of covariates, ranging from 5 to 50 in steps of 5. The average over 1000 simulation replicates of training and validating data is shown as solid black lines. The environmental variance, equal to 5 squared units is shown as horizontal dashed lines

Table 6.1 Average (over 2000 simulated training data sets) leave-one-out estimate of $MSE(X)$ and average over training and validating data of the validating $MSE(Y)$, for three sizes of training and validating records N . Figures in body of table correspond to $X; Y$

N	Number of covariates (p)		
	10	20	50
55	6.3; 6.3	8.3; 8.0	100.8; 59.2
100	5.6; 5.6	6.4; 6.2	10.4; 10.1
200	5.3; 5.3	5.6; 5.6	6.7; 6.7

Chapter 7

Shrinkage Methods



Expression (6.66) indicates how prediction ability is governed by bias and variance. As models become more complex, local noise can be captured but coefficient estimates suffer from higher variance as more terms are included in the model. In the context of the traditional regression model $y = Xb + e$, $e \sim N(0, I\sigma^2)$, when the number of covariates (number of columns of X) p is large relative to the number of records/individuals n (number of rows of X), the columns of matrix X may become rank deficient (X may not be or is close to not being of full column rank). In this case, even when $p < n$, it is difficult to separate the effect of individual covariates.

Shrinkage methods are typically used to confront this problem that emerges in highly dimensional models. Shrinkage estimators are obtained as the solution to an optimisation problem that balances bias and variance or model complexity. The general form of the optimisation problem is

$$\hat{b}_{\arg \min b} = \{L(y, b) + \lambda C(b)\}, \quad (7.1)$$

where $L(y, b)$ is a function that measures the lack of fit of the model to the data, $C(b)$ is a measure of model complexity and $\lambda \geq 0$ is a regularisation parameter controlling the trade-off between model fit and model complexity. This chapter starts by describing one of the simplest shrinkage estimators known as ridge regression. The balance between model fit and model complexity is achieved by shrinking all coefficients towards a common point; none is set to zero and coefficients tend to resemble each other.

The second shrinkage estimator described in this chapter is the *lasso* (least absolute shrinkage and selection operator). Lasso solves a critically different optimisation problem (lasso and ridge regression use different expressions for the model complexity parameter) and generates solutions where some of the regression estimates are exactly zero. Efficient algorithms have been developed that allow application of the lasso in models with a vast number of variables (typically larger

than the number of records). The lasso can be used for both prediction and model selection.

The chapter concludes with a description of a fully Bayesian-McMC spike and slab model. The model assigns a two-component prior mixture distribution to the covariate parameters. One of the components is a point mass at zero; the other is a normal distribution with mean zero and unknown variance. The model outputs the posterior probability that a covariate has no effect on the observations, for each covariate. The spike and slab model can be used for prediction and as a tool to isolate a handful of promising covariates for further study, among thousands or hundred of thousands present. Examples with simulated data mimicking genomic models illustrate various features of these models.

7.1 Ridge Regression

Ridge regression (Hoerl and Kennard 1970a,b) is a particular case of (7.1) with $L(y, b) = (y - Xb)'(y - Xb)$ and $C(b) = \sum_{j \in S} b_j^2$, where S is the set of coefficients to be penalised. Typically, some of the regression coefficients such as the intercept are not penalised. The ridge coefficients are obtained as

$$\hat{b}_{rr} = \underset{\arg \min b}{(y - Xb)'(y - Xb) + \lambda b' \Delta b}. \quad (7.2)$$

Here, Δ is a diagonal matrix whose entries are 1 for $j \in S$ and zero elsewhere. Taking first derivatives of (7.2) with respect to b , setting the linear system equal to zero and solving for b yields

$$\hat{b}_{rr} = (X'X + \lambda \Delta)^{-1} X'y, \quad (7.3)$$

a linear function of y . An equivalent way of writing (7.2) is

$$\hat{b}_{rr} = \underset{\arg \min b}{(y - Xb)'(y - Xb)}$$

subject to

$$b' \Delta b \leq t, \quad t \geq 0,$$

showing the size constraint on the parameters. There is a one-to-one correspondence between t and λ . The constrained minimisation uses Lagrange multipliers.

Relative to least squares, shrinkage adds a constant λ to the diagonal entries of $X'X$ which guarantees that the inverse in (7.3) exists even when $X'X$ is singular; there is always a unique solution to $\hat{b}_{rr}$. The form of the solution shows that as $\lambda \rightarrow \infty$, all the coefficients tend to zero $\hat{b}_{rr} \rightarrow 0$ and when $\lambda \rightarrow 0$, $\hat{b}_{rr}$ approaches the

least squares estimator. The coefficients are shrunk towards zero at a rate depending of λ that controls the size of the coefficients.

The relationship between the ridge estimator and the least squares estimator $\hat{b}$ is

$$\hat{b}_{rr} = \left[I + \lambda(X'X)^{-1} \Delta \right]^{-1} \hat{b}$$

that can be verified by replacing above $\hat{b} = (X'X)^{-1} X'y$ and using $B^{-1}A^{-1} = (AB)^{-1}$. Here $X'X$ is assumed to be non-singular.

The ridge solutions depend on the scaling of the inputs and therefore X is typically standardised (to have sample variance 1) and centred, so that $1'X = 0$, $\frac{1}{n}x_i'x_i = 1$, $i = 1, 2, \dots, p$ and $X'X$ is in “correlation form”. Also, in order to avoid fitting the intercept but leaving it unpenalised (Brown 1977), y is often also centred. When this is the case, matrix X is $n \times p$ instead of $n \times (p + 1)$ when the intercept is included and $\Delta = I$. If X is orthonormal such that $X'X = I$ and in models without the intercept,

$$\hat{b}_{rr} = [I + \lambda I]^{-1} \hat{b} = [(1 + \lambda) I]^{-1} \hat{b} = \frac{1}{1 + \lambda} \hat{b}$$

showing that in this situation, the ridge estimator is a downweighted version of the least squares estimator $\hat{b}$.

Ridge estimation leads to biased estimators of b . With $E(y|X) = Xb$ and $\Delta = I$, using (7.3),

$$\begin{aligned} E(\hat{b}_{rr}|X) &= (X'X + \lambda I)^{-1} X'Xb \\ &= (X'X + \lambda I)^{-1} (X'X + \lambda I - \lambda I) b \\ &= b - \lambda (X'X + \lambda I)^{-1} b \end{aligned}$$

with bias

$$E(\hat{b}_{rr}|X) - b = \lambda (X'X + \lambda I)^{-1} b,$$

proportional to λ . On the other hand, the ridge estimator has smaller variance than the least squares estimator. Let $X = UDV'$ be the singular value decomposition (SVD) of X of dimension $n \times p$. In the SVD, for $n > p$, U is $n \times p$, $U'U = I$, $D = \text{diag}(d_1, d_2, \dots, d_p)$, $d_i > 0$, $i = 1, \dots, p$ is a diagonal matrix with positive eigenvalues (when X is of full column rank) and V is $p \times p$, $V'V = I$. The variance of the least squares estimator is

$$\begin{aligned} \text{Var}(\hat{b}|X) &= \sigma^2 (X'X)^{-1} \\ &= \sigma^2 (VDU'UDV')^{-1} \end{aligned}$$

$$\begin{aligned}
&= \sigma^2 \left(V D^2 V' \right)^{-1} \\
&= \sigma^2 V D^{-2} V',
\end{aligned}$$

where D is a diagonal matrix with elements $1/d_i^2$. To go from the third to the fourth line use $(ABC)^{-1} = C^{-1}B^{-1}A^{-1}$ and $V^{-1} = V'$. On the other hand, the variance of the ridge estimator is

$$\begin{aligned}
\text{Var}(\hat{b}_{rr}|X) &= \sigma^2 (X'X + \lambda I)^{-1} X'X (X'X + \lambda I)^{-1} \\
&= \sigma^2 \left(V D^2 V' + \lambda I \right)^{-1} V D^2 V' \left(V D^2 V' + \lambda I \right)^{-1} \\
&= \sigma^2 \left[V \left(D^2 + \lambda I \right) V' \right]^{-1} V D^2 V' \left[V \left(D^2 + \lambda I \right) V' \right]^{-1} \\
&= \sigma^2 V \left(D^2 + \lambda I \right)^{-1} D^2 \left(D^2 + \lambda I \right)^{-1} V' \\
&= \sigma^2 V W V',
\end{aligned} \tag{7.4}$$

where $W = (D^2 + \lambda I)^{-1} D^2 (D^2 + \lambda I)^{-1}$ is a diagonal matrix with elements $d_i^2/(d_i^2 + \lambda)^2$ indicating that $\text{Var}(\hat{b}) \geq \text{Var}(\hat{b}_{rr})$, with equality if $\lambda = 0$ in the absence of shrinkage. The variance decreases with λ and vanishes as $\lambda \rightarrow \infty$.

Ridge regression performs particularly well in terms of mean squared error (or prediction error variance) relative to standard regression when some of the regression parameters are small or even zero. This advantage is less marked when the true parameters take larger values.

A Toy Example

This example is elaborated from a version in unpublished notes by G. de los Campos, Michigan State University.

Consider a simple one way classification with two levels parametrised as a linear regression model

$$y_i = x_{1i}b_1 + x_{2i}b_2 + e_i,$$

where $x_{1i} = 1$ if record i belongs in treatment 1 and 0 otherwise, and $x_{2i} = 1 - x_{1i}$. The treatment effects are b_1 and b_2 respectively. The least squares equations are

$$\begin{bmatrix} \sum_i x_{1i}^2 & \sum_i x_{1i}x_{2i} \\ \sum_i x_{1i}x_{2i} & \sum_i x_{2i}^2 \end{bmatrix} \begin{bmatrix} \hat{b}_1 \\ \hat{b}_2 \end{bmatrix} = \begin{bmatrix} \sum_i x_{1i}y_i \\ \sum_i x_{2i}y_i \end{bmatrix}.$$

Since $\sum_i x_{1i}^2$ and $\sum_i x_{2i}^2$ are the number of records in treatment 1 and 2 (denoted n_1 and n_2), $\sum_i x_{1i}x_{2i} = 0$, $\sum_i x_{1i}y_i$ and $\sum_i x_{2i}y_i$ are the sum of records in treatments 1 and 2, the least squares estimators are

$$\begin{bmatrix} \hat{b}_1 \\ \hat{b}_2 \end{bmatrix} = \begin{bmatrix} n_1 & 0 \\ 0 & n_2 \end{bmatrix}^{-1} \begin{bmatrix} \sum_i x_{1i}y_i \\ \sum_i x_{2i}y_i \end{bmatrix}.$$

The least squares estimators of the treatment effects are the treatment means

$$\begin{bmatrix} \hat{b}_1 \\ \hat{b}_2 \end{bmatrix} = \begin{bmatrix} \frac{\sum_i x_{1i}y_i}{n_1} \\ \frac{\sum_i x_{2i}y_i}{n_2} \end{bmatrix}$$

with sampling variance

$$\text{Var}(\hat{b}_j|n_j) = \frac{\sigma^2}{n_j}, \quad j = 1, 2. \quad (7.5)$$

The ridge regression estimates are

$$\begin{aligned} \begin{bmatrix} \hat{b}_{rr1} \\ \hat{b}_{rr2} \end{bmatrix} &= \begin{bmatrix} n_1 + \lambda & 0 \\ 0 & n_2 + \lambda \end{bmatrix}^{-1} \begin{bmatrix} \sum_i x_{1i}y_i \\ \sum_i x_{2i}y_i \end{bmatrix} \\ &= \begin{bmatrix} \frac{\sum_i x_{1i}y_i}{n_1 + \lambda} \\ \frac{\sum_i x_{2i}y_i}{n_2 + \lambda} \end{bmatrix} = \begin{bmatrix} \frac{n_1}{n_1 + \lambda} \hat{b}_1 \\ \frac{n_2}{n_2 + \lambda} \hat{b}_2 \end{bmatrix}. \end{aligned}$$

Adding λ on the diagonal shrinks the estimates towards zero. The extent of shrinkage depends on the value of λ relative to the sample size.

The ridge estimator is biased:

$$\text{E}(\hat{b}_{rrj}|n_j) = \frac{n_j}{n_j + \lambda} \text{E}(\hat{b}_j|n_j) = \frac{n_j}{n_j + \lambda} b_j \quad (7.6)$$

because $\text{E}(\hat{b}_j|n_j) = b_j$. The bias is

$$\text{E}(\hat{b}_{rrj}|n_j) - b_j = b_j \left(\frac{n_j}{n_j + \lambda} - 1 \right). \quad (7.7)$$

The sampling variance is

$$\text{Var}(\hat{b}_{rrj}|n_j) = \frac{n_j}{(n_j + \lambda)^2} \sigma^2, \quad (7.8)$$

which reduces to the variance of the least squares estimator when $\lambda = 0$. For $\lambda > 0$, $\text{Var}(\hat{b}_{rrj}) < \text{Var}(\hat{b}_j)$. The smoothing process will reduce mean squared error if it gets rid of more variance than it adds bias. Specifically, the mean squared error is given by adding the square of (7.7) and (7.8). This gives

$$E\left[\left(\hat{b}_{rrj} - b_j\right)^2 | n_j\right] = b_j^2 \left(\frac{n_j}{n_j + \lambda} - 1\right)^2 + \frac{n_j}{(n_j + \lambda)^2} \sigma^2. \quad (7.9)$$

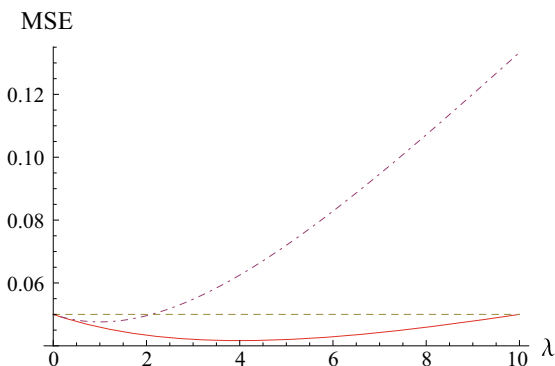
In the absence of shrinkage, $\lambda = 0$, the first term vanishes and second term is equal to (7.5). When λ is very large, the variance term approaches zero and the bias achieves its maximum value at b_j^2 (when the ridge coefficients are strongly regressed towards zero).

Figure 7.1 illustrates the trade-off variance versus bias of the ridge regression using (7.9). For a given population size, the MSE (7.9) has a minimum at $\lambda = \sigma^2/b^2$. The figure shows for $n = 20$, $\sigma^2 = 1$, that MSE of the ridge estimator, for a true $b_j = 0.5$, outperforms the MSE of the least squares estimator up to $\lambda < 10$. For a true $b_j = 1.0$, the ridge estimator does better up to $\lambda \approx 2$, with a minimum MSE at $\lambda = 1$.

Choice of Shrinkage Parameter

A natural choice of λ could be based on minimising the mean squared error of the estimator. Glancing at (7.6) and (7.8) and recalling that mean squared error is a function of the bias and the variance, indicates that the optimal choice of λ depends on the values of b and σ^2 . In a prediction context, since these are typically unknown, λ can be chosen using cross-validation.

Fig. 7.1 Mean squared error of the ridge regression estimator versus the shrinkage parameter λ . The population size is $n = 20$, the variance $\sigma^2 = 1$. Red: $b_j = 0.5$; Dot dashed: $b_j = 1.0$; Dashed: MSE of the least squares estimator ($\lambda = 0$), Eq. (7.5)



Bayesian View of Ridge Regression

In a Bayesian framework assume the regression coefficients are assigned the prior distribution $b|\sigma_b^2 \sim N(0, I\sigma_b^2)$ and the sampling model for the data is the normal process

$$y|X, b, \sigma_e^2 \sim N(Xb, I\sigma_e^2).$$

A standard result (Lindley and Smith 1972) sketched below states that the conditional posterior distribution of b in this random effects model is

$$b|X, \sigma_b^2, \sigma_e^2, y \sim N(\hat{b}_{rr}, (X'X + \lambda I)^{-1} \sigma_e^2), \quad (7.10)$$

where $\hat{b}_{rr}$ is the ridge regression estimator

$$\hat{b}_{rr} = (X'X + \lambda I)^{-1} X'y \quad (7.11)$$

and $\lambda = \sigma_e^2/\sigma_b^2$. Therefore the ridge regression estimator is equivalent to the posterior mode (or mean, given normality) of the regression coefficients in the following Bayesian model:

$$\text{Likelihood: } y|X, b, \sigma_e^2 \sim N(Xb, I\sigma_e^2)$$

$$\text{Prior: } b|\sigma_b^2 \sim N(0, I\sigma_b^2).$$

Numerically, the solution (7.11) is also the best linear unbiased predictor (BLUP) of b in this random effects model.

The ratio σ_e^2/σ_b^2 is equivalent to λ in the ridge regression context. In a fully Bayesian analysis the variance components are assigned prior distributions and can be inferred jointly with b .

Note

The proof of (7.10) is based on combining two quadratic forms, used repeatedly in the book (see also page 152). Using Bayes theorem, the posterior distribution of b is

$$\begin{aligned} p(b|X, \sigma_b^2, \sigma_e^2, y) &\propto p(y|X, b, \sigma_e^2) p(b|\sigma_b^2) \\ &\propto \exp\left[-\frac{1}{2\sigma_e^2} (y - Xb)'(y - Xb)\right] \exp\left[-\frac{1}{2\sigma_b^2} b'b\right] \end{aligned}$$

$$\begin{aligned}
&= \exp \left[-\frac{1}{2\sigma_e^2} \{ (y - Xb)' (y - Xb) + \lambda b'b \} \right] \\
&\propto \exp \left[-\frac{1}{2\sigma_e^2} (-2b'X'y + b'X'Xb + \lambda b'b) \right] \\
&= \exp \left[-\frac{1}{2\sigma_e^2} (-2b'X'y + b'(X'X + \lambda I))b \right] \quad (7.12)
\end{aligned}$$

where $\lambda = \sigma_e^2/\sigma_b^2$. Let

$$(X'X + \lambda I) \hat{b} = X'y, \quad (7.13)$$

and replace $X'y$ in (7.12) by $(X'X + \lambda I) \hat{b}$. Add and subtract $\hat{b}'(X'X + \lambda I) \hat{b}$ and keep terms in b only. Then a little manipulation yields

$$p(b|X, \sigma_b^2, \sigma_e^2, y) \propto \exp \left[-\frac{1}{2\sigma_e^2} (b - \hat{b})' (X'X + \lambda I) (b - \hat{b}) \right]$$

which is the kernel of a normal distribution with mean $\hat{b}$ and variance $(X'X + \lambda I)^{-1} \sigma_e^2$, as in (7.10).

There is a difference between the posterior variance of the Bayesian model and the variance of the ridge estimator (7.4). Using the singular value decomposition of X it is easy to show that

$$(X'X + \lambda I)^{-1} \sigma_e^2 = V \tilde{W} V' \sigma_e^2,$$

where $\tilde{W}$ is a $p \times p$ diagonal matrix with elements $(d_i^2 + \lambda)^{-1}$. On the other hand, as indicated in (7.4), the ridge regression estimator has sampling variance $\sigma^2 V W V'$, where W is a $p \times p$ diagonal matrix with elements $d_i^2/(d_i^2 + \lambda)^2$.

Example: Prediction Ability of the Ridge Estimator

The prediction ability of the ridge estimator is illustrated with simulated data ($n = 55$ observations) similar in structure to the one used in the example of page 320. In contrast to that example, the substitution effects b_ℓ of the $p_\ell = 10$ QTL are drawn from $N(0, \sigma_b^2 = 0.25)$ leading to a genomic variance (at the level of the operational model) equal to 2.5 squared units as explained below.

The data are analysed with two sets of operational models. In the first set, the QTL are part of the marker panel. In the second set the QTL are not part of the marker panel. In the latter scenario observations and covariates are uncorrelated. This creates an extreme case where the operational model has no prediction ability.

The *genomic value* of individual i is $g_i = x'_{i,\ell} b_\ell$, where $x'_{i,\ell}$ is the i th row of the $n \times p_\ell$ matrix X_ℓ of QTL genotypes. Due to centring and scaling the elements of X are random variables with mean 0 and variance 1.

At the level of the operational models

$$\text{Var}(g_i | x_i) = x'_i x_i \sigma_b^2$$

and the genomic variance is

$$\begin{aligned} \text{Var}(g_i) &= \sigma_g^2 = E_{x_i} [\text{Var}(g_i | x_i)] + \text{Var}_{x_i} [E(g_i | x_i)] \\ &= E_{x_i} [\text{Var}(g_i | x_i)] \\ &= \sigma_b^2 E_{x_i} (x'_i x_i) \\ &= p \sigma_b^2, \end{aligned} \tag{7.14}$$

where p is the number of markers of the operational model. When $p = p_\ell = 10$, $\sigma_g^2 = 10 \times 0.25 = 2.5$.

In view of (7.14) the shrinkage parameter in (7.13) is

$$\lambda = \frac{\sigma_e^2}{\sigma_b^2} = \frac{\sigma_e^2}{\sigma_g^2} p. \tag{7.15}$$

When σ_e^2 and σ_g^2 are assumed known and fixed with values 5 and 2.5, respectively, the shrinkage parameter is therefore set to increase linearly with the number of markers $p = 5, 10, \dots, 45, 50$ included in the operational model (7.13) with slope σ_e^2 / σ_g^2 .

The prediction ability of the ridge estimator using data simulated with the linear model is displayed in Fig. 7.2. When the operational model includes the first 10 covariates it coincides with the true model and the validating mean squared error reaches its minimum. Relative to the least squares predictor the increase in the validating mean squared error of the ridge predictor as a function of the number of covariates included in the operational model is attenuated by the shrinkage parameter that also increases as more covariates are included as indicated in (7.15). The training mean squared error shows a decline with increasing number of covariates. This decline is less pronounced than the one displayed in Fig. 6.1 for the least squares predictor.

Figure 7.3 displays the three components of the expected validating mean squared error. The variance of the ridge regression predictor of the validating record shows a steady increase as the number of covariates increases despite the increase of the penalty parameter that cannot compensate with increasing dimensionality of the predictive model. The bias of the predictor has a minimum when the operational model and the true model coincide and increases steadily when the operational model has more than 10 covariates. As the penalty parameter increases, the ridge-

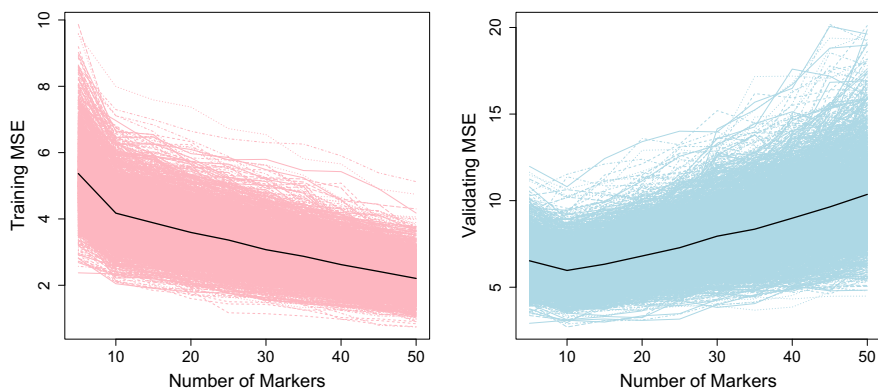
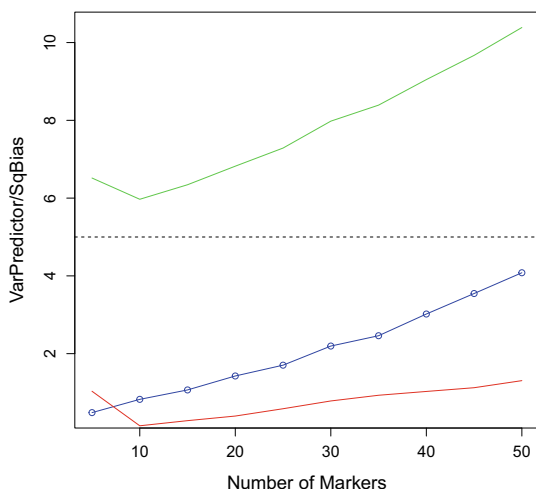


Fig. 7.2 Left (red): training mean squared error (6.67) and Right (blue): validating mean squared error (6.68), of the ridge estimator using simulated data generated with a linear model with 10 causal loci and analysed with operational linear models with increasing number of covariates that include the causal loci. Results from 2000 replicates; black lines: average over replicates

Fig. 7.3 The expected validating mean squared error (green) and its three components: expected squared bias (red), variance of predictor (blue) and residual variance (dashed lines). Data simulated using a linear model with 10 causal loci and analysed with operational linear models with increasing number of covariates, that include the causal loci



regression estimates are shrunk more heavily towards zero with a consequent increment in the bias. There is good agreement with the expected validating mean squared error and the mean validating mean squared error over the 2000 simulation replicated in Fig. 7.2.

The mean squared errors computed using validating and training data that result from fitting operational models with increasing number of covariates that do not include the causal loci are shown in Fig. 7.4. This is an example that mimics a trait whose expression is not affected by a genetic component: marker genotypes are uninformative about phenotype. Despite the lack of association between phenotypes and covariates the training mean squared errors show the expected decline due to

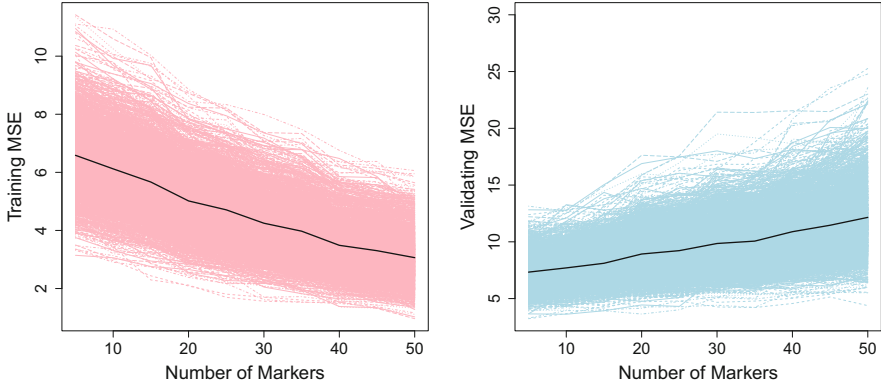
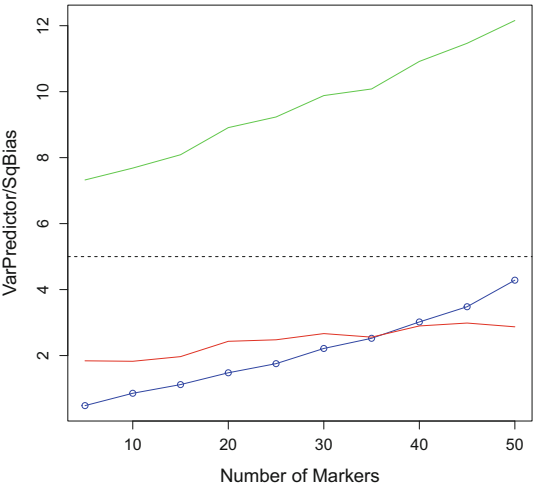


Fig. 7.4 Left (red): training mean squared error (6.67) and Right (blue): validating mean squared error (6.68), of the ridge estimator using simulated data generated with a linear model with 10 causal loci and analysed with operational linear models with increasing number of covariates that do not include the causal loci. Results from 2000 replicates; black lines: average over replicates

Fig. 7.5 The expected validating mean squared error (green) and its three components: expected squared bias (red), variance of predictor (blue) and residual variance (dashed lines). Data simulated with the linear model, 10 causal loci and analysed with operational ridge regression models with increasing number of covariates that do not include the causal loci



overfitting. The validating mean squared errors increase steadily as more covariates are added to the operational model. There is a slight squared bias contribution to this increase but most of it is driven by larger variances as the dimension of the operational model increases. Fig. 7.5 illustrates this setup.

Table 7.1 Average (over 2000 simulated training data sets) for the leave-one-out estimate of MSE (A) and average of 2000 training and validating records for the validating MSE (B), for three sizes of training and validating records, n and three different number of covariates included in the operational model, p . Figures in body of table correspond to A; B

n	Number of Covariates (p)		
	20	40	200
55	7.8; 7.5	10.4; 10.0	15.9; 15.4
100	6.5; 6.4	8.1; 8.0	11.1; 10.6
200	5.5; 5.5	6.1; 6.1	9.5; 9.1

Example: Leave-one-out Cross-validation and Shrinkage

This example illustrates the behaviour of expression (6.73), the leave-one-out cross validation mean squared error, as an estimator of the validating mean squared error, averaged over training and validating data, when it is applied using ridge regression.

The setup is similar to the one used to construct Table 6.1 on page 326. The number of training and validating records is $n = 55, 100$ or 200 and the number of markers included in the operational model is $p = 20, 40$ or 200 . The shrinkage parameter of the ridge regression estimator increases linearly with p , as indicated in expression (7.15). Table 7.1 displays the average over 2000 replications of training records for the estimates of leave-one-out cross-validation and the average over training and validating records, for the validating mean squared errors.

There is good agreement between the average estimates of leave-one-out cross-validation and the average validating mean squared error, even in situations where $p > n$ (in contrast with results in Table 6.1), with a slight tendency to overprediction.

7.2 The Lasso

Shrinkage estimators were shown to be obtained as the solution to an optimisation problem whose general form is

$$\hat{b} = \underset{\arg \min b}{\{L(y, b) + \lambda C(b)\}} \quad (7.16)$$

where $L(y, b)$ is a function that measures the lack of fit of the model to the data, $C(b)$ is a measure of model complexity and $\lambda \geq 0$ is a regularisation parameter controlling the trade-off between model fit and model complexity.

Let $b = (\beta_0, \beta) \in \mathbb{R}^{p+1}$. In ridge regression

$$L(y, b) = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2, \quad i = 1, \dots, n \quad (7.17)$$

and

$$\lambda C(b) = \lambda \sum_{j=1}^p \beta_j^2 \quad (7.18)$$

so that the ridge regression estimator is the solution (linear in y) to

$$\hat{b}_{rr} \underset{\arg \min \beta}{=} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}. \quad (7.19)$$

The term $\sum_{j=1}^p \beta_j^2$ (which does not include β_0) is known as the L_2 penalty. Relative to the classical least squares estimator, ridge regression reduces variability but can potentially cloud interpretation, as shrinkage is applied to all coefficients simultaneously. In ridge regression all coefficients “resemble each other” (shrinkage is to the same point or distribution, when interpreted from a Bayesian perspective) and none of the coefficients is set to 0.

The lasso (Tibshirani 1996, “least absolute shrinkage and selection operator”) has (7.17) in common with ridge regression but the L_2 penalty is replaced by the L_1 penalty, given by $\sum_{j=1}^p |\beta_j|$ and the lasso coefficients are the solutions (nonlinear in y) to

$$\hat{\beta}_{lasso} \underset{\arg \min \beta}{=} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}. \quad (7.20)$$

In contrast with ridge regression there is no closed form expression for the lasso.

The tuning parameter λ in (7.20) also controls the extent of the penalty. When $\lambda = 0$ the least squares solutions are obtained and when $\lambda = \infty$, $\beta_j = 0$ for all j . In between these two extremes, coefficients are shrunk but the effect of the L_1 penalty is to set some coefficients exactly to zero. As λ increases, more coefficients are set to zero (less variables are selected) and among the non-zero coefficients, shrinkage is stronger. The lasso is an attempt at combining features of subset selection and ridge regression simultaneously.

The lasso coefficients can also be obtained solving

$$\hat{\beta}_{lasso} \underset{\arg \min \beta}{=} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2$$

subject to

$$\sum_{j=1}^p |\beta_j| < t,$$

where $t \geq 0$ acts now as the tuning parameter. If t is chosen larger than $\sum_{j=1}^p |\hat{\beta}_j|$, where $\hat{\beta}_j$ is the least squares estimate, then the lasso estimates are the $\hat{\beta}'_j$ s. If t is smaller than $\sum_{j=1}^p |\hat{\beta}_j|$, shrinkage takes place and some β'_j s are set to zero.

In the context of prediction where the lasso is typically deployed, the shrinkage parameter λ is chosen using cross-validation.

If the columns of x are centred, the estimator of the intercept is $\hat{\beta}_0 = \bar{y}$. When y is also centred the model does not include an intercept term. In (7.20) the constraint treats all the coefficients equally and therefore it makes sense to scale x , as with ridge regression. For centred and scaled x , $\sum_i x_{ij} = 0$, $\sum_i x_{ij}^2 = n$.

The Subdifferential

A number of algorithms are available to obtain the solution to (7.20) that is a quadratic programming problem with linear inequality constraints. Before sketching one of the algorithms, the notion of the subderivative or subdifferential is briefly introduced. The subdifferential generalises the derivative to convex functions that may not be differentiable. This is necessary because the (convex) function defined by (7.20) does not have a derivative at the point $|\beta_j|$ for $\beta_j = 0$.

To illustrate, consider the (convex - i.e., “holds water”) absolute value function $f(x) = |x|$, $x \in \mathbb{R}$, where

$$|x| = \begin{cases} x, & x \geq 0, \\ -x, & x < 0. \end{cases} \quad (7.21)$$

For $x \neq 0$, the derivative of f with respect to x is $+1$ if $x > 0$ and -1 if $x < 0$. However for $x = 0$ (that defines the point where f achieves its absolute minimum) the derivative does not exist. However at $x = 0$ one can draw (many) lines that touch the function at $(0, f(0))$ that are everywhere below f except at the point $x = 0$. This collection of slopes tangent to the function at the point $x = 0$ is the subdifferential of f at the point $x = 0$.

One can proceed a little more formally as follows. A scalar c is a subgradient of a function f at the point x_0 if for all points x the following holds:

$$f(x) \geq f(x_0) + c(x - x_0)$$

for $x \in \mathbb{R}$ (the real line) and belonging to the domain of f . Then

$$c(x - x_0) \leq f(x) - f(x_0), \quad (7.22)$$

and the collection of all slopes c of subtangent lines is called the subdifferential of f at the point $x = x_0$. The set of subdifferentials c at x_0 is a nonempty closed interval

$[a, b]$ where a and b are the one-sided limits approached from the left and right of x , respectively, defined as

$$a = \lim_{x \rightarrow x_0^-} \frac{f(x) - f(x_0)}{x - x_0}, \quad (7.23a)$$

$$b = \lim_{x \rightarrow x_0^+} \frac{f(x) - f(x_0)}{x - x_0}, \quad (7.23b)$$

that satisfy $a \leq b$. Any c belonging in $[a, b]$ satisfies (7.22) and is a subdifferential of f at the point $x = x_0$, denoted $\partial f(x_0)$.

I return to the absolute value function $f(x) = |x|$ and find the set of subdifferentials at $x_0 = 0$. One way of approaching this is to use definition (7.22). Here $f(x) = |x|$, $f(x_0) = |0| = 0$ and using (7.22) with $x_0 = 0$

$$cx \leq |x|, \quad \text{for all } x \in \mathbb{R}.$$

Since the inequality holds for all $x \in \mathbb{R}$ it also holds for $x = \pm 1$. Then for $x = +1$

$$\begin{aligned} c(1) &\leq |1| = 1, \\ c &\leq 1 \end{aligned}$$

and for $x = -1$

$$\begin{aligned} c(-1) &\leq |-1| = 1, \\ -c &\leq 1, \\ c &> -1. \end{aligned}$$

Therefore c is any number defined in the closed interval

$$c \in [-1, 1]$$

and more generally, the subdifferential of f at $x = 0$ is written

$$\partial f(0) \subset [-1, 1].$$

For any x

$$\partial_x |x| = \begin{cases} -1, & \text{if } x < 0 \\ [-1, 1], & \text{if } x = 0 \\ 1, & \text{if } x > 0. \end{cases}$$

The limits of the closed set can also be obtained using (7.23). For the lower limit as x approaches 0 from the left

$$\begin{aligned} a &= \lim_{x \rightarrow 0^-} \frac{f(x) - f(x_0)}{x - x_0} = \lim_{x \rightarrow 0^-} \frac{|x| - |0|}{x - 0} \\ &= \lim_{x \rightarrow 0^-} \frac{|x|}{x} = \lim_{x \rightarrow 0^-} \frac{x}{-x} = \lim_{x \rightarrow 0^-} -1 = -1 \end{aligned}$$

and for the upper limit, as x approaches 0 from the right

$$\begin{aligned} b &= \lim_{x \rightarrow 0^+} \frac{f(x) - f(x_0)}{x - x_0} = \lim_{x \rightarrow 0^+} \frac{|x|}{x} \\ &= \lim_{x \rightarrow 0^+} \frac{x}{x} = \lim_{x \rightarrow 0^+} 1 = 1, \end{aligned}$$

where both cases make use of the fact that the limit of a constant (here the number -1 or 1) is the constant itself.

An Example with a Single Regression Parameter

Drawing from Friedman et al. (2007), to get a little intuition for the behaviour of the lasso, consider a linear model with p covariates, where $(p - 1)$ are fixed at some value and β_j (a scalar) is the only parameter to be estimated. The vector of covariates x is scaled and centred, so that $x'_j x_j = n$, and the observations y are expressed as deviations from the mean, so that $\bar{y} = 0$. Therefore the model does not include an intercept. Expression (7.20) now takes the form

$$\begin{aligned} f(\beta_j) &= \frac{1}{n} \sum_{i=1}^n (r_i - x_{ij} \beta_j)^2 + \lambda \sum_{k \neq j} |\beta_k| + \lambda |\beta_j| \\ &= \frac{1}{n} (r - x_j \beta_j)' (r - x_j \beta_j) + \lambda \sum_{k \neq j} |\beta_k| + \lambda |\beta_j| \end{aligned} \quad (7.24)$$

where $r_i = y_i - \sum_{k \neq j} x_{ik} \beta_k$ and the β'_k 's, $k \neq j$, are fixed at some value. The lasso estimate of β_j is the value of β_j that minimises (7.24). This gives

$$\partial f(\beta_j) = \begin{cases} \beta_j - x'_j r/n + \lambda, & \beta_j > 0, \\ \beta_j - x'_j r/n - \lambda, & \beta_j < 0, \\ [-\lambda, \lambda] - x'_j r/n, & \beta_j = 0. \end{cases}$$

Using $\hat{\beta}_j = x_j' r / n$ as the least squares estimate treating the residuals r as “data” and setting these equations equal to zero, leads to the lasso estimate of β_j

$$\beta_j^L = \begin{cases} \hat{\beta}_j - \lambda, & \hat{\beta}_j > \lambda, \\ \hat{\beta}_j + \lambda, & \hat{\beta}_j < -\lambda, \\ 0, & -\lambda \leq \hat{\beta}_j \leq \lambda, \end{cases}$$

or more compactly,

$$\beta_j^L = \begin{cases} 0, & \text{if } |\hat{\beta}_j| \leq \lambda, \\ \text{sign}(\hat{\beta}_j) (|\hat{\beta}_j| - \lambda), & \text{if } |\hat{\beta}_j| \geq \lambda. \end{cases} \quad (7.25)$$

The system (7.25) indicates that the smallest value for λ , such that all regression solutions $\beta^L = 0$, is $\max_j |\hat{\beta}_j|$, $j = 1, \dots, p$, the largest least squares estimate. Clearly, the choice of the tuning parameter λ has a strong influence on the behaviour of the lasso. The choice can be guided using cross-validation.

A General Algorithm to Obtain Lasso Solutions

The following general algorithm can be implemented to obtain lasso solutions in models with p correlated covariates. The algorithm is known as the path-wise coordinate descent algorithm (Friedman et al. 2007, 2010; Efron and Hastie 2016) and is based on updating regression coefficients one at a time as in the previous example. This is performed in an iterative fashion and the process is repeated until it converges to the lasso solution.

The algorithm has an external loop that runs over the number of iterates and an internal loop that runs over the number of covariates, updating one covariate at a time. This requires updating the residuals. A computationally efficient manner of updating residuals is as follows (Friedman et al. 2010). Consider a linear model with p covariates and write $Xb = (X_i, X_{-i})(b_i, b_{-i})'$. The residual including these p covariates is $r_1 = (y - X_i b_i - X_{-i} b_{-i})$ and the residual excluding the i th covariate is $r_0 = (y - X_{-i} b_{-i})$. Therefore

$$r_0 = r_1 + X_i b_i,$$

$$r_1 = r_0 - X_i b_i.$$

The kernel of the internal loop is simply

```
INITIALISE  $r_1 = y - \bar{y}$ ,  $b = 0$ 
LOOP OVER NUMBER COVARIATES  $i = 1, \dots, p$ 
     $r_0 = r_1 + X_i b_i$     this updates  $(y - 1\mu - X_{-i}b_{-i})$ 
    UPDATE  $b_i$           involves (7.25)
     $r_1 = r_0 - X_i b_i$     this updates  $(y - 1\mu - Xb)$ 
END LOOP
```

The following R-code illustrates the implementation of the algorithm with a toy example. The bottom of the code includes a call to the package GLMNET (Hastie and Qian 2016) as a test on the output. Details regarding this package are deferred to the Example on page 347.

```
# CODE0701
rm(list=ls()) # CLEAR WORKSPACE
set.seed(3037)
#install.packages("glmnet", .libPaths()[1])
library(glmnet)
n <- 100
p <- 100
rho <- 0.6
# GENERATE A MATRIX OF CORRELATED COVARIATES X
X <- matrix(data=0,nrow=n,ncol=p)
for (i in 1:n){
  X[i,1] <- rnorm(1)
  for (j in 1:(p-1)){
    X[i,j+1] <- X[i,j]*rho + rnorm(1)
  }
}
X <- scale(X)
beta <- rep(0,p)
betac <- rep(0,p)
beta <- sample(0:1,p,replace=TRUE,prob=c(2,1))
length(which(beta!=0))
```

```
## [1] 33
```

```
y <- X%*%beta + rnorm(n,sd=0.4)
y <- y - mean(y)
for(i in 1:p){ betac[i]=coef(lm(y~X[,i]))[2]}
lambda=max(abs(betac))*1
lambda
```

```
## [1] 0.3513841
```

```
niter <- 100
bL=matrix(nrow=niter,ncol=p)
bL[1,]=0 # initial lasso estimates set to zero
```

```

r1 <- y-mean(y)
for (i in 2:niter) {
  for(j in 1:p){
    r0 <- r1+X[,j]*bL[i-1,j]
    bLS <- crossprod(X[,j],r0)/n # LEAST SQUARES ESTIMATE
    if (abs(bLS) >= lambda){bL[i,j]<-sign(bLS)*(abs(bLS)-lambda)
    } else{
      bL[i,j] <- 0
    }
    r1 <- r0-X[,j]*bL[i,j]
  }
}
fm=glmnet(y=y,x=X,alpha=1,lambda=lambda)
# alpha=1: LASSO; alpha=0: RIDGE; 0<alpha<1: ELASTIC NET
# Number covariates included with GLMNET:
length(which(fm$beta!=0))

```

```
## [1] 41
```

```

# Number included with present code
length(which(bL[niter,]!=0))

```

```
## [1] 41
```

```

# PRINT A FEW ESTIMATES OBTAINED WITH GLMNET
round(fm$beta[which(fm$beta!=0)][1:7],3)

```

```
## [1] 0.245 0.017 0.283 0.314 0.357 0.594 0.345
```

```

# AND THE SAME WITH PRESENT CODE
round(bL[niter,which(bL[niter,]!=0)][1:7],3)

```

```
## [1] 0.244 0.014 0.304 0.304 0.366 0.582 0.345
```

Out of the $p = 100$ covariates in the full model lasso retains 41.

A Bayesian Interpretation of the Lasso

Tibshirani (1996) indicated that Lasso estimates of the elements of β can be interpreted as posterior mode estimates, when these regression parameters have independent and identical Laplace (i.e., double-exponential) priors of the form

$$p(\beta_j|\tau) = \frac{\tau}{2} \exp(-\tau |\beta_j|), \quad \tau > 0, \quad (7.26)$$

and observations are realisations from a normal process. More specifically, assume that centred data (no intercept required in the model) arise from

$$y|\beta, \sigma^2 \sim N(X\beta, I\sigma^2) \quad (7.27)$$

and

$$p(\beta|\tau) = \left(\frac{\tau}{2}\right)^p \exp\left(-\tau \sum_{j=1}^p |\beta_j|\right). \quad (7.28)$$

In (7.27), y is a column vector with n elements, X is $n \times p$ matrix and β is a column vector with p elements. Then the posterior density of β takes the form

$$p(\beta|\sigma^2, \tau, y) = (2\pi\sigma^2)^{-\frac{n}{2}} \exp\left[-\frac{1}{2\sigma^2} (y - X\beta)'(y - X\beta)\right] \left(\frac{\tau}{2}\right)^p \exp\left(-\tau \sum_{j=1}^p |\beta_j|\right)$$

and the logposterior density, including only those terms that contain β , is given by

$$\ln(p(\beta|\sigma^2, \tau, y)) = -\frac{1}{2\sigma^2} (y - X\beta)'(y - X\beta) - \tau \sum_{j=1}^p |\beta_j|. \quad (7.29a)$$

Multiplying out by $2\sigma^2$ the posterior mode results in the lasso solution

$$\begin{aligned} \hat{\beta}_L &= \underset{\arg \max b}{- (y - X\beta)'(y - X\beta) - \lambda \sum_{j=1}^p |\beta_j|} \\ &= \underset{\arg \min b}{(y - X\beta)'(y - X\beta) + \lambda \sum_{j=1}^p |\beta_j|} \end{aligned}$$

where $\lambda = 2\sigma^2\tau$. It is important to stress that the equivalence between the Bayesian lasso and the standard lasso occurs at the level of the modal value of the posterior distribution. The posterior mean is not equal to the posterior mode and does not produce the same lasso shrinkage of coefficients to zero. A fully Bayesian implementation of the lasso can be found in Park and Casella (2008).

7.3 An Extension of the Lasso: The Elastic Net

Some of the properties of the lasso are seen as shortcomings. For example, in the $p > n$ situation (n is the number of records and p is the number of covariates), lasso selects at most n covariates. This is not desirable when the analysis involves few observations and many predictors contribute to the response. Also, in models with

many highly correlated predictors, lasso tends to pick one arbitrarily and ignores the rest. On the other hand, ridge regression tends to shrink coefficients towards each other but all will be included in the model; none is set to zero. In the context of genomic models, there may be many highly correlated SNPs that cloud around unobserved causal variants. In such cases, lasso tends to choose one among the correlated set and eliminate the remaining predictors. Arguably, more of the signal can be captured by a linear combination of correlated predictors than by inclusion of only one. In contrast, ridge regression includes all the coefficients in the model despite the shrinkage, including the superfluous ones. The elastic net proposed by Zou and Hastie (2005) is a compromise between ridge regression and lasso and circumvents some of these limitations. The elastic net solves the following problem (Zou and Hastie 2005)

$$\min_{(\beta_0, \beta) \in \mathbb{R}^{p+1}} R_\lambda(\beta_0, \beta) = \min_{(\beta_0, \beta) \in \mathbb{R}^{p+1}} \left[\frac{1}{2N} \sum_{i=1}^N (y_i - \beta_0 - x_i' \beta)^2 + \lambda P_\alpha(\beta) \right] \quad (7.30)$$

where

$$P_\alpha(\beta) = \sum_{j=1}^p \left[\frac{1}{2} (1 - \alpha) \beta_j^2 + \alpha |\beta_j| \right].$$

$P_\alpha(\beta)$ is the elastic net penalty and is a compromise between ridge regression penalty ($\alpha = 0$) and the lasso penalty ($\alpha = 1$). The quadratic part of the penalty removes the limitation on the number of variables selected and encourages grouping effects. The L_1 penalty associated with the lasso generates a sparse model. The public package **glmnet** (Friedman et al. 2009) implemented in R allows the user to choose among ridge regression, lasso or elastic net and is illustrated as part of the example below.

7.4 Example: Prediction Using Ridge Regression and Lasso

This simulation example illustrates the prediction ability of ridge regression and lasso using two data sets that mimic two different genetic scenarios. The data in both scenarios consist of 1500 individuals and 1500 correlated marker genotypes at intermediate gene frequencies. The 1500 marker genotypes are in 75 independent blocks with 20 marker genotypes in each block. Marker genotypes are correlated within blocks and uncorrelated among blocks. The squared correlation between adjacent markers is approximately 0.60 and decays with marker distance.

The two data sets differ in the number of QTL. In the first set, 25 out of the 1500 markers are randomly chosen as QTL and in the second set this number is 250. The substitution effects are the same for all QTL within sets of data and are chosen such

that the equilibrium additive genetic variance (defined as $2 \sum_{QTL} p_i (1 - p_i) \beta_i^2$, where β_i is the allele substitution effect of the i th QTL, the same for all i) is equal to 10 squared units in both sets of data. The environmental variance is equal to 30 squared units and therefore the trait heritability is equal to 0.25. In the first data set each of the 25 QTL explains 1% of the phenotypic variance; in the second, each of the 250 QTL explains 0.1% of the phenotypic variance.

The 1500 individuals were divided into a training set of size 750 and a validating or testing set of size 750 and a ridge regression or lasso were fitted using the package **glmnet**. The performance of the predictive models was measured by the sample validating mean squared error (MSE) and the correlation between the predicted phenotypes (using estimates of substitution effects obtained using the training data) and observed phenotypes (from the validating data). In the data set with 25 QTL the lasso is expected to outperform ridge regression as the latter includes all 1500 markers in the model, including the redundant ones. In the data with 250 QTL the lasso limits the choice of the number of markers to be included in the predictive model within the block of correlated loci, failing in this way to capture an important part of the QTL (causal) additive genetic variance. This will lead to a relatively larger bias in the predictions. On the other hand, ridge regression includes all markers in the predictive model and captures a larger proportion of the QTL additive genetic variance. The possible increase in variance of the predictions is attenuated by a diminution of the squared bias, leading to a better predictive performance as measured by the MSE.

These expectations are borne out by the results, albeit drawn from one replication. With 25 QTL the MSE (correlation) of the lasso is 34.6 (0.30) and of the ridge regression 36.4 (0.20). The lasso includes 112 QTL in the model. With 250 QTL, the MSE (correlation) of the lasso is 43.6 (0.27) and of the ridge regression 40.9 (0.35). In this case the lasso includes 145 QTL.

These numbers can be put in perspective using the true value of the QTL substitution effects β_{true} , as predictions of the phenotypes in the validating (or test) data. The predictions are $X_{test}\beta_{true}$, where X_{test} is the matrix of QTL genotypes in the test or validating data. The MSE (correlation) is 28.5 (0.48). In this case, the MSE and the correlation measure, respectively, the $\text{Var}(y|X)$ and the heritability of the trait realised in the training sample of 750 individuals. In the population these parameters are 30 square units and $\sqrt{0.25} = 0.5$, respectively. This is as good a prediction as it gets!

An extract of the R-code (adapted from James et al. 2017, pages 251–255) used in the implementation of **glmnet** is shown below. Documentation about **glmnet** can be found in Hastie and Qian (2016) and in Friedman et al. (2010). In this example the function **cv.glmnet** is executed in a first step on the training data in order to find the value of the tuning parameter (best value of λ) of the lasso (using $\alpha = 1$ as an argument) or of the ridge regression (using $\alpha = 0$ as an argument) that optimises prediction ability measured by MSE. Execution of **predict** in a second step yields predictions of phenotypes in the testing data and the associated validation MSE (`mean((fm.pred-y.test)^2)`).

```

# Lasso solutions using package glmnet
#install.packages("glmnet", .libPaths()[1])
library(glmnet)
# UPLOAD MATRIX OF MARKER GENOTYPES X AND RECORDS y
n<-nrow(X)
Xlasso<- X
train=sample(1:nrow(Xlasso),nrow(Xlasso)/2)
test=(-train)
y.test=y[test] # y is the phenotypic data generated separately

# ***** FOR PREDICTION *****
# STEP 1

cv.out=cv.glmnet(Xlasso[train,],y[train],alpha=1)

plot(cv.out)
bestlam=cv.out$lambda.min
bestlam

# TO OBTAIN THE NUMBER OF NON-ZERO COEFFICIENTS CAN EXECUTE:

length(which(as.vector(coef(cv.out, s=bestlam))!=0))

# STEP 2

# Having obtained the best lambda fit the model
# on the testing data using this lambda to obtain predictions

fm.pred=predict(cv.out,s=bestlam,newx=Xlasso[test,])
mean((fm.pred-y.test)^2) # VALIDATION MSE
correl<-cov(fm.pred,y.test)/sqrt((var(fm.pred)*var(y.test)))
# AS GOOD AS IT GETS:
predtrue<-Xlasso[test,]%*%be # PREDICTION BASED ON TRUE MODEL
mean((predtrue-y.test)^2) # MSE USING TRUE MODEL
denominator <- sqrt(var(predtrue)*var(y[test]))
correltrue<-cov(predtrue,y[test])/denominator

```

7.5 A Bayesian Spike and Slab Model

The example on page 134, *A two component mixture model*, assumes that records are realisations from either of two unobserved mixture components and the objective is to assign each *observed* data point to a specific component. The mixture components are both Gaussian with a common variance that differ in mean value.

In the hierarchical setup discussed here, conditional on marker effects, records are realisations from a Gaussian distribution, but at the second level of the hierarchy, marker effects arise from either of two unobserved mixture components. In contrast with the example of page 134, here the objective is to allocate *unobserved* genetic marker effects to each mixture component. Genetic marker effects drawn from one

of the components have an influence on the trait of interest, while those drawn from the alternative mixture component have no detectable influence. The allocation is not unambiguous: it uses the laws of probability.

Various alternatives are available to model mixture components. One option is to assume that the mixtures are two Gaussians that differ in their means and/or variances. In this case the marker effects do not take the value zero, exactly, but their posterior means can be very small depending on the parameters of the Gaussian components. This makes it possible to distinguish the two mixture components. Another option takes the two-component mixture to be made up of a normal distribution and a degenerate distribution with a point mass at zero. This modelling option sets a number of marker effects exactly equal to zero, as in the lasso. An example of the first option implemented using Gibbs sampling is presented in George and McCulloch (1993) and an McMC implementation of the second can be found in Habier et al. (2011).

In the context of prediction, mixtures involving Gaussian distributions are likely to outperform the alternative that sets marker effects exactly equal to zero. This is so because a larger proportion of small effects variants that contribute to the phenotype are bound to be included in the final prediction model. On the other hand, the model that includes a point mass at zero constitutes arguably a better detection tool. However, this statement particularly in a prediction setup, must be interpreted in the light of the variance-bias trade-off, where size of the training sample, statistical properties of the prediction model, covariance structure of the marker/causal genotypes and the genetic architecture of the trait play important roles in defining the best choice (Hayes et al. 2010).

In the so-called spike and slab model that is considered here, the SNP effects are a priori mutually independent and drawn from a two-component mixture distribution made up of a normal distribution with mean zero and variance σ_b^2 (the slab), $N(0, \sigma_b^2)$ and a degenerate distribution with a point mass at zero, Δ_0 (the spike). Specifically, if b_i is the i th SNP effect and π denotes the a priori probability that this effect is non-zero, then

$$p(b_i | \sigma_b^2, \pi) = N(0, \sigma_b^2) \pi + \Delta_0 (1 - \pi), \quad (7.31)$$

where Δ_0 represents a point probability mass at zero.

This section provides a detailed development of a Bayesian-McMC implementation of the spike and slab model and concludes with an example in the context of prediction. The problem of detection of genetic markers that have an effect on the trait is deferred to an example on page 392.

The Mixture Model

A description of the model involves first the assumption that observed records y (vector of dimension $n \times 1$) are a realisation from the Gaussian process

$$y|\mu, b, \sigma^2 \sim N\left(1\mu + Xb, I\sigma^2\right), \quad (7.32)$$

where 1 is a vector of ones of dimension $n \times 1$, μ is a scalar, X is an observed matrix of dimension $n \times m$ representing genotypic codes of the SNP's (originally scored as 0, 1 or 2) and σ^2 is the conditional variance of y , given the location parameters of the model μ and b .

As a reminder, in the standard model ignoring the mixture, the m SNP effects are elements of the vector b (dimension $m \times 1$) that are *iid* realisations from a common distribution, assumed to be Gaussian with mean zero and variance σ_b^2 . Therefore

$$b|\sigma_b^2 \sim N\left(0, I\sigma_b^2\right). \quad (7.33)$$

The parameter σ_b^2 represents prior uncertainty about the SNP effect, the same for each SNP. The columns of matrix X are centred but not standardised. Then for each element of $X = \{x_{ij}\}$ we have

$$x_{ij}|p_j \sim (0, \text{Var}(x_{ij}|p_j)), \quad \text{Var}(x_{ij}|p_j) = 2p_j(1 - p_j).$$

A genomic variance is often associated with (7.32) and (7.33) and in the absence of a mixture is defined as (conditional on gene frequencies p_j , $j = 1, \dots, m$)

$$\begin{aligned} \sigma_g^2 &= \text{Var}(X'_i b) \\ &= E[\text{Var}(X'_i b|X_i) + \text{Var}[E(X'_i b|X_i)]] \\ &= E[X'_i X_i] \sigma_b^2 \\ &= \sigma_b^2 \sum_{j=1}^m E(x_{ij}^2), \quad \text{with } E(x_{ij}^2) = \text{Var}(x_{ij}) = 2p_j(1 - p_j). \end{aligned} \quad (7.34)$$

In these expressions X'_i is the i th row of matrix X , corresponding to the i th genomic value.

From (7.34),

$$\sigma_b^2 = \frac{\sigma_g^2}{\sum_{j=1}^m 2p_j(1 - p_j)}. \quad (7.35)$$

The term $2p_j(1 - p_j)$ can be approximated by computing the sampling variance among the elements of the j th column of X .

In order to introduce the mixture it is convenient to define the i th SNP effect as

$$b_i = \alpha_i \delta_i, \quad i = 1, \dots, m, \quad (7.36)$$

where the random variable α_i is distributed as $\alpha_i | \sigma_b^2 \stackrel{iid}{\sim} N(0, \sigma_b^2)$, independent of the binary $(0, 1)$ indicator variable δ_i that is a priori Bernoulli distributed with probability π :

$$\begin{aligned} \delta_i | \pi &\stackrel{iid}{\sim} Br(\pi), \quad i = 1, \dots, m, \\ \Pr(\delta_i = 1 | \pi) &= \pi, \quad \Pr(\delta_i = 0 | \pi) = 1 - \pi. \end{aligned} \quad (7.37)$$

This binary indicator variable specifies the a priori probability that a marker effect b_i is non-zero: $\Pr(\delta_i = 1 | \pi)$. This prior probability is the same for all markers. As indicated below, this is in contrast with the posterior probability that marker effect b_i is non-zero, $\Pr(\delta_i = 1 | y)$, distinct for each marker. One Monte Carlo estimator of this probability is given by the sampling average of the draws of δ_i from their posterior distribution.

The marginal prior density of b_i (marginalised over $[\delta_i | \pi]$) is

$$\begin{aligned} p(b_i | \sigma_b^2, \pi) &= \sum_{\delta} p(b_i, \delta_i | \sigma_b^2, \pi) \\ &= p(b_i | \delta_i = 1, \sigma_b^2, \pi) \Pr(\delta_i = 1 | \pi) \\ &\quad + p(b_i | \delta_i = 0, \sigma_b^2, \pi) \Pr(\delta_i = 0 | \pi) \\ &= N(0, \sigma_b^2) \pi + \Delta_0 (1 - \pi). \end{aligned} \quad (7.38)$$

retrieving (7.31). The mixture model specifies that when the unobserved indicator variable δ_i takes the value of 1, b_i is a realisation from a normal prior with mean zero and variance σ_b^2 ; when $\delta_i = 0$, $b_i = 0$ with probability 1. The probability π can be interpreted as the a priori proportion of marker loci with non-null effects.

The genomic variance under the mixture model is

$$\begin{aligned} \sigma_g^2 &= \text{Var} \left(\sum_{j=1}^m x_{ij} \alpha_j \delta_j \right) \\ &= E_{x\delta} \left[\text{Var} \left(\sum_{j=1}^m x_{ij} \alpha_j \delta_j | x_{ij}, \delta_j \right) \right] + \text{Var}_{x\delta} \left[E \left(\sum_{j=1}^m x_{ij} \alpha_j \delta_j | x_{ij}, \delta_j \right) \right] \\ &= E_{x\delta} \left[\sigma_b^2 \sum_{j=1}^m x_{ij}^2 \delta_j^2 \right] \\ &= \left[\sigma_b^2 \sum_{j=1}^m E(x_{ij}^2) E(\delta_j^2) \right] \\ &= \sigma_b^2 \pi \sum_{j=1}^m 2p_j (1 - p_j). \end{aligned} \quad (7.39)$$

Therefore, along the same lines as in (7.35),

$$\sigma_b^2 = \frac{\sigma_g^2}{\pi \sum_{j=1} 2p_j (1 - p_j)}. \quad (7.40)$$

The conditional distribution of the data takes the form

$$y|\mu, \alpha, \delta, \sigma^2 \sim N \left(1\mu + \sum_{i=1}^m X_i (\alpha_i \delta_i), I\sigma^2 \right), \quad i = 1, \dots, m \quad (7.41)$$

where X_i is the i th column of X .

The probability π can be assumed to follow a beta distribution, a priori, with user-tuned hyperparameters α and β ,

$$\pi|\alpha, \beta \sim Be(\alpha, \beta). \quad (7.42)$$

The hyperparameters can be chosen in order to assign a relatively higher probability mass to low values of π , reflecting prior information about the proportion of genetic markers likely to be associated with the trait.

The scalar μ can be assumed to follow an improper uniform distribution, a priori.

The two variance parameters can be assigned, a priori, scaled inverted chi-square distributions:

$$\begin{aligned} \sigma_b^2|S_b, v_b &\sim \chi^{-2}(S_b, v_b), \\ \sigma^2|S, v &\sim \chi^{-2}(S, v), \end{aligned}$$

where the S 's and the v 's are user-tuned hyperparameters.

With the above specification the prior density factorises as follows:

$$p(\mu, \alpha, \delta, \pi, \sigma_b^2, \sigma^2) \propto p(\alpha|\sigma_b^2) p(\delta|\pi) p(\pi) p(\sigma_b^2) p(\sigma^2) \quad (7.43)$$

and the posterior density takes the form

$$\begin{aligned} p(\mu, \alpha, \delta, \pi, \sigma_b^2, \sigma^2|y) &\propto p(y|\mu, \alpha, \delta, \sigma^2) \\ &\times p(\alpha|\sigma_b^2) p(\delta|\pi) p(\pi) p(\sigma_b^2) p(\sigma^2). \end{aligned} \quad (7.44)$$

McMC Implementation

The fully conditional posterior distributions (fcpd) are of standard form and the model can be implemented using Gibbs sampling. The required fcpd are

$$[\mu|D], [\alpha|D], [\delta|D], [\pi|D], [\sigma_b^2|D], [\sigma^2|D],$$

where as before (see page 244), D is a vector containing the observations y and all the parameters of the model except the one to be updated.

Updating $[\mu|D]$

From (7.44) the fcpd of the scalar μ is proportional to

$$\begin{aligned} p(\mu|D) &\propto p(y|\mu, \alpha, \delta, \sigma^2) \\ &\propto \exp\left[-\frac{1}{2\sigma^2} (y - 1\mu - Xb)' (y - 1\mu - Xb)\right] \end{aligned}$$

The quadratic form in this expression, as a function of μ , can be manipulated into

$$(y - 1\mu - Xb)' (y - 1\mu - Xb) \propto \mu 1'1\mu - 2\mu 1' (y - Xb). \quad (7.45)$$

Defining

$$(1'1) \hat{\mu} = 1' (y - Xb) \quad (7.46)$$

and replacing $1' (y - Xb)$ in (7.45) by $(1'1) \hat{\mu}$,

$$(y - 1\mu - Xb)' (y - 1\mu - Xb) \propto \mu 1'1\mu - 2\mu (1'1) \hat{\mu}.$$

Adding and subtracting $\hat{\mu} (1'1) \hat{\mu}$ and keeping terms containing μ only,

$$(y - 1\mu - Xb)' (y - 1\mu - Xb) \propto (\mu - \hat{\mu}) (1'1) (\mu - \hat{\mu})$$

and therefore,

$$p(\mu|D) \propto \exp\left[-\frac{1}{2\sigma^2} (\mu - \hat{\mu}) (1'1) (\mu - \hat{\mu})\right],$$

leading to the final result

$$[\mu|D] \sim N(\hat{\mu}, \sigma^2/n). \quad (7.47)$$

Updating $[\alpha_i|D]$

From (7.44) the fcpd is proportional to $p(y|\mu, \alpha, \delta, \sigma^2) p(\alpha|\sigma_b^2)$ where the first term is given by (7.41). There are two cases to consider. When $\delta_i = 0$ the product $\alpha_i \delta_i$ in (7.41) is equal to zero, the conditional distribution of the data is not a function of α_i and α_i is updated from

$$p(\alpha_i|D) \propto p(\alpha_i|\sigma_b^2)$$

the density of the normal distribution

$$[\alpha_i|D] \sim N\left(0, \sigma_b^2\right). \quad (7.48)$$

In this case α_i is updated from its prior distribution. When $\delta_i = 1$,

$$p(\alpha_i|D) \propto p(y|\mu, \alpha, \delta, \sigma^2) p(\alpha|\sigma_b^2)$$

which is the product of two normal densities: (7.41) and $N(0, \sigma_b^2)$. This fcpd has the same form as in (5.34), Example 3, *A regression model for correlated binary data* on page 242. Then, following the steps in that example involving the combination of two quadratic forms, leads to

$$[\alpha_i|D] \sim N\left(\hat{\alpha}_i, (X_i'X_i + k)^{-1}\sigma^2\right), \quad (7.49)$$

where $k = \sigma^2/\sigma_b^2$, X_i is the i th column of X and $\hat{\alpha}_i$ satisfies

$$(X_i'X_i + k)\hat{\alpha}_i = X_i'(y - 1\mu - X_{-i}b_{-i}). \quad (7.50)$$

In (7.50), X_{-i} is matrix X with the i th column deleted and b_{-i} is vector b (whose elements are $\alpha_i\delta_i$) with the i th element deleted.

Updating $[\delta_i|D]$

From (7.44) the fcpd of δ_i is proportional to $p(y|\mu, \alpha, \delta, \sigma^2) p(\delta|\pi)$. When $\delta_i = 0$, $\alpha_i\delta_i = b_i = 0$, $p(\delta|\pi) = \Pr(\delta_i = 0|\pi) = (1 - \pi)$ and

$$\Pr(\delta_i = 0|D) \propto \exp\left[-\frac{1}{2\sigma^2}(y - 1\mu - X_{-i}b_{-i})'(y - 1\mu - X_{-i}b_{-i})\right](1 - \pi). \quad (7.51)$$

When $\delta_i = 1$, $\alpha_i\delta_i = \alpha_i$, $p(\delta|\pi) = \Pr(\delta_i = 1|\pi) = \pi$ and

$$\Pr(\delta_i = 1|D) \propto \exp\left[-\frac{1}{2\sigma^2}(y - 1\mu - Xb)'(y - 1\mu - Xb)\right]\pi. \quad (7.52)$$

Let $(y - 1\mu - X_{-i}b_{-i})'(y - 1\mu - X_{-i}b_{-i}) = RSS0$, the residual sum of squares of the model without the i th SNP and let $(y - 1\mu - Xb)'(y - 1\mu - Xb) = RSS1$ denote the residual sum of squares of the model with all the m SNP's. Then the fcpd of the probability that $\delta_i = 0$ is the Bernoulli process with probabilities

$$\Pr(\delta_i = 0|D) = \frac{\exp\left[-\frac{1}{2\sigma^2}RSS0\right](1 - \pi)}{\exp\left[-\frac{1}{2\sigma^2}RSS0\right](1 - \pi) + \exp\left[-\frac{1}{2\sigma^2}RSS1\right]\pi} \quad (7.53)$$

and the complement

$$\Pr(\delta_i = 1|D) = \frac{\exp\left[-\frac{1}{2\sigma^2}RSS1\right]\pi}{\exp\left[-\frac{1}{2\sigma^2}RSS0\right](1-\pi) + \exp\left[-\frac{1}{2\sigma^2}RSS1\right]\pi}. \quad (7.54)$$

A Monte Carlo estimator of the (marginal) posterior probability that marker i is non-zero is given by

$$\hat{\varphi}_i = \hat{\Pr}(\delta_i = 1|y) = \frac{1}{l} \sum_{j=1}^l \delta_{ij}, \quad (7.55)$$

where δ_{ij} is the sampled value of δ for marker i at iterate j of the Gibbs sampler and l is the length of the Gibbs chain. To draw $\delta_{ij}^{[t]}$ at round t of the MCMC sampler, compute (7.53) and draw a random variable u from a uniform distribution in $(0, 1)$. If u is less than or equal to (7.53), set $\delta_{ij}^{[t]} = 0$; otherwise set $\delta_{ij}^{[t]} = 1$.

Remarks

1. Residuals $RSS0$ and $RSS1$ are efficiently updated as indicated on page 344.
2. In a computing environment (7.53) and (7.54) can be obtained as follows. Let $\theta_i = \Pr(\delta_i = 1|D)$ denote the fully conditional posterior distribution that marker i is non-zero. Then

$$\ln\left(\frac{\theta_i}{1-\theta_i}\right) = \frac{1}{2\sigma^2} (RSS0 - RSS1) - (\ln(1-\pi) - \ln(\pi)) = K_i \quad (7.56)$$

and

$$\theta_i = \frac{\exp(K_i)}{1 + \exp(K_i)}. \quad (7.57)$$

When the MCMC algorithm converges the draw at iteration j , $\theta_i^{[j]}$, is an extraction from the marginal posterior distribution $\Pr(\delta_i = 1|y)$. Therefore, the algorithm provides two characterisations of $\Pr(\delta_i = 1|y)$: one in terms of the mean of the Monte Carlo samples δ_{ij} , $\hat{\varphi}_i$ defined in (7.55) that yields a Monte Carlo point estimator of the mean of the posterior distribution $\Pr(\delta_i = 1|y)$; the other in terms of a Monte Carlo description of its complete posterior distribution through the samples $\theta_i^{[j]}$ via (7.57). Estimator (7.57) can be used to construct a Monte Carlo estimator of the marginal posterior distribution of the *false discovery rate*, a topic discussed in the next chapter on page 382.

Updating $[\pi|D]$

From (7.44) the density of the fcpd of π is proportional to $p(\delta|\pi) p(\pi)$ given by

$$\begin{aligned} p(\pi|D) &\propto \pi^{\sum_{i=1}^m \delta_i} (1-\pi)^{m-\sum_{i=1}^m \delta_i} \pi^{\eta-1} (1-\pi)^{\beta-1} \\ &= \pi^{\sum_{i=1}^m \delta_i + \eta - 1} (1-\pi)^{m-\sum_{i=1}^m \delta_i + \beta - 1}, \end{aligned}$$

which is the kernel of the density of a Beta distribution with shape parameters $\sum_{i=1}^m \delta_i + \eta$ and $m - \sum_{i=1}^m \delta_i + \beta$. Specifically

$$[\pi|D] \sim Be\left(\sum_{i=1}^m \delta_i + \eta, m - \sum_{i=1}^m \delta_i + \beta\right). \quad (7.58)$$

Updating $[\sigma_b^2|D]$

From (7.44) density of the fcpd of σ_b^2 is proportional to $p(\alpha|\sigma_b^2) p(\sigma_b^2)$,

$$\begin{aligned} p(\sigma_b^2|D) &\propto (\sigma_b^2)^{-\frac{m}{2}} \exp\left[-\frac{\alpha'\alpha}{2\sigma_b^2}\right] (\sigma_b^2)^{-\left(1+\frac{v_b}{2}\right)} \exp\left[-\frac{v_b S_b}{2\sigma_b^2}\right] \\ &= (\sigma_b^2)^{-\left(\frac{v_b+m}{2}+1\right)} \exp\left[-\frac{\alpha'\alpha + v_b S_b}{2\sigma_b^2}\right] \\ &= (\sigma_b^2)^{-\left(\frac{\tilde{v}_b}{2}+1\right)} \exp\left[-\frac{\tilde{v}_b \tilde{S}_b}{2\sigma_b^2}\right], \end{aligned} \quad (7.59)$$

where $\tilde{v}_b = v_b + m$ and $\tilde{S}_b = (\alpha'\alpha + v_b S_b) / \tilde{v}_b$. This is the kernel of a scaled inverse chi-square distribution with hyperparameters $\tilde{v}_b$ and $\tilde{S}_b$

$$[\sigma_b^2|D] \sim \chi^{-2}(\tilde{v}_b, \tilde{S}_b). \quad (7.60)$$

To obtain a sample from (7.60) draw from a chi-square distribution with $\tilde{v}_b$ degrees of freedom and the reciprocal of this number is multiplied by $\tilde{v}_b \tilde{S}_b = \alpha'\alpha + v_b S_b$.

Updating $[\sigma^2|D]$

From (7.44) the density of the fcpd of σ^2 is proportional to $p(y|\mu, \alpha, \delta, \sigma^2) p(\sigma^2)$,

$$\begin{aligned} p(\sigma^2|D) &\propto (\sigma^2)^{-\frac{n}{2}} \exp\left[-\frac{(y-1\mu-Xb)'(y-1\mu-Xb)}{2\sigma^2}\right] (\sigma^2)^{-(1+\frac{v}{2})} \\ &\quad \exp\left[-\frac{vS}{2\sigma^2}\right] \\ &= (\sigma^2)^{-\left(\frac{v+n}{2}+1\right)} \exp\left[-\frac{(y-1\mu-Xb)'(y-1\mu-Xb) + vS}{2\sigma^2}\right] \\ &= (\sigma^2)^{-\left(\frac{\tilde{v}}{2}+1\right)} \exp\left[-\frac{\tilde{v}\tilde{S}}{2\sigma^2}\right], \end{aligned}$$

where $\tilde{v} = v + n$ and $\tilde{S} = ((y - 1\mu - Xb)'(y - 1\mu - Xb) + vS) / \tilde{v}$. This is the kernel of a scaled inverse chi-square distribution with hyperparameters $\tilde{v}$ and $\tilde{S}$,

$$[\sigma^2 | D] \sim \chi^{-2}(\tilde{v}, \tilde{S}). \quad (7.61)$$

To obtain a sample from (7.61) draw from a chi-square distribution with $\tilde{v}$ degrees of freedom and the reciprocal of this number is multiplied by $\tilde{v}\tilde{S} = (y - 1\mu - Xb)'(y - 1\mu - Xb) + vS$.

Example: Spike and Slab Model

Using the example described on page 347, the spike and slab model is fitted to the same two data sets. As before, each data set is divided into 750 training (t) records and 750 validating (v) records. Let

$$W = (1 \ X),$$

$$\theta' = (\mu, b').$$

Using obvious notation a predictor of the validating records based on the spike and slab model is

$$\hat{y}_v = W_v \hat{\theta}, \quad (7.62)$$

where $\hat{\theta} = \hat{E}(\theta | X_v, y_t)$, the Monte Carlo estimate of the marginal posterior mean using the training data y_t . In common with the examples used for the ridge regression and the lasso, this is also a point prediction that does not account for uncertainty but is useful for comparing the prediction ability of various models. It addresses the general question: given the model, what is the predicted mean value of future data given $\hat{\theta}$, at the value of the covariates X_v ?

In the first data set with 25 QTL, the MSE (correlation) obtained fitting the spike and slab model is 34.1 (0.29). In the second data set with 250 QTL, the MSE (correlation) is 41.3 (0.34). The predictive performance of the spike and slab model is very similar to the lasso in the first data set and slightly better than the lasso and similar to the ridge regression in the second data set. The spike and slab model involves inferring the 4 parameters (over and above the SNP effects): the mean μ , the probability π and the two variance components σ_b^2 and σ^2 . These were estimated using the training data and not using cross-validation, as was the case with lasso and ridge regression. Arguably, a better predictive performance can be expected if these parameters were chosen via cross-validation.

One can exploit the flexibility afforded by the MCMC environment to compute the marginal posterior distribution of functions of the parameters of the model.

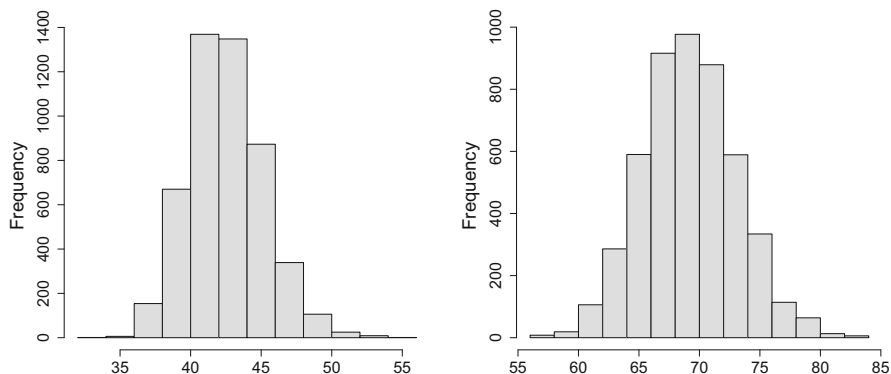


Fig. 7.6 Left: histogram of the Monte Carlo estimate of the marginal posterior distribution of the validating mean squared error accounting for posterior uncertainty of the parameters of the spike and slab model. Right: histogram of the Monte Carlo estimate of the marginal posterior distribution of the validating mean squared error accounting for posterior uncertainty of the parameters of the spike and slab model and for sampling variation of new validating data

Two such functions are mean squared errors that account for particular sources of uncertainty. An example is displayed in Fig. 7.6. The left panel is a histogram of the marginal posterior distribution of the validating mean squared error reflecting posterior uncertainty of the parameters of the Bayesian model. It is obtained by computing $\hat{y}_v^{[t]} = W_v \theta^{[t]}$ for each MCMC cycle t and the associated validation MSE.

The right panel of Fig. 7.6 shows the Monte Carlo estimate of the validating mean squared error calculated in yet another way. Here the vector of predictions of single validating records is drawn from

$$\hat{y}_v^{[t]} \sim N\left(W_v \theta^{[t]}, I \sigma^{2[t]}\right)$$

and in this manner, account is taken of the uncertainty in (θ, σ^2) and of the sampling variation of new observations. The issue of prediction uncertainty is revisited in Chap. 10.

The R-code `CODE1003` fits the spike and slab model; it can be found in the folder Codes at <https://github.com/SorensenS/SLGDS>.

An attractive property of the spike and slab model is that it may be useful for detection of QTL. Before illustrating this property, I take a detour from Prediction and provide an overview of the concept of false discovery rate in the next chapter. In general terms, the problem at hand is how to go about finding as many promising genetic markers as possible among thousands or millions observed, while incurring a user-defined expected proportion of false positives among the chosen genetic markers.

Chapter 8

Digression on Multiple Testing: False Discovery Rates



A classical single hypothesis test proceeds by specifying α , the probability of a significant result, given the null hypothesis ($H = 0$) is true. This is also known as the probability of a false discovery and more commonly of a type *I* error. If m hypothesis tests H_i are conducted, the so-called *family wise error rate* (FWER) is the probability of making one or more Type *I* errors among the family of hypothesis tests. When all m hypotheses are true nulls:

$$\begin{aligned} \Pr(\text{at least 1 false positive result in } m \text{ tests} | H_1 = 0, \dots, H_m = 0) &= \\ 1 - \Pr(\text{no false positive results in } m \text{ tests} | H_1 = 0, \dots, H_m = 0) &= \\ = 1 - (1 - \alpha)^m, &\quad (8.1) \end{aligned}$$

if the events are independent. With $\alpha = 0.05$ and $m = 50$, this yields

$$\Pr(\text{at least 1 significant result in } m \text{ tests} | H_1 = 0, \dots, H_m = 0) = 0.92$$

by chance alone. In terms of p -values, a threshold of 0.05 guarantees that the expected number of false positives over the whole family of m tests is less than or equal to $0.05m$, far too large for modern genomewide studies. Traditional methods for dealing with multiple testing call for adjusting α in some manner so that the number of false positive results over the whole family of m tests remains below a desired level. A popular method is the Bonferroni correction: for m hypotheses tests, each test is controlled so that the probability of a false positive is less than or equal to α/m . Then for the whole family of m tests, when all the m hypotheses are true nulls, the expected number of false positives is less than or equal to $m(\alpha/m) = \alpha$. This is a global test that addresses the question: is there any null hypothesis that is rejected? In the example above with $m = 50$ tests, each null hypothesis is rejected if its p -value is less than or equal to $(0.05/50) = 0.001$.

As shown below (see Note 1) the Bonferroni correction controls FWER at level $\leq \alpha$ for any number of true null hypotheses among the family of tests. Controlling FWER is useful when the number of hypotheses tested is small. However, in many modern genomewide studies where many hypotheses are tested, the Bonferroni correction can lead to a high rate of type *II* errors or false negatives (missing good candidates). The method tries too hard to make it unlikely that even one false rejection of the null is made.

An alternative that is particularly attractive for large-scale multiple testing is to identify as many significant features as possible, while keeping the proportion of false discoveries as low as desired *among these significant features*. This is the goal of the *false discovery rate* (FDR) that was first proposed from a frequentist perspective in a much celebrated paper by Benjamini and Hochberg (1995). Since its publication, the method has played a very important role in multiple testing and has undergone a number of refinements.

This chapter provides an overview of FDR starting with the classical approach proposed by Benjamini and Hochberg (1995). The method can also be anchored in a Bayesian framework and two approaches are presented. The first is an empirical Bayes approach (Efron et al. 2001) that uses as input summary statistics (such as *p*-values or *z*-values) embedded in a two-component mixture model. Secondly, a fully Bayesian hierarchical model is described and implemented with McMC. One output of this Bayesian model is a Monte Carlo estimate of the marginal posterior distribution of FDR.

The application of these methods is illustrated with two examples: one is based on the Gaussian, two-component mixture model introduced on page 134; the other is based on the spike and slab model introduced on page 349. An accessible proof of Benjamini and Hochberg's theorem is in an Appendix at the end of the chapter.

Note 1

The Bonferroni correction controls FWER at level $\leq \alpha$; this can be derived using Boole's inequality (Casella and Berger 1990). Assume that $H_1, H_2, \dots, H_m$ null hypotheses are to be tested and among these, m_0 (unknown) are true nulls. Let $p_1, p_2, \dots, p_m$ be their observed *p*-values (the random variable is P_i , $i = 1, 2, \dots, m$). The Bonferroni correction rejects the *i*th null hypothesis, $i = 1, 2, \dots, m$, when $p_i \leq \alpha/m$ and, in so doing, controls the FWER at level $\leq \alpha$. The proof is as follows (recall that for two events *A* and *B*, $\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B)$, where $\cup$ is the union symbol, $\cap$ is the intersection symbol and $\Pr(A \cap B)$ is either positive since it is a probability, or zero, if *A* and *B* are disjoint. The union of any number of events $A_1, A_2, \dots, A_n$, symbolised $\cup_{i=1}^n A_i = A_1 \cup A_2 \cup \dots \cup A_n$, is the event that at least one of the events $A_1, A_2, \dots, A_n$ is observed. Boole's inequality can be written $\Pr(\cup A_i) \leq \sum \Pr(A_i)$). The probability of at least one false positive result among the *m* tests is

$$\text{FWER}(\alpha) = \Pr\left[\cup_{i \in M_0} \left(P_i \leq \frac{\alpha}{m}\right)\right] \leq \sum_{i \in M_0} \Pr\left(P_i \leq \frac{\alpha}{m}\right) = m_0 \frac{\alpha}{m} \leq \alpha,$$

where M_0 denotes the set of indices of the m_0 true null hypotheses and P_i is uniformly distributed under the true null.

This result does not require knowledge of the number of true null hypotheses, nor does it assume independence of the tests.

An interesting alternative proof is as follows (Benjamini 2013). The starting point is as above, where m null hypotheses are to be tested and among these, m_0 (unknown) are true null hypotheses. Let R_i , $i = 1, 2, \dots, m$ denote the binary random variable equal to 1, if H_i is rejected (a discovery is made), or 0 if it is not rejected. Let $V_i = 1$, if $R_i = 1$ but H_i is a true null (a false discovery is made, or a type I error) and 0 otherwise. Then $R = \sum R_i$ is the number of null hypotheses rejected and $V = \sum_{i \in M_0} V_i$ is the number of null hypotheses rejected in error (number of false discoveries). Consider a procedure that tests each of the m null hypotheses separately at some level α_B . Then the expected number of false discoveries over the m tests is

$$E(V) = E\left(\sum_{i \in M_0} V_i\right) = \sum_{i \in M_0} E(V_i) \leq m_0 \alpha_B.$$

Therefore, to achieve a level of the expected number of false discoveries for the family of tests, $E(V)$, smaller than or equal to α , the Bonferroni method chooses $\alpha_B = \alpha/m$, as sketched out before, here for any number of true nulls m_0 .

The Bonferroni correction ensures that achieving $E(V) \leq \alpha$, also guarantees that the FWER = $\Pr(V \geq 1) \leq \alpha$. Indeed,

$$\begin{aligned} E(V) &= 0 \Pr(V = 0) + 1 \Pr(V = 1) + 2 \Pr(V = 2) + \dots + m \Pr(V = m) \\ &\geq 0 + 1 \Pr(V = 1) + 1 \Pr(V = 2) + \dots + 1 \Pr(V = m) \\ &= 0 + \Pr(V \geq 1) \leq \alpha. \end{aligned}$$

Therefore, when using $\alpha_B = \alpha/m$ for individual tests,

$$\text{FWER} = \Pr(V \geq 1) \leq E(V) \leq \alpha.$$

8.1 Introduction

Consider an experiment where m hypotheses are tested. This could be a GWAS experiment designed to isolate a few SNP's with an effect on a trait of interest out of a very large number m tested. Let δ represent the unobserved binary random variable that takes the value $\delta_i = 0$ if the i th hypothesis is a true null hypothesis ($H_i = 0$) (for example, the i th SNP has no effect on the trait), or $\delta_i = 1$ if the alternative hypothesis is true ($H_i = 1$) (the i th SNP has an effect on the trait), $i = 1, \dots, m$. Let the observed binary random variable $r_i = 1$, if the i th null hypothesis is rejected and 0 otherwise, according to a particular rule. Given observation y_i , the decision to reject the null could be based on the rule $r_i = I(y_i \geq y_t)$ for some threshold value y_t , or more commonly using p -values or “ z -values”. In a general setting the

rule can be to reject the null if a test statistic z falls in a rejection region A , $r_i = I(z_i \in A)$. The total number of rejections (an observed random variable) depends on the rule chosen and is $R = \sum_{i=1}^m r_i$ (equal to the number of discoveries or number of significant results and constitutes the discovery set). The number of true discoveries is $\sum_{i=1}^m r_i \delta_i$. The number of false discoveries or number of rejections in error is

$$V = \sum_{i=1}^m I(\delta_i = 0) \times I(r_i = 1) = \sum_{i=1}^m (1 - \delta_i) r_i. \quad (8.2)$$

The FWER is equal to $\Pr(V \geq 1)$ and the *false discovery proportion*, the proportion of false discoveries among the discoveries is

$$Q = \frac{V}{R}, \quad R > 0, \quad (8.3a)$$

$$= 0, \quad R = 0. \quad (8.3b)$$

Q is unobservable because V is unknown. It involves the unobserved parameters δ_i , the decisions taken on the basis of rule r_i and the data indirectly through the decisions.

A variety of approaches have been suggested to infer Q and three of these are outlined below. The first uses a frequentist paradigm and is the original false discovery rate of Benjamini and Hochberg (1995). It is defined as the expectation of (8.3). It treats hypotheses as not random. The second and third are based on the Bayesian paradigm where hypotheses are treated as unobserved random quantities. This chapter reviews an empirical Bayes approach (Efron et al. 2001) and presents a fully Bayesian model driven with MCMC.

8.2 Preliminaries

Before embarking on the subject two standard results are described; these will be used in the coming sections. The first concerns the distribution of p -values under a null hypothesis and the second the use of the inverse transform method to generate a random variable with a desired distribution.

1. Consider a random variable $X \in \mathbb{R}$ with probability density function $f_X(x) = \frac{d}{dx} F(x)$ and let $y = F(x)$, such that $x = F^{-1}(y)$, where the continuous function F (a cumulative distribution function, invertible and differentiable) maps x from the real line $\mathbb{R}$ to the $(0, 1)$ interval; $F : \mathbb{R} \rightarrow (0, 1)$, and F^{-1} is the inverse transformation, $F^{-1} : (0, 1) \rightarrow \mathbb{R}$. The probability density function of Y , $f_Y(y)$, is obtained using the theory of transformations. This gives

$$f_Y(y) = f_X(F^{-1}(y)) \left| \frac{dF^{-1}(y)}{dy} \right|, \quad Y \in (0, 1). \quad (8.4)$$

Noting that $\frac{dF^{-1}(y)}{dy}$ in (8.4) can be written

$$\begin{aligned}\frac{dx}{dF(x)} &= \left(\frac{dF(x)}{dx} \right)^{-1} \\ &= \frac{1}{f_X(x)}, \quad x = F^{-1}(y),\end{aligned}$$

and substituting in (8.4):

$$f_Y(y) = f_X\left(F^{-1}(y)\right) \frac{1}{f_X\left(F^{-1}(y)\right)} = 1, \quad Y \in (0, 1) \quad (8.5)$$

which is the uniform distribution $\text{Un}(0, 1)$.

The classical application of this result concerns the distribution of p -values under a null hypothesis. When the alternative hypothesis corresponds to small values of the test statistic X , the p -value is $F(x)$, where x is the observed value of the test statistic. The p -values are then uniformly distributed in the interval $(0, 1)$, whatever the (continuous) distribution of X might be. A similar argument holds when the alternative hypothesis corresponds to large values of the test statistic X .

2. The following algorithm generates a random variable with desired distribution F

- generate $U \sim \text{Un}(0, 1)$
- set $X = F_X^{-1}(U)$

and X has the desired distribution $F_X(\cdot)$. To see this note that

$$\begin{aligned}\Pr(X \leq x) &= \Pr\left(F_X^{-1}(U) \leq x\right) \\ &= \Pr(U \leq F_X(x)) \\ &= \int_0^{F_X(x)} 1 \, du \\ &= F_X(x).\end{aligned} \quad (8.6)$$

This method of generating the random variable is known as the inverse transform.

An illustration is the computation of z -values from uniformly distributed p -values. Assume two samples drawn from independent normally distributed populations with unknown means μ_i , $i = 1, 2$ and a common, unknown variance. Let $\bar{x}_i$ and $\widehat{\text{Var}}(\bar{x}_i)$ denote the sample mean and an estimate of the variance of the sample mean, respectively, calculated using n_i records, $i = 1, 2$. Then the observed so-called t -value is

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\widehat{\text{Var}}(\bar{x}_1 - \bar{x}_2)}}.$$

Given $H_0 : \mu_1 = \mu_2$, the random variable T whose realised value is t has a Student- t distribution with $n_1 + n_2 - 2$ degrees of freedom. For a level $\alpha = \Pr(\text{reject } H_0 | H_0)$ test, the null hypothesis is rejected if the p -value

-

$$p = \Pr(T \geq t | H_0) = 1 - F_0(t) \leq \alpha \quad (8.7)$$

in favour of $H_1 : \mu_1 > \mu_2$,

- if

$$p = \Pr(T \leq t | H_0) = F_0(t) \leq \alpha \quad (8.8)$$

in favour of $H_1 : \mu_1 < \mu_2$ and

- if

$$p = 2(1 - \Pr(T \leq |t| | H_0)) = 2(1 - F_0(|t|)) \leq \alpha \quad (8.9)$$

in favour of $H_1 : \mu_1 \neq \mu_2$, where F_0 is the cumulative distribution function of T under H_0 . In the last case, $\Pr(T \leq -t | H_0) = \Pr(T \geq t | H_0) = \frac{1}{2}\alpha$. Using (8.5), given H_0 , the random variable p is uniformly distributed $\text{Un}(0, 1)$. Now compute the z -value

$$z = \Phi^{-1}(p) \quad (8.10)$$

where Φ is the cumulative distribution function of the standard normal. It follows from (8.6) that

$$z \sim N(0, 1). \quad (8.11)$$

8.3 The Benjamini-Hochberg False Discovery Rate

Benjamini and Hochberg (1995) define the false discovery rate (FDR-BH) as the expectation of the false discovery proportion. From (8.3),

$$\begin{aligned} \text{FDR} &= E(Q) = E\left[\frac{V}{R} \mid R > 0\right] \Pr(R > 0) + 0 \Pr(R = 0) \\ &= E\left[\frac{V}{R} \mid R > 0\right] \Pr(R > 0), \end{aligned} \quad (8.12)$$

where the expectation is taken over replications of the experiment. When the number of unknown true null hypotheses m_0 is equal to the number of hypotheses tested m ,

$V = R$ and $Q = 1$, provided $R > 0$. Then $E\left[\frac{V}{R} \mid R > 0\right] = E[1 \mid R > 0] = 1$ and $E(Q) = \Pr(R > 0) = \Pr(V > 0) = \text{FWER}$. Therefore control of FDR implies control of FWER (in the so-called *weak* sense, i.e. when $m_0 = m$). On the other hand, if $m_0 < m$, $V \leq R$ and $\text{FDR} \leq \text{FWER}$. Therefore if FDR is controlled at level q say, FWER is larger than q and FDR is expected to result in a gain in power, on average, for any pattern of true and false null hypotheses.

A decision rule r_q controls FDR at level q , with q a pre-chosen value between 0 and 1, if

$$\text{FDR}(r_q) \leq q. \quad (8.13)$$

In other words, the expected proportion of false discoveries *among the null hypotheses rejected* is less than or equal to q . This is in contrast with multiple testing based on the traditional p -values, that leads to an expected proportion of false discoveries *among all the tests performed* less than or equal to q .

Benjamini and Hochberg (1995) show that the decision rule is as follows. Let $p_{(i)}$ be the i th element ($i = 1, 2, \dots, m$) among the ordered p -values $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$ corresponding to the m null hypotheses $H_{(1)}, H_{(2)}, \dots, H_{(m)}$. Let

$$r_q = \max \{i : p_{(i)} \leq (i/m)q\}, \quad \text{then,} \quad (8.14a)$$

$$\text{Reject } H_{(1)}, H_{(2)}, \dots, H_{(r_q)}. \quad (8.14b)$$

The rule (8.14) can be implemented as follows. Choose q . Start from the largest p -value with $i = m$ and check whether inequality (8.14a) holds. Move towards decreasing values of i one step at a time, checking for the inequality at each i . Stop at the i th value the first time $p_{(i)} \leq (i/m)q$ and set $r_q = i$. This is the largest i for which the inequality holds. Any i smaller than r_q for which the inequality holds is ignored. Then reject $H_{(1)}, H_{(2)}, \dots, H_{(r_q)}$, regardless of whether, for some $i < r_q$, $p_{(i)} > (i/m)q$. If no p -value satisfies inequality (8.14a), then no hypothesis test is declared significant. Common choices for q are 0.05 and 0.10.

Benjamini and Hochberg (1995) prove that if the p -values are independent of each other, then the rule r_q controls the expected false discovery proportion at level

$$\text{FDR}(r_q) = \pi_0 q \leq q, \quad (8.15)$$

where $\pi_0 = m_0/m$ is the proportion of true null hypotheses (the true proportion of SNP that do not have an effect on the trait). Therefore the FDR-BH controls the expected false discovery rate exactly at level $\pi_0 q$ and conservatively at level q . Since the number of true null hypotheses m_0 is not known, it is usual to specify the inequality in terms of expression (8.13). In most modern applications the ratio π_0 is close to 1, but it can be estimated as in Storey (2002), Benjamini et al. (2006) or adopting a likelihood or fully Bayesian approach. Benjamini and Yekutieli (2001) proved that the above theorem holds, with a slight modification of the rule (8.14), under certain type of stochastic dependence among the p -values.

For any given set of data, Q may or may not be less than or equal to q . Rather it is the expectation of Q that is smaller than or equal to q over conceptual replications of the experiment.

Expression (8.14a) can be written as

$$r_q = \max \left\{ i : \frac{mp_{(i)}}{i} \leq q \right\}, \quad (8.16)$$

where $mp_{(i)}$ is the expected number of false discoveries and i is the number rejected. The ratio $mp_{(i)}/i$ is an intuitive expression for the false discovery proportion. However, the ratio is not monotonically related to the p -values and moving to a lower p -value may result in a higher $mp_{(i)}/i$. Therefore, the discovery sets for a given FDR obtained using the rule (8.14) and (8.16) may differ. In order to align both and enforce monotonicity, Yekutieli and Benjamini (1999) suggest the following procedure. Let $k = \max \{ i : p_{(i)} \leq (i/m)q \}$. If such k exists reject $H_{(1)}, \dots, H_{(k)}$, as in (8.14a). The so-called q^* , the FDR-BH p -value adjustment is

$$q_i^* = q(p_i) = \min_{i \leq k} \left(p_{(k)} \frac{m}{k} \right), \quad \text{reject } H_{(i)} \text{ if } q_i^* \leq q, \quad (8.17)$$

that leads to the same decision as that based on (8.14a). In order to use the p -value adjustment one can proceed as follows. First, order all p -values from small to large, multiply each p -value by the total number of tests m and divide by the rank order i , $i = 1, \dots, m$. Start from the largest $p_{(i)}m/i$ at the bottom of the sequence and move upwards towards smaller values. Second, make sure that the resulting sequence is non-increasing: if it increases, set the increased value equal to the preceding value (repeatedly, until the whole sequence becomes non-increasing). Label the i th element of the sequence q_i^* . If q_i^* is smaller than or equal to q reject $H_{(1)}, \dots, H_{(i)}$. The kernel of the R-code to implement the algorithm is shown below.

```
# pv are the sorted p-values used as input
# smallest at the top
q=0.15 # SET FDR
qstar[m] <- pv[m] # Initialise qstar = largest p-value
minqstar <- qstar[m] # Initialise the minimum qstar
startloop <- m-1
for(i in startloop:1){ # loop towards smaller p-values
  qstar[i] <- pv[i]*(m/i)
  if(qstar[i]>=minqstar){
    qstar[i] <- minqstar # If new qstar (moving upwards towards
                        # smaller p-values) is larger than the
                        # previous qstar, set new qstar = old qstar
  }
  minqstar <- qstar[i] # update the minimum qstar
  if(minqstar <= q){break}
}
discovset <- i # SIZE OF DISCOVERY SET
```

Table 8.1 A subset of $m = 100$ null hypotheses indicating control of FDR at level $q = 0.15$. The columns are the index for the hypothesis ($i, i = 1, \dots, m$), the sorted p -values ($p_{(i)}$), rule (8.14) $((i/m)q)$, the values of (8.17), $(p_{(i)}(m/i))$ and the computation of q^* in column 5, (q_i^*)

i	$p_{(i)}$	$(i/m)q$	$p_{(i)}(m/i)$	q_i^*
15	0.01246737	0.0225	0.08311579	0.08311579
16	0.01860016	0.0240	0.11625099	0.11625099
17	0.02470245	0.0255	0.14530855	0.14071686
18	0.02532904	0.0270	0.14071686	0.14071686
19	0.02844954	0.0285	0.14973443	0.14973443
20	0.03269821	0.0300	0.16349105	0.16349105
21	0.04587966	0.0315	0.21847455	0.21847455
22	0.06910327	0.0330	0.31410579	0.30235174

As a simple illustration consider the computation of q^* and of the traditional FDR-BH rule (8.14) using as input a list of sorted p -values. A small subset of the relevant records is shown in Table 8.1.

A total of $m = 100$ null hypotheses are tested and the FDR-BH rule is applied for a value of $q = 0.15$ in (8.14). Starting from the largest p -value at $i = m = 100$, check rule (8.14) for $H_{(100)}, H_{(99)}, H_{(98)} \dots$ At $i = 19$, $p_{(i)}$ is smaller than $(i/m)q$ (coloured row in the table). Then reject $H_{(1)}, \dots, H_{(19)}$, regardless of whether, for some $i < 19$, $p_{(i)} > (i/m)q$. This rule controls FDR at level $q \leq 0.15$. Exactly the same discovery set is obtained if one chooses to use q^* . For $i = 19$, $q_{19}^* \leq q = 0.15$ which leads to the same rejection of $H_1, \dots, H_{19}$. These constitute the discovery set that is expected to include less than or equal to $19 * 0.15 \approx 3$ false discoveries. This is in contrast with the meaning attached to the p -value for $i = 19$ equal to 0.02845: in conceptual replications of the experiment where a total of m hypotheses are tested, the expected number of false discoveries among the hypotheses rejected at the cutoff 0.02845 or smaller is $0.02845 \times m$ or smaller.

Use of (8.16) in column 4 of the table generates a value of $p_{(i)}(m/i)$ for $i = 17$ that is larger than for $i = 18$. This is remedied in the computation of q_i^* shown in the 5th column of the table where q_{17}^* is set equal to q_{18}^* .

In contrast to p -values where thresholds of 0.01 or 0.05 are viewed as standards of evidence for a discovery, there is no accepted standard threshold for FDR control. FDR is used typically in an exploratory manner and thresholds are context-dependent or data-dependent. Thresholds of 20% or larger may be acceptable if the emphasis is on discovery of potentially useful covariates for further study.

Visual insight into the workings of the FDR-BH algorithm can be gained as follows. Consider an experiment that outputs m p -values; unknown to the analyst, a proportion π_0 are associated with the true null hypothesis and a proportion $\pi_1 = (1 - \pi_0)$ with the false null hypothesis. The distribution of p -values originating from the true null hypothesis is uniform $\mathcal{U}(0, 1)$, so that for p -value i , $\Pr(P_i \leq p_i | H_i = \text{true}) = \int_0^{p_i} du = p_i$. The distribution of p -values originating from the false null hypothesis tends to concentrate closer to 0 and is non-uniform; for p -value

i , $\Pr(P_i \leq p_i | H_i = \text{false}) = H(p_i)$. Marginally, the p -values are draws from the mixture distribution

$$\begin{aligned} \Pr(P_i \leq p_i) &= F(p_i) \\ &= \pi_1 \Pr(P_i \leq p_i | H_i = \text{false}) + \pi_0 \Pr(P_i \leq p_i | H_i = \text{true}) \\ &= \pi_1 H(p_i) + \pi_0 p_i. \end{aligned}$$

The sorted p -values are an approximation to the inverse distribution function $(F(p_i))^{-1}$. One can plot these sorted p -values against the indices (i/m) , $i = 1, 2, \dots, m$, and overlay a straight line with intercept equal to zero and slope equal to q . This represents a rejection line. Starting in the top-right corner (highest p -value) move towards the bottom-left corner and identify the first p -value that falls below the rejection line. Then reject all hypotheses whose p -values are smaller than or equal to this p -value (regardless of whether any of those p -values happen to be above the rejection line). The expected proportion of false positives among the rejected hypotheses (the discovery set) is smaller than or equal to q .

Figure 8.1 provides the visual representation of the FDR-BH algorithm for an experiment involving $i = 1, 2, \dots, m$, null hypothesis tests, $m = 100$, $q = 0.30$. The first blue circle, starting from the right of the figure, corresponds to the first sorted p -value that is below the rejection line $(i/m)q$. All hypotheses beyond and including this point are rejected and constitute the discovery set, despite the fact that the second blue circle starting from the right, corresponding to $i = 10$, happens to be larger than $(i/m)q$: 0.031 vs 0.030.

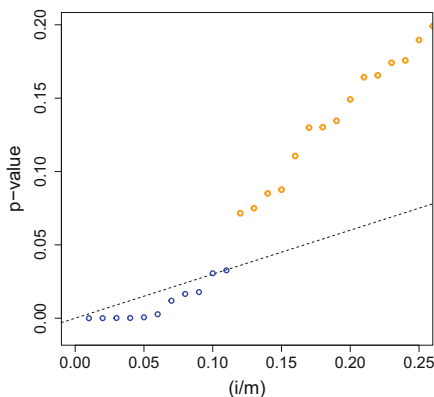


Fig. 8.1 Plot of a subset of ordered p -values from an experiment involving 100 identical hypothesis tests against the scaled index (i/m) , $m = 100$. There are 90 true null hypotheses and 10 non-null. The dashed line represents the line of rejection (intercept zero and slope $q = 0.3$). Orange circles represent ordered p -values from the non-rejected hypotheses; blue circles represent ordered p -values from the 11 rejected hypotheses and constitute the discovery set

8.4 A Bayesian Approach for a Simple Two-groups Mixture Model

Much of the material in this and the coming section has been adapted from Efron and Hastie (2016).

When the multiple testing problem is embedded in a Bayesian framework, the false discovery rate is expressed in terms of a posterior probability, rather than as the expectation (8.12) over repeated sampling of the data.

This section starts by considering a simple two-groups mixture model involving m identical hypotheses, of which m_0 are true null ($H = 0$) with probability $\Pr(H = 0) = \pi_0 = m_0/m$ and $m - m_0$ are non-null ($H = 1$) with probability $\pi_1 = 1 - \pi_0$. The probability π_0 can be construed as the a priori probability of null for each of the m hypotheses. The quantity m is observed, but m_0 is unknown. In this simple setup H is the only unknown random variable and the focus of enquiry. In modern genomewide studies involving million of genetic markers π_0 is much bigger than π_1 , say, $\pi_0 > 0.95$.

The random binary quantity H is Bernoulli distributed,

$$H \sim \text{Bernoulli}(\pi_1).$$

This is in contrast to the frequentist approach where H is considered non-random. In the simple two-groups mixture model considered here, the data are m test statistics such as “z values” associated with the m hypotheses. The density functions under the null and non-null hypotheses are

$$\begin{aligned} p(z|H = 0) &= p_0(z), \\ p(z|H = 1) &= p_1(z). \end{aligned}$$

Marginally, Z has the mixture distribution

$$p(z) = \pi_0 p_0(z) + \pi_1 p_1(z).$$

The mixture model is equivalent to the hierarchical model

$$p_j(z) = p(z|\delta = j), \quad \Pr(\delta = j) = \pi_j, \quad j = 0, 1,$$

where δ is a binary random variable representing the true state of nature.

Consider a fixed rejection region A . Let

$$F_0(A) = \Pr(Z \in A|H = 0) = \int_A p_0(z) dz,$$

$$F(A) = \Pr(Z \in A) = \int_A p(z) dz$$

denote probabilities of a set A under the null or marginal distributions.

When $Z = z$ is observed to belong in the region ($z \in A$) the hypothesis is declared as non-null, flagging a discovery. Then the probability that a false discovery has been made is

$$\text{BFDR}(A) = \Pr(H = 0 | z \in A). \quad (8.18)$$

$\text{BFDR}(A)$ is the Bayes false discovery rate for A . The subscript i denoting the i th hypothesis is not used because under the setup considered here (fixed rejection region A , the same for each test, rather than a data-dependent threshold), $\Pr(H_i = 0 | z_i \in A)$ is the same for all $i = 1, \dots, m$. A straightforward application of Bayes theorem yields

$$\begin{aligned} \Pr(H = 0 | z \in A) &= \frac{\Pr(Z \in A | H = 0) \Pr(H = 0)}{\Pr(Z \in A)} \\ &= \frac{F_0(A) \pi_0}{F(A)}, \end{aligned} \quad (8.19)$$

where

$$\begin{aligned} F(A) &= \Pr(Z \in A) = \Pr(Z \in A | H = 0) \Pr(H = 0) \\ &\quad + \Pr(Z \in A | H = 1) \Pr(H = 1). \end{aligned} \quad (8.20)$$

Expression (8.19) can be written as

$$\Pr(H = 0 | z \in A) = \frac{\Pr(\text{Type I error of } A) \pi_0}{\Pr(\text{Type I error of } A) \pi_0 + (\text{Power of } A) \pi_1}, \quad (8.21)$$

indicating that $\text{BFDR}(A)$ increases with increasing Type I errors and decreases with power (Storey 2003).

Storey (2003) shows that under the assumptions outlined in this section (m identical hypotheses H_i and test statistics Z_i arising from a random mixture of null and alternative distributions, with (H_i, Z_i) *i.i.d.* random variables, a fixed significant region A), the $\text{BFDR}(A)$ (8.18) can be written as

$$\text{pBFDR}(A) = \mathbb{E} \left[\frac{V(A)}{R(A)} | R(A) > 0 \right] \quad (8.22)$$

that he calls the *positive false discovery rate* and is a departure from (8.12) in that the expectation is conditional on the fact that at least one discovery has been made.

One proof is simple and instructive and it goes as follows (Storey 2003):

$$\begin{aligned} E \left[\frac{V(A)}{R(A)} | R(A) > 0 \right] &= \sum_{k=1}^m E \left[\frac{V(A)}{R(A)} | R(A) = k \right] \Pr(R(A) = k | R(A) > 0) \\ &= \sum_{k=1}^m E \left[\frac{V(A)}{k} | R(A) = k \right] \Pr(R(A) = k | R(A) > 0), \end{aligned}$$

where k depends on the particular outcome of the p -values in any one replication of the experiment. With the independence assumption, the number of false discoveries $V(A)$, given k hypotheses have been rejected, $R(A) = k$, is binomially distributed $Bi(k, \Pr(H = 0 | z \in A))$ with expectation $k \Pr(H = 0 | z \in A)$. Therefore the last line is

$$\begin{aligned} E \left[\frac{V(A)}{R(A)} | R(A) > 0 \right] &= \Pr(H = 0 | z \in A) \sum_{k=1}^m \Pr(R(A) = k | R(A) > 0) \\ &= \Pr(H = 0 | z \in A) \end{aligned} \quad (8.23)$$

since the last term sums to 1. It must be emphasised that the equivalence between the positive false discovery rate and the Bayesian false discovery rate holds when the hypotheses are treated as random variables.

The positive false discovery rate works by first fixing the rejection region A and then estimating the FDR, which is the opposite of the Benjamini and Hochberg's FDR-BH. However, notice that $pFDR(A)/\Pr(R(A) > 0) = FDR$. In large problems $\Pr(R(A) > 0) = 1$ and therefore numerically, $pFDR(A) = FDR$.

The BFDR (8.19) can also be interpreted as the expected proportion of false discoveries among all features declared significant. Indeed, the number of hypotheses rejected in error V is binomial $Bi(R, \Pr(H = 0 | z \in A))$. Therefore, the expected number rejected in error is $R \Pr(H = 0 | z \in A)$ and the number rejected is R . The ratio of the expected number rejected in error to the number rejected is the expected proportion of false discoveries among all features declared significant, equal to $\Pr(H = 0 | z \in A)$. As such, the BFDR is a global measure and does not provide information about each hypothesis. This is taken care of by the *local false discovery rate* (Efron and Tibshirani 2002) discussed below.

8.5 Empirical Bayes Estimation

Empirical Bayes estimation of false discovery rate is introduced in the rather idealised framework of the two-groups mixture model presented above, with m genes that are either null (an unknown number m_0 among these m have no effect on a response variable) with a priori probability $\pi_0 = m_0/m$, or non-null with a

priori probability π_1 . In the setting considered here, m is very large, m_0 is large and the objective is to reduce this vast set to a handful of scientifically interesting genes for further study.

A glance at (8.19) and (8.20) reveals that computation of BFDR requires knowledge of π_0 , F_0 and F . With very large datasets, these quantities can be estimated from the data and then the estimates can be used to approximate (8.19). This is the essence of the *empirical Bayes* approach, that in the context of false discovery was proposed by Efron et al. (2001) and summarised in Efron (2010). Efron and Tibshirani (2002) show the connection with the Benjamini and Hochberg FDR.

The method proceeds as follows. The starting point could be the availability of simple t -tests for the m genes. These t -values represent the “data”. One assumes

- F_0 is known and assumed to be the normal distribution $N(0, 1)$ that can be justified by transforming p -values derived from t -tests for each of the m hypotheses to z -values, as indicated in (8.11). Using the z -values as transformed “data”,

$$F_0(A) = \Pr(Z \in A | H = 0) = \int_A N(z|0, 1) dz. \quad (8.24)$$

- π_0 is “almost known” and in practice can be set equal to 1
- F is unknown but can be consistently estimated as follows:

$$\begin{aligned} F(A) &= \Pr(Z \in A) = \int_A p(z) dz \\ &= \int_{-\infty}^{\infty} I(z \in A) p(z) dz \\ &= E[I(z \in A)], \end{aligned} \quad (8.25)$$

then the empirical estimator of $E[I(z \in A)]$ is

$$\widehat{F}(A) = \widehat{\Pr}(Z \in A) = \frac{1}{m} \sum_{i=1}^m I(z_i \in A), \quad (8.26)$$

the proportion of the observed z_i values that fall in the rejection region A (does not depend on i and is a consistent estimator also when the Z 's are correlated).

The empirical Bayes estimator is

$$\widehat{\text{BFDR}}(A) = \frac{\widehat{\pi}_0 F_0(A)}{\widehat{F}(A)} \quad (8.27)$$

where $\widehat{\pi}_0$ is some estimate of $\pi_0 = m_0/m$ (for example, using maximum likelihood, unless it is set equal to 1 as a first approximation). This is an estimator of the

posterior probability of false discoveries among the features whose z -values fall in the rejection region A ; these constitute the discovery set.

One can also use p -values as “data”. When a fixed threshold α is chosen, $0 < \alpha \leq 1$, the features whose p -values are less than or equal to α are called significant and constitute the discovery set. Then following the same reasoning as above, with (8.24) now replaced by the uniform $\mathcal{U}(0, 1)$ distribution

$$F_0(\alpha) = \Pr(P_i \leq \alpha | H_i = 0) = \int_0^\alpha \mathcal{U}(u|0, 1) du = \alpha$$

with

$$F(\alpha) = \Pr(P \leq \alpha)$$

results in the Bayes FDR

$$\text{BFDR}(\alpha) = \Pr(H = 0 | P \leq \alpha) = \frac{\pi_0 F_0(\alpha)}{F(\alpha)}. \quad (8.28)$$

An empirical Bayes estimator is obtained using

$$\widehat{F}(\alpha) = \widehat{\Pr}(P \leq \alpha) = \frac{1}{m} \sum_{i=1}^m I(p_i \leq \alpha).$$

Then the empirical Bayes estimator of the posterior probability of a false discovery is

$$\widehat{\text{BFDR}}(\alpha) = \widehat{\Pr}(H = 0 | P \leq \alpha) = \frac{\widehat{\pi}_0 F_0(\alpha)}{\widehat{F}(\alpha)} \quad (8.29)$$

as in (8.27). A quantity closely associated to (8.28), where the fixed threshold α is replaced by p_i corresponding to hypothesis i , is

$$\text{BFDR}(p_i) = \Pr(H = 0 | P \leq p_i) = \frac{\pi_0 F_0(p_i)}{F(p_i)} \quad (8.30)$$

that is related to Storey’s q -value (Storey 2003) discussed below.

In contrast to the frequentist counterpart of Benjamini and Hochberg (1995), the derivation of the BFDR does not require independence of the hypotheses (Efron 2010). However, correlation among test statistics increases the (sampling) variance of $\widehat{\text{BFDR}}(p_i)$ (Efron 2010).

Connection with Benjamini-Hochberg False Discovery Rate

In order to establish a connection between (8.27) and FDR-BH, assume that rather than a fixed rejection region A , ordered p -values $p_{(1)} < p_{(2)} < \dots < p_{(m)}$ are available. The numerator of (8.30) is $\pi_0 p_{(i)}$. Using the ordered p -values an estimator of the denominator of (8.30) is

$$\hat{F}(p_{(i)}) = \frac{1}{m} \sum_{k=1}^m I(p_{(k)} \leq p_{(i)}) = \frac{i}{m}$$

and

$$\widehat{\text{BFDR}}(p_{(i)}) = \left(\frac{mp_{(i)}}{i} \right) \pi_0. \quad (8.31)$$

The threshold condition (8.16) of the FDR-BH (multiplied by π_0) can be written in terms of the empirical Bayes estimate

$$\widehat{\text{BFDR}}(p_{(i)}) = \pi_0 \frac{mp_{(i)}}{i} \leq \pi_0 q. \quad (8.32)$$

Expression (8.32) indicates that formulating rule (8.16) in terms of the empirical Bayes estimate of FDR, generates a discovery set that includes those cases where the empirical Bayes posterior probability that the hypothesis is a true null is smaller than or equal to q .

Under the same set of assumptions associated with (8.22), there is another way of connecting the positive false discovery rate pBFDR with the Bayesian BFDR and the classical frequentist FDR-BH (Storey 2003). The expected number of false discoveries V is $E[V(A)] = m \Pr(Z \in A | H = 0) \pi_0$ and the expected number of rejections R is $E[R(A)] = m \Pr(Z \in A)$. Then

$$\frac{E[V(A)]}{E[R(A)]} = \frac{\Pr(Z \in A | H = 0) \pi_0}{\Pr(Z \in A)} = \text{pBFDR}(A) \quad (8.33)$$

as in (8.19) and (8.23). When m is large, $E[V(A)]/E[R(A)] \approx E[V(A)/R(A)]$ and the connection with (8.12) is established. Storey (2003) remarks that despite result (8.33), pBFDR captures the joint behaviour of V and R , whereas $E[V(A)]/E[R(A)]$ does not.

8.6 Local False Discovery Rates

The Benjamini and Hochberg FDR-BH and the Bayesian FDR (8.19) are concerned with identifying a discovery set. Efron and Tibshirani (2002) define the *local false*

discovery rate

$$\text{fdr}(y_j) = \Pr(H_j = 0 | y_j) \quad (8.34)$$

that gives a probabilistic assessment of a false positive for the specific feature j given that y_j was observed, instead of providing a measure of the expected proportion of nulls among all rejections. The specificity issue is important here: the significance of the feature depends on the value y_j rather than on its inclusion in the significant region A as in (8.18). Although data y are used here, the common setup is to consider test statistics for each feature. In the context of the two-groups mixture model with densities p_0 and p_1 for the null and non-null densities of the test statistics, using Bayes theorem

$$\Pr(H_j = 0 | y_j) = \frac{p_0(y_j | H_j = 0) \pi_0}{p(y_j)} \quad (8.35)$$

where

$$p(y_j) = p_0(y_j | H_j = 0) \pi_0 + p_1(y_j | H_j = 1) (1 - \pi_0). \quad (8.36)$$

While the BFDR involves ratios of distribution functions as shown in (8.19), the local false discovery rate (8.35) involves ratios of densities. Therefore, the local false discovery rate can be interpreted as the BFDR for an infinitesimal rejection region around y_j . There is a relationship between BFDR and the local false discovery rate fdr , given by

$$\text{BFDR}(y_t) = \mathbb{E}[\text{fdr}(y) | y \geq y_t] \quad (8.37a)$$

$$= \pi_0 \frac{\int_{y_t}^{\infty} p_0(y_j | H_j = 0) dy_j}{\int_{y_t}^{\infty} p(y_j) dy_j} \quad (8.37b)$$

$$= \frac{F_0(A) \pi_0}{F(A)}, \quad (8.37c)$$

as in (8.27), with $A = [y_t, \infty)$ and where y_t is some chosen threshold value, so $\text{BFDR}(y_t)$ is the mean value of $\text{fdr}(y)$ for $y \geq y_t$. As before there is an implicit conditioning on p_0 , p_1 and π_0 , whereby these parameters can be estimated parametrically or nonparametrically as in Efron et al. (2001).

Empirical Bayes approach requires estimation of the terms in the right hand side of (8.35). Under the simple two-groups mixture model, when the y 's are interpreted as z -values, p_0 is simply the value of the standard normal density at y_j and π_0 can be set equal to 1 as an approximation. Getting a hand on the denominator is a little more delicate and requires density estimation unless the distribution under the non-null hypothesis is known. The topic is discussed in Efron (2010). The examples

starting on page 383 illustrate estimation by maximum likelihood and using a more general approach based on McMC-Bayes.

The information supplied by FDR-BH can be combined with fdr , with the former identifying a discovery set and the latter providing specific probabilities of false discovery for each feature in the discovery set.

Example

Assume that under the null hypothesis, $H = 0$, data are drawn from a standard normal distribution Φ , and under the non-null, $H = 1$, from the uniform $Un([-10, 10])$. Assume that the prior probability of the null hypothesis is $\pi_0 = 1/2$. Suppose the rejection region is $A = [2, \infty)$, so that all records falling in A are declared as discoveries. Then the Bayes false discovery rate (8.19) is

$$\begin{aligned} \text{BFDR}(A) &= \frac{\Pr(Z \in A | H = 0) \Pr(H = 0)}{\Pr(Z \in A)} \\ &= \frac{1 - \Phi(2)}{(1 - \Phi(2)) + \int_2^{10} Un(u | -10, 10) du} \\ &= 0.054. \end{aligned}$$

Using (8.35) the local false discovery rate is

$$\begin{aligned} \text{fdr}(2) &= \frac{\phi(z_i = 2)}{\phi(z_i = 2) + 1/20} \\ &= 0.52, \end{aligned}$$

where ϕ is the pdf of the standard normal and $Un(2 | -10, 10) = 1/20$. Given the rejection region A , the Bayesian FDR, $\text{BFDR}(A)$, indicates that the probability of a false discovery among the features declared significant is 5.4%. This includes implicitly all the data points equal to or larger than 2. On the other hand, for the single datum $z_i = 2$ that is on the border of the rejection region, the probability that it belongs to the null is 52%. When $z_i = 3$ this probability falls to 8%.

8.7 Storey's q -values

Consider an experiment where the estimator of the Bayes false discovery rate (8.18) together with a rejection region give rise to a discovery set. Imagine that a particular feature (for example a marker genotype) is included in the discovery set if the observed test statistic $T = t$ associated with this feature falls in the rejection region

$A = (-\infty, -c] \cup [c, \infty)$, $-\infty \leq c \leq \infty$, for a threshold c . The estimator of the Bayes false discovery rate provides a global measure informing on whole set of significant features. For example, for a threshold $c = 3$, say, the pBFDR involves tests based on an entire rejection region $|t_i| \geq 3$. However within the discovery set, a feature with $t_i = 5$ will be more significant than a feature with $t_i = 3.5$. One may be interested in a measure of significance attached to each feature in the discovery set that accounts for the fact that, within the discovery set, some features are more significant than others.

The q -value is designed to give each test statistic a measure of its significance in terms of the positive false discovery rate pBFDR. For a fixed rejection region A and an observed test statistic $T = t$, the q -value is formally defined as

$$q(t) = \min_{\{A: t \in A\}} [\text{pBFDR}(A)], \quad (8.38)$$

(Storey 2002, 2003). The q -value is the smallest pBFDR (or FDR) over all regions that reject H_i . This definition accommodates the fact that pBFDR does not necessarily increase as the significant threshold increases, in contrast with Type I error rates; it is conservative providing the minimum pBFDR to be expected among the features $1, \dots, i$ that constitute the discovery set. In other words, if the objective is to control the expected proportion of false positives at level $Y\%$, keep the features whose q -values are equal to or smaller than Y .

The computations of q^* 's defined in (8.17) and of Storey's q -values are closely related, as shown in (8.32). The empirical Bayes estimates of the q -values can be obtained using the same calculations as those for q^* except for the term π_0 . If an estimate $\hat{\pi}_0$ is available, q_i -value $= q_i^* \hat{\pi}_0$ and this aligns it with the BFDR (8.29) (the same code as shown below (8.17) can be used, with the modification immediately below the do-loop:

```
qstar[i] <- pv[i] * (m/i) * pi_0hat
```

where `pi_0hat` is an estimate of π_0).

When the test statistics are sorted p -values, the q -value is computed as

$$q(i) = \min_{j \geq i} (\text{FDR}(j))$$

For example in Table 8.1 where the test statistics are sorted p -values starting from the largest at the bottom of the list, the q_i^* is numerically equal to q_i -value ($1/\hat{\pi}_0$). Alternatively, $\hat{\pi}_0$ can be set equal to 1 and then q_i^* and q_i -value are numerically identical quantities. In this case, if the null hypothesis is rejected for features $i = 17, 16, \dots, 1$, then the figures in Table 8.1 indicate that $\text{FDR}(p_{(17)}) = p_{(17)}(m/17) = 0.14530855$; the minimum of all FDR values from rank 17 to m is 0.14071686, corresponding to $i = 18$. This is the estimate of the q -value for $i = 17$ and for the set comprising all the features that are this significant or more; we should expect at least 14.1% false positives among the $i = 17$ discoveries (this gives $17 \times 0.141 \approx 2$ false positives). On the other hand, the corresponding p -value

for $i = 17$ is 0.0247; this implies that, in conceptual replications of the experiment, the expected number of false positives among all the m tests with p -values cutoffs 0.0247 or smaller, is $0.0247 \times m$ or smaller.

To summarise, the q -value is a pBFDR analogue of the p -value. The latter is a measure of significance in terms of the false positive rate, whereas the former is a measure of significance in terms of the pBFDR. This conceptual difference is readily seen by writing

$$\begin{aligned} \text{p-value} &= \Pr(\text{Test Positive} \mid H_0) = \text{False positive rate} \\ \text{q-value} &= \Pr(H_0 \mid \text{Test Positive}) = \text{False discovery rate.} \end{aligned}$$

A $q = 0.05$, say, means that an estimated 5 % of the significant results **in the discovery set** are false discoveries. In contrast, in classical hypothesis testing, a rejection of H_0 at $\alpha = 0.05$ in a single test, does not mean that there is a 95 % probability that a discovery has been made. It rather means that an expected 5 % of **all the tests performed** are false discoveries. Further and also in contrast with the p -value, the q -value provides a measure of each marker's significance automatically accounting for multiple testing. This is so because the q -value incorporates information from the entire set of tests and conditions simultaneously on the fact that all features as or more extreme are included in the discovery set (Storey and Tibshirani 2003).

Incorporation of $\hat{\pi}_0$ in the calculations leads to a larger discovery set for the same expected proportion of false positives.

There is a **qvalue** package in R that performs FDR estimation from a collection of p -values or test-statistics. The package outputs estimates of q -values, proportion of true null hypotheses π_0 , and local false discovery rates (Storey and Bass 2021).

The false discovery rate, the local false discovery rate and the q -values can be obtained using a fully parametrised Bayesian model. This is the subject of the next section.

8.8 Fully Bayesian McMC False Discovery Rate

The Bayesian FDR (8.18) was derived within the stylised example of the two-groups mixture model and using z -values as data. The only unknown in this setting is the random quantity H and its associated “parameters”. Here an alternative development is followed that consists of inferring the parameters of (8.3) from their posterior distribution using the McMC implementation of the spike and slab model of Sect. 7.5. This allows more flexibility in modelling at the cost of heavier computations and parametrisation. Flexibility is important, particularly with sets of data involving a large number of highly correlated covariates and other explanatory variables found in modern genomic studies.

The spike and slab model is used for focusing the presentation but the approach remains valid for many massive multiple comparison problems. The indicator δ defined on page 363 is an ingredient of one estimator of the Bayesian FDR; it is inferred from its marginal posterior distribution $\Pr(\delta_i = 1|y)$, with the remaining parameters integrated out from the joint posterior distribution of the Bayesian model.

From (8.2) and (8.3)

$$\begin{aligned}\frac{V}{R} &= \frac{\sum_{i=1}^m (1 - \delta_i) r_i}{\sum_{i=1}^m r_i} \\ &= 1 - \frac{\sum_{i=1}^m \delta_i r_i}{R}.\end{aligned}\tag{8.39}$$

Conditionally on the data (indirectly, via the rule r) the only unknown quantity in (8.39) is δ that appears in the numerator, in contrast with the frequentist counterpart (8.12) that involves the expectation of a ratio of random variables. On taking expectations over the Bernoulli random variable $[\delta_i|y]$, given acceptance or rejection based on r , the fully Bayesian FDR is immediately obtained as

$$\frac{E(V|r)}{R} = 1 - \frac{\sum_{i=1}^m \Pr(\delta_i = 1|y) r_i}{R}\tag{8.40a}$$

$$= \frac{\sum_{i=1}^m \Pr(\delta_i = 0|y) r_i}{R}\tag{8.40b}$$

(Müller et al. 2007; Whittemore 2007). The i th term in the numerator, $\Pr(\delta_i = 0|y) r_i$ is the (fully) Bayesian local false discovery rate for the i th marker in the discovery set, so the Bayesian FDR is seen to be an average of the local false discovery rates in the discovery set, as in (8.37). It can also be interpreted as the expected proportion of false discoveries among the features in the discovery set which aligns it with (8.19).

In these expressions and the ones that follow y represents the complete data. When the conditional distribution of δ_i given y_i is not independent of the remaining y 's the fully Bayesian local false discovery rate $\Pr(\delta_i = 0|y) r_i$ can differ from $\Pr(\delta_i = 0|y_i) r_i$ as defined in (8.34).

The marginalisations required to obtain terms $\Pr(\delta_i = 0|y)$ can be approximated using McMC. An McMC-based estimator of the Bayesian FDR is obtained using the McMC output as follows. First, calculate

$$\hat{\varphi}_i = \hat{\Pr}(\delta_i = 1|y) = \frac{1}{l} \sum_{j=1}^l \delta_{ij},\tag{8.41}$$

where δ_{ij} is the sampled value of δ for marker i at iterate j of the Gibbs sampler and l is the length of the Gibbs chain. This yields a Monte Carlo estimator of the mean

of the posterior distribution $\widehat{\Pr}(\delta_i = 1|y)$. Then the Bayesian-McMC estimator of FDR is

$$\frac{\widehat{\mathbb{E}}(V|r, \widehat{\Pr}(\delta_i = 0|y))}{R} = \frac{\sum_{i=1}^m \widehat{\Pr}(\delta_i = 0|y) r_i}{R}, \quad (8.42)$$

where $\widehat{\Pr}(\delta_i = 0|y) = 1 - \widehat{\Pr}(\delta_i = 1|y)$. Expression (8.42) is a point estimator of the Bayesian FDR.

Posterior Distribution of False Discovery Rate

An alternative approach provides a Monte Carlo estimator of the complete posterior distribution of the fully Bayesian FDR. This distribution reflects posterior uncertainty of the parameters of the Bayesian model that propagates on to the FDR. The protocol is as follows. First execute the McMC sampler and store the draws from (7.57) at each cycle. On completion choose a discovery set; an informal approach could be based on selecting the marker genotypes whose Monte Carlo estimates of the posterior probability of non-zero effects is larger than a chosen value θ^* . These posterior probabilities could be obtained from φ in (8.41) or from the average of the draws from (7.57). The resulting discovery set includes R marker genotypes. Then, from the stored draws $\theta_i^{[j]}$ obtained from (7.57) *belonging to the discovery set*, where i refers to the marker genotype and j to the McMC cycle, compute at each McMC cycle the average local false discovery rate for all the marker genotypes in this discovery set. For example, at round j compute

$$\frac{\sum_{i=1}^m (1 - \theta_i^{[j]}) r_i}{R} \quad (8.43)$$

where m is the number of markers. The quantity (8.43) is the j th McMC draw from the posterior distribution of the Bayesian FDR for the discovery set chosen; here based on $r_i = I(\theta_i \geq \theta^*)$. Importantly, the discovery set is chosen only once and used repeatedly across all the draws of the McMC chain.

In the case of the two-component normal mixture model rules of the type $r_i = I(y_i \geq y_t)$ or $r_i = I(\theta_i \geq \theta_t)$ are illustrated in the examples below, where θ_i is the marginal posterior probability that δ_i is equal to 1, as in (7.57).

A recent McMC implementation of the Bayesian FDR was described by de los Campos et al. (2022).

8.9 Example: A Two-component Gaussian Mixture

I return to the two-component mixture model of the example on page 134. Data are realisations from two unobserved normal mixture components and the purpose of the analysis is to infer parameters and to allocate each data point to a specific component. The data could represent levels of a physiological marker in blood samples and one wishes to isolate those individuals that were exposed to a particular condition from those that were not. This condition is known to generate a higher mean level of the physiological marker. Data are simulated from a two-component Gaussian mixture, where the first component has a mean of zero (9000 observations), the second (1000 observations) a mean of 2.5 units and both a standard deviation of 1. Figure 8.2 shows a histogram of the 10,000 records (left) and plots the mixture distribution involving $N(0, 1)$ and $N(2.5, 1)$. The histogram reveals a certain amount of skewness due to the underlying mixture.

Information will be extracted from these data using five approaches:

1. a simple unadjusted test of the null hypothesis that each record is not different from zero;
2. a test of the null hypothesis that each record is not different from zero using a Bonferroni correction;
3. the classical Benjamini and Hochberg false discovery rate (FDR-BH);
4. an analysis of the two-component mixture model using maximum likelihood via de EM algorithm described on page 134;
5. a Bayesian MCMC analysis of the two-component mixture model described on page 257.

The last two analyses use fully parametrised models while the first three use p - or z -values as input.

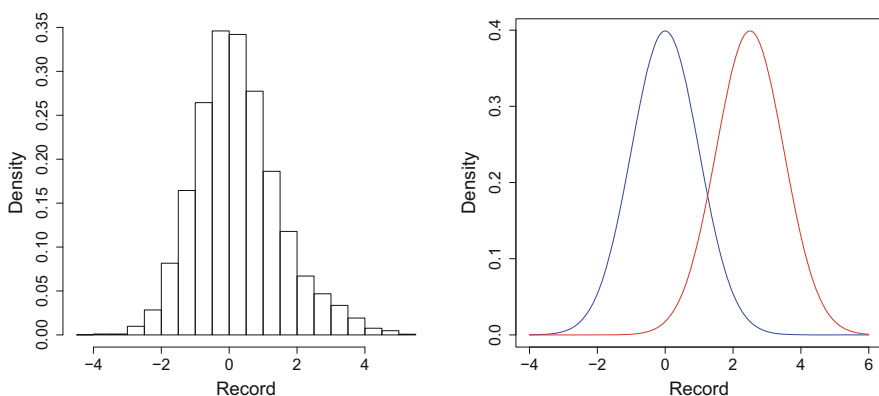


Fig. 8.2 Left: histogram of data. Right: plots of $N(0, 1)$, $N(2.5, 1)$

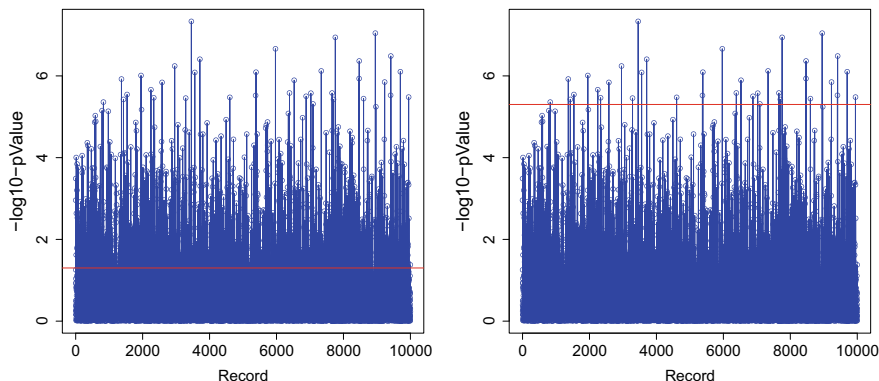


Fig. 8.3 Left: $-\log_{10} p$ -values corresponding to 10,000 records for testing the null hypothesis that each record is a draw from a single normal distribution with null mean. The horizontal red line is the $-\log_{10} p$ -value threshold corresponding to $p = 0.05$. Right: similar output where the horizontal red line is the Bonferroni threshold corresponding to $p = 0.05/10,000$

The results of the simple test and of the Bonferroni test are shown in Fig. 8.3 for a nominal probability of Type I error equal to $\alpha = 0.05$. The left plot displays the p -values against the 10,000 records and the threshold for rejecting the null hypothesis at $\alpha = 0.05$ is indicated by the horizontal line in red. This simple analysis, uncorrected for multiple testing, rejects H_0 for 1278 records out of which 470 are false discoveries. For $\alpha = 0.10$, H_0 is rejected for 1829 records, of which 928 are false discoveries. The plot on the right panel shows the results based on the Bonferroni correction using a per datum significant level equal to $0.05/10,000$. Only 35 records are declared significant with zero false discoveries among them. When the per datum significant level is $0.10/10,000$ the size of the discovery set increases to 41 with zero false discoveries.

It is interesting to look more closely at the type I and type II errors in these cases. A total of 9000 records were drawn from H_0 and 1000 from H_1 . The simple uncorrected test using $\alpha = 0.05$ results in 470 rejections of H_0 , given that H_0 is true. This yields a proportion of type I error equal to $470/9000 = 0.052$, very close to the nominal value of 0.05. The number of type II errors (false negatives) is 192 out of 1000 records drawn from $N(2.5, 1)$ or a power of the test of 81%.

For the Bonferroni test using a per datum significant level equal to $0.05/10,000$, the number of type I errors is now 0 out of 9000, but the number of observed false negatives (type II errors) is 965 out of 1000 and a power of 3.5%! (not too far off from the expected power of $pnorm(4.42, 2.5, 1, lower.tail = F) = 0.028$).

Maximum Likelihood via EM

The likelihood analysis of this problem implemented via the EM algorithm is based on the iterative system described in connection with the mixture model on page 134:

$$\begin{aligned}\hat{p}_{ij}^{[t+1]} &= \Pr\left(Z_i = j | \pi^{[t]}, \theta_j^{[t]}, y_i\right) \\ &= \frac{p_j\left(y_i | \theta_j^{[t]}\right) \pi_j^{[t]}}{\sum_j p_j\left(y_i | \theta_j^{[t]}\right) \pi_j^{[t]}},\end{aligned}\quad (8.44a)$$

$$\pi_j^{[t+1]} = \frac{1}{n} \sum_{i=1}^n \hat{p}_{ij}, \quad (8.44b)$$

$$\theta_j^{[t+1]} = \frac{\sum_{i=1}^n \hat{p}_{ij} y_i}{\sum_{i=1}^n \hat{p}_{ij}}, \quad (8.44c)$$

$$\sigma^{2[t+1]} = \frac{1}{n} \sum_{i=1}^n \sum_{j=0}^1 \hat{p}_{ij} (y_i - \theta_j)^2. \quad (8.44d)$$

The loop is over the number of iterates (t) and within each iterate, there is a loop over the number of records (i).

In this example the true value of the parameters of the mixture model are $\theta_0 = 0$, $\theta_1 = 2.5$, $\sigma^2 = 1$ and $\pi = 0.1$. When the system converges after approximately 70 iterations, the ML estimates are $\hat{\theta}_0 = 0.01$, $\hat{\theta}_1 = 2.5$, $\hat{\sigma}^2 = 1.02$ and $\hat{\pi} = 0.10$. A useful off-shot of the EM implementation are the terms $\hat{p}_{ij}$ that assign the probability that each datum i belongs to mixture component j ; this allows the calculation of a local false discovery rate along the lines in (8.35).

There is a connection between (8.44a) and the Bayesian counterpart. In a likelihood setting, (8.44a) is a conditional posterior probability, given ML estimates $\hat{\theta}$ and $\hat{\pi}$. This can be regarded as an approximation to the Bayesian marginal posterior probability $\Pr(Z_i = j | y)$ (see page 166 for an explanation).

The kernel of the R-code to implement the EM iterates is shown below.

```
CREATE DATA SET AND INITIALISE PARAMETERS
prob<-vector()
# INITIALISE THETA1 AND THETA0
y0<-which(y < mean(y))
theta0<-mean(y[y0])
y1<-which(y > mean(y))
theta1<-mean(y[y1])
iter<-100
resultML<-matrix(data=NA,nrow=iter,ncol=n+4)
for (i in 1:iter){
  for (j in 1:n){
    denom<-(dnorm(y[j],theta1,sqrt(v))*pi)
    +(dnorm(y[j],theta0,sqrt(v))*(1-pi))
    prob[j]<-(dnorm(y[j],theta1,sqrt(v))*pi)/denom
  }
  pi<-(sum(prob))/n
  theta1<-(sum(prob*y))/sum(prob)
  theta0<-(sum((1-prob)*y))/sum(1-prob)
```

```

v<-(sum(prob*(y-theta1)^2))/n+(sum((1-prob)*(y-theta0)^2))/n
resultML[i,]<-c(v,pi,theta1,theta0,prob)
}

#          COMPUTE FDR FOR ML ANALYSIS
cumavr<-rep(0,n)
fdr<-rep(0,n)
sortprob<-sort(-prob)
true<-which(dataset$label==T)
fals<-which(dataset$label==F)
for (i in 1:n){
  cumavr[i]<-mean(sortprob[1:i])
  fdr[i]<-1-cumavr[i] # THIS COMPUTES THE (CUMULATIVE) EXPECTED
# PROPORTION OF BFDR FOR EACH DISCOVERY SET OF SIZE 1 TO n
}

```

In the example it is assumed that all the non-null observations arise from a common distribution. This may not be the case, particularly if the “observations” are expression of genes or gene effects. Accounting for this requires adding extra distributions for the alternative hypotheses.

Bayesian McMC Implementation

The Bayesian computation of (8.40) is based on the McMC algorithm detailed in Sect. 5.5 on page 257. The true value of the parameters of the mixture model are $\theta_0 = 0$, $\theta_1 = 2.5$, $\sigma^2 = 1$ and $\pi_1 = 0.1$. The Monte Carlo estimates of the posterior means are $\hat{\theta}_0 = 0.01$, $\hat{\theta}_1 = 2.49$, $\hat{\sigma}^2 = 1.03$, $\hat{\pi}_1 = 0.10$, in very close agreement with the true values and with the likelihood estimates.

The kernel of the R-code to implement the McMC-based Bayesian mixture model is shown below.

```

CREATE DATA SET AND INITIALISE PARAMETERS
# READ CHAIN LENGTH
rep <- 1000
resmcmc <-matrix(data=NA,nrow=rep,ncol=5)
acprob <- rep(0,rep)
avr<-rep(0,n)
fd<-rep(0,n)
# READ HYPERPARAMETERS OF SCALE INVERSE CHI-SQUARE PRIOR FOR v
nu_v <- 4.5
Sv <- 1
# READ HYPERPARAMETERS OF THE BETA PRIOR FOR pi
a1 <- 1.5
a2 <- 10
##### GIBBS LOOP #####
for (i in 1:rep) {
  # print(i)
  # UPDATE THETA j
  theta1hat <- mean(y[z == 1])
  theta0hat <- mean(y[z == 0])
  v1 <- v / sum(z == 1)
}

```



```

v0 <- v / sum(z == 0)
theta1 <- rnorm(1, thetalhat, sqrt(v1))
theta0 <- rnorm(1, theta0hat, sqrt(v0))
# UPDATE THE n z's
K <-
  ((y-theta0)^2-(y-theta1)^2)/(2*v)-(log(1-pi)-log(pi))
un <- runif(n, 0, 1)
z <- ifelse(log(un / (1 - un)) <= K, 1, 0)
prob <- exp(K) / (1 + exp(K))
acprob <- acprob + prob
# UPDATE v
# ft <- sum((y - theta1) ^ 2)[which(z == 1)]
ft <- sum((y - theta1) ^ 2)[z == 1]
st <- sum((y - theta0) ^ 2)[z == 0]
Sc <- ft + st + (nu_v * Sv)
df <- nu_v + n
v <- Sc / rchisq(1, df)
# UPDATE pi
a1ny <- a1 + sum(z)
a2ny <- n - sum(z) + a2
pi <- rbeta(1, a1ny, a2ny)
resprob[i,] <- prob
resmcmc[i,] <- c(i, v, pi, theta1, theta0)
}
#####
postprob <- acprob/rep
#####
# COMPUTE FALSE DISCOVERY RATE USING McMC OUTPUT

ordpp<-order(-postprob)
sortpp<-postprob[ordpp]
localfdr <- 1-sortpp

for (i in 1:n){
  avr[i]<-mean(sortpp[1:i])
  fd[i]<-1-avr[i] # THIS IS THE FDR
### ALTERNATIVELY MORE COMPACTLY: MEAN OF LOCAL FDR
# IN DISCOVERY SET OF SIZE i
  fdloc[i] <- mean(localfdr[1:i])
# fd[] = fdloc[]
}

```

A summary of the main results using the 5 approaches is displayed in Table 8.2. The number and proportion of false discoveries reported in the table are true values based on the particular realisation of the simulated data. ML and Bayes-McMC give very similar results for all the features in the table. Increasing the nominal false discovery q from 0.05 to 0.20 leads to a larger discovery set (from 356 and 353 to 799 for FDR-ML and FDR-Bayes) and of course to a larger FDR. Notice that this increase in FDR is a consequence of a large increase in Type I errors (almost a tenfold increase) and a relatively smaller increase in power (almost twofold from 34% to 64%, approximately), as revealed by inspection of expression (8.21).

Table 8.2 Proportion of Type I and Type II errors, size of discovery set / true number of false discoveries (SizeDiscov/FD) and true proportion of false discoveries (FDR) for (1) the uncorrected p -values (Uncorrect), (2) the Bonferroni correction (Bonferroni). In (1) the figures in brackets correspond to nominal probability of Type I errors ($\alpha = 0.05$; $\alpha = 0.10$); in (2) these are ($\alpha = 0.05/10,000$; $\alpha = 0.10/10,000$), (3) the Benjamini-Hochberg FDR (FDR-BH), (4) the EM-likelihood analysis of the two-component mixture model (FDR-ML) and (5) the MCMC Bayesian analysis of the two-component mixture model (FDR-Bayes). In (3), (4) and (5), the figures in brackets correspond to nominal FDR $q = 0.05$; $q = 0.20$). Simulated data: 1000 records from $N(0, 1)$ and 9000 records from $N(2.5, 1)$

	Type I	Type II	SizeDiscov / FD	FDR
Uncorrect	(0.052; 0.10)	(0.19; 0.10)	(1278/470); (1829/928)	(0.37; 0.51)
Bonferroni	(0; 0)	(0.97; 0.96)	(35/0); (41/0)	(0; 0)
FDR-BH	(0.002; 0.016)	(0.65; 0.36)	(364/14); (781/144)	(0.04; 0.18)
FDR-ML	(0.002; 0.017)	(0.66; 0.35)	(356/14); (799/154)	(0.04; 0.19)
FDR-Bayes	(0.002; 0.017)	(0.66; 0.35)	(353/14); (799/155)	(0.04; 0.19)

For the given nominal FDR, FDR-BH is expected to lead to a little smaller size of discovery set for the same FDR than the other two implementations because it uses a more conservative approach (π_0 is set equal to 1 in expression (8.15), whereas it is included in the computations of the FDR in the likelihood and Bayesian approaches). This is only vaguely noticeable for a nominal $q = 0.20$ but the reverse is the case for $q = 0.05$. The explanation lies in the small overstatement of the true FDR by FDR-ML and FDR-Bayes observed at low FDR values. This is illustrated in Fig. 8.5, centre and rightmost panels. Moreover, the FDR estimated using likelihood or Bayesian methods are subject to the uncertainty (frequentist- for ML, posterior- for the Bayesian methods) of the estimates of parameters that feature in the computation of FDR.

The Bayesian MCMC implementation provides a measure of uncertainty for the FDR (conditional on the realisation of the data at hand and on the discovery set) using the draws from the Monte Carlo estimate of the posterior distribution of FDR (8.43). The Monte Carlo estimates of the posterior distributions of the FDR-Bayes for $q = 0.05$ and $q = 0.20$ are shown in Fig. 8.4 in the form of histograms. These lead to discovery sets of size 353 and 799, respectively. For the particular realisation of the data and discovery sets, the red vertical lines show the true (realised in the simulation) FDR. The Monte Carlo estimates of posterior means and of 95% posterior intervals of the FDR-Bayes estimates for $q = 0.05$, are 0.050(0.036; 0.065) and for $q = 0.20$, 0.200(0.163; 0.239).

In the case of FDR-BH, for the particular realisation of the data, the true FDR is 0.04 and 0.18 for $q = 0.05$ and $q = 0.20$, respectively (Table 8.2), column 5).

Over repeated sampling of the data, FDR-BH controls FDR at level smaller than or equal to q as indicated by expression (8.15).

Results of a little closer comparison between the Benjamini-Hochberg FDR-BH and the fully parametric likelihood or Bayesian analyses are shown in Table 8.3. The data are analysed with the FDR-BH approach using as input the 10, 000 sorted p -values. In the top half of the table, that corresponds to an expected FDR equal to

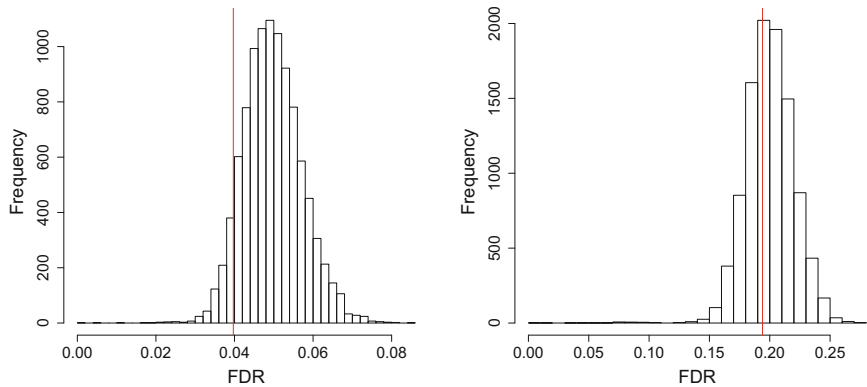


Fig. 8.4 Histograms of Monte Carlo estimates of posterior distributions of FDR. Left: nominal FDR of 0.05. Right: nominal FDR of 0.20. The vertical red lines are the true FDR (based on the simulation) for the particular realisation of the data

Table 8.3 A subset of results obtained by applying the Benjamini-Hochberg FDR-BH approach testing $m = 10,000$ hypotheses H_i with a false discovery rate set at level $q = 0.05$ (top 5 rows) or $q = 0.20$ (bottom 5 rows). The top coloured row corresponds to H_{364} (a discovery set of size $i = 364$) where condition (8.14) is satisfied and where $q_{i=364}^* \leq q = 0.05$. The bottom coloured row corresponds to H_{781} (a discovery set of size 781) where condition (8.14) is satisfied and where $q_{i=781}^* \leq q = 0.20$. The column headings are i , the index for the sorted p -values; Label=TRUE if the record is a draw from $N(2.5, 1)$, or FALSE, from $N(0, 1)$; $Pr(Z = 1|y)$ the ML estimate of the conditional probability that the record is a draw from $N(2.5, 1)$; FDR-ML, the ML estimate of the false discovery rate for the particular discovery set i and the remaining three columns as in Table 8.1

i	Label	Record	$Pr(Z = 1 y)$	FDR-ML	p -value	$(i/m)q$	q_i^*
362	TRUE	2.913	0.8612	0.05130	0.001789	0.0018	0.04936
363	TRUE	2.912	0.8609	0.05154	0.001794	0.0018	0.04936
364	TRUE	2.911	0.8608	0.05178	0.001796	0.0018	0.04936
365	TRUE	2.903	0.8583	0.05202	0.001846	0.0018	0.05058
366	TRUE	2.901	0.8579	0.05227	0.001855	0.0018	0.05068
779	TRUE	2.157	0.4978	0.1918	0.015491	0.016	0.1988
780	FALSE	2.156	0.4971	0.1922	0.01553	0.016	0.1991
781	TRUE	2.154	0.4960	0.1926	0.01560	0.016	0.1998
782	TRUE	2.151	0.4942	0.1930	0.01571	0.016	0.2007
783	FALSE	2.151	0.4942	0.1934	0.01572	0.016	0.2007

$q = 0.05$, at $i = 364$ condition (8.16) is satisfied (equivalent to condition (8.17)) and hypotheses $H_1, \dots, H_{364}$ are rejected leading to a discovery set of size 364. In the bottom half of the table that corresponds to an expected FDR equal to $q = 0.20$, the Benjamini-Hochberg FDR-BH rejects $H_1, \dots, H_{781}$. These rejection thresholds are coloured in green. The two columns with headings $Pr(Z = 1|y)$ and FDR-ML are derived from the maximum likelihood analysis (the corresponding figures from the Bayesian analysis are very similar). Column $Pr(Z = 1|y)$ displays the

estimated probability that record i is a draw from $N(2.5, 1)$ and $1 - \Pr(Z = 1|y)$ is the ML estimate of the local false discovery rate for record i . For example, for record $y_{i=364} = 2.911798$, the local false discovery rate is $1 - 0.8608679 \approx 0.14$ but the FDR (the average local false discovery rate in the discovery set of size 364 comprising $y_1, \dots, y_{364}$) is 0.052. The corresponding figures for $q = 0.20$ in the bottom half of the table are $1 - 0.4960449 \approx 0.50$ for the local FDR for record $i = 781$ and 0.19 for the FDR in the discovery set $y_1, \dots, y_{781}$. The last column of the table, q_i^* , is the Benjamini-Hochberg adjusted p -value (8.17) that in view of equivalence (8.31) is numerically identical to the empirical Bayes estimate of the FDR when π_0 is set equal to 1.

Knowledge of the true mixture proportions allows computation of the Bayesian FDR and the Bayesian local FDR. The following R-code can be used with the values in the table for $y_{364} = 2.911798$ and $y_{781} = 2.154387$:

```
# For q = 0.05:
FDR_THEOR05 <- (pnorm(2.911798,lower.tail=F)*0.9)/
  (pnorm(2.911798,lower.tail=F)*0.9 +
   pnorm(2.911798,2.5,lower.tail=F)*0.1)
LOCAL_THEOR05 <- (dnorm(2.911798)*0.9)/(dnorm(2.911798)*0.9 +
  dnorm(2.911798,mean=2.5,sd=1)*0.1)
FDR_THEOR05
```

```
## [1] 0.04537123
```

```
LOCAL_THEOR05
```

```
## [1] 0.1237628
```

```
# For q = 0.20:
FDR_THEOR20 <- (pnorm(2.154387,lower.tail=F)*0.9)/
  (pnorm(2.154387,lower.tail=F)*0.9 +
   pnorm(2.154387,2.5,lower.tail=F)*0.1)
LOCAL_THEOR20 <- (dnorm(2.154387)*0.9)/(dnorm(2.154387)*0.9 +
  dnorm(2.154387,mean=2.5,sd=1)*0.1)
FDR_THEOR20
```

```
## [1] 0.1810718
```

```
LOCAL_THEOR20
```

```
## [1] 0.4840697
```

This compares well with the ML estimates of the local FDR ≈ 0.14 and BFDR=0.052, for $q = 0.05$ and local FDR ≈ 0.50 and BFDR=0.19, for $q = 0.20$ reported in Table 8.3.

Examples of other output are shown in Fig. 8.5 for the likelihood analysis. The corresponding output from the Bayesian implementation is almost identical and is not shown. The panel on the left shows ML estimates of $\Pr(Z_i = 1|\hat{\pi}, \hat{\theta}_j, \hat{\sigma}^2, y_i)$

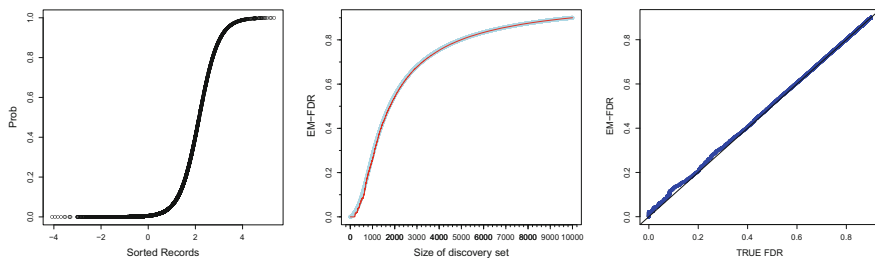


Fig. 8.5 Left: ML estimates of $\hat{p}_{i1} = \Pr(Z_i = 1 | \hat{\pi}, \hat{\theta}_j, \hat{\sigma}^2, y_i)$ versus the sorted records; Center: ML estimates of FDR (light blue) and true FDR (red) against size of discovery set; Right: plot of ML estimates of false discovery rates versus true false discovery rates. The 45° line is shown in black

for each of the 10,000 data points. Values of the data below 1 and larger than 3 are easily allocated to the correct mixture component since estimated probabilities are extreme, but there is ambiguity where the two components overlap. The panel on the centre shows FDR-ML (in light blue) and true FDR (in black) versus size of discovery set. The panel reveals a slight overestimation for low FDR values. This is also confirmed in the rightmost panel that displays a plot of FDR-ML versus true FDR. The 45° line is shown in black.

In the example the level q of the FDR-BH is set to either 0.05 or 0.20. This gives rise to particular size of the discovery set. A similar strategy can be applied for a likelihood or Bayesian-McMC implementation. An alternative is to choose various discovery sets based on Monte Carlo estimates of $\Pr(Z_i = j | y)$, obtain the associated FDR for each and decide which discovery set to keep based on this information.

The conclusion from this example based on independent data is that all three methods show similar performance. One attraction of the McMC Bayesian approach is the possibility to fit complex hierarchical models capable of accounting for various sources of variation and to generate marginal posterior distributions of the parameters of interest, or of functions of these such as FDR, in a single coherent analysis. This comes at the cost of more elaborate computations.

The following example introduces a new problem. How is false discovery rate affected when covariates are correlated? A classical example is the problem of distinguishing signals from noise in genome studies involving multiple testing of genetic markers that are in linkage disequilibrium with putative causal loci. The first part of the example deals with uncorrelated marker genotypes.

8.10 Example: The Spike and Slab Model with Genetic Markers

The problem discussed in this example concerns the detection of unobserved causal loci that have an effect on an observed continuous trait. This is studied in a variety of scenarios where in all cases, the data consist of 1500 individuals, 1500 markers and among these, 25 are randomly assigned as causal QTL. The phenotypic value for a datum is simulated by adding the contributions of a common mean, the effects of the 25 QTL and a normally distributed residual term. The operational models used for analysing the data included the 1500 marker genotypes. The objective is to identify the 25 QTL that are part of the marker panel.

The protocol is as follows:

1. The 1500 loci are independent (in linkage equilibrium).
 - (a) Heritability (h^2) is 0.10
 - (b) Heritability is 0.25
 - (c) Heritability is 0.50
2. The 1500 loci are correlated (in linkage disequilibrium)
 - (a) Heritability is 0.10
 - (b) Heritability is 0.25
 - (c) Heritability is 0.50

The 25 loci identified as QTL differ between the independent and the correlated groups, but are the same for the three heritability levels within each group.

The three levels of h^2 result in different levels of power to detect marker effects. This is achieved by setting the additive genetic variance of the trait equal to 4, 10 and 20 squared units and keeping the phenotypic variance (variance of the records, y) constant at 40 squared units. This results in effects for each of the 25 QTL (in standard deviation units of y) equal to 0.09, 0.14 and 0.20, for the three heritability levels.

The 1500 independent marker genotypes were generated by drawing each marker genotype from a binomial distribution $Bi(2, 0.5)$.

The 1500 correlated marker genotypes are in 75 independent blocks of size 20 each, where markers are correlated within blocks and uncorrelated among blocks. The squared correlation between adjacent markers is approximately 0.60 and decays with marker distance.

For all scenarios, the data are first analysed using a GWAS approach (one marker at a time) using a Bonferroni correction for the probability of a false discovery per test set equal to $0.05/1500$.

The p -values from the GWAS were used to compute false discovery rates (FDR) based on the standard Benjamini-Hochberg algorithm (FDR-BH). The levels of FDR were set equal to $q = 0.10$, $q = 0.05$, $q = 0.01$ for $h^2 = 0.10$, $h^2 = 0.25$ and $h^2 = 0.50$, respectively, that give rise to discovery sets of a given size.

Finally, the data were analysed with a fully Bayesian spike and slab model that outputs the marginal posterior distribution of FDR. In the Bayesian approach, the discovery sets were constructed either by including those markers for which the probability that the marker has an effect on the trait is larger than 0.5, $\Pr(\delta_i = 1|y) > 0.5$, or by choosing a particular size of discovery set. In the latter case the size chosen was equal to that found in the FDR-BH approach to allow comparison between both methods of computation. In all cases the true FDR realised in the simulated sample was compared to the result obtained from each of the three approaches.

Independent Marker Genotypes

The results of the GWAS using the Bonferroni correction are displayed in Fig. 8.6 for the three heritability levels and in the left block of Table 8.4. The number of loci

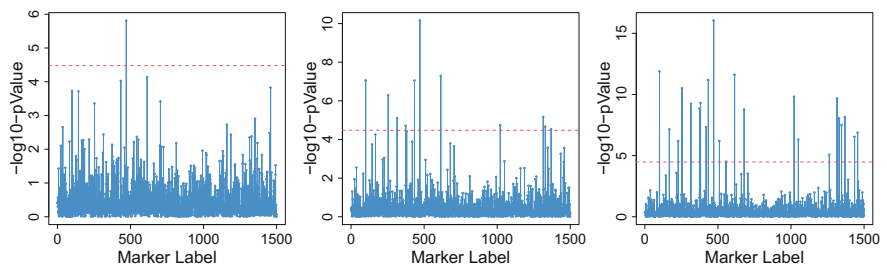


Fig. 8.6 GWAS analysis of 1500 individuals, 1500 **uncorrelated** marker genotypes with 25 causal loci and $h^2 = 0.10$ (Left), $h^2 = 0.25$ (Centre), $h^2 = 0.50$ (Right). The Y -axes shows $-\log_{10} p$ -values and the X -axes the marker labels. The markers that reach $-\log_{10} p$ -values beyond the Bonferroni threshold - horizontal line in red, set at $-\log_{10}(0.05/1500)$ are declared as discoveries (1, 11 and 23 out of 1500 markers in the Left, Centre and Right panels, respectively)

Table 8.4 Size of discovery set / true (observed) number of false discoveries, in one simulated sample, for uncorrelated and correlated marker genotypes, using (1) the Bonferroni correction (Bonferroni) with nominal probability of Type I error per marker ($\alpha = 0.05/1500$), (2) the Benjamini-Hochberg FDR (FDR-BH) and (3) the McMC Bayesian analysis (FDR-Bayes). The nominal FDR for FDR-BH is $q = 0.10$ for $h^2 = 0.10$, $q = 0.05$ for $h^2 = 0.25$ and $q = 0.01$ for $h^2 = 0.50$. 25 causal loci

	Uncorrelated			Correlated		
h^2	0.10	0.25	0.50	0.10	0.25	0.50
Bonferroni	1/0	11/0	23/0	4/0	27/12	50/25
FDR-BH	8/3	20/3	23/0	27/14	50/27	70/45
FDR-Bayes ^a	8/3	20/2	23/0	27/14	50/25	70/45
FDR-Bayes ^b	10/5	25/5	28/3	13/4	20/1	27/2

^a The discovery set for FDR-Bayes is set equal to that of FDR-BH
^b The discovery set is obtained by choosing the markers with $\hat{\Pr}(\delta_i = 1|y) > 0.5$

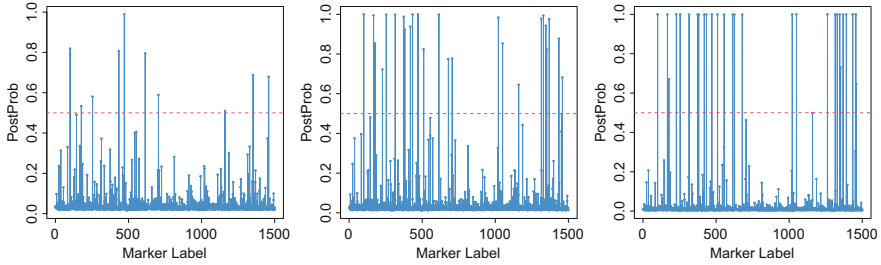


Fig. 8.7 Bayes-McMC implementation of a model with 1500 individuals, 1500 **uncorrelated** genetic markers of which 25 are chosen as QTL. The heritability is 10% (Left), 25% (Centre) and 50% (Right). The figures display the MC estimates of $\Pr(\delta_i = 1|y_i)$ against marker labels

correctly classified in the simulated sample is 1, 11 and 23 out of 25 QTL, for h^2 equal to 0.10, 0.25 and 0.50 respectively, with zero false positive results.

The FDR-BH approach identifies a larger number of QTL than the Bonferroni approach at low heritability values (8 and 20, for h^2 equal to 0.10 and 0.25, respectively), as expected due to higher power, albeit at the expense of incurring on 3 false positive results. At $h^2 = 0.5$, in the simulated sample, both methods perform equally well, with 23 loci detected out of the 25 QTL and zero false positives (Table 8.4).

Figure 8.7 shows output from the Bayesian implementation for the three heritability levels, where MC estimates of $\Pr(\delta_i = 1|y_i)$ for each marker are plotted against marker labels (see also Table 8.4). The discovery set is here arbitrarily defined by the group of markers that satisfy $\Pr(\delta_i = 1|y_i) > 0.5$. The choice of threshold would typically not be fixed but would rather depend on an eyeball judgement of the particular plot and several thresholds would be explored. Given the chosen threshold of 0.5, the size of discovery sets for h^2 equal to 0.10, 0.25 and 0.50 are 10, 25 and 28 markers with respectively 5, 5 and 3 false discoveries. The rightmost figure indicates that many markers reach $\Pr(\delta_i = 1|y_i) \approx 1$. If for this heritability level the threshold is set equal to $\Pr(\delta_i = 1|y_i) > 0.9$, then 25 markers are chosen with 0 false positives. Similarly, for the figure in the centre panel corresponding to $h^2 = 0.25$, if the threshold is $\Pr(\delta_i = 1|y_i) > 0.7$, then 23 markers are chosen as part of the discovery set with 3 false discoveries.

When the size of the discovery set of the Bayesian approach is set equal to that generated by FDR-BH, both methods yield almost identical results (Table 8.4).

The figures for FDR in Table 8.4 are based on the true quantities obtained for the sample of simulated data at hand. With real data this information is not available. In such a situation, for the FDR-BH approach one must draw conclusions based on the nominal values of q set by the user. In a discovery set of size N say, on average one expects qN false discoveries. For the three heritability levels the expected number of FD for the FDR-BH are $8 \times 0.1 \approx 1$, $20 \times 0.05 = 1$ and $23 \times 0.01 \approx 0$, not too far from the true realisations.

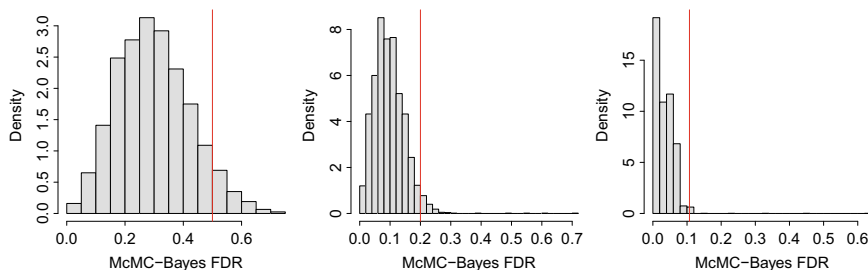


Fig. 8.8 Uncorrelated marker genotypes. The histograms are MC estimates of the marginal posterior distributions of false discovery rates for the three heritability levels. The vertical lines indicate the true number of false discoveries realised in the simulated sample. The heritability is 10% (Left), 25% (Centre) and 50% (Right). Discovery set defined by those markers that satisfy $\Pr(\delta_i = 1|y) > 0.5$

The Bayesian implementation yields the marginal posterior distribution of the FDR. Figure 8.8 displays histograms of MC estimates of these distributions for the three heritability levels, using the threshold corresponding to $\Pr(\delta_i = 1|y) > 0.5$. The vertical lines indicate the true number of false discoveries realised in the simulated sample.

Correlated Marker Genotypes

In this case account is taken of the more realistic situation where the 1500 genetic markers are in linkage disequilibrium (marker genotypes are correlated). This has important consequences for detection of QTL and on the behaviour of single marker regressions (GWAS).

The results of GWAS using the Bonferroni correction are displayed in Fig. 8.9 and in the right block of Table 8.4 for the three heritability levels. When $h^2 = 0.10$, 4 markers reach the significant threshold. When $h^2 = 0.25$, 27 markers are declared significant and 12 of these are false positive calls. At $h^2 = 0.5$, 50 markers are declared significant and 25 are false positive results. The FDR-BH approach declares 27, 50 and 70 markers as significant for $h^2 = 0.10$, $h^2 = 0.25$ and $h^2 = 0.50$, respectively, with 14, 27 and 45 false positives. These results are far from the expectations. The Bonferroni test should lead to 0 false positive results, on average and FDR-BH, given the size of the discovery sets, to $27 \times 0.1 \approx 3$, $50 \times 0.05 \approx 3$ and $70 \times 0.01 \approx 1$ false positive results, for the three heritability levels, the three values of q (equal to 0.1, 0.05, 0.01) and for the three heritability levels.

In contrast, the results based on the FDR-Bayes are more in line with those of the uncorrelated markers, as reflected in the bottom row of Table 8.4 and in Fig. 8.10.

The different behaviour of FDR-Bayes and FDR-BH is due to the consequence of fitting one marker at a time in situations where loci are correlated. As indicated on page 300, expression (6.53), fitting a single marker that has no direct effect on

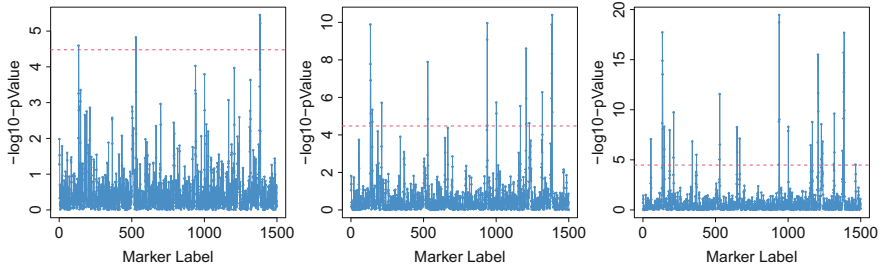


Fig. 8.9 GWAS analysis of 1500 individuals, 1500 **correlated** marker genotypes with 25 loci and $h^2 = 0.10$ (Left), $h^2 = 0.25$ (Centre), $h^2 = 0.50$ (Right). The Y-axes shows $-\log_{10} p$ -values and the X-axes the marker labels. The markers that reach $-\log_{10} p$ -values beyond the Bonferroni threshold - horizontal line in red, set at $-\log_{10}(0.05/1500)$ are declared as discoveries (4, 27 and 50 out of 1500 markers in the Left, Centre and Right panels, respectively)

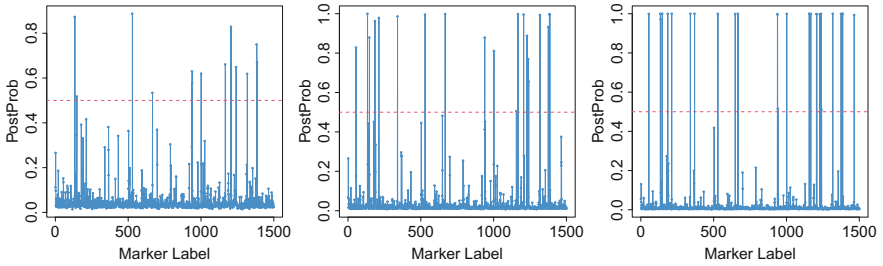


Fig. 8.10 Bayes-McMC implementation of a model with 1500 individuals, 1500 **correlated** genetic markers of which 25 are chosen as QTL. The heritability is 10% (Left), 25% (Centre) and 50% (Right). The figures display the MC estimates of $\Pr(\delta_i = 1|y_i)$ against marker labels

the dependent variable and that is correlated with a causal marker not included in the model, generates a bias that can result in a phantom significant p -value. When thousands of tests are performed in this way, the phantom p -values that are fed into the FDR-BH lead to incorrect inferences. This is not an inherent problem of FDR-BH; it is rather the single marker regression that cannot be used in this manner in conjunction with FDR-BH. An important underlying assumption of FDR-BH is that the p -values are calculated correctly. The problem is well understood by practitioners of genome studies and several approximate solutions have been proposed (Yang et al. 2012, Brzyski et al. 2017).

On the other hand, a method that fits all the markers simultaneously such as the spike and slab model should retrieve more appropriate inferences and provides a unified approach for estimation, prediction and detection of promising covariates. In practice, the computational burden of the implementation of such a model using large data sets with millions of correlated covariates poses serious challenges. Many of these challenges are being met (Patxot et al. 2021, de los Campos et al. 2022) and very likely these are going to become the methods of choice in a not too distant future.

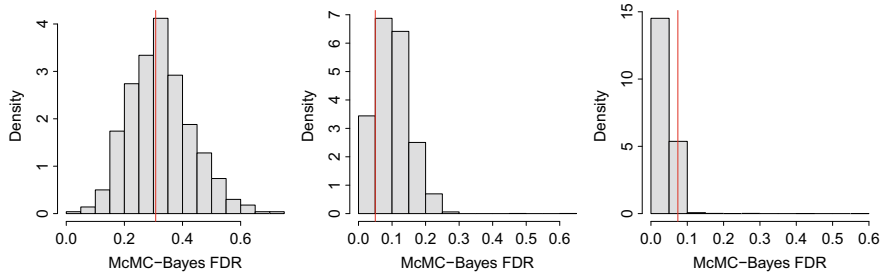


Fig. 8.11 Correlated marker genotypes. The histograms are MC estimates of the marginal posterior distributions of false discovery rates for the three heritability levels. The vertical lines indicate the true number of false discoveries realised in the simulated sample. The heritability is 10% (Left), 25% (Centre) and 50% (Right). Discovery set defined by those markers that satisfy $\Pr(\delta_i = 1|y) > 0.5$

The MC estimates of the marginal posterior distributions of FDR obtained using FDR-Bayes are displayed in Fig. 8.11.

The R-code below shows the computation of the Bayesian FDR based on (8.43), the construction of the histograms in Figs. 8.8 and 8.11 and Monte Carlo estimates of posterior means and posterior intervals .

```
# FUNCTION TO COMPUTE BAYESIAN FDR AND TO DRAW HISTOGRAM OF FDR
# INPUT:
# 1. resultprobtheta: A FILE WITH DRAWS FROM EQN. (7.57)
# 2. truefd: THE TRUE PROPORTION OF FALSE DISCOVERIES OBSERVED
#    IN THE SAMPLE (USED TO DRAW VERTICAL LINE IN HISTOGRAM)
# 3. prob: CHOSEN THRESHOLD THAT DEFINES THE DISCOVERY SET
# 4. postprob: VECTOR OF POSTERIOR PROBABILITIES
#    FOR EACH GENETIC MARKER
fdrhist <- function(resultprobtheta,truefd,prob){
  discset <- which(postprob > prob)
  fdisc <- apply(1-(resultprobtheta[,discset]),1,mean)
  # BAYESIAN FDR POSTERIOR MEAN AND POSTERIOR INTERVAL:
  avfdisc <- mean(fdisc)
  quantilefdisc <- quantile(fdisc,c(0.025,0.975))
  # HISTOGRAM OF BAYESIAN FDR:
  hist(fdisc,breaks=30,xlab='McMC-Bayes FDR',
       main=NULL,freq=FALSE)
  abline(v=truefd[length(discset)],col="red",lwd=2)
  # RETURNS: MC average of Bayesian FDR; posterior interval,
  # size of discovery set, number of false discoveries
  return(c(avfdisc,quantilefdisc,length(discset),
           truefd[length(discset)]*length(discset)))
}
out <- fdrhist(resultprobtheta,truefd,0.5)
out
```

8.11 Appendix: Proof of Benjamini and Hochberg's Theorem

Benjamini and Hochberg (1995) prove the following theorem: for continuously distributed test statistics, the false discovery rate, defined as the expectation of the false discovery proportion, is given by

$$\begin{aligned} \text{FDR} &= \mathbb{E} \left(\frac{V}{R} I(R > 0) \right) \\ &= \frac{m_0}{m} q \leq q \end{aligned}$$

where the expectation is taken over the true distribution of the data. This appendix provides an accessible proof of this result that differs from the original proof of Benjamini and Hochberg (1995) and is more aligned with the proof in Benjamini and Yekutieli (2001), for the case of jointly independent test statistics. Other sources for the proof that follows are a set of unpublished notes by Jensen (2023), and Ewens and Grant (2005), where more mathematical rigour can be found.

Consider the set of m p -values associated with the m hypotheses. An observed p -value is p_i and the random variable is P_i . The observed ordered p -values are denoted $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$. Let $(i/m)q = q_i$. Moving towards smaller p -values, $i = m, m-1, \dots, 1$; the first i where $p_{(i)} \leq q_i$ results in a discovery set $R = i$, i hypotheses are rejected and $m-i$ hypotheses with $p_{(j)} > q_j$, $j = i+1, \dots, m$ are accepted.

Let the set M_0 contain the indices of the m_0 true null hypotheses, and let the set M_1 contain the indices of the $m - m_0$ false null hypotheses. Thus H_i , $i \in M_0$, denotes the i th true null hypothesis and H_i , $i \in M_1$, denotes the i th false null.

The number of true null hypotheses rejected in error (number of false discoveries) is V , and the total number of rejections (the size of the discovery set) is R .

With this notation in place, the FDR can be written

$$\begin{aligned} \text{FDR} &= \mathbb{E} \left(\frac{V}{R} I(R > 0) \right) = \sum_{j=1}^m \frac{1}{j} \mathbb{E}[V I(R = j)] \\ &= \sum_{j=1}^m \frac{1}{j} \mathbb{E} \left[\sum_{i \in M_0} I(P_i \leq q_j) I(R = j) \right] \\ &= \sum_{i \in M_0} \sum_{j=1}^m \frac{1}{j} \mathbb{E}[I(P_i \leq q_j) I(R = j)], \end{aligned} \tag{8.45}$$

where $(P_i \leq q_j, R = j)$ is the joint event that H_i is rejected and that in total, j null hypotheses are rejected. This joint event can be rewritten as $(P_i \leq q_j, C_{j-1}^{(i)})$, where $C_{j-1}^{(i)}$ is the event that, of the remaining $m-1$ hypotheses other than H_i ,

exactly $j - 1$ are rejected. In the case of independent test statistics, $(P_i \leq q_j)$ and $C_{j-1}^{(i)}$ are independent events. Therefore

$$\begin{aligned} E \left[I(P_i \leq q_j) I(C_{j-1}^{(i)}) \right] &= \Pr(P_i \leq q_j) \Pr(C_{j-1}^{(i)}), \quad i \in M_0, \\ &= q_j \Pr(C_{j-1}^{(i)}) \\ &= \frac{j}{m} q \Pr(C_{j-1}^{(i)}). \end{aligned} \quad (8.46)$$

The second line on the right hand side follows because $P_i, i \in M_0$, is assumed to have a uniform distribution on $[0, 1]$, and the third uses the definition $(j/m)q = q_j$. Substituting in (8.45) yields the final result:

$$\begin{aligned} \text{FDR} &= \sum_{i \in M_0} \sum_{j=1}^m \frac{1}{j} \frac{j}{m} q \Pr(C_{j-1}^{(i)}) \\ &= \sum_{i \in M_0} \frac{q}{m} \sum_{j=1}^m \Pr(C_{j-1}^{(i)}) \\ &= \sum_{i \in M_0} \frac{q}{m} = \frac{m_0}{m} q. \end{aligned} \quad (8.47)$$

The third line uses the law of total probability $\sum_{j=1}^m \Pr(C_{j-1}^{(i)}) = 1$.

Benjamini and Yekutieli (2001) show that result (8.47) holds in cases where there is a certain type of correlation among the test statistics (positive regression dependence). This requires replacing the constants $(i/m)q$ by

$$\frac{iq}{m \sum_{j=1}^m \frac{1}{j}}.$$

Note

A more obvious connection between the proof of the theorem and the implementation of the FDR algorithm calls for taking a closer look into the event $C_{j-1}^{(i)}$. A little extra notation is needed. First, recall that when $R = j$, j hypotheses with ordered p -values $p_{(j)} \leq q_j$ are rejected. Consider removing the i th p -value from the ordered p -values $P_{(1)}, P_{(2)}, \dots, P_{(m)}$. Denote the remaining $m - 1$ ordered p -values as $P_{(1)}^{-i}, P_{(2)}^{-i}, \dots, P_{(m-1)}^{-i}$. For example, if $i = 3$, $P_{(1)}^{-3} = P_{(1)}$, $P_{(2)}^{-3} = P_{(2)}$, $P_{(3)}^{-3} = P_{(4)}$, $\dots$, $P_{(m-1)}^{-3} = P_{(m)}$. Let $H_{(i)}$ denote hypothesis i associated with

p -value $p_{(i)}$. With this notation the joint event $(P_i \leq q_j, R = j)$ can be expressed as $(P_i \leq q_j, C_{j-1}^{(i)})$, equal to

$$\begin{aligned} & (P_i \leq q_j, P_{(j)} \leq q_j, P_{(j+1)} > q_{j+1}, \dots, P_{(m)} > q_m) \\ &= (P_i \leq q_j, P_{(j-1)}^{-i} \leq q_j, P_{(j)}^{-i} > q_{j+1}, \dots, P_{(m-1)}^{-i} > q_m), \end{aligned} \quad (8.48)$$

that holds for $i < j$ and also for $i = j$, in which case $P_{(j-1)}^{-i} = P_{(j-1)} \leq P_{(j)} \leq q_j$. Expression (8.48) reveals that at the cut-off j with the ordered p -value $P_{(j-1)}^{-i} \leq q_j$, $j - 1$ hypotheses are rejected and $H_{(j+1)}, H_{(j+2)}, \dots, H_{(m)}$ are accepted; this is precisely how the algorithm is implemented. One can also note that

$$C_{j-1}^{(i)} = (P_{(j-1)}^{-i} \leq q_j, P_{(j)}^{-i} > q_{j+1}, \dots, P_{(m-1)}^{-i} > q_m)$$

depends only on $P_l, l \neq i$; therefore with independent test statistics $(P_i \leq q_j)$ and $C_{j-1}^{(i)}$ are independent events.

Chapter 9

Binary Data



Many of the results derived under the assumption that observations are continuously distributed extend to dichotomous and categorical responses. There are some technical details that must be observed that are specific to discontinuous data. After discussing briefly evaluation of probabilistic predictions for binary data, the chapter illustrates the behaviour of training and validating mean squared error applied to binary records using operational logistic regression models with increasing number of covariates.

As the number of covariates in a prediction model increases relative to the number of records, the bias-variance trade-off calls for the introduction of shrinkage. Three modelling scenarios that incorporate shrinkage are described: penalised logistic regression, logistic lasso and the Bayesian-McMC driven spike and slab model introduced on page 349, extended here to deal with binary records.

The performance of a classifier can be gauged by studying the *true positive rate* and *false positive rate*. The *receiver operating characteristic curve* is often used to compare the performance of different binary classifiers and is the subject of Sect. 9.7.

A topic that is peculiar to data falling into discrete and mutually exclusive categories is the computation of the probability that a future observation falls in a given category, given some previous information. Two examples are discussed. One illustrates some of the statistical challenges involved in the discovery of genes related to rare diseases and in prediction of a rare disease using genetic markers. The other deals with prediction of disease status of a genetic disease for an individual, given information on the disease status of its relatives.

The chapter ends with an appendix describing an approximate analysis of binary traits that can be useful as an initial investigating tool, before developing the full machinery needed for a more sophisticated approach.

A general framework for the analysis of binary data is introduced on page 90.

9.1 Prediction for Binary Observations

Consider data $(Y_1, x_1), \dots, (Y_n, x_n)$ where Y_i 's are binary random variables and $x_i \in \mathbb{R}^p$ covariates. A binary classifier is a function $s(x_i)$ that transforms inputs x_i into a value between 0 and 1 interpreted as a probability. Depending on the problem at hand one may be interested in estimating $\Pr(Y_i = 1|x_i)$ on its own right. Alternatively, the estimated $\Pr(Y_i = 1|x_i)$ can be expressed as a prediction $\hat{Y}_i$ that takes the values either 0 or 1, according to

$$\hat{Y}_i = \begin{cases} 1 & \text{if } s(x_i) > t \\ 0 & \text{if } s(x_i) \leq t, \end{cases} \quad (9.1)$$

where $t \in [0, 1]$ is some threshold. When $s(x_i) = E(Y_i|x_i)$ and $t = 0.5$, this is known as Bayes rule (poor choice of terminology, commonly used, not be to confused with Bayes theorem) that minimises the overall probability of a misclassification $\Pr(\hat{Y}_i \neq Y_i)$. For Bernoulli data we have

$$s(x_i) = E(Y_i|x_i) = \Pr(Y_i = 1|x_i) \quad (9.2)$$

and given data and the threshold t , one predicts $\hat{Y}_i$ according to the rule

$$\hat{Y}_i = \begin{cases} 1 & \text{if } \hat{s}(x_i) = \hat{\Pr}(Y_i = 1|x_i) > t \\ 0 & \text{if } \hat{s}(x_i) = \hat{\Pr}(Y_i = 1|x_i) \leq t, \end{cases} \quad (9.3)$$

where $\hat{s}(x_i) = \hat{\Pr}(Y_i = 1|x_i)$ is an estimate of $s(x_i)$.

In the logistic model the object of the modelling is the probability that Y is equal to 1, parametrised as

$$s(x_i) = \Pr(Y_i = 1|x_i) = \frac{\exp(x_i' \beta)}{1 + \exp(x_i' \beta)}, \quad (9.4)$$

or alternatively, as the logit or logodds

$$\ln \left[\frac{\Pr(Y_i = 1|x_i)}{\Pr(Y_i = 0|x_i)} \right] = x_i' \beta.$$

In these expressions β includes an intercept and the first element of the $(p+1)$ row vector x_i' is a 1.

Imagine that estimates $\hat{\beta}$ of the logistic coefficients are available. Conditional on $\hat{\beta}$ these yield estimates of the probability

$$\hat{\Pr}(Y_i = 1|x_i) = \frac{\exp(x_i' \hat{\beta})}{1 + \exp(x_i' \hat{\beta})}.$$

If the choice falls on $t = 0.5$, we would predict $\widehat{Y}_i = 1$ if $\widehat{\Pr}(Y_i = 1|x_i) > \widehat{\Pr}(Y_i = 0|x_i)$, that is, if $\widehat{\Pr}(Y_i = 1|x_i) > 0.5$ or if $x'_i\widehat{\beta} > 0$. The latter is readily derived as follows. Choose $\widehat{Y}_i = 1$ if

$$\frac{\widehat{\Pr}(Y_i = 1|x_i)}{\widehat{\Pr}(Y_i = 0|x_i)} > 1.$$

Taking logarithms on both sides,

$$\ln[\widehat{\Pr}(Y_i = 1|x_i)] - \ln[\widehat{\Pr}(Y_i = 0|x_i)] > 0.$$

This gives

$$\ln \left[\frac{\exp(x'_i\widehat{\beta})}{1 + \exp(x'_i\widehat{\beta})} \right] - \ln \left[\frac{1}{1 + \exp(x'_i\widehat{\beta})} \right] > 0,$$

$$x'_i\widehat{\beta} > 0.$$

Prediction of binary outcomes is often known as *classification*.

9.2 Mean Squared Error for Binary Data?

Throughout the book the mean squared error of prediction is often applied to monitor the prediction ability of a model. Its use for normally distributed data is uncontroversial. However with binary data there are other more formal measures to study model fit and prediction. The section below sketches some of the reservations and a justification for using mean squared error with binary data.

Preliminaries

In the case of normally distributed data with homogeneous variance, the mean squared error is proportional to the logscore (not to be confused with the *score* defined in Eq.(2.5)), defined as the logarithm of the probability or density of observed data given parameters and covariates, or loglikelihood. The logscore happens to be a *proper scoring rule*. A scoring rule provides a means of evaluating probabilistic predictions of an event in the light of its outcome (Bernardo and Smith 1994). The negative of the logscore can be thought of as a cost function. If the minimum of the average of the cost function is achieved when the predicted probability is equal to the true probability of the event, the scoring rule is said to be proper. Minimisation is taken with respect to the predicted probability and

the average is over all possible realisations of data. A similar line of argument holds when the logscore is not multiplied by -1 . In this case the logscore can be interpreted as a *utility*; the prescription is to maximise expected utility.

As an illustration consider binary data where Y_i can take the value 1, with probability P_i or 0 with probability $1 - P_i$. Let $\hat{P}_i$ be some estimate of the true probability P_i . For datum i the negative of the logscore evaluated at $\hat{P}_i$ for the i th term is

$$-y_i \ln(\hat{P}_i) - (1 - y_i) \ln(1 - \hat{P}_i).$$

If $y_i = 1$ (that happens with probability P_i) the cost function is $-\ln(\hat{P}_i)$ and when $y_i = 0$ (that happens with probability $(1 - P_i)$), the cost function is $-\ln(1 - \hat{P}_i)$. Averaging over Y_i results in the average cost function (or expected negative logscore)

$$-P_i \ln(\hat{P}_i) - (1 - P_i) \ln(1 - \hat{P}_i).$$

A differentiation with respect to $\hat{P}_i$ results in

$$\frac{P_i - \hat{P}_i}{\hat{P}_i(1 - \hat{P}_i)}.$$

The expected negative logscore has zero derivative at $P_i = \hat{P}_i$ and a second differentiation shows that this is a minimum. Therefore the negative of the logscore is a proper scoring rule.

As a second illustration of a proper scoring rule involving binary data, consider the *Brier score*. Let P_i denote the true probability that $Y_i = 1$. The Brier score for the i th record is

$$MSE_{B,i} = (y_i - \hat{P}_i)^2, \quad y_i = 0, 1, \quad (9.5)$$

where $\hat{P}_i$ is some estimate of the true probability P_i . The contribution to the Brier score from record i is $\hat{P}_i^2$ if $y_i = 0$ (that happens with probability $(1 - P_i)$) and $(1 - \hat{P}_i)^2$ if $y_i = 1$ (that happens with probability P_i). Averaging over Y_i yields the average cost function

$$\begin{aligned} & \hat{P}_i^2 (1 - P_i) + (1 - \hat{P}_i)^2 P_i \\ &= (\hat{P}_i - P_i)^2 + P_i (1 - P_i) \end{aligned} \quad (9.6)$$

that has a minimum when $\hat{P}_i = P_i$, indicating that the Brier score is a proper scoring rule. The Brier score discourages extreme differences between y_i and $\hat{P}_i$ less than does the logscore.

Finally consider using a predictor based on Bayes rule, that assigns to terms $(y_i - \hat{Y}_i)^2$, $\hat{Y}_i = 1$ if $\hat{P}_i \geq 0.5$ and $\hat{Y}_i = 0$ if $\hat{P}_i < 0.50$. The average cost function is again given by (9.6) with $\hat{P}_i$ replaced by $\hat{Y}_i$; the minimum cannot be attained at $\hat{Y}_i = P_i$ except when P_i is exactly 1 or exactly 0. This scoring rule is not proper.

Another consequence of using a predictor based on Bayes rule in the computation of MSE rather than using the Brier score is related to discriminatory power. Consider two models; model 1 results in an estimate $\hat{P} = 0.6$ when $y = 1$ and assigns the value 1 to $\hat{Y}$; when $y = 0$ model 1 results in an estimate $\hat{P} = 0.4$ and assigns the value 0 to $\hat{Y}$. The two estimated probabilities for $y = 1$ and $y = 0$ based on model 2 are 0.8 and 0.2, respectively. Using Bayes rule leads to the same MSE's for both models but the Brier score, that incorporates information from the estimated probabilities, assigns a smaller MSE to model 2.

Despite these limitations, in many examples in this book involving binary records, models are assessed using a predictor based on Bayes rule in the computation of MSE. An initial justification is that the assessment must be regarded as preliminary and exploratory rather than providing decisive information for model choice. The MSE using the Bayes classifier is equal to the proportion of misclassifications; this is useful in its own right and is easy to interpret. A further attraction of the Bayes classifier is that it minimises $\Pr(Y \neq \hat{Y})$, the overall true proportion of misclassifications. Notwithstanding, there may be cases where evaluation of models for the analysis of binary records using the MSE with the Bayes classifier may be a poor choice; an example is provided on page 434.

Mean Squared Error

As indicated in (6.84), the estimate of the expected validating mean squared error using independent training data is

$$\hat{E}(\text{MSE}_v) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \frac{2}{N} \sum_{i=1}^N \widehat{\text{Cov}}(y_i, \hat{y}_i), \quad (9.7)$$

where the second term is an estimate of the expected optimism. With binary observations, where both y_i and $\hat{y}_i$ take values 0 or 1,

$$(y_i - \hat{y}_i)^2 = \begin{cases} 1 & \text{if } y_i \neq \hat{y}_i, \\ 0 & \text{if } y_i = \hat{y}_i. \end{cases} \quad (9.8)$$

Therefore, the first term in the right hand side of (9.7) is the observed proportion of misclassifications (or error rate) in the training data. To compute the second term proceed as follows:

$$\text{Cov}(y_i, \hat{y}_i) = E(y_i \hat{y}_i) - E(y_i) E(\hat{y}_i), \quad (9.9)$$

where $E(y_i) = \Pr(y_i = 1)$ and all expectations are conditional on x . Also, we have

$$\begin{aligned} E(\hat{y}_i) &= E_{y_i} [E(\hat{y}_i | y_i)] \\ &= \Pr(\hat{y}_i = 1 | y_i = 0) \Pr(y_i = 0) + \Pr(\hat{y}_i = 1 | y_i = 1) \Pr(y_i = 1), \end{aligned}$$

and

$$\begin{aligned} E(y_i \hat{y}_i) &= E_{y_i} [E(y_i \hat{y}_i | y_i)] \\ &= E_{y_i} [y_i E(\hat{y}_i | y_i)] \\ &= \Pr(y_i = 1) \Pr(\hat{y}_i = 1 | y_i = 1). \end{aligned}$$

Substituting these expressions in (9.9) gives

$$\begin{aligned} \text{Cov}(y_i, \hat{y}_i) &= \Pr(y_i = 1) (1 - \Pr(y_i = 1)) \\ &\quad [\Pr(\hat{y}_i = 1 | y_i = 1) - \Pr(\hat{y}_i = 1 | y_i = 0)]. \end{aligned}$$

The estimator of the expected validating mean squared error (9.7) for binary data takes the form

$$\begin{aligned} \hat{E}(\text{MSE}_v) &= \frac{\#(y_i \neq \hat{y}_i)}{N} + \frac{2}{N} \sum_{i=1}^N \hat{\Pr}(y_i = 1) (1 - \hat{\Pr}(y_i = 1)) \\ &\quad [\hat{\Pr}(\hat{y}_i = 1 | y_i = 1) - \hat{\Pr}(\hat{y}_i = 1 | y_i = 0)] \end{aligned} \quad (9.10)$$

where $\#(y_i \neq \hat{y}_i)$ is the total number of misclassifications in the training data and the “hats” are estimates of the various probabilities. Expression (9.10) is compared to MC estimates of the expected validating mean squared error in the simulation example that follows. The second term in (9.10) is an estimator of expected optimism. Much of the material in this section is taken from Efron and Hastie (2016) where more details on the subject can be found.

Example: Training and Validation MSE with Binary Data

This example illustrates in the context of binary data, how training and validating mean squared errors are affected as the number of covariates of the operational models increases. The example is in the same spirit as that on page 320.

Binary data are simulated using the logistic model (9.4) and consist of $N = 400$ records divided in equal numbers into training and validating data. The matrix X of genetic marker codes is generated as in the Example on page 320, here with $p_\ell = 10$ loci and includes a column vector of 1s to accommodate an intercept.

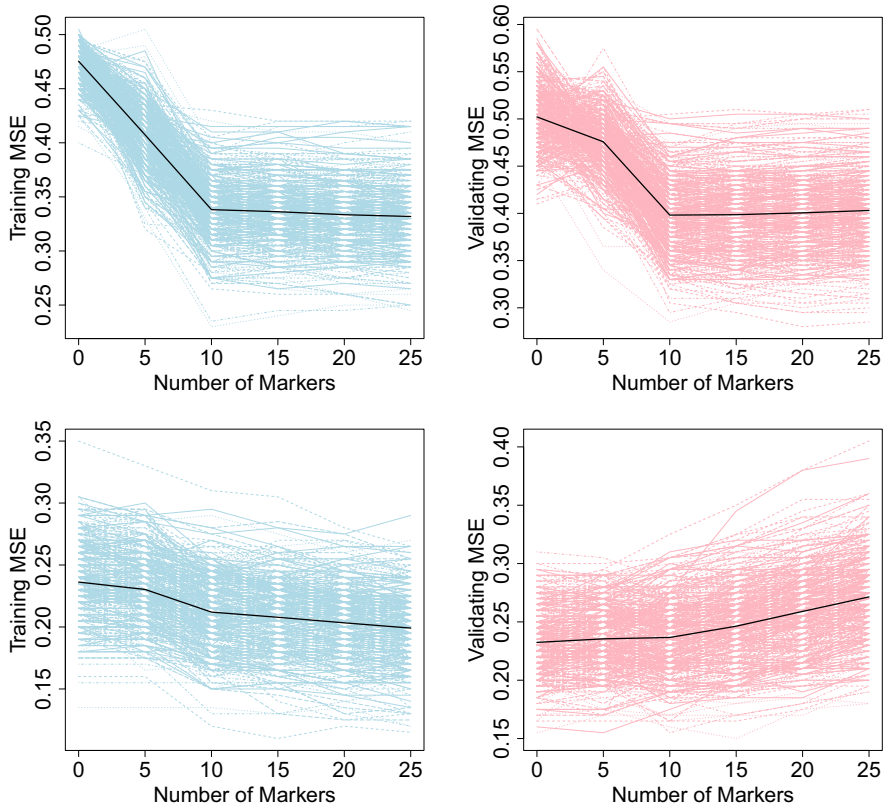


Fig. 9.1 Mean squared error (MSE) per replicate (400 Monte Carlo simulations) and average MSE shown as bold black lines. True marker effects drawn from $N(0, 0.1)$. Top panel $\Pr(Y_i = 1|\mu) = 0.5$; bottom panel: $\Pr(Y_i = 1|\mu) = 0.2$. The left panels correspond to the training data and the right to the validating data

Training data are analysed with 5 operational models with numbers of covariates equal to 5, 10, 15, 20 or 25. Model 2 with 10 covariates is the true model. The regression coefficients of the true effects of the 10 loci were sampled from $N(0, 0.1)$. The intercept μ was chosen to generate either $\Pr(Y_i = 1|\mu) = 0.5$ or $\Pr(Y_i = 1|\mu) = 0.2$. In all, 400 replicates are simulated of both training and validating data.

Figure 9.1 displays the mean squared errors for the validating and the training data for each replicate, as well as the average over replications. The top panel corresponds to $\Pr(Y_i = 1|\mu) = 0.5$ and the bottom panel to $\Pr(Y_i = 1|\mu) = 0.2$. In the training data, the mean squared error decreases with increasing number of parameters. For $\Pr(Y_i = 1|\mu) = 0.5$ the rate of decline is largest up to 10 markers (the true model) and thereafter becomes markedly smaller.

In the validating data, the pattern of the evolution of the mean squared error with increasing number of parameters depends on the true distribution of the data. At intermediate probability of success, the average mean squared error in the validating data decreases from 0.50 using a model with intercept only, to 0.396 using a model with 10 covariates (the true model). Beyond 10 covariates there is a slight increase up to a value of 0.403 with 25 covariates. The observed optimism (difference between the average validating MSE_v and average training MSE_t over Monte Carlo replicates) is 0.0623 in a model with 10 covariates and the estimate based on the second term of (9.10) is 0.0617.

At the low probability of success, the average mean squared error in the training data decreases from 0.24, fitting a model with intercept only, to 0.20 when the model has 25 covariates. On the other hand, in the validating data, the average mean squared error increases from 0.23, for a model with intercept only, to 0.27 for a model with 25 covariates. The model with only an intercept provides the best predictions. At extreme probability of success, the mean squared error is overwhelmed by the increase in the variance of the estimates of the regression coefficients as more covariates are added (see the expression for the asymptotic variance (3.37), where, conditional on X the diagonal matrix D governs the magnitude of the variance). The validating mean squared error MSE_v increases slightly from 0.232 using a model with intercept only to 0.236 using a model with 10 covariates (the true model). As the number of covariates increases beyond 10, the rate of increase is more pronounced.

In a model with 10 covariates, the observed optimism (difference between the average MSE_v and average training mean squared error MSE_t over Monte Carlo replicates) is 0.0415; the estimate based on the second term of (9.10) is 0.0477.

At a low probability of success, when the regression coefficients of the true effects of the 10 loci are sampled from $N(0, 0.25)$ instead of from $N(0, 0.1)$, thus allowing for larger absolute values that are easier to detect, the evolution of the mean squared error with increasing number of parameters in the validating data shows the more familiar pattern of a decline from an initial value when the model contains only an intercept, towards a minimum value with models with 10 covariates (the true model), followed by an increase as more covariates are added (see Fig. 9.2).

In a model with 10 covariates, the observed optimism (difference between the average MSE_v and average MSE_t over Monte Carlo replicates) is 0.0253; the estimate based on the second term of (9.10) is 0.0273.

The figures display clearly the rather large variability in MSE across replications of training and validating data.

9.3 Logistic Regression with Non-random Sampling

A major concern in epidemiological studies is to determine factors that are linked to a condition or disease in a population. A variety of designs can be considered. When the condition is at very low frequency in the population, a common study design is

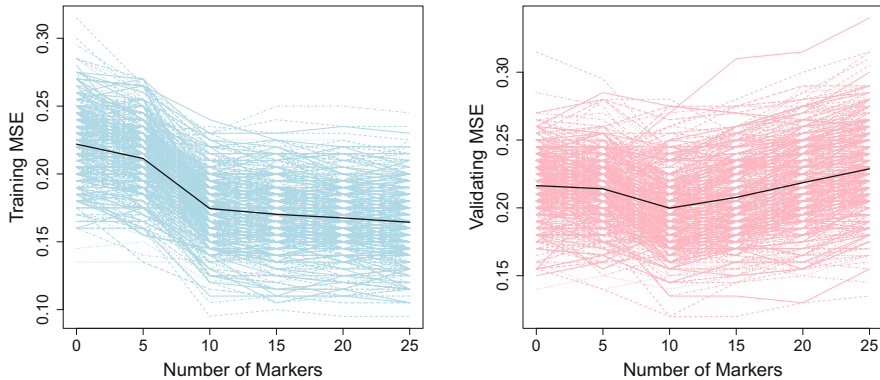


Fig. 9.2 Mean squared error (MSE) per replicate (400 Monte Carlo simulations) and average MSE in black, bold. Probability of success is $\Pr(Y_i = 1|\mu) = 0.2$. True marker effects drawn from $N(0, 0.25)$. The left panel corresponds to the training data and the right to the validating data

to over-sample individuals with the condition (known as successes or cases) and then sample a control group (failures or controls) from a similar segment of the population. This type of design is known as a case-control study. A typical example might involve collecting data from people who died from a certain condition, from other patients who act as controls and record relevant covariates. If sampling is at random, the relationship between the variables in the dataset would be representative of the same relationships in the population, but this may not be the case with non-random samples of a case-control design. Correct inferences require that the statistical analysis accounts for the non-random sampling mechanism (Prentice and Pyke 1979).

Consider a population of individuals $i = 1, \dots, n$ with disease status Y_i , covariate vector X_i and marker genotypes W_i . Assume that the conditional distribution of Y_i given $(X_i, W_i) = (x_i, w_i)$ is the logistic regression

$$\Pr(Y_i = 1|x_i, w_i) = \frac{\exp(\eta_i)}{1 + \exp(\eta_i)} \quad (9.11)$$

where

$$\eta_i = \alpha + x_i\beta + w_ib$$

is the linear predictor depending on covariate and marker effects. The parameters of the logistic model are (α, β, b) and if data were randomly sampled, they could be inferred from a likelihood constructed from (9.11) as starting point. With non-random sampling one can instead proceed as follows. Suppose (conceptually) all individuals are observed. Each individual is assigned a binary random variable S_i that takes the value 1 if the individual is sampled and 0 otherwise, with

$\Pr(S_i = 1|Y_i = 0) = p_0$ and $\Pr(S_i = 1|Y_i = 1) = p_1$. Typically p_1 is much larger than p_0 . Given $S_i = 1$, sample (X_i, W_i) from the conditional distribution $[X_i, W_i|Y_i]$ and if $S_i = 0$, (X_i, W_i) is not sampled. In the approach to be followed, inference is based on the conditional distribution of Y_i , given (X_i, W_i) and $S_i = 1$. The part of the sample for which $S_i = 0$ is ignored. As shown below this leads to the same inferences about (β, b) as an analysis using (9.11) to construct the likelihood.

The conditional distribution of Y_i given (X_i, W_i) and $S_i = 1$ is

$$\Pr(Y_i = 1|x_i, w_i, S_i = 1) = \frac{\Pr(S_i = 1|Y_i = 1, x_i, w_i) \Pr(Y_i = 1|x_i, w_i)}{\Pr(S_i = 1|x_i, w_i)}.$$

Now make the important assumption that $\Pr(S_i = 1|Y_i = 1, x_i, w_i) = \Pr(S_i = 1|Y_i = 1)$; that is, S_i and (X_i, W_i) are conditionally independent given Y_i . In a missing data framework, (X_i, W_i) are missing at random, in the sense defined by Rubin (1976). Given this assumption the above can be written

$$\begin{aligned} \Pr(Y_i = 1|x_i, w_i, S_i = 1) &= \frac{p_1 \exp(\eta_i)}{p_1 \exp(\eta_i) + p_0} \\ &= \frac{(p_1/p_0) \exp(\eta_i)}{(p_1/p_0) \exp(\eta_i) + 1}. \end{aligned} \quad (9.12)$$

Since

$$(p_1/p_0) \exp(\eta_i) = \exp(\alpha^* + x_i\beta + w_ib)$$

where $\alpha^* = \alpha + \ln(p_1/p_0)$, we have

$$\Pr(Y_i = 1|x_i, w_i, S_i = 1) = \frac{\exp(\alpha^* + x_i\beta + w_ib)}{1 + \exp(\alpha^* + x_i\beta + w_ib)}$$

as in (9.11) but with a new intercept. Therefore, the fact that the data were not sampled at random (they were collected retrospectively) can be ignored in logistic regression analyses, provided that the linear predictor contains an intercept and the sampling mechanism is independent of the explanatory variables. This is a property peculiar to the logistic link and does not carry over to other link functions.

9.4 Penalised Logistic Regression

Logistic regression can be modified in the same spirit as ridge regression to deal with the $p > n$ case (number of covariates p larger than the number of records n), by maximisation of a concave penalised loglikelihood (or, equivalently, by

minimisation of a convex penalised cost function). Here details are provided for a Newton-Raphson implementation; a full discussion can be found in Park and Hastie (2008).

The logistic regression model for the n binary responses y is parameterised as

$$\Pr(y_i = 1 | x_i, \mu, \beta) = \frac{\exp(\mu + x_i' \beta)}{1 + \exp(\mu + x_i' \beta)}, \quad i = 1, \dots, n. \quad (9.13)$$

In this expression, μ is a scalar intercept, x_i' is a row vector of p covariates and β is a column vector with p regression parameters. The convex function is the negative of the loglikelihood subject to a size constraint on the L_2 norm of β ; the intercept μ is not penalised. Then μ and β are obtained as the minimisation of the cost function

$$J(\mu, \beta) = -\ell(\mu, \beta | y, x) + \frac{\lambda}{2} \beta' \beta,$$

where λ is the regularisation parameter and

$$-\ell(\mu, \beta | y, x) = -\sum_{i=1}^n \{y_i (\mu + x_i' \beta) - \ln[1 + \exp(\mu + x_i' \beta)]\} \quad (9.14)$$

is the negative of the loglikelihood.

I shall use

$$f(x_i) = \mu + x_i' \beta$$

and

$$\pi_i = \Pr(y_i = 1 | x_i, \mu, \beta) = \frac{\exp(f(x_i))}{1 + \exp(f(x_i))}.$$

The first derivatives are

$$\frac{\partial J}{\partial \mu} = -\sum_{i=1}^n (y_i - \pi_i) = -1' (y - \pi), \quad (9.15a)$$

$$\frac{\partial J}{\partial \beta} = -\sum_{i=1}^n (y_i - \pi_i) x_i + \lambda \beta = -X' (y - \pi) + \lambda \beta, \quad (9.15b)$$

where $1'$ is a $(1 \times n)$ row vector of ones, y and π are $n \times 1$ column vectors with elements y_i and π_i , respectively and X is an $n \times p$ matrix of covariates with row vectors x_i' .

The computation of second derivatives uses

$$\begin{aligned}\frac{\partial \pi_i}{\partial \mu} &= \frac{\partial \pi_i}{\partial f(x_i)} \frac{\partial f(x_i)}{\partial \mu}, \\ \frac{\partial \pi_i}{\partial \beta} &= \frac{\partial \pi_i}{\partial f(x_i)} \frac{\partial f(x_i)}{\partial \beta}.\end{aligned}$$

Application of the chain rule yields

$$\begin{aligned}\frac{\partial^2 J}{(\partial \mu)^2} &= \sum_{i=1}^n \pi_i (1 - \pi_i) = 1' W 1, \\ \frac{\partial^2 J}{\partial \mu \partial \beta'} &= \sum_{i=1}^n \pi_i (1 - \pi_i) x_i = 1' W X, \\ \frac{\partial^2 J}{\partial \beta \partial \beta'} &= \sum_{i=1}^n x_i x_i' \pi_i (1 - \pi_i) + I \lambda = X' W X + I \lambda,\end{aligned}$$

where I is the $p \times p$ identity matrix and

$$W = \text{diag} \{ \pi_i (1 - \pi_i) \}, \quad i = 1, \dots, n,$$

a diagonal matrix of dimension $n \times n$. The matrix of second derivatives is

$$H = \begin{bmatrix} \frac{\partial^2 J}{(\partial \mu)^2} & \frac{\partial^2 J}{\partial \mu \partial \beta'} \\ \frac{\partial^2 J}{\partial \beta \partial \mu} & \frac{\partial^2 J}{\partial \beta \partial \beta'} \end{bmatrix}_{(p+1) \times (p+1)} = \begin{bmatrix} 1' W 1 & 1' W X \\ X' W 1 & X' W X + I \lambda \end{bmatrix} = Z' W Z + \Lambda \quad (9.16)$$

of order $(p+1) \times (p+1)$, where

$$Z = [1, X].$$

This is an $n \times (p+1)$ matrix whose first column is a vector of ones and

$$\Lambda = \text{diag} \{ 0, \lambda, \lambda, \dots, \lambda \},$$

is a $(p+1) \times (p+1)$ diagonal matrix. Let $\theta = (\mu, \beta')'$. The Newton-Raphson algorithm is

$$\theta_{t+1} = \theta_t - (H)^{-1} S(\theta_t) \quad (9.17)$$

where, from (9.15),

$$S(\theta_t) = -Z'(y - \pi) + \Lambda\theta_t. \quad (9.18)$$

Using (9.16) and (9.18) in (9.17) yields

$$\begin{aligned} \theta_{t+1} &= \theta_t - (Z'WZ + \Lambda)^{-1} (-Z'(y - \pi) + \Lambda\theta_t) \\ &= H^{-1} (Z'WZ + \Lambda) \theta_t + H^{-1} (Z'WW^{-1}(y - \pi) - \Lambda\theta_t) \\ &= H^{-1}Z'W(Z\theta_t) + H^{-1}\Lambda\theta_t + H^{-1}Z'W(W^{-1}(y - \pi)) - H^{-1}\Lambda\theta_t \\ &= H^{-1}Z'W(Z\theta_t + W^{-1}(y - \pi)) \\ &= H^{-1}Z'Wr, \end{aligned} \quad (9.19)$$

where $r = Z\theta_t + W^{-1}(y - \pi)$ and $H^{-1} = (Z'WZ + \Lambda)^{-1}$. This implementation of the Newton-Raphson algorithm is the *iteratively reweighted ridge regression* algorithm and requires inversion of the Hessian, an $(p + 1) \times (p + 1)$ matrix. In highly dimensional settings with $p \gg n$ alternative strategies must be sought. The aim could be either to avoid matrix inversion, or to reduce the dimensionality of the system from order p to order n . An example of the first type is the use of gradient descent that uses first derivatives of the cost function. An approach that works in an n -dimensional space involves kernel methods discussed on page 546. It is shown that use of a particular kernel known as the *linear kernel* allows a reparametrisation of the penalised logistic regression model that leads to an iterative system along the lines of (9.19), while operating in an $(n + 1)$ -dimensional space.

9.5 The Lasso with Binary Records

Use of the lasso is illustrated for the case of binary responses parametrised as in (9.4). In this formulation the loglikelihood takes the form

$$\ell(\beta_0, \beta|x, y) = \sum_{i=1}^n y_i (\beta_0 + x'_i \beta) - \ln [1 + \exp(\beta_0 + x'_i \beta)]$$

where here, contrary to (9.4), x'_i is $(1 \times p)$ and β is $(p \times 1)$. The lasso coefficients are the solutions to

$$\max_{\beta_0, \beta} \left[\sum_{i=1}^n y_i (\beta_0 + x'_i \beta) - \ln [1 + \exp(\beta_0 + x'_i \beta)] \right] - \lambda \sum_{j=1}^p |\beta_j|. \quad (9.20)$$

As before in (7.19), the intercept term is not penalised and the predictors x are standardised. An efficient computational algorithm is the path-wise coordinate descent described in connection with the linear model, with modifications to accommodate the lack of piece-wise linearity of the coefficients of the logistic likelihood (Efron and Hastie 2016).

9.6 A Bayesian Spike and Slab Model for Binary Records

An alternative to the penalised logistic regression and the lasso is the spike and slab model, that postulates a two-component mixture distribution for the SNP effects consisting of a normal distribution and a point mass at zero. Section 7.5 provides a full description of the model and an MCMC implementation for continuous data. Here I indicate the modification of the MCMC algorithm required for the analysis of binary responses.

The binary records are described with the probit model that assumes an underlying unobserved liability u . For the i th record the liability is normally distributed $u_i | \mu, x_i, b \sim N(\mu + x_i' b, \sigma^2 = 1)$. The linear model for the liability of record i can be written

$$u_i = \mu + x_i' b + e_i.$$

Above, μ is an intercept, $x_i \in \mathbb{R}^p$ is a column vector of p observed marker genotypes, b is a $p \times 1$ column vector of unobserved marker effects and residuals are $e_i \stackrel{iid}{\sim} N(0, 1)$. For the i th record ($i = 1, 2, \dots, n$)

$$\begin{aligned} \Pr(y_i = 1 | \mu, x_i, b) &= \Pr(u_i > 0 | \mu, x_i, b) \\ &= \Pr(e_i < \mu + x_i' b) \\ &= \Phi(\mu + x_i' b), \end{aligned} \tag{9.21}$$

where $\Phi(t)$ is the distribution function of the standard normal distribution and I used the symmetry of the normal distribution to go from line 1 to line 2. The conditional likelihood function for record i , given μ, x_i, b , is proportional to (9.21). The prior distributions for the binary case are assumed to be identical to those in (7.43) on page 349, except for σ^2 , that here takes the fixed value $\sigma^2 = 1$. As in the continuous case, the SNP effects are expressed as

$$b_i = \alpha_i \delta_i, \quad i = 1, \dots, p, \tag{9.22}$$

where $\alpha_i | \sigma_b^2 \stackrel{iid}{\sim} N(0, \sigma_b^2)$ is independent of the binary $(0, 1)$ indicator variable δ_i that has an a priori Bernoulli distribution with probability π :

$$\begin{aligned} \delta_i | \pi &\stackrel{iid}{\sim} Br(\pi), \quad i = 1, \dots, m, \\ \Pr(\delta_i = 1 | \pi) &= \pi, \quad \Pr(\delta_i = 0 | \pi) = 1 - \pi. \end{aligned}$$

This binary indicator variable specifies the a priori probability that a marker effect b_i is non-zero and this prior probability is the same for all markers. The variance component σ_b^2 , that describes the prior uncertainty of the SNP effect drawn from the slab component, is assumed to have the scaled inverse chi-square prior distribution

$$\sigma_b^2 | S_b, v_b \sim \chi^{-2}(S_b, v_b),$$

where the S_b and the v_b are user-tuned hyperparameters.

It is computationally convenient to augment the vector of parameters with the unobserved liabilities u (see page 237). The a priori distribution of the random vector u is

$$u | \mu, b \sim N(1\mu + Xb, 1)$$

and the conditional pmf of the data, given u , takes the degenerate form (see page 237)

$$\Pr(Y_i = y_i | u_i) = I(u_i > 0)^{y_i} + I(u_i < 0)^{1-y_i}.$$

The posterior density for the binary case is

$$p(u, \mu, \alpha, \delta, \pi, \sigma_b^2 | y) \propto p(y | u) p(u | \mu, b) p(\alpha | \sigma_b^2) p(\delta | \pi) p(\pi) p(\sigma_b^2) \quad (9.23)$$

with the b 's defined in (9.22).

The fully conditional posterior distributions for the Gibbs sampling implementation are derived from (9.23) and with the exception of the fully conditional of u , $[u | D]$ (see page 244 for definition of D) and of the fact that the conditional variance of the liability is fixed to $\sigma^2 = 1$, are identical to those for the continuous data and will not be repeated here.

The fully conditional posterior density of the liability is obtained by extracting those terms in the posterior density (9.23) that contain u . That is,

$$\begin{aligned} p(u_i | D) &\propto p(u_i | \mu, b) p(y_i | u_i) \\ &= N(u_i | 1\mu + Xb, 1) \left[I(u_i > 0)^{y_i} + I(u_i < 0)^{1-y_i} \right]. \end{aligned} \quad (9.24)$$

This expression indicates that when $y_i = 1$, $u_i > 0$ and the fully conditional posterior density of u_i is a truncated normal distribution, with mean $x_i'\beta$, variance 1 and support $(0, \infty)$. When $y_i = 0$, then $u_i < 0$ and the fully conditional posterior density of u_i is a truncated normal distribution, with mean $x_i'\beta$, variance 1 and support $(-\infty, 0)$.

A general method to sample the variable u from any univariate distribution truncated in the interval $[a, b]$ can be found in Devroye (1986), page 38, Example 10:

$$u = F^{-1} \{F(a) + U[F(b) - F(a)]\}, \quad (9.25)$$

where F is the distribution function of the untruncated variable and U is a draw from a uniform distribution in the interval $[0, 1]$.

An R-code to draw all the U 's in one pass is as follows. Assume that the mean of the vector of untruncated liability is given by $\text{mu} + \text{Xb}$, the variance is the identity matrix I and y represents the complete binary data vector. Then the code is

```
mean <- mu+Xb
sd <- 1
intermediate <- y*pnorm(0,mean=mean,sd=sd)+
  runif(length(y))*(pnorm(0,mean=mean,sd=sd)*(1-y) +
    (1-pnorm(0,mean=mean,sd=sd))*y)
u <- qnorm(intermediate,mean=mean,sd=sd)
```

The general algorithm to execute the Gibbs sampler for binary records is identical to that described on page 349, except that the liability u has to be generated with each new cycle of the Gibbs sampler. This is achieved by executing the code above. Further, the residuals have to be defined appropriately at the level of the liability, and σ^2 is fixed at the value 1.

Example: Prediction and QTL Detection Using Genetic Marker Information

This example investigates the predictive ability of the penalised logistic regression, the logistic lasso and the binary spike and slab model, using 2000 simulated binary records representing nominally unrelated individuals. The package **glmnet** described in Example 7.4 on page 347 was used to perform the computations for the logistic lasso.

The data are generated using a probit model where the unobserved liability u_i for the i th record has the linear structure

$$u_i = \mu + z_i'b + e_i, \quad i = 1, 2, \dots, 2000. \quad (9.26)$$

Above, $\mu = -0.67$ is chosen to generate a proportion of 1's of approximately 0.25, z_i is the column vector for individual i of 50 centred QTL genotype codes, b is the column vector of the 50 substitution effects and the e_i 's are $N(0, 1)$ independent residual effects. The additive genetic variance on the underlying scale is $\sigma_a^2 = \text{Var}(z_i' b | b)$. This variance was set equal to 1 and the substitution effects b_i were chosen accordingly, the same value for the 50 loci (≈ 0.23 units).

The binary records are analysed with an operational probit model involving 1500 centred genotypic marker codes. The 50 loci of the true model in (9.26) were randomly sampled from these 1500 and were assigned as QTL. For individual i , the operational model of the underlying liability is

$$u_i = m + x_i' \beta + \varepsilon_i, \quad i = 1, 2, \dots, 2000,$$

where m is an intercept, x_i is the (observed) column vector for individual i of the 1500 centred marker genotypic codes, β is the (unobserved) vector with the 1500 unknown marker effects and the ε_i 's are independently distributed standard normal random variables.

QTL Detection

Before studying prediction ability we look into the performance of the spike and slab model as a QTL detection tool and compare it with a simple GWAS approach based on a probit likelihood, one marker at a time. Having generated the binary data vector y of length 2000 and the matrix of centred marker genotypes Xc , the kernel of the R-code for the GWAS analysis is shown below:

```
# PERFORM A GWAS ON THE DATA USING A PROBIT REGRESSION
# Xc is the 2,000 by 1,500 matrix of centred marker
# genotype codes
GWAS=matrix(nrow=ncol(Xc),ncol=4)
colnames(GWAS)=c('estimate','SE','t-value','p-value')
for(i in 1:ncol(Xc)){
  fm=glm(y~Xc[,i],family = binomial(link = "probit"))
  GWAS[i,]= summary(fm)$coef[2,]
}
plot(-log10(GWAS[,4]),type='o',ylab='-log10-pValue',
     cex=.5,col=4)
abline(h=-log10(0.05/nmark),lty=2,col=2)
# log Bonferroni bound:
cat('Bonferroni',-log10(0.05/nmark),'\n')
GWASdetct<-which(-log10(GWAS[,4]) > -log10(0.05/nmark))
length(GWASdetct) # SIZE OF DISCOVERY SET
```

The computation of the p -values is obtained appealing to the asymptotic properties of the likelihood estimator of marker effects; this leads to approximate t -distributed test statistics and uniformly distributed p -values $Un(0, 1)$ under H_0 (see page 365).

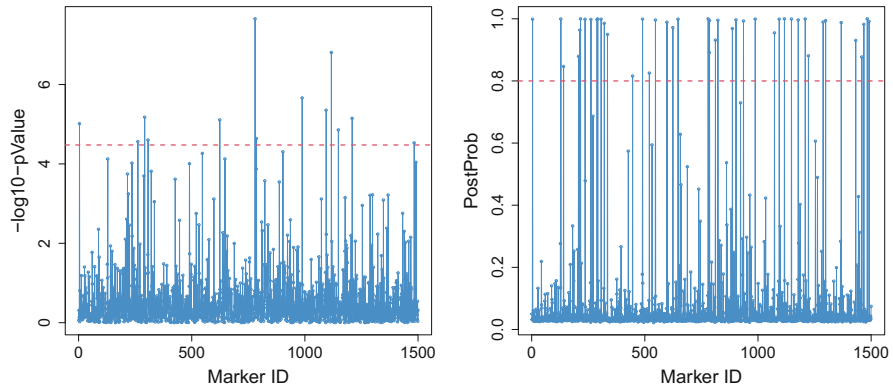


Fig. 9.3 Analysis of 2000 binary records. Left: $-\log_{10} p$ -values for testing the null hypothesis that each of 1500 marker effects is equal to zero. The horizontal red line is the Bonferroni threshold corresponding to $-\log_{10}(0.05/1500)$. Right: posterior probabilities that marker effects are not zero, for each of the 1500 markers, based on the Bayesian McMC spike and slab model. The horizontal line is the threshold corresponding to a posterior probability equal to 0.8

The result for the GWAS based on a Bonferroni bound for an individual test equal to $0.05/1500$ is shown in the left panel of Fig. 9.3. Only 13 of the 50 QTL are declared significant and in this discovery set there are no false discoveries. The classical test without the Bonferroni correction declares 133 markers as significant and among these, 86 are false discoveries.

The size of the discovery set based on the Benjamini and Hochberg FDR rule (8.14b), setting the level of the FDR $q = 0.15$, is equal to 50, and among these, the realised number of false discoveries is 11, leading to a realised false discovery proportion equal to $11/50 = 0.22$. The R-code that implements this computation is shown on page 368.

The right panel of Fig. 9.3 shows McMC estimates of the posterior probabilities that each marker effect is not equal to zero, generated from the Bayesian McMC spike and slab model using the 2000 binary records. A discovery set was obtained by arbitrarily choosing the markers with non-zero posterior probabilities larger than 0.8. The ensuing discovery set included 44 genetic markers and among these, 2 were true false positives, leading to a true FDR equal to $2/44 = 0.045$. The McMC estimate of the mean of the posterior distribution of the Bayes-FDR for this discovery set of size 44 is 0.033, based on (8.42) or on (8.43) on page 382. The 95% posterior interval obtained from the McMC draws from (8.43) is $(4.56 \times 10^{-5}; 9.11 \times 10^{-2})$.

For a discovery set of size 50, obtained by choosing the markers with non-zero posterior probabilities larger than 0.56, the Bayes-FDR estimate is 0.072 with a 95% posterior interval equal to $(0.013; 0.142)$. The true number of false discoveries is 7, resulting in a true FDR realised in the sample equal to 0.14. Figure 9.4 displays the histograms of the McMC posterior distribution of these Bayes-FDR. The vertical

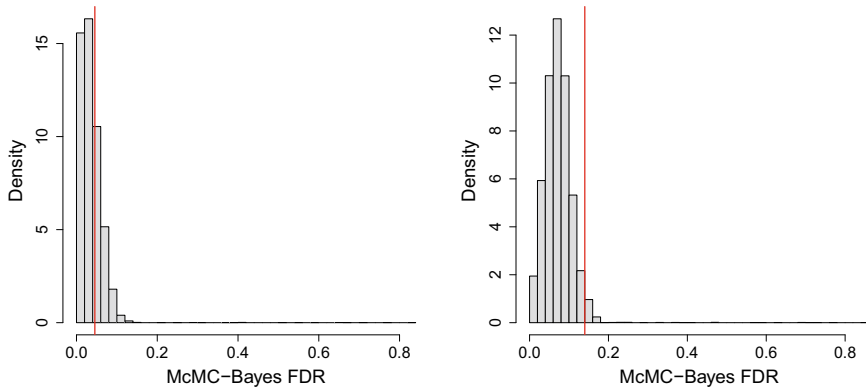


Fig. 9.4 Histograms of the McMC Bayes posterior distribution of the Bayes-FDR choosing as discovery set the set of markers whose marginal posterior probability of non-zero effects is larger than 0.80 (Left) or larger than 0.56 (Right). The vertical lines show the true FDR realised in the simulated data

lines show the realised (true) FDR obtained in the particular simulation for each discovery set.

The R-code to produce the histograms and Monte Carlo estimates of posterior means and posterior intervals of the false discovery rates is shown on page 397.

Several other pieces of information can be extracted from the Bayesian model. For each marker one can extract the probability of a non-zero effect conditional on observed data and the draws from (7.57) retrieve a Monte Carlo estimate of the posterior probability distribution of this probability of non-zero effect. For example, the Monte Carlo estimate of the mean of the posterior probability that marker 141 is non-zero is 0.83 and the Monte Carlo estimate that this probability is larger than 0.6, say, is 0.96. A similar picture can be obtained for the local false discovery rate for each marker genotype in the discovery set.

Prediction

The 2000 binary records are divided in equal proportions into training and validating sets. The criterion to evaluate predictive ability is the proportion of mistaken predictions in the validating data or error rate, given by

$$\text{MSE}_v = \frac{1}{N_v} \sum_{i=1}^{N_v} (y_i - \hat{y}_i)^2$$

where $N_v = 1000$ is the number of records in the validating set, y_i is the i th record in the validating set (0 or 1) and $\hat{y}_i$ is the predicted value computed using (9.3) using $t = 0.5$. I also used the *Brier score*

$$\text{MSE}_v^B = \frac{1}{N_v} \sum_{i=1}^{N_v} (y_i - \hat{\pi}_i)^2 \quad (9.27)$$

that uses additional information from the predicted probabilities $\hat{\Pr}(Y_i = 1|\mu) = \hat{\pi}_i$ and has therefore more discriminating power.

The validating MSE_v of the spike and slab model, the penalised logistic regression and the logistic lasso are 0.35, 0.35 and 0.30, respectively. For the Brier score (9.27), the corresponding figures are 0.25, 0.30 and 0.20.

The Bayesian McMC implementation produces automatically a Monte Carlo estimate of the marginal posterior distribution of the validating MSE_v . The McMC estimate of the 95% posterior interval of the validating MSE_v is (0.321; 0.386) and for the Brier score, (0.227; 0.275). This quantifies the uncertainty in the measures of MSE due to uncertainty in the predictors, conditional on the data.

In order to have a point of reference, these error rates can be compared to the error rate of the null model, defined as

$$u_i = \mu + \varepsilon_i, \quad i = 1, 2, \dots, 2000,$$

that results in a loglikelihood equal to

$$\ell(\mu|y) = \sum_{i=1} y_i \mu - n \ln [1 + \exp(\mu)],$$

where n is the number of records in the data set. A differentiation yields

$$\begin{aligned} \frac{d \ell(\mu|y)}{d\mu} &= \sum_{i=1} y_i - n \frac{\exp(\mu)}{1 + \exp(\mu)} \\ &= \sum_{i=1} y_i - n \Pr(Y_i = 1|\mu). \end{aligned}$$

On setting the derivative equal to zero,

$$\hat{\Pr}(Y_i = 1|\mu) = \sum_{j=1} y_j / n = \hat{\pi}, \quad \text{for all } i,$$

equal to the proportion of 1's in the data, a result that also holds for the probit model. The predicted value is

$$\hat{y}_i = \begin{cases} 1 & \text{if } \hat{\pi} > 0.5 \\ 0 & \text{if } \hat{\pi} \leq 0.5 \end{cases} \quad \text{for all } i.$$

Depending on the value of $\hat{\pi}$ this amounts to setting all the predictions to either 0 or 1. Therefore, the error rate of the null model is $\hat{\pi}$ or $1 - \hat{\pi}$, whichever is smaller. In this example, the null model leads to an error rate equal to 0.35, identical to that obtained for the spike and slab model and a little higher than the value obtained using the logistic lasso. The Brier score for the null model is $\frac{1}{n_v} \sum_i (y_i - 0.35)^2 = 0.23$.

When the data are more informative things look a little better. For example, increasing the total number of records to 5000 and keeping 1500 markers, MSE_v for the spike and slab model, the penalised logistic regression and the logistic lasso are 0.29, 0.38 and 0.27, respectively. Using the Brier score, the values for MSE_v^B are 0.19, 0.35 and 0.18.

The kernel of the R-code for running the logistic lasso using the package **glmnet** is shown below.

```
# THE TRAINING DATA y AND MATRIX OF COVARIATES X
# BOTH CREATED IN A SECTION OF THE CODE NOT SHOWN
library(glmnet)
train=sample(1:nrow(X),nrow(X)/2)
test=(-train)
y.test=y[test]

# STEP 1

cv.out=cv.glmnet(X[train,],y[train],alpha=1,intercept=TRUE,
family="binomial",type = "class")
plot(cv.out)
bestlam=cv.out$lambda.min
bestlam
#NUMBER NON-ZERO COVARIATES:
length(which(as.vector(coef(cv.out,s=bestlam))!=0))

# STEP 2

# OBTAIN PREDICTIONS BASED ON CLASS LABELS
fm.predclass=predict(cv.out,s=bestlam,newx=X[test,],
family="binomial",type="class")

# OBTAIN PREDICTIONS BASED ON PROBABILITIES (Brier score)
fm.predresp=predict(cv.out,s=bestlam,newx=X[test,],
family="binomial",type="response")
#ERROR RATE (CLASS LABELS):
mean((as.numeric(fm.predclass)-y.test)^2)
#ERROR RATE (FITTED PROBS):
mean((as.numeric(fm.predresp)-y.test)^2)

# ERROR RATE OF NULL MODEL: liability = mu + e
mean(y.test)
```

The kernel of the R-code for executing the penalised logistic regression using gradient descent is shown below.

```

nit <- 1500 # Number of gradient descent iterations

nrep <- 5 # Number of replications
# (To compute variability of MSE across replicates)

#lambda <- 0.0 # ZERO PENALTY !!!!!!!
lambda <- 0.4

gama <- 0.008 # Learning rate
c <- rep(0,nit)
newcost <- rep(0,nit)
newcostv <- rep(0,nrep)

result <- matrix(data=NA, nrow=nit,ncol=8)
resulttv <- matrix(data=NA, nrow=nit,ncol=8)
resulttvnr <- matrix(data=NA, nrow=nitnr,ncol=8)

res <- matrix(data=NA, nrow=nrep,ncol=9)
resnr <- matrix(data=NA, nrow=nrep,ncol=8)

msev <- rep(NA,nrep)
msevnr <- rep(NA,nrep)

#####
# FUNCTION TO COMPUTE Pr[Y = 1]
probl <- function(miu,beta,X){
  pr <- exp(miu+X%*%beta)/(1+exp(miu+X%*%beta))
}
# FUNCTION TO COMPUTE THE LOSS FUNCTION
cost <- function(miu,beta,X,y)
  {-sum(y*(miu+X%*%beta) - log(1 + exp(miu+X%*%beta)))
  + crossprod(beta)*(lambda/2)}
c[1]<- cost(miu,beta,X,y)
ptm <- proc.time()

#####
##### GRADIENT DESCENT #####
### FIT MODEL TO TRAINING DATA; TEST IN VALIDATING DATA

set.seed(771311)
numone <- sum(y)
numzero <- length(y) - numone
nindiv <- length(y)
ptm <- proc.time()
for (i in 1:nrep){
  cat(i, "\n",sep="")
  train=sample(1:nrow(X),floor(0.5*nrow(X)))
  Xtrain <- X[train,]
  Xval <- X[-train,]
  ytrain <- y[train]
  yval <- y[-train]
  miu <- 0.0
  beta <- rep(0.0,ncol(Xtrain))

```

```

for(j in 1:nit){
  fdmiu <- -sum(ytrain - prob1(miu, beta,Xtrain))
  fdbeta <- -t(Xtrain) %*%
    (ytrain - prob1(miu, beta,Xtrain)) + lambda * beta
  fd <- matrix(c(fdmiu,fdbeta),nrow=length(beta)+1,ncol=1)
  sol0 <- matrix(c(miu,beta),nrow=length(beta)+1,ncol = 1)
  sol1 <- sol0 - gama * fd
  miu <- sol1[1,1]
  beta <- sol1[-1,1]
  newcostv[j] <- cost(miu, beta,Xtrain,ytrain)
  resulttv[j,] <- c(j,newcostv[j],miu,beta[1:5])
}
probval <- prob1(miu,beta,Xval)
y_predval <- as.numeric(ifelse(probval > 0.5,1,0))
msev[i] <- mean((y_predval-yval)^2)
res[i,] <- c(i,j,newcostv[j],miu,beta[1:5])
}
proc.time()-ptm
tail(resulttv)
tail(res)
summary(msev) # SUMMARY MISCLASSIFICATION ACROSS REPLICATES

```

The complete R-code that performs the computations of this example is on the GitHub site in the folder named “Codes”. The web address is <https://github.com/SorensenD/SLGDS>. The specific R-code is labelled CODE0901.

Example: Rare Diseases: Gene Discovery and Prediction

An important topic of modern genomic studies is prediction of a disease before its onset to help with early diagnosis and prevention. This example illustrates some of the issues that may arise in such studies. A setup could be as follows. An analyst has access to randomly drawn data from a homogeneous population of nominally unrelated individuals, recorded for the presence/absence of a particular disease; individuals are genotyped for a large number of genetic markers. One of the objectives could be to incorporate some or all the genetic markers in a prediction model and to test the model as a tool for detecting susceptible individuals before the onset of the disease.

The prevalence of the disease in the population is approximately 9%. An estimate of heritability on the underlying scale ($h^2 = 0.33$) using traditional methods is available from previous studies. Using these data an analyst poses the following questions:

1. How many genetic markers influencing the trait, if any, are detected, and how is this detection accomplished?
2. What proportion of the variance on the underlying scale is captured by the regression on markers?

3. How do genotype profiles at the detected loci differ between two individuals with low and high estimated probabilities of showing the disease?
4. Once a prediction model is constructed, should the predictor include the “significant” markers only or should all the markers be incorporated? What justifies the decision?
5. If genetic marker information is incorporated, how should the model be tested/validated?
6. How can a measure of uncertainty be attached to the predicted probabilities of disease?

Data consist of 5000 individuals genotyped for 15,000 genetic markers with observed phenotypic binary records $y_i = (0, 1)$. The 5000 individuals are randomly divided into training (y_t) and validating data (y_v), 2500 in each. Training data y_t are analysed with the following hierarchical spike and slab model described on page 414:

$$\Pr(y_{t,i} = 1 | b, x_{t,i}) = \Phi(\mu + x'_{t,i}b),$$

where $\Phi(\cdot)$ is the cumulative distribution function of the standard normal, μ is an unobserved mean assigned an improper uniform prior distribution, $x'_{t,i}$ is the i th row of an observed matrix x of genetic markers of dimension $2500 \times 15,000$ and b is the unobserved vector of 15,000 marker effects. Marker effects are assumed independently drawn from the mixture prior

$$b_j | \sigma_b^2, \pi \sim N(0, \sigma_b^2) \pi + \Delta_0 (1 - \pi),$$

where the scalar σ_b^2 is the uncertainty variance of a marker effect drawn from the normal distribution, π is the a priori proportion of markers originating from the normal component in the population, and Δ_0 is a point probability mass centered at zero. The final layers of the hierarchy are

$$\begin{aligned} \pi | \alpha, \beta &\sim Be(\alpha, \beta), \\ \sigma_b^2 &\sim \text{scale inverted } \chi^2(S_b, v_b), \end{aligned}$$

where $Be(\alpha, \beta)$ represents a beta distribution with user-tuned parameters α and β chosen to generate a prior mode in the proximity of 0.001, S_b is the scale of the scale inverted chi-square prior distribution of σ_b^2 and v_b the degrees of freedom. Guided by Eq. (7.40), the latter is set equal to 4.1 and the scale is set equal to 0.21 to generate a prior modal value of σ_b^2 approximately equal to 0.14.

The spike and slab model is implemented using the McMC algorithm described on page 414.

The data generating model or true model, unknown to the analyst, is as follows. At the level of the liability u_i , for individual i ,

$$u_i = \mu + x'_{Qi} b_Q + a_i + \varepsilon_i, \quad i = 1, 2, \dots, n = 5000, \quad (9.28)$$

where the mean μ is chosen so that $\Pr(Y_i = 1|\mu) = 0.05$, x'_{Qi} is the i th row of a matrix x_Q of QTL (causal) genotypes of dimension $n \times (nqtl = 10)$, b_Q is the $nqtl \times 1$ vector of additive genetic effects of the QTL genotypes, a_i is the effect of additive genetic contributions from an unknown number of causal loci, and $\varepsilon_i \sim N(0, 1)$ is the effect of a normally distributed random residual term. When individuals are nominally unrelated, a_i and ε_i can be grouped in a single term $e_i \sim N(0, (1 - r_Q) \sigma_a^2 + 1)$, where $\sigma_a^2 = 0.5$ is the additive genetic variance at the level of the liability and r_Q is the true proportion of the total additive genetic variance captured by the $nqtl$ QTL . Given the liability u_i , the model that generates the data assumes

$$\begin{aligned} \Pr(y_i = 1|u_i) &= \Pr(u_i > 0|\mu, b_Q) \\ &= \int_{-\infty}^{\mu + x'_{Qi} b_Q} N(e_i|0, (1 - r_Q) \sigma_a^2 + 1) de_i \end{aligned}$$

and an observation from the training or validating set is drawn from the Bernoulli sampling model

$$y_i | \Pr(y_i = 1|u_i) \sim \text{Bin}(1, \Pr(y_i = 1|u_i)).$$

The size of the $nqtl$ effects of the vector b_Q is chosen to generate an additive genetic variance equal to $r_Q \sigma_a^2$.

The elements of the matrix x_Q of QTL genotypes are part of the matrix x of genetic markers. The $nqtl$ loci are randomly chosen among the 15,000 markers and assigned as QTL loci. The elements of the $n \times nqtl$ QTL loci are independently drawn from a binomial distribution $\text{Bin}(2, p = 0.05)$. Those from the $n \times (15,000 - nqtl)$ non- QTL marker loci, are independently drawn from a binomial distribution $\text{Bin}(2, p = 0.5)$. Thus, genetic markers are present at intermediate frequencies and QTL loci at low ($p = 0.05$) frequencies.

Two scenarios are considered that differ in the true data generating model. In scenario I, $r_Q = 1$ and the $nqtl = 10$ QTL loci explain 100% of the additive genetic variance σ_a^2 . In this case the 10 causal marker loci can be detected despite their low frequency in the sample of 2500 training observations. This illustrates a case where the experiment has high power, the prediction ability of the model is high and results are relatively clear and uncontroversial. In scenario II, $r_Q = 0.4$ and the $nqtl = 10$ QTL loci explain 40% of the additive genetic variance σ_a^2 . This leads to a lower probability of detection of the QTL loci and the prediction ability of the model is compromised.

Before addressing the six questions posed above, a little intuition for the problem at hand can be obtained as follows. Consider one of the 10 causal marker loci. Assuming Hardy-Weinberg equilibrium the three possible genotypes $x_0 = 0$, $x_1 = 1$, $x_2 = 2$ occur with expected frequencies $(0.95)^2$, $2(0.95)(0.05)$ and $(0.05)^2$. The expected number of the three genotypes in a sample of 2500 individuals is 2256.25, 237.5, 6.25. The three conditional probabilities of the event occurring, given the population mean $\mu = -1.64$ and the *QTL* locus are $\Phi(\mu + bx_0)$, $\Phi(\mu + bx_1)$, $\Phi(\mu + bx_2)$. In scenario I of the exercise, where the 10 *QTL* loci explain 100% of the additive genetic variance of the trait on the underlying scale, $b = 0.72$. This translates into the following three values for the three conditional probabilities: 0.05, 0.17, 0.42, with a posterior standard deviation of the largest of these three probabilities, accounting for uncertainty of marker effects b , of the order of 0.06. Most of the information in the data for drawing inferences about these probabilities is contributed by individuals with genotypes x_0 and x_1 (because only 6 individuals, approximately, are expected to carry genotype x_2). Very few cases are expected with individuals carrying genotypes $x = x_2$ at more than one or two of their ten *QTL* loci. In scenario II of the exercise where the 10 *QTL* loci explain 40% of the additive genetic variance of the trait on the underlying scale, the allele substitution effect $b = 0.46$. The three conditional probabilities of the event occurring are now 0.05, 0.12, 0.23. The posterior standard deviation of the largest of these three probabilities, accounting for uncertainty of marker effects b , is of the order of 0.10. This has a strong impact on *QTL* detection and on the evaluation of the model's predictive ability.

Scenario I

1. Detection of genetic variants is of interest for development of drug therapies targeted to particular metabolic pathways and for follow-up functional analysis. The results of a GWAS with a Bonferroni adjustment fitting one marker at a time for all 15,000 genetic markers are shown in the left panel of Fig. 9.5. The right panel displays similar information based on the spike and slab model. GWAS detects 9 out of the 10 *QTL* with zero true false positives (a detail unknown to the analyst); the spike and slab model detects all the 10 *QTL* with zero true false positives. This information is supplemented with the computation of false discovery rates using the Benjamini and Hochberg (FDR-BH) approach and the fully Bayesian approach. Implementation of the FDR-BH rule, setting q equal to 0.10 (see page 366) leads to a discovery set equal to 12 markers, with 2 true (unknown to the analyst) false positives, and 10 true discoveries. The fully Bayesian approach, using the right panel of Fig. 9.5 to guide the choice of the discovery set based on the estimate of the posterior probability that the marker is drawn from the slab, equal to 0.8, leads to a discovery set of 10 markers, with 0 true false positives. The estimate of the posterior mean of the Bayesian FDR is 0.01, with a 95% posterior interval equal to (0.00; 0.1). The Bayesian result indicates that the estimated average of the posterior distribution of the number

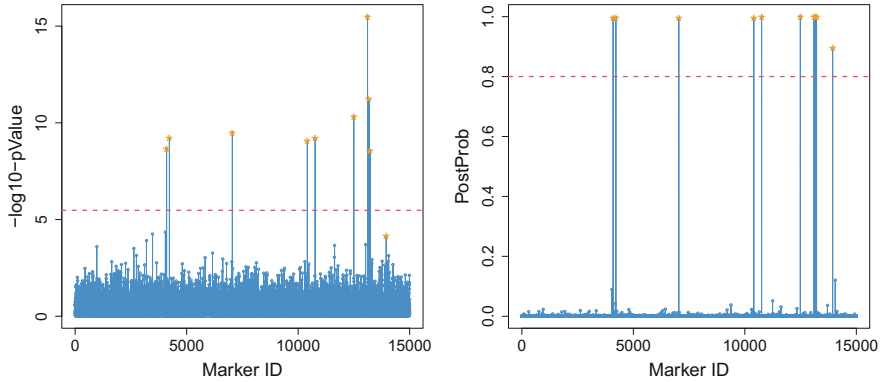


Fig. 9.5 Detection of promising marker genotypes for scenario I; 2500 training observations. Left: GWAS with a Bonferroni correction; 9 out of 10 markers detected; 0 true false discoveries. The horizontal *red line* is the Bonferroni threshold corresponding to $-\log_{10}(0.05/15,000)$. Right: spike and slab model; 10 out of 10 markers detected; 0 true false discoveries. The horizontal *red line* is the arbitrary threshold corresponding to a posterior probability equal to 0.8. *Orange stars* represent the 10 true genes (causal variants)

of false discoveries in the set of 10 markers that constitute the discovery set is $0.01 \times 10 = 0.1$. With the information available the analyst decides to build a prediction model incorporating the discovery set based on the 10 genetic markers detected by the spike and slab model.

2. The estimated posterior mean and 95% posterior interval of the genomic variance on the underlying scale based on the spike and slab model using all markers is 0.56, (0.43, 0.72). This is in quite good agreement with the true value equal to 0.5. The posterior distribution of the proportion of variance on the underlying scale explained by all the 15,000 markers (or genomic heritability) has mean and 95% posterior interval equal to $0.56/1.56 = 0.36$ (0.30; 0.42). The estimated posterior mean and 95% posterior interval of the genomic variance on the underlying scale based on the 10 detected markers is 0.52, (0.41, 0.64), and the corresponding figures for the genomic heritability are 0.34 (0.29; 0.39).
3. The spike and slab model outputs Monte Carlo estimates of $\Pr(Y_i^* = 1|x_i, y_t)$; these are posterior predictions, accounting for uncertainty of marker effects b . The posterior mean (95% posterior interval) of the highest scoring probability including all markers (that happens to correspond to individual 889) is 0.94 (0.78; 0.99). The posterior mean (95% posterior interval) of the lowest scoring probability including all markers (that happens to correspond to individual 2300) is 0.008 (0.001; 0.018). The genotype profiles at the detected marker loci for individual 889 is (before column centring)

1002001011

and for individual 2300

0000000000.

Individual 889 has one copy of a “disease causing gene” at 4 of the 10 detected loci, and two copies at one locus. Individual 2300 does not have any “disease causing alleles”. An individual that has an estimated probability $\Pr(Y_i^* = 1|x_i, y_t) = 0.085$, corresponding to the mean of the validating data, has a genotypic profile at the detected marker loci

0000001000.

This result could have implications for personalised medicine.

4. Figure 9.6, left panel, shows good agreement between predictions based on all markers and those based on detected markers only. This is hardly surprising in this example where the *QTL* account for all the genomic variance on the underlying scale of the trait. However the right panel indicates how the variance of the predictor increases with the incorporation of a large number of covariates. Each point represents either the posterior mean (left panel) or the posterior variance (right panel, variances computed using all markers) of the prediction for a particular individual, for all the 2500 individuals in the validating data. In the case of this example, the conclusion is that only the detected markers should be included in the construction of the predictor. This may not be the case when adding more markers contribute with relatively more information than noise.

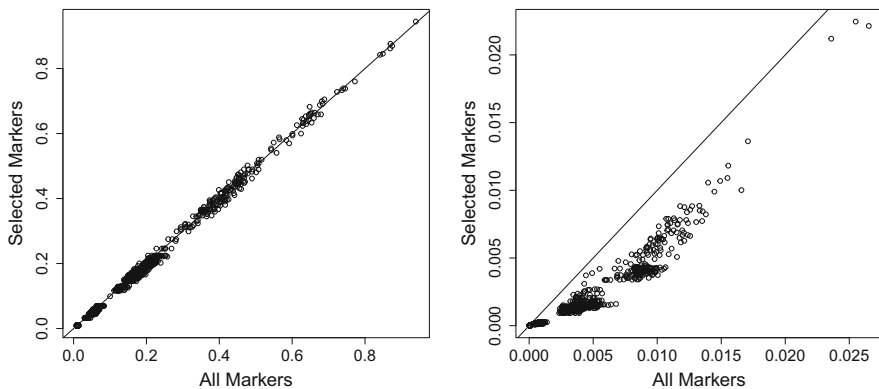


Fig. 9.6 Left: McMC estimates of posterior means of $\Pr(Y_i^* = 1|x_i, y_t)$ based on a model that includes either the 10 detected markers or the complete marker panel (scenario I). Right: McMC estimates of posterior variances of $\Pr(Y_i^* = 1|x_i, y_t)$ based on a model that includes either the 10 detected markers or the complete marker panel. True model: 10 *QTL* account for 100% of the genomic variance

Table 9.1 Point estimates of validating misclassification rates and of validating Brier scores incorporating (1) all 15, 000 markers; (2) only the 10 detected markers; (3) using a subset of the highest or (4) lowest predicted probabilities $\Pr(\hat{Y}_v = 1|x, y_t)$ computed using the 10 detected markers, and (5), using the null model with all the validating data points $\Pr(\hat{Y}_v = 1|\hat{\mu}, y_t)$, where $\hat{\mu}$ is the estimate of the posterior mean of μ

	Misclassification	Brier score
All markers	0.078	0.062
Detected markers	0.078	0.062
Highest prob	0	0.017
Lowest prob	0	0.005
Null model All	0.085	0.078

5. To study the model's prediction ability several exploratory measures are investigated. Most of these originate from various outputs of the MCMC implementation of the spike and slab model. The first two are the validating mean squared error (proportion of misclassifications) using the Bayes classifier to transform probabilities into values 0 or 1, generating in this way predicted data $\hat{Y}$, and the Brier score. Both are compared to the validating mean squared error and Brier score based on the null model $\Pr(Y = 1|\mu)$ as benchmark, where μ is replaced by the estimate of the posterior mean of μ . These two criteria are implemented using predicted probabilities computed with all the 15, 000 markers, or with the 10 selected markers only. In addition, the same criteria are applied using the highest and lowest predicted probabilities generated with the selected markers. Finally, logscores evaluated at the estimated mean of the posterior distributions of the predicted probabilities are calculated based on either all the 15, 000 markers, or with the 10 selected markers only.

Results are displayed in Table 9.1. The same qualitative conclusions are drawn from the misclassification rates or the Brier score. The point estimates of validating mean squared error are computed using

$$\text{MSE}_v = \frac{1}{n_v} \sum_{i=1}^{n_v} (y_{v,i} - \hat{y}_{v,i})^2. \quad (9.29)$$

In this expression $n_v = 2500$, $y_{v,i}$ is the i th validating datum, $\hat{y}_{v,i}$ is its prediction that takes values 0 or 1 according to $I \left[\hat{\Pr}(y_{v,i} = 1|x_i, y_t) > 0.5 \right]$, where $I(\cdot)$ is the indicator function that takes the value 1 if the argument is satisfied and 0 otherwise, y_t is the training data and $\hat{\Pr}(y_{v,i} = 1|x_i, y_t)$ is the MCMC estimate of the posterior mean $E_{b|x_i, y_t} [\Pr(y_{v,i} = 1|b, x_i, y_t)] = \Pr(y_{v,i} = 1|x_i, y_t)$ for datum i . The Brier score replaces $\hat{y}_{v,i}$ in (9.29) with the MCMC estimates of the posterior means $\hat{\Pr}(y_{v,i} = 1|x_i, y_t)$. Predictions based on the complete marker panel or on the detected markers only (first two rows of the table) lead to the same estimates of validating mean squared error and Brier score. On the face of it, parsimony would dictate choosing a predictor based on the 10

detected markers. However, the performance of the model using the complete set of markers is only marginally superior to the performance of the null model (MSE_v equal to 0.078 and 0.085, respectively). This is typically the case with binary data where the event occurs with very low probability. More convincing evidence in favour of the model's predictive ability can be sought by studying the misclassification rates and the Brier score based on the model that incorporates the 10 detected markers, choosing extreme values of the estimated predictions. The misclassification rates and Brier score among the 10 highest and lowest predictions are 0 and close to the minimum possible values, respectively (rows 3 and 4 in the table). The 10 validating data points corresponding to these 10 highest and lowest predictions are all equal to 1 and to 0, respectively.

The average of the ten highest estimated predictions $\hat{\Pr}(y_v = 1|x, y_t)$ is 0.87 and the average of the ten lowest is 0.0091. These are respectively, 9.7 times larger and 10 times lower than the prediction based on a randomly sampled individual.

The negative logscore is computed using

$$-\sum_{i=1}^{n_v} \left(y_{v,i} \log(\hat{P}_i) + (1 - y_{v,i}) \log(1 - \hat{P}_i) \right), \quad (9.30)$$

where $\hat{P}_i = \hat{\Pr}(y_{v,i} = 1|x_i, y_t)$, the McMC estimate of the mean of the posterior predictive probability of the i th validating datum. When the validating datum $y_{v,i} = 1$, if $\hat{P}_i$ is close to 1, the record makes a minimal contribution to the logscore. The same occurs when the validating datum $y_{v,i} = 0$ and $\hat{P}_i$ is close to 0. Discrepancies between the observed validating data and their predictions lead to relatively large contributions. In the present scenario, the negative logscores in a model that uses all the 15,000 markers, the 10 detected markers and the null model, are respectively 536.7, 535.9 and 760.2. These figures agree with those in Table 9.1, supporting the claim that genetic markers contribute to the model's prediction ability, and that there is no extra benefit to include other than the 10 detected markers in the construction of the predictor.

As a further analysis of the predictive ability of the model constructed with the 10 detected loci, using output from the McMC algorithm, one can obtain an estimate of the conditional probability

$$\Pr[(\Pr(\hat{y}_v = 1|x, y_t) > 2 \Pr(\hat{y}_v = 1)) | y_v = 1]. \quad (9.31)$$

In words, among those individuals in the validating data that show the disease ($y_v = 1$), what proportion of the estimated posterior predicted probabilities ($\Pr(\hat{y}_v = 1|x, y_t)$) is larger than twice the probability of observing disease in an individual randomly sampled from the population ($2 \Pr(\hat{y}_v = 1)$)? The answer

is 0.72. Similarly, given that the true state of nature is $y_v = 0$, the model predicts a disease outcome with probability

$$\Pr \left[(\Pr(\hat{y}_v = 1 | x, y_t) > 2 \Pr(\hat{y}_v = 1)) | y_v = 0 \right],$$

equal to 0.18, smaller than (9.31) by a factor of 4.

With the evidence presented one can recommend using the model as a prediction tool on an exploratory basis. Sharper inferences should be possible as further data are collected.

6. The McMC algorithm that implements the spike and slab model outputs draws from the posterior distribution $\Pr(y_{v,i} = 1 | x_i, \mu, b, y_t) = \Phi(\mu + x_i' b)$, where (μ, b) is a draw from $[\mu, b | y_t]$. The variance among these draws is a consistent estimator of the posterior variance of $\Phi(\mu + x_i' b)$, that reflects the posterior uncertainty of (μ, b) propagating on to the posterior distribution of $\Phi(\mu + x_i' b)$. As an illustration, consider again the posterior mean (95% posterior interval) of the highest scoring probability including only the 10 detected markers (that happens to correspond to individual 889, the same as in point 3 above). With only 10 markers included in the construction of $\Phi(\mu + x_i' b)$, this probability is 0.94 (0.83; 0.99); the posterior interval is a little narrower than the value (0.78; 0.99) obtained using all markers.

Scenario II

1. The results of the GWAS and of the spike and slab model are shown in Fig. 9.7, on the left and right panels, respectively. Two out of the 10 markers reach beyond the

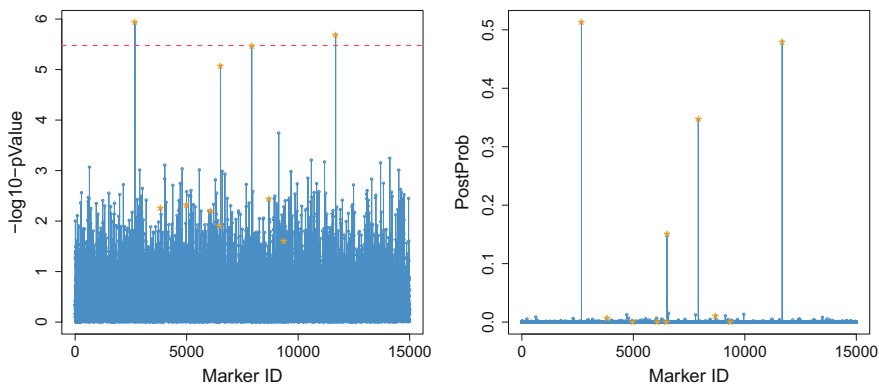


Fig. 9.7 Detection of promising marker genotypes for scenario II; 2500 training observations. Left: GWAS with a Bonferroni correction; 2 out of 10 markers detected (marker 7919 on the border of the detection threshold); 0 true false discoveries. The horizontal red line is the Bonferroni threshold corresponding to $-\log_{10}(0.05/15,000)$. Right: spike and slab model; Orange stars represent the 10 true genes (causal variants)

Bonferroni threshold and both are true positives (left subfigure). The four highest posterior probabilities that markers have an effect on the trait estimated with the spike and slab model (right subfigure, all true positives) are 0.51, 0.15, 0.35 and 0.48. Implementation of the Benjamini and Hochberg false discovery rate algorithm (FDR-BH), setting the expected proportion of false positives q equal to 0.15, leads to a discovery set of 4 markers (all happen to be true discoveries). The result is unchanged using $q = 0.5$. These are the same 4 markers that reach the highest probabilities in the analysis based on the spike and slab model. Implementation of the fully Bayesian FDR with a discovery set defined by those 4 markers that reach estimated probabilities larger than 0.15, outputs an estimate of the false discovery rate with a mean of 0.6 and an estimated 95% posterior interval equal to (0.00; 1.00). The large value of the estimated FDR is due to the low estimates of the posterior probability. For example, if the discovery set only includes the marker with highest estimate of the posterior probability (this is marker 2681, with estimate equal to 0.51), the estimate that a false discovery has been made is $1 - 0.51 = 0.49$. The large estimated uncertainty is due to the poor information content in the data to estimate the marker effects. This leads to strong fluctuations in the estimates of the posterior probabilities, that propagate on to the estimates of FDR. With these shortcomings in mind and despite the limited size of the genetic markers' posterior probabilities of having an effect on the trait, the analyst decides to construct a model based on the 4 detected markers.

2. The estimated posterior mean and 95% posterior interval of the genomic variance on the underlying scale based on the spike and slab model using all the 15,000 markers is 0.04 (0.00; 0.10). The corresponding figure based on the 4 detected markers is 0.03 (0.00, 0.08). The analyst does not feel encouraged by these figures bearing in mind the prior estimate of genomic variance equal to 0.5. The true (unknown to the analyst) proportion of the additive genetic variance explained by the 10 *QTL* is $0.5 \times 0.4 = 0.2$. Note that assuming independent loci, 4 *QTL* should generate a genomic variance equal to 0.08 square units; this is the value (unknown to the analyst) against which the estimate 0.03 should be compared.
3. The extreme values (and their 95% posterior intervals) of the estimated posterior predicted probabilities of showing the disease $\hat{E}_{b|y_t}(\Pr(Y_i^* = 1|b, x, y_t)) = \hat{\Pr}(Y_i^* = 1|x, y_t)$ using the 4 selected marker genotypes are 0.24 (0.07; 0.54) and 0.069 (0.05; 0.09), where the confidence interval reflects posterior uncertainty of marker effects b propagating on to the posterior predictive probabilities (in comparison, the probability of randomly drawing a diseased individual from the population is 0.089). The genotypes at the 4 detected loci for these extreme values are

and

0000,

respectively. The genotype combination among selected markers that generates a probability of disease approximately equal to 0.089 (corresponding to the probability of randomly drawing a diseased individual from the population) is

0200,

where the second genotype with allele content equal to 2, is the marker genotype with smallest probability of affecting the trait (≈ 0.15) among the 4 selected markers (see right panel of Fig. 9.7). These results indicate that the model cannot be discarded as a potentially useful screening tool.

- Figure 9.8, left panel, shows again good agreement between predictions based on all markers and those based on the 4 detected markers only. The right panel shows that the variance of the predictor increases with the incorporation of the complete set of covariates, although the effect is less marked than in scenario I. For example, the variance of the predictor corresponding to the highest estimated prediction based on the 4 detected markers is

$$Var_{b|y_i} [\Pr(Y_i^* = 1|b, x_i = (2\ 0\ 0\ 1), y_i)] = 0.022, \quad i = 255,$$

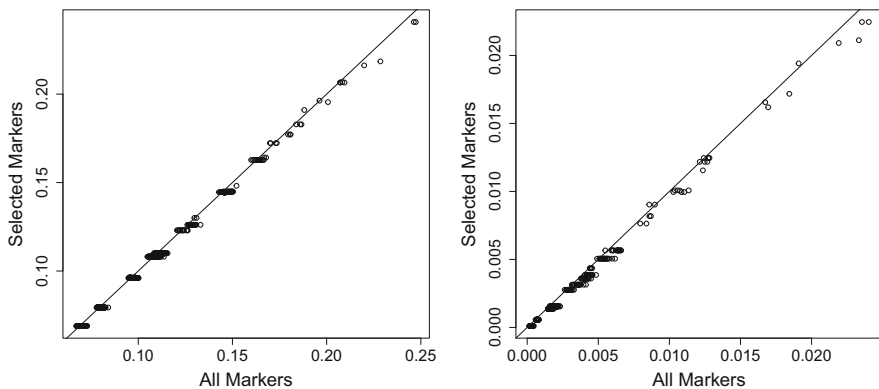


Fig. 9.8 Left: McMC estimates of posterior means of $\Pr(Y_i^* = 1|x_i, y_i)$ based on a model that includes either the 4 detected markers or the complete marker panel (scenario II). Right: McMC estimates of posterior variances of $\Pr(Y_i^* = 1|x_i, y_i)$ based on a model that includes either the 4 detected markers or the complete marker panel. True model: 10 *QTL* account for 40% of the genomic variance

whereas the value based on the 15,000 markers is 0.024. As in scenario I, the conclusion must be that only the 4 detected markers should be used in the construction of the predictor.

5. A striking difference between the behaviour of the prediction models in scenario I and scenario II is the range of the predicted probabilities. Predictions in scenario I range from close to 0 to over 0.9; in scenario II, from close to 0 to a little over 0.2 (see left panels of Figs. 9.6 and 9.8). In a situation like in scenario II, misclassification rates (based on Bayes rule, that would amount to setting all predictions equal to 0) is not a useful measure of quantifying the ability of a model to predict single records.

The negative logscore (9.30) is used instead as a global measure of the model's prediction ability. The negative logscores of a model that uses all the 15,000 markers, the 4 detected markers and the null model, are respectively 728.7, 729.3 and 754.00. The result suggests that predictions based on the 4 detected markers perform as well as those based on all 15,000 markers and both do better than the null model.

The next step is to look informally into the ability of the model to predict a single observation. The approach chosen is to compare the 10 largest estimated posterior predictions based on the 4 selected marker loci ($\hat{\Pr}(y_v = 1|x, y_t) \geq 0.18$) with the corresponding validating data. The result is shown below; the first row represents the validating records and the second the corresponding posterior predicted probabilities computed using the training data:

0	0	0	1	0	1	1	0	1	1
0.18	0.22	0.19	0.18	0.21	0.19	0.18	0.19	0.24	0.16

Predictions for $y_v = 0$ average 0.197 and for $y_v = 1$, 0.187; the proportion of diseased individuals in the selected group is 50%, 5.5 times larger than in the population. The conclusion is that individual records are predicted poorly. The model performs better at selecting a subset of the data with a proportion of affected individuals that deviate markedly from the mean proportion in the population.

The final test of the prediction ability of the model incorporating the 4 detected markers is based on calculating what proportion of the estimated posterior predicted probabilities ($\hat{\Pr}(\hat{y}_v = 1|x, y_t)$) among those individuals in the validating data that show the disease, is larger than twice the probability of observing disease in an individual randomly sampled from the population ($2 \Pr(\hat{y}_v = 1)$)? The answer now is 0.036. Similarly, given that the true state of nature is $y_v = 0$, the model predicts a disease outcome with probability

$$\Pr \left[\left(\hat{\Pr}(\hat{y}_v = 1|x, y_t) > 2 \Pr(\hat{y}_v = 1) \right) | y_v = 0 \right],$$

equal to 0.004, smaller than 0.036 by a factor of 9. The model does much better at assigning low probabilities of disease to healthy individuals, than to assign high probabilities of disease to unhealthy individuals.

6. Estimates of the 2500 complete marginal posterior distributions of the probabilities for each validating datum can be easily obtained from the output of the MCMC algorithm. For example, for the largest and smallest probabilities, using either the complete marker panel or the detected markers, the estimated means and variances (in brackets) are as follows. For the largest and smallest, all markers: 0.247 (0.0242) and 0.067 (1.94×10^{-4}). For the largest and smallest, detected markers: 0.241 (0.0224) and 0.069 (1.05×10^{-4}).

The evidence presented indicates that the model cannot be recommended for prediction of individual outcomes but can be potentially useful as a screening tool. A different design based on case-control studies could mitigate some of the shortcomings arising from the low frequency of the disease among randomly sampled data, from the small proportion of genomic variance accounted for by the marker information and from the ensuing low value of the posterior probabilities of non-null effects of individual markers. Adoption of a case-control design may require accounting for non-random sampling. An alternative recent recommendation is to avoid testing individual markers and to focus instead on identifying sets of markers that are jointly associated with the phenotype (Samaddar et al. 2024).

The R-code CODE0907.R executes the GWAS analysis, implements the spike and slab model applied to these examples and computes the classical and Bayesian FDR; it is found at <https://github.com/SorensenD/SLGDS> in the folder Codes.

9.7 Area Under the Curve

Consider binary data and a classifier such as logistic regression that generates predictions or classifications $\hat{Y}$ taking values 0 or 1 according to the rule (9.3), given covariates x . The result of applying the classifier depends critically on the threshold t used. In the case of (9.3), setting $t = 0.5$ leads to Bayes' rule that minimises $\Pr(\hat{Y} \neq Y)$, the *overall error rate*.

The outcome of the binary classifier can be displayed in the so-called *confusion matrix* shown in Table 9.2. For a particular experiment, the proportions that fall in the body of the table are empirical realisations of the 4 joint probabilities that provide 3 independent pieces of information. Different ways of summarising these joint probabilities give rise to different methods of classification.

One way of summarising the 4 joint probabilities in the body of the table is by the following 2 conditional probabilities and 1 marginal probability:

Table 9.2 A typical *confusion matrix* for a binary classifier with the 4 joint probabilities in the body of the table

		True status		Marginal
		$Y = 1$	$Y = 0$	
Predicted status	$\hat{Y} = 1$	$\Pr(\hat{Y} = 1, Y = 1)$	$\Pr(\hat{Y} = 1, Y = 0)$	$\Pr(\hat{Y} = 1)$
	$\hat{Y} = 0$	$\Pr(\hat{Y} = 0, Y = 1)$	$\Pr(\hat{Y} = 0, Y = 0)$	$\Pr(\hat{Y} = 0)$
	Marginal	$\Pr(Y = 1)$	$\Pr(Y = 0)$	
	$\Pr(\hat{Y} = k Y = k)$	$k = 1$: Sensitivity	$k = 0$: Specificity	

1. $\Pr(\hat{Y} = 1|Y = 0)$ known as the *false positive rate*, FPR (type I error), or as false discovery rate. In epidemiology the term $1 - \Pr(\hat{Y} = 1|Y = 0) = \Pr(\hat{Y} = 0|Y = 0)$ (true negative) is referred to as *specificity*
2. $\Pr(\hat{Y} = 1|Y = 1)$ known as the *true positive rate*, TPR (power) or *sensitivity* in epidemiology. The term $1 - \Pr(\hat{Y} = 1|Y = 1) = \Pr(\hat{Y} = 0|Y = 1)$ is known as the *false negative rate*, FNR. These conditional probabilities depend on the threshold t .
3. $\Pr(Y = 1)$ is sometimes known as *prevalence* or *incidence* if 1 means having the condition.

The overall probability of misclassification or *overall error rate* is

$$\begin{aligned}\Pr(\hat{Y} \neq Y) &= \Pr(\hat{Y} = 1|Y = 0) \Pr(Y = 0) + \Pr(\hat{Y} = 0|Y = 1) \Pr(Y = 1) \\ &= \text{FPR} (1 - \Pr(Y = 1)) + \text{FNR} \Pr(Y = 1),\end{aligned}\quad (9.32)$$

equal to the sum of the off-diagonal cells in the body of Table 9.2. Given the incidence in the population, the two components of the overall error rate, FPR and FNR, are critically dependent on the threshold t and they are complementary. As the threshold t is reduced, FPR increases and FNR decreases. A classifier that uses $t = 0.5$ is known to have the lowest overall error rate, but depending on the case at hand, one may choose a threshold to reduce FNR. A typical example is a rule used to classify individuals as diseased or not diseased, where one is more concerned about the error of classifying a diseased individual as not diseased, than about the error of classifying a not diseased individual as diseased. The optimal threshold must be chosen on the basis of the “cost” associated with the disease.

The receiver operating characteristic curve (ROC) for data Y is a graphic often used in engineering, psychology, medicine, to assess the performance of diagnostic systems for simultaneously displaying the two types of errors for all possible thresholds. A ROC curve is constructed by plotting true positive rate (or sensitivity), $\text{TPR}(t) = \Pr(s(x) > t|Y = 1)$, (equal to $1 - \text{FNR}$), versus the false positive rate (or $1 - \text{specificity}$), $\text{FPR}(t) = \Pr(s(x) > t|Y = 0)$, for all possible values of the

threshold t that defines disease status, where the scoring rule $s(x) = \Pr(Y = 1|x)$ is defined in (9.2). The plot is displayed on the unit square and the area under the curve (AUC) represents one measure of the performance of the classifier. Values of AUC close to 1 denote a good classifier and one that performs not better than chance, has an AUC of 0.5. When a ROC curve has an AUC close to 1, there is a high probability that a diseased individual will be correctly classified and a small probability that a healthy individual will be misdiagnosed as diseased. ROC curves for two classifiers display their ability to detect disease status.

An important detail is that TPR is computed using data from the diseased fraction only and FPR is computed using data from the non-diseased fraction only. These quantities are conditional probabilities. Therefore, if these conditional probabilities are well estimated, the ROC plot is independent of the prevalence of the disease in the population. This is in marked contrast to the so-called predicted values that involve the conditional probabilities $\Pr(Y = y|\hat{Y} = \hat{y})$, reflecting the clinical value of the test. The predicted values depend on the prevalence of the disease in the population, $\Pr(Y = y)$, as illustrated below.

Predicted Disease Status

Sensitivity and specificity are often used to describe test performance. From a diagnostic point of view one is interested in knowing how well a test result predicts disease status. The positive predicted value is equal to the conditional probability that an individual has the disease, given a positive test: $\Pr(Y = 1|\hat{Y} = 1)$. An important quantity is $\Pr(Y = 1|\hat{Y} = 0)$, the conditional probability that the disease is present given that the test is negative. These conditional probabilities are obtained using Bayes theorem,

$$\Pr(Y = 1|\hat{Y} = 1) = \frac{\Pr(\hat{Y} = 1|Y = 1) \Pr(Y = 1)}{\Pr(\hat{Y} = 1)}, \quad (9.33)$$

where

$$\begin{aligned} \Pr(\hat{Y} = 1) &= \Pr(\hat{Y} = 1|Y = 1) \Pr(Y = 1) \\ &\quad + \Pr(\hat{Y} = 1|Y = 0) \Pr(Y = 0). \end{aligned}$$

Similarly,

$$\Pr(Y = 0|\hat{Y} = 0) = \frac{\Pr(\hat{Y} = 0|Y = 0) \Pr(Y = 0)}{\Pr(\hat{Y} = 0)}, \quad (9.34)$$

where

$$\begin{aligned}\Pr(\hat{Y} = 0) &= \Pr(\hat{Y} = 0|Y = 0)\Pr(Y = 0) \\ &\quad + \Pr(\hat{Y} = 0|Y = 1)\Pr(Y = 1).\end{aligned}$$

As a specific application, consider a test that has a specificity equal to $\Pr(\hat{Y} = 0|Y = 0) = 1$ and a sensitivity equal to $\Pr(\hat{Y} = 1|Y = 1) = 0.95$. Consider a person who has been exposed to an infected relative and who is evaluated as having a pre-test probability of contracting the disease (prior probability) equal to $\Pr(Y = 1) = 0.2$. Using the complement of (9.34), the post-test probability (posterior probability) of infection, given a negative test result, is $\Pr(Y = 1|\hat{Y} = 0) = 0.012$. With a prior probability equal to $\Pr(Y = 1) = 0.5$, this changes to $\Pr(Y = 1|\hat{Y} = 0) = 0.048$.

On the other hand, if sensitivity of the test is $\Pr(\hat{Y} = 1|Y = 1) = 0.70$, with $\Pr(Y = 1) = 0.2$, the posterior probability is $\Pr(Y = 1|\hat{Y} = 0) = 0.07$ and with a prior equal to $\Pr(Y = 1) = 0.5$, the posterior probability is $\Pr(Y = 1|\hat{Y} = 0) = 0.23$.

Infection Prevalence

The prevalence of a disease is the proportion of individuals in the population that have the disease, $\Pr(Y = 1)$. Imagine that in a random sample, n individuals are tested, that predictions $\hat{Y}$ are generated using some rule $s(x)$ and that T are declared positive. Then the ratio

$$\hat{P}(\hat{Y} = 1) = \frac{T}{n} \tag{9.35}$$

is taken as an estimator of prevalence $\Pr(Y = 1)$. However this estimate is biased unless the test is perfect with specificity and sensitivity equal to 1. To see this, let $\Pr(\hat{Y} = 1)$ denote the expected value of $\hat{P}(\hat{Y} = 1)$. The relationship between $\Pr(\hat{Y} = 1)$ and $\Pr(Y = 1)$ is given by

$$\begin{aligned}\Pr(\hat{Y} = 1) &= \Pr(\hat{Y} = 1|Y = 1)\Pr(Y = 1) + \Pr(\hat{Y} = 1|Y = 0)\Pr(Y = 0) \\ &= sen \Pr(Y = 1) + (1 - spe)(1 - \Pr(Y = 1)) \\ &= (1 - spe) + (sen + spe - 1)\Pr(Y = 1),\end{aligned} \tag{9.36}$$

(Diggle 2011) where $sen = \Pr(\hat{Y} = 1|Y = 1)$ and $1 - spe = \Pr(\hat{Y} = 1|Y = 0)$. If the test is perfect, $spe = sen = 1$ and substituting above, yields $\Pr(\hat{Y} = 1) = \Pr(Y = 1)$. Otherwise with an imperfect test, $\Pr(\hat{Y} = 1)$ is a linear, increasing function of $\Pr(Y = 1)$ (assuming $sen, spe > 0.5$, implying that each is superior to the toss of a coin). A corrected estimator of prevalence (given n, sen, spe) is

$$\frac{\hat{\Pr}(\hat{Y} = 1) - (1 - spe)}{sen + spe - 1}, \hat{\Pr}(\hat{Y} = 1) \geq (1 - spe). \quad (9.37)$$

Example: Estimation of Prevalence Using an Imperfect Test

The examples that follow illustrate four approaches to estimate prevalence and to calculate confidence intervals. The starting point is to assume that the number of individuals declared positive on the basis of an imperfect test, T , is binomially distributed

$$T|n, \Pr(\hat{Y} = 1) \sim \text{Bin}(n, \Pr(\hat{Y} = 1)).$$

In all cases, the imperfect test is implemented on $n = 10,000$ individuals and that $T = 900$ show a positive result. If the test has a sensitivity equal to 0.85 and a specificity equal to 0.95 then the estimate of prevalence based on (9.37) is $\hat{\Pr}(Y = 1) = 0.05$.

In order to use more compact notation it will be convenient to define

$$\begin{aligned} \Pr(\hat{Y} = 1) &= \phi, \\ \Pr(Y = 1) &= \theta, \\ (1 - spe) &= a, \\ (sen + spe - 1) &= b, \end{aligned}$$

and therefore from (9.36)

$$\phi = a + b\theta. \quad (9.38)$$

Given the model,

$$\begin{aligned} E(T) &= n\phi, \\ \text{Var}(T) &= n\phi(1 - \phi), \end{aligned}$$

$$\begin{aligned}\widehat{\Pr}(\hat{Y} = 1) &= \hat{\phi} = \frac{T}{n}, \\ E(\hat{\phi}) &= \phi, \\ \text{Var}(\hat{\phi}) &= \frac{\phi(1-\phi)}{n}, \\ \widehat{\text{Var}}(\hat{\phi}) &= \frac{\hat{\phi}(1-\hat{\phi})}{n},\end{aligned}$$

The **first approach** uses the binomial model and computes initial bounds for the confidence interval for ϕ which are then adjusted based on Eq. (9.38). The adjustment results in a confidence interval for θ . The initial bounds are approximated using the estimate $\widehat{\Pr}(\hat{Y} = 1) = T/n$ in lieu of the true parameter $\Pr(\hat{Y} = 1)$. The unadjusted lower and upper bounds are derived from the quantiles of the binomial distribution executing the R code:

```
lb <- qbinom(0.025, n, (T/n))
ub <- qbinom(0.975, n, (T/n))
```

and the corresponding 95% (frequentist) confidence interval for $\Pr(Y = 1)$ is

$$\Pr\left[\frac{lb - n(1 - spe)}{n(sen + spe - 1)} < \theta < \frac{ub - n(1 - spe)}{n(sen + spe - 1)}\right] = 0.95.$$

For example, for $T = 900$ tested positive out of $n = 10,000$ with $spe = 0.95$ and $sen = 0.85$, the above gives

$$\Pr(0.043 < \theta < 0.057) = 0.95. \quad (9.39)$$

The calculation above assumes that spe and sen are known without error. The **second approach** incorporates uncertainty of these parameters adopting a Bayesian approach and is expected to result in a confidence interval wider than that given by (9.39).

As shown below it is quite simple to implement a Monte Carlo based Bayesian approach. The strategy adopted is to parametrise the model in terms of (ϕ, sen, spe) , to draw posterior samples from $[\phi, sen, spe|t, n]$ and to construct θ from these draws using (9.38) at each round. These are draws from the desired posterior distribution $[\theta|t, n]$. This Monte Carlo method is known as *composition* (see page 164). The Bayesian model is

$$p(\phi, sen, spe|t, n) \propto p(\phi, sen, spe) p(t|n, \phi). \quad (9.40)$$

Assume that ϕ , sen and spe are a priori independent, with $\phi \sim Un(0, 1)$, $sen \sim Un(0.8, 1)$ and $spe \sim Un(0.925, 1)$. Thus sen is constrained in the interval

$[0.8, 1]$ and spe in the interval $[0.925, 1]$. With these assumptions,

$$p(\phi, sen, spe|t, n) \propto p(sen) p(spe) p(t|n, \phi) \quad (9.41)$$

where the binomial likelihood is of the form

$$p(t|n, \phi) \propto \phi^t (1 - \phi)^{n-t}. \quad (9.42)$$

Viewed as a function of ϕ , this is the kernel of a beta distribution $Be(t + 1, n - t + 1)$. For $n = 10,000$ and $t = 900$ this is a very peaked (informative) likelihood. The algorithm proceeds as follows:

1. Draw sen from $Un(0.8, 1)$
2. Draw spe from $Un(0.925, 1)$
3. Draw ϕ from $Be(t + 1, n - t + 1)$
4. Calculate θ from (9.38) and repeat from 1 until the desired chain length is achieved.

Steps 1 and 2 could be replaced with scaled beta distributions $Be(a, b)$ where a and b are chosen to generate the appropriate mode (for example, 0.85 for sen and 0.95 for spe). The scaling is chosen to guarantee that the draws are within specific intervals (for example, $[0.8, 1]$ for sen and $[0.925, 1]$ for spe and the lower limit 0.925 guarantees that the estimator of θ is positive). The code uses an algorithm due to Devroye (1986) to sample from truncated distributions (see page 239).

It is clear from (9.41) and (9.42) that the data contain information neither about sen nor about spe . The choice of the range of values from which these parameters are sampled propagates into the posterior uncertainty of $[\theta|t, n]$.

The R-code below implements the algorithm.

```
# CODE0902
rm(list=ls()) # CLEAR WORKSPACE
set.seed(3033)
rep<-10000
se<-0.85
sp<-0.95
a_se<-4
b_se<-1.5
a_sp<-10
b_sp<-1.5

# THEORETICAL MODE OF BETA PRIORS
theoretmodese<-(a_se-1)/(a_se+b_se-2)
theoretmodesp<-(a_sp-1)/(a_sp+b_sp-2)
resulttg<-matrix(data=NA,nrow=rep,ncol=4)

# NUMBER OF TESTS: n
n<-10000
# NUMBER OF POSITIVE TESTS: t
t<-900
```

```

for (i in 1:rep){
psi<-rbeta(1,(t+1),(n-t+1))
a<-1-sp
b<-se+sp-1
if((psi-a) > 0)
{
theta <- (psi-a)/b
}
else {
theta <- 0
}

# DRAW SE SP FROM TRUNCATED BETA'S USING DEVROYE'S ALGORITHM
se<-qbeta(runif(1,pbeta(0.8,a_se,b_se),pbeta(1,a_se,b_se)),
a_se,b_se)
sp<-qbeta(runif(1,pbeta(0.925,a_sp,b_sp),pbeta(1,a_sp,b_sp)),
a_sp,b_sp)
#se <- 0.85
#sp <- 0.95
# OR
# DRAW SE AND SP FROM APPROPRIATE UNIFORM DISTRIBUTIONS
#se<-runif(1,0.8,1)
#sp<-runif(1,0.925,1)
resultg[i,]<-c(i,theta,se,sp)
}
# BAYESIAN POSTERIOR MEAN OF THETA AND POSTERIOR INTERVAL
postmean <- mean(resultg[,2])
postinterval <- quantile(resultg[,2],c(0.025,0.975))
postmean

```

```
## [1] 0.05754302
```

```
postinterval
```

```
##          2.5%          97.5%
## 0.01928466 0.09916501
```

Execution of the code choosing the scaled beta distributions for *sen* and *spe* leads to a Monte Carlo estimate of the posterior mean of θ equal to 0.058, with a 95% Bayesian posterior interval (0.019, 0.099). This is wider than the classical interval generated by (9.39) and reflects the prior uncertainty in *spe* and *sen*. When *sen* and *spe* are fixed at 0.85 and 0.95 respectively, the Monte Carlo output yields an estimate of the posterior mean of θ equal to 0.050 and a 95% Bayesian posterior interval (0.043, 0.057), in agreement with the classical interval (9.39).

The **third approach** is based on the asymptotic normal approximation

$$\frac{\hat{\phi} - \phi}{se(\hat{\phi})} \sim N(0, 1) \quad (9.43)$$

where

$$se(\hat{\phi}) = \sqrt{\widehat{Var}(\hat{\phi})}.$$

Given the sensitivity and specificity of the test are known to be 0.85 and 0.95, respectively, then,

$$a = 1 - spe = 1 - 0.85 = 0.05,$$

$$b = spe + sen - 1 = 0.95 + 0.85 - 1 = 0.8,$$

$$n = 10,000,$$

$$T = 900,$$

$$\hat{\phi} = 0.09,$$

$$\widehat{Var}(\hat{\phi}) = \frac{0.09 \times 0.91}{10,000},$$

$$se(\hat{\phi}) = \left[\frac{0.09 \times 0.91}{10,000} \right]^{\frac{1}{2}},$$

$$\hat{\theta} = \frac{\hat{\phi} - a}{b} = 0.05.$$

The confidence interval for $\phi = a + b\theta$, given (a, b) , follows from (9.43) based on the normal approximation and eventually for θ :

$$\Pr\left(-1.96 < \frac{\hat{\phi} - \phi}{se(\hat{\phi})} < 1.96\right) = 0.95, \quad (9.44a)$$

$$\Pr(\hat{\phi} - 1.96 se(\hat{\phi}) < \phi < \hat{\phi} + 1.96 se(\hat{\phi})) = 0.95, \quad (9.44b)$$

$$\Pr(\hat{\phi} - 1.96 se(\hat{\phi}) < a + b\theta < \hat{\phi} + 1.96 se(\hat{\phi})) = 0.95, \quad (9.44c)$$

$$\Pr\left(\frac{\hat{\phi} - 1.96 se(\hat{\phi}) - a}{b} < \theta < \frac{\hat{\phi} + 1.96 se(\hat{\phi}) - a}{b}\right) = 0.95, \quad (9.44d)$$

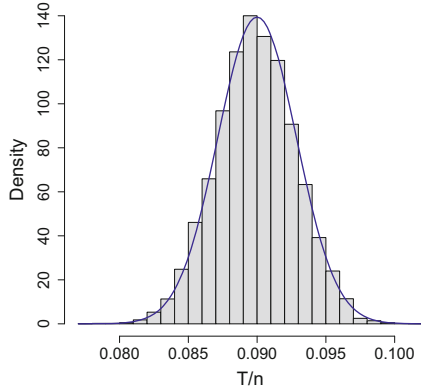
$$\Pr(0.043 < \theta < 0.057) = 0.95, \quad (9.44e)$$

the same result reported before.

The approximation based on (9.44b) is not guaranteed to generate bounds within the allowed parameter space $\phi \in (0, 1)$ particularly when ϕ takes values close to 0 or 1. This is aggravated further in the case of the confidence interval for θ given by (9.44d). If (lb, ub) is the confidence interval for ϕ , a common heuristic is to truncate the confidence interval for θ as

$$\max\left[0, \frac{lb - a}{b}\right], \min\left[1, \frac{ub - a}{b}\right] \quad (9.45)$$

Fig. 9.9 Histogram of the distribution of repeated draws T/n from a binomial distribution $Bi(n = 10,000, \Pr(\hat{Y} = 1) = 0.09)$ with a normal distribution $N(\mu = 0.09, \sigma^2 = (0.09 \times 0.91/10,000))$ overlaid



(Diggle 2011).

Figure 9.9 displays the histogram of draws T/n , $T = 900$, $n = 10,000$ from $T \sim Bi(n, \phi = 0.09)$ and a normal distribution $N(\mu = 0.09, \sigma^2 = (0.09 \times 0.91/10,000))$ overlaid. With this size of sample n the normal approximation gives adequate results despite the extreme value of ϕ .

The **fourth approach** is based on the Clopper-Pearson exact confidence interval for the binomial proportion (Clopper and Pearson 1934). Assume that for a given binomial experiment, x successes are observed out of n trials. The Clopper-Pearson lower bound for a two-sided test retrieves the smallest possible value of ϕ , ϕ_{lb} , for having an $\alpha/2$ probability of observing at least x successes. Given x , the lower bound is given by the value ϕ_{lb} satisfying the monotonically increasing function

$$\sum_{k=x}^n \binom{n}{k} \phi_{lb}^k (1 - \phi_{lb})^{n-k} = \alpha/2. \quad (9.46)$$

The probability $\alpha/2$ on the Y axis intersects the function at $\phi = \phi_{lb}$.

The upper bound provides the largest possible value of ϕ , ϕ_{ub} , for having an $\alpha/2$ probability of observing no more than x successes. Given x , the upper bound is given by the value ϕ_{ub} satisfying the monotonically decreasing function

$$\sum_{k=0}^x \binom{n}{k} \phi_{ub}^k (1 - \phi_{ub})^{n-k} = \alpha/2. \quad (9.47)$$

The probability $\alpha/2$ on the Y axis intersects the function at $\phi = \phi_{ub}$. Therefore the Clopper-Pearson interval for the proportion ϕ in the population contains the values of ϕ that are not rejected by the test at level α ($\alpha = 0.05$ for the 95% confidence interval).

Numerical methods are needed to obtain ϕ_{lb} and ϕ_{ub} in (9.46) and (9.47). The computation is greatly simplified using the equality from Johnson et al. (2005):

$$\sum_{k=x}^n \binom{n}{k} p^k (1-p)^{n-k} = \int_0^p f(u|x, n-x+1) du \quad (9.48)$$

where f is the density function of the beta distribution $Be(a, b)$; the right hand side is the cumulative distribution function of the beta distribution. When (9.48) is used in (9.46) and (9.47), the Clopper-Pearson bounds for the interval are given by the quantiles of the beta distributions (that involve inverting the cumulative distribution functions of the beta distribution):

$$(\phi_{lb}, \phi_{ub}) = (B(\alpha/2, x, n-x+1), B(1-\alpha/2, x+1, n-x)). \quad (9.49)$$

As illustrated below, these bounds are obtained using R with one line of code. Closed form solutions are obtained when x is either 0 or 1. When $x = 0$ the interval is $(0, 1 - (\alpha/2)^{\frac{1}{n}})$ and when $x = n$, the interval is $((\alpha/2)^{\frac{1}{n}}, 1)$.

The Clopper-Pearson procedure is guaranteed to have coverage probability at least $1 - \alpha$ for all values of the probability parameter. However it is known to be conservative, in the sense that for a fixed value of the parameter, the actual coverage probability can be considerably larger than the nominal confidence level unless sample size is large (Agresti and Coull 1998). (The coverage probability of an interval estimator is the probability that the interval contains the true value of the parameter).

With $n = 10,000$ and $T = 900$, the Clopper-Pearson interval for ϕ is obtained using the R-code

```
lb <- qbeta(0.025, 900, 9101)
ub <- qbeta(0.975, 901, 9100)
```

This results in an interval for a positive test given by

$$\Pr(0.08446093 < \phi < 0.09577916) = 0.95$$

and using (9.45), the interval for the prevalence in the population is

$$\Pr\left(\frac{0.08446093 - 0.05}{0.8} < \theta < \frac{0.09577916 - 0.05}{0.8}\right) = 0.95,$$

$$\Pr(0.043 < \theta < 0.057) = 0.95,$$

as in (9.44e). With $n = 10,000$ all the approaches produce similar results.

Probabilistic Interpretation of AUC

Given a randomly drawn observation $Y_i = 1$ (class 1) and a randomly drawn observation $Y'_i = 0$ (class 0) the AUC can be interpreted as the probability that the classification $s = s(x_i)$ will assign a higher score to Y_i than to Y'_i (assuming that 1's rank higher than 0's, so that if the score $s(x_i)$ is larger than a threshold t , a prediction based on x_i will be classified as a member of class 1).

It will be convenient to define the following quantities.

- TPR : $\Pr(s > t | Y = 1) = 1 - \Pr(s \leq t | Y = 1) = 1 - F_1(t)$
- FPR : $\Pr(s > t | Y = 0) = 1 - \Pr(s \leq t | Y = 0) = 1 - F_0(t)$
- $\varepsilon = 1 - F_0(t) \implies F_0(t) = 1 - \varepsilon \implies t = F_0^{-1}(1 - \varepsilon)$, where F_0^{-1} is the inverse cumulative distribution function that maps a FPR to a given threshold
- $f_0(t) = \frac{dF_0(t)}{dt} = p(t | Y = 0)$, the probability density function of s for $Y = 0$ (class 0)
- $\frac{d\varepsilon}{dt} = -f_0(t) \implies d\varepsilon = -f_0(t) dt$
- $f_1(t) = \frac{dF_1(t)}{dt} = p(t | Y = 1)$, the probability density function of s for $Y = 1$ (class 1)

Using this notation the ROC curve can be written as a function of $\varepsilon = 1 - F_0(t)$ as follows

$$ROC(\varepsilon) = 1 - F_1\left(F_0^{-1}(1 - \varepsilon)\right). \quad (9.50)$$

The area under the ROC curve (AUC) is defined as $\int_0^1 ROC(\varepsilon) d\varepsilon$. Substituting (9.50) yields

$$\begin{aligned} AUC &= \int_0^1 \left(1 - F_1\left(F_0^{-1}(1 - \varepsilon)\right)\right) d\varepsilon \\ &= \int_{-\infty}^{\infty} \left[1 - F_1\left[F_0^{-1}(F_0(t))\right]\right] (-f_0(t)) dt \\ &= \int_{-\infty}^{\infty} [1 - F_1(t)] f_0(t) dt. \end{aligned} \quad (9.51)$$

In the second line, the integration is from ∞ to $-\infty$ because $F_0^{-1}(1) = \infty$ and $F_0^{-1}(0) = -\infty$. The third line changes the sign of f_0 and the limits of integration are reversed. Substituting

$$1 - F_1(t) = \int_t^{\infty} f_1(u) du,$$

results in

$$\begin{aligned} AUC &= \int_{-\infty}^{\infty} \int_t^{\infty} f_1(u) f_0(t) du dt \\ &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} I(u > t) f_1(u) f_0(t) du dt \end{aligned} \quad (9.52)$$

that can be read as the probability that the score assigned to a draw Y from class 1 is larger than the score assigned to a draw Y' from class 0. In other words, AUC is interpreted as the probability of making the correct classification.

Example: ROC Curves

ROC curves and AUC are illustrated with two simulated binary data sets that differ in the level of heritability on the underlying scale. These are $h^2 = 0.5$ and $h^2 = 0.23$. In both sets of data, there are 2000 phenotypes and 500 genetic markers, out of which 20 are causal (have an effect on the value of liability). The proportion of 1's is approximately 0.30 and this is controlled by setting the mean μ at the level of liability equal to -0.67 . The effects of causal loci are obtained to ensure that the additive variance on the underlying scale is either 0.3 (for $h^2 = 0.23$) or 1 (for $h^2 = 0.5$). The residual effects at the level of liability are iid $N(0, 1)$ and phenotypes are simulated using a probit model. The R-code below generates the data for $h^2 = 0.23$.

```
# ***** GENERATE BINARY DATA - PROBIT MODEL *****
# CODE0903
rm(list=ls()) # CLEAR WORKSPACE
library(glmnet)
set.seed(30337)
va<-0.3
#va <- 1
p<-0.25
#p <- 0.5
mu <- qnorm(p)
ve <- 1
nindiv<-2000
nloci<-20
nmarker<-500
be<-matrix(data=0.0,nrow=nmarker,ncol=1) # parameter true model
IDq<-sample(1:nmarker,nloci,replace=F) # choose nloci as QTL
WT<-matrix(nrow=nindiv,ncol= nmarker,
           rbinom(n=nindiv*nmarker,size=2,p=.5))
XT<-matrix(data=NA,nrow=nindiv,ncol=nmarker) # NO INTERCEPT
XTi<-matrix(data=NA,nrow=nindiv,ncol=nmarker+1) # INTERCEPT
# CENTER MARKER MATRIX
cm<-colMeans(WT)
```

```

for (i in 1:nmarker)
{
  XT[,i]<-(WT[,i]-cm[i])
}
meanXT <- apply(XT,2,mean)
varXT<-apply(XT,2,var)
# Compute 2*Sum(p(1-p)); Sum over nqt1 QTL:
sumvar<-sum(varXT[IDq])
QTLeff<-sqrt(va/sumvar) # QTL effect
# so that genetic variance is VA
be[IDq] <- QTLeff # QTL EFFECT
xb<-XT%%be
p1 <- pnorm(mu+xb) # PROBIT MODEL
be[IDq]<-QTLeff # TRUE MARKER EFFECT = QTLeff; REST ARE ZERO
var(XT%%be)

```

```

##           [,1]
## [1,] 0.3103757

```

```

y<-rep(0,nindiv)
y <- rbinom(nindiv,1,p1)
mean(y)

```

```

## [1] 0.288

```

Two models are fitted to the binary data. The first is a logistic lasso implementation and the second is a standard logistic likelihood that includes the full set of 500 genetic markers (referred to as the *full model*). Both models are fitted using the package GLMNET. The full model is retrieved including $s = 0$ as an argument to the call of the function GLMNET. The code to run GLMNET, for $h^2 = 0.23$, is as follows:

```

#CODE0903(cont)
##### FITTING FULL MODEL AND LASSO MODEL WITH GLMNET #####
set.seed(3337)
#library(glmnet)

train=sample(1:nrow(XTi),nrow(XTi)/2)
test=(-train)
y.test=y[test]
y.train=y[train]
# ***** FIT GLMNET TO FIND THE BEST LAMBDA *****
# STEP 1
cv.out=cv.glmnet(
  XT[train,],y[train],alpha=1,standardize=TRUE,family="binomial")
#plot(cv.out)
bestlam=cv.out$lambda.min
length(which(as.vector(coef(cv.out,s=bestlam))!=0))

```

```

## [1] 56

```

```
# STEP 2

fm.pred0=predict(
  cv.out,s=0,newx=XT[test,],family="binomial",type="class")
fm.pred=predict(
  cv.out,s=bestlam,newx=XT[test,],family="binomial",
  type="class")

mean((as.numeric(fm.pred)-y.test)^2) # VALIDATION MSE: LASSO
```

```
## [1] 0.273
```

```
# VALIDATION MSE: FULL MODEL:

mean((as.numeric(fm.pred0)-y.test)^2)
```

```
## [1] 0.374
```

```
# ERROR RATE OF NULL MODEL:  $y = \mu + e$ 
pnull<-mean(y.test)
pnull
```

```
## [1] 0.296
```

```
ynull<-rep(0,length(y.test))
if(pnull > 0.5){ynull<-1}
mean((ynull-y.test)^2) # VALIDATION MSE BASED ON NULL MODEL
```

```
## [1] 0.296
```

ROC curves can be generated using the package `pROC`. This package can use the output from GLMNET (step 2, `fm` or `fm0`). Various pieces of information can be extracted and one of these is displayed in Fig. 9.10. The curves illustrate that the classifiers' performance is higher at heritability $h^2 = 0.5$ than at $h^2 = 0.23$ due to an increase in power. The lasso classifier is consistently superior to the full model classifier (that includes the 500 marker covariates) over the full range of the values of the threshold t .

The R-code below generates Fig. 9.10 using the package `pROC`.

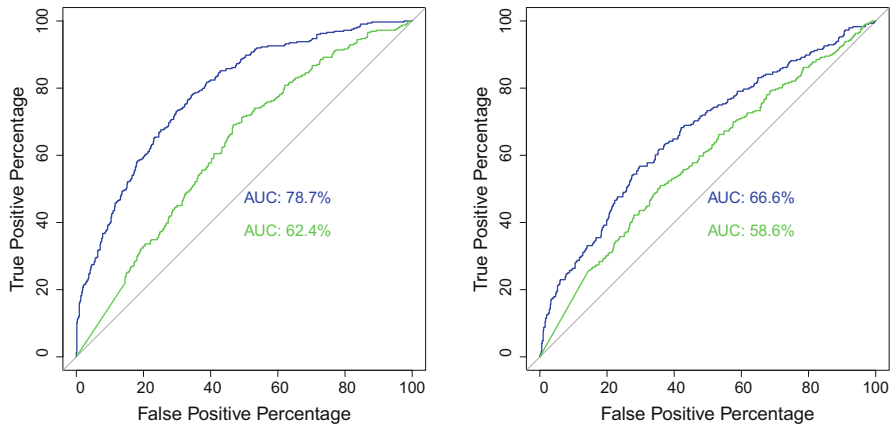


Fig. 9.10 ROC curves using simulated data. Left: $h^2 = 0.50$. Right: $h^2 = 0.23$. Blue line: lasso regression; green line: full likelihood model with all 500 covariates

```
# CODE0903(cont)
#install.packages("pROC", .libPaths()[1])
library(pROC)
set.seed(420)
par(pty="s") # GENERATES ‘‘PRETTY’’ FIGURES

fm.pred=predict(
  cv.out,s=bestlam,newx=XT[test,],family="binomial",
  type="response")
fm.pred0=predict(
  cv.out,s=0,newx=XT[test,],family="binomial",type="response")
pred<-as.numeric(fm.pred)
pred0<-as.numeric(fm.pred0)
roc.info<-roc(y.test, pred,plot=FALSE,legacy.axes=TRUE) # LASSO
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
# FULL MODEL:
roc.info0<-roc(y.test, pred0,plot=FALSE,legacy.axes=TRUE)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
s <-roc(y.test, pred,plot=FALSE,legacy.axes=TRUE,percent=TRUE,
  xlab="False Positive Percentage",
  ylab="True Positive Percentage",
  col="blue",print.auc=TRUE,cex.lab=1.3)
```



```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases

# CREATE DATA FRAME TO BE USED IN THE NEXT CODE
# THAT CONTAINS TPOS, FPOS FOR THE COMPLETE RANGE
# OF THRESHOLDS t
roc.df<-data.frame(tpos=roc.info$sensitivities*100,
                   fpos=(1-roc.info$specificities)*100,
                   thresholds=roc.info$thresholds)
roc.df0<-data.frame(tpos0=roc.info0$sensitivities*100,
                    fpos0=(1-roc.info0$specificities)*100,
                    thresholds0=roc.info0$thresholds)
head(roc.df)
```

```
##      tpos      fpos thresholds
## 1 100.00000 100.00000      -Inf
## 2 100.00000  99.85795  0.04341405
## 3 100.00000  99.71591  0.06056188
## 4  99.66216  99.71591  0.06751315
## 5  99.66216  99.57386  0.06976092
## 6  99.66216  99.43182  0.07345414
```

```
f50<-roc.df[min(which(roc.df$thresholds > 0.49995)),]
f50_0<-roc.df0[min(which(roc.df0$thresholds > 0.49995)),]
par(pty="m")
f50
```

```
##      tpos      fpos thresholds
## 950 12.5 1.988636  0.50000075
```

```
mean(y.test)
```

```
## [1] 0.296
```

The estimate of AUC can be retrieved typing `s$auc`. This yields 66.597 in agreement with the value displayed in the right panel of Fig. 9.10 corresponding to the lasso regression.

Estimates of AUC can also be obtained by numerical integration of the AUC curve. The following code performs this operation using the trapezoid rule:

```
# CODE0903(cont)
# ***** COMPUTE AUC BY NUMERICAL INTEGRATION *****
tpos <- roc.df$tpos/100
fpos <- roc.df$fpos/100
n<-length(tpos)
sum((tpos[-1]+tpos[-n])/2*(fpos[-n]-fpos[-1]))
```

```
## [1] 0.6659724
```

```
# OR AS A FUNCTION
auc <- function(fpos, tpos){
  n <- length(tpos)
  abs(sum((tpos[-1]+tpos[-n])/2*(fpos[-n]-fpos[-1])))
}
with(roc.df, auc(fpos/100, tpos/100)) # AUC FOR LASSO
```

```
## [1] 0.6659724
```

```
with(roc.df0, auc(
  fpos0/100, tpos0/100)) # AUC FULL MODEL: glmnet: LAMBDA=0
```

```
## [1] 0.5932389
```

A threshold t equal to 0.5 minimises the overall (validation) error rate (9.32). This can be confirmed by extracting the necessary information from data frames generated by the package **pROC**, executed by the section of R-code CODE0903 starting on page 450. The last two lines of this code outputs the proportion of *true positives*, *false positives* for the complete range of values of the *thresholds*, t , for the LASSO (f50) and for the full model (f50_0). For example, using the output linked to f50, the estimate of the validating mean squared error or error rate is

$$(100 - 12.5) \times 0.296 + 1.988636 \times (1 - 0.296) = 27.3$$

where the proportion of 1's in the validating data is 0.296. The result is in agreement with `mean((as.numeric(fm.pred)-y.test)^2)=0.273` in the R-code starting on page 448.

9.8 Prediction of Disease Status of Individual Given Disease Status of Relatives

The question addressed here is how to compute the probability that an individual will show a genetic disease, given sources of information that can be incorporated in a probability model. This could include family history and known susceptibility loci (So et al. 2011). In the development that follows, information is restricted to the disease status of the individual's father and mother.

The Model

A simple binary trait (disease/not-disease) and a threshold probit model is assumed. In the parametrisation used (see page 116 for the alternative parametrisation), if u_i

exceeds the threshold t then the individual shows the disease and $Y_i = 1$; otherwise, if u_i is smaller than t , $Y_i = 0$. Assume the linear model for the liability

$$u_i = \mu + g_i + e_i, \quad (9.53)$$

where $g_i \sim N(0, \sigma_g^2)$ is the additive genetic value of individual i , σ_g^2 is the additive genetic variance, $e_i \sim N(0, 1)$ is the environmental deviation and μ is the population mean. The phenotypic variance of the marginal distribution of the liability is $\sigma^2 = \sigma_g^2 + 1$ and the heritability on the underlying scale is $h^2 = \sigma_g^2 / \sigma^2$.

Calculation of Disease Status

The general result is motivated with a specific case. Consider the joint distribution of the liabilities of an offspring (u_o) and of its unrelated parents (u_f, u_m),

$$(u_o, u_f, u_m) \sim N \left(\begin{bmatrix} \mu \\ \mu \\ \mu \end{bmatrix}, \begin{bmatrix} 1 & 0.5h^2 & 0.5h^2 \\ 0.5h^2 & 1 & 0 \\ 0.5h^2 & 0 & 1 \end{bmatrix} \sigma^2 \right). \quad (9.54)$$

An additive genetic model is assumed, where the covariance of the offspring with each parent is $\sigma_g^2/2 = h^2\sigma^2/2$.

Given this model, what is the probability that the offspring will show the disease, given the disease status of father and mother? This conditional probability model, given information on parents, will be referred to as the *selection model*, introduced by the set of Eq. (2.57) on page 76. Two approaches are described. One uses standard probability theory where computations rely on multivariate normal numerical integration. The other is an approximation that avoids the numerical computation of multiple integrals; it is based on results of truncation selection of normal variates dating back to Pearson (1903) and Aitken (1934). The essentials of the latter approach are summarised in Note 2 on page 461.

Consider the case where the father has the disease and the mother does not. The conditional probability that the offspring is affected by the disease, given this information is

$$\begin{aligned} \Pr(Y_o = 1 | Y_f = 1, Y_m = 0) &= \frac{\Pr(Y_o = 1, Y_f = 1, Y_m = 0)}{\Pr(Y_f = 1, Y_m = 0)} \\ &= \Pr(u_o > t | u_f > t, u_m < t) \\ &= \frac{\int_t^\infty \int_t^\infty \int_{-\infty}^t p(u_o, u_f, u_m) du_m du_f du_o}{\int_{-\infty}^\infty \int_t^\infty \int_{-\infty}^t p(u_o, u_f, u_m) du_m du_f du_o} \\ &= \frac{\int_t^\infty \int_t^\infty \int_{-\infty}^t p(u_o, u_f, u_m) du_m du_f du_o}{\int_t^\infty \int_{-\infty}^t p(u_f, u_m) du_m du_f}. \end{aligned} \quad (9.55)$$

In these expressions, $p(u_o, u_f, u_m)$ is the joint probability density function of the distribution defined in (9.54), and $p(u_f, u_m) = \int_{-\infty}^{\infty} p(u_o, u_f, u_m) du_o$. The evaluation of (9.55) requires multivariate normal numerical integration. Several software routines are available to compute the numerical integration; here I use the R function `pmvnorm` that is part of the library `mvtnorm`.

A little insight into expression (9.55) can be gained as follows. Under the selection model, the joint probability density function of the trivariate normal distribution is

$$p^s(u_o, u_f, u_m) = \frac{p(u_o, u_f, u_m)}{\int_{-\infty}^{\infty} \int_t^{\infty} \int_{-\infty}^t p(u_o, u_f, u_m) du_m du_f du_o}$$

and the marginal density of u_o under the selection model is

$$p^s(u_o) = \int_t^{\infty} \int_{-\infty}^t p^s(u_o, u_f, u_m) du_m du_f.$$

Integration of $p^s(u_o)$ over the interval $(t < u_o < \infty)$ yields (9.55).

To compute the marginal probability (under the selection model) that $Y_o = 1$ using the Pearson-Aitken formula (see Note 2 on page 461), proceed as follows. In terms of expression (9.73), let $x = (u_f, u_m)$ and $y = u_o$. Then using (9.69) and (9.71)

$$\mu_x^* = E[(u_f, u_m) | u_f > t, u_m < t] = \begin{bmatrix} \mu + \sigma \frac{\phi(\alpha)}{1 - \Phi(\alpha)} \\ \mu - \sigma \frac{\phi(\beta)}{\Phi(\beta)} \end{bmatrix} \quad (9.56)$$

and using (9.70) and (9.72),

$$\begin{aligned} V_x^* &= \text{Var}[(u_f, u_m) | u_f > t, u_m < t] = \\ &= \begin{bmatrix} \sigma^2 \left[1 - \frac{\phi(\alpha)}{1 - \Phi(\alpha)} \left(\frac{\phi(\alpha)}{1 - \Phi(\alpha)} - \alpha \right) \right] & 0 \\ 0 & \sigma^2 \left[1 - \frac{\phi(\beta)}{\Phi(\beta)} \left(\frac{\phi(\beta)}{\Phi(\beta)} + \beta \right) \right] \end{bmatrix}. \end{aligned} \quad (9.57)$$

(see Note 1 on page 457 for the definition of α and β). Then using (9.74) the mean of the offspring in the selection model is

$$E(u_o | u_f > t, u_m < t) = \mu_y^* = \mu_y + C_{yx} V_x^{-1} (\mu_x^* - \mu_x), \quad (9.58)$$

and using the block in the second row and second column of (9.75), the variance of the offspring in the selection model is

$$\text{Var}(u_o | u_f > t, u_m < t) = V_y - C_{yx} (V_x^{-1} - V_x^{-1} V_x^* V_x^{-1}) C_{xy}. \quad (9.59)$$

Finally, the marginal probability that the offspring is affected (in the selection model) is obtained by numerically integrating the (univariate) normal distribution

with mean given by (9.58) and variance given by (9.59) over the range: lower limit = t , upper limit = infinity.

It is important to notice that results (9.56) and (9.57) are exact. However, due to the correlation of u_0 with the truncated variables u_f and u_m , the marginal distribution of u_0 after truncation is not normal. The Pearson-Aitken formula is used to compute the mean and variance of this distribution and then integrate over the normal distribution with this mean and variance. The assumption of normality in this last step is an approximation that works remarkably well in the example below, despite the rather extreme incidence for one of the cases considered.

Example: Prediction of a Genetic Disease

The computations are illustrated with a binary trait whose heritability on the underlying scale is $h^2 = 1/3$ and the phenotypic variance is $\sigma^2 = 3/2$. The objective is to predict the disease status of an offspring, given knowledge of the disease status of the father and the mother. The incidence in the population is either 2% or 0.2%.

The R-code to perform the “exact” computations and the Pearson-Aitken formula is shown below. The code is divided in two parts; the first part performs numerical integration and outputs the conditional probability that an offspring shows the disease given

1. CASE 1: father affected, mother affected
2. CASE 2: father affected, mother not affected
3. CASE 3: father unaffected, mother unaffected

```
# CODE0904
# NUMERICAL INTEGRALS
rm(list=ls()) # CLEAR WORKSPACE
library(mvtnorm)
# EXAMPLE: FATHER-MOTHER-CHILD
p<-0.02 # INCIDENCE IN THE POPULATION
# p <- 0.002
mean<-c(0,0,0) # MEAN OF THE THREE LIABILITIES
her <- 1/3 # heritability
var <- 1.5 # variance of liability
cov <- 0.5*her*var # covariance single parent-child
t<-qnorm((1-p),mean=0,sd=sqrt(1.5)) # THRESHOLD
# # VAR-COV MATRIX OF LIABILITY:
sigma <- matrix(c(var,cov,cov,cov,var,0,cov,0,var),3,3)
sigma
```

```
##      [,1] [,2] [,3]
## [1,] 1.50 0.25 0.25
## [2,] 0.25 1.50 0.00
## [3,] 0.25 0.00 1.50
```

```

# CASE 1: FATHER AFFECTED, MOTHER AFFECTED
den<-pmvnorm(lower=c(-Inf,t,t),upper=c(Inf,Inf,Inf),
             mean=mean,sigma=sigma)
num<-pmvnorm(lower=c(t,t,t),upper=c(Inf,Inf,Inf),
             mean=mean,sigma=sigma)
proboffsprcase1 <- num/den
#proboffsprcase1
# CASE 2: FATHER AFFECTED, MOTHER UNAFFECTED
den<-pmvnorm(lower=c(-Inf,t,-Inf),upper=c(Inf,Inf,t),
             mean=mean,sigma=sigma)
num<-pmvnorm(lower=c(t,t,-Inf),upper=c(Inf,Inf,t),
             mean=mean,sigma=sigma)
proboffsprcase2 <- num/den
#proboffsprcase2
# CASE 3: FATHER UNAFFECTED, MOTHER UNAFFECTED
den<-pmvnorm(lower=c(-Inf,-Inf,-Inf),upper=c(Inf,t,t),
             mean=mean,sigma=sigma)
num<-pmvnorm(lower=c(t,-Inf,-Inf),upper=c(Inf,t,t),
             mean=mean,sigma=sigma)
proboffsprcase3 <- num/den
#proboffsprcase3

```

The three conditional probabilities are CASE 1: 0.101, CASE 2: 0.046, CASE 3: 0.019.

The second part computes the same conditional probabilities using the Pearson-Aitken formula.

```

# CODE0905
#           Pearson-Aitken formula
# Input
mean<-0 # marginal mean of liability
var<-1.5 # marginal variance of liability
varg<-0.5 # additive genetic variance
p<-0.02 # Incidence in population
t<-qnorm((1-p),mean=0,sd=sqrt(1.5)) # THRESHOLD
varfm<-var*diag(2) # VAR-COV PARENTS
covop<-matrix(0.5*varg,nrow=1,ncol=2) # COV OFFS-PARENTS
# CASE 1: FATHER AFFECTED, MOTHER AFFECTED
# FATHER:
alfa<-(t-mean)/sqrt(var) # LOWER TRUNCATION
p_alfa<-dnorm(alfa) # PDF AT LOWER TRUNCATION POINT
cum_alfa<-pnorm(alfa) # CUM. DIST. FUNCTION AT LOWER TRUNCATION
intsel<-p_alfa/(1-cum_alfa) # "INTENSITY OF SELECTION"
# MEAN AND VARIANCE OF (SELECTED) FATHER (MOTHER BELOW)
minfather<-mean+sqrt(var)*intsel
varfather<-var*(1-intsel*(intsel-alfa))
# MOTHER
minmother<-minfather
varmother<-varfather
# MEAN AND VARIANCE OF SELECTED FATHER AND MOTHER
expcase1 <- matrix(c(minfather,minmother),2,1)
varcase1 <- matrix(c(varfather,0,0,varmother),2,2)

```

```

# CONDITIONAL MEAN AND VARIANCE OF LIABILITY OF OFFSPRING
condmin<-mean+covop%%solve(varfm)%%(expcase1)
int1<-solve(varfm)-(solve(varfm)%%varcase1%%solve(varfm))
condvar<-var-(covop%%(int1)%%t(covop))
proboffPAcase1<-1-pnorm(t,mean=condmin,sd=sqrt(condvar))
# *****
# CASE 2: FATHER AFFECTED, MOTHER UNAFFECTED
# FATHER: AS IN CASE 1
beta<-(t-mean)/sqrt(var) # upper TRUNCATION
p_beta<-dnorm(beta)
cum_beta<-pnorm(beta)
# MOTHER MEAN AND VARIANCE
minmother<-mean-sqrt(var)*(p_beta/cum_beta)
varmother<-var*(1-(p_beta/cum_beta)*((p_beta/cum_beta)+beta))
expcase2 <- matrix(c(minfather,minmother),2,1)
varcase2 <- matrix(c(varfather,0,0,varmother),2,2)
# CONDITIONAL MEAN AND VARIANCE OF LIABILITY OF OFFSPRING
condmin<-mean+covop%%solve(varfm)%%(expcase2)
int2<-solve(varfm)-(solve(varfm)%%varcase2%%solve(varfm))
condvar<-var-(covop%%(int2)%%t(covop))
proboffPAcase2<-1-pnorm(t,mean=condmin,sd=sqrt(condvar))
# *****
# CASE 3: FATHER UNAFFECTED, MOTHER UNAFFECTED
# FATHER: AS MOTHER IN CASE 2
expcase3 <- matrix(c(minmother,minmother),2,1)
varcase3 <- matrix(c(varmother,0,0,varmother),2,2)
# CONDITIONAL MEAN AND VARIANCE OF LIABILITY OF OFFSPRING
condmin<-mean+covop%%solve(varfm)%%(expcase3)
int3<-solve(varfm)-(solve(varfm)%%varcase3%%solve(varfm))
condvar<-var-(covop%%(int3)%%t(covop))
proboffPAcase3<-1-pnorm(t,mean=condmin,sd=sqrt(condvar))

```

The three conditional probabilities are CASE 1: 0.1, CASE 2: 0.046, CASE 3: 0.019, in good agreement with the results based on numerical integration.

The results are summarised in Table 9.3. The results show that for this example, both ways of computing the probabilities lead to identical results, to 3 decimal digits. Moving through the rows from right to left indicates that the probability of disease status of the offspring increases from a value that corresponds to the population incidence, when neither parent is affected, to a value that is 5 times larger or 15 times larger, for incidences in the population of 2% or 0.2%, respectively, when both parents are affected.

Note 1: Mean and Variance of the Truncated Normal Distribution

The pdf of the normal distribution with mean μ and variance σ^2 truncated between a and b , $a < b$, is

$$p(y|a < y < b) = \frac{(2\pi\sigma^2)^{-\frac{1}{2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right)}{\Pr(a < y < b)} \quad (9.60)$$

Table 9.3 Probability of disease of an offspring, given the disease status of its parents. $\Pr(Y = 1)$: incidence in population; PA : computation with the Pearson-Aitken formula; N : numerical integration. *Disease status of parents: for example, $(Y_f = 1; Y_m = 0)$ symbolises the case where the father shows the disease ($Y_f = 1$) and the mother does not ($Y_m = 0$)

$\Pr(Y = 1)$		$(Y_f = 1; Y_m = 1)$	$(Y_f = 1; Y_m = 0)^*$	$(Y_f = 0; Y_m = 0)$
0.02	PA	0.100	0.046	0.019
	N	0.100	0.046	0.019
0.002	PA	0.031	0.0086	0.002
	N	0.031	0.0086	0.002

where

$$\begin{aligned}
 \Pr(a < y < b) &= \Pr\left(\frac{a - \mu}{\sigma} < \frac{y - \mu}{\sigma} < \frac{b - \mu}{\sigma}\right) \\
 &= \Pr(\alpha < z < \beta), \quad \alpha = \frac{a - \mu}{\sigma}; \beta = \frac{b - \mu}{\sigma}; z = \frac{y - \mu}{\sigma} \\
 &= \Phi(\beta) - \Phi(\alpha).
 \end{aligned}$$

The expected value is

$$E(y|a < y < b) = \frac{(2\pi\sigma^2)^{-\frac{1}{2}}}{\Phi(\beta) - \Phi(\alpha)} \int_a^b y \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right) dy. \quad (9.61)$$

Since $y = \mu + \sigma z$ and $dy = \sigma dz$, this can be written as

$$\begin{aligned}
 E(y|a < y < b) &= \frac{(2\pi\sigma^2)^{-\frac{1}{2}}}{\Phi(\beta) - \Phi(\alpha)} \int_{\alpha}^{\beta} (\mu + \sigma z) \exp\left(-\frac{z^2}{2}\right) \sigma dz \\
 &= \frac{(2\pi)^{-\frac{1}{2}}}{\Phi(\beta) - \Phi(\alpha)} \mu \int_{\alpha}^{\beta} \exp\left(-\frac{z^2}{2}\right) dz \\
 &\quad + \frac{(2\pi)^{-\frac{1}{2}}}{\Phi(\beta) - \Phi(\alpha)} \sigma \int_{\alpha}^{\beta} z \exp\left(-\frac{z^2}{2}\right) dz \\
 &= \mu + \frac{(2\pi)^{-\frac{1}{2}}}{\Phi(\beta) - \Phi(\alpha)} \sigma \int_{\alpha}^{\beta} z \exp\left(-\frac{z^2}{2}\right) dz. \quad (9.62)
 \end{aligned}$$

Let $f(z) = \exp\left(-\frac{z^2}{2}\right)$; then $f'(z) = -z \exp\left(-\frac{z^2}{2}\right)$ and $\int_{\alpha}^{\beta} z \exp\left(-\frac{z^2}{2}\right) dz = -\int_{\alpha}^{\beta} f'(z) dz = -(f(\beta) - f(\alpha))$. Expression (9.62) can be written as

$$\begin{aligned} E(y|a < y < b) &= \mu + \frac{(2\pi)^{-\frac{1}{2}}}{\Phi(\beta) - \Phi(\alpha)} \sigma \left[\exp\left(-\frac{\alpha^2}{2}\right) - \exp\left(-\frac{\beta^2}{2}\right) \right] \\ &= \mu + \sigma \frac{\phi(\alpha) - \phi(\beta)}{\Phi(\beta) - \Phi(\alpha)}, \end{aligned} \quad (9.63)$$

where $\phi(x)$ is the pdf of the standard normal distribution evaluated at x .

The conditional variance is

$$\begin{aligned} \text{Var}(y|a < y < b) &= \text{Var}(\mu + \sigma z | \alpha < z < \beta) \\ &= \sigma^2 \text{Var}(z | \alpha < z < \beta). \end{aligned} \quad (9.64)$$

Then

$$\text{Var}(z | \alpha < z < \beta) = E\left[z^2 | \alpha < z < \beta\right] - [E(z | \alpha < z < \beta)]^2,$$

where $E(z | \alpha < z < \beta) = \frac{\phi(\alpha) - \phi(\beta)}{\Phi(\beta) - \Phi(\alpha)}$. The first term is

$$E\left[z^2 | \alpha < z < \beta\right] = \frac{(2\pi)^{-\frac{1}{2}}}{\Phi(\beta) - \Phi(\alpha)} \int_{\alpha}^{\beta} z^2 \exp\left(-\frac{z^2}{2}\right) dz. \quad (9.65)$$

A simple way of computing the integration is to note that

$$(2\pi)^{-\frac{1}{2}} z^2 \exp\left(-\frac{z^2}{2}\right) = \phi''(z) + \phi(z),$$

where $\phi''(z)$ is the second derivative of the pdf of the standard normal random variable z with respect to z . Then (9.65) can be written as

$$\begin{aligned} E\left[z^2 | \alpha < z < \beta\right] &= \frac{1}{\Phi(\beta) - \Phi(\alpha)} \int_{\alpha}^{\beta} [\phi''(z) + \phi(z)] dz \\ &= \frac{1}{\Phi(\beta) - \Phi(\alpha)} \left[\phi'(z) \Big|_{\alpha}^{\beta} + (\Phi(\beta) - \Phi(\alpha)) \right]. \end{aligned} \quad (9.66)$$

Using

$$\begin{aligned} \phi'(z) \Big|_{\alpha}^{\beta} &= (2\pi)^{-\frac{1}{2}} \alpha \exp\left(-\frac{\alpha^2}{2}\right) - (2\pi)^{-\frac{1}{2}} \beta \exp\left(-\frac{\beta^2}{2}\right) \\ &= \alpha \phi(\alpha) - \beta \phi(\beta), \end{aligned}$$

and substituting in (9.66), yields

$$E\left[z^2|\alpha < z < \beta\right] = \frac{\alpha\phi(\alpha) - \beta\phi(\beta)}{\Phi(\beta) - \Phi(\alpha)} + 1.$$

Then

$$\text{Var}(z|\alpha < z < \beta) = 1 + \frac{\alpha\phi(\alpha) - \beta\phi(\beta)}{\Phi(\beta) - \Phi(\alpha)} - \left(\frac{\phi(\alpha) - \phi(\beta)}{\Phi(\beta) - \Phi(\alpha)}\right)^2$$

and

$$\text{Var}(y|a < y < b) = \sigma^2 \left[1 + \frac{\alpha\phi(\alpha) - \beta\phi(\beta)}{\Phi(\beta) - \Phi(\alpha)} - \left(\frac{\phi(\alpha) - \phi(\beta)}{\Phi(\beta) - \Phi(\alpha)}\right)^2 \right]. \quad (9.67)$$

As a special case of (9.63) and (9.67), consider first $y > t$. Then $a = t$ and $b = +\infty$. Use the following result:

$$\lim_{z \rightarrow \infty} z \exp\left(-\frac{z^2}{2}\right) = 0. \quad (9.68)$$

Then $\phi(\beta) = 0$, $\Phi(\beta) = 1$ and using (9.68), $\beta\phi(\beta) = 0$. The conditional expectation is

$$E(y|y > t) = \mu + \sigma \frac{\phi(\alpha)}{1 - \Phi(\alpha)}, \quad (9.69)$$

where $1 - \Phi(\alpha)$ is the proportion larger than t (the proportion selected, in quantitative genetic parlance) and $\phi(\alpha)/(1 - \Phi(\alpha))$ is known as the intensity of selection. The conditional variance is

$$\begin{aligned} \text{Var}(y|y > t) &= \sigma^2 \left[1 + \frac{\alpha\phi(\alpha)}{1 - \Phi(\alpha)} - \left(\frac{\phi(\alpha)}{1 - \Phi(\alpha)}\right)^2 \right] \\ &= \sigma^2 \left[1 - \frac{\phi(\alpha)}{1 - \Phi(\alpha)} \left(\frac{\phi(\alpha)}{1 - \Phi(\alpha)} - \alpha\right) \right]. \end{aligned} \quad (9.70)$$

As a second case, assume $y < t$. Then $a = -\infty$, $b = t$, $\phi(\alpha) = 0$, $\Phi(\alpha) = 0$, $\alpha\phi(\alpha) = 0$. The conditional expectation is

$$\begin{aligned} E(y|y < t) &= \mu + \sigma \frac{-\phi(\beta)}{\Phi(\beta)} \\ &= \mu - \sigma \frac{\phi(\beta)}{\Phi(\beta)}. \end{aligned} \quad (9.71)$$

The conditional variance is

$$\begin{aligned}\text{Var}(y|y < t) &= \sigma^2 \left[1 - \frac{\beta \phi(\beta)}{\Phi(\beta)} - \left(\frac{\phi(\beta)}{\Phi(\beta)} \right)^2 \right] \\ &= \sigma^2 \left[1 - \frac{\phi(\beta)}{\Phi(\beta)} \left(\frac{\phi(\beta)}{\Phi(\beta)} + \beta \right) \right].\end{aligned}\quad (9.72)$$

Note 2: The Pearson-Aitken Formula

The Pearson-Aitken formula describes how a mean vector and a covariance matrix of a set of variables are affected by selection on a subset of variables. It has been much used in animal breeding, notably by Henderson (1975). The normally distributed random variables are the vectors x and y with joint distribution

$$(x, y) \sim N \left[\begin{pmatrix} \mu_x \\ \mu_y \end{pmatrix}, \begin{pmatrix} V_x & C_{xy} \\ C_{yx} & V_y \end{pmatrix} \right]. \quad (9.73)$$

If selection operates on x and its expected mean changes to μ_x^* , then the expected mean of vector y changes to

$$\mu_y^* = \mu_y + C_{yx} V_x^{-1} (\mu_x^* - \mu_x). \quad (9.74)$$

If selection changes the variance of x to V_x^* then the covariance matrix of (x, y) is changed to

$$\begin{pmatrix} V_x^* & V_x^* V_x^{-1} C_{xy} \\ C_{yx} V_x^{-1} V_x^* & V_y - C_{yx} (V_x^{-1} - V_x^{-1} V_x^* V_x^{-1}) C_{xy} \end{pmatrix}. \quad (9.75)$$

In the absence of selection (or with random selection), $V_x^* = V_x$ and the covariance (9.75) reduces to the covariance structure of (9.73).

9.9 Appendix: Approximate Analysis of Binary Traits

Binary traits can be analysed ignoring their discrete nature using a linear model. This can be a useful approximation for an initial analysis that may work adequately in large data sets, where the proportion of 1's is not extreme. Here I provide the rationale for such an approximation and establish the relationship between the parameters of the Bayesian linear (approximate) model and the Bayesian (true) probit model, where parameters are defined on the unobserved underlying scale.

As before the binary datum is classified as “survival” versus “death”, say. Let u represent the unobserved liability and t the threshold. If $u > t$ then the individual

survives and the binary variable (the observed datum) takes the value $y = 1$. If $u \leq t$ the individual dies and the observed datum is $y = 0$. The liability associated with datum ij is u_{ij} ; expressed in terms of the linear model it takes the form

$$u_{ij} = \mu + f_i + e_{ij}, \quad i = 1, 2, \dots, n_f, \quad j = 1, \dots, n, \\ f_i | \sigma_f^2 \stackrel{iid}{\sim} N(0, \sigma_f^2), \quad e_{ij} \stackrel{iid}{\sim} N(0, 1), \quad (9.76)$$

where μ is the mean, f_i is a random full-sib family effect and e_{ij} is a random residual $N(0, 1)$. There are n_f full-sib families and n full-sibs per family. The heritability on the underlying scale is $h^2 = 2\sigma_f^2 / (\sigma_f^2 + 1)$.

The probability of survival of individual ij (which is the pmf of the random variable Y_{ij}) is

$$\begin{aligned} \Pr(Y_{ij} = 1 | \mu, f_i) &= \Pr(u_{ij} > t | \mu, f_i) = \Pr(u_{ij} - \mu - f_i > t - \mu - f_i | \mu, f_i) \\ &= \Pr(e_{ij} > t - \mu - f_i | \mu, f_i) = \int_{t - \mu - f_i}^{\infty} p(e_{ij}) de_{ij} \\ &= \int_{-\infty}^{\mu + f_i - t} p(e_{ij}) de_{ij} = \Phi(\mu + f_i - t). \end{aligned} \quad (9.77)$$

The liabilities cannot be observed and a convenient origin is to set the value of the threshold to $t = 0$. Hence

$$\Pr(Y_{ij} = 1 | \mu, f_i) = \Phi(\mu + f_i). \quad (9.78)$$

This constraint makes the likelihood model identifiable and the Hessian becomes negative definite. For the ij th datum the conditional pmf is

$$[\Phi(\mu + f_i)]^{y_{ij}} [1 - \Phi(\mu + f_i)]^{1 - y_{ij}}.$$

To derive the approximation the following is needed:

$$\begin{aligned} \Phi(\mu + f_i) |_{f_i=0} &= \Phi(\mu), \\ \frac{\partial \Phi(\mu + f_i)}{\partial f_i} \Big|_{f_i=0} &= (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{\mu^2}{2}\right) = \phi(\mu), \end{aligned}$$

where Φ and ϕ are the cumulative distribution function and the density function of the standard normal distribution, respectively. Expanding the conditional likelihood (9.78) using a first order Taylor series about $f_i = 0$, one obtains

$$\begin{aligned} \Pr(Y_{ij} = 1 | \mu, f_i) &\approx \Phi(\mu) + f_i \frac{\partial \Phi(\mu + f_i)}{\partial f_i} \Big|_{f_i=0} \\ &= \Phi(\mu) + \phi(\mu) f_i. \end{aligned} \quad (9.79)$$

Since the data stem from a binary process,

$$\begin{aligned}
 E(Y_{ij}|\mu, f_i) &= \Pr(Y_{ij} = 1|\mu, f_i) \approx \Phi(\mu) + \phi(\mu) f_i, \\
 \text{Var}(Y_{ij}|\mu, f_i) &= \Pr(Y_{ij} = 1|\mu, f_i) (1 - \Pr(Y_{ij} = 1|\mu, f_i)) \\
 &\approx [\Phi(\mu) + \phi(\mu) f_i] [(1 - \Phi(\mu)) - \phi(\mu) f_i] \\
 &= \Phi(\mu) (1 - \Phi(\mu)) + \phi(\mu) f_i - \phi(\mu)^2 f_i^2 - 2\Phi(\mu) \phi(\mu) f_i. \tag{9.80}
 \end{aligned}$$

Also,

$$\text{Cov}(Y_{ij}, Y_{ik}|\mu, f_i) = 0.$$

Marginally with respect to f_i ,

$$E(Y_{ij}|\mu) = E_{f_i}[E(Y_{ij}|\mu, f_i)] \approx \Phi(\mu). \tag{9.81}$$

$$\text{Var}(Y_{ij}|\mu) = \text{Var}_{f_i}(E(Y_{ij}|\mu, f_i)) + E_{f_i}(\text{Var}(Y_{ij}|\mu, f_i))$$

The first term in the right hand side is

$$\text{Var}_{f_i}(E(Y_{ij}|\mu, f_i)) \approx \text{Var}_{f_i}[\Phi(\mu) + \phi(\mu) f_i] = \phi(\mu)^2 \sigma_f^2.$$

The second term in the right hand side is

$$E_{f_i}(\text{Var}(Y_{ij}|\mu, f_i)) \approx \Phi(\mu) (1 - \Phi(\mu)) - \phi(\mu)^2 \sigma_f^2. \tag{9.82}$$

Therefore,

$$\begin{aligned}
 \text{Var}(Y_{ij}|\mu) &= \phi(\mu)^2 \sigma_f^2 + \Phi(\mu) (1 - \Phi(\mu)) - \phi(\mu)^2 \sigma_f^2 \\
 &= \Phi(\mu) (1 - \Phi(\mu)). \tag{9.83}
 \end{aligned}$$

Similarly,

$$\begin{aligned}
 \text{Cov}(Y_{ij}, Y_{ik}|\mu) &= \text{Cov}[E(Y_{ij}|\mu, f_i), E(Y_{ik}|\mu, f_i)] + E[\text{Cov}(Y_{ij}, Y_{ik}|\mu, f_i)] \\
 &= \phi(\mu)^2 \sigma_f^2. \tag{9.84}
 \end{aligned}$$

Invoking the normal approximation to the likelihood and replacing the conditional variance (9.80) by the average conditional variance (9.82), the hierarchical model becomes

$$\begin{aligned}
 Y_{ij}|\mu, f_i, \sigma_f^2 &\sim N\left([\Phi(\mu) + \phi(\mu) f_i], [\Phi(\mu) (1 - \Phi(\mu)) - \phi(\mu)^2 \sigma_f^2]\right), \\
 f_i|\sigma_f^2 &\sim N\left(0, \sigma_f^2\right),
 \end{aligned}$$

that can be expressed as

$$Y_{ij}|\mu^*, f_i^*, \sigma_e^{2*} \sim N\left(\mu^* + f_i^*, \sigma_e^{2*}\right),$$

$$f_i^*|\sigma_f^{2*} \sim N\left(0, \sigma_f^{2*}\right),$$

where

$$\begin{aligned}\mu^* &= \Phi(\mu), \\ f_i^* &= \phi(\mu) f_i, \\ \sigma_e^{2*} &= \Phi(\mu)(1 - \Phi(\mu)) - \sigma_f^{2*}, \\ \sigma_f^{2*} &= \phi(\mu)^2 \sigma_f^2, \\ h_o^2 &= \frac{2\sigma_f^{2*}}{\sigma_f^{2*} + \sigma_e^{2*}} = \frac{2\phi(\mu)^2 \sigma_f^2}{\Phi(\mu)(1 - \Phi(\mu))}.\end{aligned}\tag{9.85}$$

The last equation reveals the dependence of the heritability (on the observed scale) on the frequency of the trait in the population. These equations establish the association between the parameters of the linear approximation and those of the probit model on the underlying scale.

Assuming improper uniform prior distributions for (μ, σ_f^2) , the approximate posterior density is

$$\begin{aligned}p(\mu, f, \sigma_f^2|y) &\propto p(y|\mu, f, \sigma_f^2) p(f|\sigma_f^2) \\ &\propto (\sigma_e^{2*})^{-\frac{N}{2}} \exp\left[-\frac{1}{2\sigma_e^{2*}}(y - 1\mu^* - Zf^*)'(y - 1\mu^* - Zf^*)\right] \\ &\quad (\sigma_f^{2*})^{-\frac{n_f}{2}} \exp\left(-\frac{1}{2\sigma_f^{2*}}f^{*'}f^*\right), \quad N = n_f n,\end{aligned}\tag{9.86}$$

where f^* is the column vector with n_f elements f_i^* .

Example: Binary Data With a Full-Sib Family Structure

The approximation is implemented to analyse simulated binary data with a full-sib family structure. The focus is to infer parameters of the full-sib model using the spike and slab probit model and the normal approximation. There are 1000 full-sib families, 3 offspring per family. On the underlying scale of the true model, the intraclass correlation between full-sibs is 0.15 leading to a heritability on the

underlying scale equal to 0.30. The proportion of 1's among the 3000 binary records is approximately 0.16.

The fitted correlated probit model is executed using the R-code CODE1312 on page 709 and is described on page 414; details of the data-generating model are on page 242, except that here, the probability that individual j from family i is equal to 1 is parametrised as

$$\Pr(y_{ij} = 1 | \mu, f_i) = \Phi(\mu + f_i).$$

Then the record is generated from the Bernoulli distribution (binomial with $n = 1$):

$$y_{ij} | \mu, f_i \sim \text{Bin}(1, \Phi(\mu + f_i)).$$

Based on a single chain of length 2000, the Gibbs sampler implemented on the spike and slab probit model generated the following estimates of posterior means:

$$\begin{aligned}\widehat{\text{E}}(\mu | y) &= -1.07, \\ \widehat{\text{E}}(\sigma_f^2 | y) &= 0.173, \\ \widehat{\text{E}}(h^2 | y) &= 0.293.\end{aligned}$$

The analysis based on the normal approximation generates the following estimates of posterior means of parameters on the observed scale:

$$\begin{aligned}\widehat{\text{E}}(\mu^* | y) &= 0.160, \\ \widehat{\text{E}}(\sigma_f^{2*} | y) &= 0.0102, \\ \widehat{\text{E}}(\sigma_e^{2*} | y) &= 0.126, \\ \widehat{\text{E}}(h_o^2 | y) &= 0.149.\end{aligned}$$

These can be transformed to estimates on the underlying scale using (9.85):

$$\begin{aligned}\widehat{\text{E}}(\mu | y) &= -0.995, \\ \widehat{\text{E}}(\sigma_f^2 | y) &= 0.174, \\ \widehat{\text{E}}(h^2 | y) &= 0.291,\end{aligned}$$

in good agreement with the results of the exact analysis. However uncertainty is a little underestimated with the normal model. For example for $[\sigma_f^2 | y]$, the 95% posterior interval obtained with the exact analysis is (0.090; 0.276) and with the

approximation, (0.106; 0.268). This is probably because the approximation used is based on a first order Taylor expansion and the variance is a second order term. In general the approximation deteriorates when the proportion of 1's is more extreme and data sets are smaller.

The R-code below fits the normal approximation.

```
# CODE0906
# GAUSSIAN FULL SIB-FAMILY MODEL; SINGLE-SITE GIBBS SAMPLING
# AS AN APPROXIMATION TO THE TRUE PROBIT MODEL
rm(list=ls()) # Clear the workspace
set.seed(12345)
require(graphics)
# GENERATE CORRELATED (FULL-SIBS DATA
#install.packages("MCMCpack", .libPaths()[1])
#install.packages("mvtnorm", .libPaths()[1])
library(MCMCpack)
# INITIALISE PARAMETERS
p0 <- 0.15
mu <- qnorm(p0)
iccfs<-0.15 #INTRAClass CORRELATION FS
# VARIANCE BETWEEN FAMILIES: iccfs /(1- iccfs)
vfs <- iccfs/(1-iccfs)
nfs<-1000 # NUMBER OF FULL-SIB FAMILIES

fs<-3 #FULL-SIB FAMILY SIZE
N<-nfs*fs

c<-0
#####
### GENERATE BINARY RECORDS Y
f<-rnorm(nfs,mean=0,sd=sqrt(vfs))
p <- pnorm(mu+f)
y <- rbinom(N,1,rep(p,each=fs))
w <- rep(1:nfs,each=fs)
d<-data.frame(w,y)
family <- as.factor(w)
Z<-model.matrix(~0+family)
#####
ztz<-t(Z)%*%Z
rep<-2000
resultap<-matrix(data=NA,nrow=rep,ncol=5)
transf<-matrix(data=NA,nrow=rep,ncol=4)

#INITIALISE THE VECTOR OF FAMILIY EFFECTS fe
# (Not to be confused with the TRUE family effects f)
fe<-rep(0,nfs)
# INITIALISE BETWEEN FAMILY VARIANCE COMPONENT vf
vf<-1
# INITILISE RESIDUAL VARIANCE
ve<-1
# INITIALISE k
k<-ve/vf
# INITIALISE THE MEAN
```



```

mu<-0
sumpyinv<-0
#START GIBBS LOOP NORMAL MODEL
ptm <- proc.time()
for (i in 1:rep)
{
  print(i)
  # SAMPLE mu
  meanmu<-sum(y-Z**fe)/(nfs*fs)
  mu<-rnorm(1,mean=meanmu,sd=sqrt(ve/(nfs*fs)))
  # SAMPLE FAMILY EFFECTS f
  varf<-(k+fs)^(-1)
  fmean<- varf*(t(Z)**(y-mu))
  fe<-rnorm(nfs,mean=fmean, sd=sqrt(varf*ve))
  #SAMPLE vf
  #COMPUTE SCALE
  ftf<-sum(fe*fe)
  vfx<-ftf/rchisq(1,nfs-2)
  vf<-as.numeric(vfx)
  # SAMPLE ve
  # COMPUTE SCALE
  e<-(y-mu-Z**fe)
  ete<-t(e)**e
  vex<-ete/rchisq(1,N-2)
  ve<-as.numeric(vex)
  k<-ve/vf
  her <- (2*vf)/(vf+ve)
  resultap[i,]<-c(i,mu,vf,ve,her)
  # TRANSFORM TO PARAMETERS ON UNDERLYING SCALE
  mut <- qnorm(mu)
  vft <- vf/(dnorm(mut)**2)
  hert <- (2*vft)/(vft+1)
  transf[i,] <- c(i,mu,vft,hert)
}
proc.time()-ptm

```

Example: Binary Data With Genetic Markers

The focus of this example is to compare the prediction ability of the spike and slab probit model and of the normal approximation when both are fitted to binary data consisting of 1000 individuals (the training data) recorded each for 5000 genetic markers. The quality of the predictions is studied with various criteria using validating data drawn independently from the same distribution as the training data.

The simulated data are generated assuming that the liability of individual i and the causal genotypes (QTL) are related by the linear structure

$$u_i = z_i' b_{qtl} + e_i, \quad i = 1, 2, \dots, 1000,$$

where z'_i is a row vector of allele contents (taking values 0, 1 or 2) of causal genotypes of dimension (1×25) , b_{qtl} is a column vector of dimension (25×1) of additive genetic effects and e_i is a residual term, independent of z_i , drawn from $N(0, 1)$. The 25 QTL are assumed to act additively on the liability and the elements of vector b_{qtl} are chosen to generate an additive genetic variance on the underlying scale equal to 0.5. The probability that a record on the observed scale is equal to 1 is given by

$$\begin{aligned}\Pr(y_i = 1|z_i) &= \Pr(u_i > 0|z_i) \\ &= \Phi(z'_i b_{qtl}).\end{aligned}$$

A record is then generated from the Bernoulli process

$$y_i|z_i \sim \text{Bin}(1, \Phi(z'_i b_{qtl})).$$

Two set of data are simulated. In data set 1, marker allele frequencies and QTL allele frequencies are both equal to 0.5 and the frequency of 1s in the data is approximately 0.5. In data set 2, marker allele frequencies are 0.5, QTL allele frequencies are 0.2 and the frequency of 1s in the data is approximately 0.25. The motivation is to illustrate the prediction ability of the probit model and of the normal approximation when power is adequate (data set 1) and when it is compromised (data set 2).

The fitted spike and slab model assumes that genetic markers are associated to the liability by the relationship

$$u_i = \mu + x'_i b + \epsilon_i, \quad i = 1, 2, \dots, 1000,$$

where the scalar μ is an overall mean, x'_i is a row vector of observed marker allele contents of dimension (1×5000) , b is an unobserved column vector of marker additive effects of dimension (5000×1) , and ϵ_i is random residual, independent of x_i , drawn from $N(0, 1)$. The 25 (causal) QTL are part of the panel of 5000 markers; the analyst does not know neither their number nor their location.

The spike and slab model is implemented as described on page 414. The normal approximation is implemented as described on page 249. Both models are implemented executing the R-code CODE0908 that can be downloaded from <https://github.com/SorensenD/SLGDS> in the folder Codes. The initial part of the computer programme simulates the binary data and fits the probit spike and slab model to the training data; the bottom part fits the normal approximation using the same training data. The models are evaluated using 1000 validating records independently drawn from the same distribution as the training data.

Tables 9.4 and 9.5 show the results based on data set 1 and 2, respectively. In both tables, the headings of the first column are defined as follows:

Table 9.4 Prediction ability of the normal approximation and of the spike and slab probit model fitted to data set 1. The prediction ability is gauged using criteria specified in the first column of the table and defined in the text. Figures in brackets in rows 2 and 3 are 95% posterior intervals, reflecting posterior uncertainty of the parameters of the model

	Normal approximation	Spike and slab
–Logscore	674	672
Brier score	0.30(0.29;0.32)	0.29(0.25;0.34)
Misclass (full)	0.46(0.43;0.48)	0.41(0.39;0.44)
Misclass (null)	0.50	0.50
H	0.72	0.88
L	0.37	0.22

Table 9.5 Prediction ability of the normal approximation and of the spike and slab probit model fitted to data set 2. The prediction ability is gauged using criteria specified in the first column of the table and defined in the text. Figures in brackets in rows 2 and 3 are 95% posterior intervals, reflecting posterior uncertainty of the parameters of the model

	Normal approximation	Spike and slab
–Logscore	567	551
Brier score	0.26(0.24;0.27)	0.22(0.19;0.27)
Misclass (full)	0.36(0.34;0.39)	0.30(0.26;0.33)
Misclass (null)	0.26	0.26
H	0.52	0.60
L	0.10	0.10

- Logscore: $-\left(\sum_{i=1}^{n_v} y_{v,i} \log(\hat{P}_i) + (1 - y_{v,i}) \log(1 - \hat{P}_i)\right)$, the negative of the logscore, where $y_{v,i}$ is the i th validating datum, $i = 1, 2, \dots, n_v$; $\hat{P}_i = \overline{\Pr}(y_{v,i} = 1 | x_i, y_t)$, the McMC estimate of the mean of the posterior predictive probability that $y_{v,i} = 1$, averaged over the samples from the posterior distributions of the remaining parameters of the model; y_t is the vector of 1000 training records.
- Brier score: $\frac{1}{n_v} \sum_{i=1}^{n_v} (y_{v,i} - \hat{y}_{v,i})^2$, where $\hat{y}_{v,i}$ is the McMC estimate of the posterior predictive probability that $y_{v,i} = 1$.
- Misclass (full): validating mean squared error $\frac{1}{n_v} \sum_{i=1}^{n_v} (y_{v,i} - \hat{y}_{v,i})^2$, where, assuming Bayes rule is applied, $\hat{y}_{v,i} = I\left(\hat{\Pr}(y_{v,i} = 1 | x_i, y_t) > 0.5\right)$ and $\hat{\Pr}(y_{v,i} = 1 | x_i, y_t)$ is an McMC estimate of the posterior predictive probability that the i th validating datum is equal to 1. The mean squared error gives the proportion of misclassifications.
- Misclass (null): validating mean squared error based on the null model that assumes $\Pr(y_{v,i} = 1 | \mu) = \Phi(\mu)$ for all i . This amounts to setting all the predictions equal to $I(\Pr(y_v = 1) > 0.5)$, where I is the indicator function equal to 1 if the observed frequency of 1's in the validating data is larger than 0.5, and equal to zero, otherwise.

- H: proportion of 1's among the 50 largest estimated mean posterior predictive probabilities $\overline{\Pr}(y_{v,i} = 1|x_i, y_t)$.
- L: proportion of 1's among the 50 smallest estimated mean posterior predictive probabilities $\overline{\Pr}(y_{v,i} = 1|x_i, y_t)$.

When power is adequate and the frequency of 1's in the data is intermediate, the results in Table 9.4 indicate that the prediction ability of the normal approximation is similar to the prediction obtained with the probit spyke and slab model. However the last two rows of the table indicate that the probit model does better at detection of extremes.

When power is compromised and the frequency of 1's in the data departs from intermediate values, both models perform poorly and the discrepancy in the quality of predictions in favour of the probit model is more noticeable (Table 9.5). With the exception of the last row of the table, all the criteria show an advantage for the probit model.

Chapter 10

Bayesian Prediction and Model Checking



Aspects of Bayesian prediction have been addressed in previous chapters. In particular, Chaps. 7 and 9 show a Bayesian implementation of the spike and slab model for continuous and binary records, respectively, and illustrate how the marginal posterior distribution of validating mean squared errors can easily be computed in an MCMC environment (pages 358 and 420).

This chapter brings several elements of Bayesian prediction into focus. After a brief discussion of the type of uncertainty accounted for by different predictors, the chapter introduces the prior and the posterior predictive distributions that are key ingredients of Bayesian prediction. Two examples, one using count data and the other continuous data, illustrate the kind of predictive inferences that are possible when posterior distributions are known. One example uses an analytical approach, whereas the other extracts Monte Carlo samples from known distributions using the method of composition. The Monte Carlo samples drawn from either the marginal posterior distributions of the parameters used to construct the predictor, or from the posterior predictive distribution of the predictor, are used to generate Monte Carlo estimates of the complete marginal posterior distributions of validating mean squared errors. This makes use of the important property of ergodic averages mentioned on page 200.

Section 10.2 offers a breakdown of the Bayesian expectation of training and validating mean squared errors of prediction. This is followed by an example of a logistic model implemented using both, a Bayesian-MCMC algorithm and using maximum likelihood. The predictive ability of the model quantified using validating mean squared errors is obtained with both approaches. The uncertainty associated with the mean squared errors is also illustrated with both methods of inference.

Cross-validation has been used repeatedly to evaluate the performance of various prediction models. The chapter describes Bayesian versions of one of its variants, leave-one-out cross-validation, that require fitting the model to the complete data only once. The chapter concludes with the topic of model checking using posterior predictive simulations with an application in an MCMC environment.

10.1 Levels of Uncertainty

One of the attractions of Bayesian methods is the natural manner of accounting for uncertainty and the possibility of doing so in a single, coherent analysis. Consider the classical regression model with the $n \times 1$ training data vector y modelled as

$$y|b, \sigma^2 \sim N(xb, I\sigma^2), \quad (10.1)$$

where b is a $p \times 1$ vector of unknown regression coefficients, x is an observed $n \times p$ matrix of covariates, I is the identity matrix of dimension $n \times n$ and the scalar σ^2 is the unknown conditional variance component of a datum, the same for all data.

Data are divided into training and validating sets, fitting the model using the former and evaluating its predictive performance with the latter.

Let $\hat{b}$ represent an estimator of b using training data y and covariates x . One possible predictor (given the sampling model (10.1)) evaluated at $x = x_0$ (here a $p \times 1$ vector) that has been explored in previous chapters is

$$\hat{y}_0 = x_0' \hat{b}. \quad (10.2)$$

This predictor represents a point estimate of the average value of y_0 (evaluated at $x = x_0$) and ignores uncertainty.

A second possibility is to construct a predictor that accounts for the uncertainty associated with the unknown b . A Bayesian could do this in an MCMC environment by replicating the following two steps:

1. Draw b^* from $[b|y]$
2. Construct the predictor $\hat{y}_0 = x_0' b^*$

Rather than a point prediction as in the previous case, the draws from $[b|y]$ and the derived $\hat{y}_0$'s generate a distribution of $\hat{y}_0$. This distribution represents the propagated posterior uncertainty of b on to $\hat{y}_0$, but as (10.2), $\hat{y}_0 = x_0' b^*$ predicts an average value y_0 , given covariate x_0 and b^* .

The predictor can be used to compute posterior distributions of any function, such as validating mean squared errors. The inferential uncertainty in b is a component of the expected validating mean squared error.

A third possibility is to construct a predictor of new validating records y_0 that accounts for uncertainty about b and now also for sampling uncertainty of the new records. Again in an MCMC environment this is achieved repeating the following:

1. Draw b^* from $[b|y]$
2. Draw the predictor $\hat{y}_0^*$ from $[y_0|b^*, x_0, y]$

This generates draws from the joint posterior distribution $[y_0, b|x_0, y]$. In this case, both sources of uncertainty are components of the expected validating mean squared error. Generation of new data from $[y_0|b^*, x_0, y]$ is accounting for

sampling variation and therefore, frequency properties of the Bayesian procedure are implicitly incorporated.

The Bayesian approach leads to a partitioning of the expected validating mean squared error in line with the frequentist counterpart. However, the interpretation is different. For example in the third case in the Bayesian context, the expectation is taken over the posterior predictive distribution of predicted data, with training and validating data fixed. This results in three terms contributing to the expected validating mean squared error. In common with the frequentist approach shown in (6.66) on page 306, the first term represents sampling uncertainty of replicated (validating) data. The second represents posterior uncertainty of the predictor (see expression (10.4b) below) and the third is the average (over the posterior predictive distribution) squared discrepancy between the predictor and the observed validating data, that is held fixed.

In a frequentist setting, the sampling uncertainty of the estimate $\hat{b}$ can be quantified by replicating simulation of data, estimating b , and constructing a prediction $\hat{y}_0$ for each replicate. Using the $\hat{y}_0$'s in the construction of the mean squared error accounts for the sampling uncertainty of $\hat{b}$. An alternative to simulating new data is to resample the observed data. When a simulation strategy is chosen, parameters are replaced by their estimated values obtained from the original training data. Under both strategies, the sampling uncertainty of a new datum can be accounted for by adding an extra step consisting of drawing the predictors $\hat{y}_0^*$ from $[y_0|\hat{b}, x_0, y]$.

The concept of uncertainty and its propagation in the computation of validating mean squared errors, from a Bayesian and from a frequentist perspective, is illustrated with the logistic model example on page 485. Details of the algebra of the Bayesian expectation can be found on page 482 and in the Prediction problems on pages 626, 719 and 722.

10.2 Prior and Posterior Predictive Distributions

In the standard setup we observe an n -dimensional response vector y and the associated $(n \times p)$ matrix of covariates x . The conditional mean of y is $f(x)$ for some function f and the objective could be to obtain a prediction for a yet-to-be-observed response y_0 , a scalar drawn from the same distribution as y , using the estimated function evaluated at a value of the covariate equal to x_0 . The classical solution to this problem has been to use the training data $z_i = (x_i, y_i)$, $i = 1, 2, \dots, n$ and to construct the predictor $\hat{y}_0 = \hat{f}(z, x_0)$, where $\hat{f}$ is some estimate of f . For example, in standard least squares linear regression, $f(x_i) = x_i' b$, $\hat{f}(z, x_0) = x_0' \hat{b}$, where $\hat{b} = (x'x)^{-1} x'y$ and x_i' is the i th row of the $(n \times p)$ matrix x . In other words, the frequentist solution is to use $\hat{y}_0 = E(y_0|x_0, \hat{b})$ as the predictor, where the parameter b that indexes the conditional distribution is replaced by its least squares estimator $\hat{b}$ in this case. This prediction can be regarded as a point estimate of the average response in the population, conditional on x_0 and $\hat{b}$.

In a more general formulation, given a vector of unknown parameters θ the Bayesian solution to this prediction problem is to construct the *posterior predictive density* of y_0 ,

$$\begin{aligned} p(y_0|y, x_0) &= \int p(y_0, \theta|y, x_0) d\theta \\ &= \int p(y_0|\theta, y, x_0) p(\theta|y) d\theta \\ &= \int p(y_0|\theta, x_0) p(\theta|y) d\theta, \end{aligned} \quad (10.3)$$

where in going from line 2 to line 3, it is assumed that given θ , y_0 is conditionally independent of y and that θ and x_0 are independent. Notice that even though y_0 is conditionally independent of y , they are not marginally independent. If in the last line, $p(\theta|y)$ is replaced by the prior density $p(\theta)$, (10.3) is known as the *prior predictive distribution* of y_0 .

Having obtained (10.3), one may wish to use the mean or the mode as the point predictor $\hat{y}_0$. Consider the linear regression example, where $y_0|b, x_0, \sigma^2 \sim N(x'_0 b, \sigma^2)$. Assume that the variance σ^2 is known and that there is a prior distribution for b which is left unspecified. The mean and variance of the posterior predictive distribution are

$$E(y_0|y, x_0, \sigma^2) = E_{b|y, \sigma^2} [E(y_0|b, x_0)] = E_{b|y, \sigma^2} (x'_0 b), \quad (10.4a)$$

$$\begin{aligned} \text{Var}(y_0|y, x_0, \sigma^2) &= E_{b|y, \sigma^2} [\text{Var}(y_0|b, x_0, \sigma^2)] + \text{Var}_{b|y, \sigma^2} [E(y_0|b, x_0)] \\ &= \sigma^2 + \text{Var}_{b|y, \sigma^2} (x'_0 b). \end{aligned} \quad (10.4b)$$

The mean can be seen to be an average, over the posterior distribution of b (given σ^2 since this variance is assumed known), of the conditional mean of y_0 given b . The variance contains two terms; one representing the sampling uncertainty of the new record and the other the posterior uncertainty of $x'_0 b$. As the dimension of y increases and that of b remains constant, the second term in the last line (the variance of the posterior distribution of $x'_0 b$) vanishes but the first term does not. When the variance is not known, marginalisation of Eq. (10.4b) over $[\sigma^2|y]$ yields

$$\text{Var}(y_0|y, x_0) = E_{\sigma^2|y} (\sigma^2) + \text{Var}_{b|y} (x'_0 b).$$

Posterior predictive distributions account for sampling uncertainty of records that could have been observed, given a model. In this sense, frequency properties of the Bayesian procedure are implicitly incorporated. This applies to any function of replicated data, such as mean squared errors.

The classical frequentist approach to prediction can be regarded as an approximation to Bayesian prediction based on the posterior predictive density. Details are found in the Note on page 166 where it is shown that in large samples, for the case of the present normal linear regression,

$$p(y_0|y, x_0) \approx p(y_0|\hat{b}, \hat{\sigma}^2, x_0)$$

and therefore

$$\begin{aligned} E(y_0|y, x_0) &= \int y_0 p(y_0|y, x_0) dy_0 \\ &\approx \int y_0 p(y_0|\hat{b}, \hat{\sigma}^2, x_0) dy_0 \\ &= E[y_0|\hat{b}, \hat{\sigma}^2, x_0] \\ &= x_0' \hat{b}, \end{aligned}$$

where $(\hat{b}, \hat{\sigma}^2)$ is the maximum likelihood estimator of (b, σ^2) ($\hat{b}$ is also the least squares estimator of b in this linear regression setting).

Example: Binary Data

The following stylised example based on binary records ($y \in 0, 1$) illustrates properties of the prior and posterior predictive distributions.

Before data Y are observed, the distribution of a future binary observation, given its prior distribution, takes the form $\Pr(Y = y|\theta) = \theta^y (1 - \theta)^{1-y}$, where $E(Y|\theta) = \Pr(Y = 1|\theta) = \theta$ is the probability that the outcome is equal to 1. Then the pmf of the *prior predictive distribution* of the future record is

$$\begin{aligned} \Pr(Y = y) &= \int_0^1 p(y, \theta) d\theta \\ &= \int_0^1 p(y|\theta) p(\theta) d\theta \\ &= \int_0^1 \theta^y (1 - \theta)^{1-y} p(\theta) d\theta. \end{aligned} \tag{10.5}$$

I will assign a beta distribution with hyperparameters a and b as the prior for θ , $Be(\theta|a, b)$. These hyperparameters define how the probability mass is allocated

through the support of the distribution and define its moments. The pdf of $Be(\theta|a, b)$ is

$$p(\theta|a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{a-1} (1-\theta)^{b-1} \propto \theta^{a-1} (1-\theta)^{b-1}, \quad \theta \in [0, 1], \quad (10.6)$$

where $\Gamma(x+1) = x!$ is the gamma function that generalises the factorial function to all real numbers $x > 0$. The mean and variance are

$$\begin{aligned} E(\theta|a, b) &= \frac{a}{a+b}, \\ \text{Var}(\theta|a, b) &= \frac{ab}{(a+b)^2(a+b+1)}. \end{aligned}$$

When $a = 1, b = 1$ the beta distribution becomes a proper uniform distribution in $[0, 1]$.

Using the beta distribution as prior, (10.5) becomes

$$\begin{aligned} \Pr(Y = y) &= \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \int_0^1 \theta^{y+a-1} (1-\theta)^{b-y} d\theta \\ &= \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \int_0^1 \theta^{a^*-1} (1-\theta)^{b^*-1} d\theta, \\ &\quad a^* = a + y, b^* = 1 + b - y, \\ &= \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \frac{\Gamma(a^*)\Gamma(b^*)}{\Gamma(a^*+b^*)} \int_0^1 \frac{\Gamma(a^*+b^*)}{\Gamma(a^*)\Gamma(b^*)} \theta^{a^*-1} (1-\theta)^{b^*-1} d\theta \\ &= \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \frac{\Gamma(a^*)\Gamma(b^*)}{\Gamma(a^*+b^*)} = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \frac{\Gamma(a+y)\Gamma(b+1-y)}{\Gamma(a+b+1)}. \end{aligned} \quad (10.7)$$

In going from the second to the third line, I multiplied and divided by the constant of integration and the fourth line follows because the integral is over a proper pdf and equals 1. Expression (10.7) is a special case of the beta-binomial distribution $Bb(x|a, b, n)$ that is generated by the mixture

$$Bb(x|a, b, n) = \int_0^1 Bi(x|n, \theta) Be(\theta|a, b) d\theta, \quad (10.8)$$

where $n = 1$ for the case of the Bernoulli trial.

The probability that $Y = 1$ derived from the prior predictive distribution is obtained from (10.7), setting $y = 1$ and using $\Gamma(x+1)/\Gamma(x) = x\Gamma(x)$,

$$\begin{aligned}\Pr(Y = 1) &= \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \frac{\Gamma(a+1)\Gamma(b)}{\Gamma(a+b+1)} \\ &= \frac{a}{a+b},\end{aligned}$$

which is equal to the prior mean, as expected, since the prior is the only source of information. This result can also be arrived at more expediently using the mixture formulation (10.8),

$$\begin{aligned}\Pr(Y = 1) &= E(y) = E_{\theta}[E(y|\theta)] \\ &= E_{\theta}(n\theta) = \frac{na}{a+b},\end{aligned}$$

where $n = 1$ in the case of the Bernoulli random variable. Since the Bernoulli distribution is completely characterised by knowledge of $\Pr(Y = 1)$ (from which $\Pr(Y = 0) = 1 - \Pr(Y = 1)$ is obtained), one can also write

$$\begin{aligned}\Pr(Y = 1) &= \int_0^1 \Pr(Y = 1|\theta) p(\theta) d\theta \\ &= \int_0^1 \theta p(\theta) d\theta = E(\theta),\end{aligned}\tag{10.9}$$

again indicating the dependence of the prior prediction on the prior distribution.

Imagine now that m binary responses $y = (y_1, y_2, \dots, y_m)'$ have been observed in a first sample. What is the *posterior predictive distribution* for a scalar y_{m+1} in a second sampling? With y representing the $m \times 1$ data vector from the first sample, the posterior predictive pmf for the Bernoulli example is characterised by computing

$$\begin{aligned}\Pr(Y_{m+1} = 1|y) &= \int_0^1 \Pr(Y_{m+1} = 1|\theta) p(\theta|y) d\theta \\ &= \int_0^1 \theta p(\theta|y) d\theta = E(\theta|y),\end{aligned}\tag{10.10}$$

equal to the posterior mean. Also, using (10.8)

$$E(Y_{m+1}|y) = E_{\theta|y}[E(Y_{m+1}|\theta)] = E(\theta|y).$$

Applying Bayes theorem, the posterior density is

$$\begin{aligned}
 p(\theta|y) &= \frac{p(y|\theta) p(\theta)}{p(y)} \\
 &= \frac{1}{p(y)} \prod_{i=1}^m \theta^{y_i} (1-\theta)^{1-y_i} \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{a-1} (1-\theta)^{b-1} \\
 &= K \theta^{\sum_{i=1}^m y_i} (1-\theta)^{m-\sum_{i=1}^m y_i} \theta^{a-1} (1-\theta)^{b-1} \\
 &\propto \theta^{a+\sum_{i=1}^m y_i-1} (1-\theta)^{b+m-\sum_{i=1}^m y_i-1}, \\
 K &= \frac{\Gamma(a+b)}{p(y) \Gamma(a) \Gamma(b)},
 \end{aligned} \tag{10.11}$$

which is the kernel of a $Be(\tilde{a}, \tilde{b})$ distribution, where

$$\begin{aligned}
 \tilde{a} &= a + \sum_{i=1}^m y_i, \\
 \tilde{b} &= b + m - \sum_{i=1}^m y_i.
 \end{aligned}$$

Then (10.10) is

$$\begin{aligned}
 \Pr(Y = 1) &= E(\theta|y) \\
 &= \frac{\tilde{a}}{\tilde{a} + \tilde{b}} = \frac{a + \sum_{i=1}^m y_i}{a + b + m}.
 \end{aligned} \tag{10.12}$$

Given the model, the predictive posterior distribution is affected by the prior input through a and b , but the influence dissipates as sample size m increases, since a and b are usually small numbers relative to m and $\sum_{i=1}^m y_i$.

Example: Continuous Data

This example considers the linear regression model

$$y|b, \sigma^2 \sim N(xb, I\sigma^2) \tag{10.13}$$

with independent improper uniform prior distributions associated with the parameters (b, σ^2) .

The objective of this example is to illustrate Monte Carlo estimation of the distribution of 3 different validating mean squared errors that account for different

sources of uncertainty of the predictor. The example is essentially an illustration of the construction of the distribution of one random variable (the mean squared error) using the known distribution of another random variable (the predicted values) and how easily this is accomplished using Monte Carlo methods.

The data (a vector with 2000 records) are simulated as follows:

- the vector b of regression coefficients, of dimension 150×1 , is drawn from a normal distribution $b \sim N(0, 4 \times I)$
- the elements of the full rank matrix x of covariates, of dimension 2000×151 , are drawn from a binomial distribution $x \sim \text{Bin}(2, 0.5)$, with an extra column vector of 1's added to account for an intercept which is set equal to $\mu = 10$
- vector of records y , of dimension the 2000×1 , is drawn from a normal distribution $y \sim N(xb, 20 \times I)$

The 2000 records are divided into a training (t) and a validating (v) set of equal size, so that $y = (y'_t, y'_v)'$.

The parameters of the model are drawn from the appropriate posterior distribution, given training data y_t , using the method of composition (see page 164) as follows:

- Draw σ^{2*} from the marginal posterior distribution (see Eq. (4.40))

$$[\sigma^2 | y_t] \sim \chi^{-2}(v, S),$$

a scale inverted chi-square distribution with $n - (p + 1) - 2$ degrees of freedom and scale parameter

$$\frac{1}{v} (y - x\hat{b})' (y - x\hat{b}),$$

where $\hat{b} = (x'x)^{-1} x'y$, as in (4.70) on page 175 and $p = 150$

- Draw b^* from the conditional posterior distribution

$$[b | \sigma^{2*}, y_t] \sim N(\hat{b}, (x'x)^{-1} \sigma^{2*}),$$

a normal distribution (see page 167 for a derivation)

- Draw $y_{v,pred}^*$, the 1000×1 vector of predicted validating records from the conditional posterior distribution

$$[y_{v,pred} | b^*, \sigma^{2*}] \sim N(x_v b^*, I \sigma^{2*}),$$

a normal distribution (given (b^*, σ^{2*}) , $y_{v,pred}$ is independent of y_t).

These steps are repeated 1000 times to obtain the required sample. In each cycle, $(\sigma^{2*}, b^*, y_v^*)$ is a sample from the joint posterior distribution $[\sigma^2, b, y_v | y_t]$, while y_v^* is a sample from the posterior predictive distribution $[y_{v,pred} | y_t]$.

Using the draws from the joint posterior distribution the following functions are calculated. First

$$\text{MSE}_{v,1} = \frac{1}{750} \left(y_v - x_v \bar{b}^* \right)' \left(y_v - x_v \bar{b}^* \right), \quad (10.14)$$

a point estimator of the validating mean squared error of average predictions, where $\bar{b}^* = \hat{E}(b|y)$, the Monte Carlo estimate of the posterior mean of b obtained using the Monte Carlo draws b^* from the marginal posterior distribution $[b|y]$.

Secondly, for each of the 1000 cycles using the draws b^* compute

$$\text{MSE}_{v,2} = \frac{1}{750} \left(y_v - x_v b^* \right)' \left(y_v - x_v b^* \right). \quad (10.15)$$

The elements of (10.15) constructed in this way, constitute extractions from the Monte Carlo estimate of the marginal posterior distribution of $\text{MSE}_{v,2}$. This posterior distribution reflects the propagated posterior uncertainty of b .

Thirdly, for each of the 1000 cycles, compute

$$\text{MSE}_{v,3} = \frac{1}{750} \left(y_v - y_{v,pred}^* \right)' \left(y_v - y_{v,pred}^* \right), \quad (10.16)$$

This estimator of validating mean squared error of predictions of individual data points accounts for posterior uncertainty of the parameters of the Bayesian model and for sampling uncertainty of the new data. The draws (10.16) constitute extractions from the Monte Carlo estimate of the marginal posterior distribution of $\text{MSE}_{v,3}$. The Monte Carlo estimates of the marginal posterior distributions of $\text{MSE}_{v,2}$ and $\text{MSE}_{v,3}$ are displayed as histograms in Fig. 10.1.

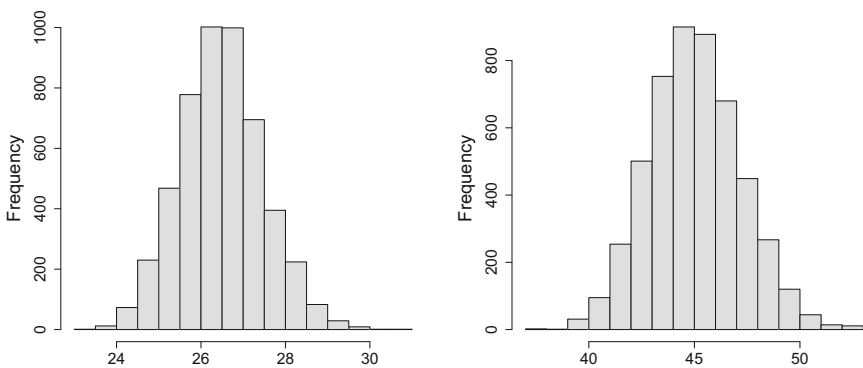


Fig. 10.1 Left: histogram of the marginal posterior distribution of $\text{MSE}_{v,2}$; the validating mean squared error accounting for posterior uncertainty of the parameters of the Bayesian model. Right: histogram of the marginal posterior distribution of $\text{MSE}_{v,3}$; the validating mean squared error accounting for posterior uncertainty of the parameters of the Bayesian model and for sampling uncertainty of new validating records

The R-code below simulates the data, executes the method of composition and computes the validating mean squared errors.

```
# CODE1001
rm(list=ls()) # CLEAR WORKSPACE
set.seed(123)

nindiv <- 2000
nmark <- 150
nsamples <- nindiv*nmark
# GENERATE COVARIATE MATRIX FROM BINOMIAL DISTRIBUTION
X<-matrix(nrow=nindiv,ncol=nmark,
          rbinom(n=nsamples,size=2,p=.5))
#####
# CHOOSE VALUE FOR MEAN mu
mu <- 10
# CHOOSE VALUE FOR ENVIRONMENTAL VARIANCE ves
ves<-20
b<-matrix(data=0.0,nrow=nmark,ncol=1) # b from operational model

et<- rnorm(nindiv,mean=0,sd=sqrt(ves))
b <- rnorm(nmark,mean=0,sd=2)
y <- mu + X %*%b + et
train <- sample(1:nrow(X),floor(0.5*nrow(X)))
Xt <- X[train,]
yt <- y[train]
Xv <- X[-train,]
yv <- y[-train]
Zt <- cbind(1,Xt)
Zv <- cbind(1,Xv)
#####
##### coefficient matrix LHSt, rhs & solution solt
RHSt <- crossprod(Zt,yt)
LHSt <- crossprod(Zt)
solt <- solve(LHSt,RHSt)
e <- yt-Zt%*%solt
#####
rep <- 5000 # NUMBER OF DRAWS USING COMPOSITION
ystartrain <- matrix(data=NA,nrow=length(yt),ncol=1)
ystarval <- matrix(data=NA,nrow=length(yt),ncol=1)

resMSE <- matrix(data=NA,nrow=rep,ncol=3)
res <- matrix(data=NA,nrow=rep,ncol=(nmark+2))

scale <- sum(e^2)
Cinv <- solve(LHSt)
ch <- chol(Cinv)
ptm<-proc.time()

for (i in 1:rep){
  # print(i)
  df <- length(yt)-(nmark+1)-2
  # DRAW RESIDUAL VARIANCE
```

```

varstar <- scale/rchisq(1,df)
resid <- rnorm(length(solt),0,1)
# DRAW LOCATION PARAMETERS
bstar <- solt + t(ch)%*% resid*sqrt(varstar)
ystartrain <- Zt%*%bstar +
  rnorm(length(ystartrain),0,sd=sqrt(varstar))
# # DRAW VALIDATING DATA
ystarval <- Zv%*%bstar +
  rnorm(length(ystarval),0,sd=sqrt(varstar))
msevalystar <- mean((ystarval-yv)^2) # MSE_3
msetrainystar <- mean((ystartrain-yt)^2)
msevalbstar <- mean((Zv%*%bstar-yv)^2) # MSE_2
resMSE[i,] <- c(msetrainystar,msevalystar,msevalbstar)
res[i,] <- c(varstar,bstar)
}
proc.time()-ptm

```

```

##      user  system elapsed
##    17.62    0.02    4.23

```

```

av <- apply(res,2,mean)
bstarhat <- av[2:(nmark+2)]
ystarhatval <- Zv%*%bstarhat
mse_1 <- mean((ystarhatval-yv)^2) # POINT PREDICTOR
mserrors <- apply(resMSE,2,mean)
mse_2 <- mserrors[3]
mse_3 <- mserrors[2]
ci_2 <- quantile(resMSE[,3],c(0.025,0.975))
ci_3 <- quantile(resMSE[,2],c(0.025,0.975))

```

The Monte Carlo estimate of the point estimator $MSE_{v,1}$ is equal to 23.2. The mean of the marginal posterior distributions of $MSE_{v,2}$ and the 95% posterior interval is 26.5, (24.6, 28.5) and the corresponding figures for $MSE_{v,3}$ are 45, (41, 49.4), the latter reflecting, as expected, the extra uncertainty contributed by sampling of new records $y_{v,pred}$.

10.3 Bayesian Expectations of MSE

In the Bayesian setting the expected values of the validating mean squared errors are composed of terms that generate a structure similar to the frequentist counterpart discussed on page 305. The expectations of (10.15) and (10.16) can be obtained as follows. The focus is on single terms of the sum, since the expectation of the sum is equal to the sum of the expectations of each record. From (10.15) for the i th term,

$$E_{b|y}(MSE_{i,v,2}) = E_{b|y}(y_{i,v} - \hat{y}_{i,v})^2 \quad (10.17)$$

where the predictor is $\hat{y}_{i,v} = x'_{i,v}b$, b is a draw from $[b|y_t]$, $E_{b|y}$ stands for the expectation over the posterior distribution $[b|y]$ and y includes the vector of training (y_t) and validating (y_v) records. The random variable in (10.17) is $\hat{y}_{i,v}$, a function of the random variable b . Expanding the square and taking expectations yields

$$\begin{aligned} E_{b|y} (\text{MSE}_{i,v,2}) &= y_{i,v}^2 + \text{Var}_{b|y_t} (\hat{y}_{i,v}) + (E_{b|y_t} (\hat{y}_{i,v}))^2 - 2y_{i,v} E_{b|y_t} (\hat{y}_{i,v}) \\ &= \text{Var}_{b|y_t} (\hat{y}_{i,v}) + (y_{i,v} - E_{b|y_t} (\hat{y}_{i,v}))^2 \\ &= \text{Var}_{b|y_t} (x'_{i,v}b) + (y_{i,v} - x'_{i,v}\hat{b})^2, \end{aligned} \quad (10.18)$$

where $\hat{b} = E(b|y_t)$. The first term is the contribution from the posterior variance of the predictor; the second term is the i th term of $\text{MSE}_{v,1}$ (where $\hat{b}$ replaces the Monte Carlo estimate), the point predictor that ignores all sources of uncertainty.

Using similar algebra, the expectation of single terms in (10.16) is

$$\begin{aligned} E_{y_v^*|y} (\text{MSE}_{i,v,3}) &= \text{Var}_{y_v^*|y_t} (y_{i,v}^*) + (y_{i,v} - E_{y_v^*|y_t} (y_{i,v}^*))^2 \\ &= E_{\sigma^2|y_t} (\sigma^2) + \text{Var}_{b|y_t} (x'_{i,v}b) + (y_{i,v} - x'_{i,v}\hat{b})^2, \end{aligned} \quad (10.19)$$

where I used $y_{i,v}^*$ instead of $y_{i,v,pred}^*$ to simplify the notation. The random variable in this case is the posterior predicted value of a future validating record $y_{i,v}^*$. The step from the first to the second line uses the decomposition of the posterior predictive variance (10.4b), here accounting for the unknown variance σ^2 (see page 723 for details). The first term in (10.19) is the posterior expectation of the irreducible error: the sampling variance of the future validating datum; the second is the contribution from the variance of the conditional mean of y_v^* under its sampling model (due to posterior uncertainty of b) and the third term, equal to the second term in (10.18) as revealed by (10.4a), is the squared discrepancy between the validating datum y_v and the mean of the posterior predictive distribution of y_v^* .

Define

$$E_{b|y} (\text{MSE}_{v,1}) = \frac{1}{N} \sum_{i=1}^N (y_{i,v} - E_{b|y} (x'_i b))^2$$

where N is the number of records in the validating data. Then the relation between the expectations of the three MSE's is

$$E_{b|y} (\text{MSE}_{v,2}) = E_{b|y} (\text{MSE}_{v,1}) + \frac{1}{N} \sum_{i=1}^N \text{Var}_{b|y} (x'_i b),$$

$$E_{y_v^*|y} (\text{MSE}_{v,3}) = E_{b|y} (\text{MSE}_{v,2}) + E_{\sigma^2|y} (\sigma^2).$$

Let $y_{i,t}^*$ represent the i th draw from the posterior predictive distribution $[\cdot|y_t, x_t]$ involving the training data. The Bayesian expectation of the training mean squared error for the i th term takes the form

$$\begin{aligned} E_{y_t^*|y_t} (\text{MSE}_{3,t,i}) &= \text{Var}_{y_t^*|y_t} (y_{i,t}^*) + (y_{i,t} - E_{y_t^*|y_t} (y_{i,t}^*))^2 \\ &= E_{\sigma^2|y_t} (\sigma^2) + \text{Var}_{b|y_t} (x'_{i,t}b) + (y_{i,t} - x'_{i,t}\hat{b})^2. \end{aligned} \quad (10.20)$$

This has exactly the same structure as (10.19) with x_v and y_v replaced by x_t and y_t . The important difference between (10.19) and (10.20) is in their third terms. In (10.20), prediction and estimation use the same training data. On the other hand, in (10.19), estimation is based on training data and prediction on validating data. Consequently, the squared term corresponding to the training mean squared error is smaller than the squared term of the validating mean squared error, reflecting overfitting.

A little more specifically, overfitting is reflected in the in-sample “correlation” between the vectors of data (considered fixed in the Bayesian framework) and the means of the draws from the posterior predictive distributions. The in-sample “correlation” is a component of the third terms of (10.19) and (10.20). It contributes negatively to these third terms and is larger in the training data than in the validating data.

The development in this subsection assumes so far that data y are conditionally normally distributed. However the same ideas apply for the case of binary records where $y_i = 0, 1$. Let b denote a draw from the posterior distribution of regression parameters $[b|y]$. From these draws a predicted probability is constructed that could take the form

$$\hat{y}_i = \hat{\pi}_i = \Phi(x'_i b)$$

where $\Phi(x'_i b) = \Pr(y_i = 1|x_i, b)$. The contribution from record i to the expectation of the Brier score is

$$\begin{aligned} E_{b|y} (\text{MSE}_{i,v,2}) &= E_{b|y} (y_{i,v} - \hat{\pi}_i)^2 \\ &= \text{Var}_{b|y_t} [\Phi(x'_i b)] + (y_{i,v} - E_{b|y_t} [\Phi(x'_i b)])^2 \end{aligned}$$

equal to the second line in (10.18). Both the variance and the expectation are taken over the posterior distribution $[b|y]$; this distribution describes the posterior uncertainty of b that propagates on to the validating mean squared error.

An expression along the lines of (10.19) is obtained when the binary predictor $\hat{y}_v$ is a draw from the posterior predictive distribution. This is achieved by

drawing b from $[b|y]$, and given b , drawing $\hat{y}_{i,v}$ from the binary distribution $\text{Bin}(n = 1, \Phi(x'_i b))$. In this case,

$$\begin{aligned} E_{\hat{y}_v|y}(\text{MSE}_{i,v,3}) &= E_{\hat{y}_v|y} (y_{i,v} - \hat{y}_{i,v})^2 \\ &= E_{b|y_i} [\Phi(x'_i b) (1 - \Phi(x'_i b))] \\ &\quad + \text{Var}_{b|y_i} [\Phi(x'_i b)] + (y_{i,v} - E_{b|y_i} [\Phi(x'_i b)])^2. \end{aligned}$$

The validating mean squared error includes sampling (binary) uncertainty from the draws $\hat{y}_{i,v}$ as well as posterior uncertainty contributed by the draws from $[b|y]$. There is a third term due to the squared difference between a validating datum and its expectation under the posterior predictive distribution, corresponding to the point predictor.

The prediction models for binary data can also be evaluated using logscores, as illustrated in the example on page 423.

10.4 Example: Bayesian and Frequentist Measures of Uncertainty

This example illustrates the computation of two measures of validating mean squared error for binary predictions. The first accounts only for uncertainty of the estimated parameters and again is labelled MSE_2 . The second accounts for both, uncertainty of the estimated parameters and uncertainty due to sampling of single records and is labelled MSE_3 .

Bayesian and frequentist approaches are used for inferences; the Bayesian approach is implemented with MCMC (Metropolis–Hastings) and the frequentist with classical likelihood using the R-function GLM. The objective is to show how uncertainty can be accounted for using both schools of inference and to make a comparison within the settings of the example.

Binary data (2000 records) are simulated using a logistic model with two covariates, x_1 and x_2 . The parameters of the logistic model are $\theta = (\beta_0, \beta_1, \beta_2)$. Specifically for the i th datum, $y_i \sim \text{Br}(p_i)$, a Bernoulli distribution with parameter p_i , where

$$\begin{aligned} p_i &= \Pr(y_i = 1 | x_{1i}, x_{2i}) \\ &= \frac{\exp(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i})}{1 + \exp(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i})}. \end{aligned} \tag{10.21}$$

The covariates are simulated from uniform distributions $x_{1i} \sim \text{Un}(-1, 1)$, $x_{2i} \sim \text{Un}(-1, 1)$ and the three regression parameters are set equal to $\beta_0 = -1.3863$, $\beta_1 = 2$, $\beta_2 = 2$. The intercept β_0 is chosen to generate a proportion of 1's in the

vicinity of 20%. The 2000 records were randomly divided into two sets of 1000 records each corresponding to training and validating data.

The R-code below generates the binary data.

```
# CODE1002
##### SIMULATE BINARY RECORDS FROM LOGISTIC MODEL
### AND FIT LOGISTIC MODEL (2 COVARIATES) WITH M-H
### CREATE TRAINING AND TESTING/VALIDATING DATA
rm(list=ls()) # CLEAR WORKSPACE
set.seed(77111)
require(graphics)
# THE CODE WILL USE THE PACKAGE MVTNORM; IT IS INSTALLED BELOW
#install.packages("mvtnorm", .libPaths()[1])
library(mvtnorm)
library(MASS)
#CHOOSE LENGTH OF CHAIN rep
rep<-6000
result<-matrix(data=NA,nrow=rep,ncol=6)
nindiv <- 2000
x1 <- rep(0,nindiv)
x2 <- rep(0,nindiv)
y <- rep(0,nindiv)
p<-0.2
x1 <- runif(nindiv,-1,1)
x2 <- runif(nindiv,-1,1)

b_0 <- log(p/(1-p))
b_1 <- 2
b_2 <- 2
z <- b_0 + b_1*x1 + b_2*x2
p1 <- exp(z)/(1+exp(z))
## GENERATE THE BINARY RECORDS
y <- rbinom(length(p1),1,p1)
# CONSTRUCT THE DATA SET
dat1 <- matrix(c(y,x1,x2),nrow = nindiv, ncol = 3)
colnames(dat1) <- c("Y", "X1", "X2")
d<-data.frame(dat1)
#attach(d)
Y <- d$Y
X1 <- d$X1
X2 <- d$X2
set.seed(771)
train=sample(1:length(X1),length(X1)/2)
test=(-train)
y.test=Y[test]
y.train<-Y[train]
length(y.train)
```

```
## [1] 1000
```

The Bayesian approach is implemented using Metropolis-Hastings. The algorithm is very similar to the one used on page 697, with a modification to account for the extra covariate.

The R-code to implement the Metropolis-Hastings algorithm is shown below.

```
# CODE1002 (cont)
# CHOOSE TUNING PARAMETER LAMBDA AND COVARIANCE MATRIX C
# OF THE METROPOLIS-HASTINGS ALGORITHM
lambda<-0.015
c <- diag(c(1.5,0.6,0.6))
# INITIALISE THE MEAN OF THE TRIVARIATE DISTRIBUTION
theta<-c(-5,1.0,1.0)

## FUNCTION TO COMPUTE THE LOG-POSTERIOR
logpost <- function(data,theta)
{
  interm <- theta[1] + theta[2]*data$X1 + theta[3]*data$X2
  with(data=data,sum(Y*( interm )-log(1+exp(interm))))
}
#START M-H LOOP
ptm <- proc.time()
accept<-0
for (i in 1:rep)
{
  # print(i)
  #SAMPLE PROPOSAL FOR THETA (Ytheta) FROM N(theta,lamdaC)
  Ytheta<- rmvnorm(1,mean=theta,sigma=lambda*c)
  logalfa<-logpost(d[train,],Ytheta) - logpost(d[train,],theta)
  unif<-runif(1)
  if (unif<exp(logalfa))
  {
    theta[1]<-Ytheta[1]
    theta[2]<-Ytheta[2]
    theta[3]<-Ytheta[3]
    interm <- theta[1] + theta[2]*X1[test] + theta[3]*X2[test]

    proby1 <- exp(interm)/(1+exp(interm))
    yhat <- rbinom(length(Y[test]),1,proby1)
    yhatBR <- ifelse(proby1 > 0.5, 1, 0)
    accept<-accept+1
  }
  else
  {
    interm <- theta[1] + theta[2]*X1[test] + theta[3]*X2[test]
    proby1 <- exp(interm)/(1+exp(interm))
    yhat <- rbinom(length(Y[test]),1,proby1)
    yhatBR <- ifelse(proby1 > 0.5, 1, 0)
  }
  misclas <- mean((yhat-Y[test])**2)
  misclasBR <- mean((yhatBR-Y[test])**2)
  brier <- mean((Y[test]-proby1)**2)
  vyhat <- var(yhat)
  logscYV <- sum(Y[test]*log(proby1)+(1-Y[test])*log(1-proby1))
  result[i, ]<-c(i,theta[1],theta[2],theta[3],misclas,misclasBR)
}
proc.time()-ptm
```

```
##      user  system elapsed
##      5.77    0.09    5.86
```

```
acceptrato <- accept/rep
# PRINT ACCEPTANCE RATIO OF THE JOINT UPDATING
acceptrato
```

```
## [1] 0.454
```

```
# PRINT THE MCMC ESTIMATES OF POSTERIOR MEANS
apply(result[501:rep,5:6],2,mean)
```

```
## [1] 0.2933327 0.2184742
```

```
# PRINT 95% POSTERIOR INTERVALS FOR THE MISCLASSIFICATION RATES
misclasQ <- quantile(result[500:rep,5],c(0.025,0.975))
misclasQ
```

```
##      2.5% 97.5%
##      0.268 0.320
```

```
misclasBRQ <- quantile(result[500:rep,6],c(0.025,0.975))
misclasBRQ
```

```
##      2.5% 97.5%
##      0.212 0.229
```

```
# THE MCMC ESTIMATES OF MEAN OF POSTERIOR DISTRIBUTION OF
# REGRESSION PARAMETERS ARE
meanbetas <- apply(result[500:rep,2:4],2,mean)
meanbetas
```

```
## [1] -1.470091 2.037410 2.063063
```

```
# AND THE 95% POSTERIOR INTERVALS ARE
quantile(result[500:rep,2],c(0.025,0.975))
```

```
##      2.5%      97.5%
## -1.678812 -1.272822
```

```
quantile(result[500:rep,3],c(0.025,0.975))
```

```
##      2.5%      97.5%
## 1.716287 2.389274
```

```
quantile(result[500:rep,4],c(0.025,0.975))
```

```
##      2.5%      97.5%
## 1.688001 2.393371
```

The Monte Carlo estimates of posterior means of β_0 , β_1 and β_2 (-1.47 , 2.04 , 2.06), are in good agreement with the simulated values ($\beta_0 = -1.39$, $\beta_1 = 2$, $\beta_2 = 2$).

The frequentist implementation maximises the logistic loglikelihood

$$\ell(\theta|y_t, x) = \sum_{i=1}^{n_t} \{y_{t,i} x'_i \theta - \ln(1 + \exp(x'_i \theta))\} \quad (10.22)$$

with respect to θ . In (10.22), y_t is the vector of $n_t = 1000$ training records and x'_i is the i th row of the $n_t \times 3$ matrix of covariates x with a column vector of 1's to account for β_0 . The R function `glm` is used to perform the likelihood analysis. Frequentist uncertainty is incorporated generating 1000 bootstrap replications of training/validating splits. The R-code shown below implements the logistic likelihood.

```
# CODE1002 (cont)
# FIT MODEL BY ML
set.seed(771)
nrepl <- 1000
resLik <- matrix(data=NA, nrow=nrepl, ncol=3)
ptm <- proc.time()
for (i in 1:nrepl){
  train=sample(1:nrow(d), nrow(d)/2)
  f<-glm(Y ~., data=d[train,], family="binomial")
  # summary(f)
  #f<-glm(Y ~1, data=d[train,], family="binomial")
  # summary(f)
  # PRED PROBABILITIES:
  predV <- predict(f, d[-train,], type="response")
  BrierFreq <- mean((predV-d$Y[-train])^2)
  # ASSIGN Y TO ITS CLASS ACCORDING TO BAYES RULE
  yhatBR <- ifelse(predV > 0.5, 1, 0)
  mseBR <- mean((yhatBR-d$Y[-train])^2) # MSE_1
  ## OR SAMPLE Y FROM ITS PREDICTIVE DISTRIBUTION
  ## CONDITIONAL ON ML ESTIMATES - THIS MAKES
  ## IT COMPARABLE TO THE mcmc APPROACH
  yhatPpd <- rbinom(length(Y[-train]), 1, predV)
  msePpd <- mean((yhatPpd-d$Y[-train])^2) # MSE_2
  resLik[i,] <- c(i, mseBR, msePpd)
}
proc.time()-ptm
```

```
##      user  system elapsed
##      4.44      0.09      4.53
```

```
apply(resLik[,2:3],2,mean)

## [1] 0.210431 0.288090

quantile(resLik[,2],c(0.025,0.975))

## 2.5% 97.5%
## 0.192 0.229

quantile(resLik[,3],c(0.025,0.975))

## 2.5% 97.5%
## 0.264 0.313
```

Results are shown in Table 10.1 that includes in the bottom row the predictive performance of the null model with the intercept β_0 as the only parameter of the loglikelihood (10.22).

The uncertainty in θ contributes to MSE_2 whereas both, uncertainty in θ and sampling variation of validating records y_v^* contribute to MSE_3 . This is reflected in the difference in mean values of MSE_2 and MSE_3 .

The figures in the table show that the full model does better than the null model on all accounts. Both methods of inference produce very similar results, not only in terms of point estimates, but also in terms of measures of uncertainty. This is not a general conclusion. In more complicated hierarchical models, marginalisation of nuisance parameters as part of the standard Bayesian machinery leads to larger (and more appropriate) measures of uncertainty than the standard likelihood approach. This often translates into larger values of MSE and larger uncertainty intervals.

The exercises section on page 626 includes several examples of the computation of mean squared errors, from a frequentist and Bayesian perspective, accounting for different sources of uncertainty.

Table 10.1 Posterior means (for the Bayesian model), means over bootstrap replicates (for the likelihood model) and 95% intervals (in brackets; Monte Carlo estimates of posterior intervals for the Bayesian model; bootstrap frequentist intervals for the likelihood model) of validating mean squared errors MSE_2 and MSE_3 , obtained from a Bayesian MCMC implementation (Bayesian MCMC) and from a frequentist, maximum likelihood implementation (ML), for a logistic model with two predictors. The corresponding results obtained fitting the null model (a model containing only a mean at the level of the logit) are displayed in the row labelled Null model. The null model acts as a benchmark

	MSE_2	MSE_3
Bayesian MCMC	0.22	0.29
	(0.21; 0.23)	(0.27; 0.32)
ML	0.21	0.29
	(0.19; 0.23)	(0.26; 0.31)
Null model	0.30	0.42
	(0.28; 0.32)	(0.39; 0.44)

10.5 Bayesian Leave-one-out Cross-validation

Chapter 6 describes an implementation shortcut for leave-one-out cross-validation for linear smoothers that requires running through the training data only once. This section describes Bayesian analogues to this idea that are not restricted to linear smoothers. Relevant background literature is Rubin (1988), Smith and Gelfand (1992), Gelfand (1996), Vehtari and Lampinen (2002), Albert (2009) and Gianola and Schön (2016).

In a general setting the leave-one-out validating mean squared error is equal to

$$\text{MSE}_v = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (10.23)$$

where N is the total number of records and y_i is the i th observed record.

A first approach that accounts for posterior uncertainty about the parameters of the Bayesian model and for sampling variation of an individual record, is to draw a predicted value $\hat{y}_i$ from the posterior predictive distribution with density

$$p(y_i | y_{-i}) = \int p(y_i | \theta, y_{-i}) p(\theta | y_{-i}) d\theta, \quad (10.24)$$

where often, $p(y_i | \theta, y_{-i}) = p(y_i | \theta)$ assuming that given θ , y_i is independent of y_{-i} . In the expression above, θ is the vector of parameters of the Bayesian model and the vector of complete data y is $y = (y_i, y_{-i})$, where y_{-i} is y excluding the i th datum. From a Bayesian perspective inferences are conditional on the observed data y and therefore the only stochastic term in (10.23) is $\hat{y}_i$. Since MSE_v is a transformation of $\hat{y}_i$, (10.23) defines a posterior predictive distribution. An estimate of this distribution can be obtained using MCMC: once the Markov chain has converged, vector $(\hat{y}_1^{(t)}, \hat{y}_2^{(t)}, \dots, \hat{y}_N^{(t)})$, $t = 1, 2, \dots, T$, drawn from the posterior predictive distribution at cycle t of a Markov chain of length T is used to compute $\text{MSE}_v^{(t)}$. This single draw $\text{MSE}_v^{(t)}$ constitutes one approximate sample from the marginal posterior distribution of MSE_v . A Monte Carlo estimate of the complete marginal posterior distribution of MSE_v is constructed using the collection of T samples.

The computing protocol to obtain a sample from the posterior predictive distribution of (10.23) is straightforward; at cycle t of an MCMC sampler that has reached convergence to its stationary distribution, for datum i , $i = 1, 2, \dots, N$,

- Draw $\theta^{(t)}$ from $[\theta | y_{-i}]$,
- Given $\theta^{(t)}$, draw $\hat{y}_i^{(t)}$ from $[y_i | \theta^{(t)}]$ if y_i is continuous or generate $\hat{y}_i^{(t)}$ from $\text{Pr}(Y_i = y_i | \theta^{(t)})$ if Y is binary; set $i \leftarrow i + 1$ and go back to the first step until $i = N$,

- Once all the N observations have been processed, compute $\text{MSE}_v^{(t)} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i^{(t)})^2$.

When the Markov chain has converged to the posterior distribution, each vector $(\theta^{(t)}, \hat{y}_i^{(t)})$ is an approximate draw from $[\theta, y_i | y_{-i}]$ and the coordinates are approximate draws from $[\theta | y_{-i}]$ and $[y_i | y_{-i}]$. Further, $\text{MSE}_v^{(t)}$ is a draw from the posterior predictive distribution of MSE_v . The computation of this estimator of the leave-one-out validating mean squared error is shown in the R-code below, `sumloo` and `mselooBayes`.

A second approach is to construct a Bayesian leave-one-out validating mean squared error that incorporates posterior uncertainty about the parameters of the model. This is achieved by defining the predictor $\hat{y}_i^{(t)}$ as a function of $\theta^{(t)}$ after the first step in the protocol. For example, in the Gaussian linear regression model where $\theta = (b, \sigma^2)$, $\hat{y}_i^{(t)} = x_i' b^{(t)}$, where b is a vector of unknown regression parameters and x_i is an observed vector of covariates. In a probit model for the analysis of binary records, $\theta = b$ and $\hat{y}_i^{(t)}$ can be constructed using Bayes rule $I[\Phi(x_i' b^{(t)}) > 0.5]$, where $I[\cdot]$ is the indicator function that takes the value 1 if the argument is satisfied, and 0 otherwise, and $\Phi(\cdot)$ is the standard normal integral. The computation of this estimator of the leave-one-out validating mean squared error is shown in the R-code below, `sumXb` and `mselooXb`.

These approaches can be applied to a range of models of different complexity, including hierarchical models for analysis of binary and categorical data. *The challenge is to obtain draws from the distribution $[\theta | y_{-i}]$, $i = 1, 2, \dots, N$, running through the data only once rather than N times.* A useful option is to use importance sampling (see Chap. 4, page 208). In order to obtain approximate draws from the distribution $[\theta | y_{-i}]$ using the available collection of samples $\theta^* = (\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(T)})$, each from the distribution $[\theta | y]$, first calculate the importance ratio

$$w_{ij} = \frac{p(\theta^{(j)} | y_{-i})}{p(\theta^{(j)} | y)}, \quad i = 1, 2, \dots, N; \quad j = 1, 2, \dots, T, \quad (10.25)$$

and then resample with replacement from the vector θ^* with probabilities proportional to w_{ij} . This is a weighted bootstrap where samples with replacement from θ^* are drawn with unequal probabilities $w_{i1}, w_{i2}, \dots, w_{iT}$. The resulting vector is an approximate sample from the distribution $[\theta | y_{-i}]$ (Gelfand 1996). The importance ratio (10.25) takes the following simple form:

$$\begin{aligned} \frac{p(\theta^{(j)} | y_{-i})}{p(\theta^{(j)} | y)} &= \frac{p(y_{-i} | \theta^{(j)}) p(\theta^{(j)}) / p(y_{-i})}{p(y | \theta^{(j)}) p(\theta^{(j)}) / p(y)} \\ &= \frac{p(y_{-i} | \theta^{(j)})}{p(y_i, y_{-i} | \theta^{(j)})} \frac{p(y)}{p(y_{-i})} \end{aligned}$$

$$\begin{aligned}
&= \frac{1}{p(y_i|y_{-i}, \theta^{(j)})} \frac{p(y)}{p(y_{-i})} \\
&\propto \frac{1}{p(y_i|\theta^{(j)})}
\end{aligned} \tag{10.26}$$

that holds also for probability mass functions associated with discrete data. The final step assumes that y_i and y_{-i} are conditionally independent, given θ . A vector with T elements representing probabilities (that sum to 1) is obtained by scaling (10.26); element (ij) is equal to

$$w_{ij} = \frac{p^{-1}(y_i|\theta^{(j)})}{\sum_{j=1}^T p^{-1}(y_i|\theta^{(j)})}, \quad i = 1, 2, \dots, N; \quad j = 1, 2, \dots, T. \tag{10.27}$$

A third Bayesian approach, developed by Gianola and Schön (2016), is a leave-one-out cross-validation point estimator of validating mean squared error (a point mass) for Bayesian linear models of the form $E(y_i|b, x_i) = x_i' b$, where x_i' is an observed row vector of covariates and b is an unobserved column vector of regression parameters. The point estimator holds irrespective of the prior adopted for b . The expression for the validating mean squared error is given by (10.23) with $\hat{y}_i = x_i' \hat{E}(b|y_{-i})$, where

$$\hat{E}(b|y_{-i}) = \sum_{j=1}^T w_{ij} b^{(j)}, \quad i = 1, 2, \dots, N, \tag{10.28}$$

w_{ij} is given by (10.27) and $b^{(j)}$ is the j th draw ($j = 1, 2, \dots, T$) from a Markov chain that at convergence represents an approximate sample from the posterior distribution $[b|y]$ based on the complete data y . The computation of the point estimator of the leave-one-out validating mean squared error based on (10.28) is shown in the R-code below, `sumdg` and `mseloodg`.

An alternative to (10.28) is to calculate

$$\tilde{E}(b|y_{-i}) = \frac{1}{T} \sum_{j=1}^T b_{-i}^{(j)}, \tag{10.29}$$

where $b_{-i}^{(j)}$ is the j th McMC draw ($j = 1, 2, \dots, T$) obtained using the approximation to $[b|y_{-i}]$ from the weighted bootstrap. A point estimator of (10.23) is derived using $\hat{y}_i = x_i' \tilde{E}(b|y_{-i})$. The computation of the point estimator based on (10.29) is shown in the R-code below, `sumavrb` and `mselooavrb`.

These sampling-based approaches are sensitive to the distribution of the weights w_{ij} . The presence of extreme large values will lead to noisy estimates and some form of smoothing of the weights may be required. An entry to the literature on

the subject can be found in Vehtari and Lampinen (2002), Gelman et al. (2014a) and Vehtari et al. (2017). One way of examining the distribution of the weights is by plotting the sorted values in increasing order of a heuristic measure of effective sample size

$$m^{(i)} = \frac{1}{\sum_{j=1}^T w_{ij}^2}, \quad i = 1, 2, \dots, N, \quad (10.30)$$

(Vehtari and Lampinen 2002). Many small values will be indicative of instability of Bayesian leave-one-out cross-validation and the results should be supplemented with other cross-validation strategies. The latter should always be considered; the absence of small values of (10.30) does not guarantee that the method is working adequately.

The methods are easy to implement in an MCMC environment. Imagine that at cycle i , $i = 1, 2, \dots, rep$, of a Monte Carlo Markov chain, draws of the vector of the conditional variance are stored in a vector `ve` of length `rep`, draws of the vector $\theta = \mu + Zb$ are stored in a matrix `storeMean` of dimension $(rep \times nindiv)$, where `nindiv` is the length of the vector of data y , μ is an overall mean, Z is a matrix of observed covariates of dimension $(nindiv \times p)$ and b is a column vector with p regression parameters. The vector θ could also represent draws from probability functions $\Phi(\mu + Zb)$ in the case of binary data analysed with a hierarchical probit model. Then the kernel of the R-code to compute Bayesian leave-one-out validating mean squared errors for a linear model is as follows (the code processes, for each i , $i = 1, 2, \dots, nindiv$, all the `rep` replicates in one pass):

```
##### COMPUTE LOOCV BAYES #####
sumloo <- 0
sumdg <- 0
sumavrb <- 0
sumXb <- 0
for(i in 1:nindiv){
  w <- 1/dnorm(y.train[i],storeMean[,i],sqrt(ve))
  w <- w/sum(w)
  meff[i] <- 1/sum(w*w)
  index <- sample(1:rep,rep,replace=T,prob=w)
  yhatminusi <- rnorm(rep,storeMean[index,i],sqrt(ve[index]))
  sumloo <- sumloo + (y.train[i]-yhatminusi)^2
  expvalueXbminusi <- w%*%storeMean[,i]
  sumdg <- sumdg + (y.train[i]-expvalueXbminusi)^2
  sumavrb <- sumavrb + (y.train[i]-mean(storeMean[index,i]))^2
  # (y - Xb^hat)^2; Xb^hat: post mean
  sumXb <- sumXb + (y.train[i]-storeMean[index,i])^2
  # (y - Xb)^2; b~[b|y_i]
}
plot(sort(meff)) # effective weight sample size
mselooBayes <- sumloo/length(y.train)
# estimate based on y^hat = y^*~[y|y_i]
summary(mselooBayes)
quantile(mselooBayes,c(0.025,0.975))
```

```

mselooXb <- sumXb/length(y.train)
# estimate based on  $\hat{y} = xb$ ,  $b \sim [b|y_i]$ 
summary(mselooXb)
quantile(mselooXb,c(0.025,0.975))

mseloodg <- sumdg/(length(y.train))
# estimate of mseloodg (Gianola et al. 2016):
mseloodg
mselooavrb <- sumavrb/length(y.train)
# MCMC estimate of mseloodg (Gianola et al. 2016):
mselooavrb

```

Example: Bayesian Leave-one-out Cross-validation

This example illustrates applications of Bayesian leave-one-out cross-validation (BLOOCV) in a simplified genomic context. Two models are fitted to two sets of data consisting of 2500 phenotypic records y_i , $i = 1, 2, \dots, 2500$, and 5000 observed genetic marker covariates X_{ij} , $i = 1, 2, \dots, 2500$, $j = 1, 2, \dots, 5000$: a spike and slab model and a genomic model that is a form of ridge regression. Details of these two fitted models and of their McMC implementation are in Chap. 5, page 249 and Chap. 7, page 349. Each covariate X_{ij} is independently drawn from a binomial distribution $Bi(X_{ij}|n = 2, p = 0.5)$ and can take the value 0, 1 or 2 with probabilities $(1 - p)^2$, $2p(1 - p)$ and p^2 . The elements of the resulting (2500×5000) matrix of marker covariates are subsequently centred and scaled.

The true models that give rise to the two sets of data are as follows. For both models, observations are the result of the additive effects of a genetic component and of an environmental component. In the first model, the genetic component is due to the additive and equal effects of 25 QTL and in the second is due to 500 additive QTL. The QTL are randomly chosen from the 5000 covariates. The total additive genetic variance contributed by these QTL is the same in both scenarios and is equal to 10 squared units. The additive genetic effect of each of the 25 QTL in the first data set is equal to 0.14 phenotypic standard deviations of y . In the second data set, for the model of 500 QTL, this figure is equal to 0.03. The environmental variance in both sets of data is set equal to 30 squared units and the heritability of the quantitative trait is therefore equal to 0.25.

The data available to the analyst consist of two sets of 2500 observed phenotypic records and 5000 observed genetic marker covariates. In the first data set with 25 QTL (information unknown to the analyst), the spike and slab model is expected to lead to better predictions than the ridge regression. This is so provided that a significant proportion of the 25 QTL are detected and the redundant QTL are partly eliminated. This results in a decline in the variance of the predictor relative to the variance expected from the ridge regression model, that incorporates all the 5000 covariates in the construction of the predictor. A smaller variance of the predictor

Table 10.2 MCMC estimates of validating mean squared errors (10.23) (first four rows: estimated means and 95% posterior intervals of marginal posterior distributions in brackets) derived from three forms of Bayesian leave-one-out cross-validation (BLOOCV) using a spike and slab model and a ridge regression model (genomic model). Details of the two models are in Chaps. 5 and 7. The two models were fitted to two simulated data sets. Data set 1 is generated using 25 additive QTL and data set 2 using 500 QTL. LOO_{y^*} : predictor $\hat{y}_i$ drawn from the posterior predictive distribution (10.24); LOO_θ : predictor of the form $\hat{y}_i = x_i' b^*$, $b^* \sim [b|y]$; LOO : point estimate of the BLOOCV. The row has entries u/w ; u corresponds to the form $\hat{y}_i = x_i' \hat{E}(b|y_{-i})$ and $\hat{E}(b|y_{-i})$ is given by (10.28); w corresponds to the form $\hat{y}_i = x_i' \tilde{E}(b|y_{-i})$ and $\tilde{E}(b|y_{-i})$ is given by (10.29)

	Spike and slab		Ridge	
QTL	25	500	25	500
LOO_{y^*}	61.0	75.9	75.8	75.3
	(58.1;63.8)	(72.3;79.6)	(72.0;79.3)	(71.7;79.0)
LOO_θ	32.0	44.6	46.0	45.4
	(31.5;32.6)	(43.3;45.9)	(44.7;47.6)	(43.8;46.9)
LOO	30.5/30.5	38.0/38.0	37.9/37.9	37.7/37.6

leads to a smaller mean squared error, on average. On the other hand, with 500 QTL each emitting weak signals that are difficult to detect by the spike and slab model, both models are likely to perform similarly.

The performance of the three measures of Bayesian leave-one-out cross-validation computed using the spike and slab model and the ridge regression model, fitted to data set 1 generated assuming 25 QTL and to data set 2 generated assuming 500 QTL, is summarised in Table 10.2. The first row (LOO_{y^*}) corresponds to the BLOOCV based on the predictor drawn from the posterior predictive distribution (10.24). The corresponding validating mean squared error (MSE_v) incorporates the posterior uncertainty of the parameters of the Bayesian model θ and sampling variation of individual records. The second row (LOO_θ) corresponds to the BLOOCV where the predictor has the form $\hat{y}_i = x_i' b^*$, where b^* is an extraction from the posterior distribution $[b|y]$. The corresponding MSE_v incorporates posterior uncertainty of the parameters of the Bayesian model. The third row (LOO) shows the point estimates of BLOOCV with entries u/w , where u corresponds to the BLOOCV where the predictor has the form $\hat{y}_i = x_i' \hat{E}(b|y_{-i})$ and $\hat{E}(b|y_{-i})$ is given by (10.28), and w corresponds to the BLOOCV where the predictor has the form $\hat{y}_i = x_i' \tilde{E}(b|y_{-i})$ and $\tilde{E}(b|y_{-i})$ is given by (10.29). The first four rows of the table display the MCMC estimates of the posterior means and underneath, the 95% posterior intervals (in brackets) of LOO_{y^*} and of LOO_θ .

As anticipated, the spike and slab model outperforms the genomic model in the case of data set 1 with 25 QTL, and both models give similar results when fitted to data set 2 with 500 QTL. Within each column, the size of the MSE_v increases (moving from the bottom of the column) as more sources of variation are incorporated into the calculations.

The results indicate that the difference in predictive performance between the two fitted models is consistent within each of the three measures of BLOOCV. If there

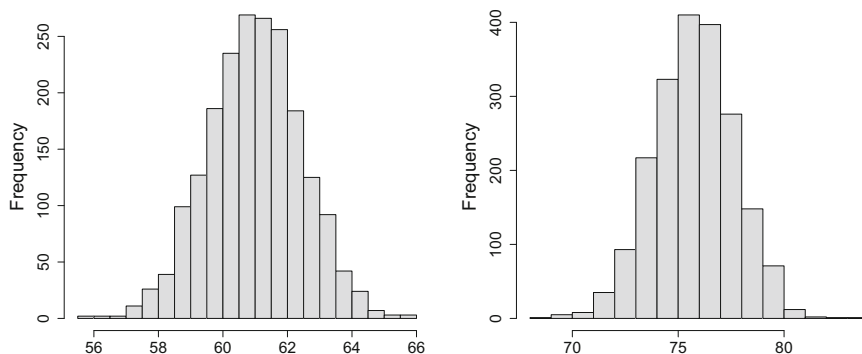


Fig. 10.2 Estimated marginal posterior distributions of validating mean squared errors obtained using the Bayesian leave-one-out cross-validation estimator LOO_{y^*} (see text for details) derived from a spike and slab model (left subfigure) and from the ridge regression model (right subfigure). Both models fitted to simulated data set 1, generated using 25 QTL

is interest in a preliminary comparison of model predictive performance, ignoring uncertainty, then the point predictor of BLOOCV is an option. If there is interest in learning how models will perform to predict an average value of y , given a particular value of the covariates, accounting for the fact that the predictor is constructed from a finite set of data, one would consider using LOO_{θ} . If there is interest in the ability of models to predict new individual records y_i , given a particular value of the covariates, the choice could fall on LOO_{y^*} .

As a further example of an output from an McMC implementation, Fig. 10.2 shows the estimated marginal posterior distributions of BLOOCV based on LOO_{y^*} when the spike and slab model (left subfigure) and the ridge regression model (right subfigure) are fitted to data set 1, with 25 QTL. The information provided by the uncertainty of the BLOOCV in the form of the estimates of marginal posterior distributions enriches inferences. In the case of data set 1, a simple eye-ball comparison of the non-overlapping supports of the estimated posterior distributions of BLOOCV in Fig. 10.2, or a glance at the corresponding posterior intervals of Table 10.2, leaves little doubt concerning which is the model with best likely predictive performance.

Figure 10.3 displays the results of a GWAS on data set 1 with 25 QTL (first row, first column) and on data set 2 with 500 QTL (second row, first column) and of the spike and slab model (second column). The y-axes represent the Bonferroni adjusted p -values for the GWAS analysis, and the posterior probability that a given genetic marker has an effect on the trait, for all 5000 markers, for the study based on the spike and slab model. The analysis of data set 1 reveals that GWAS detects 15 out of the 25 QTL, with 0 false positives whereas the spike and slab model detects all the 25 QTL, and includes extra 2 false positive results (given the arbitrarily chosen threshold of the posterior probability for declaring a result as “significant” equal to 0.5). On the other hand, with 500 QTL, GWAS fails to detect any QTL, and

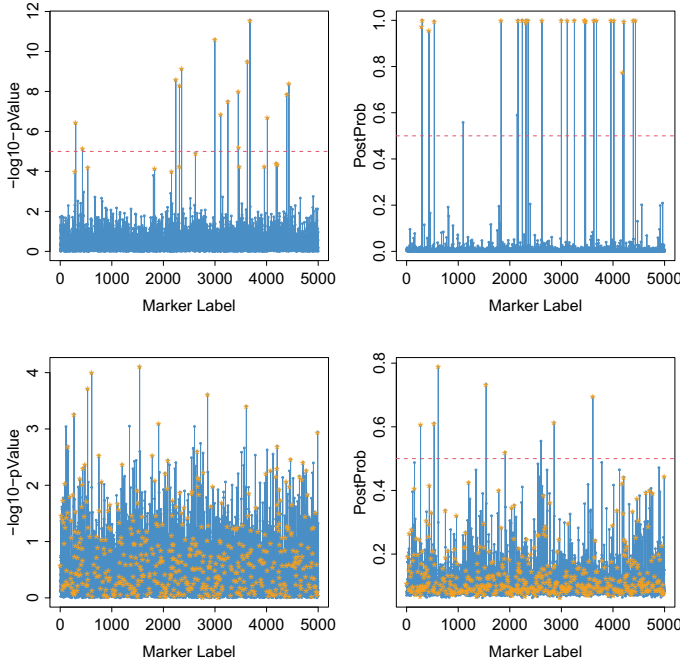


Fig. 10.3 Top row: Data set 1 with 25 QTL; bottom row: Data set 2 with 500 QTL. Top row, left subfigure: $-\log_{10} p$ -values for testing the null hypothesis that the gene has no effect on the trait using GWAS with a Bonferroni adjustment. The horizontal red line is the Bonferroni threshold corresponding to $-\log_{10}(0.05/5000)$. Top row, right subfigure: posterior probability that the gene has an effect on the trait obtained using a Bayesian, Markov chain Monte Carlo implementation of a spike and slab model, for each of the 5000 genetic marker loci. Horizontal red line is the arbitrarily chosen threshold corresponding to a posterior probability of 0.5. Bottom row: similar description as top row, except that the models are fitted to data set 2, with 500 QTL. Orange stars represent the 25 QTL in the top subfigures and the 500 QTL in the bottom subfigures

the spike and slab model detects only 8 QTL including one false positive (given the arbitrarily chosen threshold of the posterior probability equal to 0.5). There is a large proportion of the 500 QTL that are assigned similar posterior probabilities as the non-QTL. Therefore the spike and slab model behaves similarly to the ridge regression; the latter allocates similar values to all the 5000 marker loci. This results in approximately equal estimated predictive performance for both fitted models.

The two Bayesian leave-one-out mean squared errors in Table 10.2, LOO_{y^*} and LOO_{θ} , account for the same sources of variation as the standard cross-validation estimators (10.16) and (10.15), respectively. Estimator LOO and estimator $MSE_{v,1}$ defined in (10.14) are both point masses ignoring uncertainty. Table 10.3 shows Monte Carlo estimates of posterior means and of 95% posterior intervals for the 3 validating mean squared errors defined in Eqs. (10.14), (10.15) and (10.16), calculated using standard cross-validation (here the quadratic forms are divided by 2500). This was achieved using a holdout set of validating data consisting of

Table 10.3 McMC estimates of validating mean squared errors (10.14), (10.15) and (10.16); (first four rows: estimated means and 95% posterior intervals of marginal posterior distributions in brackets) derived from a spike and slab model and a ridge regression model (genomic model). Details of the two models are in Chaps. 5 and 7. The two models were fitted to two simulated data sets. Data set 1 is generated using 25 additive QTL and data set 2 using 500 QTL

QTL	Spike and slab		Ridge	
	25	500	25	500
MSE _{v,3}	60.8 (57.4;64.3)	74.0 (69.8;78.5)	74.0 (69.6;78.4)	73.0 (68.9;77.4)
MSE _{v,2}	31.9 (30.9;33.1)	42.6 (41.1;44.4)	44.2 (42.3;46.2)	42.8 (40.9;44.8)
MSE _{v,1}	30.4	37.1	37.7	35.6

2500 phenotypic records. These records were not used in the computation of the Bayesian leave-one-out cross-validation. A comparison with the estimates derived using Bayesian leave-one-out cross-validation in Table 10.2 reveals that, with the exception of the first column corresponding to the spike and slab model fitted to the data generated with 25 QTL, the estimates based on the Bayesian leave-one-out cross-validation tend to be a little larger than the respective estimates based on the standard cross-validation.

The R-code CODE1003 fits the spike and slab model and computes BLOOCV. The ridge regression model is fitted using R-code CODE1313; the extra lines required to compute BLOOCV are not part of this second code. These two R-codes can be found in the folder Codes at <https://github.com/SorensenS/SLGDS>.

10.6 Model Checking Using Posterior Predictive Distributions

Predicted data y_{pred} drawn from posterior predictive distributions were used to compute validating mean squared errors, by comparing the average squared difference between predicted (or simulated) data and observed validating data y_v . The MSE is a function of the parameters of the model θ (conditional on training data y_t); its distribution reflects the propagated posterior uncertainty of these parameters and of the sampling uncertainty of the predicted data.

Predicted data can also be used for inferences by checking whether a particular model properly accounts for aspects of the data that may be of scientific relevance. Let M denote the model that is to be questioned and that is presumed to have generated observed data y . The basic idea is to compare a function of y to the same function of predicted (or replicated) data generated under model M . These functions are constructed to address the specific feature of the model under enquiry; they depend on the observed data and often also on the parameters of the model. In the literature they are known as *discrepancy measures* (Gelman et al. 1995) and labelled

$T(y, \theta_M)$, a function of data and parameters, where θ_M is the parameter vector under model M . The discrepancy measure $T(y, \theta_M)$ based on observed data is compared to $T(y_{pred}, \theta_M)$, based on predicted data y_{pred} generated under model M . If zero is an atypical value in the posterior predictive distribution of $T(y, \theta_M) - T(y_{pred}, \theta_M)$, then model M is making predictions that do not fit that aspect of the data described by the discrepancy measure. Model checking performed in this way is a diagnostic tool for assessing the usefulness of a model for a specific purpose rather than for studying its global fit. Key literature on the subject is Rubin (1984), Gelman et al. (1995) and Gelman et al. (1996).

An Example with a Genetically Structured Heterogeneous Variance Model

The use of posterior predictive model checking is explained using an example adapted from Sorensen and Waagepetersen (2003). Imagine that unknown to the experimenter, observed data had been generated by the following model. At a first stage, the sampling model is Gaussian of the form

$$y_{ij} | \mu, f_i, \sigma_i^2 \sim N(\mu + f_i, \sigma_i^2), \quad i = 1, \dots, n_f; \quad j = 1, \dots, n. \quad (10.31)$$

In (10.31) y_{ij} is a record, μ is the mean, f_i is a random family effect and there is a structured residual term that gives rise to a different residual variance σ_i^2 for each of the n_f families; there are n offspring per family. The pattern of data generated by this model is quite common in animal breeding, where a family could represent a cohort of half-sibs produced by sires that mate each to n randomly chosen females, that in turn have each one offspring.

The model for the residual variances takes the form

$$\sigma_i^2 = \exp(\mu^* + f_i^*), \quad (10.32)$$

where μ^* is a parameter in $\mathbb{R}$ and f_i^* is a random family effect acting on the variance. When the elements of f^* are all zero or alternatively, all of equal magnitude (in which case the effects can be absorbed in μ^*), the residual variance reduces to the usual homogeneous variance $\sigma^2 = \exp(\mu^*)$.

The random variables $(f_i, f_i^*), i = 1, \dots, n_f$, are taken to be *iid* draws from

$$(f_i, f_i^* | \sigma_f^2, \sigma_{f^*}^2, \rho) \sim N \left[\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \sigma_f^2 & \rho \sigma_f \sigma_{f^*} \\ \rho \sigma_f \sigma_{f^*} & \sigma_{f^*}^2 \end{pmatrix} \right], \quad (10.33)$$

where σ_f^2 is the variance between half-sibs acting at the level of the mean, σ_{f*}^2 is the variance between half-sibs acting at the level of the variance and ρ is the coefficient of correlation.

The model defined by (10.31), (10.32) and (10.33) postulates that the residual variance has a genetic component because it varies across families. In the half-sib model assumed, variation between half-sib families is entirely genetic and explains 25% of the additive genetic variance.

This model induces a relationship between mean and variance through (10.33). Write

$$\begin{aligned} f_i^* &= E(f_i^* | f_i) + (f_i^* - E(f_i^* | f_i)) \\ &= \rho \frac{\sigma_{f*}}{\sigma_f} f_i + u \end{aligned} \quad (10.34)$$

where u is independent of f_i . Then

$$u | \sigma_{f*}, \rho \sim N\left(0, (1 - \rho^2) \sigma_{f*}^2\right). \quad (10.35)$$

Therefore the distribution $[f_i, f_i^* | \sigma_f^2, \sigma_{f*}^2, \rho]$ is the same as the distribution

$$\left[f_i, \left(\rho \frac{\sigma_{f*}}{\sigma_f} f_i + u \right) | \sigma_f^2, \sigma_{f*}^2, \rho \right]. \quad (10.36)$$

When $\rho = 1$ the relationship between mean and variance is deterministic; when $\rho = 0$ the variance is genetically determined and homogeneous, independent of the mean. Otherwise, the relationship between mean and variance is controlled by the sign and size of ρ .

The focus of enquiry in this example is to study whether the data provide evidence of a relationship between residual variance and family mean, supporting cases one or three.

Taking expectations of (10.31) over the distribution $[f_i, f_i^* | \sigma_f^2, \sigma_{f*}^2, \rho]$, the marginal distribution of the data is

$$y_{ij} | \mu, \mu^*, \sigma_f^2, \sigma_{f*}^2 \sim N\left(\mu, \sigma_f^2 + \exp\left[\mu^* + \frac{\sigma_{f*}^2}{2}\right]\right), \quad (10.37)$$

using the fact that if $x \sim N(\mu, \sigma^2)$, $\exp(x)$ is lognormally distributed with mean $E[\exp(x)] = \exp\left(\mu + \frac{\sigma^2}{2}\right)$.

Consider regressing $(\mu^* + f_i^*)$, the log-residual variance of family i , on $(\mu + f_i)$, the mean of family i . The regression coefficient is

$$\beta = \frac{\text{Cov}_{f_i, f_i^*}(\mu^* + f_i^*, \mu + f_i)}{\text{Var}_{f_i}(f_i)} = \rho \frac{\sigma_{f*}}{\sigma_f}. \quad (10.38)$$

The choice of discrepancy measure described below aimed at establishing an association between family effects and within family variance is motivated by (10.38).

Constructing Discrepancy Measures

The mechanism that generated a particular set of data is seldom known. The closest one may come to knowing this mechanism is when data originate from a carefully designed and controlled experiment.

In the specific situation considered here, we imagine that using field records y , there is interest in investigating a possible genetic source of residual variation inducing a relationship between within family variance and family effect. Before committing resources to the development of the software needed to fit the model defined by (10.31), (10.32) and (10.33), one may start by fitting a model labelled M , that assumes homogeneity of residual variance. In a second stage one can define a discrepancy measure designed to reveal whether the lack of association between family mean and within family variance implied by model M holds. Given model M , replicated or predicted data y_{pred} are generated, $T(y_{pred}, \theta_M)$ is constructed and compared to $T(y, \theta_M)$. If zero is an atypical value in the posterior predictive distribution of $T(y, \theta_M) - T(y_{pred}, \theta_M)$, then there is justification to extend model M , because model's M assumption of lack of association between family mean and within family variance is not supported.

The starting point is to fit model M of homogeneous variance to data y ,

$$y_{ij} | \mu, f_i, \sigma^2, M \sim N(\mu + f_i, \sigma^2), \quad i = 1, \dots, n_f; \quad j = 1, \dots, n, \quad (10.39a)$$

$$f_i | \sigma_f^2 \sim N(0, \sigma_f^2). \quad (10.39b)$$

The model is implemented using McMC; one may assign improper prior distributions to $(\mu, \sigma_f^2, \sigma^2)$.

The discrepancy measure to check for an association between family mean and within family variance in the observed data y is constructed as follows:

1. Using observed data first compute for each family

$$S_i(y_{ij}, \theta_i) = \frac{1}{n} \sum_{j=1}^n (y_{ij} - \mu - f_i)^2, \quad \theta_i = (\mu, f_i). \quad (10.40)$$

The quantity S_i is the average squared residual for family i .

2. Secondly, fit a simple linear regression of $\log S_i(y_{ij}, \theta_i)$ on $f_i, i = 1, \dots, n_f$, leading to the discrepancy measure

$$T(y, \theta) = \beta(y, \theta), \quad (10.41)$$

where β is the regression coefficient which is a function of data y and θ .

3. Repeat using predicted data generated using the homogeneous variance model

$$[y_{pred,ij} | \mu, f_i, \sigma^2] \sim N(\mu + f_i, \sigma^2).$$

Regress $\log S_i(y_{pred,ij}, \theta_i)$ on $f_i, i = 1, \dots, n_f$, leading to the discrepancy measure

$$T(y_{pred}, \theta) = \beta(y_{pred}, \theta), \quad (10.42)$$

the regression coefficient based on predicted data.

4. Compute

$$T(y, \theta) - T(y_{pred}, \theta). \quad (10.43)$$

These steps are carried out using MCMC. At round t the algorithm outputs draws $\theta^{[t]}$ and $y_{pred}^{[t]}$ from $[\theta, y_{pred} | y, M]$ based on model M of homogeneous variance. These draws are used to construct a Monte Carlo estimate of the posterior distribution of the discrepancy measure (10.43). If zero is excluded from the posterior distribution or is very unlikely, then the conjecture of an association between family mean and within family variance in the observed data is supported.

Generating Data Based on the Genetic Heterogeneous Variance Model

Data available for analysis are generated using the model defined in (10.31), (10.32) and (10.33). This model is not known to the researcher.

To create the data the following set of parameters are used:

1. Number of half-sib families $n_f = 400$; number of records per family $n = 30$, the same for all families
2. $\mu = 10$; $\mu^* = 3$; $\sigma_f^2 = 5$; $\sigma_{f^*}^2 = 1.8$; $\rho = 0.15$. Based on these input parameters the marginal mean and variance of a datum is expected to be 10 and $\exp(3 + 0.9) = 49.4$, respectively.

The R-code used to generate the data is shown below.

```

# HETVARMODEL MODEL CHECKING
rm(list=ls()) # Clear the workspace
require(graphics)
# GENERATE CORRELATED (FULL-SIB/HALF-SIB) DATA

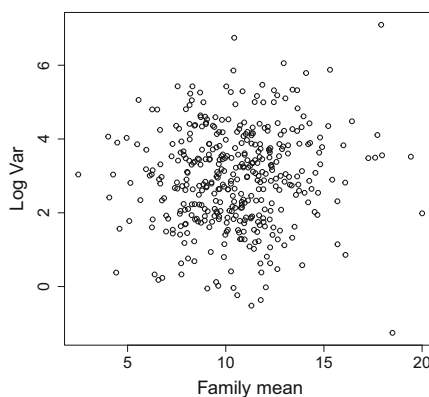
# INITIALISE PARAMETERS
mu<-10 # MEAN
mu_v <- 3 # MEAN VAR

#VARIANCE BETWEEN FULL-SIBS/HALF-SIBS
vfs <- 5 # mimicking half-sibs
vfs_v <- 1.8 # VARIANCE BETWEEN FULL-SIBS/HALF-SIBS
#           AT THE LEVEL OF THE VAR
rho <- 0.15 # CORRELATION BETWEEN FAMILY EFFECTS (fs,fs_v)
###           AT THE LEVEL OF MEAN AND VARIANCE
# RESIDUAL VARIANCE (NOT USED)
ves<-exp(log(50))
nf<-400 # NUMBER OF FULL-SIB/HALF-SIB FAMILIES
n<-30 #FULL-SIB/HALF-SIB FAMILY SIZE
N<-nf*n
y<-matrix(data=0,nrow=nf*n,ncol=1)
# z IS COLUMN MATRIX WITH FAMILY ID (ID=1,..,nfs)
z<-matrix(data=0,nrow=nf*n,ncol=1)
# GENERATE nf FULL-SIB/HALF-SIB EFFECTS f
fs<-rnorm(nf,mean=0,sd=sqrt(vfs))
#####
## GENERATING A FULL-SIB/HALF-SIB STRUCTURE
## VIA z
z <- rep(1:nf,each=n)
#####
u <- rnorm(nf,0,sqrt((1-rho^2)*vfs_v))
## SIB EFFECTS AT THE LEVEL OF VARIANCE
fs_v <- (rho*sqrt(vfs_v)/sqrt(vfs))*fs + u
# GENERATE nf*n RESIDUAL EFFECTS
es<-rnorm(nf*n,mean=0,sd=sqrt(exp(mu_v + fs_v[z])))
mean(es)
var(es)
y <- mu + fs[z] + es
f <- as.factor(z)
d <- data.frame(y,f)

#####
#### USE THE FUNCTION AGGREGATE TO COMPUTE MEANS AND
#### VARIANCES FOR EACH FAMILY
agm <- aggregate(y~f,d,FUN=function(y){mean(y)})
agv <- aggregate(y~f,d,FUN=function(y){var(y)})

```

Fig. 10.4 The relationship between the logarithm of the variance within families and the family means induced by the heterogeneous variance model



Detecting a Relationship Between Within Family Variance and Family Mean Using Observed Data

What does a simple analysis based on observed data reveal? Figure 10.4 displays the relationship between the logarithm of the estimated within family variance of the 400 families and the family means based on 30 records per family using the observed (simulated) data (generated under the heterogeneous variance model). A simple least squares regression of log-variance within families on family means yields an estimate of the regression coefficient equal to 0.038 with a standard error equal to 0.025. This analysis based on raw averages discloses considerable heterogeneity of residual variance, but provides a very tenuous and inconclusive relationship between mean and variance, despite a reasonably sized and well structured data set. The question is whether inferences of association between mean and variance are sharper using posterior predictive model checking.

Detecting a Relationship Between Within Family Variance and Family Mean Using Discrepancy Measures

The data set generated under the genetically heterogeneous variance model defined in (10.31), (10.32) and (10.33) is analysed using a homogeneous variance model. A Gibbs sampling implementation of the homogeneous variance model is described in Chap. 5, page 273. In the current application the code generates 5000 draws from the MCMC estimates of the posterior distribution of discrepancies (10.42) and (10.43).

The Monte Carlo estimates of the marginal posterior distributions of the two discrepancy measures (10.42) and (10.43) are shown in Fig. 10.5. The left panel shows

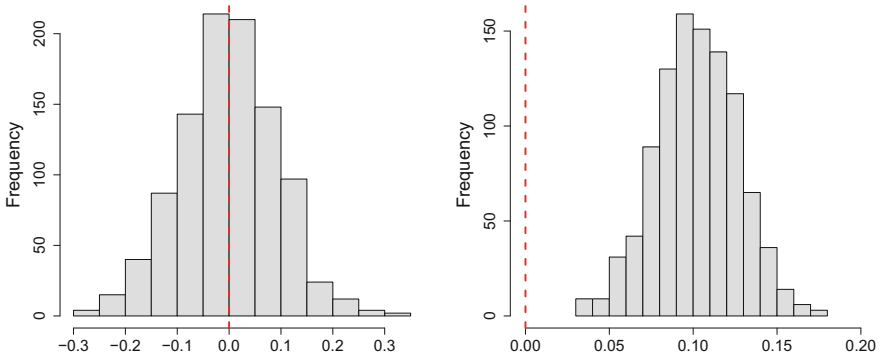


Fig. 10.5 Left: Monte Carlo estimate of posterior distribution of discrepancy measure (10.42) based on predicted data y_{pred} generated using a model of homogeneous variance across families (the reference distribution); Right: Monte Carlo estimate of posterior distribution of discrepancy measure (10.43). The vertical red lines at zero are indicative of a lack of association between family mean and within family variance

that the Monte Carlo estimate of the reference distribution, based on predicted data generated with the homogeneous variance model (discrepancy measure (10.42)) is centred at zero. This is evidence of no signs of association between family mean and within family variance in the replicated data y_{pred} , as expected. The panel on the right displays the Monte Carlo estimate of the posterior distribution of discrepancy measure (10.43). A value of zero is very unlikely under this distribution, supporting the conjecture of a relationship between family mean and within family variance in the observed data y .

This posterior prediction-based analysis, given the model, reveals an association between within family variance and family effects in the observed data, that could not be revealed by the analysis based on simple means displayed in Fig. 10.4. Does this provide convincing evidence for a genetically structured residual variance in the data? No; it does not. One can only conclude that the results are not in conflict with a model posing genetic control of residual variance and justify further experimentation.

I conclude with a remark that is particularly relevant in an inferential framework. Modern computing tools allow fitting complex models to investigate subtle structures in large observational data sets that may be scientifically relevant. The heterogeneous variance model can serve as an illustration. The coefficient of skewness generated by this model is directly proportional to ρ , the correlation coefficient between family effects acting on mean and variance (Ros et al. 2004). Therefore, if the observed data have a skewed distribution in either direction not necessarily due to genetically structured variance heterogeneity, a model that postulates variance heterogeneity will result in a better fit than the simple alternative assuming homogeneous variance. The analysis would even output an estimate of the non-existent correlation ρ , leading to spurious inference. This makes it strikingly clear that an attempt to understand the state of nature via a statistical analysis of

data, particularly observational data, must be regarded only as a first step until more fundamental knowledge becomes available. Model checking using a well designed discrepancy measure is a contribution to this process.

Identifiability of parameters is also an important consideration. In highly complex models, there is always the pitfall that parameters may be unidentified or very weakly identified and it may not always be possible to check lack of identifiability. Often for convenience, one chooses prior distributions of parameters that are improper (as practised repeatedly in this book!). Such prior assumptions can lead to improper posterior distributions that may go undetected in an MCMC environment. This can lead to misleading inferences. An interesting example, initially taken from Carlin and Louis (1996), can be found on page 543 of Sorensen and Gianola (2002) and a useful reference is Natarajan and Kass (2000). Use of proper prior distributions will dispose of the potential problem of posterior impropriety and unidentifiability and, if chosen judiciously, may lead to Bayesian learning. However, the problem of the influence of prior information on posterior inference still needs to be addressed.

It is important to learn as much as possible about the model by experimenting with it before launching a full analysis using modern computational tools.

Chapter 11

Nonparametric Methods: A Selected Overview



Throughout this book a phrase like “assume the data have been generated by the following probability model” has been abundantly used. Indeed, the standard parametric assumption is that observed data represent one realisation from some given probability model and the goal can be to infer the parameters of the model. Alternatively and from a classical frequentist setting, conditionally on estimated parameters, the goal may be to predict future observations. For quantitative responses the parametric model can be written as

$$y_i = m(x_i, \theta) + e_i, \quad i = 1, 2, \dots, n, \quad (11.1)$$

where m is some function that relates observations y_i (outputs) to observed covariates x_i (inputs, also referred to as *features*) constituting the data set $z_i \stackrel{iid}{=} (y_i, x_i)$, $i = 1, 2, \dots, n$. The function $m(x_i, \theta) = E(y_i|x_i, \theta)$, a conditional expectation, will be referred to as the *regression function* or the *prediction function*: its estimate uses as input the feature x and produces an output $\hat{y}$. The residual terms e_i are *iid* from some distribution and θ is a vector of parameters. The form of m is often simple, as in linear regression and when the model is approximately correct, inferences about θ are straightforward and efficient. If $\hat{\theta}$ is an estimate of θ , a predicted value given x_{0i} is $\hat{y}_{0i} = m(x_{0i}, \hat{\theta})$. For example in linear regression, $m(x_i, b) = x_i' b$, $\hat{y}_{0i} = x_{0i}' \hat{b}$, where $\hat{b}$ is the estimated value of b . For data arising from binary trials, modelling takes place at the level of $\Pr(y_i = 1|x_i, \theta)$ as in logistic regression.

Nonparametric models on the other hand, do not commit to a specific probability model nor to the form of m and instead regard m as an algorithm that for future x_0 leads to a good predictor of y_0 . The prediction algorithm is constructed or trained in the *training set* of observations and its predictive ability is studied in the *validating or testing set*. Nonparametric theory focuses on the properties of the algorithms, their predictive power and convergence properties if they are iterative and on factors that affect their accuracy. There is a distinction between *supervised*

learning problems, where the objective is typically to make a prediction (or a classification in the case of discrete data) and *unsupervised learning*, where the aim is to explore how the data fall into different clusters. The latter is not dealt with here.

Nonparametric methodology offers a great deal of flexibility and can handle model complexities such as a large number of feature variables, complex interactions involving the feature variables and unknown nonlinear relationships between the feature variables and the response. Nonparametric methods have tuning parameters that control their adaptation to the data and therefore the degree of overfitting via the bias-variance trade-off. These tuning parameters can be set by the user or can be estimated from the data.

Prediction algorithms are largely the product of research in fields outside of statistics, especially computer science and a vast number have been developed. This chapter provides an overview and examples including nonparametric regression methods, kernel methods using basis expansions, a special kind of neural networks known as multilayer perceptrons, decision trees and random forests.

11.1 Local Kernel Smoothing

The first part of this section provides a description of some selected traditional nonparametric methods that possess great flexibility for the estimation of the regression function. The objective is to predict the regression function m at some target value x . The examples and motivation in this first part assume that the response y is a function of a scalar predictor x .

All the nonparametric models described here are of the form $\hat{y} = Hy$, where $\hat{y}$ is the prediction and H is a hat matrix (i.e., page 291), not a function of y .

The Binned Estimator

A convenient starting point is nonparametric estimation of the conditional expectation of a random variable Y , $E(Y|X = x) = m(x)$ using a random sample (training data) $(y_1, x_1), (y_2, x_2), \dots, (y_n, x_n)$, without assuming a specific form for m . The estimate at the point $X = x$ is the mean of the observations y for which $X_i = x$, $i = 1, 2, \dots$. An approximate result is to average the observations y associated with X 's close to x , such that $|x_i - x| \leq h$ for a small $h > 0$ called the *bandwidth*. Essentially, the X axis is divided into *bins* and the estimate for the bin is the average of the y 's belonging to the bin. The value of the bandwidth determines the width of the bins: large size with more observations produces a smoother fit as a function

of X . When $x \pm h$ defines the bin, the estimator of the regression function can be written

$$\begin{aligned}\hat{m}(x) &= \frac{\sum_{i=1}^n I(|x_i - x| \leq h) y_i}{\sum_{i=1}^n I(|x_i - x| \leq h)} \\ &= \sum_{i=1}^n w_i(x) y_i, \quad w_i(x) = \frac{I(|x_i - x| \leq h)}{\sum_{i=1}^n I(|x_i - x| \leq h)},\end{aligned}\quad (11.2)$$

where $I(u)$ is the indicator function equal to 1 if the argument holds and 0 otherwise. For a given x , the sum in the denominator is the number of observations that fall in the corresponding *bin*. Since $\sum_{i=1}^n w_i(x) = 1$, (11.2) is a weighted average of the training observations y_i and the i th weight determines the contribution of y_i to the estimator $\hat{m}(x)$.

The $n \times 1$ column vector of fitted values is

$$\hat{m} = (\hat{m}(x_1), \hat{m}(x_2), \dots, \hat{m}(x_n))',$$

and it follows that

$$\hat{m} = Wy, \quad (11.3)$$

where $y = (y_1, y_2, \dots, y_n)'$ is the training data and W is the $n \times n$ matrix whose i th row contains the weights given to y_i to estimate $\hat{m}(x_i)$. Expression (11.3) highlights that the estimate is linear in the data. The i th row of W is

$$w(x_i)' = \left[\frac{I(|x_1 - x_i| \leq h)}{\sum_{j=1}^n I(|x_j - x_i| \leq h)} \quad \frac{I(|x_2 - x_i| \leq h)}{\sum_{j=1}^n I(|x_j - x_i| \leq h)} \quad \cdots \quad \frac{I(|x_n - x_i| \leq h)}{\sum_{j=1}^n I(|x_j - x_i| \leq h)} \right]. \quad (11.4)$$

The matrix W is the *hat matrix* encountered in (6.30), page 291.

To illustrate suppose that $n = 7$, $x_i = i/7$, $i = 1, 2, \dots, 7$ and $h = 1/7$. Then for $i = 1$ and $x = 1/7$, the denominator in (11.2) is 2, the first row of W is

$$\left[\frac{1}{2} \quad \frac{1}{2} \quad 0 \quad 0 \quad 0 \quad 0 \right],$$

and $\hat{m}(x_1 = 1/7) = \frac{1}{2}y_1 + \frac{1}{2}y_2$. For $i = 2$, $x = 2/7$, the denominator in (11.2) is 3, the second row of W is

$$\left[\frac{1}{3} \quad \frac{1}{3} \quad \frac{1}{3} \quad 0 \quad 0 \quad 0 \right],$$

and $\hat{m}(x_2 = 2/7) = \frac{1}{3}y_1 + \frac{1}{3}y_2 + \frac{1}{3}y_3$, and so on. The hat matrix for this example is

$$W = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 & 0 & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{2} & \frac{1}{2} \end{bmatrix}. \quad (11.5)$$

If instead $h = 2/7$ is used, the hat matrix is

$$W = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 & 0 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & 0 & 0 \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & 0 & 0 \\ 0 & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & 0 \\ 0 & 0 & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ 0 & 0 & 0 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ 0 & 0 & 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}. \quad (11.6)$$

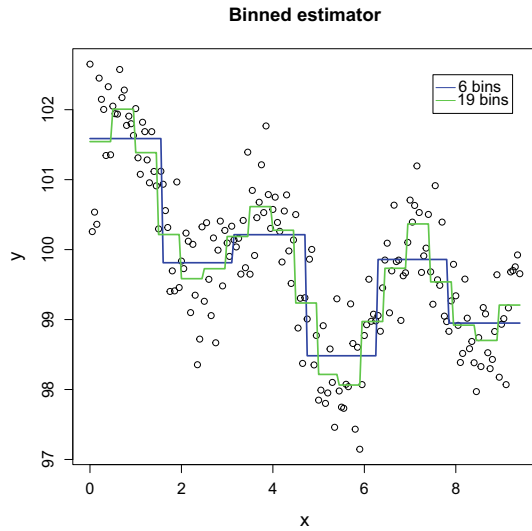
Note that as h increases, more observations enter into the bin and smoothing is said to be more “global”. Conversely as h decreases, the neighbourhood size is smaller and the smoothing is more “local”.

The R-code below displays an example of a binned estimator. Figure 11.1 shows that as the number of bins increases from 6 (blue line) to 19 (green line) the fit becomes more jagged leading to overfitting. The dots represent the response.

```
# CODE1101
rm(list=ls()) # CLEAR WORKSPACE
set.seed(195021)
x<-seq(from=0, to=2*pi,by=0.2)
f0<-function(x){ 100*sin(2*x)+cos(x/2) }
R2<-2/3
y<-f0(x)+rnorm(n=length(x),sd=sqrt(var(f0(x))*(1-R2)/R2))
z1 <- cut(x,breaks=seq(from=min(x),to=max(x+.01),
                        length=7),right=F)
f1 <- lm(y~z1)
z2 <- cut(x,breaks=seq(from=min(x),to=max(x+.01),
                        length=20),right=F)
f2 <- lm(y~z2)

plot(y~x,main='Binned estimator')
lines(x=x,y=predict(f1),col='blue',lwd=2)
lines(x=x,y=predict(f2),col='green',lwd=2)
legend(5, 103, legend=c("6 bins", "19 bins"),
      col=c("blue", "green"), lty=1:1, cex=0.8)
```

Fig. 11.1 An example of a binned estimator illustrating how the smoothness of the fit is controlled by the number of bins. The dots represent the response



Other Kernel Smoothing Methods

Estimator (11.2) is a step function, discontinuous at each $x = x_i$, due to the weights given by the indicator functions. If the indicator function in (11.2) is replaced by a *kernel function*, $K(\cdot)$, then the estimator can be written as

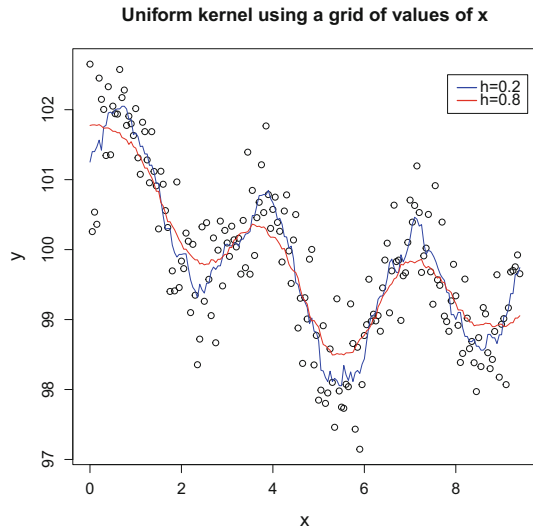
$$\hat{m}(x) = \frac{\sum_{i=1}^n K\left(\frac{x_i - x}{h}\right) y_i}{\sum_{i=1}^n K\left(\frac{x_i - x}{h}\right)}. \quad (11.7)$$

This is the form of a *kernel regression* estimator, known as the Nadaraya-Watson estimator that does not necessarily eliminate the discontinuity unless the kernel function used is continuous. For example, the uniform density $K(u) = 1/2$ for $u = (x_i - x)/h$ in the support $[-1, 1]$, is a *kernel function* that is a proper probability density function symmetric at zero. The uniform kernel in particular, is an alternative representation of the binned estimator (11.2) and suffers from the same discontinuity.

A kernel function satisfies $0 \leq K(u) < \infty$, $\int_{-\infty}^{\infty} K(u) du = 1$, or equal to a constant, $K(u) = K(-u)$, $\int_{-\infty}^{\infty} u K(u) du = 0$ and $\sigma_K^2 = \int_{-\infty}^{\infty} u^2 K(u) du < \infty$. A normalised kernel function satisfies $\int_{-\infty}^{\infty} u^2 K(u) du = 1$.

The size of h in (11.7) plays a central role in the degree of smoothness of $\hat{m}(x)$. As h approaches 0, x_i approaches x , $\hat{m}(x_i)$ approaches y_i and the fitted values go through the observed data. This is an extreme case of overfitting. When h is very large the computation of $\hat{m}(x)$ involves all the x'_i 's and the estimator $\hat{m}(x)$

Fig. 11.2 An example of a binned estimator generated using a uniform kernel on a grid of values of x . The degree of smoothness is controlled by the smoothing parameter. The dots represent the response



approaches the mean of the observations for all i , an extreme case of underfitting. The R-code below illustrates this using a uniform kernel on a grid of values of x and the result is displayed in Fig. 11.2.

```
# CODE1102
rm(list=ls()) # CLEAR WORKSPACE
set.seed(195021)
x<-seq(from=0, to=2*pi,by=0.2)
f0<-function(x){ 100+sin(2*x)+cos(x/2) }
R2<-2/3
y<-f0(x)+rnorm(n=length(x),sd=sqrt(var(f0(x))*(1-R2)/R2))
d <- as.matrix(dist(x))
h <- 0.2
d2 <- ifelse(d <= h,1,0)
div <- apply(d2,1,sum)
rx <- d2%*%y/div

h <- 0.8
d2 <- ifelse(d <= h,1,0)
div <- apply(d2,1,sum)
rx <- d2%*%y/div
```

As indicated, the construction of (11.2), either as expressed in terms of indicator functions or in terms of the uniform kernel, places equal weights to those points within a distance h and those outside h are ignored. This generates a discontinuity when the x 's are grouped in bins, although a smoother graph results if a grid of values of the x 's is chosen instead.

One may wish to place more weights to those observations that are close to x . There are a number of continuous kernel functions that can be used with (11.7) that address these issues and a common one is the Gaussian kernel

$$K\left(\frac{x_i - x}{h}\right) = (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{(x_i - x)^2}{2h^2}\right).$$

The constant term $(2\pi)^{-\frac{1}{2}}$ is typically omitted and the term in the denominator of the exponential function can be replaced by h . The fitted value at $x_1 = 1/7$ is given by

$$\hat{m}(x_1 = 1/7) = \frac{\sum_{i=1}^7 \exp\left[-\frac{(1/7-x_i)^2}{2h^2}\right] y_i}{\sum_{i=1}^7 \exp\left[-\frac{(1/7-x_i)^2}{2h^2}\right]} = \sum_{i=1}^7 w_i(x) y_i \quad (11.8)$$

that has the form given by (11.7). A prediction for a new scalar value of the covariate $x = x^*$ is

$$\hat{m}(x = x^*) = \frac{\sum_{i=1}^7 \exp\left[-\frac{(x^*-x_i)^2}{2h^2}\right] y_i}{\sum_{i=1}^7 \exp\left[-\frac{(x^*-x_i)^2}{2h^2}\right]} = \sum_{i=1}^7 w_i(x^*) y_i, \quad (11.9)$$

where the weight function is

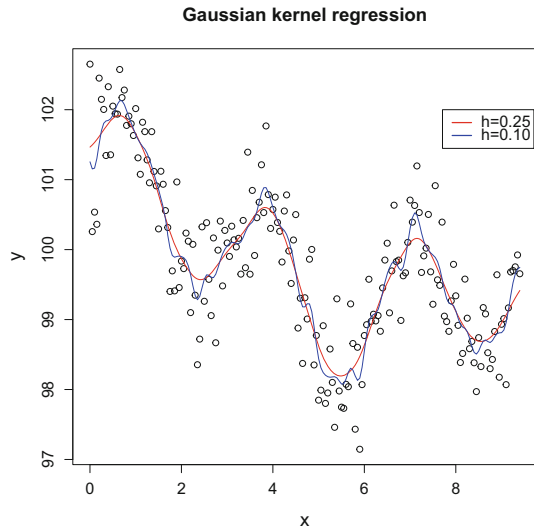
$$w_i(x^*) = \frac{\exp\left[-\frac{(x^*-x_i)^2}{2h^2}\right]}{\sum_{i=1}^7 \exp\left[-\frac{(x^*-x_i)^2}{2h^2}\right]}. \quad (11.10)$$

An example of the implementation of a Gaussian kernel applied to the same data used in Figs. 11.1 and 11.2 is shown in the R-code below that produces Fig. 11.3 as output. Compared to the uniform kernel, the Gaussian kernel generates a smoother fit. As shown in the figure, a higher degree of smoothness is associated with larger values of the bandwidth h .

```
# CODE1103
# GAUSSIAN KERNEL REGRESSION
rm(list=ls()) # CLEAR WORKSPACE
set.seed(195021)
# GENERATE DATA
x<-seq(from=0, to=2*pi, by=0.2)
#x<-seq(from=0, to=2*pi, length.out=33)

f0<-function(x){ 100*sin(2*x)+cos(x/2) }
R2<-2/3
y<-f0(x)+rnorm(n=length(x), sd=sqrt(var(f0(x))*(1-R2)/R2))
```


Fig. 11.3 The Gaussian kernel estimator computed with the R-code CODE1103, using two values for the bandwidth parameter



```
# CHOOSE h
h <- 0.25
# CONSTRUCT DISTANCE MATRIX AND GAUSSIAN KERNEL
d <- as.matrix(dist(x))^2
Kh25 <- exp(-(1/(2*h^2))*d)
sc25 <- apply(Kh25,1,sum)
mhgaus25 <- Kh25%*%y/sc25
plot(y~x,main='Gaussian kernel regression')
lines(x,mhgaus25,col="red")
# CHOOSE h
h <- 0.10
Kh10 <- exp(-(1/(2*h^2))*d)
sc10 <- apply(Kh10,1,sum)
mhgaus10 <- Kh10%*%y/sc10
lines(x,mhgaus10,col="blue")
legend(5, 103, legend=c("h=0.25", "h=0.10"),
      col=c("red", "blue"),lty=1:1, lwd=c(1.5,1.5), cex=0.8)
```

Kernel smoothing methods discussed so far suffer from boundary bias. For example, an estimate for a decreasing function on the left boundary includes points to the right of the boundary and since the function is decreasing, this creates a downward bias. Bias can also occur in the interior of the function if it has substantial curvature and is aggravated when covariates are multidimensional or unequally spaced. These problems can be alleviated using a generalisation of kernel regression: local polynomial regression.

Local Polynomial Regression

The Nadaraya-Watson kernel regression estimator of the conditional expectation $m(x)$ can be framed in terms of a weighted regression. Given the weight function like the one defined in (11.10), choose $\mu = \hat{m}(x)$ to minimise the weighted sum of squares

$$\sum_{i=1}^n w_i(x) (y_i - \mu)^2. \quad (11.11)$$

Setting the derivative with respect to μ equal to zero and solving for μ recovers the kernel estimator

$$\hat{m}(x) = \frac{\sum_i w(x_i) y_i}{\sum_i w(x_i)}. \quad (11.12)$$

This is a weighted regression with a mean only implying the approximation $m(x) \approx \mu$. A generalisation is to replace the local constant μ by a local polynomial of degree p . This polynomial is fitted to each target value x and generates estimates of regression parameters by minimising the weighted sum of squares along the same lines as in (11.11):

$$\sum_{i=1}^n w_i(x) \left(y_i - \sum_{j=0}^p b_{jx} (x_i - x)^j \right)^2. \quad (11.13)$$

This weighted sum of squares can be written

$$\sum_{i=1}^n w_i(x) \left(y_i - \sum_{j=0}^p b_{jx} (x_i - x)^j \right)^2 = (y - X_x b_x)' W_x (y - X_x b_x).$$

Minimisation of this expression with respect to b_x gives the standard weighted least squares estimator

$$\hat{b}_x = (X_x' W_x X_x)^{-1} X_x' W_x y, \quad (11.14)$$

where

$$X_x = \begin{bmatrix} 1 & (x_1 - x) & \cdots & (x_1 - x)^p \\ \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x) & \cdots & (x_n - x)^p \end{bmatrix} \quad (11.15)$$

is an $n \times (p + 1)$ matrix and

$$W_x = \text{diag} [w_1(x) \cdots w_n(x)] \quad (11.16)$$

is an $n \times n$ diagonal matrix of weights $w_i(x)$. In (11.14) the dependence on the target value x has been made explicit by writing $\hat{b}_x = (\hat{b}_{0x}, \hat{b}_{1x}, \dots, \hat{b}_{px})'$. At the target value $x_i = x$ the terms in $(x_i - x)$ drop out and the fitted value is

$$\hat{m}(x) = z_1' (X_x' W_x X_x)^{-1} X_x' W_x y \quad (11.17)$$

$$= H_x y \quad (11.18)$$

$$= \hat{b}_{0x}, \quad (11.19)$$

where z_1 is the $(p + 1) \times 1$ column vector with 1's in the first entry and zeroes elsewhere and H_x is a hat vector. It is important to notice that the complete regression function must be computed for each target value, despite the fact that at $x_i = x$, $\hat{m}(x) = \hat{b}_0(x)$. Setting $p = 0$ yields the kernel regression estimator (11.7).

The same result is obtained if (11.13) is replaced by

$$\sum_{i=1}^n w_i(x) \left(y_i - \sum_{j=0}^p b_{jx} x_i^j \right)^2 = (y - Xb_x)' W_x (y - Xb_x),$$

where now

$$X = \begin{bmatrix} 1 & x_1 & \cdots & x_1^p \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^p \end{bmatrix}_{n \times (p+1)}.$$

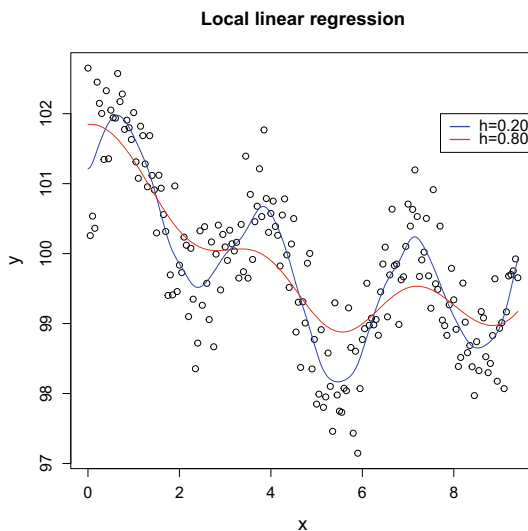
A predicted value at the target $x^* = (x^*, x^{*2}, \dots, x^{*p})$ is

$$\begin{aligned} \hat{m}(x^*) &= (1, x^*) (X' W_{x^*} X)^{-1} X' W_{x^*} y \\ &= \hat{b}_{0x^*} + \sum_{j=1}^p \hat{b}_{jx^*} (x^*)^j, \end{aligned}$$

where $(1, x^*)$ is $(p + 1) \times 1$ and the weight function W_{x^*} is an $n \times n$ diagonal matrix with i th diagonal element

$$w_i(x^*) = \frac{\exp \left[-\frac{(x^* - x_i)^2}{2h^2} \right]}{\sum_{i=1}^n \exp \left[-\frac{(x^* - x_i)^2}{2h^2} \right]}.$$

Fig. 11.4 A local linear regression estimator computed with the R-code CODE1104, using two values for the bandwidth parameter



The R-code below implements a local linear regression ($p = 1$) and the output is displayed in Fig. 11.4.

```
# CODE1104
rm(list=ls()) # CLEAR WORKSPACE
set.seed(195021)
# LOCAL LINEAR REGRESSION SETS  $p = 1$ 
p <- 1
x<-seq(from=0, to=2*pi,by=0.2)
f0<-function(x){ 100+sin(2*x)+cos(x/2) }
R2<-2/3
y<-f0(x)+rnorm(n=length(x),sd=sqrt(var(f0(x))*(1-R2)/R2))

w <- matrix(data=NA,nrow=length(x),ncol=length(x))
X <- matrix(data=NA, nrow=length(y), ncol=(p+1))
W <- matrix(data=NA, nrow=length(y), ncol=length(y))

one <- rep(1,length(y))

X <- cbind(one,x)
Xt <- t(X)
k <- seq(1:length(x))

mhx <- function(k,w,one,X){
  W <- diag(w[k,])
  Xt <- t(X)
  solve(Xt%*%W%*%X,Xt%*%W%*%y)
}

mpred <- function(k,estx,x){
  estx[1,k]+estx[2,k]*x[k]
```

```

}

# CONSTRUCT DISTANCE MATRIX
dst <- as.matrix(dist(x))
d <- as.matrix(dist(x))^2

# CONSTRUCT GAUSSIAN KERNEL
# CHOOSE h
h <- 0.2
Kh25 <- exp(-(1/(2*h^2))*d)
div <- apply(Kh25,1,sum)

# SCALE GAUSSIAN KERNEL: PLACE RESULT IN w
for( i in 1:nrow(Kh25)){
  w[i,] <- Kh25[i,]/div[i]
}

estx <- sapply(k,mhx,w,one,X)

fitx <- sapply(k,mpred,estx,x)
## APPROX PREDICTION FOR A NEW X=5.12:
fitx[which.min(abs(x-5.12))]]
fitx[which.min(abs(x-5.0))]] ## FIT FOR X=5.0

plot(x,y,main='Local linear regression')
lines(x,fitx,lty=1,col="blue")

h <- 0.5
Kh25 <- exp(-(1/(2*h^2))*d)
div <- apply(Kh25,1,sum)

# SCALE GAUSSIAN KERNEL: PLACE RESULT IN w
for( i in 1:nrow(Kh25)){
  w[i,] <- Kh25[i,]/div[i]
}

estx <- sapply(k,mhx,w,one,X)

fitx <- sapply(k,mpred,estx,x)
## APPROX PREDICTION FOR A NEW X=5.12:
fitx[which.min(abs(x-5.12))]]
fitx[which.min(abs(x-5.0))]] ## FIT FOR X=5.0

lines(x,fitx,lty=1,col="red")
legend(5, 103, legend=c("h=0.20", "h=0.50"),
      col=c("blue", "red"),lty=1:1, lwd=c(1.5,1.5), cex=0.8)

```

The R-code below computes a prediction based on the local linear regression for a new value of the feature $x = z$.

```

# CODE1105
rm(list=ls()) # CLEAR WORKSPACE
set.seed(195021)
# LOCAL LINEAR REGRESSION SETS p = 1
p <- 1
x<-seq(from=0, to=2*pi,by=0.2)
f0<-function(x){ 100+sin(2*x)+cos(x/2) }
R2<-2/3
y<-f0(x)+rnorm(n=length(x),sd=sqrt(var(f0(x))*(1-R2)/R2))

z <- 5.12
bz <- matrix(c(1,z),nrow=2)
bzt <- t(bz)
one <- rep(1,length(y))

X <- matrix(data=NA, nrow=length(y), ncol=(p+1))
Wz <- matrix(data=NA, nrow=length(y), ncol=length(y))

X <- cbind(one,x)
Xt <- t(X)
# CHOOSE h
h <- 0.2
d <- x-z
ele <- exp(-(d^2)/(2*h^2))
sm <- sum(ele)
Wz <- diag(ele/sm)
pred_z <- bzt%%solve(Xt%%Wz%%X,Xt%%Wz%%y)
cat("x =",z,"\n")

```

```
## x = 5.12
```

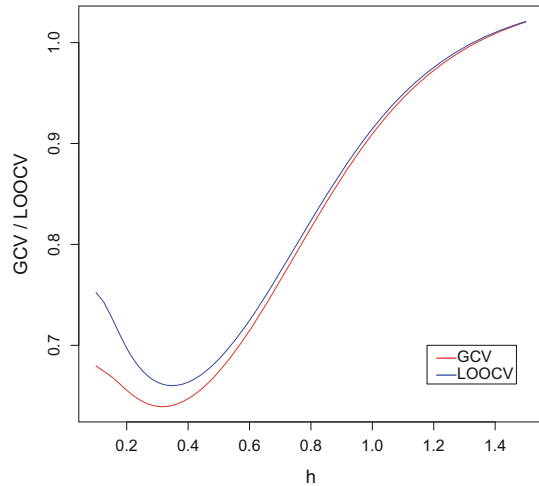
```
cat("Prediction =", pred_z,"\n")
```

```
## Prediction = 98.37111
```

Choice of Bandwidth Parameter

Leave-one-out cross validation can be used to choose the bandwidth parameter h . Since the methods described here are linear smoothers, use can be made of the exact expression (6.73) or of the approximate but more robust generalised cross validation estimator (6.74). The R-code below computes both for a range of values of h . Figure 11.5 illustrates that for the present data, both estimators produce very similar values of leave-one-out mean squared error (0.64 vs 0.66) and choose a similar amount of smoothing ($h = 0.35$ for (6.73) and $h = 0.325$ for (6.74)).

Fig. 11.5 Computation of leave-one-out mean squared error for a range of values of the smoothing parameter h using the generalised cross-validation estimator (6.74) and using estimator (6.73). The former chooses $h = 0.325$ and the latter $h = 0.35$



```
# CODE1106
# COMPUTE HAT MATRIX FOR THE LOCAL LINEAR REGRESSION
rm(list=ls()) # CLEAR WORKSPACE
set.seed(195021)

x<-seq(from=0, to=3*pi,by=0.05)

f0<-function(x){ 100+sin(2*x)+cos(x/2) }
noise <- rnorm(n=length(x),sd=sqrt(var(f0(x))*0.5))
y<- f0(x) + noise

one <- rep(1,length(x))

X <- matrix(data=NA, nrow=length(y), ncol=2))
W <- matrix(data=NA, nrow=length(y), ncol=length(y))
HatMat <- matrix(data=NA, nrow=length(x), ncol=length(x))

h <- seq(from = 0.1, to = 1.5, by = 0.025)
GCV <- rep(0,length(h))
LOOCV <- rep(0,length(h))
form40 <- rep(0,length(h))
dsq <- rep(0,length(h))

tr <- rep(0,length(h))
Trmse <- rep(0,length(h))

j <- seq(1:length(x))

X <- cbind(one,x)
Xt <- t(X)

Hat <- function(j,h,k,x,X){
```

```

d <- x - x[j]
ele <- exp(-(d^2)/(2*h[k]^2))
sm <- sum(ele)
w <- diag(ele)/sm
X[j,] %*% solve(Xt %*% w %*% X) %*% Xt %*% w
}

for (k in 1:length(h)) {
  HatMat <- t(sapply(1:length(x), Hat, h, k, x, X))
  predy <- HatMat %*% y
  tr[k] <- sum(diag(HatMat))
  Trmse[k] <- mean((predy - y) ^ 2)
  dsq[k] <- (1 - mean(diag(HatMat))) ^ 2
  GCV[k] <- Trmse[k] / dsq[k] # generalised LOOCV
  omd <- 1 - diag(HatMat)
  LOOCV[k] <- mean(((predy - y) / omd) ^ 2) # LOOCV
}

h[which.min(LOOCV)]

```

```
## [1] 0.35
```

```
h[which.min(GCV)]
```

```
## [1] 0.325
```

```
min(LOOCV)
```

```
## [1] 0.660054
```

```
min(GCV)
```

```
## [1] 0.639314
```

Extension to Several Dimensions

Kernel smoothing methods such as the Nadaraya-Watson estimator and local regression estimators can be extended to deal beyond the one-dimensional case ($d = 1$) discussed so far. For example, for $d = 2$ and $p = 1$ (polynomial of degree one for each of the two coordinates, a linear regression), terms of the form $(x_i - x)$ in the kernel (11.7) are replaced by

$$(x_i - x) = (x_{1i} - x_1)(x_{2i} - x_2).$$

When a single bandwidth parameter h is used, the kernel reduces to the product of the kernels of each component

$$K\left(\frac{(x_i - x)'(x_i - x)}{h}\right) = K_1\left(\frac{x_{1i} - x_1}{h}\right) K_2\left(\frac{x_{2i} - x_2}{h}\right) \quad (11.20)$$

for target values (x_1, x_2) . However, a multivariate kernel of the form

$$K\left(\frac{(x_i - x)' A (x_i - x)}{h}\right),$$

for some positive semidefinite matrix A can also be used. For diagonal A , this would allow to control the influence of certain predictors by assigning different values to h in (11.20). The fit at the target values $x = (x_1, x_2)$ requires minimising with respect to α, b_1, b_2

$$\sum_{i=1}^n w_i (y_i - \alpha(x_1, x_2) - b_1(x_1, x_2)x_{1i} - b_2(x_1, x_2)x_{2i})^2.$$

The fitted value at the target becomes

$$\hat{m}(x_1, x_2) = (1, x_1, x_2)' \hat{b}(x_1, x_2)$$

where $\hat{b}'(x_1, x_2) = (\hat{\alpha}(x_1, x_2), \hat{b}_1(x_1, x_2), \hat{b}_2(x_1, x_2))$ and

$$w_i(x) = \frac{K\left(\frac{(x_i - x)'(x_i - x)}{h}\right)}{\sum_{i=1}^n K\left(\frac{(x_i - x)'(x_i - x)}{h}\right)}.$$

Local polynomials become less attractive when the dimension exceeds beyond $d = 2$ or 3. Other nonparametric approaches are required to deal with higher dimensions.

11.2 Kernel Methods Using Basis Expansions

The nonparametric methods discussed so far provide flexibility by fitting a model repeatedly for each target value x . The model could be a simple mean model (polynomial of degree zero: the binned estimator), or polynomials of any degree, such as the simple linear regression, a polynomial of degree one. The contribution that the data features $x_i, i = 1, \dots, n$ make to the fit at the target x is controlled by a kernel function.

This section introduces kernel methodology where kernel functions expand the original set of features into an implicitly high-dimensional space. The method can involve regularisation in a similar spirit as in ridge regression and can capture the effect on the data of complex interactions and nonlinear terms involving the original features, without explicit modelling. Here the treatment is heuristic and based on examples. An authoritative reference is Wahba (1990). The topic is also discussed at length in the books of Bishop (2006) and Hastie et al. (2009).

Preliminaries

I review briefly the concepts of *dual representation*, *basis functions* and *kernel functions* and note how a *dual representation* leads to *kernel functions*. These provide flexible mechanisms to implicitly expand the feature space of a regression model. Kernels are introduced using ridge regression as an example.

The following notation and definitions will be needed.

- The p -norm of a vector $x \in \mathbb{R}^n$ is $(|x_1|^p + \cdots + |x_n|^p)^{\frac{1}{p}} = \|x\|_p$
- For $p = 1$, the ℓ_1 -norm is $\|x\|_1 = |x_1| + \cdots + |x_n|$
- For $p = 2$, the ℓ_2 -norm or Euclidean norm or length of vector x is $\|x\|_2 = (x_1^2 + \cdots + x_n^2)^{\frac{1}{2}} = \langle x, x \rangle^{\frac{1}{2}} = \sqrt{x'x} \geq 0$. The Euclidean norm is sometimes denoted without any subscript: $\|x\|$
- For vectors $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^n$, the squared distance between x and y is

$$\|x - y\|_2^2 = \|x\|_2^2 + \|y\|_2^2 - 2x'y.$$

For example, for $x = (x_1, x_2)$ and $y = (y_1, y_2)$,

$$\begin{aligned} \|x - y\|_2^2 &= [(x_1 - y_1), (x_2 - y_2)] [(x_1 - y_1), (x_2 - y_2)]' \\ &= (x_1 - y_1)^2 + (x_2 - y_2)^2 \\ &= x_1^2 + x_2^2 + y_1^2 + y_2^2 - 2x_1y_1 - 2x_2y_2 \\ &= \|x\|_2^2 + \|y\|_2^2 - 2x'y. \end{aligned}$$

and if x and y are orthogonal, $x'y = 0$.

- A vector whose length is 1 is called the *unit vector*. If a non-zero vector x is divided by its length (or multiplied by $1/\|x\|$) a unit vector results because

$$\|u\| = \frac{1}{\|x\|} \|x\| = 1$$

- The inner product of vectors x and y is a function that can be represented as $\sum_{i=1}^n x_i y_i = x'y = y'x = \langle x, y \rangle = \langle y, x \rangle$ (the inner product is symmetrical).

- As noted in the following sections, vectors can be infinitely dimensional. Such infinitely dimensional vector spaces (*Hilbert spaces*) must be endowed with certain structure in order to derive mathematically useful results. This structure guarantees that properties of Hilbert spaces apply also to finite dimensional settings. Technical details are omitted in this overview and I draw instead on a result based on the *Representer theorem* on page 536 that has far reaching applications. A similar result can be arrived at informally using the concept of dual representation. The general idea is to express the solution of a linear system in terms of kernel functions. A variety of kernel functions can be used. This provides a flexible mechanism to expand the feature space without explicit modelling. In this way, nonlinear associations between the covariates and the records can be explored.

Dual Representation

Linear models admit a *dual representation* where kernel functions arise naturally. For example consider a training data set $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ where $x_i \in \mathbb{R}^p$ are p -dimensional feature vectors and $y_i \in \mathbb{R}$ are responses. The regression model can be written

$$y = 1\mu + Xb + e, \quad e \sim (0, I\sigma^2). \quad (11.21)$$

In ridge regression μ and b are obtained as the solution to the minimisation of the cost function

$$J(\mu, b) = (y - 1\mu - Xb)'(y - 1\mu - Xb) + \lambda \|b\|^2, \quad (11.22)$$

where the scalar $\lambda \geq 0$ is a parameter that controls the amount of shrinkage. Differentiation with respect to μ and b gives

$$\begin{bmatrix} 1'1 & 1'X \\ X'X & X'X + \lambda I \end{bmatrix} \begin{bmatrix} \hat{\mu} \\ \hat{b} \end{bmatrix} = \begin{bmatrix} 1'y \\ X'y \end{bmatrix} \quad (11.23)$$

that results in the closed form solution

$$\hat{b} = (X'X + I\lambda)^{-1} X'(y - 1\hat{\mu}). \quad (11.24)$$

A scalar prediction at some target value x (a $p \times 1$ column vector) is equal to

$$m(x) = \hat{\mu} + x'\hat{b}. \quad (11.25)$$

Matrix $X'X + I\lambda$ is of dimension $p \times p$. If $\lambda > 0$ the matrix $X'X + I\lambda$ is positive definite and therefore the solution $\hat{b}$ is unique.

Solution (11.24) can be expressed in what is known as the *dual form*. The partial derivative of $J(\mu, b)$ with respect to b results in

$$-X'y + X'1\mu + X'Xb + \lambda b = 0$$

Multiplying by λ^{-1} and rearranging yields

$$\hat{b} = \lambda^{-1} X' (y - 1\hat{\mu} - X\hat{b}) = X'\hat{\alpha} = \sum_{i=1}^n \hat{\alpha}_i x_i \quad (11.26)$$

where x_i is the i th column of X' with p entries and the *dual variable* $\hat{\alpha}$ is equal to

$$\begin{aligned} \hat{\alpha} &= \lambda^{-1} (y - 1\hat{\mu} - X\hat{b}) \\ \Rightarrow \lambda\hat{\alpha} &= (y - 1\hat{\mu} - XX'\hat{\alpha}) \\ \Rightarrow \hat{\alpha} &= (XX' + I\lambda)^{-1} (y - 1\hat{\mu}). \end{aligned} \quad (11.27)$$

Expression (11.26) indicates that $\hat{b}$ can be expressed as a linear combination of the training features x_i and $\hat{\alpha}$. The dual variable $\hat{\alpha}$ involves the solution of a linear system of n equations. In contrast, (11.24) requires the solution of a linear system of p equations. The choice between both depends on the relative sizes of n and p . An important observation is that the *dual solution* of the ridge regression (11.27) uses inner products between data points x_i via XX' . Indeed, the ij th element of XX' is

$$[XX']_{i,j} = x'_i x_j,$$

where x'_i is the i th row of X and x_j is the j th column of X' . A predicted value at the target x is of the form

$$\begin{aligned} m(x) &= \hat{\mu} + x'\hat{b} = \hat{\mu} + x' \sum_{i=1}^n \hat{\alpha}_i x_i \\ &= \hat{\mu} + \sum_{i=1}^n \hat{\alpha}_i x' x_i \\ &= \hat{\mu} + \sum_{i=1}^n \hat{\alpha}_i \langle x, x_i \rangle, \end{aligned} \quad (11.28)$$

again involving the inner product between the training features x_i and the target x . In general a method will be referred to as *kernelised*, if the training features x appear only inside inner products of functions of x , the *basis functions*. These are dealt with

on page 530. The inner product $\langle x, x_i \rangle$ is the simplest example of a *kernel function*, often referred to as *linear kernel*.

Kernelised Predictions

Consider representing the inner product $\langle x, x_i \rangle$ in the last line of (11.28) by $k(x, x_i)$, a *kernel function*. This kernel function takes the form

$$k(x, x_i) = (x_1, \dots, x_p) \begin{pmatrix} x_{i1} \\ \vdots \\ x_{ip} \end{pmatrix}.$$

The kernelised prediction formulation of (11.28) at the target x becomes

$$m(x) = \hat{\mu} + \sum_{i=1}^n \hat{\alpha}_i k(x, x_i), \quad (11.29)$$

a linear combination of $k(x, x_1), \dots, k(x, x_p)$ considered as a function of x . The predicted values for new features $x'_i, i = 1, \dots, m$, can be written as

$$\begin{bmatrix} m(x'_1) \\ \vdots \\ m(x'_m) \end{bmatrix} = 1\hat{\mu} + \begin{bmatrix} \hat{\alpha}_1 k(x'_1, x_1) + \dots + \hat{\alpha}_n k(x'_1, x_n) \\ \vdots \\ \hat{\alpha}_1 k(x'_m, x_1) + \dots + \hat{\alpha}_n k(x'_m, x_n) \end{bmatrix} = 1\hat{\mu} + K\hat{\alpha} \quad (11.30)$$

generalising expression (11.29). Matrix $K = XX' = (k(x_i, x_j))_{ij} \in \mathbb{R}^{n \times n}$, which is symmetric and positive semidefinite, is known as the *kernel matrix* or *Gram matrix* for any kernel k in the set $\{x_1, \dots, x_n\}$. More details on this matrix follow shortly. At this point note that the Gram matrix contains the evaluation of the kernel function on all pairs of feature points x_i . All the information about the data x is contained in K . New predictions require the input of the complete feature points x_i of the training data.

Kernelised Cost Functions

The cost function (11.22) can be written in terms of the $n \times n$ symmetric kernel matrix K . Using (11.26),

$$\begin{aligned} (y - 1\mu - Xb) &= (y - 1\mu - XX'\alpha) \\ &= (y - 1\mu - K\alpha). \end{aligned}$$

Therefore the residual sum of squares takes the form

$$\|y - 1\mu - Xb\|^2 = \|y - 1\mu - K\alpha\|^2.$$

The penalised term $\lambda \|b\|^2$ can be expressed as

$$\begin{aligned}\lambda \|b\|^2 &= \lambda \|X'\alpha\|^2 \\ &= \lambda \alpha' X X' \alpha \\ &= \lambda \alpha' K \alpha.\end{aligned}$$

Therefore, the cost function (11.22) is equal to the convex and differentiable cost function

$$(y - 1\mu - K\alpha)'(y - 1\mu - K\alpha) + \lambda \alpha' K \alpha. \quad (11.31)$$

Setting the partial derivatives with respect to μ and α equal to zero yields

$$\begin{bmatrix} 1'1 & 1'K \\ K1 & K(K + \lambda I) \end{bmatrix} \begin{bmatrix} \hat{\mu} \\ \hat{\alpha} \end{bmatrix} = \begin{bmatrix} 1'y \\ Ky \end{bmatrix}. \quad (11.32)$$

Rearranging the second equation, results in $K(K + \lambda I)\alpha = K(y - 1\hat{\mu})$. One solution is

$$\hat{\alpha} = (K + \lambda I)^{-1} (y - 1\hat{\mu}), \quad (11.33)$$

as can be confirmed by replacing (11.33) in the partial derivative of the cost function with respect to α . If K is positive definite, (11.33) is the only solution to the minimisation of (11.31). The vector of n fitted values is given by (11.30):

$$\begin{aligned}\hat{y} &= 1\hat{\mu} + K\hat{\alpha} \\ &= 1\hat{\mu} + K(K + \lambda I)^{-1} (y - 1\hat{\mu}).\end{aligned} \quad (11.34)$$

The vector of fitted values $\hat{y}$ is invariant to whatever solution of $K(K + \lambda I)\alpha = K(y - 1\hat{\mu})$ is used. The right hand side of (11.34) can be written as

$$\begin{bmatrix} 1 & K \end{bmatrix} \begin{bmatrix} 1'1 & 1'K \\ K1 & K(K + \lambda I) \end{bmatrix}^{-1} \begin{bmatrix} 1' \\ K \end{bmatrix} y = Hy$$

where H is not a function of y , so this is a linear smoother in the observations.

The subsection concludes with two important messages. First, the algorithm for solving (11.31) involves only the computation of inner products of feature vectors x . All the information about the training data is contained in the matrix K and the output vector y . Secondly, the kernelised cost function (11.31) with solution

(11.33) can be used with a variety of kernel matrices K , providing flexibility. Kernel functions automatically incorporate nonlinear associations between the features x and the records y , without explicit modelling, as discussed in the next subsection.

Nonlinear Feature Mappings Using Kernel Functions

Consider a regression function

$$m(x) = \alpha + bx, \quad (11.35)$$

where b is a scalar parameter and feature $x \in \mathbb{R}$ is also a scalar. One may wish to expand the model to explore nonlinear structures in the training data and, to this end, a *basis function* $\phi(x)$ is used that maps x onto $\phi(x) = (x, x^2, x^3) \in \mathbb{R}^3$, a three dimensional space. The effect of ϕ is to recode the data from $\{(x_1, y_1), \dots, (x_n, y_n)\}$ to $\{(\phi(x_1), y_1), \dots, (\phi(x_n), y_n)\}$. The regression function in the new space is

$$m(\phi(x)) = \alpha + b_1x + b_2x^2 + b_3x^3 = \alpha + b'\phi(x),$$

where now b is a vector of parameters with three elements. This operation allows the output y to be represented by a nonlinear function of the original features x , but y is still linear in the parameters b in the expanded model. The only difference with (11.35) is that $\phi(x)$ is substituted for x . This linearity in the parameters greatly simplifies the analysis of this class of models. As we shall see, these models lead to convex functions that are relatively easy to optimise. The algorithm that solves (11.31) proceeds in exactly the same manner as in the standard linear model, with x replaced by $\phi(x)$. This requires computation of the inner products $\phi(x)'\phi(x)$.

A more general case is to consider a feature vector $x = (x_1, \dots, x_p) \in \mathbb{R}^p$ involving p variables, where the basis function ϕ transforms x into the product of all monomial terms. The notation (x, z) denotes two different values of the feature x , (x_i, x_j) and therefore $x, z \in \mathbb{R}^p$. The basis function is

$$\begin{aligned} \phi(x) &= (x_1x_1, x_1x_2, \dots, x_1x_n, \dots, x_px_p)' \in \mathbb{R}^{p^2}, \\ \phi(z) &= (z_1z_1, z_1z_2, \dots, z_1z_n, \dots, z_pz_p)' \in \mathbb{R}^{p^2}. \end{aligned} \quad (11.36)$$

The time required to compute $\phi(x)$ or the inner product $\phi(x)'\phi(z)$ is of order p^2 (because there are p^2 terms and p^2 products) while the time required to compute the inner product in the original feature space $x'z$ is of order p (there are p products). So the complexity of the evaluation of the inner products is proportional to the

dimension of the feature space. The function that for all x, z computes the inner products $\phi(x)' \phi(z)$ is a *kernel function* k . Specifically,

$$k(x, z) = \langle \phi(x), \phi(z) \rangle = \phi(x)' \phi(z) = \sum_{i=1}^p \sum_{j=1}^p (x_i x_j) (z_i z_j). \quad (11.37)$$

The kernel function involves p^2 products, where ϕ is a mapping from X to an inner product feature space F . When ϕ is the identity mapping and the new feature space involves the covariates $x_i \in \mathbb{R}^p$ of the training set, $K = XX'$. The ij th element of XX' is $(XX')_{ij} = \langle x_i, x_j \rangle = x_i' x_j$, where x_i' is the i th row of the incidence matrix $X \in \mathbb{R}^{n \times p}$; this particular kernel is known as the *linear kernel*. The kernel matrix K has dimension $n \times n$.

At face value, it seems that use of the basis function ϕ to explore a higher dimensional space comes at the cost of increased computation. The next section shows that this is not the case.

The Kernel Trick

As indicated above, the computation of inner products involving ϕ in (11.37) can be very demanding when the inner product feature space F is high dimensional. In the case of (11.37) there are p^2 products. As shown below, regardless of the dimension of F the use of kernel functions allows computation of $\phi(x)' \phi(z)$ using the original features (x, z) without explicitly computing the mapping ϕ . The claim is that the inner product (11.37) involving the basis function (11.36) can be computed using

$$k(x, z) = \phi(x)' \phi(z) = (x'z)^2 \quad (11.38)$$

requiring p products. To prove the claim write

$$\begin{aligned} k(x, z) &= (x'z)^2 \\ &= \left(\sum_{i=1}^p x_i z_i \right) \left(\sum_{j=1}^p x_j z_j \right) \\ &= \sum_{i=1}^p \sum_{j=1}^p x_i x_j z_i z_j \\ &= \sum_{i=1}^p \sum_{j=1}^p (x_i x_j) (z_i z_j) \\ &= \phi(x)' \phi(z). \end{aligned} \quad (11.39)$$

The first line in (11.39) requires order p computations involving the inner product of vectors x and z , each with p elements, while the last line requires order p^2 . The inner product between the new feature space (high dimensional) has been computed without explicitly evaluating the new feature space, using the original (lower dimensional) feature space. The function that performs this direct computation is the kernel function k and the operation of swapping the linear kernel with the new kernel is known as the kernel trick.

Choice of Kernel Functions

The use of kernel functions provides flexibility: the same algorithm can be applied replacing one kernel function with another. Also, very high dimensional feature spaces involving nonlinear terms among the features and complex interactions can be explored, without their explicit evaluation. The question is, how should a kernel be chosen and what function qualifies as a kernel function?

A function $k(x, z)$ can be used as a kernel function if there exists some ϕ such that $k(x, z) = \phi(x)' \phi(z)$. For a given set of feature vectors $\{x_1, \dots, x_n\}$, $x_i \in \mathbb{R}^p$, define the symmetric matrix $K \in \mathbb{R}^{n \times n}$ to be the Gram matrix or kernel matrix such that $[K_{ij}]_{i,j=1,\dots,n}$ where, $K_{ij} = \langle \phi(z_i), \phi(z_j) \rangle$. Then

$$\begin{aligned} z' K z &= \sum_{i=1}^n \sum_{j=1}^n z_i z_j K_{ij} \\ &= \sum_{i=1}^n \sum_{j=1}^n z_i z_j \langle \phi(z_i), \phi(z_j) \rangle \\ &= \left\langle \sum_{i=1}^n z_i \phi(z_i), \sum_{j=1}^n z_j \phi(z_j) \right\rangle \\ &= \left\| \sum_{i=1}^n z_i \phi(z_i) \right\|^2 \geq 0 \end{aligned}$$

and therefore K is a positive semidefinite matrix. This result indicates that the construction of a valid kernel involves first, finding a feature mapping ϕ and then computing its inner product to generate K . There is a powerful result known as *Mercer's Theorem* that allows going the other way: choose an appropriate K and do not worry about ϕ . An appropriate K is simply any symmetric, positive semidefinite matrix.

A few examples of commonly used kernels are

- the linear kernel $k(x, z) = \langle x, z \rangle$

- the polynomial kernel of degree d , $k(x, z) = (\langle x, z \rangle + 1)^d$
- the Laplacian radial basis kernel $k(x, z) = \exp \left[-\frac{\|x-z\|_1}{2\sigma^2} \right]$
- the Gaussian or radial basis function kernel $k(x, z) = \exp \left[-\frac{\|x-z\|_2^2}{2\sigma^2} \right]$.

(Above, the term $2\sigma^2$ is often replaced by the positive scalar h , the bandwidth parameter).

Any of these kernel functions and many others can be used to minimise (11.31) with solution given by (11.33).

An observation regarding the Gaussian kernel is that, in order to express it as $k(x, z) = \langle \phi(x), \phi(z) \rangle$, the new feature vector $\phi(x)$ must be of infinite dimension. The inner product space of this infinitely dimensional feature vector is referred to as a *Reproducing kernel Hilbert space* (RKHS). The finite dimensional inner product space $\mathbb{R}^p$ is a special case. The Gaussian kernel is a square, *positive definite* matrix of dimension $p \times p$, where the original feature space is $\mathbb{R}^p$, despite the fact that $\phi \in \mathbb{R}^\infty$. To illustrate, assume a single scalar feature per response. The basis function $\phi(x)$ corresponding to the Gaussian kernel is an implicitly infinite vector with the form

$$\phi(x) = \exp \left[-\frac{x^2}{2\sigma^2} \right] \left[1, \frac{x}{\sigma\sqrt{1!}}, \frac{x^2}{\sigma^2\sqrt{2!}}, \frac{x^3}{\sigma^3\sqrt{3!}}, \dots \right]'$$

and $\langle \phi(x), \phi(z) \rangle$ is a series that converges to $k(x, z)$. Indeed,

$$\begin{aligned} \langle \phi(x), \phi(z) \rangle &= \sum_{j=0}^{\infty} \frac{1}{j!} \exp \left[-\frac{(x^2 + z^2)}{2\sigma^2} \right] \frac{x^j z^j}{(\sigma^2)^j} \\ &= \exp \left[-\frac{(x^2 + z^2)}{2\sigma^2} \right] \sum_{j=0}^{\infty} \frac{(xz)^j}{(\sigma^2)^j j!} \\ &= \exp \left[-\frac{(x^2 + z^2)}{2\sigma^2} \right] \exp \left[\frac{xz}{\sigma^2} \right] \\ &= \exp \left[-\frac{(x - z)^2}{2\sigma^2} \right] = k(x, z) \end{aligned}$$

where the step from the second to third line follows from the characterisation of the exponential function as a Maclaurin series.

Interestingly, the finite dimensional cost function (11.31) with solution (11.33) can be used despite the infinite dimensionality of the inner product space associated with the Gaussian kernel. Positive definite kernel functions give rise to implicitly infinite dimensional feature spaces.

Note

The term $2\sigma^2$ in the Laplacian basis kernel and in the Gaussian kernel is the bandwidth parameter and can be replaced by the customary notation h .

Kernel Matrices as Similarity Matrices

Kernel matrices are also known as *similarity matrices* or *covariance functions*. For example, consider two p dimensional vectors x_i and x_j , where p could represent the number of features, while i and j could represent two individuals. The Gaussian kernel function is

$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{\sigma^2}\right),$$

where

$$\|x_i - x_j\| = \left[(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \cdots + (x_{ip} - x_{jp})^2\right]^{\frac{1}{2}} \quad (11.40)$$

is the Euclidean distance and σ^2 (or written also as h) is the bandwidth of the kernel. The kernel function $k(x_i, x_j)$ evaluates to 1 if x_i and x_j are identical and approaches 0 as x_i and x_j become increasingly different. In a genomics context when x represents SNP's, the elements of the kernel matrix are larger for pairs of individuals that are more genetically alike.

The bandwidth parameter controls how fast the Euclidean distance falls with increasing difference between features x_i and x_j . The choice of the bandwidth parameter has important consequences for inference and prediction. Large values of σ^2 lead to entries of the Gaussian kernel approaching 1 resulting in underfitting. Small values of σ^2 generate entries approaching 0. In this case, in a model including a random effect and a residual term, the random effect associated with the kernel matrix becomes confounded with the residual term. The bandwidth parameter can be chosen using cross-validation.

A few remarks are in order.

- Kernel matrices are dense and in large data sets can be computationally challenging to construct.
- Concerning the influence of the bandwidth parameter on predictions based on (11.30): If the elements of K are very small the predicted values are all approximately equal to $\hat{\mu}$. Such a kernel model has little or no predictive power.
- On the other hand, if all elements of K are very close to 1, the predicted values are very similar and again, predictive power is compromised.

- Kernels can be used for many types of inputs such as strings, discrete structures, images, time series and more, provided the kernel matrices corresponding to any finite training set are positive semidefinite. They are an extremely flexible tool.
- A new kernel can be generated from existing kernels by addition, element-wise multiplication, or by multiplication by a positive scalar. The requirement is that these operations result in a symmetric, positive semidefinite matrix. This property provides great flexibility. In a genetics context, a new kernel can be constructed from the sum of existing covariance matrices derived from specific parametric assumptions. The resulting kernel can be used to exploit potential contributions from, for example, non-additive genetic sources of variation or from SNP $\times$ environment interactions, despite the fact that the nonparametric model is not explicitly built upon specific mechanistic considerations (Gianola and de los Campos 2008; de los Campos et al. 2009).
- The bandwidth parameter of a Gaussian kernel can be selected by cross-validation or can be inferred as in Perez-Elizalde et al. (2015).
- As illustrated on page 540, kernels can be viewed as prior information describing covariance structures in the data. The choice of kernel should reflect prior knowledge, but this may be difficult to accomplish. One may want to consider instead a family of kernels weighted by parameters inferred from the data (for example, de los Campos et al. 2010), or attempt to infer the kernel from the data (for example, Duvenaud et al. 2013 and Gianola et al. 2020).
- As is clear from (11.40) diagonal elements of the Euclidean distance can become very large when the number of features p is large. In such a situation the Gaussian kernel matrix is diagonally dominant and the model may predict poorly. Typically this calls for some form of standardisation at the level of the inputs and perhaps at the level of the Euclidean distance leading to different kernel matrices. This may impinge on the predictive capacity of the model and can be gauged using cross-validation
- Kernel methods are essentially prediction tools. The kernel function expands the original feature space into a new, high dimensional feature space via inner products of basis functions that are not explicitly constructed. It is therefore not straightforward to measure the importance of a specific feature in the new feature space, contrary to parametrised regression models. However, there are kernels that are constructed in a manner that resemble known parameters of fully parametric models and this provides a framework for interpretation. An example on page 540 is the linear kernel that takes the form of a genomic relationship matrix and the smoothing or regularisation parameter is interpreted as a ratio of variance components.

It becomes clear that the linear kernel can be viewed as a similarity matrix if it is transformed into a correlation matrix where the diagonal elements, equal to 1, describe the similarity between the individual with itself. A genomic relationship matrix is also a linear kernel that can be scaled in a variety of ways. A standard approach is to transform the marker genotype x_{ij} into $w_{ij} \sim (0, 1)$, where i refers to individual (row of X) and j to marker genotype (column of X). The

genomic variance under such a model is $p\sigma_b^2$ (this is the marginal variance of $w_i'b$, where vector $b \in \mathbb{R}^p$ has *iid* elements representing p marker effects distributed as $N(0, I\sigma_b^2)$).

Another approach is to centre x_{ij} and divide by $\left(\sum_{j=1}^p \text{Var}(x_j)\right)^{\frac{1}{2}}$, where $\text{Var}(x_j)$ is the variance of marker j , equal to $2p_j(1 - p_j)$. In practice the variance of marker j is replaced by the empirical variance of the elements of column j of matrix X . With this scaling and centring of x_{ij} , the genomic variance is equal to σ_b^2 , independent of the number of markers.

Reducing Infinitely Dimensional Problems to Finite Dimensional Problems

This short tour of kernel methods using basis expansions took ridge regression as starting point from which a number of results were derived. The model is specified in (11.21) with solution (11.23). The dual representation allows to express the solution and predictions in terms of inner products involving the original feature data, as indicated in (11.26) and (11.30). Kernel functions, with associated kernel matrices, are defined as inner products of basis functions. The basis functions expand the original feature space. In this way, one can explore the effects of nonlinear terms of the original feature space, on the observations. Kernel methodology creates a very flexible setup, because the same algorithm to solve the linear system can be applied with different kernel matrices.

A more rigorous approach to arrive at the same result invokes the *Representer theorem* (Kimeldorf and Wahba 1971). Consider a training data set $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ where $x_i \in \mathbb{R}^p$ are p -dimensional feature vectors, $y_i \in \mathbb{R}$ are responses. Assume a regression function $m(x_i) = \mu + g(x_i)$, where, for example, $g(x_i) = x_i'b$ as in the linear model, but could also accommodate other nonlinear structures. A flexible general formulation for estimating m when it belongs to some possibly infinite dimensional Hilbert space $\mathcal{H}$ of functions, is to minimise the cost function

$$J = \min_{m \in \mathcal{H}} \sum_{i=1}^n \ell(y_i, m(x_i)) + \lambda \|m\|_{\mathcal{H}}^2 \quad (11.41)$$

where ℓ is some function measuring goodness of fit to data such as $\ell(y, m) = (y - m)^2$, a quadratic function, although other functions (such as the negative of the log-likelihood) can be used, λ is the smoothing or regularisation parameter and the last term $\|m\|_{\mathcal{H}}^2$ is a norm in the space of functions $\mathcal{H}$ generated by a positive definite kernel $K(x_i, x_j)$. Solving for m directly from (11.41) involves an optimisation in a possibly infinite dimension; this is a difficult task.

The representer theorem finds a solution of an equivalent problem in a finite dimension of the form

$$g(x) = \sum_{i=1}^n \alpha_i k(x_i, x), \quad (11.42)$$

that requires finding the values of n scalar coefficients α_i (and of the intercept μ). Writing

$$\begin{aligned} m(x_i) &= \mu + g(x_i) \\ &= \mu + \sum_{i=1}^n \alpha_i k(x_i, x), \end{aligned} \quad (11.43)$$

this translates into solving

$$\begin{aligned} &\min_{\mu, \alpha} \sum_{i=1}^n \ell(y_i, \mu + g(x_i)) + \lambda \|g\|^2 = \\ &\min_{\mu, \alpha} \sum_{i=1}^n \ell\left(y_i, \mu + \sum_{j=1}^n \alpha_j k(x_i, x_j)\right) + \lambda \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j k(x_i, x_j). \end{aligned} \quad (11.44)$$

For example in the case of ridge regression, the cost function is

$$J = \sum_{i=1}^n (y_i - m(x_i))^2 + \lambda \|g\|^2.$$

Plugging (11.43) into J and writing $\|g\|^2 = \alpha' K \alpha$ yields the finite dimensional optimisation problem of the form

$$J = \|y - 1\mu - K\alpha\|^2 + \lambda \alpha' K \alpha,$$

as in (11.31), where $[K]_{i,j} = k(x_i, x_j)$ is an $n \times n$ Gram matrix. The minimiser over α is (11.33) and the fitted values are given by (11.34); these are obtained by direct application of the Representer theorem, without the need for a dual form.

A couple of examples follow, starting with a toy example that highlights details of some of the matrix operations and illustrates how the Gaussian kernel regression can cope with strong nonlinear relationships between the feature x and the output y .

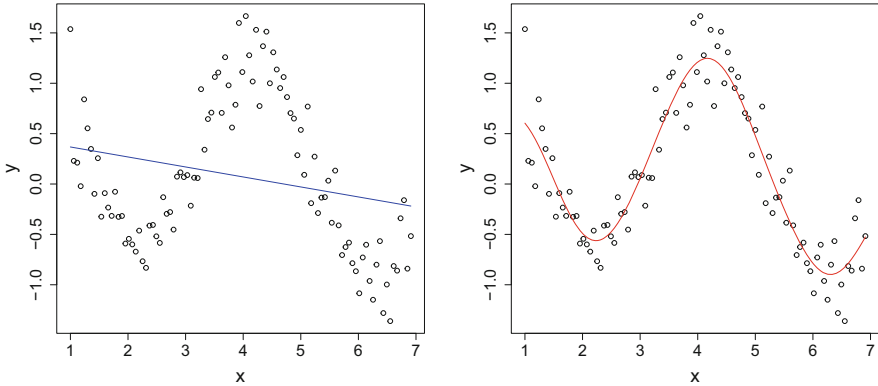


Fig. 11.6 Left: fitted values using a classical linear ridge regression. Right: fitted values using a Gaussian kernel

Example: Classical, Kernelised and Gaussian Kernel Ridge Regression

The R-code below generates a scalar covariate x and a vector of responses y of dimension $N = 100$, nonlinearly associated with x . In a first stage, it performs an analysis using standard ridge scalar regression solving the linear system (11.23). Model (11.21) includes an intercept μ and a regression parameter b , so the left hand side of the system (11.23) is of dimension 2×2 . The fitted values are obtained from $X\hat{b}$, where $\hat{b}$ is the solution to (11.23). The estimate of the regression parameter is $\hat{b} = 0.399$ and the code prints the first 10 fitted values. Figure 11.6, left panel, displays the output y as a function of the feature vector x and the fitted values.

```
# CODE1107
rm(list=ls()) # CLEAR WORKSPACE
set.seed(195021)
library(MASS)
N <- 100
# GENERATE DATA
x<-seq(from=1, to=2.2*pi,length=N)
signal <- cos(1.5*x)+ exp(-0.4*x)
noise <- rnorm(N,0,0.25)
y <- signal + noise
lambda <- 0.01
X <- cbind(1,x)
RHS <- crossprod(X,y)
LHS <- crossprod(X)
LHS[-1,-1] <- LHS[-1,-1]+diag(c(rep(1,1)))*lambda # assumes
#           one covariate x
#### Classical LS solution
bh1 <- solve(LHS,RHS)
bh1
```

```
##           [,1]
##      0.46692087
## x -0.09918769
```

```
yhatclassic <- X%*%bh1
yhatclassic[1:5]
```

```
## [1] 0.3677332 0.3618105 0.3558878 0.3499650 0.3440423
```

```
yhatclassic[6:10]
```

```
## [1] 0.3381196 0.3321969 0.3262742 0.3203515 0.3144288
```

The R-code below fits the same model in dual form with $K = XX'$. The left hand side of the system (11.32) is now 101×101 and the code prints out the first 10 fitted values, which are identical to those generated with the classical representation. The estimates of the intercept and of the regression parameter are the same as in the classical parametrisation.

```
# CODE1107 (cont)
#### kernelised (dual) solution with kernel matrix XX'
K <- x%*%t(x) # LINEAR KERNEL
X <- cbind(1,K)
RHS <- crossprod(X,y)
LHS <- crossprod(X)
LHS[-1,-1] <- LHS[-1,-1]+K*lambda
diag(LHS) <- diag(LHS) + c(0,rep(1e-8,N))
sol <- as.matrix(solve(LHS,RHS))
yhatkernellin <- sol[1]+K%*%sol[-1]
yhatkernellin[1:5]
```

```
## [1] 0.3677332 0.3618105 0.3558878 0.3499650 0.3440423
```

```
yhatkernellin[6:10]
```

```
## [1] 0.3381196 0.3321969 0.3262742 0.3203515 0.3144288
```

```
alfa <- sol[-1]
bhkernellin <- sum(alfa*x)
bhkernellin
```

```
## [1] -0.09918769
```

```
muhatkernellin <- sol[1]
muhatkernellin
```

```
## [1] 0.4669209
```


Finally, I fit the Gaussian kernel regression using the R-code below. The same system of Eq. (11.32) is solved, where now K represents the Gaussian kernel with $\sigma^2 = h = 0.7$. The fitted values are displayed in Fig. 11.6, right panel. The nonlinear relationship between the covariate and the data is well captured by the Gaussian kernel.

```
# CODE1107(cont)
##### GAUSSIAN KERNEL SOLUTION #####
# CONSTRUCT GAUSSIAN KERNEL
#w <- matrix(data=NA,nrow=length(x),ncol=length(x))
K <- matrix(data=NA,nrow=length(x),ncol=length(x))

d <- as.matrix(dist(x))^2
# CHOOSE h and lambda
h <- 0.7
lambda <- 0.5

K <- exp(-(1/(2*h^2))*d)

X <- cbind(1,K)
RHS <- crossprod(X,y)
LHS <- crossprod(X)
LHS[-1,-1] <- LHS[-1,-1]+K*lambda

diag(LHS) <- diag(LHS)+c(0,rep(1e-8,N))

sol <- solve(LHS,RHS)
fgaus <- sol[1]+K%*%sol[-1]
alfa <- sol[-1]
```

Bayesian View of Kernelised Regression

Consider the derivation of BLUP (best linear unbiased predictor) that can be found in Henderson et al. (1959), based on finding the maximiser of the joint density of y (vector of dimension $n \times 1$) and a (vector of dimension $q \times 1$) with respect to b (vector of dimension $p \times 1$) and a , under the assumption of normality. When vector a represents additive genetic values, this probability model is known as the additive genetic model. With known covariances matrices R and G , the logarithm of the joint distribution takes the form (ignoring an additive constant and multiplying by -1)

$$\begin{aligned} \log [p(y, a|b, R, G)] &= \log [p(y|a, b, R)] + \log [p(a|G)] \\ &\propto (y - Xb - Za)' R^{-1} (y - Xb - Za) + a' G^{-1} a. \end{aligned} \quad (11.45)$$

Differentiating with respect to b and a leads to the linear system, which arranged in the usual mixed model format is

$$\begin{bmatrix} X'R^{-1}X & X'R^{-1}Z \\ Z'R^{-1}X & Z'R^{-1}Z + G^{-1} \end{bmatrix} \begin{bmatrix} \hat{b} \\ \hat{a} \end{bmatrix} = \begin{bmatrix} X'R^{-1}y \\ Z'R^{-1}y \end{bmatrix}. \quad (11.46)$$

An alternative interpretation based on a Bayesian argument, is to write the joint posterior density $p(b, a|y, G, R)$ as

$$\begin{aligned} p(b, a|y, G, R) &\propto p(y|a, b, R) p(a, b|G) \\ &= p(y|a, b, R) p(a|G) \end{aligned} \quad (11.47)$$

assuming $p(a, b|G) \propto p(a|G)$. Then $[\hat{b}, \hat{a}]$ is the mode (and the mean under normality) of the posterior distribution $[b, a|y, G, R]$.

This same structure (11.45) can be applied to the kernel function (replacing vector b with scalar μ)

$$\frac{1}{\sigma_e^2} (y - 1\mu - K\alpha)' (y - 1\mu - K\alpha) + \frac{1}{\sigma_a^2} \alpha' K \alpha \quad (11.48)$$

which implies $[y|\mu, \alpha, \sigma_e^2] \sim N(1\mu + K\alpha, I\sigma_e^2)$ and $[\alpha|K, \sigma_a^2] \sim N(0, K^{-1}\sigma_a^2)$, where σ_a^2 is the variance captured by the specific kernel adopted. Differentiation with respect to μ and α leads to the linear system

$$\begin{bmatrix} 1'1 & 1'K \\ K'1 & K'K + \lambda K \end{bmatrix} \begin{bmatrix} \hat{\mu} \\ \hat{\alpha} \end{bmatrix} = \begin{bmatrix} 1'y \\ K'y \end{bmatrix}, \quad (11.49)$$

where $\lambda = \sigma_e^2/\sigma_a^2$. This system is identical to (11.32). One can argue that this interpretation brings us back to the parametric setup and offers a framework to infer the regularisation parameter λ from data, using likelihood or Bayesian methods. Parameter λ is expressed as a ratio of the two variance components, i.e., $\lambda = \sigma_e^2/\sigma_a^2$.

Genetic Models Using Kernelised Regressions

The classical additive genetic model assumes that, in (11.45), $G = A\sigma_a^2$ where A is a positive definite $q \times q$ matrix of additive genetic relationships among additive genetic values of q individuals, given a pedigree, and the scalar σ_a^2 is the additive genetic variance of some trait.

In the kernel model (11.48), consider the transformation $g = K\alpha$. Since α is normal with mean zero and variance $K^{-1}\sigma_a^2$, it follows that $g \sim N(0, K\sigma_a^2)$. The

cost function to be minimised along the same lines as in (11.48) is now

$$\frac{1}{\sigma_e^2} (y - 1\mu - g)' (y - 1\mu - g) + \frac{1}{\sigma_\alpha^2} g' K^{-1} g. \quad (11.50)$$

Differentiation with respect to μ and g leads to the linear system

$$\begin{bmatrix} 1' & 1' \\ 1 & I + \lambda K^{-1} \end{bmatrix} \begin{bmatrix} \hat{\mu} \\ \hat{g} \end{bmatrix} = \begin{bmatrix} 1'y \\ y \end{bmatrix} \quad (11.51)$$

where $\lambda = \sigma_e^2 / \sigma_\alpha^2$ is the regularisation parameter. If $K = A$ and $\sigma_\alpha^2 = \sigma_a^2$, the linear system (11.51) becomes identical to that of the classical (infinitesimal) additive genetic model using the numerator relationship matrix as kernel (de los Campos et al. 2009). In this case the random variable g represents additive genetic values. Of course the solution for $\hat{g}$ in (11.51) is identical to the solution for $K\hat{\alpha}$ in (11.49). To see this, write the second equation in (11.49) as

$$K' (K + \lambda I) \hat{\alpha} = K' (y - 1\hat{\mu}).$$

Multiplying both sides by K^{-1} and using $K' = K$ due to symmetry, gives

$$(K + \lambda I) \hat{\alpha} = (y - 1\hat{\mu}). \quad (11.52)$$

Similarly from (11.51),

$$(I + \lambda K^{-1}) \hat{g} = (y - 1\hat{\mu})$$

and replacing $\hat{g}$ by $K\hat{\alpha}$ recovers (11.52). A similar result applies to the equations for $\hat{\mu}$.

This development was based on the classical infinitesimal model; a genomic model incorporating a large number of markers leads to the same results. In this case, g are genomic values, σ_a^2 is interpreted as the genomic variance and K as a genomic relationship matrix.

Example: A Bayesian Kernelised Regression

A Bayesian implementation of a RKHS (reproducing kernel Hilbert space) model is applied to the wheat data set downloaded from BGLR (Perez and de los Campos 2014) consisting of grain yields from $n = 599$ wheat inbred lines genotyped for 1279 genetic markers. A Gaussian kernel with three values of the bandwidth parameter h is used to illustrate the effect on inferences.

The Bayesian RKHS model assumes the following hierarchical structure

$$y|\mu, g, \sigma_e^2 \sim N\left(1\mu + g, I\sigma_e^2\right),$$

with

$$g|K_h, \sigma_g^2 \sim N\left(0, K_h\sigma_g^2\right),$$

and improper uniform prior distributions for μ , σ_e^2 and σ_g^2 . The positive definite $n \times n$ matrix K_h is the Gaussian kernel with bandwidth parameter h .

In order to improve computational efficiency, K is expressed as

$$K = UDU', \quad (11.53)$$

(de los Campos et al. 2010) using the eigenvalue decomposition shown on page 97. Details of the Gibbs sampling implementation can be found on page 249 (here, K is positive definite whereas in the example of page 249, G is singular; this requires a very slight modification of the MCMC implementation). In (11.53) U is the $n \times n$ orthogonal matrix of eigenvectors of K and D is an $n \times n$ diagonal matrix with eigenvalues $d_i > 0$, for all i .

A chain of length 25, 000 was run and the Monte Carlo estimates of posterior means of σ_g^2 , σ_e^2 and $\lambda = \sigma_e^2/\sigma_g^2$ for $h = 0.5$, $h = 1$ and $h = 3$ are shown in Table 11.1.

Inferences about the components of variance are sensitive to the chosen value of the bandwidth parameter h , but the regularisation parameter λ is less affected. The impact on prediction varying h for constant λ is illustrated in an example on page 573.

The R-code to execute the Gibbs sampler is shown below. The lines at the bottom compute the Monte Carlo (sampling) error of features of the posterior samples based on the method of batching.

Table 11.1 MCMC-based Bayesian inferences of the components of variance conditional on three values of the bandwidth parameters h for grain yield data of 599 wheat inbred lines from the BGLR package, using a Gaussian kernel RKHS regression. The regularisation parameter λ is the ratio of the variance components σ_e^2 and σ_g^2 . The elements in brackets in the fourth column show the 95% posterior intervals for the MC estimates of the posterior distribution of the regularisation parameter λ . The Monte Carlo standard error of the posterior mean of the λ 's is approximately 0.002

h	σ_g^2	σ_e^2	λ
0.5	1.20	0.33	0.28 (0.17;0.45)
1.0	0.83	0.27	0.33 (0.20;0.52)
3.0	0.72	0.20	0.28 (0.15;0.46)

```

# CODE1108
# BAYESIAN KERNELISED REGRESSION
# WHEAT DATA FROM BGLR

# CODE ASSUMES K IS OF FULL RANK
# THEREFORE IT DOES NOT WORK WITH CENTRED G.
# IT WORKS WITH GAUSSIAN KERNEL

rm(list=ls()) # CLEAR WORKSPACE
set.seed(37111)
library(BGLR)
data(wheat)
### USE BGLR MATRIX X
X <- wheat.X
y<- wheat.Y[,1]
nindiv<-length(y)
nmark<-ncol(X)

#### A GAUSSIAN KERNEL #####

kgaus <- function(X,h){
  X <- scale(X,center=TRUE,scale=FALSE)
  S=sqrt(sum(apply(FUN=var, X=X,MARGIN=2)))
  X <- X/S
  D <- as.matrix(dist(X))^2
  K <- exp(-h*D)
}
##### CHOOSE GAUSSIAN KERNEL #####
#h <- 0.5
h <- 1
#h <- 3.0

K <- kgaus(X,h)
dim(K)
qr(K)$rank
G<-K
#####
# EIGEN DECOMPOSITION OF G
EVD <- eigen(G)
names(EVD)
head(EVD$values)
U <- EVD$vector
tU<-t(U)
val <- EVD$values
summary(val)
D <- diag(val,nrow=nindiv)
#Dp IS A VECTOR WITH NON-ZERO EIGENVALUES
Dp<-c(val[1:nindiv])
#INITIALISE Ve
Ve<-0.5
#INITIALISE Vg

```

```

Vg<-0.5
#INITIALISE k
k<-Ve/Vg
#INITIALISE VECTOR ALFA
alfa<-rep(0,nindiv)
# CHOOSE LENGTH OF GIBBS CHAIN
rep<-25000
#INITIALISE result
result<-matrix(data=NA,nrow=rep,ncol=5)
# START GIBBS CHAIN
ptm <- proc.time()
for (i in 1:rep)
{
  cat(i, "\n",sep="")
  # SAMPLE mu
  avmu<-sum(y-U%%alfa)/nindiv
  varmu<-Ve/nindiv
  mu<-rnorm(1,mean=avmu,sd=sqrt(varmu))
  # SAMPLE alfa1 (VECTOR OF LENGTH nindiv)
  meanalfa1<-(Dp/(Dp+k))*tU%%(y-mu)
  varalfa1<-((Dp)/(Dp+k))*Ve
  alfa1<-rnorm(nindiv,meanalfa1,sqrt(varalfa1))
  alfa<-alfa1
  # SAMPLE Vg
  # COMPUTE SCALE
  scVg<-sum(alfa1*alfa1*(1/Dp))
  Vg<-scVg/rchisq(1,nindiv-2)
  #Vg<-0.0001
  # SAMPLE Ve
  # COMPUTE SCALE
  ystar<-y-mu-U%%alfa
  scVe<-sum(ystar*ystar)
  Ve<-scVe/rchisq(1,nindiv-2)
  k<-Ve/Vg
  ualfa <- U%%alfa
  result[i,<-c(i,mu,Vg,Ve,k)
# print(result[i,])
}
proc.time()-ptm
apply(result[2000:rep,2:5],2,mean)
#####
#CODE FOR THE MC VARIANCE BASED ON BATCHING
y <- result[,5] # READS IN ALL DRAWS STORED IN RESULT
#choose number of batches b
b<-500
batch_size <- length(y)/b
batch_size
x<-matrix(y,ncol=b, byrow=FALSE)
avrb<-apply(x,2,mean)
mcvarb<-var(avrb)/length(avrb)
sqrt(mcvarb)
efchsizebatch<-var(y)/mcvarb
efchsizebatch

```

```
#####
### PLOT AUTOCORRELATION VERSUS LAG USING
### R-FUNCTION acf
require(graphics)
acf(y) ## AUTOCORRELATION OF McMC DRAWS
acf(avrb) ## AUTOCORRELATION OF BATCH MEANS
```

Kernel Logistic Regression

Section 9.4 introduced penalised logistic regression, where regression coefficients are shrunk as in ridge regression allowing the analysis when $p > n$, where p is the number of features and n is the length of the vector of binary records y . Kernel methods also make use of penalisation and work instead in an n -dimensional space that can be an advantage when $p \gg n$. Additionally, kernel methods extend the feature space exploring complicated associations among the original features without explicit modelling. Here I describe an application to logistic regression. The starting point is to use $f(x_i) = \mu + x_i' \beta$ in the negative of the log-likelihood (9.14)

$$-\ell(\mu, \beta | y, x) = - \sum_{i=1}^n \{y_i (f(x_i)) - \ln[1 + \exp(f(x_i))]\},$$

and use the representer theorem result (11.43) replacing $f(x_i)$ by

$$f(x_i) = \mu + \sum_{j=1}^n \alpha_j k(x_i, x_j). \quad (11.54)$$

The next step is to minimise the convex cost function with respect to μ and α_i , $i = 1, 2, \dots, n$

$$\begin{aligned} J(\mu, \alpha | \lambda) = & - \sum_{i=1}^n \{y_i (f(x_i)) - \ln[1 + \exp(f(x_i))]\} \\ & + \frac{\lambda}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j k(x_i, x_j). \end{aligned} \quad (11.55)$$

Using similar algebra as in Sect. 9.4, the expressions for the first derivatives are

$$\frac{\partial J}{\partial \mu} = -1'(y - \pi), \quad (11.56a)$$

$$\frac{\partial J}{\partial \alpha} = -K(y - \pi) + \lambda K\alpha, \quad (11.56b)$$

where $1'$ is a row vector of n ones, $y = \{y_i\}$ is a column vector with the n responses y_i , $i = 1, \dots, n$, $\pi = \{\pi_i\}$ is a column vector with the n terms having the form

$$\pi_i = \Pr(y_i = 1|x, \mu, \alpha) = \frac{\exp(f(x_i))}{1 + \exp(f(x_i))},$$

and $K = \{k(x_i, x_j)\}$ is an $n \times n$ Gram matrix whose ij th element is the kernel function $k(x_i, x_j)$. The second derivatives are

$$\begin{aligned} \frac{\partial^2 J}{(\partial \mu)^2} &= \sum_{i=1}^n \pi_i (1 - \pi_i) = 1' W 1, \\ \frac{\partial^2 J}{\partial \alpha \partial \alpha'} &= K W K + \lambda K, \\ \frac{\partial^2 J}{\partial \alpha \partial \mu} &= K W 1. \end{aligned}$$

The matrix of second derivatives can be written

$$H = \begin{bmatrix} 1' W 1 & 1' W K \\ K W 1 & K W K + \lambda K \end{bmatrix},$$

where $W = \text{diag}\{\pi_i(1 - \pi_i)\}$, $i = 1, 2, \dots, n$, an $n \times n$ diagonal matrix. Let

$$Z = [1, K], \text{ of dimension } n \times (n + 1),$$

$$M = \begin{bmatrix} 0 & 0 \\ 0 & K \end{bmatrix}, \text{ of dimension } (n + 1) \times (n + 1),$$

$$\Lambda = \text{diag}\{0, \lambda, \lambda, \dots, \lambda\}, \text{ of dimension } (n + 1) \times (n + 1),$$

$$\theta = (\mu, \alpha')', \text{ of dimension } (n + 1) \times 1.$$

The matrix of second derivatives can be compactly expressed as

$$H = Z' W Z + \Lambda M$$

and the vector of first derivatives as

$$S(\theta) = -Z'(y - \pi) + \Lambda M\theta.$$

The iterative system of the Newton-Raphson algorithm is

$$\begin{aligned}
 \theta_{t+1} &= \theta_t - HS(\theta_t) \\
 &= \theta_t - (Z'WZ + \Lambda M)^{-1} (-Z'(y - \pi) + \Lambda M\theta_t) \\
 &= H^{-1} (Z'WZ + \Lambda M) \theta_t + H^{-1} (Z'WW^{-1}(y - \pi) - \Lambda M\theta_t) \\
 &= H^{-1} Z'W (Z\theta_t) + H^{-1} \Lambda M\theta_t + H^{-1} Z'W (W^{-1}(y - \pi)) - H^{-1} \Lambda M\theta_t \\
 &= H^{-1} Z'W (Z\theta_t + W^{-1}(y - \pi)) \\
 &= H^{-1} Z'W r,
 \end{aligned} \tag{11.57}$$

where $H^{-1} = (Z'WZ + \Lambda M)^{-1}$ and $r = Z\theta_t + W^{-1}(y - \pi)$. This implementation of Newton-Raphson is the *iteratively reweighted ridge regression* algorithm that has a similar structure as (9.19) and at face value requires inversion of the $(n + 1) \times (n + 1)$ matrix of second derivatives.

In connection with the penalised logistic regression model described on page 410, it was indicated that use of the linear kernel $K = XX'$ generates a reparametrised penalised logistic regression model that operates in the $n + 1$ -dimensional space. This kernel leads to a dual representation of the original logistic regression model (9.13), which operates in the $p + 1$ -dimensional space. To see this multiply the second line of (9.19) by $H = (Z'WZ + \Lambda)$

$$(Z'WZ + \Lambda) \theta_{t+1} = (Z'WZ + \Lambda) \theta_t - (-Z'(y - \pi) + \Lambda \theta_t), \tag{11.58}$$

where $Z = [1, X]$ and $\theta' = [\mu, \beta']'$ as in the penalised logistic regression of page 410. This system can be written as

$$\begin{aligned}
 \begin{bmatrix} 1'W1 & 1'WX \\ X'W1 & X'WX + I\lambda \end{bmatrix} \begin{bmatrix} \mu_{t+1} \\ \beta_{t+1} \end{bmatrix} &= \begin{bmatrix} 1'W1 & 1'WX \\ X'W1 & X'WX + I\lambda \end{bmatrix} \begin{bmatrix} \mu_t \\ \beta_t \end{bmatrix} \\
 &\quad + \begin{bmatrix} 1'(y - \pi) \\ X'(y - \pi) - \lambda\beta_t \end{bmatrix}.
 \end{aligned}$$

Expanding yields

$$(1'W1) \mu_{t+1} + (1'WX) \beta_{t+1} = (1'W1) \mu_t + (1'WX) \beta_t + 1'(y - \pi), \tag{11.59}$$

$$\begin{aligned}
 (X'W1) \mu_{t+1} + (X'WX + I\lambda) \beta_{t+1} &= (X'W1) \mu_t + (X'WX + I\lambda) \beta_t \\
 &\quad + X'(y - \pi) - \lambda\beta_t.
 \end{aligned} \tag{11.60}$$

Now perform the following

- Replace $\beta_t = X'\alpha_t$
- Premultiply (11.60) by X and let $K = XX'$

This results in the iterative system

$$\begin{bmatrix} \mu_{t+1} \\ \alpha_{t+1} \end{bmatrix} = \begin{bmatrix} \mu_t \\ \alpha_t \end{bmatrix} + \begin{bmatrix} 1'W1 & 1'WK \\ KW1 & KWK + \lambda K \end{bmatrix}^{-1} \begin{bmatrix} 1'(y - \pi) \\ K(y - \pi) - \lambda K\alpha_t \end{bmatrix}, \quad (11.61)$$

the same as the second line of (11.57). This is a dual representation of the penalised logistic regression model of Sect. 9.4.

Example: Analysis of Binary Observations Using Kernelised Logistic Regression, Penalised Logistic Regression and Logistic Lasso

The performance of the kernelised logistic regression (KLR) using a Gaussian kernel, the penalised logistic regression (PLR) and logistic lasso (LL) is compared for a particular simulated data set consisting of 599 binary records. The binary records were simulated using a logistic model. Genetic marker data from 599 wheat inbred lines genotyped for 1279 genetic markers were downloaded from the statistical package BGLR described in Perez and de los Campos (2014). The simulated “true” model on an underlying scale involves 20 loci randomly sampled from these 1279 markers and assigned as QTL. The substitution effects of these QTL were chosen so that the additive genetic variance between lines σ_a^2 on the underlying scale was equal to 1 squared unit. The underlying liability for line i has the linear structure

$$u_i = m + z_i'b + e_i, \quad i = 1, 2, \dots, 599,$$

where $m = 0.0$, z_i is the column vector for line i of the 20 scaled and standardised QTL genotypes and b is the column vector of the 20 substitution effects. Vector z_i' of dimension 1×20 has (0, 1) elements for each of the two homozygote genotypes of line i and the e_i 's are independently distributed standard logistic random variables. The heritability h^2 between lines on the underlying scale ($\sigma_a^2 / (\sigma_a^2 + \pi^2/3)$) is equal to 0.23 and the proportion of 1's in the data is approximately 0.5.

The three operational models used to analyse the data assumed that the liability can be written in terms of the linear structure:

$$u_i = \mu + x_i'\beta + \varepsilon_i, \quad i = 1, 2, \dots, 599,$$

where x'_i is the 1×1279 row vector for line i of the 1279 marker genotypes, β is the vector with the 1279 marker effects and the ε'_i 's are independently distributed standard logistic random variables.

The 599 lines were divided into training and validating sets of sizes 299 and 300, respectively. The logistic regression model using the 1279 markers was fitted to the training data using KLR, PLR and LL and the estimates of the marker effects were used to predict the binary phenotypes of the validating dataset. The package **glmnet** described in Example 7.4 was used to fit the logistic lasso.

The criterion used to evaluate prediction ability was the proportion of misclassifications in the validating data or error rate, given by

$$\frac{1}{N_v} \sum_{i=1}^{N_v} (y_i - \hat{y}_i)^2 \quad (11.62)$$

where $N_v = 300$ is the number of records in the validating set, y_i is the i th record in the validating set (0 or 1) and $\hat{y}_i$ is the predicted value computed using (9.3) with $t = 0.5$.

The average proportion of misclassifications (min, max) over 10 random replicates of training/testing samples was 0.33 (0.30, 0.37) for LL, 0.40 (0.38, 0.43) for PLR and 0.33 (0.28, 0.37) for KLR. In principle, LL has the advantage of performing model selection. However, in this particular data set, the number of covariates not set to zero was highly variable across replicates ranging from 1 to 52 with an average of 14. Therefore little inferential meaning can be assigned to the choice of covariate.

The R-code to fit the KLR to the wheat data set using the Newton-Raphson algorithm is shown below.

```
# CODE1109
# SIMULATING BINARY DATA TO BE ANALYSED WITH KLR
# USES THE X MATRIX FROM WHEAT IN BGLR
rm(list=ls()) # CLEAR WORKSPACE
library(BGLR)
data(wheat)
X <- wheat.X
set.seed(371)
#####
### USE BGLR MATRIX X
nindiv <- nrow(X)
nmark <- ncol(X)
#####
nloci <- 20
p <- 0.5
mu <- log(p/(1-p))

##### GENERATE LIABILITY #####
va <- 1.0 # additive variance of liability
Xc <- matrix(data=NA, nrow=nindiv, ncol=nmark)
```

```

# parameter from true model:
be<-matrix(data=0.0,nrow=nmark,ncol=1)
y<-rep(0,nindiv)
cm<-colMeans(X)
#### CENTER AND SCALE X #####
for (i in 1:nmark)
{Xc[,i]<- (X[,i]-cm[i])/sd(X[,i])
}
QTLeff<-sqrt(va/nloci)# calculate the QTL effect so that the
# total genetic variance is VA
IDq<-sample(1:nmark,nloci,replace=F) # from the nmark markers,
# choose nloci as QTL
be[IDq]<-QTLeff # the only b's that are not zero are those
# associated with QTL.
##### GENERATE PHENOTYPIC BINARY DATA #####
xb<-Xc*%be
pr <- exp(mu+xb)/(1+exp(mu+xb))
y <- rbinom(nindiv,1,pr)
#sum(y)/length(y) # OBSERVED PROPORTION OF 1'S IN SAMPLE
mean(y)

```

```
## [1] 0.5025042
```

```

nitnr <- 10 # NUMBER OF N-R ITERATIONS
nrep <- 10 # NUMBER OF TRAINING / TESTING REPLICATES

#lambda <- 0.0 # ZERO PENALTY !!!!!!!
lambda <- 0.4

newcostvnr <- rep(0,nrep)
res <- matrix(data=NA, nrow=nrep,ncol=9)
resulttvnr <- matrix(data=NA, nrow=nitnr,ncol=8)

msev <- rep(NA,nrep)
msevnr <- rep(NA,nrep)

#### A GAUSSIAN KERNEL #####
kgaus <- function(X,h){
  X <- scale(X,center=TRUE,scale=FALSE)
  S=sqrt(sum(apply(FUN=var, X=X,MARGIN=2)))
  X <- X/S
  D <- as.matrix(dist(X))^2
  K <- exp(-h*D)
}
##### CHOOSE KERNEL #####
h <- 0.5
K <- kgaus(Xc,h)
#dim(K)
#qr(K)$rank
#####
probl <- function(miu,alfa,K){
  pr <- exp(miu+K*%alfa)/(1+exp(miu+K*%alfa))
}

```

```

cost <- function(miu,alfa,K,y){-sum(y*(miu+K%%alfa) -
  log(1 + exp(miu+K%%alfa))) + lambda*crossprod(alfa)}

##### NEWTON-RAPHSON #####
### FIT MODEL TO TRAINING DATA AND TEST IN VALIDATING DATA ###
set.seed(77131111)
ptm <- proc.time()
for (i in 1:nrep) {
#   cat(i, "\n",sep="")
  train=sample(1:nrow(K),floor(0.5*nrow(K)))
  Ktrain <- K[train,train]
  Kval <- K[-train,train]
  ytrain <- y[train]
  yval <- y[-train]

  delta <- diag(c(0,rep(lambda,ncol(Ktrain))))
  M <- cbind(0,Ktrain)
  M <- rbind(0,M)
  ##### START VALUES FOR MIU AND ALFA #####
  miu <- 0.0
  alfa <- rep(0.0, ncol(Ktrain))
  W <- matrix(data = 0,nrow = ncol(Ktrain),ncol = ncol(Ktrain))
  for (j in 1:nitr) {
    fdmiu <- -sum(ytrain - prob1(miu, alfa, Ktrain))
    fdalfa <-
      -Ktrain %*% (ytrain - prob1(miu, alfa, Ktrain)) +
      lambda * Ktrain %*% alfa
    fd <- matrix(c(fdmiu, fdalfa), nrow = length(alfa) +
      1, ncol = 1)
    W <-
      diag(c(prob1(miu, alfa, Ktrain) *
        (1 - prob1(miu, alfa, Ktrain))))
    Z <- cbind(1, Ktrain)
    zwz <- t(Z) %*% W %*% Z
    LHS <- zwz + lambda * M
    RHS <- -fd
    sol0 <- matrix(c(miu, alfa), nrow =
      length(alfa) + 1, ncol = 1)
    sol1 <- sol0 + solve(LHS,RHS)
    miu <- sol1[1, 1]
    alfa <- sol1[-1, 1]
    newcostvnr[j] <- cost(miu, alfa, Ktrain, ytrain)
    resulttvnr[j, ] <- c(j, newcostvnr[j], miu, alfa[1:5])
  }
  probval <- prob1(miu, alfa, Kval)
  y_predval <- as.numeric(ifelse(probval > 0.5, 1, 0))
  msev[i] <- mean((y_predval - yval) ^ 2)
  res[i,] <- c(i,j,newcostvnr[j],miu,alfa[1:5])
}
proc.time()-ptm

```

```

##      user  system elapsed
##      2.11    0.11    0.58

```

```
tail(resulttvnr[,2:6])
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [5,] 236.2372 0.1062297 -0.9584729 0.685112 0.8690137
## [6,] 236.2372 0.1062297 -0.9584729 0.685112 0.8690137
## [7,] 236.2372 0.1062297 -0.9584729 0.685112 0.8690137
## [8,] 236.2372 0.1062297 -0.9584729 0.685112 0.8690137
## [9,] 236.2372 0.1062297 -0.9584729 0.685112 0.8690137
## [10,] 236.2372 0.1062297 -0.9584729 0.685112 0.8690137
```

```
#tail(res)
summary(msev)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.2833  0.3233   0.3283   0.3310  0.3442   0.3700
```

The program outputs the last 6 iterations of the Newton-Raphson algorithm, replicate 10, for the value of the cost function, μ and the estimates of first 3 elements of α . The `summary` function at the end displays summary statistics of the misclassification rate over the 10 replicates.

The R-code for execution of the penalised logistic regression and the logistic lasso is CODE0901.R; it is found at <https://github.com/SorensenD/SLGDS> in the folder Codes.

11.3 Neural Networks

Neural networks (NN) have a long history (McCulloch and Pitts 1943) and have become the state of the art technique for many machine learning problems involving complex data. The subject has been rebranded along its development and presently it is also known as *machine learning* or *deep learning* and it is being successfully applied in many software areas including object, image and speech processing, robotics, video games and search engines. This section provides a very basic introduction to the subject concentrating exclusively on fully connected NN, also known as multilayer perceptrons (MLP) and illustrates via examples, how they can be used for classification and regression. MLP or fully connected networks are the oldest and classic neural architecture invented in the 1960s. MLP are used today as components of other neural network architectures, such as convolutional neural networks, transformers, Bayesian neural networks, variational autoencoders and others. Useful references are the books by Bishop (2006), Goodfellow et al. (2016) and Bernard (2021) and a review by LeCun et al. (2015), where more modern networks are discussed. A very comprehensive reference is López et al. (2022) that also provides detailed descriptions and implementation of many methods used for genomic prediction with special emphasis on plant breeding. An early Bayesian perspective on the subject can be found in Neal (1996) and a recent neural network Bayesian MCMC implementation in Zhao et al. (2021).

The subject is motivated by interpreting logistic regression as a special case of a fully connected NN.

Preliminaries: A Logistic Regression

Consider binary observations y and features x that constitute training records $\{(x_1, y_1), \dots, (x_n, y_n)\}$, $x_i \in \mathbb{R}^p$, $y_i \in \{0, 1\}$ and assume the model adopted is

$$E(y_i | z_i) = \Pr(y_i = 1 | z_i) = m(x_i) = \frac{\exp(z_i)}{1 + \exp(z_i)}. \quad (11.63)$$

The linear transformation $z_i = w'x_i + b$ consists of the input: vector x with p features (covariates), parameters: w (a $(p \times 1)$ vector of unknown regression coefficients) and b (a scalar intercept). In machine learning the parameters w are known as *weights*, the intercept b as the *bias* and the training records as *training examples*. Both w and b are unknown.

In Chap. 3 maximum likelihood estimates of the parameters of the logistic model w and b were obtained using the Newton-Raphson and the EM algorithms. Chapter 5 discusses a Bayesian implementation. Here the problem is approached regarding the logistic model as a special case of a NN and the cost function (negative of the loglikelihood, a convex function) is minimised using a gradient descent algorithm. In what follows, unless otherwise stated, the subscript i identifying the i th training record will be omitted.

Figure 11.7 displays a logistic regression model represented as a neural network. The x 's are the $p = 2$ features (covariates). These constitute the first layer. A linear function $z^{(2)}$ of the features and parameters is fed into what is known as *neuron* or *node* (the superscript denotes the layer). The neuron is activated by an *activation function* σ , that results in an output of layer 2, $a^{(2)}$, known as an *activation unit*. For the logistic regression the activation function is the logistic or *sigmoid function* σ (11.63)

$$\sigma(z^{(2)}) = \frac{\exp(z^{(2)})}{1 + \exp(z^{(2)})} = \frac{1}{1 + \exp(-z^{(2)})}, \quad (11.64)$$

where the linear function $z^{(2)} = w'x + b$. In Fig. 11.7 there are 2 layers: an input layer (layer 1), where the activation unit is just the input, $a_1^{(1)} = x_1$, $a_2^{(1)} = x_2$, and a layer 2 that includes a single *activation unit* $a^{(2)} = \sigma(z^{(2)}) = \tilde{y}$, the output. The output is here a predicted or estimated value of the probability $m(x_i)$ in (11.63). With some abuse of notation, this output will also be denoted by $h(x)$ or $h_\theta(x)$, where θ is a vector of parameters of the NN and x represents the vector of covariates.

In linear models the regression function takes the form $m(x) = w'x$. Kernels introduce nonlinear functions of the features x , via basis functions $\phi(x)$ that

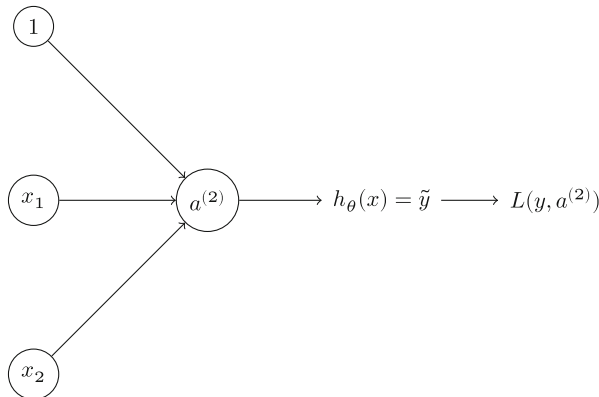


Fig. 11.7 Logistic regression model represented as a 2-layered neural network for a single training record (y, x_1, x_2) . The first layer (outer circles) corresponds to the input: $p = 2$ covariates x_j , $j = 1, 2$, plus 1 for the intercept. A linear transformation of the input, $z^{(2)} = w'x + b$, where $w' = (w_1, w_2)$, is fed into a neuron on the second layer. The neuron is activated by the activation function σ (here, the sigmoid function), producing an output $a^{(2)}$, the activation unit $a^{(2)} = \sigma(z^{(2)})$. In this simple example of a shallow network, the output or prediction $h_\theta(x) = \tilde{y}$ is equal to $a^{(2)}$. This output is evaluated using the loss function L that indicates how well $\tilde{y} = a^{(2)}$ compares with the observation y

could capture more complex structures leading to $m(x) = w'\phi(x)$. Kernel methods require the user to supply the form of the kernel function. In the present representation of logistic regression as a NN, $\phi(x) = \sigma(w'x + b)$, where w and b are estimated from the training data and σ is a given nonlinear function such as the sigmoid. If σ is the identity function the neural network reduces to a linear model.

Training the Neural Network

Training the NN in Fig. 11.7 requires first, a *forward propagation* step that runs from left to right. It starts by feeding the features x and the parameters (w, b) , via a linear transformation, into a neuron that gets activated by the activation function σ generating a prediction or output $a^{(2)} = h(x) = \tilde{y}$ (the notation distinguishes the regression function or conditional expectation of the data m , from the output of a forward propagation step that results in a prediction $h = \tilde{y}$). Secondly, a *back propagation* step (Rumelhart et al. 1986) is used to compute the gradient of the cost function with respect to all parameters of the neural network. The back propagation step involves the chain rule of calculus. Once gradients are computed the cost function can be minimised using for example, gradient descent or stochastic gradient descent, yielding estimates of parameters. This is an iterative process which on completion, outputs a prediction.

Notation

The simple NN of Fig. 11.7 can be made more complex by increasing the number of layers and the number of neurons per layer. Identification of parameters in such a complex system requires a more involved notation. In general, let

- n_ℓ : number of neurons in layer ℓ . For the input layer, $\ell = 1$, $n_1 = p$, the number of features in the input datum
- $a_i^{(j)}$: activation unit i (or output produced by the activation function) in layer j . For the input layer $j = 1$, there is no activation function and the notation is interpreted as $a_i^{(1)} = x_i$
- $a^{(j)}$: layer j . This is a column vector with number of elements equal to the number of neurons in layer j
- $W^{(j)}$: matrix of weights (parameters) of the function that controls the mapping from layer j to layer $j + 1$. The number of rows of $W^{(j)}$ is equal to the number of neurons in layer $j + 1$, and the number of columns is equal to the number of neurons in layer j . If a network has n_j neurons in layer j and n_{j+1} neurons in layer $j + 1$, $W^{(j)}$ is of dimension $n_{j+1} \times n_j$
- $b^{(j)}$: column vector of intercept terms of the function that controls the mapping from layer j to layer $j + 1$, with number of elements equal to the number of neurons in layer $j + 1$

Forward Propagation

The logistic regression example displayed in Fig. 11.7 considers a single training datum consisting of the two covariates (x_1, x_2) and the observation y . A scheme of forward propagation is displayed in Fig. 11.8.

The variable $z^{(2)}$ is a linear function of the input features (the two covariates $x_1 = a_1^{(1)}, x_2 = a_2^{(1)}$) and is fed into the activation function σ , which generates

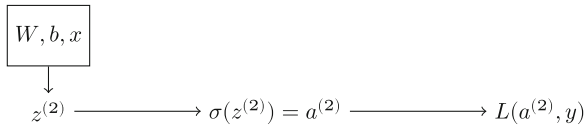


Fig. 11.8 Forward propagation scheme for the logistic regression model interpreted as a neural network. The linear function of parameters W, b and feature vector $x = a^{(1)}$ give rise to $z^{(2)}$ that is fed into the activation function σ (the logistic function). This activation function outputs the prediction $a^{(2)} = \tilde{y}$. The loss function L quantifies the discrepancy between $a^{(2)}$ and the observed record y

the output $a^{(2)} = \tilde{y}$ (a predicted probability). The *forward propagation* step can be written as

$$z^{(2)} = W_{11}^{(1)} a_1^{(1)} + W_{12}^{(1)} a_2^{(1)} + b^{(1)}, \quad (11.65a)$$

$$a^{(2)} = \sigma(z^{(2)}) = \tilde{y}. \quad (11.65b)$$

The dimension of $W^{(1)} = (W_{11}^{(1)}, W_{12}^{(1)})$ is 1×2 because this network has 2 features in layer 1 (i.e., x_1 and x_2) and 1 single neuron in layer 2 (the output $a^{(2)} = \tilde{y}$). The parameters $W^{(1)}$ and $b^{(1)}$ must be initialised to start the iteration.

Loss Function

The loss function L measures the degree to which the output $\tilde{y}$ matches the observation y (following the machine learning tradition, here *cost function* is a measure involving an entire dataset—that may or may not include a regularisation term; *loss function* involves a single datum). The overall objective is to minimise the cost function by minimising the sum of the loss functions for each datum. This requires computation of the gradients $\partial L / \partial W^{(1)}$, $\partial L / \partial b^{(1)}$, that can be accomplished using back propagation. The present simple example provides the background for dealing with more complex situations.

Back propagation is an application of the chain rule of calculus applied in a computationally efficient manner. This efficiency is achieved in part by re-using derivatives computed for higher layers of the network, in the computation of derivatives of the lower layers.

The contribution of the datum (y, x) to the (convex) loss function (i.e., the negative of the loglikelihood—see page 403) to be minimised with respect to (W, b) follows from the Bernoulli distribution and is

$$L(\tilde{y}, y) = -(y \log \tilde{y} + (1 - y) \log (1 - \tilde{y})). \quad (11.66)$$

An intuition for the loss function is as follows. For $y = 1$, when the predicted probability (the output of the forward propagation step) is $a^{(2)} = \tilde{y} = 1$, the loss function is zero and when the predicted probability is $\tilde{y} = 0$, the loss function is infinity. Similarly, for $y = 0$, when $\tilde{y} = 0$ the loss function is zero and when $\tilde{y} = 1$ the loss function is infinity.

Although hidden from the notation, L depends not only on y but also on the input

$$(b^{(1)}, W_{11}^{(1)}, W_{12}^{(1)}, a_1^{(1)}, a_2^{(1)}).$$

Some of this input, $(a_1^{(1)}, a_2^{(1)})$, is given (in this simple example, $a_i^{(1)} = x_i$), but the parameters $(b^{(1)}, W_{11}^{(1)}, W_{12}^{(1)})$ are not observed and must be initialised to begin iteration.

Chain Rule

Before deriving the back propagation algorithm for the simple example of Fig. 11.7, I consider a general application of the chain rule of calculus. Figure 11.9 shows a flowchart where input $a \in \mathbb{R}^p$ is fed into a function g that outputs $b \in \mathbb{R}^n$. In turn, b is fed into a function f that outputs $c \in \mathbb{R}^m$. Specifically,

$$\begin{aligned} a &= (a_1, a_2, \dots, a_p), \\ b &= g(a) = (b_1, b_2, \dots, b_n), \\ c &= f(b) = (c_1, c_2, \dots, c_m). \end{aligned}$$

The function g performs a mapping from $\mathbb{R}^p$ to $\mathbb{R}^n$ and the function f a mapping from $\mathbb{R}^n$ to $\mathbb{R}^m$. Consider the computation of the change in c_i due to a change in a_j . A change in a_j may induce a change in $(b_1, b_2, \dots, b_n)$, which in turn induces a change in each c_i . The chain rule states that to first order, the net change in c_i due to a change in a_j is given by the sum of the changes induced along each path from a_j to c_i . Letting k denote one of the m paths,

$$\frac{\partial c_i}{\partial a_j} = \sum_{k=1}^n \frac{\partial c_i}{\partial b_k} \frac{\partial b_k}{\partial a_j}. \quad (11.67)$$

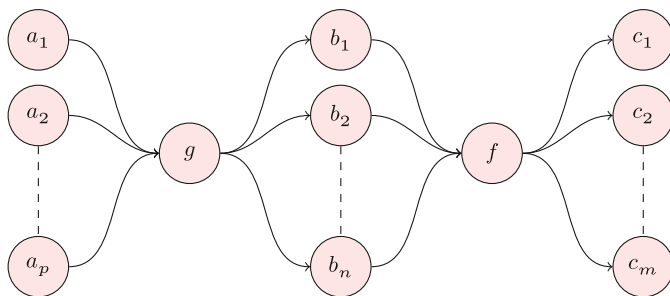


Fig. 11.9 Flowchart where input a is fed into a function g that outputs b . This output is fed into a function f that outputs c

Back Propagation

The back propagation step computes the gradient of the cost function with respect to the parameters of the NN. Once the gradient is available, the cost function can be minimised using gradient descent.

Minimisation of the cost function involves minimising the sum of the loss functions for each datum (11.66). Using (11.65), this takes the form (see also Fig. 11.8)

$$\begin{aligned}\frac{\partial L}{\partial a^{(2)}} &: (1 \times 1), \\ \frac{\partial L}{\partial z^{(2)}} &= \frac{\partial L}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} : (1 \times 1), \\ \frac{\partial L}{\partial W^{(1)}} &= \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial W^{(1)}} : (2 \times 1), \\ \frac{\partial L}{\partial b^{(1)}} &= \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial b^{(1)}} : (1 \times 1).\end{aligned}$$

More specifically,

1. For the output $a^{(2)} = \tilde{y}$ in layer 2 compute

$$\begin{aligned}\delta^{(2)} &= \frac{\partial L}{\partial z^{(2)}} = \frac{\partial L}{\partial a^{(2)}} \frac{da^{(2)}}{dz^{(2)}} \\ &= \frac{\partial L}{\partial a^{(2)}} \sigma' \left(z^{(2)} \right).\end{aligned}\tag{11.68}$$

2. The partial derivatives with respect to $W_{11}^{(1)}, W_{12}^{(1)}$ are

$$\begin{aligned}\frac{\partial L}{\partial W_{11}^{(1)}} &= \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial W_{11}^{(1)}} \\ &= \delta^{(2)} a_1^{(1)},\end{aligned}\tag{11.69}$$

$$\begin{aligned}\frac{\partial L}{\partial W_{12}^{(1)}} &= \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial W_{12}^{(1)}} \\ &= \delta^{(2)} a_2^{(1)},\end{aligned}\tag{11.70}$$

$$\begin{aligned}\frac{\partial L}{\partial b^{(1)}} &= \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial b^{(1)}} \\ &= \delta^{(2)}.\end{aligned}\tag{11.71}$$

3. For the sigmoid function in (11.68)

$$\frac{da^{(2)}}{dz^{(2)}} = \sigma'(z^{(2)}) = \sigma(z^{(2)}) (1 - \sigma(z^{(2)})) = a^{(2)} (1 - a^{(2)}),$$

and

$$\frac{\partial L}{\partial a^{(2)}} = -\frac{y}{a^{(2)}} + \frac{1-y}{1-a^{(2)}}. \quad (11.72)$$

Therefore, substituting in (11.68),

$$\delta^{(2)} = a^{(2)} - y. \quad (11.73)$$

Cost Function

The above computations and the loss function (11.66) pertain to a single training record. For n records the *cost function* is

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L(\tilde{y}_i, y_i) = -\frac{1}{n} \sum_{i=1}^n (y_i - \log \tilde{y}_i + (1 - y_i) \log (1 - \tilde{y}_i)), \quad (11.74)$$

an average of the loss functions over the n records (scaling by $1/n$ does not change the optimisation of J but avoids numerical problems during the computations). In this equation θ is the vector of parameters $(W^{(1)}, b^{(1)})$. Minimisation of the cost function requires computation of the gradients with respect to $(W^{(1)}, b^{(1)})$. These are the average of the gradients of each datum calculated using (11.69), (11.70) and (11.71). For example, for $W_{11}^{(1)}$,

$$\frac{\partial J(\theta)}{\partial W_{11}^{(1)}} = \frac{1}{n} \sum_{i=1}^n \frac{\partial L(\tilde{y}_i, y_i)}{\partial W_{11}^{(1)}},$$

where now $\partial L(\tilde{y}_i, y_i) / \partial W_{11}^{(1)}$ is the partial derivative of the loss function for the i th record with respect to $W_{11}^{(1)}$.

The general form of a gradient descent update is given by expression (3.57) on page 105. In the present case, once the partial derivatives are available, the gradient descent update is

$$W_{11}^{(1)} := W_{11}^{(1)} - \alpha \frac{\partial J}{\partial W_{11}^{(1)}},$$

$$W_{12}^{(1)} := W_{12}^{(1)} - \alpha \frac{\partial J}{\partial W_{12}^{(1)}},$$

$$b^{(1)} := b^{(1)} - \alpha \frac{\partial J}{\partial b^{(1)}},$$

for a user chosen value of the learning rate α .

A Single Hidden Layer Neural Network

Figure 11.10 shows an example of a neural network with three layers: the input layer showing a single training example, a hidden layer (absent in the previous

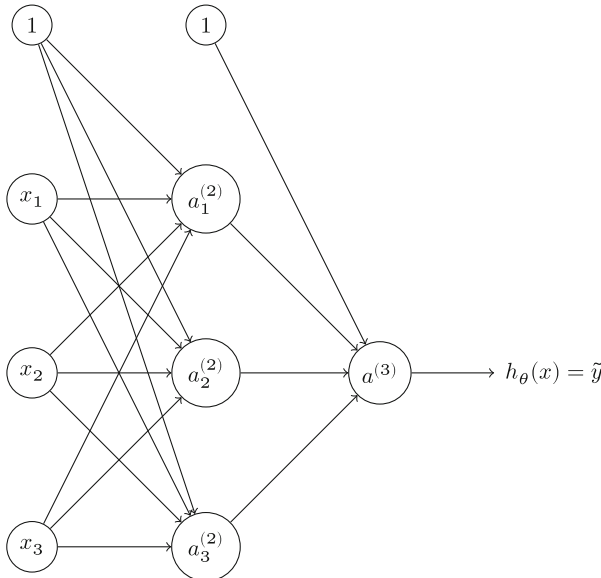


Fig. 11.10 A neural network diagram with three layers: an input layer representing the feature vector with three predictors $x = (x_1, x_2, x_3)$, with 1 to account for the bias term; a hidden layer with 3 nodes representing activation units ($a_1^{(2)}$, $a_2^{(2)}$, $a_3^{(2)}$), with the extra node 1 to account for the bias term and an output layer $a^{(3)} = h_{\theta}(x) = \tilde{y}$ that yields a prediction

example) and an output layer with a single output node. The input layer has 3 nodes $(x_i, i = 1, 2, 3)$ plus an extra node 1 for the bias term. The hidden layer has also 3 nodes representing activation units $(a_i^{(2)}, i = 1, 2, 3)$, plus a node 1 to account for the bias. Finally, the output layer has a single node $a^{(3)}$, the prediction. For this example the hidden layer is generated with the sigmoid function, but other activation functions can be used instead, as discussed on page 564.

The single node output layer applied here is appropriate for a regression model where the output is a real number, or for a binary (1/0) outcome. Here the latter is assumed and the sigmoid function (11.64) is used in the generation of the final layer. For a continuous variable the output is typically generated using the identity function. For a K class classification the final output can be generated using the *softmax* function

$$a_k^{(L)} = g(z_k) = \frac{\exp(z_k)}{\sum_{k=1}^{K} \exp(z_k)}, \quad (11.75)$$

also used for a multilogit model and interpreted as a probability. In this expression L denotes the number of layers in the network, so that $a_k^{(L)}$ is the k th node of the last layer corresponding to class k . The number of nodes in the final layer is K and the sum of terms (11.75) equals 1.

Reverting to the NN of Fig. 11.10 and following the notation defined above, consider the forward propagation step. For a single training datum the step can be represented by the equations below where input predictors x output the scalar prediction $\tilde{y}$,

$$\begin{aligned} a^{(1)} &= (a_1^{(1)}, a_2^{(1)}, a_3^{(1)})' = (x_1, x_2, x_3)', \\ z_1^{(2)} &= W_{11}^{(1)} a_1^{(1)} + W_{12}^{(1)} a_2^{(1)} + W_{13}^{(1)} a_3^{(1)} + b_1^{(1)}, \\ z_2^{(2)} &= W_{21}^{(1)} a_1^{(1)} + W_{22}^{(1)} a_2^{(1)} + W_{23}^{(1)} a_3^{(1)} + b_2^{(1)}, \\ z_3^{(2)} &= W_{31}^{(1)} a_1^{(1)} + W_{32}^{(1)} a_2^{(1)} + W_{33}^{(1)} a_3^{(1)} + b_3^{(1)}, \\ a_i^{(2)} &= \sigma(z_i^{(2)}), \quad i = 1, 2, 3, \\ z^{(3)} &= W_{11}^{(2)} a_1^{(2)} + W_{12}^{(2)} a_2^{(2)} + W_{13}^{(2)} a_3^{(2)} + b_1^{(2)}, \\ a^{(3)} &= \sigma(z^{(3)}) = \tilde{y}. \end{aligned} \quad (11.76)$$

The forward propagation can be written more compactly in vectorised form along the lines in (11.65), yielding

$$\begin{aligned}
 z^{(2)} &= W^{(1)}a^{(1)} + b^{(1)}, & z^{(2)} &= [z_1^{(2)}, z_2^{(2)}, z_3^{(2)}]', \\
 W^{(1)} &: (3 \times 3), & a^{(1)} &: (3 \times 1), & b^{(1)} &: (3 \times 1), \\
 a^{(2)} &= \sigma(z^{(2)}), & a^{(2)} &: (3 \times 1), \\
 z^{(3)} &= W^{(2)}a^{(2)} + b^{(2)}, \\
 W^{(2)} &: (1 \times 3), & a^{(2)} &: (3 \times 1), & b^{(2)} &: (1 \times 1), \\
 a^{(3)} &= \sigma(z^{(3)}), & a^{(3)} &: (1 \times 1).
 \end{aligned} \tag{11.77}$$

The logistic regression model was interpreted as a two-layered neural network of the form $f(x) = \Pr(Y = y|x)$. The three-layered neural network represented by the system (11.77) can be written

$$f(x) = f^{(2)}\left(f^{(1)}(x)\right),$$

with a straightforward extension for a deeper layered network. Above, $f^{(1)}$ is the activation function from the first layer and $f^{(2)}$ is the activation function of the second layer. Each of these functions is indexed by parameters that are learned by the network in order to output/predict a value as close as possible to the observed record. The effectiveness with which this is achieved is measured by the cost function.

From a prediction perspective, closeness between predictions and outputs may be the result of overfitting. This suggests the need for regularisation.

The Cost Function

The cost function (11.74) for the logistic regression is a convex function easy to minimise. In the example there are p parameters and n data points and when $n \gg p$ there are no issues with the computation. Typically neural networks are very heavily parametrised and prone to overfitting. One way to avoid overfitting is to include a regularisation term as part of the cost function.

Consider a neural network with m training records $(x_1, y_1), \dots, (x_m, y_m)$ and assume an output with K classes, so that the output $h_{W,b}(x) \in \mathbb{R}^K$ (with binary

outcomes $K = 1$). Let $(h_{W,b}(x^{(i)}))_k$ be the k th output corresponding to datum i . The cost function is

$$\begin{aligned}
 J(\theta) = & -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(h_{W,b}(x^{(i)}) \right)_k \right. \\
 & \left. + \left(1 - y_k^{(i)} \right) \log \left(1 - \left(h_{W,b}(x^{(i)}) \right)_k \right) \right] \\
 & + \frac{\lambda}{2} \sum_{l=1}^{L-1} \sum_{i=1}^{n_l} \sum_{j=1}^{n_{l+1}} \left(W_{ji}^{(l)} \right)^2,
 \end{aligned} \tag{11.78}$$

where θ represents the parameters of the neural network. In this expression, L is the number of layers, n_l is the number of nodes in layer l , n_{l+1} the number of nodes in layer $l + 1$ and λ is the regularisation or tuning parameter. The bias term is not regularised. Expression (11.78) features the quadratic penalisation, but other penalties such as the lasso penalty are also used.

Contrary to what has been the case so far, the cost function (11.78) and of most neural networks is not convex but typically has multiple optima. Global minimisation of such a multidimensional surface is an insurmountable task. Notwithstanding, finding near local optima is not difficult. Recent theoretical and empirical results appear to indicate that, in general, the many local optima found using mini-batch gradient descent or stochastic gradient descent do not constitute an issue in neural network optimisation (LeCun et al. 2015).

Minimisation of the cost function requires the gradients $\partial J(\theta) / \partial W_{ji}^{(l)}$, $\partial J(\theta) / \partial b_j^{(l)}$. This is achieved in a computationally efficient manner using back propagation along the same lines indicated for the simple logistic regression model. Details are a little laborious and are relegated to an Appendix on page 597. The Appendix also discusses vectorisation of back propagation to optimise matrix multiplications and provides an example of back propagation in the presence of multiple connected paths.

Activation Functions

Activation functions are a fundamental building block of neural networks. They turn the neural network model into a highly nonlinear function of the input variables whose parameters are learned as the network is implemented. It is easy to see that use of an identity function (rather than of a nonlinear activation function) in the

hidden layer and output layer of the neural network of Fig. 11.10 transforms the neural network into a linear model. Indeed, setting in (11.77)

$$\begin{aligned}
 a^{(2)} &= z^{(2)} = W^{(1)}a^{(1)} + b^{(1)} \\
 a^{(3)} &= W^{(2)}a^{(2)} + b^{(2)} \\
 &= W^{(2)} \left[W^{(1)}a^{(1)} + b^{(1)} \right] + b^{(2)} \\
 &= W^{(2)}W^{(1)}a^{(1)} + \left(W^{(2)}b^{(1)} + b^{(2)} \right) \\
 &= \tilde{W}a^{(1)} + \tilde{b},
 \end{aligned} \tag{11.79}$$

transforms the network into a linear function of the input $a^{(1)}$. This lacks the ability to extract nonlinear patterns from the input data.

The examples discussed so far included as activation functions the sigmoid function, the softmax function and the identity function. The sigmoid function can be used in the hidden layers of a neural network and is used in the outer layer for binary classifiers. It transforms the input to values in $(0, 1)$. For multiclass problems the outer layer uses the softmax function and for regression the outer layer uses the identity function.

Another activation function is the tanh function or hyperbolic function

$$\tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)},$$

that outputs values in $(-1, 1)$. The range of values of z generated by this function has the same effect as standardizing the output. This often leads to better behaviour of the neural network relative to that of the sigmoid function. The derivative is

$$\frac{\partial}{\partial z} \tanh(z) = 1 - (\tanh(z))^2.$$

One disadvantage of the sigmoid function and of the tanh function is that if z becomes a very large positive number or a very small negative number, the derivatives are very close to zero. This can slow down gradient descent. An activation function that does not suffer from this property is the ReLU (rectified linear unit) function that outputs positive values. It takes the form

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} 0, & z \leq 0, \\ z, & z > 0, \end{cases}$$

with derivative

$$\frac{d\text{ReLU}(z)}{dz} = I(z > 0).$$

This derivative does not exist at $z = 0$, but a computer implementation is not sensitive to this since numerically z is never exactly equal to zero. Despite the fact that the derivative is close to zero when $z < 0$, there are enough z values larger than zero so that gradient descent moves at an acceptable pace. A variant is the leaky ReLU

$$\text{leaky ReLU}(z) = \max(cz, z),$$

where c is a small number (for example, $c = 0.01$) with derivative

$$\frac{\partial \text{leaky ReLU}(z)}{\partial z} = \begin{cases} 1, & z > 0, \\ c, & z < 0, \end{cases}$$

so the derivative, numerically speaking, is never zero.

In modern neural networks hidden layers often use the ReLU or the leaky ReLU functions rather than the sigmoid function because they often lead to better numerical behaviour and are cheaper to compute. Several other activation functions can be found in the literature but the ones mentioned here are the most commonly used.

Remarks on Fitting Neural Networks

Neural networks are characterised by having many parameters: the weights and biases (the current version of ChatGPT, GPT-3, is a giant neural network with 175 billion parameters (Wolfram 2023)), the choice of activation functions, the regularisation parameter, the learning rate, the number of hidden layers and of neurons per layer. For good performance neural networks require fine tuning of these parameters. This calls for considerable experimentation and expertise. In addition, back propagation, a central operation of the NN, can become very challenging in real-world implementations and is an area with many subtleties. The following is a minimal list of issues that require attention.

- Standardising the features/covariates to mean zero and variance one is particularly advisable when the inputs have different distributions. The standardisation has the effect of speeding up gradient descent and simplifies computations since a common learning rate can be applied.
- Starting values of covariates, parameters and regularisation parameter. The range of values of the covariates has a direct effect on the values of the parameters. Standardisation of the covariates facilitates initialising the parameters; a general advice is to choose starting values at random in the proximity of zero (for example, random uniform weights over the range $[-0.7, +0.7]$ (Hastie et al. 2009), or smaller). Weights (parameters) are typically initialised drawing randomly from Gaussian or uniform distributions. The choice of scale of the initial distribution

may require considerable experimentation. One suggestion for initialising $W^{(\ell)}$ is to use $Un(-k, k)$, $k = 1/\sqrt{n_\ell}$, or $N(0, 1) * 1/\sqrt{n_\ell}$ (where n_ℓ is the number of columns of $W^{(\ell)}$) and if a ReLU activation function, is used $N(0, 1) * 2/\sqrt{n_\ell}$ (Hu et al. 2021). It is important not to set the initial value of the W 's exactly equal to zero. If this is the case, the activation units as well as the δ terms of each layer take the same values and this symmetry cannot be broken. This creates a redundancy (irrespective of the number of neurons per layer, this would be equivalent to a network with only one neuron per layer since all neurons take the same value) and poor behaviour of the neural network. Judicious choice of starting values for the parameters of a neural network reduces the problem of exploding or vanishing gradients that may arise in deep neural networks (Glorot and Bengio 2010). The value of the regularisation parameter is very influential, with larger values of λ leading to smaller values of the parameters tending to reduce overfitting but to increase bias. An interesting case arises in connection with the use of tanh or sigmoid. Small weights lead to small values of z . Within a fairly broad range of values of z the relationship between tanh or sigmoid with z is almost linear. In this situation the neural network reduces to a linear model (see (11.79)) and may perform poorly if there are complex nonlinearities in the data

- Number of hidden layers/number of neurons per layer. For any given problem it is difficult to know in advance how deep (how many layers) a neural network must be. The number of hidden layers is best considered as a hyperparameter to be determined via experimentation and cross-validation. The number of neurons per layer is also a model feature to be determined by experimentation. It is common to use in the order of tens or hundreds of neurons, with this number increasing with the number of features. Shallow (few layers) neural networks would typically require more neurons per layer in order to fit the same function.
- Tuning gradient descent. Depending on the size of the data set, gradient descent can be applied to the complete data (*batch gradient descent*), or to mini-batches of the data (*mini-batch gradient descent*). For datasets of the order of say up to 3000 features, batch gradient descent should be a reasonable choice. For larger sizes the features can be divided into mini-batches of size 1024, or 512 features. Within each mini-batch, a single step of gradient descent is computed (as if the whole training set consisted of this unique mini-batch). This requires computation of forward propagation, computation of the cost function, computation of back propagation to obtain the gradients of the cost function with respect to the parameters and finally updating of parameters. This completes the processing of the mini-batch. The calculations are repeated for each mini-batch, until the complete training data have been processed leading to a single pass of the complete training set. The cost function changes for each mini-batch and the algorithm will never reach a minimum; it will rather fluctuate around a (local) minimum. In contrast with batch gradient descent where a pass through the training data leads to one updating step of the parameters, in mini-batch gradient descent many updating steps are performed, one for each mini-batch.

The complete mini-batch algorithm has an outer loop allowing many passes through the complete training data until satisfactory convergence is reached. For very large datasets mini-batch gradient descent is considerably faster than batch gradient descent. The noise associated with mini-batch gradient descent is beneficial in avoiding that the algorithm gets stuck in local minima.

- **Gradient checking.** During the debugging phase of the development of an algorithm, it is good practice to check the results from the coded gradient with those obtained with numerical derivatives. The latter are easily obtained as follows. Let $\theta \in \mathbb{R}^p$ be the parameters of the neural network and let $J(\theta)$ be the cost function. Then for small ϵ (for example, $\epsilon = 10^{-4}$), the numerical partial derivatives are approximated by

$$\begin{aligned} \frac{\partial}{\partial \theta_1} J(\theta) &\approx \frac{J((\theta_1 + \epsilon), \theta_2, \dots, \theta_p) - J((\theta_1 - \epsilon), \theta_2, \dots, \theta_p)}{2\epsilon}, \\ &\vdots \\ \frac{\partial}{\partial \theta_p} J(\theta) &\approx \frac{J(\theta_1, \theta_2, \dots, (\theta_p + \epsilon)) - J(\theta_1, \theta_2, \dots, (\theta_p - \epsilon))}{2\epsilon}. \end{aligned}$$

Eyeballing a plot of the exact versus the numerical derivatives should give a first impression (a straight line with slope 1 passing through the origin should be a good indication) or more precisely checking the size of

$$\frac{\|\partial\theta_{\text{approx}} - \partial\theta\|_2}{\|\partial\theta_{\text{approx}}\|_2 + \|\partial\theta\|_2}$$

which for $\epsilon = 10^{-6}$ should be 10^{-6} or 10^{-5} , though this will depend on the number of iterations.

- **Software.** There is a wealth of software available to fit neural networks. For the simple multilayer perceptron `neuralnet` proved straightforward to use in the small datasets of the examples below. Other popular choices include `deepnet`, `h2o` and for more ambitious users, Google released the open-source software TensorFlow with the friendly user-interface Keras.

Example: Analysis of Binary Observations Using a Neural Network

The objective is to study the predictive ability of a neural network and to make a comparison with the kernelised logistic regression, the logistic lasso and the penalised logistic regression.

The data consist of 599 binary records simulated with a logistic model using the wheat inbred lines genotyped for 1279 binary genetic markers, as in the example of page 549. The true (simulated) model on the underlying scale involves 20 loci randomly sampled from these 1279 markers and assigned as QTL. The proportion of 1's in the observations is approximately 50%. A neural network with 1 hidden layer is implemented in R with the vectorised code described on page 602. The neural network is also fitted using the R software `neuralnet` as a comparison. The single layered neural network has 5 neurons in the hidden layer and the sigmoid function is used in the hidden and output layers. An $\ell-2$ regularisation term is added to the logistic loss function and the regularisation parameter is set to 0.002. The learning rate of the gradient descent is set to 0.008 and for each of 20 replicates of training/validating splits, the number of gradient descent iterations is 5000. The predictive ability of the neural network was quantified using the validation MSE (proportion of misclassifications in the validation data, after transforming the predicted probabilities into predicted observations on the 0, 1 scale using Bayes rule, as defined on page 402).

Over the 20 replications, the minimum, median, mean and maximum values of the validating MSE were 0.32, 0.36, 0.36 and 0.40, respectively. The corresponding figures for the training MSE were 0.08, 0.10, 0.10 and 0.15. The analysis was repeated using the R software `neuralnet` and the results for the minimum, median, mean and maximum values of the validation MSE over 20 splits were 0.31, 0.36, 0.36 and 0.41. For the training MSE, the corresponding results were 0.01, 0.03, 0.04 and 0.10.

The predictive performance of the neural work for these data quantified as the average (over 20 replications) validating MSE, was approximately similar to that obtained using the penalised logistic regression (0.40), the kernelised logistic regression (0.33) and the logistic lasso (0.33), as reported on page 549.

Increasing the number of neurons in the hidden layer from 5 to 10 and to 50 deteriorated prediction performance. The same was noted by increasing the number of layers to two and three (with 5 neurons per layer) using the software `neuralnet`. These were exploratory modifications to the original architecture without aiming at a proper optimisation of the neural network.

The R-code to fit the neural network to the simulated wheat data set is shown below. The first code represents the vectorised version and the second code describes the implementation with `neuralnet`.

```
# CODE1110
# FIT THE NEURAL NETWORK TO SIMULATED BINARY PHENOTYPES
# USING THE WHEAT INBRED LINES WITH THE 1,279 GENETIC MARKERS

rm(list=ls()) # CLEAR WORKSPACE
set.seed(37111)

library(BGLR)
data(wheat)
#####
```

```

### USE BGLR MATRIX X

X <- wheat.X
nindiv <- nrow(X)
nmark <- ncol(X)
#####
# NUMBER OF LOCI AFFECTING THE SIMULATED DATA
nloci<-20
p<-0.5
mu<-log(p/(1-p)) # BIAS TERM FOR SIMULATED DATA

##### INITIALISE PARAMETERS AND ALLOCATE MATRICES #####
va<-1.0 # additive variance of liability
ve<-1.0 # environmental variance
Xc<-matrix(data=NA,nrow=nindiv,ncol= nmark)
be<-matrix(data=0.0,nrow=nmark,ncol=1) # parameter of true model
y<-rep(0,nindiv)
cm<-colMeans(X)
for (i in 1:nmark)
{Xc[,i]<- (X[,i]-cm[i])/sd(X[,i])
}
QTLeff<-sqrt(va/nloci)# QTL effect so that the total
# genetic variance is VA
IDq<-sample(1:nmark,nloci,replace=F) # from the nmark markers,
# choose nloci as QTL
be[IDq]<-QTLeff # the only b's that are not zero are those
# associated with QTL.
##### GENERATE BINARY DATA y #####
xb<-Xc%%be
pr <- exp(mu+xb)/(1+exp(mu+xb))
y <- rbinom(nindiv,1,pr)
#####
df = data.frame(cbind(X,y))
m <- length(y)
# BIG X!
X <- t(X)
p <- nrow(X)

Y <- matrix(y,nrow=1,ncol=length(y))

n_1 <- p # NUMBER OF FEATURES IN INPUT DATA
# READ NUMBER OF NEURONS IN LAYER 2
n_2 <- 5
# READ NUMBER OF NEURONS IN LAYER 3
n_3 <- 1
#####
### FUNCTIONS:
# SIGMOID FUNCTION
sigm <- function(par){
  1/(1+exp(-par))
}
# COST FUNCTION EXCLUDING REGULARISATION TERM
cost <- function(A,Y){-(tcrossprod(Y,log(A))+

```

```

    tcrossprod((1-Y),(log(1-A)))/m}
#####

# READ REGULARISATION PARAMETER delta

delta <- 0.02

# READ GD LEARNING RATE (HERE LABELLED gamma)
gamma <- 0.08

# eps: range of initial values of elements of W_1: (-eps,eps)
eps <- 0.85
# READ NUMBER OF GRADIENT DESCENT ITERATIONS
nit <- 5000
# READ NUMBER OF TRAINING / VALIDATING REPS
nitval <- 20

resultval <- matrix(data=NA, nrow=nitval,ncol=3)
result <- matrix(data=NA, nrow=nit,ncol=8+n_2)

ptm<-proc.time()

for (j in 1:nitval) {
  # INITIALISE MATRIX OF WEIGHT W_1 (n_2 x n_1), n_1=p=rows of X
  # INITIALISE MATRIX OF WEIGHT W_2 (n_3 x n_2)
  W_1 <- matrix(nrow=n_2,ncol=n_1,runif(n_2*n_1,-eps,eps))
  W_2 <- matrix(nrow=n_3,ncol=n_2,runif(n_3*n_2,-eps,eps))
  train=sample(1:ncol(X),floor(0.5*ncol(X)))
  Yt <- matrix(Y[train],nrow=1,ncol=length(Y[train]))
  Yv <- matrix(Y[-train],nrow=1,ncol=length(Y[-train]))
  Xt <- X[,train]
  Xv <- X[,-train]
  b_1 <- matrix(0,nrow=n_2,ncol=ncol(Xt))
  b_2 <- matrix(0,nrow=n_3,ncol=ncol(Xt))
  for (i in 1:nit) {
    cat("j=",j, " ", "i=",i, "\n", sep = "")
    # FORWARD PROPAGATION
    Z_2 <- W_1 %*% Xt + b_1
    A_2 <- sigm(Z_2) # SIGMOID FUNCTION
    # A_2 <- pmax(Z_2,0.01*Z_2) # Leaky ReLU function
    Z_3 <- W_2 %*% A_2 + b_2
    A_3 <- sigm(Z_3)
    # A_3 <- Z_3 # USE THE IDENTITY FUNCTION FOR CONTINUOUS DATA
    # BACK PROPAGATION
    DZ_3 <- A_3 - Yt
    DW_2 <- (DZ_3 %*% t(A_2) / m) + delta * W_2
    Db_2 <- mean(DZ_3)
    DZ_2 <- t(W_2) %*% DZ_3 * A_2 * (1 - A_2) # sigmoid function
    # Leaky ReLU function:
    # DZ_2 <- t(W_2)%*%DZ_3 * ifelse(Z_2 > 0,1,0.01)

    DW_1 <- (DZ_2 %*% t(Xt) / m) + delta * W_1
    Db_1 <- apply(DZ_2, 1, mean)
  }
}

```



```

# GRADIENT DESCENT ON TRAINING DATA Xt
W_1 <- W_1 - gamma * DW_1
W_2 <- W_2 - gamma * DW_2
b_1 <- b_1 - gamma * Db_1
b_2 <- b_2 - gamma * Db_2
# BELOW: ADD PENALTY TERM TO THE LOSS FUNCTION
newcost <- cost(A_3, Yt) + (delta/2)*(sum(W_2^2)+sum(W_1^2))
result[i,] <- c(i,newcost,DW_2[1:5],W_1[1],W_2)

}
ytrain <- as.numeric(ifelse(A_3 > 0.5, 1, 0))
msetrain <- mean((ytrain - Yt) ^ 2)
# print(table(yh, Y))
# VALIDATION STAGE WITH DATA Xv
# A LITTLE TWIST: IN LINES 400-404 b_1 & b_2 MUST BE ADJUSTED
# BECAUSE/IF # TRAINING RECORDS < # VALIDATING RECORDS !!!!!
a <- floor(0.5*ncol(X))
b <- 0.5*ncol(X)
bind <- function(b){cbind(b,b[,1])}
b_1 <- if(a<b) {bind(b_1)}
b_2 <- if(a<b) {bind(b_2)}
Z_2 <- W_1 %*% Xv + b_1
A_2 <- sigm(Z_2)
Z_3 <- W_2 %*% A_2 + b_2
A_3 <- sigm(Z_3)
yval <- as.numeric(ifelse(A_3 > 0.5, 1, 0))
mseval <- mean((yval - Yv) ^ 2)
resultval[j, ] <- c(j, msetrain,mseval)

}
proc.time() - ptm
print(table(yval,Yv))
plot(resultval[,2],type="l",ylim=c(min(resultval[,2]),
max(resultval[,3])))
lines(resultval[,3],col="red")
summary(resultval[,3])

```

R-code to fit the neural network using `neuralnet` is shown below.

```

# CODE1110 (cont)
# FIT \texttt{neuralnet} ON TRAINING DATA AND
# TEST ON VALIDATING DATA
# REPEAT nrepnn TIMES
set.seed(371111)
library(neuralnet)

nrepnn <- 20

resmsenn <- matrix(data=NA,nrow=nrepnn,ncol=3)

for (i in 1:nrepnn){
  cat(i, "\n",sep="")
  train=sample(1:ncol(X),floor(0.5*ncol(X))) # ASSUMES X

```

```

# HAS BEEN TRANSPOSED!
# SETTING BELOW hidden = c(5,6) FITS TWO LAYERS OF 5 NEURONS
# IN LAYER 1 AND 6 NEURONS IN LAYER 2. hidden =c(5) FITS
# A SINGLE LAYER WITH 5 NEURONS
nn=neuralnet(y~.,data=df[train,],hidden=c(5),linear.output=
FALSE,act.fct="logistic")
pnv <- as.numeric(predict(nn,df[-train,]))
predv <- ifelse(pnv > 0.5, 1, 0)
msev <- mean((pnv-Y[-train])^2)
pnt <- as.numeric(predict(nn,df[train,]))
predt <- ifelse(pnt > 0.5, 1, 0)
mset <- mean((pnt-Y[train])^2)
resmsenn[i,] <- c(i,msev,mset)
}
plot(resmsenn[,2],type="l",ylim=c(min(resmsenn[,3]),
max(resmsenn[,3])))
lines(resmsenn[,3],col="red")
summary(resmsenn[,2])

```

Example: Prediction Using a Neural Network, a RKHS Regression and a Genomic BLUP Model

In this second example prediction ability is evaluated for the neural network and for the RKHS (Reproducing kernel Hilbert space) regression with a Gaussian kernel. A genomic BLUP model is also fitted, which is a special case of the RKHS regression with a linear kernel appropriately scaled. In contrast with the previous two models, the genomic BLUP model captures only linear relationships of the feature space.

Grain yields (centred and standardised to unit variance) from the 599 inbred lines were downloaded from the BGLR package and, as was the case in the previous example, 1279 binary genetic markers were available as input features for each line. The neural network architecture was as in the previous example, with the only difference that the output neuron was generated using the identity function since grain yield data are continuous. The cost function for this example has terms of the form $(y - a^{(3)})^2$ rather than the logistic likelihood employed for the binary records. The expression for $z^{(3)}$ on page 602 is unchanged.

The analysis of grain yield using the RKHS regression was based on the model described in the example of page 542. The Bayesian RKHS model used $\lambda = 0.3$ and three values of the bandwidth parameter h were entertained. The best prediction result (reported in Table 11.2) was obtained using $h = 1$.

The prediction ability of the models was quantified using the validating MSE (average sum of squared differences between observed and predicted validating records) and the correlation between observed and predicted validating phenotypes, over 20 replicates of training/validating splits. For the RKHS regression, predictions were obtained from expression (11.30) and details are disclosed in the R-code

Table 11.2 Grain yield of 599 inbred lines for wheat data from the BGLR package. Prediction ability quantified as the *Minimum*, *Mean* and *Maximum* over 20 random training/validating splits (50% of each) for the validation MSE (X ; for example for the first entry, $X = 0.72$) and for the correlation between observed and predicted validating phenotypes (Y ; for example for the first entry, $Y = 0.36$) for the neural network model (NN), the RKHS regression model (RKHS) and for the genomic BLUP model. The results in the table are in the form X/Y

	<i>Minimum</i>	<i>Mean</i>	<i>Maximum</i>
NN	0.72/0.36	0.83/0.43	0.91/0.51
RKHS	0.68/0.42	0.78/0.49	0.90/0.56
BLUP	0.70/0.35	0.83/0.43	0.93/0.51

below. Results in the form of *Minimum*, *Mean* and *Maximum* over the 20 replicates, for the validating MSE and for the correlation between observed and predicted validating phenotypes, are displayed in Table 11.2.

For these data the RKHS regression with the Gaussian kernel was a slightly better prediction machine than the neural network and the BLUP model, both of which performed similarly. The RKHS regression model was also implemented using $\lambda = 0.3$, $h = 0.5$ and $\lambda = 0.3$, $h = 3.0$. The mean validation MSE and correlations between observed and predicted validation phenotypes for these two RKHS variants were (0.79; 0.47) and (0.86; 0.42), respectively.

The R-code used to fit the RKHS model is shown below.

```
# CODE1111
# TRAINING AND TESTING OF GRAIN YIELD -
# WHEAT INBRED LINES FROM BGLR PACKAGE
rm(list=ls()) # CLEAR WORKSPACE
set.seed(37111)
library(BGLR)
data(wheat)
#####
### USE BGLR MATRIX X
x <- wheat.X
y<- wheat.Y[,4]
#### A GAUSSIAN KERNEL #####

kgaus <- function(X,h){
  X <- scale(X,center=TRUE,scale=FALSE)
  S=sqrt(sum(apply(FUN=var, X=x,MARGIN=2)))
  X <- X/S
  D <- as.matrix(dist(X))^2
  K <- exp(-h*D)
}
#####
##### A LINEAR KERNEL #####
klin1 <- function(X){
  X=scale(X,center=TRUE,scale=FALSE)
  S=sqrt(sum(apply(FUN=var, X=X,MARGIN=2)))
  X=X/S
  K=tcrossprod(X)
```

```

}
##### CHOOSE GAUSSIAN KERNEL #####

#h <- 0.5
h <- 1
# <- 3

K <- kgaus(x,h)
#dim(K)
#qr(K)$rank

##### CHOOSE LINEAR KERNEL #####
#K <- klin1(x)
#dim(K)
#qr(K)$rank
#####
# READ NUMBER OF TRAINING / VALIDATING SPLITS nitval
nitval <- 20
# READ REGULARISATION PARAMETER lambda
lambda <- 0.3

result <- matrix(data=NA, nrow=nitval,ncol=5)

ptm <- proc.time()
for (j in 1:nitval) {
  cat(j, "\n",sep="")
  train = sample(1:nrow(x), floor(0.5 * nrow(x)))
  xt <- x[train, ]
  yt <- y[train]
  xv <- x[-train, ]
  yv <- y[-train]
  YHATt <- matrix(nrow = length(yt), ncol = 1)
  YHATv <- matrix(nrow = length(yv), ncol = 1)
  Ktrain <- K[train, train]
  Kval <- K[-train, train]
  Xt <- cbind(1, Ktrain)
  RHSt <- crossprod(Xt, yt)
  LHSt <- crossprod(Xt)
  LHSt[-1, -1] <- LHSt[-1, -1] + Ktrain * lambda
  # diag(LHSt) <- diag(LHSt) + c(0, rep(1e-8,ncol(Xt)-1))
  solt <- solve(LHSt, RHSt)
  YHATt <- solt[1] + Ktrain %*% solt[-1]
  YHATv <- solt[1] + Kval %*% solt[-1]
  mset <- mean((YHATt - yt) ^ 2)
  msev <- mean((YHATv - yv) ^ 2)
  cort <- cor(YHATt, yt)
  corv <- cor(YHATv, yv)
  result[j, ] <- c(j, mset, msev, cort, corv)
}
proc.time() - ptm
summary(result[,3])
summary(result[,5])

```

11.4 Classification and Regression Trees

This section describes an approach to modelling where the feature space is partitioned into regions within which the responses are relatively homogeneous. The partitioning of the feature space follows a set of binary splitting rules that gives rise to a decision tree. Decision trees can be used for continuous as well as for categorical variables. In the former they are known as regression trees; in the latter as classification trees. The tree has terminal nodes or leaves that arise when a stopping criterion for further partitioning has been met. Finally, a prediction associated to a particular terminal node is obtained based on the mean or mode of the training output in the terminal node, in the case of regression trees, or based on the most frequent class, in the case of classification trees. The method lends itself to a graphical representation that possesses pedagogical quality and ease of interpretation.

The construction of the tree and its application is illustrated with the South African Heart Disease Data (Rousseeuw et al. 1983) used also in Hastie et al. (2009), where a more detailed account can be found. The data is a subset of a larger data set and includes 462 males in a high-risk heart disease region of the Western Cape in South Africa. There are $p = 9$ sources of information (features or covariates or inputs) available on each patient: systolic blood pressure (sbp), cumulative tobacco (tobacco), low density lipoprotein cholesterol (ldl), adiposity, family history of heart disease (famhist), type-A behaviour (type-a), obesity, alcohol, age. The output or response is coronary heart disease (chd). Adiposity is a measure of percent body fat, whereas obesity is measured as body-mass-index, bmi. Type-A refers to a behaviour pattern characterised by an excessive competitive drive, impatience and anger/hostility. Coronary heart disease (chd) is a classification variable that takes values “Diseased” and “Healthy”. Out of the 462 patients 160 are classified as “Diseased”.

Figure 11.11 shows the output of a classification tree fitted to these data. The tree arises from questions associated to the inputs/covariates. These questions are known as *splits*. Starting at the top of the tree the first split assigns observations to the left if age is less than 31.5 years. These observations do not undergo further splits and constitute a *terminal node* or *leaf*. Observations whose age is older than 31.5 years are moved to the right. This branch undergoes a further split, again based on age, where the threshold now is 50.5 years; those younger define a left branch and those older a right branch. The left branch has a split according to scoring for type-a behaviour. Those scoring less than 68.5 are moved to the left and constitute another terminal node; those scoring higher than 68.5 are moved to the right and give rise to a third terminal node. Observations whose age is older than 50.5 years undergo a split due to family history, (absent/present), a categorical variable. The left branch includes observations that do not show family history for chd, while the right branch includes those observations that do. These constitute a terminal node. The branch on the left has a further split according to tobacco: less than an accumulated amount

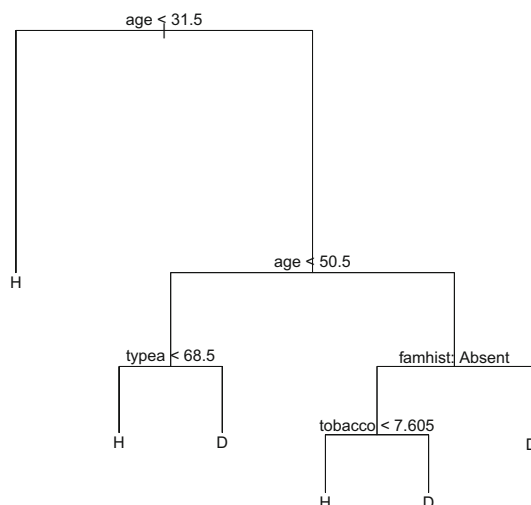


Fig. 11.11 Classification tree fit to the South African Heart Disease data. The goal is to predict the health status of a patient (“Healthy” or “Diseased,” “H” and “D”, respectively), based on 9 features or covariates. The split at the top of the tree results in two branches: one on the left for age < 31.5 years and one on the right for age > 31.5 years. The left split does not undergo further subdivision and the group is classified as “Healthy”. The split on the right undergoes a further split whereby those with age < 50.5 years are moved to the left and those over 50.5 years to the right. The split on the left has a further split governed by type-a score whereby those < 68.5 are moved to the left and are classified as “Healthy” and those scoring > 68.5 are moved to the right and are classified as “Diseased”. The final tree has 6 terminal nodes or leaves with a certain number of observations in each terminal node (not shown in the figure). The predictions “D” or “H” at each terminal node depend on the proportion of observations falling into each category

of 7.605 go to the left and give rise to a terminal node. Those with a value larger than 7.605 comprise another terminal node. The tree has 6 terminal nodes.

The shape of the final tree is governed by the splits. The data consist of p variables or features (in the case of the heart data, $p = 9$) and a response: that is (x_i, y_i) , $i = 1, 2, \dots, n$, with $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})$. For each split, the algorithm must decide which of the p variables to choose and then must perform a search to find the *optimal* point to split (the first split in the tree of Fig. 11.11 is based on the feature age and the split occurs at 31.5 years). One must also decide how deep the tree must be. In other words, when to stop splitting and thereby declaring the records from this last split as belonging to a terminal node. The predictions for each terminal node shown in Fig. 11.11 arise by counting the number of D 's and H 's among the observations in the terminal node and choosing as the predicted value the most frequent.

Among the 462 observations, the proportion “Diseased” is 0.346. Consider the first decision at the top of the tree (actually, the root, since the tree is upside down). For each of the 9 covariates the algorithm finds the binary partition that optimises one of two criteria (*Gini index* or *deviance*, described at the end of this

subsection) that measure the *impurity* among the response in the binary partition (if all the observations in the partition belong to the same class the impurity is zero and it is maximum when 50% belong to each class). The aim is ideally, to find a partition of the covariate within which the responses have the same label. For the tree in Fig. 11.11 the choice falls on age. There are 49 “age classes” among the 462 records and the age 31.5 years minimises the *impurity*. At this point there are 117 observations whose age is less than 31.5 years and among these, 10 are classified as “Diseased” resulting in a proportion of “Healthy” equal to 0.915 and in a prediction equal to H . This constitutes the first terminal node of the tree. After this split the algorithm works with those observations on the right and a new split is found by the same procedure. The same covariate, age, now at the point 50.5 years is chosen as the one minimising impurity. Those records whose age is less than 50.5 years are moved to the left. Among these records, 119 are “Healthy” and 54 are “Diseased” resulting in a prediction at this stage of H .

A characteristic of decision trees is that optimality is defined locally for each split, one at a time, and the algorithm searches for all possible splitting points for each covariate. The process is repeated until a stopping criterion is reached (based on the number of observations in a splitting region, or on the reduction of impurity). A compromise must be found between growing very deep trees, leading to overfitting, or shallow trees that do not capture enough of the variability of the response. The common practice is first, to grow a deep tree stopping when the number of observations in a node reaches a minimum (5, say). This tree will typically overfit the data. This is then followed by tree *pruning* which requires cross-validation: the bottom branches of the tree are removed until the cross-validation error increases. The tree in Fig. 11.11 is a result of this pruning procedure. The original tree had 15 terminal nodes and included 7 of the 9 predictors. The pruned tree has only 6 terminal nodes and 4 predictors. Both trees were fitted to the complete set of 462 observations. The misclassification rate of these trees using the complete data was 0.21 and 0.23 for the complete and pruned tree, respectively.

Regression trees follow the same rationale as the classification trees, except that the residual sum of squares is used instead of the Gini index or the deviance as criteria to split the nodes or to prune the tree. The response y now is quantitative and, initially, the residual sum of squares before the first split is $\sum_{i=1}^n (y_i - \bar{y})^2$ where $\bar{y}$ is the mean of the n responses. For a first partition around a splitting point s , the n_l observations falling on the left of s have mean $\hat{\mu}_l$ and those n_r falling on the right of s , $\hat{\mu}_r$. These means constitute the predicted values for the first partition. The residual sum of squares for this node for this value of s is reduced to

$$\sum_{i: x_{ij} \leq s} (y_i - \hat{\mu}_l)^2 + \sum_{i: x_{ij} > s} (y_i - \hat{\mu}_r)^2. \quad (11.80)$$

The algorithm searches the covariate j and the split point s that minimises Eq. (11.80). For each branch a new split is carried out to refine predictions. Once a final tree has been constructed there will be a predicted value attached to each terminal node given by the node’s mean. For a new set of covariates for a new

individual, a predicted value is obtained by starting at the top of the tree and following the splits downwards until the terminal node with its predicted value is reached. For a regression tree, the new individual is assigned the prediction given by that terminal node's mean.

The Gini Index and the Deviance

Let R_j denote the j th node and let n_{jk} denote the number of observations of class k , $k = 1, 2, \dots, K$ in node j . The estimate of the probability of observing class k in node j is the proportion of observations y that fall in class k

$$\hat{p}_{jk} = \frac{1}{n_{jk}} \sum_{i: x_i \in R_j} I(y_i = k),$$

and the class assigned to node j is the class with the largest estimated probability, $\arg \max \hat{p}_{jk}$. The *Gini index* associated with node j is

$$G_j = \sum_{k=1}^K \hat{p}_{jk} (1 - \hat{p}_{jk}). \quad (11.81)$$

G takes on small values when the estimated probabilities are extreme and reaches a maximum when estimated probabilities are intermediate.

An alternative to the Gini index is the *deviance* or *entropy* given by

$$D_j = - \sum_{k=1}^K \hat{p}_{jk} \log \hat{p}_{jk}. \quad (11.82)$$

Since $\lim_{p \rightarrow 0} p \log p = 0$, we take $\hat{p}_{jk} \log \hat{p}_{jk} = 0$ whenever $\hat{p}_{jk} = 0$. Therefore entropy is at a minimum value of 0 when one of the $\hat{p}_{jk} = 1$ and all other equal to 0 and it attains a maximum when all $\hat{p}_{jk}$ in node j are equal and $\hat{p}_{jk} = 1/K$ for all k .

As shown in the examples that follow, for classification trees, the function `tree` in R uses $2n_j D_j = -2 \sum_{k=1}^K n_{jk} \log \hat{p}_{jk}$ instead of (11.82), where n_j is the number of responses in node j .

Evaluating Prediction Performance with Cross-Validation

The misclassification rates of the full and pruned trees quoted above (21 and 23%, respectively) were estimated using the complete data. This is likely to underestimate the misclassification rate when evaluated on new data. Here the predicted ability of

the classification tree is investigated further using cross-validation. The approach is as follows:

1. Split the data into training and validating sets (50% of the observations in each)
2. Construct a full tree using the training data
3. Compute the misclassification rate for this full tree in the training and validating data
4. Prune the full tree
5. Compute the misclassification rate of this pruned tree in the validating data

For a particular replicate, the full tree from step 2 has 25 terminal nodes and includes 7 of the 9 variables (dropped *famhist* and *alcohol*). The training and validating misclassification rates of this tree are 14 and 38% respectively. The misclassification rate of the null model obtained by classifying all the observations in the complete data as healthy is 34.6% (equal to the proportion of diseased individuals). The result is not encouraging.

The pruned tree from step 4 has 3 terminal nodes and includes only *age* and *ldl*, a remarkable simplification from the full tree. The training and validating misclassification rates of this pruned tree are 24 and 34.2% respectively, better than the full tree but not an improvement over the performance of the null model.

Example: Analysis of the Heart Data Using a Classification Tree

The code below reads the South African Heart Disease data set and executes the function `tree` in R. The presentation follows closely the format in James et al. (2017, pp. 323–327). We look a little more closely into the output of the `tree` function and of other related functions that reveal some of the mechanics of the constructed trees.

```
# CODE1112
# READ SOUTH AFRICAN HEART DISEASE DATA
rm(list=ls()) # CLEAR WORKSPACE
library(sda)
library(tree)
library(glmnet)
library(randomForest)
library(loon.data) # MUST INSTALL PACKAGE loon.data
data("SAheart")
sahd <- SAheart
sahd$chd <- factor(as.numeric(sahd$chd), levels=c(2,1),
                  labels=c("D", "H"))
#length(which(sahd$chd=="H")) # NO HEART DISEASE
#length(which(sahd$chd=="D")) # HEART DISEASE
#####
# FUNCTION 'ACCURACY':
accuracy = function(actual, predicted) {
  mean(actual == predicted)
}
```

```
# FIT TREE TO TRAINING AND VALIDATING HEART DATA
replicate <- 1
result <- matrix(data=NA, nrow=replicate,ncol=2)
# REPLICATION
for(i in 1:replicate){
  # SPLIT DATA INTO TRAINING / VALIDATING SET
  set.seed(31)
  train=sample(1:nrow(sahd),nrow(sahd)/2)
  treetrain <- sahd[train,]
  treevalid <- sahd[-train,]
  # FIT TREE TO THE TRAINING DATA
  treetr <- tree(sahd$chd[train] ~. ,data=treetrain)
  trtrpred <- predict(treetr,treetrain,type="class")
  trvpred <- predict(treetr,treevalid,type="class")
  table(predicted = trvpred, actual = sahd$chd[-train] )
  table(predicted = trtrpred, actual = sahd$chd[train] )
  acval <- accuracy(actual=sahd$chd[-train],predicted=trvpred)
  actst <- accuracy(actual=sahd$chd[train],predicted=trtrpred)
  result[i,] <- c(1-acval,1-actst)
}
summary(treetr)
```

```
##
## Classification tree:
## tree(formula = sahd$chd[train] ~ ., data = treetrain)
## Variables actually used in tree construction:
## [1] age, typea, tobacco, obesity, ldl, sbp,
## [7] adiposity
## Number of terminal nodes: 25
## Residual mean deviance: 0.5449 = 112.2 / 206
## Misclassification error rate: 0.1342 = 31 / 231
```

```
result
```

```
##           [,1]      [,2]
## [1,] 0.3809524 0.1385281
```

The code reads the 462 records and randomly samples two sets of 231 records that constitute the training and validating data. The `tree` function is fit to the training data (*treetrain*) and creates the object *treetr*. The `summary` function displays information about the resulting tree. I discuss this function below in connection with the pruned tree. At this stage we note that the resulting tree has 25 terminal nodes and includes 7 out of the 9 features of the data set. The reported misclassification error rate of 13% corresponds to the training data; it is therefore likely to be underestimated. This is confirmed applying the `predict` function using as arguments the training and validating data. The misclassification error rate using the validating data is 38% and using the training data slightly under 14% (showing a mild discrepancy with the result of a little over 13% reported in the `summary` function).

The next step is to investigate whether a pruned tree leads to a better prediction performance. The code below fits the function `cv.tree` using as one of the arguments the object generated by fitting the tree to the training data. It uses cross-validation in order to find the size of the tree that produces the smallest misclassification rate. Size here refers to the number of terminal nodes.

```
# CODE1112 (cont)
# PRUNE TREE FROM TRAINING RUN
set.seed(33)
treetr_cv <- cv.tree(treetr, FUN = prune.misclass)
#treetr_cv
# index of tree with minimum error
min_idx = which.min(treetr_cv$dev)
#min_idx
# number of terminal nodes in that tree
treetr_cv$size[min_idx]
```

```
## [1] 3
```

```
# misclassification rate of each tree
#treetr_cv$dev / length(train)
# IT APPEARS THAT A TREE WITH treetr_cv$size[min_idx]
# TERMINAL NODES HAS THE SMALLER MISCLASSIFICATION
# EXECUTE PRUNE.MISCLASS: TREE WITH LOWEST C-V ERROR RATE
# SET best = TO NR TERMINAL NODES FROM THIS BEST TREE
tree_prune<-prune.misclass(treetr,best=treetr_cv$size[min_idx])
#summary(tree_prune)
# OBTAIN PREDICTIONS USING THIS PRUNED TREE
tree_prune_trn <- predict(tree_prune, sahd[train,],type="class")
table(predict = tree_prune_trn, actual = sahd$chd[train] )
```

```
##          actual
## predict    D    H
##          D  35  10
##          H  45 141
```

```
1-accuracy (predict = tree_prune_trn, actual = sahd$chd[train])
```

```
## [1] 0.2380952
```

```
tree_prune_val<-predict(tree_prune,sahd[-train,],type="class")
table(predict = tree_prune_val, actual = sahd$chd[-train] )
```

```
##          actual
## predict    D    H
##          D  20  19
##          H  60 132
```

```
1-accuracy (predict = tree_prune_val, actual = sahd$chd[-train])
```

```
## [1] 0.3419913
```

```
tree_prune
```

```
## node), split, n, deviance, yval, (yprob)
##      * denotes terminal node
##
## 1) root 231 298.10 H ( 0.3463 0.6537 )
## 2) age < 51.5 149 155.00 H ( 0.2148 0.7852 ) *
## 3) age > 51.5 82 111.30 D ( 0.5854 0.4146 )
## 6) ldl < 4.92 37 47.97 H ( 0.3514 0.6486 ) *
## 7) ldl > 4.92 45 47.67 D ( 0.7778 0.2222 ) *
```

The training and validating misclassification rates ($1 - \text{accuracy}$) of this pruned tree (that has 3 terminal nodes only) are 24 and 34.2% respectively. The function `table` displays the number of misclassifications in the off-diagonals and the correctly classified records in the diagonals. For example, in the case of the *training* data, the first table, the number of misclassified observations is $10 + 45 = 55$ and the number of correctly classified observations is $35 + 141 = 176$, out of a total of 231 observations.

Turning attention to the output from the pruned tree and typing the `tree` object `tree_prune` discloses details about each branch of the tree. The first branch of the tree (the *split* at `age < 51.5` years; corresponds to the second *node*) has $n = 149$ observations. The *deviance* for the split is 155.00 and the predicted value is `yval = H`, because the proportion of *H*'s, 0.7852, is larger than the proportion of *D*'s, 0.2148. This split is a *terminal node* as indicated by the asterisk *. As a check on the reported proportions, compute:

```
# CODE1112 (cont)
# A LITTLE CALCULATION: PROPORTION OF H'S IN LEFT BRANCH
# OF THE FIRST SPLIT OF THE PRUNED TREE
trtrlt <- treetrain[which(treetrain$age < 51.5),]
dim(trtrlt)
```

```
## [1] 149 10
```

```
trtrltH <- trtrlt[which(trtrlt$chd == "H"),]
dim(trtrltH)[1]/dim(trtrlt)[1]
```

```
## [1] 0.7852349
```

The reported *deviance* for the split equal to 155.00 results from the following computation. The starting point is the entropy (11.82); for $j = 2$, the second node,

$$\begin{aligned} D_2 &= -(0.7852 \times \log(0.7852) + 0.2148 \times \log(0.2148)) \\ &= 0.5202 \end{aligned}$$

and the printed *deviance* is $2(149)(0.5202) = 155.02$.

It is also possible to obtain the deviance of the pruned tree by executing:

```
summary(tree_prune)
```

```
##
## Classification tree:
## snip.tree(tree = treetr, nodes = c(6L, 7L, 2L))
## Variables actually used in tree construction:
## [1] "age" "ldl"
## Number of terminal nodes: 3
## Residual mean deviance: 1.099 = 250.7 / 228
## Misclassification error rate: 0.2381 = 55 / 231
```

The numerator of the printed *Residual mean deviance* is

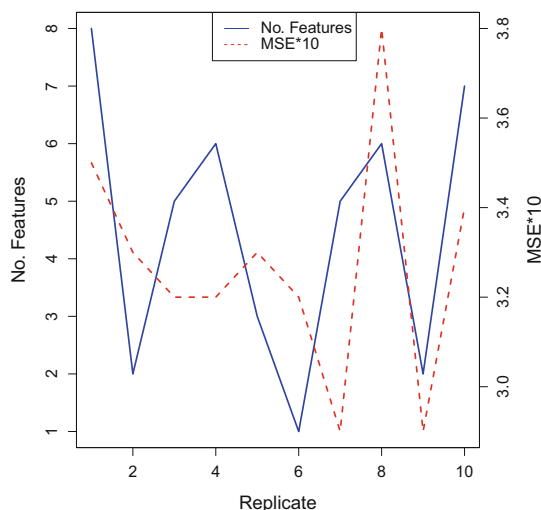
$$\begin{aligned} 2 \sum_j n_j D_j &= -2 \times 149 \times (0.7852 \times \log(0.7852) + 0.2148 \times \log(0.2148)) + \\ &\quad -2 \times 37 \times (0.6486 \times \log(0.6486) + 0.3514 \times \log(0.3514)) + \\ &\quad -2 \times 45 \times (0.2222 \times \log(0.2222) + 0.7778 \times \log(0.7778)) \\ &= 250.68, \end{aligned}$$

where D_j is given by expression (11.82), n_j is the number of observations in terminal node j and there are 3 terminal nodes in the pruned tree. The *Residual mean deviance* is obtained dividing the numerator by the number of observations in the data minus the number of terminal nodes. In the example, this gives $250.68/(231 - 3) = 1.099$.

Variability of Estimated Trees

Figure 11.12 shows the number of features and the validating misclassification error rate versus replicate number for 10 pruned trees obtained by resampling the complete data to construct 10 training/validating sets of data, fitting a tree to the training data in each replicate, pruning the tree obtained from the training data and obtaining predictions on the validating data. The number of features included in the final pruned tree ranged from 8 in replicate 1, with $MSE = 0.35$, to 1 in replicate 6,

Fig. 11.12 Left y-axis: number of features included in the pruned tree (in blue). Right y-axis: misclassification error ($MSE \times 10$) (dashed red), versus replicate number. The smallest $MSE = 0.29$ over the 10 replicates was for replicates 7 and 9. The pruned trees for these replicates included 5 and 2 features, respectively. The smallest number of features is 1 (*age* < 49.5 years) for replicate 6, with a $MSE = 0.32$



with a $MSE = 0.32$. The smallest $MSE = 0.29$ was observed in replicates 7 and 9, with 5 and 2 features, respectively. The figures display large variation, particularly in the number of features included in the final pruned tree.

There are two sources of variability at play here. One is the data. The number of records, the relative contribution of each covariate and the correlation structure among covariates makes it difficult to find a “best” predictive model. More than one combination of covariates leads to similar predictive ability. The other important source of variability is methodological: trees are known to be non-robust, such that small changes in the data cause large changes in the final estimated tree. The large variability of decision trees makes them relatively poor prediction machines. This limitation is in contrast to other attractive properties such as ease of interpretation, coupled with a highly pedagogical graphical display. As indicated in the next section, the non-robustness issue can be improved upon by constructing many trees and averaging the final predictions.

11.5 Bagging and Random Forests

As indicated in (6.66), the variance of the predictor is one of the three terms that influence the expected validation prediction error. Bootstrap aggregation or *bagging* (Breiman 1996) improves prediction ability by reducing this variance. This is achieved by constructing a prediction averaged over several bootstrap samples whose variance is smaller than the variance of an estimate based on a single sample.

The idea of bagging is illustrated here with decisions trees but it can be used with other approaches. The algorithm is quite straightforward: create B bootstrap samples of training data each of size n_t (sampling the n_t rows of the training data

$(y_i, x_i), i = 1, 2, \dots, n_t$ with replacement n_t times) and grow a tree from each. Let $\hat{f}_b(x_0)$ be the prediction from the b th bootstrap sample at the prediction point x_0 and let $\hat{f}_B(x_0)$ be the prediction obtained using the B samples. In the case of regression trees, $\hat{f}_B$ is a simple average over the B samples and for classification trees $\hat{f}_B$ is based on a majority vote: the most frequent class is chosen as the label for $\hat{f}_B$. Each tree is grown deep and not pruned. This ensures low bias and by averaging the B predictions the large variance of each tree is substantially reduced. Specifically we have

$$\begin{aligned} \text{Var}(\hat{f}_B(x_0)) &= \text{Var}\left[\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x_0)\right] \\ &= \frac{1}{B^2} [B \text{Var}(\hat{f}_b(x_0)) + B(B-1) \text{Cov}(\hat{f}_b(x_0), \hat{f}_b'(x_0))] \\ &= \frac{(1-\rho)}{B} \text{Var}(\hat{f}_b(x_0)) + \rho \text{Var}(\hat{f}_b(x_0)) \end{aligned} \quad (11.83)$$

where ρ is the pairwise correlation between predictors from the different trees. With large number of bootstrap samples B , the first term in the right hand side is reduced, while the second depends on the correlation between the predictor functions.

The initial use of bagging with decision trees is based on inclusion of all the p features each time a split is considered. The trees fitted to the bootstrap samples constructed in this way tend to be correlated because the same feature is likely to be chosen in the different trees, particularly at the top of the tree. In a *random forest* (Breiman 2001) only $m < p$ features are randomly selected (without replacement) as candidates for splitting and a different set of m features is selected at each split. This has the effect of reducing the correlation between the trees and the second term in (11.83). Typical values for m are $p/3$ for regression trees and $\sqrt{p}$ for classification. The essentials of a random forest construction is that each tree is based on a random subset of the observations (the bootstrapping step) and each split is based on a random subset of candidate features.

Out-of-Bag Error Estimates

The use of bagging provides an opportunity to study the prediction ability of a model without resorting to cross-validation. Consider data $z_i = (y_i, x_i), i = 1, 2, \dots, n$. The probability of selecting a particular observation in each of the B bootstrap samples is $1/n$ and the probability of not selecting it is $(1 - 1/n)$. When n random samples are taken with replacement to form the bootstrap sample, the probability of not selecting the observation in the n samples is $(1 - 1/n)^n$, which in the limit when $n \rightarrow \infty$, is equal to $\exp(-1) \approx 0.37$. Therefore the observation is absent in approximately one third of the B bootstrap samples, on average (the number of bootstrap samples with the missing observation is binomially distributed, with

parameters B and $\exp(-1)$). In random forests, *out-of-bag* error estimates for classification or regression are obtained by constructing random forest predictors of each observation y_i , by averaging the predictors from trees grown in which z_i is absent. The error estimates obtained in this manner agree well with those obtained using cross-validation.

Chapter 6 describes a number of approaches to compute validating mean squared errors using cross-validation. The idea of bagging can also be exploited to estimate validating mean squared error with the bootstrap, without splitting the data into training and validating sets. There are several possible variants; one of these, the so-called ".632 method", outperforms the traditional estimator based on cross-validation in terms of sampling variability (Efron 1983; Efron and Tibshirani 1997). See also Efron and Hastie (2016), p. 232.

Variable Importance

Although the primary objective of random forests is prediction, it may be of interest to obtain an overall summary of the importance of each predictor or feature. There are two popular measures of importance of the predictor variables. One is based on recording the decrease of the residual sum of squares, in the case of regression trees, or of the Gini index in the case of classification trees, each time a particular predictor is used in a split. Averaging this decrease over the B trees reveals the important predictors as those leading to large decreases. The other approach addresses the contribution of a feature to prediction more directly: it considers the decrease in prediction accuracy when the feature is randomly permuted in the out-of-bag samples, thereby removing its association with the response. The rationale is that a permutation is supposed to mimic the absence of the predictor variable from the model. When the permuted predictor, together with the remaining nonpermuted predictors, is used to predict the response with these out-of-bag observations, the prediction accuracy (proportion of observations classified correctly) decreases if the permuted predictor is associated with the response. In regression problems, classification rate is replaced by mean squared error. The difference between the two prediction accuracies (permuted/non-permuted) associated with a particular feature averaged over the B trees constitutes its variable importance score.

Example: Analysis of the Heart Data Using a Random Forest

The R-code below fits a random forest to the heart data using the function `randomForest`. In order to look more closely into the output, the argument `mtry` that specifies the number of features randomly selected in each split is set to a single value (3 in the example; setting `mtry = 9` chooses bagging).


```

# CODE1113
# RANDOM FOREST WITH HEART DATA
#rm(list=ls()) # CLEAR WORKSPACE
library(sda)
library(tree)
library(glmnet)
library(randomForest)
# sahd <- read.table(
# "http://www-stat.stanford.edu/~tibs/ElemStatLearn/
# datasets/SAheart.data", sep=",", head=T, row.names=1)
# FUNCTION ACCURACY:
accuracy = function(actual, predicted) {
  mean(actual == predicted)
}

# CHANGE RESPONSE VARIABLE AND FAMHIST TO FACTOR
#sahd$chd<-as.factor(ifelse(sahd$chd == 1,
#                           "Disease", "Healthy"))
#sahd$famhist <- as.factor(sahd$famhist)

#head(sahd)
#str(sahd)
sumd <- data.frame()
nrep <- 1
reslt <- rep(NA,nrep)
mtry <- c(1,3,5,9)
mtry <- 3
set.seed(2)
for (m in mtry){
  cat("mtry ",m,"\n",sep="")
  for ( rep in 1:nrep ) {
    cat("Replicate ",rep,"\n",sep="")
    train=sample(1:nrow(sahd),nrow(sahd)/2)
    rf <- randomForest(sahd$chd ~.,data=sahd,
                      subset=train,mtry=m,importance=TRUE)
    predict <- predict(rf,sahd[-train,])
    observed <- sahd$chd[-train]
    t <- table(observed,predict)
    print(t)
    reslt[rep]<-1-accuracy(predicted=predict,actual=observed)
  }
  sumd <- rbind(
    sumd,c(m,min(reslt),mean(reslt),median(reslt),max(reslt),
            var(reslt)))
}

```

```

## mtry 3
## Replicate 1
##      predict
## observed   D   H
##          D  30  47
##          H  25 129

```

```
names(sumd) <- c("mtry", "min", "mean", "median", "max", "var")
sumd[3]
```

```
##           mean
## 1 0.3116883
```

The data are divided into a training and a validating set (each of 231 observations), the random forest is fitted to the training set and the resulting validating mean squared error is $MSE \approx 0.31$ (misclassification rate: $(47+25)/(30+47+25+129)$). This is a little improvement over the validating MSE of the pruned tree, with $MSE = 0.34$, reported on page 583.

When `randomForest` is executed on the complete data, details of the output can be displayed typing the object `rf` (not shown).

```
# CODE1113 (cont)
library(randomForest)
rf <- randomForest(sahd$chd ~., data=sahd, mtry=3, importance=TRUE)
rf$mtry
```

```
## [1] 3
```

```
rf$ntree
```

```
## [1] 500
```

```
rf$confusion
```

```
##      D      H class.error
## D 68   92    0.5750000
## H 54  248    0.1788079
```

Particular elements of the output file can be extracted. The output indicates that 3 variables are used per split (`mtry`) and that by default $B = 500$ trees are constructed (`ntree`). The estimate of the out-of-bag error rate computed from the complete data $((92 + 54)/(92 + 54 + 248 + 68) \approx 0.32)$, is in good agreement with the misclassification error rate using cross-validation reported above (31%).

The importance of each feature can be obtained using the `importance` function:

```
# CODE1113 (cont)
importance(rf, type=1)
```

```
##           MeanDecreaseAccuracy
## sbp                3.869507
## tobacco            13.272608
## ldl                 4.080764
## adiposity          4.032976
## famhist             7.679644
```

```
## typea          1.714666
## obesity       -3.309007
## alcohol       -3.556895
## age          17.041426
```

The output indicates that on average over the 500 trees, *age*, *tobacco* and *family history* are the three most important features affecting prediction accuracy.

Variability of Random Forests

Variation over repeated samples of training/testing data can be obtained using the code above by modification of the parameter `nrep`. The performance of random forest for different choices of the number of features sampled per split can be studied setting `mtry <- c(1,3,5,9)`. The code above was run setting `nrep=100` and the prediction accuracy (misclassification rate) is shown in Table 11.3.

For this particular data set the results are similar across scenarios with a slight advantage for the choice `mtry=1`.

Example: An Analysis Involving Genetic Epistatic Interactions

An attractive property of decision trees and random forests is their potential ability to capture interactions involving covariables. This is examined here with a stylised example in the context of gene discovery and genomic prediction.

A matrix of genotypic markers (of order *number of individuals* (sample size) $\times$ *number of markers* = 1000) is generated from a binomial distribution $Bi(n = 2, p = 0.5)$. These marker data are part of the operational models used for inferences and prediction. Three sample sizes are studied: 1000, 5000 and 10,000 observations.

Observations are simulated using two models that differ in their genetic architectures. In the first model, the *additive model*, 10 marker genotypes are randomly sampled from the 1000 genotypic markers and designated as QTL. The 10 QTL

Table 11.3 Prediction accuracy of random forests over 100 samples of training/testing splits and for 4 different numbers of features sampled out of 9 per split (`mtry`; `mtry=9` indicates *bagging*). The figures in the table display the minimum, mean and maximum prediction accuracy over 100 replicates

<i>mtry</i>	<i>min</i>	<i>mean</i>	<i>max</i>
1	0.22	0.30	0.35
3	0.27	0.32	0.36
5	0.26	0.32	0.40
9	0.26	0.32	0.38

are assigned additive genetic effects equal to 0.22σ each, where $\sigma = \sqrt{5}$ is the standard deviation of the phenotypic distribution. The 10 QTL combine additively to determine genetic values. This model generates an additive genetic variance equal to 1.25 squared units. Setting the environmental variance equal to $(5 - 1.25)$, results in a heritability or proportion of variance of the observations captured by the linear regression on QTL genotypes equal to 0.25.

In the second model, the *interaction model*, the genetic values are the result of the following process. First, one locus contributing to additive genetic variation is randomly chosen among the 10 QTL. This additive locus contributes an additive genetic variance equal to 0.125 squared units. Secondly, among the 45 possible combinations of two loci taken from the 10 QTL, 10 are randomly sampled to generate additive $\times$ additive epistatic interactions. The interaction effects are all set equal to 0.44σ where, as before, $\sigma = \sqrt{5}$. When gene frequencies are intermediate and loci are in linkage equilibrium, the epistatic part of the interaction model constructed in this example does not generate additive genetic variation; the only source of additive genetic variation stems from the single additive locus. The genetic variance arising from the interacting genotypes is of additive $\times$ additive type only and equal to 2.5 squared units. As in the first model, the phenotypic variance is equal to 5 squared units. Details of the interaction model are described below in the **Note: The epistatic model**.

Three operational models used for analysis of the data are compared. The first two are fully parametric models and include the lasso and a Bayesian spike and slab mixture described on page 349. In both, the conditional mean of the data is assumed to be linearly related to the 1000 marker genotypes. The third is a nonparametric model: a random forest. The three models are fitted to data generated using both genetic architectures (the *additive* and the *interaction models*) and to the 3 sample sizes.

The two parametric models assume a multiple linear regression of the response y on the 1000 marker genotypes. It follows that, for the additive model, the 10 QTL are a subset of the marker panel of the operational model. For the interaction model, only the single QTL is a subset of the marker panel. The interaction terms that define the remaining proportion of the true genotypic values are not members of the operational models.

The models do not intend to mimic any particular trait. Rather, the intention is to examine the following conjectures:

1. with a purely additive genetic model, genomic operational models that postulate a linear regression of response on marker genotypes (encoded additively) should perform as well as or better than random forests, in terms of prediction ability and in terms of QTL detection.
2. In the presence of genetic interactions between QTL, random forests should outperform linear models, both, in prediction ability and in QTL detection because linear models do not account for interactions.

The justification for point 1 is that operational models that postulate a linear regression of data on additively encoded marker genotypes are in close agreement with the true model. In such a set up, the linear models should be difficult to outperform. While the exact location of QTL is unknown, their detection depends on size of the effects and on the amount of data. In the case of point 2, the linear models are at a disadvantage: there are no interaction terms in the operational models. On the other hand, the building process of a decision tree (that is an integral part of the random forests) is designed to capture interactions provided signals are strong enough.

The R-code below generates the data for 10,000 individuals and for the *interaction model*. The *additive model* is retrieved by minor changes of the code.

```
# CODE1114
# GENERATING AN EPISTATIC MODEL AND FIT RANDOM FOREST,
# LASSO, BAYESIAN MIXTURE
rm(list=ls()) # CLEAR WORKSPACE
set.seed(303371)
nindiv<-10000
nmark <- 1000
nqtl <- 10
nintqtl <- 10
mu_y<-0
Xq<-matrix(data=NA,nrow= nindiv,ncol= nqtl)
##### GENETIC MARKERS Xm #####
Xm<-matrix(nrow= nindiv,ncol= nmark,
           rbinom(nindiv*nmark,size=2,p=.5)-1)
# from the nmark markers, choose nqtl as QTL:
IDq<-sample(1:nmark,nqtl,replace=F)
Xq <- Xm[,IDq] # QTL GENOTYPIC MATRIX
# INTERACTION GENOTYPIC MATRIX:
Xi<-matrix(data=NA,nrow= nindiv,ncol= nintqtl)
b <- rep(0,nmark+nintqtl)
nr <- ncol(Xq)
i1 <- combn(nr,2)
i2 <- sample(ncol(i1),nintqtl,replace=FALSE)
i3 <- as.matrix(i1[,i2])
for (i in 1:nintqtl){
  Xi[,i] <- Xq[,i3[1,i]]*Xq[,i3[2,i]]+1
}
# GENERATE GENOTYPIC VALUES g
b[IDq] <- 0.5
#####
# BELOW: ZERO OUT length(idzero) ADDITIVE EFFECTS
idzero <- sample(IDq,floor(0.9*nqtl),replace=FALSE)
b[idzero] <- 0
#####
lb <- nmark+1
ub <- nmark+nintqtl
b[lb:ub] <- 1.0
gi <-Xi%*%b[lb:ub]
ga <- Xq%*%b[IDq]
```

```

g <- ga + gi
va <- var(ga)
vi <- var(gi)
vg <- var(g)
y <- ga+gi+rnorm(nindiv,0,sqrt(5-va-vi))
vy <- var(y)
her_a <- va/vy
her_i <- vi/vy
V <- vy*(1-her_a-her_i) # CONDITIONAL VARIANCE
cov(ga,gi)

```

```

##                [,1]
## [1,] -0.0001627963

```

```
va
```

```

##                [,1]
## [1,] 0.1254941

```

```
vi
```

```

##                [,1]
## [1,] 2.553642

```

```
vy
```

```

##                [,1]
## [1,] 4.922617

```

The results for the *additive Model* are displayed in Table 11.4. In terms of prediction ability quantified by the validating mean squared error (MSE), there is little difference between the lasso and the Bayesian mixture although the latter is consistently superior. Both outperformed the random forest. In terms of detection of causal loci, the proportion of true and false positives in the discovery set differs markedly among methods and population sizes (see the table legend for the definition of discovery set). With $n = 1000$ observations the random forest and the Bayesian mixture detect 4 and 7 of the 10 causal loci, respectively. The random forest flags 6 false positives and the Bayesian mixture none. Lasso on the other hand detects 9 of the 10 causal loci and generates 10 false positives. With the two largest population sizes, the three methods successfully detect the 10 causal loci (with the exception of the random forest that detects 9 out of the 10 with 1 false positive when fitted to $n = 5000$ observations). However, lasso, in contrast with the other two methods, flags a considerable number of false positive calls with the larger population sizes.

The results for the *interaction Model* are displayed in Table 11.5. The prediction ability of lasso and Bayesian mixture is slightly better to that of the random forest for $n = 1000$ but the reverse is true for the two larger sample sizes.

Table 11.4 *Additive Model*. Validating mean square error (MSE), True positive calls (TP) and false positive calls (FP) among flagged loci in the discovery set (TP+FP) using lasso, the Bayesian mixture and the random forest for three sample sizes (n). The discovery set was arbitrarily defined as follows. For lasso, the set of regression estimates not set equal to zero. For the Bayesian mixture, the loci whose McMC estimate of the probability of belonging to the non-zero mixture component is larger than 0.5. For random forest, the 10 loci that had the highest importance score. NULL MODEL refers to the validation MSE of predictions based on the mean of the training observations, a benchmark

n	MSE			TP/FP		
	1000	5000	10,000	1000	5000	10,000
LASSO	4.59	3.98	3.96	9/10	10/17	10/33
MIXTURE	4.12	3.89	3.71	7/0	10/0	10/0
R FOREST	5.25	4.53	4.32	4/6	9/1	10/0
NULL MODEL	5.55	5.03	5.03			

Table 11.5 *Interaction Model*. Validating mean square error (MSE), True positive calls (TP) and false positive calls (FP) among flagged loci in the discovery set (TP+FP) using lasso, the Bayesian mixture and the random forest for three sets of number of individuals (n). The discovery set was arbitrarily defined as follows. For lasso, the set of regression estimates not set equal to zero. For the Bayesian mixture, the loci whose McMC estimate of the probability of belonging to the non-zero mixture component is larger than 0.5. For random forest, the 10 loci that had the highest importance score. As point of reference, NULL MODEL refers to the validation MSE of predictions based on the mean of the training observations, a benchmark

n	MSE			TP/FP		
	1000	5000	10,000	1000	5000	10,000
LASSO	4.47	5.04	4.89	1/15	1/2	1/0
MIXTURE	4.46	5.03	4.87	1/1	1/0	1/0
R FOREST	4.61	4.95	4.69	1/9	4/6	8/2
NULL MODEL	4.61	5.16	5.01			

When $n = 1000$, all three methods detect the single additive QTL. Lasso and random forest produce a substantial amount of false positive calls; the Bayesian mixture only 1. When $n = 5000$ or 10,000, lasso and the Bayesian mixture detect the single additive QTL. On the other hand, the random forest detects not only the single additive QTL, but also flags 4 and 8 of the 10 genotypes involved in epistatic interactions, for sample sizes $n = 5000$ or 10,000, respectively. The number of false positive results in the random forest discovery set is 6 and 2, for $n = 5000$ and $n = 10,000$, respectively.

We conclude the example with a few remarks:

- The analysis confirms both conjectures in terms of prediction ability. When it comes to identification of causal loci involved in epistatic interactions, the conjecture holds provided $n > p$.
- When the relationship between signal and noise is unfavourable for detection of loci involved in epistatic interactions, a possible strategy is to fit a random forest

locally, in specific chromosome segments, searching for loci that are not captured by a regression based on linear terms.

- A common way of searching for interactions is, first, to fit a regression based on linear terms and secondly, using the detected marker loci, fit the quadratic terms. This strategy does not work when the QTL emit signals only through epistatic combinations.
- The random forest, as implemented in the example, can potentially capture individual QTL that are involved in epistatic interactions but does not flag the interaction itself. A simple discovery strategy, particularly when the number of single loci discovered is not large relative to n , is to include these individual loci in a second step constructing all possible interaction combinations using for example, simple least squares regression, or alternatively, a Bayesian mixture.

Note: The Epistatic Model

The epistatic model used in the example induces additive genetic variation through the single additive QTL. The 10 epistatic loci generate additive $\times$ additive epistatic variance only. This is stylised and holds provided that loci are in linkage equilibrium and gene frequencies are all intermediate.

The details of the model for a specific interaction genotype are as follows. Each biallelic locus with genotypes A_1A_1 , A_1A_2 and A_2A_2 is coded as $X_{11} = 1$, $X_{12} = X_{21} = 0$ and $X_{22} = -1$, respectively. Then the quadratic term is obtained as $X_{ij}X_{kl} + 1$, $i, j, k, l = 1, 2$. This generates the following genotypic codes for the additive $\times$ additive epistatic model:

	A_1A_1	A_1A_2	A_2A_2
B_1B_1	2	1	0
B_1B_2	1	1	1
B_2B_2	0	1	2

The R-code below provides an example with 10^6 individuals in the sample and a single additive $\times$ additive genotype.

```
# CODE1115
###      GENETIC MODELS
rm(list=ls()) # CLEAR WORKSPACE
set.seed(303371)
nindiv<-1000000
nqtl <- 2
nintqtl <- 1
ba <- rep(1,nqtl)
bi <- rep(1,nintqtl)
mu_y<-0
# INTERACTION GENOTYPIC MATRIX
```



```

Xi<-matrix(data=NA,nrow= nindiv,ncol= nintqtl)
Xq<-matrix(nrow= nindiv,ncol= nqtl,
  rbinom(nindiv*nqtl,size=2,p=.5)-1) # LINEAR TERMS
nr <- ncol(Xq)
i1 <- combn(nr,2)
i2 <- sample(ncol(i1),nintqtl,replace=FALSE)
i3 <- as.matrix(i1[,i2])
# CONSTRUCT INTERACTION GENOTYPE
for (i in 1:nintqtl){
  Xi[,i] <- Xq[,i3[1,i]]*Xq[,i3[2,i]]+1
}
gi <-Xi%*%bi # INTERACTION GENETIC VALUES
ga <- Xq%*%ba # ADDITIVE GENETIC VALUES
g <- ga + gi # TOTAL GENETIC VALUES
va <- var(ga)
vi <- var(gi)
vg <- var(g)
y_i <- gi+rnorm(nindiv,0,sqrt(5-vi))
y_ai <- ga + gi+rnorm(nindiv,0,sqrt(5-vi))
vy <- var(y_i)
cor(Xq)

```

```

##           [,1]           [,2]
## [1,] 1.0000000000 0.0003593763
## [2,] 0.0003593763 1.0000000000

```

```
vy
```

```

##           [,1]
## [1,] 4.982423

```

```
vi
```

```

##           [,1]
## [1,] 0.2497092

```

```
va
```

```

##           [,1]
## [1,] 0.9998697

```

```
vg
```

```

##           [,1]
## [1,] 1.249443

```

```

# FIT LINEAR REGRESSION OF
# INTERACTION GENETIC VALUES ON QTL LOCI
fa <- lm(gi ~ Xq)
# CONFIRM THAT THE MODEL DOES NOT CAPTURE ANY

```

```
# (ADDITIVE GENETIC) VARIATION
res <- summary(fa)
res$r.squared
```

```
## [1] 9.507938e-07
```

```
res$coefficients[,1:3]
```

```
##              Estimate   Std. Error   t value
## (Intercept) 1.0001798275 0.0004997101 2001.5199827
## Xq1         -0.00005506352 0.0007067582  -0.7790999
## Xq2          0.0004147291 0.0007069803   0.5866204
```

The 2×2 correlation matrix of the QTL genotypes indicates virtually no linkage disequilibrium. The output of the summary indicates that the linear regression of epistatic genetic values on the two QTL genotypes fails to capture epistatic variation and, in agreement with this, the two linear regression coefficients do not differ from zero. In other words, for these data the additive genetic variance is zero. This is no longer the case if matrix X_q of QTL genotypes is generated from a binomial distribution with $n = 2$ and $p \neq 0.5$. The subject is further explored by, for example, Hill et al. (2008) and Mäki-Tanila and Hill (2014).

11.6 Appendix

This appendix provides the details of back propagation for the single hidden layer neural network described on page 561. It is shown how matrix multiplications can be optimised computationally by means of vectorisation. The last subsection provides an example of minimisation of a cost function, when the architecture of the NN requires computation of partial derivatives following more than a single path.

Minimisation of the Cost Function: Back Propagation

Finding local minima of (11.78) requires the gradients $\partial J(\theta) / \partial W_{ji}^{(l)}$, $\partial J(\theta) / \partial b_j^{(l)}$. For the neural network of Fig. 11.10 $K = 1$, $L = 3$, $n_1 = 3$, $n_2 = 3$, $n_3 = 1$ and the predicted value for the i th datum is $h_{W,b}(x^{(i)}) = a_i^{(3)}$. Then (11.78) reduces to

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log a_i^{(3)} + (1 - y^{(i)}) \log (1 - a_i^{(3)}) \right] + \frac{\lambda}{2} \sum_{l=1}^2 \sum_{i=1}^{n_l} \sum_{j=1}^{n_{l+1}} (W_{ji}^{(l)})^2. \quad (11.84)$$

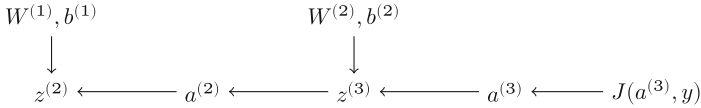


Fig. 11.13 Scheme of back propagation for the neural network displayed in Fig. 11.10, moving from right to left. Forward propagation moves from left to right. Back propagation computes partial derivatives of the cost function J with respect to $W^{(2)}, b^{(2)}, W^{(1)}, b^{(1)}$. This involves 16 parameters

In this neural network, as indicated in (11.77), the dimension of the matrix of parameters is:

$$\begin{aligned} W^{(1)} &= (3 \times 3), \\ W^{(2)} &= (1 \times 3), \\ b^{(1)} &= (3 \times 1), \\ b^{(2)} &= (1 \times 1). \end{aligned}$$

There are a total of 16 parameters including 4 bias terms.

The back propagation for this neural network is derived in detail. The different steps pertain to Fig. 11.13 that displays a flowchart of the operations and make reference to Eq. (11.76). In a first stage I start with a single record/example and the computations involve only the first term on the right hand side of (11.84) (partial derivatives for the regularisation term are of the form $\lambda W_{ji}^{(\ell)}$).

- Step 1. Start from the last layer (layer 3) and compute $\delta^{(3)}$ in two steps

$$\begin{aligned} \delta^{(3)} &= \frac{\partial J}{\partial z^{(3)}} = \frac{\partial J}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}} \\ &= - \frac{\partial [y \log a^{(3)} + (1 - y) \log (1 - a^{(3)})]}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial z^{(3)}}. \end{aligned}$$

From the first term in (11.84), dropping the superscript (i) denoting the record,

$$\begin{aligned} \frac{\partial J}{\partial a^{(3)}} &= -\frac{y}{a^{(3)}} + \frac{(1 - y)}{(1 - a^{(3)})}, \\ \frac{\partial a^{(3)}}{\partial z^{(3)}} &= a^{(3)} (1 - a^{(3)}), \\ \Rightarrow \delta^{(3)} &= a^{(3)} - y. \end{aligned}$$

These partial derivatives take the same form as (11.72) and (11.73).

- Step 2. Compute partial derivatives of J with respect to $W^{(2)} = [W_{11}^{(2)}, W_{12}^{(2)}, W_{13}^{(2)}], b^{(2)}$,

$$\begin{aligned}\frac{\partial J}{\partial W^{(2)'}} &= \frac{\partial J}{\partial z^{(3)}} \frac{\partial z^{(3)}}{\partial W^{(2)'}} , \\ \frac{\partial z^{(3)}}{\partial W_{11}^{(2)}} &= a_1^{(2)} , \\ \frac{\partial z^{(3)}}{\partial W_{12}^{(2)}} &= a_2^{(2)} , \\ \frac{\partial z^{(3)}}{\partial W_{13}^{(2)}} &= a_3^{(2)} ,\end{aligned}$$

and therefore

$$\frac{\partial J}{\partial W^{(2)}} = \delta^{(3)} a^{(2)} , \quad (11.85)$$

where $\delta^{(3)} = \partial J / \partial z^{(3)}$ is a scalar and $a^{(2)} = [a_1^{(2)}, a_2^{(2)}, a_3^{(2)}]$ is a (3×1) column vector. Above, $W^{(2)}$ is a matrix with a single row and 3 columns consistent with the definition given for $W^{(j)}$. We need also

$$\frac{\partial J}{\partial b^{(2)}} = \frac{\partial J}{\partial z^{(3)}} \frac{\partial z^{(3)}}{\partial b^{(2)}} = \delta^{(3)} \quad (11.86)$$

because $\partial z^{(3)} / \partial b^{(2)} = 1$.

- Step 3. Compute $\delta^{(2)}$ in two steps. The first step is

$$\begin{aligned}\delta^{(2)} &= \frac{\partial J}{\partial z^{(2)}} = \frac{\partial J}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} , \\ \frac{\partial J}{\partial a^{(2)}} &= \frac{\partial J}{\partial z^{(3)}} \frac{\partial z^{(3)}}{\partial a^{(2)}} , \\ \frac{\partial z^{(3)}}{\partial a_1^{(2)}} &= W_{11}^{(2)} , \\ \frac{\partial z^{(3)}}{\partial a_2^{(2)}} &= W_{12}^{(2)} , \\ \frac{\partial z^{(3)}}{\partial a_3^{(2)}} &= W_{13}^{(2)} .\end{aligned}$$

The second step is

$$\begin{aligned}\frac{\partial \mathbf{a}^{(2)}}{z^{(2)}} &= \boldsymbol{\sigma}'\left(z^{(2)}\right) = \begin{bmatrix} a_1^{(2)} \left(1 - a_1^{(2)}\right) \\ a_2^{(2)} \left(1 - a_2^{(2)}\right) \\ a_3^{(2)} \left(1 - a_3^{(2)}\right) \end{bmatrix} \\ &= \mathbf{a}^{(2)} \cdot \left(1 - \mathbf{a}^{(2)}\right),\end{aligned}$$

where $\cdot$ denotes the Hadamard (or elementwise) product and $\mathbf{1}$ is a column vector with 3 elements. Finally,

$$\boldsymbol{\delta}^{(2)} = \mathbf{W}^{(2)'} \boldsymbol{\delta}^{(3)} \cdot \boldsymbol{\sigma}'\left(z^{(2)}\right).$$

This can be written compactly as follows

$$\boldsymbol{\delta}^{(2)} = \frac{\partial J}{\partial z^{(2)}} = \begin{bmatrix} \delta_1^{(2)} \\ \delta_2^{(2)} \\ \delta_3^{(2)} \end{bmatrix} = \boldsymbol{\delta}^{(3)} \begin{bmatrix} W_{11}^{(2)} \\ W_{12}^{(2)} \\ W_{13}^{(2)} \end{bmatrix} \cdot \begin{bmatrix} a_1^{(2)} \left(1 - a_1^{(2)}\right) \\ a_2^{(2)} \left(1 - a_2^{(2)}\right) \\ a_3^{(2)} \left(1 - a_3^{(2)}\right) \end{bmatrix}.$$

- Step 4. Compute partial derivatives of J with respect to $\mathbf{W}^{(1)}$, $\mathbf{b}^{(1)}$.

$$\begin{aligned}\frac{\partial J}{\partial W_{11}^{(1)}} &= \frac{\partial J}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial W_{11}^{(1)}} = \delta_1^{(2)} a_1^{(1)}, \\ \frac{\partial J}{\partial W_{12}^{(1)}} &= \frac{\partial J}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial W_{12}^{(1)}} = \delta_1^{(2)} a_2^{(1)}, \\ &\vdots \\ \frac{\partial J}{\partial W_{33}^{(1)}} &= \frac{\partial J}{\partial z_3^{(2)}} \frac{\partial z_3^{(2)}}{\partial W_{33}^{(1)}} = \delta_3^{(2)} a_3^{(1)},\end{aligned}$$

and

$$\begin{aligned}\frac{\partial J}{\partial b_1^{(1)}} &= \frac{\partial J}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial b_1^{(1)}} = \delta_1^{(2)}, \\ \frac{\partial J}{\partial b_2^{(1)}} &= \frac{\partial J}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial b_2^{(1)}} = \delta_1^{(2)}, \\ \frac{\partial J}{\partial b_3^{(1)}} &= \frac{\partial J}{\partial z_3^{(2)}} \frac{\partial z_3^{(2)}}{\partial b_3^{(1)}} = \delta_3^{(2)}.\end{aligned}$$

For the ij th element of W of layer ℓ and for the i th bias term b of layer ℓ , we have

$$\frac{\partial J}{\partial W_{ij}^{(\ell)}} = \delta_i^{(\ell+1)} a_j^{(\ell)},$$

$$\frac{\partial J}{\partial b_i^{(\ell)}} = \delta_i^{(\ell+1)}.$$

In vectorised form, for any matrix $W^{(\ell)}$, vector $b^{(\ell)}$ in layer ℓ and activation function g , the back propagation for a single record, in standard multilayer neural networks, reduces to the following 3 equations:

$$\delta^{(\ell)} = \left(W^{(\ell+1)} \delta^{(\ell+1)} \right) \cdot g' \left(z^{(\ell)} \right),$$

$$\frac{\partial J}{\partial W^{(\ell)}} = \delta^{(\ell+1)} a^{(\ell)'} + \lambda W^{(\ell)}, \quad (11.87)$$

$$\frac{\partial J}{\partial b^{(\ell)}} = \delta^{(\ell+1)},$$

where λ is the regularisation term in Eq. (11.84). For example,

$$\begin{aligned} \frac{\partial J}{\partial W^{(1)}} &= \begin{bmatrix} \delta_1^{(2)} \\ \delta_2^{(2)} \\ \delta_3^{(2)} \end{bmatrix} \begin{bmatrix} a_1^{(1)} & a_2^{(1)} & a_3^{(1)} \end{bmatrix} + \lambda W^{(1)} \\ &= \begin{bmatrix} \delta_1^{(2)} a_1^{(1)} & \delta_1^{(2)} a_2^{(1)} & \delta_1^{(2)} a_3^{(1)} \\ \delta_2^{(2)} a_1^{(1)} & \delta_2^{(2)} a_2^{(1)} & \delta_2^{(2)} a_3^{(1)} \\ \delta_3^{(2)} a_1^{(1)} & \delta_3^{(2)} a_2^{(1)} & \delta_3^{(2)} a_3^{(1)} \end{bmatrix} + \lambda W^{(1)}, \end{aligned}$$

and in particular

$$\frac{\partial J}{\partial W_{23}^{(1)}} = \delta_2^{(2)} a_3^{(1)} + \lambda W_{23}^{(1)}.$$

With the gradients available from (11.87) and averaging the gradients over the m training records, implementation of gradient descent to find a local minimum of J requires the updating steps

$$W^{(\ell)} : W^{(\ell)} - \alpha \frac{\partial J}{\partial W^{(\ell)}},$$

$$b^{(\ell)} : b^{(\ell)} - \alpha \frac{\partial J}{\partial b^{(\ell)}},$$

where gradients are interpreted as averages over the m training records.

Vectorising Forward and Back Propagation for the Complete Training Data

Much of the material in this section is guided by Andrew Ng's course on Machine Learning, Stanford University.

Most neural network computations involve matrix multiplications. This can be exploited in order to improve computational efficiency by means of vectorisation.

The system of Eq. (11.77) for forward propagation and (11.87) for back propagation involves vectorised forms for a single input datum. Therefore the complete propagation requires a loop over all the training records. Such a loop is computationally inefficient and can be avoided by vectorising over the complete set of training records.

The complete vectorisation of forward propagation is achieved as follows. Assume that the m training records are $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$, where $x^{(i)} \in \mathbb{R}^p$ and $y^{(i)} \in \mathbb{R}$. First, stack the input feature vectors columnwise and generate the matrix X of dimension $p \times m$,

$$X = [x^{(1)} \ x^{(2)} \ \dots \ x^{(m)}]$$

where each $x^{(i)}$ is a column vector with p elements. I illustrate with the neural network of Fig. 11.10, with $p = n_1 = 3, n_2 = 3, n_3 = 1$ and X is $n_1 \times m$. Then the complete forward propagation involves computation of

1. $Z^{(2)} = W^{(1)}X + b^{(1)}$, $Z^{(2)} : n_2 \times m = 3 \times m$; $W^{(1)} : n_2 \times n_1 = 3 \times 3$; $X : n_1 \times m = 3 \times m$; $b^{(1)} : n_2 \times m = 3 \times m$
2. $A^{(2)} = \sigma(Z^{(2)})$, $A^{(2)} : n_2 \times m = 3 \times m$
3. $Z^{(3)} = W^{(2)}A^{(2)} + b^{(2)}$, $Z^{(3)} : n_3 \times m = 1 \times m$; $W^{(2)} : n_3 \times n_2 = 1 \times 3$; $b^{(2)} : n_3 \times m = 1 \times m$
4. $A^{(3)} = \sigma(Z^{(3)})$, $A^{(3)} : n_3 \times m = 1 \times m$

In step 1, $b^{(1)}$ is the original 3×1 column vector with the single column repeated m times. Similarly in step 3, $b^{(2)}$ is the original scalar, repeated columnwise m times. Computation of the 4 steps generates the forward propagation for the complete training data. The activation function σ in steps 2 and 4 can be arbitrary but in the present example it is the sigmoid function.

To compute back propagation in vectorised form, define

- $\Delta Z^{(\ell)}$: vectorised version of $\delta^{(\ell)} = \partial J / \partial z^{(\ell)}$
- $\Delta W^{(\ell)}$: vectorised version of $\partial J / \partial W^{(\ell)}$
- $\Delta b^{(\ell)}$: vectorised version of $\partial J / \partial b^{(\ell)}$
- $Y = (y^{(1)}, y^{(2)}, \dots, y^{(m)})$: $1 \times m$ vector of observed records

The complete back propagation involves computation of

1. $\Delta Z^{(3)} = A^{(3)} - Y$, $\Delta Z^{(3)} : 1 \times m$

2. $\Delta W^{(2)} = \frac{1}{m} \Delta Z^{(3)} A^{(2)'} + \lambda W^{(2)}$, $\Delta W^{(2)} : 1 \times 3$; $A^{(2)'} : m \times 3$; $W^{(2)} : 1 \times 3$
3. $\Delta b^{(2)} = \frac{1}{m} \Delta Z^{(3)} 1$, $\Delta b^{(2)} : 1 \times 1$ and 1 is a $m \times 1$ column vector of 1's ($\Delta b^{(2)}$ is the result of averaging the elements of the row vector $\Delta Z^{(3)}$ over the number of training records)
4. $\Delta Z^{(2)} = W^{(2)'} \Delta Z^{(3)} \cdot \sigma'(Z^{(2)})$, $\Delta Z^{(2)} : 3 \times m$; $W^{(2)'} : 3 \times 1$; $\Delta Z^{(3)} : 1 \times m$
 $\sigma'(Z^{(2)}) : 3 \times m$
5. $\Delta W^{(1)} = \frac{1}{m} \Delta Z^{(2)} X' + \lambda W^{(1)}$, $\Delta W^{(1)} : 3 \times 3$; $\Delta Z^{(2)} : 3 \times m$; $X' : m \times 3$; $\lambda W^{(1)} : 3 \times 3$
6. $\Delta b^{(1)} = \frac{1}{m} \Delta Z^{(2)} 1$, $\Delta b^{(1)} : 3 \times 1$; 1 is a $m \times 1$ column vector of 1's ($\Delta b^{(1)}$ is the result of averaging the elements of each of the 3 rows of matrix $\Delta Z^{(2)}$ over the number of columns).

Computation of the 6 steps generate the back propagation for the complete training data and as in the case of forward propagation, the activation function in step 4 is arbitrary. The result in step 1 is specific for the sigmoid function used in the output layer.

The R-code CODE1110 fits the neural network in vectorised form. The evolution of the cost function can be displayed plotting `result[,2]`.

Back Propagation with Multiple Paths

Neither of the two examples of back propagation discussed so far involved the computation of partial derivatives following more than one path along the lines indicated in (11.67). To illustrate a case where this is necessary, consider the 3-layer neural network displayed in Fig. 11.14 with two output classes rather than the single output class neural network of Fig. 11.10 (another example could involve a neural network with a single output neuron and more than 3 layers).

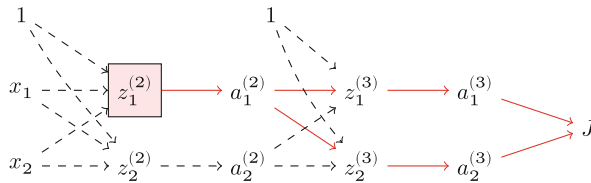


Fig. 11.14 A three layer neural network with 2 output classes, $a_1^{(3)}$, $a_2^{(3)}$. The computation of the partial derivative of the cost function J with respect to $z_1^{(2)}$ must trace two paths (marked in red), since a change in $z_1^{(2)}$ has an effect on J via $z_1^{(3)}$ and via $z_2^{(3)}$

The forward propagation for a single data point for this network generates

$$\begin{aligned} z_i^{(2)} &= W_{i1}^{(1)} x_1 + W_{i2}^{(1)} x_2 + b_i^{(1)}, \quad i = 1, 2, \\ z_i^{(3)} &= W_{i1}^{(2)} a_1^{(2)} + W_{i2}^{(2)} a_2^{(2)} + b_i^{(2)}, \quad i = 1, 2, \\ a_i^{(2)} &= \sigma \left(z_i^{(2)} \right) \quad i = 1, 2, \\ a_i^{(3)} &= \sigma \left(z_i^{(3)} \right) \quad i = 1, 2. \end{aligned}$$

Consider the computation of $\partial J / \partial z_1^{(2)} = \delta_1^{(2)}$. This requires

$$\begin{aligned} \frac{\partial J}{\partial z_1^{(2)}} &= \frac{\partial J}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\ &= \left[\frac{\partial J}{\partial z_1^{(3)}} \frac{\partial z_1^{(3)}}{\partial a_1^{(2)}} + \frac{\partial J}{\partial z_2^{(3)}} \frac{\partial z_2^{(3)}}{\partial a_1^{(2)}} \right] \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\ &= \left[\delta_1^{(3)} W_{11}^{(2)} + \delta_2^{(3)} W_{21}^{(2)} \right] a_1^{(2)} \left(1 - a_1^{(2)} \right). \end{aligned} \quad (11.88)$$

This can be generated directly using (11.87). Indeed, for this example, $\ell = 2$,

$$W^{(2)} = \begin{bmatrix} W_{11}^{(2)} & W_{12}^{(2)} \\ W_{21}^{(2)} & W_{22}^{(2)} \end{bmatrix}; \quad \delta^{(3)} = \begin{bmatrix} \delta_1^{(3)} \\ \delta_2^{(3)} \end{bmatrix}; \quad g' \left(Z^{(2)} \right) = \begin{bmatrix} a_1^{(2)} \left(1 - a_1^{(2)} \right) \\ a_2^{(2)} \left(1 - a_2^{(2)} \right) \end{bmatrix}$$

and substituting in

$$\delta^{(\ell)} = \left(W^{(\ell)'} \delta^{(\ell+1)} \right) \cdot g' \left(z^{(\ell)} \right)$$

yields

$$\delta^{(2)} = \frac{\partial J}{\partial z^{(2)}} = \begin{bmatrix} \delta_1^{(2)} \\ \delta_2^{(2)} \end{bmatrix} = \begin{bmatrix} \delta_1^{(3)} W_{11}^{(2)} + \delta_2^{(3)} W_{21}^{(2)} \\ \delta_1^{(3)} W_{12}^{(2)} + \delta_2^{(3)} W_{22}^{(2)} \end{bmatrix} \cdot \begin{bmatrix} a_1^{(2)} \left(1 - a_1^{(2)} \right) \\ a_2^{(2)} \left(1 - a_2^{(2)} \right) \end{bmatrix}.$$

Part III

Exercises and Solutions

Chapter 12

Exercises



12.1 Likelihood Exercises I

Exercise 1

In a binomial experiment with n trials and probability of success θ , x successes are observed. The setup could represent a trial designed to estimate the proportion of individuals in a population that suffer from a particular disease. A sample of n randomly selected individuals is taken and the number diseased, x , is recorded.

- (a) Write down the probability mass function of the data
- (b) Write down the likelihood function and the loglikelihood function of θ .
- (c) Derive the maximum likelihood estimator of θ . Assume $n = 10$ and $x = 8$. What is the maximum likelihood estimate of θ ?
- (d) Derive a 95% confidence interval for θ using the asymptotic approximation to the ML estimator of θ .

Exercise 2

In the above experiment use the transformation (known as the logit or logodds)

$$\beta = g(\theta) = \ln \left(\frac{\theta}{1 - \theta} \right). \quad (12.1)$$

The inverse transformation is

$$\theta = g^{-1}(\beta) = \frac{\exp(\beta)}{1 + \exp(\beta)}. \quad (12.2)$$

- Write the likelihood based on β .
- Plot the likelihood based on β and the likelihood based on θ and compare both.
- Derive the ML estimator and the ML estimate of β based on the likelihood (a).
- What is the 95% confidence interval for β ?
- Finally, transform the confidence interval obtained in (d) back in terms of θ . How does this confidence interval compare with that obtained in (1d)? Which do you believe is more reliable?

Exercise 3

Suppose you observe the following $n = 10$ iid records each from $N(\mu, \sigma^2)$

0.88; 1.07; 1.27; 1.54; 1.91; 2.27; 3.84; 4.50; 4.64; 5.41,

$\sum_{i=1}^n y_i = 27.33$; $\sum_{i=1}^n (y_i - \hat{\mu})^2 = 25.8052$ where $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n y_i$, $n = 10$.

- Write down the likelihood and the loglikelihood of (μ, σ^2) .
- Derive the ML estimators and the ML estimates of μ and σ^2 .
- Derive the observed information matrix $I(\mu, \sigma^2)$ and evaluate it at the ML estimates.
- Derive the asymptotic variance of $(\hat{\mu}, \hat{\sigma}^2)$ based on 3c.
- Derive Fisher's (expected) information of (μ, σ^2) and evaluate it at the ML estimates.
- Find the asymptotic variance of $(\hat{\mu}, \hat{\sigma}^2)$ based on Fisher's (expected) information obtained in 3e.
- According to asymptotic theory, $\hat{\sigma}^2$ is normally distributed with mean equal to σ^2 and variance given by the result you derived in 3f. What is the exact (small sample) distribution of $\hat{\sigma}^2$ and what is its mean and variance?
- Compute an exact (small sample) 95% confidence interval for σ^2 and an approximate 95% confidence interval based on asymptotic results.

Exercise 4

Consider the following hypothetical data collected from the field:

$$x_1, x_2, \dots, x_m, x_{m+1}, \dots, x_n,$$

$$y_1, y_2, \dots, y_m.$$

There are m bivariate observations $\{(x_i, y_i), i = 1, 2, \dots, m\}$ and $(n - m)$ univariate records. However sampling is not at random: the structure of the data

mimics records on first and second lactation, where only individuals with the highest first lactation records $(x_1, \dots, x_m)$ were allowed to produce a second lactation $(y_1, \dots, y_m)$.

Using these data, the analyst wishes to estimate various parameters associated with milk production, such as the mean and variance of first and second lactation records, their correlation, and the regression coefficient of second lactation records on first lactation records. The objective of the exercise is to show (here, by means of example) how the construction of the “correct” likelihood model avoids the potential problem associated with the non-random sampling mechanism that generated the data.

Let $\theta = (\mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx})$. Ignoring the selection mechanism and the $n - m$ univariate records $x_{m+1}, \dots, x_n$, one can assume that each of the m pairs of records is bivariate normal

$$(y_i, x_i) | \theta \sim N \left(\begin{bmatrix} \mu_y \\ \mu_x \end{bmatrix}, \begin{bmatrix} \sigma_{yy} = 4 & \sigma_{yx} = 2.4 \\ \sigma_{yx} = 2.4 & \sigma_{xx} = 4 \end{bmatrix} \right), \quad (12.3)$$

$$i = 1, 2, \dots, m,$$

with $\mu_y = 10$, $\mu_x = 15$ and the pairs (y_i, x_i) are *iid*. A standard result is that, given (12.3),

$$y_i | x_i, \theta \sim N (E(y_i | x_i, \theta), \text{Var}(y_i | x_i, \theta)), \quad (12.4)$$

where

$$\begin{aligned} E(y_i | x_i, \theta) &= \mu_y + \frac{\sigma_{yx}}{\sigma_{xx}} (x_i - \mu_x) \\ &= 10 + 0.6 (x_i - 15) \end{aligned}$$

and

$$\begin{aligned} \text{Var}(y_i | x_i, \theta) &= \sigma_{yy} - \sigma_{yx} (\sigma_{xx})^{-1} \sigma_{yx} \\ &= 4 - 2.4^2/4. \end{aligned}$$

For the m pair of records the joint pdf is

$$p(x, y | \mu_y, \theta) = \left(2\pi \sigma_{yy} \sigma_{xx} (1 - \rho^2) \right)^{-\frac{m}{2}} \quad (12.5)$$

$$\exp \left[-\frac{1}{2(1 - \rho^2)} \right] \quad (12.6)$$

$$\left(\frac{\sum_{i=1}^m (x_i - \mu_x)^2}{\sigma_{xx}} + \frac{\sum_{i=1}^m (y_i - \mu_y)^2}{\sigma_{yy}} - 2\rho \frac{\sum_{i=1}^m (x_i - \mu_x)(y_i - \mu_y)}{(\sigma_{xx})^{\frac{1}{2}} (\sigma_{yy})^{\frac{1}{2}}} \right) \Bigg], \quad (12.7)$$

where the coefficient of correlation $\rho = \sigma_{xy}/(\sigma_{xx}\sigma_{yy})^{0.5}$. Ignoring the sampling mechanism that gave rise to the data, the likelihood is proportional to (12.7) and the closed form likelihood estimators are given by

$$\hat{\mu}_x = \frac{1}{m} \sum_{i=1}^n x_i, \quad \hat{\mu}_y = \frac{1}{m} \sum_{i=1}^m y_i, \quad (12.8a)$$

$$\hat{\sigma}_{xx} = \frac{1}{m} \sum_{i=1}^m (x_i - \hat{\mu}_x)^2, \quad \hat{\sigma}_{yy} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{\mu}_y)^2, \quad (12.8b)$$

$$\hat{\rho} = \frac{\hat{\sigma}_{xy}}{\sqrt{\hat{\sigma}_{xx}\hat{\sigma}_{yy}}}, \quad \hat{\sigma}_{xy} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{\mu}_y)(x_i - \hat{\mu}_x). \quad (12.8c)$$

The R-code below generates bivariate observations where some of the records in one of the variables are missing. The strategy makes use of the factorisation

$$p(x, y) = p(y|x)p(x).$$

```
rm(list=ls()) # Clear the workspace
set.seed(772231)
n <- 5000 # Number of records generated for Y_1
p <- 0.20 # Proportion to be selected
m <- round(p*n) # Number of records selected
y1 <- rnorm(n, mean=10, sd=2) # Generate random variable Y_1
y1_sort <- sort(y1, decreasing=TRUE) # Sort Y_1
y1s <- y1_sort[1:m] # The highest m Y_1 records are kept
#length(y1) # Number of records in Y_1
#length(y1s) # Number of records in the selected subset of Y_1
# Generate Y_2
y2 <- rnorm(length(y1s), mean=15+0.6*(y1s-10), sd=sqrt(4-(2.4^2)/4))
```

- (a) Using only the fraction of the data that has no missing offspring (i.e., the m pairs (y_i, x_i)), estimate the parameters with estimators in (12.8). From the same subset of data estimate also the regression of y on x , using

$$\hat{b}_{yx} = \frac{\hat{\sigma}_{xy}}{\hat{\sigma}_{xx}}. \quad (12.9)$$

The regression is a parameter that can be directly derived from the set of parameters in likelihood (12.7) or from (12.3). That is

$$b_{yx} = \frac{\sigma_{xy}}{\sigma_{xx}} = 0.6.$$

Similarly, the correlation coefficient is

$$\rho = \frac{\sigma_{xy}}{\sigma_{xx}\sigma_{yy}} = 0.6. \quad (12.10)$$

How do the estimates obtained using this approach compare with the parameters defined in (12.3)? An eyeball evaluation is adequate at this stage.

- (b) Write down the correct likelihood and loglikelihood accounting for the observed pattern of data based on parameterisation $(\mu_x, \sigma_{xx}, \beta_0, \beta_1, \sigma_{y..x})$.
- (c) Using the likelihood in (b) obtain ML estimators and ML estimates of $\phi = (\mu_x, \sigma_{xx}, \beta_0, \beta_1, \sigma_{y..x})$ by analytical maximisation of the loglikelihood. Back-transform these estimates in terms of $\theta = (\mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx})$. How do the estimates obtained using this second approach compare with the true value of the parameters defined in (12.3)?

Exercise 5

This is a theoretical exercise designed to train the skills necessary to construct and to maximise a likelihood function in the absence of random sampling. An exponential distribution is used often applied to study waiting times until the next event. For example in a clinical trial, one may be interested in studying how a new drug affects time until a relapse of the disease. At the end of the trial, some patients may not have shown signs of relapse; this gives rise to censoring.

In the simple exponential sampling model used here, expectations can be obtained in closed form. The censoring mechanism is as follows: each observation is compared to a censoring or truncation point c (the termination time of the clinical trial). If it is smaller than or equal to c it is registered as such; if it is larger than c (no relapse observed), it is set equal to c . Since selection depends on the censored records, it is not ignorable and must be incorporated in the likelihood in order to draw inferences correctly. To set the background, the first part of the example deals with standard inferences based on a random sample from an exponential distribution in the absence of censoring.

1. A random sample of size n is obtained from *iid* draws from an exponential distribution. The pdf of one observation is

$$p(y_i|b) = b \exp(-by_i), \quad b, y_i > 0.$$

The parameter b is known as the rate, the focus of inference.

It can readily be verified that

$$\int_0^{\infty} p(y_i|b) dy_i = 1.$$

The mean and variance of Y_i are

$$E(Y_i|b) = \frac{1}{b}$$

and

$$\text{Var}(Y_i|b) = \frac{1}{b^2},$$

respectively, (here I distinguish the random variable Y from its realised value y). The expression for the mean indicates that mean relapse time is inversely proportional to the rate, as expected.

The pdf associated with the n independent observations is

$$\begin{aligned} p(y|b) &= \prod_{i=1}^n p(y_i|b) \\ &= b^n \exp\left(-b \sum_{i=1}^n y_i\right). \end{aligned} \quad (12.11)$$

- (a) Write the loglikelihood
 - (b) Write the score
 - (c) Derive the maximum likelihood estimator of b
 - (d) Derive Fisher's information from $-E[l''(b|Y)]$
2. Assume that after the random sample of size n has been drawn, $(n-r)$ observations larger than a known point c are censored. Clearly, $r \leq n$. There are r observed records which happen to be smaller than c , and $(n-r)$ records whose values are equal to c .

The data generating mechanism can be written as follows. To derive the pdf of a single observation, define a latent variable X_i whose density is $p(x_i|b) = b \exp(-bx_i)$, $b, x_i > 0$ and let the data be the observed values of the random variable

$$Y_i = \begin{cases} c & \text{if } X_i \geq c \\ X_i & \text{if } X_i < c. \end{cases}$$

Note that

$$\begin{aligned}
 \Pr(X_i \geq c) &= 1 - \int_0^c p(x_i|b) dx_i \\
 &= \exp(-bc) \\
 &= 1 - F(c), \\
 \Pr(X_i < c) &= \int_0^c p(x_i|b) dx_i \\
 &= 1 - \exp(-bc) \\
 &= F(c).
 \end{aligned}$$

Then the pdf of Y_i is

$$\begin{aligned}
 p_{Y_i}(y_i|b) &= p_{X_i}(y_i|Y_i < c, b) \Pr(Y_i < c|b) \\
 &\quad + \Pr(Y_i = c|b, Y_i = c) \Pr(Y_i = c|b) \\
 &= I(y_i < c) p_{X_i}(y_i|b) + I(y_i = c) \Pr(Y_i = c|b) \\
 &= I(y_i < c) b \exp(-by_i) + I(y_i = c) \exp(-bc), \tag{12.12}
 \end{aligned}$$

and due to independence, $p(y|b) = \prod_{i=1}^n p(y_i|b)$. Expression (12.12) is a proper pdf. Indeed,

$$\begin{aligned}
 \int p(y_i|b) dy_i &= \int I(y_i < c) b \exp(-by_i) dy_i + \exp(-bc) \\
 &= 1 - \exp(-bc) + \exp(-bc) = 1.
 \end{aligned}$$

- (a) Write the loglikelihood
- (b) Write the score (loglikelihood)
- (c) Derive the maximum likelihood estimator of b
- (d) Derive Fisher's information from $-E[l''(b|Y)]$

12.2 Likelihood Exercises II

Exercise 1

The R-code below generates 30 binary records mimicking the presence-1/absence-0 of disease among subjects exposed to (scaled) levels of a drug, the covariate. The analyst could be interested in learning how the probability of the disease is affected by the levels of the drug.

```
rm(list=ls()) # Clear the workspace
set.seed(12371)
# CREATE BINARY DATA
mu <- -2
beta <- 0.7
cov <- rnorm(30,2,3) # GENERATE THE COVARIATE
xb <- cov*beta
p1 <- pnorm(mu+xb) # PROBABILITIES ACCORDING TO PROBIT MODEL
#p1 <- rbeta(30,2,2)
dat1 <- cbind(rbinom(30,1,p1),round(cov,digits=0)) # CREATE DATA
colnames(dat1) <- c("Y", "X")
d <- data.frame(dat1)
head(d)
```

```
##   Y X
## 1 1 7
## 2 1 6
## 3 1 3
## 4 0 1
## 5 1 4
## 6 0 0
```

In an initial analysis one can study the following approximate relationship between the probability of disease and the level of the drug. Let

$$\theta_i = \Pr(Y_i = 1 | \mu, \beta, x_i),$$

be the probability that the i th individual shows the disease, given the covariate and parameters μ and β .

- Sort the covariate x and then create 5 groups where group 1 includes the 6 individuals exposed to the lowest levels of the drug and group 5 includes individuals exposed to the highest levels of the drug. Plot the proportion of individuals that show the disease in each group versus the mean level of the group. Does the relationship look approximately linear?
- Fit a model of the form

$$Y_i = \mu + \beta x_i + e_i, \quad (12.13)$$

and estimate μ and β by least squares. In this expression Y_i is a proxy for θ_i and represents the number of cases that show the disease in group i , divided by the number of cases in group i (i.e., 6 cases) and x_i is the average level of the drug in group i over the 6 cases. What is the probability that a future individual will show the disease when exposed to a scaled drug level equal to -3 , 1 , or 9 ?

- Fit the logistic regression

$$\text{logit}(\theta_i) = \ln\left(\frac{\theta_i}{1 - \theta_i}\right) = \mu + \beta x_i. \quad (12.14)$$

From this expression, solving for the probability θ_i (the probability that $Y_i = 1$)

$$\text{logit}^{-1}(\theta_i) = \theta_i = \frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)}. \quad (12.15)$$

The parametrisation used is

$$Y_i = \begin{cases} 1 & \text{if } u_i > 0, \\ 0 & \text{if } u_i < 0, \end{cases}$$

$$u_i = \mu + \beta x_i + e_i,$$

where the independent errors e_i follow the standard logistic distribution. Therefore,

$$\begin{aligned} \Pr(Y_i = 1 | \mu, \beta, x_i) &= \Pr(u_i > 0 | \mu, \beta, x_i) \\ &= \Pr(\mu + \beta x_i + e_i > 0 | \mu, \beta, x_i) \\ &= \Pr(e_i > -\mu - \beta x_i | \mu, \beta, x_i) \\ &= \Pr(e_i < \mu + \beta x_i | \mu, \beta, x_i) \\ &= \int_{-\infty}^{\mu + \beta x_i} p(e_i) de_i \\ &= \int_{-\infty}^{\mu + \beta x_i} \frac{\exp(e_i)}{[1 + \exp(e_i)]^2} de_i \\ &= \frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)} \end{aligned} \quad (12.16)$$

as in (12.15), indicating that in the binary case, the logit and the threshold are equivalent models.

With the present parametrisation the joint density of the data is

$$p(y | \mu, \beta, x) = \prod_{i=1}^{30} \left[\frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)} \right]^{y_i} \left[\frac{1}{1 + \exp(\mu + \beta x_i)} \right]^{1-y_i}. \quad (12.17)$$

- (c1) Write the loglikelihood of the parameters
- (c2) Calculate first and second derivatives of the loglikelihood
- (c3) Fit the model (12.14) using Newton-Raphson and obtain ML estimates of μ and of β .
- (c4) What is the asymptotic variance covariance matrix of the ML estimators of μ and of β ?
- (c5) What is the probability that a future individual will show the disease when exposed to a scaled drug level equal to -3 , 1 , or 9 ? Compare with what you obtained in 1b.

Table 12.1 Data from a bioassay experiment

Dose	Deaths	Sample size
-0.86	0	5
-0.30	1	5
-0.05	3	5
0.73	5	5

Exercise 2

Fit the probit regression model to the previous data using the EM algorithm.

Exercise 3

Table 12.1, taken from Gelman et al. 1995, shows data from a bioassay experiment. Four different doses of a drug (measured as $\ln(g/ml)$) are administered to 20 animals, 5 in each dose and the number of deaths is recorded. The objective of the experiment is to study the toxicity of the drug and how it varies with dose.

Let y_i represent the number of deaths observed out of n_i with dose level x_i . It will be assumed that y_i is binomial $Bi(n_i, \theta_i)$, where θ_i is modelled with the logistic regression

$$\text{logit}(\theta_i) = \ln\left(\frac{\theta_i}{1 - \theta_i}\right) = \beta_0 + \beta_1 x_i, \quad i = 1, \dots, 4. \quad (12.18)$$

The inverse function yields

$$\text{logit}^{-1}(\theta_i) = \theta_i = \frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)}. \quad (12.19)$$

The likelihood takes the form

$$\begin{aligned} L(\beta_0, \beta_1 | n, x) &\propto \prod_{i=1}^4 \left[\frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)} \right]^{y_i} \\ &\times \left[1 - \frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)} \right]^{n_i - y_i} \end{aligned} \quad (12.20)$$

where $n = (n_1, \dots, n_4)'$, $x = (x_1, \dots, x_4)'$. The loglikelihood, after a little simplification reduces to

$$\ell(\beta_0, \beta_1 | n, x) = \sum_{i=1}^4 \{y_i (\beta_0 + \beta_1 x_i) - n_i \ln(1 + \exp(\beta_0 + \beta_1 x_i))\}. \quad (12.21)$$

- (i) Obtain ML estimates of β_0 and β_1 for the logit model using Newton-Raphson.
- (ii) Obtain ML estimates of β_0 and β_1 for the probit model using the EM algorithm.
- (iii) Suppose that within each dose, the five animals happen to be full-sibs. What modifications to the loglikelihood (12.20) are necessary to account for the correlated structure of the data?

Example 4

The R-code below generates genomic data: 2000 nominally unrelated individuals (meaning distantly related) with 20,000 genetic markers available for each individual. Modern data sets include of the order of 200,000 individuals and 2 million marker genotypes per individual. This example scales down by factor 100. The simulated data assumes that all the genetic markers have an effect on the trait of the same magnitude.

In a preliminary analysis one could be interested in fitting a classic genomic model to estimate the genomic variance.

Assuming that records have zero mean, the genomic model is

$$y|g, \sigma_e^2 \sim N(g, I\sigma_e^2), \quad (12.22a)$$

$$g|W, \sigma_g^2 \sim SN(0, G\sigma_g^2), \quad (12.22b)$$

$$G = \frac{1}{m}WW', \quad (12.22c)$$

$$W = \{W_{ij}\}. \quad (12.22d)$$

In these expressions, m is the number of SNPs and W_{ij} is the label for the j th maker in individual i , ($i = 1, \dots, n$; $j = 1, \dots, m$; $m > n$)

$$W_{ij} = \frac{X_{ij} - E(X_{ij})}{\sqrt{\text{Var}(X_{ij})}}, \quad X_{ij} = 0, 1, 2.$$

The eigenvalue decomposition of WW' is

$$\begin{aligned} WW' &= U\Delta U' \\ &= \sum_{i=1}^n \lambda_i U_i U_i', \end{aligned}$$

where $U = [U_1, U_2, \dots, U_n]$, of order $n \times n$ is the matrix of eigenvectors of WW' , U_j is the j th column (dimension $n \times 1$) and Δ is a diagonal matrix with elements equal to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ associated to the n eigenvectors.

The loglikelihood, up to an additive constant, is

$$\ln p(k, \sigma_e^2 | y, W) = -\frac{1}{2} \left\{ n \ln \sigma_e^2 + \sum_{i=1}^n \ln(\lambda_i k + 1) + \frac{1}{\sigma_e^2} \sum_{i=1}^n \frac{\tilde{y}_i^2}{\lambda_i k + 1} \right\}, \quad (12.23)$$

where $\tilde{y}_i$ is the i th element of the $n \times 1$ column vector $\tilde{y} = U'y$.

- (i) Obtain ML estimates of σ_e^2 and k using Newton-Raphson.
- (ii) Obtain ML estimates using the R-function `OPTIM` and compare with the Newton-Raphson computations.
- (iii) The loglikelihood (12.23) is parameterised in terms of σ_e^2 and $k = \sigma_g^2 / \sigma_e^2$. Obtain the ML estimate of σ_g^2 and its asymptotic variance from the Newton-Raphson output in (i).
- (iv) Obtain ML estimates of σ_e^2 , k and σ_g^2 using the EM algorithm.

```
# LIKELIHOOD PROBLEMS II QUESTION 4
# DATA BASED ON GENOMIC MODEL; OBTAIN SVD OF WW'(1/m)
rm(list=ls()) # CLEAR WORKSPACE
set.seed(1327)
nindiv<-2000
nmark<-20000
nt <- nindiv*nmark
X<-matrix(nrow=nindiv,ncol=nmark,rbinom(n=nt,size=2,p=.5))
stdev <- matrix(data=NA,nrow= nmark,ncol=1)
W <- matrix(data=NA,nrow= nindiv,ncol=nmark)
U <- matrix(data=NA,nrow= nindiv,ncol= nindiv)
G<-matrix(data=NA,nrow= nindiv,ncol= nindiv)
cm <- colMeans(X)
# MATRIX OF STANDARDISED MARKER GENOTYPE CODES
for (i in 1:nmark)
{
  W[,i] <- ( X[,i]-cm[i]) / sd(X[,i])
}
# COULD USE INSTEAD:
# W <- scale(X)
#qr(X)$rank
#qr(W)$rank
# GENERATE nindiv GENOMIC VALUES N(0,(1/nmark)WW'*10); Vg=10
g <- (1/sqrt(nmark))*W%*%rnorm(nmark,mean=0,sd=sqrt(10))
# GENERATE nindiv PHENOTYPES WITH MEAN 0, VAR=10+15,
# HERITABILITY=10/(10+15)=0.4
#PARAMETER k = Vg/Ve = 10/15 =0.67
y <- g+rnorm(nindiv,mean=0,sd=sqrt(15))
# GENOMIC RELATIONSHIP MATRIX G
G <- (1/nmark)*W%*%t(W)
```

```
# SVD OF G
EVD <- eigen(G)
names(EVD)
head(EVD$values)
U <- EVD$vector
val <- EVD$values
val[length(y)] <- 0
D <- diag(val, nrow=nindiv)
ytilde <- t(U)%*%y
dim(ytilde)
#END OF GENERATION OF DATA
```

12.3 Bayes Exercises I

Exercise 1

You are given the following data y drawn from a normal distribution

$$y = (45.83, 50.37, 50.06, 51.59, 48.43, 52.75, 42.92, 48.57, 46.18, 50.20)'.$$

Assuming the likelihood is proportional to

$$y|\mu, \sigma^2 \sim N(1\mu, I\sigma^2),$$

that μ and σ^2 are a priori independent and their prior distributions are

$$[\mu] \propto \text{constant}, \quad (12.24a)$$

$$[\sigma^2] \propto \frac{1}{\sigma^2}, \quad (12.24b)$$

then the posterior distribution can be written

$$\begin{aligned} p(\mu, \sigma^2|y) &\propto p(y|\mu, \sigma^2) p(\sigma^2) p(\mu) \\ &\propto \exp\left[-\frac{\sum_{i=1}^n (y_i - \mu)^2}{2\sigma^2}\right] (\sigma^2)^{-(\frac{n}{2}+1)}. \end{aligned} \quad (12.25)$$

- (i) Derive analytically the marginal distributions $[\mu|y]$ and $[\sigma^2|y]$.

- (ii) From these marginal distributions obtain $E[\mu|y]$, $\text{Var}[\mu|y]$, $E[\sigma^2|y]$, $\text{Var}[\sigma^2|y]$ and the mode of $[\sigma^2|y]$.
- (iii) Compute 95% posterior intervals for μ and σ^2 .
- (iv) Derive the asymptotic posterior distribution $[\mu, \sigma^2|y]$
- (v) Obtain the 95% posterior intervals for μ and σ^2 based on the asymptotic posterior distribution $[\mu, \sigma^2|y]$.

Exercise 2

Using data y in Exercise 1, write a computer program to implement the above model with

- (i) A Metropolis-Hastings, single-site updating of μ and σ^2 . From the output obtain Monte Carlo estimates of $E[\mu|y]$, $\text{Var}[\mu|y]$, $E[\sigma^2|y]$, $\text{Var}[\sigma^2|y]$ and 95% posterior intervals for μ and σ^2 .
- (ii) A Metropolis-Hastings, joint updating of μ and σ^2 . From the output obtain Monte Carlo estimates of $E[\mu|y]$, $\text{Var}[\mu|y]$, $E[\sigma^2|y]$, $\text{Var}[\sigma^2|y]$ and 95% posterior intervals for μ and σ^2 .
- (iii) For the Metropolis-Hastings single-site updating algorithm, use two sets of tuning parameters. In the first set, try $k_\mu = 0.19$, $k_{\sigma^2} = 1$. In the second set, try $k_\mu = 19$, $k_{\sigma^2} = 9$. Compute the Monte Carlo variance, the effective chain length (effective sample size) and the integrated autocorrelation for μ and for σ^2 for both strategies. What do you observe? Can you provide an explanation?

Exercise 3

Use the vector of records y from Exercise 1 and write a computer program to implement the model using Gibbs sampling.

- (i) From the output of the Gibbs sampler compute Monte Carlo estimates of $E[\mu|y]$, $E[\sigma^2|y]$ and 95% posterior intervals for μ and σ^2 . Calculate also Monte Carlo variances of the estimator of $E[\mu|y]$ and $E[\sigma^2|y]$.
- (ii) Compute the effective chain length and the integrated autocorrelation for μ and for σ^2 .

12.4 Bayes Exercises II

Exercise 1

Consider the data generated in Likelihood Exercises II, Exercise 1, on page 613. As before, let

$$\theta_i = \Pr(Y_i = 1 | \mu, \beta, x_i),$$

be the probability that the i th individual shows the disease, given the covariate and parameters μ and β .

The logistic regression model is

$$\text{logit}(\theta_i) = \ln\left(\frac{\theta_i}{1 - \theta_i}\right) = \mu + \beta x_i. \quad (12.26)$$

Solving for θ_i gives

$$\theta_i = \text{logit}^{-1}(\theta_i) = \frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)}.$$

This model can be interpreted directly as a nonlinear model for the probability θ_i or also, indirectly, in terms of an unobserved latent variable (or underlying variable) u_i . In this second formulation, the discrete observation Y_i is associated with the latent variable u_i as follows:

$$Y_i = \begin{cases} 1 & \text{if } u_i > 0 \\ 0 & \text{if } u_i < 0 \end{cases}, \text{ and} \\ u_i = \mu + \beta x_i + e_i,$$

where the e_i 's are *iid* residuals from a logistic probability distribution. Therefore in this case,

$$\begin{aligned} \theta_i &= \Pr(Y_i = 1 | \mu, \beta, x_i) = \Pr(u_i > 0 | \mu, \beta, x_i) \\ &= \frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)}, \end{aligned} \quad (12.27)$$

as in (12.16). The likelihood is proportional to

$$p(\mu, \beta | y, x) = \prod_{i=1}^{30} \left[\frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)} \right]^{y_i} \left[\frac{1}{1 + \exp(\mu + \beta x_i)} \right]^{1-y_i} \quad (12.28)$$

and assuming a uniform prior for (μ, β) , (12.28) is also proportional to the posterior distribution of (μ, β) .

- (i) Write a program to implement this logistic model using the Metropolis-Hastings algorithm with joint updating for (μ, β) .
- (ii) Compute MC estimators of the mean, variance and 95% posterior intervals of $[\mu|y]$ and $[\beta|y]$.
- (iii) Compute MC standard errors and effective chain lengths of $\hat{E}(\mu|y)$ and $\hat{E}(\beta|y)$.
- (iv) Compute a MC estimate of $\Pr(Y = 1|x = 3, y)$.

Exercise 2

Fit the probit regression model to the previous data using the Metropolis-Hastings algorithm with joint updating for (μ, β) .

- (i) Write a program to implement this probit model using the Metropolis-Hastings algorithm with joint updating of (μ, β) .
- (ii) Compute MC estimators of the mean, variance and 95% posterior intervals of $[\mu|y]$ and $[\beta|y]$.
- (iii) Compute MC standard errors and effective chain lengths of $\hat{E}(\mu|y)$ and $\hat{E}(\beta|y)$.
- (iv) Compute the MC estimate of $\Pr(Y = 1|x = 3, y)$. How does this estimate differ from the one computed in Exercise 1, iv ?

Exercise 3

Fit the probit regression model to the previous data using a Gibbs sampling algorithm and data augmentation.

- (i) Write a program to implement this probit model using the Gibbs sampling algorithm.
- (ii) Compute MC estimators of the mean, variance and 95% posterior intervals of $[\mu|y]$ and $[\beta|y]$.
- (iii) Compute MC standard errors and effective chain lengths of $\hat{E}(\mu|y)$ and $\hat{E}(\beta|y)$.

Exercise 4

The R-code below generates correlated binary data based on a probit threshold model. A full-sib family structure is assumed: $nfs = 500$ full-sib families with $fs = 3$ full-sibs per family. At the level of liability the (true) hierarchical model is

$$u_{ij} = f_i + e_{ij}, \quad i = 1, \dots, nfs, j = 1, \dots, fs,$$

$$f_i | \sigma_f^2 \stackrel{iid}{\sim} N(0, \sigma_f^2 = 0.18),$$

$$e_{ij} \stackrel{iid}{\sim} N(0, 1).$$

```
#SINGLE-SITE GIBBS - CORRELATED PROBIT MODEL
# DOES NOT USE THE SVD OF ZZ'
rm(list=ls()) # Clear the workspace
set.seed(7713)

require(graphics)
# GENERATE CORRELATED (FULL-SIBS) BINARY DATA (THRESHOLD MODEL)
# THE CODE WILL USE THE PACKAGE MVTNORM; IT IS INSTALLED BELOW
#install.packages("MCMCpack", .libPaths()[1])
#install.packages("mvtnorm", .libPaths()[1])
library(mvtnorm)
library(MCMCpack)
# INITIALISE PARAMETERS
#p0<-0.2
p0 <- 0.5
mu <- qnorm(p0)
iccfs<-0.15 #INTRACLASS CORRELATION FS
# VARIANCE BETWEEN FAMILIES: iccfs /(1- iccfs)
# PHENOTYPIC VARIANCE: 1/(1-iccfs)
nfs<- 500 # NUMBER OF FULL-SIB FAMILIES
fs<-3 #FULL-SIB FAMILY SIZE
y<-matrix(data=0,nrow=fs*nfs,ncol=1)
x<-matrix(data=0,nrow=fs*nfs,ncol=1)
# GENERATE NFS FULL-SIB EFFECTS f
f<-rnorm(nfs,mean=0,sd=sqrt(iccfs/(1-iccfs)))
#####
#### GENERATE BINARY RECORDS Y
f<-rnorm(nfs,mean=0,sd=sqrt(iccfs/(1-iccfs)))
p <- pnorm(mu+f)
y <- rbinom(nfs*fs,1,rep(p,each=fs))
w <- rep(1:nfs,each=fs)
d<-data.frame(w,y)
family <- w
family <- as.factor(family)
Z<-model.matrix(~0+family)
head(d)
```

```
## w y
```

```
## 1 1 0
## 2 1 0
## 3 1 0
## 4 2 0
## 5 2 0
## 6 2 1
```

The Bayesian model used to analyse the data assumes

$$u_{ij} = \mu + f_i + e_{ij}, \quad i = 1, \dots, nfs, j = 1, \dots, fs,$$

$$f_i | \sigma_f^2 \stackrel{iid}{\sim} N(0, \sigma_f^2),$$

$$e_{ij} \stackrel{iid}{\sim} N(0, 1).$$

The prior distributions for μ and σ_f^2 are assumed to be proportional to constants. In matrix notation the model for the vector of liabilities is

$$u = X\mu + Zf + e$$

where X (a column vector of 1's) and Z are observed incidence matrices.

- (i) Write a computer program to draw inferences about μ and σ_f^2 using a single-site updating Gibbs sampling algorithm and data augmentation.
- (ii) Compute MC estimators of the mean and 95% posterior intervals of $[\mu|y]$, $[\sigma_f^2|y]$ and $[h^2|y]$.
- (iii) Compute MC standard errors and effective chain lengths for $\hat{E}(\mu|y)$, $\hat{E}(\sigma_f^2|y)$ and $\hat{E}(h^2|y)$.

Exercise 5

The R-code below generates genomic data: 500 nominally unrelated individuals with 1000 genetic markers available for each individual. The genomic model is

$$y|\mu, g, \sigma_e^2 \sim N(1\mu + g, I\sigma_e^2), \quad (12.29a)$$

$$g|W, \sigma_g^2 \sim SN(0, G\sigma_g^2), \quad (12.29b)$$

$$G = \frac{1}{m} W W', \quad (12.29c)$$

$$W = \{W_{ij}\}. \quad (12.29d)$$

In these expressions, $\mu = 0$, $\sigma_e^2 = 25$, $\sigma_g^2 = 10$, $m = 1000$ is the number of SNPs and W_{ij} is the label for the j th marker genotype of individual i , ($i = 1, \dots, n$; $j = 1, \dots, m$; $m > n$)

$$W_{ij} = \frac{X_{ij} - E(X_{ij})}{\sqrt{\text{Var}(X_{ij})}}, \quad X_{ij} = 0, 1, 2.$$

The eigenvalue decomposition of G is

$$\begin{aligned} G &= U \Delta U' \\ &= \sum_{i=1}^n \lambda_i U_i U_i', \end{aligned}$$

where $U = [U_1, U_2, \dots, U_n]$, of order $n \times n$ is the matrix of eigenvectors of G , U_j is the j th column (dimension $n \times 1$) and Δ is a diagonal matrix with elements equal to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ associated to the n eigenvectors.

The loglikelihood, up to an additive constant, is (see expression (3.48) on page 101)

$$\begin{aligned} \ln p(\mu, k, \sigma_e^2 | y, W) &= -\frac{1}{2} \left\{ n \ln \sigma_e^2 + \sum_{i=1}^n \ln(\lambda_i k + 1) \right. \\ &\quad \left. + \frac{1}{\sigma_e^2} (y - 1\mu)' U (\Delta k + I)^{-1} U' (y - 1\mu) \right\}, \end{aligned} \quad (12.30)$$

where $k = \sigma_g^2 / \sigma_e^2$.

```
# BAYES PROBLEMS II. GENOMIC MODEL
# DATA BASED ON GENOMIC MODEL; OBTAIN SVD OF WW'(1/m)
rm(list=ls()) # CLEAR WORKSPACE
nindiv<-500
nmark<-1000
nt <- nindiv*nmark
X<-matrix(nrow=nindiv,ncol=nmark,rbinom(n=nt,size=2,p=.5))
stdev <- matrix(data=NA,nrow= nmark,ncol=1)
W <- matrix(data=NA,nrow= nindiv,ncol=nmark)
U <- matrix(data=NA,nrow= nindiv,ncol= nindiv)
G<-matrix(data=NA,nrow= nindiv,ncol= nindiv)
cm <- colMeans(X)
#CHOOSE VALUE FOR GENOMIC VARIANCE vgs
vgs<-10
#CHOOSE VALUE FOR ENVIRONMENTAL VARIANCE ves
ves<-25
# CREATE MATRIX OF STANDARDISED MARKER GENOTYPE CODES
for (i in 1:nmark)
{
  W[,i] <- ( X[,i]-cm[i]) / sd(X[,i])
}
```

```

}
# CAN USE INSTEAD:
# W <- scale(X)
# GENERATE nindiv GENOMIC VALUES FROM N(0, (1/nmark)WW'*vgs,)
# nmark MARKER VALUES: REALISATIONS FROM N(0, I sqrt(vgs/nmark))
g <- (1/sqrt(nmark))*W%*%rnorm(nmark,mean=0,sd=sqrt(vgs))
# GENERATE nindiv PHENOTYPES WITH MEAN 0, VAR=vgs+ves,
# HERITABILITY=vgs/(vgs+ves)
e<- rnorm(nindiv,mean=0,sd=sqrt(ves))
y <- g+ e
# GENOMIC RELATIONSHIP MATRIX G
G <- (1/nmark)*W%*%t(W)
# SVD OF G
EVD <- eigen(G)
#names(EVD)
#head(EVD$values)
U <- EVD$vector
tU<-t(U)
val <- EVD$values
val[length(y)] <-0
D <- diag(val,nrow=nindiv)
# Dp IS A VECTOR WITH NON-ZERO EIGENVALUES
Dp<-c(val[1:nindiv-1])

```

- (i) Write a computer program to draw inferences about μ , σ_e^2 and σ_g^2 using a single-site updating Gibbs sampling algorithm.
- (ii) Compute MC estimators of the mean and 95% posterior intervals of $[\mu|y]$, $[\sigma_g^2|y]$, $[\sigma_e^2|y]$ and $[h^2|y]$.
- (iii) Compute MC standard errors and effective chain lengths for $\hat{E}(\mu|y)$, $\hat{E}(\sigma_g^2|y)$, $\hat{E}(\sigma_e^2|y)$ and $\hat{E}(h^2|y)$.
- (iv) Obtain ML estimates of μ , σ_e^2 and σ_g^2 using the R-function `OPTIM` to compare with the Bayesian results. Obtain the asymptotic covariance matrix and compare the 95% frequentist interval with the 95% Bayesian posterior interval of σ_g^2 .

12.5 Prediction Exercises

The expectations of the validating mean squared errors on pages 305 and 482 were obtained by simple expansion of the squared term of the MSE_v . The same results can be obtained by expressing the MSE as a quadratic form. It is quite simple to check whether the quadratic form is chi-square distributed and therefore an analytical form of the complete distribution of MSE becomes available. The result rests on the assumption of normality.

The following result is used (used also on page 650):

- If the random vector $x \sim N(\mu, V)$, then the random variable $x'Ax \sim \chi^2(r(A), \lambda)$, a chi-square distribution with $r(A)$ degrees of freedom and non-centrality parameter λ , if AV is idempotent, where $r(A)$ denotes the rank of matrix A . The rank and trace of an idempotent matrix are equal. The non-centrality parameter λ is equal to

$$\lambda = \mu' A \mu.$$

The mean and variance are

$$E(x'Ax) = \mu' A \mu + \text{tr}(AV), \quad (12.31a)$$

$$\text{Var}(x'Ax) = 4\mu' AV A \mu + 2 \text{tr}(AV)^2. \quad (12.31b)$$

In the special case when $\mu = 0$, then $E(x'Ax) = \text{tr}(AV)$ and $\text{Var}(x'Ax) = 2 \text{tr}(AV)^2 = 2 \text{tr}(AV)$. Typically, $r(A) < r(V)$ and $\text{tr}(AV) = r(AV) = r(A)$. Then for $\mu = 0$, $E(x'Ax) = r(A)$ and $\text{Var}(x'Ax) = 2r(A)$.

- For a symmetric matrix A and a random vector $x \sim (\mu, V)$, the quadratic form $x'Ax$ has mean given by

$$E(x'Ax) = \mu' A \mu + \text{tr}(AV). \quad (12.32)$$

If x is normally distributed

$$\text{Var}(x'Ax) = 4\mu' AV A \mu + 2 \text{tr}(AV)^2.$$

In the above expressions expectations are taken with respect to $[x|u, V]$.

Exercise 1

A classical problem in quantitative genetics is to predict a future phenotypic value or an unobserved genetic value, given available information. The starting point is to construct a model of the joint distribution of observables and unobservables. To be specific, consider a **training-validating scenario** where the training data consist of n_1 observed phenotypic records y_1 and the scalar to be predicted could be either a hold-out phenotypic record y_2 or an unobserved genetic value g_2 . A hierarchical genetic model assumes

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1\mu_1 \\ \mu_2 \end{bmatrix} + \begin{bmatrix} g_1 \\ g_2 \end{bmatrix} + \begin{bmatrix} e_1 \\ e_2 \end{bmatrix}, \quad (12.33)$$

where y_1 is the vector of observed phenotypic records in the training set with n_1 elements. The scalar y_2 is a hold-out validating datum, μ_1 and μ_2 are unobserved means, g_1 (a column vector with n_1 elements) and g_2 (a scalar) are normally distributed unobserved (additive) genetic values

$$\begin{bmatrix} g_1 \\ g_2 \end{bmatrix} \sigma_g^2 \sim N \left[\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} G_{11} & G_{12} \\ G_{21} & G_{22} \end{pmatrix} \sigma_g^2 \right], \quad (12.34)$$

where σ_g^2 is the additive genetic variance (here, assumed known). In (12.34) G_{11} is the matrix of dimension $n_1 \times n_1$ of expected additive genetic relationships in the training data constructed on the basis of a given pedigree. The dimension of the remaining blocks are

$$G_{12} : n_1 \times 1,$$

$$G_{21} : 1 \times n_1,$$

$$G_{22} : 1 \times 1.$$

The off-diagonal block G_{12} , a column vector, specifies the coefficient of additive genetic relationships between the n_1 individuals in the training data and the individual in the validating data. The scalar G_{22} is the coefficient of additive genetic relationship of the individual in the validating data with itself, equal to 1, in the absence of inbreeding. Residual terms are assumed to follow the independent normal distributions

$$\begin{bmatrix} e_1 \\ e_2 \end{bmatrix} \sigma_e^2 \sim N \left[\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} I & 0 \\ 0 & 1 \end{pmatrix} \sigma_e^2 \right],$$

where σ_e^2 is a residual variance $\text{Var}(y_i | \mu, g_i)$ (assumed known) and I is the $n_1 \times n_1$ identity matrix.

The purpose of the exercise is to obtain predictors of y_2 and of g_2 , given the observable y_1 , the training data. Assume that the predictors take the form of the conditional means,

$$\hat{y}_2 = E(y_2 | y_1),$$

$$\hat{g}_2 = E(g_2 | y_1).$$

1. Derive $\text{Var}(\hat{y}_2)$, $\text{Cov}(y_2, \hat{y}_2)$, $\text{Var}(\hat{g}_2)$, $\text{Cov}(g_2, \hat{g}_2)$.
2. Derive the prediction error variances $\text{Var}(y_2 - \hat{y}_2)$, $\text{Var}(g_2 - \hat{g}_2)$
3. Derive the squared correlations $R^2(y_2, \hat{y}_2)$, $R^2(g_2, \hat{g}_2)$, where

$$R^2(x, y) = \frac{[\text{Cov}(x, y)]^2}{\text{Var}(x) \text{Var}(y)}$$

Show that

$$R^2(y_2, \hat{y}_2) = h^2 R^2(g_2, \hat{g}_2), \quad h^2 = \frac{\text{Var}(g_2)}{\text{Var}(y_2)}$$

4. Argue that as n_1 increases, $R^2(g_2, \hat{g}_2)$ approaches 1 and $R^2(y_2, \hat{y}_2)$ approaches h^2 .
5. Using the hold-out single data point y_2 derive the expected value of the validating mean squared error

$$E(\text{MSE}_v) = E_{y_1 y_2} (y_2 - \hat{y}_2)^2$$

Exercise 2

The components of the expected validating mean squared error MSE_v were discussed on page 304. From a classical frequentist point of view, when the expectation is taken over the distribution of training and validating data, three components contribute to the expected validating mean squared error as indicated in (6.66) on page 306.

Let y denote training data, y_0 validating data and let $\hat{y}_0 = x'_0 \hat{\theta}$ denote a frequentist point prediction of the validating datum y_0 , where x'_0 is a row vector of p covariates and $\hat{\theta}$ is some estimate based on the training data of parameter θ that indexes the linear model. Then, given x_0

$$\begin{aligned} E_{yy_0}(\text{MSE}_v) &= E_{yy_0} (y_0 - \hat{y}_0)^2 \\ &= E_{y_0} (y_0^2) + E_y (\hat{y}_0^2) - 2 E_{yy_0} (y_0 \hat{y}_0). \end{aligned} \quad (12.35)$$

1. Obtain an analytical expression for (12.35) by calculating the expectations of its three terms and in so doing, reproduce result (6.66). Expectations are taken over the distributions of training and validating data.
2. Adopt a Bayesian perspective and the linear model implicit in eqns. (10.4). Conceptually, this involves drawing θ^* from $[\theta|y, \sigma^2]$ and given θ^* , drawing a predicted validating datum $\hat{y}_0^*$ from $[\hat{y}_0|\theta^*, y, x_0, \sigma^2]$ that has the same distribution as the validating datum $[y_0|\theta^*, y, x_0, \sigma^2]$. The draws $(\theta^*, \hat{y}_0^*)$ are extractions from $[\theta, \hat{y}_0|y, x_0, \sigma^2]$, while the margins are draws from $[\theta|y, \sigma^2]$ and from $[\hat{y}_0|y, x_0, \sigma^2]$. The latter is the posterior predictive distribution of the Bayesian predictor. The mean (dropping the conditioning on x_0 and on σ^2 to avoid cluttering the notation) is

$$E(\hat{y}_0|y) = E_{\theta|y} [E(\hat{y}_0|\theta, y)] = E_{\theta|y} (x'_0 \theta)$$

and the variance

$$\text{Var}(\hat{y}_0|y) = \sigma^2 + \text{Var}_{\theta|y}(x'_0\theta)$$

where σ^2 , assumed known, represents sampling uncertainty of the draws from $[\hat{y}_0|\theta^*, y, x_0, \sigma^2]$. Using the linear model implicit in eqns. (10.4), obtain an analytical expression for the Bayesian expected validating mean squared error

$$\begin{aligned} E(\text{MSE}_v | y, y_0) &= E(y_0 - \hat{y}_0 | y, y_0)^2 \\ &= y_0^2 + E(\hat{y}_0^2 | y) - 2y_0 E(\hat{y}_0 | y). \end{aligned}$$

that involves expectations over the posterior predictive distribution of $\hat{y}_0$, assuming that σ^2 is known.

3. Repeat the exercise assuming that σ^2 is not known.

Exercise 3

Let the $n \times 1$ vector of observations y be a realisation from the probability model

$$y|\mu \sim N(1\mu, I\sigma^2). \quad (12.36)$$

In what follows σ^2 is assumed known and μ is the only unknown parameter.

Partition $y' = (y'_t, y'_v)$ into training and validating sets of lengths n_t, n_v . Let

$$\hat{\mu} = \frac{1'y_t}{n_t} \quad (12.37)$$

be the ML estimate of μ based on the training records, where 1 is a column vector of 1's with n_t elements. The sampling distribution of the ML estimator is

$$\hat{\mu} \sim N\left(\mu, \frac{\sigma^2}{n_t}\right). \quad (12.38)$$

A vector of predicted validating records $\hat{y}_v$ is

$$\hat{y}_v = 1\hat{\mu}$$

and the validating MSE_v is

$$\text{MSE}_v = \frac{1}{n_v} (\hat{y}_v - y_v)' (\hat{y}_v - y_v).$$

1. Derive the expected value of MSE_v under the following three scenarios:
 - (a) Case 1: the training data y_t are treated as fixed and the validating data y_v as random. Here the MSE_v quantifies the ability of the model to predict new records accounting for their sampling variation, without accounting for sampling variation of $\hat{\mu}$.
 - (b) Case 2: the training data y_t are treated as random and the validating data y_v as fixed. Here the MSE_v quantifies the ability of the model to predict an average validating datum, accounting for sampling variation of $\hat{\mu}$.
 - (c) Case 3: Derive the expected value of MSE_v and of MSE_t when training data y_t and validating data y_v are treated as random. Here the MSE_v quantifies the ability of the model to predict a new record accounting for its sampling variation and also for sampling variation of $\hat{\mu}$.
2. Obtain an expression for $E(\text{MSE}_v) - E(\text{MSE}_t)$. Formulate any insights from this calculation, perhaps with help from expression (6.83).

Exercise 4

A Bayesian perspective is adopted extending model (12.36) by assigning an improper uniform prior distribution to μ . Bayesian inferences are conditional on the data and therefore y_t and y_v are treated as fixed observed quantities.

1. Derive the expected value of the validating mean squared error under the following three scenarios:
 - (a) Compute the mean of the posterior distribution of $[\mu|y_t]$ that takes the same form as the ML estimator (12.37); label this mean $\hat{\mu}$. Generate validating predictions y_v^* drawing from $[y_v|\hat{\mu}, y_t]$. Define the validating mean squared error

$$\text{MSE}_v = \frac{1}{n_v} (y_v^* - y_v)' (y_v^* - y_v). \quad (12.39)$$

Show that MSE_v has a scaled chi-square distribution.

This approach does not account for the posterior uncertainty of μ but accounts for sampling uncertainty of y_v^* .

- (b) Account for the uncertainty about μ that is fully captured by its posterior distribution. Given the likelihood (12.36) and an improper uniform prior distribution for μ , the posterior distribution is

$$\mu|y_t \sim N\left(\hat{\mu}, \frac{\sigma^2}{n_t}\right).$$

Construct the vector of predicted validating records

$$\hat{y}_v = \mathbf{1}_{n_v} \mu$$

and using these, calculate

$$\text{MSE}_v = \frac{1}{n_v} (\hat{y}_v - y_v)' (\hat{y}_v - y_v) \quad (12.40)$$

and its expectation.

- (c) Account for the uncertainty about μ and for sampling uncertainty of new validating data y_v^* (the predictors), fitting the hierarchical Bayesian model

$$\mu | y_t \sim N \left(\hat{\mu}, \frac{\sigma^2}{n_t} \right), \quad (12.41a)$$

$$y_v | \mu, y_t \sim N \left(\mathbf{1} \mu, I \sigma^2 \right), \quad (12.41b)$$

where $\hat{\mu}$ is the posterior mean; obtain an analytical expression for $E(\text{MSE}_v)$.

A Monte Carlo approach based on the method of composition is to draw the pairs $(\mu_1^*, y_{v1}^*), \dots, (\mu_N^*, y_{vN}^*)$, where μ_i^* is a draw from the distribution $[\mu | y_t]$ and y_{vi}^* is a draw from the distribution $[y_v | \mu_i^*, y_t]$. The quantities $y_{v1}^*, \dots, y_{vN}^*$ are an *iid* sample from the posterior predictive distribution $[y_v | y_t]$ and can be used to compute the validating mean squared error (12.39).

2. Discuss the results obtained using a, b and c. Provide an interpretation of the distribution of the MSE_v under the Bayesian and frequentist perspectives.

Exercise 5

This exercise is in the same spirit as the previous one; the model is a little more parametrised.

The R-code below generates n individuals each genotyped for p covariates (genetic markers). Among these genetic markers `nqt1` are defined as causal genotypes. The size of gene substitution effects of these causal loci is chosen to generate an additive genetic variance equal to 10 squared units and the heritability of the continuous trait is set equal to 0.5. The data vector y with n elements is divided into a training set y_t and a validating set y_v , each of size $n/2$. Let W represent the centred and scaled matrix X , where $X = \{X_{ij}\}$ is an $n/2 \times p$ observed matrix with genotypic codes X_{ij} equal to 0, 1, 2 according to the number of the arbitrarily chosen allele of individual i and marker j .

The operational statistical model is as follows:

$$y_i | \mu, b, \sigma_e^2 \sim N(1\mu + Wb, I\sigma_e^2), \quad (12.42a)$$

$$b | \sigma_b^2 \sim N(0, I\sigma_b^2), \quad (12.42b)$$

where y_i is the vector of training records of length $n/2$, μ is an unobserved mean, b is a vector of unknown genetic marker effects of length p , 1 is a vector of 1's of length $n/2$, σ_e^2 is the residual variance and σ_b^2 reflects prior uncertainty for each element of b . In other words, σ_b^2 is the prior variance of the effect of one marker, the same for all p markers. These two variance components are assumed known.

```
# PREDICTION EXERCISE 5
rm(list=ls()) # CLEAR WORKSPACE
set.seed(123)
nindiv<-100
nmark <- 500
nt <- nindiv*nmark
# NUMBER QTL
nqtl <- 50

# GENERATE MARKER MATRIX FROM BINOMIAL DISTRIBUTION
X<-matrix(nrow= nindiv,ncol= nmark,rbinom(n=nt,size=2,p=.5))
#####
# CHOOSE VALUE FOR MEAN mu AND GENOMIC VARIANCE vgs
mu <- 10
vgs<-10
# CHOOSE VALUE FOR ENVIRONMENTAL VARIANCE ves
ves<-20
her <- vgs/(vgs+ves)

btrue<-matrix(data=0.0,nrow=nmark,ncol=1) # parameter from
# true model
IDq<-sample(1:nmark,nqtl,replace=F) # from nmark markers, choose
# nqtl as QTL
QTLeff<-sqrt(vgs/nqtl)# QTL effect so that the total genetic
# variance is VA
btrue[IDq]<-QTLeff # QTL b's are not zero
W <- matrix(data=NA,nrow= nindiv,ncol=nmark)
cm <- colMeans(X)
# CREATE MATRIX OF STANDARDISED MARKER GENOTYPE CODES
for (i in 1:nmark)
{
  W[,i] <-( X[,i]-cm[i]) / sd(X[,i])
}
# can use more compactly:
# W <- scale(X)
# GENERATE nindiv PHENOTYPES
e<- rnorm(nindiv,mean=0,sd=sqrt(ves))
y <- mu + W%*%btrue+ e
k <- (ves/vgs)*nmark # ratio of residual to genomic variance
```

```

#      Vb = vgs/nmark
train <- sample(1:nrow(W), floor(0.5*nrow(W)))
Xt <- W[train,]
yt <- y[train]
Xv <- W[-train,]
yv <- y[-train]
Zt <- cbind(1, Xt)
Zv <- cbind(1, Xv)
#####
##   ridge regression coefficient matrix, rhs & solution solt
RHSt <- crossprod(Zt, yt)
LHSt <- crossprod(Zt)
LHSt[-1, -1] <- LHSt[-1, -1] + diag(k, nrow=nrow(LHSt)-1)
solt <- solve(LHSt, RHSt)
# PREDICTION, CONDITIONAL ON ESTIMATED PARAMETERS (solt)
predval <- Zv %*% solt # VALIDATING
predtrain <- Zt %*% solt # TRAINING

```

A standard ridge regression is fitted to the training data y_t and assuming known dispersion parameters, the posterior mean of $[\mu, b | y_t, \sigma_e^2, \sigma_b^2]$ satisfies the linear system

$$\begin{bmatrix} 1'1 & 1'W_t \\ W_t'1 & W_t'W_t + I\lambda \end{bmatrix} \begin{bmatrix} \hat{\mu} \\ \hat{b} \end{bmatrix} = \begin{bmatrix} 1'y_t \\ W_t'y_t \end{bmatrix}, \quad (12.43)$$

where $\lambda = \frac{\sigma_e^2}{\sigma_b^2}$ and matrix W has been appropriately partitioned into training and validating blocks:

$$W = \begin{bmatrix} W_t \\ W_v \end{bmatrix},$$

each of dimension $n/2 \times p$.

In a Bayesian setting, predictors y_v^* of validating records are drawn from the posterior predictive distribution $[y_v | y_t]$ and the validating mean squared error is computed with the usual expression

$$\text{MSE}_v = \frac{1}{n_v} (y_v^* - y_v)' (y_v^* - y_v), \quad (12.44)$$

1. Derive the exact distribution of MSE_v (12.44) conditional on the solutions $(\hat{\mu}, \hat{b})$ from (12.43). In this case, y_v^* is a draw from $[y_v | y_t, \hat{\mu}, \hat{b}]$. Write a computer program to generate a Monte Carlo estimate of the posterior distribution of MSE_v , drawing validating predictions y_v^* from $[y_v | y_t, \hat{\mu}, \hat{b}]$. Compare with the analytic results.

2. Derive an analytic expression for the expected value of (12.44) based on the hierarchical Bayesian model (12.42), using the formula for the expected value of a quadratic form (12.32). This provides a description of MSE based on a single point; its mean. Here, as in 1. above, account is taken of the contribution from sampling variation of new records, but in contrast with 1., account is also taken of the posterior uncertainty of $[\mu, b]$.
3. Allowing for uncertainty in (μ, b) , write a code that uses the method of composition to obtain draws (y_v^*, μ^*, b^*) , by sampling repeatedly from

$$\begin{aligned}(\mu^*, b^*) &\sim [\mu, b|y_t], \\ y_v^* &\sim [y_v|\mu^*, b^*, y_t],\end{aligned}$$

from which the validating mean squared error (12.44) is calculated. This generates a second MC estimate of the posterior distribution of MSE_v . In contrast with 2. above, this provides a complete description of the marginal posterior distribution of MSE. Compute the Monte Carlo estimates of the mean and variance and compare the mean with the exact result obtained in 2.

Exercise 6

The exercise involves estimation of validating mean squared error using training data only. This entails obtaining an expression for the expected optimism followed by use of expression (6.84).

Consider the hierarchical linear model,

$$y|b, f, \sigma^2 \sim N(Xb + Zf, I\sigma^2), \quad (12.45a)$$

$$f|\sigma_f^2 \sim N(0, I\sigma_f^2). \quad (12.45b)$$

Above, y is the vector of observed training data with n elements, X and Z are observed incidence matrices of dimension $(n \times p)$ and $(n \times n_f)$, b is a vector of unobserved p elements representing fixed effects and f is a vector of unobserved *iid* random effects with n_f elements. Assume that there are n_o records in each of the n_f random effects, so that the total number of records is $n = n_f n_o$. A typical example arises when data cluster in full-sib families, where f is a vector of random family effects (here, assumed to be unrelated across families) and there are n_o offspring per family. Given known variance components σ^2 and σ_f^2 , BLUP of f and BLUE of b are obtained solving the linear system

$$\begin{bmatrix} X'X & X'Z \\ Z'X & Z'Z + Ik \end{bmatrix} \begin{bmatrix} \hat{b} \\ \hat{f} \end{bmatrix} = \begin{bmatrix} X'y \\ Z'y \end{bmatrix}, \quad k = \frac{\sigma^2}{\sigma_f^2}. \quad (12.46)$$

The vector of predictors (fitted values, linear in y) is

$$\hat{y} = W\hat{\theta}$$

where $W = [X \ Z]$, $\hat{\theta}' = [\hat{b}', \hat{f}']$ and $\hat{\theta}$ is the solution to the linear system

$$[W'W + \Sigma]\hat{\theta} = W'y$$

with

$$\Sigma = \begin{bmatrix} 0 & 0 \\ 0 & Ik \end{bmatrix}.$$

The R-code below generates data based on the model, with $n_f = 500$ full-sib families, $n_o = 2$ full-sibs per family and $n = n_f n_o = 1000$ records. The vector of fixed effects b has $p = 2$ elements, here mimicking 2 breeds. The variance components σ^2 and σ_f^2 are set equal to 5 and 1 squared units respectively, leading to a heritability equal to 1/3.

The code below constructs and solves the mixed model equations.

```
#FULL-SIB CONTINUOUS DATA
rm(list=ls()) # Clear the workspace
set.seed(123771)
ptm<-proc.time()
require(graphics)
# INITIALISE PARAMETERS
mus<-10 # MEAN
vfs<-1 #VARIANCE BETWEEN FULL-SIBS
#vfs<-0.5 #VARIANCE BETWEEN FULL-SIBS
#vfs <- 0.1
# RESIDUAL VARIANCE
ves<-5
k <- ves/vfs
nf<-500 # NUMBER OF FULL-SIB FAMILIES
n<-2 # FULL-SIB FAMILY SIZE
nb <- 2 # NUMBER OF BREEDS
N<-nf*n # TOTAL NUMBER OF RECORDS
y<-matrix(data=0,nrow=nf*n,ncol=1)
z<-matrix(data=0,nrow=nf*n,ncol=1)
# GENERATE nf FULL-SIB EFFECTS fs
fs<-rnorm(nf,mean=0,sd=sqrt(vfs))
# BREED EFFECTS
br <- rep(0,nb)
br[1] <- 5
br[2] <- 8
# GENERATE nf*n RESIDUAL EFFECTS
es<-rnorm(nf*n,mean=0,sd=sqrt(ves))
#####
## GENERATING A FULL-SIB STRUCTURE
```



```

b <- rep(1:nb,each=N/2)
z <- rep(1:nf,each=n)
y <- br[b] + fs[z] + es
d <- data.frame(y,z)
#####
d<-data.frame(y,z)
# GENERATE INCIDENCE MATRICES X & Z
family <- z
breed <- b
family <- as.factor(family)
breed <- as.factor(breed)
X<-model.matrix(~0+breed)
Z<-model.matrix(~0+family)
W <- cbind(X,Z)
LHS <- crossprod(W) # LHS OF MME
LHS[-(1:2),-(1:2)] <- LHS[-(1:2),-(1:2)]+
  diag(k,nrow=nrow(LHS)-2)
RHS <- crossprod(W,y) # RHS OF MME
SOL <- solve(LHS,RHS) # SOLUTION

```

1. Derive the analytical form for the expected optimism,

$$\frac{2}{n} \sum_{i=1}^n \text{Cov}(y_i, \hat{y}_i). \quad (12.47)$$

Data are correlated within the training and validating data due to the full-sib structure. However training and validating data are independent because it is assumed that different *iid* families are assigned to both. Therefore the expression for expected optimism given in (6.83) is valid.

2. Write a computer program to obtain a Monte Carlo estimate of (12.47) using a parametric bootstrap and compare with the exact result obtained in 1.

Exercise 7

Suppose there is a set of data consisting of samples (y_i, x_i) , $i = 1, 2, \dots, n$, where scalars y_i are observed normally distributed records and $x_i \in \mathbb{R}^p$ are vectors of observed genetic markers, p marker genotypes per individual. The set of data is a realisation from some unknown probability model; the analyst fits one or more operational models and wishes to study their prediction ability using the validating mean squared error as evaluation criterion. The purpose of this exercise is to use the data available to obtain a *predictor of the validating mean squared error*, without splitting the available data into training and validating sets.

Preliminaries

- Assume that the pdf of the operational probability model is $p(y|x, b, \sigma^2)$, where b is a vector of unobserved multiple regression coefficients associating y to x and σ^2 is an unobserved variance component. The sampling distribution of the operational model can take the Gaussian form

$$y_i | x_i, b, \sigma^2 \sim N(x_i' b, \sigma^2).$$

- Assume that the density of the prior distribution of (b, σ^2) is $p(b, \sigma^2)$.
- Let $\theta = (b, \sigma^2)$ and let $\theta^* = (b^*, \sigma^{2*})$ be an independent realisation from the posterior distribution $[\theta|y]$.
- Define a posterior predictive distribution for a new observation y^* with probability density

$$p(y^*|y) = \int p(y^*|\theta^*, y) p(\theta^*|y) d\theta^* \quad (12.48)$$

where usually $p(y^*|\theta^*, y) = p(y^*|\theta^*)$. The same form (12.48) can be defined for unobserved validating data y_v , replacing y^* by y_v , where subscript v refers to validating data.

- Given observed $x_i, i \in v$, define the following three predictors of validating data:

1.

$$\hat{y}_i = x_i' \hat{b}, \quad \hat{b} = E(b|y), \quad i \in v. \quad (12.49)$$

2.

$$\hat{y}_i = x_i' b^*, \quad b^* \sim [b|y], \quad i \in v. \quad (12.50)$$

3.

$$\hat{y}_i = y_i^* \sim [y^*|y], \quad i \in v. \quad (12.51)$$

- Define the validating mean squared errors corresponding to each of the three predictors

$$\text{MSE}_v = \frac{1}{n} \sum_{i \in v} (y_i - \hat{y}_i)^2. \quad (12.52)$$

The validating data is not observed because the complete training data y is to be used to compute the predicted validating mean squared error, as explained below. When (12.49) is used in (12.52), the MSE_v is a point mass that ignores

uncertainty. When (12.50) is used in (12.52), the MSE_v accounts for posterior uncertainty of b . When (12.51) is used in (12.52), the MSE_v accounts for posterior uncertainty of b and for sampling variance of the draws from $[y^*|\theta^*, y]$.

Posterior Distribution of the Predicted Validating Mean Squared Error

1. Using (12.49) in (12.52), the posterior distribution of the predicted validating mean squared error is obtained integrating the unobserved validating data y_v in (12.52) over its posterior predictive distribution $[y_v|y]$, treating the predictor $\hat{y}_i$ as an observed quantity. Perform this integration. Note that the integration is over the probability distribution of the operational model. Show that the resulting predicted validating mean squared error, a point mass, takes the form

$$E_{\sigma^2|y}(\sigma^2) + \frac{1}{n} \sum_{i=1}^n \text{Var}_{b|y}(x'_i b).$$

2. Using (12.50) in (12.52), the posterior distribution of the predicted validating mean squared error is obtained integrating the unobserved validating data y_v in (12.52) over its posterior predictive distribution $[y_v|y]$, treating the predictor $\hat{y}_i$ as an observed quantity. Perform this integration. Note that the integration is over the probability distribution of the operational model. The resulting expression yields the posterior distribution of the predicted validating mean squared error. The distribution reflects posterior uncertainty of b through the stochastic term $\hat{y}_i = x'_i b^*$ as shown in (12.50).
3. Compute the mean of the posterior predictive distribution of the validating mean squared error, averaging the stochastic terms in $x'_i b^*$ over $[b|y]$, and show that this mean is equal to

$$E_{\sigma^2|y}(\sigma^2) + \frac{2}{n} \sum_{i=1}^n \text{Var}_{b|y}(x'_i b). \quad (12.53)$$

4. Using (12.51) in (12.52), the posterior distribution of the predicted validating mean squared error is obtained integrating the unobserved validating data y_v in (12.52) over the posterior predictive distribution $[y_v|y]$ as in the previous step, treating the predictor $\hat{y}_i$ as an observed quantity. Perform this integration. As above, the integration is over the probability distribution of the operational model. The resulting expression yields the posterior distribution of the predicted validating mean squared error. The distribution reflects uncertainty of the posterior predictive distribution of $\hat{y}_i$ through the term $\hat{y}_i = y_i^* \sim [y^*|y]$, as shown in (12.51).

5. Compute the mean of the posterior predictive distribution of the validating mean squared error, averaging the stochastic terms in y_i^* over $[y^*|y]$, and show that this mean is equal to

$$2 E_{\sigma^2|y} \left(\sigma^2 \right) + \frac{2}{n} \sum_{i=1}^n \text{Var}_{b|y} \left(x_i' b \right). \quad (12.54)$$

6. Extend the methodology to accommodate binary data.

The answer to this exercise will be uploaded in the repository at <https://github.com/SorensenD/SLGDS> and placed in the folder ErrataNotes. The file will be labelled `note1301.pdf`

Chapter 13

Solution to Exercises



13.1 Likelihood Exercises I

Exercise 1

- (a) The probability mass function of the data x is

$$p(x|n, \theta) = \binom{n}{x} \theta^x (1 - \theta)^{n-x}, \quad x = 0, 1, \dots, n \quad (13.1)$$

where

$$\binom{n}{x} = \frac{n!}{(n-x)!x!}$$

is the binomial coefficient. In (13.1) the random variable is x , for fixed values of n and θ .

- (b) The likelihood function is proportional to (13.1) and can be written

$$L(\theta|n, x) \propto \theta^x (1 - \theta)^{n-x}, \quad 0 < \theta < 1. \quad (13.2)$$

which is a function of θ , for fixed values of n and x .

The loglikelihood is obtained by taking the natural logarithm of (13.2):

$$\ell(\theta|n, x) = x \ln \theta + (n - x) \ln(1 - \theta), \quad 0 < \theta < 1. \quad (13.3)$$

- (c) The ML estimator of θ is obtained by maximising (13.3). Taking the first derivative of $\ell(\theta|n, x)$ with respect to θ and setting the resulting expression equal to zero yields

$$\frac{\partial \ell(\theta|n, x)}{\partial \theta} = \frac{x}{\theta} - \frac{n-x}{1-\theta} = 0. \quad (13.4)$$

The ML estimator of θ is

$$\hat{\theta} = \frac{x}{n}. \quad (13.5)$$

With $n = 10$ and $x = 8$, the ML estimate is

$$\hat{\theta} = \frac{8}{10} = 0.8.$$

- (d) First-order asymptotic theory asserts that

$$\hat{\theta} \sim N\left(\theta, i(\theta)^{-1}\right), \quad (13.6)$$

where Fisher's expected information is

$$\begin{aligned} i(\theta) &= -E_x \left[\frac{\partial^2 \ell(\theta|n, x)}{(\partial \theta)^2} \right] \\ &= -E_x \left[-\frac{x}{\theta^2} - \frac{n-x}{(1-\theta)^2} \right] \\ &= \frac{n\theta}{\theta^2} + \frac{n-n\theta}{(1-\theta)^2} \\ &= \frac{n}{\theta(1-\theta)}. \end{aligned}$$

The asymptotic variance of the ML estimator is obtained using

$$i(\hat{\theta})^{-1} = \frac{\hat{\theta}(1-\hat{\theta})}{n} = 0.016.$$

The asymptotic 95% confidence interval for θ is

$$\Pr\left(0.8 - 1.96 \times \sqrt{0.016} < \theta < 0.8 + 1.96 \times \sqrt{0.016}\right) = 0.95,$$

equal to

$$\Pr(0.55 < \theta < 1.05) = 0.95. \quad (13.7)$$

In repeated sampling, the random interval (13.7) includes θ with probability 95%. The asymptotic confidence interval (13.7) includes values of θ outside its bounds.

Exercise 2

(a) The transformation is

$$\beta = g(\theta) = \ln\left(\frac{\theta}{1-\theta}\right). \quad (13.8)$$

with inverse transformation

$$\theta = g^{-1}(\beta) = \frac{\exp(\beta)}{1 + \exp(\beta)}. \quad (13.9)$$

Under the transformation (13.8), the likelihood is

$$\begin{aligned} L(\theta|n, x) &= L(g^{-1}(\beta)|n, x) \propto \left[\frac{\exp(\beta)}{1 + \exp(\beta)}\right]^x \left(1 - \frac{\exp(\beta)}{1 + \exp(\beta)}\right)^{n-x}, \\ &= \left[\frac{\exp(\beta)}{1 + \exp(\beta)}\right]^x \left[\frac{1}{1 + \exp(\beta)}\right]^{n-x} \quad -\infty < \beta < \infty. \end{aligned} \quad (13.10)$$

(b) The result of using the transformed parameter (13.8) translates into a more symmetric likelihood function, as displayed in Fig. 13.1. This in turn has consequences for the quality of inferences.

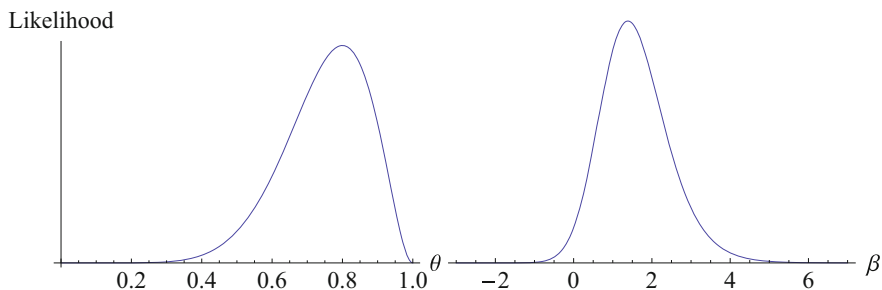


Fig. 13.1 Left panel: plot of $L(\theta|n, x)$ (13.2). Right panel: plot of $L(g^{-1}(\beta))$ (13.10)

- (c) There are two ways of deriving the MLE of β . The simplest is using the invariance property of the MLE. The MLE of β is

$$\hat{\beta} = \ln \left(\frac{\hat{\theta}}{1 - \hat{\theta}} \right) = \ln \frac{0.8}{0.2} = 1.386. \quad (13.11)$$

The asymptotic variance of $\hat{\beta}$ is

$$\begin{aligned} \text{Var}(\hat{\beta}) &= \text{Var}(\hat{\theta}) \left(\frac{dg(\theta)}{d\theta} \right) \Big|_{\theta=\hat{\theta}}^2 \\ &= 0.016 \left(\frac{1}{\hat{\theta}(1-\hat{\theta})} \right)^2 \\ &= 0.625. \end{aligned} \quad (13.12)$$

A longer calculation requires maximising (13.10) with respect to β . It is easier to work with the loglikelihood. A simple calculation shows that the loglikelihood is

$$\ell(g^{-1}(\beta) | n, x) = x\beta - n \ln[1 + \exp(\beta)]. \quad (13.13)$$

Then

$$\frac{\partial}{\partial \beta} \ell(g^{-1}(\beta) | n, x) = x - \frac{n \exp(\beta)}{1 + \exp(\beta)},$$

whose root is

$$\hat{\beta} = -\ln \left(\frac{n-x}{x} \right) = 1.386$$

as in (13.11).

To compute the asymptotic variance of $\hat{\beta}$, take second derivatives of the loglikelihood with respect to β . This gives

$$\begin{aligned} \frac{\partial^2}{(\partial \beta)^2} \ell(g^{-1}(\beta) | n, x) &= \frac{\partial}{\partial \beta} \left[x - \frac{n \exp(\beta)}{1 + \exp(\beta)} \right] \\ &= -\frac{n \exp(\beta)}{(1 + \exp(\beta))^2}. \end{aligned}$$

Evaluated at $\beta = \hat{\beta}$, taking the reciprocal and multiplying by -1 yields the asymptotic variance

$$\text{Var}(\hat{\beta}) = \frac{(1 + \exp(\beta))^2}{n \exp(\beta)} \Big|_{\beta=\hat{\beta}} = \frac{(1 + \exp(1.386))^2}{10 \exp(1.386)} = 0.625$$

as in (13.12).

(d) The asymptotic 95% confidence interval (CI) for β is

$$\Pr\left(1.386 - 1.96 \times \sqrt{0.625} < \beta < 1.386 + 1.96 \times \sqrt{0.625}\right) = 0.95,$$

equal to

$$\Pr(-0.16 < \beta < 2.94) = 0.95. \quad (13.14)$$

(e) Using (13.9), interval (13.14) can be transformed back in terms of θ

$$\Pr\left(g^{-1}(-0.16) < g^{-1}(\beta) < g^{-1}(2.94)\right)$$

yielding

$$\Pr(0.46 < \theta < 0.95). \quad (13.15)$$

This interval based on the more regular likelihood function (13.10) is more reliable than that obtained with the original likelihood function (13.2). Interval (13.15) includes values of θ within the permissible parameter space.

Note

There is a (pure) likelihood-based confidence interval. Useful references from two advocates of the method are Edwards (1992) and Royall (1997) and a tutorial is given by Meeker and Escobar (1995). In this approach to inference, the likelihood function is regarded as conveying a complete measure of uncertainty about the parameter of interest, without the need to invoke sampling distributions (like in inferences based on a posterior distribution). In the present example of the binomial model with $x = 8$ successes out of $n = 10$ trials, the value of the loglikelihood (13.3) evaluated at $\theta = \hat{\theta}$ is equal to -5.004024 . In the words of Edwards (1992), this is the *maximum support*, at $\hat{\theta} = 0.8$. The pure likelihood approach considers the values of θ which lie within, for example, two units of support of the best supported value (the choice of “two” units is arbitrary, like the choice of two units of standard deviations of the “best” estimate in conventional analyses). Then the MLE and the

“confidence interval” that arises from such an exercise in the present example are 0.8 (0.494; 0.965), quite close to the value reported in (13.15).

The R-code below evaluates first, the loglikelihood when the parameter is replaced by its MLE. Secondly, the code executes the function OPTIM twice, with judicious choice of the parameter `lower` and `upper`, generating the two values of θ that define the lower and upper limits within two units of support of the best supported value.

In multiparameter settings this approach is less transparent, in contrast with Bayesian methods that handle multidimensional parameters in a natural way via marginalisation. In general, this is a vast subject with considerable statistical and philosophical controversy.

```
rm(list=ls()) # Clear the workspace
set.seed(123771)
n <- 10
x <- 8
loglik<-function(n,x,theta)
{
  llik <- (x*log(theta)+(n-x)*log(1-theta))
}
LL <- loglik(10,8,0.8) # loglikelihood at ML of theta
LL
```

```
## [1] -5.004024
```

```
# The value "-7" below, is the (approximate) value of
# the loglikelihood two units of support from its value
# at the MLE (which is approx, -5)
fn <- function(theta){
  (-7+(8*log(theta)+(10-8)*log(1-theta)))^2
}
# EXECUTE OPTIM TWICE, ADJUSTING LOWER AND UPPER LIMITS
# TO GENERATE THE TWO ROOTS
lower<-optim(0.4,fn,method="Brent",lower=0.01,upper=0.8)$par
upper<-optim(0.4,fn,method="Brent",lower=0.8,upper=0.999)$par
# LOWER BOUND OF CI
lower
```

```
## [1] 0.4943429
```

```
# UPPER BOUND OF CI
upper
```

```
## [1] 0.9652074
```

As a closing comment, from a Bayesian perspective, (13.3) is the logarithm of the beta density $Be(a, b)$, with $a = x + 1$ and $b = n - x + 1$. Therefore this “pure likelihood” approach in this unidimensional example leads to the same inferences

about θ as those drawn from the beta posterior $Be(x + 1, n - x + 1)$. To illustrate, the 95 % posterior interval for θ from $Be(x + 1, n - x + 1)$, using R is obtained executing `qbeta(c(0.025, 0.975), 9, 3)` that yields (0.482, 0.940).

Exercise 3

- (a) First write the model using the more general formulation

$$y = Xb + e,$$

where X is a column vector of ones of length $n = 10$, b is a vector with a single element equal to μ and the random vector $e \sim N(0, I\sigma^2)$. With this notation the likelihood is proportional to

$$L(b, \sigma^2 | y) \propto (2\pi\sigma^2)^{-\frac{n}{2}} \exp\left[-\frac{1}{2\sigma^2} (y - Xb)' (y - Xb)\right],$$

and the loglikelihood, up to an additive constant is equal to

$$\ell(b, \sigma^2 | y) = -\frac{n}{2} \ln(\sigma^2) - \frac{1}{2\sigma^2} (y - Xb)' (y - Xb). \quad (13.16)$$

- (b) Derivation of the ML estimators of b and of σ^2 requires a joint maximisation of (13.16) with respect to b and σ^2 . The partial derivative with respect to b is

$$\begin{aligned} \frac{\partial}{\partial b} \ell(b, \sigma^2 | y) &= \frac{\partial}{\partial b} \left[-\frac{1}{2\sigma^2} (y' y - 2y' Xb + b' X' Xb) \right] \\ &= \frac{1}{2\sigma^2} (2X' y - 2X' Xb) \\ &= \frac{1}{\sigma^2} (X' y - X' Xb). \end{aligned}$$

Setting this equal to zero and solving for b yields the ML estimator

$$\hat{b} = (X' X)^{-1} X' y. \quad (13.17)$$

The partial derivative of (13.16) with respect to σ^2 is

$$\begin{aligned} \frac{\partial}{\partial \sigma^2} \ell(b, \sigma^2 | y) &= \frac{\partial}{\partial \sigma^2} \left[-\frac{n}{2} \ln(\sigma^2) - \frac{1}{2\sigma^2} (y - Xb)' (y - Xb) \right] \\ &= -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} (y - Xb)' (y - Xb). \end{aligned}$$

On setting this equation equal to zero and multiplying out by $2\sigma^2$ yields the MLE of σ^2

$$\hat{\sigma}^2 = \frac{(y - X\hat{b})' (y - X\hat{b})}{n} \quad (13.18)$$

where $X\hat{b} = \hat{y}$ is the predicted or fitted value of y .

In the special case of this simple model with the $n \times 1$ matrix $X = 1$ and $b = \mu$, the MLE (13.17) is

$$\begin{aligned} \hat{\mu} &= (1'1)^{-1} 1'y \\ &= \frac{1}{10} \sum_{i=1}^{10} y_i = \bar{y} = 2.733 \end{aligned}$$

and (13.18) is

$$\begin{aligned} \hat{\sigma}^2 &= \frac{(y - 1\hat{\mu})' (y - 1\hat{\mu})}{n} \\ &= \frac{\sum_{i=1}^{10} (y_i - \hat{\mu})^2}{10} = 2.581. \end{aligned}$$

(c) Let $\theta' = (b, \sigma^2)$. The observed information is equal to minus the Hessian

$$I(b, \sigma^2) = -\frac{\partial^2 \ell(b, \sigma^2 | y)}{\partial \theta \partial \theta'}. \quad (13.19)$$

The Hessian requires

$$\frac{\partial^2 \ell(b, \sigma^2 | y)}{\partial b \partial b'}, \frac{\partial^2 \ell(b, \sigma^2 | y)}{\partial b \partial \sigma^2}, \frac{\partial^2 \ell(b, \sigma^2 | y)}{(\partial \sigma^2)^2}.$$

These second derivatives are

$$\begin{aligned} \frac{\partial^2 \ell(b, \sigma^2 | y)}{\partial b \partial b'} &= \frac{\partial \left[\frac{1}{\sigma^2} (X'y - X'Xb) \right]}{\partial b} = -\frac{X'X}{\sigma^2}, \\ \frac{\partial^2 \ell(b, \sigma^2 | y)}{\partial b \partial \sigma^2} &= \frac{\partial \left[\frac{1}{\sigma^2} (X'y - X'Xb) \right]}{\partial \sigma^2} = -\frac{X' (y - Xb)}{\sigma^2}, \end{aligned}$$

$$\begin{aligned}\frac{\partial^2 \ell(b, \sigma^2 | y)}{(\partial \sigma^2)^2} &= \frac{\partial \left[-\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} (y - Xb)' (y - Xb) \right]}{\partial \sigma^2} \\ &= \frac{n}{2} (\sigma^2)^{-2} - (y - Xb)' (y - Xb) (\sigma^2)^{-3}.\end{aligned}$$

Therefore the observed information matrix is

$$\begin{aligned}I(b, \sigma^2) &= \begin{bmatrix} \frac{X'X}{\sigma^2} & -\frac{X'(y-Xb)}{\sigma^2} \\ -\frac{(y-Xb)'X}{\sigma^2} & -\frac{n}{2(\sigma^2)^2} + \frac{(y-Xb)'(y-Xb)}{(\sigma^2)^3} \end{bmatrix} \\ &= \begin{bmatrix} \frac{n}{\sigma^2} & \frac{\sum_{i=1}^n (y_i - \mu)}{\sigma^2} \\ \frac{\sum_{i=1}^n (y_i - \mu)}{\sigma^2} & -\frac{n}{2(\sigma^2)^2} + \frac{\sum_{i=1}^n (y_i - \mu)^2}{(\sigma^2)^3} \end{bmatrix},\end{aligned}\quad (13.20)$$

which evaluated at the ML estimates $\hat{\mu} = 2.733$ and $\hat{\sigma}^2 = 2.581$ is equal to

$$I(\hat{\mu}, \hat{\sigma}^2) = \begin{bmatrix} 3.874 & 0 \\ 0 & 0.7503 \end{bmatrix}.$$

(d) The asymptotic variance of $(\hat{\mu}, \hat{\sigma}^2)$ based on (13.19) is

$$\begin{aligned}\text{Var}(\hat{\mu}, \hat{\sigma}^2) &= I(\hat{\mu}, \hat{\sigma}^2)^{-1} \\ &= \begin{bmatrix} 3.874^{-1} & 0 \\ 0 & 0.7503^{-1} \end{bmatrix} \\ &= \begin{bmatrix} 0.258 & 0 \\ 0 & 1.333 \end{bmatrix}.\end{aligned}\quad (13.21)$$

(e) The expected information is

$$i(b, \sigma^2) = E_y \left[-\frac{\partial^2 \ell(b, \sigma^2 | y)}{\partial \theta \partial \theta'} \right],$$

which requires the expectation of each of the 4 elements of (13.20). These are

$$\begin{aligned}E_y \left(\frac{n}{\sigma^2} \right) &= \frac{n}{\sigma^2}, \\ E_y \left(\frac{\sum_{i=1}^n (y_i - \mu)}{\sigma^2} \right) &= 0,\end{aligned}$$

$$\begin{aligned} E_y \left(-\frac{n}{2(\sigma^2)^2} + \frac{\sum_{i=1}^n (y_i - \mu)^2}{(\sigma^2)^3} \right) &= -\frac{n}{2(\sigma^2)^2} + \frac{n\sigma^2}{(\sigma^2)^3} \\ &= \frac{n}{2(\sigma^2)^2}. \end{aligned}$$

Therefore,

$$\begin{aligned} i(\hat{\mu}, \hat{\sigma}^2) &= \begin{bmatrix} \frac{n}{\hat{\sigma}^2} & 0 \\ 0 & \frac{n}{2(\hat{\sigma}^2)^2} \end{bmatrix} \\ &= \begin{bmatrix} 3.874 & 0 \\ 0 & 0.7503 \end{bmatrix}, \end{aligned}$$

the same as (13.21).

- (f) The asymptotic variance of $(\hat{\mu}, \hat{\sigma}^2)$ based on Fisher's (expected) information is the same as (13.21). The form of the expression is

$$\text{Var}(\hat{\mu}, \hat{\sigma}^2) = \begin{bmatrix} \frac{\hat{\sigma}^2}{n} & 0 \\ 0 & \frac{2(\hat{\sigma}^2)^2}{n} \end{bmatrix}. \quad (13.22)$$

- (g) The following results are useful:

- (i) If vector x is $N(\mu, V)$, then $x'Ax$ is $\chi^2(r(A), \mu'A\mu)$, a chi-square distribution with $r(A)$ degrees of freedom and non-centrality parameter $\mu'A\mu$, if AV is idempotent. An idempotent matrix is a matrix which, when multiplied by itself, yields itself. If K is idempotent, $K^2 = K$ and $\text{rank}(K) = r(K) = \text{tr}(K)$.
- (ii) Write $\frac{\sum_{i=1}^n (y_i - \mu)^2}{\sigma^2} = \frac{1}{\sigma^2} (y - 1\mu)'(y - 1\mu)$. Given $y \sim N(1\mu, I\sigma^2)$, then

$$E(y - 1\mu) = 0; \quad V = \text{Var}(y - 1\mu) = I\sigma^2; \quad A = \frac{1}{\sigma^2}I$$

and $AV = I$ which is idempotent. It follows that

$\frac{1}{\sigma^2} (y - 1\mu)'(y - 1\mu) \sim \chi^2[n, 0]$ because $r\left(\frac{1}{\sigma^2}I\right) = n$. The mean and variance are

$$\begin{aligned} E\left(\frac{1}{\sigma^2} (y - 1\mu)'(y - 1\mu)\right) &= n \\ \text{Var}\left(\frac{1}{\sigma^2} (y - 1\mu)'(y - 1\mu)\right) &= 2n. \end{aligned}$$

Given the linear model $y = Xb + e$, with $e \sim N(0, I\sigma^2)$, the MLE of b is

$$\hat{b} = (X'X)^{-1} X'y.$$

and the MLE of σ^2 is

$$\hat{\sigma}^2 = \frac{(y - X\hat{b})' (y - X\hat{b})}{n}.$$

Write

$$\begin{aligned} y - X\hat{b} &= y - X(X'X)^{-1} X'y \\ &= (I - X(X'X)^{-1} X')y. \end{aligned}$$

Therefore

$$\begin{aligned} (y - X\hat{b})' (y - X\hat{b}) &= y' (I - X(X'X)^{-1} X') y \\ &= y' Py \end{aligned}$$

where P is idempotent. Then

$$\frac{1}{\sigma^2} (y - X\hat{b})' (y - X\hat{b}) = \frac{1}{\sigma^2} y' Py.$$

Since $V = \text{Var}(y) = I\sigma^2$ and $A = \frac{1}{\sigma^2} P$, $AV = P$, and idempotent matrix. Therefore

$$\begin{aligned} \frac{1}{\sigma^2} (y - X\hat{b})' (y - X\hat{b}) &\sim \chi^2 \left(r(P), b' X' P X b \frac{1}{2\sigma^2} \right) \\ &= \chi^2 (n - r(X)), \end{aligned} \tag{13.23}$$

and the exact sampling distribution of the MLE of σ^2 is

$$\frac{(y - X\hat{b})' (y - X\hat{b})}{n} \sim \frac{\sigma^2}{n} \chi^2 (n - r(X)), \tag{13.24}$$

which is proportional to a chi-square distribution. The second line follows because $PX = 0$ and $r(P) = r(I - X(X'X)^{-1} X') = r(I) - r(X(X'X)^{-1} X') =$

$n - r(X)$. In view of (13.24), since $X = 1$ (a vector of ones whose rank is 1) and $b = \mu$, a scalar, the expected value and variance of $\hat{\sigma}^2$ are

$$\begin{aligned} E \left[\frac{(y - X\hat{b})' (y - X\hat{b})}{n} \right] &= \frac{\sigma^2}{n} (n - 1), \\ \text{Var} \left[\frac{(y - X\hat{b})' (y - X\hat{b})}{n} \right] &= \left(\frac{\sigma^2}{n} \right)^2 2(n - 1). \end{aligned} \quad (13.25)$$

Therefore the MLE of σ^2 is biased. The small sample variance of $\hat{\sigma}^2$ using (13.25) evaluated at $\sigma^2 = \hat{\sigma}^2$ is

$$2 \left(\frac{\hat{\sigma}^2}{n} \right)^2 (n - 1) = \left(\frac{2.581}{10} \right)^2 2(9) = 1.199,$$

a little smaller than the asymptotic value $2(\hat{\sigma}^2)^2/n$ reported in (13.22).

Exercise 4

The R-code that generates the data is reproduced below.

```
rm(list=ls()) # Clear the workspace
set.seed(772231)
n <- 5000 # Number of records generated for Y_1
p <- 0.20 # Proportion to be selected
m <- round(p*n) # Number of records selected
y1 <- rnorm(n, mean=10, sd=2) # Generate random variable Y_1
y1_sort <- sort(y1, decreasing=TRUE) # Sort Y_1
y1s <- y1_sort[1:m] # The highest m Y_1 records are kept
#length(y1) # Number of records in Y_1
#length(y1s) # Number of records in the selected subset of Y_1
# Generate Y_2
y2 <- rnorm(length(y1s), mean=15+0.6*(y1s-10), sd=sqrt(4-(2.4^2)/4))
```

Let $\theta = (\mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx})$. If the selection mechanism is (incorrectly) ignored and only the m bivariate records are considered, these can be assumed to be a realisation from

$$(y_i, x_i) | \theta \sim N \left(\begin{bmatrix} \mu_y \\ \mu_x \end{bmatrix}, \begin{bmatrix} \sigma_{yy} = 4 & \sigma_{yx} = 2.4 \\ \sigma_{yx} = 2.4 & \sigma_{xx} = 4 \end{bmatrix} \right), \quad (13.26)$$

$$i = 1, 2, \dots, m,$$

with $\mu_y = 10$, $\mu_x = 15$. The closed form of the ML estimators are

$$\hat{\mu}_x = \frac{1}{m} \sum_{i=1}^m x_i, \quad \hat{\mu}_y = \frac{1}{m} \sum_{i=1}^m y_i, \quad (13.27a)$$

$$\hat{\sigma}_{xx} = \frac{1}{m} \sum_{i=1}^m (x_i - \hat{\mu}_x)^2, \quad \hat{\sigma}_{yy} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{\mu}_y)^2, \quad (13.27b)$$

$$\hat{\rho} = \frac{\hat{\sigma}_{xy}}{\sqrt{\hat{\sigma}_{xx} \hat{\sigma}_{yy}}}, \quad \hat{\sigma}_{xy} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{\mu}_y)(x_i - \hat{\mu}_x). \quad (13.27c)$$

(a) The ML estimates based on (13.27) can be computed with the R-code below:

```
m <- length(y2)
n <- length(y1)
round(mean(y1s), digits=2)
```

```
## [1] 12.8
```

```
round(mean(y2), digits=2)
```

```
## [1] 16.56
```

```
round(mean((y1s-mean(y1s))^2), digits=2)
```

```
## [1] 0.91
```

```
round(mean((y2-mean(y2))^2), digits=2)
```

```
## [1] 2.85
```

```
round((cov(y1s,y2)*(m-1)/m), digits=2)
```

```
## [1] 0.54
```

```
round(cov(y1s,y2)/(sqrt(var(y1s)*var(y2))), digits=2)
```

```
## [1] 0.34
```

```
round(cov(y1s,y2)/var(y1s),digits=2)
```

```
## [1] 0.59
```

The ML estimates (13.27) are

$$\begin{aligned}\hat{\mu}_x &= 12.8, \\ \hat{\mu}_y &= 16.56, \\ \hat{\sigma}_{xx} &= 0.91, \\ \hat{\sigma}_{yy} &= 2.85, \\ \hat{\sigma}_{xy} &= 0.54, \\ \hat{\rho} &= 0.34, \\ \hat{b}_{yx} &= 0.59.\end{aligned}$$

With the exception of $\hat{b}_{yx}$ these estimates are in very poor agreement with the true parameter values defined in (13.26).

If the individuals that produced a second lactation had been randomly selected, this approach using a subset of the data would have performed adequately. The topic of inferences under selection is a delicate one; important references are Rubin (1976) and Little and Rubin (1987).

- (b) The density function of the **complete** data under parametrisation $\phi = (\mu_x, \sigma_{xx}, \beta_0, \beta_1, \sigma_{y.x})$ is

$$\begin{aligned}p(x, y|\phi) &= \prod_{i=1}^m p(x_i, y_i|\phi) \prod_{i=1+m}^n p(x_i|\phi) \\ &= \left[\prod_{i=1}^m p(y_i|x_i, \phi) p(x_i|\phi) \right] \left[\prod_{i=1+m}^n p(x_i|\phi) \right] \\ &= \prod_{i=1}^n p(x_i|\mu_x, \sigma_{xx}) \prod_{i=1}^m p(y_i|x_i, \beta_0 + \beta_1 x_i, \sigma_{y.x}) \\ &= (2\pi\sigma_{xx})^{-\frac{n}{2}} \exp\left[-\frac{\sum_{i=1}^n (x_i - \mu_x)^2}{2\sigma_{xx}}\right] (2\pi\sigma_{y.x})^{-\frac{m}{2}} \\ &\quad \exp\left[-\frac{\sum_{i=1}^m (y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma_{y.x}}\right]\end{aligned}\tag{13.28}$$

where

$$\sigma_{y.x} = \sigma_{yy} - \frac{(\sigma_{xy})^2}{\sigma_{xx}},\tag{13.29}$$

is the variance of the conditional distribution of Y given X . The likelihood is proportional to (13.28) and the loglikelihood is

$$\begin{aligned} \ell(\mu_x, \sigma_{xx}, \beta_0, \beta_1, \sigma_{y.x}|x, y) = & -\frac{n}{2} \ln \sigma_{xx} - \frac{\sum_{i=1}^n (x_i - \mu_x)^2}{2\sigma_{xx}} - \frac{m}{2} \ln \sigma_{y.x} \\ & - \frac{\sum_{i=1}^m (y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma_{y.x}}. \end{aligned} \quad (13.30)$$

- (c) The first two terms in the right hand side of (13.30) are the loglikelihood of the $N(\mu, \sigma^2)$ linear model. The ML estimates of μ and σ^2 are given for the general case in (13.17) and (13.18), which in this example with $X = 1$ (a vector of ones) and $b = \mu_x$ reduce to

$$\hat{\mu}_x = \frac{\sum_{i=1}^n x_i}{n}, \quad (13.31a)$$

$$\hat{\sigma}_{xx} = \frac{\sum_{i=1}^n (x_i - \hat{\mu})^2}{n}. \quad (13.31b)$$

To obtain the MLE of $\sigma_{y.x}$, β_0 and β_1 , differentiate (13.30) with respect to $\sigma_{y.x}$, β_0 and β_1 . After a little simplification the three equations with the three unknowns are

$$-m + \frac{\sum_{i=1}^m (y_i - \beta_0 - \beta_1 x_i)^2}{\sigma_{y.x}} = 0,$$

$$\sum_{i=1}^m (y_i - \beta_0 - \beta_1 x_i) = 0,$$

$$\sum_{i=1}^m x_i (y_i - \beta_0 - \beta_1 x_i) = 0.$$

From the second equation

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}^*, \quad (13.32)$$

where

$$\bar{y} = \frac{\sum_{i=1}^m y_i}{m},$$

$$\bar{x}^* = \frac{\sum_{i=1}^m x_i}{m},$$

In these expressions, $\bar{x}^*$ is the mean of the selected first lactation records. From the third equation,

$$\hat{\beta}_1 = \frac{\sum_{i=1}^m x_i y_i - m \bar{x}^* \bar{y}}{\sum_{i=1}^m x_i^2 - m \bar{x}^{*2}} = \frac{S_{xy}}{S_{xx}}. \quad (13.33)$$

Finally from the first equation,

$$\hat{\sigma}_{y \cdot x} = \frac{\sum_{i=1}^m (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2}{m}. \quad (13.34)$$

Substituting (13.32) and (13.33) in (13.34), leads to the alternative form (see Note below)

$$m \hat{\sigma}_{y \cdot x} = S_{yy} - \frac{S_{xy}^2}{S_{xx}} \quad (13.35)$$

which is more in line with the parameter (13.29), where

$$S_{yy} = \sum_{i=1}^m (y_i - \bar{y})^2.$$

Note

Substituting (13.32) in (13.34) gives

$$\begin{aligned} \sum_{i=1}^m (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2 &= \sum_{i=1}^m (y_i - (\bar{y} - \hat{\beta}_1 \bar{x}^*) - \hat{\beta}_1 x_i)^2 \\ &= \sum_{i=1}^m ((y_i - \bar{y}) - \hat{\beta}_1 (x_i - \bar{x}^*))^2 \\ &= \sum_{i=1}^m (y_i - \bar{y})^2 - 2\hat{\beta}_1 \sum_{i=1}^m (y_i - \bar{y})(x_i - \bar{x}^*) + \hat{\beta}_1^2 \sum_{i=1}^m (x_i - \bar{x}^*)^2 \\ &= S_{yy} - 2\hat{\beta}_1 S_{xy} + \hat{\beta}_1^2 S_{xx} \\ &= S_{yy} - \frac{S_{xy}^2}{S_{xx}}, \end{aligned}$$

where the last line is obtained by replacing $\hat{\beta}_1 = S_{xy}/S_{xx}$ in the 4th line.

These MLE's of $\phi = (\mu_x, \sigma_{xx}, \beta_0, \beta_1, \sigma_{y \cdot x})$ can be expressed in terms of the MLE's of $\theta = (\mu_y, \mu_x, \sigma_{xy}, \sigma_{yy}, \sigma_{xx})$. First, MLE's of μ_x and σ_{xx} are displayed in (13.31). The remaining MLE's of the parameters in ϕ are obtained as follows. Recall that

$$\begin{aligned}\beta_0 &= \mu_y - \beta_1 \mu_x = 1, \\ \beta_1 &= \frac{\sigma_{xy}}{\sigma_{xx}} = 0.6, \\ \sigma_{y \cdot x} &= \sigma_{yy} - \frac{(\sigma_{xy})^2}{\sigma_{xx}} = 2.56.\end{aligned}$$

Then from the first equation,

$$\hat{\mu}_y = \hat{\beta}_0 + \hat{\beta}_1 \hat{\mu}_x = 14.9.$$

From the second equation,

$$\hat{\sigma}_{xy} = \hat{\beta}_1 \hat{\sigma}_{xx} = 2.36,$$

and from the third

$$\hat{\sigma}_{yy} = \hat{\sigma}_{y \cdot x} + \frac{(\hat{\sigma}_{xy})^2}{\hat{\sigma}_{xx}} = \hat{\sigma}_{y \cdot x} + \hat{\beta}_1^2 \hat{\sigma}_{xx} = 4.25.$$

The R-code that performs the computations is shown below

```
sxy <- cov(y1s,y2)/var(y1s)
syy <- sum((y2-mean(y2))^2)
sxx <- mean((y1-mean(y1))^2)
betalhat <- cov(y1s,y2)/var(y1s)
sigmaxxhat <- mean((y1-mean(y1))^2)
sigmaxyhat <- betalhat*sigmaxxhat
beta0hat <- mean(y2)-betalhat*mean(y1s)
muxhat <- mean(y1)
muyhat <- beta0hat+betalhat*muxhat
sigmay.xhat <- (syy-sxy^2/sxx)/length(y2)
sigmayyhat <- sigmay.xhat + ( (sigmaxyhat^2)/sigmaxxhat)
rhothathat <- sigmaxyhat/sqrt(sigmaxxhat*sigmayyhat)
byxhat <- cov(y1s,y2)/var(y1s)
```

and generates the ML estimates:

$$\begin{aligned}\hat{\mu}_x &= 10, \\ \hat{\mu}_y &= 14.9, \\ \hat{\sigma}_{xx} &= 3.97,\end{aligned}$$

$$\hat{\sigma}_{yy} = 4.25,$$

$$\hat{\sigma}_{xy} = 2.36,$$

$$\hat{\rho} = 0.57,$$

$$\hat{b}_{yx} = 0.59.$$

These estimates are all in good agreement with the true value of the parameters defined in (13.26).

Exercise 5

1. Uncensored model.

(a) The loglikelihood is obtained taking logarithms of (12.11):

$$l(b|y) = n \ln b - b \sum_{i=1}^n y_i. \quad (13.36)$$

(b) The score is

$$l'(b|y) = \frac{n}{b} - \sum_{i=1}^n y_i. \quad (13.37)$$

(c) The maximum likelihood estimator is obtained setting the derivative with respect to b equal to zero and solving for b . This results in

$$\hat{b} = \frac{n}{\sum_{i=1}^n y_i}. \quad (13.38)$$

(d) Fisher's information is computed from

$$l''(b|y) = -\frac{n}{b^2},$$

that yields

$$\begin{aligned} I(b) &= -E[l''(b|Y)] \\ &= \frac{n}{b^2}. \end{aligned} \quad (13.39)$$

2. Censored model

- (a) From (12.12), the contribution to the loglikelihood from one observation is

$$l(b|y_i) = I(y_i < c) (\ln b - by_i) - I(y_i = c) bc. \quad (13.40)$$

- (b) Associated first and second derivatives are

$$l'(b|y_i) = I(y_i < c) \left(\frac{1}{b} - y_i \right) - I(y_i = c) c \quad (13.41)$$

and

$$l''(b|y_i) = -I(y_i < c) \frac{1}{b^2}. \quad (13.42)$$

The score based on all the data is

$$\begin{aligned} \sum_{i=1}^n l'(b|y_i) &= \sum_{i=1}^n \left\{ I(y_i < c) \left(\frac{1}{b} - y_i \right) - I(y_i = c) c \right\} \\ &= \frac{r}{b} - \sum_{i=1}^r y_i - (n-r)c. \end{aligned} \quad (13.43)$$

- (c) Setting to zero and solving leads to MLE of b

$$\hat{b} = \frac{r}{\sum_{i=1}^r y_i + (n-r)c}. \quad (13.44)$$

In the absence of censoring, $r = n$ and (13.44) is equal to (13.38).

- (d) Information is computed using (13.42):

$$\begin{aligned} I(b) &= -E \left\{ \sum_{i=1}^n \left[l''(b|Y_i) \right] \right\} \\ &= E \left\{ \sum_{i=1}^n \left[I(Y_i < c) \frac{1}{b^2} \right] \right\} \\ &= \sum_{i=1}^n \frac{1}{b^2} \Pr(Y_i < c) \\ &= \frac{n}{b^2} - \frac{n \exp(-bc)}{b^2} \\ &= \frac{n}{b^2} \Pr(Y_i < c). \end{aligned} \quad (13.45)$$

In the absence of censoring $c = \infty$, $\Pr(Y_i < c) = 1$ and (13.45) is equal to (13.39). Also, $n \Pr(Y_i < c) = E(r)$ (the number of non-censored records, r , is binomially distributed, i.e., $r \sim Bi(n, \Pr(Y_i < c))$).

Censoring reduces the amount of information in the data to infer b .

13.2 Likelihood Exercises II

Exercise 1

- (a) See Fig. 13.2.
- (b) Figure 13.2 depicts the relationship between Y (the rough measure of θ) and X , the covariate (average drug level). A linear fit seems like a good approximation. Here, Y is an 5×1 vector with the averages of the 5 groups of the 0/1 observations and X is an 5×1 vector with the average drug level of the 5 groups. The linear fit is based on the model $E(Y_i | x_i) = \mu + \beta x_i$.

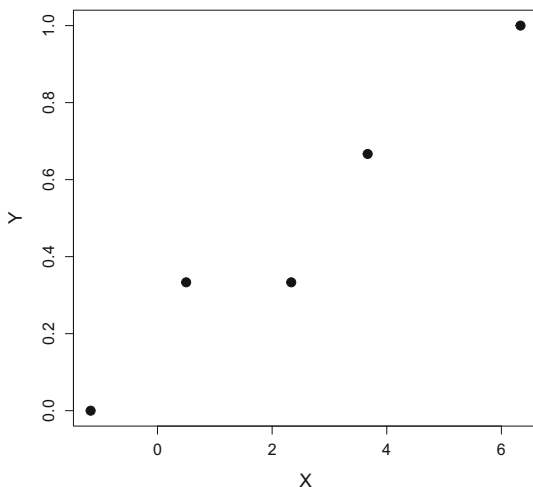
The least squares estimates of μ and β are

$$\hat{\beta} = \frac{\hat{\text{Cov}}(X, Y)}{\hat{\text{Var}}(X)} = \frac{1.07}{8.35} = 0.13$$

$$\hat{\mu} = \bar{X} - \hat{\beta} \bar{Y} = 0.47 - \hat{\beta} 2.33 = 0.17.$$

In these expressions, $\hat{\text{Cov}}(X, Y)$ is the sample covariance between the elements of vectors X and Y , $\hat{\text{Var}}(X)$ is the sample variance among the 5 observations in X , $\bar{X}$ is the mean of the 5 observations in X and $\bar{Y}$ is the mean of the 5 observations in Y .

Fig. 13.2 Plot of Y versus X —the data collected in 5 groups with 6 observations in each group



According to the linear approximation, the estimated probabilities that future individuals will show the disease when exposed to a drug levels equal to $-3, 1, 9$ are

$$\hat{\mu} + \hat{\beta}(-3) = -0.22$$

$$\hat{\mu} + \hat{\beta}1 = 0.30$$

$$\hat{\mu} + \hat{\beta}9 = 1.32.$$

With the exception of the second, the probabilities do not lie within the interval $(0, 1)$.

(c1) Taking logarithms of the likelihood results in the loglikelihood

$$\begin{aligned} \ell(\mu, \beta|y, x) &= \sum_{i=1}^{30} \left\{ y_i \ln \left[\frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)} \right] \right. \\ &\quad \left. + (1 - y_i) \ln \left[\frac{1}{1 + \exp(\mu + \beta x_i)} \right] \right\} \\ &= \sum_{i=1}^{30} \{ y_i (\mu + \beta x_i) - \ln(1 + \exp(\mu + \beta x_i)) \}. \end{aligned} \quad (13.46)$$

(c2) The first derivatives of $\ell(\mu, \beta|y, x)$ with respect to μ and β are

$$\begin{aligned} \frac{\partial \ell(\mu, \beta|y, x)}{\partial \mu} &= \sum_{i=1}^{30} \left(y_i - \frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)} \right), \\ \frac{\partial \ell(\mu, \beta|y, x)}{\partial \beta} &= \sum_{i=1}^{30} \left(x_i y_i - \frac{x_i \exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)} \right). \end{aligned}$$

The second derivatives are

$$\begin{aligned} \frac{\partial^2 \ell(\mu, \beta|y, x)}{(\partial \mu)^2} &= - \sum_{i=1}^{30} \frac{\exp(\mu + \beta x_i)}{(1 + \exp(\mu + \beta x_i))^2}, \\ \frac{\partial^2 \ell(\mu, \beta|y, x)}{(\partial \beta)^2} &= - \sum_{i=1}^{30} \frac{x_i^2 \exp(\mu + \beta x_i)}{(1 + \exp(\mu + \beta x_i))^2}, \\ \frac{\partial^2 \ell(\mu, \beta|y, x)}{\partial \beta \partial \mu} &= - \sum_{i=1}^{30} \frac{x_i \exp(\mu + \beta x_i)}{(1 + \exp(\mu + \beta x_i))^2}. \end{aligned}$$

(c3) The iterative system based on Newton-Raphson is

$$\begin{bmatrix} \mu \\ \beta \end{bmatrix}_{t+1} = \begin{bmatrix} \mu \\ \beta \end{bmatrix}_t + \left[\begin{array}{cc} \sum_{i=1}^{30} \frac{\exp(\mu + \beta x_i)}{(1 + \exp(\mu + \beta x_i))^2} & \sum_{i=1}^{30} \frac{x_i \exp(\mu + \beta x_i)}{(1 + \exp(\mu + \beta x_i))^2} \\ \sum_{i=1}^{30} \frac{x_i \exp(\mu + \beta x_i)}{(1 + \exp(\mu + \beta x_i))^2} & \sum_{i=1}^{30} \frac{x_i^2 \exp(\mu + \beta x_i)}{(1 + \exp(\mu + \beta x_i))^2} \end{array} \right]^{-1} \begin{bmatrix} \sum_{i=1}^{30} \left(y_i - \frac{\exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)} \right) \\ \sum_{i=1}^{30} \left(x_i y_i - \frac{x_i \exp(\mu + \beta x_i)}{1 + \exp(\mu + \beta x_i)} \right) \end{bmatrix} \Bigg|_{\substack{\mu = \mu_t \\ \beta = \beta_t}} \quad (13.47)$$

The following R-code implements the Newton-Raphson algorithm.

```
# CODE1301
rm(list=ls()) # Clear the workspace
set.seed(12371)
# CREATE BINARY DATA
mu <- -2
beta <- 0.7
cov <- rnorm(30,2,3) # GENERATE THE COVARIATE
xb <- cov*beta
p1 <- pnorm(mu+xb) # PROBABILITIES ACCORDING TO PROBIT MODEL
# CREATE DATA:
dat1 <- cbind(rbinom(30,1,p1),round(cov,digits=0))
colnames(dat1) <- c("Y", "X")
### END OF GENERATION OF DATA
nit <- 10 # NUMBER OF N-R ITERATIONS
miu <- matrix(data=NA, nrow=nit+1,ncol=1)
beta <- matrix(data=NA, nrow=nit+1,ncol=1)
resultnr <- matrix(data=NA,nrow=nit,ncol=3)
# START VALUES FOR MIU AND BETA
miu [1]<- 0.17
beta[1] <- 0.13
for (i in 1:nit)
{
  vc11 <- - sum(exp(miu[i]+beta[i]*dat1[,2]))/((1+exp(miu[i]+
    beta[i]*dat1[,2]))^2))
  vc22 <- - sum(dat1[,2]^2*exp(miu[i]+ beta[i]*dat1[,2])/
    ((1+exp(miu[i]+ beta[i]*dat1[,2]))^2))
  vc12 <- - sum(dat1[,2]*exp(miu[i]+ beta[i]*dat1[,2])/
    ((1+exp(miu[i]+beta[i]*dat1[,2]))^2))
  vcmat <- matrix(c(vc11,vc12,vc12,vc22),nrow=2,ncol=2)
  vcmatinv <- solve(vcmat)
  fd1 <- sum((dat1[,1]-(exp(miu[i]+ beta[i]*dat1[,2]))/
    (1+exp(miu[i]+beta[i]*dat1[,2]))))
  fd2 <- sum(((dat1[,1]*dat1[,2])-(dat1[,2]*exp(miu[i]+
    beta[i]*dat1[,2]))/(1+exp(miu[i]+beta[i]*dat1[,2]))))
  fd <- matrix(c(fd1,fd2),nrow=2,ncol=1)
```

```

sol0 <- matrix(c(miu[i], beta[i]),nrow=2,ncol=1)
sol1 <- sol0+(-vcmatinv%%fd)
miu[i+1] <- sol1[1]
beta[i+1] <- sol1[2]
resultnr[i,] <- c(i,sol1[1],sol1[2])
}
resultnr

```

```

##           [,1]      [,2]      [,3]
## [1,]      1 -1.366182 0.5567651
## [2,]      2 -2.014602 0.7949203
## [3,]      3 -2.278267 0.8957553
## [4,]      4 -2.314792 0.9096833
## [5,]      5 -2.315397 0.9099107
## [6,]      6 -2.315397 0.9099107
## [7,]      7 -2.315397 0.9099107
## [8,]      8 -2.315397 0.9099107
## [9,]      9 -2.315397 0.9099107
## [10,]     10 -2.315397 0.9099107

```

```

# ASYMPTOTIC COVARIANCE MATRIX
-vcmatinv

```

```

##           [,1]      [,2]
## [1,] 0.8641485 -0.2427031
## [2,] -0.2427031 0.1003475

```

```

## COMPUTE PROBABILITIES THAT Y=1, GIVEN X = -3, 1, 9
p1<-exp(miu[i+1]+beta[i+1]*(-3))/(1+exp(miu[i+1]+
                                         beta[i+1]*(-3)))
p1

```

```

## [1] 0.006399413

```

```

p2<-exp(miu[i+1]+beta[i+1]*1)/(1+ exp(miu[i+1]+beta[i+1]*1))
p2

```

```

## [1] 0.196947

```

```

p3<-exp(miu[i+1]+beta[i+1]*9)/(1+ exp(miu[i+1]+beta[i+1]*9))
p3

```

```

## [1] 0.9971957

```

After a few iterations the Newton-Raphson method converges to $\hat{\mu} = -2.32$ and $\hat{\beta} = 0.91$

- (c4) The asymptotic covariance matrix of the ML estimators is shown above. The 95% confidence interval for β based on the asymptotic variance is

$$\Pr\left(0.91 - 1.96 \times \sqrt{0.100} < \beta < 0.91 + 1.96 \times \sqrt{0.100}\right) = 0.95,$$

equal to

$$\Pr(0.29 < \beta < 1.53) = 0.95. \quad (13.48)$$

- (c5) The three probabilities (also displayed at the bottom of the code) are

$$\Pr(Y = 1 | \hat{\mu}, \hat{\beta}, x = -1) = 0.006,$$

$$\Pr(Y = 1 | \hat{\mu}, \hat{\beta}, x = 3) = 0.20,$$

$$\Pr(Y = 1 | \hat{\mu}, \hat{\beta}, x = 9) = 0.99,$$

all within the allowed range (0, 1).

As a comparison, the following code uses the R function `OPTIM` to maximise the loglikelihood and to obtain asymptotic variances (`OPTIM` minimises the cost function defined as the negative of the loglikelihood). The bottom part of the code presents the simplest solution to this problem using the standard R function `glm`.

```
# CODE1301 (cont)
### USE THE R-FUNCTION OPTIM TO COMPARE WITH THIS PROGRAMME
dat <- data.frame(dat1)
logl <- function(data,par)
{
  with(data,-sum(Y*(par[1]+par[2]*X)-
    log(1+exp(par[1]+par[2]*X))))
}
result<-optim(par=c(-3.5,0.05),logl,data=dat,
  hessian=TRUE,method="BFGS")
# IF METHOD IS NOT INCLUDED IN THE CALL, OPTIM USES
# THE NELDER-MEAD ALGORITHM
# result <- optim(par=c(-3.5,0.05),logl,data=dat,hessian=TRUE,
# method="BFGS", control=list(trace=1,REPORT=1))
# THIS CALL INCLUDES control=list(trace=1,REPORT=1) WHICH
# PROVIDES THE PROGRESS OF THE ITERATION
result$par

## [1] -2.3154794  0.9099291
```

```
solve(result$hessian)
```

```
##           [,1]      [,2]
## [1,]  0.8641949 -0.2427165
## [2,] -0.2427165  0.1003514
```

```
#####
##### USE GLM in R
d <- data.frame(dat1)
logreg <- glm(d$Y~d$X, data=d, family=binomial(link="logit"))
resglm <- summary(logreg)
pred <- predict(logreg,d,type="response")
logscore <- sum(d$Y*log(pred)+(1-d$Y)*log(1-pred)) # LogLik of
#           full model evaluated at ML estimates
-2*logscore # DEVIANCE FULL MODEL
```

```
## [1] 22.81623
```

```
resglm$deviance
```

```
## [1] 22.81623
```

```
resglm$coefficients[,1:3]
```

```
##           Estimate Std. Error  z value
## (Intercept) -2.3153968  0.9295501 -2.490879
## d$X          0.9099107  0.3167592  2.872563
```

```
# NULL MODEL
logregnull <- glm(d$Y~1,data=d,family=binomial(link="logit"))
resglmnull <- summary(logregnull)
prednull <- predict(logregnull,d,type="response")
logscorenull <- sum(d$Y*log(prednull)+(1-d$Y)*log(1-prednull))
# LogLik of null model evaluated at ML estimate
-2*logscorenull # DEVIANCE NULL MODEL
```

```
## [1] 41.4554
```

```
resglmnull$deviance
```

```
## [1] 41.4554
```

```
resglmnull$coefficients[1:3]
```

```
## [1] -0.1335314  0.3659621 -0.3648776
```

Exercise 2

The model is parametrised as

$$\begin{aligned}\Pr(Y_i = 1|x_i, \beta) &= \Pr(u_i > 0|x_i, \beta) \\ &= \Phi(x'_i\beta)\end{aligned}\tag{13.49}$$

and

$$\Pr(Y_i = 0|x_i, \beta) = 1 - \Phi(x'_i\beta).\tag{13.50}$$

The conditional expectations are

$$\begin{aligned}E(u_i|\beta, x_i, y_i = 1) &= x'_i\beta + \frac{\phi(x'_i\beta)}{\Phi(x'_i\beta)}, \\ E(u_i|\beta, x_i, y_i = 0) &= x'_i\beta - \frac{\phi(x'_i\beta)}{1 - \Phi(x'_i\beta)}.\end{aligned}$$

The likelihood is

$$L(\beta|x, y) \propto \prod_{i=1}^N \left[(\Phi(x'_i\beta))^{y_i} (1 - \Phi(x'_i\beta))^{1-y_i} \right]$$

and the loglikelihood

$$\ell(\beta|x, y) = \sum_{i=1}^N \left[y_i \ln \Phi(x'_i\beta) + (1 - y_i) \ln (1 - \Phi(x'_i\beta)) \right].$$

The R-code below implements the EM algorithm using the probit model. The result is in good agreement with that obtained with the R function OPTIM shown at the bottom.

```
# CODE1302
# LikIIQ2 EM algorithm with probit model
rm(list=ls()) # Clear the workspace
set.seed(12371)
# CREATE BINARY DATA
mu <- -2
beta <- 0.7
cov <- rnorm(30,2,3) # GENERATE THE COVARIATE
xb <- cov*beta
p1 <- pnorm(mu+xb) # PROBABILITIES ACCORDING TO PROBIT MODEL
#p1 <- rbeta(30,2,2)
dat1 <- cbind(rbinom(30,1,p1),round(cov,digits=0)) # CREATE DATA
colnames(dat1) <- c("Y", "X")
```

```

d <- data.frame(dat1)
attach(d)
m <- cbind(1,X)
mt <- t(m)
mtm <- mt%%m
miu <- -1
beta <- 1
# FUNCTION expu THAT COMPUTES CONDITIONAL EXPECTATIONS
# HERE WE ASSUME THAT Y=1 IF u>0
expu <- function(data,par,i)
{
  miu <- par[1]
  beta <- par[2]
  if(Y[i]==1)
  {
    with(data,miu+beta*X[i]+
          (dnorm(miu+beta*X[i])/pnorm(miu+beta*X[i])))
  } else
  {
    with(data,miu+beta*X[i]-
          (dnorm(miu+beta*X[i])/(1-pnorm(miu+beta*X[i]))))
  }
}
## FUNCTION loglik TO COMPUTE THE LOG-LIKELIHOOD
loglik <- function(data,par)
{
  miu <- par[1]
  beta <- par[2]
  with(data,sum(Y*log(pnorm(miu+beta*X))+
                (1-Y)*log(1-pnorm(miu+beta*X))))
}
iter <- 100
euvec <- matrix(data=NA, nrow=30,ncol=1)
sol <- matrix(data=NA, nrow=iter, ncol=2)
llik <- matrix(data=NA,nrow=iter,ncol=1)
result <- matrix(data=NA,nrow=iter,ncol=3)
# PLACE 30 CONDITIONAL EXPECTATIONS IN euvec AND ITERATE
for (i in 1:iter)
{
  for (j in 1:length(X))
  {
    euvec[j] <- expu(d,c(miu,beta),j)
  }
  sol[i,] <- t(solve(mtm)%%mt%%euvec)
  miu <- sol[i,1]
  beta <- sol[i,2]
  llik[i] <- loglik(d,c(miu,beta))
  result[i,] <- c(miu,beta,llik[i])
}
# FINAL ITERATES OF THE EM ALGORITHM
tail(result)

```

```
##           [,1]      [,2]      [,3]
```

```
## [95,] -1.386253 0.5462922 -11.22912
## [96,] -1.386253 0.5462921 -11.22912
## [97,] -1.386253 0.5462921 -11.22912
## [98,] -1.386253 0.5462920 -11.22912
## [99,] -1.386253 0.5462920 -11.22912
## [100,] -1.386253 0.5462919 -11.22912

#####
### USE THE R-FUNCTION OPTIM TO COMPARE WITH THIS PROGRAMME
logl <- function(data,par)
{
  miu <- par[1]
  beta <- par[2]
  with(data,-sum(Y*log(pnorm(miu+beta*X))+
    (1-Y)*log(1-pnorm(miu+beta*X))))
}
result1<-optim(par=c(-1,1),logl,data=d,hessian=TRUE,
  method="BFGS")
# ML ESTIMATES USING OPTIM
result1$par

## [1] -1.386252 0.546292
```

Exercise 3

- (i) To implement the Newton-Raphson algorithm first and second derivatives of the loglikelihood are needed. These are

$$\frac{\partial \ell(\beta_0, \beta_1 | n, x)}{\partial \beta_0} = \sum_{i=1}^{30} \left(y_i - \frac{n_i \exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)} \right),$$

$$\frac{\partial \ell(\beta_0, \beta_1 | n, x)}{\partial \beta} = \sum_{i=1}^{30} \left(x_i y_i - \frac{n_i x_i \exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)} \right).$$

and

$$\frac{\partial^2 \ell(\beta_0, \beta_1 | n, x)}{(\partial \beta_0)^2} = - \sum_{i=1}^{30} \frac{n_i \exp(\beta_0 + \beta_1 x_i)}{(1 + \exp(\beta_0 + \beta_1 x_i))^2},$$

$$\frac{\partial^2 \ell(\beta_0, \beta_1 | n, x)}{(\partial \beta)^2} = - \sum_{i=1}^{30} \frac{n_i x_i^2 \exp(\beta_0 + \beta_1 x_i)}{(1 + \exp(\beta_0 + \beta_1 x_i))^2},$$

$$\frac{\partial^2 \ell(\beta_0, \beta_1 | n, x)}{\partial \beta \partial \beta_0} = - \sum_{i=1}^{30} \frac{n_i x_i \exp(\beta_0 + \beta_1 x_i)}{(1 + \exp(\beta_0 + \beta_1 x_i))^2}.$$

Therefore the iterative system based on Newton-Raphson is

$$\begin{bmatrix} \beta_0 \\ \beta \end{bmatrix}_{t+1} = \begin{bmatrix} \beta_0 \\ \beta \end{bmatrix}_t + \left[\begin{array}{cc} \sum_{i=1}^{30} \frac{n_i \exp(\beta_0 + \beta_1 x_i)}{(1 + \exp(\beta_0 + \beta_1 x_i))^2} & \sum_{i=1}^{30} \frac{n_i x_i \exp(\beta_0 + \beta_1 x_i)}{(1 + \exp(\beta_0 + \beta_1 x_i))^2} \\ \sum_{i=1}^{30} \frac{n_i x_i \exp(\beta_0 + \beta_1 x_i)}{(1 + \exp(\beta_0 + \beta_1 x_i))^2} & \sum_{i=1}^{30} \frac{x_i^2 \exp(\beta_0 + \beta_1 x_i)}{(1 + \exp(\beta_0 + \beta_1 x_i))^2} \end{array} \right]^{-1} \begin{bmatrix} \beta_0 = \beta_{0t} \\ \beta = \beta_t \end{bmatrix} + \begin{bmatrix} \sum_{i=1}^{30} \left(y_i - \frac{n_i \exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)} \right) \\ \sum_{i=1}^{30} \left(x_i y_i - \frac{n_i x_i \exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)} \right) \end{bmatrix}_{\beta_0 = \beta_{0t}, \beta = \beta_t}. \quad (13.51)$$

To obtain start values for the iteration, rough estimates of β_0 and β_1 can be obtained from $\theta_i \approx y_i/n_i$. Then,

$$\text{logit}(\theta_i) = \ln\left(\frac{\theta_i}{1 - \theta_i}\right) = \beta_0 + \beta_1 x_i, \quad i = 1, \dots, 4. \quad (13.52)$$

and

$$\text{logit}(y_i/n_i) = z = \beta_0 + \beta_1 x_i + e_i, \quad i = 1, \dots, 4. \quad (13.53)$$

The logits of 0 and 1 are not defined, so for the purpose of this approximation, change $y_1 = 0.5$ and $y_4 = 4.5$. Writing (13.53) as $z = Xb + e$, the least squares estimates are

$$\hat{\beta}_0 = 0.1; \quad \hat{\beta}_1 = 2.9.$$

The R-code below fits the likelihood using Newton-Raphson. As before, the function OPTIM is included as a check.

```
# CODE1303
## NEWTON-RAPHSON IMPLEMENTATION OF LIKELIHOOD PROBLEM II,
rm(list=ls()) # Clear the workspace
set.seed(771133)
## BINOMIAL DATA SET (Y "successes" out of n=5 "trials")
dat1 <-
matrix(c(0, -0.86, 5, 1, -0.3, 5, 3, -0.05, 5, 5, 0.73, 5),
       nrow=4, ncol=3, byrow=T)
colnames(dat1) <- c("Y", "X", "n")
dat1
```

```
##      Y      X n
## [1,] 0 -0.86 5
## [2,] 1 -0.30 5
## [3,] 3 -0.05 5
## [4,] 5  0.73 5
```

```

nit <- 10
miu <- matrix(data=NA, nrow=nit+1,ncol=1)
beta <- matrix(data=NA, nrow=nit+1,ncol=1)
resultnr <- matrix(data=NA, nrow=nit,ncol=2)

miu [1]<- 0.1
beta[1] <- 2.9
for (i in 1:nit)
{
  vc11 <- - sum(5*exp(miu[i]+beta[i]*dat1[,2])/
    ((1+exp(miu[i]+ beta[i]*dat1[,2]))^2))
  vc22 <- - sum(5*dat1[,2]^2*exp(miu[i]+ beta[i]*dat1[,2])/
    ((1+exp(miu[i]+ beta[i]*dat1[,2]))^2))
  vc12 <- - sum(5*dat1[,2]*exp(miu[i]+ beta[i]*dat1[,2])/
    ((1+exp(miu[i]+ beta[i]*dat1[,2]))^2))
  vcmat <- matrix(c(vc11,vc12,vc12,vc22),nrow=2,ncol=2)
  vcmatinv <- solve(vcmat)
  fd1 <- sum((dat1[,1]-(5*exp(miu[i]+ beta[i]*dat1[,2]))/
    (1+exp(miu[i]+ beta[i]*dat1[,2]))))
  fd2 <- sum(((dat1[,1]*dat1[,2])-(5*dat1[,2]*exp(miu[i]+
    beta[i]*dat1[,2]))/(1+exp(miu[i]+ beta[i]*dat1[,2]))))

  fd <- matrix(c(fd1,fd2),nrow=2,ncol=1)
  sol0 <- matrix(c(miu[i], beta[i]),nrow=2,ncol=1)
  sol1 <- sol0+(-vcmatinv%%fd)
  miu[i+1] <-sol1[1]
  beta[i+1] <- sol1[2]
  resultnr[i,] <- c(miu[i+1],beta[i+1])
}
resultnr

```

```

##           [,1]      [,2]
## [1,] 0.3048018 4.425389
## [2,] 0.5560914 5.914432
## [3,] 0.7610883 7.169775
## [4,] 0.8394160 7.696086
## [5,] 0.8465271 7.748401
## [6,] 0.8465802 7.748817
## [7,] 0.8465802 7.748817
## [8,] 0.8465802 7.748817
## [9,] 0.8465802 7.748817
## [10,] 0.8465802 7.748817

```

```
-vcmatinv
```

```

##           [,1]      [,2]
## [1,] 1.038535 3.545987
## [2,] 3.545987 23.743865

```

```

### USE THE R-FUNCTION OPTIM TO COMPARE WITH THIS PROGRAMME
dat <- data.frame(dat1)
logl <- function(data,par)
{
  with(data,-sum(Y*(par[1]+par[2]*X)-
                 5*log(1+exp(par[1]+par[2]*X))))
}
result <- optim(par=c(0.1,2.9),logl,data=dat,
                hessian=TRUE,method="BFGS")
# IF METHOD NOT INCLUDED, OPTIM USES NELDER-MEAD ALGORITHM
# THIS CALL INCLUDES control=list(trace=1,REPORT=1):
# PROVIDES THE PROGRESS OF THE ITERATION
result$par

```

```
## [1] 0.8468243 7.7496258
```

```
solve(result$hessian)
```

```

##           [,1]      [,2]
## [1,] 1.038684  3.546793
## [2,] 3.546793 23.748757

```

(ii) The EM algorithm is constructed assuming the probit model

$$\begin{aligned}
 \Pr(Y_i = 1|x_i, \beta) &= \Pr(u_i > 0|x_i, \beta) \\
 &= \Pr(x'_i\beta + e_i > 0|x_i, \beta) \\
 &= \Pr(e_i > -x'_i\beta|x_i, \beta) \\
 &= \Phi(x'_i\beta),
 \end{aligned}$$

$$\Pr(Y_i = 0|x_i, \beta) = 1 - \Phi(x'_i\beta),$$

where $e_i \sim N(0, 1)$ and $\Phi(z)$ is the cdf of the standard normal distribution evaluated at z . The binomial likelihood is proportional to

$$L(\beta|n, N, x) \propto \prod_{i=1}^C [\Phi(x'_i\beta)]^{n_i} [1 - \Phi(x'_i\beta)]^{N_i - n_i}$$

and the loglikelihood is

$$\ell(\beta|n, N, x) = \sum_{i=1}^C n_i \ln \Phi(x'_i\beta) + (N_i - n_i) \ln (1 - \Phi(x'_i\beta))$$

where C is the number of levels of the covariate (in the example, $C = 4$).

After 950 iterations the system converges to $\hat{\beta}_0 = 0.484$ and $\hat{\beta}_1 = 4.459$. At this point the loglikelihood is equal to -5.869818 . The R-code below fits the likelihood using the EM algorithm based on the iterative system (3.94) on page 123. The R-function OPTIM is included as a comparison. It provides the same estimates and in addition, an estimate of the asymptotic covariance matrix. This is

$$\text{Var}(\hat{\beta}_0, \hat{\beta}_1) = \begin{bmatrix} 0.434 & 1.575 \\ 1.575 & 9.733 \end{bmatrix}.$$

Based on $\text{Var}(\hat{\beta}_0, \hat{\beta}_1)$, the 95% confidence intervals for β_0 and β_1 are

$$\Pr(-0.807 < \beta_0 < 1.775) = 0.95,$$

$$\Pr(-1.655 < \beta_0 < 10.574) = 0.95,$$

reflecting large uncertainty in the inference.

```
# CODE1304
## EM PROBIT MODEL; LIKELIHOOD PROBLEMS II, QUESTION 3ii
rm(list=ls()) # Clear the workspace
set.seed(12371)
### BINOMIAL DATA (Y successes out of n=5 trials)
dat1 <-
matrix(c(0,-0.86,5,1,-0.3,5,3,-0.05,5,5,0.73,5),
       nrow=4,ncol=3,byrow=T)
colnames(dat1) <-c("n","X","N")
dat1

##      n      X N
## [1,] 0 -0.86 5
## [2,] 1 -0.30 5
## [3,] 3 -0.05 5
## [4,] 5  0.73 5

nit <- 1000
miu <- 0.1
beta <- 2.9
dat <- data.frame(dat1)
coll <- matrix(rep(1:1,length(dat1[,1])),
              nrow=length(dat1[,1]),ncol=1)
m <- matrix(c(coll,dat1[,2]),nrow=length(dat1[,1]),ncol=2)
mt <- t(m)
dN <- diag(dat$N)
dn <- diag(dat$n)
dd <- dN-dn
## FUNCTION loglik TO COMPUTE THE LOG-LIKELIHOOD
loglik <- function(data,par)
{
  miu <-par[1]
```

```

beta <- par[2]
with(data, sum(n*log(pnorm(miu+beta*X))+
               (N-n)*log(1-pnorm(miu+beta*X))))
}
# CONSTRUCT FUNCTIONS expu0 and expu1 THAT COMPUTE
# CONDITIONAL EXPECTATIONS
# ASSUME: Y=1 IF u>0; THEREFORE Pr[Y=1]=F (CDF of std. normal)
expu1 <- function(data,par,i)
{
  miu <- par[1]
  beta <- par[2]
  with(data, (miu+beta*X[i]+
              dnorm(miu+beta*X[i])/pnorm(miu+beta*X[i])))
}
expu0 <- function(data,par,i)
{
  miu <- par[1]
  beta <- par[2]
  with(data, (miu+beta*X[i]-
              dnorm(miu+beta*X[i])/(1-pnorm(miu+beta*X[i]))))
}
sol <- matrix(data=NA, nrow=nit, ncol=2)
e1 <- matrix(data=NA, nrow=length(dat1[,1]), ncol=1)
e0 <- matrix(data=NA, nrow=length(dat1[,1]), ncol=1)
llik <- matrix(data=NA, nrow=nit, ncol=1)
result <- matrix(data=NA, nrow=nit, ncol=3)
for (i in 1:nit)
{
  for (j in 1:length(dat1[,1]))
  {
    e1[j] <- expu1(data=dat, c(miu, beta), j)
    e0[j] <- expu0(data=dat, c(miu, beta), j)
  }
  sol[i,] <- t(solve(mt%**dN%**m)%%(mt%**dn%**e1+mt%**dd%**e0))
  miu <- sol[i,1]
  beta <- sol[i,2]
  llik[i] <- loglik(dat, c(miu, beta))
  result[i,] <- c(miu, beta, llik[i])
}
tail(result)

```

```

##           [,1]      [,2]      [,3]
## [995,] 0.4839674 4.45866 -5.869818
## [996,] 0.4839674 4.45866 -5.869818
## [997,] 0.4839674 4.45866 -5.869818
## [998,] 0.4839674 4.45866 -5.869818
## [999,] 0.4839674 4.45866 -5.869818
## [1000,] 0.4839674 4.45866 -5.869818

```

```

### USE THE R-FUNCTION OPTIM TO COMPARE WITH THIS PROGRAMME
logl <- function(data,par)
{

```

```

miu <- par[1]
beta <- par[2]
with(data, -sum(n*log(pnorm(miu+beta*X))+
               (N-n)*log(1-pnorm(miu+beta*X))))
}
result1 <- optim(par=c(0.1,2.9),logl,data=dat,
                   hessian=TRUE,method="BFGS")
result2 <- optim(par=c(0.1,2.9),logl,data=dat,
                   hessian=TRUE)
result1$par

```

```
## [1] 0.4839727 4.4587022
```

```
solve(result1$hessian)
```

```
##           [,1]      [,2]
## [1,] 0.4343711 1.575394
## [2,] 1.5753937 9.732975
```

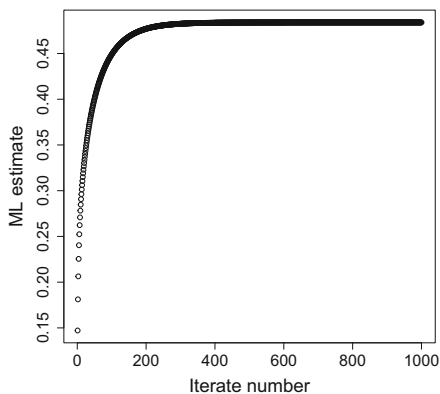
Figure 13.3 displays the trajectory of the EM iterates for μ . The corresponding plot for β shows a similar pattern.

- (iii) Data (number of deaths) are assumed to be binomially distributed. This requires that the binary observations contributing to the number of deaths are independent. This will not be the case if the data consist of related individuals: in the presence of genetic variation or of effects of common environment records of relatives are correlated.

When observations within a class consist of full sibs, the liability for a given individual (assume the probit model for the example) can be written

$$u_{ij} = x_i' \beta + f_i + e_{ij}, \dots j = 1, \dots, N_i \quad (13.54)$$

Fig. 13.3 Trajectory of the EM iterates for μ



where now f_i is the effect of the i th full-sib family common to the N_i records that belong to covariate level i and $e_{ij} \stackrel{iid}{\sim} N(0, 1)$. Assume that $f_i \stackrel{iid}{\sim} N(0, \sigma_f^2)$, where σ_f^2 is the covariance between full-sibs. Then

$$\text{Cov}(u_{ij}, u_{ik}) = \sigma_f^2.$$

Conditioning on f_i , $[u_i|x_i, \beta, f_i] \stackrel{iid}{\sim} N(x'_i\beta + f_i, 1)$ and

$$\begin{aligned} \Pr(Y_{ij} = 1, Y_{ik} = 1|x_i, \beta, f_i) &= \Pr(u_{ij} > 0, u_{ik} > 0|x_i, \beta, f_i) \\ &= \Pr(u_{ij} > 0|x_i, \beta, f_i) \Pr(u_{ik} > 0|x_i, \beta, f_i) \\ &= \Pr(Y_{ij} = 1|x_i, \beta, f_i) \Pr(Y_{ik} = 1|x_i, \beta, f_i). \end{aligned}$$

In view of the conditional independence of Y_{ij} , given f_i , the sum of independent binary random variables $n_i = \sum_{j=1}^{N_i} Y_{ij}$ is binomial. In the probit model the probability that $Y_{ij} = 1$ is

$$\begin{aligned} \Pr(Y_{ij} = 1|x_i, \beta, f_i) &= \Pr(u_i > 0|x_i, \beta, f_i) \\ &= \Pr(x'_i\beta + f_i + e_{ij} > 0|x_i, \beta, f_i) \\ &= \Pr(e_{ij} > -x'_i\beta - f_i|x_i, \beta, f_i) \\ &= \Phi(x'_i\beta + f_i) \end{aligned}$$

and

$$\Pr(Y_{ij} = 0|x_i, \beta, f_i) = 1 - \Phi(x'_i\beta + f_i).$$

The conditional distribution of the data $n = \{n_i\}_{i=1}^C$ given vector $f = \{f_i\}_{i=1}^C$ is binomial and takes the form

$$\Pr(n|x_i, \beta, f) = \prod_{i=1}^C [\Phi(x'_i\beta + f_i)]^{n_i} [1 - \Phi(x'_i\beta + f_i)]^{N_i - n_i}$$

and the marginal distribution of the data is obtained averaging over the distribution $[f|\sigma_f^2]$

$$\begin{aligned} \Pr(n|x_i, \beta) &= \int \prod_{i=1}^C [\Phi(x'_i\beta + f_i)]^{n_i} [1 - \Phi(x'_i\beta + f_i)]^{N_i - n_i} p(f|\sigma_f^2) df \\ &= \prod_{i=1}^C \int [\Phi(x'_i\beta + f_i)]^{n_i} [1 - \Phi(x'_i\beta + f_i)]^{N_i - n_i} p(f_i|\sigma_f^2) df_i \quad (13.55) \end{aligned}$$

where the equality in the second line holds if the full-sib families are independent. The likelihood is proportional to (13.55) whose construction requires the solution of univariate integrals. In such univariate cases numerical methods such as Gaussian quadrature work well and the problem does not pose a computational obstacle. A benchmark paper pre McMC is Anderson and Aitkin (1985).

Exercise 4

The first R-code below generates genomic data (assumes genomic variance is 10 and residual variance is 15).

```
# CODE1305
# LIKELIHOOD PROBLEMS II QUESTION 4
# DATA BASED ON GENOMIC MODEL AND OBTAIN THE SVD OF WW'(1/m)
rm(list=ls()) # CLEAR WORKSPACE
set.seed(1327)
nindiv<-2000
nmark<-20000
nt<-nindiv*nmark
X<-matrix(nrow= nindiv,ncol= nmark,rbinom(n=nt,size=2,p=.5))
stdev <- matrix(data=NA,nrow= nmark,ncol=1)
W <- matrix(data=NA,nrow= nindiv,ncol=nmark)
U <- matrix(data=NA,nrow= nindiv,ncol= nindiv)
G<-matrix(data=NA,nrow= nindiv,ncol= nindiv)
cm <- colMeans(X)
# CREATE MATRIX OF STANDARDISED MARKER GENOTYPE CODES
for (i in 1:nmark)
{
  W[,i] <- ( X[,i]-cm[i]) / sd(X[,i])
}
# COULD USE INSTEAD:
# W <- scale(X)
#qr(X)$rank
#qr(W)$rank
#nmark MARKER VALUES: REALISATIONS FROM N(0,I sqrt(0.001))
g <- (1/sqrt(nmark))*W%*%rnorm(nmark,mean=0,sd=sqrt(10))
# GENERATE nindiv PHENOTYPES WITH MEAN 0, VAR=10+15,
# GENOMIC HERITABILITY=10/(10+15)=0.4
#PARAMETER k = Vg/Ve = 10/15 =0.67
y <- g+rnorm(nindiv,mean=0,sd=sqrt(15))
# GENOMIC RELATIONSHIP MATRIX G
G <- (1/nmark)*W%*%t(W)
# SVD OF G
EVD <- eigen(G)
names(EVD)
```

```
## [1] "values" "vectors"
```



```
head(EVD$values)
```

```
## [1] 1.731427 1.724233 1.722874 1.715446 1.710233 1.707764
```

```
U <- EVD$vector
val <- EVD$values
val[length(y)] <- 0
D <- diag(val,nrow=nindiv)
ytilde <- t(U)%%y
dim(ytilde)
```

```
## [1] 2000    1
```

```
#END OF GENERATION OF DATA
```

- (i) The R-code below performs the Newton-Raphson computations and the top part includes a call to OPTIM as a check.

```
#####
# CODE1305(cont)
# FUNCTIONS loglik AND logliktransf TO COMPUTE LOG-LIKELIHOODS
loglik <- function(data,par)
{
  ve <- par[1]
  k <- par[2]
  ll <- -0.5*(length(ytilde)*log(ve)+sum(log(val*k+1))+
    (1/ve)*sum(ytilde^2/(val*k+1)))
  return(-ll)
}
# FUNCTION logliktransf TO COMPUTE TRANSFORMED LOG-LIKELIHOOD
logliktransf <- function(data,par)
{
  nue <- par[1]
  nug <- par[2]
  lltransf<- -0.5*(length(ytilde)*nue+sum(log(val*exp(nug-nue)+1))+
    (1/exp(nue))*sum(ytilde^2/(val*exp(nug-nue)+1)))
  return(-lltransf)
}
#####
# FUNCTION OPTIM TO COMPARE WITH RESULTS TO COME
result1 <- optim(par=c(5,0.5),loglik,data =ytilde,hessian=TRUE)
result1$par
```

```
## [1] 18.2720286 0.2991165
```

```
# OBTAIN ASYMPTOTIC VARIANCES BY INVERSION OF THE -HESSIAN
solve(result1$hessian)
```

```
##           [,1]           [,2]
## [1,]  5.6228947 -0.38289962
## [2,] -0.3828996  0.02772035
```

```
# USE OPTIM TO MAXIMIZE TRANSFORMED LOG-LIKELIHOOD
result2 <-
optim(par=c(exp(5),exp(0.5)),logliktransf,
      data=ytilde,hessian=TRUE)
result2$par
```

```
## [1] 2.901324 1.714862
```

```
solve(result2$hessian)
```

```
##           [,1]           [,2]
## [1,]  0.01619379 -0.04998708
## [2,] -0.04998708  0.17211488
```

```
#####
# NEWTON-RAPHSON COMPUTATIONS
nit <- 20
resultnr<-matrix(data=NA,nrow=nit,ncol=3)
llike<-matrix(data=NA,nrow=nit,ncol=1)
ve <- matrix(data=NA, nrow=nit+1,ncol=1)
k <- matrix(data=NA, nrow=nit+1,ncol=1)
ve[1] <- 7
k[1] <- 0.4
llike[1] <- -loglik(ytilde,c(ve[1],k[1]))
for (i in 1:nit)
{
  vc11 <- - 0.5*((2/ve[i]^3)*sum(ytilde^2/(1+k[i]*val))-
    length(ytilde)/ve[i]^2)
  vc12 <- -0.5*(1/ve[i]^2)*sum(val*ytilde^2/(1+k[i]*val)^2)
  vc22 <- -0.5*((1/ve[i])*sum(2*val^2*ytilde^2/(1+k[i]*val)^3)-
    sum(val^2/(1+k[i]*val)^2))
  vcmat <- matrix(c(vc11,vc12,vc12,vc22),nrow=2,ncol=2)
  vcmatinv <- solve(vcmat)
  fd1 <- -0.5*((length(ytilde)/ve[i])-
    (1/ve[i]^2)*sum(ytilde^2/(1+k[i]*val))))
  fd2<--0.5*(sum(val/(1+k[i]*val))-
    (1/ve[i])*sum(val*ytilde^2/((1+k[i]*val)^2)))
  fd <- matrix(c(fd1,fd2),nrow=2,ncol=1)
  sol0 <- matrix(c(ve[i],k[i]),nrow=2,ncol=1)
  sol1 <- sol0 - (vcmatinv%*%fd)
  ve[i+1] <- sol1[1]
  k[i+1] <- sol1[2]
```

```

llike[i+1] <- loglik(ytilde,c(ve[i+1],k[i+1]))
resultnr[i,] <- c(sol1[1],sol1[2],llike[i])
}
#PRINT RESULTS
tail(resultnr)

```

```

##           [,1]      [,2]      [,3]
## [15,] 18.2763 0.2986934 4164.218
## [16,] 18.2763 0.2986934 4164.218
## [17,] 18.2763 0.2986934 4164.218
## [18,] 18.2763 0.2986934 4164.218
## [19,] 18.2763 0.2986934 4164.218
## [20,] 18.2763 0.2986934 4164.218

```

```

-vcmatinv

```

```

##           [,1]      [,2]
## [1,]  5.6031133 -0.38126712
## [2,] -0.3812671  0.02758818

```

- (i) After 8–10 iterations the Newton-Raphson algorithm converges to $\hat{\sigma}_e^2 = 18.28$ and $\hat{k} = 0.30$. The asymptotic variance-covariance matrix is

$$\text{Var}(\hat{\sigma}_e^2, \hat{k}) = \begin{bmatrix} 5.603 & -0.381 \\ -0.381 & 0.028 \end{bmatrix}.$$

- (ii) The values obtained using OPTIM are $\hat{\sigma}_e^2 = 18.27$ and $\hat{k} = 0.30$ with asymptotic variances

$$\text{Var}(\hat{\sigma}_e^2, \hat{k}) = \begin{bmatrix} 5.622 & -0.383 \\ -0.383 & 0.028 \end{bmatrix}.$$

- (iii) The ML estimate of $\sigma_g^2 = k\sigma_e^2$ is $\hat{k}\hat{\sigma}_e^2 = 5.48$. The asymptotic variance is

$$\text{Var}(g(\hat{\theta})) = \left. \frac{\partial \lambda}{\partial \theta'} \right|_{\theta=\hat{\theta}} \text{Var}(\hat{\theta}) \left. \frac{\partial \lambda'}{\partial \theta} \right|_{\theta=\hat{\theta}}. \quad (13.56)$$

In this expression, the original parameters are $\theta' = (\sigma_e^2, k)$ and we are interested in

$$\lambda = g(\theta) = (\sigma_e^2, k\sigma_e^2)',$$

where $k\sigma_e^2 = \sigma_g^2$. Now,

$$\frac{\partial \lambda}{\partial \theta'} = \begin{bmatrix} \frac{\partial \sigma_e^2}{\partial \sigma_e^2} & \frac{\partial \sigma_e^2}{\partial k} \\ \frac{\partial k\sigma_e^2}{\partial \sigma_e^2} & \frac{\partial k\sigma_e^2}{\partial k} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ k & \sigma_e^2 \end{bmatrix}.$$

Therefore (13.56) is

$$\begin{aligned} \text{Var}(\hat{\sigma}_e^2, \hat{k}\hat{\sigma}_e^2) &= \begin{bmatrix} 1 & 0 \\ 0.30 & 18.28 \end{bmatrix} \begin{bmatrix} 5.603 & -0.381 \\ -0.381 & 0.028 \end{bmatrix} \begin{bmatrix} 1 & 0.30 \\ 0 & 18.28 \end{bmatrix} \\ &= \begin{bmatrix} 5.603 & -5.266 \\ -5.266 & 5.693 \end{bmatrix} \end{aligned}$$

and $\text{Var}(\hat{\sigma}_g^2) = 5.693$.

- (iv) The R-code below performs the EM computations based on the iterative system (3.141).

```
# CODE1305 (cont)
# EM COMPUTATIONS
emiter<-1000
vgem<-5
veem<-10
kem<-vgem/veem
resultem<-matrix(data=NA,nrow=emiter,ncol=4)
for (i in 1:emiter)
{
  expalfa<-kem*(val/(val*kem+1))*ytilde
  tol<-0.00001
  trdiv<-vgem*sum(1/(val[val>tol]*kem+1))
  trv<-vgem*sum(val/(val*kem+1))
  vgem<-((kem^2)*sum(ytilde^2*val/(val*kem+1)^2)+trdiv)/(nindiv-1)
  veem<-((sum(y^2)-2*sum(expalfa*ytilde)+sum(expalfa^2)+trv)/nindiv)
  kem<-vgem/veem
  loglike<-loglik(ytilde,c(veem,kem))
  resultem[i,]<-c(vgem,veem,kem,-loglike)
}
tail(resultem)
```

```
##           [,1]      [,2]      [,3]      [,4]
## [995,]  5.464388 18.27113 0.2990722 -4164.218
## [996,]  5.464361 18.27116 0.2990703 -4164.218
## [997,]  5.464335 18.27118 0.2990684 -4164.218
## [998,]  5.464308 18.27121 0.2990666 -4164.218
## [999,]  5.464282 18.27124 0.2990647 -4164.218
## [1000,] 5.464256 18.27126 0.2990629 -4164.218
```

After approximately 950 iterations the EM algorithm converges to the same estimates obtained using Newton-Raphson (to 2 decimal places). These are $\hat{\sigma}_g^2 = 5.46$, $\hat{\sigma}_e^2 = 18.27$ and $\hat{k} = 0.30$.

13.3 Bayes Exercises I

Exercise 1

Start by reading in the data:

```
y <- c(45.83, 50.37, 50.06, 51.59, 48.43, 52.75, 42.92, 48.57,
       46.18, 50.20)
ybar <- mean(y)
ssq <- var(y)
```

The sample mean $\bar{y}$ and the sample variance $S^2 = \frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n-1}$ are 48.69 and 8.859, respectively, with sample size $n = 10$.

- (i) Derivation of the density of $[\mu|y]$ requires integration of (12.25) with respect to σ^2 . Using

$$\sum_{i=1}^n (y_i - \mu)^2 = (n-1) S^2 + n (\bar{y} - \mu)^2,$$

where

$$S^2 = \frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n-1},$$

leads to

$$p(\mu, \sigma^2 | y) \propto (\sigma^2)^{-(\frac{n}{2}+1)} \exp\left(-\frac{1}{2\sigma^2} \left[(n-1) S^2 + n (\bar{y} - \mu)^2\right]\right). \quad (13.57)$$

Let $\alpha = n/2$. Then

$$p(\mu | y) = \int_0^\infty K(\sigma^2)^{-\alpha-1} \exp\left(-\frac{1}{2\sigma^2} \left[(n-1) S^2 + n (\bar{y} - \mu)^2\right]\right) d\sigma^2, \quad (13.58)$$

where K is a constant of integration. Let

$$\begin{aligned} A &= (n-1)S^2 + n(\bar{y} - \mu)^2, \\ x &= \frac{A}{2\sigma^2}, \\ d\sigma^2 &= -\frac{A}{2}x^{-2}dx. \end{aligned}$$

Substituting in (13.58)

$$\begin{aligned} p(\mu|y) &= -\frac{A}{2} \int_0^\infty K \left(\frac{A}{2x} \right)^{-\alpha-1} \exp(-x) x^{-2} dx \\ &= -\left(\frac{A}{2} \right) \left(\frac{A}{2} \right)^{-\alpha-1} \int_0^\infty K \left(\frac{1}{x} \right)^{-\alpha-1} \exp(-x) x^{-2} dx \\ &\propto A^{-\alpha} \int_0^\infty x^{\alpha-1} \exp(-x) dx \\ &\propto A^{-\alpha} \end{aligned}$$

where the last line results from the fact that the integral, known as the Gamma function is

$$\int_0^\infty x^{\alpha-1} \exp(-x) dx = \Gamma(\alpha), \quad \alpha > 0,$$

a constant with respect to μ . Then

$$\begin{aligned} p(\mu|y) &\propto \left((n-1)S^2 + n(\bar{y} - \mu)^2 \right)^{-\frac{n}{2}} \\ &= \left((n-1)S^2 + n(\bar{y} - \mu)^2 \right)^{-\left(\frac{v+1}{2}\right)}, \end{aligned}$$

where $v = n - 1$. Finally,

$$p(\mu|y) \propto \left(1 + \frac{n(\bar{y} - \mu)^2}{vS^2} \right)^{-\left(\frac{v+1}{2}\right)}, \quad (13.59)$$

that is the kernel of the density of a t -distribution with $v = n - 1$ degrees of freedom, mean $\bar{y}$ and scale S^2/n ; that is

$$[\mu|y] \sim t \left(n-1, \bar{y}, \frac{S^2}{n} \right). \quad (13.60)$$

The mean and variance are

$$\begin{aligned} E(\mu|y) &= \bar{y}, \\ \text{Var}(\mu|y) &= \frac{S^2}{n} \frac{n-1}{n-3}. \end{aligned}$$

Derivation of the density of $[\sigma^2|y]$, requires the following integration of (12.25) with respect to μ :

$$\begin{aligned} p(\sigma^2|y) &\propto \int_{-\infty}^{\infty} (\sigma^2)^{-\left(\frac{n}{2}+1\right)} \exp\left(-\frac{1}{2\sigma^2} \left[(n-1)S^2 + n(\bar{y} - \mu)^2\right]\right) d\mu \\ &= (\sigma^2)^{-\left(\frac{n}{2}+1\right)} \exp\left(-\frac{1}{2\sigma^2} (n-1)S^2\right) \int_{-\infty}^{\infty} \exp\left(-\frac{1}{2\sigma^2} n(\bar{y} - \mu)^2\right) d\mu \\ &= (\sigma^2)^{-\left(\frac{n}{2}+1\right)} \exp\left(-\frac{1}{2\sigma^2} (n-1)S^2\right) \left(2\pi \frac{\sigma^2}{n}\right)^{\frac{1}{2}} \\ &\propto (\sigma^2)^{-\left(\frac{n}{2}+1\right)} (\sigma^2)^{\frac{1}{2}} \exp\left(-\frac{1}{2\sigma^2} (n-1)S^2\right) \\ &= (\sigma^2)^{-\left(\frac{n-1}{2}+1\right)} \exp\left(-\frac{1}{2\sigma^2} (n-1)S^2\right), \end{aligned} \tag{13.61}$$

which is the kernel of the density of a scaled inverted chi-square distribution with $(n-1)$ degrees of freedom and scale S^2 . Therefore,

$$[\sigma^2|y] \sim \chi^{-2}(n-1, S^2). \tag{13.62}$$

(ii) The expected value and variance of the t -distribution (13.60) are given by

$$\begin{aligned} E(\mu|y) &= \bar{y} = 48.69, \\ \text{Var}(\mu|y) &= \frac{n-1}{n} \frac{S^2}{n} = \frac{9}{7} \frac{8.859}{10} = 1.139. \end{aligned}$$

The modal value is equal to the mean.

The mean, variance and mode of the scale inverted chi-square distribution (13.62) are

$$\begin{aligned} E(\sigma^2|y) &= \frac{n-1}{n-3} S^2 = 11.39, \\ \text{Var}(\sigma^2|y) &= \frac{2(n-1)^2}{(n-3)^2(n-5)} S^4 = \frac{162}{49 \times 5} 8.86^2 = 51.91, \\ \arg \max [\sigma^2|y] &= \frac{n-1}{n+1} S^2 = \frac{9}{11} 8.86 = 7.25. \end{aligned}$$

(iii) To compute confidence intervals for the t -distribution, it is easiest to transform μ to the standard t -distribution as follows.

Let

$$\begin{aligned} t &= f(\mu) = \frac{\mu - \bar{y}}{S/\sqrt{n}}; \\ \mu &= f^{-1}(t) = \frac{S}{\sqrt{n}}t + \bar{y}; \\ \frac{df^{-1}(t)}{t} &= S/\sqrt{n}; \text{ not a function of } t; \\ \Rightarrow p(t) &\propto \left(1 + \frac{n \left(\bar{y} - \frac{S}{\sqrt{n}}t - \bar{y} \right)^2}{v S^2} \right)^{-\left(\frac{v+1}{2}\right)} \\ &= \left(1 + \frac{t^2}{v} \right)^{-\left(\frac{v+1}{2}\right)} \end{aligned}$$

which is the kernel of the density of a standard t -distribution $t(v)$, with $v = n - 1$. Then using the R-function `qt(c(.025, .975), df=9)` the higher and lower bounds are

$$\begin{aligned} h &= 2.262157, \\ \ell &= -2.262157 \end{aligned}$$

which in the original scale (obtained as $\left(h \times \frac{S}{\sqrt{n}}\right) + \bar{y}$ and $\left(\ell \times \frac{S}{\sqrt{n}}\right) + \bar{y}$) results in the 95% posterior interval

$$\Pr(46.56 < \mu < 50.82|y) = 0.95. \quad (13.63)$$

To compute the 95% posterior interval of σ^2 , it seems easy to use the function `qigamma` in R. First recall that the relationship between the scaled inverse chi-square and the inverse gamma distributions is

$$\chi^{-2}\left(v, S^2\right)=IG\left(\frac{v}{2}, \frac{v S^2}{2}\right).$$

Therefore in terms of (13.62) requires working with

$$IG\left(\frac{n-1}{2}, \frac{(n-1) S^2}{2}\right).$$

The R function `qigamma` is part of the `pscl` package that must be downloaded and installed. The R-code below does this and computes the 95% posterior interval:

```
#install.packages("pscl", .libPaths()[1])
library(pscl)
a <- (length(y)-1)/2
b <- (length(y)-1)*var(y)/2
qigamma(c(0.025,0.975),a,b)
```

```
## [1] 4.191167 29.524482
```

The 95% posterior interval is

$$\Pr\left(4.19 < \sigma^2 < 29.52|y\right) = 0.95. \quad (13.64)$$

(iv) According to (first order) asymptotic theory the posterior distribution of $[\mu, \sigma^2|y]$ is

$$[\mu, \sigma^2|y] \sim N\left(\begin{bmatrix} \hat{\mu} \\ \hat{\sigma}^2 \end{bmatrix}, -\begin{bmatrix} \left(\frac{\partial^2 p(\mu, \sigma^2|y)}{(\partial \mu)^2}\right) & \left(\frac{\partial^2 p(\mu, \sigma^2|y)}{\partial \mu \partial \sigma^2}\right) \\ \left(\frac{\partial^2 p(\mu, \sigma^2|y)}{\partial \mu \partial \sigma^2}\right) & \left(\frac{\partial^2 p(\mu, \sigma^2|y)}{(\partial \sigma^2)^2}\right) \end{bmatrix}^{-1}\right), \quad (13.65)$$

where $\hat{\mu}$ and $\hat{\sigma}^2$ are the mode of $[\mu, \sigma^2|y]$.

The density of the joint posterior distribution was given in (13.57):

$$p\left(\mu, \sigma^2|y\right) \propto \left(\sigma^2\right)^{-\left(\frac{n}{2}+1\right)} \exp\left(-\frac{1}{2\sigma^2}\left[(n-1) S^2+n(\bar{y}-\mu)^2\right]\right),$$

and the logposterior density, up to an additive constant is

$$\ln p\left(\mu, \sigma^2|y\right) = -\left(\frac{n}{2}+1\right) \ln \sigma^2 - \frac{1}{2\sigma^2}\left[(n-1) S^2+n(\bar{y}-\mu)^2\right]. \quad (13.66)$$

First derivatives are

$$\begin{aligned}\frac{\partial \ln p(\mu, \sigma^2|y)}{\partial \mu} &= \frac{n(\bar{y} - \mu)}{\sigma^2}, \\ \frac{\partial \ln p(\mu, \sigma^2|y)}{\partial \sigma^2} &= -\frac{n+2}{2\sigma^2} + \frac{(n-1)S^2 + n(\bar{y} - \mu)^2}{2\sigma^4}.\end{aligned}$$

Setting these equations equal to zero and solving for μ and σ^2 gives

$$\begin{bmatrix} \hat{\mu} \\ \hat{\sigma}^2 \end{bmatrix} = \begin{bmatrix} \bar{y} \\ \frac{n-1}{n+2} S^2 \end{bmatrix} = \begin{bmatrix} \bar{y} \\ \frac{\sum (y_i - \bar{y})^2}{n+2} \end{bmatrix}. \quad (13.67)$$

Second derivatives are

$$\begin{aligned}\frac{\partial^2 p(\mu, \sigma^2|y)}{(\partial \mu)^2} &= -\frac{n}{\sigma^2}, \\ \frac{\partial^2 p(\mu, \sigma^2|y)}{(\partial \sigma^2)^2} &= \frac{n+2}{2\sigma^4} - \frac{(n-1)S^2 + n(\bar{y} - \mu)^2}{\sigma^6}, \\ \frac{\partial^2 p(\mu, \sigma^2|y)}{\partial \mu \partial \sigma^2} &= -\frac{n(\bar{y} - \mu)}{\sigma^4}.\end{aligned}$$

Evaluated at the mode, these second derivatives are (and substituting $(n-1)S^2 = (n+2)\hat{\sigma}^2$)

$$\begin{aligned}\left. \frac{\partial^2 p(\mu, \sigma^2|y)}{(\partial \mu)^2} \right|_{\mu=\hat{\mu}, \sigma^2=\hat{\sigma}^2} &= -\frac{n}{\hat{\sigma}^2}, \\ \left. \frac{\partial^2 p(\mu, \sigma^2|y)}{(\partial \sigma^2)^2} \right|_{\mu=\hat{\mu}, \sigma^2=\hat{\sigma}^2} &= \frac{n+2}{2\hat{\sigma}^4} - \frac{n+2}{\hat{\sigma}^4} = -\frac{n+2}{2\hat{\sigma}^4}, \\ \left. \frac{\partial^2 p(\mu, \sigma^2|y)}{\partial \mu \partial \sigma^2} \right|_{\mu=\hat{\mu}, \sigma^2=\hat{\sigma}^2} &= 0.\end{aligned}$$

Therefore the asymptotic approximation to $[\mu, \sigma^2|y]$ is

$$[\mu, \sigma^2|y] \sim N \left(\begin{bmatrix} \bar{y} \\ \frac{\sum (y_i - \bar{y})^2}{n+2} \end{bmatrix}, \begin{bmatrix} \frac{\hat{\sigma}^2}{n} & 0 \\ 0 & \frac{2\hat{\sigma}^4}{n+2} \end{bmatrix} \right) \quad (13.68)$$

Another approximation based on the likelihood function is

$$[\theta|y] \sim N\left(\hat{\theta}, I(\theta)^{-1}\Big|_{\theta=\hat{\theta}}\right) \quad (13.69)$$

where $\hat{\theta}$ is the ML estimator and $I(\theta)$ is the observed information. I derive this approximation and compare it to (13.68). Ignoring the prior, write

$$\begin{aligned} p(\mu, \sigma^2|y) &\propto p(y|\mu, \sigma^2) \\ &\propto \exp\left[-\frac{\sum_{i=1}^n (y_i - \mu)^2}{2\sigma^2}\right] (\sigma^2)^{-\left(\frac{n}{2}\right)} \\ &= \exp\left[-\frac{(n-1)S^2 + n(\bar{y} - \mu)^2}{2\sigma^2}\right] (\sigma^2)^{-\left(\frac{n}{2}\right)}, \end{aligned}$$

and the logposterior is

$$\ln p(\mu, \sigma^2|y) = -\left(\frac{n}{2}\right) \ln \sigma^2 - \frac{1}{2\sigma^2} \left[(n-1)S^2 + n(\bar{y} - \mu)^2\right].$$

First derivatives are

$$\begin{aligned} \frac{\partial \ln p(\mu, \sigma^2|y)}{\partial \mu} &= \frac{n(\bar{y} - \mu)}{\sigma^2}, \\ \frac{\partial \ln p(\mu, \sigma^2|y)}{\partial \sigma^2} &= -\frac{n}{2\sigma^2} + \frac{(n-1)S^2 + n(\bar{y} - \mu)^2}{2\sigma^4}. \end{aligned}$$

Setting these equations equal to zero and solving for μ and σ^2 gives

$$\begin{bmatrix} \hat{\mu} \\ \hat{\sigma}^2 \end{bmatrix} = \begin{bmatrix} \bar{y} \\ \frac{n-1}{n} S^2 \end{bmatrix} = \begin{bmatrix} \bar{y} \\ \frac{\sum (y_i - \bar{y})^2}{n} \end{bmatrix}.$$

Second derivatives are

$$\begin{aligned} \frac{\partial^2 p(\mu, \sigma^2|y)}{(\partial \mu)^2} &= -\frac{n}{\sigma^2}, \\ \frac{\partial^2 p(\mu, \sigma^2|y)}{(\partial \sigma^2)^2} &= \frac{n}{2\sigma^4} - \frac{(n-1)S^2 + n(\bar{y} - \mu)^2}{\sigma^6}, \\ \frac{\partial^2 p(\mu, \sigma^2|y)}{\partial \mu \partial \sigma^2} &= -\frac{n(\bar{y} - \mu)}{\sigma^4}. \end{aligned}$$

Evaluated at the mode, these second derivatives are (substituting $(n - 1) S^2 = n\hat{\sigma}^2$) are

$$\begin{aligned}\frac{\partial^2 p(\mu, \sigma^2 | y)}{(\partial \mu)^2} \bigg|_{\mu=\hat{\mu}, \sigma^2=\hat{\sigma}^2} &= -\frac{n}{\hat{\sigma}^2}, \\ \frac{\partial^2 p(\mu, \sigma^2 | y)}{(\partial \sigma^2)^2} \bigg|_{\mu=\hat{\mu}, \sigma^2=\hat{\sigma}^2} &= \frac{n}{2\hat{\sigma}^4} - \frac{n}{\hat{\sigma}^4} = -\frac{n}{2\hat{\sigma}^4}, \\ \frac{\partial^2 p(\mu, \sigma^2 | y)}{\partial \mu \partial \sigma^2} \bigg|_{\mu=\hat{\mu}, \sigma^2=\hat{\sigma}^2} &= 0.\end{aligned}$$

Therefore the asymptotic approximation to $[\mu, \sigma^2 | y]$ is now

$$[\mu, \sigma^2 | y] \sim N \left(\begin{bmatrix} \bar{y} \\ \frac{\sum (y_i - \bar{y})^2}{n} \end{bmatrix}, \begin{bmatrix} \frac{\hat{\sigma}^2}{n} & 0 \\ 0 & \frac{2\hat{\sigma}^4}{n} \end{bmatrix} \right) \quad (13.70)$$

As n increases, (13.68) and (13.70) approach the same value.

- (v) The approximate 95% posterior intervals for μ and σ^2 based on (13.68) can be obtained using the R functions

```
qnorm(c(0.025, 0.975), 48.69, sqrt(6.644/10))
```

```
## [1] 47.09242 50.28758
```

```
qnorm(c(0.025, 0.975), 6.644, sqrt(7.357))
```

```
## [1] 1.327835 11.960165
```

that yield

$$\Pr(47.09 < \mu < 50.28 | y) = 0.95$$

$$\Pr(1.33 < \sigma^2 < 11.96 | y) = 0.95.$$

The approximate 95% posterior intervals for μ and σ^2 based on (13.70) are obtained using the R functions

```
qnorm(c(0.025, 0.975), 48.69, sqrt(7.973/10))
```

```
## [1] 46.93992 50.44008
```

```
qnorm(c(0.025,0.975),7.973,sqrt(12.714))
```

```
## [1] 0.9844159 14.9615841
```

that yield

$$\Pr(46.94 < \mu < 50.44|y) = 0.95,$$

$$\Pr(0.98 < \sigma^2 < 14.96|y) = 0.95.$$

Comparison with the exact results (13.63) and (13.64), shows that the interval for μ is well approximated by the normal distribution but this is not the case for σ^2 . Clearly a sample size of $n = 10$ is not big enough for the normal approximation to hold in the case of σ^2 .

Exercise 2

- (i) The R-code below implements the Bayesian model with the Metropolis-Hastings single-site updating algorithm.

```
# CODE1306
#METROPOLIS-HASTINGS SINGLE-SITE UPDATING - USES ACF
rm(list=ls()) # Clear the workspace
set.seed(123456)
require(graphics)
y<-c(45.83,50.37,50.06,51.59,48.43,52.75,42.92,48.57,
     46.18,50.20)
# SET LENGTH OF CHAIN rep
rep<-10000
result<-matrix(data=NA,nrow=rep,ncol=3)

mu<- 15
v<- 10
#CHOOSE TUNING PARAMETERS kmu AND kv
#kmu<-0.19
#kv<-0.1
kmu<-19
kv<-9
acceptv<-0
acceptmu<-0
ptm <- proc.time()
for (i in 1:rep)
{
  #UPDATING THE VARIANCE
  logYv<-rnorm(1,mean=log(v),sd=sqrt(kv))
  logalfav <-sum((y-mu)^2)/(2*v)-sum((y-mu)^2)/
```

```

      (2*exp(logYv))+(length(y)/2)*(log(v)-logYv)
unif<-runif(1)
if (unif<exp(logalfav))
{
  v<-exp(logYv)
  acceptv<-acceptv+1
}
#UPDATING THE MEAN
Ymu<-rnorm(1,mean=mu,sd=sqrt(kmu))
logalfamu<- sum((y-mu)^2)/(2*v)-sum((y-Ymu)^2)/(2*v)
unif<-runif(1)
if (unif<exp(logalfamu))
{
  mu<-Ymu
  acceptmu<-acceptmu+1
  result[i, ]<-c(i,mu,v)
}
else {
  result[i, ]<-c(i,mu,v)
}
}
proc.time()-ptm

```

```

##      user  system elapsed
##      0.15    0.00    0.19

```

```

acceptrationmu<-acceptmu/rep
acceptratiov<-acceptv/rep
# DISCARD THE FIRST 1500 DRAWS
mu<-result[1501:rep,2]
v<-result[1501:rep,3]
meanmus<-mean(mu)
meanmus

```

```
## [1] 48.67492
```

```

varmus<-var(mu)
varmus

```

```
## [1] 1.315822
```

```

cimus<-quantile(mu,c(0.025,0.975))
cimus

```

```

##      2.5%    97.5%
## 46.35049 50.93522

```

```
meanvs<-mean(v)
meanvs
```

```
## [1] 12.15146
```

```
varvs<- var(v)
varvs
```

```
## [1] 66.8197
```

```
civs<-quantile(v,c(0.025,0.975))
civs
```

```
##      2.5%      97.5%
##  4.29519 32.92001
```

The single-site updating algorithm generates the following output. The chain length is 10,000, and the first 1500 samples are discarded.

$$\begin{aligned}\hat{E}(\mu|y) &= 48.67 \\ \widehat{\text{Var}}(\mu|y) &= 1.32 \\ \hat{\text{Pr}}(46.35 < \mu < 50.94|y) &= 0.95 \\ \hat{E}(\sigma^2|y) &= 12.15 \\ \widehat{\text{Var}}(\sigma^2|y) &= 66.82 \\ \hat{\text{Pr}}(4.3 < \sigma^2 < 32.92|y) &= 0.95.\end{aligned}$$

- (ii) The R-code below implements the Bayesian model with the Metropolis-Hastings joint updating algorithm.

```
# CODE1307
#METROPOLIS-HASTINGS JOINT UPDATING
rm(list=ls()) # Clear the workspace
set.seed(123456)
require(graphics)
y<-c(45.83,50.37,50.06,51.59,48.43,52.75,42.92,48.57,
     46.18,50.20)
# SET LENGTH OF CHAIN rep
rep<-10000
result<-matrix(data=NA,nrow=rep,ncol=3)
#INITIALISE mu AND v
mu<-15
v<-10
```

```
#CHOOSE TUNING PARAMETERS kmu AND kv
kmu<-0.15
kv<-0.08
accept<-0
ptm <- proc.time()
for (i in 1:rep)
{
  Ymu<-rnorm(1,mean=mu,sd=sqrt(kmu))
  logYv<-rnorm(1,mean=log(v),sd=sqrt(kv))
  logalfa <-sum((y-mu)^2)/(2*v)-sum((y-Ymu)^2)/
    (2*exp(logYv))+(length(y)/2)*(log(v)-logYv)
  unif<-runif(1)
  if (unif<exp(logalfa))
  {
    mu<-Ymu
    v<-exp(logYv)
    result[i, ]<-c(i,mu,v)
    accept<-accept+1
  }
  else
  {
    result[i, ]<-c(i,mu,v)
  }
}
proc.time()-ptm
```

```
##      user  system elapsed
##      0.13    0.00    0.13
```

```
acceptraio<-accept/rep
mu<-result[1501:rep,2]
v<-result[1501:rep,3]
meanmuj<-mean(mu)
meanmuj
```

```
## [1] 48.62203
```

```
varmuj<-var(mu)
varmuj
```

```
## [1] 1.175362
```

```
cimuj<-quantile(mu,c(0.025,0.975))
cimuj
```

```
##      2.5%    97.5%
## 46.34522 50.69246
```



```
meanvj<-mean(v)
meanvj
```

```
## [1] 11.91832
```

```
varvj<- var(v)
varvj
```

```
## [1] 55.73139
```

```
civj<-quantile(v,c(0.025,0.975))
civj
```

```
##      2.5%      97.5%
##  4.242654 30.864260
```

The joint updating algorithm produces the following output. The chain length is 10,000, and the first 1500 samples are discarded.

$$\begin{aligned}\hat{E}(\mu|y) &= 48.62 \\ \widehat{\text{Var}}(\mu|y) &= 1.18 \\ \hat{\Pr}(46.35 < \mu < 50.69|y) &= 0.95 \\ \hat{E}(\sigma^2|y) &= 11.92 \\ \widehat{\text{Var}}(\sigma^2|y) &= 55.73 \\ \hat{\Pr}(4.24 < \sigma^2 < 30.86|y) &= 0.95.\end{aligned}$$

A glance at these figures confirms that they are in good agreement with the exact results.

(iii) For $k_\mu = 0.19$, $k_{\sigma^2} = 0.1$ the following results are obtained:

- (a) acceptance ratios for μ and σ^2 are 0.87 and 0.47, respectively;
- (b) The MC variances for μ and σ^2 are 0.0095 and 0.63 respectively;
- (c) effective chain length for μ and σ^2 are 160 and 194, respectively;
- (d) integrated autocorrelations for μ and σ^2 are 53 and 43, respectively.

For $k_\mu = 19$, $k_{\sigma^2} = 9$ the following results are obtained:

- (a) acceptance ratios for μ and σ^2 are 0.29 and 0.19, respectively;
- (b) The MC variances for μ and σ^2 are 0.00097 and 0.093, respectively;
- (c) effective chain length for μ and σ^2 are 1357 and 720, respectively;
- (d) integrated autocorrelations for μ and σ^2 are 6.3 and 11.8, respectively.

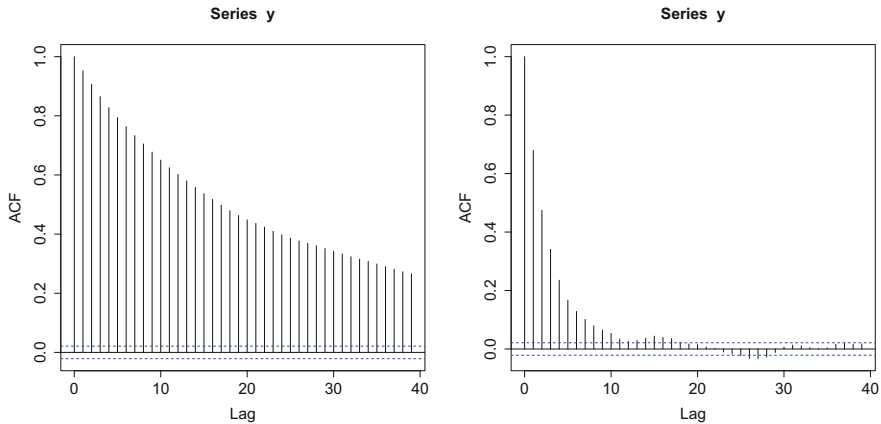


Fig. 13.4 Plots of autocorrelations versus lag of draws from $[\mu|y]$ using $k_\mu = 0.19$, $k_{\sigma^2} = 0.1$ (left panel) and $k_\mu = 19$, $k_{\sigma^2} = 9$ (right panel)

When the tuning parameters (variances) are very small, the proposed values differ very little from the previous values and are accepted with high probability; this generates a highly correlated chain. When the tuning parameters are larger, proposed values differ, on average, relatively more from previous values, are accepted with smaller probability and the degree of autocorrelations is relatively smaller. This is also illustrated in Fig. 13.4 that displays autocorrelations versus lag. The left panel corresponds to $k_\mu = 0.19$, $k_{\sigma^2} = 0.1$ that generates high autocorrelations decreasing at a low rate with lag. The right panel corresponds to $k_\mu = 19$, $k_{\sigma^2} = 9$; the autocorrelations fall at a higher rate.

Exercise 3

- (i) The R- code below implements the Bayesian model with the single-site Gibbs sampling algorithm.

```
# CODE1308
#GIBBS SAMPLING ALGORITHM
rm(list=ls()) # Clear the workspace
set.seed(12345)
require(graphics)
dat<-c(45.83,50.37,50.06,51.59,48.43,52.75,42.92,48.57,
       46.18,50.20)
# SET LENGTH OF CHAIN rep
rep<-10000
result<-matrix(data=NA,nrow=rep,ncol=3)
#INITIALISE mu AND v
#mu<-mean(dat)
```

```

#v<-var(dat)
mu<-1
v<-2
n<-length(dat)
# START GIBBS CHAIN
ptm <- proc.time()
for (i in 1:rep)
{
  # GENERATE MIU
  mu<-rnorm(1,mean=mean(dat),sd=sqrt(v/n))
  # COMPUTE SCALE
  s<-((n-1)*var(dat)+n*(mean(dat)-mu)^2)/n
  # GENERATE V
  v<-n*s/rchisq(1,n)
  result[i,]<-c(i,mu,v)
}
proc.time()-ptm

```

```

##      user  system elapsed
##    0.45    0.00    0.46

```

```

# END OF GIBBS CHAIN
#mu<-result[1000:rep,2]
mu<-result[,2]

#v<-result[1000:rep,3]
v<-result[,3]

meanmu<-mean(mu)
meanmu

```

```

## [1] 48.70177

```

```

varmu<-var(mu)
varmu

```

```

## [1] 1.132319

```

```

cimu<-quantile(mu,c(0.025,0.975))
cimu

```

```

##      2.5%    97.5%
## 46.59150 50.84902

```

```

meanv<-mean(v)
meanv

```

```
## [1] 11.35799
```

```
varv<- var(v)
varv
```

```
## [1] 48.91507
```

```
civ<-quantile(v,c(0.025,0.975))
civ
```

```
##      2.5%      97.5%
## 4.184609 29.596819
```

The Gibbs sampling algorithm yields the following estimates

$$\hat{E}(\mu|y) = 48.7$$

$$\widehat{\text{Var}}(\mu|y) = 1.13$$

$$\hat{E}(\sigma^2|y) = 11.36$$

$$\widehat{\text{Var}}(\sigma^2|y) = 48.92$$

$$\hat{\text{Pr}}(46.59 < \mu < 50.85|y) = 0.95$$

$$\hat{\text{Pr}}(4.18 < \sigma^2 < 29.6|y) = 0.95,$$

in good agreement with those obtained with the Metropolis-Hastings algorithms. The Monte Carlo variances are

$$\widehat{\text{Var}}[\hat{E}(\mu|y)] = 0.0001,$$

$$\widehat{\text{Var}}[\hat{E}(\sigma^2|y)] = 0.0070.$$

(ii) The effective chain lengths for μ and σ^2 are 9931 and 7512, respectively.

The integrated autocorrelations for μ and σ^2 are 1.0 and 1.33, respectively.

The R-code that computes these quantities is shown below.

```
# CODE1308 (cont)
#POST-McMC ANALYSIS
# CODE FOR THE MC VARIANCE BASED ON GEYER

# CHOOSE MU OR V AND PLACE IN VECTOR Y
y<-result[,2] # READS IN mu
#y <- result[,3] # READS IN v
ns <- length(y)
svar<-var(y)*(ns-1)/ns
```

```

tau<-1
tausum<-0
for(i in 0:ns)
{
  gamaj<-0.0
  gamak<-0.0
  j<-2*i
  k<-(2*i)+1
  lag1<-j
  lag2<-k
  #USE THE R-FUNCTION ACF TO COMPUTE AUTOCORRELATIONS
  cov1<-acf(y,type="covariance",lag.max=lag1,plot=FALSE)
  cov2<-acf(y,type="covariance",lag.max=lag2,plot=FALSE)
  gamaj<-cov1$acf[lag1+1]
  gamak<-cov2$acf[lag2+1]
  tau<-gamaj+gamak
  if(tau<0)
  {
    break
  }
  tausum<-tausum+tau
}

varch<- -svar+2*tausum
mcvar<-varch/ns
mcvar

```

```
## [1] 0.0001140094
```

```

efchsize<-svar/mcvar
efchsize

```

```
## [1] 9930.804
```

```

integrautoc<-varch/svar
integrautoc

```

```
## [1] 1.006968
```

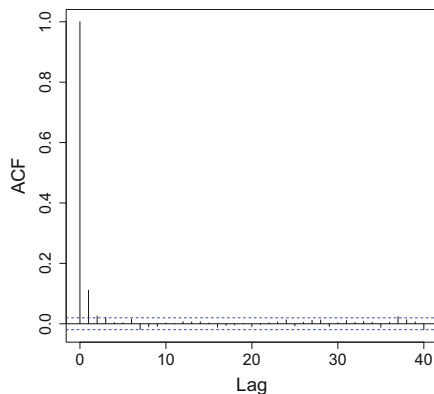
For this toy model and data set, the draws from the MCMC chain generated by the Gibbs sampler have very small autocorrelation. This is illustrated in Fig. 13.5.

13.4 Bayes Exercises II

Exercise 1

- (i) The R-code below fits the Bayesian logistic regression model implemented with a Metropolis-Hastings algorithm with joint updating of (μ, β) .

Fig. 13.5 Plots of autocorrelations versus lag of draws from $[\sigma^2|y]$ using the Gibbs sampler



```
# CODE1309
#BINARY DATA - M-H JOINT UPDATING - LOGISTIC MODEL
rm(list=ls()) # Clear the workspace
set.seed(123)
#require(graphics)
# THE CODE WILL USE THE PACKAGE MVTNORM; IT IS INSTALLED BELOW
#install.packages("mvtnorm", .libPaths()[1])
library(mvtnorm)
#CHOOSE LENGTH OF CHAIN rep
rep<-10000
result<-matrix(data=NA,nrow=rep,ncol=4)
#####
# CREATE BINARY DATA
mu <- -2
beta <- 0.7
nrec <- 30
cov <- rnorm(nrec,2,3) # GENERATE THE COVARIATE
xb <- cov*beta
p1 <- pnorm(mu+xb) # PROBABILITIES ACCORDING TO PROBIT MODEL
# CREATE DATA:
d <- data.frame(Y=rbinom(nrec,1,p1),X=round(cov,digits=0))
mean(d$Y)
```

```
## [1] 0.3
```

```
# CHOOSE TUNING PARAMETER LAMBDA AND COVARIANCE MATRIX C
lambda<-1
c<-matrix(c(1,0.0,0.0,0.1),nrow=2,ncol=2,byrow=T)

# INITIALISE THE MEAN OF THE BIVARIATE DISTRIBUTION
theta<-c(-2,1)
accept<-0
## FUNCTION TO COMPUTE THE LOG-POSTERIOR
```

```

logpost <- function(data,par)
{
  theta[1] <-par[1]
  theta[2] <- par[2]
  with(data=d,sum(Y*(theta[1]+theta[2]*X)-log(1+exp(theta[1]+
                                                    theta[2]*X))))
}
#START MH LOOP
ptm <- proc.time()
for (i in 1:rep)
{
  #SAMPLE PROPOSAL FOR THETA (Ytheta) FROM N(theta,lamdaC)
  Ytheta<- rmvnorm(1,mean=theta,sigma=lambda*c)
  logalfa<-logpost(d,c(Ytheta[1],Ytheta[2])) -
    logpost(d,c(theta[1],theta[2]))
  unif<-runif(1)
  if (unif<exp(logalfa))
  {
    theta[1]<-Ytheta[1]
    theta[2]<-Ytheta[2]
    result[i, ]<-c(i,theta[1],theta[2],exp(theta[1]+3*theta[2])/
      (1+exp(theta[1]+3*theta[2])))
    accept<-accept+1
  }
  else
  {
    result[i, ]<-c(i,theta[1],theta[2],exp(theta[1]+3*theta[2])/
      (1+exp(theta[1]+3*theta[2])))
  }
}
proc.time()-ptm

```

```

##      user  system elapsed
##      2.56    0.03    2.59

```

```

acceptrato<-accept/rep
acceptrato

```

```
## [1] 0.4313
```

```

meanmu<-mean(result[,2])
meanmu

```

```
## [1] -3.996897
```

```

varmu<-var(result[,2])
varmu

```

```
## [1] 1.986835
```

```
cimu<-quantile(result[,2],c(0.025,0.975))
cimu
```

```
##      2.5%      97.5%
## -7.311877 -1.717778
```

```
meanbeta<-mean(result[,3])
meanbeta
```

```
## [1] 1.168351
```

```
varbeta<- var(result[,3])
varbeta
```

```
## [1] 0.1899689
```

```
cibeta<-quantile(result[,3],c(0.025,0.975))
cibeta
```

```
##      2.5%      97.5%
## 0.4968853 2.1923000
```

```
meanprob <- mean(result[,4])
meanprob
```

```
## [1] 0.3901256
```

```
ciprob <- quantile(result[,4],c(0.025,0.975))
ciprob
```

```
##      2.5%      97.5%
## 0.1421850 0.6921409
```

- (ii) The following features of the posterior distribution are obtained from the output:

$$\begin{aligned}\hat{E}(\mu|y) &= -3.997, \\ \hat{E}(\beta|y) &= 1.168, \\ \widehat{\text{Var}}(\mu|y) &= 1.987, \\ \widehat{\text{Var}}(\beta|y) &= 0.19, \\ \hat{\text{Pr}}(-7.312 < \mu < -1.718|y) &= 0.95,\end{aligned}$$

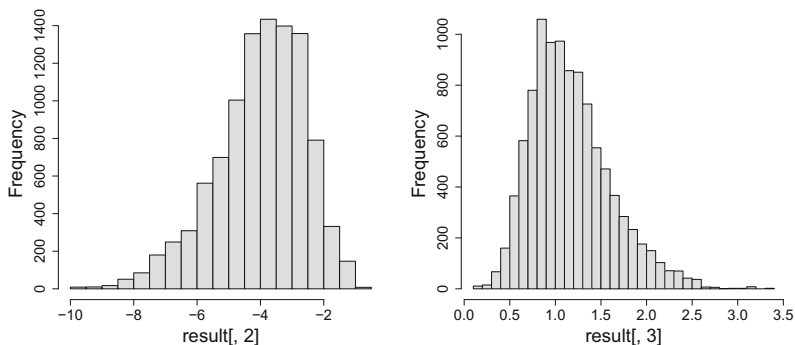


Fig. 13.6 Histograms of MC estimates of $[\mu|y]$ (left panel) and of $[\beta|y]$ (right panel)

$$\hat{\Pr}(0.497 < \beta < 2.192|y) = 0.95,$$

$$\overline{\hat{\Pr}}(Y = 1|x = 3, y) = 0.39,$$

where in the last line, $\overline{\hat{\Pr}}(Y = 1|x = 3, y)$ is the mean of the McMC draws from $\Pr(Y = 1|x = 3, y)$. The McMC estimate of the 95% posterior interval of the distribution of $\Pr(Y = 1|x = 3, y)$ is (0.142; 0.692). Given x , this probability is only a function of (μ, β) and the confidence interval reflects the posterior uncertainty associated with (μ, β) .

Figure 13.6 displays histograms of MC estimates of the marginal posterior distributions $[\mu|y]$ (left panel) and of $[\beta|y]$ (right panel). Both histograms reveal considerable asymmetry.

It is interesting to compare the output from the Bayesian implementation with that derived from a maximum likelihood analysis. The R-function below fits the OPTIM function

```
# CODE1309 (cont)
### USE THE R-FUNCTION OPTIM TO COMPARE WITH THIS PROGRAMME
## FUNCTION TO COMPUTE THE NEGATIVE OF THE LOG-POSTERIOR
logpostoptim <- function(data,par)
{
  theta[1] <-par[1]
  theta[2] <- par[2]
  with(data=d,-sum(X*(theta[1]+theta[2]*X)-log(1+exp(theta[1]+
                                                         theta[2]*X))))
}
result1 <- optim(par=c(-3,1),logpostoptim,
                 data=d,hessian=TRUE,method="BFGS")
result1$par
```

```
## [1] -3.2918405 0.9354898
```

```
solve(result1$hessian)
```

```
##           [,1]      [,2]
## [1,]  1.3069454 -0.3367133
## [2,] -0.3367133  0.1197200
```

The ML estimates of uncertainty are based on asymptotic results that do not quite hold in the small sample setting of this example. There is considerable difference between the uncertainty reported by both methods of inference.

The ML estimates of μ and β are -3.292 and 0.935 , respectively. These differ from the McMC estimates of posterior means due, in part, to the asymmetry referred to above that results in different values for mean and mode. Further, the likelihood approach relies on a joint maximisation of the loglikelihood function, whereas marginal inferences are reported with the Bayesian method. One can bring both approaches a little closer by computing the modal value of the estimates of the marginal posterior distributions $[\mu|y]$ and $[\beta|y]$. The R-code below uses the function `modeest` and calculates these modal values using a kernel estimator.

```
# CODE1309 (cont)
# computing MODE with package modeest
# and via density estimation
#install.packages("modeest", .libPaths()[1])
y<- result[,2]
x<-result[,3]
library(modeest)
myDensity<-density(y)
modeEstmu <- mlv(y,method = "kernel", kernel = "gaussian")
modeEstmu
```

```
## [1] -3.498042
```

```
modeEstbeta <- mlv(x,method = "kernel", kernel = "gaussian")
modeEstbeta
```

```
## [1] 0.9147319
```

The estimate of the posterior mode of $[\mu|y]$ is seen to be -3.498 and of $[\beta|y]$, 0.915 , more in line with the results reported from the likelihood approach.

(iii) The R-code below calculates MC variances, effective chain lengths and integrated autocorrelations.

```
# CODE1309 (cont)
#POST-McMC ANALYSIS
# CODE FOR THE MC VARIANCE BASED ON GEYER
ns<-rep
# CHOOSE MU OR V AND PLACE IN VECTOR Y
y<- result[,2]
```

```

#y<-result[,3]
svar<-var(y)*(ns-1)/ns
tau<-1
tausum<-0
for(i in 0:ns)
{
  gamaj<-0.0
  gamak<-0.0
  j<-2*i
  k<-(2*i)+1
  lag1<-j
  lag2<-k
  #USE THE R-FUNCTION ACF TO COMPUTE AUTOCORRELATIONS
  cov1<-acf(y,type="covariance",lag.max=lag1,plot=FALSE)
  cov2<-acf(y,type="covariance",lag.max=lag2,plot=FALSE)
  gamaj<-cov1$acf[lag1+1]
  gamak<-cov2$acf[lag2+1]
  tau<-gamaj+gamak
  if(tau<0)
  {
    break
  }
  tausum<-tausum+tau
}
varch<- svar+2*tausum
mcvar<-varch/ns
mcvar

```

```
## [1] 0.009202506
```

```

efchsize<-svar/mcvar
efchsize

```

```
## [1] 215.88
```

```

integrautoc<-varch/svar
integrautoc

```

```
## [1] 46.32203
```

The MC variance of the estimated mean of $[\mu|y]$, the effective chain length and integrated autocorrelation of the draws from $[\mu|y]$ obtained from the output are 0.0092, 215.88 and 46.32, respectively. For $[\beta|y]$ the respective values are 0.00088, 213.57 and 46.82, respectively. These figures indicate that the MCMC samples display a relatively high degree of autocorrelation.

Exercise 2

- (i) The R-code below fits the probit regression model with the Metropolis-Hastings joint updating algorithm.

```
# CODE1310
#BINARY DATA - METROPOLIS-HASTINGS JOINT UPDATING - PROBIT MODEL
rm(list=ls()) # Clear the workspace
set.seed(123)
#require(graphics)
# CODE USES PACKAGE MVTNORM; IT IS INSTALLED BELOW
#install.packages("mvtnorm", .libPaths()[1])
library(mvtnorm)
#CHOOSE LENGTH OF CHAIN rep
rep<-10000
result<-matrix(data=NA,nrow=rep,ncol=4)
#####
# CREATE BINARY DATA BASED ON PROBIT MODEL
mu <- -2
beta <- 0.7
nrec <- 30
cov <- rnorm(nrec,2,3) # GENERATE THE COVARIATE
xb <- cov*beta
p1 <- pnorm(mu+xb) # PROBABILITIES ACCORDING TO PROBIT MODEL
# CREATE DATA:
d <- data.frame(Y=rbinom(nrec,1,p1),X=round(cov,digits=0))
mean(d$Y)
```

```
## [1] 0.3
```

```
#####
# CHOOSE TUNING PARAMETER LAMBDA AND COVARIANCE MATRIX C
lambda<-0.25
c<-matrix(c(1,0.0,0.0,0.1),nrow=2,ncol=2,byrow=T)

# INITIALISE THE MEAN OF THE BIVARIATE DISTRIBUTION
theta<-c(-2,1)

accept<-0
## FUNCTION TO COMPUTE THE LOG-POSTERIOR
logpost <- function(data,par)
{
  theta[1] <-par[1]
  theta[2] <- par[2]
  with(data=d,sum(Y*log(pnorm(theta[1]+theta[2]*X))+
    (1-Y)*log(1.000001-pnorm(theta[1]+theta[2]*X))))
}
#START MH LOOP
ptm <- proc.time()
for (i in 1:rep)
{
```

```

#SAMPLE PROPOSAL FOR THETA (Ytheta) FROM N(theta, lamdaC)
Ytheta<- rmvnorm(1,mean=theta,sigma=lambda*c)
lp1<- logpost(d,c(Ytheta[1],Ytheta[2]))
lp2<- logpost(d,c(theta[1],theta[2]))
logalfa<-lp1-lp2
unif<-runif(1)
if (unif<exp(logalfa))
{
  theta[1]<-Ytheta[1]
  theta[2]<-Ytheta[2]
  result[i, ]<-c(i,theta[1],theta[2],
    pnorm(theta[1]+3*theta[2]))
  accept<-accept+1
}
else
{
  result[i, ]<-c(i,theta[1],theta[2],
    pnorm(theta[1]+3*theta[2]))
}
}
proc.time()-ptm

```

```

##      user  system elapsed
##      2.73    0.06    2.80

```

```

acceptrato<-accept/rep
meanmu<-mean(result[,2])
meanmu

```

```
## [1] -2.230753
```

```

varmu<-var(result[,2])
varmu

```

```
## [1] 0.4690703
```

```

cimu<-quantile(result[,2],c(0.025,0.975))
cimu

```

```

##      2.5%      97.5%
## -3.696849 -1.062495

```

```

meanbeta<-mean(result[,3])
meanbeta

```

```
## [1] 0.6450377
```

```
varbeta<- var(result[,3])
varbeta
```

```
## [1] 0.04395467
```

```
cibeta<-quantile(result[,3],c(0.025,0.975))
cibeta
```

```
##      2.5%      97.5%
## 0.2879808 1.0886044
```

```
meanprob <- mean(result[,4])
meanprob
```

```
## [1] 0.3900007
```

```
ciprob <- quantile(result[,4],c(0.025,0.975))
ciprob
```

```
##      2.5%      97.5%
## 0.1653514 0.6497098
```

- (ii) The following features of the posterior distribution are obtained from the output:

$$\begin{aligned}\hat{E}(\mu|y) &= -2.231, \\ \hat{E}(\beta|y) &= 0.645, \\ \widehat{\text{Var}}(\mu|y) &= 0.469, \\ \widehat{\text{Var}}(\beta|y) &= 0.044, \\ \hat{\text{Pr}}(-3.697 < \mu < -1.062|y) &= 0.95, \\ \hat{\text{Pr}}(0.288 < \beta < 1.089|y) &= 0.95, \\ \widehat{\overline{\text{Pr}}}(Y = 1|x = 3, y) &= 0.39,\end{aligned}$$

where in the last line, $\widehat{\overline{\text{Pr}}}(Y = 1|x = 3, y)$ is the mean of the MCMC draws from $\text{Pr}(Y = 1|x = 3, y)$. The MCMC estimate of the 95% posterior interval of the distribution of $\text{Pr}(Y = 1|x = 3, y)$ is (0.165; 0.65). Given x this probability is only a function of (μ, β) and the confidence interval reflects the posterior uncertainty associated with (μ, β) . Despite the difference in estimates of the parameters between the logit and the probit models, the estimates of the posterior probability $\text{Pr}(Y = 1|x = 3, y)$ are in good agreement.

- (iii) The MC variance of the mean of $[\mu|y]$, the effective chain length and integrated autocorrelation of the draws from $[\mu|y]$ are 0.0018, 264.0 and 37.9, respectively. For $[\beta|y]$ these figures are 0.00017, 264.92 and 37.75, respectively.

Exercise 3

- (i) The R-code below generates the same 30 binary records used in the previous examples and fits a probit model implemented with a single-site updating Gibbs sampling algorithm.

```
# CODE1311
# BINARY DATA; GIBBS WITH DATA AUGMENTATION PROBIT MODEL
# LIABILITIES SAMPLED IN ONE GO AFTER JA pg 241
rm(list=ls()) # Clear the workspace
set.seed(123)
#require(graphics)
# THE CODE WILL USE THE PACKAGE MVTNORM; IT IS INSTALLED BELOW
#install.packages("mvtnorm", .libPaths()[1])
library(mvtnorm)
#CHOOSE LENGTH OF CHAIN rep
rep<-10000
result<-matrix(data=NA,nrow=rep,ncol=3)
#####
# CREATE BINARY DATA
mu <- -2
beta <- 0.7
nrec <- 30
cov <- rnorm(nrec,2,3) # GENERATE THE COVARIATE
xb <- cov*beta
p1 <- pnorm(mu+xb) # COMPUTE PROBABILITIES FOR PROBIT MODEL
#p1 <- rbeta(30,2,2)
# CREATE DATA:
d <- data.frame(Y=rbinom(nrec,1,p1),X=round(cov,digits=0))
mean(d$Y)
```

```
## [1] 0.3
```

```
#####
One<-rep(1,nrec)
X<-matrix(c(One,d[,2]),nrow=nrec,ncol=2)
Y<-matrix(d[,1],nrow=length(d[,1]),ncol=1)
#INITIALISE THETA (THE VECTOR WITH MU AND BETA)
theta<-matrix(data=0,nrow=2,ncol=1)
# INITIALISE VECTOR u
u<-rep(0,nrec)
#COMPUTE XTX INVERSE
```

```

xtxinv<-solve(t(X)%*%X)
#START GIBBS LOOP
ptm <- proc.time()
for (i in 1:rep)
{
  thetahat<-xtxinv%*%t(X)%*%u
  #SAMPLE THETA FROM MVN(thetahat,xtxinv)
  theta<- t(rmvnorm(1,mean=thetahat,sigma= xtxinv))
  # SAMPLE LIABILITIES u_j FROM A TN(beta+Uf,1)

  av<-X%*%theta # MEAN OF UNTRUNCATED NORMAL(mu+U[j,]alfa,1)
  cutoff<-pnorm(-av)
  interm<-(cutoff*(1-Y)+(1-cutoff)*Y)*runif(length(Y))+cutoff*Y
  interm[interm==1]<-0.999
  u<-qnorm(interm)+av
  # END SAMPLING LIABILITIES u
  result[i,<-c(i,t(theta))
}
proc.time() - ptm

```

```

##      user  system elapsed
##    2.64    0.09    2.73

```

```

meanmu<-mean(result[,2])
meanmu

```

```
## [1] -2.177854
```

```

varmu<-var(result[,2])
varmu

```

```
## [1] 0.4747578
```

```

cimu<-quantile(result[,2],c(0.025,0.975))
cimu

```

```

##      2.5%    97.5%
## -3.728438 -1.014262

```

```

meanbeta<-mean(result[,3])
meanbeta

```

```
## [1] 0.6290715
```

```

varbeta<- var(result[,3])
varbeta

```



```
## [1] 0.04445811
```

```
cibeta<-quantile(result[,3],c(0.025,0.975))
cibeta
```

```
##          2.5%          97.5%
## 0.2846762 1.1175002
```

- (ii) The following features of the posterior distribution are obtained from the output:

$$\begin{aligned}\hat{E}(\mu|y) &= -2.178, \\ \hat{E}(\beta|y) &= 0.629, \\ \widehat{\text{Var}}(\mu|y) &= 0.475, \\ \widehat{\text{Var}}(\beta|y) &= 0.044, \\ \hat{\Pr}(-3.728 < \mu < -1.014|y) &= 0.95, \\ \hat{\Pr}(0.285 < \beta < 1.118|y) &= 0.95,\end{aligned}$$

that can be seen to be in good agreement with the output in Exercise 13.4.

- (iii) The MC variance of the estimated mean of $[\mu|y]$, the effective chain length and integrated autocorrelation of the draws from $[\mu|y]$ are 0.0014, 340.8 and 29.3, respectively. For $[\beta|y]$ these figures are 0.00013, 336.3 and 29.7, respectively. A comparison with Exercise 13.4 indicates that the Gibbs sampler generated slightly less correlated draws from $[\mu, \beta|y]$ than the Metropolis-Hastings algorithm, as implemented in this example. The latter can be tuned to improve its behaviour; this was not attempted here.

Exercise 4

- (i) The R-code below generates binary data from 500 full-sib families each with three full-sibs and fits a probit threshold model implemented via a Gibbs sampling algorithm using a single chain of length 2000.

```
# CODE1312
#SINGLE-SITE GIBBS - CORRELATED PROBIT MODEL
# DOES NOT USE THE SVD OF ZZ'
rm(list=ls()) # Clear the workspace
set.seed(7713)

require(graphics)
# GENERATE CORRELATED (FULL-SIBS) BINARY DATA (THRESHOLD MODEL)
```

```
#install.packages("MCMCpack", .libPaths()[1])
#install.packages("mvtnorm", .libPaths()[1])
library(mvtnorm)
library(MCMCpack)
# INITIALISE PARAMETERS
p0 <- 0.5
mu <- qnorm(p0)
iccfs<-0.15 #INTRACLASSE CORRELATION FS
# VARIANCE BETWEEN FAMILIES: iccfs /(1- iccfs)
iccfs /(1- iccfs)
```

```
## [1] 0.1764706
```

```
# PHENOTYPIC VARIANCE: 1/(1-iccfs)
nfs<-500 # NUMBER OF FULL-SIB FAMILIES
fs<-3 #FULL-SIB FAMILY SIZE
# SET DATA Y= 0
y<-matrix(data=0,nrow=fs*nfs,ncol=1)
# x IS COLUMN MATRIX WITH FAMILY ID (ID=1,..,nfs)
x<-matrix(data=0,nrow=fs*nfs,ncol=1)
# GENERATE NFS FULL-SIB EFFECTS f
f<-rnorm(nfs,mean=0,sd=sqrt(iccfs/(1-iccfs)))
# SET COUNTER c EQUAL TO ZERO
# (c = nfs*fs IS EQUAL TO THE LENGTH OF BINARY DATA VECTOR y)
c<-0
#####
#### GENERATE BINARY RECORDS Y
f<-rnorm(nfs,mean=0,sd=sqrt(iccfs/(1-iccfs)))
p <- pnorm(mu+f)
y <- rbinom(nfs*fs,1,rep(p,each=fs))
w <- rep(1:nfs,each=fs)
d<-data.frame(w,y)
family <- w
family <- as.factor(family)
Z<-model.matrix(~0+family)
# WITH INDEPENDENT FAMILIES Z'Z IS DIAGONAL
ztz<-rep(fs,nfs)
#CHOOSE LENGTH OF CHAIN rep
rep<-2000
result<-matrix(data=NA,nrow=rep,ncol=3)
# INITIALISE LIABILITY VECTOR u
u<-rep(0,fs*nfs)
#INITIALISE THE VECTOR OF FAMILIY EFFECTS fam
fam<-rep(0,nfs)
# INITIALISE BETWEEN FAMILY VARIANCE COMPONENT vf
vf<-0.2
# INITIALISE THE MEAN (HERE BETA, A SCALAR)
beta<-0
#START GIBBS LOOP
ptm<-proc.time()
for (i in 1:rep)
{
```

```

zfam <- Z%%fam
# SAMPLE BETA
betahat<-sum(u-zfam)/(fs*nfs)
beta<-rnorm(1,mean=betahat,sd=sqrt(1/(fs*nfs)))
# SAMPLE LIABILITIES u FROM A TN(beta+Zf,1)
av<-beta+ zfam # MEAN OF UNTRUNCATED NORMAL(mu+Zf,1)
prob<-pnorm(-av)
interm<-( prob *(1-y)+(1- prob)*y)*runif(fs*nfs)+ prob*y
interm[interm==1]<-0.999
u<-qnorm(intermed)+av
# END SAMPLING LIABILITIES u
# SAMPLE FAMILY EFFECTS fam
varfam<-1/(fs+(1/vf))
fammean<-varfam*(t(Z)%(u-beta))
fam<-rnorm(nfs,mean=fammean, sd=sqrt(varfam))
#SAMPLE vf
#COMPUTE SCALE
ftf<-sum(fam*fam)
vf<-ftf/rchisq(1,nfs-2)
result[i,]<-c(i,beta,vf)
}
proc.time()-ptm

```

```

##      user  system elapsed
##    38.89    1.46    11.68

```

```

# CONSTRUCT THE DERIVED PARAMETER "HERITABILITY" herit
herit <- 2*result[,3]/(result[,3]+1)
meanbeta <- mean(result[,2])
meanbeta

```

```
## [1] -0.01897901
```

```

meanvf <- mean(result[,3])
meanvf

```

```
## [1] 0.216644
```

```

meanher <- mean(herit)
meanher

```

```
## [1] 0.3512363
```

```

cibeta <- quantile(result[,2],c(0.025,0.975))
cibeta

```

```

##           2.5%           97.5%
## -0.09912916  0.05794092

```

```
civf <- quantile(result[,3],c(0.025,0.975))
civf
```

```
##          2.5%          97.5%
## 0.08737675 0.35402913
```

```
ciher <- quantile(herit,c(0.025,0.975))
ciher
```

```
##          2.5%          97.5%
## 0.1607111 0.5229269
```

(ii) The code outputs the following results:

$$\begin{aligned}\hat{E}(\mu|y) &= -0.02, \\ \hat{E}(\sigma_f^2|y) &= 0.22, \\ \hat{E}(h^2|y) &= 0.35, \\ \hat{\Pr}(-0.099 < \mu < 0.058|y) &= 0.95, \\ \hat{\Pr}(0.087 < \sigma_f^2 < 0.354|y) &= 0.95, \\ \hat{\Pr}(0.161 < h^2 < 0.523|y) &= 0.95.\end{aligned}$$

The parameter h^2 , the heritability on the underlying scale, is constructed from the output of the MCMC, using σ_f^2 , the variance component between families and the variance of the residual term of the liability, equal to 1: $h^2 = 2\sigma_f^2/(\sigma_f^2 + 1)$.

(iii) Execution of the R-code on page 702, generates the following results.

The MC variances of the estimate of the mean of $[\mu|y]$, the mean of $[\sigma_f^2|y]$ and the mean of $[h^2|y]$ are 3.03×10^{-6} , 1.36×10^{-4} and 2.67×10^{-4} , respectively.

The effective chain lengths of $[\mu|y]$, $[\sigma_f^2|y]$ and $[h^2|y]$ are 521, 32 and 30, respectively.

The integrated autocorrelations of $[\mu|y]$, $[\sigma_f^2|y]$ and $[h^2|y]$ are 3.8, 61.6 and 65.9, respectively.

Due to the considerable degree of autocorrelation among samples, particularly for $[\sigma_f^2|y]$ and $[h^2|y]$, a chain of length 2000 results in effective chain lengths equivalent to approximately 30 independent draws.

Exercise 5

- (i) The R-code below fits the genomic model using a single-site Gibbs sampling algorithm.

```
# CODE1313
# BAYES PROBLEMS II Exercise 5. GENOMIC MODEL
# IMPORTANT: ASSUMES NUMBER OF MARKERS > NUMBER INDIVIDUALS
# OPTIONAL CHOICE OF PROPER SCALED INVERSE CHI SQUARE PRIORS
# FOR VARIANCES
# DATA BASED ON GENOMIC MODEL; OBTAIN SVD OF WW'(1/m)
rm(list=ls()) # CLEAR WORKSPACE
set.seed(12345)
nindiv<-500
nmark<-1000
nt <- nindiv*nmark

# GENERATE MARKER MATRIX FROM BINOMIAL DISTRIBUTION
X<-matrix(nrow= nindiv,ncol= nmark,rbinom(n=nt,size=2,p=.5))
stdev <- matrix(data=NA,nrow= nmark,ncol=1)
W <- matrix(data=NA,nrow= nindiv,ncol=nmark)
U <- matrix(data=NA,nrow= nindiv,ncol= nindiv)
G<-matrix(data=NA,nrow= nindiv,ncol= nindiv)
cm <- colMeans(X)
#CHOOSE VALUE FOR GENOMIC VARIANCE vgs
vgs<-10
#CHOOSE VALUE FOR ENVIRONMENTAL VARIANCE ves
ves<-25

#####
### CODE r=1 IF VARIANCE COMPONENTS ASSIGNED IMPROPER UNIFORM
### PRIORS
### ELSE: VARIANCE COMPONENTS ASSIGNED SCALED INVERTED CHI
### SQUARE DISTRIBUTION
#####
r <- 1
if(r!=1){
  ## HYPERPARAMETERS OF THE SCALED INVERTED PRIORS FOR va AND ve
  nua <- 4.1
  nue <- 4.1
  Sa <- 10
  Se <- 30
  ## THIS GENERATES PRIOR MODES FOR va AND ve equal to
  ## 10*(4.1/6.1) = 6.7
}
#####

# CREATE MATRIX OF STANDARDISED MARKER GENOTYPE CODES
for (i in 1:nmark)
{
  W[,i] <- ( X[,i]-cm[i]) / sd(X[,i])
}
```

```

}
# GENERATE nindiv GENOMIC VAL FROM N(0,(1/nmark)WW'*vgs)
#NOTE: MARKER VALUES ARE DRAWS FROM N(0,I sqrt(vgs/nmark))
g <- (1/sqrt(nmark))*W**rnorm(nmark,mean=0,sd=sqrt(vgs))
# GENERATE nindiv PHENOTYPES WITH MEAN 0,
# VAR=vgs+ves, HERITABILITY=vgs/(vgs+ves)
e<- rnorm(nindiv,mean=0,sd=sqrt(ves))
y <- g+ e
# GENOMIC RELATIONSHIP MATRIX G
G <- (1/nmark)*W**%t(W)
# SVD OF G
EVD <- eigen(G)
names(EVD)

```

```
## [1] "values" "vectors"
```

```

#head(EVD$values)
U <- EVD$vector
tU<-t(U)
val <- EVD$values
val[length(y)] <-0
D <- diag(val,nrow=nindiv)
#Dp IS A VECTOR WITH NON-ZERO EIGENVALUES
Dp<-c(val[1:nindiv-1])
#INITIALISE Ve
Ve<-5
#INITIALISE Vg
Vg<-5
#INITIALISE k
k<-Ve/Vg
#INITIALISE VECTOR ALFA
alfa<-rep(0,nindiv)
# CHOOSE LENGTH OF GIBBS CHAIN
rep<-4000
#INITIALISE result
result<-matrix(data=NA,nrow=rep,ncol=7)
# START GIBBS CHAIN
ptm<-proc.time()

for (i in 1:rep)
{
  # print (i)
  # SAMPLE mu
  avmu<-sum(y-U**%alfa)/nindiv
  varmu<-Ve/nindiv
  mu<-rnorm(1,mean=avmu,sd=sqrt(varmu))
  #mu<-0
  # SAMPLE alfa1 (VECTOR OF LENGTH nindiv-1)
  meanalfa1<-(Dp/(Dp+k))*tU[1:nindiv-1]**%(y-mu)
  varalfa1<-((Dp)/(Dp+k))*Ve
  alfa1<-rnorm(nindiv-1,meanalfa1,sqrt(varalfa1))

```

```

alfa<-c(alfa1,0)
ystar<-y-mu-U**%alfa

# SAMPLE Vg and Ve
if(r==1) # IMPROPER UNIFORM PRIORS FOR Vg, Ve
{ scVg<-sum(alfa1*alfa1*(1/Dp))
  scVe<-sum(ystar*ystar)
  Vg<-scVg/rchisq(1,nindiv-3)
  Ve<-scVe/rchisq(1,nindiv-2)
}
if (r!=1) # SCALED INVERTED CHI SQUARE PRIORS FOR Vg,Ve
{ scVg<-sum(alfa1*alfa1*(1/Dp)) + nua*Sa
  scVe<-sum(ystar*ystar) + nue*Se
  Vg<-scVg/rchisq(1,nindiv-1+nua)
  Ve<-scVe/rchisq(1,nindiv + nue)
}

k<-Ve/Vg
result[i,<-c(i,mu,Vg,Ve,Vg/(Vg+Ve),1/k,mean(alfa*alfa))
# print(result[i,])
}
proc.time()-ptm

```

```

##      user  system elapsed
##    28.87    0.67    7.39

```

```

# FUNCTION LOGLIK TO CONSTRUCT THE LOGLIKELIHOOD
# TO COMPARE THE BAYESIAN RESULTS WITH OPTIM
#NOTE: k IN THE LOGLIKELIHOOD IS Vg/Ve
loglik<-function(data,par)
{
  mu<-par[1]
  ve<-par[2]
  k<-par[3]
  dkiinv<-diag(1/((val*k)+1),nrow=nindiv,ncol=nindiv)
  ll<- -0.5*(length(y)*log(ve)+sum(log((val*k)+1))+
        (1/ve)*t(y-mu)**U**dkiinv**tU*(y-mu))
  return(-ll)
}
result1<-optim(par=c(0.5,12,2),loglik,data=y,hessian=TRUE)
result1$par

```

```
## [1] -0.02250109 28.28775687 0.32911418
```

```

vgml <- result1$par[2]*result1$par[3]
solve(result1$hessian)

```

```

##           [,1]           [,2]           [,3]
## [1,]  5.657552e-02  0.0003634452 -1.420182e-05
## [2,]  3.634452e-04  13.2109992444 -5.162269e-01
## [3,] -1.420182e-05 -0.5162269466  2.661989e-02

```

```
meanmu <- mean(result[,2])
meanmu
```

```
## [1] -0.02252344
```

```
#apply(result[,2:6],2,mean)
cimu <- quantile(result[,2],c(0.025,0.975))
cimu
```

```
##      2.5%      97.5%
## -0.4810972  0.4374214
```

```
meanvg <- mean(result[,3])
meanvg
```

```
## [1] 10.11903
```

```
civg <- quantile(result[,3],c(0.025,0.975))
civg
```

```
##      2.5%      97.5%
##  3.210285 16.971752
```

```
meanve <- mean(result[,4])
meanve
```

```
## [1] 28.27113
```

```
cive <- quantile(result[,4],c(0.025,0.975))
cive
```

```
##      2.5%      97.5%
## 22.05225 35.56460
```

```
meanher <- mean(result[,5])
meanher
```

```
## [1] 0.2624171
```

```
ciher <- quantile(result[,5],c(0.025,0.975))
ciher
```

```
##      2.5%      97.5%
## 0.08389514 0.41867397
```



```
meank <- mean(result[,6])
meank
```

```
## [1] 0.3749238
```

```
cik <- quantile(result[,6],c(0.025,0.975))
cik
```

```
##          2.5%          97.5%
## 0.09157809 0.72020512
```

```
meanalfasq <- mean(result[,7])
meanalfasq
```

```
## [1] 9.928053
```

```
cialfasq <- quantile(result[,7],c(0.025,0.975))
cialfasq
```

```
##          2.5%          97.5%
## 3.17666 15.95473
```

(ii) The code outputs the following quantities:

$$\hat{E}(\mu|y) = -0.02,$$

$$\hat{E}(\sigma_g^2|y) = 10.12,$$

$$\hat{E}(\sigma_e^2|y) = 28.27,$$

$$\hat{E}(h^2|y) = 0.26,$$

$$\hat{\Pr}(-0.481 < \mu < 0.437|y) = 0.95,$$

$$\hat{\Pr}(3.21 < \sigma_g^2 < 16.972|y) = 0.95,$$

$$\hat{\Pr}(22.052 < \sigma_e^2 < 35.565|y) = 0.95,$$

$$\hat{\Pr}(0.084 < h^2 < 0.419|y) = 0.95.$$

The last lines at the bottom of the code (meanalfasq and cialfasq) show the Monte Carlo estimate of the mean and posterior interval of the genomic variance defined in (5.55) on page 256.

- (iii) Execution of the R-code on page 702 yields MC variances of $\hat{E}(\mu|y)$, $\hat{E}(\sigma_g^2|y)$, $\hat{E}(\sigma_e^2)$ and $\hat{E}(h^2|y)$. These are 1.47×10^{-5} , 0.23, 0.14 and 1.41×10^{-4} , respectively. The associated effective chain lengths are 3758, 55, 84 and 54, revealing a high degree of autocorrelation for the draws from $\sigma_e^2|y$, $\sigma_g^2|y$ and from the posterior distribution $h^2|y$. A longer chain length than the one used here would result in less noisy estimates of MC estimates of posterior means. For example, executing the code setting the length of the Gibbs chain to 40,000 results in a MC variance of $\hat{E}(\sigma_g^2|y)$ equal to 0.04 and an effective chain length of 326, an increase of factor 6 for the latter. This leads to more precise estimates of features of posterior distributions.
- (iv) The likelihood is parameterised in terms of $\theta' = (\mu, \sigma_e^2, k)$. In order to derive the ML estimate of σ_g^2 one proceeds as follows. Let $\lambda' = g(\theta)' = (\mu, \sigma_e^2, k\sigma_e^2)$, where $k\sigma_e^2 = \sigma_g^2$. Using the transformation invariance property (see page 60),

$$\hat{\sigma}_g^2 = \hat{k}\hat{\sigma}_e^2 = 9.31.$$

The asymptotic variance of λ is

$$\text{Var}(\hat{\lambda}) = \frac{\partial \lambda}{\partial \theta'} \Big|_{\theta=\hat{\theta}} \text{Var}(\hat{\theta}) \frac{\partial \lambda'}{\partial \theta} \Big|_{\theta=\hat{\theta}}. \quad (13.71)$$

In this expression (see page 58)

$$\frac{\partial \lambda'}{\partial \theta} = \begin{bmatrix} \frac{\partial \mu}{\partial \mu} & \frac{\partial \sigma_e^2}{\partial \mu} & \frac{\partial k\sigma_e^2}{\partial \mu} \\ \frac{\partial \mu}{\partial \sigma_e^2} & \frac{\partial \sigma_e^2}{\partial \sigma_e^2} & \frac{\partial k\sigma_e^2}{\partial \sigma_e^2} \\ \frac{\partial \mu}{\partial k} & \frac{\partial \sigma_e^2}{\partial k} & \frac{\partial k\sigma_e^2}{\partial k} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & k \\ 0 & 0 & \sigma_e^2 \end{bmatrix}.$$

When evaluated at the ML estimates, this gives

$$\frac{\partial \lambda'}{\partial \theta} \Big|_{\theta=\hat{\theta}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0.3291142 \\ 0 & 0 & 28.2877569 \end{bmatrix}.$$

Using $\text{Var}(\hat{\theta})$ from the OPTIM output, (13.71) is

$$\text{Var}(\hat{\mu}, \hat{\sigma}_e^2, \hat{\sigma}_g^2) = \begin{bmatrix} 0.00566 & 0.00036 & -0.00028 \\ 0.00036 & 13.21010 & -10.25614 \\ -0.00028 & -10.25614 & 13.12319 \end{bmatrix}.$$

The frequentist 95% confidence interval for σ_g^2 is

$$\begin{aligned} & \Pr \left(9.31 - 1.96\sqrt{13.12319} < \sigma_g^2 < 9.31 + 1.96\sqrt{13.12319} \right) \\ &= \Pr \left(2.21 < \sigma_g^2 < 16.41 \right) = 0.95 \end{aligned}$$

that is numerically in reasonable agreement with the McMC estimate of the Bayesian posterior interval. The chain of length 40,000 leads to the 95% Bayesian posterior interval $\hat{\Pr} \left(2.25 < \sigma_g^2 < 16.92 | y \right) = 0.95$, quite in line with the frequentist confidence interval.

13.5 Prediction Exercises

Exercise 1

The following results are useful:

$$E(\hat{y}_2) = E[E(y_2|y_1)] = E(y_2), \quad (13.72)$$

and

$$\begin{aligned} \text{Cov}(\hat{y}_2, y_2) &= E(\hat{y}_2 y_2) - E(\hat{y}_2) E(y_2), \\ E(\hat{y}_2 y_2) &= E[E(\hat{y}_2 y_2 | y_1)] \\ &= E[\hat{y}_2 E(y_2 | y_1)] \\ &= E(\hat{y}_2^2). \end{aligned} \quad (13.73)$$

Using (13.72) in (13.73)

$$\text{Cov}(\hat{y}_2, y_2) = \text{Var}(\hat{y}_2). \quad (13.74)$$

Similar algebra yields

$$\text{Cov}(\hat{g}_2, g_2) = \text{Var}(\hat{g}_2).$$

1. Given the model, the variance in the training data is

$$\text{Var}(y_1) = \sigma_g^2 (G_{11} + I\lambda) = \sigma_g^2 T, \quad \lambda = \frac{\sigma_e^2}{\sigma_g^2}, \quad T = (G_{11} + I\lambda).$$

$$\hat{y}_2 = E(y_2|y_1) = \mu_2 + G_{21}T^{-1}(y_1 - 1\mu_1).$$

Then

$$\begin{aligned}\text{Var}(\hat{y}_2) &= G_{21}T^{-1} \left(T\sigma_g^2 \right) T^{-1}G_{12} \\ &= G_{21}T^{-1}G_{12}\sigma_g^2,\end{aligned}$$

Also, given the model,

$$\hat{g}_2 = E(g_2|y_1) = G_{21}T^{-1}(y_1 - 1\mu_1).$$

Then

$$\begin{aligned}\text{Var}(\hat{g}_2) &= G_{21}T^{-1}G_{12}\sigma_g^2 \\ &= \text{Var}(\hat{y}_2).\end{aligned}$$

2. The prediction error variances are

$$\text{Var}(y_2 - \hat{y}_2) = \text{Var}(y_2) - \text{Var}(\hat{y}_2) = E[\text{Var}(y_2|y_1)] = \text{Var}(y_2|y_1),$$

Similarly,

$$\text{Var}(g_2 - \hat{g}_2) = \text{Var}(g_2) - \text{Var}(\hat{g}_2) = E[\text{Var}(g_2|y_1)] = \text{Var}(g_2|y_1),$$

where the last equalities in the right hand sides hold in the Gaussian setup because the conditional variances do not depend on y .

3. The squared correlations or prediction accuracies are

$$\begin{aligned}R^2(g_2, \hat{g}_2) &= \frac{[\text{Cov}(g_2, \hat{g}_2)]^2}{\text{Var}(g_2) \text{Var}(\hat{g}_2)} \\ &= \frac{\text{Var}(\hat{g}_2)}{\text{Var}(g_2)},\end{aligned}$$

equal to the proportion of the additive genetic variance captured by the linear predictor $\hat{g}_2$. Similarly

$$\begin{aligned}R^2(y_2, \hat{y}_2) &= \frac{[\text{Cov}(y_2, \hat{y}_2)]^2}{\text{Var}(y_2) \text{Var}(\hat{y}_2)} \\ &= \frac{\text{Var}(\hat{y}_2)}{\text{Var}(y_2)} \\ &= \frac{\text{Var}(\hat{g}_2) \text{Var}(g_2)}{\text{Var}(y_2) \text{Var}(g_2)} \\ &= h^2 R^2(g_2, \hat{g}_2).\end{aligned}\tag{13.75}$$

4. $R^2(g_2, \widehat{g}_2)$ can be written as

$$\begin{aligned} R^2(g_2, \widehat{g}_2) &= \frac{\text{Var}(\widehat{g}_2)}{\text{Var}(g_2)} \\ &= \frac{\text{Var}(g_2) - \mathbb{E}[\text{Var}(g_2|y_1)]}{\text{Var}(g_2)}. \end{aligned}$$

As the size of the training data n_1 increases, given that the data structure is such that this leads to a steady increase of information about the scalar g_2 , the posterior variance $\text{Var}(g_2|y_1)$ decreases and $R^2(g_2, \widehat{g}_2)$ will increase towards 1. Therefore in (13.75), the upper limit of $R^2(y_2, \widehat{y}_2)$ is equal to h^2 .

5.

$$\begin{aligned} \mathbb{E}(\text{MSE}_v) &= \mathbb{E}_{y_1, y_2} (y_2 - \widehat{y}_2)^2 \\ &= \mathbb{E}_{y_1, y_2} (y_2^2 + \widehat{y}_2^2 - 2y_2\widehat{y}_2) \\ &= \mathbb{E}_{y_2} (y_2^2) + \mathbb{E}_{y_1} (\widehat{y}_2^2) - 2\mathbb{E}_{y_1, y_2} (y_2\widehat{y}_2) \\ &= \text{Var}(y_2) + [\mathbb{E}(y_2)]^2 + \text{Var}(\widehat{y}_2) + [\mathbb{E}(\widehat{y}_2)]^2 \\ &\quad - 2[\text{Cov}(y_2, \widehat{y}_2) + \mathbb{E}(y_2)\mathbb{E}(\widehat{y}_2)] \\ &= \text{Var}(y_2) + \text{Var}(\widehat{y}_2) + [\mathbb{E}(y_2) - \mathbb{E}(\widehat{y}_2)]^2 - 2\text{Cov}(y_2, \widehat{y}_2) \\ &= \text{Var}(y_2) - \text{Var}(\widehat{y}_2) \end{aligned}$$

because $\mathbb{E}(y_2) = \mathbb{E}(\widehat{y}_2)$ and $\text{Cov}(y_2, \widehat{y}_2) = \text{Var}(\widehat{y}_2)$. This expression is equal to $\text{Var}(y_2 - \widehat{y}_2)$, the prediction error variance. The equality holds when the predictor is based on the conditional mean. Indeed, by definition of variance

$$\begin{aligned} \text{Var}(y_2 - \widehat{y}_2) &= \mathbb{E}[(y_2 - \widehat{y}_2)^2] - [\mathbb{E}(y_2 - \widehat{y}_2)]^2 \\ &= \mathbb{E}(\text{MSE}) - [\mathbb{E}(y_2) - \mathbb{E}(\widehat{y}_2)]^2 \\ &= \mathbb{E}(\text{MSE}) \end{aligned}$$

when $\mathbb{E}(y_2) = \mathbb{E}(\widehat{y}_2)$.

The assumption throughout is that expectations are computed over the true model. For example, the conditional mean is

$$\widehat{y}_2 = \mathbb{E}(y_2|y_1) = \int y_2 p(y_2|y_1) dy_2$$

and this requires knowledge of $[y_2|y_1]$, not only its form but also of the parameters that index this distribution.

Exercise 2

1. The frequentist expectation of the validating mean squared error (12.35), given x_0 , is

$$\begin{aligned} E_{y y_0} (\text{MSE}_v) &= E_{y_0} (y_0^2) + E_y (\hat{y}_0^2) - 2 E_{y_0} (y_0) E_y (\hat{y}_0) \\ &= \text{Var} (y_0) + (E (y_0))^2 + \text{Var} (\hat{y}_0) + (E (\hat{y}_0))^2 \\ &\quad - 2 E_{y_0} (y_0) E_y (\hat{y}_0) \\ &= \text{Var} (y_0) + \text{Var} (\hat{y}_0) + (E (y_0 - \hat{y}_0))^2 \end{aligned}$$

as in (6.66). Due to independence, $E_{y y_0} (y_0 \hat{y}_0) = E_{y_0} (y_0) E_y (\hat{y}_0)$. The first term represents sampling variation of a new record; the second accounts for variation of the predictor over replications of training data and the third is an average squared bias.

2. The algebra of the Bayesian calculation (given x_0 and here also σ^2 , both omitted in the notation) is based on taking expectations over the posterior predictive distribution of $\hat{y}_0$, $[\hat{y}_0|y]$,

$$\begin{aligned} E (\text{MSE}_v | y, y_0) &= y_0^2 + E (\hat{y}_0^2 | y) - 2 y_0 E (\hat{y}_0 | y) \\ &= y_0^2 + \sigma^2 + \text{Var}_{\theta|y} (x'_0 \theta) + (E (\hat{y}_0 | y))^2 \\ &\quad - 2 y_0 E (\hat{y}_0 | y) \\ &= \sigma^2 + \text{Var}_{\theta|y} (x'_0 \theta) + (y_0 - E (\hat{y}_0 | y))^2. \quad (13.76) \end{aligned}$$

The first term accounts for sampling variation of a new validating datum; the second represents the propagated posterior uncertainty of parameter θ on to the conditional mean of the predictor $\hat{y}_0$, given θ (the sharper the posterior distribution of θ the smaller the size of this term); the third is the squared discrepancy between the observed validating datum y_0 and the expected value of the predictor, taken over its posterior predictive distribution (see (10.4a)). In the second line I used (see (10.4b))

$$\begin{aligned} \text{Var} (\hat{y}_0 | y) &= E (\hat{y}_0^2 | y) - (E (\hat{y}_0 | y))^2 \\ &= \sigma^2 + \text{Var}_{\theta|y} (x'_0 \theta), \\ E (\hat{y}_0^2 | y) &= \sigma^2 + \text{Var}_{\theta|y} (x'_0 \theta) + (E (\hat{y}_0 | y))^2. \end{aligned}$$

On the other hand, if instead of predicting single records, a Bayesian predictor $\hat{y}_0 = x'_0 \theta^*$ is constructed drawing θ^* from the posterior distribution $[\theta|y]$, then,

taking expectations over $[\theta|y]$

$$\begin{aligned} E(\text{MSE}_v | y, y_0) &= y_0^2 + E\left((x'_0 \theta)^2 | y\right) - 2y_0 E(x'_0 \theta | y) \\ &= \text{Var}_{\theta|y}(x'_0 \theta) + (y_0 - E(x'_0 \theta | y))^2, \end{aligned}$$

which is equal to the last two terms in (13.76). This is so because in the linear model, with $\hat{y}_0 = x'_0 \theta$,

$$E_{y_0|y}(\hat{y}_0 | y) = E_{\theta|y}(x'_0 \theta | y)$$

(see (10.4a)). The expectation of MSE_v in (13.76) has an extra term accounting for the sampling variance of the draw of the predictor from $[\hat{y}_0 | \theta, y]$.

3. To recapitulate and modifying the notation used in part 2, the Bayesian linear model under consideration assumes that the i th record of the training data y is a draw from the sampling distribution

$$y_i | b, x_i, \sigma^2 \sim N(x'_i b, \sigma^2). \quad (13.77)$$

The parameters (b, σ^2) are assumed independent a priori, with prior densities $p(b) p(\sigma^2)$ left unspecified.

A validating datum y_0 is an extraction from the sampling model

$$y_0 | b, x_0, \sigma^2 \sim N(x'_0 b, \sigma^2).$$

Denote the predictor of y_0 as y_0^* ; this is a random variable that has the same sampling distribution as y_0 and that is independently drawn from the posterior predictive distribution $[y_0^* | x_0, y]$. The construction of y_0^* can be described as follows:

- (a) Randomly sample (b^*, σ^{2*}) from the posterior distribution (given training data y) with density $p(b, \sigma^2 | y, x_0)$
- (b) Given (b^*, σ^{2*}) , draw y_0^* from the posterior distribution with density $p(y_0^* | b^*, x_0, \sigma^{2*}) = N(y_0^* | x'_0 b^*, \sigma^{2*})$
- (c) It follows that y_0^* is a draw from the posterior predictive distribution with density

$$p(y_0^* | y, x_0) = \int p(y_0^* | b^*, x_0, \sigma^{2*}) p(b^*, \sigma^{2*} | y, x_0) db^* d\sigma^{2*}$$

(that happens to be a multivariate t -distribution; this needs not concern us when operating in an MCMC environment)

- (d) The random vectors (b^*, σ^{2*}) and (b, σ^2) are independent, given y , and also, the scalars y_0^* and y_0 are independent, given y .

The expected value and variance of y_0^* , taken over the posterior predictive distribution are

$$\begin{aligned} E(y_0^*|x_0, y) &= E_{b, \sigma^2|y} \left[E(y_0^*|b, x_0, \sigma^2) \right] \\ &= E_{b|y}(x'_0 b), \end{aligned} \quad (13.78)$$

(y is omitted in the inner expectation in the first line because given (b, x_0, σ^2) , y_0^* is independent of y) and

$$\begin{aligned} \text{Var}(y_0^*|x_0, y) &= E_{b, \sigma^2|y} \left[\text{Var}(y_0^*|b, x_0, \sigma^2) \right] + \text{Var}_{b, \sigma^2|y} \left[E(y_0^*|b, x_0, \sigma^2) \right] \\ &= E_{\sigma^2|y}(\sigma^2) + \text{Var}_{b|y}(x'_0 b). \end{aligned} \quad (13.79)$$

The expected value of the validating mean squared error of prediction based on y_0^* (conditional on training and validating data) is

$$\begin{aligned} E_{y_0^*|x_0, y}(\text{MSE}_v) &= E_{y_0^*|x_0, y} (y_0 - y_0^*)^2 \\ &= y_0^2 + E_{y_0^*|x_0, y} (y_0^{*2}) - 2y_0 E_{y_0^*|x_0, y} (y_0^*) \\ &= \text{Var}(y_0^*|x_0, y) + \left[y_0 - E_{y_0^*|x_0, y} (y_0^*) \right]^2 \\ &= E_{\sigma^2|y}(\sigma^2) + \text{Var}_{b|y}(x'_0 b) + \left[y_0 - E_{b|y}(x'_0 b) \right]^2. \end{aligned} \quad (13.80)$$

The first term accounts for the sampling variance of y_0^* , the second for posterior uncertainty of b , and the third term is the usual point mass (where $E_{b|y}(x'_0 b)$ replaces an estimate of $E_{b|y}(x'_0 b)$) that is commonly used in the calculation of validating mean squared error when all sources of uncertainty are ignored.

Exercise 3

Case 1 When training data y_t are treated as fixed, $E(\hat{y}_v - y_v) = \hat{y}_v - E(y_v)$, $\text{Var}(\hat{y}_v - y_v) = \text{Var}(y_v)$ and

$$\begin{aligned} E(\hat{y}_v - y_v) &= 1\hat{\mu} - 1\mu. \\ \text{Var}(\hat{y}_v - y_v) &= V = I\sigma^2. \end{aligned}$$

Let $A = \frac{1}{\sigma^2} I$. Then $AV = I$, an $n_v \times n_v$ identity matrix (idempotent). It follows that

$$\begin{aligned} & (\hat{y}_v - y_v)' A (\hat{y}_v - y_v) \\ &= \frac{1}{\sigma^2} (\hat{y}_v - y_v)' (\hat{y}_v - y_v) \sim \chi^2(r(A), \lambda). \end{aligned}$$

Since $r(A) = n_v$, $\lambda = \frac{1}{\sigma^2} (1\hat{\mu} - 1\mu)' (1\hat{\mu} - 1\mu)$, then,

$$\text{MSE}_v = \frac{\sigma^2}{n_v} \chi^2(n_v, \lambda).$$

Using (12.31a)

$$\begin{aligned} E(\text{MSE}_v) &= \frac{\sigma^2}{n_v} \left(n_v + \frac{1}{\sigma^2} (1\hat{\mu} - 1\mu)' (1\hat{\mu} - 1\mu) \right) \\ &= \sigma^2 + (\hat{\mu} - \mu)^2. \end{aligned} \quad (13.81)$$

The first term in the right hand side of (13.81) represents sampling uncertainty of y_v and the second represents the squared difference between the predictor and the expected value of the predictand.

Using (12.31b) the variance is

$$\text{Var}(\text{MSE}_v) = \left(\frac{\sigma^2}{n_v} \right)^2 \left(2n_v + \frac{4}{\sigma^2} (1\hat{\mu} - 1\mu)' (1\hat{\mu} - 1\mu) \right). \quad (13.82)$$

Case 2 When training data are treated as random and validating data as fixed,

$$\begin{aligned} E(\hat{y}_v - y_v) &= 1_{n_v} \mu - y_v \\ \text{Var}(\hat{y}_v - y_v) &= 1_{n_v} 1'_{n_v} \frac{\sigma^2}{n_t}, \end{aligned}$$

where 1_{n_v} is a column vector of 1's with n_v elements.

Consider the quadratic form

$$\frac{1}{\sigma^2} (\hat{y}_v - y_v)' (\hat{y}_v - y_v) = (\hat{y}_v - y_v)' A (\hat{y}_v - y_v)$$

where $A = I \frac{1}{\sigma^2}$. Then AV is idempotent, as shown below

$$\begin{aligned} AVAV &= \begin{pmatrix} 11' & 1 \\ & n_t \end{pmatrix} \begin{pmatrix} 11' & 1 \\ & n_t \end{pmatrix} \\ &= \begin{pmatrix} 11' & 1 \\ & n_t \end{pmatrix}. \end{aligned}$$

It follows that

$$\frac{1}{\sigma^2} (\hat{y}_v - y_v)' (\hat{y}_v - y_v) \sim \chi^2 \left(r(A), \frac{1}{\sigma^2} (1\mu - y_v)' (1\mu - y_v) \right)$$

a chi-square distribution with $r(A) = n_v$ degrees of freedom and non-centrality parameter $\frac{1}{\sigma^2} (1\mu - y_v)' (1\mu - y_v)$. Then the sample validating mean squared error is a draw from

$$\text{MSE}_v = \frac{\sigma^2}{n_v} \chi^2 \left(r(A), \frac{1}{\sigma^2} (1\mu - y_v)' (1\mu - y_v) \right)$$

and has expected value

$$\begin{aligned} E(\text{MSE}_v) &= \frac{\sigma^2}{n_v} \left[\frac{n_v}{n_t} + \frac{1}{\sigma^2} (1\mu - y_v)' (1\mu - y_v) \right] \\ &= \frac{\sigma^2}{n_t} + \frac{1}{n_v} \sum_{i=1}^{n_v} (y_{v_i} - \mu)^2. \end{aligned}$$

The first term represents sampling variance of $\hat{\mu}$ and the second is the average squared difference between the validating records and the expected value of their predictions.

Case 3 When both the training and the validating data are treated as random, the mean squared error is not chi-square distributed. However, its expected value can be computed using (12.32).

In this case,

$$\begin{aligned} E(\hat{y}_v - y_v) &= E(1_{n_v} \hat{\mu} - 1_{n_v} \mu) \\ &= (1_{n_v} \mu - 1_{n_v} \mu) = 0, \\ \text{Var}(\hat{y}_v - y_v) &= 1_{n_v} 1'_{n_v} \frac{\sigma^2}{n_t} + I_{n_v} \sigma^2. \end{aligned}$$

Then

$$\begin{aligned} E(\text{MSE}_v) &= \frac{1}{n_v} \text{tr} [\text{Var}(\hat{y}_v - y_v)] \\ &= \frac{\sigma^2}{n_v} \left(\frac{n_v}{n_t} + n_v \right) \\ &= \sigma^2 \left(\frac{1}{n_t} + 1 \right). \end{aligned} \tag{13.83}$$

The first term accounts for the sampling variance of the ML estimator and the second for the sampling variance of the validating records. There is no third term because the expectation of $(\hat{y}_v - y_v)$ over the distribution of (y_v, y_t) is zero.

One can make further progress as follows. Expand the MSE_v , adding and subtracting 1μ and replacing $\hat{y}_v = 1\hat{\mu}$:

$$\begin{aligned} & \frac{1}{n_v} (y_v - \hat{y}_v)' (y_v - \hat{y}_v) \\ &= \frac{1}{n_v} ((y_v - 1\mu) - (1\hat{\mu} - 1\mu))' ((y_v - 1\mu) - (1\hat{\mu} - 1\mu)) \\ &= \frac{1}{n_v} (y_v - 1\mu)' (y_v - 1\mu) - \frac{2}{n_v} (y_v - 1\mu)' 1 (\hat{\mu} - \mu) \\ & \quad + (\hat{\mu} - \mu)' (\hat{\mu} - \mu). \end{aligned}$$

With large n_v , $\frac{1}{n_v} (y_v - 1\mu)' (y_v - 1\mu)$ tends to σ^2 , $(y_v - 1\mu)' 1 (\hat{\mu} - \mu)$ tends to zero in probability, and we are left with the expression

$$\frac{1}{n_v} (y_v - \hat{y}_v)' (y_v - \hat{y}_v) \rightarrow \sigma^2 + (\hat{\mu} - \mu)' (\hat{\mu} - \mu). \quad (13.84)$$

Let $A = \frac{n_v}{\sigma^2}$. If $n_v = n_t = n$, with $V = \frac{\sigma^2}{n_t}$, $AV = 1$. Define

$$z'Az = \frac{n}{\sigma^2} (\hat{\mu} - \mu)' (\hat{\mu} - \mu) \sim \chi^2(1),$$

a chi-square distribution with one degree of freedom. Then

$$\frac{\sigma^2}{n} z'Az = (\hat{\mu} - \mu)' (\hat{\mu} - \mu) \sim \frac{\sigma^2}{n} \chi^2(1)$$

and

$$\frac{1}{n_v} (y_v - \hat{y}_v)' (y_v - \hat{y}_v) \rightarrow \sigma^2 + \frac{\sigma^2}{n} \chi^2(1),$$

with expected value $\sigma^2 + \frac{\sigma^2}{n}$ as in (13.83). So in this case, in the limit with very large sample size, MSE_v is distributed as a scaled chi-square distribution plus an additive constant.

To compute $E(\text{MSE}_t)$, I use

$$\begin{aligned} \hat{y}_t &= 1_{n_t} (1_{n_t} 1'_{n_t})^{-1} 1_{n_t} y_t \\ &= H y_t \end{aligned}$$

where H is idempotent. Then with $H1_{n_t} = 1_{n_t}$,

$$E(\hat{y}_t - y_y) = 0.$$

The variance is

$$\begin{aligned} \text{Var}(Hy_t - y_y) &= H\sigma^2 + I\sigma^2 - 2H\sigma^2 \\ &= (I_{n_t} - H)\sigma^2. \end{aligned}$$

Therefore,

$$\begin{aligned} E(\text{MSE}_t) &= \frac{\sigma^2}{n_t} \text{tr}(I_{n_t} - H) \\ &= \frac{\sigma^2}{n_t} (n_t - 1). \end{aligned}$$

The average difference between validating and training mean squared errors

$$\begin{aligned} E(\text{MSE}_v) - E(\text{MSE}_t) &= \sigma^2 \left(\frac{1}{n_t} + 1 \right) - \frac{\sigma^2}{n_t} (n_t - 1) \\ &= 2 \frac{\sigma^2}{n_t}. \end{aligned}$$

This is the expected optimism (see (6.83)) as coined by Efron (1986), the extent to which the MSE based on training data underestimates the validating MSE, on average.

Exercise 4

1. (a) In the first scenario, assuming y_v^* is a draw from $y_v|\hat{\mu}$, $y_t \sim N(1\hat{\mu}, I\sigma^2)$,

$$E(y_v^* - y_v) = 1\hat{\mu} - y_v.$$

Define

$$z = (y_v^* - y_v).$$

Let $A = I_{n_v} \frac{1}{\sigma^2}$ and let $V = \text{Var}(y_v^* - y_v) = I_{n_v} \sigma^2$. Then $AV = I_{n_v}$, an idempotent matrix and

$$z'Az \sim \chi^2(n_v, \lambda),$$

where the non-centrality parameter is

$$\lambda = \frac{1}{\sigma^2} (1\hat{\mu} - y_v)' (1\hat{\mu} - y_v).$$

The validating mean squared error is

$$\text{MSE}_v = \frac{\sigma^2}{n_v} z' A z \sim \frac{\sigma^2}{n_v} \chi^2(n_v, \lambda)$$

and therefore, using (12.31),

$$\text{E}(\text{MSE}_v) = \frac{\sigma^2}{n_v} \left(n_v + \frac{1}{\sigma^2} (1\hat{\mu} - y_v)' (1\hat{\mu} - y_v) \right), \quad (13.85a)$$

$$\text{Var}(\text{MSE}_v) = \left(\frac{\sigma^2}{n_v} \right)^2 \left(2n_v + \frac{4}{\sigma^2} (1\hat{\mu} - y_v)' (1\hat{\mu} - y_v) \right). \quad (13.85b)$$

The first term in the right hand side of (13.85a) accounts for the sampling variance associated with the generation of predicted validating records y_v^* .

(b) In this second scenario the posterior distribution of μ is

$$\mu|y_t \sim N\left(\hat{\mu}, \frac{\sigma^2}{n_t}\right),$$

where in this model, the posterior mean $\hat{\mu}$ takes the same form as the ML estimator. The vector of predicted validating records is

$$\hat{y}_v = 1_{n_v} \mu.$$

The computation of MSE_v requires

$$\begin{aligned} \text{E}(\hat{y}_v|y_t) &= \text{E}_{\mu|y_t} [\text{E}(\hat{y}_v|\mu)] \\ &= \text{E}_{\mu|y_t} [1_{n_v} \mu] = 1_{n_v} \hat{\mu} \end{aligned}$$

and

$$\begin{aligned} \text{Var}(\hat{y}_v|y_t) &= \text{E}_{\mu|y_t} [\text{Var}(\hat{y}_v|\mu)] + \text{Var}_{\mu|y_t} [\text{E}(\hat{y}_v|\mu)] \\ &= \text{Var}_{\mu|y_t} (1_{n_v} \mu) = 1_{n_v} 1_{n_v}' \frac{\sigma^2}{n_t}. \end{aligned}$$

Therefore,

$$\begin{aligned} \text{E}(\hat{y}_v - y_v) &= 1_{n_v} \hat{\mu} - y_v, \\ \text{Var}(\hat{y}_v - y_v) &= 1_{n_v} 1_{n_v}' \frac{\sigma^2}{n_t}, \end{aligned}$$

and

$$\begin{aligned}
 E(\text{MSE}_v) &= \frac{1}{n_v} E[(\hat{y}_v - y_v)' (\hat{y}_v - y_v)] \\
 &= \frac{1}{n_v} \left[\frac{\sigma^2}{n_t} \text{tr}(1'_{n_v} 1_{n_v}) + (1_{n_v} \hat{\mu} - y_v)' (1_{n_v} \hat{\mu} - y_v) \right] \\
 &= \frac{\sigma^2}{n_t} + \frac{1}{n_v} (1_{n_v} \hat{\mu} - y_v)' (1_{n_v} \hat{\mu} - y_v). \tag{13.86}
 \end{aligned}$$

The first term on the right hand side is the contribution from the posterior uncertainty of μ and the second term is the average squared discrepancy between the validating observations and the validating mean predictions.

If $n_v = n_t = n$, $\text{MSE}_v \sim \frac{\sigma^2}{n} \chi^2(1, \lambda)$, where $\lambda = \frac{1}{\sigma^2} (1\hat{\mu} - y_v)' (1\hat{\mu} - y_v)$. The mean is given by (13.86), replacing n_v and n_t by n .

- (c) The third scenario is based on the hierarchical model defined in (12.41). Under this model, the validating mean squared error does not reduce to a chi-square variable. An analytic expression for the expected value of MSE_v is derived using the formula for the expectation of a quadratic form (12.32). Under the hierarchy defined in (12.41), dropping the subscript n_v in 1_{n_v}

$$\begin{aligned}
 E(y_v | y_t) &= E_{\mu | y_t} [E(y_v | \mu, y_t)] \\
 &= E_{\mu | y_t} (1\mu) = 1\hat{\mu}
 \end{aligned}$$

and

$$\begin{aligned}
 \text{Var}(y_v | y_t) &= E[\text{Var}(y_v | \mu, y_t)] + \text{Var}[E(y_v | \mu, y_t)] \\
 &= I\sigma^2 + 11' \frac{\sigma^2}{n_t}.
 \end{aligned}$$

Then

$$\begin{aligned}
 E(y_v^* - y_v) &= (1\hat{\mu} - y_v), \\
 \text{Var}(y_v^* - y_v) &= I\sigma^2 + 11' \frac{\sigma^2}{n_t},
 \end{aligned}$$

where the random variable y_v^* , the vector of predicted validating records, is drawn from the posterior predictive distribution $[y_v | y_t]$. Since $A = I$, using

(12.32), it follows that the expected value of the validating mean squared error is

$$\begin{aligned} E(\text{MSE}_v) &= \frac{1}{n_v} \left[(1\hat{\mu} - y_v)' (1\hat{\mu} - y_v) + \sigma^2 \left(\text{tr} \left(I_{n_v} + \frac{1}{n_t} (11') \right) \right) \right] \\ &= \sigma^2 + \frac{\sigma^2}{n_t} + \frac{1}{n_v} (1\hat{\mu} - y_v)' (1\hat{\mu} - y_v). \end{aligned} \quad (13.87)$$

2. The difference in MSE_v between (13.85) and (13.87) is the term $\frac{\sigma^2}{n_t}$ that reflects posterior uncertainty of μ .

As indicated in (13.83), the frequentist expectation of MSE_v taken over the distribution of training and validating data generates a term that accounts for the sampling uncertainty of $\hat{\mu}$ (the term $\frac{\sigma^2}{n_t}$). The estimator is unbiased and therefore there is no third term contributing the $E(\text{MSE}_v)$.

Exercises 1 and 2 illustrate that the expectation of MSE derived either from frequentist or from Bayesian perspectives contain similar terms. The distribution over which the expectation is computed determines the breakdown of $E(\text{MSE})$. In the frequentist approach, expectations are taken over the training data, or the validating data, or both. The Bayesian counterpart conditions on training and validating data; expectations are computed over the posterior distribution of parameters, or over the conditional distribution of simulated records, given parameters. These are draws from the same distribution as the validating records. Alternatively and to account for both sources of uncertainty, one resorts to the hierarchical model (12.41) and expectations are taken over the posterior predictive distribution of predicted values of validating data.

Exercise 5

All computations are conditional on the variance components that are assumed known.

1. The exact distribution of MSE_v when validating predictions y_v^* are drawn from the distribution $[y_v | \hat{\mu}, \hat{b}, y_t]$ can be obtained as follows. Let

$$\begin{aligned} z &= (y_v^* - y_v), \\ &\Rightarrow z | \hat{\mu}, \hat{b}, y_v \sim N \left(1_v \hat{\mu} + W_v \hat{b} - y_v, I_{n_v} \sigma_e^2 \right). \end{aligned}$$

Let $A = I_{n_v} \sigma_e^{-2}$; since $V = \text{Var}(z) = I_{n_v} \sigma_e^2$, $AV = I_{n_v}$, an idempotent matrix. Then

$$z' A z = \frac{1}{\sigma_e^2} (y_v^* - y_v)' (y_v^* - y_v) \sim \chi^2(n_v, \lambda)$$

where the non-centrality parameter is

$$\lambda = \frac{1}{\sigma_e^2} (1_v \hat{\mu} + W_v \hat{b} - y_v)' (1_v \hat{\mu} + W_v \hat{b} - y_v).$$

The validating mean squared error is

$$\text{MSE}_v = \frac{\sigma_e^2}{n_v} z' A z \sim \frac{\sigma_e^2}{n_v} \chi^2(n_v, \lambda). \quad (13.88)$$

Using (12.31) and (13.88) it follows that

$$\begin{aligned} E(\text{MSE}_v) &= \\ &= \left(\frac{\sigma_e^2}{n_v} \right) \left(n_v + \frac{1}{\sigma_e^2} (1_v \hat{\mu} + W_v \hat{b} - y_v)' (1_v \hat{\mu} + W_v \hat{b} - y_v) \right) \end{aligned} \quad (13.89a)$$

$$\begin{aligned} \text{Var}(\text{MSE}_v) &= \\ &= \left(\frac{\sigma_e^2}{n_v} \right)^2 \left(2n_v + \frac{4}{\sigma_e^2} (1_v \hat{\mu} + W_v \hat{b} - y_v)' (1_v \hat{\mu} + W_v \hat{b} - y_v) \right). \end{aligned} \quad (13.89b)$$

The R-code below generates n individuals each genotyped for p covariates (genetic markers). Among these genetic markers `nqtl` are chosen as causal genotypes. The gene substitution effects of these causal loci is chosen to generate an additive genetic variance equal to 10 squared units and the heritability of the continuous trait is set equal to 0.5. The data y are divided into a training y_t and a validating set y_v , each of size $n/2$. Let W represent the centred and scaled matrix X , where $X = \{X_{ij}\}$ is an $n/2 \times p$ observed matrix with genotypic codes X_{ij} equal to 0, 1, 2 according to the number of the arbitrarily chosen allele of individual i and marker j .

The operational model is defined in terms of the following distributions:

$$y_t | \mu, b, \sigma_e^2 \sim N(1\mu + Wb, I\sigma_e^2), \quad (13.90a)$$

$$b | \sigma_b^2 \sim N(0, I\sigma_b^2), \quad (13.90b)$$

where y_t is a vector of records of length $n/2$, μ is an unobserved mean, b is a vector of unknown genetic marker effects of length p , 1 is a vector of 1's of length $n/2$, σ_e^2 is the residual variance and σ_b^2 reflects prior uncertainty for each element of b . The two variance components are assumed known.

```
# CODE1314
# PREDICTION EXERCISE 5
rm(list=ls()) # CLEAR WORKSPACE
```



```

set.seed(123)
nindiv<-100
nmark <- 500
nt <- nindiv*nmark
# NUMBER QTL
nqtl <- 50

# GENERATE MARKER MATRIX FROM BINOMIAL DISTRIBUTION
X<-matrix(nrow=nindiv,ncol=nmark,rbinom(n=nt,size=2,p=.5))
#####
# CHOOSE VALUE FOR MEAN mu AND GENOMIC VARIANCE vgs
mu <- 10
vgs<-10
# CHOOSE VALUE FOR ENVIRONMENTAL VARIANCE ves
ves<-20
her <- vgs/(vgs+ves)

btrue<-matrix(data=0.0,nrow=nmark,ncol=1) # parameter from
# true model
IDq<-sample(1:nmark,nqtl,replace=F) # from the nmark markers,
# choose nqtl as QTL
QTLeff<-sqrt(vgs/nqtl)# calculate the QTL effect so that the
# total genetic variance is VA
btrue[IDq]<-QTLeff # the only b's that are not zero are those
# associated with QTL.
W <- matrix(data=NA,nrow= nindiv,ncol=nmark)
cm <- colMeans(X)
# CREATE MATRIX OF STANDARDISED MARKER GENOTYPE CODES
for (i in 1:nmark)
{
  W[,i] <-( X[,i]-cm[i]) / sd(X[,i])
}
# more efficiently, could use:
# W <- scale(X)
# GENERATE nindiv PHENOTYPES WITH MEAN 0, VAR=vgs+ves,
# HERITABILITY=vgs/(vgs+ves)
e<- rnorm(nindiv,mean=0,sd=sqrt(ves))
y <- mu + W%*%btrue+ e
k <- (ves/vgs)*nmark # ratio of residual to
# genomic variance Vb = vgs/nmark
train <- sample(1:nrow(W),floor(0.5*nrow(W)))
Xt <- W[train,]
yt <- y[train]
Xv <- W[-train,]
yv <- y[-train]
Zt <- cbind(1,Xt)
Zv <- cbind(1,Xv)
#####
# ridge regression coefficient matrix, rhs & solution solt
RHSt <- crossprod(Zt,yt)
LHSt <- crossprod(Zt)
LHSt[-1,-1] <- LHSt[-1,-1]+diag(k,nrow=nrow(LHSt)-1)
solt <- solve(LHSt,RHSt)

```

```
# PREDICTION, CONDITIONAL ON ESTIMATED PARAMETERS (solt)
predval <- Zv**%solt # VALIDATING
predtrain <- Zt**%solt # TRAINING
```

The R-code below generates a MC estimate of the posterior distribution of the MSE_v , where the validating predictions are drawn from the distribution $[y_v | \hat{\mu}, \hat{b}, y_t]$.

```
# CODE1314 (cont)
# COMPUTE SAMPLING DISTRIBUTION OF MSE, CONDITIONAL ON
# (mu_hat,b_hat) AND VARIANCES
rep <- 10000
res1 <- matrix(data=NA, nrow=rep,ncol=1)

meany <- predval
vary <- diag(ves,nrow=length(yv))
ptm <- proc.time()
for (i in 1:rep){
  yrep <- rnorm(length(yv),meany,sqrt(ves))
  msel <- mean((yrep-yv)^2)
  ztz <- (1/ves)*sum((yrep-yv)^2)
  res1[i,] <- msel
}
proc.time()-ptm
```

```
##      user  system elapsed
##      0.70    0.01    0.21
```

```
meanmsev <- apply(res1,2,mean)
meanmsev
```

```
## [1] 52.34182
```

```
varmsev <- apply(res1,2,var)
varmsev
```

```
## [1] 67.52094
```

```
ncp <- sum((yv-meany)^2)/ves
expmse <- (ves/length(yv))*(length(yv)+ncp)
expmse
```

```
## [1] 52.35257
```

```
varmse <- (2* length(yv) + 4*ncp)*(ves/length(yv))^2
varmse
```

```
## [1] 67.76411
```

```
expQF <- ves + (mean((yv-meany)^2))
expQF
```

```
## [1] 52.35257
```

The MC estimates of the mean and variance are 52.342 and 67.521. These agree well with the exact results, 52.353 and 67.764 obtained from (13.89). The last line of the code computes the expected value of MSE_v using the formula for the expectation of a quadratic form, which of course, agrees with the mean of the scaled chi-square distribution.

2. The validating mean squared error arising from this model does not reduce to a chi-square variable. However an analytic expression for the expected value of MSE_v based on the expectation of a quadratic form can be derived as follows. First express the hierarchical model as

$$\begin{aligned}\theta^*|y_t &\sim N(\hat{\theta}, C_t^{-1}\sigma_e^2), \\ y_v^*|\theta^*, y_t &\sim N(Z_v\theta^*, I\sigma_e^2),\end{aligned}$$

where

$$\begin{aligned}\theta &= (\mu, b), \\ Z_v &= (1 \ W_v).\end{aligned}$$

The solution to the linear system, $\hat{\theta}$, is

$$C_t \hat{\theta} = Z_t' y_t,$$

and

$$C_t = \begin{bmatrix} 1'1 & 1'W_t \\ W_t'1 & W_t'W_t + Ik \end{bmatrix}, \quad k = \frac{\sigma_e^2}{\sigma_b^2}.$$

Then

$$E(y_v^*|y_t) = E_{\theta^*|y_t} [E(y_v^*|\theta^*, y_t)] = E_{\theta^*|y_t} (Z_v\theta^*) = Z_v\hat{\theta},$$

and

$$\begin{aligned}\text{Var}(y_v^*|y_t) &= E_{\theta^*|y_t} [\text{Var}(y_v^*|\theta^*, y_t)] + \text{Var}_{\theta^*|y_t} [E(y_v^*|\theta^*, y_t)] \\ &= I\sigma_e^2 + Z_v C_t^{-1} Z_v' \sigma_e^2.\end{aligned}$$

Therefore, conditional on the complete data $y' = (y'_t, y'_v)$,

$$\begin{aligned} E(\text{MSE}_v) &= \frac{1}{n_v} E \left[(y_v^* - y_v)' (y_v^* - y_v) \right] \\ &= \frac{1}{n_v} \left[\text{tr} \left(\sigma_e^2 \left(I + Z_v C_t^{-1} Z_v' \right) \right) + (Z_v \hat{\theta} - y_v)' (Z_v \hat{\theta} - y_v) \right] \\ &= \sigma_e^2 + \frac{\sigma_e^2}{n_v} \text{tr} \left(Z_v C_t^{-1} Z_v' \right) + \frac{1}{n_v} (Z_v \hat{\theta} - y_v)' (Z_v \hat{\theta} - y_v). \end{aligned} \quad (13.91)$$

The first term is the contribution from sampling variation of validating predictions, the second reflects the propagated posterior uncertainty of θ and the third is an average squared discrepancy between the observed validating records and the predictions.

3. The R-code below applies the method of composition to obtain draws from the posterior predictive distribution of y_v and constructs the validating mean squared error based on these draws.

```
# CODE1314 (cont)
# METHOD OF COMPOSITION:
# (ACCOUNTING FOR UNKNOWN LOCATION PARAMETERS)
# 1. USING TRAINING DATA Yt, SAMPLE THETA* ~ THETA|Yt
# 2. SAMPLE VALIDATING DATA Yv* ~ Yv|THETA*
# 3. COMPUTE VALIDATION MSEv = MEAN((Yv*-Yv)^2)
# 4. GOTO 1 UNTIL ENOUGH SAMPLES
rep <- 1000
res2 <- matrix(data=NA, nrow=rep, ncol=2)
theta <- solt
Cinv <- solve(LHSt)
ch <- chol(Cinv*ves)
varcov <- Cinv*ves
ptm <- proc.time()
for (i in 1:rep){
  # print(i)
  theta <- solt + t(ch)%*%rnorm(length(theta),0,1)
  # DRAWS FROM THE VALIDATING DATA:
  ystarval <- rnorm(length(yv), Zv%*%theta, sqrt(ves))
  # DRAWS FROM THE TRAINING DATA:
  ystartrain <- rnorm(length(yt), Zt%*%theta, sqrt(ves))

  mse2val <- mean((ystarval-yv)^2) # VALIDATION MSE
  mse2train <- mean((ystartrain-yt)^2) # TRAINING MSE
  res2[i,] <- c(mse2val, mse2train)
}
proc.time()-ptm
```

```
##      user  system elapsed
##      6.51    0.15    1.50
```

```
# hist(res2[,1])
apply(res2,2,mean)
```

```
## [1] 62.99294 43.39716
```

```
meanmse2val <- mean(res2[,1])
varmse2val <- var(res2[,1])
interm <- Zv%*%Cinv%*%t(Zv)
expQF <- ves + (ves*sum(diag(interm)))/length(yv) +
  mean((predval-yv)^2)
expQF
```

```
## [1] 62.65027
```

The MC estimate of the mean validating mean squared error 62.993 agrees well with the theoretical result 62.65 obtained from (13.91). The posterior uncertainty of $[\theta|y_r]$ propagates on to the distribution of the validating mean squared error. As a consequence, the mean validating mean squared error is larger than the one obtained when the calculation is conditional on $\theta = \hat{\theta}$. The posterior variance of MSE_v , equal to 116.464 is also considerably larger.

The attraction of the Monte Carlo approach is that it generates an estimate of the complete marginal posterior distribution of MSE.

Exercise 6

1. The linear predictor is

$$\begin{aligned}\hat{y} &= W [W'W + \Sigma]^{-1} W'y \\ &= Hy.\end{aligned}$$

Contrary to the non-hierarchical case the ‘hat’ matrix H is not idempotent.

The expected optimism is

$$\begin{aligned}\frac{2}{n} \text{tr} [\text{Cov} (y, \hat{y}')] &= \frac{2}{n} \text{tr} (\text{Cov} (y, \hat{y}')) \\ &= \frac{2}{n} \text{tr} (\text{Cov} (y, y' H')) \\ &= \frac{2}{n} \text{tr} [\text{Cov} (y, y' W [W'W + \Sigma]^{-1} W')] \\ &= \frac{2}{n} \text{tr} [V W [W'W + \Sigma]^{-1} W']\end{aligned}\tag{13.92}$$

where $V = \text{Var}(y) = ZZ'\sigma_f^2 + I\sigma^2 = \sigma^2 \left(ZZ'\frac{\sigma_f^2}{\sigma^2} + I \right) = \sigma^2 \tilde{V}$. An additive genetic model without non-genetic sources of covariation between full-sibs imposes the constraint $0 < \sigma_f^2 \leq \sigma^2$.

The R-code below generates the full-sib family data, constructs the mixed model equations, the predictor (fitted values) and obtains an expression for expected optimism. The bottom part of the code generates a Monte Carlo estimate of expected optimism.

```
# CODE1315
#FULL-SIB CONTINUOUS DATA
rm(list=ls()) # Clear the workspace
set.seed(123771)
ptm<-proc.time()
require(graphics)
# INITIALISE PARAMETERS
mus<-10 # MEAN
vfs<-1 #VARIANCE BETWEEN FULL-SIBS
#vfs<-0.5 #VARIANCE BETWEEN FULL-SIBS
#vfs <- 0.1
# RESIDUAL VARIANCE
ves<-5
k <- ves/vfs
nf<-500 # NUMBER OF FULL-SIB FAMILIES
n<-2 # FULL-SIB FAMILY SIZE
nb <- 2 # NUMBER OF BREEDS
N<-nf*n # TOTAL NUMBER OF RECORDS
y<-matrix(data=0,nrow=nf*n,ncol=1)
z<-matrix(data=0,nrow=nf*n,ncol=1)
# GENERATE nf FULL-SIB EFFECTS fs
fs<-rnorm(nf,mean=0,sd=sqrt(vfs))
# BREED EFFECTS
br <- rep(0,nb)
br[1] <- 5
br[2] <- 8
# GENERATE nf*n RESIDUAL EFFECTS
es<-rnorm(nf*n,mean=0,sd=sqrt(ves))
#####
## GENERATING A FULL-SIB STRUCTURE
b <- rep(1:nb,each=N/2)
z <- rep(1:nf,each=n)
y <- br[b] + fs[z] + es
d <- data.frame(y,z)
#####
d<-data.frame(y,z)
# GENERATE INCIDENCE MATRICES X & Z
family <- z
breed <- b
family <- as.factor(family)
breed <- as.factor(breed)
X<-model.matrix(~0+breed)
Z<-model.matrix(~0+family)
```

```

W <- cbind(X,Z)
LHS <- crossprod(W) # LHS OF MME
LHS[-(1:2),-(1:2)]<-LHS[-(1:2),-(1:2)]+diag(k,nrow=nrow(LHS)-2)
RHS <- crossprod(W,y) # RHS OF MME
SOL <- solve(LHS,RHS) # SOLUTION
HAT <- W%%solve(LHS)%%t(W)
V <- Z%%t(Z)*vfs + diag(vfs,nrow=length(y))
COVyyhat <- sum(diag(V%%t(HAT)))
lambda <- 1/k
Vtilde <- (Z%%t(Z)*lambda + diag(1,nrow=length(y)))
df <- sum(diag(Vtilde%%HAT))
yhat <- HAT%%y
MSEt <- mean((y-yhat)^2)
MSEt

```

```
## [1] 4.754883
```

```

optim1 <- 2*COVyyhat/length(y)
optim1

```

```
## [1] 2.02
```

```

MSEv <- MSEt + optim1
MSEv

```

```
## [1] 6.774883
```

The analytical results yields an expression for expected optimism (parameter `optim1` in the bottom of the code) based on (13.92) equal to 2.02. The sample training mean squared error is

$$\text{MSE}_t = \frac{1}{n} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

equal to 4.75. Therefore the estimate of the validating mean square error is

$$\widehat{\text{MSE}}_v = \text{MSE}_t + \frac{2}{n} \text{tr}(\text{Cov}(y, \hat{y}'))$$

equal to 6.77 (parameter `MSEv` at the bottom of the code).

2. The R-code below generates a Monte Carlo estimate of expected optimism using a parametric bootstrap.

```

# CODE1315 (cont)
# MONTE CARLO ESTIMATE OF OPTIMISM
## SIMULATE DATA & STORE Y, YHAT
rep <- 1000
gemY <- matrix(data=NA,nrow=rep,ncol=length(y))
gemYHAT <- matrix(data=NA,nrow=rep,ncol=length(y))

br[1] <- SOL[1]
br[2] <- SOL[2]
for (i in 1:rep){
  fs<-rnorm(nf,mean=0,sd=sqrt(vfs))
  es<-rnorm(nf*n,mean=0,sd=sqrt(ves))
  y <- br[b] + fs[z] + es
  RHS <- crossprod(W,y)
  SOL <- solve(LHS,RHS)
  yhat <- W%SOL
  gemY[i,] <- y
  gemYHAT[i,] <- yhat
}
sumcov <- 0
## COMPUTE SUM(COV(Y,YHAT))
for (i in 1:length(y)){
  sumcov <- sumcov + cov(gemY[,i],gemYHAT[,i])
}
#####
## A MORE EFFICIENT AND LESS TRANSPARENT CODE IS
# sumcov <-
# sum(sapply(1:length(y),FUN=function(i)
# {cov(gemY[,i],gemYHAT[,i])}))
#####
sumcov

```

```
## [1] 1010.313
```

```

optim2 <- 2*sumcov/length(y)
optim2

```

```
## [1] 2.020626
```

The Monte Carlo estimate of expected optimism, conditional on the model (parameter `optim2` in the bottom of the code), based on 1000 replications of data, is equal to 2.0206. This compares well with the analytical result.

Degrees of Freedom

An expression for the model's *degrees of freedom* (or *effective number of parameters*) is

$$df = \frac{1}{\sigma^2} \text{tr}(\text{Cov}(y, \hat{y}')), \quad (13.93)$$

(Hastie et al. 2009) that for the present setup gives

$$df = \text{tr} \left[\tilde{V} W [W'W + \Sigma]^{-1} W' \right].$$

Setting $\sigma_f^2 = 1$ and $\sigma^2 = 5$ results in an estimate of the model's degrees of freedom or effective number of parameters based on (13.93) equal 202 (parameter df in the code).

This estimate depends on the ratio σ_f^2 / σ^2 . To illustrate, setting σ_f^2 to a very small number in the code above (say, $\sigma_f^2 = 10^{-5}$), produces an estimate of the model's degrees of freedom of 2.00 (rounded-off to two decimal places). This is equal to the number of breeds. A simple way to understand this result is by studying the linear system (12.46). The model for a scalar record can be written

$$y_{ijk} = b_i + f_{ij} + e_{ijk},$$

where b_i is the fixed effect of breed i , f_{ij} is the random effect of family j nested in breed i and e_{ijk} is the residual associated with record k of family j in breed i . A closer look at the mixed model equations (12.46) reveals that the equation for the ij th family is

$$n_o \hat{b}_i + \left(n_o + \frac{\sigma_f^2}{\sigma^2} \right) \hat{f}_{ij} = y_{ij.}$$

where $y_{ij.}$ is the sum of the records belonging to the ij th family. A little algebra shows that for the data structure of the example

$$\hat{f}_{ij} = (y_{ij.} - n_o \hat{b}_i) - \frac{(n_o - 1) \sigma_f^2 + \sigma^2}{n_o \sigma_f^2 + \sigma^2} (y_{ij.} - n_o \hat{b}_i).$$

When $\sigma_f^2 \rightarrow 0$, $\hat{f}_{ij} \rightarrow 0$ and when $\sigma_f^2 \rightarrow \sigma^2$ (its maximum value, given the constraint),

$$\hat{f}_{ij} \rightarrow \frac{1}{(n_o + 1)} (y_{ij.} - n_o \hat{b}_i).$$

The first case, $\sigma_f^2 \rightarrow 0$, sets family effects equal to zero and the model's degrees of freedom correspond to the number of breeds, equal to 2.

Imagine that instead of parametrising the model as in (12.45), the random family effects are integrated out yielding the alternative form

$$y|b \sim N(Xb, \sigma^2 \tilde{V}).$$

The generalised least squares estimator of b is

$$\hat{b} = \left(X' \tilde{V}^{-1} X \right)^{-1} X' \tilde{V}^{-1} y.$$

The vector of fitted values is $\hat{y} = X\hat{b}$ and the degrees of freedom of the integrated parametrisation is

$$\begin{aligned} df &= \frac{1}{\sigma^2} \text{tr} [\text{Cov}(y, \hat{y})] \\ &= \frac{1}{\sigma^2} \text{tr} \left[\text{Cov} \left(y, y' \tilde{V}^{-1} X \left(X' \tilde{V}^{-1} X \right)^{-1} X' \right) \right] \\ &= \frac{1}{\sigma^2} \text{tr} \left[\sigma^2 \tilde{V} \tilde{V}^{-1} X \left(X' \tilde{V}^{-1} X \right)^{-1} X' \right] \\ &= \text{tr} \left[X' X \left(X' \tilde{V}^{-1} X \right)^{-1} \right]. \end{aligned}$$

Setting $\sigma_f^2 = 1$ and $\sigma^2 = 5$ results in an estimate of the model's degrees of freedom equal to 2.8, that differs from the estimate 202 obtained with the alternative parametrisation. Different parametrisations of the model result in different measures of its complexity, as pointed out by Spiegelhalter et al. (2002). On the other hand, when $\sigma_f^2 \rightarrow 0$, $\tilde{V} \rightarrow I$ and the model's degrees of freedom approaches $\text{tr} \left[X' X \left(X' X \right)^{-1} \right] = 2$, the number of breeds, as with the previous parametrisation.

The concept of degrees of freedom has not been discussed in the book and is not pursued further. The example illustrates that the topic deserves considerable reflection (see Janson et al. 2015). The definition used here is based on the optimism of the training mean squared error as an estimate of validating error and is due to Efron (1986). Spiegelhalter et al. (2002, 2014) discuss the subject and propose an alternative expression. Further elaborations of the concept can be found in Gelman et al. (2014b).

References

- Agresti A, Coull BA (1998) Approximate is better than "exact" for interval estimation of binomial proportions. *Am Stat* 52:119–126
- Aitken AC (1934) Note on selection from a multivariate normal population. *Proc Edinb Math Soc* 4:106–110
- Albert J (2009) Bayesian computation with R. Springer, Berlin
- Anderson TW (1957) Maximum likelihood estimation for the multivariate normal distribution when some observations are missing. *J Am Stat Assoc* 52:200–203
- Anderson DA, Aitkin M (1985) Variance component models with binary response: interviewer variability. *J R Stat Soc Ser B* 47:203–210
- Benjamini Y (2013) Selective inference and false discovery rate. Public Lecture at UC Berkeley, Berkeley
- Benjamini Y, Hochberg Y (1995) Controlling false discovery rate: a practical and powerful approach to multiple testing. *J R Stat Soc Ser B Methodol* 57:289–300
- Benjamini Y, Yekutieli Y (2001) The control of false discovery rate in multiple testing under dependency. *Ann Stat* 29:1165–1188
- Benjamini Y, Krieger AM, Yekutieli D (2006) Adaptive linear step-up procedures that control the false discovery rate. *Biometrika* 93:491–507
- Bernard E (2021) Introduction to machine learning. Wolfram Media, Champaign
- Bernardo JM, Smith AFM (1994) Bayesian theory. Wiley
- Berndt E, Hall B, Hall R, Hausman J (1974) Estimation and inference in nonlinear structural models. *Ann Econ Soc Measur* 3:653–665
- Bishop CM (2006) Pattern recognition and machine learning. Springer, Berlin
- Bottou L (2012) Stochastic gradient descent tricks. In: Montavon G, Orr GB, Müller KR (eds) *Neural networks: tricks of the trade*, chap 18. Springer, Berlin, pp 421–436
- Box GEP, Tiao GC (1973) Bayesian inference in statistical analysis. Wiley
- Boyle EA, Li YI, Pritchard JK (2017) An expanded view of complex traits: from polygenic to omnigenic. *Cell* 169:1177–1186
- Breiman L (1996) Bagging predictors. *Mach Learn* 24:123–140
- Breiman L (2001) Random forests. *Mach Learn* 45:5–32
- Breiman L, Friedman JH, Olshen RA, Stone CJ (1984) Classification and regression trees. Chapman and Hall, London
- Brown PJ (1977) Centering and scaling in ridge regression. *Technometrics* 19:35–36
- Brzyski D, Peterson CB, Sobczyk P, Candes E, Bogdan M, Sabatti C (2017) Controlling the rate of GWAS false discoveries. *Genetics* 205:61–75
- Bulmer MG (1971) The effect of selection on genetic variability. *Am Nat* 105:201–211

- Bulmer MG (1980) The mathematical theory of quantitative genetics. Oxford University Press, Oxford
- Carlin BP, Louis TA (1996) Bayes and empirical bayes methods for data analysis. Chapman and Hall, London
- Casella G, Berger RL (1990) Statistical inference. Brooks–Cole, Pacific Grove
- Chen MH, Shao QM, Ibrahim JG (2000) Monte Carlo methods in Bayesian computation. Springer, Berlin
- Chib S, Greenberg E (1995) Understanding the Metropolis-Hastings algorithm. *Am Stat* 49:327–335
- Clopper CJ, Pearson ES (1934) The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika* 26:404–413
- Cowles MK, Carlin BP (1996) Markov chain Monte Carlo convergence diagnostics: a comparative review. *J Am Stat Assoc* 91:883–904
- Curnow RN (1961) The estimation of repeatability and heritability from records subject to culling. *Biometrics* 17:553–566
- Dahlquist BÅ, Björck Å (1974) Numerical methods. Prentice Hall, Saddle River
- de los Campos G, Gianola D, Rosa GJM (2009) Reproducing kernel Hilbert spaces regression: a general framework for genetic evaluation. *J Anim Sci* 87:1883–1887
- de los Campos G, Gianola D, Rosa GJM, Weigel KA, Crossa J (2010) Semi-parametric genomic-enabled prediction of genetic values using reproducing kernel Hilbert spaces methods. *Genet Res* 92:295–308
- de los Campos G, Vazquez AI, Fernando R, Klimentidis YC, Sorensen D (2013a) Prediction of complex human traits using the genomic best linear unbiased predictor. *PLOS Genetics* 9(3), e1003608
- de los Campos G, Hickey J, Pong-Wong R, Daetwyler HD, Calus MPL (2013b) Whole genome regression and prediction methods applied to plant and animal breeding. *Genetics* 193:327–345. <https://doi.org/10.1534/genetics.112.143313>
- de los Campos G, Sorensen D, Gianola D (2015) Genomic heritability: what is it? *PLOS Genet* 11(5):e1005048
- de los Campos G, Grueneberg A, Funkhouser S, Perez-Rodriguez P, Samaddar A (2022) Fine mapping and accurate prediction of complex traits using Bayesian Variable Selection models applied to biobank-size data. *Eur J Hum Genet* <https://doi.org/10.1038/s41431-022-01135-5>
- Dempfle L (1977) Relation entre BLUP (best linear unbiased prediction) et estimateurs Bayesiens. *Ann Génét Sél Anim* 9:27–32
- Dempster AP, Laird NM, Rubin DB (1977) Maximum likelihood from incomplete data via de EM algorithm (with discussion). *J R Stat Soc Ser B* 39:1–38
- Devroye L (1986) Non-uniform random variate generation. Springer, Berlin
- Diggle PJ (2011) Estimating prevalence using an imperfect test. *Epidemiol Research Int.* <https://doi.org/10.1155/2011/608719>
- Duvenaud D, Lloyd JR, Grosse R, Tenenbaum JB, Ghahramani Z (2013) Structure discovery in nonparametric regression through compositional kernel search. arxiv:1302.4922
- Edwards AWF (1992) Likelihood. The John Hopkins University Press, Baltimore
- Efron B (1983) Estimating the error rate of a prediction rule: improvement on cross-validation. *J Am Stat Assoc* 78:316–331
- Efron B (1986) How biased is the apparent rate of a prediction rule? *J Am Stat Assoc* 81:461–470
- Efron B (2010) Large-scale inference. Cambridge University Press, Cambridge
- Efron B (2020) Prediction, estimation, and attribution. *J Am Stat Assoc* 115:636–655
- Efron B, Hastie T (2016) Computer age statistical inference. Cambridge University Press, Cambridge
- Efron B, Tibshirani R (1997) Improvements on cross-validation: the .632+ bootstrap method. *J Am Stat Assoc* 92:548–560
- Efron B, Tibshirani R (2002) Microarrays, empirical Bayes methods and false discovery rates. *Genet Epidemiol* 1:70–86

- Efron B, Tibshirani R, Storey JD, Tusher V (2001) Empirical Bayes analysis of a microarray experiment. *J Am Stat Assoc* 96:1151–1160
- Ewens WJ, Grant GR (2005) Statistical methods in bioinformatics. Springer, Berlin
- Falconer DS (1965) The inheritance of liability to certain diseases, estimated from the incidence among relatives. *Ann Hum Gen* 29:51–76
- Falconer DS, Mackay TFC (1996) Introduction to quantitative genetics. Longman, London
- Fisher RA (1922) On the mathematical foundations of theoretical statistics. *Philos Trans R Soc Lond Ser A* 222:309–368
- Friedman J, Hastie T, Höfling H, Tibshirani R (2007) Pathwise coordinate optimization. *Ann Appl Stat* 1(2):302–332
- Friedman J, Hastie T, Tibshirani R (2009) Glmnet: lasso and elastic-net regularized generalized linear models. R package version 1.1-4
- Friedman J, Hastie T, Tibshirani R (2010) Regularization paths for generalized linear models via coordinate descent. *J Stat Softw* 33(1):1–22
- Galton F (1885) Regression towards mediocrity in hereditary stature. *J Anthropol Inst* 15:246–263
- García-Cortés LA, Sorensen D (1996) On a multivariate implementation of the Gibbs sampler. *Genet Sel Evol* 28:121–126
- Gelfand AE (1996) Model determination using sampling-based methods. In: Gilks WR, Richardson S, Spiegelhalter DJ (eds) Markov chain Monte Carlo in practice. Chapman and Hall, London, pp 145–161
- Gelfand AE, Dey DK, Chang H (1992) Model determination using predictive distributions with implementation via sampling-based methods. In: Bernardo JM, Berger JO, Dawid AP, Smith AFM (eds) Bayesian statistics 4. Oxford University Press, Oxford, pp 147–167
- Gelman A, Carlin JB, Stern HS, Rubin DB (1995) Bayesian data analysis. Chapman and Hall, London
- Gelman A, Meng XL, Stern H (1996) Posterior predictive assessment of model fitness via realized discrepancies (with discussion). *Stat Sin* 6:733–807
- Gelman A, Carlin JB, Stern HS, Rubin DB (2014a) Bayesian data analysis. Chapman and Hall, Boca Raton
- Gelman A, Hwang J, Vehtari A (2014b) Understanding predictive information criteria for Bayesian models. *Stat Comput* 24:997–1016
- George EI, McCulloch RE (1993) Variable selection via Gibbs sampling. *J Am Stat Assoc* 88:881–889
- Geweke J (1989) Bayesian inference in econometric models using Monte Carlo integration. *Econometrica* 57:1317–1339
- Geyer CJ (1992) Practical Markov chain Monte Carlo. *Stat Sci* 7:473–511
- Gianola D, de los Campos G (2008) Inferring genetic values for quantitative traits non-parametrically. *Genet Res* 90:525–540
- Gianola D, Schön CC (2016) Cross-validation without doing cross-validation in genome-enabled prediction. *G3* 6:3107–3128
- Gianola D, de los Campos G, Hill WG, Manfredi E, Fernando R (2009) Additive genetic variability and the Bayesian alphabet. *Genetics* 183:347–363
- Gianola D, Fernando R, Schön CC (2020) Inferring trait-specific similarity among individuals from molecular markers and phenotypes with Bayesian regression. *Theor Popul Biol* 132:47–59
- Glorot X, Bengio Y (2010) Understanding the difficulty of training deep feedforward neural networks. *J Mach Learn Res Proc Track* 9:249–256
- Goodfellow I, Bengio Y, Courville A (2016) Deep learning. MIT Press, Cambridge
- Green P (1995) Reversible jump MCMC computation and Bayesian model determination. *Biometrika* 82:711–732
- Habier D, Fernando R, Decker J (2007) The impact of genetic relationship information on genome-assisted breeding values. *Genetics* 177:2389–2397
- Habier D, Fernando R, Kizilkaya K, Garrik DJ (2011) Extension of the Bayesian alphabet for genomic selection. *BMC Bioinform* 12(186):1–12
- Hald A (1998) A history of mathematical statistics, from 1750 to 1930. Wiley, New York

- Halldorsson BV, Eggertsson HP, et al (2022) The sequences of 150,119 genomes in the UK Biobank. *Nature* 607:732–740
- Hammersley JM, Handscomb DC (1964) Monte Carlo methods. Wiley, Hoboken
- Harville DA (1977) Maximum likelihood approaches to variance component estimation and to related problems. *J Am Stat Assoc* 72:320–340
- Hastie T, Qian J (2016) Glmnet vignette. https://web.stanford.edu/~hastie/Papers/Glmnet_Vignette.pdf
- Hastie T, Tibshirani R, Friedman J (2009) The elements of statistical learning. Springer, New York, 745pp
- Hastings WK (1970) Monte Carlo sampling methods using Markov chains and their application. *Biometrika* 57:97–109
- Hayes B, Price J, Chamberlain AJ, Bowman PJ, Goddard ME (2010) Genetic architecture of complex traits and accuracy of genomic prediction: coat colour, milk-fat percentage, and type in Holstein cattle as contrasting model traits. *PLoS Genet* 6(9), e1001139
- Henderson CR (1953) Estimation of variance and covariance components. *Biometrics* 9:226–252
- Henderson CR (1963) Selection index and expected selection advance. In: Hanson WD, Robinson HF (eds) Statistical genetics and plant breeding. National Academy of Sciences, National Research Council Publication No. 982, Washington, DC, pp 141–163
- Henderson CR (1975) Best linear unbiased estimation and prediction under a selection model. *Biometrics* 31:423–447
- Henderson CR, Kempthorne O, Searle SR, Von Krosigk CN (1959) Estimation of environmental and genetic trends from records subject to culling. *Biometrics* 15:192–218
- Hill WG, Weir BS (2011) Variation in actual relationship as a consequence of Mendelian sampling and linkage. *Genet Res* 93:47–64
- Hill WG, Goddard ME, Visscher PM (2008) Data and theory point to mainly additive genetic variance for complex traits. *PLoS Genet* 4:e1000008
- Robert JP, Casella G (1996) The effect of improper priors on Gibbs sampling in hierarchical linear models. *J Am Stat Assoc* 91:1461–1473
- Hoerl AE, Kennard RW (1970a) Ridge regression: applications to nonorthogonal problems. *Technometrics* 12:69–82
- Hoerl AE, Kennard RW (1970b) Ridge regression: biased estimation for nonorthogonal problems. *Technometrics* 12:55–67
- Hu Z, Zhang J, Ge Y (2021) Handling vanishing gradient problem using artificial derivative. *IEEE Access* 9:22371–22377
- James G, Witten D, Hastie T, Tibshirani R (2017) An introduction to statistical learning. Springer, Berlin
- Jensen, J. L. (2023) Statistical inference for high dimensional data. Mathematical Institute, Aarhus University
- Janson L, Fithian W, Hastie TJ (2015) Effective degrees of freedom: a flawed metaphor. *Biometrika* 102:479–485
- Johnson NL, Kemp AW, Kotz S (2005) Univariate discrete distributions. Wiley, New York
- Kass RE, Carlin BP, Gelman A, Neal RM (1998) Markov chain Monte Carlo in practice: a roundtable discussion. *Am Stat* 52:93–100
- Kimeldorf G, Wahba G (1971) Some results on Tchebycheffian spline functions. *J Math Anal Appl* 33:82–95
- LeCun Y, Bengio Y, Hinton G (2015) Deep learning. *Nature* 521:436–444
- Lehmann EL, Casella G (1998) Theory of point estimation. Springer, Berlin
- Lindley DV, Smith AFM (1972) Bayesian estimates for the linear model. *J R Stat Soc Ser B* 34:1–41
- Little RJA, Rubin DB (1987) Statistical analysis with missing data. Wiley, New York
- Liu JS, Wong HW, Kong A (1994) Covariance structure of the Gibbs sampler with applications to the comparisons of estimators and augmentation schemes. *Biometrika* 81:27–40
- López OAM, López AM, Crossa J (2022) Multivariate statistical machine learning methods for genomic prediction. Springer, Berlin

- Lush JL, Shrode RR (1950) Changes in milk production with age and milking frequency. *J Dairy Sci* 33:338–357
- Mäki-Tanila A, Hill WG (2014) Influence of gene interaction on complex trait variation with multilocus models. *Genetics* 198:355–367
- Mallows CL (1973) Some comments on *Cp*. *Technometrics* 15:661–675
- Mardia KV, Kent JT, Bibby JM (1979) *Multivariate analysis*. Academic Press, Cambridge
- McCulloch WS, Pitts W (1943) A logical calculus of the ideas immanent in nervous activity. *Bullet Math Biophys* 5:115–133
- McLachlan GJ, Krishnan T (1997) *The EM algorithm and extensions*. Wiley, Hoboken
- Meeker WQ, Escobar LA (1995) Teaching about approximate confidence regions based on maximum likelihood estimation. *Am Stat* 49:48–53
- Meng X (2018) Statistical paradises and paradoxes in big data (i): law of large populations, big data paradox, and the 2016 presidential election. *Ann Appl Stat* 12:685–726
- Metropolis N, Rosenbluth AW, Rosenbluth MN, Teller AH, Teller E (1953) Equations of state calculations by fast computing machines. *J Chem Phys* 21:1087–1092
- Meuwissen THE, Hayes BJ, Goddard ME (2001) Prediction of total genetic value using genome-wide dense marker maps. *Genetics* 157:1819–1829
- Mitchell AJ, Beauchamp JJ (1988) Bayesian variable selection in linear regression (with discussion). *J Am Stat Assoc* 83:1023–1036
- Müller P, Parmigiani G, Rice K (2007) FDR and Bayesian multiple comparison rules. In: Bernardo JM, Bayarri MJ, Berger JO, Dawid AP, Heckerman D, Smith AFM, West M (eds) *Bayesian statistics*, vol 8. Oxford University Press, Oxford, pp 349–370
- Natarajan R, Kass RE (2000) Reference Bayesian methods for generalized linear models. *J Am Stat Assoc* 95:227–237
- Neal RM (1996) *Bayesian learning for neural networks*. Springer, Berlin
- Park T, Casella G (2008) The Bayesian LASSO. *J Am Stat Assoc* 103:681–686
- Park MY, Hastie T (2008) Penalized logistic regression for detecting gene interactions. *Biostatistics* 9:30–50
- Patterson HD, Thompson R (1971) Recovery of inter-block information when block sizes are unequal. *Biometrika* 58:545–554
- Patxot M, Banos TD, Kousathanas A, Orliac EJ, Ojavee SE, Moser G, Holloway A, Sidorenko J, Kutalik Z, Mägi R, Visscher PM, Rönnegård L, Robinson MR (2021) Probabilistic inference of the genetic architecture underlying functional enrichment of complex traits. *Nat Commun* 12:6972
- Pearson K (1894) Contributions to the mathematical theory of evolution. *Philos Trans R Soc Lond Ser A* 185:71–110
- Pearson K (1903) Mathematical contributions to the theory of evolution. XI. On the influence of natural selection on the variability and correlation of organs. *Philos Trans R Soc Lond Ser A* 200:1–66
- Perez P, de los Campos G (2014) Genome-wide regression and prediction with the BGLR statistical package. *Genetics* 198:483–495
- Perez-Elizalde S, Cuevas J, Perez-Rodriguez P, Crossa J (2015) Selection of the bandwidth parameter in a Bayesian regression model for genomic-enabled prediction. *J Agric Biol Environ Stat* 20:512–532
- Peskun PH (1973) Optimum Monte Carlo sampling using Markov chains. *Biometrika* 60:607–612
- Prentice RL, Pyke R (1979) Logistic disease incidence models and case-control studies. *Biometrika* 66:403–411
- Priestley MB (1981) *Spectral analysis and time series*. Academic Press, Cambridge
- Ripley B (1987) *Stochastic simulation*. Wiley, Hoboken
- Roberts GO, Gelman A, Gilks WR (1997) Weak convergence and optimal scaling of random walk Metropolis algorithms. *Ann Appl Probab* 7:110–120
- Ros M, Sorensen D, Waagepetersen R, Dupont-Nivet M, SanCristobal M, Bonnet JC, Mallard J (2004) Evidence for genetic control of adult weight plasticity in the snail *Helix aspersa*. *Genetics* 168:2089–2097

- Rousseauw J, du Plessis J, Benade A, Jordaan P, Kotze J, Jooste P (1983) Coronary risk factor screening in three rural communities. *South African Med J* 64:430–436
- Royall R (1997) *Statistical evidence*. Chapman and Hall, London
- Rubin DB (1976) Inference and missing data. *Biometrika* 63:581–592
- Rubin DB (1984) Bayesianly justifiable and relevant frequency calculations for the applied statistician. *Ann Stat* 12:1151–1172
- Rubin DB (1987) A noniterative sampling/importance resampling alternative to the data augmentation algorithm for creating a few imputations when fractions of missing information are modest: the SIR algorithm. Discussion of Tanner and Wong. *J Am Stat Assoc* 82:543–546
- Rubin DB (1988) Using the SIR algorithm to simulate posterior distributions (with discussion). In: Bernardo JM, DeGroot MH, Lindley DV, Smith AFM (eds) *Bayesian statistics 3*. Oxford University Press, Oxford, pp 395–402
- Rubin DB (2002) *Statistical analysis with missing data*. Wiley, New York
- Rumelhart D, Hinton G, Williams R (1986) Learning representations by back-propagation. *Nature* 323:533–536
- Samaddar A, Maiti T, de los Campos G (2024) Bayesian hierarchical hypothesis testing in large-scale genome-wide association analysis. *Genetics* <https://doi.org/10.1101/2024.02.26.582204>
- San Cristobal-Gaudy M, Elsen JM, Bodin L, Chevalet C (1998) Prediction of the response to a selection for canalisation of a continuous trait in animal breeding. *Genet Select Evol* 30:423–451
- Searle SR (1971) *Linear models*. Wiley, Hoboken
- Searle SR, Casella G, McCulloch CE (1992) *Variance components*. Wiley, Hoboken
- Seber GAF, Lee AJ (2003) *Linear regression analysis*. Wiley, Hoboken
- Singh D, Febbo PG, Ross K, Jackson DG, Manola J, Ladd C, Tamayo P, Renshaw AA, D'Amico AV, Richie JP, Lander ES, Loda M, Kantoff PW, Golub TR, Sellers WR (2002) Gene expression correlates of clinical prostate cancer behavior. *Cancer Cell* 1:203–209
- Smith AFM, Gelfand AE (1992) Bayesian statistics without tears: a sampling–resampling perspective. *Am Stat* 46:84–88
- So HC, Kwan JSH, Cherny SS, Sham PC (2011) Risk prediction of complex diseases from family history and known susceptibility loci, with applications for cancer screening. *Am J Hum Genet* 88:548–565
- Sorensen D, Gianola D (2002) *Likelihood, Bayesian, and MCMC methods in quantitative genetics*. Springer, Berlin, 740 pp. Reprinted with corrections (2006)
- Sorensen D, Waagepetersen R (2003) Normal linear models with genetically structured residual variance heterogeneity: a case study. *Genet. Res.* 82:207–222
- Sorensen D, Fernando RL, Gianola D (2001) Inferring the trajectory of genetic variance in the course of artificial selection. *Genet Res* 77:83–94
- Speed D, Hermani G, Johnson MR, Balding DJ (2012) Improved heritability estimation from genome-wide SNPs. *Am J Hum Genet* 91:1011–1021
- Speed D, Cai N, Johnson SMRN, Balding DJ (2017) Reevaluation of SNP heritability in complex human traits. *Nat Genet* 49:986–992
- Speed D, Kaphle A, Balding DJ (2021) SNP-based heritability and selection: improved models and new results. *BioEssays*. <https://doi.org/10.1002/bies.202100170>
- Spiegelhalter DJ, Best NG, Carlin BP, van der Linde A (2002) Bayesian measures of model complexity and fit (with discussion). *J R Stat Soc Ser B* 64:583–639
- Spiegelhalter DJ, Best NG, Carlin BP, van der Linde A (2014) The deviance information criterion: 12 years on. *J R Stat Soc Ser B* 76:485–493
- Storey JD (2002) A direct approach to false discovery rates. *J R Stat Soc Ser B* 64:479–498
- Storey JD (2003) The positive false discovery rate: a Bayesian interpretation and the q -value. *Ann Stat* 31:2013–2035
- Storey JD, Bass AJ (2021) Bioconductor's q value package. <https://www.bioconductor.org/packages/devel/bioc/vignettes/qvalue/inst/doc/qvalue.pdf>
- Storey JD, Tibshirani R (2003) Statistical significance for genomewide studies. *Proc Natl Acad Sci* 100:9440–9445

- Sun X, Qu L, Garrick DJ, Dekkers JCM, Fernando RL (2012) A fast EM algorithm for Bayes-like prediction of genomic breeding values. *PLoS One*. <https://doi.org/10.1371/journal.pone.0049157>
- Swendsen R, Wang J (1987) Non-universal critical dynamics in Monte Carlo simulations. *Phys Rev Lett* 58:86–88
- Tanner MA (1996) Tools for statistical inference. Springer, Berlin
- Tanner MA, Wong W (1987) The calculation of posterior distributions by data augmentation. *J Am Stat Assoc* 82:528–550
- Tibshirani R (1996) Regression shrinkage and selection via the LASSO. *J R Stat Soc Ser B* 58:267–288
- VanRaden PM (2008) Efficient methods to compute genomic predictions. *J Dairy Sci* 91:4414–4423
- Vehtari A, Lampinen J (2002) Bayesian model assessment and comparison using cross-validation predictive densities. *Neural Comput* 14:2439–2468
- Vehtari A, Gelman A, Gabry J (2017) Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. *Stat Comput* 27:1413–1432
- Visscher PM (2008) Sizing up human height variation. *Nat Genet* 40:489–490
- Waagepetersen R, Sorensen D (2001) A tutorial on reversible jump MCMC with a view towards applications in QTL-mapping. *Int Stat Rev* 69:49–61
- Wahba G (1990) Spline models for observational data. SIAM, Philadelphia
- Wainshtein P, Jain D, et al (2022) Assessing the contribution of rare variants to complex trait heritability from whole-genome sequence data. *Nat Genet* 54:263–273
- Wei GCG, Tanner MA (1990) A Monte Carlo implementation of the EM algorithm and the poor man's data augmentation algorithm. *J Am Stat Assoc* 85:699–704
- Weir BS (1996) Genetic data analysis II. Sinauer Associates, Sunderland
- Weir BS, Cockerham CC (1977) Two-locus theory in quantitative genetics. In: Pollak E, Kempthorne O, Bailey TB (eds) Proceedings of the international conference on quantitative genetics. The Iowa State University Press, Ames, pp 247–269
- Weir B, Cockerham CC, Reynolds J (1980) The effects of linkage and linkage disequilibrium on the covariance of noninbred relatives. *Heredity* 45:351–359
- Whittemore AS (2007) A Bayesian false discovery rate for multiple testing. *J Appl Stat* 34:1–9
- Wolfram S (2023) What is ChatGPT doing ... and why does it work? Wolfram Media, Champaign
- Wright S (1934) An analysis of variability in number of digits in an inbred strain of guinea pigs. *Genetics* 19:506–536
- Yang J, Benyamin B, McEvoy BP, Gordon S, Henders AK, Nyholt DR, Madden PA, Heath AC, Martin NG, Montgomery GW, Goddard ME, Visscher PM (2010) Common SNP's explain a large proportion of the heritability for human height. *Nat Genet* 42:565–569
- Yang J, Ferreira T, Morris AP, Medland SE, Genetic Investigation of Anthropometric Traits (GIANT) Consortium DGR, Meta-analysis (DIAGRAM) Consortium PAF Madden, Heath AC, Martin GN, Montgomery GW, Weedon MN, Loos RJ, Frayling TM, McCarthy MI, Hirschhorn JN, Goddard ME, Visscher PM (2012) Conditional and joint multiple-SNP analysis of GWAS summary statistics identifies additional variants influencing complex traits. *Nat Genet* 44:369–378
- Yekutieli D, Benjamini Y (1999) Resampling-based false discovery rate controlling multiple tests procedures for correlated test statistics. *J Stat Plann Inference* 82:171–196
- Yengo L, Vedantam S, Marouli E, et al (2022) A saturated map of common genetic variants associated with human height. *Nature* 610:704–712
- Zhao T, Fernando R, Cheng H (2021) Interpretable artificial neural networks incorporating Bayesian alphabet models for genome-wide prediction and association studies. *G3*. <https://doi.org/10.1093/g3journal/jkab228>
- Zou H, Hastie T (2005) Regularization and variable selection via the elastic net. *J R Stat Soc Series B* 67:301–320

Author Index

A

Agresti and Coull (1998), 445, 743
Aitken (1934), 453, 743
Albert (2009), 9, 212, 491, 743
Anderson (1957), 70, 743
Anderson and Aitkin (1985), 676, 743

B

Benjamini (2013), 363, 743
Benjamini and Hochberg (1995), 32, 33, 362, 364, 366, 367, 375, 398, 743
Benjamini and Yekutieli (2001), 367, 398, 399, 743
Benjamini et al. (2006), 367, 743
Bernard (2021), 553, 743
Bernardo and Smith (1994), 184, 403, 743
Berndt et al. (1974), 85, 743
Bishop (2006), 525, 553, 743
Bottou (2012), 106, 743
Box and Tiao (1973), 152, 153, 172, 743
Boyle et al. (2017), 47, 743
Breiman (1996), 585, 743
Breiman (2001), 30, 586, 743
Breiman et al. (1984), 26, 743
Brown (1977), 329, 743
Brzyski et al. (2017), 396, 743
Bulmer (1971), 39, 743
Bulmer (1980), 79, 743

C

Carlin and Louis (1996), 13, 507, 744
Casella and Berger (1990), 362, 744
Chen et al. (2000), 198, 744

Chib and Greenberg (1995), 198, 744
Clopper and Pearson (1934), 444, 744
Cowles and Carlin (1996), 198, 744
Curnow (1961), 74, 744

D

Dahlquist and Björck (1974), 83, 744
de los Campos et al. (2009), 535, 542, 744
de los Campos et al. (2010), 535, 543, 744
de los Campos et al. (2013a), 744
de los Campos et al. (2013b), 47, 49, 744
de los Campos et al. (2015), 43, 44, 46, 50, 78, 744
de los Campos et al. (2022), 382, 396, 744
Dempfle (1977), 171, 744
Dempster et al. (1977), 109, 744
Devroye (1986), 240, 416, 441, 744
Diggle (2011), 439, 444, 744
Duvenaud et al. (2013), 535, 744

E

Edwards (1992), 645, 744
Efron (1983), 587, 744
Efron (1986), 304, 313, 314, 728, 742, 744
Efron (2010), 374, 375, 377, 744
Efron (2020), 32, 744
Efron and Hastie (2016), 314, 315, 343, 371, 406, 414, 587, 744
Efron and Tibshirani (1997), 587, 744
Efron and Tibshirani (2002), 373, 374, 376, 744
Efron et al. (2001), 362, 364, 374, 377, 744
Ewens and Grant (2005), 398, 745

F

- Falconer (1965), [92](#), [745](#)
 Falconer and Mackay (1996), [64](#), [286](#), [745](#)
 Fisher (1922), [55](#), [56](#), [745](#)
 Friedman et al. (2007), [342](#), [343](#), [745](#)
 Friedman et al. (2009), [24](#), [347](#), [745](#)
 Friedman et al. (2010), [24](#), [343](#), [348](#), [745](#)

G

- Galton (1885), [64](#), [745](#)
 García-Cortés and Sorensen (1996), [195](#), [745](#)
 Gelfand (1996), [270](#), [271](#), [491](#), [492](#), [745](#)
 Gelfand et al. (1992), [270](#), [745](#)
 Gelman et al. (1995), [19](#), [150](#), [176](#), [499](#), [500](#),
[616](#), [745](#)
 Gelman et al. (1996), [19](#), [500](#), [745](#)
 Gelman et al. (2014a), [311](#), [494](#), [745](#)
 Gelman et al. (2014b), [742](#), [745](#)
 George and McCulloch (1993), [47](#), [350](#), [745](#)
 Geweke (1989), [209](#), [745](#)
 Geyer (1992), [198](#), [202](#), [745](#)
 Gianola and de los Campos (2008), [535](#), [745](#)
 Gianola and Schön (2016), [491](#), [493](#), [745](#)
 Gianola et al. (2009), [44](#), [745](#)
 Gianola et al. (2020), [535](#), [745](#)
 Glorot and Bengio (2010), [567](#), [745](#)
 Goodfellow et al. (2016), [106](#), [553](#), [745](#)
 Green (1995), [213](#), [218](#), [219](#), [745](#)

H

- Habier et al. (2007), [42](#), [745](#)
 Habier et al. (2011), [47](#), [350](#), [745](#)
 Hald (1998), [55](#), [745](#)
 Halldorsson et al. (2022), [3](#), [745](#)
 Hammersley and Handscomb (1964), [209](#), [746](#)
 Harville (1977), [128](#), [746](#)
 Hastie and Qian (2016), [24](#), [344](#), [348](#), [746](#)
 Hastie et al. (2009), [294](#), [525](#), [566](#), [576](#), [741](#),
[746](#)
 Hastings (1970), [203](#), [217](#), [746](#)
 Hayes et al. (2010), [350](#), [746](#)
 Henderson (1953), [167](#), [746](#)
 Henderson (1963), [169](#), [746](#)
 Henderson (1975), [74](#), [170](#), [461](#), [746](#)
 Henderson et al. (1959), [72](#), [148](#), [167–169](#),
[540](#), [746](#)
 Hill and Weir (2011), [49](#), [78](#), [746](#)
 Hill et al. (2008), [597](#), [746](#)
 Hobert and Casella (1996), [255](#), [746](#)
 Hoerl and Kennard (1970a), [328](#), [746](#)

- Hoerl and Kennard (1970b), [328](#), [746](#)
 Hu et al. (2021), [567](#), [746](#)

J

- James et al. (2017), [348](#), [580](#), [746](#)
 Janson et al. (2015), [742](#), [746](#)
 Jensen (2023), [398](#), [746](#)
 Johnson et al. (2005), [445](#), [746](#)

K

- Kass et al. (1998), [197](#), [746](#)
 Kimeldorf and Wahba (1971), [536](#), [746](#)

L

- Lehmann and Casella (1998), [56](#), [746](#)
 LeCun et al. (2015), [106](#), [553](#), [564](#), [746](#)
 Lindley and Smith (1972), [170](#), [333](#), [746](#)
 Little and Rubin (1987), [71](#), [654](#), [746](#)
 Liu et al. (1994), [194](#), [746](#)
 Lush and Shrode (1950), [72](#), [746](#)
 López et al. (2022), [553](#), [746](#)

M

- Mallows (1973), [311](#), [318](#), [747](#)
 Mardia et al. (1979), [99](#), [747](#)
 McCulloch and Pitts (1943), [553](#), [747](#)
 McLachlan and Krishnan (1997), [109](#), [747](#)
 Meeker and Escobar (1995), [645](#), [747](#)
 Meng (2018), [32](#), [747](#)
 Metropolis et al. (1953), [218](#), [747](#)
 Meuwissen et al. (2001), [47](#), [78](#), [747](#)
 Mitchell and Beauchamp (1988), [47](#), [747](#)
 Mäki-Tanila and Hill (2014), [597](#), [747](#)
 Müller et al. (2007), [381](#), [747](#)

N

- Natarajan and Kass (2000), [507](#), [747](#)
 Neal (1996), [553](#), [747](#)

P

- Park and Casella (2008), [346](#), [747](#)
 Park and Hastie (2008), [411](#), [747](#)
 Patterson and Thompson (1971), [128](#), [175](#), [747](#)
 Patxot et al. (2021), [47](#), [396](#), [747](#)
 Pearson (1894), [134](#), [747](#)
 Pearson (1903), [453](#), [747](#)
 Perez and de los Campos (2014), [542](#), [549](#), [747](#)

Perez-Elizalde et al. (2015), [535](#), [747](#)
 Peskun (1973), [217](#), [747](#)
 Prentice and Pyke (1979), [409](#), [747](#)
 Priestley (1981), [202](#), [747](#)

R

Ripley (1987), [198](#), [747](#)
 Roberts et al. (1997), [198](#), [747](#)
 Ros et al. (2004), [506](#), [747](#)
 Rousseauw et al. (1983), [576](#), [747](#)
 Royall (1997), [645](#), [748](#)
 Rubin (1976), [67](#), [71](#), [76](#), [410](#), [654](#), [748](#)
 Rubin (1984), [19](#), [500](#), [748](#)
 Rubin (1987), [212](#), [748](#)
 Rubin (1988), [212](#), [491](#), [748](#)
 Rubin (2002), [67](#), [72](#), [748](#)
 Rumelhart et al. (1986), [555](#), [748](#)

S

Samaddar et al. (2024), [435](#), [748](#)
 San Cristobal-Gaudy et al. (1998), [291](#), [748](#)
 Searle et al. (1992), [168](#), [748](#)
 Searle (1971), [127](#), [748](#)
 Seber and Lee (2003), [169](#), [310](#), [748](#)
 Singh et al. (2002), [21](#), [28](#), [33](#), [748](#)
 Smith and Gelfand (1992), [212](#), [491](#), [748](#)
 So et al. (2011), [452](#), [748](#)
 Sorensen and Gianola (2002), [55](#), [59](#), [61](#), [64](#),
[109](#), [111](#), [125](#), [148](#), [176](#), [195](#), [210](#),
[211](#), [214](#), [219](#), [507](#), [748](#)
 Sorensen and Waagepetersen (2003), [500](#), [748](#)
 Sorensen et al. (2001), [47](#), [748](#)
 Speed et al. (2012), [49](#), [748](#)
 Speed et al. (2017), [47](#), [748](#)
 Speed et al. (2021), [47](#), [748](#)
 Spiegelhalter et al. (2002), [742](#), [748](#)
 Spiegelhalter et al. (2014), [742](#), [748](#)
 Storey and Bass (2021), [380](#), [748](#)
 Storey and Tibshirani (2003), [380](#), [748](#)
 Storey (2002), [367](#), [379](#), [748](#)

Storey (2003), [372](#), [373](#), [375](#), [376](#), [379](#), [748](#)
 Sun et al. (2012), [261](#), [262](#), [748](#)
 Swendsen and Wang (1987), [231](#), [749](#)

T

Tanner and Wong (1987), [231](#), [749](#)
 Tanner (1996), [164](#), [749](#)
 Tibshirani (1996), [24](#), [339](#), [345](#), [749](#)

V

VanRaden (2008), [49](#), [749](#)
 Vehtari and Lampinen (2002), [491](#), [494](#), [749](#)
 Vehtari et al. (2017), [213](#), [494](#), [749](#)
 Visscher (2008), [45](#), [749](#)

W

Waagepetersen and Sorensen (2001), [213](#), [219](#),
[749](#)
 Wahba (1990), [525](#), [749](#)
 Wainshtein et al. (2022), [46](#), [749](#)
 Wei and Tanner (1990), [111](#), [749](#)
 Weir (1996), [88](#), [233](#), [749](#)
 Weir and Cockerham (1977), [41](#), [749](#)
 Weir et al. (1980), [41](#), [749](#)
 Whittemore (2007), [381](#), [749](#)
 Wolfram (2023), [566](#), [749](#)
 Wright (1934), [92](#), [749](#)

Y

Yang et al. (2010), [46](#), [749](#)
 Yang et al. (2012), [45](#), [396](#), [749](#)
 Yekutieli and Benjamini (1999), [368](#), [749](#)
 Yengo et al. (2022), [45](#), [749](#)

Z

Zhao et al. (2021), [553](#), [749](#)
 Zou and Hastie (2005), [347](#), [749](#)

Subject Index

A

Acceptance probability, 185, 215
Activation function, *see* Neural networks
Additive $\times$ additive genetic variance, 595
Additive genetic effects, 286
Additive genetic model, 540–542
Additive genetic relationship matrix, 541
Additive genetic value, 38, 179, 285, 286, 541
Additive genetic variance, 38
Allele, 36
Allele content, 37, 285
Area under the curve, *see* Binary data
Autocorrelation of sampled values, 199, 703
Autoregressive model, 16, 204
Average effect of the gene substitution, 38, 286

B

Back propagation, 555, 559, 597
 back propagation with multiple paths, 603
 vectorising back propagation, 602
Back propagation with multiple paths, *see*
 Back propagation
Bagging, 30
 See also Random forest
Bandwidth, 510
 choice of bandwidth parameter, 521
 leave-one-out cross-validation, 521
Basis function, 530
Batching, 203
Bayes factor, 177, 270
Bayes false discovery rate, *see* false discovery
 rate
Bayesian asymptotics, 183
Bayesian learning, 176, 179, 507

Bayesian local false discovery rate, *see* False
 discovery rate
Bayesian MCMC estimator of FDR, *see* False
 discovery rate
Bayesian prediction, *see* Prediction
Bayesian view of kernelised regression, 540
Bayes rule, 402, 435
Bayes theorem, 9, 148
Benjamini and Hochberg FDR, *see* False
 discovery rate
Best linear approximation, 283, 286
Best linear predictor, 283, 285
Best linear unbiased estimator (BLUE), 170,
 635
Best linear unbiased predictor (BLUP), 540,
 635
Best predictor, 282
 properties of best predictor, 284
Bias, *see* Neural networks
Bias-variance trade-off, 304
Binary classifier, *see* Binary data
Binary data, 20, 402, 429, 478, 490, 549, 613,
 707
 approximate analysis, 461
 area under the curve, 435
 binary classifier, 402
 Brier score, 420, 484
 confusion matrix, 435
 correlated binary data, 623, 709
 estimator of expected validating mean
 squared error, 406
 false negative rate, 436
 false positive rate, 436
 incidence, 436
 lasso for binary records, 413

logistic regression with non-random sampling, 408
 mean squared error, 405
 overall error rate, 436
 penalised logistic regression, 410
 posterior predictive distribution, 477
 prevalence, 436
 prior predictive distribution, 475
 receiver operating characteristic (ROC) curve, 436
 sensitivity, 436
 specificity, 436
 spike and slab model, 414
 training mean squared error, 406
 validating mean squared error, 406
 Binned estimator, 510
 Binomial proportion, 439
 Bonferroni correction, 44, 361
 Bootstrap, 315, 489
 nonparametric bootstrap, 315
 parametric bootstrap, 315, 637, 739
 Breeding value, 38
 Brier score, 404, 429
 See also Binary data
 Burn-in period, 194

C

Censored data, 611
 Central limit theorem, 149, 199
 Chain rule, *see* Neural networks
 Choice of bandwidth parameter, *see* Bandwidth
 Combining quadratic forms, 152, 153
 Complete data loglikelihood, 113
 Composition, 164, 440, 479, 632, 635, 736
 Conditional posterior distribution, 151, 174
 Conditional predictive distribution, 270
 Conditional predictive ordinate (CPO), 270
 Confidence interval, 7, 439, 608
 asymptotic approximation, 442
 binomial proportion, 443
 Clopper-Pearson confidence interval, 444
 Confusion matrix, *see* Binary data
 Conjugacy, 151
 Conjugate prior distribution, 150
 Convergence diagnosis, 197
 Correlated binary data, 242
 Cost function, 106, 411, 526, 529, 536, 537, 542, 546, 554, 563
 Covariance between relatives, 40
 Cramér-Rao theorem, 60
 Cumulative distribution function, 11
 Curvature, 7

D

Data augmentation, 231, 243, 622
 Deep learning, 553
 Degrees of freedom, 740
 Detailed balance, 215
 Dimension matching, 218
 Diploid, 36
 Discovery set, *see* False discovery rate
 Discrepancy measures, *see* Posterior predictive model checking
 Discrete Markov chain, 214
 Distribution
 Bernoulli distribution, 3, 257, 485
 beta-binomial distribution, 476
 beta distribution, 11, 12
 binomial distribution, 4, 112, 607, 641
 bivariate normal distribution, 67, 609
 chi-square distribution, 627, 651
 Dirichlet distribution, 229
 double exponential distribution, 345
 exponential distribution, 611, 658
 censoring, 612, 658
 lognormal distribution, 63, 187, 227
 mean and variance of a truncated normal distribution, 457
 multinomial distribution, 86, 111
 multivariate normal distribution, 123, 173, 180
 multivariate t -distribution, 162
 normal distribution, 55, 149
 normal inverse chi square distribution, 151
 scaled inverted chi square distribution, 150, 154, 256, 683
 singular normal distribution, 79, 99
 standard normal distribution, 118
 Student- t distribution, 155, 159, 262, 365, 682
 truncated distributions, 77, 239
 Dominance variance, 38
 Dual representation, 526
 Dual solution, *see* Ridge regression
 Dual variable, 527

E

Effective chain length, 198, 620, 622, 624, 626, 693, 696, 702, 712
 Effective number of parameters, 294, 740
 Effective sample size, 198, 201
 Eigenvalues, 98
 Eigenvectors, 98
 Elastic net, 346

Empirical Bayes estimator of FDR, *see* False discovery rate
 Environmental value, [37](#)
 Epistatic genetic variance, [595](#)
 Epistatic loci, [595](#)
 Epistatic model, [595](#)
 Epoch, [108](#)
 Equilibrium additive genetic variance, [39](#)
 Ergodic Markov chain, [215](#)
 Estimator of expected validating mean squared error, *see* Binary data
 Expectation-maximisation (EM) algorithm, [109](#), [616–618](#), [666](#), [671](#), [680](#)
 Expected (training) mean squared error, [303](#), [631](#), [728](#)
 Expected (validating) mean squared error, [303](#), [631](#), [634](#), [725](#), [726](#), [729](#), [730](#), [732](#)
 Bayesian interpretation, [722](#)
 Expected additive genetic relationship, [37](#)
 Expected information, [642](#)
 Expected mean squared error, [303](#)

F

Factorised likelihood, [68](#)
 False discovery proportion, [364](#)
 False discovery rate, [366](#)
 Bayes false discovery rate, [372](#)
 Bayesian McMC estimator of FDR, [382](#)
 Benjamini and Hochberg FDR, [366](#)
 connection between the empirical Bayes estimator and the classical false discovery rate, [376](#)
 discovery set, [364](#)
 empirical Bayes estimation, [373](#)
 empirical Bayes estimator, [374](#)
 fully Bayesian false discovery rate, [380](#)
 fully Bayesian local false discovery rate, [381](#)
 local false discovery rate, [373](#), [376](#)
 marginal posterior distribution, [388](#), [395](#), [397](#)
 McMC draw from the posterior distribution of FDR, [382](#)
 McMC estimate of average local false discovery rate, [382](#)
 mixture model, [371](#)
 positive false discovery rate, [372](#)
 spike and slab model, [392](#)
 Storey's q -values, [378](#)
 two Gaussian mixtures, [383](#)
 two Gaussain mixtures: inferences using Bayes-McMC, [386](#)

two Gaussain mixtures: inferences using EM maximum likelihood, [384](#)
 False negative rate, *see* Binary data
 False positive rate, *see* Binary data
 Family wise error rate, [361](#)
 Fisher's expected information, [59](#), [60](#), [608](#), [612](#), [613](#), [650](#), [659](#)
 Forward propagation, [555](#), [557](#)
 vectorising forward propagation, [602](#)
 Full-sib mixed model, [273](#)
 Fully Bayesian false discovery rate, *see* False discovery rate
 Fully conditional posterior distribution, [194](#), [232](#)
 Functional invariance, [60](#), [66](#), [644](#), [679](#), [718](#)

G

Gamma function, [12](#), [157](#), [476](#), [682](#)
 Gaussian kernel, [515](#), [533](#), [549](#)
 Gene, [36](#)
 Generalised inverse, [99](#)
 Genetic markers, [24](#), [42](#), [45](#), [49](#), [78–80](#), [359](#), [371](#), [447](#), [542](#), [549](#), [569](#), [617](#), [624](#), [632](#), [732](#)
 Genetic value, [37](#), [285](#)
 Genetic variance, [38](#)
 Genomewide association study (GWAS), [44](#), [300](#), [392](#), [393](#), [395](#), [417](#), [418](#), [426](#)
 Genomic heritability, [43](#), [80](#)
 Genomic likelihood, [101](#)
 Genomic model, [42](#), [78](#), [97](#), [138](#), [249](#), [261](#), [542](#), [617](#), [624](#), [676](#), [713](#)
 Genomic relationship matrix, [46](#), [535](#), [542](#)
 Genomic value, [43](#), [542](#)
 Genomic variance, [43](#), [542](#)
 Genotype, [36](#)
 Geyer's estimator of Monte Carlo variance, [202](#)
 Gibbs sampling algorithm, [193](#), [237](#), [620](#), [622](#), [624](#), [626](#), [694](#), [707](#), [713](#)
 Gradient checking, [568](#)
 Gradient descent, [105](#), [554](#), [555](#), [561](#), [564](#), [567](#)
 batch gradient descent, [567](#)
 mini-batch gradient descent, [567](#)
 Gram matrix, [528](#)

H

Haploid, [36](#)
 Hardy-Weinberg law, [37](#)
 Hat matrix, *see* Least squares
 Heritability, [38](#)

Hessian, 59, 648
 Heterogeneous variance model, *see* Least squares; Posterior predictive model checking, 500
 Heteroscedastic error, 291
 Homoscedastic error, 291
 Hyperbolic function, *see* Neural networks
 Hypothesis test, 361

I

Idempotent matrix, 292, 293, 627, 650, 651, 725, 728, 737
 Identifiability, 93, 462, 507
 Identity by descent (IBD), 37
 Ignorable selection, 67, 72
 Importance sampling, 208
 Improper prior distribution, 174
 Incidence, *see* Binary data
 Indicator function, 267, 511
 Infection prevalence, 438
 Infinitesimal model, 79, 180, 542
 Inner product of two vectors, 525
 Integrated autocorrelation, 201, 693, 696, 702, 712
 Intensity of selection, 118
 Interaction genetic model, 591, 595
 Inverse transform, 364, 365
 Iterative reweighted least squares, 97

J

Jacobian of the transformation, 219

K

Kernel function, 513, 528, 531, 532
 Kernelised cost function, 528
 Kernelised logistic regression prediction, 569
 Kernelised predictions, 528
 Kernel logistic regression, 546, 549
 Kernel matrix, 528
 Kernel regression estimator, 513
 Kernel trick, 531
 K-fold cross-validation, 309

L

Lag-covariance, 199
 Lagrange multiplier, 136
 Lag-*t* autocovariance, 202

Lasso, 24, 338
 Bayesian interpretation, 345
 binary records, 413
 logistic regression, 549
 subdifferential, 340
 tuning parameter, 339
 Law of iterated expectations, 282
 Least squares, 287, 511
 estimator, 287, 660
 hat matrix, 291
 heterogeneous variance model, 291
 overfitting, 294
 prediction, 317
 residual sum of squares, 295
 Leave-one-out cross-validation, 308, 338, 521
 Bayesian leave-one-out cross-validation, 491
 generalised cross-validation, 311
 implementation shortcut, 310
 mean squared error
 See also Bandwidth; Ridge regression
 Leverage, 294
 Liability, 92, 116
 Liability model, 92
 Likelihood
 marginal likelihood, 270
 Likelihood function, 5, 58, 607, 608, 611, 616, 621, 641, 643, 647, 655, 666, 671, 718
 Linear kernel, 531
 Linear regression model, 287, 478, 647
 Linear smoother, 301, 529
 Linkage disequilibrium, 38
 Local false discovery rate, *see* False discovery rate
 Local polynomial regression, 517
 Locus, 36
 Logistic lasso, 549
 prediction, 569
 Logistic model, 20, 485, 614, 616, 621, 697
 Logistic regression, 20
 See also Metropolis-Hastings algorithm; Neural networks 486
 Logistic regression with non-random sampling, *see* Binary data
 Logit, 94, 402, 607
 Logit model, 616
 Loglikelihood, 5, 58, 64, 607, 611–613, 615, 616, 618, 625, 641, 644, 647, 655, 658, 659, 661, 666, 671, 702
 Logodds, 94, 259, 402, 607
 Logscore, 403, 429
 Loss function, 557

M

Machine learning, 553
 Marginal distribution, 619
 Marginal posterior distribution, 153, 174
 Markov chain Monte Carlo (MCMC), 185
 Markov property, 214
 Maximum likelihood estimate, 5, 58, 618, 626, 648
 Maximum likelihood estimator, 5, 607, 608, 612, 613, 642, 653, 659
 Mean and variance of a truncated normal distribution, *see* Distribution
 Mean squared error, 21, 282, 303, 305, 336, 348, 478, 490
 accounting for sources of uncertainty, 478
 Bayesian interpretation, 629, 722
 marginal posterior distribution, 359, *See also* Binary data
 Mercer's theorem, 532
 Method of moments, 146
 Method of scoring, 84
 Metropolis-Hastings algorithm, 186, 622
 general acceptance ratio, 219
 joint updating, 186, 620, 622, 691, 697, 704
 logistic regression, 486
 single-site updating, 190, 620, 689
 Microarray study, 21
 Misclassification error, 21
 Missing data, 67, 72, 109
 Misspecified likelihood, 78
 Mixed linear model, 125, 167, 180
 Mixed model equations (BLUP), 168, 636, 738
 Mixture distribution, 155
 Student-*t* distribution, 155
 Mixture model, 134, 257, 371
 Gaussian mixture, *See also* false discovery rate
 Model checking, 19
 Model comparison, 270
 Monte Carlo sampling error, 197, 620, 622, 624, 626, 693, 696, 702, 712
 Multilogic model, 562
 Multiple testing, 44

N

Nadaraya-Watson estimator, 513
 Neural networks, 553
 activation function, 554, 564
 bias, 554
 chain rule, 558
 hyperbolic function, 565
 logistic regression, 554
 neuron, 554

 prediction, 569
 rectified linear unit (ReLU) function, 565
 leaky ReLU function, 566
 sigmoid function, 554
 softmax function, 562
 tanh function, 565
 weights, 554

Neuron, *see* Neural networks

Newton-Raphson, 83, 615, 617, 618, 662, 668, 669, 677

Normal mixtures, *see* False discovery rate

Norm of a vector, 525

Notation, 57

Nuisance parameter, 128, 175

Null hypothesis, 361

Null model, 420

O

Observed information, 59, 608, 648, 649

Odds ratio, 94

Operational model, 633

OPTIM, 11, 89, 626, 664, 666, 669, 672, 677, 701, 718

Optimism, 304, 314, 322, 484, 637, 728

 Bayesian optimism, 484

 estimation of optimism, 315, 738

 Monte Carlo estimator, 637, 739

Out-of-bag error rate, *see* Random forest

Overall error rate, *see* Binary data

Overfitting, *see* Least squares

P

Pearson-Aitken formula, 455, 461

Penalised logistic regression, 410, 549

 prediction, *See also* Binary data

Phenotype, 36

Phenotypic value, 285

Phenotypic variance, 38

Positive false discovery rate, *see* False discovery rate

Posterior distribution, 8, 147, 619, 685

 asymptotic theory, 183, 685

 improper posterior distribution, 507

 modal value, 702

Posterior interval, 12, 620, 688, 719

Posterior mean, 153, 166

Posterior mode, 166, 702

Posterior predictive distribution, 19, 157, 473,
 See also Binary data

Posterior predictive model checking, 499

 discrepancy measures, 499

 marginal posterior distribution, 505

heterogeneous variance model, 500
 Posterior variance, 153
 Predicted disease status, 437
 Prediction, 281, 472
 Bayesian prediction, 472
 Prediction error variance, 170, 296
 Prediction of a genetic disease, 455
 Prediction of disease status, 452
 Prediction with binary records, 419, 485
 Prevalence, *see* Binary data
 Prior distribution, 8, 147, 619
 Prior information, 8, 18
 Prior odds, 177
 Prior predictive distribution, 474
 See also Binary data
 Probability density function, 12, 57, 149, 654
 Probability distribution, 4
 Probability mass function, 58, 92, 607, 641
 Probability of a false discovery, 361
 Probability of a significant result, 361
 Probability of a type I error, 361
 Probit model, 116, 242, 616, 622, 666, 671, 704
 correlated data, 674
 Proposal distribution, 185, 214
p-value, 364

Q

QTL detection with binary records, 417, 427
 QTL detection with continuous data, 392
 Quadratic forms
 expectation of a quadratic form, 141, 627
 Quantitative trait loci, QTL, 37

R

Random forest, 30, 585
 bagging, 585
 capturing interaction, 590
 out-of-bag error estimate, 586
 variability of random forest, 590
 variable importance, 587
 Random variable, 4
 Receiver operating characteristic (ROC) curve,
 see Binary data
 Rectified linear unit (ReLU) function, *see*
 Neural networks
 Representer theorem, 536
 Reproducing kernel Hilbert space (RKHS),
 533
 Restricted likelihood, 175
 Restricted maximum likelihood, 128
 Reversibility, 220
 Reversible Markov chain, 215

Ridge regression, 328, 527, 634
 Bayesian view of ridge regression, 333
 bias-variance trade-off, 332
 choice of shrinkage parameter, 332
 dual solution, 527
 leave-one-out cross-validation, 338
 mean squared error, 330
 prediction ability, 334

S

Sample mean squared error, 303
 Sampling from truncated distributions, 239
 Sampling importance resampling (SIR), 212
 Sampling variance of the maximum likelihood
 estimator, 6
 Score function, 59, 64, 95, 613, 659, 661
 Scoring rule, 403
 Sensitivity, *see* Binary data
 Sensitivity analysis, 208
 Shrinkage, *see* Ridge regression
 Sigmoid function, *see* Neural networks
 Similarity matrix, 534
 Single nucleotide polymorphisms (SNP), 42
 Softmax function, *see* Neural networks
 Specificity, *see* Binary data
 Spectral decomposition, 50, 98, 329, 543, 617,
 625
 Spike and slab model, 349, 392
 genomic variance, 352
 MCMC implementation, 353
 mean squared error, *See also* False
 discovery rate
 Spike and slab model for binary records, *see*
 Binary data
 State space, 214
 Stationarity, 214, 219
 Stationary distribution, 185, 214, 216
 Stochastic gradient descent, 106
 Storey's *q*-values, *see* False discovery rate
 Subdifferential, *see* Lasso
 Supervised learning, 510
 Support, 58, 76

T

Tanh function, *see* Neural networks
 Target distribution, 186
 Taylor series expansion, 83, 166, 462
 Threshold model, 92
 Training data, 21, 295, 302
 Training mean squared Error
 See also Binary data
 Training mean squared error, 21, 308

Transformation of random variables, [189](#), [364](#)

Transition probability, [215](#)

Tree

 classification tree, [26](#), [576](#)

 deviance, [579](#), [584](#)

 Gini index, [579](#)

 impurity, [578](#)

 prediction, [28](#)

 split, [576](#)

 terminal node, [28](#), [576](#)

 pruning, [578](#)

 regression tree, [578](#)

 variability of estimated trees, [584](#)

True model, [78](#)

Truncated data, [77](#), [142](#), [266](#)

Truncated distribution, *see* Distribution

Truncated normal model, [266](#)

t-test, [44](#), [366](#)

U

Unsupervised learning, [510](#)

V

Validating data, [21](#), [302](#)

Validating datum, [295](#)

Validating mean squared error, [21](#), [321](#)

See also Binary data

Vectorising back propagation, *see* Back propagation

Vectorising forward propagation, *see* Forward propagation

W

Weighted bootstrap, [492](#), [493](#)

Weights, *see* Neural networks

Z

z-values, [365](#)